

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ  
НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ  
Физический факультет

С. В. Дубров

## **АРХИТЕКТУРА И ЭВОЛЮЦИЯ ЭВМ**

Учебное пособие

Новосибирск

2021

УДК 004.2(075)  
ББК 3-971.32я73-1  
Д79

**Дубров, С. В.**

Д79 Архитектура и эволюция ЭВМ : учеб. пособие / С. В. Дубров ; Новосиб. гос. ун-т. – Новосибирск : ИПЦ НГУ, 2021. – 654 с.

В пособии представлен курс лекций «Архитектура и эволюция ЭВМ», читаемый бакалаврам физического факультета НГУ. В тексте даётся большое количество материала по истории развития вычислительной техники.

Цель курса – ознакомление с основами и принципами построения и функционирования современных электронно-вычислительных машин (компьютеров), изучение их архитектуры, системы команд, устройства системы ввода-вывода.

Материал организован по принципу «снизу-вверх» – от основ построения цифровых устройств, включая обоснование выбора двоичной системы счисления для представления в «цифре». Приведены основы булевой алгебры, дано описание представления чисел в знаковом целочисленном формате и в формате двоичной арифметики с плавающей точкой (IEEE 754). Приведено краткое описание основных технологий цифровых интегральных схем и принципы построения на основе базовых элементов более крупных узлов, из которых в итоге строится полноценная вычислительная система.

Предназначено для студентов математических, физических, естественнонаучных факультетов университетов, инженерно-физических и других специальностей вузов.

УДК 004.2(075)  
ББК 3-971.32я73-1

Рекомендовано к печати кафедрой физико-технической информатики ФФ НГУ  
(протокол № 6 от 23.10.2020 г.)

© Новосибирский государственный  
университет, 2021  
© Дубров С.В., 2021

## Оглавление

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| Введение.....                                                               | 7  |
| Глава 1. Основы цифровой электроники.....                                   | 9  |
| Понятие алгебры логики (булева алгебра).....                                | 9  |
| Комбинационная логика.....                                                  | 11 |
| Таблицы истинности для основных двоичных логических функций.....            | 12 |
| Обозначение на логических схемах.....                                       | 13 |
| Карты Карно.....                                                            | 13 |
| Секвенциальная (последовательностная) логика.....                           | 19 |
| Триггеры.....                                                               | 19 |
| Регистры.....                                                               | 25 |
| Счётчики.....                                                               | 26 |
| Запоминающие устройства.....                                                | 27 |
| Технологии цифровых логических микросхем.....                               | 29 |
| Резисторно-транзисторная логика.....                                        | 31 |
| Диодно-транзисторная логика.....                                            | 32 |
| Транзисторно-транзисторная логика.....                                      | 33 |
| Интегрально-инжекционная логика.....                                        | 39 |
| N-МОП/Р-МОП.....                                                            | 41 |
| КМОП.....                                                                   | 42 |
| Эмиттерно-связанная логика.....                                             | 45 |
| Глава 2. Архитектура цифровых вычислительных устройств.....                 | 48 |
| Архитектура фон Неймана.....                                                | 48 |
| История возникновения термина.....                                          | 48 |
| Принципы фон Неймана.....                                                   | 49 |
| Узкое место архитектуры фон Неймана.....                                    | 49 |
| Гарвардская архитектура.....                                                | 50 |
| История.....                                                                | 50 |
| Классическая гарвардская архитектура.....                                   | 51 |
| Модификации.....                                                            | 51 |
| Использование.....                                                          | 52 |
| CISC/RISC архитектуры.....                                                  | 52 |
| CISC.....                                                                   | 52 |
| RISC.....                                                                   | 53 |
| Компьютерные шины.....                                                      | 60 |
| Описание шин.....                                                           | 61 |
| История.....                                                                | 62 |
| Организация ввода-вывода информации.....                                    | 66 |
| Программно управляемый ввод/вывод.....                                      | 66 |
| Ввод/вывод по прерываниям.....                                              | 67 |
| Организация прямого доступа к памяти (ПДП/DMA).....                         | 70 |
| Глава 3. Арифметические основы ЭВМ.....                                     | 72 |
| Выбор системы счисления ЭВМ.....                                            | 72 |
| Понятия разрядности в архитектуре ЭВМ.....                                  | 75 |
| Представление целых чисел со знаком.....                                    | 76 |
| Представления действительных чисел (стандарт IEEE 754).....                 | 77 |
| Представление числа в нормализованном экспоненциальном виде.....            | 78 |
| Представление числа в денормализованном экспоненциальном виде.....          | 78 |
| Преобразование десятичного числа в двоичное число с плавающей точкой.....   | 78 |
| Полный диапазон чисел одинарной точности (32 бит) по стандарту IEEE754..... | 81 |
| Полный диапазон чисел двойной точности (64 бит) по стандарту IEEE754.....   | 81 |
| Точность представления вещественных чисел в формате IEEE754.....            | 82 |

|                                                        |     |
|--------------------------------------------------------|-----|
| Порядок следования байт в многобайтовых словах .....   | 83  |
| Варианты записи.....                                   | 83  |
| Структуры данных.....                                  | 85  |
| Основные структуры данных .....                        | 85  |
| Глава 4. Методы увеличения производительности ЭВМ..... | 91  |
| Power Wall .....                                       | 91  |
| Иерархия памяти в архитектуре ЭВМ, Memory Wall .....   | 93  |
| Регистры .....                                         | 96  |
| Кэш память.....                                        | 97  |
| Оперативная память (RAM) .....                         | 104 |
| 3D XPoint (Intel Optane) .....                         | 106 |
| Дисковая память .....                                  | 107 |
| Ленточные накопители .....                             | 115 |
| Методы увеличения производительности CPU.....          | 117 |
| Параллелизм на уровне команд.....                      | 117 |
| Глава 5. Однокристалльные микроЭВМ MCS-48/MCS-51 ..... | 126 |
| Семейство MCS-48 .....                                 | 126 |
| Архитектура микроконтроллера MCS-48 .....              | 126 |
| Блок управления и синхронизации .....                  | 128 |
| Арифметико-логическое устройство .....                 | 129 |
| Память микроконтроллера.....                           | 130 |
| Организация ввода/вывода .....                         | 133 |
| Система обработки прерываний .....                     | 135 |
| Таймер/счётчик .....                                   | 136 |
| Система команд MCS-48.....                             | 137 |
| Семейство MCS-51 .....                                 | 145 |
| Архитектура микроконтроллера MCS-51 .....              | 145 |
| Арифметико-логическое устройство .....                 | 147 |
| Память микроконтроллера.....                           | 148 |
| Параллельные порты ввода/вывода .....                  | 150 |
| Система прерывания MCS-51.....                         | 151 |
| Таймеры/счётчики .....                                 | 153 |
| Последовательный интерфейс.....                        | 155 |
| Сброс в начальное состояние и синхронизация .....      | 159 |
| Система команд MCS-51.....                             | 160 |
| Программирование микроконтроллера.....                 | 167 |
| Глава 6. Мини ЭВМ DEC PDP-11/VAX-11.....               | 169 |
| Семейство PDP-11/LSI-11.....                           | 170 |
| Архитектура LSI-11.....                                | 171 |
| Q-bus .....                                            | 171 |
| Система команд PDP-11/LSI-11 .....                     | 182 |
| Диспетчер памяти (MMU) .....                           | 192 |
| Программное обеспечение.....                           | 195 |
| Супермини ЭВМ VAX-11.....                              | 199 |
| Программная архитектура VAX-11 .....                   | 199 |
| Общая схема VAX-11.....                                | 200 |
| Система команд VAX-11 .....                            | 204 |
| Защита и привилегии .....                              | 211 |
| Программное обеспечение.....                           | 218 |
| Развитие архитектур DEC.....                           | 219 |
| Глава 7. ЭВМ на основе процессоров Intel .....         | 226 |
| Архитектура микроЭВМ на базе i8080 .....               | 227 |



|                                                                                   |     |
|-----------------------------------------------------------------------------------|-----|
| Система команд i8080/85 .....                                                     | 230 |
| Ядро системы на базе процессора i8080, шина .....                                 | 234 |
| Подключение периферийных устройств .....                                          | 239 |
| Программное обеспечение.....                                                      | 243 |
| Архитектура компьютеров на базе Intel 8086/88 .....                               | 252 |
| Архитектура микропроцессора i8086/88 .....                                        | 255 |
| Система команд i8086/88 .....                                                     | 258 |
| Подключение периферийных устройств, шина.....                                     | 262 |
| Программное обеспечение.....                                                      | 266 |
| Архитектура компьютеров на базе Intel 80286 .....                                 | 270 |
| Третье поколение IBM PC/AT .....                                                  | 271 |
| Архитектура микропроцессора i80286 .....                                          | 272 |
| Система команд i80286 .....                                                       | 294 |
| Ядро системы на базе процессора i80286 .....                                      | 302 |
| Подключение периферийных устройств .....                                          | 308 |
| Программное обеспечение.....                                                      | 314 |
| Архитектура компьютеров на базе Intel 80386/486.....                              | 319 |
| Поколение PC на основе процессоров i80386/486 .....                               | 320 |
| Попытка IBM вернуться на рынок PC, семейство IBM PS/2 .....                       | 322 |
| Архитектура микропроцессоров i80386/486 .....                                     | 328 |
| Организация памяти в 80386/486 .....                                              | 343 |
| Система команд i80386/486 .....                                                   | 354 |
| Построение вычислительной системы .....                                           | 356 |
| Программное обеспечение.....                                                      | 369 |
| Архитектура компьютеров на базе Intel Pentium поколений P5...P6/M...NetBurst..... | 375 |
| PC на основе процессоров Pentium поколений P5...P7 .....                          | 377 |
| Архитектура микропроцессоров Pentium поколений P5...P7 .....                      | 382 |
| 64-битное расширение архитектуры x86.....                                         | 403 |
| Организация памяти в P5...P7 .....                                                | 408 |
| Система команд .....                                                              | 412 |
| Построение вычислительной системы .....                                           | 415 |
| Программное обеспечение.....                                                      | 434 |
| Архитектура компьютеров на базе поколений процессоров Intel Core 2 и далее .....  | 437 |
| PC на основе процессоров поколений Intel Core 2, Nehalem, Sandy Bridge.....       | 437 |
| Архитектура процессоров Intel Core 2...Skylake.....                               | 445 |
| Intel Core 2, Nehalem, Sandy Bridge... — а что же AMD?.....                       | 475 |
| Система команд .....                                                              | 489 |
| Построение вычислительной системы .....                                           | 490 |
| Программное обеспечение.....                                                      | 517 |
| Глава 8. Суперкомпьютеры .....                                                    | 535 |
| Определение понятия суперкомпьютер .....                                          | 535 |
| Классификация параллельных машин.....                                             | 537 |
| Таксономия Флинна .....                                                           | 538 |
| Основные классы современных параллельных компьютеров .....                        | 543 |
| Краткая история появления параллелизма в архитектуре ЭВМ.....                     | 544 |
| Использование параллельных вычислительных систем.....                             | 547 |
| Закон Амдала-Уэра и его следствия .....                                           | 547 |
| Параллельные вычисления на GPU .....                                              | 549 |
| Архитектура параллельных вычислений CUDA .....                                    | 550 |
| Ты помнишь, как всё начиналось – ILLIAC IV, Cray, CDC Cyber .....                 | 555 |
| ILLIAC IV .....                                                                   | 555 |
| CDC Cyber .....                                                                   | 559 |

|                                                                      |     |
|----------------------------------------------------------------------|-----|
| CRAY .....                                                           | 561 |
| СуперЭВМ в России .....                                              | 573 |
| Современное состояние .....                                          | 574 |
| Глава 9. Архитектура виртуальных машин.....                          | 580 |
| Виртуализация .....                                                  | 580 |
| Классические критерии виртуализуемости .....                         | 580 |
| Виды виртуализации .....                                             | 585 |
| Виртуальная машина.....                                              | 588 |
| Гипервизор .....                                                     | 591 |
| История гипервизоров.....                                            | 592 |
| Типы гипервизоров.....                                               | 592 |
| Под «капотом» у виртуальной машины .....                             | 596 |
| Аппаратная поддержка виртуализации в x86 архитектурах .....          | 599 |
| Intel VT-x .....                                                     | 602 |
| AMD Virtualization – AMD-V (Secure Virtual Machine «Pacifica») ..... | 604 |
| Виртуализация ввода-вывода .....                                     | 608 |
| Облачные технологии .....                                            | 612 |
| Глава 10. Архитектуры ЭВМ, разработанных в ИЯФ .....                 | 615 |
| МикроЭВМ Одрёнок.....                                                | 615 |
| Одрёнок .....                                                        | 616 |
| Особенности архитектуры .....                                        | 617 |
| Системные средства .....                                             | 618 |
| Схемная реализация .....                                             | 620 |
| Заключение.....                                                      | 621 |
| Универсальный арифметический процессор АП-32 .....                   | 622 |
| Введение .....                                                       | 622 |
| Архитектура АП-32 .....                                              | 623 |
| Система команд АП-32 .....                                           | 629 |
| Технические характеристики АП-32 .....                               | 633 |
| Канал ввода/вывода и управления АП-32.....                           | 635 |
| Драйвер АП-32.....                                                   | 642 |
| Комплекс программного обеспечения.....                               | 644 |
| Заключение.....                                                      | 649 |
| Список литературы.....                                               | 652 |

## ВВЕДЕНИЕ

До появления первых электронных цифровых вычислительных машин человечество прошло долгий путь: от домеханических устройств (как пример такого устройства – логарифмическая линейка), через механическую эпоху (первая вычислительная машина «Паскалина»), через электромеханическую эпоху (табулятор Холлерита, машина MARK-1 Айкена), до официально признанного первым цифровым вычислительным устройством, без движущихся частей – компьютера Атанасова — Берри (Atanasoff-Berry Computer, ABC)<sup>1</sup> и, наконец, до первого электронного цифрового вычислителя общего назначения ENIAC<sup>2</sup> (Джон Преспер Эккерт и Джон Уильям Моучли, 1945 год).

Во введении были использованы материалы из доступного публично цикла презентаций преподавателя Томского государственного университета Б.А.Гладких. Из четырёх частей под общим названием «ИНФОРМАТИКА. Историческое введение в специальность» были использованы материалы из первых двух: «Доэлектронная история вычислительной техники» и «Электронные вычислительные машины», в основном иллюстрации.

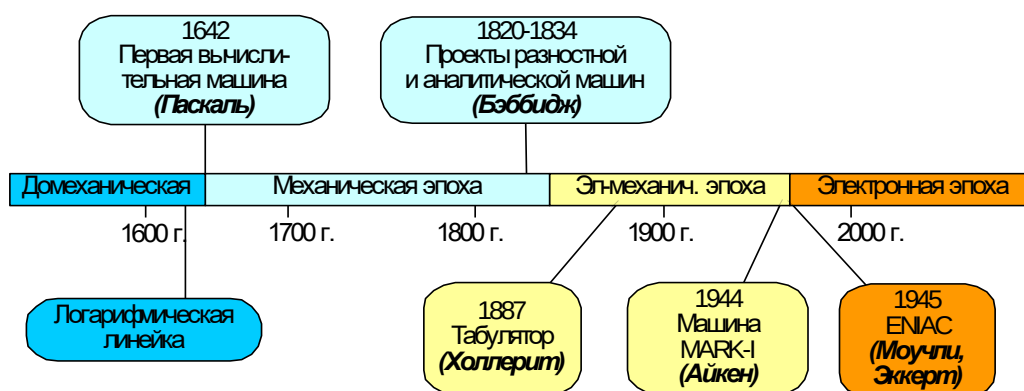


Рис.1 – эпохи

Первыми в этом ряду исторически принято считать древнеримский абак, к этой же категории «вычислительных устройств» относятся китайские счёты «суаньпань» и русские счёты, которые ещё лет 25-30 назад вполне можно было встретить вживую на кассах в наших новосибирских магазинах:

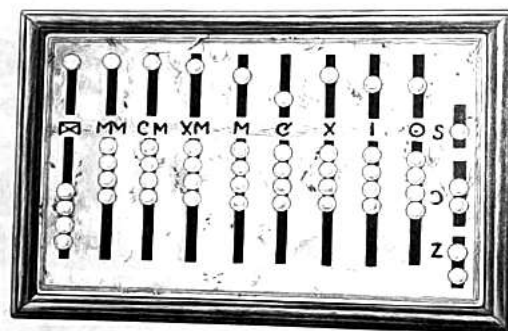


Рис.2 - Древнеримский абак

<sup>1</sup> Формально ABC машина не была программируемой, она создавалась только для решения систем линейных уравнений.

<sup>2</sup> Сокр. от Electronic Numerical Integrator and Computer.



Рис.3 - Китайские счёты «суанпань»

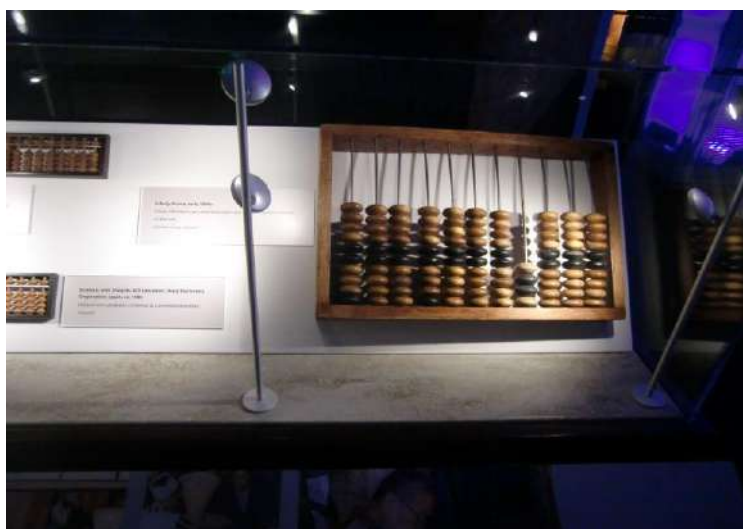


Рис.4 - Русские счёты<sup>1</sup>

Дальше последовала логарифмическая линейка, которая реализовала в «железе» (точнее, в «дереве») теорию логарифмов, изложенную Джоном Непером в 1614 г. в его книге, озаглавленной, как какое-то сказочное издание: «Описание удивительных таблиц логарифмов». По сути, логарифмическая линейка была реализацией **аналогового** вычислителя – входные данные и результат представлялись не в дискретном виде, как, например, на счётах, а в виде непрерывных, аналоговых значений, использовавших свойства логарифмов:  $\log(a \cdot b) = \log a + \log b$  – произведение двух значений вычислялось как (аналоговая) сумма двух отрезков, отградуированных в логарифмическом масштабе.

Более подробные материалы по теме представлены в указанных выше PowerPoint презентациях (первые две части) Б.А.Гладких, полностью переносить (копировать) их в этот документ было бы нерационально из-за большого объёма: 45 слайдов ч.1 и 124 слайда ч.2. Следует отметить, что вторая часть презентации Б.А.Гладких – «Электронные вычислительные машины» – заканчивается описанием образцов вычислительной техники примерно 2005 года и не включает множество революционных прорывных технологий, произошедших после этой даты.

<sup>1</sup> Фотография из музея истории вычислительной техники в Маунтин-Вью, Калифорния.

# ГЛАВА 1. ОСНОВЫ ЦИФРОВОЙ ЭЛЕКТРОНИКИ

Все современные вычислительные машины построены исключительно с использованием электронных цифровых схем. В этой главе будут изложены теоретические основы цифровой логики, её реализации в «кремнии», показано, как из простейших базовых элементов строятся более крупные узлы: АЛУ (арифметически-логическое устройство), регистры, ОЗУ, мультиплексоры, организуются связи между ними («шина») для обмена данными и т.д.

В третьей главе приводится обоснование выбора системы счисления, которую используют современные ЭВМ – на сегодня это практически повсеместно позиционная система с основанием 2, она же – двоичная, самая простая в реализации и одновременно самая «широкая» и «многословная». Одно и то же число, представленное в различных системах счисления, будет иметь наибольшее количество разрядов именно в двоичной нотации. В качестве примера приведём представление десятичного числа 128 в разных системах счисления (в формате языка Си):

- 128 – десятичная (dec)
- 0200 – восьмеричная (octal)
- 0x80 – шестнадцатеричная (hex)
- 2000 – четверичная
- 100000000 – двоичная (bin)

## Понятие алгебры логики (булева алгебра)

Алгебра логики (булева алгебра) — это раздел математики, изучающий высказывания, рассматриваемые со стороны их логических значений (истинности или ложности) и логических операций над ними. Алгебра логики позволяет закодировать любые утверждения, истинность или ложность которых нужно доказать, а затем манипулировать ими подобно обычным числам в математике. Создание алгебры логики в середине XIX века в трудах Джорджа Буля представляло собой попытку решать традиционные логические задачи алгебраическими методами.

Формальное определение: **булевой алгеброй** называется непустое множество  $A$  с двумя бинарными операциями  $\wedge$  (аналог конъюнкции, логическое «И»),  $\vee$  (аналог дизъюнкции, логическое «ИЛИ»), одной унарной операцией  $\neg$  (аналог отрицания, логическое «НЕ») и двумя выделенными элементами: 0 (или Ложь) и 1 (или Истина) такими, что для любых  $a, b$  и  $c$  из множества  $A$  верны следующие **аксиомы**:

1. Булева переменная всегда равна либо нулю, либо единице  
 $x=0$ , если  $x \neq 1$   
 $x=1$ , если  $x \neq 0$
2. Инверсное значение переменной  $x$  противоположно ее прямому значению  
 $x=0$ , если  $\neg x=1$   
 $x=1$ , если  $\neg x=0$
3. Правила выполнения логического умножения (конъюнкции)  
 $0 \wedge 0 = 0$   
 $1 \wedge 1 = 1$   
 $0 \wedge 1 = 0$
4. Правила выполнения логического сложения (дизъюнкции)  
 $0 \vee 0 = 0$   
 $1 \vee 1 = 1$   
 $0 \vee 1 = 1$

## Законы:

### 1. Ассоциативный (сочетательный) закон.

Ассоциативность конъюнкции и дизъюнкции выражается следующими формулами:

$$x_1 \vee (x_2 \vee x_3) = (x_1 \vee x_2) \vee x_3;$$

$$x_1 \wedge (x_2 \wedge x_3) = (x_1 \wedge x_2) \wedge x_3$$

На практике это означает, что можно опускать те скобки, которые определяют, в каком порядке должна выполняться конъюнкция и дизъюнкция.

### 2. Коммутативный (переместительный) закон.

Если операция коммутативна, то результат ее применения не зависит от того, какой из операндов был первым, а какой — вторым. Операнды коммутативных операций можно менять друг с другом местами, получая тождественный результат.

$$x_1 \vee x_2 = x_2 \vee x_1; \quad x_1 \wedge x_2 = x_2 \wedge x_1$$

### 3. Дистрибутивный (распределительный) закон

Свойство дистрибутивности одной операции относительно другой позволяет «раскрывать» скобки аналогично процедуре из элементарной алгебры. Конъюнкция и дизъюнкция дистрибутивны друг относительно друга, что выражается в следующих формулах:

$$(x_1 \wedge x_2) \vee x_3 = (x_1 \vee x_3) \wedge (x_2 \vee x_3)$$

$$(x_1 \vee x_2) \wedge x_3 = (x_1 \wedge x_3) \vee (x_2 \wedge x_3)$$

### 4. Законы де Моргана (законы общей инверсии или дуальности)

Законы де Моргана позволяют применять отрицания к целой скобке, позволяя перейти к так называемым тесным отрицаниям, когда ни одно отрицание не стоит перед скобкой.

$$\overline{x_1 \wedge x_2} = \overline{x_1} \vee \overline{x_2}; \quad \overline{\overline{x_1 \wedge x_2}} = \overline{\overline{x_1} \vee \overline{x_2}}$$

$$\overline{x_1 \vee x_2} = \overline{x_1} \wedge \overline{x_2}; \quad \overline{\overline{x_1 \vee x_2}} = \overline{\overline{x_1} \wedge \overline{x_2}}$$

### 5. Закон поглощения (элиминации)

Если к выражению применяется с одним и тем же операндом сначала одна операция, а потом, с тем же самым операндом, поглощающая её, то значение выражения поглощается, становясь равно операнду. Таким образом поглощающие друг друга пары операций можно «выкидывать» во время упрощения.

$$x_1 \vee (x_1 \wedge x_2) = x_1; \quad x_1 \wedge (x_1 \vee x_2) = x_1$$

### 6. Закон склеивания (исключения)

$$(x_1 \wedge x_2) \vee (x_1 \wedge \overline{x_2}) = x_1 \wedge (x_2 \vee \overline{x_2}) = x_1$$

$$(x_1 \vee x_2) \wedge (x_1 \vee \overline{x_2}) = x_1$$

### 7. Свойства единицы и нуля

Конъюнкция и дизъюнкция «по-особому» реагируют на единицу или ноль в качестве одного из операндов, независимо от значения второго. Эти свойства похожи на знакомые из элементарной алгебры умножение на единицу, умножение на ноль, сложение с нулем:

$$A \wedge 0 = 0$$

$$A \wedge 1 = A$$

$$A \vee 0 = A$$

$$A \vee 1 = 1$$

#### 8. Идемпотентность

Операция называется идемпотентной, если, применяя ее к двум равным операндам, получается тот же самый операнд. Идемпотентность позволяет «выбрасывать» лишние повторные применения операции из формулы. Конъюнкция и дизъюнкция идемпотентны:

$$A \wedge A = A$$

$$A \vee A = A$$

#### 9. Дополнение

Отрицание (инверсия) операнда называется его дополнением. Конъюнкция или дизъюнкция операнда со своим дополнением дает однозначные результаты независимо от значения операнда:

$$A \wedge \neg A = 0$$

$$A \vee \neg A = 1$$

#### 10. Двойное отрицание

Двойное отрицание компенсирует само себя. Таким образом в форме с тесными отрицаниями у каждой переменной в выражении либо не стоит ни одного отрицания, либо только одно.

$$\neg\neg A = A$$

### Правила:

С помощью законов алгебры логики можно производить равносильные преобразования логических выражений с целью их упрощения. В алгебре логики на основе принятого соглашения установлены следующие правила (приоритеты) для выполнения логических операций:

первыми выполняются операции в скобках, затем в следующем порядке:

инверсия (отрицание  $\neg$ ),

конъюнкция ( $\wedge$ ),

дизъюнкция ( $\vee$ ),

импликация ( $\rightarrow$ ), эквиваленция ( $=$ ).

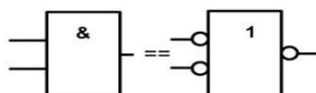
На логических элементах, реализующих булевы функции, строятся **цифровые логические схемы** электронных устройств.

### Комбинационная логика

**Комбинационная логика** (*комбинационная схема*) в теории цифровых устройств — двоичная логика функционирования устройств комбинационного типа. У комбинационных устройств состояние выхода однозначно определяется набором входных сигналов, что отличает комбинационную логику от *секвенциальной* (последовательностной) логики, в рамках которой выходное значение зависит не только от текущего входного воздействия, но и от предыстории функционирования цифрового устройства. Другими словами, секвенциальная логика предполагает наличие памяти, которая в комбинационной логике не предусмотрена.

Значениям переменной в булевой алгебре соответствуют состояниям цифровых элементов компьютера или любого другого электронного устройства: например, сигнал присутствует – можно интерпретировать как логическую "1", сигнал отсутствует – логический "0". В соответствии с законами де Моргана можно сказать, что оказывается совершенно непринципиальным, что именно будет «исполнять» роль логической «1» или логического «0» в реальной электронной схеме – можно, наоборот, наличие сигнала интерпретировать как логический «0», а его отсутствие – как логическую «1». Такая интерпретация «наоборот» называется *инверсной логикой*.

Как пример: в позитивной логике (есть сигнал – логическая «1», сигнал отсутствует – логический «0»), схема 2И «на единицы» в инверсной логике будет трактоваться 2ИЛИ «на нули»:



Ещё пример, где активно используется *инверсная логика*: т.н. *монтажное ИЛИ*, использующееся для объединения нескольких выходов логических схем. Функция ИЛИ здесь реализуется в инверсной логике, в позитивной логике – это схема И.



Кроме алгебраического есть несколько других способов представления логических функций: в виде таблицы истинности или в графическом виде (карты Карно), все способы описания эквивалентны друг другу.

## Таблицы истинности для основных двоичных логических функций

Конъюнкция  
(AND)

| $a$ | $b$ | $a \wedge b$ |
|-----|-----|--------------|
| 0   | 0   | 0            |
| 1   | 0   | 0            |
| 0   | 1   | 0            |
| 1   | 1   | 1            |

Дизъюнкция  
(OR)

| $a$ | $b$ | $a \vee b$ |
|-----|-----|------------|
| 0   | 0   | 0          |
| 1   | 0   | 1          |
| 0   | 1   | 1          |
| 1   | 1   | 1          |

Сложение по модулю 2  
(XOR)

| $a$ | $b$ | $a \oplus b$ |
|-----|-----|--------------|
| 0   | 0   | 0            |
| 1   | 0   | 1            |
| 0   | 1   | 1            |
| 1   | 1   | 0            |

Импликация

| $a$ | $b$ | $a \rightarrow b$ |
|-----|-----|-------------------|
| 0   | 0   | 1                 |
| 1   | 0   | 0                 |
| 0   | 1   | 1                 |
| 1   | 1   | 1                 |

Эквиваленция

| $a$ | $b$ | $a \leftrightarrow b$ |
|-----|-----|-----------------------|
| 0   | 0   | 1                     |
| 1   | 0   | 0                     |
| 0   | 1   | 0                     |
| 1   | 1   | 1                     |

Штрих Шеффера Стрелка Пирса

| $a$ | $b$ | $a \downarrow b$ |
|-----|-----|------------------|
| 0   | 0   | 1                |
| 1   | 0   | 1                |
| 0   | 1   | 1                |
| 1   | 1   | 0                |

| $a$ | $b$ | $a \downarrow b$ |
|-----|-----|------------------|
| 0   | 0   | 1                |
| 1   | 0   | 0                |
| 0   | 1   | 0                |
| 1   | 1   | 0                |

Отрицание  
(NOT)

| $a$ | $\neg a$ |
|-----|----------|
| 0   | 1        |
| 1   | 0        |

В программировании:

- Конъюнкция = AND = И =  $\wedge$  = &
- Дизъюнкция = OR = ИЛИ =  $\vee$  = |
- Сложение по модулю 2 = XOR = ИСКЛЮЧАЮЩЕЕ ИЛИ =  $\oplus$  = ^
- Отрицание = NOT = НЕ =  $\neg$  = !

Глядя на приведённые выше таблицы, можно понять, например, что штрих Шеффера ( $\downarrow$ ) – это 2И-НЕ, а стрелка Пирса – 2ИЛИ-НЕ ( $\downarrow$ ). Это т.н. базисные функции, используя только их (любую из двух), можно построить сколь угодно сложную комбинационную логическую схему.



На Рис.5 показаны варианты представления некоторых логических функций в виде таблиц истинности и теории множеств:

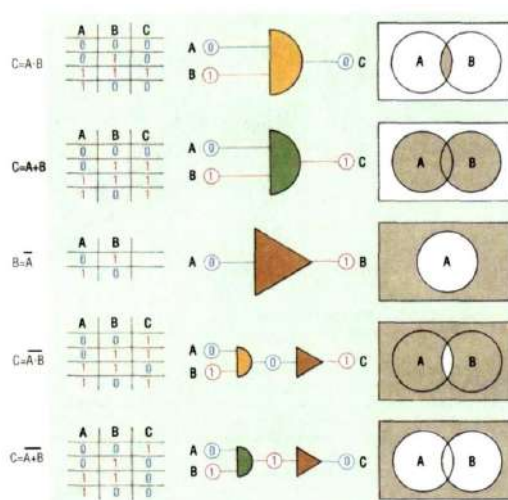


Рис.5 - Примеры представления для некоторых логических функций

### Обозначение на логических схемах

Для обозначения логических элементов используется несколько стандартов. Наиболее распространёнными являются американский (ANSI), европейский (DIN), международный (IEC) и российский (ГОСТ). На Рис.6 ниже приведены обозначения логических элементов на схемах в этих стандартах.

| Логика     |        | «НЕ» | «И» | «ИЛИ» | «И-НЕ» | «ИЛИ-НЕ» |
|------------|--------|------|-----|-------|--------|----------|
| ГОСТ и IEC | Полож. |      |     |       |        |          |
|            | Отриц. |      |     |       |        |          |
| ANSI       | Полож. |      |     |       |        |          |
|            | Отриц. |      |     |       |        |          |
| DIN        | Полож. |      |     |       |        |          |
|            | Отриц. |      |     |       |        |          |

Рис.6 - Обозначение логических элементов.

### Карты Карно

**Куб Карно́ (ка́рта Карно, диагра́мма Карно)** — графический способ представления переключательных булевых функций с целью наглядной и удобной их минимизации, обеспечивающий упрощение сложных логических функций многих переменных и устранение потенциальных логических гонок.

Является одним из эквивалентных способов описания или задания логической функции наряду с таблицей истинности или выражениями булевой алгебры. Преобразование

представления логической функции, заданной в виде карты Карно в таблицу истинности или в булеву формулу и обратное преобразование элементарно по простоте.

Удобство и наглядность такого представления логической функции обусловлено тем, что логические термы, к которым могут быть применены *операции попарного неполного склеивания* и *элементарного поглощения* группируются в карте Карно в виде массивов соседних ячеек по вертикалям и горизонталям логических единиц или нулей, что человек сразу наглядно видит всю картину.

Карты Карно можно рассматривать как развертку на плоскость  $n$ -мерного булева куба, причем размерность этого гиперкуба совпадает с количеством переменных представляемой функции. Графически карта Карно изображается в виде прямоугольника или квадрата из ячеек, число которых равно  $2^N$ , причем любые две соседние ячейки по вертикали или по горизонтали или, иными словами — в *окрестности фон Неймана*, описывают термы, различающиеся *только по одной* переменной — с логическим отрицанием и без логического отрицания. Также соседним являются первая и последняя строки, крайний левый и крайний правый столбцы таблицы, поэтому таблица Карно является фактически разверткой логического гиперкуба на поверхность *тороида*. Возможно построение самых различных карт для одной и той же функции, удовлетворяющих условию: геометрическое соседство ячеек в смысле фон Неймана — логическое соседство термов — то есть с *расстоянием Хэмминга*<sup>1</sup> между термами соседних ячеек равным 1. Любая из таких таблиц одинаково удобна для минимизации функции, но обычно переменные по строкам и столбцам в карте Карно упорядочивают по *рефлексивному коду Грея*<sup>2</sup> из-за *мнемоничности* и наглядности.

Карты Карно были изобретены в 1952 Эдвардом В. Вейчем и усовершенствованы в 1953 Морисом Карно, физиком из «Bell Labs», и были призваны помочь упростить цифровые электронные схемы.

В качестве примера на рис.6 приведена карта Карно для логической функции четырех переменных. Задача — «графически» преобразовать функцию на карте в минимизированное алгебраическое представление.

Процедура начинается с выявления ячеек, граничащих друг с другом по горизонтали или по вертикали, содержащих одинаковые данные, причём количество подряд идущих ячеек с одинаковым содержанием должно быть степенью 2. С такими ячейками производится операция «склеивания», склеиваемые области могут быть прямоугольными или квадратными. Если «склеивать» ячейки с 1, то в итоге получится алгебраическое выражение в т.н. ДНФ<sup>3</sup>. Если «склеивать» ячейки с 0, получится выражение в КНФ<sup>4</sup>.

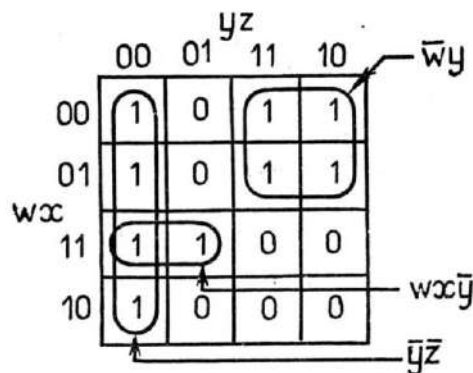
---

<sup>1</sup> **Расстояние Хэмминга** — число позиций, в которых соответствующие символы двух слов одинаковой длины различны.

<sup>2</sup> **Код Грея** — двоичный код, иначе зеркальный код, он же код с отражением, в котором две «соседние» (в упорядоченном, то есть лексикографическом, наборе) кодовые комбинации различаются только цифрой в одном двоичном разряде. Иными словами, расстояние Хэмминга между соседними кодовыми комбинациями равно 1.

<sup>3</sup> **Дизъюнктивная нормальная форма (ДНФ)** в булевой логике — нормальная форма, в которой булева формула имеет вид *дизъюнкции* (ИЛИ) *конъюнкций* (И) литералов (или их инверсий). Любая булева формула может быть приведена к ДНФ.

<sup>4</sup> **Конъюнктивная нормальная форма (КНФ)** в булевой логике — нормальная форма, в которой булева формула имеет вид *конъюнкции* (И) *дизъюнкции* (ИЛИ) литералов (или их инверсий). Любая булева формула может быть приведена к КНФ.



Группы на карте Карно для функции четырех переменных.

Рис.7 - Функция четырех переменных на карте Карно.

В нашем примере мы будем «склеивать» ячейки с «1»:

1. Сначала выделим (склеим) самый левый вертикальный столбец – как видно, в выражение, описывающем эти четыре ячейки, не будет переменных  $W$  и  $X$  – при любом сочетании этих двух литералов выходное значение не меняется и  $=1$ . Т.е., от первого склеивания мы получаем произведение  $\neg Y \wedge \neg Z$ .
2. На втором шаге выделим квадратную группу в правом верхнем углу. В выражении, которое ей соответствует, не будут входить переменные  $X$  и  $Z$ , т.е. результатом второго «склеивания» будет произведение  $\neg W \wedge Y$ .
3. И, наконец, выделим самую маленькую группу, две левых ячейки в предпоследнем ряду. Т.к. в этом случае объединяется только две клетки, это означает, что из выражения, описывающего эту группу, удастся исключить только одну переменную (литерал), в итоге получим  $W \wedge X \wedge \neg Y$ .

Итоговое выражение в ДНФ (на карте не должно остаться «несклеенных» ячеек, содержащих «1»):

$$\neg Y \wedge \neg Z \vee \neg W \wedge Y \vee W \wedge X \wedge \neg Y$$

или в чуть более привычном виде:

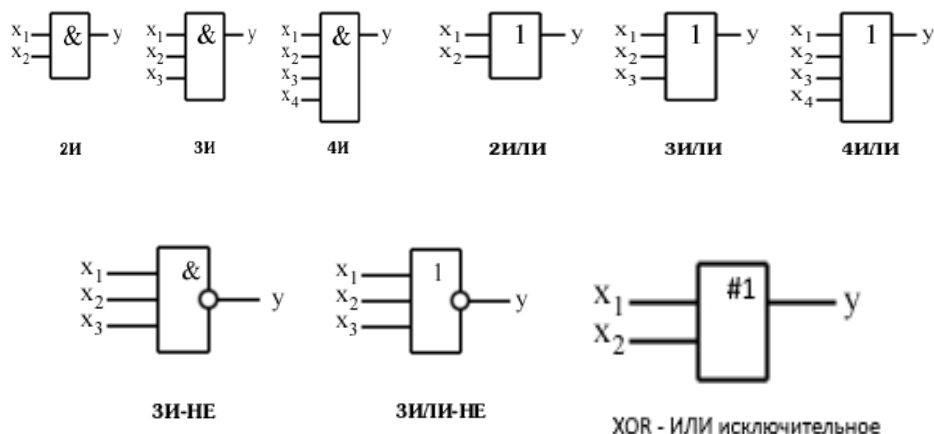
$$\bar{y} * \bar{z} + \bar{w} * y + w * x * \bar{y}$$

Как уже упоминалось выше, на основе любого из двух базисов – штриха Шеффера (И-НЕ) или стрелки Пирса (ИЛИ-НЕ) можно построить сколь угодно сложную комбинационную логическую схему. Но такие схемы будут неоптимальными ни по количеству элементов, ни по производительности (быстродействию). Поэтому для реальной цифровой электроники (обычно это кремний) производилось большое количество самых разнообразных, более сложных комбинационных логических устройств.

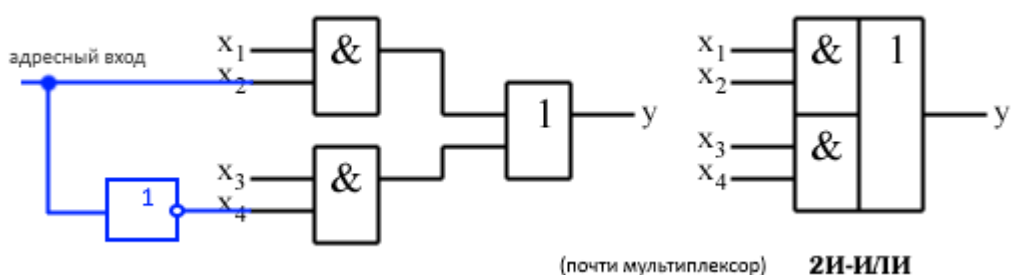
Напомню, что комбинационная схема – это логическая схема, сигнал на выходе которой определяется только значением сигналов на её входах.

В качестве примера более сложных комбинационных схем, чем приведённые выше 2И-НЕ/2ИЛИ-НЕ/XOR первым надо упомянуть, если можно так сказать, «естественное» усложнение базовых элементов (2И-НЕ, 2ИЛИ-НЕ) «вширь» – с увеличенным количеством входов: 3И, 4И-НЕ, 5ИЛИ, 8ИЛИ-НЕ и т.п.

Простые логические элементы:



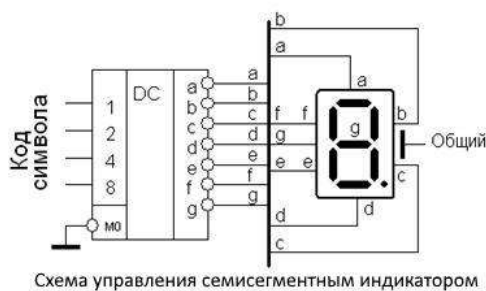
Логическому элементу 2И-ИЛИ, чтобы стать полноценным мультиплексором 2-в-1, не хватает одного инвертора (выделено цветом):



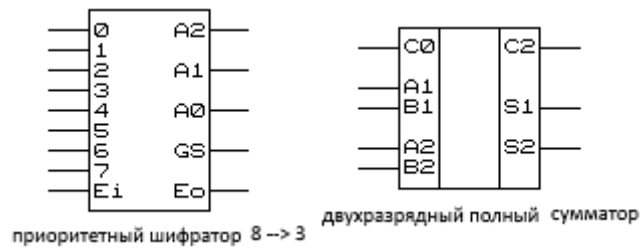
Мультиплексор 8-в-1, двоянный мультиплексор 4-в-1, дешифратор 4-в-16:



Схема управления семисегментным индикатором (дешифратор DC слева на рисунке):



Приоритетный шифратор (кодер) 8-в-3, двухразрядный полный сумматор (AB – входы, S – выход суммы, C0 – вход переноса, C2 – выход переноса):



### Интересные свойства функции XOR

Если сравнить таблицы истинности ИЛИ и «ИЛИ исключительного» (XOR), то можно увидеть, что отличаются они всего одной последней строкой. Но это небольшое отличие на самом деле очень существенно: XOR это не базовая функция (как ИЛИ-НЕ или И-НЕ), и используя уже, в свою очередь, XOR, как базу, можно получить много интересных решений, иногда для этого достаточно просто изменить интерпретацию входов.

| A | B | A или B | A | B | A xor B |
|---|---|---------|---|---|---------|
| 0 | 0 | 0       | 0 | 0 | 0       |
| 0 | 1 | 1       | 1 | 0 | 1       |
| 1 | 0 | 1       | 0 | 1 | 1       |
| 1 | 1 | 1       | 1 | 1 | 0       |

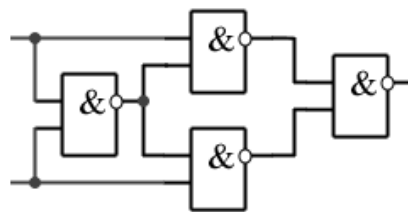
или

ИЛИ исключительное (XOR)

В алгебраической форме (ДНФ и КНФ) выражение для XOR выглядит просто:

- $X_1 * \bar{X}_2 + \bar{X}_1 * X_2$  – ДНФ (сумма произведений)
- $(X_1 + X_2) * (\bar{X}_1 + \bar{X}_2)$  – КНФ (произведение сумм)

На Рис.8 ниже приведён пример реализации XOR на одном из возможных базисов, 2И-НЕ (предлагается воспроизвести XOR на базисе 2ИЛИ-НЕ):



Или исключительное (XOR) на 2И-НЕ

Рис.8 - Реализация XOR в базисе И-НЕ.

Ниже перечислены некоторые функции, которые реализуются на XOR:

1. Цифровой одноразрядный компаратор – если числа на входе равны, на выходе будет 0, если входные числа разные – на выходе будет 1. Иногда этот функционал приводят как основное определение функции XOR.
2. Полусумматор. Опять-таки из таблицы истинности XOR видно, что она реализует арифметический одноразрядный полусумматор (полу- – потому, что XOR не формирует перенос в старший разряд)

$$\begin{aligned}
 0 + 0 &= 0 \\
 0 + 1 &= 1 \\
 1 + 0 &= 1 \\
 1 + 1 &= 10 \text{ (перенос в старший разряд)}
 \end{aligned}$$

Ниже, на Рис.9, приведена схема полного одноразрядного сумматора, с входом и выходом переноса.

3. Симметричное шифрование. Если число  $A$  два раза проXORить одним и тем же ключом (с разрядностью ключа равной разрядности входных данных), то мы получим исходное число  $A$ :

$$A \oplus K = C \text{ (зашифрованные данные)}$$

$$C \oplus K = A \text{ (расшифрованные данные)}$$

$$A \oplus K \oplus K = A$$

4. Управляемый инвертор. Функция XOR симметрична по отношению к входам, но можно для примера вход  $A$  назвать управляющим, а  $B$  – входом данных (выход обозначим как  $Y$ ). Тогда:

$$\text{Если } A = 0 \Rightarrow Y = B$$

$$\text{Если } A = 1 \Rightarrow Y = \bar{B}$$

5. Восстановление данных. Предположим мы пишем данные на два диска, половину на один диск ( $A$ ), половину на второй ( $B$ ) (например, чётные блоки пишем на  $A$ , нечётные – на  $B$ ), считаем их «продольную» сумму (сумма по модулю 2) и пишем её на третий (избыточный) диск  $S$ :

$$A \oplus B = S$$

Предположим, что вышел из строя диск с контрольной суммой ( $S$ ), данные не пострадали, мы имеем полный поток, собранный из двух «половинок». XOR здесь никак не проявляется. Если же выходит из строя один из дисков с данными, пусть это будет  $B$ , мы, воспользовавшись свойствами функции XOR, можем восстановить данные с отказавшего диска  $B$ :

$$A \oplus S = A \oplus A \oplus B = B$$

Т.е., одну «половину» данных (с диска  $A$ ) мы читаем напрямую, вторую – восстанавливаем с помощью XOR (суммы по модулю два, диск  $S$ ). Дисков с данными не обязательно должно быть ровно два (два – это минимум), обычно их собирают в группы до 8—12 дисков, из которых один диск выделяется под контрольную сумму. Такая технология носит название RAID<sup>1</sup>. Схема с выделенным диском для контрольной суммы (RAID3) в реальности используется редко (из-за повышенной нагрузки на диск с контрольной суммой), гораздо большее распространение получила технология RAID5, в которой контрольная сумма равномерно распределяется по всем дискам, т.е. каждый следующий блок XOR пишется на следующий диск.

---

<sup>1</sup> **RAID** (*Redundant Array of Independent Disks* — избыточный массив независимых (самостоятельных) дисков) — технология виртуализации данных для объединения нескольких физических дисковых устройств в единый логический модуль для повышения отказоустойчивости и производительности.

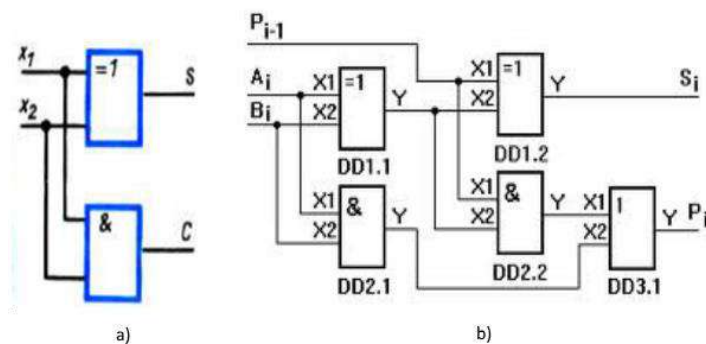


Рис.9 - Одноразрядные сумматоры: а – полусумматор; б – полный сумматор.

### Секвенциальная (последовательностная) логика

**Секвенциальная логика** — это логика памяти цифровых устройств. Название «секвенциальная» восходит к англ. *sequential*. Соответствующая логика может именоваться также как *последовательностная*, хотя последний термин чаще употребляется в связи с логическими автоматами.

Секвенциальная логика отличается от *комбинационной логики* тем, что моделирует цифровые устройства с учётом предыстории их функционирования (то есть предполагается наличие памяти, которой в комбинационной логике просто нет).

Секвенциальная логика является разделом дискретной математики. Она развивается в рамках теории цифровых схем в тесной связи с *комбинационной логикой*, *булевой алгеброй* и *конечными автоматами*<sup>1</sup>. В зависимости от регламента функционирования цифровые устройства подразделяются на синхронные и асинхронные. Соответственно их поведение подчиняется либо синхронной, либо асинхронной логике.

К секвенциальным (последовательностным) относятся следующие типы цифровых устройств:

1. Триггеры
2. Регистры
3. Счётчики
4. Запоминающие устройства (ЗУ)

### Триггеры

Триггером называется устройство, имеющее два устойчивых состояния и сохраняющее любое из них сколь угодно долго после снятия внешнего воздействия, вызвавшего переход триггера из одного состояния в другое. Поэтому говорят, что триггер обладает памятью. Триггер можно в общем случае представить состоящим из ячейки памяти и устройства управления (порой весьма сложного), преобразующего входную информацию в комбинацию сигналов, под воздействием которых ячейка памяти принимает одно из двух устойчивых состояний.

По способу записи информации триггеры могут быть *асинхронными* и *синхронными*. Триггер называют асинхронным, если сам сигнал, несущий информацию, вызывает его переключение. В синхронных (тактируемых) триггерах информация записывается при одновременном воздействии информационного сигнала и синхронизирующего (разрешающего) импульса. Синхронизация может осуществляться импульсом (потенциалом<sup>2</sup>) или перепадом потенциала (фронтом  или срезом  импульса). В первом случае (т.н. статическое управление) сигналы на информационных входах оказывают влияние на состояние триггера в течение всего времени наличия синхроимпульса. Во втором случае (динамическое

<sup>1</sup> **Конечный автомат** — абстрактный автомат, число возможных внутренних состояний которого конечно.

<sup>2</sup> Такие триггеры называются «защёлками» (англ. **latch**).

управление) воздействие информационных сигналов проявляется только в моменты изменения потенциала на входе синхронизации, т.е. при переходе его от 0 к 1 (фронт) или от 1 к 0 (срез).

По функциональному признаку различают **RS-триггеры**, **D-триггеры**, **T-триггеры** и самые универсальные **JK-триггеры**, а также их комбинации. Основу любого триггерного устройства составляет элементарная ячейка памяти на двух инверторах с петлёй положительной обратной связи.

**RS-триггер** – простейший тип триггеров, на основе которого можно создавать другие типы триггеров. Он может быть построен как на логических элементах 2ИЛИ-НЕ (прямые входы) или 2И-НЕ (инверсные входы).

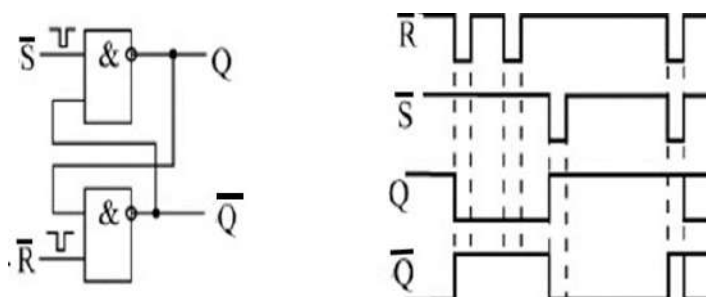


Рис.10 - RS-триггер на основе 2И-НЕ

В асинхронном (неактивируемом) RS-триггере (Рис.10), выполненном на логических элементах 2И-НЕ, состояниями ячейки памяти можно управлять, подавая логический 0 либо на вход  $\bar{S}$  (Set) для установки в состояние  $Q=1/\bar{Q}=0$ , либо на вход  $\bar{R}$  (Reset) для сброса в состояние  $Q=0/\bar{Q}=1$ . Если одновременно подать на оба входа логический 0 (оба выхода при этом перейдут в состояние  $Q=\bar{Q}=1$ ), а затем одновременно его снять (переключить в 1) с обоих входов, то состояние, в которое перейдут выходы триггера после такого синхронного переключения, будет неопределенным<sup>1</sup>. Такую комбинацию на входах триггера иногда называют запрещённой<sup>2</sup>. Минимальная длительность установочных импульсов должна вдвое превышать время задержки используемых логических элементов, чтобы по петле положительной обратной связи пришло подтверждение фиксации нового состояния триггера.

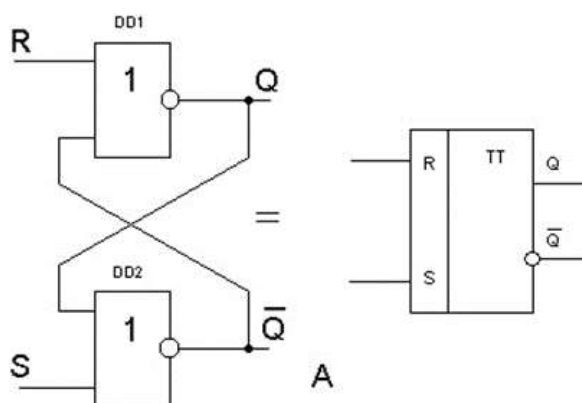


Рис.11 - RS-триггер на основе 2ИЛИ-НЕ

Для RS-триггера на базе логических элементов 2ИЛИ-НЕ (Рис.11) верны все приведённые выше высказывания с точностью до инверсии: активными уровнями входов будут

<sup>1</sup> В реальной (не «бумажной») схеме в итоге какой-то из выходов триггера перейдёт в полноценную 1, а второй выход, соответственно, в 0. В состоянии, когда оба выхода =1 устройство точно не останется, но какой выход конкретно в какое состояние перейдёт – не определено.

<sup>2</sup> В реальной схемотехнике ничто не запрещает подачу двух активных сигналов (0 в нашем примере) на оба входа одновременно, поэтому термин «запрещённый» применительно к этой ситуации не вполне адекватен.



логические 1, при одновременной подаче на входы R и S логической 1 выходы перейдут в состояние  $Q=\bar{Q}=0$ .

На Рис.12 приведена схема подавления дребезга контактов на основе асинхронного RS-триггера:

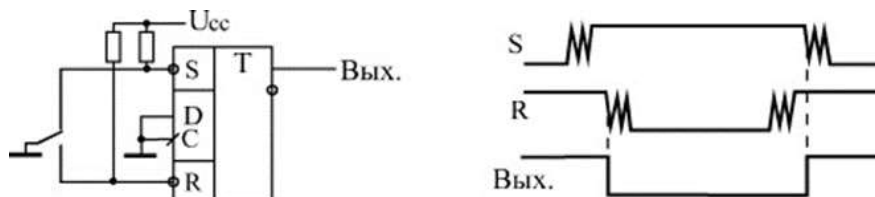


Рис.12 - Схема устранения влияния дребезжания контактов

**Тактируемый (синхронный) RS-триггер** (Рис.13) может изменить свое состояние только с приходом тактового импульса на вход  $C^1$ . Помехи, действующие на информационных входах R и S между тактовыми импульсами, не влияют на работу триггера. Запрещенным (неопределённым) является состояние  $CRS=1$ .

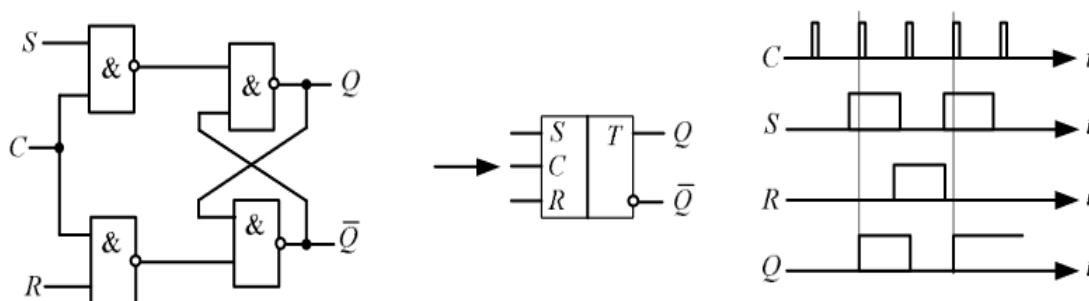


Рис.13 - Синхронный RS-триггер со статическим управлением

В триггерах с динамическим управлением переключение осуществляется по переднему ( $\nearrow$  переход из 0 в 1) или по заднему ( $\searrow$  переход из 1 в 0) фронту (срезу) синхросигнала C. В первом случае говорят, что у триггера прямой динамический вход (Рис.14, а), во втором случае – инверсный (Рис.14, б).

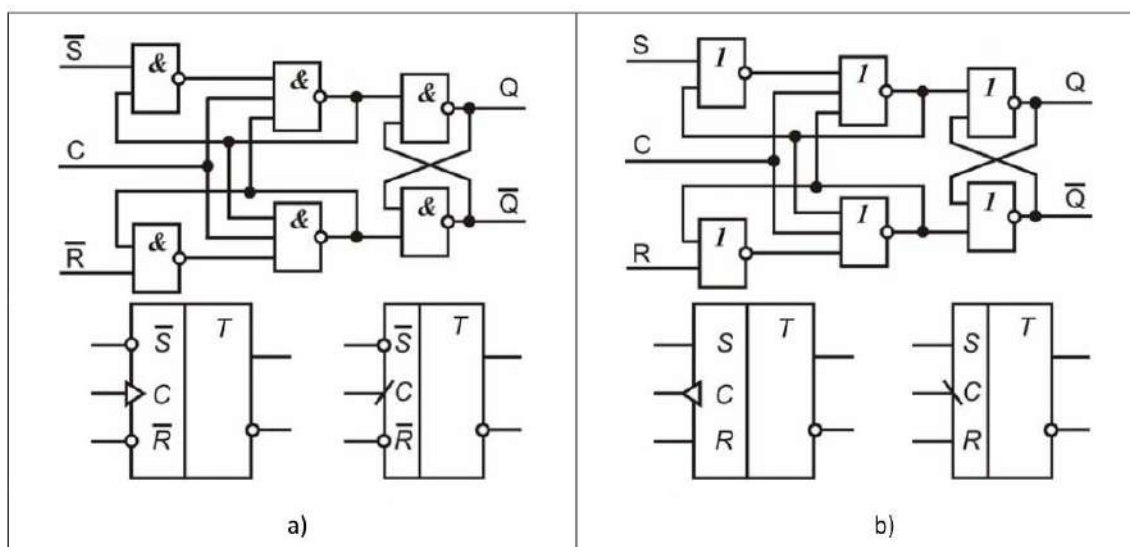


Рис.14 - RS-триггер а) с прямым динамическим входом; б) с инверсным динамическим входом

<sup>1</sup> C – от англ. Clock

**D-триггер** – (от английского **Delay** – задержка) имеет один информационный (D – Data – данные) и один тактируемый (C – Clock – тактовая последовательность) вход. Такой триггер можно получить из RS-триггера, подав на R-вход инвертированный сигнал с S-входа (Рис.15, а). Условное обозначение D-триггера со статическим управлением (latch) показано на Рис.15, б. Из временных диаграмм, приведенных на Рис.15, в) можно увидеть, что при  $C=1$  триггер работает как повторитель ( $Q = D$ ), а при переходе сигнала на входе C с уровня логической 1 к уровню логического 0 триггер «защёлкивает» (запоминает) текущее состояние входа D и переходит в режим хранения. В отличие от RS-триггера, D-триггер не имеет запрещенного (неопределённого) состояния.

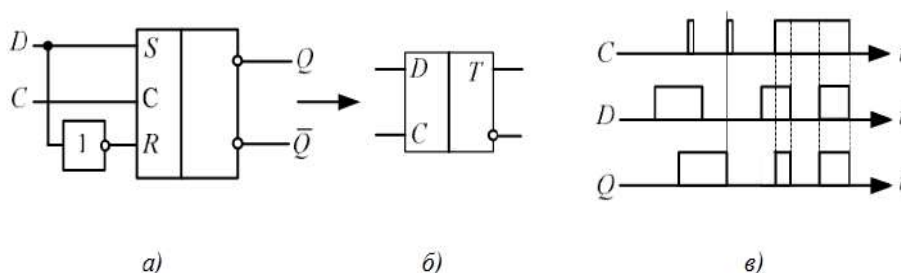


Рис.15 - D-триггер типа «защёлка» (latch): а – схема; б – обозначение; в – временная диаграмма

На Рис.16 приведён ещё один вариант построения D-триггера типа «защёлка» на базе 2И-НЕ вентиляей:

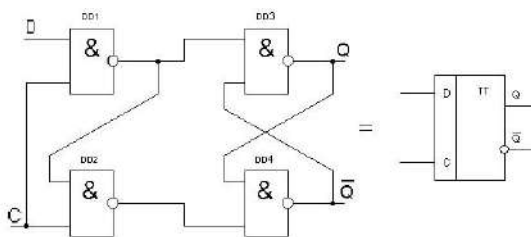


Рис.16 - Ещё один вариант D-триггера типа «защёлка»

**D-триггер с динамическим управлением** – отличается от статического D-триггера свойствами C(lock) входа. Запись информации происходит только в момент перехода тактового сигнала C из 0 в 1 ( $\nearrow$ ) (или в момент среза, перехода сигнала на C из 1 в 0 –  $\searrow$ ). При постоянном значении  $C=0$ ,  $C=1$  или отрицательном перепаде триггер не меняет состояние выхода и хранит предыдущую информацию. На Рис.17 приведена схема и обозначение комбинированного триггера: D-триггер + RS-триггер, с прямым динамическим входом C (Рис.17 б).

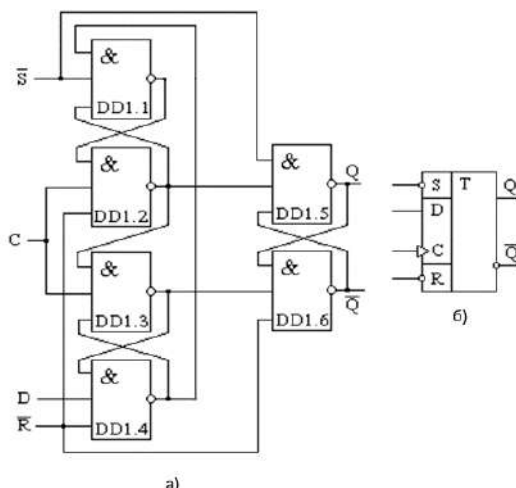


Рис.17 - Комбинированный динамический D-триггер +  $\overline{R}\overline{S}$  триггер (555TM2/7474)

D-триггер с динамическим управлением, тактируемый фронтом тактового импульса, можно выполнить по двухступенчатой схеме (т.н. схема master-slave или «ведомый-ведущий»), показанной на Рис.18, а). При  $C=0$  триггер первой ступени повторяет сигнал на входе D, но триггер второй ступени защёлкнут (находится в режиме хранения). При переходе к  $C=1$  первый триггер защёлкивает (фиксирует) на своем выходе информацию со входа D, а второй триггер начинает повторять её на своем выходе. Таким образом, двухступенчатый триггер по фронту тактового импульса защёлкивает на своем выходе уровень сигнала со входа D и сохраняет его до следующего фронта. Триггеры с динамическим управлением используются при построении счетчиков и сдвиговых регистров.

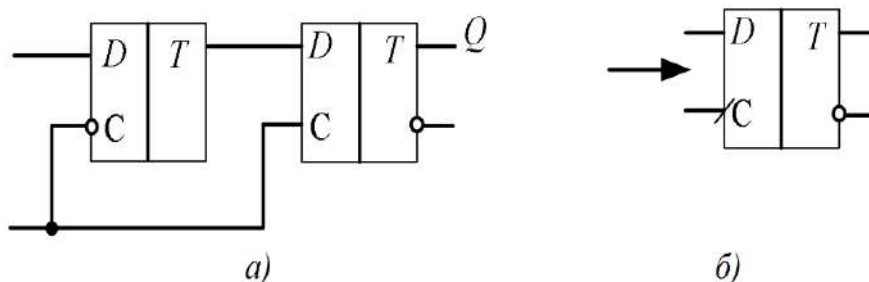


Рис.18 - Динамический D-триггер, построенный по двухступенчатой схеме.

**Т-триггер** – (от англ. Toggle — *переключатель*) называют счётным триггером, так как он является простейшим счётчиком по модулю 2. Он имеет только тактовый вход T и меняет свое состояние на противоположное по фронту (или срезу) каждого нового тактового импульса (Рис.19, г). На Рис.19 приведены схемы Т-триггера, выполненного на основе D-триггера – Рис.19, а) и RS-триггера – Рис.19, б) с прямым динамическим управлением. Как видно на Рис.19, г) частота изменения сигнала на выходе Т-триггера ровно в два раза меньше частоты импульсов на его тактовом входе, т.е., Т-триггер «умеет» считать до двух. Включённые последовательно N Т-триггеров, соответственно, будут делить частоту входного сигнала на  $2^N$ .

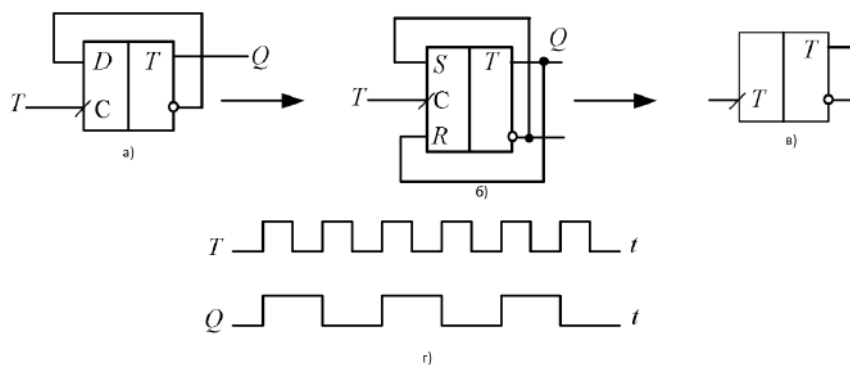


Рис.19 - Т-триггер на основе D-триггера – а); RS-триггера – б); обозначение Т-триггера – в); временная диаграмма – г).

**JK-триггер** – (от англ. J – Jerk – резкое движение, толчок; K – Kill – ликвидировать) наиболее универсальный триггер. Коммутацией входов его можно перевести в режим Т-триггера (синхронного или асинхронного), синхронного D-триггера. JK-триггер строится на базе RS-триггера с динамическим тактовым входом (Рис.20), но, в отличие от него, в JK-

триггере состояние информационных входов  $J=K=1$  не является запрещённым (неопределённым).

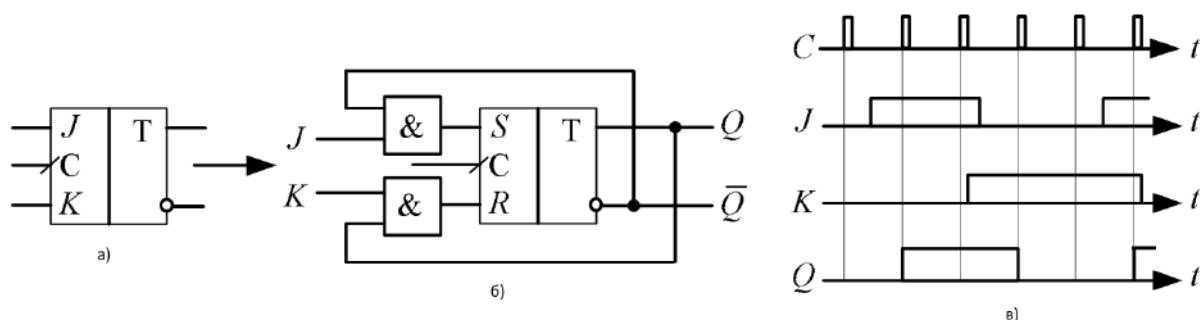


Рис.20 - JK-триггер на основе динамического RS-триггера

У JK-триггера два информационных входа (J и K) и вход синхронизации C. 1 на входе J устанавливает выход Q в единицу (аналог Set), 1 на входе K – в ноль (аналог Reset). Если на обоих входах 0 – триггер находится в режиме хранения, если на обоих входах 1 – работает в режиме T-триггера (по синхросигналу состояние выхода меняется на противоположное – Рис.20, в).

В качестве примера практической реализации JK-триггера на Рис.21 приведено обозначение и временная диаграмма микросхемы K555TB9 (74112) – это двойной комбинированный JK-триггер + RS-триггер.

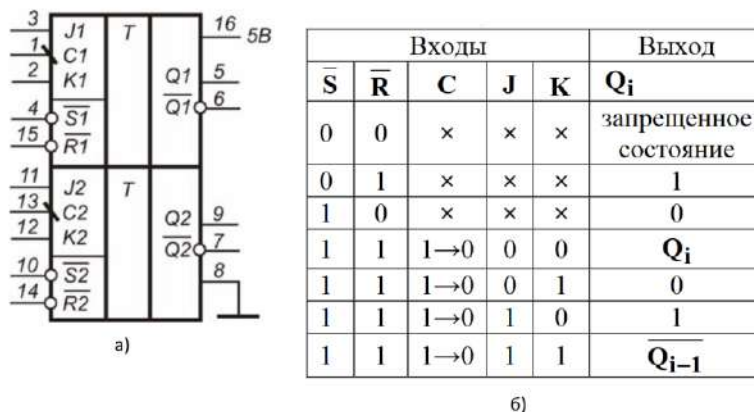


Рис.21 – а) двойной JK-триггер K555TB9; б) его таблица истинности.

На Рис.22 приведены варианты использования JK-триггера: а) асинхронный T-триггер; б) синхронный T-триггер; в) D-триггер.

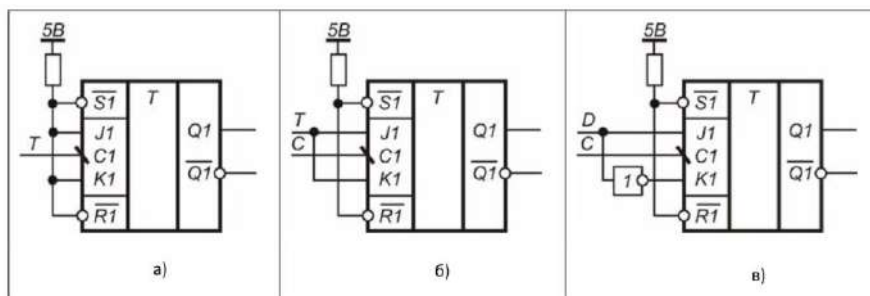


Рис.22 - JK-триггер в различных режимах: а) асинхронного T-триггера; б) синхронного T-триггера; в) D-триггера.

## Регистры

Регистры представляют собой цепочки триггеров и предназначены для записи, хранения, сдвига и считывания из них двоичной информации. Различают регистры *сдвиговые* (со сдвигом вправо, влево и реверсивные), *параллельные* (с параллельной загрузкой), *универсальные*, *кольцевые*, *регистровые файлы* (группы).

*Регистры сдвига* строятся на D-триггерах с динамическим управлением, по сути это цифровая линия задержки. Тактовые входы объединяются параллельно, вход D первого триггера служит для приема информации, а входы D последующих триггеров подключаются к прямым выходам предыдущих (Рис.23):

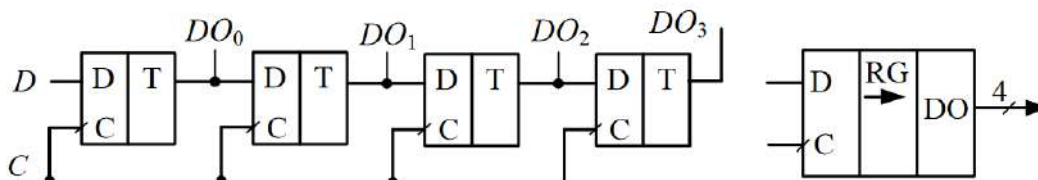


Рис.23 - Четырёхразрядный сдвиговый регистр

За четыре такта C (четыре синхроимпульса) последовательную информацию со входа D можно преобразовать в параллельную на выходах DO0–DO3.

Регистры с *параллельной загрузкой* также чаще всего строятся на D-триггерах, объединяя их тактовые входы. Параллельные регистры могут тактироваться уровнем (регистры «защёлки», latch) или фронтом/срезом. Пример n-разрядного параллельного регистра, срабатывающего по фронту на входе C:

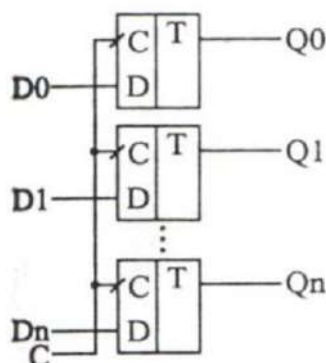


Рис.24 - n-разрядный параллельный регистр

*Универсальные регистры* – регистры, режимы работы которых можно изменять в процессе работы. Пример микросхемы универсального регистра K155ИР13 приведён на Рис.25:

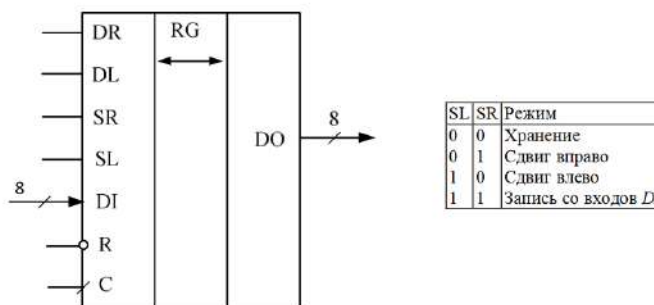


Рис.25 - Универсальный регистр K155ИР13

## Счётчики

Цифровое устройство, циклически меняющее свои состояния под действием импульсов, подаваемых на счётный вход, называется счётчиком. Количество тактов, через которое повторяется исходное состояние счетчика, называют коэффициентом пересчёта (модулем счета)  $K_{сч}$ . Счётчики строят из цепочек триггеров с динамическим управлением.

По коэффициенту пересчёта счётчики бывают двоичными –  $K_{сч}$  степень 2-ки, десятичными –  $K_{сч}$  степень 10-ти и с произвольным  $K_{сч}$ .

По направлению счёта счётчики делятся на суммирующие, вычитающие, реверсивные. По способу организации внутренних связей между триггерами счетчики могут быть асинхронными (с последовательным переносом) и синхронными (с параллельным переносом). Синхронные счетчики обладают большим быстродействием.

*Асинхронные* двоичные счётчики строят из цепочки счётных триггеров, соединяя выход предыдущего со входом последующего. Такой счетчик, например, был реализован в микросхеме К155ИЕ5 (Рис.26). При совпадении логических единиц на выходах 2 и 3 счётные триггеры сбрасываются в нулевое состояние. При соединении выводов 1 и 12 получаем четырёхразрядный двоичный счётчик. Частота счётных импульсов последовательно делится на два каждым триггером. При этом счётчик проходит 16 состояний (от 0 по 15), каждому из которых соответствует четырёхразрядный код на выходах с весовыми коэффициентами 8, 4, 2, 1.

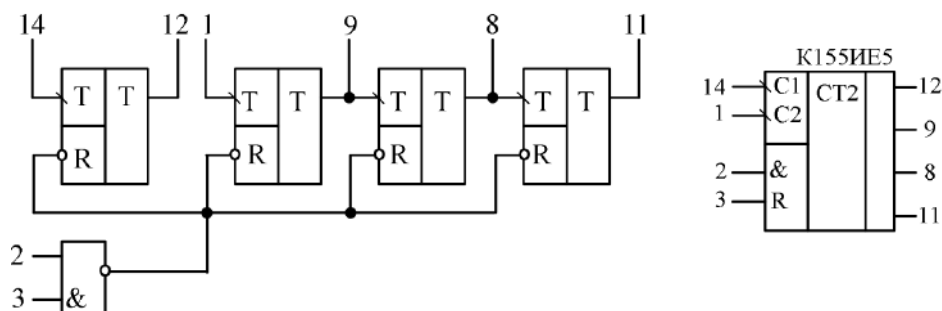


Рис.26 - Асинхронный двоичный четырёхразрядный счётчик на базе К155ИЕ5

В *синхронном* двоичном счётчике (Рис.27) счётный импульс воздействует сразу на все триггеры. Первый триггер работает как обычный счётный триггер ( $J=K=1$ ). Каждый последующий триггер меняет свое состояние на противоположное, когда все предыдущие находятся в единичном состоянии. Устройство реализует алгоритм работы суммирующего двоичного счетчика  $K_{сч} = 16$ .

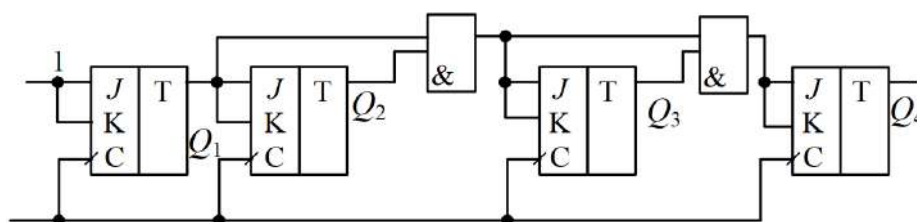


Рис.27 - Синхронный двоичный счётчик

*Реверсивные счётчики* могут работать на суммирование входных импульсов (суммирующие счётчики) или на вычитание.

*Кольцевой счётчик Джонсона* – по сути представляет собой регистр сдвига с обратной связью, с подачей сигнала на вход с инверсного выхода последнего триггера,  $K_{сч} = 2 * N$  ( $N$  – количество триггеров в кольце). Пример пятиразрядного –  $K_{сч} = 2 * 5 = 10$  – счётчика Джонсона приведён на Рис.28:

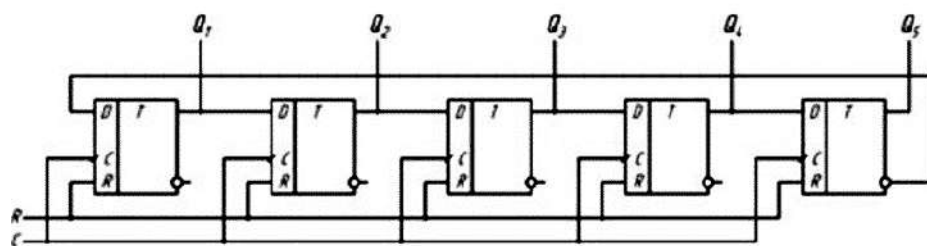


Рис.28 - Пятиразрядный счётчик Джонсона

Как видно, при счёте сначала от первого разряда до последнего распространяется волна единиц, а затем волна нулей («шторка»). В счётчике Джонсона, как и в других кольцевых счётчиках, возможны сбои в виде лишних волн нулей или единиц. При сбое, потеряв единицу, схема сама не исправляется, но если внести корректирующую цепь, то получится схема с самовосстановлением, устойчивая к случайным сбросам триггеров (Рис.29).

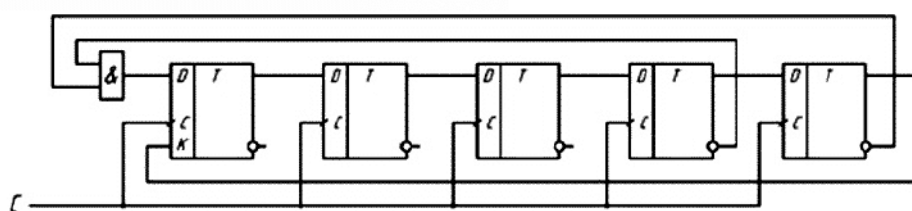


Рис.29 - Кольцевой счётчик с корректирующей цепью

## Запоминающие устройства

*Запоминающие устройства (ЗУ)* различаются по функциональному признаку: постоянные запоминающие устройства (ПЗУ), хранящие информацию, предназначенную только для чтения и оперативные запоминающие устройства (ОЗУ), предназначенные для записи, хранения и считывания цифровой информации.

ПЗУ относятся к *комбинационным* цифровым устройствам, ОЗУ относятся к цифровым устройствам последовательного типа. ПЗУ сохраняют информацию при отключенном питании, т.е. обладают свойством энергонезависимости, в ОЗУ информация теряется при отключении питания.

*Постоянные запоминающие устройства (ПЗУ).* ПЗУ по принципу работы являются преобразователями  $n$ -разрядного кода адреса ячейки  $A$  в  $m$ -разрядный код хранящегося в ней слова  $D$  (Рис.30). Данные считываются при подаче разрешающего уровня на вход  $CS$ .

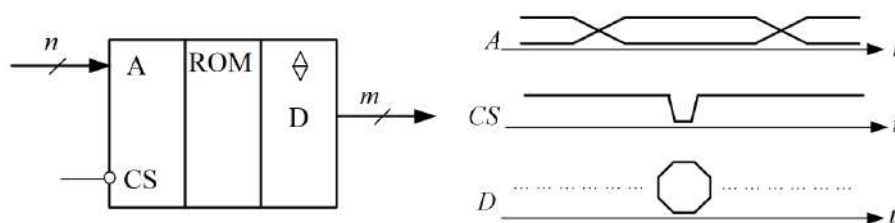


Рис.30 – ПЗУ

По способу занесения информации в ПЗУ различают:

- *масочные ПЗУ*, в которых перемычки в накопителе формируются в заводских условиях на заключительной стадии изготовления микросхем памяти;



- *программируемые ПЗУ*, в которых пользователь имеет возможность с помощью специального устройства (программатора) один раз осуществить пережигание плавких перемычек, исходя из собственной программы или кода;
- *репрограммируемые (многократно программируемые) ПЗУ*, допускающие многократное (сотни и тысячи циклов) перепрограммирование (EEPROM, Flash), которые в свою очередь подразделяются на РПЗУ с электрическим программированием и ультрафиолетовым стиранием (РПЗУ УФ или EPROM), с электрическим программированием и электрическим стиранием информации (РПЗУ ЭС или EEPROM).

По способу хранения информации выделяют две разновидности ОЗУ:

- *статическое ОЗУ*, в которых состояние элементов памяти при хранении информации остается неизменным. Статическое ОЗУ строится на транзисторах (Рис.31);
- *динамическое ОЗУ*, в котором состояние элементов памяти (обычно полупроводниковых емкостей) не остается неизменным и требует периодического выполнения процесса регенерации (восстановления) исходных уровней сигналов (Рис.33).

По технологии выполнения полупроводниковые ЗУ можно разделить на следующие виды:

- на основе полевых транзисторов с изолированным затвором: *p*-МОП, *n*-МОП и КМОП (Рис.31);
- на основе биполярных структур, использующих схемотехнику ТТЛ, ЭСЛ и др. (Рис.32);
- на основе приборов с зарядовой связью.

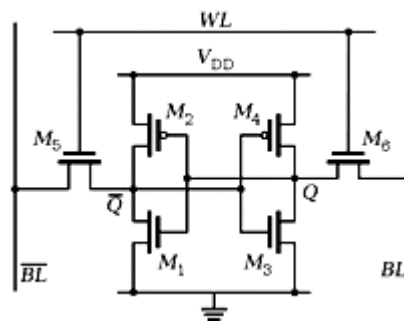


Рис.31 - Ячейка статического ОЗУ на КМОП транзисторах

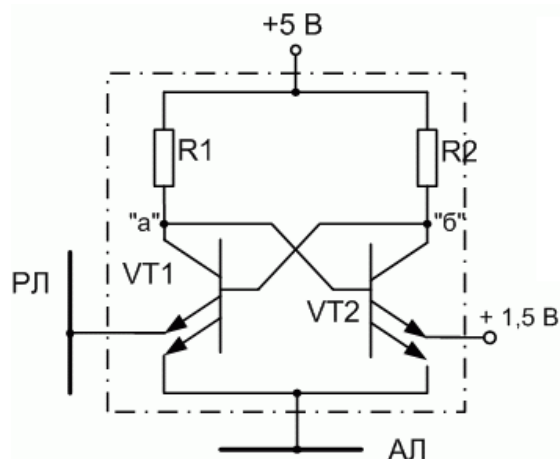


Рис.32 - Ячейка статического ОЗУ на биполярных транзисторах.



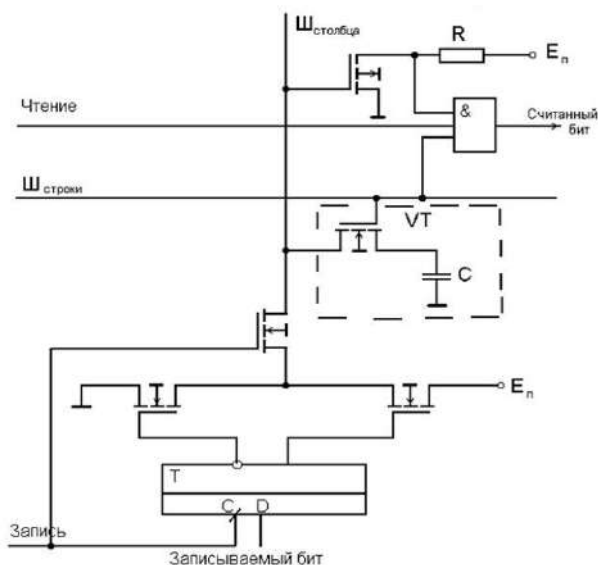


Рис.33 - Схема 3У с однотранзисторным запоминающим элементом.

На Рис.34 приведён упрощённая структура схемы динамической оперативной памяти:

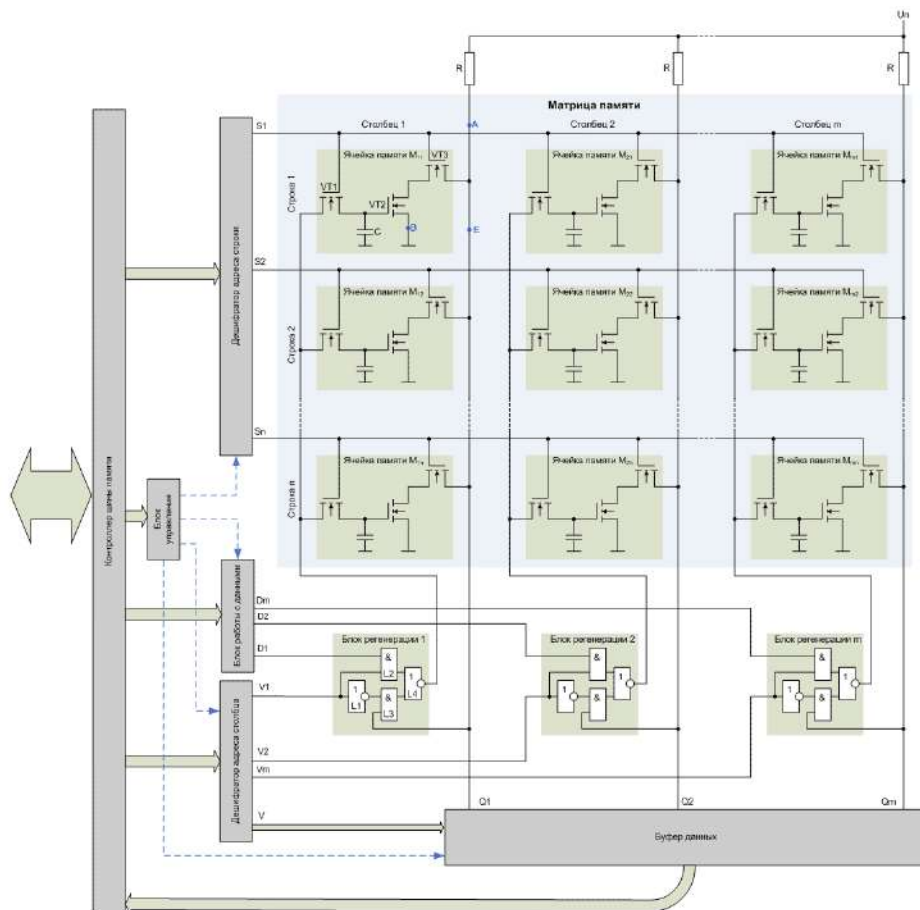


Рис.34 - Упрощённая структура динамической оперативной памяти.

### Технологии цифровых логических микросхем

16 декабря 1947 года физик-экспериментатор *Уолтер Браттейн*, работавший с теоретиком *Джоном Бардином*, собрал первый работоспособный точечный транзистор. Спустя полгода, но до обнародования работ *Бардина* и *Браттейна*, немецкие физики *Герберт Матаре* и *Генрих Велькер* представили разработанный во Франции точечный транзистор

(«транзистрон»). Так из безуспешных попыток создать сначала твердотельный аналог вакуумного триода, а затем *полевой транзистор*, родился первый несовершенный точечный *биполярный транзистор*.

Точечный транзистор, выпускавшийся серийно около десяти лет, оказался тупиковой ветвью развития электроники — ему на смену пришли *германиевые* плоскостные транзисторы. Теорию *p-n-перехода* и плоскостного транзистора создал в 1948—1950 годах *Уильям Шокли*. У *Уолтера Браттейна* и *Джона Бардина* с одной стороны, и *Уильяма Шокли* с другой, были довольно непростые взаимоотношения. После присуждения этому трио Нобелевской премии по физике за 1956 год отношения *Шокли* и *Бардина* отчасти нормализовались. Четвёртый основной соавтор изобретения, *Роберт Джинни*, ушёл из Bell Labs в начале 1948 года и Нобелевской премии не получил.

Первый плоскостной транзистор был изготовлен 12 апреля 1950 года методом выращивания из расплава. За ним последовали сплавной транзистор, «электрохимический» транзистор и диффузионный меза-транзистор.

На заре производства транзисторов основным полупроводниковым материалом был германий (Ge). В плане энергозатрат он весьма экономичен, напряжение отпирания его *p-n-перехода* составляет всего 0,1...0,3В, но многие его параметры были весьма нестабильны, и, со временем, на смену ему пришел кремний (Si). Германиевые транзисторы значительно уступали кремниевым в быстродействии, имея значительно более низкие предельные рабочие частоты. Также полупроводниковые приборы на основе германия намного хуже переносили высокие температуры (нагрев), уступая по этому параметру кремниевым изделиям десятки градусов.

В 1954 году компания *Texas Instruments* выпустила первый *кремниевый* транзистор. Открытие процесса мокрого окисления кремния сделало возможным выпуск в 1958 году первых кремниевых меза-транзисторов, а в марте 1959 года *Жан Эрн* создал первый кремниевый *планарный транзистор*. Кремний вытеснил германий, а планарный процесс стал основной технологией производства транзисторов и сделал возможным создание *монолитных интегральных схем*<sup>1</sup>.

Транзисторы по структуре, принципу действия и параметрам делятся на два класса — биполярные и полевые (униполярные). В *биполярном транзисторе* используются полупроводники с обоими типами проводимости, он работает за счет взаимодействия двух, близко расположенных на кристалле, *p-n переходов* и управляется изменением тока через переход база-эмиттер, при этом вывод эмиттера всегда является общим для управляющего и выходного токов. В *полевом транзисторе* используется полупроводник только одного типа проводимости, расположенный в виде тонкого канала, на который воздействует электрическое поле изолированного от канала затвора, управление осуществляется изменением напряжения между затвором и истоком. Полевой транзистор, в отличие от биполярного, управляется напряжением, а не током. В настоящее время в аналоговой технике доминируют *биполярные транзисторы* (БТ) (международный термин — BJT, bipolar junction transistor). В *цифровой технике*, в составе микросхем (логика, память, процессоры, компьютеры, цифровая связь и т. п.), напротив, биполярные транзисторы почти полностью вытеснены полевыми. В 1990-е годы был разработан новый тип гибридных биполярно-полевых транзисторов — IGBT, которые сейчас широко применяются в силовой электронике.

Интересный факт: полевой транзистор в каком-то смысле проще биполярного — и по физическим принципам работы, и по технологии. Очень примерная аллегория: принцип работы полевого транзистора можно описать, как регулировку потока жидкости через трубу (шланг) путём изменения диаметра трубы (подачи запирающего напряжения на затвор).

---

<sup>1</sup> Не существует единого мнения о том, кто именно является изобретателем ИС. Американская пресса 1960-х годов признавала изобретателями ИС четырёх человек: Килби, Леговца, Нойса и Эрн. В 1970-е годы список изобретателей сократился до двух имён: Килби и Нойс, а в популярной литературе — до одного Килби. Именно Килби был удостоен в 2000 году Нобелевской премии по физике «за личный вклад в изобретение интегральной схемы».

Первый патент на принципы работы полевого транзистора были получены в 1925-ом году (автор патента – австро-венгерский физик *Юлий Эдгар Лилиенфельд*).

Но первый работоспособный полевой транзистор был создан уже после открытия биполярного транзистора. В 1952 году *Уильям Шокли* теоретически описал модель полевого транзистора другого типа, модуляция тока в котором, в отличие от ранее предложенных МДП<sup>1</sup> (МОП) структур, осуществлялась изменением толщины проводящего канала за счёт расширения или сужения обеднённой области, прилегающего к каналу р-п-перехода. Это происходило при подаче на переход управляющего напряжения запирающей полярности затворного диода. Транзистор получил название «полевой транзистор с управляющим р-п-переходом» (мешающие работе поверхностные явления устранялись, так как проводящий канал находился внутри кристалла).

Первый полевой МДП-транзистор, запатентованный ещё в 1920-е годы и сейчас составляющий основу компьютерной индустрии, впервые был создан в 1960 году после работ американцев *Канга* и *Аталлы*, предложивших в качестве слоя затворного диэлектрика формировать на поверхности кремниевого кристалла с помощью окисления поверхности кремния тончайший слой *диоксида кремния*, изолирующий металлический затвор от проводящего канала, такая структура получила название МОП-структура (Металл-Оксид-Полупроводник).

В 90-х годах XX века МОП-технология стала доминировать над биполярной.

## Резисторно-транзисторная логика

**Резисторно-транзисторная логика (РТЛ)** — это первая технология построения логических цифровых электронных схем, основанная на простых биполярных транзисторных ключах.

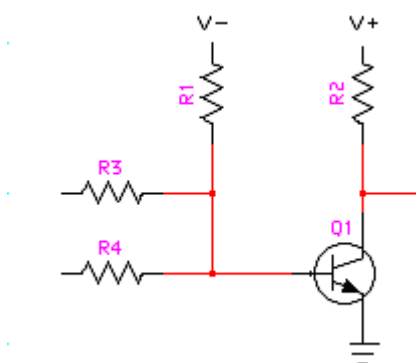


Рис.35 - Элемент 2ИЛИ-НЕ на основе РТЛ (в позитивной логике)

Коллектор транзистора (ключа) соединён через резистор с шиной питания (как правило +3.15В — половина накала лампы), а эмиттер с корпусом. К базе подключены резисторы, являющиеся входами.

При отсутствии напряжения на всех входах транзистор закрыт и на выход через резистор поступает напряжение близкое к напряжению питания, то есть логическая единица на выходе при нулях на входе при позитивной логике. При наличии положительного напряжения хотя бы на одном из входов ключ открывается и закорачивает выход на корпус. При этом он «просаживает» выходное напряжение почти до нуля.

Таким образом РТЛ-элемент выполняет функцию ИЛИ-НЕ в позитивной логике или И-НЕ в инверсной.

<sup>1</sup> МДП-транзистор – металл, диэлектрик, полупроводник. Иногда говорят МОП-транзистор – металл, окисел, полупроводник.

## Резисторно-емкостная транзисторная логика (РЕТЛ)

Резисторно-емкостная транзисторная логика аналогична РТЛ, но в ней есть конденсаторы на входах, пропускающие только импульсы. Как правило предусматривается возможность использовать РЕТЛ-элемент как РТЛ. Таким образом чёткой грани между ними нет. Просто РЕТЛ более «интегрирована».

### Достоинства и недостатки РТЛ и РЕТЛ

Достоинства:

- Конструктивная простота;
- Низкая стоимость.

Недостатки:

- Высокая рассеиваемая мощность (как на открытом ключе, так и на резисторах);
- Нечёткий уровень сигналов (уровень единицы от  $\sim 0,9V$  до напряжения питания);
- Крайне низкое быстродействие;
- Низкая помехоустойчивость;
- Сложность разработки;
- Низкая нагрузочная способность выходов (обычно не более трёх входов других элементов).

## Диодно-транзисторная логика

Диодно-транзисторная логика (ДТЛ) — технология построения цифровых схем на основе биполярных транзисторов, диодов и резисторов. Своё название технология получила благодаря реализации логических функций (например, И) с помощью диодных цепей, а усиления сигнала — с помощью транзистора.

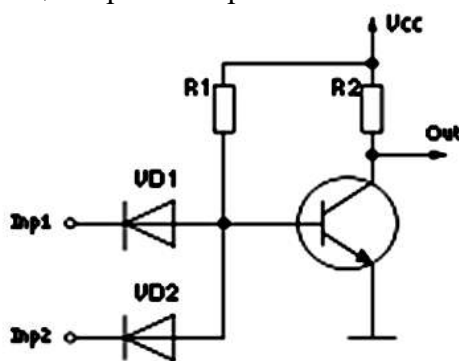


Рис.36 - Упрощённая схема двухвходового ДТЛ-элемента 2И-НЕ

Показанная на рисунке схема представляет собой типичный элемент 2И-НЕ: если хотя бы на одном из входов уровень логического 0, то ток через R1 течет через диод во входную цепь. На анодах диодов напряжение 0.7В, которого недостаточно для открывания транзистора, для того чтобы его ввести в режим насыщения. На выходе при этом формируется уровень логической “1”.

Если на все входы поступает уровень логической единицы, ток через R1 втекает в базу транзистора. Диоды оказываются заперты (обратное смещение) и не участвуют в работе схемы. Транзистор при этом открыт полностью (насыщение) и на выходе Out формируется низкое напряжение, соответствующее уровню логического “0”.

## Достоинства и недостатки ДТЛ

Основное преимущество ДТЛ над более ранней технологией РТЛ — возможность создания большого числа входов. Задержка прохождения сигнала по-прежнему достаточно велика (транзистор включён по схеме с общим эмиттером), из-за медленного процесса рассасывания заряда в базе при переходе из режима насыщения транзистора (когда все входы имели высокий уровень) в режим закрытого транзистора (при подаче на один из входов низкого уровня). Эту задержку можно уменьшить подключением базы транзистора через резистор к общему проводу или к источнику отрицательного напряжения.

В более современной и эффективной технологии ТТЛ данная проблема решена путём замены диодов на многоэмиттерный транзистор. Это также уменьшает площадь кристалла (в случае реализации в виде интегральной схемы), и соответственно позволяет добиться более высокой плотности элементов.

## Транзисторно-транзисторная логика

**Транзисторно-транзисторная логика (ТТЛ, TTL)** — разновидность цифровых логических микросхем, построенных на основе биполярных транзисторов и резисторов. Название *транзисторно-транзисторный* возникло из-за того, что транзисторы используются как для выполнения логических функций (например, **И**, **ИЛИ**), так и для усиления выходного сигнала (в отличие от резисторно-транзисторной и диодно-транзисторной логики).

Простейший базовый элемент ТТЛ выполняет логическую операцию **И-НЕ**, в принципе повторяет структуру ДТЛ микросхем и в то же время за счёт использования многоэмиттерного транзистора, объединяет свойства диода и транзисторного усилителя, что позволяет увеличить быстродействие<sup>1</sup>, снизить потребляемую мощность и усовершенствовать технологию изготовления микросхемы.

ТТЛ получила широкое распространение в компьютерах, электронных музыкальных инструментах, а также в контрольно-измерительной аппаратуре и автоматике (КИПиА). Благодаря широкому распространению ТТЛ входные и выходные цепи электронного оборудования часто выполняются совместимыми по электрическим характеристикам с ТТЛ. Максимальное напряжение в схемах с ТТЛ может достигать 24 В, однако, это приводит к большому уровню паразитного сигнала. Достаточно малый уровень паразитного сигнала при сохранении достаточной эффективности достигается при напряжении 5 В, поэтому данная цифра и вошла в технический регламент ТТЛ.

ТТЛ стала популярной среди разработчиков электронных систем после того, как в 1965 году фирма *Texas Instruments* представила серию интегральных микросхем 7400. Данная серия микросхем стала промышленным стандартом, но ТТЛ-микросхемы производились и другими компаниями. Более того, фирма *Texas Instruments* не была первой, кто начал выпуск ТТЛ-микросхем, несколько ранее его начали фирмы *Sylvania* и *Transitron*. Тем не менее промышленным стандартом стала именно серия 74 фирмы *Texas Instruments*, что в значительной мере объясняется большими производственными мощностями фирмы *Texas Instruments*, а также её усилиями по продвижению 74-ой серии. Поскольку биполярные ИМС серии 74 фирмы *Texas Instruments* стали наиболее распространёнными, их функционально и параметрически повторяет продукция других фирм (*Advanced Micro Devices*, серия 90/9N/9L/9H/9S *Fairchild*, *Harris*, *Intel*, *Intersil*, *Motorola*, *National* и многих других).

Важность ТТЛ заключается в том, что ТТЛ-микросхемы оказались более пригодны для массового производства и при этом превосходили по параметрам ранее выпускавшиеся серии микросхем (резисторно-транзисторная и диодно-транзисторная логика).

---

<sup>1</sup> Входной многоэмиттерный транзистор включён по схеме с общей базой, что позволяет транзистору работать на максимальной (граничной) частоте переключения, которая в схеме с общей базой примерно в  $\beta$  раз выше, чем в схеме с общим эмиттером.

## Принцип работы ТТЛ

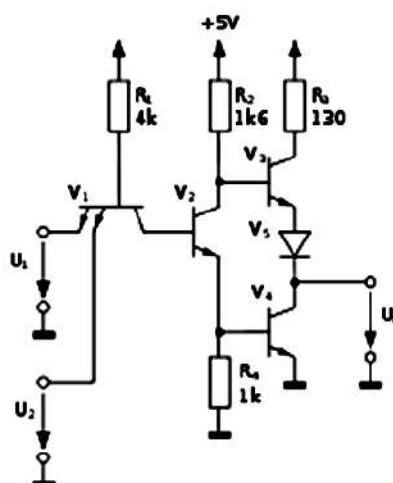


Рис.37 - Принципиальная схема ТТЛ элемента 2И-НЕ

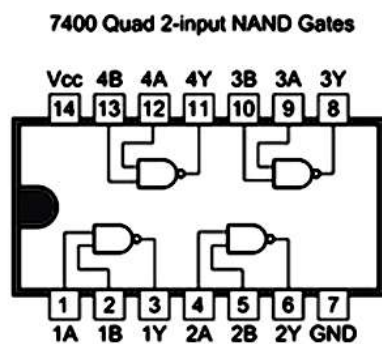


Рис.38 - Цоколёвка первого ТТЛ элемента 2И-НЕ семейства 7400

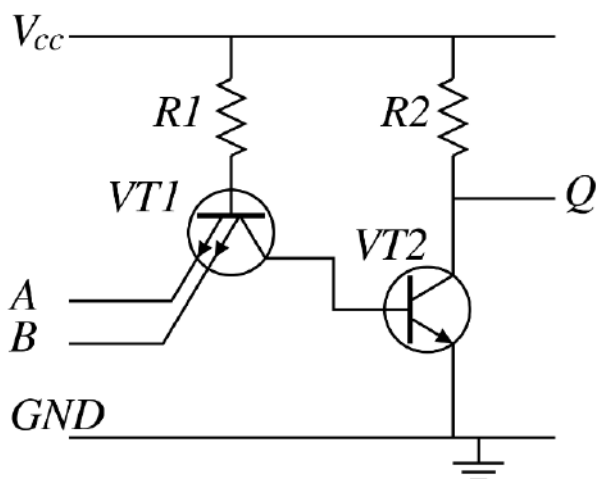


Рис.39 - Упрощённая схема элемента 2И-НЕ

### Принцип работы ТТЛ с простым инвертором

Биполярные транзисторы могут работать в нескольких режимах: отсечки, насыщения, нормально активный, инверсно активный. В инверсно активном режиме эмиттерный переход закрыт, а коллекторный переход открыт. В инверсном активном режиме коэффициент усиления транзистора по току значительно меньше, чем в нормальном режиме, из-за несимметричности конструктивного исполнения переходов база-коллектор и база-эмиттер, в частности, из-за разницы в их площадях и степени легирования коллекторного и эмиттерного слоёв полупроводника.

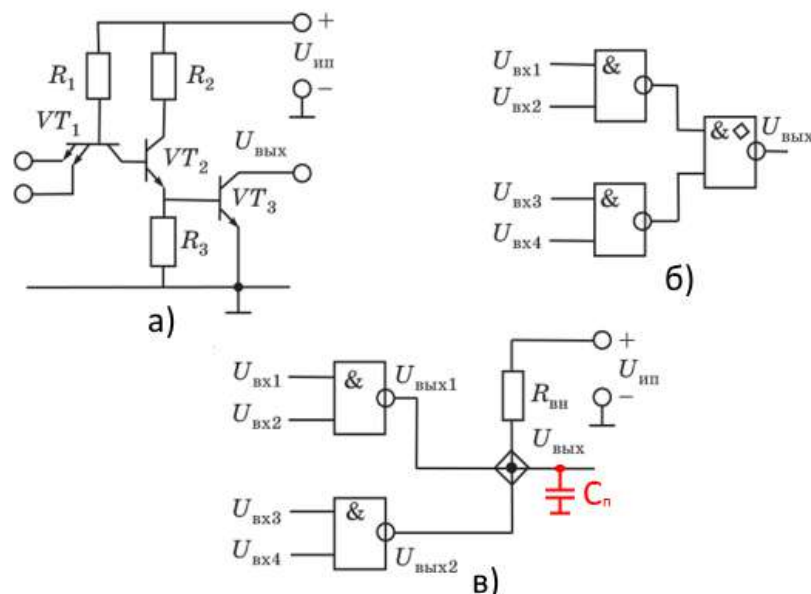
При нулевом потенциале на любом эмиттере многоэмиттерного транзистора VT1 он работает в нормальном режиме насыщения, так как в базу втекает ток резистора R1 (Рис.39), поэтому потенциал коллектора VT1 и базы VT2 близок к нулю ( $V_{бэ1} = (A|B=0) + 0,7В \approx V_{бк1} + V_{бэ2}$ ), что переводит VT2 в режим отсечки, поэтому на коллекторе VT2 потенциал близок к потенциалу источника питания  $V_{cc}$ , — на выходе элемента логическая “1”. В этом состоянии изменение потенциала другого эмиттера не изменяет состояние элемента. Через эмиттер (вход), присоединённый к «земле» на землю вытекает ток  $I = (V_{cc} - 0,7)/R1$ , 0,7 В — падение напряжения на прямосмещённом эмиттерном переходе VT1.

Если отключить все эмиттеры, или подать на них напряжения логической “1” (более 2,4 В), то через прямосмещённый коллекторный переход VT1 в базу VT2 будет втекать ток резистора R1,  $I = (V_{cc} - 1,4)/R1$ , 1,4 В — сумма падений напряжений на прямосмещённом эмиттерном переходе VT2 и прямосмещённом коллекторном переходе VT1, при этом VT2 переходит в состояние насыщения, его коллекторный потенциал становится близок к нулю (логический “0”).

Таким образом, на выходе будет логический “0” только если все входы имеют состояние логической “1”, это соответствует логической функции И-НЕ.

ТТЛ имеет повышенное, по сравнению с ДТЛ-логикой быстродействие, даже если используемые транзисторы имеют равное быстродействие. Это обусловлено тем, что при переходе выхода из состояния логического нуля в логическую “1” транзистор выходит из насыщения, неосновные носители, накопленные в базе VT2, не только самопроизвольно рассасываются, но и стекают в коллектор насыщенного VT1 (как было ранее сказано, его потенциал близок к нулю). Типовая задержка на элемент ранних серий ТТЛ-микросхем около 22 нс.

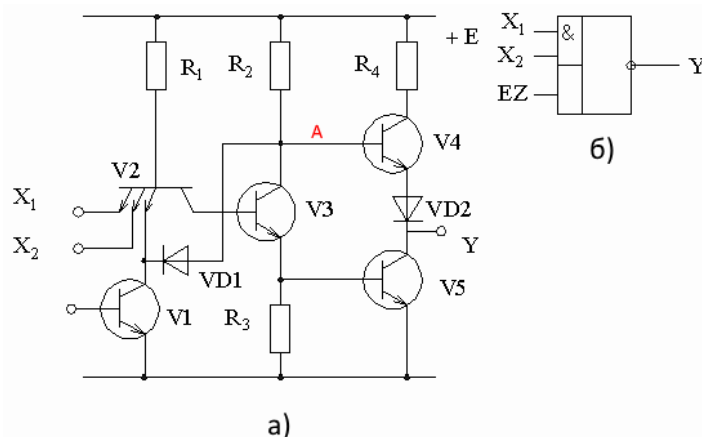
Некоторые микросхемы в каждой серии ТТЛ изготавливаются без резистора R2, на выход выведен коллектор VT2, так называемые элементы с «открытым коллектором». Группу таких выходов можно электрически соединить, снабдив единственным внешним резистором, другим концом присоединённом к  $V_{cc}$ , при этом реализуется логическая функция «И» — такое соединение иногда называют «проводным И» (чаще — «монтажным ИЛИ» в инверсной логике (Комбинационная логика)). На электрических принципиальных схемах в условном обозначении элементов с открытым коллектором используется дополнительный символ (Рис.40, б). На Рис.40, а) приведена принципиальная схема логического ТТЛ-элемента 2И-НЕ с открытым коллектором, а на Рис.40, в) приведена реализация монтажной (проводной) логики, выполненная путём объединения *выходов* логических элементов. Для формирования высокого уровня необходим внешний нагрузочный резистор  $R_{вн}$ . Такое объединение позволяет выполнить пространственно распределённое *мультиплексирование* сигналов (шина).



**Рис.40** - Логический элемент 2И-НЕ с открытым коллектором

Т.к., переход из “0” в “1” на выходе в схеме с ОК формируется пассивными элементами, фронт получается затянутым (экспонента с постоянной времени  $R_{вн}C_{п}$ ).

Более качественный (более короткий) положительный фронт при объединении выходов позволяют формировать схемы с тремя состояниями (tri-stable) выхода (Рис.41).



**Рис.41** – Логический элемент 2И-НЕ с тремя состояниями на выходе

Третье состояние (не путать с троичной логикой) достигается переводом обоих выходных транзисторов ( $V4$  и  $V5$ ) в закрытое высокоимпедансное Z-состояние. Управляет третьим состоянием выхода дополнительная цепь: транзистор  $V1$  и диод  $VD1$  (Рис.41 а)). При подаче уровня логического “0” на вход базы  $V1$ , этот транзистор будет закрыт и не оказывает никакого влияния на работу остальной схемы, реализующей функцию 2И-НЕ. При подаче на базу  $V1$  уровня логической “1” транзистор откроется и в точке **А** будет напряжение близкое к нулю, что приведёт к закрытию верхнего транзистора  $V4$ . Ток через транзистор  $V3$ , следовательно, в базе  $V5$ , будет близок к нулю, и транзистор  $V5$  тоже закроется. Таким образом оба выходных транзистора  $V4/V5$  будут одновременно закрыты при любых уровнях входных сигналов на входах  $X1/X2$ . Это и есть третье высокоимпедансное состояние.

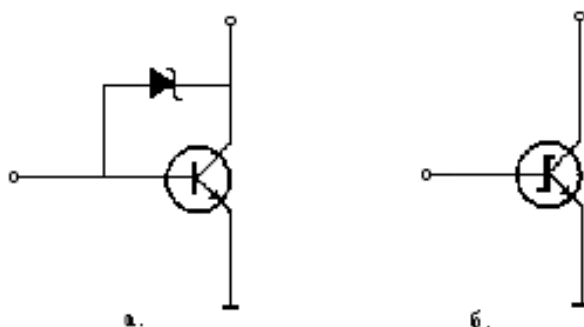
ТТЛ-логика (как и ТТЛШ) является прямым наследником ДТЛ и использует тот же принцип действия. Входной ТТЛ-транзистор (в отличие от обычного) имеет несколько,



обычно от 2 до 8, эмиттеров. Эти эмиттеры выполняют роль входных диодов (если сравнивать с ДТЛ). Многоэмиттерный транзистор по сравнению с применявшейся в схемах ДТЛ сборкой из отдельных диодов занимает меньше места на кристалле и обеспечивает более высокое быстродействие. Следует отметить, что в микросхемах ТТЛШ, начиная с серии 74LS, вместо многоэмиттерного транзистора используется сборка диодов Шоттки (серия 74LS) или PNP транзисторы в сочетании с диодами Шоттки (серии 74AS, 74ALS), так что фактически произошёл возврат к ДТЛ. Название ТТЛ заслуженно носят лишь серии 74, 74Н, 74L, 74S, содержащие многоэмиттерный транзистор. Все более поздние серии многоэмиттерного транзистора не содержат, фактически являются ДТЛ и носят название ТТЛШ (ТТЛ Шоттки) лишь «по традиции», будучи развитием именно диодно-транзисторной логики.

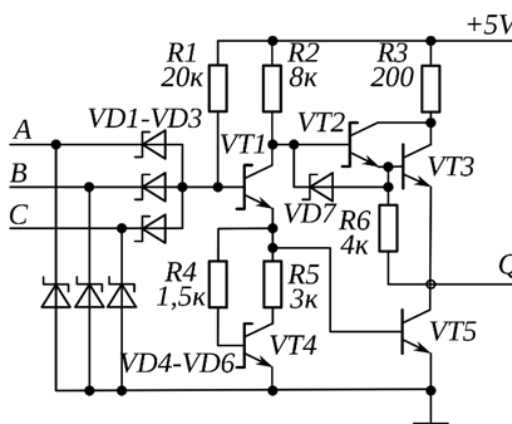
### **Транзисторно-транзисторная логика с диодами Шоттки (ТТЛШ)**

В ТТЛШ используются диоды Шоттки, в которых барьер Шоттки не позволяет транзистору войти в режим насыщения, в результате чего диффузионная ёмкость относительно небольшая, задержки переключения малы, быстродействие высокое. Такая комбинация (биполярный транзистор-диод Шоттки в цепи база-коллектор (Рис.42, а)) считается отдельным компонентом — транзистором Шоттки — и имеет собственное обозначение на электрических принципиальных схемах (Рис.42, б).



**Рис.42** - Транзистор, в котором переход база-коллектор шунтируется диодом Шоттки

На Рис.43 приведена принципиальная схема логического элемента 3И-НЕ ТТЛ с диодами Шоттки:



**Рис.43** - Логический элемент 3И-НЕ в серии микросхем 74LS(K555)

ТТЛШ-логика отличается от классической ТТЛ наличием диодов Шоттки в цепях база — коллектор, что исключает насыщение транзистора, а также наличием демпфирующих диодов Шоттки на входах (редко на выходах) для подавления импульсных помех, об-

разующихся из-за отражений в длинных линиях связи (длинной считается линия, время распространения сигнала в которой больше длительности его фронта, для самых быстрых ТТЛШ микросхем линия становится длинной, начиная с длины в несколько сантиметров).

### ***Серии ТТЛ-микросхем зарубежного производства***

В скобках указаны типовые значения времени задержки ( $T_{pd}$ ) и потребляемой мощности ( $P_d$ ) для каждой серии, взятые из документа SDAA010.PDF фирмы Texas Instruments, кроме 74F, для которой данные взяты из AN-661 фирмы Fairchild.

- 74 — базовая ТТЛ-серия. Несмотря на то, что была первой серией, выпускавшейся фирмой Texas Instruments, до сих пор находится в производстве ( $T_{pd} = 10$  нс,  $P_d = 10$  мВт);
- 74L — серия с пониженным энергопотреблением, заменена серией LS, а также КМОП-микросхемами, значительно превосходящими её по экономичности ( $T_{pd} = 33$  нс,  $P_d = 1$  мВт);
- 74H — повышенная скорость. Использовалась в 1960-е — начале 1970-х годов и была заменена S-серией ( $T_{pd} = 6$  нс,  $P_d = 22$  мВт);
- 74S — с диодами Шоттки (Schottky). Хотя устарела (её превосходят серии 74AS и 74F), до сих пор производится фирмой Texas Instruments ( $T_{pd} = 3$  нс,  $P_d = 19$  мВт);
- 74LS — с диодами Шоттки и пониженным энергопотреблением (Low Power Schottky) ( $T_{pd} = 9$  нс,  $P_d = 2$  мВт);
- 74AS — улучшенная с диодами Шоттки (Advanced Schottky) ( $T_{pd} = 1.7$  нс,  $P_d = 8$  мВт);
- 74ALS — улучшенная с диодами Шоттки и пониженным энергопотреблением (Advanced Low Power Schottky) ( $T_{pd} = 4$  нс,  $P_d = 1.2$  мВт);
- 74F — быстрая (Fast) с диодами Шоттки (Fast) ( $T_{pd} = 1.7$  нс,  $P_d = 4$  мВт, в действительности 74F несколько уступает по быстродействию 74AS);

Префикс серии «74» обозначает коммерческий вариант микросхем, «54» — промышленный или военный, с расширенным температурным диапазоном  $-55\text{ }^{\circ}\text{C} \dots +125\text{ }^{\circ}\text{C}$ . Тип корпуса, как правило, указывается последней буквой в обозначении, например для Texas Instruments тип корпуса пластиковый DIP кодируется буквой N (SN7400N).

### ***Советские серии ТТЛ-микросхем***

- 106 — ранняя серия ТТЛ-микросхем пониженной степени интеграции (не более 2 логических элементов в одном корпусе), предназначенная для тяжёлых условий эксплуатации (военная аппаратура, космос, и т. п.). Не имеет аналогов среди микросхем фирмы Texas Instruments.
- 133 и 155 соответствуют сериям 54 и 74;
- 130, 131 — сериям 54H и 74H;
- 134, КР134 — сериям 54L и 74L;
- 136 и 158 — близки к сериям 54L и 74L, но имеют вдвое большее энергопотребление при несколько большем быстродействии;
- 530 и 531 — сериям 54S и 74S;
- 533 и 555 — сериям 54LS и 74LS;
- 1530, КР1530 — сериям 54AS и 74AS (приблизительное соответствие);
- 1531, КР1531 — сериям 54F и 74F;
- 1533, КР1533 — сериям 54ALS и 74ALS;

### ***Особенности применения микросхем ТТЛ логики***

При работе ТТЛ-логики наблюдаются достаточно сильные всплески токов (особенно на выходе), которые могут создавать паразитные наводки на цепях питания, приводя к

сбоям самих ТТЛ элементов. Для борьбы с этим явлением необходимо руководствоваться следующими правилами:

- Питание ТТЛ микросхем организуется в виде двух шин (обычно из медных или латунных полос) с короткими отводами печатных дорожек к выводам питания. При многослойном монтаже выделяются отдельные слои для шин питания. Применение разветвлённых дорожек питания запрещено.
- Между шинами питания устанавливаются блокировочные конденсаторы с малой паразитной индуктивностью (керамические или слюдяные). Минимальная ёмкость, количество блокировочных конденсаторов определяется инструкцией по монтажу ТТЛ-микросхемы.

Не всегда все имеющиеся входы ТТЛ элемента используются в конкретной схеме. Если по логике работы на входе необходим нулевой сигнал, то неиспользуемые входы соединяются с общим проводом.

- Для собственно ТТЛ-схемы неиспользуемые входы можно никуда не подключать, но из-за паразитной ёмкости будет появляться задержка прохождения сигнала (около 2 нс на один вход), кроме того, на неиспользуемые входы могут наводиться помехи. У ТТЛШ оставлять не подключёнными неиспользуемые входы запрещено.
- Можно соединить используемые и неиспользуемые входы (если это допустимо по схеме), но это увеличивает нагрузку на источник сигнала и также увеличивает задержку.
- Неиспользуемые входы можно присоединить к выходу инвертирующего элемента, входы которого подсоединяются к общему проводу.
- Неиспользуемые входы можно присоединить к источнику питания через резистор — для одних серий (количество одновременно подключаемых входов к одному резистору и величина этого сопротивления регламентируются соответствующими инструкциями) или непосредственно к питанию — для других.

## Интегрально-инжекционная логика

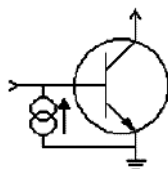
**Интегрально-инжекционная логика** (распространённые аббревиатуры ИИЛ, И<sup>2</sup>Л, И2Л, ИЗЛ, I2L, ИЛ, I<sup>2</sup>L) — схемотехника и технология изготовления логических элементов на биполярных транзисторах.

Интегрально-инжекционная логика предложена в конце 1971 г. — начале 1972 г. почти одновременно сотрудниками фирмы *IBM X*. Бергером и З. Видманом (Бёблинген, ФРГ) и сотрудниками фирмы *Philips* В. К. Хартом и А. Слобом (Эйндховен, Голландия).

## Принцип работы

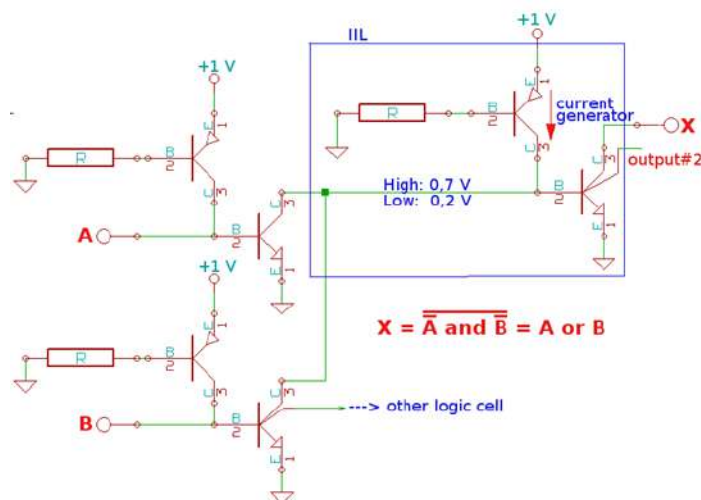
ИИЛ является развитием технологии НСТЛ (логика с непосредственными связями между транзисторами, которая в иностранной литературе называется MTL — Merged Transistor Logic или DCTL — Direct-Coupled Transistor Logic). В свою очередь, НСТЛ можно рассматривать как предельный вариант РТЛ, в НСТЛ отсутствуют резисторы между выходом (коллектором) n-p-n транзистора логического элемента-источника и входом (базой) элемента-приёмника.

Вообще говоря, ИИЛ не является разновидностью *транзисторно-транзисторной логики*. В основе функционирования ИИЛ лежит использование «особых» транзисторов с объединённой базой. Эти транзисторы не способны проводить ток из-за низкой концентрации носителей зарядов в базе. Поэтому рядом с основным транзистором, входящим в логический элемент, находится «инжектор» — электрод, инжектирующий носители заряда в базу. При инжекции транзистор включается и может выполнять заданную логическую функцию.



**Рис.44** - Упрощенная символическое обозначение И2Л инвертора

При проектировании микросхем ИИЛ основную роль отводят именно инжекторам. Эмиттеры, как правило, соединены — ими является подложка микросхемы. На поверхности кристалла выходят базовые области транзисторов со сформированными на них коллекторными областями. Благодаря такой технологии ИИЛ-транзистор по размеру (если не считать инжектора) занимает на кристалле меньшую площадь чем МОП-транзистор. Так как один инжектор может использоваться для инжекции в несколько транзисторов, в результате получается выигрыш в степени интеграции.



**Рис.45** - Принципиальная схема логического элемента «ИЛИ». Р-п-р транзисторы в базах служат для инжекции зарядов в базы основных транзисторов.

## Достоинства и недостатки

Преимущества И2Л:

- Высокая степень интеграции (например, «проводное ИЛИ» осуществляется простым соединением коллекторов транзисторов, то есть логическая функция ИЛИ-НЕ реализуется вообще без дополнительных ключей), высокая степень интеграции, иногда выше, чем у микросхем с МОП-логикой.
- Иногда меньшая стоимость, чем у устройств, построенных по принципам других логик. Высокий выход годных при производстве.
- Малое потребление энергии на одно элементарное переключение вентиля  $\sim 10^{-12}$  Дж и как следствие — сниженное энергопотребление.
- Низкое напряжение питания: 1...3 В.
- Хотя логические уровни ИИЛ очень близки (логический ноль  $\sim 0.2$  В, логическая единица  $\sim 0.7$  В) схемы ИИЛ имеют высокую помехоустойчивость, поскольку логические уровни передаются током, а не напряжением.

К недостаткам относят:

- Относительно низкое быстродействие, максимальная рабочая частота  $\sim 50$  МГц.

## Применение

В СССР выпускались микропроцессорные наборы серий К582 и К584 на основе технологии ИИЛ. Также выпускались серии микросхем статических запоминающих устройств К558 и К541 небольшой ёмкости, которые благодаря встроенным преобразователям логических уровней были совместимы по уровням входных и выходных сигналов с ТТЛ.

## N-МОП/Р-МОП

**МОП (металл-оксид-полупроводник)** — один из видов полевого транзистора (самый простой), в котором управляющий электрод (затвор) отделён от канала слоем диэлектрика, в простейшем случае, *диоксида кремния*. Транзисторы МОП-структуры лучше других активных полупроводниковых приборов подходили для создания логических БИС и СБИС, и ранний прогресс цифровой техники обусловлен микросхемами на транзисторах с МОП-структурой. В отличие от биполярного транзистора, выходной ток которого управляется входным током, МОП-транзистор, как и другие полевые транзисторы, управляется напряжением, этим он напоминает электровакуумный триод. В зависимости от типа носителей зарядов, МОП-транзисторы могут быть n-канальными или p-канальными, в первых в качестве основных носителей используются электроны, во втором типе — дырки.

Логический МОП-элемент представляет собой несколько транзисторов, соединённых либо последовательно (для получения функции «И-НЕ»), либо параллельно (для получения функции «ИЛИ-НЕ»). Такой же транзистор, только постоянно включённый, служит нагрузочным резистором логического элемента. Увеличение сопротивления открытого канала этого транзистора уменьшает потребляемую мощность, но одновременно и скорость действия логического элемента. Управлять этим параметром можно, изменяя геометрические размеры, например, ширину канала.

МОП-транзисторы занимают на кристалле микросхемы в 6—9 раз меньшую площадь, чем транзисторы, используемые в ТТЛ за счёт упрощения топологии, так, наиболее простой тип транзистора, с индуцированным каналом, требует всего одной операции легирования и одной — металлизации (в англоязычной литературе такой тип называется «обогашённый канал»). Это позволяло добиться высокой степени интеграции. Первые микропроцессоры были созданы как раз на основе МОП технологии. Однако, схемы на транзисторах с индуцированным каналом требуют весьма высокого напряжения питания (27 вольт для типичных p-МОП-структур, и 12 вольт для типичных n-МОП), и обладают невысоким быстродействием, задержка переключения для p-МОП схем составляла десятки, в лучшем случае — единицы микросекунд, а для n-МОП — сотни наносекунд. Увеличить быстродействие с одновременным снижением питающего напряжения удалось с применением транзисторов со встроенным каналом, работающим в режиме обеднения. Такие транзисторы требуют на одну операцию легирования больше, но позволили n-МОП схемам работать от одного источника напряжением 5 вольт.

Дальнейшее повышение быстродействия было связано с отказом от металлических затворов и переходом на затворы из поликристаллического кремния. Чтобы ещё увеличить быстродействие, для изоляции затвора от канала были использованы диэлектрики с меньшим коэффициентом диэлектрической проницаемости, чем у оксида кремния, поэтому полевые транзисторы современных цифровых СБИС уже неправомерно называть МОП-транзисторами. С целью радикального снижения энергопотребления в одном логическом элементе могут применяться транзисторы обоих типов проводимости, как с каналом n-типа, так и с каналом p-типа. Такая схема называется КМОП — «комплементарная». В отличие от схем на одном типе проводимости, КМОП схемы практически не потребляют тока в статическом режиме, так как в них в цепочке последовательно соединённых транзисторов хотя бы один всегда закрыт, и лишь во время переходного процесса все транзисторы оказываются ненадолго открытыми. Однако, КМОП структуры требуют большего числа технологических операций при изготовлении, что первоначально ограничивало достижимую степень интеграции (довольно долго по этому параметру лидировали n-МОП структуры).

Существуют и иные разновидности интегральных полевых транзисторов. В частности, с барьером Шоттки — обычно это МеП (Металл-полупроводник). ЛИЗМОП (МОП с Лавинной Инжекцией Заряда) с «плавающим» затвором без вывода. В нём затвор представляет собой изолированный островок, который состоит из молибдена или поликремния. При инжекции заряда информация запоминается. Для стирания используется облучение ультрафиолетом. Аналогично устроена и Flash-память, но там затворы тоньше для возможности «вытягивания» заряда в подложку. Для поблочной адресации и для исключения возможности повреждения транзисторов при стирании данная память часто имеет встроенные цепи программирования/стирания/управления. МНОП (металл-нитрид-оксид-полупроводник) — транзистор с двойным затвором и двойным изолятором из нитридного и оксидного слоёв. В процессе заряда ток проходит через оксид, но не проходит через нитрид, что позволяет сохранять информацию. Второй затвор сверху позволяет электрически стирать информацию в таком транзисторе. Применялись они до появления Flash-памяти.

Условные графические обозначения МОП транзисторов различного типа

|                                                                                           | Индукцированный канал | Встроенный канал |
|-------------------------------------------------------------------------------------------|-----------------------|------------------|
| <b>P-канал</b>                                                                            |                       |                  |
| <b>N-канал</b>                                                                            |                       |                  |
| Условные обозначения: З — затвор (G — Gate), И — исток (S — Source), С — сток (D — Drain) |                       |                  |

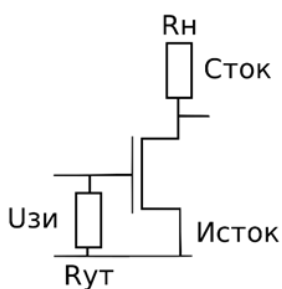


Рис.46 - Схема включения МОП-транзистора

## КМОП

**КМОП** (комплементарная структура металл-оксид-полупроводник; англ. *CMOS, complementary metal-oxide-semiconductor*) — набор полупроводниковых технологий построения интегральных микросхем и соответствующая ей схемотехника микросхем. Подавляющее большинство современных цифровых микросхем — КМОП.

В более общем случае — КМДП (со структурой металл-диэлектрик-полупроводник). В технологии КМОП используются полевые транзисторы с изолированным затвором

с каналами разной проводимости. Отличительной особенностью схем КМОП по сравнению с биполярными технологиями (ТТЛ, ЭСЛ и др.) является очень малое энергопотребление в статическом режиме (в большинстве случаев можно считать, что энергия потребляется только во время переключения состояний). Отличительной особенностью структуры КМОП по сравнению с другими МОП-структурами (N-МОП, P-МОП) является наличие как n-, так и p-канальных полевых транзисторов в одной локации кристалла; вследствие меньшего расстояния между элементами КМОП-схемы обладают более высокой скоростью действия и меньшим энергопотреблением, однако при этом характеризуются более сложным технологическим процессом изготовления и меньшей плотностью упаковки.

По аналогичной технологии выпускаются дискретные полевые транзисторы с изолированным затвором (MOSFET, metal-oxide-semiconductor field-effect transistor).

Схемы КМОП в 1963 изобрёл Фрэнк Вонлас из компании Fairchild Semiconductor, первые микросхемы по технологии КМОП были созданы в 1968. Долгое время КМОП рассматривалась как энергосберегающая, но медленная альтернатива ТТЛ, поэтому микросхемы КМОП в первую очередь нашли применение в электронных часах, калькуляторах и других устройствах с батарейным питанием, где энергопотребление было критичным.

К 1990 году с повышением степени интеграции микросхем встала проблема рассеивания энергии на элементах. В результате технология КМОП оказалась в выигрышном положении. Со временем были достигнуты скорость переключения и плотность монтажа, недостижимые в технологиях, основанных на биполярных транзисторах.

Ранние КМОП-схемы были очень уязвимы для электростатических разрядов. Сейчас эта проблема в основном решена, но при монтаже КМОП-микросхем рекомендуется принимать меры по снятию электрических зарядов.

Для изготовления затворов в КМОП-ячейках на ранних этапах применялся алюминий. Позже, в связи с появлением так называемой самосовмещённой технологии, которая предусматривала использование затвора не только как конструктивного элемента, но одновременно как маски при получении сток-источковых областей, в качестве затвора стали применять поликристаллический кремний.

## Схемотехника

Инвертор (элемент НЕ) на КМОП (Рис.47) – проще невозможно.

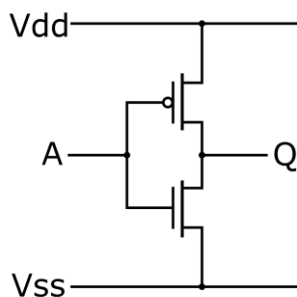


Рис.47 - Логический элемент НЕ на КМОП

Схема вентиля 2И-НЕ, построенного по технологии КМОП (Рис.48).

- Если на оба входа А и В подан высокий уровень, то оба транзистора снизу на схеме открыты, а оба верхних закрыты, то есть выход соединён с землёй.
- Если хотя бы на один из входов подать низкий уровень, соответствующий транзистор сверху будет открыт, а снизу - закрыт. Таким образом, выход будет соединён с напряжением питания и отсоединён от земли.

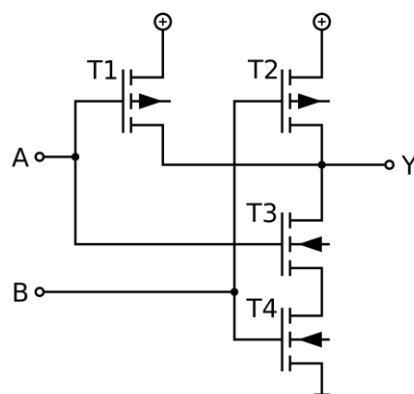


Рис.48 - Логический элемент 2И-НЕ на КМОП

В схеме нет никаких нагрузочных сопротивлений, поэтому в статическом состоянии через КМОП-схему протекают только токи утечки через закрытые транзисторы, и энергопотребление очень низкое. При переключениях электрическая энергия тратится в основном на заряд емкостей затворов и проводников, так что потребляемая (и рассеиваемая) мощность пропорциональна частоте (точнее, квадрату частоты) этих переключений (например, тактовой частоте процессора).

На рисунке с логическим элементом 2И-НЕ (Рис.48) можно заметить, что в ней используются два двухзатворных полевых транзистора разных конструкций. Верхний двухзатворный полевой транзистор отвечает за формирование высокого уровня на выходе логического элемента, если любой из входов имеет низкий уровень, а нижний двухзатворный полевой транзистор отвечает за формирование низкого уровня на выходе, если оба входа имеют высокий уровень.

Следует отметить, что, поскольку переключение n-канальных и p-канальных транзисторов имеет конечное время, на короткое время оба типа транзисторов могут быть открыты и между цепями питания возникает ток короткого замыкания. Это приводит к повышению энергопотребления при переключениях.

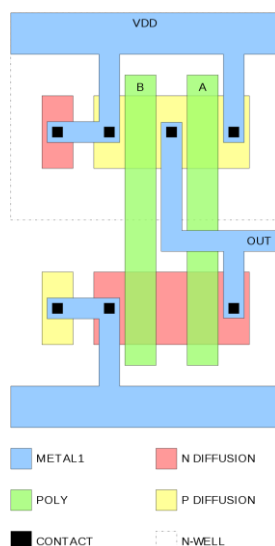


Рис.49 - Топология логического элемента 2И-НЕ (схема)

## Серии и семейства логических КМОП-микросхем

- На КМОП-транзисторах (CMOS):
  - 4000 — CMOS с питанием от 3 до 15 В, 200 нс;
  - 4000В — CMOS с питанием от 3 до 18 В, 90 нс;



- 74C — семейство в серии 7400, аналогичное 4000B;
- 74HC — высокоскоростное CMOS, по скорости аналогично семействам LS, 12 нс;
- 74HCT — высокоскоростное, совместимо по выходам с биполярными сериями;
- 74AC — улучшенное CMOS, скорость в целом между семействами S и F;
- 74ACT — улучшенное CMOS, совместимо по выходам с биполярными сериями;
- 74AHC — улучшенное высокоскоростное CMOS, втрое быстрее HC;
- 74AHCT — улучшенное высокоскоростное CMOS, совместимо по выходам с биполярными сериями;
- 74ALVC — с низким напряжением питания (1,65 — 3,3 В), время срабатывания 2 нс;
- 74AUC — с низким напряжением питания (0,8 — 2,7 В), время срабатывания < 1,9 нс при  $V_{пит}=1,8$  В;
- 74FC — быстрое CMOS, скорость аналогична F;
- 74FCT — быстрое CMOS, совместимо по выходам с биполярными сериями;
- 74LCX — CMOS с питанием 3 В и 5В-совместимыми входами;
- 74LVC — с пониженным напряжением (1,65 — 3,3 В) и 5В-совместимыми входами, время срабатывания < 5,5 нс при  $V_{пит}=3,3$  В, < 9 нс при  $V_{пит}=2,5$  В;
- 74LVQ — с пониженным напряжением (3,3 В);
- 74LVX — с питанием 3,3 В и 5В-совместимыми входами;
- 74VHC — сверхвысокоскоростное CMOS-семейство — быстродействие сравнимо с S;
- 74VHCT — сверхвысокоскоростное CMOS, совместимая по выходам с биполярными сериями;
- 74G — суперсверхвысокоскоростное для частот выше 1 ГГц, питание 1,65 В — 3,3 В, 5В-совместимые входы;
- BiCMOS
  - 74BCT — BiCMOS, TTL-совместимые входы, используется для буферов;
  - 74ABT — улучшенное BiCMOS-семейство, TTL-совместимые входы, быстрее ACT и BCT;

*Для более гибкого применения у ряда производителей существуют также особые семейства КМОП, в которых каждая ИМС включает всего 1 логический элемент в 5...6-пиновом корпусе, что бывает полезно для конструкций с малым количеством разных элементов и минимальным размером платы (например: **74LVC1G00GW** фирмы NXP; SOT353-1 Single 2-Input Positive-AND Gate).*

## Эмиттерно-связанная логика

**Эмиттерно-связанная логика (ЭСЛ, ECL)** — способ построения логических элементов на основе *дифференциальных транзисторных каскадов*. ЭСЛ является самой быстродействующей из всех типов логики, построенной на биполярных транзисторах. Это объясняется тем, что транзисторы в ЭСЛ работают в линейном режиме, не переходя в режим насыщения, выход из которого замедлен. Низкие значения логических перепадов в ЭСЛ-логике способствуют снижению влияния на быстродействие паразитных ёмкостей.

Основная деталь ЭСЛ-логики — схема потенциального сравнения, собранная не на диодах (как в ДТЛ), а на транзисторах. Схема (Рис.50) представляет собой транзисторы, соединённые эмиттерами и подключённые к корпусу (или питанию) через резистор (или генератор тока). При этом транзистор, у которого напряжение на базе выше, пропускает через себя основной ток. Как правило, один транзистор в схеме сравнения подключен к

опорному уровню, равному напряжению логического порога, а остальные транзисторы являются входами. Выходные цепи схемы сравнения поступают на усилительные транзисторы, а с них — на выходные *эмиттерные повторители*.

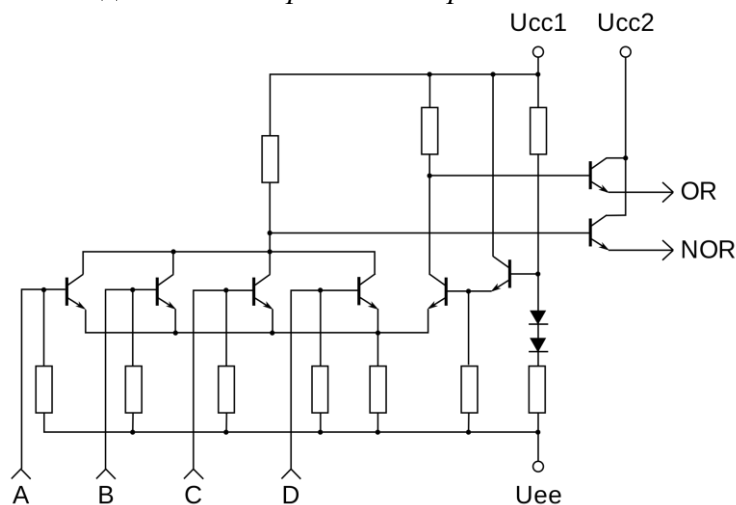


Рис.50 - Логический элемент 4ИЛИ/ИЛИ-НЕ на ЭСЛ

Особенностью ЭСЛ является повышенные скорость (150 МГц уже в первых образцах 1960-х годов и 0,5...2 ГГц в 1970-1980-х) и энергопотребление по сравнению с ТТЛ и КМОП (на низких частотах, на высоких — примерно равное), низкая помехоустойчивость, низкая степень интеграции (ограниченная, в частности, большой потребляемой мощностью каждого элемента, что не позволяет разместить в одном корпусе много элементов, так как это приведёт к перегреву), и, как следствие — высокая стоимость.

## История

ЭСЛ была изобретена в августе 1956 года инженером IBM *Хэноном Йорком* (англ. *Hannon S. Yourke*). Первоначально имела название «управляемая током логика», применялась в компьютерах Stretch, IBM 7090, и IBM 7094. Также использовалось название *схема токового режима* (токовые ключи).

Переключатель тока Йорка представлял собой *дифференциальный усилитель*, в котором входные логические уровни сигналов отличались от выходных. В схеме Йорка отличие опорных уровней напряжения составляло 3 вольта. В связи с этим использовались две взаимодополняющих версии логических элементов: NPN и PNP. Выход NPN версии мог управлять входами PNP версии и наоборот. Недостатками схемы являлись использование дополнительных источников напряжения и использование как PNP, так и NPN транзисторов.

Позже, вместо чередования NPN и PNP версий логических элементов, был предложен метод с использованием стабилитронов и резисторов для сдвига выходных логических уровней к значениям входных логических уровней. В ЭВМ БЭСМ-6 для этой же цели в состав переключателя тока был введён вторичный источник электропитания на основе трансформатора и двухполупериодного выпрямителя со средней точкой, названный «подвешенным источником питания».

С появлением цифровых интегральных микросхем ЭСЛ логики для сдвига выходных логических уровней стал использоваться эмиттерный повторитель, также обеспечивающий повышение коэффициента разветвления.

Первая серия микросхем ЭСЛ логики, MECL I, была представлена фирмой Motorola в 1962 году. Motorola разработала улучшенные серии MECL II в 1966 году и MECL III в 1968 году. MECL III обладала задержкой распространения сигнала в 1 наносекунду и частотой переключения триггеров до 500 МГц. В 1971 году выпущена серия 10000 с пониженным энергопотреблением и быстродействием.

Высокое энергопотребление ЭСЛ ограничило её применение только в схемах, где было важно максимальное быстродействие. ЭСЛ применялась в мейнфреймах IBM серии IBM System/390, суперкомпьютере Cray-1, первом поколении мейнфреймов Amdahl, ЕС ЭВМ ряда 2, ЭВМ «Эльбрус-2».

## Цифровые микросхемы ЭСЛ

Серии микросхем отечественного производства:

- 137, 187, 229 — совместимые серии, 137 отличается более высоким быстродействием, 187 — меньшей потребляемой мощностью. Давно устарели.
- 138 — серия с лучшими параметрами, чем 137, 187. Устарела с появлением серий 100 и 500.
- 100, 500, 700 — это одна и та же серия в плоских корпусах (100), DIP-корпусах (500), бескорпусная для гибридных микросхем (700). Аналог серии Motorola 10000.
- 1500 — серия с субнаносекундным быстродействием, использовалась в ЭВМ «Эльбрус-2».
- 1590 — последняя отечественная серия ЭСЛ-логики, аналоги серии Motorola MC10Hxxx. По сравнению с предыдущей, 1500-ой серией, при том же быстродействии энергопотребление снижено примерно втрое.
- 1800 — разрядно-модульный (bit-slice) микропроцессорный комплект, некоторые микросхемы использовались в ЭВМ серии ЕС.

## ГЛАВА 2. АРХИТЕКТУРА ЦИФРОВЫХ ВЫЧИСЛИТЕЛЬНЫХ УСТРОЙСТВ

Под *архитектурой* компьютера (ЭВМ) понимается его абстрактное представление в терминах основных функциональных модулей, т.е. его логическая организация. В это понятие входят структура, перечень программно-доступных элементов, система команд, методы адресации и т.д. Архитектура не определяет особенности технологии, аппаратной реализации, временные диаграммы, тактовую частоту, параметры электропитания и т.д. Одним словом, термин «архитектура» используется для описания возможностей, предоставляемых ЭВМ, а термин «организация» определяет, как эти возможности реализованы.

Все ЭВМ содержат следующие функциональные блоки, имеющие свою архитектуру: процессор, состоящий из арифметико-логического устройства (АЛУ) и устройства управления (УУ), память (оперативная – ОЗУ и постоянная – ПЗУ), устройства ввода/вывода информации. Устройство управления формирует управляющие сигналы для работы АЛУ. Объединение функциональных блоков в ЭВМ осуществляется посредством системы шин<sup>1</sup>: шины данных, по которой осуществляется обмен данными между блоками ЭВМ, шины адреса, используемой для передачи адресов, по которым осуществляется обращение к различным устройствам ЭВМ, и шины управления для передачи управляющих сигналов.

### Архитектура фон Неймана

**Архитектура фон Неймана** (модель фон Неймана, *Принстонская архитектура*) — широко известный принцип совместного хранения команд и данных в памяти компьютера. Вычислительные машины такого рода часто обозначают термином «машина фон Неймана», однако соответствие этих понятий не всегда однозначно. В общем случае, когда говорят об архитектуре фон Неймана, подразумевают принцип хранения данных и инструкций в одной памяти.

### История возникновения термина

Основы учения об архитектуре вычислительных машин заложил *фон Нейман* в 1944 году, когда подключился к созданию первого в мире лампового компьютера ЭНИАК. В процессе работы над ЭНИАКом в Институте Мура в Пенсильванском Университете во время многочисленных дискуссий фон Неймана с его коллегами *Джоном Уильямом Мокли*, *Джоном Эккертом*, *Германом Голдстейном* и *Артуром Бёрксом*, возникла идея более совершенной машины под названием EDVAC. Исследовательская работа над EDVAC продолжалась параллельно с конструированием ЭНИАКа.

В марте 1945 года принципы логической архитектуры были оформлены в документе, который назывался «Первый проект отчёта о EDVAC» — отчёт для Баллистической Лаборатории Армии США, на чьи деньги осуществлялась постройка ЭНИАКа и разработка EDVACa. Отчёт, поскольку он являлся всего лишь наброском, не предназначался для публикации, а только для распространения внутри группы, однако Герман Голдстейн — куратор проекта со стороны Армии США — размножил эту научную работу и разослал её широкому кругу учёных для ознакомления. Так как на первой странице документа стояло только имя фон Неймана, у читавших документ сложилось ложное впечатление, что автором всех идей, изложенных в работе, является именно он. Документ давал достаточно информации для того, чтобы читавшие его могли построить свои компьютеры, подобные EDVACу на тех же принципах и с той же архитектурой, которая в результате стала называться [«архитектурой фон Неймана»](#).

---

<sup>1</sup> **Компьютерная шина** (англ. *computer bus*) в архитектуре компьютера — соединение, служащее для передачи данных между функциональными блоками компьютера. В устройстве шины можно различить механический, электрический (физический) и логический (управляющий) уровни.

В 1946 году трое учёных — Артур Бёркс (англ. *Arthur Burks*), Герман Голдстейн (англ. *Herman Goldstine*) и Джон фон Нейман — опубликовали статью «Предварительное рассмотрение логического конструирования электронного вычислительного устройства». В статье обосновывалось использование двоичной системы для представления данных в ЭВМ (преимущественно для технической реализации, простота выполнения арифметических и логических операций — до этого машины хранили данные в десятичном виде), выдвигалась идея использования общей памяти для программы и данных. Имя фон Неймана было достаточно широко известно в науке того времени, что отодвинуло на второй план его соавторов, и данные идеи получили название «принципы фон Неймана».

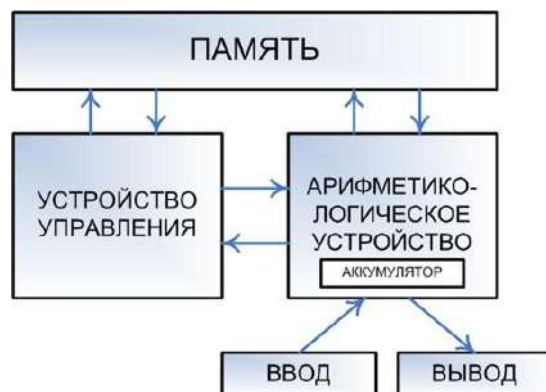


Рис.51 - Схематичное изображение машины фон Неймана

## Принципы фон Неймана

### Принцип двоичного кодирования

Для представления данных и команд используется двоичная система счисления.

### Принцип однородности памяти

Как программы (команды), так и данные хранятся в одной и той же памяти (и кодируются в одной и той же системе счисления — чаще всего **двоичной**). Над командами можно выполнять такие же действия, как и над данными.

### Принцип адресуемости памяти

Структурно основная память состоит из пронумерованных ячеек; процессору в произвольный момент времени доступна любая ячейка; память внутренняя.

### Принцип последовательного программного управления

Все команды располагаются в памяти и выполняются последовательно, одна после завершения другой, в последовательности, определяемой программой.

### Принцип жёсткости архитектуры

Неизменяемость в процессе работы топологии, архитектуры, списка команд.

### Узкое место архитектуры фон Неймана

Совместное использование шины для памяти программ и памяти данных приводит к *узкому месту* архитектуры фон Неймана, а именно ограничению *пропускной способности* между процессором и памятью. Из-за того, что память программ и память данных не могут быть доступны в одно и то же время, пропускная способность канала «процессор-память»

и скорость работы памяти существенно ограничивают скорость работы процессора — гораздо сильнее, чем если бы программы и данные располагались в разных адресных пространствах. Так как скорость процессора и объём памяти увеличивались гораздо быстрее, чем пропускная способность между ними, узкое место стало большой проблемой, серьёзность которой возрастает с каждым новым поколением процессоров.

Данная проблема решается совершенствованием систем *кэширования*, что в свою очередь усложняет архитектуру систем и увеличивает риск возникновения побочных ошибок (например, в 2018 году были обнаружены уязвимости *Meltdown* и *Spectre*, присутствовавшие в современных процессорах в течение десятилетий, но не обнаруженные ранее из-за огромной сложности современных вычислительных систем, в частности, при весьма не тривиальном взаимодействии процессора с кэш-памятью при спекулятивном выполнении инструкций, предсказании ветвлений и т.п.).

Термин «узкое место архитектуры фон Неймана» ввёл Джон Бэкус в 1977 в своей лекции «*Можно ли освободить программирование от стиля фон Неймана?*», которую он прочитал при вручении ему *Премии Тьюринга*.

## Гарвардская архитектура

**Гарвардская архитектура** — архитектура ЭВМ, отличительными признаками которой являются:

- хранилище инструкций и хранилище данных представляют собой разные физические устройства;
- канал инструкций и канал данных также физически разделены.

Архитектура была разработана *Говардом Эйкеном* в конце 1930-х годов в *Гарвардском университете*.

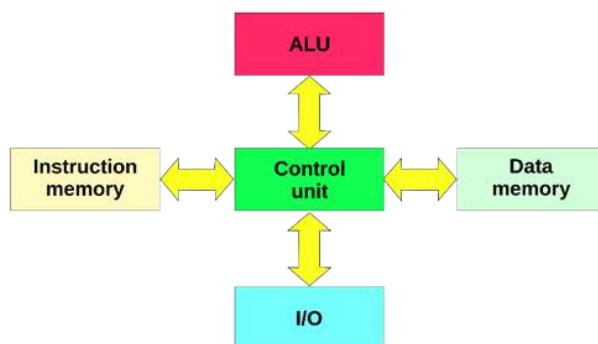


Рис.52 - Схематичное изображение машины с гарвардской архитектурой

## История

В 1930-х годах правительство США поручило Гарвардскому и Принстонскому университетам разработать архитектуру ЭВМ для военно-морской артиллерии. В конце 1930-х годов в Гарвардском университете Говардом Эйкеном была разработана архитектура компьютера Марк I, в дальнейшем называемая по имени этого университета. Оригинальная идея была продемонстрирована Эйкеном компании IBM в октябре 1937 года. Однако победила более простая в реализации разработка Принстонского университета (более известная как *архитектура фон Неймана*, названная так по имени авторитетного учёного-консультанта и разработчика, первым предоставившего отчёт об архитектуре, к которой пришли в ходе плодотворных дискуссий в команде создателей; авторами же идей, заложенных в этой архитектуре, являлись Джон Преспер Эккерт и Джон Уильям Мокли).

Гарвардская архитектура использовалась советским учёным А. И. Китовым в ВЦ-1 МО СССР.

## Классическая гарвардская архитектура

Типичные операции (н-р, сложение и умножение) требуют от любого вычислительного устройства нескольких действий:

- выбор *инструкции*;
- выборку двух *операндов* и выполнение инструкции;
- сохранение *результата*.

Идея, реализованная Эйкеном, заключалась в физическом разделении линий передачи *команд* и *данных*. В первом компьютере Эйкена «Марк I» для хранения инструкций использовалась *перфорированная лента*, а для работы с данными — электромеханические *регистры*. Это позволяло одновременно пересылать и обрабатывать команды и данные, благодаря чему значительно повышалось общее быстродействие компьютера.

В гарвардской архитектуре характеристики устройств памяти для инструкций и памяти для данных не обязательно должны быть одинаковыми. В частности, ширина слова, тактирование, технология реализации и структура адресов памяти могут различаться. В некоторых системах инструкции могут храниться в памяти только для чтения, в то время как для сохранения данных обычно требуется память с возможностью чтения и записи. В некоторых системах требуется значительно больше памяти для инструкций, чем памяти для данных, поскольку данные обычно могут подгружаться с внешней или более медленной памяти. Такая потребность увеличивает битность (ширину) шины адреса памяти инструкций по сравнению с шиной адреса памяти данных.

## Отличие от архитектуры фон Неймана

В чистой неймановской архитектуре процессор в каждый момент времени может либо читать инструкцию, либо читать/записывать единицу данных из/в памяти. Оба действия одновременно происходить не могут, поскольку инструкции и данные используют один и тот же поток (*шину*).

В компьютере с использованием гарвардской архитектуры процессор может считывать очередную команду и оперировать памятью данных одновременно и без использования кэш-памяти. Таким образом, компьютер с гарвардской архитектурой при определенной сложности схемы быстрее, чем компьютер с архитектурой фон Неймана, поскольку потоки команд и данных расположены на отдельных физически не связанных между собой аппаратных каналах.

Исходя из физического разделения шин команд и данных, разрядности этих шин могут различаться и *физически* не могут пересекаться.

## Модификации

### Модифицированная гарвардская архитектура

Соответствующая схема реализации доступа к памяти имеет один очевидный недостаток — высокую стоимость. При разделении каналов передачи команд и данных на кристалле процессора последний должен иметь почти вдвое больше интерфейсных выводов, так как шина адреса и шина данных составляют основную часть выводов микропроцессора. Способом разрешения этой проблемы стала идея использовать общие шину данных и шину адреса для всех внешних данных, а внутри процессора использовать шину данных, шину команд и две шины адреса. Такую концепцию стали называть *модифицированной гарвардской архитектурой*.

Такая схемотехника применяется в современных сигнальных процессорах. Ещё дальше по пути уменьшения стоимости пошли при создании однокристалльных микро-ЭВМ — микроконтроллеров. В них одна шина команд и данных применяется и внутри кристалла.

Разделение шин в модифицированной гарвардской структуре осуществляется при помощи отдельных управляющих сигналов: чтения, записи или выбора области памяти.

## Расширенная гарвардская архитектура

Часто при выполнении команды требуется выбрать три составляющие: два операнда и инструкцию (в алгоритмах цифровой обработки сигналов – это наиболее распространенная задача в БПФ-<sup>1</sup>, КИХ-<sup>2</sup> и БИХ-фильтрах<sup>3</sup>). Для этого существует кэш-память. В ней может храниться инструкция — следовательно, обе шины остаются свободными и появляется возможность передать два операнда одновременно. Использование кэш-памяти вместе с разделёнными шинами получило название «Super Harvard Architecture» («SHARC») — расширенная гарвардская архитектура.

Примером могут служить процессоры компании «Analog Devices»: ADSP-21xx — модифицированная гарвардская архитектура, ADSP-21xxx (SHARC) — расширенная гарвардская архитектура.

## Гибридные модификации с архитектурой фон Неймана

Существуют гибридные архитектуры, сочетающие достоинства как гарвардской, так и фон-неймановской архитектур. Современные CISC-процессоры обладают отдельной кэш-памятью 1-го уровня для команд и данных, что позволяет им за один рабочий такт получать одновременно и команду, и данные для её выполнения. То есть процессорное ядро аппаратно гарвардское, но программно оно фон-неймановское, что упрощает написание программ. Обычно в данных процессорах одна шина используется и для передачи команд, и для передачи данных, что схемотехнически упрощает систему. Современные варианты таких процессоров могут иногда содержать встроенные контроллеры сразу нескольких разнотипных шин для работы с различными типами памяти — например, DDR RAM и Flash. Тем не менее, и в этом случае шины, как правило, используются и для передачи команд, и для передачи данных без разделения, что делает данные процессоры ещё более близкими к фон-неймановской архитектуре при сохранении достоинств гарвардской архитектуры.

## Использование

Первым компьютером, в котором была использована идея гарвардской архитектуры, был Марк I.

Гарвардская архитектура используется в ПЛИК и микроконтроллерах, таких, как Microchip PIC, Atmel AVR, Intel 4004, Intel 8051, а также в кэш-памяти первого уровня x86-микропроцессоров, делящейся на два равных либо различных по объёму блока для данных и команд.

## CISC/RISC архитектуры

### CISC

CISC (*complex instruction set computing* или *complex instruction set computer*) — тип процессорной архитектуры, которая характеризуется следующим набором свойств:

- нефиксированное значение длины команды;
- арифметические действия кодируются в одной команде;

---

<sup>1</sup> БПФ – Быстрое Преобразование Фурье

<sup>2</sup> Фильтр КИХ – фильтр с Конечной Импульсной Характеристикой

<sup>3</sup> Фильтр БИХ – фильтр с Бесконечной Импульсной Характеристикой



- небольшое число регистров, каждый из которых выполняет строго определённую функцию.

Методика построения системы команд CISC противостоит методике, применяемой в другом распространённом типе процессорных архитектур — RISC, где используется набор упрощённых инструкций.

Типичными представителями CISC-архитектуры являются процессоры на основе команд x86, процессоры Motorola MC680x0, процессоры мейнфреймов zSeries.

Благодаря распространённости процессоров архитектур x86 и x86-64 CISC-системы являются самыми распространённым в мире вычислительной техники — они доминируют в сегментах рабочих станций, персональных компьютеров, серверов начального и среднего уровня, а также мейнфреймов (RISC-системы превалируют в сегментах мобильных устройств, микроконтроллеров и Unix-серверов старшего уровня).

При этом поздние x86-процессоры (Intel Pentium 4, Pentium D, Core, AMD Athlon, Phenom и новее), хотя и CISC-совместимы, но являются процессорами с RISC-ядром, и в формальном смысле считаются гибридными. В таких гибридных CISC-процессорах CISC-инструкции преобразовываются в набор внутренних RISC-команд, при этом одна команда x86 может порождать несколько RISC-команд (в случае процессоров типа P6 — до четырёх RISC-команд в большинстве случаев), исполнение команд происходит на *суперскалярном*<sup>1</sup> конвейере одновременно по несколько штук.

Основной недостаток CISC-архитектуры в сравнении с RISC — более сложный подход к распараллеливанию вычислений.

## RISC

**RISC** (*reduced instruction set computer* — компьютер с набором коротких (простых, быстрых) команд) — архитектура процессора, в которой быстродействие увеличивается за счёт упрощения инструкций, чтобы их декодирование было более простым, а время выполнения — меньшим. Первые RISC-процессоры даже не имели инструкций умножения и деления. Это также облегчает повышение тактовой частоты и делает более эффективной *суперскалярность* (распараллеливание инструкций между несколькими исполнительными блоками).

## Отличия от CISC

Наборы инструкций в более ранних архитектурах, для облегчения ручного написания программ на *языках ассемблеров* или прямо в *машинных кодах*, а также для упрощения реализации компиляторов, выполняли как можно больше работы на каждую инструкцию. Нередко в наборы включались инструкции для прямой поддержки конструкций языков высокого уровня. Другая особенность этих наборов — большинство инструкций, как правило, допускало все возможные методы адресации (т. н. «*ортогональность системы команд*»<sup>2</sup>) — к примеру, и операнды, и результат в арифметических операциях доступны не только в *регистрах*, но и через непосредственную адресацию, и прямо в *памяти*. Позднее такие архитектуры были названы CISC (*Complex instruction set computer* (Компьютер со сложным набором команд)).

<sup>1</sup> **Суперскалярный процессор** (англ. *superscalar processor*) — процессор, поддерживающий параллелизм на уровне инструкций (то есть, процессор, способный выполнять несколько инструкций одновременно) за счёт включения в состав его вычислительного ядра нескольких функциональных узлов (блоков) (таких как АЛУ, FPU, умножитель (*integer multiplier*), сдвигающее устройство (*integer shifter*) и другие устройства). Планирование исполнения потока инструкций осуществляется динамически вычислительным ядром (не статически компилятором).

<sup>2</sup> **Ортогональная система команд** — набор инструкций, в которой код команды не зависит от методов адресации операндов.

Однако многие компиляторы не задействовали все возможности таких наборов инструкций, а на сложные методы адресации уходит много времени из-за дополнительных (последовательных) обращений к медленной памяти. Было показано, что такие функции лучше заменять последовательностью более простых инструкций, при этом исполнительная часть процессора упрощается и в нём остаётся место для большего числа регистров, за счёт которых можно сократить количество обращений к памяти. В первых архитектурах, причисляемых к RISC, большинство инструкций для упрощения декодирования имеет одинаковую длину и схожую структуру, арифметические операции работают только с регистрами, а работа с памятью идёт через отдельные инструкции загрузки (load) и сохранения (store). Эти свойства и позволили лучше сбалансировать этапы *конвейеризации*, сделав конвейеры в RISC значительно более эффективными и позволив поднять тактовую частоту.

## Философия RISC

В середине 1970-х разные исследователи (в частности, из IBM) показали, что большинство комбинаций инструкций и ортогональных методов адресации не использовалось в большинстве программ, порождаемых компиляторами того времени. Также было обнаружено, что в некоторых архитектурах с микрокодовой реализацией сложные операции зачастую были медленнее последовательности более простых операций, выполняющих те же действия. Это было вызвано, в частности, тем, что многие архитектуры разрабатывались в спешке и хорошо оптимизировался микрокод только тех инструкций, которые использовались чаще.

Поскольку многие реальные программы тратят большинство своего времени на выполнение простых операций, многие исследователи решили сфокусироваться на том, чтобы сделать эти операции максимально быстрыми. Производительность процессора ограничена временем, которое процессор тратит на выполнение наиболее медленных шагов в процессе обработки любой инструкции; уменьшение длительности таких шагов даёт общее повышение производительности, а также зачастую ускоряет выполнение инструкций за счёт более эффективной конвейеризации. Фокусирование на простых инструкциях и привело к архитектуре RISC, цель которой — сделать инструкции настолько простыми, чтобы они легко конвейеризировались и тратили не более одного такта на каждом шаге конвейера на высоких частотах.

Позднее было отмечено, что наиболее значимая характеристика RISC в разделении инструкций для обработки данных и обращения к памяти — обращение к памяти идёт только через инструкции load и store, а все прочие инструкции ограничены внутренними регистрами. Это упростило архитектуру процессоров: позволило инструкциям иметь фиксированную длину, упростило конвейеры и изолировало логику, имеющую дело с задержками при доступе к памяти, только в двух инструкциях. В итоге RISC-архитектуры стали называть также *архитектурами load/store*.

## Количество инструкций

Нередко слова «сокращённый набор команд» понимаются как минимизация количества инструкций в системе команд. В действительности, инструкций у многих RISC-процессоров больше, чем у CISC-процессоров. Некоторые RISC-процессоры – вроде транзисторов компании INMOS – имеют наборы команд не меньше, чем, например, у CISC-процессоров IBM System/370; и обратный пример — CISC-процессор DEC PDP-8 имеет только 8 основных и несколько расширенных инструкций.

На самом деле, термин «сокращённый» в названии описывает тот факт, что сокращён объём (и время) работы, выполняемый каждой отдельной инструкцией — как максимум один цикл доступа к памяти, — тогда как сложные инструкции CISC-процессоров могут требовать сотен циклов доступа к памяти для своего выполнения.

Некоторые архитектуры, специально разработанные для минимизации количества инструкций, сильно отличаются от классических RISC-архитектур и получили другие

названия: Minimal instruction set computer (MISC), Zero instruction set computer (ZISC), Ultimate RISC (также называемый OISC), Transport triggered architecture (TTA) и т. п.

## Характерные особенности RISC-процессоров

- Фиксированная длина машинных инструкций (например, 32 бита) и простой формат команды.
- Специализированные команды для операций с памятью — чтения или записи. Операции вида Read-Modify-Write («прочитать-изменить-записать») отсутствуют. Любые операции «изменить» выполняются только над содержимым регистров (т. н. архитектура load-and-store).
- Большое количество регистров общего назначения (32 и более<sup>1</sup>).
- Отсутствие поддержки операций вида «изменить» над укороченными типами данных — байт, 16-разрядное слово. Так, например, система команд DEC Alpha содержала только операции над 64-разрядными словами, и требовала разработки и последующего вызова процедур для выполнения операций над байтами, 16- и 32-разрядными словами.
- Отсутствие *микропрограмм* внутри самого процессора. То, что в CISC-процессоре выполняется микропрограммами, в RISC-процессоре выполняется как обыкновенный (хотя и помещённый в специальное хранилище) машинный код, не отличающийся принципиально от кода ядра ОС и приложений. Так, например, обработка отказов страниц в DEC Alpha и интерпретация таблиц страниц содержалась в так называемом *PALcode* (Privileged Architecture Library), помещённом в ПЗУ. Заменой PALCode можно было превратить процессор Alpha из 64-разрядного в 32-разрядный, а также изменить порядок байтов в слове и формат входов таблиц страниц виртуальной памяти.

## Другие архитектуры

За годы после появления архитектуры RISC были реализованы и другие альтернативы — например, VLIW, MISC, OISC, массово-параллельная обработка, систолическая матрица (*Systolic array*), переконфигурируемые вычисления (*Reconfigurable computing*), потоковая архитектура (*Dataflow architecture*).

- Суперскалярные архитектуры (первоначально — большие ЭВМ конца 1960-х годов, в микропроцессорах — Sun SPARC, начиная с Pentium использованы в семействе x86). Распараллеливание исполнения команд между несколькими устройствами исполнения, причём решение о параллельном исполнении двух или более команд принимается аппаратурой процессора на этапе исполнения. Эффективное использование такой архитектуры требует специальной оптимизации машинного кода в компиляторе для генерации пар независимых команд (когда результат одной команды не является аргументом другой).
- Архитектуры VLIW (Very Long Instruction Word — очень длинное слово команды). Отличаются от суперскалярной архитектуры тем, что решение о распараллеливании принимается не аппаратурой на этапе исполнения, а компилятором на этапе генерации кода. Команды очень длинные и содержат явные инструкции по распараллеливанию нескольких субкоманд на несколько устройств исполнения. Элементы архитектуры содержались в серии HP PA-RISC. Процессорами с архитектурой VLIW в

---

<sup>1</sup> В современных RISC-архитектурах количество регистров общего назначения достигает нескольких сотен. Например, в процессоре «Эльбрус» их насчитывается 226 штук.

её классическом виде являются процессоры с архитектурой Эльбрус 2000. Процессор Itanium использует архитектуру *EPIC*<sup>1</sup>, основанную на VLIW. Разработка эффективного компилятора для VLIW является сложнейшей задачей. Преимущество VLIW перед суперскалярной архитектурой заключается в том, что компилятор может быть более развитым, нежели устройства управления процессора, и он способен хранить больше контекстной информации для принятия более верных решений по оптимизации.

Машины VLIW заимствуют один из принципов работы RISC, возлагая на компилятор работу по планировке и упаковке в машинный код VLIW-инструкций, сгенерированных из высокоуровневого исходного кода.

В своё время Intel решила, что архитектура VLIW выглядит очень привлекательно — и начала разработку процессора VLIW, который станет её заявкой на вхождение в неминуемо наступающий мир 64-битных процессоров. Свою VLIW ISA<sup>2</sup> компания Intel назвала IA-64. Как обычно, Intel разработала собственную номенклатуру и свои названия для привычных терминов. На интел-жаргоне VLIW превратилась в EPIC. Архитектура EPIC не должна быть основана на наборе инструкций x86, отчасти для предотвращения копирования со стороны AMD.

Позже инженеры HP PA-RISC тоже решили, что у RISC потенциал развития почти исчерпан — и тоже заразились болезнью VLIW. В 1994 году компания HP объединила усилия с Intel для разработки совместной 64-битной архитектуры VLIW/EPIC. Результат назовут Itanium. Объявили целью выпустить первый процессор Itanium в 1998 году.

Однако вскоре стало ясно, что будет невероятно трудно разрабатывать процессоры и компиляторы для VLIW архитектуры. Intel не объявляла название Itanium до 1999 года (острословы в Usenet сразу же окрестили процессор «Итаником»), а первый рабочий процессор выпустили только в 2001 году. «Итаник» в итоге благополучно утонул в 2017 году, когда Intel объявила о завершении работ над IA-64. Окончательный финал этой истории: в 2019 году компания предупредила, что не будет принимать заказы на Itanium, начиная с 2020 года, а его поставки прекратятся (уже прекращены) с 29 июля 2021 года.

Архитектура EPIC стала по сути эпичным провалом, хотя в своё время казалась хорошей идеей. Процессоры Itanium, EPIC и VLIW умерли по нескольким причинам, говорит Паттерсон:

- Непредсказуемые ветвления, усложняющие планирование и упаковку параллельных операций в командные слова VLIW.
- Непредсказуемые промахи кэша замедляли выполнение и приводили к переменным задержкам выполнения.
- Наборы команд VLIW раздували объём кода.
- Оказалось слишком сложным создать хорошие оптимизирующие компиляторы для машин VLIW.

Пожалуй, самый известный в мире специалист по компьютерным алгоритмам Дональд Кнут заметил: «Подход Itanium... казался таким великолепным — пока не выяснилось, что желанные компиляторы по сути невозможно написать». Как оказалось, компиляторы гораздо лучше справляются с простыми архитектурами типа RISC.

---

<sup>1</sup> **EPIC** (Explicitly Parallel Instruction Computing) - архитектура процессора с явным параллелизмом команд. Термин введен в 1997 году фирмами HP и Intel для разрабатываемой архитектуры Intel Itanium. EPIC позволяет микропроцессору выполнять инструкции параллельно, опираясь на работу компилятора, а не выявляя возможность параллельной работы инструкций при помощи специальных схем. Это могло упростить масштабирование вычислительной мощности процессора без увеличения тактовой частоты.

<sup>2</sup> **ISA** — Instruction Set Algorithm.

## Иные архитектурные решения, типичные для RISC

- Спекулятивное исполнение. При встрече с командой условного перехода процессор исполняет (или, по крайней мере, читает в кэш инструкций) сразу обе ветви до тех пор, пока не окончится вычисление управляющего выражения перехода. Позволяет отказаться от простоев конвейера при условных переходах<sup>1</sup>.
- Переименование регистров. Каждый регистр процессора на самом деле представляет собой несколько параллельных регистров, хранящих несколько версий значения. Используется для реализации спекулятивного выполнения.

## Начало развития архитектуры RISC

Первая система, которая может быть названа системой RISC — суперкомпьютер CDC 6600, который был создан в 1964 году, практически за десять лет до появления соответствующего термина. CDC 6600 имел архитектуру RISC всего с двумя режимами адресации («регистр+регистр» и «регистр+непосредственное значение») и 74 кодами команд (тогда как, к примеру, i8086 имел 400 кодов команд). В CDC 6600 было 11 конвейерных устройств арифметической и логической обработки, а также пять устройств загрузки и два устройства хранения. Память была многоблочной (многовходовой), поэтому все устройства загрузки-хранения могли работать одновременно. Базовая тактовая частота/скорость выдачи команд была в 10 раз выше, чем времена доступа к памяти. Джим Торнтон и *Сеймур Крэй*<sup>2</sup>, разработчики CDC 6600, создали для него мощный процессор, позволявший быстро обрабатывать большие объёмы цифровых данных. Главный процессор поддерживался десятью простыми периферийными процессорами, выполнявшими операции ввода-вывода и другие функции ОС. Позднее появилась шутка, что термин RISC на самом деле расшифровывается как «Really invented by Seymour Cray» («на самом деле придуман *Сеймуром Крэем*»).

Ещё одна ранняя машина с архитектурой RISC — мини-компьютер «Data General Nova», разработанный в 1968 году.

Первая попытка создать процессор с архитектурой RISC на микросхеме была принята IBM в 1975 году. Эта работа привела к созданию семейства процессоров IBM 801, которые широко использовались в различных устройствах IBM. 801-й, в конце концов, был выпущен в форме микросхемы под именем ROMP в 1981 году. ROMP расшифровывается как «Research OPD (Office Product Division) Micro Processor», то есть «исследовательский микропроцессор», разработанный в подразделении офисных разработок. Как следует из названия, процессор был разработан для мини-задач, и когда в 1986 году IBM выпустила на его основе компьютер IBM RT-PC, он работал не слишком хорошо. Однако за выпуском 801-го процессора последовало несколько исследовательских проектов, в результате одного из которых появилась система «POWER».

Однако наиболее известные системы RISC в США были разработаны в рамках университетских исследовательских программ, финансировавшихся программой «DARPA VLSI».

## RISC в Беркли

Проект «RISC» в Университете Беркли был начат в 1980 году под руководством Дэвида Паттерсона и Карло Секвина. Исследования основывались на использовании конвейерной обработки и агрессивного использования *техники регистрового окна*. В обычном

<sup>1</sup> Ошибки в реализации спекулятивного (упреждающего) чтения привели к уязвимостям типа Meltdown и Spectre.

<sup>2</sup> **Сеймур Роджер Крэй** (*Seymour Roger Cray*; 28 сентября 1925 года, Чиппеуа Фоллс, Висконсин, США — 5 октября 1996 года, Колорадо-Спрингс, США) — американский инженер в области вычислительной техники, создатель ряда американских суперкомпьютеров. Славу изобретателю принесли компьютеры CDC 6600, CDC 7600, Cray.

процессоре имеется небольшое количество регистров, и программа может использовать любой регистр в любое время. В процессоре, использующем технологии регистрового окна, очень большое количество регистров (например, 128), но программы могут использовать ограниченное количество (например, только 8 в каждый момент времени).

Программа, ограниченная лишь восемью регистрами для каждой процедуры, может выполнять очень быстрые вызовы процедур: «окно» просто сдвигается к 8-регистровому блоку нужной процедуры, а при возврате из процедуры сдвигается обратно, к регистрам вызвавшей процедуры (в обычном процессоре большинство процедур при вызове вынуждены сохранять значения некоторых регистров в стеке для того, чтобы пользоваться этими регистрами при исполнении процедуры. При возврате из процедуры значения регистров восстанавливаются из стека).

Проект RISC произвёл на свет процессор RISC-I в 1982 году. В нём было 44 420 транзисторов (для сравнения: в процессорах CISC того времени их было около 100 тыс.). RISC-I имел всего 32 инструкции, но превосходил по скорости работы любой однокристальный процессор того времени. Через год, в 1983 году, был выпущен RISC-II, который состоял из 40 760 транзисторов, использовал 39 инструкций и работал в три раза быстрее RISC-I. Проект RISC-Беркли оказал влияние на RISC-процессоры семейства SPARC<sup>1</sup> и DEC Alpha.

### ***RISC в Стэнфорде***

Практически в то же время, в 1981 году, Джон Хеннесси начал аналогичный проект, названный архитектура MIPS в Стэнфордском университете. Создатель MIPS практически полностью сосредоточился на конвейерной обработке, попытавшись выжать всё из этой технологии. Конвейерная обработка использовалась и в других продуктах, некоторые идеи, реализованные в MIPS, позволили разработанному процессору работать значительно быстрее аналогов. Наиболее важным было требование выполнения любой из инструкций процессора за один такт. Это требование позволило конвейеру работать на гораздо больших скоростях передачи данных и привело к значительному ускорению работы процессора. С другой стороны, исполнение этого требования имело отрицательный побочный эффект в виде удаления из набора инструкций таких полезных операций, как умножение или деление.

В первые годы попытки развития архитектуры RISC были хорошо известны, однако оставались в рамках породивших их университетских исследовательских лабораторий. Многие в компьютерной отрасли считали, что преимущества процессоров RISC не проявятся при использовании в реальных продуктах из-за низкой эффективности использования памяти в составных инструкциях. Однако с 1986 года исследовательские проекты RISC начали выпускать первые коммерческие работающие изделия. RISC-процессор из Стэнфорда был реализован в процессорах семейства Rxxxx компании MIPS Technologies.

### **Последние годы**

Как оказалось, в начале 1990-х годов, RISC-архитектуры позволяют получить большую производительность, чем CISC, за счёт использования *суперскалярного* и *VLIW*-подхода, а также за счёт возможности серьёзного повышения тактовой частоты и упрощения кристалла с высвобождением площади под кэш, достигающего огромных объёмов. Также RISC-архитектуры позволили значительно снизить энергопотребление процессора за счёт уменьшения числа транзисторов.

---

<sup>1</sup> **SPARC** (Scalable **P**rocessor **AR**chitecture — *масштабируемая архитектура процессора*) — архитектура RISC-микропроцессоров, первоначально разработанная в 1985 году компанией Sun Microsystems. Любопытно, что в свою очередь Sun — аббревиатура от Stanford University Network.

Первое время RISC-архитектуры с трудом принимались рынком из-за отсутствия программного обеспечения для них. Эта проблема была решена переносом UNIX-подобных операционных систем (SunOS) на RISC-архитектуры.

В настоящее время многие архитектуры процессоров являются RISC-подобными, к примеру, ARM, DEC Alpha, SPARC, AVR, MIPS, POWER и PowerPC. Наиболее широко используемые в настольных компьютерах процессоры архитектуры x86 исходно были классическими CISC-процессорами, однако новые процессоры, начиная с Intel Pentium Pro (с 1995 г.), являются CISC-процессорами с RISC-ядром. Они непосредственно перед исполнением преобразуют CISC-инструкции x86-процессоров в более простой набор внутренних инструкций RISC.

После того, как процессоры архитектуры x86 были переведены на суперскалярную RISC-архитектуру, можно сказать, что большинство существующих ныне процессоров основано на архитектуре RISC.

На Рис.53 ниже приведён рисунок из книги Джона Хеннесси и Дэвида Паттерсона<sup>1</sup>, «Компьютерная архитектура. Количественный подход», 6-е изд. 2018, на котором графически показана история роста производительности процессоров:

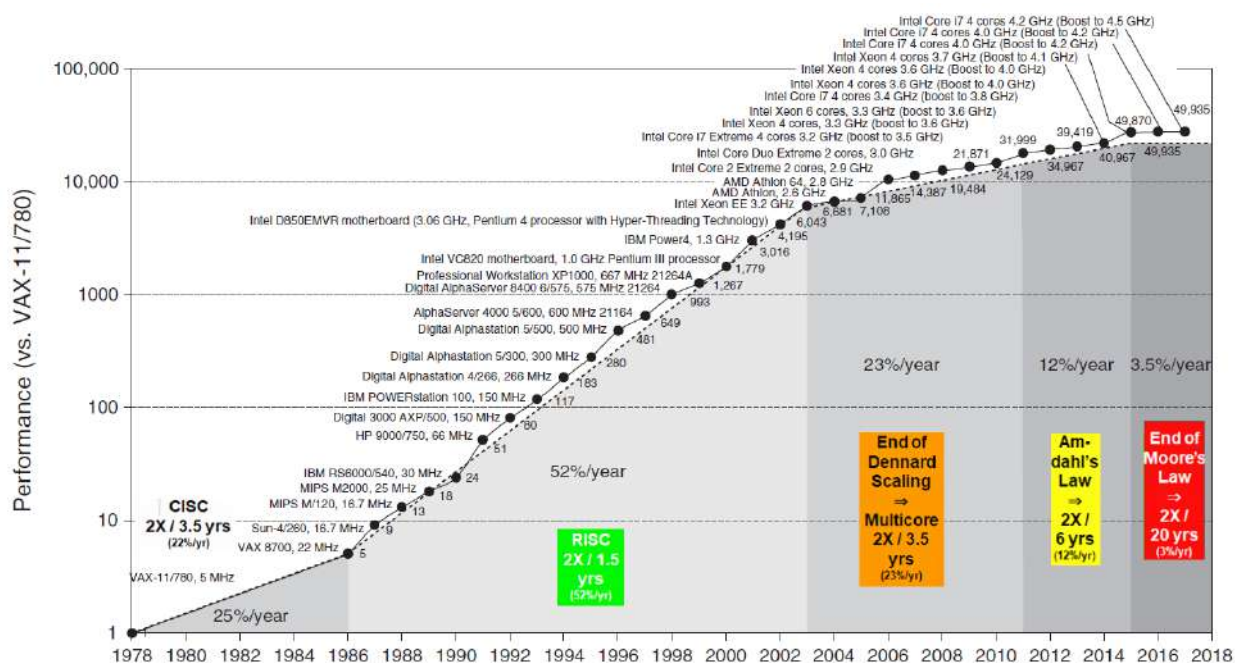


Рис.53 - 40 лет роста производительности процессоров

На графике видно, что микропроцессоры RISC обеспечили почти двадцать лет быстрого прироста производительности с 1986 по 2004 год, поскольку они эволюционировали по закону Мура (вдвое больше транзисторов на каждом новом витке техпроцесса) и закону масштабирования Деннарда (удвоение быстродействия при двукратном снижении энергопотребления на транзистор на каждом новом витке техпроцесса). Затем закон масштабирования Деннарда умер — и отдельные процессоры перестали ускоряться. Потребление энергии на транзистор также перестало уменьшаться вдвое на каждом этапе.

Промышленность компенсировала это, полагаясь исключительно на закон Мура по удвоению количества транзисторов — и быстро увеличивала количество процессоров на чипе, вступив в многоядерную эру. Интервал удвоения производительности вырос с 1.5 до

<sup>1</sup> Рисунок и текст ниже него (курсив) взяты из лекции «50 лет компьютерной архитектуры: от центральных процессоров до DNN TPU и Open RISC-V» д-ра Дэвида Паттерсона, которую он прочитал 15 марта 2018г. в большом конференц-зале здания в кампусе Texas Instruments в Санта-Кларе.

3,5 лет в течение этой эпохи, длившейся менее десяти лет, прежде чем вступил в силу закон Амдала (перефразированный как «в каждом приложении эксплуатируемый параллелизм ограничен»). Немногие приложения способны полностью загрузить десятки процессоров.

Тогда скончался и закон Мура.

По словам Паттерсона, результат такой, что с 2015 года рост производительности процессоров упал до ничтожных 3% в год. Удвоение по закону Мура теперь происходит не за 1,5 и даже не за 3,5 года. Теперь это двадцать лет.

Конец игры? «Нет», — говорит Паттерсон. В архитектуре процессоров можно попробовать ещё некоторые интересные вещи.

Один пример: специализированные архитектуры (DSA, Domain Specific Architectures) — специально разработанные процессоры, которые пытаются ускорить выполнение небольшого количества задач для конкретных приложений. Архитектуры VLIW не подошли для универсальных процессоров, но вполне имеют смысл для приложений DSP с гораздо меньшим числом ветвлений. Другой пример: Google TPU (Tensor Processing Unit), ускоряющий выполнение DNN (Deep Neural Network, глубокие нейросети) с помощью блока из 65 536 юнитов умножения-сложения (MAC) на одной микросхеме.

Оказывается, матричные вычисления с пониженной точностью — ключ к реализации действительно быстрых DNN. 65 536 восьмибитных блоков MAC в Google TPU работают на 700 МГц и выдают производительность 92 TOPS (тераопераций в секунду). Это примерно в 30 раз быстрее, чем серверный CPU и в 15 раз быстрее, чем GPU. Умножьте на сокращённое вдвое энергопотребление в 28-нанометровом TPU по сравнению с серверным CPU или GPU — и получите преимущество по энергопотреблению/мощности в 60 и 30 раз, соответственно.

Еще одна интересная вещь — создание архитектур ISA с открытым исходным кодом, говорит Паттерсон. Предыдущие такие попытки, в том числе OpenRISC и OpenSPARC, не взлетели, но Паттерсон рассказал о совершенно новой ISA с открытым исходным кодом — это RISC-V, которую он помогал разрабатывать в Беркли. Посмотрите на SoC, говорит Паттерсон, и вы увидите много процессоров с разными ISA. «Почему?», — задаёт он вопрос.

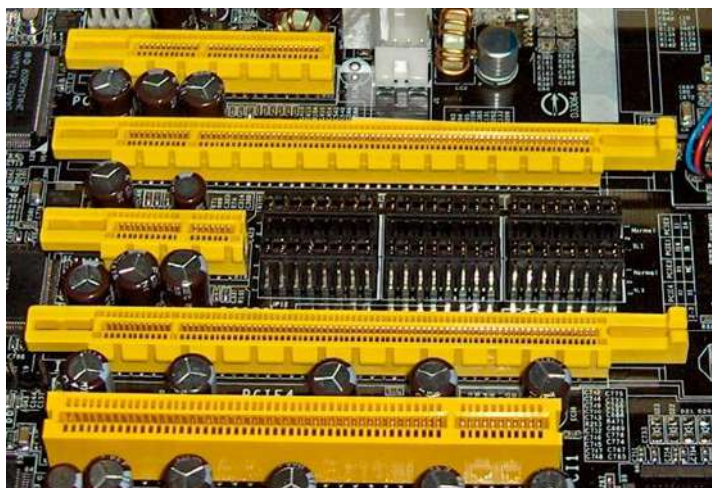
Зачем нам универсальная ISA, ещё одна ISA для обработки изображений, а также ISA для видеообработки, для аудиообработки и DSP ISA на одном кристалле? Почему бы не сделать только одну или несколько простых ISA (и один набор инструментов разработки программного обеспечения), которые можно многократно использовать для конкретных приложений? Почему бы не сделать ISA с открытым исходным кодом, чтобы каждый мог использовать эту архитектуру безвозмездно и улучшать её? Единственный ответом Паттерсона на эти вопросы — RISC-V ISA.

Недавно сформирован Фонд RISC-V, похожий по концепции на успешный Linux Foundation. В него вошли уже более 100 компаний, и он взял на себя работу по стандартизации RISC-V ISA. Миссия Фонда — способствовать внедрению RISC-V ISA и её будущему развитию. Так совпало, что «отошедший от дел» доктор Дэвид Паттерсон занимает должность заместителя председателя Фонда RISC-V.

## Компьютерные шины

**Компьютерная шина** (computer bus) в архитектуре компьютера — соединение, служащее для передачи данных между функциональными блоками компьютера. В устройстве шины можно различить механический, электрический (физический) и логический (управляющий) уровни.





**Рис.54** - Разъёмы шины PCI Express (сверху вниз: x4, x16, x1 и x16). Ниже — обычный 32-битный разъём параллельной шины PCI.

В отличие от соединения точка-точка, к шине обычно можно подключить несколько устройств по одному набору проводников (н-р, используя объединение выходов по технологии открытого коллектора или выхода с тремя состояниями). Каждая шина определяет свой набор разъёмов (соединений) для физического подключения устройств, карт и кабелей.

Компьютерные шины ранних вычислительных машин представляли собой жгуты (пучки соединительных проводов — сигнальных и питания, для компактности и удобства обслуживания увязанных вместе), реализующие параллельные электрические шины с несколькими подключениями. В современных вычислительных системах данный термин используется для любых физических механизмов, предоставляющих такую же логическую функциональность, как параллельные компьютерные шины.

Современные компьютерные шины используют как параллельные, так и *последовательные соединения*<sup>1</sup> и могут иметь параллельные (*multidrop*) и цепные (*daisy chain*) топологии. В случае USB и некоторых других шин могут также использоваться хабы (концентраторы).

Некоторые виды скоростных шин (Fibre Channel, InfiniBand, скоростной Ethernet, SDH) для передачи сигналов используют не электрические соединения, а оптические.

Присоединители к шине, разнообразные разъёмы, как правило, унифицированы и позволяют подключить различные устройства к шине.

Управление передачей по шине реализуется как на уровне прохождения сигнала (мультиплексоры, демультиплексоры, буферы, регистры, шинные формирователи), так и со стороны ядра операционной системы — в таком случае в его состав входит соответствующий драйвер.

## Описание шин

Шины бывают параллельными (данные переносятся по словам, распределённые между несколькими проводниками) и последовательными (данные переносятся побитово).

Шины могут быть *синхронными* (осуществляющими передачу данных только по тактовому импульсам) и *асинхронными* (осуществляющими передачу данных без привязки к тактовой частоте).

Большинство компьютеров имеет как внутренние, так и внешние шины. Внутренняя шина подключает все внутренние компоненты компьютера к материнской плате (и, следо-

<sup>1</sup> В области телекоммуникаций и информатике под термином **последовательное соединение** понимают процесс пересылки данных по одному биту за раз (последовательно) по каналу связи или компьютерной шине

вательно, к процессору и памяти). Такой тип шин также называют локальной шиной, поскольку она служит для подключения локальных устройств. Внешняя шина подключает внешнюю периферию к материнской плате.

Сетевые соединения, такие, как *Ethernet*, обычно не рассматриваются как шины, хотя разница больше концептуальная, чем практическая. Появление технологий *InfiniBand* и *HyperTransport* ещё больше размыло границу между сетями и шинами.

## История

### Первое поколение

Ранние компьютерные шины были группой проводников, подключающей компьютерную память и периферию к процессору. Почти всегда для памяти и периферии использовались разные шины, с разным способом доступа, задержками, протоколами.

Одним из первых усовершенствований стало использование прерываний. До их внедрения компьютеры выполняли операции ввода-вывода в цикле ожидания готовности периферийного устройства. Это было бесполезной тратой времени для программ, которые могли делать другие задачи. Также, если программа пыталась выполнить другие задачи, она могла проверить состояние устройства слишком поздно и потерять данные. Поэтому инженеры дали возможность периферии прерывать процессор. Прерывания имели приоритет, так как процессор может выполнять только код для одного прерывания в один момент времени, а некоторые устройства требовали меньших задержек, чем другие.

Классический простой способ обеспечить приоритеты прерываний или доступа к шине заключался в цепочечном подключении устройств – чем дальше от процессора, тем ниже приоритет устройства.

DEC отмечала, что две разные шины могут быть избыточными и дорогими для малых, серийных компьютеров, и предложила отображать периферийные устройства на шину памяти, так, что они выглядели как область памяти. В то время это было очень смелым решением, и критики предсказывали ему провал.

Первые мини-компьютерные шины представляли пассивные объединительные платы, подключенные к контактам микропроцессора. Память и другие устройства подключались к шине с использованием тех же контактов адреса и данных, что и процессор. Все контакты были подключены параллельно. В некоторых случаях, например, в IBM PC, необходимы дополнительные инструкции процессора для генерации сигналов, чтобы шина была настоящей шиной ввода-вывода.

Во многих микроконтроллерах и встраиваемых системах шины ввода-вывода до сих пор не существуют. Процесс передачи контролируется ЦПУ, который в большинстве случаев читает и пишет информацию в устройства, так, как будто они являются блоками памяти. Все устройства используют общий источник тактового сигнала. Периферия может запросить обработку информации путём подачи сигналов на специальные контакты ЦПУ, используя какие-либо формы прерываний. Например, контроллер жёсткого диска уведомит процессор о готовности новой порции данных для чтения, после чего процессор должен считать их из области памяти, соответствующей контроллеру. Почти все ранние компьютеры были построены по таким принципам, начиная от Altair с шиной S-100, заканчивая IBM PC в 1980-х.

Такие простые шины имели серьёзный недостаток для универсальных компьютеров. Всё оборудование на шине должно было передавать информацию на одной скорости и использовать один источник синхросигнала. Увеличение скорости процессора было непростым, так как требовало такого же ускорения всех устройств. Это часто приводило к ситуации, когда очень быстрым процессорам приходилось замедляться для возможности передачи информации некоторым устройствам. Хотя это допустимо для встраиваемых систем,

данная проблема непозволительна для коммерческих компьютеров. Другая проблема состоит в том, что процессор требуется для любых операций, и когда он занят другими операциями, реальная пропускная способность шины может значительно страдать.

Такие компьютерные шины были сложны в настройке, при наличии широкого спектра оборудования. Например, каждая добавляемая карта расширения могла требовать установки множества переключателей для задания адреса памяти, адреса ввода-вывода, приоритетов и номеров прерываний.

## Второе поколение

Компьютерные шины «второго поколения», например, **NuBus** решали некоторые из вышеперечисленных проблем. Они обычно разделяли компьютер на две «части», процессор и память в одной и различные устройства в другой. Между частями устанавливался специальный контроллер шин (*bus controller*). Такая архитектура позволила увеличивать скорость процессора без влияния на шину, разгрузить процессор от задач управления шиной. При помощи контроллера устройства на шине могли взаимодействовать друг с другом без вмешательства центрального процессора. Новые шины имели лучшую производительность, но также требовали более сложных карт расширения. Проблемы скорости часто решались увеличением разрядности шины данных, с 8-битных шин первого поколения до 16- или 32-битных шин во втором поколении. Также появилась программная настройка устройств для упрощения подключения новых устройств, ныне стандартизованная как **Plug-n-play**.

Однако новые шины, так же, как и предыдущее поколение, требовали одинаковых скоростей от устройств на одной шине. Процессор и память теперь были изолированы на собственной шине, и их скорость росла быстрее, чем скорость периферийной шины. В результате шины были слишком медленны для новых систем, и машины страдали от нехватки данных. Один из примеров данной проблемы: видеокарты быстро совершенствовались, и им не хватало пропускной способности даже новых шин **Peripheral Component Interconnect (PCI)**. Компьютеры стали включать в себя **Accelerated Graphics Port (AGP)** только для работы с видеоадаптерами. В 2004 году AGP снова стала недостаточно быстрой для мощных видеокарт, и AGP стал замещаться новой шиной **PCI Express**.

Для обслуживания увеличивающегося количества внешних устройств, контроллеры стали применять собственные шины. Когда были изобретены приводы дисков, они присоединялись к машине при помощи карты, подключаемой к шине. Для этого компьютеры имели много слотов расширения. Но в 1980-х и 1990-х были изобретены новые шины **SCSI** и **IDE**, решившие эту проблему, оставив большую часть разъёмов расширения в новых системах пустыми. В наше время типичная машина поддерживает около пяти различных шин.

Шины стали разделять на внутренние (*local bus*) и внешние (*external bus*). Первые разработаны для подключения внутренних устройств, таких, как видеоадаптеры и звуковые платы, а вторые предназначались для подключения внешних устройств, например, сканеров. IDE является внешней шиной по своему предназначению, но почти всегда используется внутри компьютера.

## Третье поколение

Шины «третьего поколения» (например, **PCI-Express**) обычно позволяют сочетать как высокие скорости, требуемые при обмене с памятью, видеокартами или для межпроцессорного взаимодействия, так и относительно невысокие при работе с медленными устройствами, например, приводами дисков. Также шины третьего поколения стремятся к большей гибкости в терминах физических подключений, позволяя использовать себя и как внутренние, и как внешние шины, например, для объединения компьютеров. Это приводит к сложным проблемам при удовлетворении различных требований, так что большая часть работ по данным шинам связана с программным обеспечением, а не с самой аппаратурой.

В общем, шины третьего поколения больше похожи на компьютерные сети, чем на изначальную идею шин, с большими накладными расходами, чем у ранних систем. Также они позволяют использовать шину нескольким устройствам одновременно.

Современные интегральные схемы часто «собираются» из ранее разработанных компонентов, и для более простой интеграции этих частей в такой схеме, часто используются специальные шины, например, *Wishbone*.

## Примеры внутренних компьютерных шин

### *Параллельные*

- Проприетарная Asus Media Bus, использовалась на некоторых материнских платах ASUS с Socket 7 и представляла собой шину ISA в специфическом разъеме, размещенном в одну линию с разъемом шины PCI.
- CAMAC (КАМАК) для измерительных систем (*instrumentation systems*)
- Extended ISA или EISA
- Industry Standard Architecture или ISA
- Low Pin Count или LPC
- MicroChannel или MCA
- MBus
- Multibus для промышленных систем
- NuBus или IEEE 1196
- OPTi local bus, использовалась для ранних материнских плат для Intel 80486
- Peripheral Component Interconnect или PCI, также PCI-X
- S-100 bus или IEEE 696, использовалась в Altair и подобных микрокомпьютерах
- SBus или IEEE 1496
- VESA Local Bus или VLB или VL-bus, использовалась в основном на материнских платах для 80486 процессоров и была подключена непосредственно к выводам микропроцессора. Однако встречалась и реализация этой шины в сочетании с ЦПУ IBM BL3 (аналог i386SX) и ранними Pentium
- VMEbus, VERSAmodule Eurocard bus
- STD Bus для 8- и 16-битных микропроцессорных систем
- Unibus
- Q-bus

### *Последовательные*

- 1-Wire
- HyperTransport
- I<sup>2</sup>C
- PCI Express или PCIe
- Serial Peripheral Interface Bus или шина SPI
- USB, Universal Serial Bus, чаще используется как внешняя
- FireWire, i.Link, IEEE 1394, чаще используется как внешняя
- Direct Media Interface (DMI)
- Intel QuickPath Interconnect или просто QuickPath (QPI)
- SATA/SAS

## Примеры внешних компьютерных шин

- Advanced Technology Attachment или ATA (также известна как PATA, IDE, EIDE, ATAPI) — шина для подключения дисковой и ленточной периферии.

- SATA, Serial ATA — последовательный (serial) вариант ATA
- USB, Universal Serial Bus, используется для подключения внешних устройств
- HIPPI, HIgh Performance Parallel Interface
- IEEE-488, GPIB (General-Purpose Instrumentation Bus), HPIB, (Hewlett-Packard Instrumentation Bus)
- PC card, ранее известная как *PCMCIA*, часто используется в ноутбуках и других портативных компьютерах, но теряет своё значение с появлением USB и встраиванием сетевых карт и модемов
- SCSI, *Small Computer System Interface*, шина для подключения дисковых и ленточных накопителей
- Serial Attached SCSI, SAS — последовательный вариант SCSI

## Примеры универсальных компьютерных шин

- Futurebus
- InfiniBand
- QuickRing
- SCI

На примере эволюции архитектуры РС можно проследить однозначную тенденцию по переходу от параллельных шин к последовательным. При повышении тактовых частот у параллельных шин всё сильнее начинают сказываться две основных проблемы:

- Точность (синхронность) фронтов/срезов сигналов на линиях разных разрядов относительно друг-друга. На очень высоких тактовых частотах, с очень короткими ( $<1\text{нс}$ ) фронтами в задачу начинают входить разница по времени распространения сигналов. Усугубляет эту проблему «дрожание» сигнала — т.н. джиттер<sup>1</sup>. На модулях динамической памяти DDR, работающих на частотах в сотни мегагерц, для выравнивания времён распространения сигналов используют разводку проводников не только под прямым углом или под  $45^\circ$ , а в виде извилистой дорожки, для увеличения длины.
- Взаимные наводки от очень коротких перепадов напряжения/тока на соседние проводники.

В своё время долго разработчики долго пытались сделать очередной шаг по повышению производительности (удвоение скорости) параллельной шины (интерфейса) SCSI. Но надёжно обеспечить скорость передачи выше 320 Мбайт/сек им так и не удалось. Полоса пропускания 320 Мбайт/сек делилась между всеми подключёнными к шине устройствами (максимум до 15 штук). Последовательный **Serial Attached SCSI (SAS)**, разработанный на замену параллельного SCSI, по сути реализует подключение точка-точка, обеспечивая каждому подключённому устройству индивидуальную полосу пропускания 3 Гбит/с, 6 Гбит/с или 12 Гбит/с.

SAS обратно совместим с интерфейсом SATA: устройства 3 Гбит/с и 6 Гбит/с SATA могут быть подключены к контроллеру SAS, но не наоборот. SATA пришёл на замену параллельному (P)ATA/IDE, который смог достичь максимальной скорости 133 Мбайт/с (делилась между двумя устройствами).

Когда-то на первой РС был один последовательный интерфейс, стандарта RS-232-C, всё остальное было реализовано на параллельных шинах, а сейчас практически все внут-

---

<sup>1</sup> Джиттер (*jitter* — дрожание) или **фазовое дрожание цифрового сигнала данных** — нежелательные фазовые или частотные отклонения передаваемого сигнала.

рение и внешние шины в современных стали последовательными. Даже, казалось бы, «параллельный» 16x PCI Express на самом деле представляет собой шестнадцать последовательных независимых линий (lane), каждая из которых подключена по схеме точка-точка.

Но параллельные, обычно очень «широкие», шины в современных архитектурах, всё-таки присутствуют, там, где важно передавать сразу очень широкие слова за как можно более короткое время. Такие сверширокие шины могут работать на высоких частотах только на очень коротких расстояниях<sup>1</sup>, и реализованы, например, в шине данных для обмена CPU <=> Кэш-память. В некоторых моделях серверных процессоров Intel ширина шины CPU <=> Кэш достигает 576 бит! Ещё пример: во втором поколении процессоров архитектуры Zen 2 (Ерус 2) компании AMD межпроцессорная шина Infinity Fabric имеет ширину 512 бит.

## Организация ввода-вывода информации

Под вводом–выводом в ЭВМ понимают передачу данных из периферийного устройства в оперативную память (ввод) или наоборот – из памяти на периферийное устройство (вывод). Ввод/вывод (V/B, I/O) обычно работает медленнее, чем производится обработка данных на процессоре, поэтому его организации уделяется значительное внимание. В идеале архитектура ввода–вывода должна обеспечивать обмен информацией между процессором и периферийными устройствами без снижения производительности системы в целом.

Адресное пространство ЭВМ может быть организовано по схеме с отдельными адресными пространствами ОЗУ и ввода-вывода или по схеме с совмещённым (общим) адресным пространством. Примеры первого варианта: микроЭВМ на базе i8080, ранние PC на базе i8086/8 (в более новых поколениях PC ввод-вывод стал реализовываться по схеме с общим адресным пространством). Классический пример архитектур с отображением пространства ввода-вывода в конец адресного пространства ОЗУ: DEC-овские архитектуры PDP-/LSI-11, VAX-11.

Различают три способа организации ввода–вывода:

- Программно управляемый
- Работа по прерываниям
- Прямой доступ к памяти (DMA)



Рис.55 - Обобщённая схема устройства ввода-вывода

## Программно управляемый ввод/вывод

Самым простым методом организации ввода/вывода является программно управляемый ввод–вывод, ещё называемый вводом/выводом с *опросом флага* или *условным*. В этом

<sup>1</sup> Линия передачи цифровых сигналов считается электрически *короткой*, если время распространения сигнала от передатчика к приёмнику меньше половины длительности его фронта.

случае ввод–вывод происходит под полным контролем центрального процессора. Основой метода является опрос процессором флага ПУ (периферийного устройства) в регистре состояния (проверка флага готовности). Когда устройство готово к обмену, оно устанавливает этот флаг в 1. Процессор опрашивает ПУ и, обнаружив в регистре состояния «Флаг готовности» = 1, выполняет обмен (чтение – ввод, запись – вывод из/в регистр(-а) данных).



Рис.56 - Пример ввода (чтения) данных по готовности ПУ

После выполнения операции чтения данных в данном примере флаг готовности сбрасывается в 0. Метод очень прост, требует минимальных затрат с точки зрения аппаратуры. Основным недостатком метода является очень нерациональное расходование процессорного времени на ожидание готовности устройства к обмену<sup>1</sup>; особенно если учесть, что обычно скорость работы большинства ПУ существенно меньше скорости работы ЦП.

Предположим, что мы работаем на компьютере, CPU которого функционирует с тактовой частотой 1 ГГц (период 1нс), и запустили на нём программу, в которой циклически опрашиваем клавиатуру, чтобы определить факт нажатия клавиши. При зажатой постоянно клавише клавиатура РС генерирует факт «нажатия» с частотой ~30 Гц или с периодом ~33 мс. Т.е., при работе по готовности (опрос) наш процессор выполнит около  $33\text{мс}/1\text{нс} = 3.3 \cdot 10^{-2} / 10^{-9} = 33.000.000$  – тридцати трёх миллионов операций чтения регистра состояния, из которых только одно будет «полезным», а остальные – по сути будут просто разогревать CPU, создавая для него 100%-ую нагрузку.

## Ввод/вывод по прерываниям

Идея **прерывания** была предложена в середине 1950-х годов и основная цель введения прерываний – реализация синхронного режима работы и реализация параллельной работы отдельных устройств ЭВМ.

**Прерывания и обработка прерываний** зависят от типа ЭВМ, поэтому их реализацию относят к машинно-зависимым свойствам операционных систем.

<sup>1</sup> При попытке запустить исполняемый файл MS-DOS (н-р, текстовый редактор MultiEdit) в многозадачной ОС типа Windows приведёт к 100%-ой загрузке CPU, т.к. для однозадачной MS-DOS обычным методом ввода-вывода был программный опрос готовности ПУ. Текстовый редактор просто бесконечно циклически опрашивает готовность клавиатуры, ожидая нажатия/отжатия клавиш, полностью загружая при этом процессор.

**Прерывание** (interrupt) – это сигнал, заставляющий ЭВМ менять обычный порядок выполнения команд процессором. Прерывания могут быть синхронными (возникающими синхронно с выполнением основного потока программы, например, команда INT в системе команд x86 или исключения/exception, например, деление на ноль) или асинхронными – обычно это внешние события от оборудования, на которые процессор должен «обратить внимание» – это могут быть события (флаг) нажатия на клавишу, готовность данных от дискового контроллера, прерывания от сетевого адаптера и т.п.

Если в случае программного ввода/вывода (опрос) процессор обнаруживает готовность устройства, многократно опрашивая его статусный регистр, то в случае прерывания само ПУ сообщает о своей готовности, выставляя сигнал «запроса/требования прерывания». Процессор в это время может заниматься совсем другими задачами, никак не связанными с вводом/выводом. В большинстве архитектур реакция на запрос прерывания (Interrupt Request, IRQ) реализуется в виде микроветвления, когда, после окончания выполнения текущей инструкции (во время обработки которой возник запрос прерывания) процессор принимает решение – перейти к обработке следующей инструкции или «отвлечься» на обработку возникшего события (запроса прерывания).

В некоторых архитектурах (например, в ранних LSI-11) некоторые «длинные» инструкции, которые выполнялись очень долго по сравнению с более простыми базовыми командами, было разрешено прерывать, не выполняя их до конца. Иначе, если бы реакция на запрос прерывания следовала бы, как обычно, по окончании выполнения таких «длинных» инструкций, это привело бы к неоправданно большим задержкам по времени реакции на события (в таких случаях говорят, что система имеет плохую *реактивность*). В случае прерываемых «длинных» инструкций процессор, после обработки прерывания, возвращается к началу выполнения прерванной команды, пытаясь выполнить её полностью заново.

При реакции на прерывание процессор должен запомнить точку возврата, в которую он вернётся после обработки прерывания – это адрес следующей инструкции, которую процессор бы выполнил, если бы его не «отвлекли» на обработку возникшего события. Также необходимо запомнить текущий статус процессора, его флаги – перенос, переполнение и т.п. – чтобы, вернувшись в точку программы, где он был прерван, процессор продолжил дальнейшее выполнение программы так, как он бы работал в отсутствие прерывания (и как бы ничего не заметил).

Очень часто, для корректной работы, при обработке прерывания приходится сохранять весь контекст процесса – не только адрес возврата, слово состояния процессора (флаги, PSW), но и регистры общего назначения. Обычно в качестве места сохранения контекста используют стек<sup>1</sup>.

На Рис.57 приведена последовательность действий при обработке прерывания:

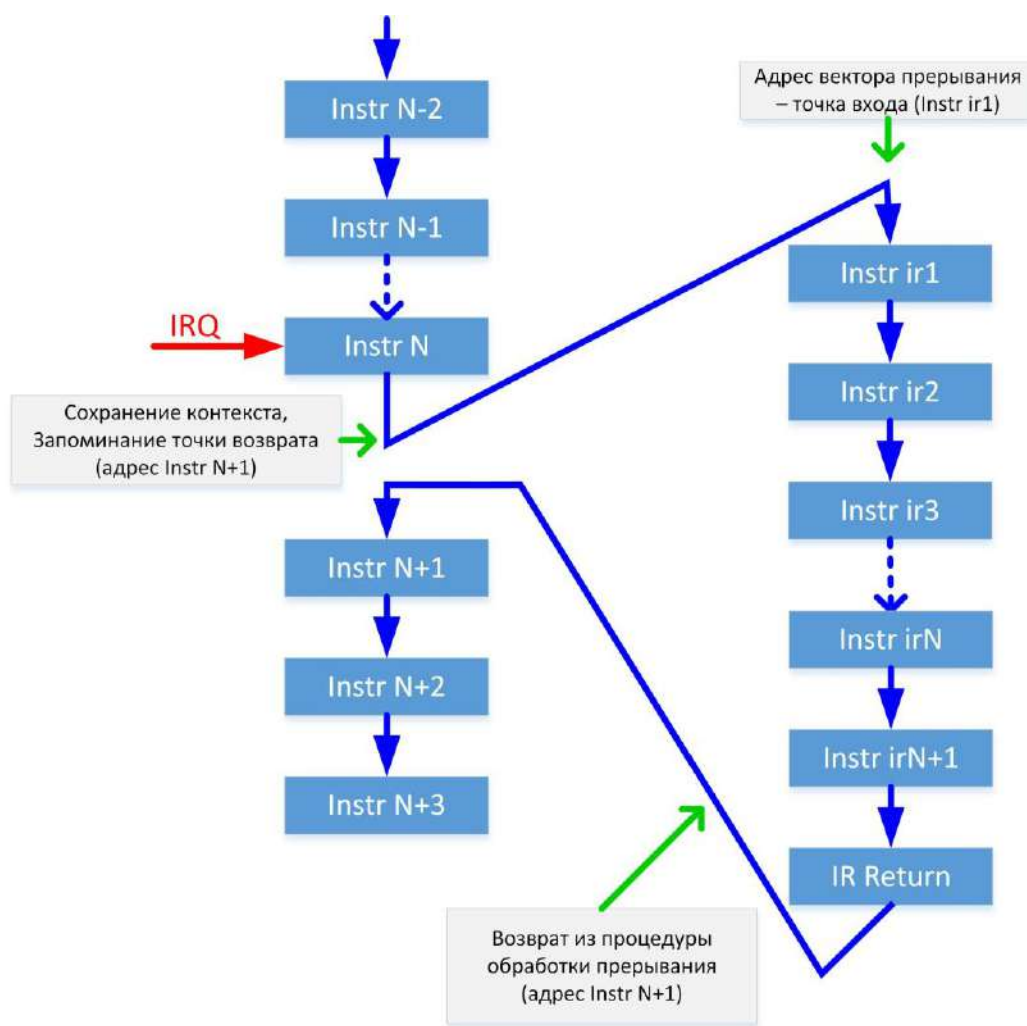
- Запрос на прерывание IRQ (асинхронный) возник во время выполнения инструкции “Instr N”;
- Если прерывания от этого устройства разрешены, процессор, завершив выполнение текущей инструкции “Instr N”, запоминает точку возврата (адрес следующей инструкции “Instr N+1”) и слово состояния процессора (PSW);
- Осуществляется переход по адресу *вектора прерывания* – адресу точки входа, первой инструкции, с которого начинается подпрограмма обработки данного события (“Instr ir1”). Каждое устройство, генерирующее прерывание, должно однозначно обозначить себя, т.е. каждому событию (прерыванию) должна соответствовать своя уникальная процедура обработки этого события (адрес вектора прерывания). Так как обработку ведёт процессор, он должен точно знать, кто конкретно его прерывает;

---

<sup>1</sup> **Стековой** называют память, доступ к которой организован по принципу: «последним записан - первым считан» (LIFO).



- В начале процедуры обработки прерывания (“Instr ir1”) программа может сохранить дополнительные данные контекста, например, содержимое регистров общего назначения (РОН);
- Обработав событие, последней инструкцией в обработке должна быть команда «возврат из прерывания» (“IR Return”), которая осуществляет переход к выполнению «отложенной на время» инструкции “Instr N+1”;
- Результатом обработки должна стать деактивация запроса прерывания, иначе при возврате из обработки произойдёт повторное прерывание по тому же самому событию.



**Рис.57 - Процедура обработки прерывания**

Если внимательно посмотреть на последовательность действий при обработке прерывания, то можно увидеть, что она практически неотличима от последовательности при вызове подпрограммы (CALL): точно также запоминается адрес возврата, контекст – и осуществляется переход по адресу подпрограммы, а после выполнения подпрограммы выполняется возврат из подпрограммы и возобновление выполнения основной программы с инструкции, следующей за инструкцией CALL.

Эта схожесть механизмов обработки прерывания и CALL была поддержана, можно сказать, на аппаратном уровне в контроллере прерываний Intel 8259: процессор, при обработке прерывания, фактически вычитывал из контроллера прерывания (который должен был быть запрограммирован соответствующим образом) команду “CALL vector\_address”. Одним из этапов программирования контроллера прерывания i8259 была загрузка таблицы, сопоставляющей номер входа запроса прерывания с адресом вектора.

Реализация обработки прерываний – это комплекс аппаратно-программных средств и далеко не во всех архитектурах обработка прерывания реализована быстро и эффективно. Например, в интеловских архитектурах x86/x64 обработка прерывания довольно ресурсоёмкий и затратный процесс. Современный многогигагерцовый многоядерный процессор интеловской архитектуры вряд ли справится с потоком прерываний даже в несколько сотен килогерц. Любопытно, что гораздо менее производительные одноядерные процессоры DEC LSI-11 (работавшие на частотах на три порядка ниже современных интеловских), легко обеспечивали гарантированное время реакции на прерывание в 20-30 мкс.

Большие накладные расходы на обработку прерывания иногда даже вынуждают разработчиков драйверов<sup>1</sup> переходить на, казалось бы, совершенно неэффективный метод обмена с опросом готовности. Так поступили, в частности разработчики драйверов для гигабитных сетевых адаптеров Ethernet Intel – при работе с передатчиком по готовности (опрос) на системах с не очень производительным CPU удаётся достичь более высокой общей производительности, чем при работе по прерываниям, хотя при этом растёт суммарная нагрузка процессора.

## Организация прямого доступа к памяти (ПДП/DMA)

При переходе от программного ввода-вывода (с опросом) к обработке по прерываниям мы разгружаем процессор от большого объёма, по сути, бесполезной работы с чрезвычайно низким к.п.д. (пример выше с опросом клавиатуры). Теперь процессору есть чем заняться, вместо того чтобы 33млн./раз в секунду «вхолостую» читать статусный регистр.

Но при обмене данными с блочными устройствами (например, с дисковым контроллером) процессору всё равно приходится заниматься рутинной работой по пересылке данных из ПУ в память или обратно. Чтобы ещё больше освободить процессор от такого типа простых операций ввода-вывода, был разработан механизм прямого доступа к памяти (ПДП, DMA – Direct Memory Access), в котором устройство ввода/вывода самостоятельно (минуя процессор) осуществляет пересылку данных между ОЗУ и ПУ.

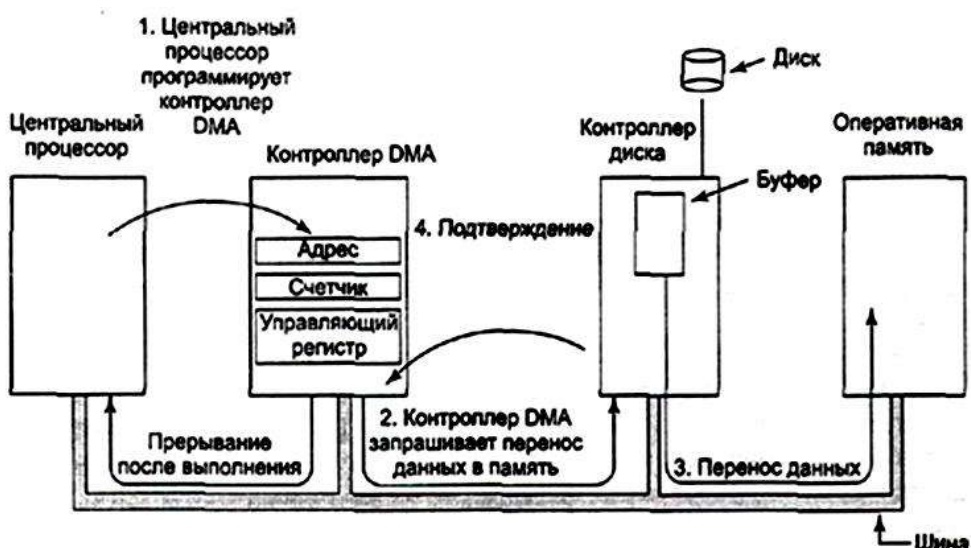


Рис.58 - Ввод-вывод с использованием прямого доступа

На Рис.58 приведена последовательность действий при вводе-выводе с использованием прямого доступа к памяти (DMA):

<sup>1</sup> **Драйвер** (*driver*, мн. ч. *драйверы*) — компьютерное программное обеспечение, с помощью которого другое программное обеспечение (операционная система) получает доступ к аппаратному обеспечению некоторого устройства.

- Процессор инициализирует контроллер DMA. Минимальная инициализация включает в себя:
  - Задание начального адреса в ОЗУ, начиная с которого будут записаны (прочитаны) байты данных (регистр адреса);
  - Задание счётчика (количества) байт, которые будут пересланы при обмене по прямому доступу, обычно этот счётчик работает на вычитание;
  - Задание направления передачи – запись/чтение (относительно ОЗУ).
- После инициализации контроллер DMA готов к обмену, который может быть инициирован CPU или ПУ;
- Цикл обмена по прямому доступу начинается с запроса контроллера DMA на «захват шины». Если процессор готов освободить шину, он отключится от неё (переведя свои линии адреса/данных в третье высокоимпедансное состояние) и подтверждает запрос на «захват шины»;
- Захватив шину, контроллер начинает генерировать циклы обмена на шине, неотличимые от циклов, генерируемых процессором, пересылая данные из буфера контроллера дисков в ОЗУ (запись) или наоборот, читая данные из ОЗУ (инкрементируя адрес при каждой следующей пересылке) и пересылая их в буфер контроллера диска;
- После завершения цикла обмена (антипереполнение счётчика, переход из 00...00 → 11...11) контроллер обычно генерирует прерывание, сообщаящее процессору, что цикл DMA завершён.

Контролер DMA работает фактически как ещё один «хозяин шины» (bus master) по отношению к CPU. Вариация прямого доступа, при которой контроллер DMA инициализируется не CPU, а самостоятельно (по прямому доступу) забирает свои параметры (канальная программа) из ОЗУ, делает его ещё более похожим по поведению на центральный процессор. Такая способ организации ввода-вывода носит название ПВВ – процессор ввода-вывода.

## ГЛАВА 3. АРИФМЕТИЧЕСКИЕ ОСНОВЫ ЭВМ

### Выбор системы счисления ЭВМ

В механических и электромеханических вычислительных устройствах было довольно просто реализовать десятичную систему счисления, наиболее понятную и естественную для человека – сделать шестерёнку с нужным (десять) количеством зубьев, с организацией «переноса» в следующий разряд – задача относительно несложная. Но при реализации «цифры» в электронике – практически нереализуемая. Наиболее простым и естественным для цифровой электроники оказалось использование двоичной системы счисления. Логические '0' и '1' легко могут быть представлены различными физическими сущностями, например, током: тока нет – '0', ток есть – '1', напряжением: нулевое напряжение – логический ноль, напряжение отлично от нуля – единица, включено/выключено и т.п.

Под *системой счисления* понимается способ представления числовых данных с помощью некоторого ограниченного алфавита символов.

В вычислительной технике и программировании применяются только *позиционные системы счисления*. В таких системах счисления позиция цифры в числе определяет её вес, а общее количество символов алфавита – *основание системы счисления*.

*Непозиционные системы* (к ним, например, относится римская система счисления) из-за сложности и неформализуемости в ЭВМ использоваться не могут.

Из позиционных систем счисления, кроме общепринятой (при взаимодействии человек-человек) десятичной, на различных этапах подготовки и выполнения программ на ЭВМ применяются *двоичная, восьмеричная, шестнадцатеричная* системы счисления. Необходимо подчеркнуть, что все системы счисления, кроме двоичной, используются, если можно так сказать, «на бумаге», на упомянутых выше этапах подготовки программ. При выполнении программ, в «железе», все современные ЭВМ, практически без исключения, используют двоичные логические элементы, двоичную систему счисления.

Не сужая области применения ЭВМ, зададимся величиной самого большого (целого) числа. Обозначим его как *M*. Если воспользоваться позиционной системой счисления с *основанием p*, тогда нам потребуется *n* разрядов, чтобы представить все *M* чисел:

$$M = p^n - 1 \quad (\text{от } 0 \text{ до } p^n - 1)$$

$$M \approx p^n$$

$$\log_p M = n \cdot \log_p p, \text{ где } \log_p p = 1, \text{ тогда } n = \log_p M$$

Оборудование<sup>1</sup>, которое нужно для хранения любого числа от 0 до *M* пропорционально произведению *основания системы счисления* на количество *разрядов* (произведение «глубины» разряда на «ширину» слова).

Таким образом, при заданном числе *M* количество цифроразрядов при *основании p*:

$$p \cdot n = p \cdot \log_p M (*), \text{ где:}$$

*цифроразряд* – эквивалент (количества) оборудования,  
*p \* n* – число устойчивых состояний элемента памяти,  
*n* – число *разрядов* в числе.

<sup>1</sup> Здесь имеется в виду некоторая количественная характеристика – количество транзисторов, вентилях и т.п.

**Рассмотрим пример:** пусть у нас есть 24 цифроразряда.

| Основание $p$ | Возможное число цифроразрядов | Наибольшее число $M$                                                                                                         |
|---------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| 2             | $2 \cdot 12$                  | $1 \cdot 1^* \dots \cdot 1_{12} = 4095_{10}$<br>$\quad \quad \quad \backslash \quad \quad \quad /$<br>$\quad \quad \quad 12$ |
| 3             | $3 \cdot 8$                   | $2 \cdot 2^* \dots \cdot 2_8 = 6560_{10}$<br>$\quad \quad \quad \backslash \quad \quad \quad /$<br>$\quad \quad \quad 8$     |
| 4             | $4 \cdot 6$                   | $3 \cdot 3^* \dots \cdot 3_6 = 4095_{10}$<br>$\quad \quad \quad \backslash \quad \quad \quad /$<br>$\quad \quad \quad 6$     |
| 6             | $6 \cdot 4$                   | $5 \cdot 5^* \dots \cdot 5_6 = 1295_{10}$<br>$\quad \quad \quad \backslash \quad \quad \quad /$<br>$\quad \quad \quad 4$     |
| 8             | $8 \cdot 3$                   | $7 \cdot 7^* \dots \cdot 7_8 = 511_{10}$<br>$\quad \quad \quad \backslash \quad \quad \quad /$<br>$\quad \quad \quad 3$      |

Количество цифроразрядов говорит как о величине оборудования, так и является некой характеристикой быстродействия. В позиционной *системе счисления* времена выполнения операций могут быть выражены через количество *разрядов* в числе.

Считаем  $p$  – величиной непрерывной. Находим первую и вторую производную от (\*) по величине  $p$ . И оказывается, что первая производная обращается в нуль, а вторая – больше нуля при  $p = e$ . Т.е. минимум функции (\*) достигается при  $p = e$ .

Таким образом, оптимальной по оборудованию и быстродействию является система с основанием  $e$ .

Но  $e$ , как известно, число не очень простое, а скорее иррациональное ☺:

$$e = \lim_{x \rightarrow \infty} \left( 1 + \frac{1}{x} \right)^x \quad \text{или} \quad e = \lim_{n \rightarrow \infty} \frac{n}{\sqrt[n]{n!}}$$

$$e = 2,7182818284590452353602874713527\dots$$

Поэтому в реальной жизни оптимальной является система с основанием  $p = 3$ , ближайшим целым к  $e$ . Если построить функцию, характеризующую отношение оборудования в системе с основанием  $p$  относительно системы с основанием 2...

| $p$    | 2     | 3     | 4     | 5     | 6     | 7     | 8     | 9     | 10    |
|--------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| $f(p)$ | 1,000 | 0,946 | 1,000 | 1,078 | 1,148 | 1,247 | 1,333 | 1,420 | 1,595 |

то становится видно, что десятичная система счисления является более чем в 1,5 менее экономичной по сравнению с двоичной, а самая эффективная троичная оказывается лишь на 5% экономичнее двоичной.

Реализовать в электронике работу в троичной логике на практике оказалось не на много сложнее, чем работу в более простой двоичной. В истории отечественных компьютеров была даже серийно выпускавшаяся ЭВМ «Сетунь», работавшая в троичной логике (для обозначения одного разряда для неё использовался термин **трит**<sup>1</sup>, вместо **бит** в двоичной). Но 5% экономии (количества) оборудования по сравнению с двоичными машинами

<sup>1</sup> **Трит** — логарифмическая единица измерения в теории информации, минимальная целая единица измерения количества информации источников с тремя равновероятными сообщениями. Энтропию в 1 трит имеет источник информации с тремя равновероятными состояниями. Проще говоря, по аналогии с битом, который «уменьшает незнание» об исследуемом объекте в два раза, трит «уменьшает незнание» в три раза.

явно не стали определяющим при выборе системы счисления для основной массы вычислительных систем, и троичная система в «Сетунь» осталась своего рода экзотическим концептом, не получив дальнейшего развития.

В действительное обоснование экономичности той или иной системы выглядит несколько сложнее.

Когда мы говорим об экономичности, то, прежде всего, имеем ввиду объем оборудования, сосредоточенный в АЛУ (арифметически-логическое устройство) и ЗУ (запоминающее устройство). Объем оборудования УУ (устройство управления) не находится в столь простой зависимости от  $p$ , да и в АЛУ учитывается лишь оборудование, связанное с элементами хранения информации, но не сопутствующее логическое оборудование. Более детальный анализ показывает, что наиболее эффективными являются системы с основанием, кратным степени 2, т.е. 2, 4, 8, 16.

В реальности, на практике, наиболее эффективной считается система с основанием 2, по причине её наибольшего распространения. Основные соображения в пользу двоичной системы:

1. Высокая информационная эффективность.
2. Простота и надежность работы двоичного элемента хранения информации (имеющего два устойчивых состояния)
3. Совпадение максимального числа состояний элемента с максимальным числом значений двоичной переменной, дающее возможность не строить специальные устройства для выполнения логических операций.
4. Простота построения схем для выполнения простых операций.
5. Более высокая скорость выполнения основных арифметических операций.

П.5 требует дополнительного пояснения. В данном случае рассматриваются не отрезки времени, необходимые для выполнения тех или иных операций, а скорость, определяемая косвенно по относительному числу операций, которые требуется провести для выполнения, например, деления или умножения в двоичной или прочих системах.

Если  $p$  – основание системы счисления, то максимальная цифра в одном разряде (по определению) равна  $p-1$ .

Если  $N$  – максимальное число, то для его изображения требуется  $\log_p N$  разрядов (это, по сути, определение логарифма).

Для того чтобы выполнить операцию умножения, например, потребуется  $(p-1) * \log_p N$  операций сложения. Если сравнить это число операций в системе с основанием  $p$  и отнести его к числу операций в системе с основанием 2, то может получиться следующая функция:

$$f(p) = \frac{(p-1) * \log_p N}{1 * \log_2 N} = \frac{p-1}{\log_2 p}$$

В табличном виде:

| n    | 2     | 3     | 4     | 5     | 6     | ... | 10    |
|------|-------|-------|-------|-------|-------|-----|-------|
| f(n) | 1,000 | 1,262 | 1,500 | 1,725 | 1,913 | ... | 2,709 |

Это были лишь основные соображения в пользу выбора в качестве основы двоичную систему счисления. Существуют и другие (контроль, диагностика неисправностей), но здесь мы их рассматривать не будем.



## Понятия разрядности в архитектуре ЭВМ

Когда говорится, что та или иная ЭВМ имеет разрядность  $N$  (или  $M$  ☺), обычно имеется в виду максимальная разрядность *целого* числа, которая поддержана системой команд конкретной архитектуры. Эту характеристику можно назвать *native* (естественной) разрядностью архитектуры. Чаще всего (но не обязательно) эта разрядность совпадает с разрядностью АЛУ (CPU – Central Processing Unit). Но, кроме «целочисленной» разрядности, термин разрядности (ширина) к архитектуре ЭВМ может быть применён к разрядности адресного пространства, *физического* или *виртуального*, или к разрядности (ширине) шин(-ы), через которые(-ую) CPU «общается» с устройствами ввода-вывода или ОЗУ. Несколько примеров архитектур, где «под одной крышей» присутствует несколько разных разрядностей:

1. Первый персональный компьютер фирмы IBM PC использовал *шестнадцатиразрядный* микропроцессор фирмы Intel i8088 и арифметический сопроцессор Intel i8087:
  - a. Разрядность АЛУ – 16;
  - b. Разрядность ОЗУ – 20 (в максимальном режиме);
  - c. Разрядность шины данных i8088 для общения с «внешним» миром – 8.
  - d. Разрядность сопроцессора i8087 для обработки данных в формате с плавающей запятой:
    - i. «Внешняя» – 64;
    - ii. «Внутренняя» – 80.
2. Один из первых в мире *шестидесятичетырёхразрядных* микропроцессоров DEC Alpha (1992 год) и компьютеры на его основе:
  - a. Разрядность АЛУ – 64;
  - b. Разрядность виртуального адресного пространства – 48;
  - c. Физическое адресуемое адресное пространство – 40 (в реальности тех лет, конечно же, доступна была память намного меньшего объёма – размеры ОЗУ в терабайт ( $2^{40}$ ) и более стали привычными относительно недавно, в 1990-ые годы такие объёмы физической памяти были недостижимы).
3. Первые микропроцессорные архитектуры DEC LSI-11/03:
  - a. «Внешняя» (native) разрядность АЛУ – 16;
  - b. «Внутренняя», скрытая от пользователя разрядность микропроцессора –  $8^1$ .
4. Архитектуры DEC с менеджером памяти (MMU), LSI-11/23, LSI-11/73:
  - a. Разрядность данных целочисленного АЛУ – 16;
  - b. Разрядность данных сопроцессора для данных в формате с плавающей запятой – 64;
  - c. Разрядность виртуального адресного пространства – 16 ( $2^{16}$ ) (или 17 – два раза по  $2^{16}$  для LSI-11/73);
  - d. Разрядность физического адресного пространства – 22.
5. Сервер интеловской архитектуры, на базе Intel(R) Xeon(R) CPU X5690:
  - a. Разрядность данных целочисленного АЛУ – 64;
  - b. Разрядность виртуального адресного пространства – 48;
  - c. Разрядность физического адресного пространства – 43.
6. МикроЭВМ на базе микропроцессора i8080:
  - a. Разрядность АЛУ – 8;
  - b. Разрядность адресного пространства – 16.

---

<sup>1</sup> Шестнадцатиразрядность, «видимая» внешнему пользователю/программисту, достигалась средствами т.н. микропрограммирования – данные обрабатывались аппаратно *восьмиразрядным* АЛУ, на котором *микропрограмно* была реализована работа с шестнадцатиразрядными целочисленными данными.

В качестве примера можно привести совсем уж «скрытую» разрядность в архитектуре компьютера – в некоторых моделях серверных процессоров Intel Xeon «ширина» шины обмена CPU ↔ Cache имеет невероятные 768 разрядов, при всё том же 64-х разрядном АЛУ.

Коротко можно резюмировать – когда говорится о N-разрядной архитектуре, обычно имеется в виду максимальная «ширина» N машинного слова (целого), аппаратно поддерживаемого в АЛУ. Программными средствами возможна обработка целых чисел с разрядностью, превосходящей разрядность АЛУ.

### **Представление целых чисел со знаком**

Существует несколько вариантов представления отрицательных целочисленных данных (чисел со знаком). Один из очевидных способов, который, пожалуй, первым приходит в голову – формат (знак).(модуль числа) – это т.н. прямой код. Но по причинам, которые приведены ниже, наиболее распространённым и удобным оказался дополнительный двоичный код (пример для восьмиразрядной арифметики):

| Десятичное представление | Двоичное представление (8 бит) |           |                |
|--------------------------|--------------------------------|-----------|----------------|
|                          | прямой                         | обратный  | дополнительный |
| 127                      | 0111 1111                      | 0111 1111 | 0111 1111      |
| 1                        | 0000 0001                      | 0000 0001 | 0000 0001      |
| 0                        | 0000 0000                      | 0000 0000 | 0000 0000      |
| -0                       | 1000 0000                      | 1111 1111 |                |
| -1                       | 1000 0001                      | 1111 1110 | 1111 1111      |
| -2                       | 1000 0010                      | 1111 1101 | 1111 1110      |
| -3                       | 1000 0011                      | 1111 1100 | 1111 1101      |
| -4                       | 1000 0100                      | 1111 1011 | 1111 1100      |
| -5                       | 1000 0101                      | 1111 1010 | 1111 1011      |
| -6                       | 1000 0110                      | 1111 1001 | 1111 1010      |
| -7                       | 1000 0111                      | 1111 1000 | 1111 1001      |
| -8                       | 1000 1000                      | 1111 0111 | 1111 1000      |
| -9                       | 1000 1001                      | 1111 0110 | 1111 0111      |
| -10                      | 1000 1010                      | 1111 0101 | 1111 0110      |
| -11                      | 1000 1011                      | 1111 0100 | 1111 0101      |
| -127                     | 1111 1111                      | 1000 0000 | 1000 0001      |
| -128                     | ---                            | ---       | 1000 0000      |

Как аргумент против использования прямого кода, например, можно назвать наличие двух нулей: +/-0. Кто из них больше, кто меньше? Или, если из положительного нуля в прямом представлении вычесть единицу (представим, что мы декрементируем обнулённый двоичный счётчик), то мы получим естественные 1111 1111 (займ при вычитании из каждого разряда), что в трактовке «прямого» кода даёт в десятичном представлении -127, вместо ожидаемой -1.

В итоге, дополнительный код стал наиболее распространённым способом представления отрицательных целых чисел в компьютерах. Он позволяет заменить операцию вычитания на операцию сложения и сделать операции сложения и вычитания **одинаковыми** для знаковых и беззнаковых чисел, чем заметно упрощает архитектуру ЭВМ.

Дополнительный код для отрицательного числа можно получить инвертированием его двоичного модуля (первое дополнение) и прибавлением к инверсии единицы (второе дополнение), либо вычитанием числа из нуля.



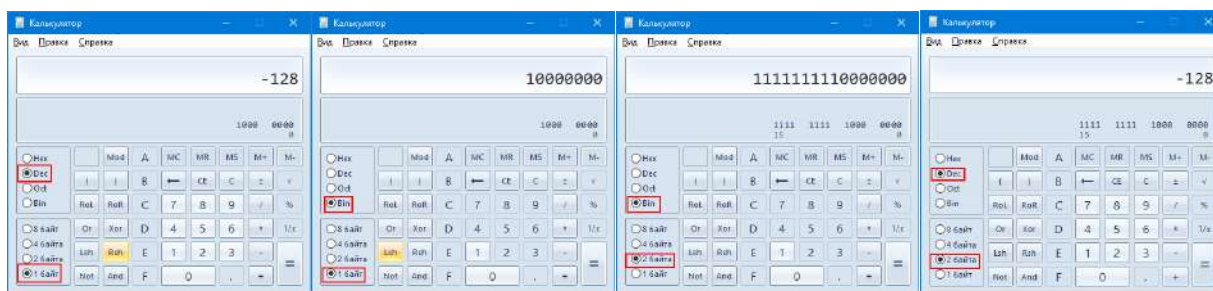
В качестве примера «естественности» и удобства работы с дополнительным кодом можно привести реализацию работы со знаковой арифметикой для неарифметических команд в архитектуре DEC PDP-11/LSI-11:

```
MOV #0, R0
MOVB #200, R0
```

Первая команда пересылки зануляет шестнадцатиразрядный регистр R0, а вторая пересылает в его *младший* байт константу 0200 (восьмеричное представление) ( $= -128$  десятичное,  $= 10000000$  – двоичное). И, внезапно, вдруг выясняется, что после выполнения второй инструкции *старший* байт регистра R0 оказывается заполнен единицами – 11111111. Но, при ближайшем рассмотрении, становится понятно, что теперь в R0 содержится -128 как в младшем восьмиразрядном байте, так и в целом шестнадцатиразрядном слове. Явное приведение типа, например, в смысле языка программирования C, *без явного использования АЛУ* – не иначе магия ☺, имеющая, впрочем, вполне официальный термин – *расширение знака*:

```
char C = -128;
X = (int)C;
```

В качестве иллюстрации эта же операция в «исполнении» стандартного калькулятора Windows:



## Представления действительных чисел (стандарт IEEE 754)

В представлении целых чисел в дополнительном коде задействуются все сочетания двоичных битов, от 0 до  $2^N-1$ , причём все целые числа из этого диапазона представлены точно. В отличие от этого, в представлении действительных чисел существуют диапазоны, которые нельзя сопоставить никакому числу, они называются не числа (NaN). Числа, представленные в формате IEEE754, представляют конечное множество, на которое отображается бесконечное множество вещественных чисел. Поэтому исходное число может быть представлено в формате IEEE754 с ошибкой, приближённо.

Стандарт IEEE 754 был разработан ассоциацией IEEE (Institute of Electrical and Electronics Engineers) и используется для представления действительных чисел (чисел с плавающей точкой) в двоичном коде. Это наиболее используемый стандарт для вычислений с плавающей точкой, используется многими микропроцессорами и логическими устройствами, а также программными средствами. В 2008 года ассоциация IEEE выпустила стандарт IEEE 754-2008, который включил в себя стандарт IEEE 754-1985.

### Стандарт IEEE 754-1985 определяет:

- как представлять нормализованные положительные и отрицательные числа с плавающей точкой;
- как представлять денормализованные положительные и отрицательные числа с плавающей точкой;
- как представлять нулевые числа;

- как представлять специальную величину бесконечность (Infinity);
- как представлять специальную величину «Не число» (NaN или NaNs);
- четыре режима округления.

**IEEE 754-1985 определяет четыре формата представления чисел с плавающей запятой:**

- с одинарной точностью (single-precision) 32 бита;
- с двойной точностью (double-precision) 64 бита;
- с одинарной расширенной точностью (single-extended precision)  $\geq 43$  бит (редко используемый);
- с двойной расширенной точностью (double-extended precision)  $\geq 79$  бит (обычно используют 80 бит).

### **Представление числа в нормализованном экспоненциальном виде**

Возьмем, к примеру, десятичное число 155,625. В нормализованном экспоненциальном виде это число будет выглядеть так:  $1,55625 \cdot 10^{+2} = 1,55625 \cdot \exp_{10}^{+2}$ . Число  $1,55625 \cdot \exp_{10}^{+2}$  состоит из двух частей: мантиссы  $M=1,55625$  и экспоненты  $\exp_{10}=+2$ . Если мантисса находится в диапазоне  $1 \leq M < 10$ , то число считается *нормализованным*. Экспонента представлена основанием системы исчисления (в данном случае 10) и порядком (в данном случае +2). Порядок экспоненты может иметь отрицательное значение, например, число  $0,0155625 = 1,55625 \cdot \exp_{10}^{-2}$ .

### **Представление числа в денормализованном экспоненциальном виде.**

То же самое десятичное число 155,625 в денормализованном экспоненциальном виде:  $0,155625 \cdot 10^{+3} = 0,155625 \cdot \exp_{10}^{+3}$ . Число  $0,155625 \cdot \exp_{10}^{+3}$  состоит из двух частей: мантиссы  $M=0,155625$  и экспоненты  $\exp_{10}=+3$ . Если мантисса находится в диапазоне  $0,1 \leq M < 1$ , то число считается денормализованным. Экспонента представлена основанием системы исчисления (в данном случае 10) и порядком (в данном случае +3). Порядок экспоненты может иметь отрицательное значение, например, число  $0,0155625 = 0,155625 \cdot \exp_{10}^{-3}$ .

### **Преобразование десятичного числа в двоичное число с плавающей точкой.**

Для преобразования десятичного числа в двоичное число с плавающей точкой в экспоненциальном нормализованном виде необходимо разложить заданное число по двоичным разрядам:

$$155,625 = 1 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3}$$

$$155,625 = 128 + 0 + 0 + 16 + 8 + 0 + 2 + 1 + 0,5 + 0 + 0,125$$

$$155,625_{10} = 10011011,101_2 \text{ — число в десятичной и в двоичной системе с плавающей точкой.}$$

Приведем полученное число к нормализованному виду в десятичной и двоичной системе:

$$1,55625 \cdot \exp_{10}^{+2} = 1,0011011101 \cdot \exp_2^{+111}$$

Т.е., в результате были получены основные составляющие экспоненциального нормализованного двоичного числа:

$$\text{Мантисса } M = 1,0011011101$$

$$\text{Экспонента } \exp_2 = +111$$

## Преобразование двоичного нормализованного числа в 32 битный формат IEEE 754:

1. Число может быть положительным отрицательным, поэтому один бит (самый старший) отводится для обозначения знака числа: 0 – положительное, 1 – отрицательное.
2. Далее выделено 8 бит (байт) для битов экспоненты. Экспонента тоже может быть со знаком; чтобы не вводить ещё один бит знака, добавляют смещение к экспоненте на половину байта +127 (0111 1111). То есть, если наша экспоната = +7 (+111 в двоичной), то смещенная экспонента = 7+127=134. Если бы экспонента была, к примеру, -7, то смещенная экспонента = 127-7 =120. Смещенную экспоненту записывают в отведенные 8 бит. При этом, когда нам будет нужно получить экспоненту двоичного числа, мы просто отнимаем 127 от этого байта.
3. Оставшиеся 23 бита отведены для мантииссы. Но, т.к. у нормализованной двоичной мантииссы первый бит всегда равен 1 (число лежит в диапазоне  $1 \leq M < 2$ ), поэтому нет смысла записывать единицу в отведенные 23 бита,

## Преобразования числа формата 32 бит IEEE 754 в десятичное число:

Чтобы записать число в стандарте IEEE 754 или восстановить его, необходимо знать три параметра:

- S - бит знака (31-й бит)
- E - смещенная экспонента (30-23 биты)
- M - остаток от мантииссы (22-0 биты)

Формула для для получения десятичного числа из числа IEEE754 одинарной точности:

$$F = (-1)^S 2^{(E-127)} (1 + M/2^{23})$$

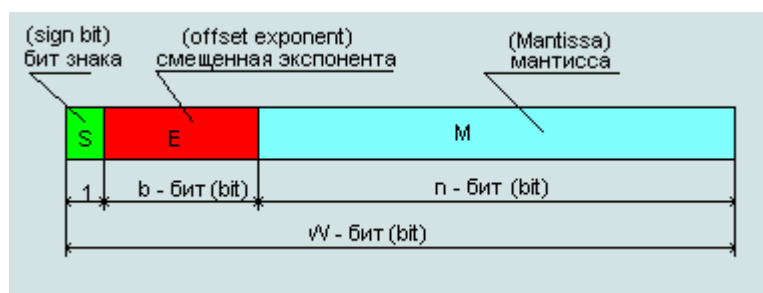


Рис.59 - Представление числа в формате IEEE 754

Формула вычисления десятичных чисел с плавающей точкой, из числа, представленного в стандарте IEEE754:

$$F = (-1)^S 2^{(E-2^{(b-1)}+1)} (1 + M/2^n) \quad (\text{№1})$$

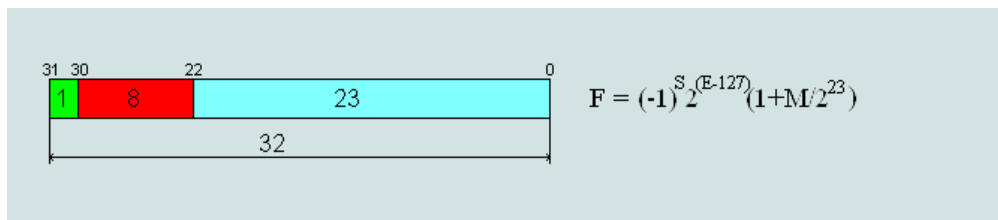


Рис.60 - Формат числа одинарной точности (single-precision) 32 бита

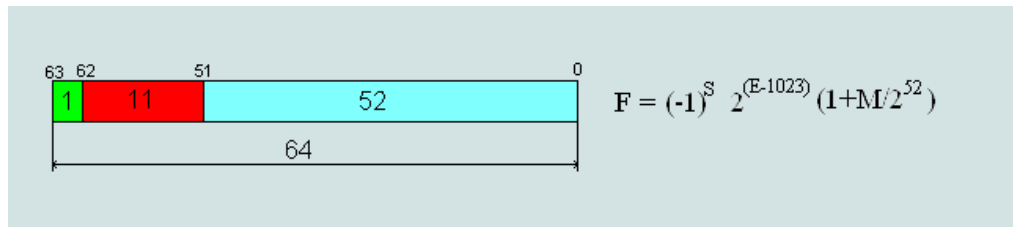
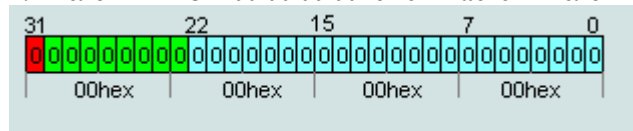


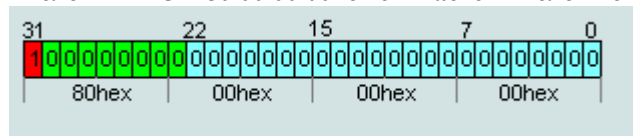
Рис.61 - Формат числа двойной точности (double-precision) 64 бита

Используя формулу №1, невозможно представить число нуль или бесконечность в заданном формате, поэтому она не применяется в следующих случаях:

1. число IEEE754=00 00 00 00hex считается числом +0



число IEEE754=80 00 00 00hex считается числом -0



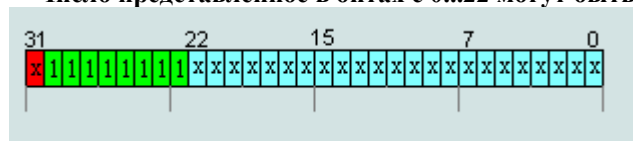
2. число IEEE754=7F 80 00 00hex считается числом +∞



число IEEE754=FF 80 00 00hex считается числом -∞



3. числа IEEE754=FF (1xxx)X XX XXhex не считаются числами (NaN), кроме случая п.2  
числа IEEE754=7F (1xxx)X XX XXhex не считаются числами (NaN), кроме случая п.2  
Число представленное в битах с 0...22 могут быть любым числом кроме 0 (т.е.+∞ и -∞ ).



4. числа IEEE754=(x000) (0000) (0xxx)X XX XXhex считаются денормализованными числами, за исключением чисел п.1(то есть -0 и +0)



## Точность представления вещественных чисел в формате IEEE754.

Как уже говорилось выше, числа в формате IEEE754 представляют конечное множество, на которое отображается бесконечное множество вещественных чисел. Поэтому исходное число может быть представлено в формате IEEE754 неточно, с ошибкой.

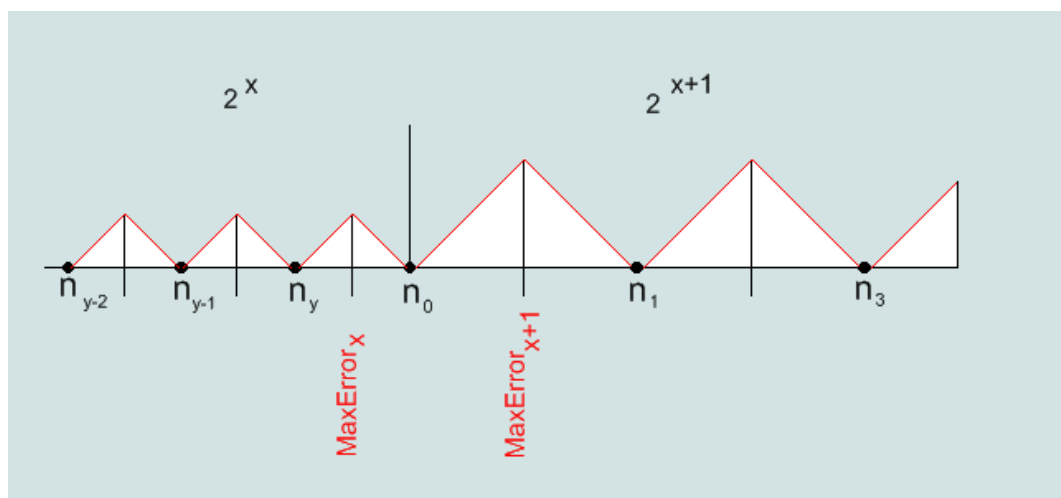


Рис.62 - Функция ошибки точности представления числа в IEEE754

Максимальная относительная ошибка для денормализованного числа(single/double):

$$\frac{2^{(E-150)}}{2^{(E-126)} \frac{M}{2^{23}}} = \frac{1}{2M}$$

Максимальная относительная ошибка нормализованного числа(single):

$$\frac{2^{(E-151)}}{2^{(E-127)} \left(1 + \frac{M}{2^{23}}\right)} = \frac{1}{2^{24} + 2M}$$

Максимальная относительная ошибка нормализованного числа(double):

$$\frac{2^{(E-1076)}}{2^{(E-1023)} \left(1 + \frac{M}{2^{52}}\right)} = \frac{1}{2^{53} + 2M}$$

Дальнейшее, более полное описание стандарта IEEE 754 можно найти по ссылке [IEEE 754 - стандарт двоичной арифметики с плавающей точкой](#).

## Порядок следования байт в многобайтовых словах

В современной вычислительной технике и цифровых системах связи информация обычно представлена в виде последовательности байтов. В том случае, если число не может быть представлено одним байтом, имеет значение, в каком порядке байты записываются в памяти компьютера или передаются по линиям связи. Выбор порядка записи байтов произволен и определяется только соглашениями.

В общем случае, для представления числа, большего 255 ( $2^8 - 1 = 255$  — максимальное целое число, записываемое одним байтом) приходится использовать несколько байтов. При этом число  $M$  записывается в позиционной системе счисления по основанию 256:

$$M = \sum_{i=0}^n A_i \cdot 256^i = A_0 \cdot 256^0 + A_1 \cdot 256^1 + A_2 \cdot 256^2 + \dots + A_n \cdot 256^n.$$

Набор целых чисел  $A_0, \dots, A_n$ , каждое из которых лежит в интервале от 0 до 255, является последовательностью байтов для записи  $M$ .

## Варианты записи

### Порядок от старшего к младшему

Порядок *от старшего к младшему* или (*big-endian*, дословно: «тупоконечный»):  $A_n, \dots, A_0$ , запись начинается со старшего и заканчивается младшим. Этот порядок является стандартным для протоколов TCP/IP, он используется в заголовках пакетов данных и во многих протоколах более высокого уровня, разработанных для использования поверх TCP/IP. Поэтому, порядок байтов от старшего к младшему часто называют сетевым порядком байтов (англ. *network byte order*). Этот порядок байтов используется процессорами IBM 360/370/390, Motorola 68000, SPARC (отсюда третье название — *порядок байтов Motorola*, *Motorola byte order*).

В этом же виде (используя представление в десятичной системе счисления) записываются числа индийско-арабскими цифрами в письменностях с порядком знаков слева направо (латиница, кириллица). Для письменностей с обратным порядком (арабская) та же запись числа воспринимается как «от младшего к старшему».

Порядок байтов от старшего к младшему применяется во многих форматах файлов — например, PNG, FLV, EBML.

### Порядок от младшего к старшему

Порядок *от младшего к старшему* или (*little-endian*, дословно: «остроконечный»):  $A_0, \dots, A_n$ , запись начинается с младшего и заканчивается старшим. Этот порядок записи принят в памяти персональных компьютеров с x86-процессорами, в связи с чем иногда его называют **интеловский порядок байтов** (по названию фирмы-создателя архитектуры x86). В некоторых кругах используется название *VAX order*, например, в документации Perl.

По сравнению с «тупоконечным» вариантом, кросс-платформенные протоколы и форматы данных с «остроконечным» порядком байт, пожалуй, менее распространены, хотя здесь можно отметить некоторые весьма популярные исключения: шина USB, конфигурация PCI, таблица разделов GUID, рекомендации FidoNet.

## Переключаемый порядок

Многие процессоры могут работать и в порядке от младшего к старшему, и в обратном, например, ARM, PowerPC (но не PowerPC 970), DEC Alpha, MIPS, PA-RISC и IA-64. Обычно порядок байтов выбирается программно во время инициализации операционной системы, но может быть выбран и аппаратно переключками на материнской плате. В этом случае правильнее говорить о порядке байтов операционной системы. Переключаемый порядок байтов иногда называют *bi-endian*.

## Смешанный порядок

Смешанный порядок байтов (*middle-endian*) иногда используется при работе с числами, длина которых превышает машинное слово. Число представляется последовательностью машинных слов, которые записываются в формате, естественном для данной архитектуры, но сами слова следуют в обратном порядке.

Классический пример *middle-endian* — представление 4-байтных целых чисел на 16-битных процессорах семейства PDP-11 (известен как PDP-endian). Для представления двухбайтных значений (слов) использовался порядок *little-endian*, но 4-хбайтное двойное слово записывалось от старшего слова к младшему.

В процессорах VAX и ARM используется смешанное представление для длинных вещественных чисел.

Ниже приведён пример, в котором описано размещение четырёхбайтового числа в ОЗУ ЭВМ, доступ к которому может производиться как к 32-разрядному слову, так и по байтно. Числа в HEX-формате:

Число: 0xA1B2C3D4

|                                |                          |                                                                              |
|--------------------------------|--------------------------|------------------------------------------------------------------------------|
| Представление                  |                          | $D4 \times 0x01 + C3 \times 0x100 + B2 \times 0x10000 + A1 \times 0x1000000$ |
| Порядок от младшего к старшему | ( <i>little-endian</i> ) | 0xD4, 0xC3, 0xB2, 0xA1                                                       |
| Порядок от старшего к младшему | ( <i>big-endian</i> )    | 0xA1, 0xB2, 0xC3, 0xD4                                                       |
| Порядок, принятый в PDP-11     | (PDP-endian)             | 0xB2, 0xA1, 0xD4, 0xC3                                                       |

Более наглядно, на картинках ниже:



Рис.63 - Порядок байтов в ОЗУ

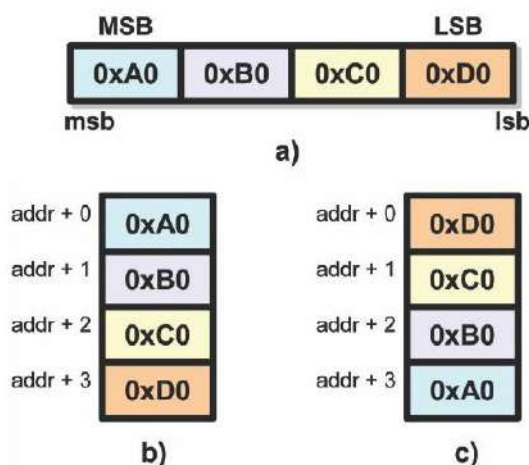


Рис.64 - Размещение байтов слова в памяти Big-endian и Little-endian ЭВМ

Существенным достоинством *little-endian* по сравнению с *big-endian* порядком записи считается возможность «неявной типизации» целых чисел при чтении меньшего объ-



ёма байт (при условии, что читаемое число помещается в диапазон). Так, если в ячейке памяти содержится число 0x00000022, то прочитав его как int16 (два байта) мы получим число 0x0022, прочитав один байт — число 0x22. Однако, это же может считаться и недостатком, потому что провоцирует ошибки потери данных.

Недостатком little-endian, по сравнению с big-endian, считается «неочевидность» значения байтов памяти при отладке (последовательность байтов (A1, B2, C3, D4) на самом деле значит 0xD4C3B2A1, для big-endian эта последовательность (A1, B2, C3, D4) читалась бы «естественным» для арабской записи чисел образом: 0xA1B2C3D4). Наименее удобным в работе считается middle-endian формат записи; но он сохранился только на старых платформах.

Для записи длинных чисел (чисел, длина которых существенно превышает разрядность машины) обычно предпочтительнее порядок *слов в числе* little-endian (поскольку арифметические операции над длинными числами производятся от младших разрядов к старшим). Порядок *байтов в слове* — обычный для данной архитектуры.

### Этимология названия

Термины big-endian и little-endian первоначально не имели отношения к информатике. В сатирическом произведении Джонатана Свифта «Путешествия Гулливера» описываются вымышленные государства Лилипутия и Блефуску, в течение многих лет ведущие между собой войны из-за разногласия по поводу того, с какого конца следует разбивать варёные яйца. Тех, кто считает, что их нужно разбивать с тупого конца, в произведении называют Big-endians («тупоконечники»). Споры между сторонниками big-endian и little-endian в информатике также часто носят характер т. н. «религиозных войн».

Термины big-endian и little-endian ввёл Коэн (Danny Cohen) в 1980 году в своей статье «On Holy Wars and a Plea for Peace».

## Структуры данных

Что такое структуры данных? По сути, это способы хранить и организовывать данные, для более эффективного решения различных задач. Данные можно представить по-разному, и в зависимости от того, что это за данные и что вы собираетесь с ними делать, одно представление подойдёт лучше других.

Компьютерная память — довольно простая сущность. Это группа упорядоченных слотов, в которых хранится информация. Чтобы получить к ней доступ, вы должны знать её адрес в памяти. Естественно, здесь речь идёт о прямоадресуемой памяти, которую мы знаем под названием ОЗУ (RAM). Только в англоязычном термине RAM отражается это свойство — память с произвольным доступом. Чтобы прочитать содержимое ячейки памяти с адресом N, нет необходимости читать (пропускать) N-1 ячеек. Пример содержимого фрагмента ОЗУ:

|           |   |      |   |      |   |      |   |      |   |      |   |      |   |      |   |      |   |      |   |      |     |
|-----------|---|------|---|------|---|------|---|------|---|------|---|------|---|------|---|------|---|------|---|------|-----|
| Значения: |   | 1001 |   | 0110 |   | 1000 |   | 0100 |   | 0101 |   | 1010 |   | 0010 |   | 0001 |   | 1101 |   | 1011 | ... |
| Адреса:   | 0 |      | 1 |      | 2 |      | 3 |      | 4 |      | 5 |      | 6 |      | 7 |      | 8 |      | 9 |      | ... |

Адресная арифметика в программировании начинается с 0, не бывает ячеек памяти с отрицательным абсолютным адресом.

Примеры структур данных: линейные — элементы образуют последовательность или линейный список, обход узлов линейен — к этой категории относятся массивы, связанные списки, стеки и очереди. Нелинейные структуры — если обход узлов нелинейный, а данные не последовательны. Пример: графы, деревья.

## Основные структуры данных

1. Массивы
2. Стеки

3. Очереди
4. Связный список
5. Графы
6. Деревья
7. Префиксные деревья
8. Хэш таблицы

## Массивы

Массив — это самая простая и широко используемая структура данных. Другие структуры данных, такие как стеки и очереди, являются производными от массивов.

Изображение простого массива размера 4, содержащего элементы (1, 2, 3 и 4) (Рис.65).

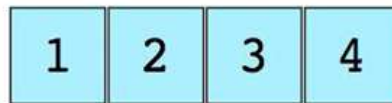


Рис.65 - Пример массива

Массивы бывают одномерными (Рис.65) и многомерными (массивы внутри массива).

## Стек

Стек — это коллекция, элементы которой получают по принципу «последний вошёл, первый вышел» (*Last-In-First-Out или LIFO*). Это значит, что мы будем иметь доступ только к последнему добавленному элементу («верхушка стека»).

В отличие от списков, мы не можем получить доступ к произвольному элементу стека. Мы можем только добавлять или удалять элементы с помощью специальных методов. По сути это очередь, придумана в своё время Аланом Тьюрингом.

Наиболее часто встречающаяся аналогия для объяснения работы стека — стопка тарелок (Рис.66). Вне зависимости от того, сколько тарелок в стопке, мы всегда можем снять только верхнюю. Чистые тарелки точно так же кладутся на верх стопки, и мы всегда будем первой брать ту тарелку, которую положили последней.



Рис.66 - Стек как стопка тарелок

Если мы положим, например, красную тарелку, затем синюю, а затем зеленую, то сначала надо будет снять зеленую, потом синюю, и, наконец, красную. Главное, что надо запомнить — тарелки всегда ставятся на верх стопки. Когда кто-то берет тарелку, он также снимает её сверху. Получается, что тарелки разбираются в порядке, обратном тому, в котором ставились.

Программный стек реализуется на ОЗУ с помощью указателей, которые при помещении в стек (PUSH) декрементируются, а при извлечении из стека (POP) – инкрементируются<sup>1</sup>.

Изображение стека, в три элемента (1, 2 и 3), где 3 находится наверху и будет удален (POP) первым (Рис.67).

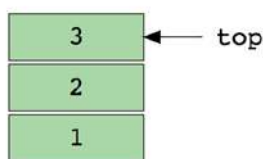


Рис.67 - Трёхэлементный стек

## Очередь

**FIFO** (*first in, first out* — «первым пришёл — первым ушёл») — способ организации и манипулирования данными относительно времени и приоритетов. Это выражение описывает принцип технической обработки очереди или обслуживания конфликтных требований путём упорядочения процесса по принципу: «первым пришёл — первым обслужен» (ПППО). Тот, кто приходит первым, тот и обслуживается первым, пришедший следующим ждёт, пока обслуживание первого не будет закончено, и так далее.

Этот принцип аналогичен поведению лиц, стоящих в очереди, когда люди получают обслуживание в том порядке, в котором они занимали очередь. То же самое происходит, например, на нерегулируемом перекрёстке, когда водители ожидают своей очереди на продолжение движения (в ПДД США нет правила «помеха справа», приоритет определяется по принципу FIFO). ПППО также используется как сокращённое название для алгоритма **FIFO** планирования работы операционной системы, по которому процессорное время выделяется каждому процессу в порядке поступления запросов на обслуживание.

Изображение очереди, в четыре элемента (1, 2, 3 и 4), где 1 находится наверху и будет удален первым (Рис.68).

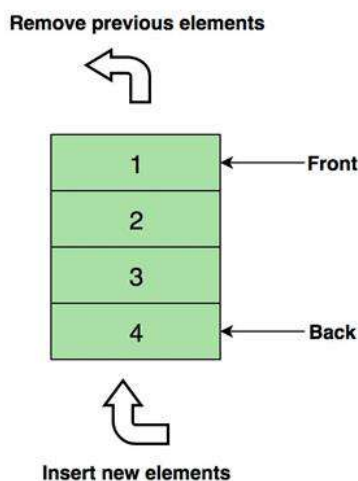


Рис.68 – Очередь

Легко запомнить: устройство для преобразования стека в очередь – автомат Калашникова ☺ (Рис.69).

<sup>1</sup> Направление продвижения указателя, в сторону больших или меньших адресов при операциях PUSH/POP зависит от конкретной архитектуры ЭВМ.



Рис.69 - Устройство для преобразования стека в очередь

## Связный список

**Связный список** — базовая динамическая структура данных в информатике, состоящая из узлов, каждый из которых содержит как собственно данные, так и одну или две ссылки («связки») на следующий и/или предыдущий узел списка. Принципиальным преимуществом перед массивом является структурная гибкость: порядок элементов связного списка может не совпадать с порядком расположения элементов данных в памяти компьютера, а порядок обхода списка всегда явно задаётся его внутренними связями.

Связные списки бывают:

**Однонаправленные**, каждый узел хранит адрес или ссылку на следующий узел в списке и последний узел имеет следующий адрес или ссылку на NULL.

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow \text{NULL}$

**Двунаправленные**, две ссылки, связанные с каждым узлом, одним из опорных пунктов на следующий узел и один к предыдущему узлу.

$\text{NULL} \leftarrow 1 \leftrightarrow 2 \leftrightarrow 3 \rightarrow \text{NULL}$

**Кольцевые**, все узлы соединяются, образуя кольцо. В конце нет NULL. Циклический связанный список может быть одно-или двукратным циклическим связанным списком.

$1 \rightarrow 2 \rightarrow 3 \rightarrow 1$

## Графы

Граф — это набор узлов (вершин), которые соединены друг с другом в виде сети ребрами (дугами) (Рис.70).

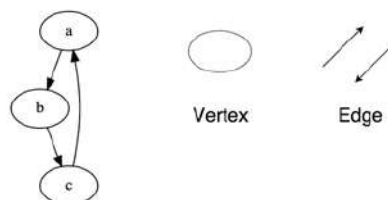


Рис.70 - Граф

Графы бывают **ориентированными**, где ребра являются направленными, т.е. существует только одно доступное направление между двумя связными вершинами. Или **неориентированными**, когда к каждому из ребер можно осуществлять переход в обоих направлениях. Есть вариант смешанного графа, комбинация ориентированного и неориентированного.

## Деревья

Дерево – это иерархическая структура данных, состоящая из узлов (вершин) и рёбер (дуг).  
Деревья – это по сути связанные графы без циклов (Рис.71).

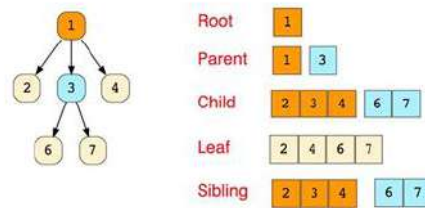


Рис.71 – Дерево

Типы деревьев:

- N-дерево
- Сбалансированное дерево
- Бинарное дерево
- Дерево Бинарного Поиска
- AVL дерево
- 2-3-4 деревья

## Trie (префиксное дерево)

Разновидность дерева для строк, быстрый поиск (Словари, T9). Вот как такое дерево хранит слова «top», «thus» и «their» (Рис.72).

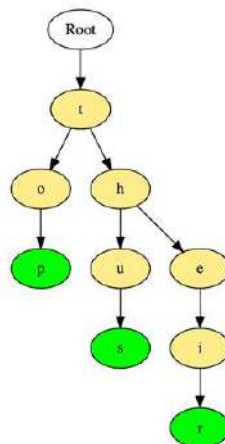


Рис.72 - Префиксное дерево

## Хэш-таблицы

Хэширование — это процесс, используемый для уникальной идентификации объектов и хранения каждого объекта в заранее рассчитанном уникальном индексе (ключе). Объект хранится в виде пары «ключ-значение», а коллекция таких элементов называется «словарём». Каждый объект можно найти с помощью этого ключа. По сути это массив, в котором ключ представлен в виде хэш-функции.

Пример сопоставления хэша в массиве. Индекс этого массива вычисляется через хэш-функцию (Рис.73).

|    |       |        |
|----|-------|--------|
|    |       |        |
| 3  | <key> | <data> |
| ⋮  |       |        |
| 16 | <key> | <data> |
| 17 | <key> | <data> |
|    |       |        |

Рис.73 - Хэш-таблица

## ГЛАВА 4. МЕТОДЫ УВЕЛИЧЕНИЯ ПРОИЗВОДИТЕЛЬНОСТИ ЭВМ

Общая производительность компьютера зависит от производительности всех его составляющих. Основные компоненты, определяющие быстродействие ЭВМ – это её АЛУ (CPU) и ОЗУ (RAM). Долгое время главным и практически единственным способом повышения производительности системы было повышение тактовой частоты (кроме очевидного увеличения разрядности обрабатываемых данных). Например, 8-разрядный i8080 работал на тактовой частоте всего 2,5 МГц, а три десятилетия спустя его праправнук разогнался уже до частот на три порядка более высоких – до 4 ГГц и более!

### Power Wall

На этом понятном и простом пути «гонки мегагерц» разработчики микропроцессоров упёрлись в объективные фундаментальные физические проблемы. Эти проблемы проиллюстрированы на Рис.74 (взято из работ Patterson & Hennessy, повторно воспроизведён Рис.53). Производительность SPECint самых «горячих» чипов росла примерно на 52% в год в период с 1986 по 2002, а затем рост производительности замедлился до ~20% в следующие три года (примерно 6% в год).

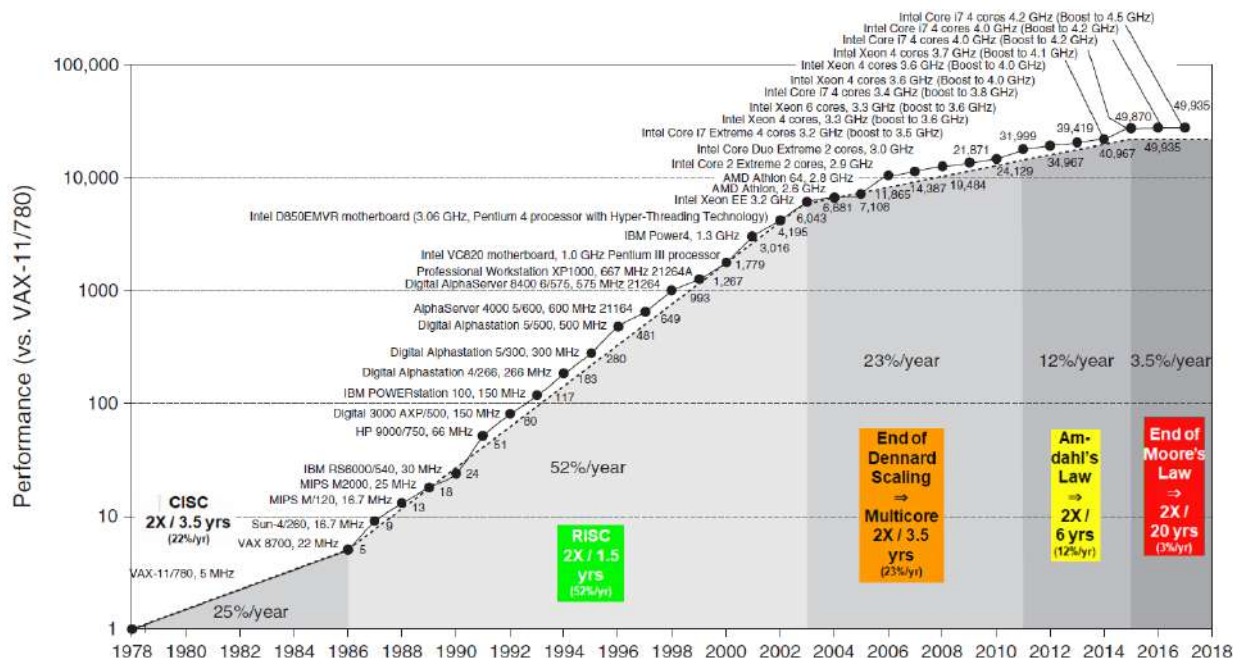


Рис.74 – SPECint самых производительных процессоров

Проблему, с которой столкнулись разработчики, стали называть «Power Wall» (взято из работ Patterson & Hennessy) Рис.75:

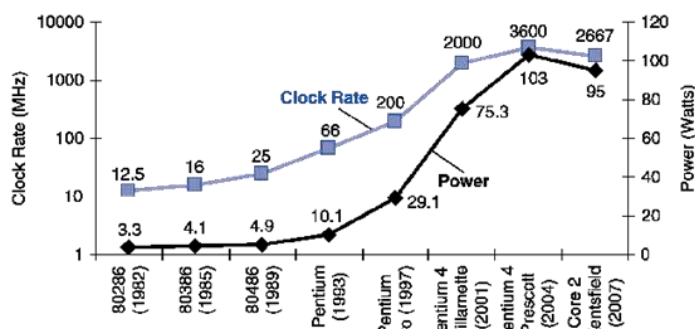
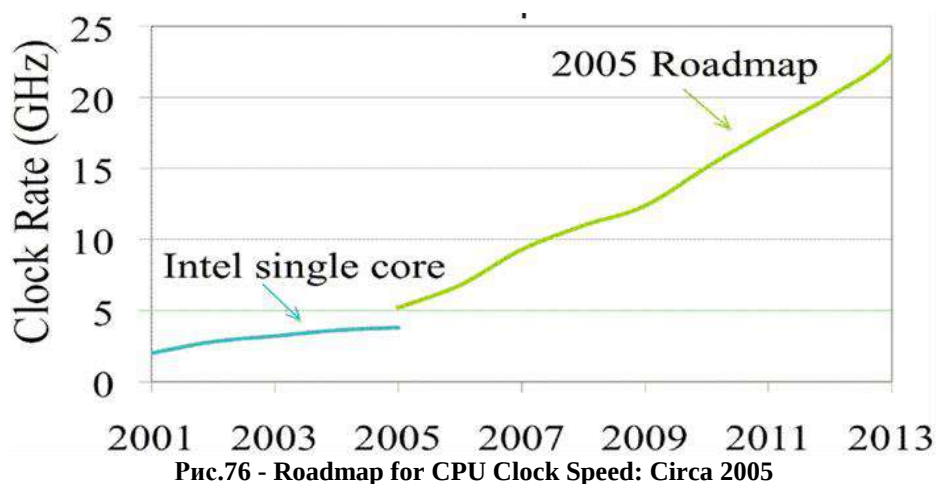
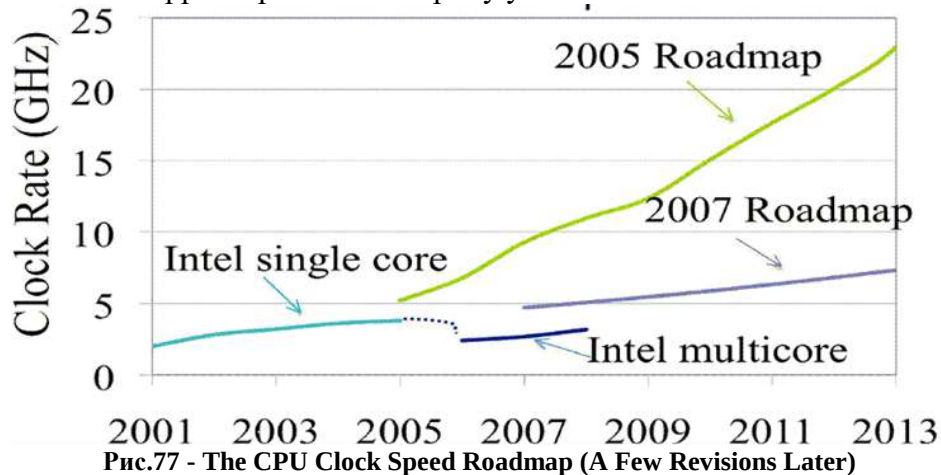


Рис.75 - Частоты и рассеиваемая мощность для некоторых микропроцессоров

Любопытно сравнить реалии жизни с ожиданиями – ниже приведёт рисунок с прогнозами (roadmap) тактовых частот топовых процессоров образца 2005 года:



По самым оптимистичным прикидкам (Рис.76) в 2015г. ожидалось горячие процессоры, которые работали бы на частотах 12...15 ГГц. Несколько ревизий спустя ожидания были существенно скорректированы в сторону уменьшения:



Это отражает практический опыт, полученный в работе со сверхплотными чипами – они становились всё горячее и их было всё труднее охлаждать. Никуда не денешься, это законы природы: вся потребляемая электроэнергия в конечном итоге излучается в виде тепла.

Ниже, на (Рис.78) приведена ещё одна таблица, иллюстрирующая проблему «Power Wall»:



## Частота ЦПУ

Разработчики аппаратуры столкнулись с "Power Wall"

|                                           | 1980 | 1990  | 1995    | 2000  | 2003 | 2005   | 2010               | 2015               | 2015<br>÷<br>1980 |
|-------------------------------------------|------|-------|---------|-------|------|--------|--------------------|--------------------|-------------------|
| ЦПУ                                       | 8086 | 80386 | Pentium | P-III | P-4  | Core 2 | Core i7<br>Nehalem | Core i7<br>Haswell |                   |
| Частота (МГц)                             | 1    | 20    | 150     | 600   | 3300 | 2000   | 2500               | 3000               | 3000              |
| Длительность<br>такта (нс.)               | 1000 | 50    | 6       | 1.6   | 0.3  | 0.5    | 0.4                | 0.25               | 3000              |
| Количество<br>ядер                        | 1    | 1     | 1       | 1     | 1    | 2      | 4                  | 8                  | 8                 |
| Эффективная<br>длительность<br>такта (нс) | 1000 | 50    | 6       | 1.6   | 0.3  | 0.25   | 0.1                | 0.04125            | ~24250            |

Рис.78 - Частоты ЦПУ и ядра

Как двигаться дальше, в сторону повышения производительности вычислительных машин, если прямолинейное повышение тактовых частот упирается в объективные физические пределы? Ответ на вопрос можно найти в сравнении колонки таблицы (Рис.78), выделенной красным и соседней с ней справа – индустрия пошла по пути увеличения количества вычислительных ядер. Более высокочастотный (3,3 ГГц) одноядерный Pentium-4 имеет бóльшую эффективную длительность такта, чем более низкочастотный (2 ГГц) двухядерный Core 2: 0,3 нс против 0,25.

С этого времени тактовые частоты практически перестали расти, более того, начиная с 2010 года, каждый следующий процессор Intel имел меньшую производительность в пересчёте на ядро, по сравнению с предшественником, но общая производительность систем росла за счёт увеличения количества ядер.

### Иерархия памяти в архитектуре ЭВМ, Memory Wall

Ещё раньше проблемы, названной «Power Wall», компьютерная индустрия столкнулась с проблемой всё увеличивающегося разрыва в производительности ЦПУ и ОЗУ (обычно точкой отсчёта называется 1980 год).

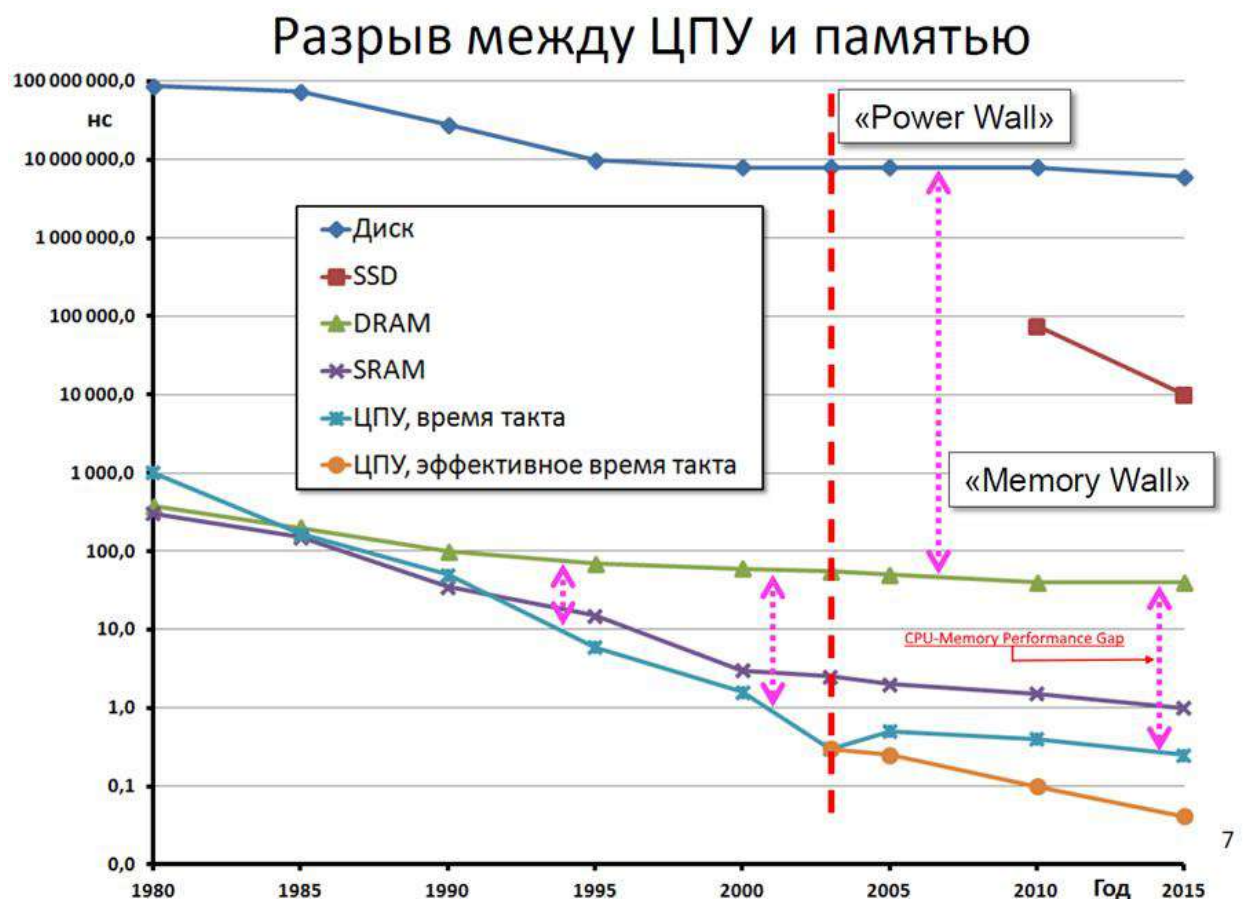


Рис.79 - Разрыв производительности CPU-memory

В правой части Рис.79 показан (красной стрелкой) разрыв, который достигает величин в несколько порядков (см. эффективную длительность такта на Рис.78). Со временем этот разрыв только увеличивается, для примера – топовый современный 64-ядерный процессор AMD Ерус (архитектура NUMA<sup>1</sup>) работает на частоте 3,2 ГГц и имеет два времени задержки доступа к памяти – 104 нс (к «своей» памяти, 201 нс – к «чужой»), а эффективная длительность такта при этом составляет – 0,0000048828125нс.

Для преодоления разрыва в производительности была предложена концепция иерархической организации памяти ЭВМ. Структуры данных, о которых было рассказано *выше*, описывают логическую организацию хранения данных (в ОЗУ), тогда как иерархия памяти даёт представление о физической организации памяти компьютера в более общем смысле – от регистров до лент.

Главная задача компьютерной системы – (по возможности быстро) выполнять программы. Программы вместе с данными, к которым они имеют доступ, в процессе выполнения должны (по крайней мере частично) находиться в оперативной памяти. Операционной системе приходится решать задачу распределения памяти между пользовательскими процессами и компонентами ОС. Эта деятельность называется управлением памятью. Таким образом, память (storage, memory) является важнейшим ресурсом, требующим тщательного управления.

В основе реализации иерархии памяти современных компьютеров лежат два принципа: принцип локальности обращений и соотношение стоимость/производительность. Принцип локальности обращений говорит о том, что большинство программ, к счастью, не

<sup>1</sup> NUMA (Non-Uniform Memory Access — «неравномерный доступ к памяти» или Non-Uniform Memory Architecture — «Архитектура с неравномерной памятью») — схема реализации компьютерной памяти, используемая в мультипроцессорных системах, когда время доступа к памяти определяется её расположением по отношению к процессору. Префикс cc означает cache coherence – свойство кэшей, означающее целостность данных, хранящихся в локальных кэшах для разделяемого ресурса.

выполняют обращений ко всем своим командам и данным равновероятно, а оказывают предпочтение некоторой части своего адресного пространства:

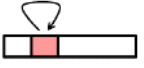
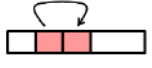
- *Основной принцип локальности*: программа стремится использовать данные и инструкции с адресами близкими (либо точно такими же) к тем, которые использовались ранее.
  - *Временная локальность* – повторные обращения. 
  - *Пространственная локальность* – в некоторый малый промежуток времени используются ячейки памяти с близкими адресами. 
- Некоторые фундаментальные свойства аппаратуры и ПО:
  - Более быстрые устройства хранения стоят дороже, имеют меньший объем и потребляют больше энергии.
  - Разрыв в скорости работы между ЦПУ и оперативной памятью увеличивается.
  - В хорошо написанных программах демонстрируется хорошая локальность.
- Данные свойства дополняют друг друга и ...
- ...выводят на идею **иерархической организации памяти** (Рис.80)



Рис.80 – Иерархия памяти.

Когда говорят о производительности вычислительной системы (обычно в терминах количества операций/сек), эта производительность всегда оценивается при работе с использованием самой быстрой и, одновременно, самой малоёмкой (и самой дорогой) памяти, находящейся ближе всех к процессору – регистров.

## Регистры

Регистры (обычно речь идёт о целочисленных регистрах) – это группа (иногда называемая регистровым файлом) параллельных регистров (РОН – Регистры Общего Назначения), с разрядностью равной разрядности АЛУ. Это самая быстрая память в компьютере, в некоторых источниках даже можно встретить её обозначение как СОЗУ – сверхоперативное ЗУ. Если мы знаем, что наш процессор работает на тактовой частоте 1 ГГц и при этом выполняет миллиард операций/сек – это будет верным, только если операнды будут располагаться в регистрах.

Количество доступных регистров – важнейшая характеристика архитектуры. Переход от CISC к RISC сопровождался значительным увеличением количества регистров – от нескольких штук до нескольких десятков и даже сотен. В компьютерных архитектурах ранних поколений бывали даже случаи полного отсутствия регистров в CPU, все манипуляции с данными проделывались в ОЗУ. Из примеров таких «безрегистровых» архитектур можно привести микропроцессор TMS9900<sup>1</sup>.

В таблице ниже приведены количественные характеристики регистровых файлов для некоторых архитектур (целочисленные и для операндов в формате с плавающей запятой):

| Архитектура        | Целочисленных регистров | FP регистров | Примечания                            |
|--------------------|-------------------------|--------------|---------------------------------------|
| x86-32             | 8                       | 8            |                                       |
| x86-64             | 16                      | 16           |                                       |
| IBM System/360     | 16                      | 4            |                                       |
| z/Architecture     | 16                      | 16           |                                       |
| Itanium            | 128                     | 128          |                                       |
| SPARC              | 31                      | 32           | Регистр 0 (глобальный) всегда занулен |
| IBM Cell           | 4~16                    | 1~4          |                                       |
| IBM POWER          | 32                      | 32           |                                       |
| Power Architecture | 32                      | 32           |                                       |
| Alpha              | 32                      | 32           |                                       |
| 6502               | 3                       | 0            |                                       |
| W65C816S           | 5                       | 0            |                                       |
| PIC                | 1                       | 0            |                                       |
| AVR                | 32                      | 0            |                                       |
| ARM 32-bit         | 16                      | различное    |                                       |
| ARM 64-bit         | 31                      | 32           |                                       |
| MIPS               | 31                      | 32           | Регистр 0 всегда занулён              |
| RISC-V             | 32                      | 32           |                                       |
| Эльбрус 2000       | 256                     |              | 84-разрядных регистров                |

Ниже, на Рис.81 приведен список всех регистров в архитектуре x86/x64:

<sup>1</sup> TMS9900 имел три внутренних 16-битных регистра: PC – счётчик команд, ST – статусный регистр и Workspace Pointer register (WP). Регистр WP использовался в качестве указателя на базовый адрес в ОЗУ, где располагались 16 шестнадцатиразрядных «регистров» общего назначения.

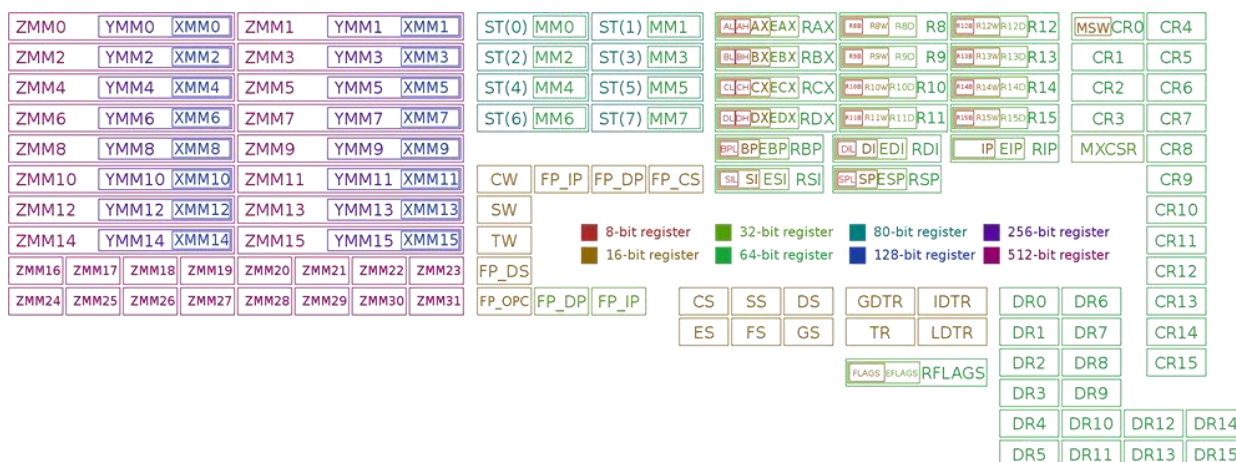


Рис.81 - Регистры x86/x64 архитектуры

## Кэш память

**Кэш микропроцессора** — кэш (иногда говорят «сверхоперативная память»), используемый микропроцессором компьютера для уменьшения среднего времени доступа к компьютерной памяти<sup>1</sup>. Является одним из верхних уровней иерархии памяти. Кэш использует относительно небольшую, быструю память (обычно типа SRAM), которая хранит копии часто используемых данных из основной памяти. Если большая часть запросов в память будет обрабатываться кэшем, средняя задержка обращения к памяти будет приближаться к задержкам работы кэша.

Когда процессору нужно обратиться в память для чтения или записи данных, он сначала проверяет, доступна ли их копия в кэше. В случае успеха проверки процессор производит операцию, используя кэш, что значительно быстрее использования более медленной основной памяти.

Данные между кэшем и памятью передаются блоками фиксированного размера, также называемые **линиями кэша** (*cache line*) или блоками кэша.

Большинство современных микропроцессоров для настольных компьютеров и серверов имеют как минимум три независимых кэша: **кэш инструкций** для ускорения загрузки машинного кода (кода команды), **кэш данных** для ускорения чтения и записи данных и буфер ассоциативной трансляции (TLB) для ускорения трансляции виртуальных (логических) адресов в физические, как для инструкций, так и для данных. Кэш данных часто реализуется в виде многоуровневой структуры кэша (L1, L2, L3, L4).

Увеличение размера кэш-памяти может положительно влиять на производительность почти всех приложений, хотя в некоторых случаях эффект может быть незначителен. Работа кэш-памяти обычно прозрачна для программиста, однако для её эффективного использования в некоторых случаях применяются специальные алгоритмические приёмы, изменяющие порядок обхода данных в ОЗУ или повышающие их локальность (например, при блочном умножении матриц).

<sup>1</sup> **Компьютерная память** (устройство хранения информации, запоминающее устройство) — часть вычислительной машины, физическое устройство или среда для хранения данных, используемая в вычислениях в течение определённого времени. Память, как и центральный процессор, является неизменной частью компьютера с 1940-х годов. Память в вычислительных устройствах имеет иерархическую структуру и обычно предполагает использование нескольких типов запоминающих устройств, имеющих различные характеристики.

## Принцип работы

Основная идея кэширования опирается на свойство локальности данных и инструкций: если происходит обращение по некоторому адресу, то велика вероятность, что в ближайшее время произойдет обращение к памяти по тому же адресу либо по соседним адресам.

Логически кэш-память представляет собой набор кэш-линий. Каждая кэш-линия хранит блок данных определенного размера и дополнительную информацию. Под размером кэш-линии понимают обычно размер блока данных, который в ней хранится. Для архитектуры x86 размер кэш-линии составляет 64 байта.

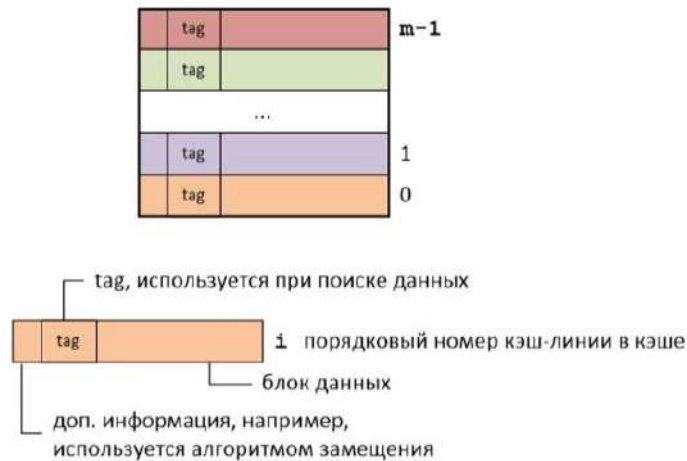


Рис.82 - Кэш - разбиение на кэш-линии

Суть кэширования состоит в разбиении RAM на кэш-линии и отображении их на кэш-линии кэш-памяти. Возможно несколько вариантов такого отображения.

## Кэш прямого отображения (Direct Mapping)

Основная идея прямого отображения (direct mapping) RAM (ОЗУ) на кэш-память состоит в следующем: ОЗУ делится на сегменты, причем размер каждого сегмента равен полному размеру кэша, а каждый сегмент в свою очередь делится на блоки, размер каждого блока равен размеру кэш-линии.

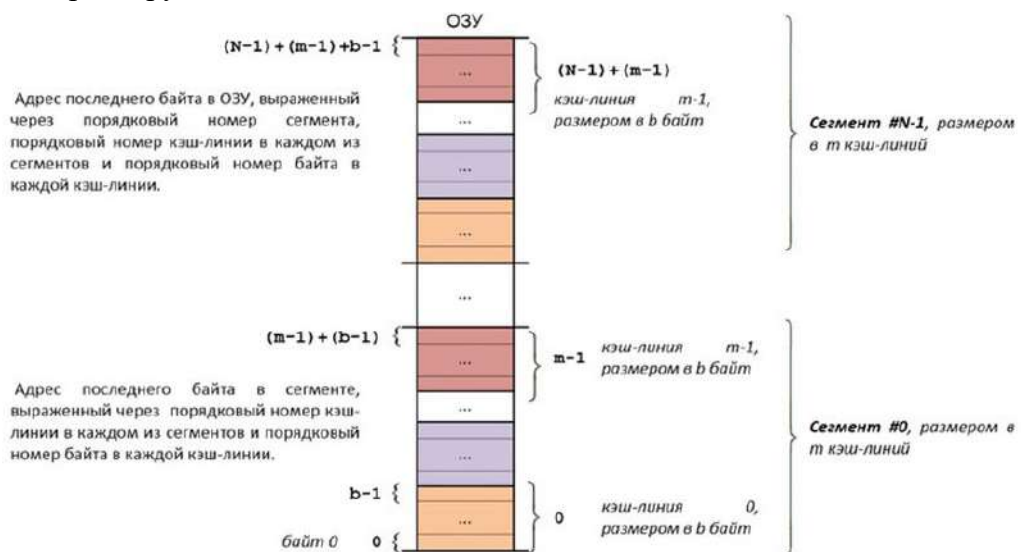
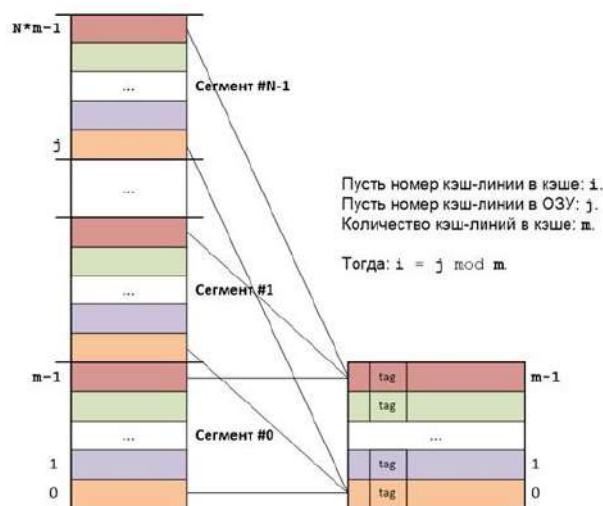


Рис.83 - Кэш прямого отображения

Блоки RAM из разных сегментов, но с одинаковыми номерами в этих сегментах, всегда будут отображаться (претендовать) на одну и ту же кэш-линию кэша:





**Рис.84 – Отображение блоков ОЗУ на линии кэша в случае direct mapping**

Адрес каждого байта представляет собой сумму порядкового номера сегмента, порядкового номера кэш-линии внутри сегмента и порядкового номера байта внутри кэш-линии. Отсюда следует, что адреса байт различаются только старшими частями, представляющими собой порядковые номера сегментов, а порядковые номера кэш-линий внутри сегментов и порядковые номера байт внутри кэш-линий повторяются.

Таким образом нет необходимости хранить полный адрес кэш-линии, достаточно сохранить только старшую часть адреса. Тэг (tag) каждой кэш-линии как раз и хранит старшую часть адреса первого байта в данной кэш-линии.

$b$  — размер кэш-линии.

$m$  — количество кэш-линий в кэше.

Для адресации  $b$  байт внутри каждой кэш-линии потребуется:  $\log_2 b$  бит. Для адресации  $m$  кэш-линий внутри каждого сегмента потребуется:  $\log_2 m$  бит.

$m = \text{Объем кэш-памяти} / \text{Размер кэш линии}.$

Для адресации  $N$  сегментов RAM:  $\log_2 N$  бит.

$N = \text{Объем RAM} / \text{Размер сегмента}.$

Для адресации байта потребуется:  $\log_2 N + \log_2 m + \log_2 b$  бит.

Этапы поиска в кэше:

1. Извлекается средняя часть адреса ( $\log_2 m$ ), определяющая номер кэш-линии в кэше.
2. Тэг кэш-линии с данным номером сравнивается со старшей частью адреса ( $\log_2 N$ ).

Если было совпадение по одному из тэгов, то произошло кэш-попадание (**cache hit**). Если не было совпадение ни по одному из тэгов, то произошел кэш-промах (**cache miss**).

## Полностью ассоциативный кэш (Fully Associative Mapping)

Основная идея полностью ассоциативного отображения (fully associative mapping) RAM на кэш-память состоит в следующем: RAM делится на блоки, размер которых равен размеру кэш-линии, а каждый блок RAM может сохраняться в любой кэш-линии кэша:

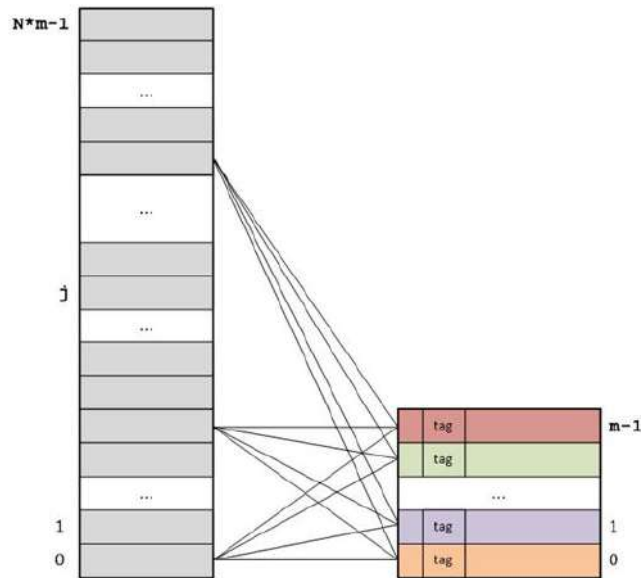


Рис.85 - Отображение блоков ОЗУ на линии кэша в случае fully associative mapping

Адрес каждого байта представляет собой сумму порядкового номера кэш-линии и порядкового номера байта внутри кэш-линии. Отсюда следует, что адреса байт различаются только старшими частями, представляющими собой порядковые номера кэш-линий. Порядковые номера байт внутри кэш-линий повторяются.

Тэг (tag) каждой кэш-линии хранит старшую часть адреса первого байта в данной кэш-линии.

$b$  — размер кэш-линии.

$m$  — количество кэш-линий, уместяющихся в RAM.

Для адресации  $b$  байт внутри каждой кэш-линии потребуется:  $\log_2 b$  бит.

Для адресации  $m$  кэш-линий:  $\log_2 m$  бит.

$m = \text{Размер RAM} / \text{Размер кэш-линии}$ .

Для адресации байта потребуется:  $\log_2 m + \log_2 b$  бит.

Этапы поиска в кэше:

Тэги всех кэш-линий сравниваются со старшей частью адреса одновременно.

Если было совпадение по одному из тэгов, то произошло кэш-попадание (**cache hit**). Если не было совпадения ни по одному из тэгов, то произошел кэш-промах (**cache miss**).

## Наборно-ассоциативный кэш (Set Associative Mapping)

Основная идея наборно ассоциативного отображения (set associative mapping) RAM на кэш-память состоит в следующем: RAM делится также, как и в прямом отображении, а сам кэш состоит из  $k$  кэшей ( $k$  каналов), использующих прямое отображение.





Рис.86 – Наборно-ассоциативный кэш.

Кэш-линии, имеющие одинаковые номера во всех каналах, образуют **set** (набор). Каждый set представляет собой кэш, в котором используется полностью ассоциативное отображение.

Блоки RAM из разных сегментов, но с одинаковыми номерами в этих сегментах, всегда будут отображаться на один и тот же set кэша. Если в данном сете есть свободные кэш-линии, то считываемый из RAM блок будет сохраняться в свободную кэш-линию, если же все кэш-линии сета заняты, то кэш-линия выбирается согласно используемому алгоритму замещения.

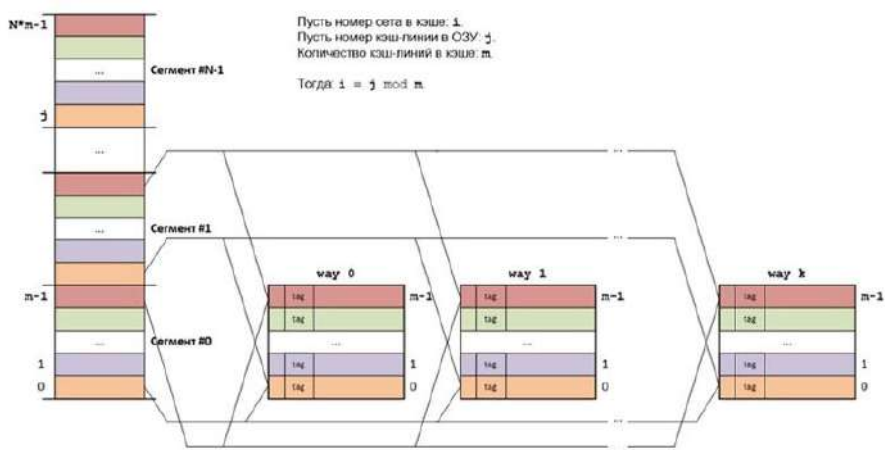


Рис.87 - Отображение в наборно-ассоциативном кэше

Структура адреса байта в точности такая же, как и в прямом отображении:  $\log_2 N + \log_2 m + \log_2 b$  бит, но т.к. set представляет собой  $k$  различных кэш-линий, то поиск в кэше немного отличается.

Этапы поиска в кэше:

1. Извлекается средняя часть адреса ( $\log_2 m$ ), определяющая номер сета в кэше.
2. Тэги всех кэш-линий данного сета сравниваются со старшей частью адреса ( $\log_2 N$ ) одновременно.

Если было совпадение по одному из тэгов, то произошло кэш-попадание (**cache hit**). Если не было совпадения ни по одному из тэгов, то произошел кэш-промах (**cache miss**).

Таким образом количество каналов (way) кэша определяет количество *одновременно* сравниваемых тэгов.

*Прямое отображение* – простой и недорогой в реализации способ отображения. Основной его недостаток – жёсткое закрепление за определенными блоками ОП одной строки в кэше. Поэтому если программа поочередно обращается к словам из двух различных блоков, отображаемых на одну и ту же строку кэш-памяти, постоянно будет происходить обновление данной строки и вероятность попадания будет низкой.

*Полностью ассоциативное* отображение позволяет преодолеть недостаток прямого, разрешая загрузку любого блока ОП в любую строку кэш-памяти. Ассоциативное отображение обеспечивает гибкость при выборе строки для вновь записываемого блока. Принципиальный недостаток этого способа – необходимость выполнения *одновременной* проверки для всех строк кэш-памяти.

*Частично-ассоциативное отображение* является одним из возможных компромиссов, сочетающим достоинства прямого и ассоциативного способов отображения и, в известной мере, свободным от их недостатков. Кэш-память разбивается на  $v$  подмножеств (наборов), каждое из которых содержит  $k$  строк (принято говорить, что набор имеет  $k$  входов –  $K$ -way). Зависимость между набором и блоками ОП такая же, как и при прямом отображении: на строки, входящие в набор  $i$ , могут быть отображены только вполне определенные блоки основной памяти, в соответствии с соотношением  $i = j \bmod v$ , где  $j$  – адрес блока ОП. В то же время размещение блоков по строкам модуля – произвольное, и для поиска нужной строки в пределах модуля используется ассоциативный принцип.

В предельных случаях, когда  $v = m$ ,  $k = 1$ , множественно-ассоциативное отображение сводится к прямому, а при  $v = 1$ ,  $k = m$  – к ассоциативному.

Логика работы кэша основана на том, что программа гораздо чаще читает данные из памяти (те же самые или расположенные рядом, принцип локальности), чем пишет в них. А как поступать с записываемыми данными? Есть два основных подхода (политики) к кэшированию записи:

- *Сквозная запись (write-through)* — запись производится непосредственно в основную память, дублируясь при этом в кэш, то есть запись не кэшируется.
- *Отложенная (обратная) запись (write-back)* — запись данных производится в кэш. Запись же в основную память производится позже (при вытеснении или по истечении времени), группируя в одной операции несколько операций записи в соседние ячейки. Технология обратной записи на некоторое время делает данные в основной памяти неактуальными («грязные» данные), для самого ЦП эта неактуальность не заметна, но перед обращением к памяти другого ведущего системной шины (контроллера DMA, bus-master-устройства шины PCI, другого ядра) кэш должен быть записан (сброшен) в память принудительно. При использовании обратной записи в многопроцессорной системе кэши различных ЦП должны быть согласованы (или процессоры должны использовать одну общую разделяемую кэш-память).

Организация и размеры кэша это, пожалуй, наиболее часто и сильно изменяющаяся характеристика при переходе на новое поколение/семейство процессоров. Более того, размер кэша и его организация (количество наборов, set) может значительно варьироваться внутри одного семейства – в младших моделях объём кэша обычно меньше, и иногда в них бывает уменьшено количество set-ов (way).

Первым процессором x86, на кристалле которого была интегрирована кэш-память, был процессор i80486. Его кэш был одноуровневым (L1), и имел скромный объём 8 КБайт (позднее 16 КБайт). Кэш имел 4-канальную наборно-ассоциативную архитектуру и работал на уровне физических адресов памяти.

Обычно разработчики архитектуры CPU выдерживают правило построения иерархической памяти – чем «дальше» от процессора, тем медленнее, но больше объём (т.е., объём  $L1 < L2$ , но  $L1$  в несколько раз быстрее). Но в жизни бывали исключения, в некоторых моделях CPU компании AMD размер кэша  $L1=128$  КБайт вдвое превосходил объём кэша  $L2=64$  КБайт.

По логике попадания данных в кэш разных уровней компании Intel и AMD используют разные подходы: Intel строит двухуровневый кэш как *инклюзивный* (inclusive) – всё, что есть в кэше  $L1$ , *обязательно* имеет копию в кэше  $L2$ . AMD практикует *эксклюзивный*

(exclusive) подход, при котором, напротив, то, что попало в L1, обязательно *не имеет* копии в L2. Можно это трактовать так: в интеловском подходе общий размер кэша равен объёму бóльшего кэша второго уровня L2, но часть этого кэша, равная по размеру L1, более быстрая, чем остальная часть L2-L1. У AMD, с эксклюзивным подходом, суммарный объём кэша равен сумме кэшей L1+L2, с двумя разными временами доступа для каждой из частей, соответствующих своему уровню.

Что интересно: когда в процессорах появился кэш третьего уровня, Intel объявила, что L3 в нём не будет ни чисто инклюзивным, ни чисто эксклюзивным. Т.е., данные попавшие в кэши L1/L2, *могут* попасть в кэш L3, а могут не попасть. Также обычно кэш L3 распределён (shared) между всеми ядрами, тогда как L1/L2 – индивидуальные (приватные) для каждого физического ядра.

Ниже приведены характеристики кэша одного из современных семейств Intel Skylake. Обратите внимание, что кэш первого уровня L1 раздельный – отдельно для кода, отдельно для данных, это обычная практика для процессоров Intel.

В серверных процессорах Skylake были сделаны некоторые значительные изменения в иерархии кэша по сравнению с тем, как это было реализовано в семействах Broadwell/Haswell. Иерархия памяти для серверных процессоров Skylake и HEDT<sup>1</sup> процессоров была перебалансирована. Обратите внимание, что L3 теперь не инклюзивный и что часть статического SRAM кэша из L3 была перенесена в приватный L2 кэш.

#### Skylake Cache:

- L0  $\mu$ OP cache:
  - 1,536  $\mu$ OPs/core, 8-way set associative
    - 32 sets, 6- $\mu$ OP line size
    - statically divided between threads, inclusive with L1I
- L1I Cache:
  - 32 KiB/core, 8-way set associative
    - 64 sets, 64 B line size
    - competitively shared by the threads/core
- L1D Cache:
  - 32 KiB/core, 8-way set associative
  - 64 sets, 64 B line size
  - competitively shared by threads/core
  - 4 cycles for fastest load-to-use (simple pointer accesses)
    - 5 cycles for complex addresses
  - 128 B/cycle load bandwidth
  - 64 B/cycle store bandwidth
  - Write-back policy
- L2 Cache:
  - 1 MiB/core, 16-way set associative
  - 64 B line size
  - Inclusive
  - 64 B/cycle bandwidth to L1 cache
  - Write-back policy
  - 14 cycles latency
- L3 Cache:
  - 1.375 MiB/core, 11-way set associative, shared across all cores
    - Note that a few models have non-default cache sizes due to disabled cores
  - 2,048 sets, 64 B line size
  - Non-inclusive victim cache
  - Write-back policy
  - 50-70 cycles latency

---

<sup>1</sup> HEDT - high-end desktop – десктопные CPU для энтузиастов.

## Оперативная память (RAM)

**Оперативная память** (*Random Access Memory, RAM*, память с произвольным доступом<sup>1</sup>) или **оперативное запоминающее устройство (ОЗУ)** — *энергозависимая* часть системы компьютерной памяти, в которой во время работы компьютера хранится выполняемый машинный код (программы), а также входные, выходные и промежуточные данные, обрабатываемые процессором.

Обмен данными между процессором и оперативной памятью производится:

- непосредственно;
- через сверхбыструю память 0-го уровня — регистры в АЛУ, либо при наличии аппаратного кэша процессора — через кэш.

Содержащиеся в полупроводниковой оперативной памяти данные доступны и сохраняются только тогда, когда на модули памяти подаётся напряжение. Выключение питания оперативной памяти, даже кратковременное, приводит к искажению либо полному разрушению хранимой информации.

Энергосберегающие режимы работы материнской платы компьютера позволяют переводить его в режим *сна*, что значительно сокращает уровень потребления компьютером электроэнергии. В режиме *гибернации* питание ОЗУ отключается. В этом случае для сохранения содержимого ОЗУ операционная система (ОС) перед отключением питания записывает содержимое ОЗУ на устройство постоянного хранения данных (как правило, жёсткий диск). Например, в ОС Windows XP содержимое памяти сохраняется в файл hiberfil.sys, в ОС семейства Unix — на специальный swap-раздел жёсткого диска (хотя возможна организовать swap в файл, как в Windows).

В общем случае ОЗУ содержит программы и данные ОС, запущенные прикладные программы пользователя и данные этих программ, поэтому от объёма оперативной памяти зависит количество задач, которые одновременно может выполнять компьютер под управлением ОС.

Оперативное запоминающее устройство (ОЗУ) — техническое устройство, реализующее функции оперативной памяти. ОЗУ может изготавливаться как отдельный внешний модуль или располагаться на одном кристалле с процессором, например, в однокристальных ЭВМ или однокристальных микроконтроллерах.

В «больших» архитектурах в качестве ОЗУ обычно используется *DRAM* (Dynamic Random Access Memory), в которой бит данных хранится в виде заряда конденсатора. Однобитовая ячейка памяти содержит конденсатор и транзистор. Конденсатор заряжается до высокого или низкого напряжения (логические 1 или 0). Транзистор выполняет функцию ключа, подключающего конденсатор к схеме управления, расположенного на том же чипе. Схема управления позволяет считывать состояние заряда конденсатора или изменять его. Так как хранение 1 бита информации в этом виде памяти дешевле, чем в статической, *DRAM* преобладает в компьютерах третьего поколения. Основное преимущество *DRAM* по сравнению со статическими триггерами в *SRAM* — гораздо более высокая плотность бит/мм<sup>2</sup>, но *DRAM* заметно медленнее, поэтому основное место, где массово используется *SRAM* — это кэш-память. Ещё один недостаток технологии *DRAM* по сравнению со *SRAM* — динамическая память требует периодической (раз в несколько миллисекунд) регенерации (чтения строки матрицы памяти), чтобы компенсировать саморазряд конденсатора.

Схема ячейки динамической памяти:

---

<sup>1</sup> В информатике под **произвольным доступом** (также называемым **случайным доступом**, англ. *random access*) понимают возможность обратиться к любому элементу последовательности за равные промежутки времени, не зависящие от размеров последовательности (в отличие от последовательного доступа, при котором чем дальше расположен элемент, тем больше требуется времени для доступа к нему).

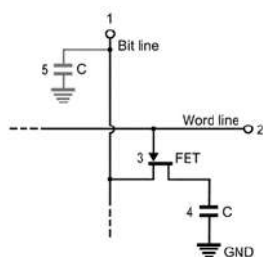


Рис.88 – Ячейка DRAM (1 транзистор, 1 конденсатор)

На протяжении долгого времени разработчики создавали различные типы DRAM с использованием различных технических, технологических и архитектурных решений. Основной движущей силой такого развития были стремление увеличить быстродействие и объём оперативной памяти.

Первые поколения динамической памяти выпускались в начале 1980-х годов и имели времена цикла порядка 250 нс. Один из первых специализированных компьютерных типов DRAM – **PM DRAM** – выпускавшийся в начале 1990-х, имел времена цикла порядка 200 нс.

Технологии организации и производства динамической памяти активно развивались, за последние десятилетия DRAM «прожила» более чем десяток поколений. В таблице Рис.89 в строке «Chip densities» («плотность чипа» ЗУ) интересна последняя колонка (подчёркнуто): 64 Gb. Для сравнения за точку отсчёта можно принять ёмкость чипа образца 1980-го года: 64 Kb. Т.е. на сегодня, за 40 лет существования, ёмкость чипов динамической памяти выросла в  $64\text{Gb}/64\text{Kb} = 10^6$  – в миллион раз!

| DDR Version          | DDR1       | DDR2      | DDR3                   | DDR 4      | DDR5            |
|----------------------|------------|-----------|------------------------|------------|-----------------|
| Released date        | 2000       | 2003      | 2007                   | 2012       | Under Progress  |
| Operating voltage    | 2.5V       | 1.8V      | 1.5V                   | 1.2V       | 1.1V            |
| Prefetch buffer size | 2          | 4         | 8                      | 8          | 16              |
| Chip densities       | 128Mb-1Gb  | 128Mb-4Gb | 512Mb-8Gb              | 2Gb-16Gb   | <u>8Gb-64Gb</u> |
| Data rate (MT/s)     | 200-400    | 400-800   | 800-2133               | 1600-3200  | 3200-6400       |
| Bank groups          | 0          | 0         | 0                      | 4          | 8               |
| Termination/ODT      | Ω on board | ODT added | Nominal, Dynamic Modes | Park Modes | Nominal Wr/Rd   |

Рис.89 - Сравнение поколений DDR

А вот скоростные характеристики DRAM (Рис.90) за это же время улучшились гораздо менее существенно, речь в первую очередь идёт о задержках (latency), временах доступа к ячейке памяти: с примерных ~250 нс в 1980г. эти цифры удалось сократить в лучшем случае до ~35-40 нс, т.е. всего в ~7 раз.

| DRAM Type                                      | Standard Power |           |           |           |                 | Low Power |           |           |            |
|------------------------------------------------|----------------|-----------|-----------|-----------|-----------------|-----------|-----------|-----------|------------|
|                                                | DDR3           | DDR4      | GDDR5     | HBM       | HMC             | LPDDR3    | LPDDR4    | Wide I/O  | Wide I/O 2 |
| Data Rate (MT/s)                               | 2133           | 3200      | 7000      | 1000      | 2500            | 2133      | 3200      | 266       | 1067       |
| Clock Frequency (MHz)                          | 1067           | 1600      | 1750      | 500       | 1250            | 1067      | 1600      | 266       | 533        |
| Maximum Bandwidth (GBps)                       | 68.3           | 102.4     | 224.0     | 128.0     | 320.0           | 68.3      | 51.2      | 17.0      | 34.1       |
| Channels/Ranks per Channel                     | 4/1            | 4/1       | 4/1       | 8/1       | 1/1             | 4/1       | 4/1       | 4/1       | 4/2        |
| Banks per Rank                                 | 8              | 16        | 16        | 16        | 256 (32 vaults) | 8         | 16        | 4         | 8          |
| Channel Width (bits)                           | 64             | 64        | 64        | 128       | 32              | 64        | 64        | 128       | 64         |
| Row Buffer Size                                | 8KB            | 8KB       | 8KB       | 2KB       | 256B            | 8KB       | 4KB       | 2KB       | 4KB        |
| Row Hit/Miss Latencies (ns)                    | 15.0/26.3      | 16.7/30.0 | 13.1/25.1 | 18.0/32.0 | 16.8/30.4       | 21.6/40.3 | 26.9/45.0 | 30.1/38.9 | 22.5/41.3  |
| Minimum Row Conflict Latency <sup>†</sup> (ns) | 37.5           | 43.3      | 37.1      | 46.0      | 44.0            | 59.1      | 61.9      | 67.7      | 60.0       |

Рис.90 - Ключевые характеристики некоторых моделей DRAM

Вот он источник memory wall или CPU↔memory performance gap – процессоры за это же время стали быстрее в ~1000 раз, и это ещё без учёта многоядерной архитектуры. А с учётом многоядерности всё становится ещё во много раз хуже, во столько раз, сколько ядер CPU будет параллельно конкурировать за доступ к памяти (реальный пример AMD

Ерус, приводившийся выше: задержка доступа к памяти ~100 нс, к которой – в худшем случае – обращаются одновременно 64 ядра, каждое работающее на частоте 3,2 ГГц (такт ~300пс)).

### 3D XPoint (Intel Optane)

Эта технология, по своим характеристикам попадающая в иерархию памяти между уровнями «Оперативная память» и «Дисковая память», появилась на рынке относительно недавно. В июле 2015 года компании Intel и Micron анонсировали принципиально новую технологию *энергонезависимой* памяти – 3D XPoint. Основанные на технологии 3D XPoint устройства компании Intel выпускаются под торговой маркой **Optane**.

Память 3D XPoint на один-два порядка быстрее традиционной энергонезависимой флэш-памяти NAND – основы большинства современных твердотельных накопителей (SSD). Кроме того, ресурс перезаписи у 3D XPoint также примерно на два порядка выше, чем у обычной флэш-памяти. И, наконец, 3D XPoint допускает побайтовое чтение и запись в отличие от NAND, которая позволяет читать и записывать данные только поблочно, блоками размером не менее 4 КБайт.

С другой стороны, 3D XPoint примерно в пять раз медленнее<sup>1</sup> стандартной *энергозависимой* памяти DRAM, используемой в качестве оперативной памяти в компьютерах и серверах.

Первое применение технологии – твердотельные NVMe-накопители Intel Optane с интерфейсом PCIe 3.0 x4, выполненные либо в виде плат расширения, устанавливаемых в слоты PCIe, либо в виде обычных накопителей 2,5”, устанавливаемых в дисковую корзину и подключаемых к специальным разъемам для накопителей NVMe (M.2). Основное назначение данной технологии – ускорение работы SATA-накопителей за счёт «умного» кэширования, с использованием драйвера технологии хранения Intel RST.

Хотя 3D XPoint существенно быстрее NAND, дополнительные задержки, вносимые интерфейсом PCIe, не позволяют накопителям Optane NVMe полностью раскрыть свой скоростной потенциал, поэтому SSD NVMe на базе 3D XPoint и SSD NVMe на базе NAND демонстрируют схожую производительность, значительно отличаясь друг от друга только по ресурсу записи.

Несколько лет спустя после появления технологии 3D XPoint, компания Intel представила новую реинкарнацию этого типа памяти, в которой сочетание её уникальных возможностей – быстродействия, выносливости, побайтовой адресации и энергонезависимости, – способно проявить себя в полной мере.

В апреле 2018 года компания Intel объявила о начале продаж модулей памяти, основанных на технологии 3D XPoint и предназначенных для установки в стандартные разъёмы оперативной памяти на материнской плате (DIMM). Такие модули получили название **Intel Optane Data Center Persistent Memory Module (DCPMM)**. Данная память, в отличие от обычной DDR, является энергонезависимой. И в данном случае правильнее говорить не о памяти RAM, а об интерфейсе этой памяти, который используется в новых DIMM. За счёт более высокой плотности 3D XPoint Intel планирует устанавливать до 512 ГБайт памяти на модуль (что в несколько раз больше максимального объёма для DDR RAM), что позволит получить до 3 ТБайт на каждый сокет CPU. В случае грядущих серверных процессоров Cascade Lake-SP Intel планирует поддерживать 12 модулей DIMM на сокет, как и в случае Ice Lake Xeon под сокет LGA 4189. При этом половину из 12 слотов DIMM могут занимать планки Optane DC Persistent Memory.

---

<sup>1</sup> Т.к. 3D XPoint медленнее DRAM, но быстрее SSD, автор решил разместить описание этой технологии в иерархии памяти между «ОЗУ» и «Дисковой памятью».

## Дисковая память

В отличие от описанных ранее первых трёх (четырёх, если считать Intel Optane) уровней в иерархии памяти – регистры, кэш, ОЗУ – дисковая подсистема – это внешняя система. Многие десятилетия<sup>1</sup> на этом уровне иерархии работали магнитные диски, но в последние годы, примерно с середины 2000-х, их всё активнее теснят твердотельные (SSD) диски.

### Жёсткие магнитные диски (HDD)

**Накопитель на жёстких магнитных дисках**, или **НЖМД** (*hard (magnetic) disk drive, HDD, HMDD*), *жёсткий диск, винчестер* — запоминающее устройство (устройство хранения информации, накопитель) произвольного доступа, основанное на принципе магнитной записи. На сегодня является основным накопителем данных в большинстве компьютеров.

В отличие от гибкого диска (дискеты), информация в НЖМД записывается на жёсткие (алюминиевые или стеклянные) пластины, покрытые слоем ферромагнитного материала, чаще всего диоксида хрома — магнитные диски. В НЖМД используется одна или несколько пластин на одной оси. Считывающие головки в рабочем режиме не касаются поверхности пластин благодаря прослойке набегающего потока воздуха, образующейся у поверхности при быстром вращении. Расстояние между головкой и диском составляет несколько нанометров (в современных дисках около 10 нм), а отсутствие механического контакта обеспечивает долгий срок службы устройства. При отсутствии вращения дисков головки находятся у шпинделя или за пределами диска в безопасной («парковочной») зоне, где исключён их нештатный контакт с поверхностью дисков.

Также, в отличие от гибкого диска, носитель информации обычно совмещают с накопителем, приводом и блоком электроники. Такие жёсткие диски часто используются в качестве несъёмного носителя информации.

На Рис.91 представлена фотография 2,5-дюймового жёсткого диска с интерфейсом SATA и 4-контактным служебным разъёмом для доступа к микропрограмме диска. Гермо-зона вскрыта.



Рис.91 - 2.5-дюймовый диск

Основные характеристики диска – ёмкость и производительность. Для сравнения с современными дисками за точку отсчёта, пожалуй, наиболее корректным будет взять первые относительно компактные диски, которые называли «винчестеровскими»<sup>2</sup>, разработанные в IBM в 1973-ем году.

<sup>1</sup> Первый жёсткий магнитный диск IBM объёмом 5 МБайт (это не опечатка) был выпущен в сентябре 1956-го года. Весила система около тонны и состояла из 50-ти(!) физических дисков диаметром 24 дюйма (610 мм).

<sup>2</sup> По одной из версий, название «винчестер» (*Winchester*) накопитель получил благодаря работавшему в фирме IBM Кеннету Хотону (*Kenneth E. Haughton*), руководителю проекта. В результате работы над этим проектом в 1973 году был выпущен жёсткий диск модели 3340, впервые объединивший в одном неразъёмном корпусе пластины диска и считывающие головки. При его разработке инженеры использовали краткое внутреннее



Ёмкость первых массовых «винчестеров» была порядка нескольких десятков мегабайт ( $\sim 10\text{-}20 \cdot 10^6$  (10-20 МБ)). Сегодняшние современные диски имеют ёмкость практически на шесть порядков больше ( $\sim 10\text{-}20 \cdot 10^{12}$  (10-20 ТБ)). Ниже (Рис.92) приведен график роста объёмов магнитных дисков компании Seagate. В 2020 году на рынке были доступны диски с максимальной ёмкостью 20ТБ, а примерно в 2023-24 году прогнозируется выпуск дисков с ёмкостью  $\sim 30\text{ТБ}$ .

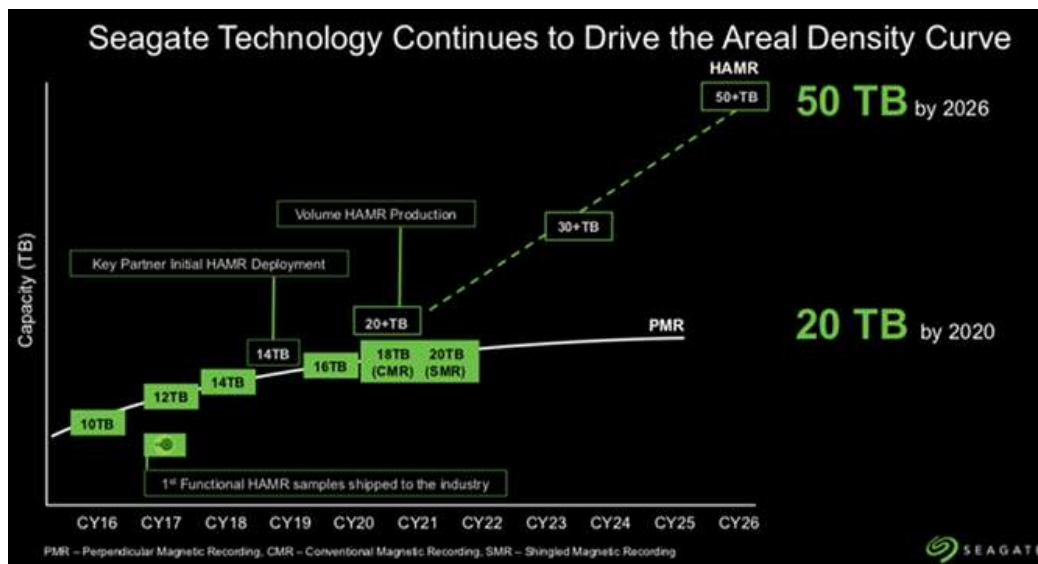


Рис.92 - Ёмкости дисков Seagate

Если сравнить с ростом объёмов DRAM на те же шесть порядков (в миллион раз), то дискам, для такого же кратного увеличения ёмкости, потребовалось несколько больше времени (1973 – 2020 vs 1980 – 2020).

Производительность жёсткого диска может быть охарактеризована двумя основными параметрами: временем доступа к данным и скоростью линейного чтения/записи.

Средние времена доступа определяются наиболее медленной (механической) составляющей конструкции дискового устройства: временем перемещения головки считывания/записи на нужный угол – это времена порядка нескольких миллисекунд, и временем ожидания, когда нужный сектор на вращающемся диске «подъедет» под считывающую/записывающую головку.

Скорость вращения «блинов» в современных дисках – это обычно наиболее распространённые цифры 5400 об/мин или 7200 об/мин (жёсткие диски вращаются с постоянной угловой скоростью, в отличие от, н-р, CD, где поддерживается постоянная линейная скорость). Более производительные диски со скоростями 10 000 и 15 000 об/мин на сегодня стремительно теряют свою долю присутствия на рынке, их активно вытесняют твердотельные SSD-диски.

По теории вероятности, диску в среднем придётся совершить пол-оборота, чтобы нужный сектор оказался под головкой. Для диска со скоростью вращения 7200 об/мин время поворота диска на пол-оборота составляет  $\sim \frac{1}{2} \cdot \frac{60}{7200} \approx 4\text{мс}$  ( $4 \cdot 10^{-3}$ ).

Скорость линейного (последовательного) чтения/записи определяется темпом поступления данных с подряд идущих секторов. Данные на жёстких дисках организованы по схеме «цилиндр-головка-сектор» (CHS) (Рис.93):

название «30-30», что означало два модуля (в максимальной компоновке) по 30 мегабайт каждый, что по созвучию совпало с обозначением популярного охотничьего оружия — винтовки Winchester Model 1894, использующего винтовочный патрон .30-30 Winchester.





Рис.93 - Цилиндр-Головка-Сектор

В ранних моделях дисков количество секторов на дорожку (цилиндр) не зависело от номера дорожки, что означает: а) постоянную скорость последовательного (линейного) чтения/записи; б) более высокую плотность секторов на внутренних дорожках.

В современных дисках количество секторов на дорожке зависит от номера цилиндра (диаметра) и меняется от максимального на самой внешней дорожке до минимальной на внутренней. Количество секторов меняется не на каждой следующей дорожке, а ступенчато – дорожки сгруппированы в зоны, все дорожки одной зоны имеют одинаковое количество секторов. Соответственно, скорость последовательного чтения (записи) на внешних дорожках будет выше скорости на внутренних, обычно это разница  $\sim$  в 2-2.5 раза. Ступенчатое изменение скорости линейного чтения при переходе от цилиндра к цилиндру хорошо видно на Рис.94:

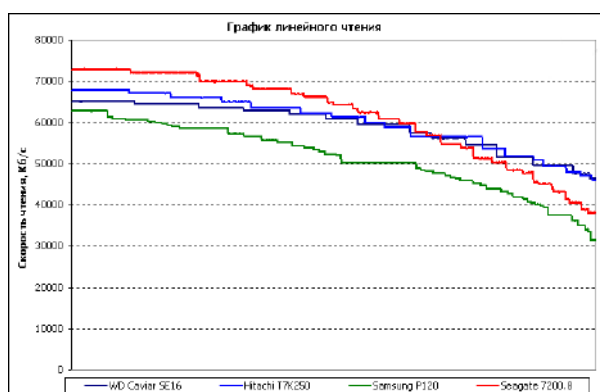


Рис.94 - График линейного чтения

Размер сектора жёстких дисков долгое время не менялся и составлял 512 байт ( $2^9$ ). Такой размер сектора ограничивает максимальный размер диска цифрой  $\sim 2^9 * 2^{32} \approx 2$  ТБ. Для преодоления этого ограничения в современных дисках используется сектор размером 4КБ ( $2^{12}$ ).

Скорость линейного чтения в современных жёстких дисках достигает значений 120-150 Мбайт/сек, очень приличная цифра ( $\sim 6,6$  нс/байт). Сравните с характерными цифрами «механической» составляющей диска, например, с временами доступа – единицы **миллисекунд**. Разрыв – в шесть порядков ( $10^{-3} / 10^{-9} = 10^6$ )! И вот эта медленная компонента (скорость вращения и скорость позиционирования головки) за все годы ускорилась совсем незначительно: первый жёсткий диск IBM вращался на скорости 1200 об/мин, а это всего в  $15000 / 1200 = 12,5$  раз медленнее современных самых скоростных жёстких дисков. Линейные скорости выросли более значительно, с примерно нескольких десятков килобайт в секунду до современных  $\sim 150$  Мбайт/сек. Объёмы выросли в миллионы раз ( $10^6$ ), линейные скорости – в тысячи ( $10^3$ ), а времена доступа уменьшились всего в несколько раз. Налицо всё нарастающий разрыв между ёмкостными характеристиками и производительностью дисков.

Самая медленная компонента (время доступа) в магнитном диске определяет очень низкую производительность диска при случайном доступе, когда каждый следующий блок информации считывается из случайного сектора – головка вынуждена «прыгать», перемещаясь в нужные позиции и ожидая каждый раз, когда диск повернётся на нужный угол, чтобы к ней «подъехал» нужный сектор. Число, обратное времени доступа, это, по сути, максимальное количество операций ввода-вывода (IOPS) в секунду, выполняемых диском при случайном доступе. Для самых производительных дисков со скоростью вращения шпинделя 15 000 об/мин, количество IOPS-ом оказываются всего... ~250 в секунду. Не миллионов, даже не тысяч, а всего несколько сотен операций в секунду.

Для хотя бы частичного сглаживания последствий медленной механики, в современных дисках используется одновременно несколько технологий:

1. Буферная память, кэш, реализованный на DRAM. Объём кэша составляет обычно 32-64МБ.
2. Очередь команд – приняв к исполнению команды для доступа к конкретным секторам, контроллер диска переупорядочивает команды так, чтобы минимизировать случайные перемещения головки и стараясь организовать доступ так, чтобы головка не «металась» по диску, а по возможности одним заходом (от края к центру или от центра к краю) прошла по всем запланированным секторам (т.н. scatter/gather).



Рис.95 - Тестирование производительности HDD диска

На Рис.95 приведена таблица результатов тестирования жёсткого диска программой CrystalDiskMark (тестировался диск модели Toshiba L200 2ТБ). Комментарии к обозначениям:

- Seq — тест последовательного чтения/записи.
- 512K — тест на случайное чтение/запись блоков по 512 КБ.
- 4K — тест на случайное чтение/запись блоков по 4 КБ (глубина очереди — 1).
- 4K (QD32) — тест на случайное чтение/запись блоков по 4 КБ (глубина очереди — 32).

Бросается в глаза колоссальная разница в скорости последовательного чтения/записи – ~150/160 МБ/сек – и скорости случайного чтения/записи (особенно для случая с глубиной очереди =1) – всего 0,4/5,2 Мбайт/сек. Следует обратить внимание также на существенную разницу в скорости случайного чтения и случайной записи – более чем в 10 раз. Здесь, очевидно, сказывается политика отложенной (обратной) записи внутреннего кэша диска. А вот при случайном чтении кэш не выручает, т.к. в этом случае нарушаются принципы локальности данных, основы работы кэша.

## Твердотельные накопители (SSD)

**Твердотельный накопитель** (*solid-state drive, SSD*) — компьютерное энергонезависимое немеханическое запоминающее устройство на основе микросхем памяти, альтернатива HDD. Кроме микросхем памяти, SSD содержит управляющий контроллер. Наиболее распространённый вид твердотельных накопителей использует для хранения информации флэш-память<sup>1</sup> типа NAND, однако существуют варианты, в которых накопитель создаётся на базе DRAM-памяти, снабжённой дополнительным источником питания — аккумулятором.

В настоящее время твердотельные накопители используются как в носимых (ноутбуках, нетбуках, планшетах) так и в стационарных компьютерах для повышения производительности. В последние годы стремительно развиваются дисковые массивы<sup>2</sup>, построенные на SSD накопителях, как в гибридном (HDD + Flash), так и в «чистом» («All Flash») исполнении.

Наиболее производительные на сегодня SSD-диски представлены в формате M.2 NVME, скорость записи/чтения данных у таких дисков может достигать 4000-5000 мегабайт в секунду.

По сравнению с традиционными жёсткими дисками (HDD) твердотельные накопители имеют меньший размер и вес, являются беззвучными, а также многократно более устойчивы к повреждениям (например, к падению) и имеют гораздо большую скорость записи и чтения. В то же время, они имеют в несколько раз большую стоимость в расчёте на гигабайт и существенно меньшую износостойкость (ресурс записи).

### Описание

SSD представляют собой устройства, хранящие данные в микросхемах флэш-памяти, вместо вращающихся металлических дисков или магнитных лент. Причина их появления отражает тот факт, что скорость обработки данных в процессоре намного превышает скорость чтения/записи данных в HDD. Магнитные диски на протяжении десятилетий доминировали в корпоративном сегменте хранения данных, за это время (с 1950-х) ёмкость носителей выросла в миллионы раз, скорость работы процессоров тоже возросла на несколько порядков, но скорость доступа к данным изменилась значительно меньше и диски давно уже стали «узким местом» системы. Проблему (частично) решают твердотельные накопители — они обеспечивают намного большие скорости работы с данными по сравнению с магнитными жёсткими дисками.

SSD за счёт использования микросхем флэш-памяти по своим характеристикам существенно отличаются от жёстких дисков с магнитными пластинами. С целью оптимизации использования SSD в 2011 году был разработан логический интерфейс NVMe (*Non-Volatile Memory Express*), поддержка которого была добавлена в Windows, начиная только с версии

---

<sup>1</sup> **Флеш-память** (*flash memory*) — разновидность полупроводниковой технологии электрически перепрограммируемой памяти (EEPROM). Это же слово используется в электронной схемотехнике для обозначения технологически законченных решений постоянных запоминающих устройств в виде микросхем на базе этой полупроводниковой технологии. В быту это словосочетание закрепилось за широким классом твердотельных устройств хранения информации.

<sup>2</sup> **Дисковый массив** — внешнее устройство хранения, состоящее из нескольких дисков, «классических» HDD и твердотельных SSD, или их комбинации (гибридные массивы). Крупные инсталляции могут содержать сотни и даже тысячи дисковых накопителей.

8.1. Не все материнские платы поддерживают протокол NVMe. Поэтому обычно модели SSD выпускаются в двух версиях — AHCI<sup>1</sup> и NVMe<sup>2</sup>.

Основные характеристики твердотельных накопителей:

- меньшее время доступа к данным: от 100 до 1000 раз быстрее, чем у механических дисков;
- высокая скорость, вплоть до нескольких гигабайт в секунду, при случайном (random) доступе к данным;
- высокие значения IOPS благодаря высокой скорости и низкому времени доступа;
- низкая цена производительности, лучшее соотношение цены к производительности среди всех устройств хранения;
- высокая надёжность, SSD дают уровень сохранности данных такой же, как другие типы полупроводниковых устройств.

На Рис.96 приведена таблица результатов тестирования SSD-диска программой CrystalDiskMark:

|              | Read [MB/s] | Write [MB/s] |
|--------------|-------------|--------------|
| SEQ1M Q8T1   | 2707.90     | 2369.50      |
| SEQ1M Q1T1   | 2199.92     | 1988.09      |
| RND4K Q32T16 | 2370.29     | 2328.92      |
| RND4K Q1T1   | 280.29      | 260.44       |

Intel Optane SSD 905P 960GB / Intel Core i9-7960X

Рис.96 - Тестирование производительности SSD диска

Комментарии к обозначениям:

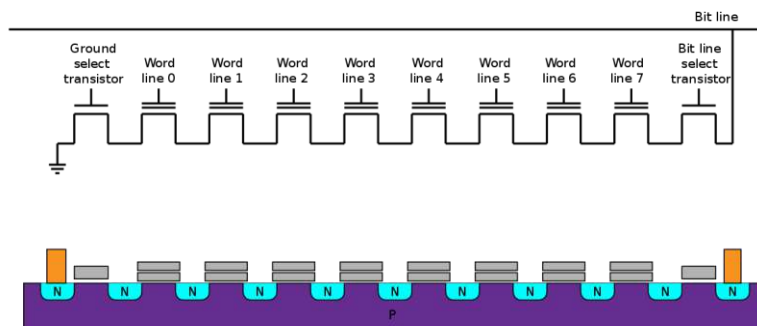
- SEQ1M Q8T1 — тест последовательного чтения/записи блоками 1МБ, глубина очереди 8, в 1 поток.
- SEQ1M Q1T1 — тест последовательного чтения/записи блоками 1МБ, глубина очереди 1, в один поток.
- RND4K Q32T16 — тест на случайное чтение/запись блоками по 4 КБ, глубина очереди 32, в 16 потоков.
- RND4K Q1T1 — тест на случайное чтение/запись блоками по 4 КБ, глубина очереди 1, в один поток).

<sup>1</sup> **Advanced Host Controller Interface (AHCI)** — механизм, используемый для подключения накопителей информации стандарта Serial ATA, позволяющий пользоваться расширенными функциями, такими, как встроенная очередность команд (NCQ) и горячая замена (hot swap). AHCI, по сути, является стандартизацией дешёвых версий контроллеров, позволяющей обойтись без установки дополнительных драйверов. AHCI также позволяет использовать твердотельные накопители SSD; однако он был разработан для механических HDD и при работе с SSD не позволяет использовать все их преимущества, в отличие от NVMe.

<sup>2</sup> **NVM Express (NVMe, NVMHCI — Non-Volatile Memory Host Controller Interface Specification)** — спецификация на протоколы доступа к твердотельным накопителям (SSD), подключённым по шине PCI Express. «NVM» в названии спецификации обозначает энергонезависимую память, в качестве которой в SSD повсеместно используется флэш-память типа NAND. Логический интерфейс NVM Express был разработан с нуля, основные цели — получение низких задержек и эффективное использование высокого параллелизма твердотельных накопителей за счёт применения нового набора команд и механизма обработки очередей, оптимизированного для работы с современными многоядерными процессорами.

Как видно, даже в самом неприятном сочетании – случайный доступ к мелким (4КБ) блокам в один поток и минимальной глубиной очереди =1, накопитель демонстрирует скорость ~в два раза выше максимальной скорости линейного чтения HDD. Ну, а «скорострельность» SSD накопителя при последовательном доступе принципиально недостижима для одиночного HDD, и превосходит последний практически в двадцать раз!

В отличие от жёстких дисков, цена SSD очень сильно зависит от доступной ёмкости, это связано с ограниченной плотностью размещения ячеек памяти и ограничением размера кристалла в микросхеме.



**Рис.97 - Структура одного столбца NAND flash с 8 ячейками**

Основным компонентом в флэш-памяти является транзистор с плавающим затвором, который является разновидностью МОП-транзисторов. Его отличие в том, что у него есть дополнительный затвор (плавающий), расположенный между управляющим затвором и p-слоем. Плавающий затвор изолирован, и сохраненный в нём отрицательный заряд будет сохраняться там долго (годы).

## SLC и MLC

Различают устройства, в которых элементарная ячейка хранит один бит информации и несколько бит. В однобитовых ячейках различают только два уровня заряда на плавающем затворе. Такие ячейки называют одноуровневыми (*single-level cell, SLC*). В многобитовых ячейках различают больше уровней заряда; их называют многоуровневыми (*multi-level cell, MLC*). MLC-устройства дешевле и более ёмкие, чем SLC-устройства, однако имеют более высокое время доступа и примерно на порядок меньшее максимальное количество циклов перезаписей.

SLC-устройства быстрее – то есть операции чтения/записи/стирания на них выполняются за меньшее количество времени. А поскольку для определения состояния ячейки памяти используется только одно пороговое значение напряжения, при использовании SLC-памяти меньше вероятность возникновения ошибки. Но это не значит, что MLC хуже. MLC-память, во-первых, позволяет сохранять большее количество информации, а во-вторых, она дешевле. То есть с точки зрения отношения цена/качество MLC, в принципе, выглядит предпочтительнее.

К 2016 году многоуровневая память доминирует на рынке. Тем не менее, SLC-изделия, несмотря на многократно меньшую ёмкость, продолжают разрабатываться и выпускаться для особо ответственных применений.

## Запись в NAND

При использовании NAND мы не имеем доступа к произвольной ячейке памяти. Все ячейки объединены в страницы, а страницы, в свою очередь, объединены в логические блоки. Каждая страница, помимо сохраненной пользовательской информации, содержит

некоторые дополнительные (избыточные) данные – информацию о «плохих» блоках, дополнительную служебную информацию для коррекции ошибок. Запись данных (Рис.98) в NAND можно производить только постранично, то есть, если мы хотим изменить какой-то бит, то нам нужно перезаписать всю страницу целиком. Но, для того, чтобы перезаписать страницу, мы сначала должны её стереть (Рис.99). Но стирать данные возможно только более крупным блоком, содержащим данную страницу. Для примера из характеристик микросхемы NAND-памяти NAND128W3A: размер страницы – 512 байт + 16 байт дополнительной служебной информации, размер блока – 16 кБайт, то есть 32 страницы.

В результате, если стирание затрагивает соседние страницы блока, содержащие актуальные данные, необходимо перед стиранием блока/записью страницы сохранить содержимое затрагиваемых соседних страниц. В результате на сильно фрагментированном и заполненном накопителе одна элементарная операция записи может породить множество сопутствующих «паразитных» операция стирания/перезаписи (*write multiplication*), что приводит к значительному снижению производительности.

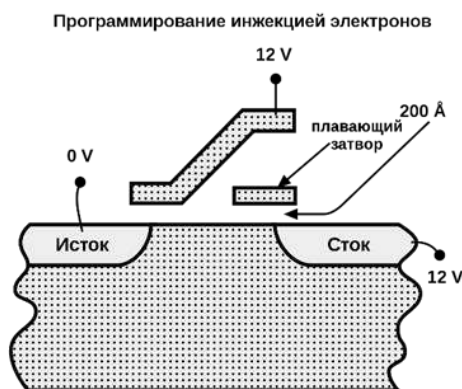


Рис.98 - Программирование ячейки флэш-памяти

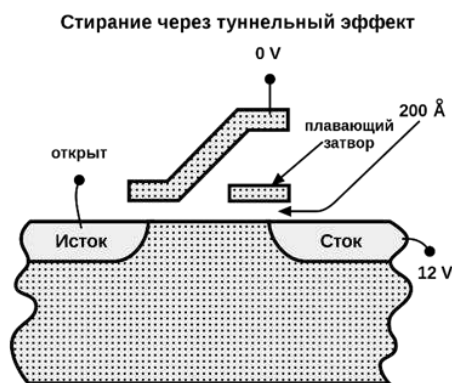


Рис.99 - Стирание ячейки флэш-памяти

Конструкция NAND — трёхмерный массив. В основе та же самая матрица, что и в NOR, но вместо одного транзистора в каждом пересечении устанавливается столбец из последовательно включенных ячеек. В такой конструкции получается много затворных цепей в одном пересечении. Плотность компоновки можно резко увеличить (ведь к одной ячейке в столбце подходит только один проводник затвора), однако алгоритм доступа к ячейкам для чтения и записи заметно усложняется. Также в каждой линии установлено два МОП-транзистора. Управляющий транзистор разрядной линии (*bit line select transistor*), расположенный между столбцом ячеек и разрядной линией. И управляющий транзистор заземления, расположенный перед «землёй» (*ground select transistor*).

Еще одной проблемой при использовании NAND является то, что количество циклов перезаписи ячеек памяти далеко не бесконечно. Если запись всегда будет производиться в одну и ту же страницу, она рано или поздно окажется повреждённой. И для того, чтобы обеспечить равномерный износ всех ячеек памяти, контроллер NAND-памяти ведёт учёт

количества циклов записи в каждый отдельный блок памяти. Если контроллер видит, что блок «плохой», то он может пропустить его и произвести запись в следующий блок. Благодаря этому срок службы носителей информации значительно увеличивается. Если мы хотим записать большой массив данных, то внутри микросхемы памяти все данные будут перемешаны по блокам (работает алгоритм записи в наименее изношенные блоки), а когда встает задача чтения этих данных, контроллер NAND-памяти упорядочит данные и выдаст их нам в первоизданном виде.

## Ленточные накопители

**Стример** (от *streamer*), также **ленточный накопитель** — запоминающее устройство на принципе магнитной записи на ленточном носителе, с последовательным доступом к данным, по принципу действия аналогичен бытовому магнитофону. Основное назначение: запись и воспроизведение информации, архивация и резервное копирование данных.

## Достоинства и недостатки

Технология хранения данных на магнитной ленте в ходе развития вычислительной техники претерпела значительные изменения. Использование современных стримеров имеет следующие отличительные черты.

Достоинства:

- большая ёмкость;
- низкая стоимость и широкие вариации условий хранения информационного носителя;
- стабильность работы;
- надёжность;
- низкое энергопотребление у ленточной библиотеки большого объёма.

Недостатки:

- низкая скорость произвольного доступа к данным из-за последовательного доступа (лента должна перемотаться к нужному месту);
- сравнительно высокая стоимость устройства записи (стримера).

## История

Магнитная лента была впервые использована для записи компьютерных данных в 1951 году в компании Eckert-Mauchly Computer Corporation на ЭВМ UNIVAC I. В качестве носителя использовалась тонкая полоска металла шириной 12,65 мм, состоящая из никелированной бронзы (называемая Vicalloy). Плотность записи была 128 символов на дюйм (198 микрометров/символ) на восемь дорожек.

В ЭВМ, выпускавшихся до момента появления и широкого распространения жёстких дисков, накопители на магнитной ленте (НМЛ), аналогичные стримерам, использовались как основной долговременный носитель информации. В дальнейшем в мейнфреймах НМЛ стали использоваться в системах иерархического управления носителями для хранения редко используемых данных. Некоторое время они достаточно широко применялись в качестве съёмного ЗУ при переносе большого количества информации.

В отличие от все предыдущих уровней в иерархии памяти, лента — устройство последовательное, соответственно, имеет очень большие времена доступа к случайным данным — секунды, даже десятки секунд и минуты — время перемотки кассеты от начала до конца. Но вот со скоростью линейного чтения/записи у современных стриммеров дела обстоят довольно неплохо — они как минимум в несколько раз превосходят по этому показателю производительность одиночного HDD диска.



На рынке в разное время существовало несколько разных ленточных технологий, на сегодня их осталось практически две: проприетарная технология IBM 3592<sup>1</sup>, с катриджами ёмкостью до 4Тбайт и технология LTO<sup>2</sup>. С 2007 года было прекращено развитие стандарта SDLT, в пользу LTO. DLT долгие годы конкурировал с LTO и некоторое время даже опережал LTO по максимально возможным объёмам записи на кассету. Максимальный объёмом SDLT-катриджа достигал величины 800 Гбайт.

В 2016-ом году прекратилось развитие ещё одной ленточной технологии, долгое время лидировавшей в индустрии по объёмам записи на катридж – StorageTek Sun/Oracle. Было объявлено о прекращении разработок в этом направлении, и ранее обещанные катриджи 10000Е объёмом 12-16 Тбайт, так и не были выпущены на рынок. Последняя версия этого стандарта – 10000D (сентябрь 2013 года) – имела катридж ёмкостью 8,5 Тбайт и обеспечивала линейную скорость чтения/записи 252 Мбайт/сек.

И LTO, и DLT, и StorageTek использовали одностороннюю схему катриджа (Рис.100) – внутри квадратного пластикового корпуса располагается единственная катушка, с намотанной на неё лентой. При вставке катриджа в стример механизм привода вытягивал ленту, цепляя крючком за кольцо на её конце, и заправлял её в приёмную бобину.



**Рис.100** - Разобранный картридж Ultrium LTO-2

---

<sup>1</sup> Стандарт **IBM 3592** – ориентирован не только на архивацию и резервное копирование, но и на произвольный доступ к данным, обеспечивает удовлетворение более жёстких требований по количеству перезаписей носителя. Также в IBM 3592 использован ряд решений для оптимизации производительности в старт-стопном режиме записи, такие, как глубокое кеширование данных и многоскоростное движение ленты (6 или 7 скоростей, в зависимости от модели стримера).

<sup>2</sup> **LTO** (Linear Tape-Open) — стандарт записи на магнитную ленту, которому удовлетворяет большинство современных стримеров. Практически используемым форматом записи данных в этом стандарте является Ultrium. Технология LTO была разработана в 1998 году совместно тремя крупнейшими производителями накопителей на магнитной ленте — IBM, Hewlett-Packard и Seagate Technology.



Объёмы LTO катриджей

|                                                              | LTO-1   | LTO-2  | LTO-3  | LTO-4  | LTO-5  | LTO-6     | LTO-7    | LTO-8    | Type M-8 | LTO-9       | LTO-10   | LTO-11 | LTO-12 |  |
|--------------------------------------------------------------|---------|--------|--------|--------|--------|-----------|----------|----------|----------|-------------|----------|--------|--------|--|
| Год появления                                                | 2000    | 2003   | 2005   | 2007   | 2010   | Дек.2012  | Дек.2015 | Дек.2017 |          | План        |          |        |        |  |
| Физическая ёмкость (raw)                                     | 100 GB  | 200 GB | 400 GB | 800 GB | 1.5 TB | 2.5 TB    | 6 TB     | 12 TB    | 9 TB     | 24 TB       | 48 TB    | 96 TB  | 192 TB |  |
| Сжатая ёмкость                                               | 200 GB  | 400 GB | 800 GB | 1.6 TB | 3 TB   | 6.25 TB   | 15 TB    | 30 TB    | 22.5 TB  | 60 TB       | 112.5 TB | 240 TB | 480 TB |  |
| Макс. скорость (МБ/с) (несжатая)                             | 20      | 40     | 80     | 120    | 140    | 160       | 300      | 360      | 300      | 708         | 1100     | План   |        |  |
| Макс. скорость (МБ/с) (сжатая)                               | 40      | 80     | 160    | 240    | 280    | 400       | 750      | 900      | 750      | 1770        | 2750     |        |        |  |
| Время записи полной ленты на макс. несжатой скорости (чч:мм) | 1:25    | 1:25   | 1:25   | 1:50   | 3:10   | 4:35      | 5:55     | 9:15     | 8:20     | План        |          |        |        |  |
| Поддержка сжатия                                             | Да, 2:1 |        |        |        |        | Да, 2.5:1 |          |          |          | План, 2.5:1 |          |        |        |  |
| Поддержка WORM                                               | Нет     |        | Да     |        |        |           |          |          | Нет      |             | План     |        |        |  |
| Поддержка шифрования                                         | Нет     |        |        | Да     |        |           |          |          |          |             |          |        |        |  |

Стриммеры могут использоваться как автономно, в небольших конфигурациях, так и в составе ленточных библиотек<sup>1</sup>. Ленточная библиотека начального уровня содержит один привод (стриммер) и 12-24 слотов для хранения катриджей. Крупные библиотеки, например, StorageTek 8500, содержат до 64-х приводов, которые могут работать одновременно, и до 100 тыс. слотов для хранения катриджей.

## Методы увеличения производительности CPU

Некоторые методы повышения производительности вычислительных систем были рассмотрены выше: увеличение тактовой частоты, увеличение количества одновременно исполняемых потоков, иерархическая организация памяти компьютера и др.

Применительно конкретно к центральному процессору (CPU) – центру «Вселенной ЭВМ» – это были описанные выше технологии кэширование, многоядерные архитектуры. Ниже будет приведено описание некоторых методов, реализованных на микроархитектурном уровне, максимально «близко» к вычислительному блоку компьютера.

## Параллелизм на уровне команд

**Параллелизм на уровне команд** (*Instruction-level parallelism — ILP*) является мерой того, какое множество операций в компьютерной программе может выполняться одновременно. Потенциальное совмещение выполнения команд называется «*параллелизмом на уровне команд*».

## Описание

Есть два подхода к выявлению параллелизма (*parallelism extraction*) на уровне команд:

<sup>1</sup> **Ленточная библиотека** представляет собой запоминающее устройство, которое содержит один или несколько ленточных накопителей, несколько слотов для хранения ленточных катриджей, считыватель штрих-кода для идентификации ленточных катриджей и автоматизированный способ загрузки лент (робот).

- аппаратные средства — выявлением параллелизма в потоке операций занимаются специальные схемы процессора при исполнении кода программ;
- программное обеспечение — выявлением параллелизма занимается компилятор, который формирует исполняемый код программы под специальный процессор (EPIC архитектура).

Уровень аппаратного обеспечения осуществляет динамический параллелизм, тогда как уровень программного обеспечения реализует статический параллелизм. Современные высокопроизводительные процессоры x86 (Intel Pentium 4, Intel Core и новее) работают на динамической последовательности параллельного выполнения (*Внеочередное исполнение*). Статический параллелизм применялся в процессорах Itanium (однако в Itanium 2 применяется гибридный подход).

Рассмотрим следующую программу:

1.  $e = a + b$
2.  $f = c + d$
3.  $m = e * f$

Операция 3 зависит от результатов операций 1 и 2, поэтому она не может быть вычислена, пока не будут завершены 1 и 2. Однако операции 1 и 2 не зависят от других операций, поэтому они могут быть вычислены одновременно. Если предположить, что каждая операция может быть завершена за одну единицу времени, то эти три инструкции могут быть завершены в общей сложности за две единицы времени, обеспечивая параллелизм равный 3/2.

Цель разработчиков компилятора и процессора заключается в выявлении параллелизма и получении от него максимального выигрыша. Обычные программы, как правило, написаны под последовательную модель исполнения, где команды выполняются одна за другой в порядке, установленном программистом. ILP<sup>1</sup> позволяет компилятору и/или процессору распараллеливать выполнение нескольких инструкций или даже изменять порядок их выполнения.

Сколько ILP присутствует в программе, сильно зависит от области её применения. В некоторых областях, таких как компьютерная графика и научные вычисления, число может быть очень большим. А такие задачи, как криптография, могут демонстрировать гораздо меньше параллелизма.

Аппаратные методы выявления и использования параллелизма:

- *Вычислительный конвейер*, где выполнение нескольких инструкций может частично перекрываться;
- *Суперскалярное выполнение операций*, в которой несколько функциональных блоков используется для выполнения нескольких команд одновременно;
- *Внеочередное исполнение*, где инструкции выполняются в любом порядке, который не нарушает зависимости данных. Обратите внимание, что этот метод не зависит ни от конвейера, ни от суперскалярного выполнения. Текущие реализации внеочередного исполнения динамически (т.е. во время выполнения программы и без помощи со стороны компилятора) получают ILP из обычных программ. Альтернативой является получение параллелизма во время компиляции и передача этой информации аппаратному обеспечению. Из-за сложности масштабирования техники внеочередного исполнения отрасль пересмотрела набор команд, которые явно кодируют несколько независимых операций в инструкции;

<sup>1</sup> ILP – Instruction-Level Parallelism

- *Переименование регистров* — метод, используемый для устранения ненужной сериализации (последовательного выполнения) операций, к которой приводит переиспользование регистров этими операциями, и используемый для внеочередного исполнения;
- *Спекулятивное исполнение*, которое позволяет выполнять инструкции целиком или частично до того, как станет ясно, нужно ли это выполнение. Часто используемой формой спекулятивного исполнения является спекулятивное исполнение потока управления, при котором инструкции (например, переход) за потоком управления выполняются до того, как определится та ветка программы, которую определяет инструкция перехода. Были предложены и используются несколько других форм спекулятивного исполнения, в том числе для спекулятивного исполнения, обусловленного предсказанием значения, предсказание зависимости по памяти и предсказание задержек кэша;
- *Предсказание переходов*, которое используется, чтобы избежать простаивания для разрешения управления зависимостями. Предсказатель переходов используется со спекулятивным исполнением.

## Вычислительный конвейер

**Конвейер** — способ организации вычислений, используемый в современных процессорах и контроллерах с целью повышения их производительности (увеличения числа инструкций, выполняемых в единицу времени — эксплуатация параллелизма на уровне инструкций), технология, используемая при разработке компьютеров и других цифровых электронных устройств.

На Рис.101 показан простой пятиуровневый конвейер в RISC-процессорах. Здесь:

- IF (*Instruction Fetch*) — получение инструкции,
- ID (*Instruction Decode*) — декодирование инструкции,
- EX (*Execute*) — выполнение,
- MEM (*Memory access*) — доступ к памяти,
- WB (*Register write back*) — запись в регистр.



Рис.101 - Простой пятиуровневый конвейер в RISC-процессорах

Вертикальная ось — последовательные независимые инструкции, горизонтальная — время. Зелёная колонка описывает состояние процессора в один момент времени, в ней самая ранняя, верхняя инструкция уже находится в состоянии записи в регистр, а самая последняя, нижняя инструкция — только в процессе выборки инструкции.

Свойство конвейеризированной архитектуры — чем длиннее конвейер, тем до больших частот его можно «разогнать». Но, обратная сторона медали — длинный конвейер не очень «любит» команды перехода, т.к. при переходе необходимо сбрасывать весь конвейер и заполнять его заново, и чем больше ступеней (стадий) в конвейере (чем он длиннее), тем больше на него влияют переходы (необходимо заполнять большее количество стадий) и *простой конвейера* займёт больше времени). Обработка прерывания в этом смысле тоже может рассматриваться, как команда перехода, сбрасывающая весь конвейер.

## Суперскалярное выполнение операций

**Суперскалярный процессор** (*superscalar processor*) — процессор, поддерживающий так называемый параллелизм на уровне инструкций (ILP) (то есть, процессор, способный выполнять несколько инструкций одновременно) за счёт включения в состав его вычислительного ядра нескольких одинаковых функциональных узлов (таких как АЛУ, FPU, умножитель (*integer multiplier*), сдвигающее устройство (*integer shifter*) и другие устройства). Планирование исполнения потока инструкций осуществляется динамически вычислительным ядром (а не статически компилятором).

Способы увеличения производительности, которые могут использоваться совместно:

- Использование конвейера (*pipelining*)
- увеличение количества функциональных узлов процессора (*суперскалярность*)
- увеличение количества ядер (*многоядерность*)
- увеличение количества процессоров (*многопроцессорность*)

При использовании конвейера количество узлов остаётся прежним; увеличение производительности достигается за счёт одновременной работы узлов, ответственных за разные стадии обработки инструкций одного потока. При использовании *суперскалярности* увеличение производительности достигается за счёт одновременной работы большего количества одинаковых узлов, независимо обрабатывающих инструкции одного потока (в том числе, и большего количества конвейеров). При использовании нескольких ядер каждое ядро выполняет инструкции отдельного потока, причем каждое из них может быть суперскалярным и/или конвейерным. При использовании нескольких процессоров каждый процессор может быть многоядерным.

В суперскалярном процессоре инструкция извлекается из потока инструкций (находящегося в памяти), определяется наличие или отсутствие *зависимости инструкции по данным* от других инструкций, затем инструкция выполняется. Одновременно, в течение одного такта, может выполняться несколько независимых инструкций.

По *классификации Флинна* одноядерные суперскалярные процессоры относят к группе процессоров SISD (*single instruction stream, single data stream* — один поток инструкций, один поток данных). Подобные процессоры, поддерживающие инструкции для работы с короткими векторами, могут быть отнесены к группе SIMD (*single instruction stream, multiple data streams* — один поток инструкций, несколько потоков данных). Многоядерные суперскалярные процессоры относят к группе MIMD (*multiple instruction streams, multiple data streams* — несколько потоков инструкций, несколько потоков данных).

## Внеочередное исполнение

**Внеочередное исполнение** (*out-of-order execution*) машинных инструкций — исполнение машинных инструкций не в порядке их следования в машинном коде (как было при выполнении инструкций по порядку (*in-order execution*)), а в порядке готовности к выполнению. Реализуется с целью повышения производительности вычислительных устройств.

## Описание

При выполнении инструкций по порядку (*in-order*) осуществляются следующие действия:

1. считывание инструкции из ОЗУ (кэша);
2. остановка процессора в ожидании для загрузки операндов, указанных в инструкции;
3. передача инструкции на выполнение соответствующему исполнительному модулю;
4. выполнение инструкции соответствующим модулем;

5. запись результата выполнения инструкции модулем в регистровую ячейку (или ОЗУ).

При выполнении инструкций не по порядку (*out-of-order*) осуществляются следующие действия:

1. считывание инструкции из ОЗУ (кэша);
2. помещение инструкции в очередь инструкций I (называемую *instruction queue*, *instruction buffer* или *reservation stations*);
3. ожидание, пока операнды любой инструкции, находящейся в очереди I, станут доступны;
4. передача инструкции, операнды которой доступны, на выполнение соответствующему исполнительному модулю;
5. выполнение инструкции соответствующим модулем;
6. запись результата выполнения инструкции модулем в очередь II;
7. извлечение из очереди II результатов выполнения тех инструкций, перед которыми в очереди I не осталось невыполненных инструкций, и запись результатов в регистровый файл (или ОЗУ); удаление таких инструкций из очереди I.

Особенности внеочередного исполнения:

- раньше других может быть выполнена инструкция, операнды которой будут готовы раньше операндов других инструкций;
- инструкция, поставленная в очередь I позже, может быть выполнена раньше;
- время простоя, вызванное ожиданием готовности операндов инструкций, уменьшается (по сравнению с процессором, выполняющим инструкции по порядку) за счёт ожидания готовности операндов *нескольких* инструкций;
- за счёт использования очереди II запись результатов в регистровый файл осуществляется в порядке следования инструкций в машинном коде (создаётся видимость выполнения инструкций по порядку, *re-order*).

Основная идея внеочередного выполнения заключается в возможности избежать простоя процессора в тех случаях, когда данные, необходимые для выполнения очередной инструкции, недоступны. Как можно видеть выше, процессор избегает простоев, которые возникают в пункте 2 для очередного выполнения инструкций в случае, когда инструкция не может быть выполнена из-за недоступности её операндов.

Процессор с внеочередным выполнением кода выполняет те инструкции, операнды которых готовы, вне зависимости от порядка следования инструкций в коде программы. Инструкции выполняются не в порядке их следования в коде программы, а в порядке доступности данных в регистрах процессора. С помощью очереди II процессор записывает результаты выполнения инструкций так, чтобы создавалась видимость нормального, очередного выполнения.

Внеочередное выполнение тем эффективнее, чем длиннее конвейер процессора, и чем больше разница между скоростью работы памяти (оперативной или кэшем) и скоростью работы процессора. В современных компьютерах процессоры работают намного быстрее памяти, поэтому во время ожидания поступления данных процессор может выполнить множество инструкций, операнды которых доступны.

### **Переименование регистров**

При реализации п.4 при внеочередном выполнении может оказаться так, что есть свободный исполнительный модуль, но нет свободного регистра (регистров) для хранения

операнда. Для обхода этой проблемы была придумана технология *переименования регистров*.

**Переименование регистров** (*register renaming*) — метод ослабления взаимозависимостей команд, применяемый в процессорах с внеочередным исполнением команд. Один из методов, применяемых в вычислительных конвейерах для реализации параллелизма на уровне команд.

В том случае, если в соответствии с двумя или более командами необходимо осуществить запись данных в один регистр, их корректное внеочередное исполнение<sup>(выше)</sup> становится невозможным (более поздняя команда не может быть обработана до завершения более ранней) даже в том случае, если при этом нет зависимости по данным. Такие взаимозависимости часто называют *ложными* (в случае *истинной* зависимости существует зависимость и по данным).

Так как количество архитектурных регистров обычно ограничено (например, стандартная архитектура **x86** имеет только восемь регистров общего назначения), вероятность возникновения ложных взаимозависимостей достаточно велика, что может привести к снижению общей производительности процессора.

Переименование регистров представляет собой преобразование программных ссылок на архитектурные регистры в ссылки на физические регистры и позволяет ослабить влияние ложных взаимозависимостей за счёт использования большого количества физических регистров вместо ограниченного количества архитектурных (так, например, x86-совместимые процессоры архитектуры Intel P6 содержат 40 физических регистров, т.е. физически регистровый файл в процессоре присутствует в пяти экземплярах). При этом процессор отслеживает, состояние каких физических регистров соответствует состоянию архитектурных, а выдача результатов осуществляется в порядке, который предусмотрен программой.

Ниже (Рис.102) приведён скриншот из видеоурока, иллюстрирующего технику переименования регистров. Ссылка на Youtube:

[Register Renaming Quiz Solution](#)

## REGISTER RENAMING QUIZ SOLUTION

| FETCHED        | RENAMED        | RAT |    |
|----------------|----------------|-----|----|
| MUL R2, R2, R2 | MUL P7, P2, P2 | R1  | P8 |
| ADD R1, R1, R2 | ADD P8, P1, P7 | R2  | P7 |
| MUL R2, R4, R4 | MUL , 1        | R3  | P3 |
| ADD R3, R3, R2 |                | R4  | P9 |
| MUL R2, R6, R6 |                | R5  | P5 |
| ADD R5, R5, R2 |                | R6  | P6 |

Рис.102 - Переименование регистров.

## **Спекулятивное исполнение**

**Спекулятивное выполнение**<sup>1</sup> — это метод оптимизации, когда компьютерная система выполняет некоторую задачу, результаты которой могут не потребоваться. Работа выполняется до того, как станет известно, действительно ли она необходима (заранее, «на всякий случай»), чтобы предотвратить задержку, которая могла бы возникнуть при выполнении работы после того, как стало известно, что она всё-таки необходима. Если выясняется, что в конце концов работа не нужна, большинство изменений, внесенных этой работой, отменяются, а результаты игнорируются.

Цель состоит в том, чтобы обеспечить больше параллелизма при наличии дополнительных ресурсов. Этот подход используется в различных областях, включая прогнозирование ветвлений в конвейерных процессорах, прогнозирование стоимости для использования значения локальности, предварительную выборку (prefetching) из памяти и файлов, и при оптимистическом управлении параллелизмом в СУБД. Спекулятивная многопоточность является частным случаем спекулятивного исполнения.

## **Предсказание переходов**

**Модуль предсказания переходов (прогнозирования ветвлений)** (*branch prediction unit*) — устройство, входящее в состав микропроцессоров, имеющих конвейерную архитектуру, предсказывающее, будет ли выполнен условный переход в исполняемой программе. Предсказание ветвлений позволяет сократить время простоя конвейера за счёт предварительной загрузки и исполнения инструкций, которые должны выполняться после выполнения инструкции условного перехода. Прогнозирование ветвлений играет критическую роль, так как в большинстве случаев (точность предсказания переходов в современных процессорах превышает 90%) позволяет оптимально использовать вычислительные ресурсы процессора.

Без предсказания переходов конвейер должен дожидаться выполнения инструкции условного перехода, чтобы произвести следующую выборку. Предсказатель переходов позволяет избежать траты времени, пытаясь выявить ветвление. Ветвление выбирается по предыдущим результатам проверки условия. Предполагаемое ветвление затем загружается и частично выполняется. Если затем обнаруживается, что предсказание было выполнено неверно, отменяются результаты неверного ветвления, и в конвейер загружается правильное ответвление, создавая тем самым задержку. Величина задержки зависит от длины конвейера. Для процессора Intel Core i7 глубина конвейера составляет 14 стадий.

Следует отличать «предсказание переходов» от «предсказания адреса перехода». Цель предсказания адреса перехода состоит в выборе адреса условного или безусловного перехода до декодирования и выполнения инструкции перехода.

Существует два основных метода предсказания переходов:

- статический;
- динамический.

Статические методы предсказания ветвлений являются наиболее простыми. Суть этих методов состоит в том, что различные типы переходов:

- либо выполняются всегда;
- либо не выполняются никогда.

---

<sup>1</sup> Серьёзные ошибки в реализации спекулятивного выполнения в некоторых процессорах привели к выявлению в начале 2018-го года совершенно нового класса уязвимостей: Spectre, Meltdown, SPOILER, Foreshadow, Microarchitectural Data Sampling. Несмотря на то, что эти уязвимости стали широко известны сравнительно недавно, фундаментальные ошибки, «благодаря» которым они стали возможны, были допущены много лет назад, впервые это случилось ещё в архитектуре Pentium Pro, 1995 год.

В современных процессорах статические методы используются лишь тогда, когда использование динамических методов невозможно.

**Пример** тривиального статического предсказания, реализованного в первых процессорах архитектур **SPARC** и **MIPS**.

- В конвейер всегда загружаются инструкции, расположенные после инструкции условного перехода. `if(условие) {<предсказанный переход>} else {...}`

В данном случае принимается предположение о том, что условный переход никогда не выполнится.

**Пример** статического предсказания, реализованного в современных процессорах.

- Если инструкция перехода выполняет переход назад (на младшие адреса), в конвейер загружаются инструкции, расположенные по адресу перехода. `do {<предсказанный переход>} while(условие); ...`
- Если инструкция перехода выполняет переход вперёд (на старшие адреса), в конвейер загружаются инструкции, расположенные после инструкции перехода. `if(условие) {<предсказанный переход>} else {...}`

В данном случае принимается предположение о том, что любой переход назад будет выполнен, а любой переход вперёд выполнен не будет. Метод используется в качестве «подстраховки».

Динамические методы, широко используемые в современных процессорах, подразумевают анализ истории ветвлений.

### **Счётчик с насыщением или бимодальный счётчик**

Анализируется таблица истории переходов. Таблица содержит:

- младшие значимые биты адреса инструкции;
- соответствующую им вероятность условного перехода:
  - «скорее всего, будет выполнен»;
  - «возможно, будет выполнен»;
  - «возможно, не будет выполнен»;
  - «скорее всего, не будет выполнен».

Таблица обновляется после каждого перехода. Алгоритм изменяет выбор ветвления, если результат условия отклонился два раза от предыдущих результатов. Использование младших битов адреса инструкции позволяет производить предсказания нескольких инструкций до их декодирования.

### **Адаптивный двухуровневый предсказатель**

Для первого уровня выполняются история последних  $k$  ветвлений, для второго уровня  $k$  указывает на таблицу шаблонов.

### **Локальное предсказание перехода**

Каждый условный переход в области имеет собственную историю переходов. Шаблоны переходов могут быть общими или отдельными.



### **Глобальное предсказание перехода**

Глобальное предсказание переходов не хранит истории отдельно для каждого перехода, а использует общую историю. Любые закономерности в переходах сказываются на этой истории, но историю могут загрязнять нерелевантные записи.

### **Гибридный предсказатель**

Гибридный предсказатель может выбирать результаты наиболее успешных предсказателей на основе истории либо использовать мажоритарную функцию нечётного количества предсказателей.

### **Предсказатель для цикла**

Предсказатель для цикла может использовать счетчик цикла для отсчета количества переходов в начало цикла. Этот предсказатель может использоваться в гибридном предсказателе.

### **Предсказание косвенных переходов**

Косвенный переход может иметь больше двух ветвлений. Новейшие процессоры имеют возможность выбора более двух условий, используя двухуровневый адаптивный предсказатель. Процессоры без поддержки предсказаний косвенных переходов могут использовать статическое предсказание или брать предыдущее значение.

## ГЛАВА 5. ОДНОКРИСТАЛЛЬНЫЕ МИКРОЭВМ MCS-48/MCS-51

МикроЭВМ (микроконтроллер) – устройство, содержащее на одном кристалле все элементы архитектуры ЭВМ: центральный процессор, память, порты ввода/вывода, возможность работать по прерываниям. Основное предназначение вычислительного устройства такого класса – встраиваемые системы<sup>1</sup>. Это не универсальные компьютеры, это специализированная система управления, выполняющая обычно одну задачу по управлению одним конкретным устройством или процессом: телевизором, принтером, клавиатурой<sup>2</sup> вашего десктопного универсального компьютера, soho-маршрутизатором и т.п. Поэтому часто в отношении таких систем используется термин микроконтроллер<sup>3</sup>.

### Семейство MCS-48

#### Архитектура микроконтроллера MCS-48

MCS-48 – это первое семейство восьмиразрядных однокристальных микроконтроллеров, которое было разработано корпорацией Intel в 1976 и производилось до 1990-х годов, имеет встроенную память, таймер и интерфейсы для связи с внешними устройствами.

Семейство MCS-48 основано на модифицированной гарвардской архитектуре. Память программ реализуется внутренним или внешним ПЗУ, память данных — внутреннее ОЗУ с возможностью подключения внешнего. Порты ввода/вывода вынесены в отдельное адресное пространство.

В рамках семейства MCS-48 выпускалось несколько версий микроконтроллеров, отличающихся объёмом ОЗУ, наличием/отсутствием ПЗУ на кристалле, его объёмом, тактовыми частотами, способами записи программ в ПЗУ. Например, более дешёвый i8035 отличался от i8048 отсутствием встроенного ПЗУ на кристалле.

В СССР выпускались клоны интеловских i8048 микроконтроллеров (один из крупнейших производителей этого контроллера, кстати, находился в Новосибирске), под названием 1816BE48. Это обозначение также будет использоваться в дальнейшем.

1816BE48 изготавливался по n-МОП технологии и упаковывался в 40-выводной DIP-корпус. Сорока выводов не хватало, чтобы вывести каждый сигнал индивидуально, поэтому часть выводов выполняли более, чем одну функцию.

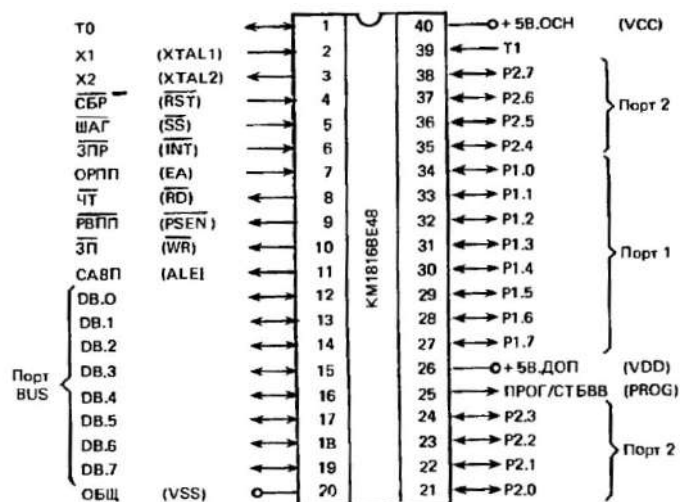


Рис.103 - Цоколёвка корпуса 1816BE48

<sup>1</sup> Встраиваемая система (встроенная система, *embedded system*) — специализированная микропроцессорная система управления, контроля и мониторинга, концепция разработки которой заключается в том, что такая система будет работать, будучи встроенной непосредственно в устройство, которым она управляет.

<sup>2</sup> Микроконтроллер i8048 использовался в качестве контроллера клавиатуры первых IBM PC.

<sup>3</sup> Контроллер (*controller* — регулятор, управляющее устройство).

Основу микроконтроллера (Рис.104) составляет двунаправленная восьмиразрядная системная шина, связывающая между собой все основные компоненты схемы:

- блок синхронизации и управления;
- арифметико-логическое устройство (АЛУ);
- блок таймеров/счётчиков;
- блок прерываний;
- программный счётчик;
- резидентную память данных (РПД) и память программ (РПП).

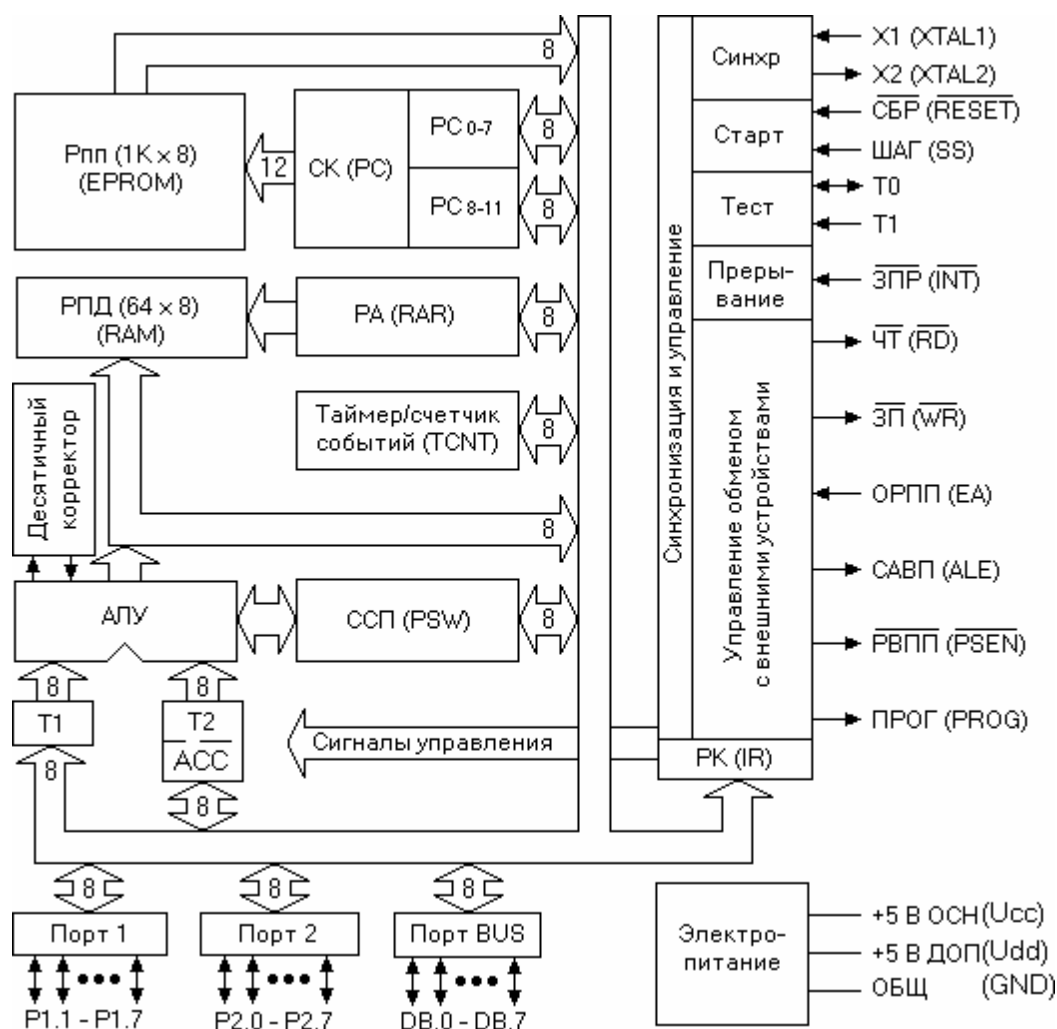


Рис.104 - Структурная схема микроконтроллера 1816BE48

Сигнал SS (ШАГ) совместно с сигналом ALE (САВП) позволяет при отладке выполнять программу с остановом после исполнения очередной команды. Выводы T1 и T0 опрашиваются командами условного перехода и могут использоваться для подсчёта внешних событий. Сигнал запроса прерывания ЗПР от внешнего источника вызывает подпрограмму обслуживания прерывания. Высокий уровень на входе EA (ОРПП) заставляет МК выполнять выборку команд только из внешней памяти программ, что позволяет тестировать прикладную программу при её отладке. Этот вывод также используется для подачи напряжения программирования +25В. Контроллер имеет три параллельных восьмиразрядных порта ввода/вывода, из которых порт BUS может использоваться в качестве внешней шины данных центрального процессора контроллера.

## Блок управления и синхронизации

Совместную работу блоков микроконтроллера во всех режимах работы координирует блок управления и синхронизации. В его состав входят: устройство синхронизации, логика ввода-вывода, регистр команд, дешифратор команд, логика управления ЭВМ.

Опорную частоту синхронизации определяет либо кварцевый резонатор, подключаемый к выводам X1 и X2, либо LC-цепь (Рис.105а и Рис.105б, соответственно). X1 является входом, а X2 – выходом генератора, способного работать в диапазоне частот от 1 до 6 МГц. На вход X1 может подаваться сигнал от источника внешней синхронизации Рис.105в. В состав генератора МК входят два счётчика с модулями пересчёта 3 и 5. Первый используется для формирования сигнала системной синхронизации (0,5 мкс). Этот же сигнал поступает на счётчик машинных циклов, на выходе которого через каждые пять сигналов синхронизации формируется сигнал ALE (2,5 мкс), идентифицирующий машинный цикл и использующийся в расширенных МК-системах для стробирования адреса внешней памяти.

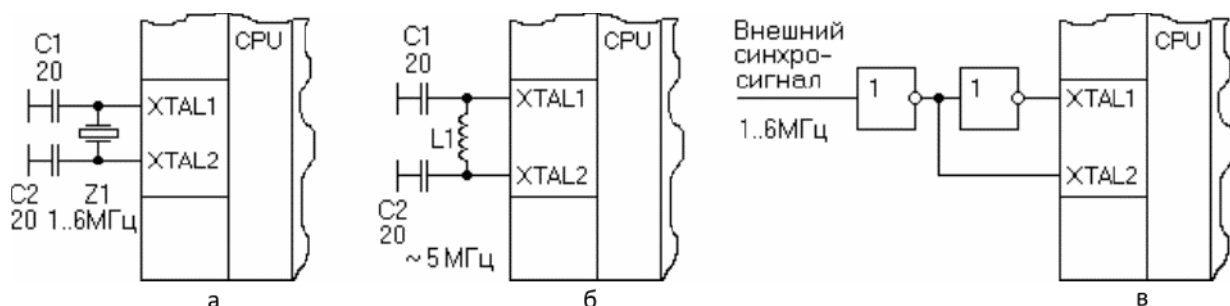


Рис.105 - Варианты схем синхронизации MCS-48

Сброс микроконтроллера в обслуживаемых МК-системах осуществляется с помощью кнопки «Сброс», которая заземляет вход RESET (Рис.106б). В необслуживаемых МК-системах, ко входу RESET подключается конденсатор ёмкостью 1 мкФ, что обеспечивает подачу сигнала близкого к потенциалу земли, длительностью не менее 50 мкс после установившегося номинального значения напряжения питания (Рис.106а). Сигнал RESET производит следующие действия: сбрасывает счётчик команд (PC) и указатель стека (SP); устанавливает порт BUS в высокоимпедансное состояние, а порты P1 и P2 – в режим ввода (FF в выходном буфере); выбирает банк регистров 0 и банк памяти 0; запрещает прерывания; останавливает таймер и выдачу синхросигнала на вывод T0; сбрасывает флаг переполнения таймера TF и флаги пользователя F0 и F1.

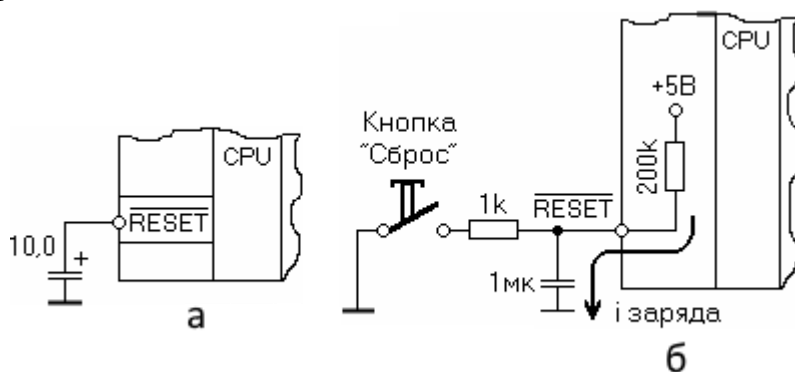


Рис.106 - Схемы начальной установки MCS-48

Для записи и хранения восьмиразрядного кода операции выполняемой команды предназначен регистр команд. Код операции, с помощью дешифратора команд и логики управления, преобразуется в микропрограмму выполнения команды, считанной из ПЗУ.

## Арифметико-логическое устройство

Восьмиразрядное арифметико-логическое устройство обеспечивает выполнение арифметических и логических операций и состоит из регистров временного хранения T1 и T2, аккумулятора (ACC), ПЗУ констант, арифметико-логического устройства и регистра слова-состояния программы (PSW – Program Status Word).

Программно недоступные восьмиразрядные регистры временного хранения (T1 и T2) используются для временного хранения второго операнда при выполнении двухоперандных команд. Результат выполнения арифметико-логических операций или операций сдвига сохраняется в восьмиразрядном аккумуляторе. ПЗУ констант обеспечивает генерацию корректирующего кода при двоично-десятичном представлении данных, кода маски при битовых операциях и кода констант.

Арифметико-логическое устройство представляет собой схему комбинационного типа с последовательным переносом, предназначенную для выполнения операций сложения и логических операций И, ИЛИ, исключающего ИЛИ, инкремента, декремента, инверсии, циклического сдвига вправо и влево, десятичной коррекции содержимого аккумулятора и обмена тетрад в байте.

При выполнении операций обработки данных в АЛУ, формируются флаги:

- переноса (C);
- вспомогательного переноса (AC);
- нулевого содержимого аккумулятора (Z);
- наличия единицы в заданном бите аккумулятора.

Необходимо отметить, что в регистре слова состояния программы PSW (Рис.107) сохраняются только два из этих флагов (C, AC). Остальные флаги формируются комбинационной схемой и не фиксируются, но логика условных переходов позволяет выполнять команды передачи управления (JZ, JNZ, JB0...JB7) без их фиксации триггерами. Кроме этого имеются условные переходы по «флагам пользователя» F0 и F1, по флагу переполнения таймера TF, а также по сигналам на тестируемых входах T0 и T1. Программист также имеет доступ к флагам рабочего банка регистров BS и выбранного банка внешней памяти программ MB. Кроме того, после окончания каждого машинного цикла опрашивается флаг разрешения/запрета прерывания.

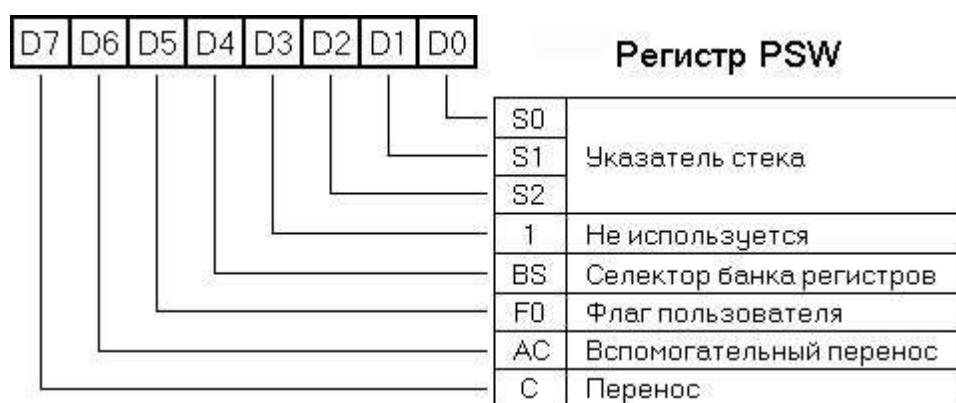


Рис.107 - Формат слова состояния (PSW)

Счётчик команд (Program Counter) формирует текущий 12-разрядный адрес памяти программ. В состав счётчика команд входят регистр PC и схема инкремента одиннадцати разрядов адреса (старший разряд PC не инкрементируется).

## Память микроконтроллера

В семействе микроконтроллеров MCS-48 подсистема памяти реализована по принципам гарвардской архитектуры, что предполагает её физическое и логическое разделение на память программ и память данных с отдельным адресным пространством. В этих микроконтроллерах имеется три адресных пространства: память программ, внешняя память данных (ВПД) и внутренняя (резидентная) память данных (РПД).

Схема подключения внешних микросхем памяти к микроконтроллерам семейства MCS-48 показана на Рис.108. Регистр адреса D2 на этой схеме предназначен для запоминания младших восьми бит адреса, передаваемых через шину данных/памяти. Старшие четыре бита адреса передаются через шину адреса, совмещенную с портом P2. Выдача адреса сопровождается синхроимпульсом ALE, который позволяет запомнить младший байт адреса в регистре D2.

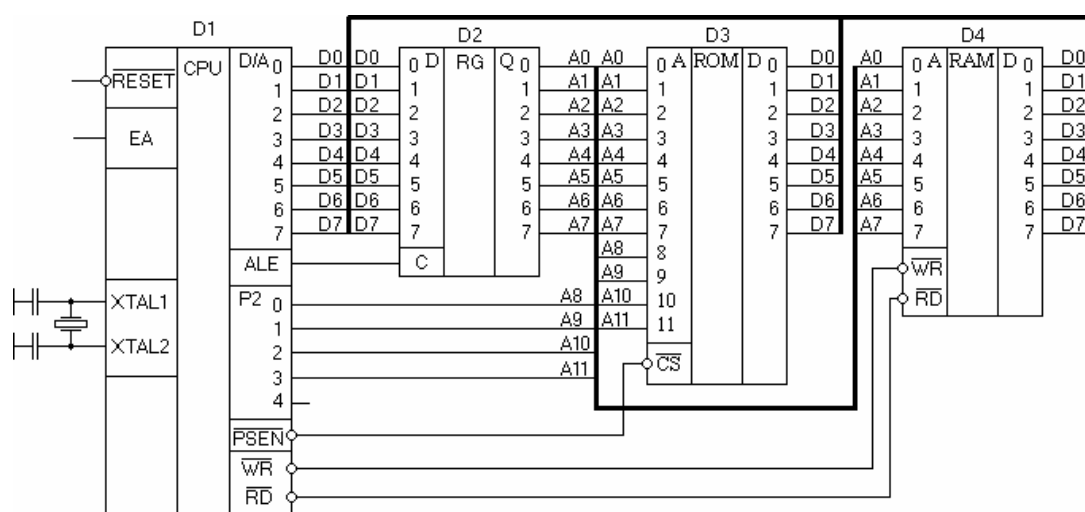


Рис.108 - Подключение внешней памяти к микроконтроллерам MCS-48

Для обращения к памяти программ D3 и к памяти данных D4 используется совмещённая шина данных/адреса BUS, а её текущее назначение определяется управляющими сигналами. При чтении памяти программ используется сигнал PSEN, а при чтении памяти данных – сигнал RD. Запись информации в память данных осуществляется по сигналу WR.

## Память программ

Память программ, размещённая на кристалле микроконтроллера, называется резидентной (РПП) и представляет собой постоянное запоминающее устройство. В семейство MCS-48 входят микроконтроллеры с различным объёмом резидентной памяти программ, а у 1816BE35 РПП отсутствует, поэтому они могут работать только с внешней микросхемой ПЗУ ёмкостью до 4 Кбайт. Максимальное адресное пространство памяти программ составляет 4 Кбайта, поэтому счётчик команд двенадцатиразрядный ( $4 \text{ Кбайт} = 4096 = 2^{12} \text{ байт}$ ). Адреса РПП находятся в диапазоне 000h..3FFh. После сброса микроконтроллера выполнение программы начинается с нулевого адреса, соответствующего внутреннему ПЗУ микроконтроллера.

Микроконтроллеры семейства MCS-48 имеют вывод EA, с помощью которого можно запретить работу внутренней памяти, подав на этот вход логический “0”. При этом, начиная с нулевого адреса, все обращения происходят к внешней памяти программ. Поэтому доступ к внешней памяти программ осуществляется в двух случаях:

1. при подаче сигнала EA=0 независимо от адреса обращения,
2. в случае, если счётчик команд (PC) содержит число большее, чем 3FFh.

Необходимо отметить, что при выполнении программы инкрементируются только младшие 11 бит счётчика команд. Поэтому программа из предельного адреса 7FFh (если только по этому адресу не расположена команда передачи управления) перейдёт по адресу 0000h. Состояние старшего бита счётчика команд может быть изменено специальными командами (SEL MB0 и SEL MB1) совместно с командами безусловного перехода JMP и вызова подпрограмм CALL. При этом следует учитывать, что команды SEL MB0 и SEL MB1 изменяют только флаг MB и не воздействуют непосредственно на старший бит счётчика команд. Изменение этого бита осуществляется командами JMP и CALL, которые копируют текущее содержимое MB в старший бит счётчика команд. Поэтому для перехода из нулевого банка в первый требуется выполнить последовательность команд SEL MB1, JMP adr1.

При выполнении переходов между банками по командам SEL MB, необходимо следить за тем, чтобы подпрограммы, взаимно вызывающие друг друга, располагались в пределах одного банка памяти. Иначе возникает необходимость модификации флага MB в вызываемой подпрограмме и восстановления его при возврате в вызывавшую подпрограмму. Но если вызов такой подпрограммы носит условный характер, то проблема восстановления флага может оказаться неразрешимой.

Неполная (11-битная) инкрементация счётчика команд приводит к тому, что память программ разделяется на два банка ёмкостью по 2 Кбайта. Кроме того, в командах условного перехода задаётся 8-битный адрес, что обуславливает разбиение памяти программ на 16 страниц по 256 байт. В случае, когда в программе необходимо иметь много условных переходов, из-за небольшого размера страницы возникает проблема размещения соответствующих программных модулей на двух страницах. Для решения этой проблемы используют комбинации команд условного перехода с 8-разрядным адресом и команды безусловного перехода с 11-разрядным адресом. Адресное пространство памяти программ при этом распределено, как показано на Рис.109.

В резидентной памяти программ имеется три специализированных адреса:

- адрес 0, которому передаётся управление по сигналу RESET; по этому адресу должна находиться команда безусловного перехода к началу программы;
- адрес 3, по которому передаётся управление по сигналу внешнего прерывания;
- адрес 7, по которому передаётся управление по прерыванию от таймера/счётчика.

При обработке запросов прерываний в МК48 старший бит счётчика команд принудительно устанавливается в 0. Это приводит к необходимости располагать подпрограмму обслуживания прерывания и все вызываемые ею подпрограммы в нулевом банке памяти.

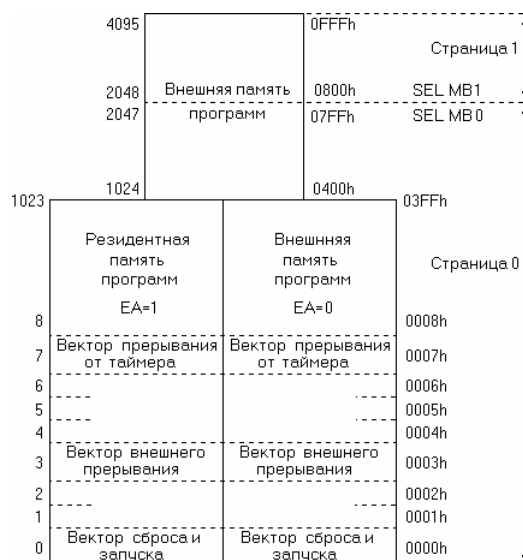


Рис.109 - Карта адресов памяти программ 1816BE48

Семейство MCS-48 имеет несколько модификаций, отличающихся объёмом резидентной памяти программ. При использовании портов P1 и P2 в качестве дополнительных адресных расширителей объём внешней памяти программ может быть увеличен до 16 Мбайт.

## Память данных

Резидентная память данных (РПД) ёмкостью 64 байта имеет в своём составе два банка рабочих регистров R0...R7, причём программист может выбрать один из них в качестве активного. Для размещения регистров используются ячейки памяти 0-7 и 24-31 (Рис.110), поэтому символу R0 в программе может соответствовать как ячейка памяти 0, так и ячейка с адресом 24. Такая организация позволяет сократить время обработки прерываний, так как вместо традиционного сохранения содержимого регистров в стеке можно переключить активный банк регистров. Выбор активного банка регистров выполняется командой SEL RB.

Любая ячейка РПД доступна по командам с косвенной адресацией через регистры R0, R1. При этом используются символы @R0 и @R1. Однако при адресации ячеек 0-7 и 24-31 предпочтительно использовать символы R0...R7, соответствующие прямой регистравой адресации.

В РПД размещается также стек объёмом 16 байт, имеющий 8 уровней по 2 байта (область памяти с адресами 8...23). В качестве указателя стека используются три разряда слова состояния программы PSW. В стеке сохраняется адрес возврата из подпрограммы (12 разрядов) и старшая тетрада PSW (4 разряда).

При уровне вложенности подпрограмм меньше восьми, незадействованные в стеке ячейки могут использоваться как ячейки РПД. При переполнении стека трёхбитный указатель стека переходит из состояния 7 в состояние 0.

Семейство MCS-48 не имеет команд загрузки байта в стек или его извлечения из стека, обычно используемые при обработке прерываний. Поэтому программист должен следить за тем, чтобы вложенные подпрограммы не использовали одни и те же рабочие регистры.

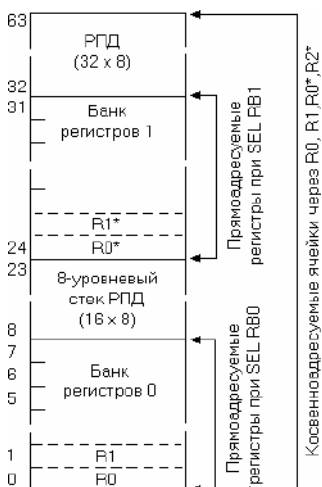


Рис.110 - Карта адресов памяти данных

Практически все команды с обращением к РПД оперируют с одним байтом. Однако по командам вызова подпрограмм и возврата осуществляется доступ к двухбайтным словам. В памяти данных слова хранятся так,



что старший байт располагается в ячейке с большим адресом. Отметим, что в памяти программ порядок расположения байтов по старшинству при хранении двухбайтных слов обратный.

В МК-системах с внешней памятью данных адресация осуществляется также через регистры R0 и R1 (символы @R0 и @R1), что обеспечивает доступ к памяти объемом 256 байт.

## Организация ввода/вывода

Для организации ввода/вывода информации микроконтроллеры семейства MCS-48 имеют 27 линий, в том числе три параллельных порта (BUS, P1 и P2) по восемь линий в каждом, а также три линии, которые могут использоваться для изменения хода программы по командам условного перехода и сигналу прерывания:

- INT – для ввода в МК сигнала запроса прерывания от внешнего источника либо для ввода тестируемого сигнала;
- T0 – для ввода тестируемого сигнала от двоичного датчика (по команде ENT0 CLK по этой линии выдается сигнал синхронизации);
- T1 – для ввода тестируемого сигнала или ввода импульсов для счётчика событий, который запускается по команде STRT CNT.

**Порт BUS** является двунаправленным и имеет буферные каскады с тремя состояниями выходов. Он может использоваться либо в качестве шины данных/адреса при подключении внешней памяти, либо как параллельный порт ввода/вывода.

При подключении внешней памяти данных на линиях порта BUS сначала выставляется 8-разрядный адрес, сопровождаемый сигналом ALE. Затем по тем же линиям порта BUS производится передача байта данных, стробируемых сигналами RD или WR (при чтении или записи соответственно). Поэтому на время передачи данных адрес должен фиксироваться во внешнем регистре (см. Рис.108, регистр D2).

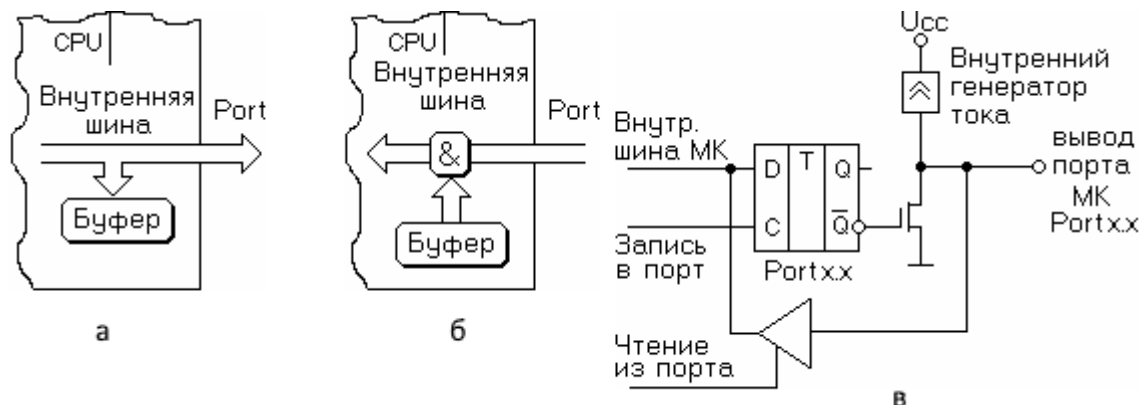
При подключении внешней памяти программ требуется формировать 12-разрядный адрес, младшие 8 разрядов которого передаются аналогично (по линиям порта BUS в сопровождении сигнала ALE) и также запоминаются во внешнем регистре. Старшие 4 разряда передаются через порт P2 (линии P2.0...P2.3) и в течение машинного цикла не изменяются, поэтому для старших разрядов адреса буферизация не требуется. Данные из внешней памяти программ передаются в микроконтроллер в сопровождении сигнала PSEN (РВПП); очевидно, что запись в память программ невозможна (по определению).

При работе порта BUS в режиме параллельного интерфейса вывод/вывод выполняется командами OUTL и INS. Кроме того, вводимые и выводимые через порт BUS данные можно модифицировать с помощью команд ORL BUS, #d и ANL BUS, #d, что позволяет выделять и обрабатывать в байте отдельный бит или группу бит. Такая операция называется маскированием.

Заметим, что обмен данными через порт BUS осуществляется командами пересылки MOVX, а также ввода/вывода OUTL и INS. При этом в случае записи во внешнюю память данных командой MOVX @Ri, A или выводе данных через порт командой OUTL BUS, A соответствующий байт фиксируется в буферном регистре и сохраняется на выводах микроконтроллера до выполнения следующей команды такого типа. Однако при чтении данных командой MOVX A, @Ri содержимое буфера уничтожается, в то время, как команда ввода INS не изменяет содержимое буферного регистра. Поэтому при работе с портом BUS не рекомендуется попеременное использование команд пересылки данных и команд ввода/вывода.

В отсутствие передач данных линии порта BUS отключаются от нагрузки, переходя в высокоимпедансное состояние.

**Порты ввода/вывода P1 и P2** являются *квазидвунаправленными*. Это означает, что вывод информации осуществляется аналогично порту BUS, но при вводе производится операция поразрядного логического «И» между входными сигналами и содержимым буфера (Рис.111). Поэтому для ввода данных без изменений перед операцией чтения IN необходимо записать во все разряды буфера логические единицы. Заметим, что по сигналу RESET в буферные регистры портов P1 и P2 автоматически записывается FFh.



**Рис.111 - Квазидвунаправленный порт: а - в режиме вывода; б - в режиме ввода; в - для одного разряда**

Для ввода/вывода информации через порты P1 и P2 используются команды IN и OUTL:

- IN A, P1 ; ввести данные из порта P1 в аккумулятор
- OUTL P2, A ; вывести данные из аккумулятора в порт P2.

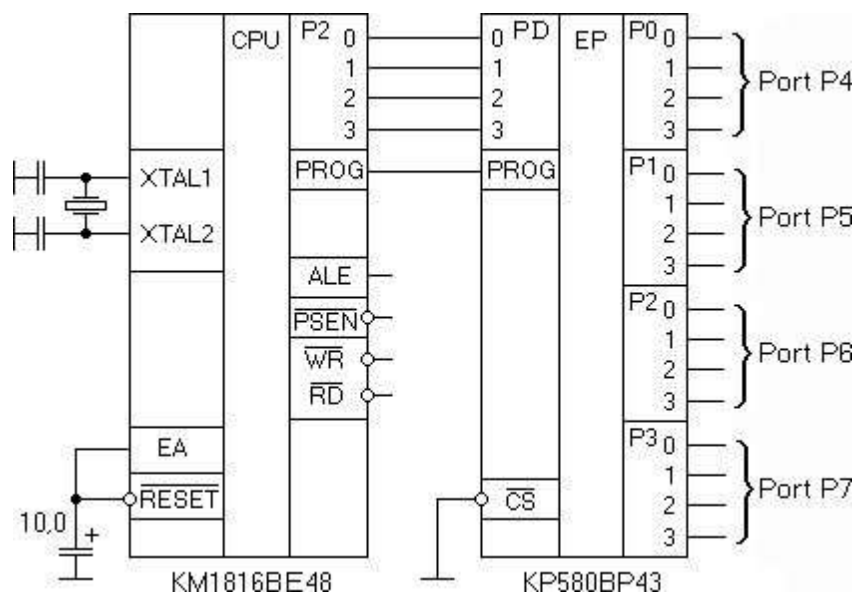
Квазидвунаправленная структура портов позволяет аппаратно выполнять *маскирование* требуемых линий при вводе, т.е. запись нулей в заданный разряд или группу разрядов. Например, если требуется обрабатывать только младшие 4 линии порта P2, то можно использовать последовательность команд:

- MOV A, #0Fh ; запись маски в аккумулятор;
- OUTL P2, A ; загрузка маски в буфер порта P2;
- IN A, P2 ; ввод младшей тетрады и маскирование старшей,

выполнение которых обеспечивает запись нулей в старшую тетраду аккумулятора. Следует учитывать, что текущее содержимое буфера P1 и P2 для чтения не доступно, поэтому программист должен быть уверен, что в буфере находится требуемая маска (в противном случае её необходимо загрузить повторно).

Порты P1 и P2 архитектурно идентичны, за исключением четырёх младших линий порта P2, которые могут использоваться для:

- выдачи старшей тетрады адреса ВПП,
- подключения расширителя КР580ВР43 (Рис.112), позволяющего увеличить количество линий ввода/вывода на шестнадцать (четыре четырёхразрядных порта P4-P7).



**Рис.112 - Подключение расширителя ввода/вывода**

Кроме обычных команд ввода/вывода IN, OUTL для изменения состояния линий P1, P2 можно использовать и логические команды ANL, ORL. Например:

ANL P1, #11100111b ; Установить логический “0” на линиях P1.3 и P1.4  
 ORL P2, #00011000b ; Установить логическую “1” на линиях P2.3 и P2.4

Такой приём позволяет заменить одной командой последовательность из трёх команд (ввод, логическая операция с аккумулятором, вывод).

## Система обработки прерываний

Система прерываний представляет собой механизм, позволяющий реагировать на особые состояния, возникающие при работе системы управления (срабатывание датчика аварийной ситуации, пропадание питания и т.д.). С точки зрения программиста прерывание – это принудительная приостановка текущей программы и переход на выполнение другой программы (обработчика прерывания), после завершения которой управление возвращается прерванной программе.

Микроконтроллеры семейства MCS-48 позволяют обрабатывать прерывания от двух источников:

- внешнего сигнала на входе INT (адрес вектора прерывания 3);
- флага переполнения таймера-счётчика (адрес вектора прерывания 7).

Прерывание от внешнего источника возникает, когда на входе INT появляется низкий уровень. Состояние сигнала на этом входе анализируется во время выдачи сигнала ALE (в каждом машинном цикле), а обработка прерывания начинается после завершения текущей команды. При обработке внешнего прерывания выполняются следующие действия:

- содержимое счётчика команд (адрес возврата, 12 разрядов) и старшей тетрады слова состояния программы (флаги, 4 разряда) сохраняется в стеке;
- в счётчик команд записывается значение 003h (по этому адресу должна находиться команда JMP, обеспечивающая переход к обработчику прерывания, расположенному в MB0).

Обработка прерывания должна заканчиваться командой RETR, по которой:

- восстанавливается из стека содержимое счётчика команд (адрес возврата) и старшей тетрады слова состояния программы;
- продолжается выполнение прерванной программы.

Ко входу INT микроконтроллера через «монтажное ИЛИ» могут быть подключены несколько источников прерываний, однако другие запросы прерываний игнорируются до выхода из подпрограммы обработки по команде RETR. В этом случае источник прерывания определяется программно (необходимо предусмотреть возможность ввода сигналов от всех источников прерываний через порт P1 или P2).

Поскольку система прерываний MCS-48 реагирует на уровень сигнала на входе INT (а не на переход из “1” в “0”), то для исключения повторных прерываний сигнал INT должен быть снят до выполнения команды RETR. Эта операция должна выполняться автоматически, в противном случае обработчик прерываний должен снять низкий уровень с линии INT через какую-либо линию порта P1 или P2.

Прерывания от таймера/счётчика возникают, когда флаг переполнения TF устанавливается в “1”. Они обрабатываются аналогично, но переход осуществляется по адресу 007h. Кроме того, флаг TF сбрасывается автоматически при переходе на подпрограмму обработки прерывания.

Приоритет внешнего прерывания выше приоритета прерывания от внутреннего таймера. Это означает, что в случае одновременного запроса прерываний от внешнего источника и запроса от флага переполнения таймера приоритет остается за источником, воздействующим на вход INT.

При необходимости, в МК можно создать двухуровневую систему прерываний. Для этого необходимо загрузить в таймер/счётчик число FFh и перевести его в режим счётчика. Тогда изменение сигнала на входе T1 из «1» в «0» приведёт к прерыванию по вектору 007h.

Внешние прерывания могут быть запрещены командой DIS I и разрешены командой EN I. Для разрешения и запрещения прерываний от таймера используются команды EN TCNTI и DIS TCNTI. Сигнал RESET запрещает прерывания от обоих источников. Если внешние прерывания запрещены, то вход INT может быть проверен командой условного перехода JNI (при этом он используется в качестве дополнительного тестируемого входа, подобно T0 и T1).

При входе в подпрограммы обслуживания прерываний старший бит счётчика команд PC11 принудительно устанавливается в нуль. Поэтому обработчики прерываний должны размещаться в банке памяти MB0.

## Таймер/счётчик

MCS-48 имеет встроенный восьмиразрядный таймер/счётчик (Рис.113), который может подсчитывать импульсы от внутреннего генератора (режим таймера) или импульсы от внешнего источника (режим счётчика).

*В режиме таймера* через делитель частоты на 32 поступают сигналы основной синхронизации с периодом 2,5 мкс (400 кГц), поэтому состояние счётчика увеличивается на 1 каждые 80 мкс (12,5 кГц). Путём программной установки таймера/счётчика в исходное состояние и анализа флага переполнения могут быть реализованы различные временные задержки, лежащие в диапазоне от 80 мкс до 20,48 мс. Временные задержки, превышающие по длительности 256 состояний счётчика (20,48 мс), могут быть получены накоплением переполнений в рабочем регистре. Заметим, что период работы таймера 20,48 мс примерно соответствует частоте 50 Гц.

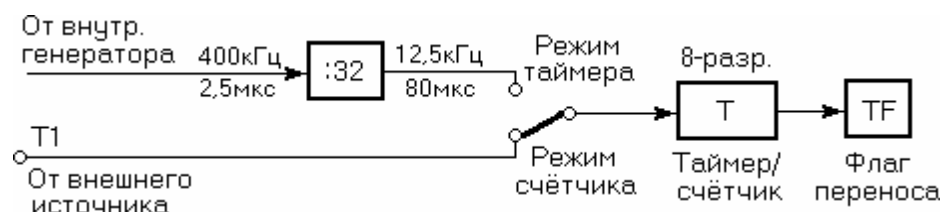


Рис.113 - Таймер микроконтроллеров семейства MCS-48

В режиме счётчика производится аппаратный подсчёт внешних импульсов, поступающих на тестируемый вход T1. При этом состояние счётчика изменяется в момент перехода входного сигнала из “1” в “0”. Количество подсчитанных импульсов можно прочесть командой MOV A, T. Минимальная длительность импульса на входе T1 равна 0,5 мкс, а минимальное время между импульсами – 7,5 мкс (при частоте кварцевого резонатора 6 МГц).

Из максимального состояния FFH таймер/счётчик переходит в начальное состояние 00H, при этом устанавливается флаг переполнения TF, что вызывает прерывание по вектору 007h (если оно разрешено командой EN TCNTI). Если прерывания от таймера/счётчика запрещены, то флаг переполнения может быть опрошен командой условного перехода JTF. Выполнение команды JTF, как и переход к подпрограмме обработки прерывания, сбрасывает флаг TF.

Выбор режима работы таймера/счётчика осуществляется специальными командами. Команда STRT T включает режим таймера, команда STRT CNT включает режим счётчика, а команда STOP TCNT останавливает как таймер, так и счётчик. Заметим, что сигнал RESET также останавливает таймер/счётчик.

Если требуется реализовать временную задержку, кратную 80 мкс, но меньшую 20,48 мс, то можно предварительно загрузить в таймер число, соответствующее количеству тактов в дополнительном коде. Например, для формирования временного интервала 4 мс (50 тактов по 80 мкс) загружается константа  $256 - 50 = 206$ , что соответствует CEh:

```
MOV A, #CEh ; количество периодов (в дополнительном коде)
MOV T, A ; загрузка таймера
STRT T ; запуск таймера
M0: JTF M1 ; ожидание переполнения таймера
JMP M0
M1: STOP TCNT ; остановка таймера
. . . . .
```

На основе этой программы также можно реализовать задержки большей длительности. Для этого используется регистр, в который заносится требуемое число переполнений таймера, и организуется цикл, внутри которого осуществляется ожидание переполнения флага TF. Следует заметить, что наиболее эффективно таймер используется в режиме прерываний, когда для обнаружения переполнения не используются ресурсы процессора (при этом процессор может выполнять другую программу, обеспечивающую, например, отображение информации или анализ нажатия клавиш).

## Система команд MCS-48

Система команд микроконтроллеров семейства MCS-48 содержит 96 базовых команд, имеющих длину один или два байта. Большинство команд являются однобайтными и выполняются за один машинный цикл длительностью 2,5 мкс (при частоте 6 МГц). Двухбайтные команды выполняются за 5 мкс. Команды, обращающиеся к ОЗУ, также выполняются за 5 мкс.

В микроконтроллерах реализована как побайтовая (8 бит) так и потетрадная (4 бита) обработка данных. К достоинствам системы команд MCS-48 можно отнести: эффективный ввод/вывод, включая маскирование и возможность управления отдельными битами портов; возможность ветвления по значению отдельных бит; возможность обработки как двоичных, так и десятичных двоично-кодированных чисел. Форматы команд приведены на Рис.114 (КОП – код операции, #d – непосредственный операнд, a10..0 – адрес):

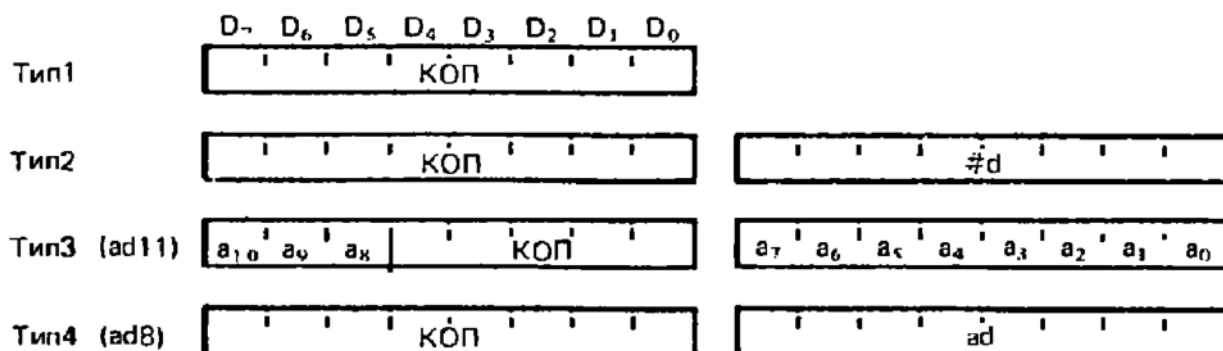


Рис.114 - Типы команд МК48

Систему команд микроконтроллера условно можно разбить на пять групп: арифметические, логические, команды передачи данных, команды ветвления программ и передачи управления, команды управления режимами. В MCS-48 реализовано четыре способа адресации:

1. **Непосредственная** – в теле команды содержится восьмибитный операнд (во втором байте команды):

MOV A, #05h ; (A) ← 05h

2. **Прямая** – в теле команды содержится адрес операнда (номер регистра):

MOV A, Rn ; (A) ← (Rn)

3. **Косвенная** – в теле команды содержится адрес адреса операнда (номер регистра, в котором расположен адрес ячейки памяти):

MOV A, @R0 ; (A) ← ((R0))

Косвенная адресация допускается только через два регистра: @R0 или @R1.

4. **Неявная**, – операнд или его адрес не указывается в явном виде, а определяется кодом операции (по умолчанию). Обычно таким операндом является содержимое аккумулятора:

DA A ; двоично-десятичная коррекция A

Хотя в мнемонике этой команды аккумулятор указывается явно, в формате команды нет адресного поля (т.е. неявно подразумевается, что операндом может быть только аккумулятор).

## Команды пересылки данных

С помощью этих команд можно произвести копирование либо обмен данных. Ячейка памяти или регистр, в котором располагается исходный байт называется *источником S* (source), а ячейка или регистр, в который производится копирование, – *приёмником D* (destination). Для рассматриваемого микроконтроллера команды пересылки записываются в виде MOV D, S, т.е. сначала указывается приёмник, а затем источник.

В семействе MCS-48 источником и приёмником могут быть аккумулятор A, регистры R0...R7, ячейки памяти, порты ввода/вывода, а также регистр таймера T и слово состояния процессора PSW. Однако система команд не позволяет переслать данные из любого источника в любой приёмник, поэтому большинство пересылок выполняется через аккумулятор (Рис.114).

Передача данных возможна в двух режимах: *пересылки* (загрузки) и *обмена*. При этом команды пересылки не изменяют содержимого источника, а команды обмена изменяют значения обоих операндов, участвующих в операции. Команды пересылки внутри микроконтроллера выполняются за один машинный цикл, обмен с внешней памятью и портами требует двух машинных циклов. Все команды пересылки (кроме MOV PSW, A) не изменяют флаги.

Схема возможных пересылок представлена на Рис.115, где показаны 9 типов операндов: аккумулятор A, регистры общего назначения R0...R7, PSW, таймер, порты, непосредственный операнд #d, внешняя и резидентная память данных, а также память программ (только для чтения). Большинство команд выполняет пересылку 8-битных операндов. Существуют также несколько команд, оперирующих с четырёхбитными операндами (при обращении к 4-битным портам расширителя ввода/вывода P4...P7).

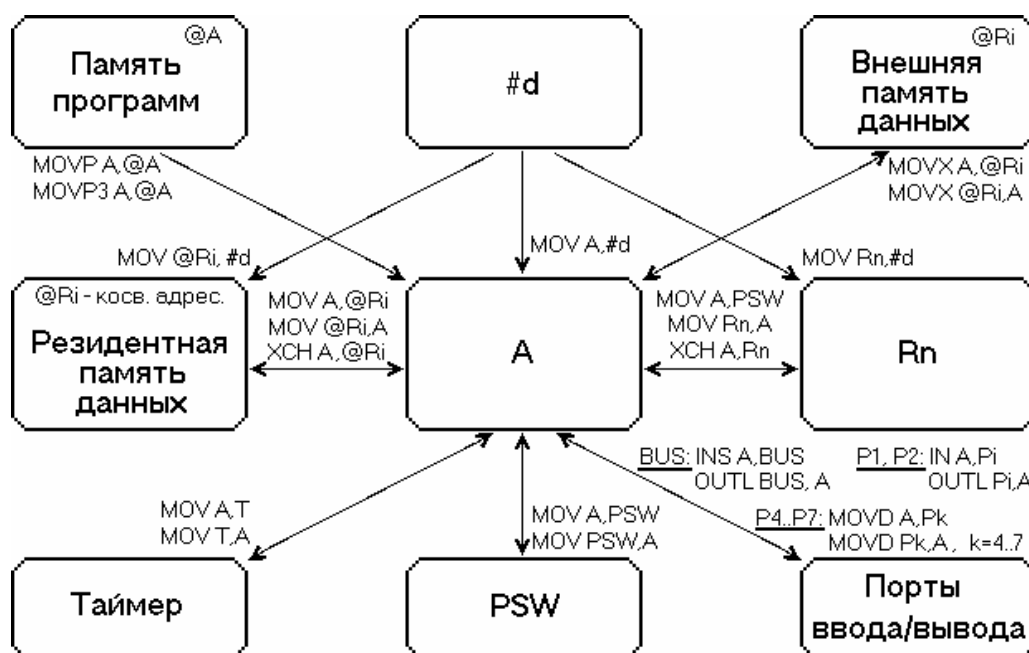


Рис.115 - Команды пересылки данных MCS-48

Так как в микроконтроллере существуют три независимых области памяти, а также порты ввода/вывода, то для обращения к ним используются различные команды:

- MOV, XCH – пересылка данных внутри микроконтроллера;
- MOVX – пересылка данных между аккумулятором и ВПД;
- MOVP – пересылка данных из памяти программ;
- IN, OUTL, MOVD – пересылка из портов в аккумулятор и наоборот.

Для обращения к резидентной памяти данных используется косвенная адресация через регистры R0 и R1:

**MOV A, @R0** ; переслать в аккумулятор содержимое ячейки РПД с адресом, хранящемся в R0;  
**MOV @R1, A** ; переслать содержимое аккумулятора в ячейку РПД с адресом, хранящемся в R1.

Для аккумулятора и регистров также возможно выполнение команды обмена:

**XCH A, R0** ; обменять содержимое аккумулятора с содержимым регистра

R0 текущего банка.

В памяти программ наряду с командами можно располагать таблицы функций  $\sin x$ ,  $\cos x$ ,  $\sqrt{x}$ , а также константы  $\pi$ ,  $e$  и др. Их можно скопировать в аккумулятор с помощью команд с косвенной адресацией:

**MOVP A, @A** ; переслать в аккумулятор байт из текущей страницы памяти программ (адрес внутри страницы содержится в аккумуляторе);

**MOVPR A, @A** ; переслать в аккумулятор байт из третьей страницы памяти программ (адреса – 300h..3FFh).

Для обращения к внешней памяти данных также используется косвенная адресация через регистры R0 и R1:

**MOVX A, @R0** ; переслать в аккумулятор содержимое ячейки ВПД с адресом, хранящемся в R0;

**MOVX @R1, A** ; переслать содержимое аккумулятора в ячейку ВПД с адресом, хранящемся в R1.

Для работы с параллельными портами используются команды ввода/вывода IN и OUTL (для 8-разрядных операндов) и команды MOVD (для 4-разрядных операндов):

**IN A, P0** ; ввести байт из порта P0 в аккумулятор;

**OUTL P1, A** ; вывести байт из аккумулятора в порт P1;

**MOVD A, P4** ; ввести данные из порта P4 в младшую тетраду аккумулятора (старшая тетрада обнуляется);

**MOVD P7, A** ; вывести данные из младшей тетрады аккумулятора в порт P7.

При использовании команд пересылки данных следует учитывать, что большинство из них не изменяет флагов переноса C и полупереноса AC (за исключением команды MOV PSW, A). Кроме того, поскольку символы R0...R7 могут обозначать две различные области РПД, то необходимо корректно управлять выбором текущего банка регистров (команды SEL MB0 и SEL MB1, а также MOV PSW, A).

## Арифметические команды

В системе команд микроконтроллера имеются следующие арифметические команды, позволяющие выполнять операции над 8-битными целыми числами:

- ADD – сложение однобайтных чисел;
- ADDC – сложение с учетом флага переноса;
- INC – инкрементирование (увеличение на 1);
- DEC – декрементирование (уменьшение на 1);
- DA – двоично-десятичная коррекция.

Команды сложения ADD, ADDC и DA формируют два флага: C (перенос из старшего разряда) и AC (полуперенос, т.е. перенос из младшей тетрады в старшую). Эти флаги используются для выполнения условных переходов, двоично-десятичной коррекции, а также для сложения многобайтных чисел.

Любопытный факт: в системе команд MCS-48 отсутствует команда вычитания, что само по себе довольно удивительно, т.к. в системе команд были реализованы гораздо более сложные инструкции, например, XRL («ИЛИ исключительное»). Для дополнительного



кода (ДК) команда вычитания может быть заменена последовательностью их трёх инструкций:

$$a - b = a + DK(b) = a + (\bar{b} + 1)$$

Команды сложения ADD и ADDC выполняют операции сложения над однобайтными двоичными числами, которые могут интерпретироваться двумя способами:

- 8-разрядное число без знака (unsigned) в диапазоне  $0 \dots 255_{10}$ ;
- 8-разрядное число со знаком (signed) с диапазоном  $-128_{10} \dots +127_{10}$ , в котором старший разряд является знаковым («1» означает минус, «0» – плюс).

При использовании арифметических операций следует учитывать, что команды INC и DEC не изменяют флаги C и AC.

### Логические команды

Система команд микроконтроллеров MCS-48 может выполнять следующие логические операции над байтами:

- ANL – логическое «И» аккумулятора с операндом;
- ORL – логическое «ИЛИ» аккумулятора с операндом;
- XRL – логическое «Исключающее ИЛИ» аккумулятора с операндом;
- CLR A – очистка содержимого аккумулятора (обнуление);
- CPL A – инвертирование содержимого аккумулятора.

Эти операции выполняются побитно, а результат помещается в аккумулятор. Команды ANL, ORL, XRL и CPL A не воздействуют на флаги. Команда CLR A очищает флаг C.

Существуют также логические команды (ANL, ORL), результат выполнения которых помещается в буфер порта ввода/вывода BUS, P1, P2. Аналогичные команды (ANLD, ORLD) имеются для четырёхразрядных портов P4..P7. Это позволяет эффективно управлять значениями отдельных бит при вводе/выводе информации.

ORL P1, #00001000b ; установить третий бит порта P1;  
ANL P2, #10111111b ; сбросить шестой бит порта P2.

Кроме того, имеются команды, позволяющие выполнять операции над отдельными битами:

- CLR C; CLR F0; CLR F1 – сброс флагов;
- CPL C; CPL F0; CPL F1 – инвертирование флагов.

Для установки флагов в “1” используется последовательность команд CLR, CPL.

К группе логических команд относятся и команды циклического сдвига содержимого аккумулятора RL, RR, RLC, RRC (Рис.116). На основе команд RRC, RLC можно реализовать арифметический сдвиг, т.е. умножение и деление на 2 (при умножении перед сдвигом во флаг C необходимо занести 0, а при делении – скопировать в него знаковый разряд аккумулятора).

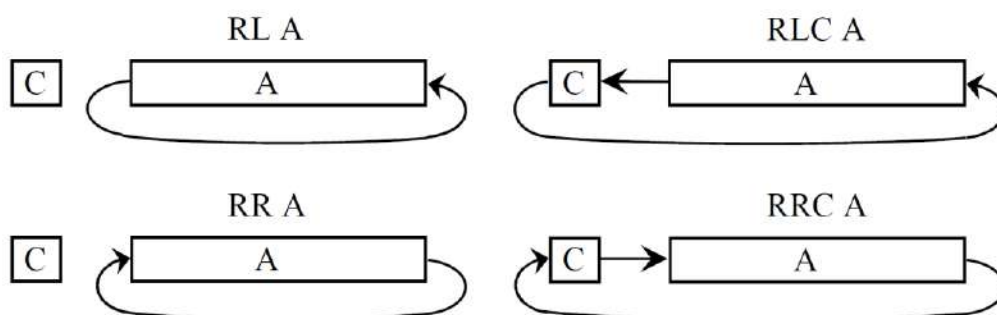


Рис.116 - Команды циклического сдвига

## Команды передачи управления

Команды передачи управления позволяют изменить естественный ход выполнения программы. Обычно при выполнении программы содержимое счётчика команд последовательно увеличивается на 1 или 2 (для однобайтных и двухбайтных команд соответственно). А команды передачи управления изменяют эту последовательность и приводят к записи в счётчик команд нового значения.

В микроконтроллерах семейства MCS-48 реализовано две команды безусловного перехода:

- JMP ad11 – безусловный переход (с 11-разрядным адресом);
- JMPP @A – безусловный переход в границах текущей страницы ПП

и 20 команд условного перехода (с 8-разрядным адресом):

- JZ, JNZ – переход по нулевому/ненулевому содержимому аккумулятора;
- JC, JNC – переход по установленному/сброшенному флагу C;
- JB0...JB7 – переход по единичному значению бита аккумулятора;
- JF0, JF1 – переход по единичному значению флагов F0 и F1;
- JTF – переход по переполнению таймера (установке флага TF);
- JT0, JNT0 – переход по высокому/низкому уровню на входе T0;
- JT1, JNT1 – переход по высокому/низкому уровню на входе T1;
- JN1 – переход по низкому уровню на входе прерывания INT.

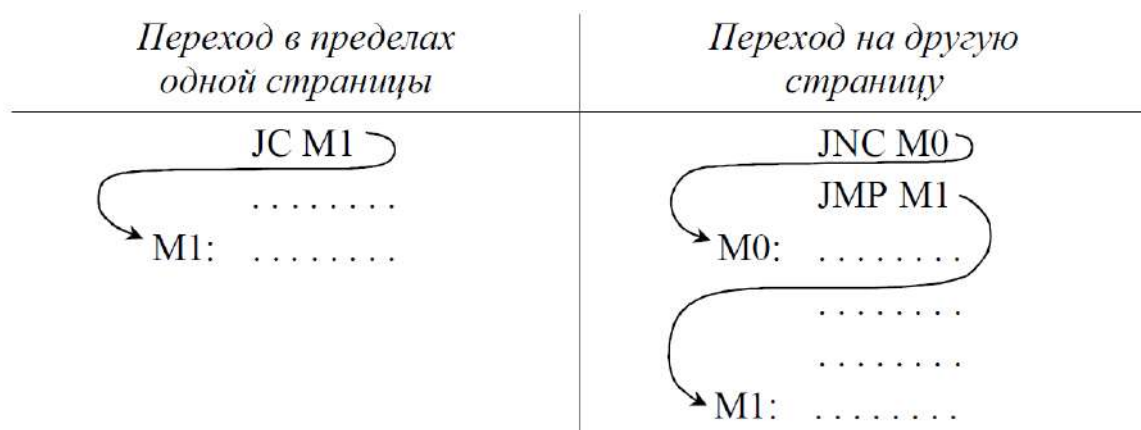
Команда безусловного перехода JMP задаёт 11-разрядный адрес ad11, поэтому она позволяет осуществить переход в пределах банка MB0 или MB1 объёмом по 2 Кбайт. Однако для полной адресации памяти программ объёмом 4 Кбайт требуется 12 разрядов. Номер банка определяется флагом DBF, значение которого копируется в старший бит счётчика команд (PC<sub>11</sub>). А сам флаг DBF управляется командами SEL MB0 и SEL MB1. Такая логика формирования адреса перехода приводит к тому, что для переключения банков памяти программ требуется выполнить две команды:

SEL MB1 ; изменить номер банка во флаге DBF;  
JMP ad11 ; перейти по адресу ad11 в банке MB1.

При этом собственно переключение банка (PC<sub>11</sub> = 1) происходит по команде JMP, а команда SEL используется для подготовки переключения.

Остальные команды ветвления содержат только 8-разрядный адрес, который копируется в младшие разряды счётчика команд PC<sub>0</sub>...PC<sub>7</sub>. При этом его старшие разряды PC<sub>8</sub>..PC<sub>11</sub> не изменяются. Поэтому переход осуществляется только в пределах одной страницы объёмом 256 байт. Это порождает проблему перехода с одной страницы на другую,

которая решается путём комбинирования команд условного перехода и команды безусловного перехода JMP, например, вместо команды JC ad8 выполняется последовательность команд:



Если команда перехода с 8-разрядным адресом расположена на границе двух страниц (т.е. первый байт команды на одной странице, а второй – на следующей), то переход будет выполнен в пределах той страницы, где располагается второй байт команды.

В отличие от классических процессоров микроконтроллеры семейства MCS-48 не имеют команд перехода по положительному и отрицательному результату. Однако имеются команды JB0...JB7, позволяющие выполнить анализ любого бита аккумулятора. Поэтому для условного перехода по отрицательному результату используется команда JB7, а для перехода по положительному результату предварительно инвертируется содержимое аккумулятора.

Команды JC и JNC совместно с командой сложения ADD позволяют реализовать переходы по условиям «больше/меньше» некоторой пороговой величины. Например, для перехода по условию  $A > 05$  используются команды:

```
ADD A, #0FBh ; сложить аккумулятор с числом -6;
JC M0 ; если до сложения  $A > 05$ , то перейти на M0.
```

Но следует учитывать, что при этом изменяется содержимое аккумулятора.

Команда безусловного перехода с косвенной адресацией JMPP @A позволяет реализовать *множественные ветвления*, аналогично операторам SWITCH или CASE языков высокого уровня. При этом в памяти программ располагается таблица команд JMP, а в аккумулятор помещается номер ячейки таблицы, содержащей команду JMP с требуемым адресом перехода. Эта таблица должна располагаться в той же странице памяти программ, что и команда JMPP, а ячейки таблицы нумеруются числами 0, 2, 4, ... .

Для организации *циклов* удобно использовать команду DJNZ, сочетающую в себе декремент и переход по ненулевому результату. При этом счётчик циклов организуется в одном из регистров:

```

M0:  MOV R4, #0Ah      ; инициализация счётчика цикла
      .....
      .....          ; тело цикла
      .....
      DJNZ R4, M0      ; декремент R4 и повторение цикла, если R4 ≠ 0

```

Тело цикла от метки M0 до команды DJNZ включительно должны находиться в пределах одной страницы памяти программ.

Сложные алгоритмы управления можно реализовать, применяя структурный стиль программирования, при котором задача разбивается на мелкие подзадачи, каждая из которых решается своей процедурой или функцией. Для этого используются следующие команды:

- CALL – вызов подпрограммы (с 11-разрядным адресом);
- RET – возврат из подпрограммы (без восстановления PSW);
- RETR – возврат из прерывания (с восстановлением PSW).

При вызове подпрограммы выполняются следующие действия:

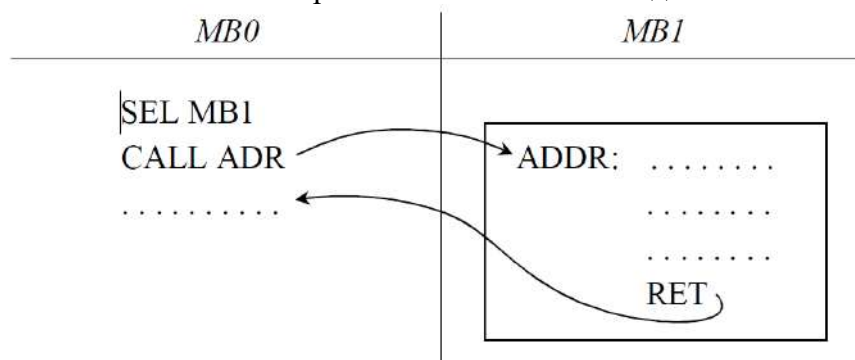
- содержимое счётчика команд (адрес возврата, 12 разрядов) и старшей тетрады слова состояния (флаги, 4 разряда) сохраняются в стеке;
- в счётчик команд записывается 12-разрядный адрес подпрограммы, старший бит которого копируется из флага DBF, а остальные 11 разрядов являются аргументом команды CALL.

Глубина вложений подпрограмм ограничена ёмкостью стека (16 байт) и не должна превышать восьми. Подпрограмма должна заканчиваться командой RET, по которой:

- восстанавливается из стека содержимое счётчика команд (адрес возврата);
- продолжается выполнение основной программы.

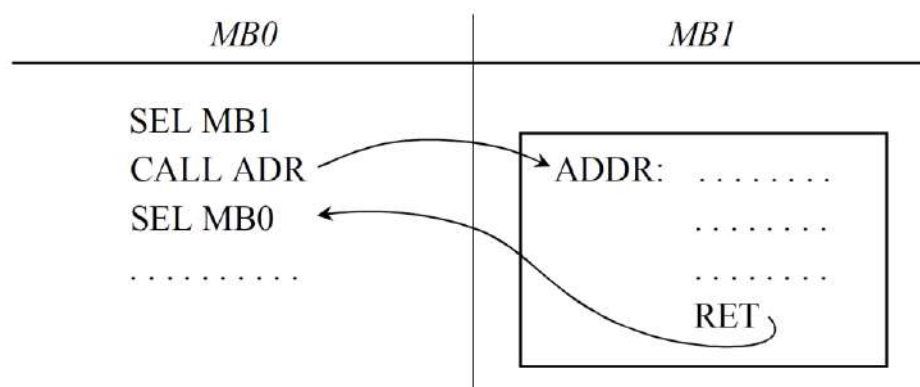
Подпрограмма может заканчиваться командой возврата из прерывания RETR, которая кроме адреса возврата восстанавливает PSW.

Подобно команде JMP, команда CALL позволяет обратиться в любое место текущего банка памяти программ. Кроме того, используя команды SEL MB0 и SEL MB1, можно вызвать подпрограммы из альтернативного (по отношению к текущему) банка. В этом случае перед вызовом подпрограммы необходимо выбрать соответствующий банк памяти, а перед возвратом – восстановить старое значение DBF командой SEL:



Если в подпрограмме не восстановить значение номера банка во флаге DBF командой SEL MB0, то возврат все же будет выполнен правильно, в банк MB0 (так как в стеке сохранен полный 12-разрядный адрес возврата). Однако первая же команда JMP или CALL переключит банк на MB1.

Если к подпрограмме обращаться из разных банков памяти, то необходимо восстанавливать DBF в основной программе вслед за командой вызова:



Следует также учитывать, что подпрограммы обработки прерывания должны располагаться только в нулевом банке памяти MB0.

## Команды управления режимами работы микроконтроллера

В эту группу входят команды управления таймером/счётчиком, прерываниями и флагами переключения банков регистров и банков памяти программ:

- STRT CNT – запуск счётчика;
- STRT T – запуск таймера;
- STOP TCNT – останов таймера/счётчика;
- EN I, DIS I – разрешение/запрещение внешних прерываний;
- EN TCNTI, DIS TCNTI – разрешение/запрещение прерывания от таймера/счётчика;
- SEL RB0, SEL RB1 – выбор текущего банка регистров;
- SEL MB0, SEL MB1 – выбор текущего банка памяти программ;
- ENT0 CLC – разрешение выдачи синхросигнала на выход T0.

Последняя из этих команд ENT0 используется для выдачи на вывод T0 синхроимпульсов с частотой  $f_0 / 3$ , где  $f_0$  – частота кварцевого резонатора. Этот сигнал используется для синхронизации внешних устройств. Он может быть отключен только сбросом микроконтроллера.

Наличие команд переключения банков регистров позволяет при вызове подпрограмм и обработке прерываний эффективно использовать второй банк регистров в качестве рабочего, сохраняя параметры вычислительного процесса не в стеке, а в исходном банке регистров. При этом возврат к исходному банку регистров будет выполнен автоматически, если подпрограмма обработки прерывания оканчивается командой возврата RETR (с восстановлением PSW).

## Семейство MCS-51

### Архитектура микроконтроллера MCS-51

Семейство микроконтроллеров MCS-51 основано на 8-разрядном процессорном ядре. В его состав входит целый ряд микросхем, отличающихся производительностью, объёмом встроенной памяти, количеством портов ввода/вывода, характеристиками таймеров и интерфейсных модулей. Все микроконтроллеры этого семейства работают с одной и той же системой команд, большинство из них выполняется в одинаковых корпусах с совпадающей цоколевкой. Это позволяет использовать для разработанного устройства микросхемы разных производителей без изменения принципиальной схемы и программы.

Первым представителем этого семейства является микросхема 8051 фирмы Intel, выпущенная в 1980 году на базе n-МОП технологии. В более поздних модификациях 80C51

использовалась технология CMOS, отличающаяся пониженным энергопотреблением. Отечественными аналогами семейства MCS-51 являются микросхемы 1816BE31/51, 1830BE31/51, 1834BE31 и 1850BE31. Типичный микроконтроллер этого семейства имеет 1 Кбайт внутренней памяти программ и 128 байт внутренней памяти данных (16 байт из которых допускают побитовую адресацию).

Первоначально при выполнении команд использовались простейшие методы синхронизации, при которых каждый машинный цикл состоял из 12 тактов. Поэтому при тактовой частоте 12 МГц команда выполнялась за 1...2 мкс. В дальнейшем число тактов на цикл удалось сократить до четырёх, двух, и даже одного, а тактовую частоту довести до 100 МГц.

На сегодняшний день существует более 200 модификаций микроконтроллеров семейства MCS-51, выпускаемых почти двадцатью компаниями (Philips, Siemens, Intel, Atmel, Dallas, Temic, Oki, AMD, MHS, Gold Star, Winbond, Silicon Systems и др.). Эти модификации включают в себя кристаллы с широчайшим спектром периферии: от простых 20-выводных устройств с одним таймером и 1К программной памяти до сложнейших 100-выводных кристаллов с 10-разрядными АЦП, массивами таймеров-счетчиков, аппаратными 16-разрядными умножителями и 64 Кбайт программной памяти на кристалле. Основными направлениями их развития являются: повышение быстродействия, снижение напряжения питания и энергопотребления, увеличение объема внутренней памяти данных, использование внутренней FLASH-памяти программ, а также введение в состав периферии сложных устройств таких, как схемы управления электроприводами, CAN-интерфейсов распределённых систем управления, USB-интерфейсов и т.п.

Основными структурными элементами базовой модели семейства микроконтроллеров MCS-51 являются Рис.117:

- арифметико-логическое устройство;
- внутренняя память программ (4 Кбайт);
- внутренняя память данных (128 байт);
- два 16-разрядных таймера/счётчика;
- последовательный интерфейс (UART);
- блок обработки прерываний;
- четыре 8-разрядных порта ввода/вывода;
- тактовый генератор.

Конструктивно микроконтроллер был выполнен в корпусе DIP-40 с внешними выводами, электрически совместимыми с уровнями ТТЛ. Он имеет 32 вывода для подключения внешних устройств (4 восьмиразрядных порта P0...P3), два вывода питания, два вывода для кварцевого резонатора и четыре вывода для управляющих сигналов. Порт P3 может быть запрограммирован пользователем на выполнение специальных (альтернативных) функций обмена информацией с внешней средой: последовательный ввод/вывод, чтение/запись, ввод тестируемых сигналов и сигналов прерывания.

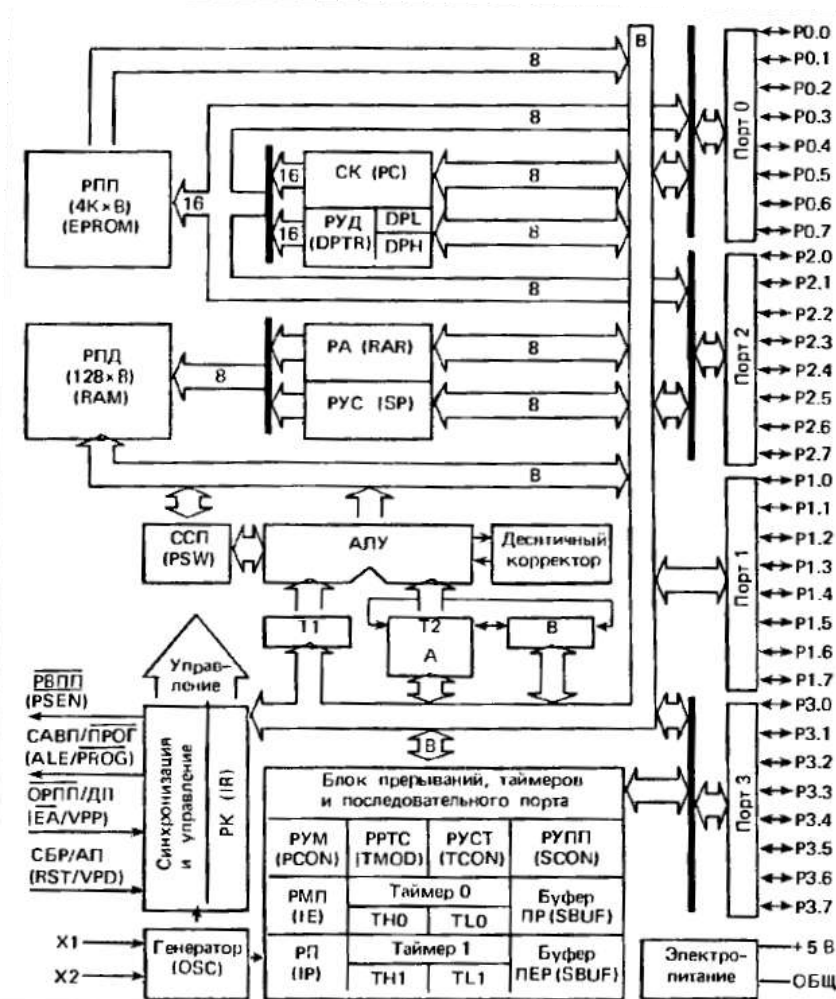


Рис.117 - Структурная схема микроконтроллера 1816BE51

## Арифметико-логическое устройство

Восьмиразрядное арифметико-логическое устройство может выполнять арифметические операции сложения, вычитания, умножения и деления; логические операции И, ИЛИ, исключающее ИЛИ, а также операции циклического сдвига, сброса, инвертирования и т.п. В нём имеются программно недоступные регистры T1 и T2, предназначенные для временного хранения операндов, схема десятичной коррекции и схема формирования признаков.

Для реализации сложных команд в АЛУ используется механизм каскадного выполнения простейших операций, которые автоматически образуют «тандемы». Так, например, при выполнении одной из команд условной передачи управления по результату сравнения трижды инкрементируется счётчик команд, дважды производится чтение из РПД, выполняется арифметическое сравнение двух переменных, формируется 16-битный адрес перехода и принимается решение об условном переходе. Все перечисленные операции выполняются в АЛУ за 2 мкс.

Арифметико-логическое устройство может оперировать четырьмя типами данных: однобитными, четырёхбитными (тетрадами), 8-битными (байтами) и 16-битными. При создании прикладных программ разработчику доступны любые ячейки памяти данных и памяти программ, аккумулятор A, расширитель аккумулятора B, восемь регистров общего назначения R0...R7, слово состояния программы PSW, указатель стека SP, а также ряд регистров специальных функций (Рис.119).

Важной особенностью АЛУ является его способность оперировать не только байтами, но и битами. Эти операции выполняются специальным модулем АЛУ – «булевым процессором», который использует в качестве аккумулятора флаг переноса С. Этот процессор имеет побитово-адресуемое ОЗУ объёмом 128 бит, а также позволяет обращаться к 128 битовым полям регистров специальных функций. Побитно-адресуемыми являются все порты ввода-вывода, каждая линия которых может рассматриваться как однобитовый порт. Отдельно программно-доступные биты могут быть установлены, сброшены, инвертированы, переданы, проверены и использованы в логических операциях.

## Память микроконтроллера

В семействе микроконтроллеров MCS-51 память физически и логически разделена на память программ и память данных. В этих микроконтроллерах имеется три адресных пространства (Рис.118): память программ (ПП) – 64 Кбайт, внешняя память данных (ВПД) – 64 Кбайт, а также пространство внутренней (резидентной) памяти данных (РПД) и регистров специальных функций (РСФ) – 256 байт.

Память программ (ROM или EPROM) имеет суммарное адресное пространство объёмом 64 Кбайт, из которых 4 Кбайт размещается на кристалле микроконтроллера (РПП). Область её нижних адресов используется при сбросе и обработке прерываний:

- 0000h – адрес перехода по сигналу RESET;
- 0003h – адрес вектора внешнего прерывания INT0;
- 000Bh – адрес вектора прерывания от таймера T0;
- 0013h – адрес вектора внешнего прерывания INT1;
- 001Bh – адрес вектора прерывания от таймера T1;
- 0023h – адрес вектора прерывания от последовательного порта.

При необходимости память программ может быть расширена за счёт дополнительных микросхем памяти, подключаемых к портам P0, P2 с помощью управляющих сигналов PSEN и ALE. Основное назначение памяти программ – хранение команд, которые считываются автоматически. Кроме того, в памяти программ хранятся константы, таблицы функций и т.д., доступ к которым осуществляется командами MOVC.

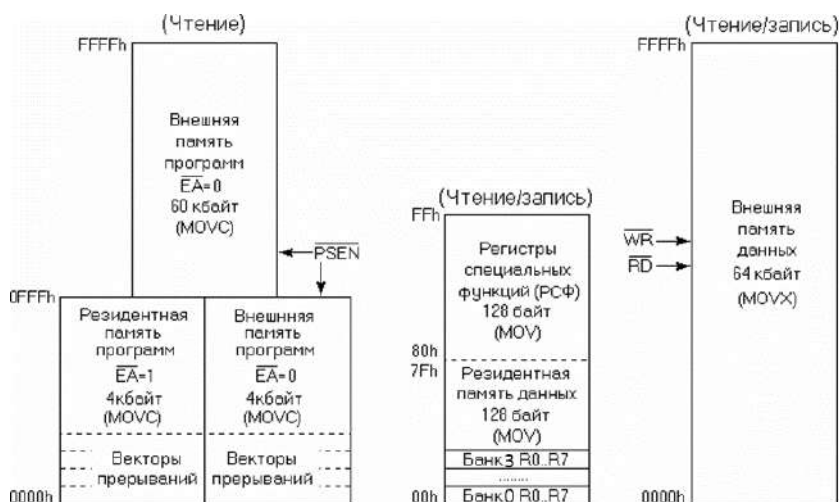


Рис.118 - Организация памяти микроконтроллера MCS-51

Резидентная память данных (RAM) предназначена для хранения переменных в процессе выполнения прикладной программы. Она адресуется одним байтом и имеет ёмкость 128 байт. В ней размещается 4 банка рабочих регистров R0...R7, а также стек. Для банков



регистров используются первые байта. Регистры R0 и R1 в любом из банков могут использоваться в качестве регистров косвенного адреса @R0 и @R1. Следующие за регистровыми банками 16 байт образуют блок побитно-адресуемого пространства (128 бит с адресами от 00h до 7Fh). Все байты в нижней 128-байтной половине памяти могут адресоваться как прямо, так и косвенно. По умолчанию область стека начинается с адреса 0008h, так как при сбросе указатель стека SP = 0007h. Поэтому при использовании дополнительных банков регистров программист должен переопределить начальное значение SP.

К адресному пространству РПД примыкают адреса регистров специальных функций, которые перечислены на Рис.119. Они включают в себя регистры портов, таймеры, средства управления периферией и т.д. Часть из них допускает побитовую адресацию (128 бит с адресами 80h..FFh). Например, порт P0 можно адресовать двумя способами: как байт с адресом 80h, либо как набор битов P0.0...P0.7 с адресами 80h...87h. Карта адресов РПД и РСФ приведена на Рис.120. При обращении к РПД и РСФ используются команды MOV.

| Обозначение | Наименование                          | Адрес |
|-------------|---------------------------------------|-------|
| ACC*        | Аккумулятор                           | E0h   |
| B*          | Регистр-расширитель аккумулятора      | F0h   |
| PSW*        | Слово состояния программы             | D0h   |
| SP          | Регистр-указатель стека               | 81h   |
| DPTR        | Регистр-указатель данных (DPL)        | 82h   |
|             | Регистр-указатель данных (DPH)        | 83h   |
| P0*         | Порт 0                                | 80h   |
| P1*         | Порт 1                                | 90h   |
| P2*         | Порт 2                                | A0h   |
| P3*         | Порт 3                                | B0h   |
| IP*         | Регистр приоритетов                   | B8h   |
| IE*         | Регистр маски прерываний              | A8h   |
| TMOD        | Регистр режима таймера/счетчика       | 89h   |
| TCON*       | Регистр управления/статуса таймера    | 88h   |
| TH0         | Таймер 0 (старший байт)               | 8Ch   |
| TL0         | Таймер 0 (младший байт)               | 8Ah   |
| TH1         | Таймер 1 (старший байт)               | 8Dh   |
| TL1         | Таймер 1 (старший байт)               | 8Bh   |
| SCON*       | Регистр управления приемопередатчиком | 98h   |
| SBUF        | Буфер приемопередатчика               | 99h   |
| PCON        | Регистр управления мощностью          | 87h   |

Примечание. Регистры, имена которых отмечены знаком (\*), допускают адресацию отдельных бит.

**Рис.119 - Регистры специальных функций**

Существуют варианты микроконтроллеров семейства MCS-51 с резидентной памятью данных объемом 256 байт. В этом случае при обращении к области РСФ используется прямая адресация, а при обращении к верхней части области РПД – косвенная адресация.

|     |                                                    |     |     |     |     |     |     |     |
|-----|----------------------------------------------------|-----|-----|-----|-----|-----|-----|-----|
| 7Fh | Побайтно-адресуемая область ОЗУ (direct, indirect) |     |     |     |     |     |     |     |
| 30h |                                                    |     |     |     |     |     |     |     |
| 2Fh | 7Fh                                                | 7Eh | 7Dh | 7Ch | 7Bh | 7Ah | 79h | 78h |
| 2Eh | 77h                                                | 76h | 75h | 74h | 73h | 72h | 71h | 70h |
|     | Побитно-адресуемая область ОЗУ (direct)            |     |     |     |     |     |     |     |
| 21h | 0Fh                                                | 0Eh | 0Dh | 0Ch | 0Ah | 0Bh | 09h | 08h |
| 20h | 07h                                                | 06h | 05h | 04h | 03h | 02h | 01h | 00h |
| 1Fh | RB3                                                |     |     |     |     |     |     |     |
| 18h |                                                    |     |     |     |     |     |     |     |
| 17h | RB2                                                |     |     |     |     |     |     |     |
| 10h |                                                    |     |     |     |     |     |     |     |
| 0Fh | RB1                                                |     |     |     |     |     |     |     |
| 08h |                                                    |     |     |     |     |     |     |     |
| 07h | RB0(R7+R0)                                         |     |     |     |     |     |     |     |
| 00h | SP после RESET                                     |     |     |     |     |     |     |     |

|                     |      |       |     |     |     |     |     |      |     |
|---------------------|------|-------|-----|-----|-----|-----|-----|------|-----|
| Побитовая адресация |      | 8байт |     |     |     |     |     |      |     |
| F8h                 |      |       |     |     |     |     |     |      | FFh |
| F0h                 | B    |       |     |     |     |     |     |      | E7h |
| E8h                 |      |       |     |     |     |     |     |      | EFh |
| E0h                 | ACC  |       |     |     |     |     |     |      | E7h |
| D8h                 |      |       |     |     |     |     |     |      | DFh |
| D0h                 | PSW  |       |     |     |     |     |     |      | D7h |
| C8h                 |      |       |     |     |     |     |     |      | CFh |
| C0h                 |      |       |     |     |     |     |     |      | C7h |
| B8h                 | IP   |       |     |     |     |     |     |      | B7h |
| B0h                 | P3   |       |     |     |     |     |     |      | B7h |
| A8h                 | IE   |       |     |     |     |     |     |      | AFh |
| A0h                 | P2   |       |     |     |     |     |     |      | A7h |
| 98h                 | SCON | SBUF  |     |     |     |     |     |      | 9Fh |
| 90h                 | P1   |       |     |     |     |     |     |      | 97h |
| 88h                 | TCON | TMOD  | TL0 | TL1 | TH0 | TH1 |     |      | 8Fh |
| 80h                 | P0   | SP    | DPL | DPH |     |     |     | PCON | 87h |
|                     | 0/8  | 1/9   | 2/A | 3/B | 4/C | 5/D | 6/E | 7/F  |     |

Рис.120 - Побайтовая и побитовая адресация РПД и РСФ

Внешняя память данных (RAM) имеет независимое адресное пространство объёмом 64 Кбайт, для обращения к ней используются команда MOVX. Микросхемы внешней памяти данных подключаются к портам P0, P2 с помощью управляющих сигналов RD, WR и ALE. При необходимости адресное пространство ВПД можно совместить с адресным пространством ВПП, объединяя сигналы RD и PSE.

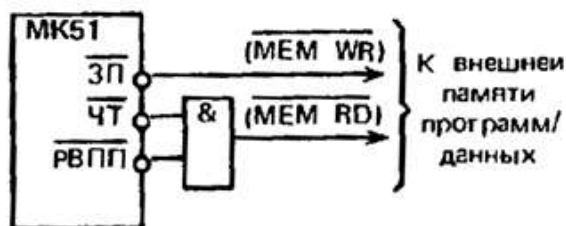


Рис.121 - Схема совмещения адресного пространства ВПП и ВПД

функцию «ИЛИ» в инверсной логике.

MCS-51, как уже несколько раз упоминалось выше, имеет гарвардскую архитектуру, но на этапе отладки машина неймановской архитектуры оказывается очень удобной, т.к. позволяет разработчику оперативно менять код программы, расположенной в ОЗУ. С этой целью МК-система может быть модифицирована для совмещения адресного пространства ВПП и ВПД путём подключения внешней логики (Рис.121). Схема «И» здесь по сути реализует

## Параллельные порты ввода/вывода

Линии ввода/вывода микроконтроллеров семейства MCS-51 сгруппированы в четыре 8-разрядных порта P0...P3. Эти порты могут использоваться для функций обмена данными (допуская управление отдельными линиями), так и для выполнения альтернативных функций, таких как обращение к внешней памяти, приём запросов прерываний и др. Порт P0 является двунаправленным и аналогичен порту BUS микроконтроллеров MCS-48. Порты P1, P2, P3 являются квазидвунаправленными и также аналогичны портам P1, P2 семейства MCS-48. Чтобы вывести значение «0» или «1» на линию квазидвунаправленного порта, необходимо записать это значение в соответствующий разряд буферного регистра, а для использования этой линии в качестве входа её регистр должен содержать «1».

При обращении к внешней памяти используются порты P0 и P2. Через порт P0 выдаётся младший байт адреса, который должен быть зафиксирован во внешнем регистре по сигналу ALE, так как затем на линиях P0 появляются данные для записи или чтения. Через порт P0 выдаётся старший байт адреса, внешняя фиксация которого не требуется.

Порт P1 предназначен для задания младшего байта адреса при программировании и проверке РПП микроконтроллера. При этом старший байт адреса передаётся через порт P2, а байт кода команды – через порт P0.

Линии порта P3 могут быть использованы для альтернативных функций, которые требуются в сложных системах управления:

- P3.7 – строб чтения из внешней памяти данных RD;
- P3.6 – строб записи во внешнюю память данных WR;
- P3.5 – вход тестируемого сигнала T1;
- P3.4 – вход тестируемого сигнала T0;
- P3.3 – вход внешнего прерывания INT1;
- P3.2 – вход внешнего прерывания INT0;
- P3.1 – выход последовательного порта TxD;
- P3.0 – вход последовательного порта RxD.

Альтернативные функции могут быть задействованы путём записи «1» в соответствующие биты выходного буфера P3.0...P3.7.

## Система прерывания MCS-51

Базовые модели микроконтроллеров 8051 имеют пять источников прерываний – два внешних (входы INT0, INT1) и три внутренних (от двух таймеров/счётчиков и один от последовательного порта). С появлением микроконтроллеров типа 8052 число источников прерываний увеличилось до 15 (в модели 8XC51GP). Адреса соответствующих векторов прерываний 0003h..0023h расположены в нижней области памяти программ (Рис.118). Поскольку интервал между адресами векторов прерываний составляет всего 8 байт, то по этим адресам располагается короткая процедура, либо команда JMP, обеспечивающая переход на подпрограмму обслуживания прерывания (Рис.122).

Внешние прерывания INT0 и INT1 могут быть вызваны либо уровнем, либо переходом сигнала из «1» в «0» (в зависимости от значений управляющих бит IT0 и IT1 в регистре TCON). В обоих случаях прерывание устанавливает флаги IE0 и IE1 в регистре TCON, которые инициируют вызов соответствующей подпрограммы. Если прерывание вызвано уровнем, то для исключения повторных прерываний сигнал INT0 или INT1 должен стать пассивным до завершения подпрограммы обслуживания. Эта операция должна выполняться источником прерывания автоматически, в противном случае обработчик прерываний должен снять низкий уровень с линий INT через какую-либо линию портов. Кроме того, перед возвратом из прерывания необходимо сбросить флаги IE0 и IE1. Во втором случае (прерывание по срезу импульса INT) сброс этих флагов выполняется аппаратно и дополнительных операций не требуется.

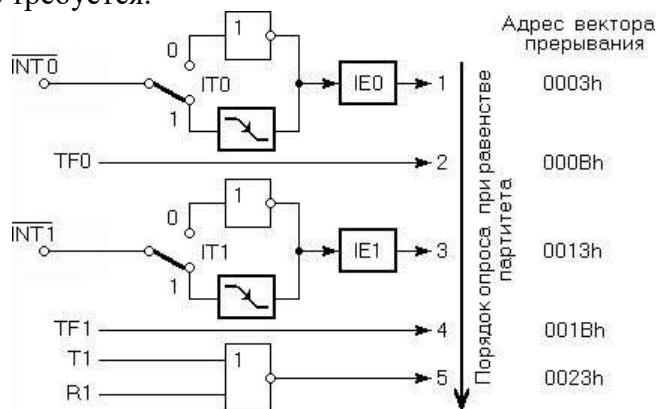


Рис.122 - Система прерываний MCS-51

*Прерывания от таймеров* инициируются при установке их флагов переполнения TF0 и TF1 в регистре TCON. Эти флаги сбрасываются автоматически при передаче управления подпрограмме обслуживания.

*Прерывания от последовательного порта* возникают в двух случаях: при загрузке байта во входной буфер (в режиме приёма) или освобождение выходного буфера (в режиме передачи). При этом аппаратно устанавливается один из флагов RI или TI регистра SCON, а конкретный источник определяется процедурой обслуживания посредством опроса. Сброс этих флагов выполняется программно.

Любое из прерываний может быть инициировано программно, путём установки соответствующего флага: IE0, IE1, TF0, TF1, RI, TI. В результате будет вызвана та же подпрограмма, что и при установке этих флагов аппаратными средствами.

Каждый из источников прерываний может быть разрешен или запрещён при помощи соответствующего бита регистра маски прерываний IE, формат которого приведен на Рис.123. В этом регистре также имеется бит общего запрета прерываний от всех источников EA.

При появлении запросов от нескольких источников очередность их обслуживания определяется двухступенчатым механизмом приоритетов. На первой ступени любому источнику можно присвоить «высокий» («1») или «низкий» («0») уровень приоритета. Для этого используются соответствующие биты регистра приоритетов IP (Рис.124). На второй ступени очередность обслуживания определяется адресом вектора прерывания, т.е. запросы с меньшим адресом обслуживаются в первую очередь. Если какое-либо прерывание уже обслуживается, то его процедуру может прервать только прерывание с более высоким приоритетом.

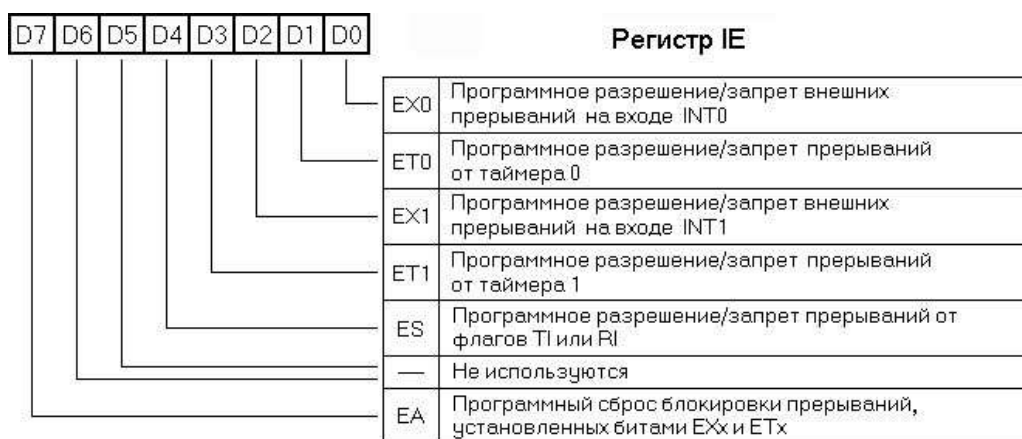


Рис.123 - Регистр маски прерываний IE

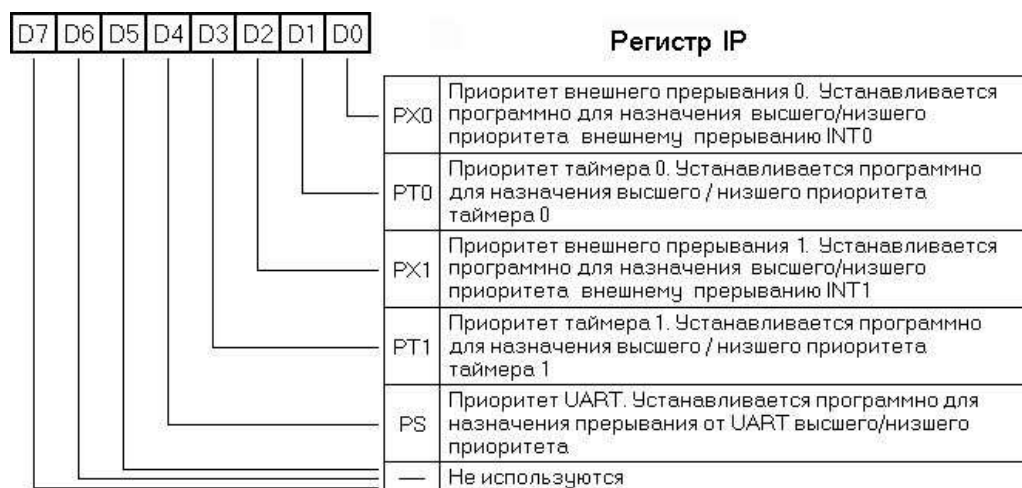


Рис.124 - Регистр приоритетов прерываний IP

Между запросом и началом обслуживания прерывания проходит не менее трёх машинных циклов. Обработка запроса может быть отложена, если:

- обслуживается запрос равного или более высокого уровня приоритета;
- текущий машинный цикл не является последним в цикле команды;
- выполняется команда RETI или любая команда, связанная с обращением к регистрам IE или IP.

Реакция на запрос, разрешённый к обслуживанию заключается в передаче управления от текущей программы специальной процедуре обслуживания прерывания. Для этого аппаратно формируется команда LCALL (заметим, что её код в программе не содержится). По команде LCALL текущее содержимое счётчика команд (адрес возврата) сохраняется в стеке. Затем в счётчик команд загружается адрес вектора соответствующего источника прерывания, по которому располагается команда JMP. Сохранение в стеке других регистров микроконтроллера (A, B, PSW и т.д.) осуществляется программно командами PUSH в начале процедуры обслуживания. В конце этой процедуры должны выполняться соответствующие процедуры восстановления из стека POP. Подпрограммы обслуживания прерывания завершаются командой возврата RETI, по которой восстанавливается из стека прежнее содержимое счётчика команд. Заметим, что команда возврата из подпрограммы RET также восстанавливает содержимое счётчика команд, но не снимает блокировку прерываний.

## Таймеры/счётчики

Микроконтроллеры MCS-51 имеют два 16-разрядных таймера/счётчика Timer0 и Timer1, каждый из которых независимо может быть запрограммирован на работу в качестве либо таймера, либо счётчика. В режиме таймера производится подсчёт импульсов внутреннего тактового генератора, что позволяет формировать временные интервалы. В режиме счётчика производится подсчёт внешних импульсов на входах T0 и T1, которые соответствуют каким-либо событиям. В обоих случаях переполнение таймеров/счётчиков приводит к формированию запроса прерывания.

При работе в режиме таймера его содержимое инкрементируется в каждом машинном цикле, т.е. через каждые 12 периодов тактовой частоты. При работе в качестве счётчика инкрементирование осуществляется при переходе из «1» в «0» внешнего входного сигнала на входах T0, T1. Так как на распознавание перехода требуется два машинных цикла, то максимальная частота подсчёта входных сигналов равна 1/24 тактовой частоты. Для надёжной работы значение «1» на входах T0, T1 должно удерживаться не менее одного машинного цикла.

При использовании таймеров программисту доступны старшие байты счётчиков TH0, TH1, их младшие байты TL0, TL1, а также их флаги переполнения TF0, TF1. Для управления режимами работы и для организации взаимодействия таймеров с системой прерывания используются два регистра специальных функций: TMOD (регистр режима таймеров) и TCON (регистр управления таймерами). Формат этих регистров и назначение отдельных разрядов приведены на Рис.125 и Рис.126. Таймеры/счётчики могут работать в четырёх режимах, причём режимы 0, 1 и 2 одинаковы для обоих таймеров, а режим 3 различается.

В режиме 0 таймер/счётчик имеет разрядность 13, причём старший байт THx используется полностью, а из младшего байта TLx используются только 5 разрядов. Этот режим аналогичен таймеру микроконтроллеров MCS-48, где имеется 8-разрядный счётчик с 5-разрядным предделителем (Рис.127), однако частота переполнения таймера в MCS-51 примерно равна 8 мс.

Режим 1 аналогичен режиму 0, но используются все 16 разрядов регистров THx и TLx (Рис.128). Поэтому максимальное значение счётчика равно 65535, а максимальный период переполнения таймера – около 65 мс.

В режиме 2 регистр TLx работает как 8-разрядный счётчик (Рис.129). После переполнения и установки флага TFx этот счётчик автоматически перезагружается значением из регистра THx, которое предварительно было задано программно. Перезагрузка не изменяет значение THx. Такая организация позволяет легко реализовать деление частоты на любой коэффициент в диапазоне от 1 до 256.

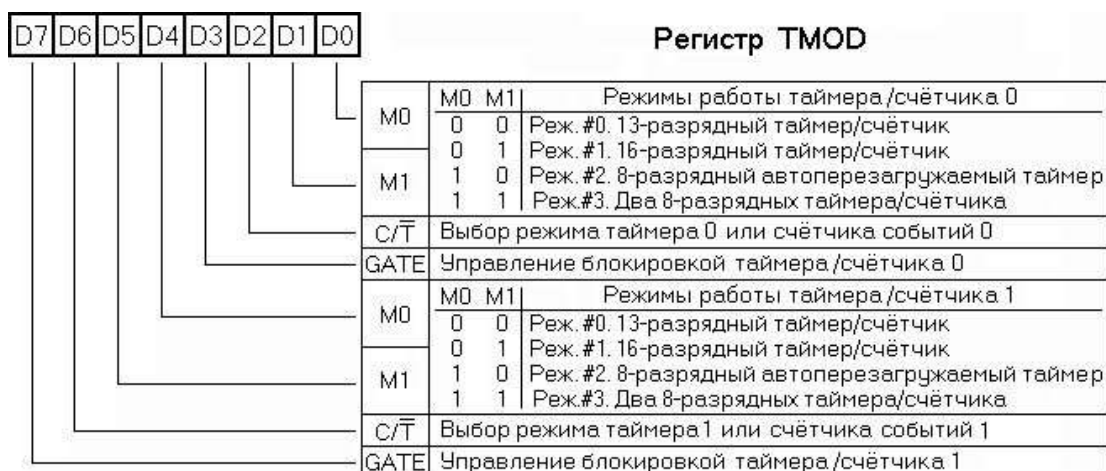


Рис.125 - Регистр режима таймеров/счётчиков TMOD

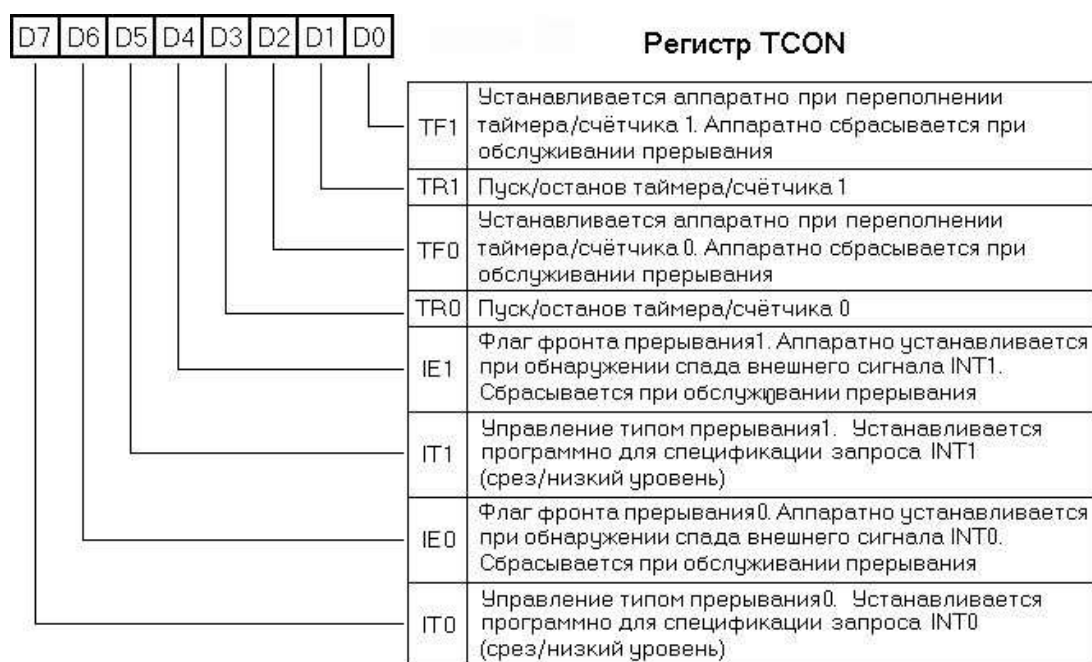


Рис.126 - Регистр управления таймеров/счётчиков TCON

В режиме 3 таймеры работают по-разному: Timer1 останавливается (эквивалентно TR1 = 0), а Timer0 разделяется на два независимых 8-разрядных счётчика TH0 и TL0 (Рис.130). При этом биты управления нулевого таймера/счётчика используются для TL0, а биты управления первого таймера/счётчика – для TH0, который может выполнять только функции таймера. Такой режим используется, если в системе управления требуется три таймера/счётчика.

В семействе MCS-51 отсутствуют специальные команды запуска и останова таймеров/счётчиков. Вместо этого используется биты TR0, TR1 («1» – запуск, «0» – останов). При этом бит GATEx (блокировка) должен быть сброшен. Если GATEx = 1, то запуском таймеров/счётчиков управляет внешний вывод запроса прерывания INTx. Это позволяет аппаратно измерять длительность импульсов на входах прерываний.

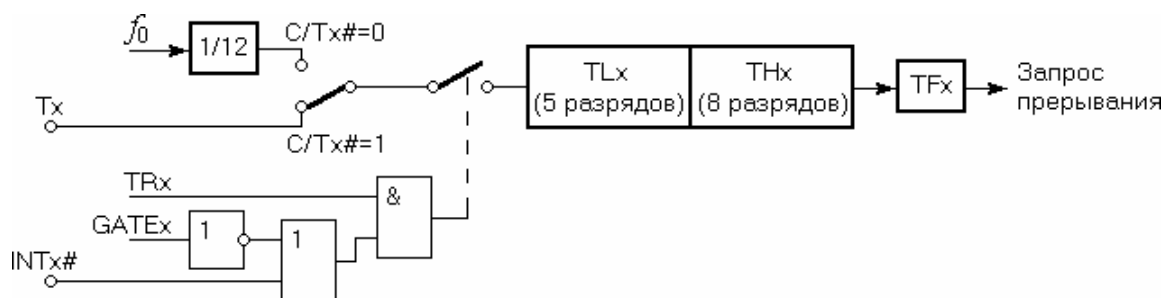


Рис.127 - Таймер/счётчик в режиме 0

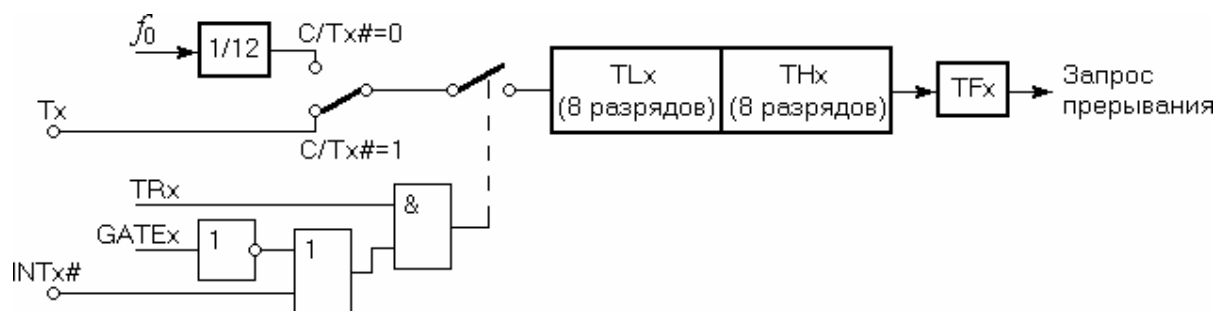


Рис.128 - Таймер/счётчик в режиме 1

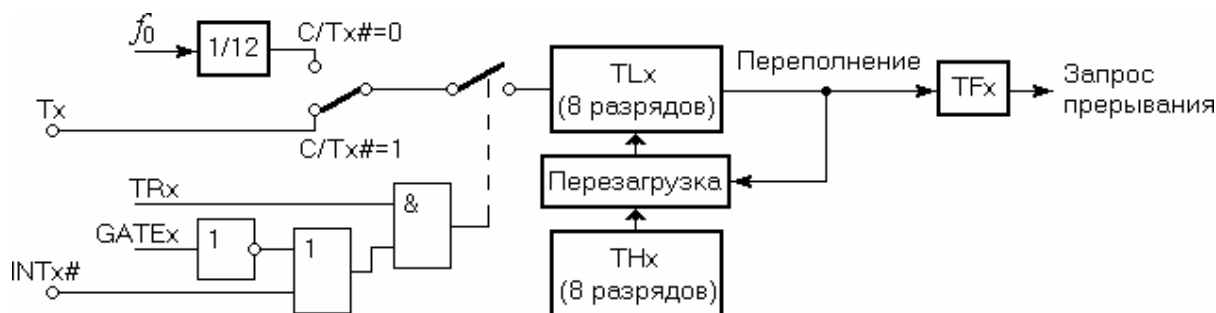


Рис.129 - Таймер/счётчик в режиме 2

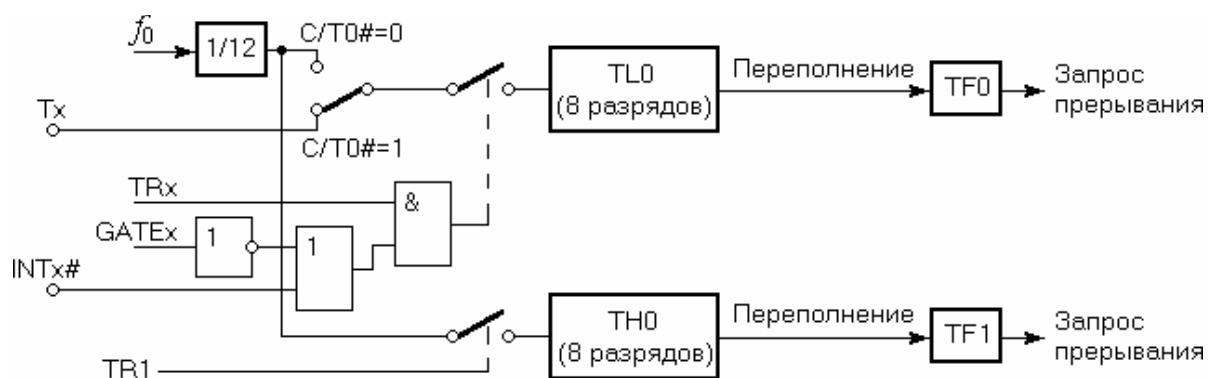


Рис.130 - Таймер/счётчик в режиме 3

На базе двух таймеров можно также измерить частоту внешних импульсов, подаваемых на вход T0. Для этого Timer0 переводится в режим счётчика и включается на определённый интервал времени (например, 1 мс), который формирует Timer1. В результате в регистры TH0, TL0 записывается частота измеряемого сигнала в килогерцах.

## Последовательный интерфейс

Последовательный порт микроконтроллеров MCS-51, называемый также UART (Universal Asynchronous Receiver/Transmitter), является дуплексным и обеспечивает работу в синхронном, либо асинхронном режимах. Его можно также использовать для обмена дан-

ными в системах управления на базе нескольких микроконтроллеров. Этот интерфейс поддерживает протокол обмена RS-232-C, используемый во многих промышленных системах управления.

В состав UART входят регистры сдвига приёмника и передатчика, а также буферный регистр SBUF. Для управления последовательным портом используется регистр SCON (Рис.131), а также старший бит регистра управления энергопотреблением PCON.7, который позволяет задать удвоенную скорость передачи. Они позволяют установить режим работы (0 – синхронный; 1, 2, 3 – асинхронный), а также задать скорость передачи. В обоих случаях данные передаются, начиная с младшего бита.

В режиме *синхронного* обмена (режим 0) по линии TxD выдаётся восемь синхроимпульсов, а по линии RxD принимается или передаётся байт данных. Поэтому линии TxD и RxD одного контроллера соединяются с соответствующими линиями другого контроллера (Рис.132). Передача очередного бита осуществляется каждый машинный цикл, поэтому обмен производится с фиксированной частотой  $f_0 / 12$ . Это максимально возможная скорость обмена через последовательный порт микроконтроллера.

При *асинхронном* обмене данные передаются по линии TxD, а принимаются по линии RxD. Поэтому линия TxD одного контроллера подключается к линии RxD другого, и наоборот (перекрёстное соединение). В этом случае используется старт/стопный принцип, при котором каждый передаваемый байт дополняется стартовым и стоповыми битами, что позволяет начать передачу в любой момент времени. Приём и передача могут осуществляться одновременно, поскольку регистр SBUF состоит из двух частей – буфера приёмника и буфера передатчика. В режиме ожидания на линиях TxD удерживается высокий уровень, поэтому переход из “1” в “0” воспринимается как начало передачи старт-бита.

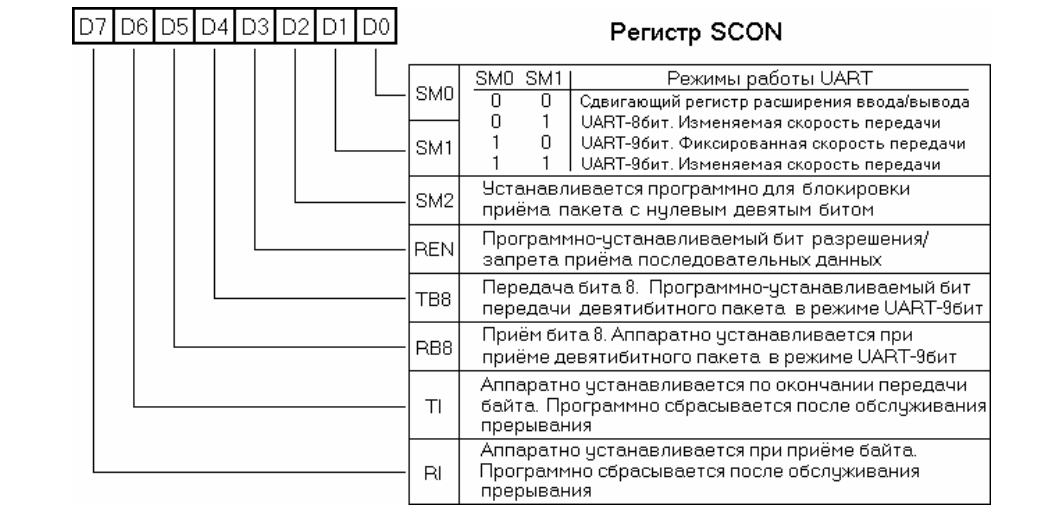


Рис.131 - Регистр управления режимами последовательного порта SCON

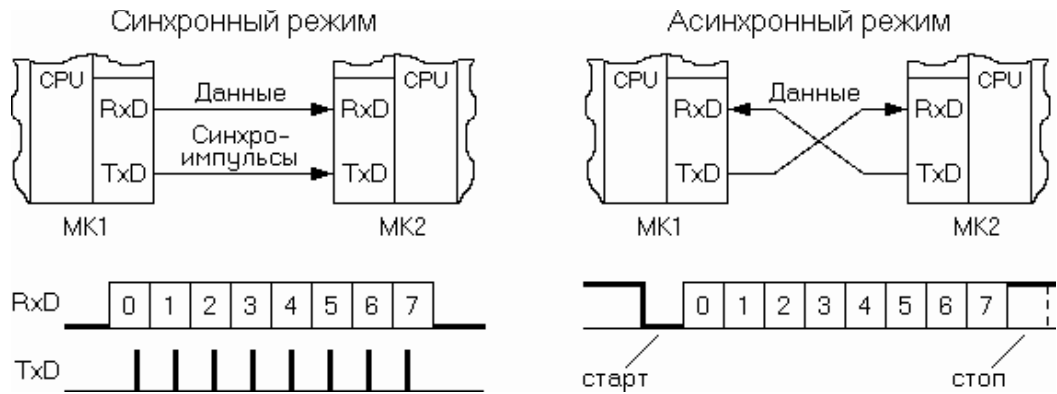


Рис.132 - Соединение микроконтроллеров при последовательной передаче данных



Чтобы передать данные необходимо записать байт в регистр SBUF, а для приёма информации необходимо прочитать содержимое этого регистра. После загрузки SBUF передаваемый байт копируется в регистр сдвига передатчика и побитно выдвигается на линию TxD. При приёме осуществляется обратная операция. Наличие буферного регистра приемника позволяет совмещать операцию чтения ранее принятого байта с приемом очередного байта. Если к моменту окончания приема байта предыдущий байт не был считан из SBUF, то он будет потерян.

Последовательный интерфейс может работать в четырех различных режимах.

В *режиме 0* осуществляется передача данных в синхронном режиме. Данные передаются и принимаются по линиям RxD, а по линиям TxD – синхросигналы. Передача начинается любой командой, по которой в SBUF поступает байт данных. Затем он копируется в сдвигающий регистр передатчика и в каждом машинном цикле сдвигается вправо (младшими битами вперёд), после чего очередной бит поступает на вывод RxD. В освобождающиеся старшие биты сдвигающего регистра передатчика записываются нули. При обнулении этого регистра устанавливается флаг TI (конец передачи) и линия RxD переводится в высокоуровневое состояние. При приёме этого байта в микроконтроллере должен быть сброшен флаг RI и установлен бит REN (разрешение приёма). После окончания приёма всех восьми бит устанавливается флаг RI (конец приёма), после чего принятый байт может быть прочитан из регистра SBUF.

В *режиме 1* осуществляется асинхронная передача данных. При этом пакет данных состоит из 10 бит (Рис.132): старт-бит («0»), 8 бит данных и стоп-бит («1»). При приёме в разряд RB8 регистра SCON записывается стоп-бит. Для передатчика в регистре SCON необходимо сбросить бит REN, а для приёмника – установить его.

Передача инициируется загрузкой SBUF передатчика. Приём начинается при обнаружении перехода сигнала на входе RxD приёмника из состояния 1 в состояние 0, причём вход RxD опрашивается 16 раз за время передачи старт-бита. Если при этом обнаруживается значение, не равное 0, то приемник вновь возвращается к поиску перехода из 1 в 0. Такой механизм обеспечивает подавление ложных (сбойных) старт-бит.

Далее, при приёме каждого из информационных битов и стоп-бита, вход RxD опрашивается трижды (в состояниях 7, 8 и 9 внутреннего счётчика с коэффициентом деления 16). Это позволяет настроить приёмник на середину периода передачи бита и повысить помехоустойчивость за счет использования мажоритарного принципа (принятым значением считается то, которое было получено по меньшей мере дважды из трех раз). Если последний принятый бит равен 1, то приёмник устанавливает флаг RI, в противном случае принятый байт теряется и вновь начинается поиск переход из 1 в 0 на входе RxD. Потеря принятого байта также происходит, если из регистра SBUF не был прочитан предыдущий байт, т.е. не был сброшен флаг RI. Это должно выполняться программно.

В *режимах 2 и 3* также осуществляется асинхронный обмен, но пакет данных состоит из 11 бит: старт-бит («0»), 8 бит данных, программируемый девятый бит и стоп-бит («1»). При передаче девятый бит записывается в разряд TB8 регистра SCON (перед загрузкой регистра SBUF инициирующей передачу). При приёме девятый бит может быть прочитан из разряда TB8 регистра SCON. Программист может использовать девятый бит по своему усмотрению, например, для контроля по чётности. Алгоритмы работы приёмника и передатчика в этих режимах аналогичны режиму 1, но имеются некоторые отличия в условиях окончания цикла приёма. Приём считается успешным, если принятый стоп-бит равен «1» и из регистра SBUF был прочитан предыдущий байт (сброшен флаг RI). Кроме того, если установлен разряд SM2 регистра SBUF, то независимо от состояния флага RI принимается и загружается в SBUF байт данных, который сопровождается «1» в девятом бите. Если при этом предыдущий байт не прочитан, то он теряется. Последний случай используется для организации обмена в многоконтроллерных системах.

Скорость обмена в асинхронном режиме определяется частотой переполнения первого таймера  $f_T$  либо частотой синхросигнала  $f_0$ . В режиме 1, 3 скорость обмена равна  $f_T / 16$

или  $f_T / 32$ , а в режиме 2 –  $f_0 / 32$  или  $f_0 / 64$ . При этом выбор конкретного значения скорости определяется флагом SCON – старшим разрядом регистра PCON. Например, если первый таймер настроить на режим 2 и использовать константу перезагрузки  $F_{416} = 244_{10}$ , то при  $f_0 = 12$  МГц этот таймер будет переполняться каждые  $256 - 244 = 12$  мкс. Это соответствует частоте  $f_T = 1/12$  мкс  $\approx 83,3$  кГц, откуда скорость передачи равна 2,6 кбит/с (при SCON = 0). Поэтому при использовании кварцевого резонатора с частотой **11,059** МГц удастся получить стандартное значение скорости передачи 2400 бит/с. Общая формула вычисления скорости передачи имеет вид:

$$f = \frac{f_0 / 12}{32 \cdot (256 - c)} \quad \text{при SMOD} = 0;$$

$$f = \frac{f_0 / 12}{16 \cdot (256 - c)} \quad \text{при SMOD} = 1;$$

где  $c$  – константа перезагрузки таймера. Очевидно, что при  $f_0 = 12$  МГц максимальная скорость передачи достигает 62,5 кбит/с в режимах 1, 3, а режиме 2 – 375 кбит/с (для сравнения, в синхронном режиме 0 скорость составляет 1 Мбит/с).

Режимы 2, 3 позволяют построить многоконтроллерную систему, в которой обмен осуществляется через витую пару или коаксиальный кабель. При этом один контроллер должен быть ведущим (master), а остальные – ведомыми (slave). Поскольку команды ведущего контроллера должны восприниматься ведомыми, то выход master-передатчика должен быть подключен ко входу slave-приёмников и наоборот (Рис.133).

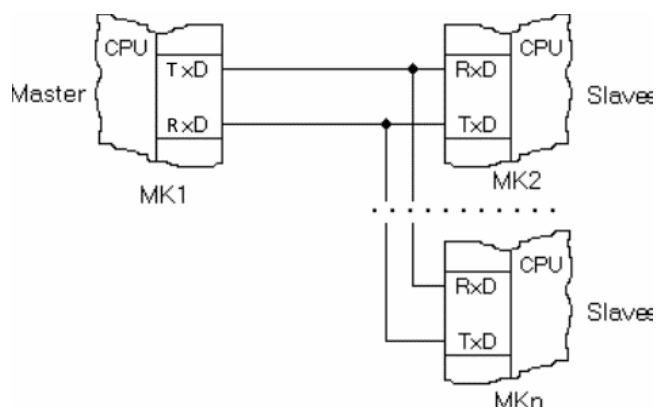


Рис.133 - Многоконтроллерная система

Команды ведущего контроллера передаются всем подчинённым контроллерам, но воспринимаются только тем контроллером, которому они адресованы. Для этого используется девятый бит, позволяющий отличать адресную информацию от данных. Обычно при передаче адреса девятый бит равен «1», а при передаче данных и команд – «0». Временная диаграмма работы многоконтроллерной шины приведена на Рис.134.

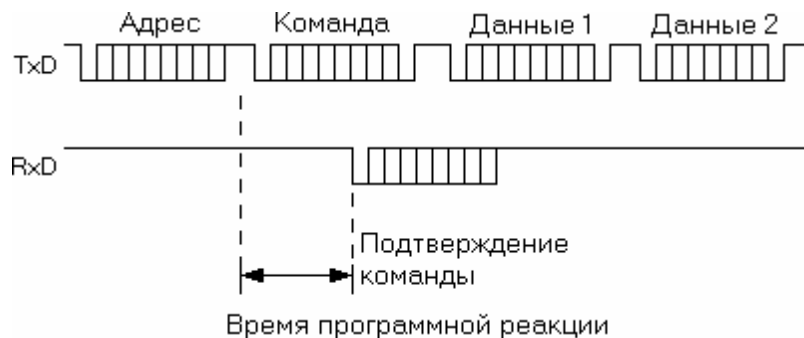


Рис.134 - Временная диаграмма многоконтроллерной системы

Если ведущему микроконтроллеру требуется передать блок данных, то он в режиме «широковещательной» передачи (всем ведомым) сначала выдаёт код адреса получателя (в его девятом бите содержится «1»). При получении адреса во всех ведомых микроконтроллерах происходят прерывания и вызов подпрограммы сравнения полученного адреса с кодом собственного сетевого адреса. Адресуемый микроконтроллер сбрасывает свой управляющий бит SM2 в «0» и готовится к приёму блока данных. Остальные ведомые микроконтроллеры, адрес которых не совпал с принятым, оставляют неизменным состояние SM2 = 1 и передают управление прерванной программе. В дальнейшем байты данных, поступающие в последовательный порт ведомых микроконтроллеров у которых SM2= 1, прерывание не вызывают (не устанавливается флаг RI), т.е. игнорируются. Этот процесс продолжается до тех пор, пока ведущий микроконтроллер не передаст новый адрес получателя данных.

## Управление энергопотреблением

У микроконтроллеров семейства MCS-51, выполненных по технологии CMOS (80C51), существуют два режима, позволяющих уменьшить потребление энергии в пассивном состоянии:

- режим холостого хода IDL (Idle), ток потребления 4,2 мА.
- режим пониженного энергопотребления PD (Power Down), ток потребления 50 мкА;

## Сброс в начальное состояние и синхронизация

В отличие от MCS-48, сброс микроконтроллеров семейства MCS-51 осуществляется путём подачи на вход RESET сигнала «1». Для надёжности, этот сигнал должен быть удержан на входе RESET по меньшей мере в течение двух машинных циклов (24 периода резонатора).

Под воздействием сигнала RESET сбрасывается содержимое счётчика команд PC, аккумулятора ACC и его расширителя B, слова-состояния PSW, регистра-указателя DPTR, а также регистра специальных функций TMOD, TCON, THx, TLx, IE, IP и SCON. Начальное состояние остальных регистров определяется следующим образом: в регистре PCON сбрасывается только старший бит SMOD, в указатель стека SP загружается код 07h, а в буферы портов P0...P3 – код FFh. Состояние регистра SBUF не определено. Сигнал RESET не воздействует на содержимое ячеек РПД. Когда включается электропитание (V<sub>CC</sub>), содержимое РПД неопределенно, за исключением операции возврата из режима пониженного энергопотребления.

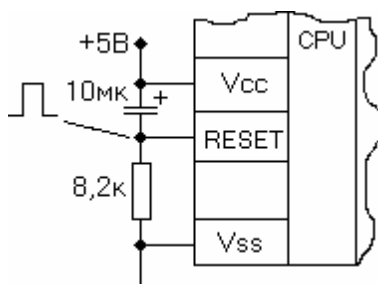


Рис.135 - Схема сброса при включении электропитания

На Рис.135 показана схема автоматического формирования сигнала RESET при включении электропитания. Время, необходимое для полного заряда ёмкости, обеспечивает уверенный запуск резонатора и его работу в течение более чем двух машинных циклов.

Синхронизация микроконтроллеров (Рис.136) семейства MCS-51 осуществляется от внешнего тактового генератора или внутреннего генератора с внешним кварцевым резонатором, подключаемым к выводам XTAL1 и XTAL2. Отметим, что в NMOS-кристаллах сигнал внешнего генератора подаётся на вывод XTAL2, а в CMOS – на вывод XTAL1.

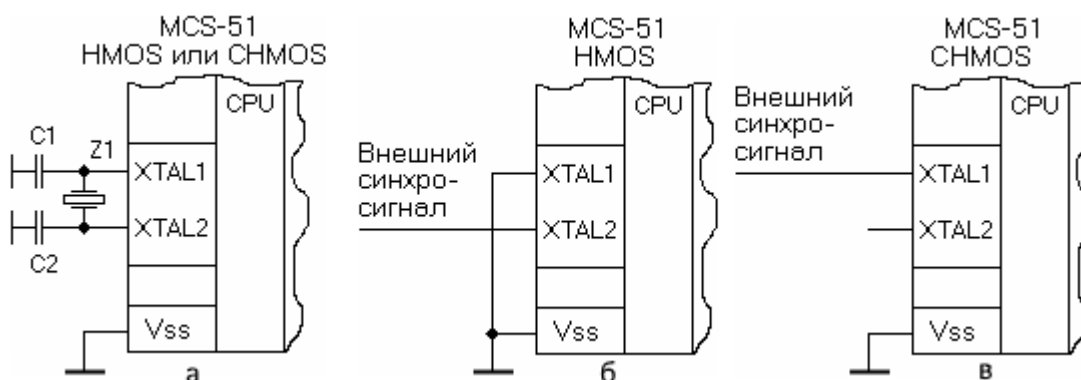


Рис.136 - Синхронизация контроллера: а – от внутреннего генератора; б, в – от внешнего генератора

Внутри микроконтроллера частота синхросигнала делится на 12, образуя машинный цикл. Машинный цикл состоит из шести последовательных состояний S1...S6, каждое из которых делится на две фазы P1 и P2. При описании последовательности тактов используются обозначения S1P1, S1P2, S2P1,...,S6P2. Цикл выполнения каждой команды состоит из одного, двух или четырёх машинных циклов.

## Система команд MCS-51

Система команд семейства MCS-51 содержит 111 базовых команд, которые подразделяются на пять групп:

- команды пересылки данных (Data Transfer);
- арифметические команды (Arithmetic);
- логические команды (Logic);
- команды ветвления (Program Branching);
- команды операций с битами (Boolean Variable Manipulation).

В отличие от MCS-48 в системе команд MCS-51 присутствуют команды умножения, деления, вычитания, операций над битами, операций со стеком и расширенный набор команд передачи управления.

Команда может иметь длину один, два или три байта, причем первый байт содержит код операции, а второй и третий - адреса операндов либо непосредственные операнды. Большинство команд (94 из 111) состоят из одного или двух байт.

Время выполнения команды составляет один, два или четыре машинных цикла. При тактовой частоте 12 МГц длительность машинного цикла равна 1 мкс, при этом 64 команды выполняются за 1 мкс, 45 команд - за 2 мкс и 2 команды (умножение и деление) - за 4 мкс. Интересно, что в отличие от MCS-48, здесь нет прямой связи между длиной команды и числом машинных циклов, требуемых для ее выполнения.

В командах MCS-51 используется четыре типа операндов - биты, тетрады (4 бита), байты и 16-битные слова. Однобитный операнд может содержаться в любом из 128 флагов РПД, а также в отдельных разрядах буферов ввода/вывода и некоторых регистров специальных функций. Четырехбитные операнды используются только при операциях обмена (команды SWAP и XCHD). Восьмибитные операнды могут размещаться в аккумуляторе, регистрах R0...R7, в ячейках памяти программ или данных (резидентной или внешней), в регистрах специальных функций, а также в буферах портов ввода/вывода. Двухбайтные операнды - это константы и прямые адреса, для представления которых используются второй и третий байты команды.

Система команд MCS-51 поддерживает те же методы адресации, что и MCS-48, однако имеет некоторую специфику, обусловленную большим объемом внешней памяти программ и данных. При задании адреса операнда используются следующие методы:

- **непосредственная адресация** - операнд содержится непосредственно в команде и может занимать один или два байта (data<sub>8</sub>, data<sub>16</sub>);
- **прямая адресация** – в команде содержится однобайтный адрес операнда adr<sub>8</sub> (используется только для резидентной памяти данных и регистров специальных функций);
- **регистровая адресация** – в команде имеется трехбитовое поле, определяющее номер регистра, содержащего операнд; выбор одного из четырех регистровых банков осуществляется программированием битов селектора банка (RS1, RS0) в PSW.
- **косвенно-регистровая адресация** - в команде содержится ссылка на регистр, содержащий адрес операнда (используется при обращении к РПД и ВПД через 8-разрядные регистры R0 и R1, указатель стека SP, а также через 16-разрядный регистр-указатель DPTR);
- **индексная адресация** – адрес операнда находится сложением 8–разрядного индекса, содержащегося в аккумуляторе, и 16-разрядной базовой величины, содержащейся в DPTR или PC (используется только при чтении из памяти программ, при выборке из таблиц).
- **неявная адресация** – в команде отсутствует специальное адресное поле, определяющее адрес операнда, но его расположение полностью определяется кодом операции (используется в командах, которые по умолчанию выполняют операции только с аккумулятором или DPTR).

Следует отметить, что, в отличие от других процессоров, преимущества регистровой адресации по времени выполнения команд здесь не столь существенные (за один машинный цикл выполняются и некоторые команды с прямой либо косвенной адресацией).

При программировании на ассемблере для ASM51 обычно используются символические адреса регистров специальных функций и их отдельных бит. Например, ACC.5 – символическое имя пятого бита аккумулятора; SMOD (либо PCON.7) – символическое имя бита удвоенной скорости передачи последовательного интерфейса. Эти символические адреса являются зарезервированными словами ASM51, и их не надо определять с помощью директив ассемблера.

## Команды пересылки данных

Команды этой группы обеспечивают выполнение операций копирования (пересылка байтов и битов между памятью и регистрами процессора). Они делятся на три типа:

- команды пересылки, использующие РПД и РСФ (команды типа MOV);
- команды пересылки, использующие ВПД (команды типа MOVX);
- команды пересылки, использующие ПП (команды типа MOVC).

Эти команды не модифицируют флаги результата, за исключением команд загрузки PSW и аккумулятора (флаг паритета). Большую часть команд данной группы составляют команды передачи и обмена байтов. Команды пересылки бит представлены далее в группе команд битовых операций.

Команды MOV (таблица Рис.137) позволяют пересылать данные между аккумулятором, регистрами общего назначения, ячейками резидентной памяти данных и регистрами специальных функций. В приведенной таблице используются следующие обозначения: src (source) – источник; dest (destination) – приемник; Rn – регистр общего назначения R0...R7; Ri – регистр-указатель адреса при косвенной адресации R0, R1; #ad8 – восьмиразрядный адрес; #d8 – восьмиразрядный непосредственный операнд; #d16 – 16-разрядный непосредственный операнд.

Необходимо заметить, что в отличие от MCS-48 пересылки между ячейками памяти здесь могут выполняться без участия аккумулятора. Кроме того, в зависимости от метода адресации аккумулятор имеет два различных имени: А – при неявной адресации (например, MOV A, R0) и ACC – при использовании прямого адреса. В последнем случае аккумулятор рассматривается как регистр специальных функций с адресом E0h, для которого также допускается побитовый доступ. Использование неявной адресации аккумулятора предпочтительнее, но не всегда возможно (например, при обращении к его отдельным битам).

| Мнемокод       | Операция                                                                   | Операнды                                  |
|----------------|----------------------------------------------------------------------------|-------------------------------------------|
| MOV A, src     | $A \leftarrow \text{src}$                                                  | src = Rn, @Ri, ad8                        |
| MOV dest, A    | $\text{dest} \leftarrow A$                                                 | dest = Rn, @Ri, ad8                       |
| MOV dest, src  | $\text{dest} \leftarrow \text{src}$                                        | src = Rn, @Ri, ad8<br>dest = Rn, @Ri, ad8 |
| MOV DPTR, #d16 | $\text{DPTR} \leftarrow \#d16$                                             | DPTR                                      |
| PUSH src       | $\text{SP} \leftarrow \text{SP} + 1$<br>$@\text{SP} \leftarrow \text{src}$ | src = ad8                                 |
| POP dst        | $@\text{SP} \leftarrow \text{dst}$<br>$\text{SP} \leftarrow \text{SP} - 1$ | dst = ad8                                 |
| XCH A, byte    | $A \leftrightarrow \text{byte}$                                            | byte = Rn, @Ri, ad8                       |
| XCHD A, @Ri    | $\text{Low\_A} \leftrightarrow \text{Low\_byte}$                           | byte = @Ri                                |

Рис.137 - Команды пересылки, использующие РПД и РСФ

В первую группу также входят команды PUSH и POP, обеспечивающие соответственно запись данных в стек и чтение из стека. В MCS-51 стек располагается в резидентной памяти данных, поэтому размер стека ограничен лишь её объемом. При этом следует учитывать, что по сигналу RESET в указатель стека заносится код 07h, поэтому первый элемент стека располагается в ячейке памяти с адресом 08h, используемой для регистра R0 банка RB1. Поэтому в сложных системах после сброса следует переопределить начальное значение стека. Кроме пересылок команды первой группы могут осуществлять обмен байтами между аккумулятором и ячейками РПД (команда XCH), а также обмен младшими тетрадами, т.е. битами 0 - 3 (команда XCHD).

Необходимо отметить, что MCS-51 не имеет специальных команд ввода-вывода и команд для работы с регистрами специальных функций. Доступ к ним осуществляется обычными командами MOV с использованием прямого адреса (допускается также использование символических адресов). Например, пересылка PSW в аккумулятор выполняется командой MOV A, PSW, которая преобразуется ассемблером к виду MOV A, 0D0h. Аналогично, вывод из аккумулятора в порт P0 осуществляется командой MOV P0, A эквивалентной MOV A, 80h. Ещё одна особенность команд типа MOV проявляется в микроконтроллерах с расширенным объемом РПД, где обращение к верхним 128 байтам резидентной памяти данных осуществляется только в режиме косвенной адресации (при использовании прямого адреса выполняется обращение к РСФ).

Команды MOVX (таблица Рис.138) позволяют пересылать данные между аккумулятором и ячейками ВПД. При этом используется косвенная адресация через 16-разрядный регистр-указатель DPTR либо 8-разрядные регистры R0, R1. В первом случае обеспечивается доступ к любой из 65 536 ячеек ВПД, в втором – к странице объемом 256 байт (старший байт адреса, т.е. номер страницы, определяется текущим содержимым буфера порта P2).

| Мнемокод      | Операция             | Разрядность адреса |
|---------------|----------------------|--------------------|
| MOVX A, @Ri   | $A \leftarrow @Ri$   | 8                  |
| MOVX @Ri, A   | $@Ri \leftarrow A$   | 8                  |
| MOVX A, @DPTR | $A \leftarrow @DPTR$ | 16                 |
| MOVX @DPTR, A | $@DPTR \leftarrow A$ | 16                 |

Рис.138 - Команды пересылки, использующие ВПД

Команды MOVC (таблица Рис.139) используются для чтения данных, размещенных в памяти программ (констант и таблиц функций). При обращении к таблице обычно применяется первая из этих команд, причём регистр DPTR устанавливается на точку начала таблицы, а в аккумулятор помещается индекс табличной ячейки. При использовании второй команды следует учитывать, что в момент вычисления суммы  $A+PC$  содержимое счётчика команд PC уже увеличено на 1 (т.е. указывает на следующую команду).

| Мнемокод        | Операция                  |
|-----------------|---------------------------|
| MOVC A, @A+DPTR | $A \leftarrow @ (A+DPTR)$ |
| MOVC A, @A+PC   | $A \leftarrow @ (A+PC)$   |

Рис.139 - Команды пересылки из памяти программ

## Команды арифметических операций

Микроконтроллеры MCS-51 содержат следующие арифметические команды: сложение ADD, сложение с учётом переноса ADC, вычитание с заёмом SUBB, инкремент INC, декремент DEC, десятичная коррекция DA A, умножение MUL и деление DIV (таблица Рис.140). Большинство из них выполняет операции над 8-разрядными операндами (кроме INC DPTR, которая обеспечивает инкремент 16-разрядного регистра-указателя).

| Мнемокод     | Операция                                      | Операнды                 | Флаги        |
|--------------|-----------------------------------------------|--------------------------|--------------|
| ADD A, byte  | $A \leftarrow A + \text{byte}$                | byte = Rn, @Ri, ad8, #d8 | C, AC, OV, P |
| ADDC A, byte | $A \leftarrow A + \text{byte} + \text{flagC}$ | byte = Rn, @Ri, ad8, #d8 | C, AC, OV, P |
| SUBB A, byte | $A \leftarrow A - \text{byte} - \text{flagC}$ | byte = Rn, @Ri, ad8, #d8 | C, AC, OV, P |
| INC byte     | $\text{byte} \leftarrow \text{byte} + 1$      | byte = A, Rn, @Ri, ad8   | —            |
| DEC byte     | $\text{byte} \leftarrow \text{byte} - 1$      | byte = A, Rn, @Ri, ad8   | —            |
| INC DPTR     | $DPTR \leftarrow DPTR + 1$                    | DPTR                     | —            |
| MUL AB       | $BA \leftarrow B \cdot A$                     | A, B                     | C=0, OV      |
| DIV AB       | $A(B) \leftarrow A/B$                         | A, B                     | C=0, OV      |
| DA A         | Двоично- десятичная коррекция                 | A                        | C, AC, OV, P |

Рис.140 - Команды арифметических операций



При выполнении арифметических операций формируется флаги переноса C, дополнительного переноса AC, переполнения OV и паритета P, которые помещаются в регистр PSW (Рис.141). Они формируются следующим образом:

- C = 1, если в старшем бите результата возникает перенос или заем;
- AC = 1, если возникает перенос/заём между тетрадами;
- P = 1, если число единичных бит в аккумуляторе нечетно;
- OV = 1, если старший бит результата не может быть интерпретирован как знаковый (например, при суммировании положительных чисел получен отрицательный результат).



Рис.141 - Формат слова состояния PSW

Отметим, что все четыре флага формируются только при выполнении операций сложения и вычитания, а операции умножения и деления флаг C сбрасывают. При умножении флаг OV устанавливается, если результат больше 255. При делении флаг OV устанавливается при делении на ноль. Особенности формирования флагов C и OV поясняются примерами, приведенными ниже (Рис.142, Рис.143).

$$\begin{array}{llll}
 FF_{16} + FF_{16} = FE_{16} & 7F_{16} + 7F_{16} = FE_{16} & 01_{16} + 01_{16} = 02_{16} & 81_{16} + 81_{16} = 02_{16} \\
 (-1) + (-1) = (-2) & (+127) + (+127) = (-2) & (+1) + (+1) = (+2) & (-127) + (-127) = (+2) \\
 C=1, OV=0 & C=0, OV=1 & C=0, OV=0 & C=1, OV=1
 \end{array}$$

Рис.142 - Формирование флагов C и OV при сложении

$$\begin{array}{llll}
 FF_{16} - 01_{16} = FE_{16} & 7F_{16} - 81_{16} = FE_{16} & 01_{16} - FF_{16} = 02_{16} & 81_{16} - 7F_{16} = 02_{16} \\
 (-1) - (+1) = (-2) & (+127) - (-127) = (-2) & (+1) - (-1) = (+2) & (-127) - (+127) = (+2) \\
 C=0, OV=0 & C=1, OV=1 & C=1, OV=0 & C=0, OV=1
 \end{array}$$

Рис.143 - Формирование флагов C и OV при вычитании

Поскольку в MCS-51 отсутствует команда вычитания без учета заёма, то для выполнения такой операции используется команда SUBB (перед этим программно сбрасывается флаг переноса).

Команда десятичной коррекции DAA позволяет выполнить сложение чисел, представленных в двоично-десятичном коде. Она корректирует 8-битовое число, полученное в результате выполненной ранее команды сложения ADD или ADDC. Если содержимое младшей тетрады аккумулятора превышает 9 или AC=1, то к ней прибавляется 06h, при этом получается младшая двоично-десятичная цифра. Указанное сложение не изменяет содержимое флага AC, но может установить флаг C, если перенос из младшей тетрады распространяется через всю старшую тетраду (в противном случае C не изменяется). Далее, если C=1 или содержимое старшей тетрады превышает 9, то содержимое аккумулятора увели-



чивается на 60h, создавая старшую двоично-десятичную цифру. При этом флаг С устанавливается (не изменяется), если имеется (отсутствует) перенос из старшего бита аккумулятора.

Операция умножения MUL выполняется над содержимым аккумулятора А и регистра В, полученный результат размещается в двух байтах АВ (младший байт в В, старший – в А). Для операции деления DIV в аккумулятор помещается частное, а в регистр В – остаток. При этом надо учитывать, что операнды интерпретируются как 8-разрядные целые числа без знака.

## Команды логических операций

Система команд MCS-51 включает в себя стандартные логические операции И, ИЛИ, исключающее ИЛИ (ANL, ORL, XRL), а также операции инвертирования CPL, сброса CLR, циклического сдвига RL, RLC, RR, RRC и обмена тетрадами SWAP (таблица Рис.144). Операции ANL, ORL, XRL выполняются поразрядно над байтами, располагаемыми в аккумуляторе, в ячейках резидентной памяти данных или регистрах специальных функций. Остальные операции выполняются только над содержимым аккумулятора.

В отличие от MCS-48, в архитектуре MCS-51 здесь имеется возможность производить операцию XRL с содержимым буферов портов P0...P3, что может быть эффективно использовано для инверсии их отдельных линий.

| Мнемокод      | Операция                                                                       | Операнды                 | Флаги |
|---------------|--------------------------------------------------------------------------------|--------------------------|-------|
| ANL A, byte   | $A \leftarrow A \& \text{byte}$                                                | byte = Rn, @Ri, ad8, #d8 | P     |
| ANL ad8, byte | $\text{ad8} \leftarrow \text{ad8} \& \text{byte}$                              | byte = A, #d8            | –     |
| ORL A, byte   | $A \leftarrow A \vee \text{byte}$                                              | byte = Rn, @Ri, ad8, #d8 | P     |
| ORL ad8, byte | $\text{ad8} \leftarrow \text{ad8} \vee \text{byte}$                            | byte = A, #d8            | –     |
| XRL A, byte   | $A \leftarrow A \oplus \text{byte}$                                            | byte = Rn, @Ri, ad8, #d8 | P     |
| XRL ad8, byte | $\text{ad8} \leftarrow \text{ad8} \oplus \text{byte}$                          | byte = A, #d8            | –     |
| CLR A         | $A \leftarrow 00h$                                                             | A                        | P=1   |
| CPL A         | $A \leftarrow \text{NOT}(A)$                                                   | A                        | P=1   |
| RL A          | $A_{k+1} \leftarrow A_k; k=0\div6, A_0 \leftarrow A_7$                         | A                        | –     |
| RLC A         | $A_{k+1} \leftarrow A_k; k=0\div6, A_0 \leftarrow C;$<br>$C \leftarrow A_7$    | A                        | C, P  |
| RR A          | $A_{k+1} \rightarrow A_k; k=0\div6, A_0 \rightarrow A_7$                       | A                        | –     |
| RRC A         | $A_{k+1} \rightarrow A_k; k=0\div6, A_0 \rightarrow C;$<br>$C \rightarrow A_7$ | A                        | C, P  |
| SWAP A        | $A_{7\ldots4} \rightarrow A_{3\ldots0}$                                        | A                        | –     |

Рис.144 - Команды логических операций

## Команды операций над битами

Микроконтроллеры MCS-51 содержат в своем составе «булевый» процессор, имеющий своё побитово-адресуемое ОЗУ, свои порты ввода/вывода и выполняющий свой набор команд. При выполнении этих команд флаг С выполняет функции аккумулятора.

Внутреннее ОЗУ «булевого» процессора имеет объём 256 бит. В него входят 128 прямоадресуемых бит из области РПД (ячейки с адресами 20h-2Fh), а также битовые поля некоторых регистров специальных функций (адреса которых кратны восьми): ACC, В,

P0...P3, TCON, SCON, IE, IP, PSW. Побитно-адресуемыми являются, в частности, все линии ввода-вывода, каждая линия которых может рассматриваться как однобитовый порт. При работе с битами используется только прямая адресация.

Каждый из отдельно адресуемых бит может быть установлен в «1», сброшен в «0», инвертирован, передан во флаг C или принят из него. Между любым прямоадресуемым битом и флагом C могут быть произведены логические операции И, ИЛИ, результат которых заносится во флаг C. Кроме того, по состоянию адресуемого бита можно проверить и выполнить соответствующий условный переход. Список команд операций над битами приведён в таблице Рис.145, где bit обозначает прямой восьмиразрядный адрес.

| Мнемокод    | Операция                                     | Мнемокод | Операция                                       |
|-------------|----------------------------------------------|----------|------------------------------------------------|
| ANL C, bit  | $C \leftarrow C \& \text{bit}$               | CLRC     | $C \leftarrow 0$                               |
| ANL C, /bit | $C \leftarrow C \& \text{NOT}(\text{bit})$   | CLR bit  | $\text{bit} \leftarrow 0$                      |
| ORL C, bit  | $C \leftarrow C \vee \text{bit}$             | SETB C   | $C \leftarrow 1$                               |
| ORL C, /bit | $C \leftarrow C \vee \text{NOT}(\text{bit})$ | SETB bit | $C \leftarrow \text{bit}$                      |
| MOV C, bit  |                                              | CPL C    | $C \leftarrow \text{NOT}(C)$                   |
| MOV bit, C  | $\text{bit} \leftarrow C$                    | CPL bit  | $\text{bit} \leftarrow \text{NOT}(\text{bit})$ |

Рис.145 - Команды побитовой обработки

## Команды передачи управления

В эту группу входят команды условного и безусловного перехода, а также команды вызова подпрограмм и возврата из них (Рис.146). Большинство из них используется прямая адресация, т.е. адрес перехода целиком (или его часть) содержится в самой команде передачи управления.

В зависимости от разрядности указываемого адреса и способа его формирования различают четыре типа команд безусловного перехода:

- **LJMP ad16** - *длинный переход* (в команде содержится полный 16-битный адрес, обеспечивается переход по всему адресному пространству объёмом 64 Кбайт, длина команды – три байта);
- **AJMP ad11** - *абсолютный переход* (в команде содержится 11 младших бит адреса, обеспечивается переход в пределах текущей страницы объёмом 2048 байт, длина команды – два байта);
- **SJMP rel8** - *короткий относительный переход* (в команде задается 8-разрядное смещение адреса относительно содержимого счётчика команд, обеспечивается переход в пределах -128...+127 байт относительно адреса следующей команды, длина команды – два байта);
- **JMP @A+DPTR** - *косвенный переход* (в аккумуляторе задается 8-разрядное смещение адреса относительно DPTR, обеспечивается переход в пределах -128...+127 байт относительно этого регистра, длина команды – один байт).

На практике редко возникает необходимость перехода в пределах всего адресного пространства и чаще используются укороченные команды перехода, занимающие меньше места в памяти программ. При программировании на ассемблере допускается использование мнемонического обозначения JMP, при этом выбор конкретного варианта команды безусловного перехода LJMP, AJMP или SJMP осуществляется компилятором.

| Мнемокод      | Операция                                     | Мнемокод            | Операция                               |
|---------------|----------------------------------------------|---------------------|----------------------------------------|
| LJMP ad16     | Длинный безусловный переход                  | DJNZ Rn, rel8       | DEC Rn и переход, если byte $\neq 0$   |
| AJMP ad11     | Абсолютный безусловный переход               | DJNZ byte, rel8     | DEC byte и переход, если byte $\neq 0$ |
| SJMP rel8     | Короткий безусловный переход (относительный) | CJNE A, byte, rel8  | Переход, если A $\neq$ byte            |
| JZ rel8       | Переход, если A = 0                          | CJNE A, #d8, rel8   | Переход, если A $\neq$ #d8             |
| JNZ rel8      | Переход, если A $\neq 0$                     | CJNE Rn, #d8, rel8  | Переход, если Rn $\neq$ #d8            |
| JC rel8       | Переход, если flagC = 1                      | CJNE @Ri, #d8, rel8 | Переход, если @Ri $\neq$ #d8           |
| JNC rel8      | Переход, если flagC = 0                      | LCALL ad16          | Вызов подпрограммы (длинный)           |
| JB bit, rel8  | Переход, если bit = 1                        | ACALL ad11          | Вызов подпрограммы (абсолютный)        |
| JNB bit, rel8 | Переход, если bit = 0                        | RET                 | Возврат из подпрограммы                |
| JBC bit, rel  | Переход, если bit = 1; затем CLR bit         | RETI                | Возврат из прерывания                  |

**Рис.146 - Команды передачи управления**

Команды условных переходов используют только относительную адресацию, аналогичную SJMP. Они осуществляют ветвление по следующим условиям: аккумулятор содержит нуль (JZ); содержимое аккумулятора не равно нулю (JNZ); перенос равен единице (JC); перенос равен нулю (JNC); адресуемый бит равен единице (JB); адресуемый бит равен нулю (JNB).

Команда DJNZ предназначена для организации циклов, при этом в отличие от MCS-48 в качестве счётчика может использоваться не только регистр, но и любой прямоадресуемый байт (например, ячейка РПД). В процедурах ожидания какого-либо события удобно использовать команду CJNE, обеспечивающую сравнение байта с однобайтным операндом и переход в случае неравенства.

Все команды переходов, за исключением CJNE и JBC, не оказывают воздействия на флаги. Команда CJNE устанавливает флаг C, если первый операнд оказывается меньше второго. Команда JBC сбрасывает флаг C в случае перехода.

Для обращения к подпрограммам используются команды LCALL, ACALL. Эти команды в отличие от команд перехода LJMP, AJMP сохраняют в стеке адрес возврата в основную программу. Для возврата из подпрограммы необходимо выполнить команду RET. Команда RETI отличается от команды RET тем, что разрешает прерывания обслуживаемого уровня.

## Программирование микроконтроллера

Разработка, трансляция программного обеспечения для микроконтроллеров осуществляется в специализированных средах разработки, на обычных компьютерах с универсальной ОС (Windows, Linux), а не на целевой системе. В среду разработки обычно входят текстовый редактор с подсветкой синтаксиса, транслятор (компилятор) с ассемблера и других языков программирования, редактор связей (компоновщик), который собирает отдельные объектные модули и библиотеки, в исполняемый двоичный файл.

Получившийся загрузочный файл (обычно в формате HEX) загружается во внутреннее ПЗУ MCS системы, используя специальное аппаратное и программное обеспечение

(программатор). В оригинальных i8048/i8051 на кристалле размещалось ПЗУ с ультрафиолетовым стиранием.

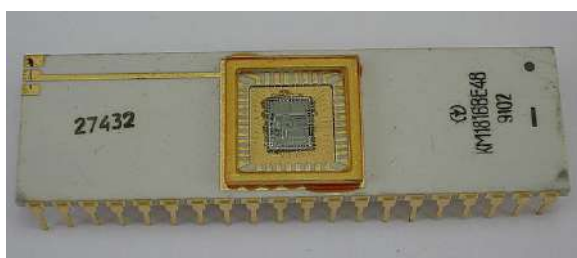


Рис.147 – МК KM1816BE48 с окошком для стирания содержимого ПЗУ

Запись в такие МК производилась с помощью программатора, и была довольно медленной – запись нескольких килобайт занимала десятки секунд и даже несколько минут. Перед записью ПЗУ должно было быть очищено (стёрто), процедура стирания осуществлялась засветкой источником УФ прозрачного окошка (Рис.147), расположенного на верхней поверхности микросхемы. Окошко было выполнено из кварцевого стекла, прозрачного для УФ, излучение попадало на кристалл и приводило к рассасыванию сохранённых зарядов под «плавающим» затвором n-МОП транзисторов. Время стирания было ещё более продолжительным, чем время записи – для надёжного гарантированного стирания содержимого ПЗУ необходимо было светить на кристалл не менее 10-15 мин. Количество циклов стирания-записи было небольшим, несколько десятков, дальше ячейки ПЗУ «изнашивались», не обеспечивая надёжного хранения записываемых в них данных.

С развитием технологий ПЗУ с УФ стиранием было заменено сначала на EEPROM – электрически стираемое ПЗУ, а потом на всем нам сегодня хорошо знакомую flash-память.

Из-за очень примитивной системы команд, малого количества методов адресации, ограниченных ресурсов и т.п. для МК семейства MCS-48 единственным реальным языком программирования был ассемблер. В MCS-51 ресурсов в системе значительно прибавилось, сильно выросли объёмы ПЗУ (до 64 Кбайт), ОЗУ (внутреннее ОЗУ увеличилось до 128...256 байт, внешнее ОЗУ – до 64 Кбайт), появилось много новых мощных методов адресации, в частности, в системе команд появились *относительные*<sup>1</sup> методы адресации операндов. Это позволило писать программы для MCS-51 в т.н. «позиционно-независимом коде» (PIC), в котором работа участка кода не зависит от абсолютных адресов, в которых размещается код. Этот подход очень удобен при создании библиотек, когда заранее неизвестно, в какие конкретные адреса будет загружен тот или иной модуль.

Учитывая это (расширенные методы адресации) и существенно увеличившиеся ресурсы системы, в первую очередь 3У, для MCS-51 оказалось возможным разработать кросс-компиляторы с языков высокого уровня: Паскаль, С, BASIC и др. Если для MCS-48 основным и единственным языком программирования был ассемблер, то для i8051 основным языком разработки программ был язык С. Кроме того, объёмы 3У и архитектура 51-го позволили создавать для него даже компактные операционные системы (по сути мониторы), которые брали под своё управление аппаратные ресурсы микроконтроллера, давая возможность программисту-разработчику сконцентрироваться на алгоритмах, не погружаясь при этом глубоко в «железо».

<sup>1</sup> **Относительная** адресация – при *относительной адресации* для получения исполнительного адреса операнда содержимое поля *адреса* команды складывается с содержимым счётчика команд. Т.е., исполнительный адрес вычисляется как «расстояние» (смещение) относительно РС.



## ГЛАВА 6. МИНИ ЭВМ DEC PDP-11/VAX-11

В самом начале 1970-хх компания DEC<sup>1</sup> выпустила на рынок новое семейство своих миникомпьютеров – PDP-11. Это семейство было развитием 12-битного PDP-8, запущенного в производство в 1964, вполне заслуженно считающегося первым *серийно* выпускавшимся «миникомпьютером», из-за довольно компактных размеров (с небольшой шкаф, см.Рис.148) и невысокой, по тем временам, цены (около 18 тыс. долл. США (1965) в базовой комплектации); благодаря отличному соотношению цены и производительности, этот компьютер смог составить достойную конкуренцию знаменитым мэйнфреймам IBM, и к 1968г. было собрано около 1450 машин (не считая поздних многочисленных модификаций).

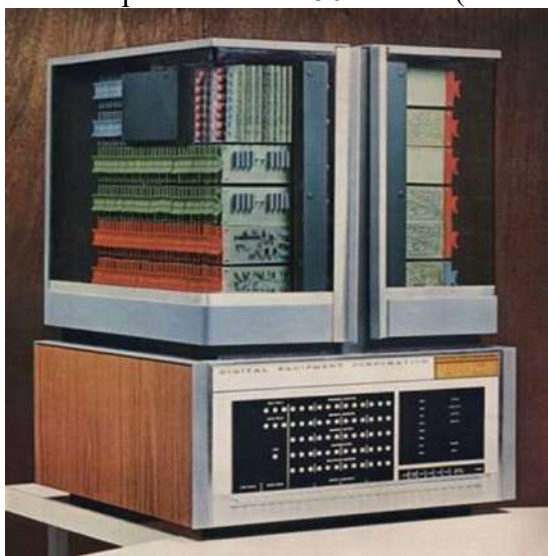


Рис.148 – МиниЭВМ PDP-8 компании DEC

Тогда же, в 1968-ом, была представлена 36-битная PDP-10, основанная на дизайне экспериментальной PDP-6, рассчитанной на использование в центрах обработки данных, исследовательских лабораториях, и в военных целях; модификации PDP-10 производились вплоть до 1983. Работы над усовершенствованием 36-битной архитектуры продолжались в рамках проекта «Единорог» (the Unicorn project), под руководством Леонарда Хьюджеса (Leonard Hughes) и Дэвида Роджерса (David Rogers), но в июне 1975 проект был закрыт, и все ресурсы были переведены на поддержку другой, 32-битной, архитектуры – «Звезда» («Star»), которая в итоге стала знаменитыми VAX-11.

16-битные PDP-11, запущенные в производство в начале 1970-х, были первыми компьютерами DEC, в котором использовались 8-битные байты<sup>2</sup>, и прямыми наследниками линейки PDP-8. Благодаря простой и удачной архитектуре, основанной на системной шине Unibus (или её модификации, Q-bus<sup>3</sup>), довольно эффективному набору инструкций, и, что немаловажно, невысокой стоимости, линейка PDP-11 очень быстро обрела популярность. Вполне закономерно, что PDP-11 вскоре стали широко клонировать: СМ-4 (СССР, Болгария, Венгрия), СМ-1420 (СССР, Болгария, ГДР), СМ-1600 (СССР), ИЗОТ-1016 (Болгария), ДБК (СССР). Для PDP-11 было разработано множество операционных систем: DEC предлагала P/OS, RSX-11, RT-11, RSTS/E, также было доступно несколько вариантов DOS, многопользовательская/многозадачная ОС TSX-Plus<sup>4</sup>, и в конце-концов, на PDP-7 (начало) и PDP-11 (релиз) в 1971 было сделано первое издание ОС UNIX (в Bell Laboratories), тогда ещё на ассемблере. Архитектура PDP-11 ушла с рынка в течение 1980-х по одной, но неизбежной причине: недостаточный объем адресного пространства, знаменовавший переход на новую,

<sup>1</sup> **Digital Equipment Corporation (DEC)** — американская компьютерная компания, была основана в 1957 году Кеном Олсеном и Харланом Андерсоном (*Harlan Anderson*). В год основания, когда DEC только появилась на свет (1957-ой), на компьютерном рынке абсолютно доминировала IBM, они были своего рода «богами всемогущими». Видимо, чтобы не вызвать подозрения «небожителей» из IBM, что DEC сможет когда-то дорасти до конкуренции с ними, DEC даже свои миникомпьютеры назвала максимально нейтрально – **Programmed Data Processor (PDP)**.

<sup>2</sup> Когда-то байты были не только восьмиразрядными, PDP-11 и последующие поколения компьютеров постепенно приучили нас, что байт – это только восемь бит, и со временем, при упоминании байта, приставку его разрядности перестали озвучивать.

<sup>3</sup> Дальше в курсе будет рассмотрена архитектура микроЭВМ LSI-11, основанной на шине Q-bus.

<sup>4</sup> **TSX-Plus** (первоначально **TSX**) — операционная система, разрабатывалась с конца 1970-х годов американской компанией *S&H Computer Systems* для компьютеров семейств PDP-11 и LSI-11. Являясь расширением (надстройкой) операционной системы RT-11, TSX-Plus предоставляла возможности многопользовательской, многозадачной работы с существенным расширением возможностей RT-11XM.

32-битную, но все ещё CISC, архитектуру VAX-11. Хотя в мире и сейчас есть места, где до сих пор исправно работают компьютеры семейства LSI-11, например, в системах управления ядерными реакторами на АЭС в Канаде.

Следующей DEC-ской архитектурой стала **VAX (Virtual Address eXtension)**, официально утвержденная на заседании VAX Architecture Committee в апреле 1975. Детали архитектуры разрабатывались в течение нескольких месяцев в рамках проекта «Звезда» (the Star project) под руководством Гордона Белла (Gordon Bell), параллельно с вышеупомянутым проектом «Единорог», и по окончании работ над обоими проектами было решено отменить дальнейшее развитие 36-битных систем, и сконцентрировать ресурсы на поддержке 32-битных VAX-ов. Фактическим результатом работы проекта «Звезда» было обоснование необходимости как увеличения разрядности основных регистров PDP-11 до 32 бит, так и их числа (с 8 до 16), а также существенный пересмотр набора инструкций. В октябре 1977 был анонсирован первый представитель линейки – модель VAX-11/780, а в феврале 1978 была выпущена новая операционная система для VAX-ов, VMS (Virtual Memory System) версии 1.0; к слову, новая ОС, многопользовательская и многозадачная, поддерживала до 64Мб оперативной памяти, встроенные сетевые возможности (DECnet), адаптивный планировщик задач, расширенное управление процессами, и многие другие функции, значительно опередившие свое время. В апреле 1980 вышла версия 2.0, с многочисленными улучшениями, переименованная в VAX/VMS. На VAX также была вскоре портирована классическая UNIX. VAX-ы выпускались и продавались с большим успехом в течение 1980-х, и даже на рубеже веков поставлялись ограниченными партиями по спецзаказам. Модельный ряд насчитывал несколько десятков наименований: от довольно компактных рабочих станций до шестипроцессорных серверов мэйнфрейм-класса. И по сей день тысячи VAX-ов работают в структуре Министерства обороны США и NSA (National Security Agency), а также во многих коммерческих организациях. Но тем не менее, эпоха VAX-ов осталась в 1980-х, а в 1990-х компания DEC сделала ставку на новую архитектуру.

## **Семейство PDP-11/LSI-11**

**PDP-11** — серия 16-разрядных мини-ЭВМ компании DEC, серийно производившихся и продававшихся в 1970—80-х годах, это было развитием серии PDP-8 из общей линейки компьютеров PDP. В PDP-11 появилось несколько уникальных технологических инноваций, эта серия была проще в программировании, чем её предшественники. Но, несмотря на её всеобщее признание со стороны программистов, PDP-11 со временем была вытеснена персональными компьютерами, включая IBM PC и Apple II.

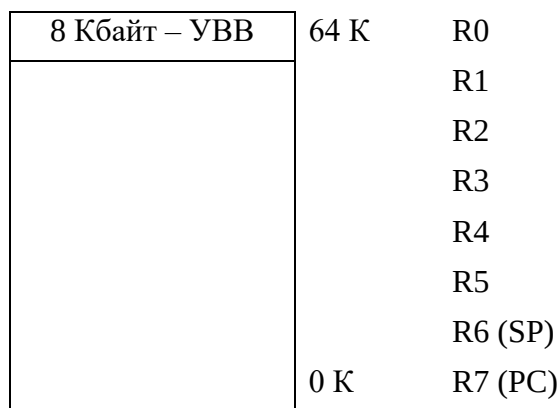
В СССР силами нескольких министерств выпускались клоны как миниЭВМ PDP-11, так и программно-совместимых с PDP-11 микроЭВМ семейства LSI-11 («Электроника-60»). Копии компьютеров выпускались как максимально похожие на оригинал, включая дюймовый шаг ламелей (контактов) печатного разъёма, так и имевшие только программную совместимость модели, с другими механическими и электрическими характеристиками. Были и разработки, даже опережавшие DEC-кие оригиналы по некоторым параметрам, в частности, наша «Кремниевая Долина» (г.Зеленоград) в начале 1980-х объявил программу выпуска LSI-11 совместимых процессоров, которая звучала примерно так – «всё, что DEC делает на одной плате, мы будем воспроизводить на одном кристалле» (микропроцессоры 1801BM1, 1801BM2, 1801BM3/4).

Дальнейший материал основан на технической документации процессора M2 («Электроника-60»), который выпускался в г.Воронеже, на заводе «Процессор». Это вариант, который был максимально похож на оригинальный DEC-кий LSI-11/03, скопированный с точностью до разводки печатной платы процессора, хотя с точки зрения «геометрии» всё-таки в «Электронике-60» совместимость с прообразом была не совсем неполной.

## Архитектура LSI-11

МикроЭВМ LSI-11 строились на базе асинхронной шины Q-bus (варианты названия: Q-22, МПИ (межмодульный параллельный интерфейс), «канал ЭВМ» - Рис.149). Q-bus была более простой и менее производительной версией шины Unibus, использовавшейся в миникомпьютерах PDP-11.

Программная модель младших LSI-11 очень простая, она представлена ниже на рисунке:



PDP-11 можно назвать канонической реализацией неймановской архитектуры. В частности, «благодаря» фон Нейману, для PDP-11 удавалось создавать удивительно красивый и компактный самомодифицирующийся код, правда разобраться в том, как «это работает», часто бывало весьма и весьма непросто.

Ресурсы архитектуры: ОЗУ (RAM) объёмом 64 Кбайт ( $2^{16}$  байт), из этого адресного пространства последние (старшие) 8 Кбайт отведены под адресацию устройств ввода/вывода. Восемь регистров общего назначения (РОН). Два регистра со старшими номерами имеют специализацию: R6 – SP (указатель стека), R7 – PC (счётчик команд, автоматически инкрементирующийся на +2 регистр-указатель, указывающий на адрес следующей инструкции<sup>1</sup>).



Рис.149 - Блок-схема "Электроники-60" (LSI-11/03)

## Q-bus

**Q-bus** (также известная как **LSI-11 Bus**) — одна из разновидностей шин, применяемых в компьютерах PDP-11 и MicroVAX фирмы Digital Equipment Corporation.

Q-bus являлась удешевленным вариантом шины Unibus и использовала мультиплексирование, так что линии данных и адреса использовали те же самые контакты и проводники. Это позволяло как уменьшить размер, так и удешевить конструкцию, при сохранении практически той же функциональности.

Спустя некоторое время адресное пространство шины было увеличено с 16 до 18 и далее до 22 бит. Также были добавлены режимы пересылки блоков данных.

<sup>1</sup> В архитектуре PDP-11 инструкция всегда имеет длину 16 бит и должна быть выровнена на чётную (словную) границу. Поэтому PC инкрементирует адрес всегда на +2.

В Q-bus было использовано 38 линий, из которых 31 линия была двунаправленная (схемы с «открытым коллектором» или с тремя состояниями выхода, использовались TTL-уровни сигналов, инверсная логика – низкий уровень соответствовал логической «1»).

## Технические особенности

Так же, как и Unibus, Q-bus использует:

- Ввод-вывод с отображением на память
- Адресацию с точностью до байта
- Строгие отношения главный-подчинённый на шине
- Асинхронный протокол взаимодействия

*Ввод-вывод с отображением на память* означает, что при обмене данными между любыми двумя устройствами на шине, такими, как ЦП, память, порты ввода-вывода, используются одни и те же протоколы. На шине Unibus специальный блок адресов был выделен для устройств ввода-вывода. В Q-bus это соглашение упростили, введя дополнительный сигнал (изначально назывался *BBS7 Bus Bank Select 7*, позднее получил более общее название *BBSIO, Bus Bank Select I/O*, в МПИ — *BH BB* или *BY*), который выбирает блок адресов, предназначенных для устройств ввода-вывода.

*Адресацию с точностью до байта* означает, что минимальной адресуемой единицей на шине является 8-битный байт. Операция чтения на шине всегда происходит пословно по чётному адресу ( $AD0=0$ ), и ненужный байт просто отбрасывается. При записи байта адресная линия  $AD0$  используется специальным образом, в качестве индексной — для указания смещения в передаваемом слове к передаваемому байту. При  $AD0=0$  байт передаётся по линиям  $AD0—AD7$  и должен быть записан в младший байт адресуемого слова, тогда как при  $AD0=1$  байт передаётся по линиям  $AD8—AD15$  и должен быть записан в старший байт адресуемого слова. Существует специальная линия — *WTBT* (на МПИ — *БАЙТ* или *ПЗП*), обозначающая, что происходит операция записи только одного байта слова. При записи целого слова ( $WTBT=0$ ) состояние адресной линии  $AD0$  игнорируется. При работе программы все 16-битные операции (операции со словом) могут обращаться только по чётным адресам, а 16-битная операция по нечётному адресу, в зависимости от типа процессора, или вызывает прерывание особого вида (ошибка шины), или осуществляется по чётному адресу.

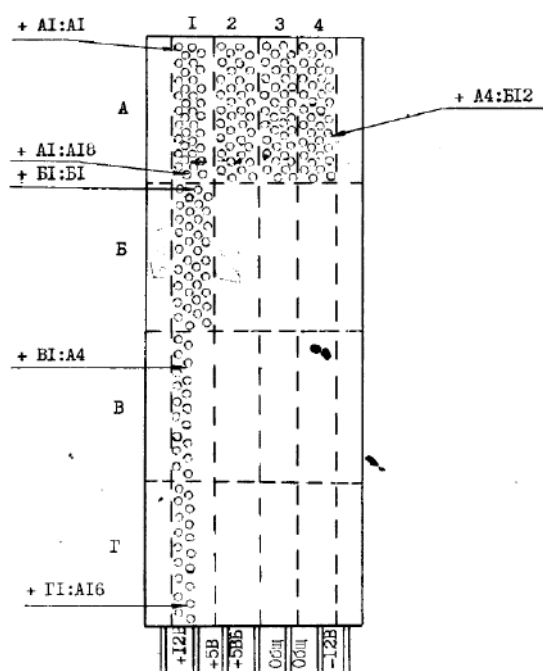
*Строгие отношения главный-подчинённый на шине* означает, что в каждый текущий момент времени только одно устройство может быть в состоянии *Главный* (*Master* или *Ведущий*) на шине. *Ведущее* устройство инициирует операцию на шине и ему отвечает максимум одно *ведомое* устройство. Ведущее устройство может инициировать любую операцию — чтение или запись. После окончания цикла шины механизм *арбитража* выбирает новое устройство, которое будет главным на шине в следующем цикле.

*Асинхронный протокол взаимодействия* означает, что длина цикла шины не зафиксирована во времени и не привязана ни к какому синхросигналу; продолжительность каждого отдельного цикла на шине определяется исключительно взаимодействием *Ведущего* и *Ведомого* (*Master* и *Slave*) устройств в текущем цикле. Эти устройства используют специальные сигналы готовности (*RPLY* или *СИП*) для управления длительностью цикла шины. Кроме того, специальная логика *Ведущего* устройства ограничивает максимальную длительность цикла для предотвращения зависания (канальный таймаут).

В зависимости от поколения, Q-bus содержала 16, 18 или 22 линии адреса/данных *BDAL (Bus Data/Address Line)*. При этом, в цикле шины 16, 18 или 22 линии использовались в фазе адреса для передачи физического адреса, а затем 8 или 16 младших линий использовались для передачи данных в фазе передачи данных. В некоторых системах по старшим линиям в фазе передачи данных передавались контрольные разряды или биты чётности. Системы нового поколения могли поддерживать *блочный режим* работы шины, когда после фазы передачи адреса следовала одна или более фаз передачи данных (при этом данные должны были располагаться в последовательных адресах). Поскольку шина Q-bus мульти-



плексирована и при передаче адреса пересылка данных невозможна, блочный режим позволял уменьшить количество фаз адреса и дать больше времени на передачу данных, позволяя увеличить пропускную способность шины.



**Рис.150 - Backplane шины Q-Bus**

Вот так – Рис.150 – выглядела задняя объединительная панель (backplane) машин с шиной Q-bus (многослойная печатная плата).

На Рис.151 приведено условное обозначение платы процессора M2 (LSI-11/03), в правом верхнем углу видны пять прямоугольников увеличенного размера – это многокристальный микропроцессор с системой команд PDP-11. Существовали варианты без пятой секции микропроцессора, в этом случае в системе команд отсутствовали команды расширенной арифметики и арифметики данных в формате с фиксированной запятой.

Синими стрелками отмечено место упомянутой выше неполной «геометрической» совместимости оригинальных DEC-ких машин и клонов, выпускавшихся в СССР: в «корзину» «Электроники-60» можно вставлять платы двух типоразмеров – одинарные (занимавшие позиции А:Б или В:Г – Рис.150) и двойные – занимавшие позиции А:Б:В:Г. Платы «двойной ширины» на самом деле по площади были больше суммарной площади

двух одинарных плат, за счёт зазора (промежутка) между «столбцами» Б и В. Несовместимость с американским оригиналом была как раз в величине зазора Б:В – у оригинала он был более узким (меньше зазора А:Б или В:Г), а в отечественном клоне – более широким, равным ширине А:Б или В:Г. Эта разница приводила к тому, что платы одинарной ширины были взаимозаменяемыми (в отечественном клоне был выдержан дюймовый шаг ламелей разъёма) – американскую плату можно было без проблем вставить в отечественную «корзину» и, наоборот – отечественная плата успешно работала в американской «корзине». Но вот с платами двойной ширины такого взаимобмена достичь было невозможно, из-за разницы в размерах среднего зазора Б:В.

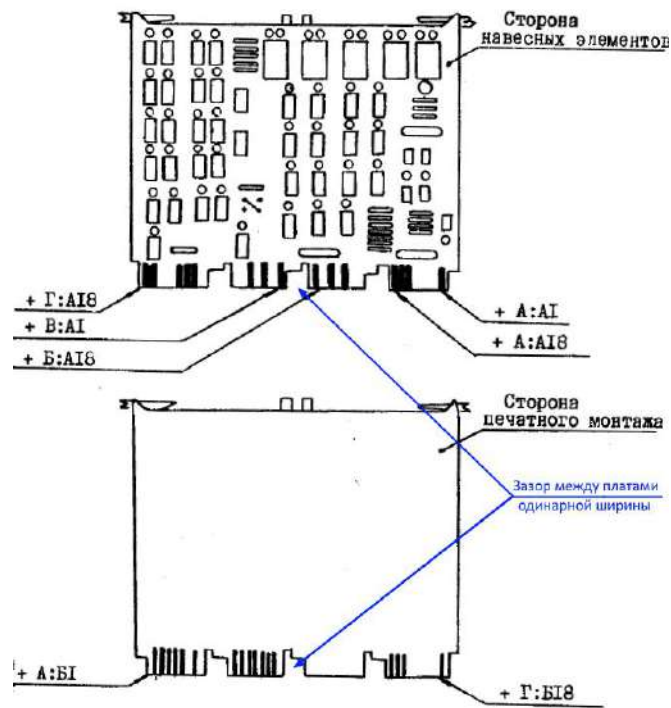


Рис.151 - Плата процессора M2

В шине Q-bus было реализовано пять основных типов циклов обмена:

- ВВОД (Чтение)
- ВЫВОД (Запись)
- ВВОД-ПАУЗА-ВЫВОД
- Предоставление прямого доступа в память (DMA)
- Прерывание

### Цикл ВВОД

Направление передачи определяется по отношению к активному устройству, т.е., ВВОД означает «ввод данных» (чтение) в активное устройство (н-р, процессор).

В шине Q-bus используется инверсная логика (TTL), т.е. более низкий уровень сигнала соответствует логической «1» (Рис.152).

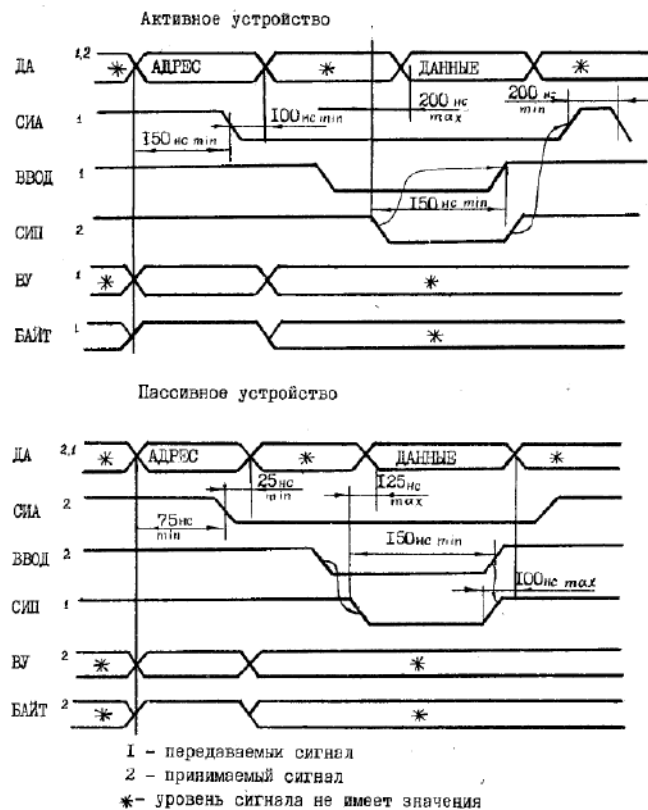


Рис.152 - Временная диаграмма цикла ВВОД

Активное устройство начинает цикл с анализа занятости шины. Признаком незанятости является неактивный уровень сигнала CIA (синхронизация активного устройства) и линий DA (адреса/данных).

Выяснив, что шина свободна, активное устройство выставляет на адресные линии адрес (16/18/22 разрядов в зависимости от модели) ячейки ОЗУ/регистра УВВ. После этого активное устройство, после задержки минимум ~150нс (чтобы на линиях закончились переходные процессы) активирует сигнал CIA, срезом которого всем, слушающим шину, сообщается, что на шине присутствует актуальный адрес (по срезу этот адрес фиксируется в регистрах-защёлках). После среза CIA, минимум через ~100нс, активное устройство снимает адрес с линий A/DA, переводя каналные передатчики в высокоимпедансное (третье) состояние, освобождая шестнадцать линий DA для передачи по ним данных.

Освободив линии DA, активное устройство активирует сигнал ВВОД, тем самым сообщая всем, кто слушает шину, о чтении ячейки/регистра по адресу, переданному в первой фазе цикла. Если ячейка памяти/регистр с таким адресом присутствует в системе и понимает команду ВВОД (чтение), это устройство (ОЗУ/ВВ) должно ответить, активировав сигнал СИП (синхронизация пассивного устройства) на двунаправленной линии («монтажное ИЛИ»). Если адрес не существует (программная ошибка или неисправность устройства) и никто не сгенерировал ответ (СИП), активное устройство, не дождавшись СИПа в течении «канального таймаута» (~10...15мкс – зависит от модели процессора), генерирует «ошибку обращения к каналу» или «обращение по несуществующему адресу». Если были разрешены прерывания, то по каналному таймауту произойдёт прерывание с адресом вектора 4.

Написанное выше означает, что в Q-bus на *канальном* уровне определяется наличие/отсутствие адреса ОЗУ/УВВ.

Если ОЗУ/УВВ с таким адресом присутствует в системе, сигналом СИП пассивное устройство: а) оповещает о своём наличии; б) выдаёт одновременно с ним данные на линии DA.

Увидев ответ СИП, *активное* устройство принимает (читает) данные с линий DA и, минимум через ~150нс, снимает сигнал ВВОД, оповещая шину о завершении этапа чтения.

Увидев снятие сигнала ВВОД, *пассивное* устройство снимает в ответ СИП и переводит линии DA в третье состояние, в ответ на снятие СИПа *активное* устройство снимает сигнал СИА. На этом цикл ВВОД завершается, шина свободна.

Асинхронность шины здесь проявляется в том, что ответить (сигналом СИП) *пассивное* устройство может с любой задержкой, не превышающий канальный таймаут, т.е. время выполнения цикла определяется более медленным устройством в паре «ведомый-ведущий». ОЗУ могло отвечать с задержкой в сотни нс, медленные УВВ – с задержкой на порядок больше, до ~10...15мкс.

## Цикл ВЫВОД

Цикл ВЫВОД (запись) (Рис.153) практически по временной диаграмме аналогичен циклу ВВОД, кроме направления передачи данных – здесь оно производится **от** активного устройства **к** пассивному. Пассивное устройство точно также должно ответить на «свой» адрес И «свою» команду (ВЫВОД) сигналом СИП («монтажное ИЛИ»).

Для примера: если активное устройство обратится к ПЗУ, то пассивное устройство (ПЗУ) ответит СИПом (и выдаст данные) при обращении к нему с командой ВВОД, но «промолчит» при попытке выполнить в него запись (ВЫВОД), в результате чего произойдет та самая «ошибка обращения к каналу» или «обращение по несуществующему адресу».

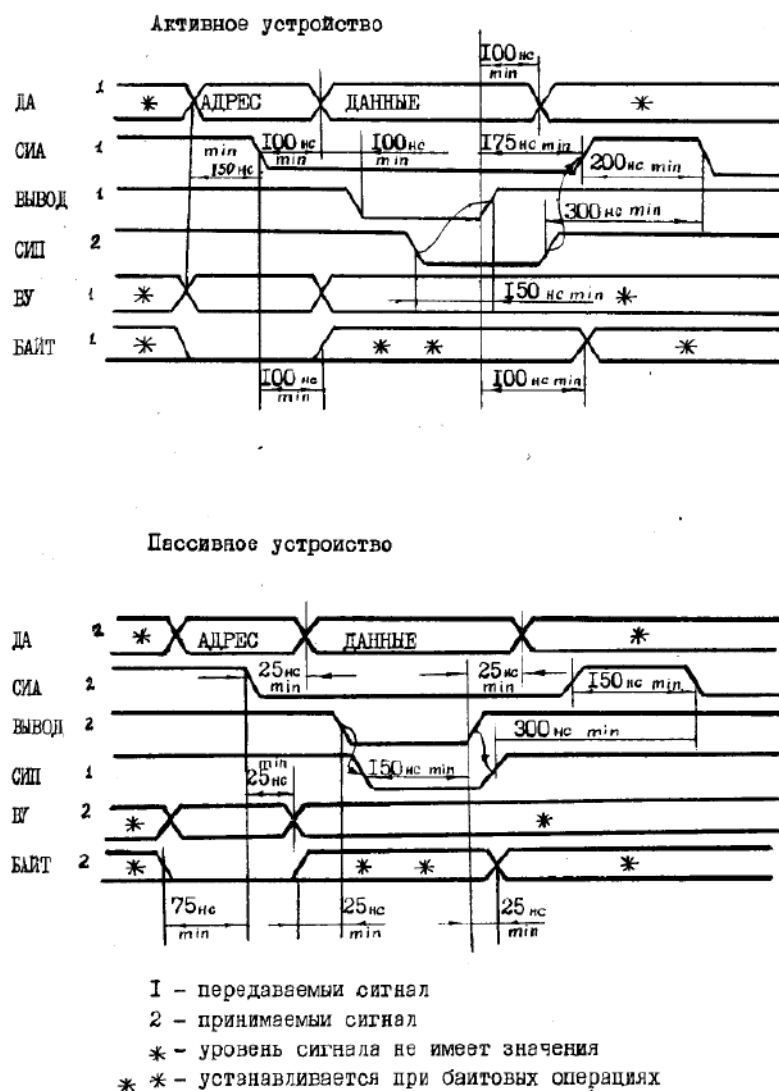


Рис.153 - Временная диаграмма цикла ВЫВОД

## Цикл ВВОД-ПАУЗА-ВЫВОД

Цикл ВВОД-ПАУЗА-ВЫВОД (Чтение-Модификация-Запись) (Рис.154) представляет собой своего рода объединение циклов ВВОД/ВЫВОД, в котором обе операции – чтения и записи – выполняются по одному и тому же адресу. Это означает два момента: а) экономится время на повторную (не)передачу того же самого адреса; б) операция атомарная, между ВВОДом (чтением) и ВЫВОДом (записью) никакое другое активное устройство не сможет модифицировать содержимое, хранящееся по этому адресу.

Сгенерировать цикл ВВОД-ПАУЗА-ВЫВОД может, к примеру, одноадресная инструкция вроде INC @R4 – инкрементировать (+1) содержимое ячейки памяти, адрес которой содержится в регистре R4 – сначала процессор читает содержимое по адресу, на который указывает R4, прибавляет к этому содержимому единицу и записывает в память по тому же самому адресу.

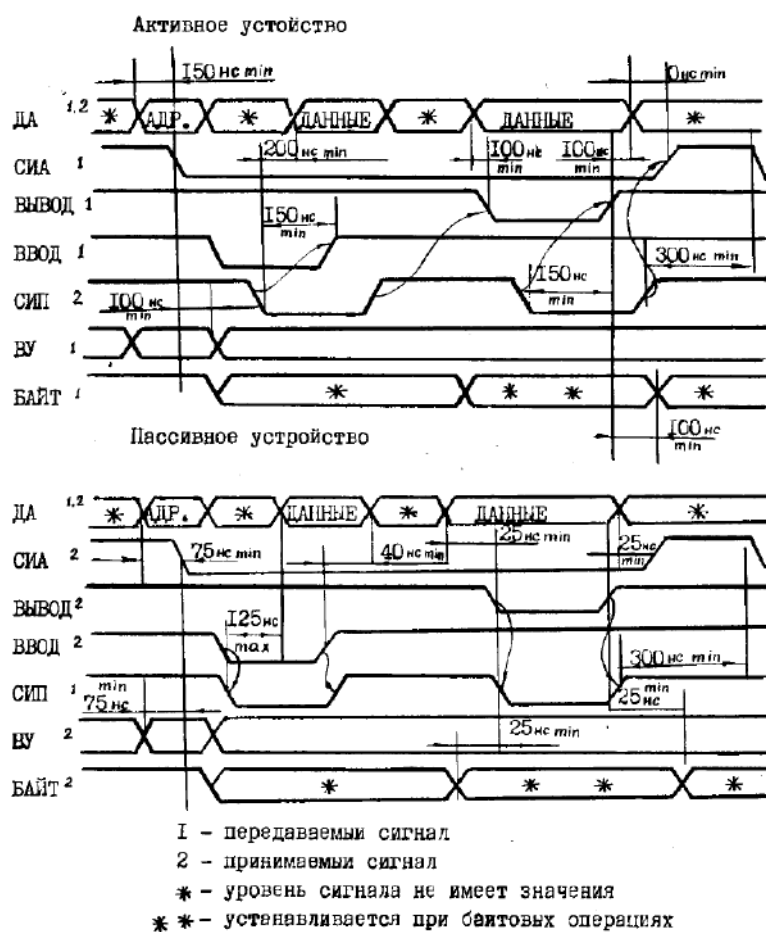


Рис.154 - Временная диаграмма ВВОД-ПАУЗА-ВЫВОД

Обратите внимание, что во время выполнения цикла ВВОД-ПАУЗА-ВЫВОД пассивное устройство генерирует ответ СИП дважды – в первый раз в ответ на команду ВВОД, второй раз – в ответ на ВЫВОД.

Из ранее не описанных сигналов необходимо упомянуть БАЙТ – признак байтовой (8-разрядной операции) и ВУ – признак обращения к старшим 8 Кбайт адресного пространства, по сути это 3И для 16-ти разрядного адреса (5И для 18-ти и 9И для 22-х, соответственно).

## Прямой доступ в память

Шина Q-bus поддерживает более одного активного устройства на шине. Альтернативное активное устройство может стать «хозяином шины» (bus master), захватив её, после



- Если устройство вообще не поддерживает работу по прямому доступу, то оно должно просто пропустить сигнал ППД дальше по цепочке. Это достигается простой электрической перемычкой, соединяющей входной сигнал ППРвх и выходной ППРвых. Никаких логических схем в этом случае не используется.
- Если устройство поддерживает работу по прямому доступу, но, на момент получения ППД не оно сгенерировало активное ТПД – устройство должно, с минимальной задержкой, пропустить ППД дальше по цепочке, «за себя» (кто-то там, дальше по цепочке, запрашивал прямой доступ).
- И, наконец, если сигнал ППД поступил на вход устройства, которое активировало ТПД (в ответ на которое CPU и сгенерировал сигнал предоставление прямого доступа) – устройство обязано прекратить распространение ППД дальше по цепочке, используя логический вентиль (gate) (микросхема D на Рис.157).

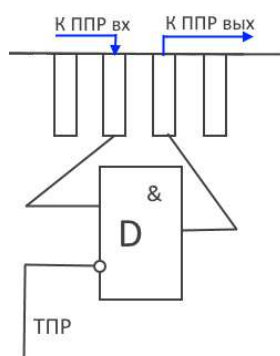


Рис.157 - Логика обработки сигнала ППД

Остановив распространение сигнала ППД, устройство: а) снимает ТПД; б) сообщает процессору, что предоставление прямого доступа добралось до «места назначения», генерируя ответный сигнал ПВ (подтверждение выбора). Процессор, «увидев» сигнал ПВ, снимает ППД и переводит все свои каналные приёмопередатчики (линии адреса, данных, командные линии, стробирующие сигналы и т.д.) в третье состояние, освобождая шину. Устройство теперь может начать цикл обмена данными, генерируя необходимые адреса, команды, стробы (СИА), анализируя ответы (СИП).

Завершив цикл обмена, устройство снимает сигнал ПВ, тем самым сигнализируя процессору, что шина свободна. К моменту снятия ПВ все обмены на всех линиях должны быть завершены, а каналные приёмопередатчики устройства переведены в высокоимпедансное состояние.

Обратите внимание, что нигде в процедуре прямого доступа (захвата шины) устройство не обозначило себя – процессор, сгенерировав сигнал ППД и получив в ответ сигнал ПВ, освобождает шину для устройства, которое в этот момент себя никак не «назвало», оставаясь в каком-то смысле анонимным.

## Прерывания

На шине Q-bus (LSI-11) обработка прерываний реализована по упрощённой схеме, по сравнению с реализацией на шине Unibus (PDP-11).

Устройство, требующее прерывание, активирует («монтажное ИЛИ») сигнал ТПР (требование прерывания). Процессор реагирует на запрос прерывания не сразу, а по окончании выполнения текущей инструкции (прерывание от устройства асинхронно по отношению к потоку выполнения и может возникнуть в любой момент в процессе обработки

текущей инструкции). Т.е., время реакции на запрос прерывания зависит от того, в какой фазе и при обработке какой конкретно инструкции возник запрос.

По окончании выполнения текущей инструкции (если прерывания разрешены), процессор принимает решение (ветвление) – переходить к адресу следующей инструкции или «отвлекаться» на обработку запроса прерывания.

В ранних версиях процессоров архитектуры LSI-11 в системе команд присутствовало несколько очень медленных инструкций – команды работы с данными в формате с фиксированной запятой (FIS, четыре инструкции: FADD, FSUB, FMUL, FDIV) и команды так называемой расширенной арифметики. Время выполнения самой медленной FDIV (до ~400 мкс) превышало время выполнения более простых базовых команд в десятки и даже в сотни раз! Если бы инженеры DEC сохранили схему реакции на запрос прерывания и для этих «тихоходных» инструкций, то время реакции могло быть недопустимо большим, если запрос на прерывания возникал бы где-то в начале выполнения такой медленной инструкции, «реактивность» системы сильно пострадала бы, и при этом мы получили бы ещё и большие непредсказуемые вариации – в сотни раз – по времени реакции на событие.

Поэтому в этой ситуации разработчики пошли на компромисс – если прерывание возникало в процессе выполнения такой медленной инструкции, то процессор тут же прекращал её обработку, сбрасывая (декрементируя) счётчик команд PC на адрес, указывающий на прерванную инструкцию, а не на следующую, как это происходит обычно в случае с более быстрыми инструкциями. Т.е., сохранённый в стеке адрес указывал на прерванную инструкцию, и, вернувшись из процедуры обработки прерывания, процессор снова пытался выполнить её с начала.

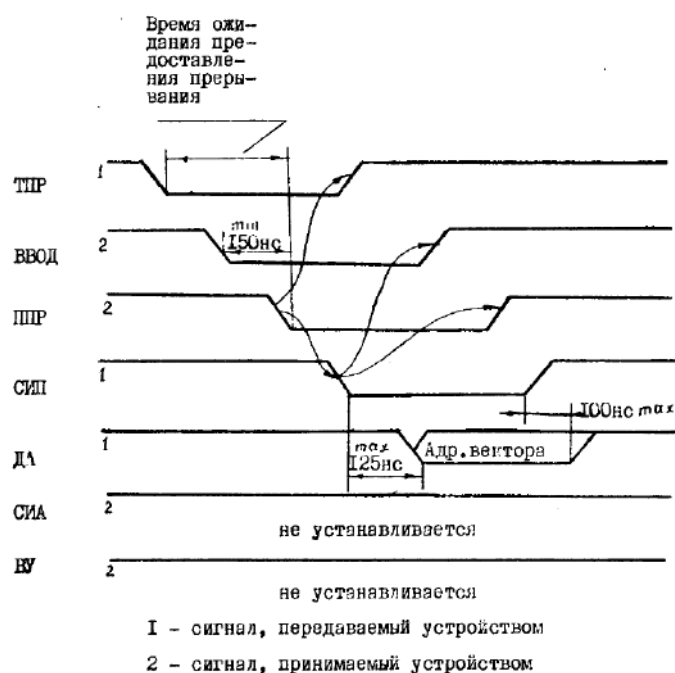


Рис.158 - Временная диаграмма прерывания программы

Процессор в ответ на сигнал ТПР генерирует два сигнала в ответ: сигнал ВВОД (чтение) и, минимум через 150нс – сигнал ППР (предоставление прерывания). ППР распространяется по цепочке, аналогичной daisy chain, используемой в обработке запроса на прямой доступ к памяти. У ППР своя цепочка, схема распространения такая же, как у сигнала ППД (Рис.155). Логика работы с ППР практически аналогична логике обработки сигнала ППД:



- Если устройство вообще не имеет логических схем для работы по прерываниям, оно должно просто пропустить ППР. Точно также, как в случае с ППД, это достигается электрической перемычкой, перемыкающей ППРвх и ППРвых.
- Если устройство имеет логику работы по прерываниям, но на момент поступления не оно сгенерировало ТПР – оно должно, с минимальными задержками, пропустить ППР дальше по цепочке.
- Если ППР поступил на вход устройство, которое на этот момент активировало ТПР, то устройство запрещает распространение сигнала ППР дальше. Точно также местоположение в цепочке распространения сигнала ППР определяет приоритет устройства при обработке запросов прерывания – «ближе» к процессору по цепочке – приоритет выше.

Запретив распространение сигнала ППР дальше по цепочке, устройство: а) снимает сигнал ТПР; б) отвечает сигналом СИП (ответ на команду ВВОД); в) выставляет на шину данных (линии DA) адрес вектора прерывания. Процессор, в ответ на СИП, читает данные (адрес вектора), и снимает сигналы ВВОД и ППР.

Принципиальное отличие в процедуре обработки запроса на прерывание от запроса прямого доступа к памяти заключается в том, что при прерывании устройство обязано себя обозначить (адресом вектора прерывания). Обработку прерывания выполняет процессор, поэтому он должен знать, какое устройство конкретно его прерывает, т.е., каждому источнику прерывания (событию) сопоставлено его «имя» – адрес вектора прерывания, адрес в ОЗУ, начиная с которого расположена подпрограмма обработки данного события. Для примера, адреса некоторых векторов прерывания в LSI-11 (восьмеричные): 060 – регистр статуса клавиатуры, 064 – регистр статуса экрана, 0100 – таймер (прерывания от него идут с частотой 100 Гц), 030 – программное прерывание, команда ЕМТ.

Есть интересный тонкий момент, как правильно должна быть написана программа, если программист хочет запретить на время (или навсегда) прерывание от какого-то одного конкретного устройства. Стандартом организации битов в регистрах УВВ было принято иметь в качестве бита готовности старший (знаковый) бит младшего байта в регистре состояния устройства (0200). В качестве бита разрешения прерывания обычно использовался предпоследний, шестой бит в байте – 0100. Логическое «И» битов 0200 и 0100 приводило к генерации сигнала ТПР. И, если требовалось запретить прерывание от какого-либо устройства, казалось бы, самое простое и естественное действие для этого – зануление шестого бита разрешения прерывания в регистре состояния устройства.

Но существует ненулевая вероятность, что запрос на прерывание от устройства возникнет как раз в момент, когда процессор выполняет команду зануления шестого бита (например, командой BIC – bit clear или MOV). Запрос на прерывание успеет возникнуть и будет зафиксирован на триггере запроса прерывания в логике процессоре. Но, результатом выполнения команды обнуления шестого бита станет сброс сигнала ТПР от устройства. И, в соответствии с Рис.158, процессор, выставив сигналы ВВОД и ППР, не получит в ответ ни сигнал СИП от устройства, ни адрес его вектора прерывания. Это будет «фантомное» прерывание, как это называют в других архитектурах.

Разные поколения LSI-11 по-разному реагировали на ситуацию, описанную выше – когда какое-то устройство выставляло запрос на прерывание, но не отвечало СИПом и адресом вектора: первые LSI-11 при этом прекращали выполнение программы и останавливались. Более поздние версии процессоров LSI-11, после таймаута (не дождавшись СИПа от устройства), продолжали выполнение программы.

Правильная последовательность команд при запрете прерываний от конкретного устройства должна быть такой:

- Сначала необходимо полностью запретить прерывания от всех источников (занулением бита 0200 в регистре состояния процессора – PSW).

- После этого можно занулять шестой бит в регистре состояния устройства. Общее зануление разрешения прерывания сбрасывает внутренний триггер запроса прерывания в процессоре, так что «фантомного» прерывания не возникнет.
- Далее можно снова разрешить прерывания от всех источников, установив бит 0200 в «1» в слове состояния процессора (PSW).

## Система команд PDP-11/LSI-11

Программисты любили PDP-11 за её «ортогональную» систему команд: можно было отдельно запоминать команды, и отдельно — методы доступа к операндам. Можно было считать, что любой метод доступа (режим адресации) будет работать с любой операцией; не нужно было запоминать список исключений и особых случаев, в которых операция имеет ограниченный набор режимов адресации, все регистры (R0...R7) могли использоваться с любой командой. Регистры R0...R4 были универсальными, а регистры R6 (SP, указатель стека, *stack pointer*) и R7 (PC, счётчик команд, *program counter*) — имели специализацию, также для нескольких команд были исключения, но все эти регистры могли использоваться как операнды в арифметико-логических операциях или использоваться при формировании адресов операндов.

В некотором смысле, набор режимов адресации образовывал «базис», а набор операций — другой базис. Каждая двухоперандная инструкция состояла из двух 6-битных идентификаторов операндов (каждый из них содержал три бита на номер регистра и три бита на режим адресации), а также из 4-битного кода операции; однооперандная инструкция содержала один 6-битный идентификатор операнда и 10-битный код операции. Благодаря логичной и структурированной системе команд коды достаточно легко читаемы непосредственно в восьмеричном представлении, например, формат двухоперандных команд:

|    |        |    |    |      |    |   |       |   |   |      |   |   |       |   |   |
|----|--------|----|----|------|----|---|-------|---|---|------|---|---|-------|---|---|
| 15 | 14     | 13 | 12 | 11   | 10 | 9 | 8     | 7 | 6 | 5    | 4 | 3 | 2     | 1 | 0 |
| B  | Opcode |    |    | Mode |    |   | R Src |   |   | Mode |   |   | R Dst |   |   |

Где старший бит B — признак байтовой инструкции.

## Слово состояния процессора

Ниже, на Рис.159 приведен формат слова состояния процессора (PSW). В старших моделях процессоров был активно задействован старший байт PSW, в младших моделях рабочие биты размещались только в младшем байте.

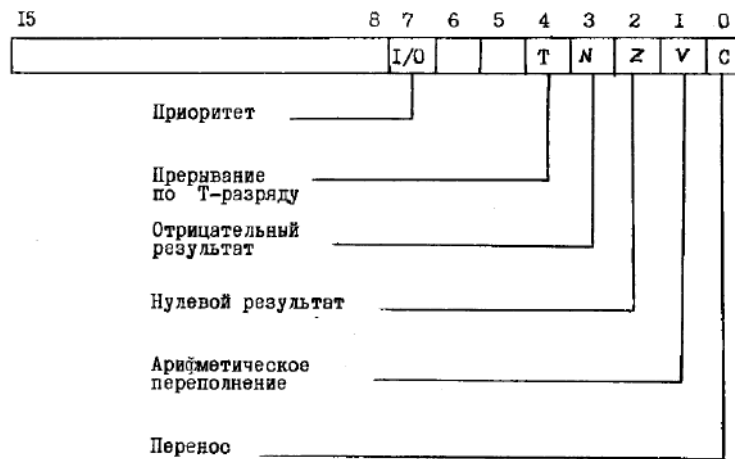


Рис.159 - Слово состояния процессора (PSW)

Нужно отметить, что сочетание бит TNZVC в PSW оказалось очень удачным и DEC перенёс эту «великолепную» пятёрку в слово состояния процессора своей следующей архитектуры – VAX-11, расположив эти биты в тех же разрядах.

### Формат одноадресных команд

Формат одноадресных команд в LSI-11 имеет следующий вид (*little-endian*):

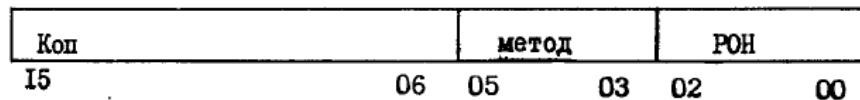


Рис.160 - Формат одноадресных команд

Разряды 15...06 содержат код операции, разряды 05...00 поле адресации приёмника, в котором:

- три младших бита 02...00 определяют один из восьми регистров общего назначения (РОН).
- три бита 05...03 определяют метод адресации.

### Формат двухадресных команд

Двухадресные команды, такие как команды пересылки, сложения и др., содержат указатели на два операнда, один из которых (левый в ассемблерной нотации) – операнд источник (src), второй (правый) – операнд приёмник (dst).

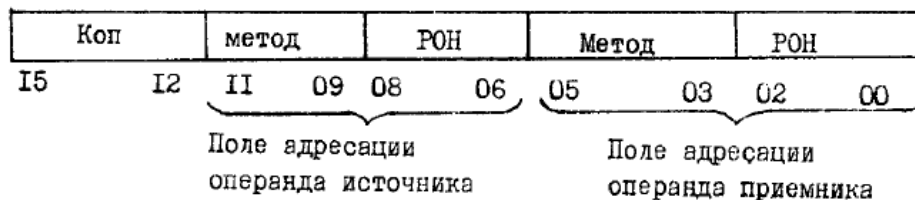


Рис.161 - Формат двухадресных команд

Биты 15...12 содержат код операции, 11...06 – поле адресации источника, 05...00 – поле адресации приёмника.

Как видно, и на номер РОН и на метод адресации отведено три бита, поэтому оказалось удобным, при работе с машинным кодом, использовать восьмеричную систему счисления ( $2^3 = 8$ ), она для PDP-11 наиболее естественная.

## Формат команд ветвления

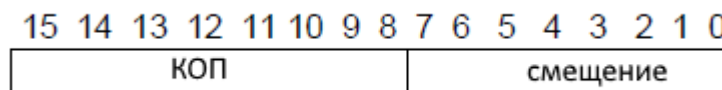


Рис.162 - Формат команд ветвления

Биты 15...08 содержат код операции, биты 07...00 – смещение адреса.

## Формат безадресных команд

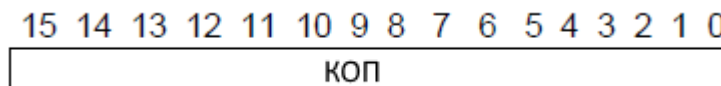


Рис.163 - Формат безадресных команд

Биты 15...00 содержат код операции.

## Список команд

| Мне-мокод                                                  | Код    | N Z V C | Операция                             | Действия                     |
|------------------------------------------------------------|--------|---------|--------------------------------------|------------------------------|
| <u>Команды пересылки и установки признаков результата:</u> |        |         |                                      |                              |
| CLR                                                        | *050DD | 0 1 0 0 | Очистка                              | D = 0 <sup>+</sup>           |
| MOV                                                        | *18SDD | + + 0 - | Пересылка                            | D = S                        |
| SWAB                                                       | 0003DD | + - 0 0 | Перестановка байтов                  | D=D[7..0]•D[15..8]           |
| MFPS                                                       | 1067DD | - - - - | Чтение PSW                           | D = PSW                      |
| MTFS                                                       | 1064SS | + + + + | Запись PSW                           | PSW = S                      |
| TST                                                        | *057DD | + + 0 0 | Проверка                             | D = D                        |
| <u>Логические операции:</u>                                |        |         |                                      |                              |
| COM                                                        | *051DD | + + 0 1 | Инвертирование                       | D = ^D                       |
| BIT                                                        | *38SDD | + + 0 - | Проверка разрядов                    | S & D                        |
| BIC                                                        | *48SDD | + + 0 - | Очистка разрядов                     | D &= ^S                      |
| BIS                                                        | *58SDD | + + 0 - | Логическое сложение                  | D  = S                       |
| XOR                                                        | 074RDD | + + 0 - | Исключающее ИЛИ                      | D ^= R                       |
| <u>Арифметические операции и сдвиги:</u>                   |        |         |                                      |                              |
| INC                                                        | *052DD | + + + - | Увеличение на 1                      | D++                          |
| DEC                                                        | *053DD | + + + - | Уменьшение на 1                      | D--                          |
| CMP                                                        | *2SSDD | + + + + | Сравнение                            | S - D                        |
| ADD                                                        | 06SSDD | + + + + | Сложение                             | D += S                       |
| SUB                                                        | 16SSDD | + + + + | Вычитание                            | D -= S                       |
| ADC                                                        | *055DD | + + + + | Добавление переноса                  | D += C                       |
| SBC                                                        | *056DD | + + + + | Вычитание переноса                   | D -= C                       |
| NEG                                                        | *054DD | + + + + | Изменение знака                      | D = 0 - D                    |
| SXT                                                        | 0067DD | + + + + | Расширение знака                     | D[*] = N                     |
| MUL                                                        | 070RSS | + + 0 + | Умножение                            | Rp = R * S                   |
| DIV                                                        | 071RSS | + + + + | Деление                              | Rp /= S                      |
| ASR                                                        | *062DD | + + + + | Арифм. сдвиг вправо                  | D•C = D[ст] • D              |
| ASL                                                        | *063DD | + + + + | Арифм. сдвиг влево                   | C•D = D•0                    |
| ROR                                                        | *060DD | + + + + | Цикл. сдвиг вправо                   | D•C = C•D                    |
| ROL                                                        | *061DD | + + + + | Цикл. сдвиг влево                    | C•D = D•C                    |
| ASH                                                        | 072RSS | + + + + | Арифм. сдвиг на заданное число разр. | S>0 ? R <<= S:<br>R >>= -S   |
| ASHC                                                       | 073RSS | + + + + | Арифм. сдвиг двойного слова          | S>0 ? Rp <<= S:<br>Rp >>= -S |

Рис.164 - Одноадресные и двухадресные команды обработки данных.

| Мне-мокод                                                      | Код    | N Z V C             | Операция                                                  | Действия                                                 |
|----------------------------------------------------------------|--------|---------------------|-----------------------------------------------------------|----------------------------------------------------------|
| <b>Безусловный переход и организация циклов и подпрограмм:</b> |        |                     |                                                           |                                                          |
| JMP                                                            | 0001DD | - - - -             | Безусловный переход                                       | PC = &D                                                  |
| SOB                                                            | 077RNN | - - - -             | Организация цикла                                         | if(--R != 0)<br>PC -= 2*NN                               |
| JSR                                                            | 004RDD | - - - -             | Обращение к подпрограмме                                  | *(--SP) = R,<br>R=PC, PC=&D                              |
| RTS                                                            | 00020R | - - - -             | Возврат из п/п                                            | PC=R, R=*SP++                                            |
| MARK                                                           | 0064NN | - - - -             | Восстановление указателя стека                            | SP = PC+2*NN<br>PC = R5<br>R5 = *SP++                    |
| <b>Программные прерывания:</b>                                 |        |                     |                                                           |                                                          |
| EMT                                                            | 104000 | из век-<br>..104377 | Прерывание для систем. программ с адресом вектора AV = 30 | *--SP = PSW<br>*--SP = PC<br>PC = *(AV)<br>PSW = *(AV+2) |
| TRAP                                                           | 104400 | - " -<br>..104777   | Прерывания пользов.                                       | то же, но адр. вектора AV=34                             |
| IOT                                                            | 000004 | - - - -             | Прерыв. для вв/выв.                                       | AV=20                                                    |
| BPT                                                            | 000003 | - - - -             | Прерыв. трассировки                                       | AV=14                                                    |
| RTI                                                            | 000002 | из стека            | Возврат из прерывания                                     | PC = *SP++<br>PSW = *SP++                                |
| RTT                                                            | 000006 | из стека            | Возврат из прерывания трассировщика                       | PC = *SP++<br>PSW = *SP++                                |

Рис.165 - Команды управления программой и прерываниями

| Если условие ветвления выполняется, то PC += 2*(short)Disp, где: Disp - младший байт команды. |                           |                      |                    |
|-----------------------------------------------------------------------------------------------|---------------------------|----------------------|--------------------|
| Мне-мокод                                                                                     | Диапазон кодов результата | Наименование условия | Значение признаков |
| BR                                                                                            | 000400..000777            | Безусловно           | нет                |
| BNE                                                                                           | 001000..001377            | Не нуль;неравенство  | Z == 0             |
| BEQ                                                                                           | 001400..001777            | Нуль; равенство      | Z == 1             |
| BPL                                                                                           | 100000..100377            | Плюс                 | N == 0             |
| BMI                                                                                           | 100400..100777            | Минус                | N == 1             |
| BVC                                                                                           | 102000..102377            | Нет переполнения     | V == 0             |
| BVS                                                                                           | 102400..102777            | Есть переполнение    | V == 1             |
| BCC                                                                                           | 103000..103377            | Нет переноса         | C == 0             |
| BCS                                                                                           | 103400..103777            | Есть перенос         | C == 1             |
| <b>Ветвления по неравенству чисел со знаком:</b>                                              |                           |                      |                    |
| BLT                                                                                           | 002400..002377            | Меньше               | N ~ V = 1          |
| BGT                                                                                           | 003000..003377            | Больше               | Z   (N ~ V) == 0   |
| BLE                                                                                           | 003400..003777            | Меньше или равно     | Z   (N ~ V) == 1   |
| BGE                                                                                           | 002000..002377            | Больше или равно     | N ~ V == 0         |
| <b>Ветвления по неравенству чисел без знака:</b>                                              |                           |                      |                    |
| BLO                                                                                           | 103400..103777            | Меньше               | C == 1             |
| BHI                                                                                           | 101000..101377            | Больше               | Z   C == 0         |
| BLOS                                                                                          | 101400..101777            | Меньше или равно     | Z   C == 1         |
| BHIS                                                                                          | 103000..103377            | Больше или равно     | C == 0             |

Рис.166 - Команды ветвления

| Мне-мокод | Код    | Операция     | Действия                    |
|-----------|--------|--------------|-----------------------------|
| HALT      | 000000 | Останов      | Переход на связь с консолью |
| WAIT      | 000001 | Ожидание     | Предоставление канала ВУ    |
| RESET     | 000005 | Сброс        | Начальная установка ВУ      |
| NOP       | 000240 | Нет операции | Пустая команда              |

Рис.167 - Команды управления

| Мне-мокод | Код    | Действие  | Мне-мокод | Код    | Действие  |
|-----------|--------|-----------|-----------|--------|-----------|
| CLN       | 000250 | N = 0     | SEN       | 000270 | N = 1     |
| CLZ       | 000244 | Z = 0     | SEZ       | 000264 | Z = 1     |
| CLV       | 000242 | V = 0     | SEV       | 000262 | V = 1     |
| CLC       | 000241 | C = 0     | SEC       | 000261 | C = 1     |
| CCC       | 000257 | N=Z=V=C=0 | SCC       | 000277 | N=Z=V=C=1 |

Рис.168 - Команды изменения признаков в PSW

## Методы прямой адресации

Инженеры DEC очень хорошо продумали набор методов адресации данных для архитектуры PDP-11. Восемь возможных методов были разделены на чётные (прямые) и нечётные (косвенные). Каждому прямому методу соответствует парный (с номером на единицу больше) косвенный метод адресации. Т.е., прямые методы имеют номера 0...2...4...6, косвенные, соответственно 1...3...5...7.

0 (**регистровый**) метод – операнд содержится в РОН, номер которого указан в поле номера регистра.

2 (**автоинкрементный**) метод – РОН используется как указатель на адрес ячейки в ЗУ, содержащей операнд. Регистр сначала используется как указатель, затем его содержимое автоматически *инкрементируется*: на +1 для байтовых операций, на +2 – для словных.

4 (**автодекрементный**) метод – РОН используется как указатель на адрес ячейки в ЗУ, содержащей операнд, но **перед** использованием РОНа как указателя, его содержимое сначала *декрементируется*: на -1 для байтовых операций, на -2 – для словных.

6 (**индексный**) метод – содержимое РОН складывается с индексным словом (которое расположено в памяти сразу за командой), полученная сумма используется как адрес операнда.

Бросается в глаза кажущаяся асимметрия второго и четвёртого методов – в автоинкрементном РОН *сначала* используется, как указатель, а затем – инкрементируется (postfix), а в автодекременте содержимое РОН *сначала* декрементируется (prefix), потом используется как указатель.

Но, при внимательном рассмотрении всё оказывается в полном порядке. Предположим, наш регистр R6 (SP, указатель стека) содержит число 01000 (адрес) и мы хотим поместить в стек (push) константу 0333:

```
MOV #0333, -(R6)
```

Указатель (словная инструкция) сначала декрементируется на -2, т.е. R6 будет указывать на адрес 0776, и по этому адресу будет записана константа 0333. Теперь, для извлечения константы из стека (pop), применим автоинкрементный метод:

```
MOV (R6)+, R1
```

Эта инструкция *сначала* извлечёт из ячейки с адресом 0776 константу 0333 и перешлёт её в регистр R1, а *потом* инкрементирует R6 на +2, вернув указатель в исходное состояние – теперь R6 вновь содержит адрес 0100.

Для корректной работы один из авто-методов должен был быть постфиксным, а второй – префиксным. Почему был выбран именно такой порядок: (R6)+ и -(R6), а не наоборот? Ответ содержится в роли регистра R7 (PC) – его работа в качестве регистра общего назначения – это не его основной функционал, главное его предназначение – указывать на адрес ячейки, содержащей следующую инструкцию (счётчик команд). Для этого R7 – и только этот регистр – всегда автоматически инкрементируется на +2 (инструкции в PDP-11 выровнены на словную границу), и инкремент выполняется в постфиксном стиле: сначала из памяти извлекается инструкция, на которую указывает счётчик команд (R7), а *после* этого его содержимое увеличивается на два. Т.е., автоинкрементный метод для всех остальных РОН повторяет поведение регистра R7.

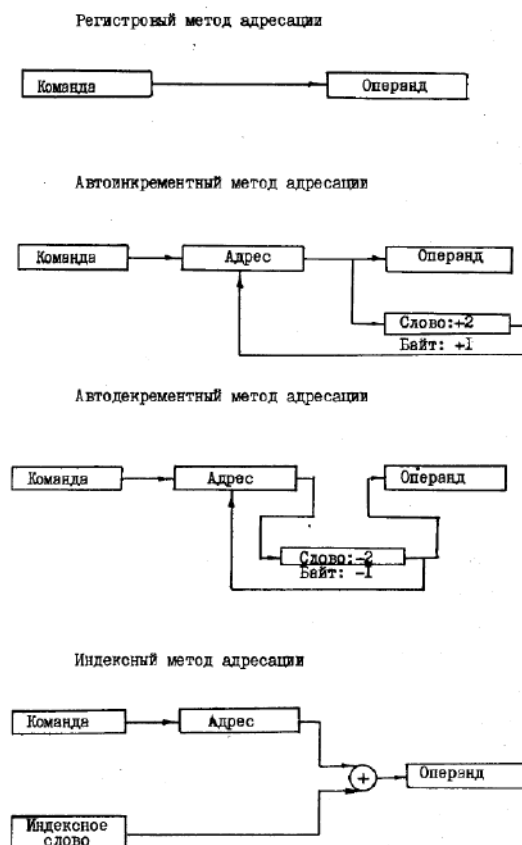


Рис.169 - Методы прямой адресации

## Методы косвенной адресации

Основное отличие косвенных методов от прямых – там, где в прямых методах получалась ссылка на операнд, в косвенных будет указатель на адрес адреса. Например, если в 0-ом методе РОН содержит операнд, то в 1-ом – адрес ячейки, содержащей операнд.

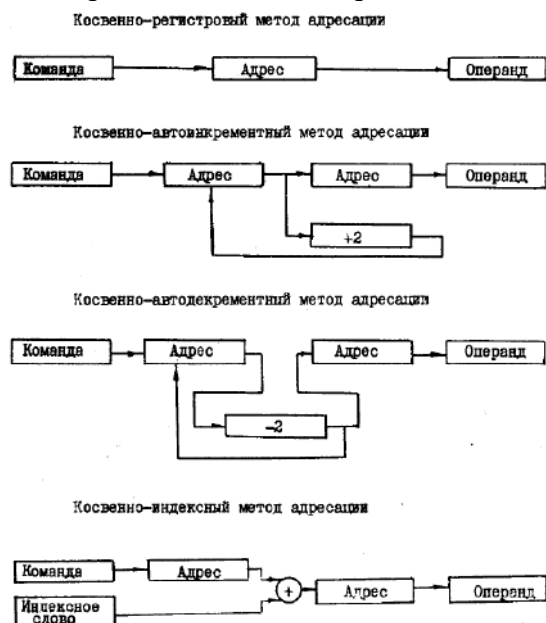


Рис.170 - Методы косвенной адресации

- 1 (косвенно-регистровый) метод – РОН содержит адрес ячейки с операндом.
- 3 (косвенно-автоинкрементный) метод – РОН используется как указатель на адрес адреса операнда, с автоинкрементом *после* использования указателя.

5 (**косвенно-автодекрементный**) метод – РОН используется как указатель на адрес операнда, с автодекрементом *перед* использованием указателя.

7 (**косвенно-индексный**) метод – содержимое РОН суммируется с индексным словом, полученная сумма используется как указатель на адрес операнда.

## Методы адресации с использованием R7

Когда центральный процессор использует счётчик команд для выборки слова из памяти, его содержимое автоматически увеличивается на 2. Новое содержимое является адресом слова, используемого при выполнении данной команды или адресом следующей инструкции, которую необходимо выполнить. Следует отметить, что при работе с байтами содержимое РС также увеличивается на 2.

R7 теоретически может быть использован во всех методах адресации. Хотя, как уже неоднократно упоминалось выше, главное предназначение регистра R7 (PC) – работа в качестве счётчика команд, а не в качестве регистра общего назначения. Этот регистр автоматически, по определению своего основного функционала, инкрементируется на +2, поэтому часть методов адресации в сочетании с РС не имеют практического смысла. Например, если в одноадресной команде применить автодекрементный метод адресации в сочетании с R7, то программа просто остановится – R7, как счётчик команд, инкрементируется на +2, но автодекрементный метод снова вернёт содержимое R7 в исходное состояние, выполнив команду вычитания числа «2» из R7, в результате чего счётчик команд снова будет указывать на ту же самую инструкцию, т.е. программа будет «вечно» исполнять одну и ту же команду.

Поэтому наиболее эффективно РС используется только ровно с половиной методов адресации – четырьмя из восьми: 2, 3, 6 и 7. Которые, если приглядеться внимательно к тому, как они работают, оказывается логичным именовать их особо:

- 27-ой метод – непосредственный;
- 37-ой метод – абсолютный;
- 67-ой метод – относительный;
- 77-ой метод – косвенно-относительный.

И ведь действительно – в 27-ом (автоинкремент) методе указатель (РС) указывает на операнд, располагающийся в ячейке *непосредственно за* ячейкой с инструкцией. В 37-ом (косвенно-автоинкрементный) методе указатель «смотрит» на *абсолютный* адрес операнда, располагающийся в ячейке, следующей за ячейкой с инструкцией. В 67-ом (индексный) методе адрес операнда формируется как сумма содержимого РС и ячейки, следующей за ячейкой с инструкцией (относительный индекс). Ну и, наконец, в 77-ом (косвенно-индексный) методе из ячейки, адрес которой формируется как сумма содержимого РС и ячейки, следующей за инструкцией, выбирается *адрес* операнда.

Если в других архитектурах абсолютные и относительные методы адресации обычно присутствовали в системе команд с разными кодами операций, то в ортогональной системе команд PDP-11 они были реализованы просто, изящно и абсолютно естественно, с использованием сочетания обычных для всех РОН методов адресации и свойства счётчика команд (автоинкремент +2).

Хотя «запрещённое» сочетание автодекрементных методов с R7 всё-таки иногда было возможным. Инженеры DEC использовали нижеследующую инструкцию для быстрого заполнения ОЗУ константой:

**MOV – (PC) , – (PC)** ; машинный код 014747

И никаких циклов, счётчиков и т.п. – всего одна инструкция! Разберём, как это работает. Для определённости пусть команда ‘mov –(pc), –(pc)’ исходно была записана в ячейку ОЗУ с адресом 01000.



Процессор извлекает инструкцию из адреса 01000 и R7 (PC) инкрементируется на +2, указывая на адрес 01002. Но чтобы добраться до первого операнда, регистр R7 декрементируется на -2 (адрес 01000) и из этого адреса извлекается src операнд (число 014747, код инструкции). После этого вычисляется адрес получателя (dst) – R7 *ещё раз* декрементируется на -2, теперь он указывает на 0776, и в этот адрес копируется src операнд – число 014747. Т.е., после однократного выполнения содержимое счётчика команд в итоге декрементировалось на два и указывает на адрес, на два меньший в сравнении с адресом, с которого началось выполнение программы. Но при этом в ячейку с адресом 0776 был скопирован код самой команды mov (восьмеричный 014747), т.е. команда «размножилась», скопировав саму себя в соседнюю ячейку (с адресом -2 от исходного) – неймановская архитектура во всей красе. Теперь, на втором шаге из ячейки с адресом 0776 (на неё указывает R7) будет извлечена... та же команда 'mov -(pc), -(pc)' (код 014747), которая произведёт аналогичные действия: скопирует себя в ячейку с адресом на два меньшим и извлечёт оттуда свою очередную копию. И т.д, пока R7 не начнёт указывать на несуществующий адрес (переход от 0 к 177776).

В итоге, без всяких циклов, мы получаем заполнение ОЗУ от 01000 до 0 константой 014747 – кодом инструкции 'mov -(pc), -(pc)'. Красивый фокус...

## Некоторые программные «трюки» для системы команд PDP-11

Используя «стандартные» для всех РОН методы адресации в сочетании со счётчиком команд PC (R7), в PDP-11 были реализованы методы адресации – непосредственный, абсолютный, относительный – для которых в других архитектурах использовались отдельные КОП. А «неправильное» сочетание автодекрементных методов и R7 позволяло создавать трюки-шедевры, вроде описанного выше 'mov -(pc), -(pc)'.

На этом возможности к написанию необычного, красивого и изящного кода в «исполнении» PDP-11 не заканчивались. Например, очень интересно в PDP-11 реализовывались **сопрограммы**<sup>1</sup>, которые получались из не совсем обычного сочетания команды JSR (Jump to Service Routine) и автоинкрементного метода адресации для указателя стека. Команды безусловного перехода JMP и перехода к подпрограмме в качестве адреса перехода используют эффективный адрес операнда. Поэтому для этих команд недопустима регистровая адресация. При входе в подпрограмму по команде 'JSR Ri, Adr' содержимое регистра Ri сохраняется в стеке, в регистр Ri заносится адрес возврата и в PC заносится адрес перехода. При возврате из подпрограммы по команде RTS адрес возврата переносится из Ri в PC и старое содержимое Ri восстанавливается из стека. Поэтому команда RTS должна ссылаться на тот же Ri, что и команда вызова данной подпрограммы. Чаще всего в качестве регистра используется PC, то есть команды имеют вид 'JSR PC, Adr' (== CALL) и RTS PC, а адрес возврата сохраняется в стеке.

Т.е., если в команде JSR в качестве регистра связи выступает счётчик команд, то она загружает исполнительный адрес в PC, а адрес возврата в системный стек. Интересная форма команды получается, если адрес программы, которой должно быть передано управление, сам был занесен в стек. В этом случае требуемый исполнительный адрес есть не (SP) (т.е. ячейка в системном стеке), а ячейка, на которую указывает (SP) (косвенность). Поэтому управление передается командой

JSR      PC, @ (SP)

<sup>1</sup> **Сопрограмма (coroutine)** — программный модуль, особым образом организованный для обеспечения взаимодействия с другими модулями по принципу кооперативной многозадачности: модуль приостанавливается в определённой точке, сохраняя полное состояние (включая стек вызовов и счётчик команд), и передаёт управление другому, тот, в свою очередь, выполняет задачу и передаёт управление обратно, сохраняя свои стек и счётчик.

Это приводит к засылке адреса возврата в стек поверх того адреса, на который только что было передано управление. Но, однако, нет никакой причины, ради которой последний должен оставаться в стеке. Лучше, прежде чем заносить текущее значение PC, его оттуда убрать. Это достигается видоизменением команды, продвигающей указатель стека на +2 за счёт косвенно-автоинкрементного метода адресации для SP:

JSR PC, @(SP) +

что приводит к обмену содержимого PC и верхней ячейки стека. Из симметрии следует, что программа, которой было передано управление, может вернуть его назад точно такой же командой, после чего в стеке будет содержаться адрес следующей за ней ячейки. Новая команда 'JSR PC, @(SP) +' передаст управление именно на эту ячейку.

Т.е., таким способом две программы могут поочередно передавать управление друг другу. Такие программы называются **сопрограммами**, что подчеркивает факт их кооперации, основанной на равноправии, а не на подчинении, как в **подпрограммах**.

Ниже на Рис.171 показана упрощённая схема взаимодействия двух сопрограмм (поток) – чтения и интерпретации, – которые протекают при работе ассемблера, выполняющего интерпретацию очередной команды

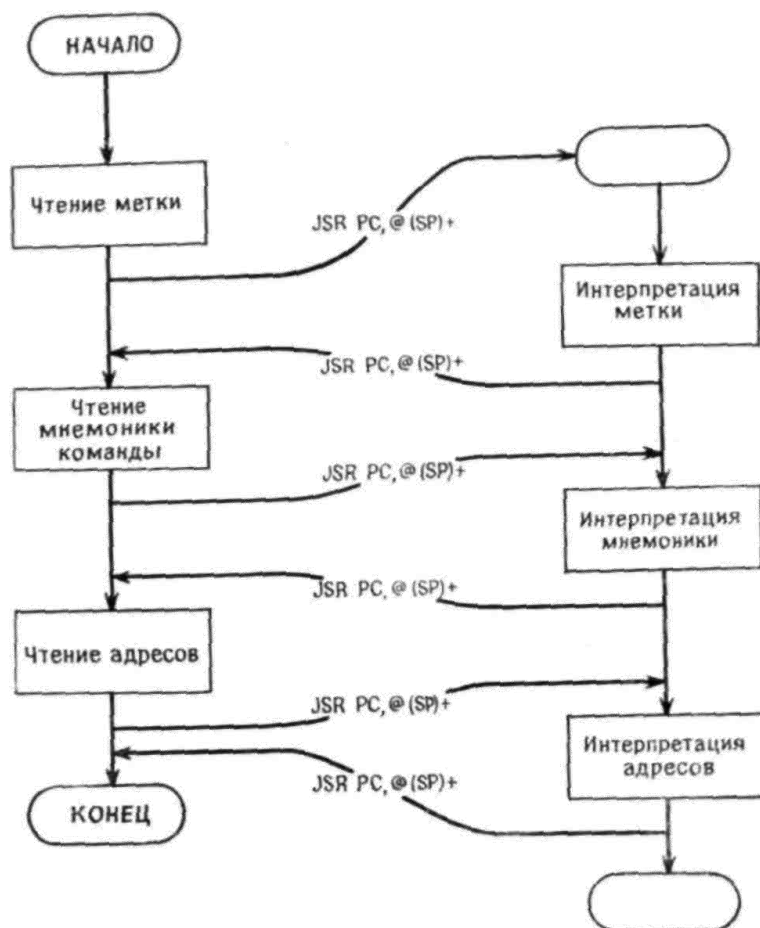


Рис.171 - Передача управления между сопрограммами в ассемблере.

Ещё один пример любопытного кода, в котором в полной мере раскрывается мощь и красота системы команд и методов адресации LSI-11, в сочетании с неймановской организацией ЭВМ – приведён ниже (Рис.172). Это реальный код, который был «вшит» на уровне микропрограммного консольного отладчика в младших моделях LSI-11, осуществлявшего загрузку программы с перфоленты в память, с последующей передачей управления загруженной программе.

Программа *не* позиционно независимая, в ней есть привязка к абсолютным адресам (это позволило сэкономить размер кода), *самомодифицирующаяся* (неймановская архитектура).

```

077744 016701 START:    MOV      77776,R1
                        000026
077750, 012702 S1:      MOV      #352,R2
                        000352
077754 005211           INC      (R1)
077756 105711 LOOP:     TSTB     (R1)
077760 100376           BPL      LOOP
077762 116162           MOVB     2(R1),77400(R2)
                        000002
                        077400
077770 005267           INC      77752
                        177756
077774 000765           BR       S1
077776 177550           .WORD    PRS

```

Рис.172 - Программа начального загрузчика для перфоленты.

Далее приведено подробное описание работы программы, приведённой на Рис.172. Эта часть не входит в экзаменационные билеты, являясь своеобразным бонусом – если студент на экзамене точно и без ошибок продемонстрирует понимание работы этого кода, ему будет гарантирована оценка не ниже «хорошо», независимо от качества ответов на другие вопросы.

Регистр устройства чтения с перфоленты имеет адрес **PRB=177552**. Регистр состояния считывающего устройства **PRS=177550**.

Программа начинается с засылки в **R1** адреса **PRS**. В цикле с меткой **LOOP** регистр состояния проверяется уже знакомым нам способом, и в случае поступления байта последний пересылается из **PRB** в память. Заметим, что перед этим командой **INC (R1)** бит готовности в **PRS** был установлен в единицу. Затем программа передает управление назад на метку **S1** для ввода следующего байта.

Меткой **S1** помечена команда, которая пересылает содержимое ячейки **77752** в **R2**. После ее выполнения **R2** содержит **352**. Поэтому первая прочитанная литера будет записана в ячейку с адресом **77400(R2) = 77400+352 = 77752**. Как видно, это то слово, куда первоначально было помещено число **352**, чтобы потом его можно было занести во второй регистр. Обратите также внимание, что следующая команда увеличивает содержимое этой ячейки.

Т.е., отсюда становится ясным, что начальный загрузчик представляет собой самомодифицирующуюся программу. Причину применения такого подхода, однако, невозможно понять, не зная, что программе на перфоленте предшествует *ракорд*, который представляет собой повторение одного и того же байта на протяжении нескольких *дюймов ленты*. Благодаря ракорду снижается риск физического повреждения самой программы и чётко обозначается начало программы — это место, где кончается ракорд. В ракорде монитора пробивается код **351**. Как мы уже видели, после считывания байт **351** пересылается в ячейку с адресом **77752**. Содержимое этой ячейки затем увеличивается на единицу, так что при выполнении команды **BR S1** она содержит код **352**, что и было в ней вначале. Таким образом, программа не изменяется при чтении одного байта **351** из ракорда, а значит, и любого количества таких байтов.

Любой иной (не **351**) прочитанный с перфоленты байт будет, однако, изменять программу. Рассмотрим, что произойдет при чтении первого, отличного от **351** байта. Когда он

считывается, второй регистр содержит код **352**, сохранившийся от предыдущего цикла чтения, и новый байт будет снова занесен в ячейку **77752**. Следовательно, после возвращения на метку **S1** ячейка **77752** содержит новый байт, увеличенный на единицу, и это значение будет загружено затем в **R2**. Второй регистр указывает теперь на новую ячейку, предназначенную для следующего байта. Её адрес будет начальным адресом программы монитора.

Монитор должен быть загружен в ячейки памяти, которые *непосредственно предшествуют* программе предзагрузчика. Допустим, что мониторная программа должна быть размещена с адреса **77600**. Это значит, что первый после ракорда байт устанавливает такое значение в **R2**, что команда **MOV** засылает следующий байт в ячейку **77600**. Как легко видеть, во второй регистр для этого должно быть записано число **200**. Так как текущее содержимое ячейки **77752** перед засылкой в **R2** увеличивается на единицу, то записанный в неё на предыдущем цикле чтения байт должен быть равен **177**.

Итак, после чтения отперфорированной последовательности байтов **351, ..., 351, 177** ячейка **77752** и регистр **R2** содержат код **200**.

Заметим, что теперь программа уже не *самомодифицирующаяся*! Ячейка **77400(R2)** больше не находится внутри программы, и на каждом цикле чтения увеличение содержимого ячейки **77752** приводит просто к увеличению на единицу значения **R2**, так что следующий байт заносится в память сразу после предыдущего. Следовательно, при вводе последовательности байтов **351, ..., 351, 177**, программа в память, начиная с адреса **77600**, загружается программа.

Программа монитора организована так, что последняя её ячейка находится как раз перед первым словом предзагрузчика. Однако если бы на этом перфолента и заканчивалась, то предзагрузчик продолжал бы ждать дальнейшего ввода: он не передал бы управления монитору. Перфолента должна содержать ещё какую-то информацию, которая позволит передать управление на некую ячейку внутри мониторной программы.

С перфоленты будут прочитаны ещё восемь байтов. Первые шесть из них совпадают с шестью первыми байтами программы предзагрузчика. При чтении содержимое ячеек **77744**, **77746** и **77750** останется таким же, как прежде. Предзагрузчик опять становится самомодифицирующимся, но эти шесть байтов, в сущности, не изменяются. Седьмой байт будет записан в младший байт ячейки **77752**. Заметим, что при входе в цикл чтения этого байта ячейка **77752** опять содержит число **352**. Седьмой байт будет равен **373**, и потому при выходе на метку **S1** по адресу **77752** будет стоять число **374**.

Предположим, что управление должно быть передано по адресу **77600**, т.е. на первое слово программы монитора. Тогда последний байт должен быть равен **701**. Перед входом в цикл чтения этого байта значение **R2** равнялось **374**, и поэтому этот байт будет занесен по адресу **77400+374 = 77774**. Следовательно, команда **MOV** изменит команду **BR S1**; с новым кодом **000701** это будет **BR 77600**.

## Диспетчер памяти (MMU)

В большинстве моделей PDP-11/LSI-11, кроме самых младших, в системе присутствовал дополнительный функциональный блок, который назывался *диспетчер памяти*<sup>1</sup>. Его функции – трансляция адресов виртуальной памяти в адреса физической памяти (то есть управление виртуальной памятью), защита памяти, и переключении блоков памяти. В младших моделях MMU не было, и виртуальный адрес в программе был полностью эквивалентен физическому адресу на шине. Кроме того, в таких CPU присутствовал ровно один комплект РОН и работать процессор мог в одном единственном режиме, в котором программам доступны все инструкции и вся память, включая страницу ввода-вывода.

---

<sup>1</sup> Диспетчер памяти или устройство управления памятью (*memory management unit*, MMU) — компонент аппаратного обеспечения компьютера, отвечающий за управление доступом к памяти, запрашиваемым центральным процессором.

Компьютеры с MMU имели как минимум два, а в старших моделях даже три, режима работы процессора: пользовательский (непривилегированный) и системный. В системном режиме процессору доступны все инструкции, а также процессор при этом имеет полный доступ к устройствам (странице) ввода/вывода. В многозадачной среде процессы, размещённые в ОЗУ, должны быть защищены от несанкционированного доступа, чтобы «чужой» процесс, специально или непреднамеренно, не «подсмотрел» или не испортил чужие данные или код. Межпрограммная защита реализуется средствами диспетчера памяти. В пользовательском режиме процессору недоступны некоторые привилегированные команды, например, при попытке выполнить команду «Сброс» («Reset») будет сгенерировано исключение с адресом вектора 010 («резервная»/«несуществующая» инструкция для пользовательского режима).

MMU в PDP-11 преобразует виртуальные (логические) адреса в физические, расширяя адресное пространство в  $2^6=64$  раза – с 64 Кбайт до 4 Мбайт. Необходимо отдельно подчеркнуть – расширяется именно доступная физическая память, процессы продолжают работать в том же 16-ти разрядном виртуальном адресном пространстве, как было без MMU, т.е. максимально возможный размер программы не меняется<sup>1</sup>.

В схеме генерации физического адреса MMU располагается между адресами, сгенерированными в CPU (указатели) и Q-bus шиной. Преобразование адреса выполняется по страничной схеме – всё виртуальное адресное пространство (64К) делится на восемь страниц равного размера по 8К. Каждой странице ставится в соответствие пара шестнадцатиразрядных регистров: регистр описания страницы (РОС, PDR) и регистр адреса страницы (РАС, PAR). Адреса каждой из восьми 8-килобайтных страниц преобразовывается в диспетчере памяти (по сути в нём выполняется операция беззнакового сложения базового адреса и адреса смещения) и отображаются на соответствующую ей страницу в физической памяти. Обратите внимание (Рис.173), что страницы могут располагаться в ОЗУ не в том порядке, в каком они расположены в виртуальном адресном пространстве.

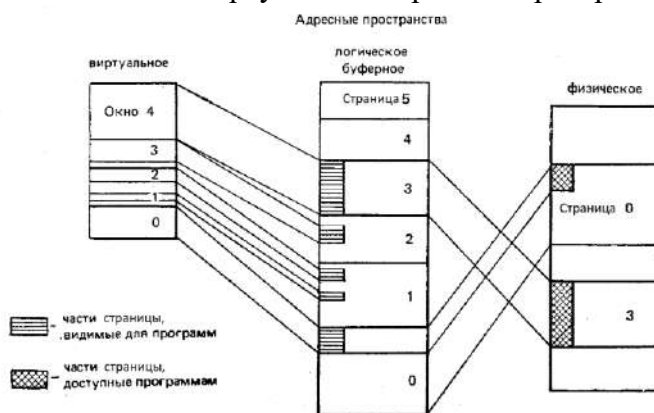


Рис.173 - Структура адресных пространств

В архитектуре PDP-11 было два варианта MMU – с 18-разрядным физическим адресом (макс. ОЗУ 256К) и с 22-разрядным (макс. ОЗУ 4М). Далее описание соответствует 22-разрядному варианту.

Более подробно: 16-разрядное поле виртуального адреса делится на три части: старшие три разряда адреса – 15...13 – определяют номер страницы, разряды 12...06 – номер блока внутри страницы, 05...00 – адрес внутри блока.

<sup>1</sup> Формально на компьютере без MMU максимально возможный размер программы составляет 64К – 8К (страница ввода/вывода) = 56К. Ещё одно исключение, уже для самых старших моделей PDP-11 – в них было реализовано раздельное адресное пространство для кодов и данных (намёк на гарвардскую архитектуру), каждое максимум по 64К, т.е. теоретически объём программы мог достигать 64К + 64К = 128К.

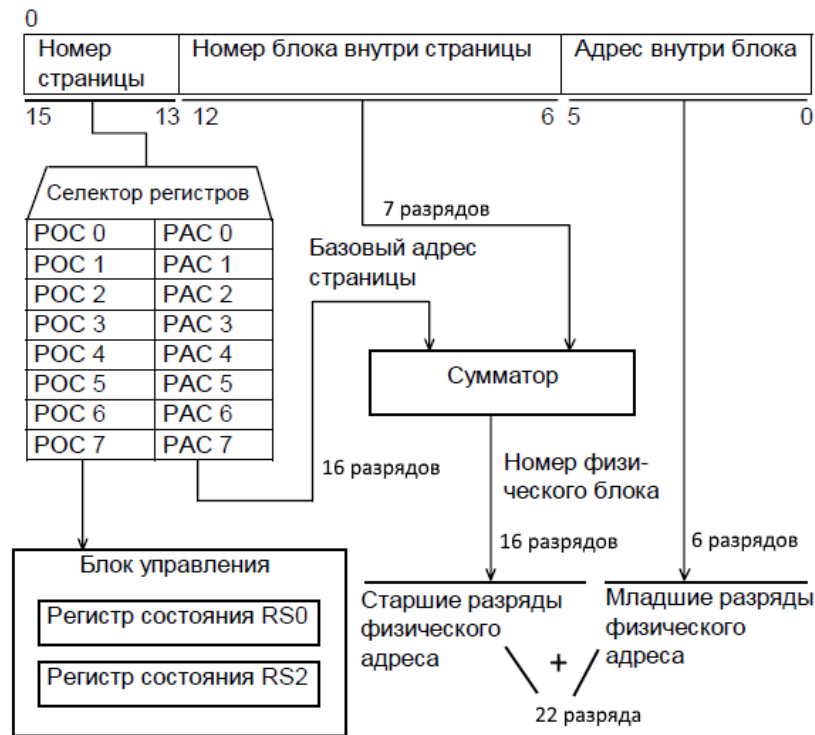


Рис.174 - Схема формирования физического адреса в диспетчере памяти PDP-11

Старшие 16 разрядов физического адреса формируются суммированием 16-разрядного содержимого соответствующего PAC (выбранного тремя старшими разрядами виртуального адреса), сдвинутого влево на 6 разрядов (базовый адрес страницы) с 7-разрядным номером блока внутри страницы. Младшие 6 разрядов виртуального адреса (адрес внутри блока) транслируются напрямую в младшие 6 разрядов физического адреса, формируя  $16 + 6 = 22$  разрядный адрес (Рис.174). Идея увеличения адресуемого объема физической памяти состоит просто в увеличенном в  $2^6 = 64$  раза размере минимально адресуемой единицы ЗУ – с одного байта до 64-х.

Формат регистра описания страницы (PDR) приведен ниже на Рис.175.



Рис.175 - Формат регистра описания страницы

- 1 – признак отсутствия страницы в памяти и атрибуты доступа:
  - РОС[0] – признак отсутствия страницы в ОЗУ;
  - РОС[2...1] – атрибуты доступа:
    - 00, 10 – доступ запрещён;
    - 01 – страница доступна только для чтения (readonly);
    - 11 – страница доступна на чтение/запись.
- 2 – направление роста страницы, РОС[3]:
  - 0 – рост в сторону увеличения адресов;
  - 1 – рост в сторону уменьшения адресов (стек).
- РОС[05...04] – 0.
- 3 – признак модификации страницы, РОС[6]:

- 0 – записи в страницу не было;
  - 1 – была запись в страницу.
- 4 – признак обращения к странице, РОС[7]:
- 0 – не было обращения к странице;
  - 1 – было обращение к странице.
- РОС[14...08] – размер страницы в блоках 64 байта.

Необходимо ещё раз подчеркнуть – MMU PDP-11 имеет переменный размер страницы, отображаемой в физическую память (ОЗУ), 8 Кбайт – это максимально возможный размер страницы. Это достаточно необычное свойство, в подавляющем большинстве других архитектур страницы физической памяти чаще всего имеют фиксированный размер (или несколько фиксированных значений размера). Разработчики архитектуры PDP-11 сделали возможным иметь размер страницы от минимальных 64 байт до 8 Кбайт, с шагом 64 байта, для этого ими было предусмотрено 7-разрядное поле в РОС[14...08] – размер страницы в 64-байтных блоках.

Вариант со страницами фиксированного размера для 16-разрядной архитектуры был бы неоптимален ни при каких реально допустимых значениях размера страницы: к примеру, если взять размер 8 Кбайт – это была бы слишком большая величина для 16-разрядного адресного пространства, минимально возможный размер программы при этом (1/8 от общего объёма ОЗУ, градация выделения блоков памяти) был бы крайне неэкономичным расходом дефицитного ресурса. Если же выбирать размер страницы существенно меньшим, например, 512 байт (хотя это тоже не особо экономичное расходование ОЗУ, в 8 раз «грубее» 64-байтной страницы), это потребовало бы  $64K/0.5K = 128$  регистровых пар (по схеме PAR/PDR) или структуры в ОЗУ эквивалентного размера  $128 * 2 * 2 = 512$  байт (page table), с неизменно сопутствующей такому решению структурой типа TLB<sup>1</sup>, без которой очень сильно падает производительность при работе системы с памятью.

Страница с переменным размером – очень красивое и изящное решение, предложенное разработчиками архитектуры PDP-11: и минимальный размер выделения блоков физической памяти достаточно экономный – 64 байта, и «таблица страниц» была реализована не в ОЗУ, а в компактном регистровом файле, для чего потребовалось добавить в архитектуру CPU всего восемь регистровых пар PAR/PDR (РАС/РОС).

В следующем разделе, посвящённом архитектуре VAX-11, будут приведены более подробные рассуждения и расчёты по выбору оптимального размера физической страницы в схеме с фиксированным размером.

## Программное обеспечение

Для PDP-11 было разработано множество операционных систем: DEC предлагала P/OS, RSX-11, RT-11, RSTS/E, также было доступно несколько вариантов DOS, и в конце-концов, на PDP-7 и PDP-11 в 1971 было сделано **первое издание ОС UNIX** (в Bell Laboratories), тогда ещё на ассемблере. PDP-11 ушла с рынка в течение 1980-х по одной, но неизбежной причине: недостаточный объём адресного пространства, знаменовавший переход на новую, 32-битную, но все ещё CISC, архитектуру VAX-11.

Очень популярная в своё время RT-11<sup>2</sup> по современным понятиям была, конечно, довольно примитивной – например, файловая система в ней была «плоской», без иерархической организации папок/каталогов. Файл на диске не мог состоять из разрозненных дисковых фрагментов и был организован в неразрывный поток байтов. Имена файлов могли

<sup>1</sup> **Буфер ассоциативной трансляции** (*Translation lookaside buffer, TLB*) — это специализированный кэш центрального процессора, используемый для ускорения трансляции адресов виртуальной памяти в адреса физической памяти.

<sup>2</sup> **RT** (Real Time) – однопользовательская ОС реального времени. Имела очень малые (гарантированные) времена реакции на прерывания, была очень «реактивной», в силу очень небольших накладных расходов на систему.

быть заданы только в формате NAME6.EXT3 – имя (в uppercase) до шести символов, расширение – до трёх. Основным «местом работы» ОС семейства RT-11 были машины без менеджера памяти, хотя старшая версия в линейке – RT-11XM – напротив, могла работать только на моделях с MMU, но программы для этой версии RT приходилось писать совершенно в другом стиле, по сравнению с машинами без MMU.

Младшие RT-11 – RT-11BL (base line), RT-11SJ (single job) – были однозадачными, зато имели самый компактный код ОС, размещённый в ОЗУ (которого «на всех», напомним, было всего максимум 56 Кбайт). Интересные факты, говорящие о качестве кода тогдашних разработчиков ОС RT-11 – консольный интерпретатор команд занимал в ОЗУ всего ~2 Кбайт, но при этом «знал» около 60 консольных команд, которые можно было вводить не полностью, а сокращать до минимальной длины, исключаящей неоднозначность. Например, команда просмотра содержимого текущего каталога могла быть введена как DIR, DIRE, DIREC или полностью DIRECTORY. При попытке выполнить команду DI интерпретатор сообщал о неоднозначности, т.к. существовала ещё команда DISABLE, начинающаяся с тех же двух символов. Если при загрузке программы в память ОЗУ не хватало, интерпретатор мог «подвинуться», освобождая 2 Кбайта, который он занимал в ЗУ. После окончания работы программы интерпретатор возвращался на место, система подгружала его с диска.

Интересно сравнить некоторые характеристики интерпретатора команд для другой ОС 16-разрядной архитектуры – MS-DOS. В первой версии DOS v1.0 command.com (интерпретатор команд в MS-DOS) занимал в ОЗУ около ~25 Кбайт, но при размере более чем в десять раз большем, чем интерпретатор команд в RT-11, распознавал только четыре или пять команд фиксированной длины: DIR, COPY, RENAME, ERASE...

В более продвинутых версиях RT – RT-11FB (foreground, background) была реализована даже многозадачная работа – без MMU, конечно, никакой межпрограммной защиты в системе быть просто не могло, все процессы работали в едином адресном пространстве. Но, тем не менее, в распоряжении программиста была возможность запускать суммарно до восьми задач – одну фоновую (ту единственную, которая была возможна в RT-11SJ) и до семи foreground задач. Система приоритетов процессов была фиксированной – пока работала задача с бóльшим номером, все задачи с меньшими номерами были остановлены. Фоновая задача имела самый низкий приоритет – 0.

Одним из главных достоинств ОС семейства RT-11 была её «легковесность» – относительно небольшие накладные расходы на систему позволяли добиваться очень малых времён реакции на события (прерывания), порядка 30-50 мкс. Скромная по сегодняшним меркам система, с тактовой частотой всего несколько мегагерц, могла обрабатывать прерывания, идущие с частотой в несколько килогерц и даже десятков килогерц. Обработка такого уровня загрузки по прерываниям весьма непростая задача и для современных многоядерных процессоров, с тактовыми частотами на три порядка более высокими.

Ещё одной любопытной ОС для архитектуры PDP-11 была система **TSX-Plus** (первоначально **TSX**) — разрабатывалась с конца 1970-х годов американской компанией S&H Computer Systems для компьютеров семейств PDP-11 и LSI-11.

Являясь расширением (фактически надстройкой) операционной системы RT-11, TSX-Plus предоставляла возможности многопользовательской, многозадачной работы с существенным расширением возможностей RT-11XM. Многозадачность системы была реализована с помощью адаптивного планировщика заданий. Работа планировщика основана на предоставлении каждой задаче выделенных квантов времени, определяемых приоритетом, установленным для каждой задачи.

TSX-Plus была многозадачной, многопользовательской системой, предоставляющей каждому пользователю как бы множество RT-11SJ – нажатием простой комбинации клавиш на клавиатуре пользователь открывал новую сессию, каждая из которых была практически неотличима от обычной однозадачной RT-11SJ. TSX не имела собственного дискового за-



грузчика, она запускалась, как простая задача, из обычной RT11 (чаще всего для этого использовалась ревизия RT-11FB), превращая однопользовательскую машину в многотерминальную, многозадачную, многопользовательскую систему разделения времени.

Самой мощной и сложной ОС, разработанной для PDP-11/LSI-11, была система RSX-11 (**R**esource **S**haring **eX**ecutor) – версия RSX-11S работала на машинах без MMU, версии RSX-11M и RSX-11M+ требовали наличия диспетчера памяти. Самая мощная и производительная версия RSX-11M+ была основной ОС для топового миникомпьютера PDP-11/70. ОС RSX-11M – это многозадачная, многопользовательская система разделения времени, с ограниченной поддержкой realtime. Система была заметно «тяжелее» RT-11 и, несмотря на обычно существенно более производительные процессоры, на которых она использовалась, обеспечивала заметно худшие времена реакции, по сравнению с более «лёгкой» RT-11.

В RSX-11 формат файловой системы заметно отличался от формата в RT-11: полное имя файла состояло из имени логического диска, директории (в ранних версиях числовой, в более поздних – символической), имени (до 8 символов), расширения (до трёх символов) и номера версии (байт):

```
RK5:[100,25]NAME.EXT;VER
```

Т.е., файловая система на каждом диске имела двухуровневую иерархию – на логическом диске «параллельно» существовали пользовательские директории, внутри которых файлы были организованы в «плоский» список. При обращении к файлу без указания версии доступ производился к файлу с наибольшим номером версии. Версия автоматически инкрементировалась на единицу при сохранении файла после редактирования/трансляции/линковки, обеспечивая многоуровневый откат в случае необходимости.

Все ОС для PDP-11 были хорошо поддержаны внушительным объёмом прикладного программного обеспечения: разнообразные текстовые редакторы, СУБД, трансляторы для всех популярных языков программирования – Фортран, Паскаль, BASIC, Lisp, Cobol и, конечно же язык Си, который по сути и был рождён на архитектуре PDP-11. Многие структуры в Си, типа `p++`, `--p`, `p+=n`, `p-=n` просто однозначно отображаются на методы адресации данных в системе команд PDP-11.

## Чему нас научила PDP-11?

Несколько любопытных фактов из истории семейства PDP-11, подобранных на сайте Хабр: [Чему нас научила PDP-11?](#) Речь идёт о середине 1970-х, когда была написана статья Гордона Белла<sup>1</sup>:

- **RISC** – вначале когда PDP-11 ещё не была разработана, и, тем более, не рассматривалась, как RISC-машина — этого термина не существовало до того, как его ввёл Джон Коук в 1976 по отношению к IBM 801. Однако для каждого, кто имел опыт программирования процессоров ARM, современных RISC-процессоров, их сходство очевидно. Так же, как дизайн языков программирования является процессом как эволюционного развития, так и заимствований, то же самое происходит с дизайном системы команд.

Также PDP-11 вбила кол прямо в сердце выделенных инструкций ввода-вывода, утвердив модель отображаемого на память ввода-вывода, что является самым распространённым механизмом на сегодня. Единственными процессорами, имевшими отдельные инструкции ввода-вывода, являются Intel 8080 и его двоюродный брат Z80.

---

<sup>1</sup> C. G. Bell, W. D. Strecker, "Computer What Have We Learned from the PDP-11," *The 3rd Annual Symposium on Computer Architecture Conference Proceedings*, pp. 1-14, 1976.

- **UNIX** – PDP-11 сильно повлияла на программное обеспечение и операционные системы. PDP-11 был машиной, на которой Кен Томпсон и Деннис Ритчи разработали UNIX в Bell Labs  
UNIX была изначально создана на PDP-7. В 1970 разработка переместилась на PDP-11/20, а позднее на PDP-11/45.  
До PDP-11 UNIX не существовал. До PDP-11 язык C не существовал, это был компьютер, для которого C и был разработан. Если вы хотите знать, почему в классическом C int имеет 16 бит, то это из-за PDP-11.  
UNIX принёс нам идеи каналов (pipes), «всё есть файл» и интерактивных вычислений.
- **VAX-11/780** – при интерактивных вычислениях, использование памяти не ограничено, и хотя PDP была хороша для управления процессами, требования интерактивных вычислений стали поводом для создания замены PDP-11. Через год после выхода статьи Белла, в 1977, был выпущен последователь PDP-11, VAX-11, что означало “virtual address extension”.
- **BSD** – UNIX пришёл в Беркли в 1974 году, ленту привёз Кен Томпсон, что стало началом развития на западном побережье BSD — Berkley Systems Distribution. Berkeley UNIX была портирована на VAX в начале 1980-х и процветала как контркультурная альтернатива DEC-овской операционной системе VMS. Berkeley UNIX породила новое поколение хакеров, которые впоследствии основали такие компании, как Sun Microsystems, и языки типа Self, который напрямую привёл к появлению Java.  
UNIX был портирован на огромное число компьютерных систем в течение 80-х и в результате «войны UNIX-ов» появились различные BSD-системы, существующие по сей день.
- **NeXT** – 4BSD, производная от оригинального дистрибутива Беркли, стала основой операционной системы семейства компьютеров NeXT от Стива Джобса. Когда Apple купила NeXT в 1997 году, NextSTEP и производная от BSD стали основой Darwin, OSX и iOS.
- **Windows NT** – как было написано (в статье на Хабре) выше, говоря об Эдисоне де Кастро, в DEC бывали и изменники.  
Дэйв Катлер, архитектор операционной системы VAX VMS, после провальной попытки начать новый комбинированный проект операционной системы и аппаратного обеспечения, предназначенного на замену VAX, ушёл в Microsoft в 1988, приведя с собой свою команду, и возглавил разработку Windows NT. Те, кто знает внутренности Windows и операционную систему VMS, возможно, заметят общие черты.
- **Xerox Alto** – чтобы завершить разговор об истории де Кастро, серия Data General Nova вдохновила Чарльза Такера и Батлера Лэмпсона, разработчиков в Ксерокс Альто, которые, в свою очередь, вдохновили разработчиков Apple Macintosh.
- **Data General Nova** – соперничество между Data General и DEC продолжилось и в 32-битную эру, о чём и рассказал Трейси Киддер, получивший в 1981 году Пулитцеровскую премию, в книге "[The Soul of a New Machine](#)".

## Чему мы научились у PDP-11?

Хотя разработка этой архитектуры была иногда хаотичной, и не лишена недостатков, PDP-11 является точкой пересечения многих нитей истории.

Аппаратное обеспечение, софт, языки программирования, операционные системы, на всё это повлияла PDP-11.

## Супермини ЭВМ VAX-11

Дэйв Катлер<sup>1</sup>, бывший в своё время главным архитектором ОС RSX-11 и одним из главных архитекторов ОС VAX VMS, в одном из комментариев в тексте программы писал: «Единственный недостаток архитектуры, который не может быть исправлен программно — это нехватка адресного пространства». Замечательная, красивая, ортогональная, продуманная и т.п. архитектура PDP-11 имела всего 64 Кбайт виртуального адреса, что быстро стало проблемой при разработке больших программных систем<sup>2</sup>.

Отчётливо осознавая ограничения, присущие PDP-11, в DEC к 1973 году было принято решение о разработке компьютера следующего поколения, концепции «wide-word machine». Первоначально рассматривалось два варианта: продолжение линии PDP-10 с шириной слова 36 бит (проект «Unicorn») и развитие линии PDP-11, но с удвоенной до 32 бит шириной слова и механизмами виртуальной памяти (проект «Star»).

Официальной датой рождения архитектуры VAX<sup>3</sup> вероятно можно считать 1 апреля 1975 года, дату первого совещания команды, впоследствии названной VAX-A. Через два месяца была готова первая версия описания архитектуры — «VAX Blue Book», а к июню 1975 года проект «Star» обогнал конкурента настолько, что руководство корпорации решило свернуть дальнейшую разработку проекта «Unicorn».

Первая серийная модель архитектуры VAX была представлена публике 25 октября 1977 года. Это была модель VAX-11/780 «Star» с процессором KA780 (29 печатных плат ТТЛ) и 4 МБ ОЗУ с коррекцией ошибок (ECC RAM). В качестве консольного процессора использовалась специализированная версия PDP-11. Машина была крайне коммерчески успешна. Первая модель серии VAX-11/780 некоторое время рассматривалась как эталон вычислительной машины с быстродействием 1 млн оп/с, который даже имел специальное название — VUP (VAX Unit Performance), а всё семейство VAX-11 стало называться супермини ЭВМ.

Далее последовала разработка «второй серии», уже не на рассыпной логике, а на вентильных массивах (gate array), и спустя примерно год после модели «Star» была представлена модель VAX-11/750 «Comet» (отличавшаяся также новой реализацией консоли на Intel 8085). В феврале 1978 была выпущена новая операционная система для VAX'ов, VMS (Virtual Memory System) версии 1.0.

## Программная архитектура VAX-11

VAX-11 разрабатывался как развитие архитектуры PDP-11, поэтому многие элементы PDP были перенесены в VAX, с увеличением их количества и расширенными характеристиками:

---

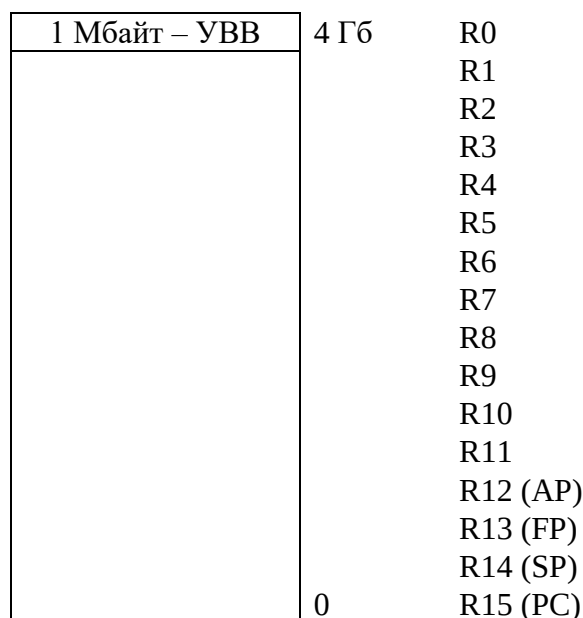
<sup>1</sup> В 1988 году Катлер, протестуя против решения руководства DEC закрыть проект **PRISM** (Parallel Instruction Set Machine), уволился из DEC, и перешёл с частью команды в Microsoft, где возглавил подразделение по разработке Windows NT.

<sup>2</sup> Чтобы «втиснуться» в маленькое адресное пространство, для больших программ приходилось пользоваться техникой оверлеев, когда два и более участков кода размещались по одним и тем же физическим адресам, экономя общий занимаемый объём памяти. Естественно, что размещающиеся в одних и тех же адресах подпрограммы не должны были напрямую обращаться друг к другу, так как одновременно в ОЗУ мог находиться только один оверлей.

<sup>3</sup> **VAX** (Virtual Address eXtension) — 32-битная компьютерная архитектура, была разработана в середине 1970-х годов Digital Equipment Corporation как развитие линии PDP-11 в рамках проекта Star.

1. Разрядность обрабатываемых данных была увеличена с 16 до 32 разрядов;
2. Прямоадресуемая виртуальная память выросла с тех же 16 разрядов до 32, т.е. виртуальное адресное пространство выросло в  $2^{16}=64K$  раз;
3. Количество 32-х разрядных РОН было увеличено вдвое – с 8 до 16;
4. Количество методов адресации увеличить вдвое не получилось – их количество возросло с 8 до 13. Интересно, что разработчики архитектуры отказались от косвенно-автодекрементного метода метода адресации – как показал анализ кода PDP-11, это был наименее востребованный метод, практически не встречающийся в реальном коде.

Программная модель VAX-11 довольно проста, главным принципиальным отличием от PDP-11 в части физической организации была многошинная структура системы, хотя существовали microVAX первого поколения, которые работали в «корзинах» с единственной шиной Q-bus, с естественными ограничениями, свойственными этой шине – ширина шины передачи данных 16 разрядов, максимальная адресуемая физическая память – 4 Мб (22 разряда), а с другой стороны, старшие модели PDP-11 также имели многошинную структуру, например в PDP-11/70 присутствовали шины Unibus и Massbus.



Ресурсы архитектуры: физический объём ОЗУ (RAM) до 1 Гбайт ( $2^{30}$  байт). Из виртуального адресного пространства  $2^{32}=4$  Гбайт последний (старший) 1 Мбайт отведён под адресацию устройств ввода/вывода. Шестнадцать регистров общего назначения (РОН). Четыре регистра со старшими номерами имеют специализацию: R12 – AP (указатель аргументов), R13 – FP (указатель блока вызова), R14 – SP (указатель стека), R15 – PC (счётчик команд).

### Общая схема VAX-11

Старшие модели первых поколений VAX-11 (11/780) были построены на основе новой синхронной шины SBI (Synchronous Backplane Interconnect), использующей технологии ТТЛШ – это основной путь обмена данными CPU↔RAM, хотя к этой шине также было возможно подключение контроллеров периферийных устройств с соответствующим интерфейсом.

SBI является основным каналом управления и передачи данных в системе 11/780. Поскольку кэш и буфер записи в значительной степени снижают зависимость производительности процессора от времени доступа к памяти, дизайн SBI был оптимизирован для пропускной способности и надёжности, а не для минимизации возможного времени доступа. SBI – синхронная шина с временем цикла 200нс. Ширина тракта передачи данных SBI составляет 32 бита. В течение каждого цикла 200нс можно передавать либо 32 бита данных, либо 30-битный физический адрес. Поскольку каждое 32-разрядное чтение или запись требует передачи как адреса, так и данных, для полной транзакции используются два цикла SBI. Протокол SBI разрешает 64-битное чтение или запись с использованием одного цикла адреса и двух циклов передачи данных: ЦПУ и подсистема ввода-вывода используют этот режим всякий раз, когда это возможно. Для транзакций чтения шина повторно используется (reacquired) памятью для отправки данных: таким образом, шина не удерживается в течение всего времени доступа к памяти.

Арбитраж на SBI статический, распределённый: каждый интерфейс в SBI имеет определённый приоритет и собственную линию запроса шины (параллельное распределённое разрешение). Когда интерфейс желает захватить шину, он инициирует (asserts) свою линию запроса шины. Если в конце цикла 200нс нет интерфейсов с более высоким приоритетом, запрашивающих шину, интерфейс получает контроль над шиной. Время реакции на запрос не зависит от физического местоположения интерфейса, как это бывает в схеме с последовательной daisy chain, где присутствует задержка распространения сигнала подтверждения по цепочке.

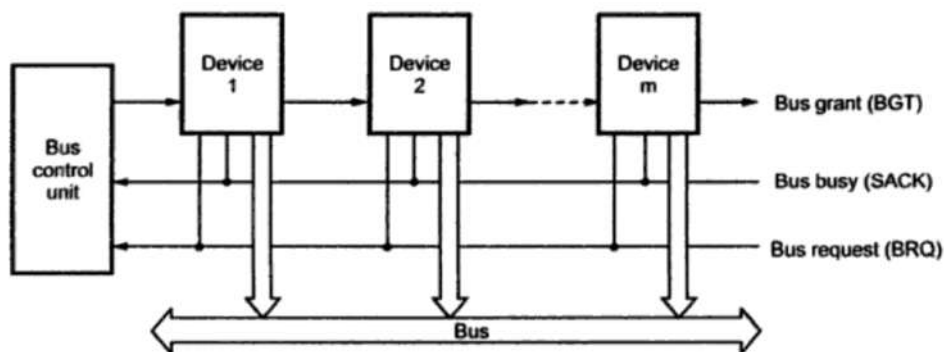


Рис.176 - Реализация статической daisy chain в системной шине

На SBI выполняются интенсивные проверки, например, в каждой пересылке осуществляется проверка на чётность. Процесс арбитража и общее соблюдение протокола SBI проверяются каждым интерфейсом SBI в течение каждого цикла SBI. Процессор поддерживает 16-тактную историю SBI: любое состояние ошибки SBI блокирует и сохраняет эту историю для диагностических целей.

Кроме SBI в VAX-11/780 присутствовали ещё два типа шин для подключения периферийных устройств: один адаптер Unibus и до четырёх Massbus адаптеров.

*Unibus* – мультиплексированная асинхронная 16-разрядная шина среднего уровня производительности. Использовалась как основная системная и периферийная шина во многих моделях PDP-11. Имеет 18-битное физическое адресное пространство, поддерживает передачу байтов и шестнадцатиразрядных слов. В дополнение к реализации обычной Unibus по передаче данных и трансляции сигналов запросов прерывания в CPU, адаптер Unibus обеспечивает две дополнительные функции. Первая функция служит для отображения 18-разрядных адресов Unibus в 30-битные физические адреса шины SBI и обратно. Преобразование осуществляется по схеме, практически идентичной схеме преобразования виртуального адреса в CPU в физический. Адрес Unibus пространства разделён на 512 512-байтовых страниц<sup>1</sup>.

<sup>1</sup> VAX-11 использовал физические страницы ОЗУ фиксированного размера 512 байт.

Каждая страница Unibus имеет запись в таблице страниц (хранящейся в адаптере Unibus), которая отображает адреса этой страницы на адреса физической памяти. По сути адаптер Unibus↔SBI вводит аппаратную поддержку защиты от несанкционированного доступа к «чужой» памяти для периферийных устройств, работающих по DMA, практически по схеме обычного MMU. В дополнение к трансляции адресов, мэпирование позволяет смежные пересылки (contiguous transfers) на Unibus, которые пересекают границы страниц, мэпируя их так, чтобы быть сопоставленными с несмежными фреймами страниц физической памяти.

Вторая функция, выполняемая адаптером Unibus, заключается в сборке 16-разрядных трансферов Unibus (как для чтения, так и для записи) в 64-разрядные пересылки SBI. Эта операция (которая применима только к блочным обменам, например, с дисковыми устройствами) значительно сокращает трафик SBI, связанный с операциями Unibus. В адаптере Unibus присутствуют 158-байтные буферы, допускающие 15 одновременных буферизованных транзакций. Кроме того, существует небуферизованный путь через адаптер Unibus, позволяющий выполнять произвольное количество одновременных небуферизованных передач.

*Massbus* – это высокоскоростная блочная шина, используемая в основном для подключения блочных дисковых и ленточных устройств. Разработана в 1970-х в компании DEC, использовалась в миниЭВМ PDP-10, PDP-11 старших моделей и VAX. Адаптер Massbus обеспечивает практически тот же функционал, что и адаптер Unibus. Физические адреса, на которые осуществляются передачи, определяются таблицей страниц: опять же, это позволяет преобразовывать смежные трансферы устройства в несмежную физическую память.

Буферизация обеспечивается в адаптере Massbus, который минимизирует вероятность переполнения устройства и компонует данные в 64-битные блоки для передачи по SBI.

Massbus логически реализована как две отдельные части:

- Асинхронная шина управления, используемая для доступа к отмэпированным на память регистрам ввода/вывода (*memory-mapped I/O registers*) в индивидуальном устройстве хранения.
- Высокоскоростная синхронная шина, использующаяся для передачи данных между устройством хранения и адаптером шины хоста (*host bus adapter*). Ширина шины данных 16/18 бит плюс паритет. 16-битная шина использовалась для PDP-11 и VAX, 18-битная – для систем DEC-10.
- Множество устройств различных типов могли пересылать данные по разделяемому пути (*shared data path*). Однако, этот функционал никогда не был поддержан в DEC-ких ОС.
- Статический двойной порт реализовывал отказоустойчивость, также давая возможность ручного переключения устройства хранения на другой CPU.

Ниже, на Рис.177 представлена типичная блок-схема VAX-11/780:

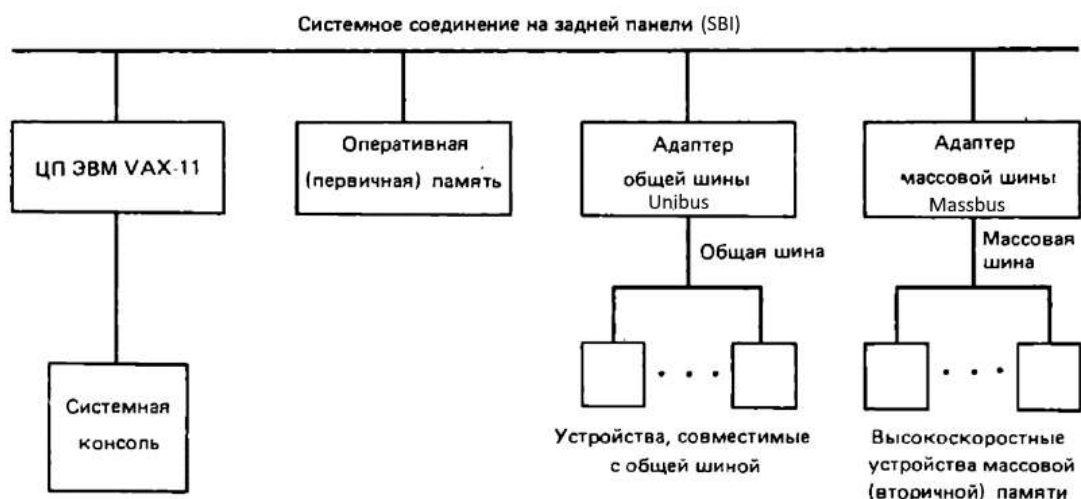
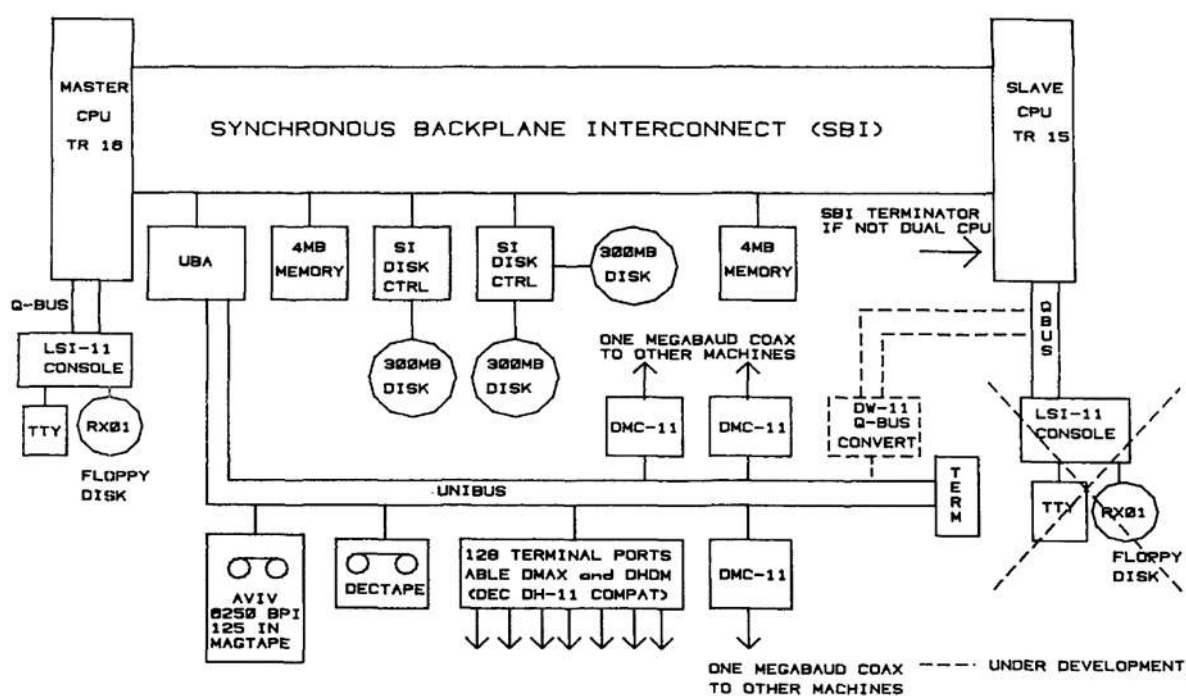


Рис.177 - Общая схема VAX-11/780

На Рис.178 приведена схема двухпроцессорного VAX-11/780, установленного в своё время в школе электротехники и вычислительной техники Университета Пердью (Purdue EE School).



Purdue EE School Dual Processor VAX-11/780.

Рис.178 - Двухпроцессорный VAX-11/780 в школе электротехники и вычислительной техники Университета Purdue

Обращает на себя внимание, что при наискромнейших на сегодня цифрах ёмкости ОЗУ  $4 + 4 = 8$  Мбайт, дисковой памяти суммарным объёмом всего  $300 + 300 + 300 = 1.2$  Гбайт и производительности  $2 * 1$  VUPs ( $\sim 2$  млн оп/сек) заявлена поддержка до 128 терминальных интерактивных пользователей. Современные ОС на таких объёмах RAM даже загрузиться не смогут, разве только самые урезанные по функционалу версии.

## Система команд VAX-11

VAX-11 поддерживал огромный набор инструкций (только основных команд насчитывалось 248 штук). Увеличенное вдвое машинное слово позволило разработчикам архитектуры реализовать команды, которые имели до шести(!) операндных модификаторов (к, например, в PDP-11 было возможно максимум два операндных модификатора). Можно сказать, что VAX-11 был, пожалуй, самым известным и мощным представителем CISC-архитектуры. Основной целью набора инструкций VAX-11 было обеспечение эффективного кода, сгенерированного трансляторами с языков высокого уровня.

Количество методов адресации по сравнению с PDP-11 формально выросло в полтора раза – с 8 до 13, основную добавку сделали три варианта смещения: байт, слово (16 бит), длинное слово (32 бита). Но вместе с тем был убран косвенно-автодекрементный метод (эквивалент 5-му методу адресации в PDP-11), как практически невостребованный в реальных программах.

### Режимы адресации с использованием регистров общего назначения

На Рис.179 приведена таблица методов (режимов) адресации с использованием регистров общего назначения:

| Режим адресации                                       | Байт специ-<br>фикатора | Код<br>операции | Пример<br>на языке<br>ассемблера |
|-------------------------------------------------------|-------------------------|-----------------|----------------------------------|
| Регистровый                                           |                         | 53 D4           | CLRL R3                          |
| Регистровый косвенный                                 |                         | 63 D4           | CLRL (R3)                        |
| С автоуменьшением                                     |                         | 73 D4           | CLRL -(R3)                       |
| С автоувеличением                                     |                         | 83 D4           | CLRL (R3)+                       |
| Со смещением (байт)                                   | xx                      | A3 D4           | CLRL D(R3)                       |
| Со смещением (слово)                                  | xxxx                    | C3 D4           | CLRL D(R3)                       |
| Со смещением (длинное слово)                          | xxxxxxxx                | E3 D4           | CLRL D(R3)                       |
| Относительный индексный<br>(смещение – байт)          | xx                      | AF 43 D4        | CLRL A[R3]                       |
| Относительный индексный<br>(смещение – слово)         | xxxx                    | CF 43 D4        | CLRL A[R3]                       |
| Относительный индексный<br>(смещение – длинное слово) | xxxxxxxx                | EF 43 D4        | CLRL A[R3]                       |

Примечание. Символ A обозначает адрес, символ D обозначает смещение. Формат смещения – байт, слово или длинное слово выбирается ассемблером в зависимости от значения смещения.

Рис.179 - Режимы адресации с использованием регистров общего назначения

### Литеральный режим адресации

На Рис.179 приведены режимы адресации, имеющие коды 4, 5, 6, 7, 8, A, C и E. Очевидно, что незадействованные коды можно использовать для других целей. Байт спецификатора операнда, имеющий значения от ^X00 до ^X3E, используется для задания *литерального режима*. Отличительным признаком этого режима является то, что два старших бита в байте спецификатора операнда содержат нули. В этом случае оставшиеся шесть битов рассматриваются как заданное непосредственно 6-битовое число без знака. Это позволяет описывать в одном байте непосредственно заданные операнды, имеющие значения в диапазоне 0-63. Значение литерала должно быть положительным. Ассемблер автоматически выбирает литеральный режим для небольших по значению положительных непосредственно заданных операндов. Например, при ассемблировании следующей инструкции:

```
MOVL    #5, R0
```

могут быть получены следующие три байта машинного кода:

```
50      05      D0
```



Инструкция с большими по значению непосредственно заданными операндами, такая как

```
MOVL    #64, R0
```

ассемблируется с использованием режима непосредственной адресации и будет занимать 7 байтов:

```
50      00000040      8F      D0
```

Поскольку литерал не имеет знака, то желательно избегать непосредственного задания небольших отрицательных операндов. Это можно сделать, если правильно выбрать инструкцию. Например, инструкция MNEGL #5, R0 более предпочтительна, чем инструкция MOVL #-5, R0, так как в машинном коде первая инструкция будет занимать три байта в ОЗУ, а вторая – семь байтов.

## Режимы косвенной адресации

В этом режиме содержимое регистра не используется как операнд, а рассматривается как адрес операнда. Существует ещё четыре режима косвенной адресации, которые могут использоваться в сочетании с режимом с автоувеличением и с тремя режимами адресации со смещением (Рис.180).

| Режим адресации                        | Байт спецификатора | Код операции | Пример на языке ассемблера |
|----------------------------------------|--------------------|--------------|----------------------------|
| Косвенный с автоувеличением            | 93                 | D4           | CLRL @(R3)+                |
| Косвенный со смещением (байт)          | B3                 | D4           | CLRL @D(R3)                |
| Косвенный со смещением (слово)         | D3                 | D4           | CLRL @D(R3)                |
| Косвенный со смещением (длинное слово) | F3                 | D4           | CLRL @D(R3)                |

Примечание. Символ D обозначает смещение. Ассемблер автоматически в зависимости от его значения выберет формат смещения – байт, слово или длинное слово.

Рис.180 - Режимы косвенной адресации

На Рис.181 приведена сводная таблица режимов адресации с использованием регистров общего назначения в ЭВМ семейства VAX:

| Режим адресации                           | Байт специ-<br>фикатора | Код<br>операции | Пример на языке<br>ассемблера |
|-------------------------------------------|-------------------------|-----------------|-------------------------------|
| Литеральный                               | 50 05                   | D0              | MOVL #5, R0*                  |
| Индексный                                 | ... xx 43               | D4**            | CLRL A[R3]                    |
| Регистровый                               | 53                      | D4              | CLRL R3                       |
| Регистровый косвенный                     | 63                      | D4              | CLRL (R3)                     |
| С автоуменьшением                         | 73                      | D4              | CLRL -(R3)                    |
| С автоувеличением                         | 83                      | D4              | CLRL (R3)+                    |
| Косвенный с автоувеличением               | 93                      | D4              | CLRL @(R3)+                   |
| Со смещением (байт)                       | xx A3                   | D4              | CLRL D(R3)                    |
| Косвенный со смещением (байт)             | xx B3                   | D4              | CLRL @D(R3)                   |
| Со смещением (слово)                      | xxxx C3                 | D4              | CLRL D(R3)                    |
| Косвенный со смещением (слово)            | xxxx D3                 | D4              | CLRL @D(R3)                   |
| Со смещением (длинное слово)              | xxxxxxxx E3             | D4              | CLRL D(R3)                    |
| Косвенный со смещением<br>(длинное слово) | xxxxxxxx F3             | D4              | CLRL @D(R3)                   |

\* Литеральный режим разрешен только для операнда-источника.

\*\* Обозначение ... xx указывает соответствующий режим адресации для A.

Примечание. Символ D обозначает смещение. Формат смещения — байт, слово или длинное слово выбирается ассемблером в зависимости от значения смещения.

Рис.181 - Режимы адресации с использованием регистров общего назначения в ЭВМ семейства VAX

В каждом случае косвенной адресации адрес вычисляется точно так же, как это делается в случае режимов прямой адресации. По этому адресу выбирается длинное слово, и содержимое этого длинного слова используется как *адрес* операнда инструкции. В случае косвенной адресации с автоувеличением значение содержимого регистра увеличивается на четыре. Это делается независимо от типа инструкции, потому что в регистре всегда содержится некоторый адрес, а адрес всегда занимает длинное слово. Такие режимы косвенной адресации используются в основном для работы с таблицами адресов. Адреса в таблице являются адресами операндов или более сложных структур данных.

## Режимы адресации с использованием программного счётчика

Режимы адресации с использованием регистра PC (^XF) называются *режимами адресации с использованием программного счётчика* и перечислены на Рис.182. Непосредственная адресация иллюстрируется инструкцией MOVL, а для других режимов адресации в качестве примера приводится инструкция CLRL. Обратите внимание на символы @# во второй строке: они используются в инструкции CLRL @#A для указания режима абсолютной адресации. Байты спецификатора операнда в этой таблице содержат коды ^X8F, ^X9F, ^XAF, ^XCF и ^XEF. Везде код ^XF указывает, что для определения адреса операнда используется программный счётчик. Символы x, следующие за байтом спецификатора операнда, показывают, сколько шестнадцатеричных цифр, обозначающих дополнительную информацию, должно стоять после байта спецификатора операнда.

| Режим адресации                             | Байт специ-<br>фикатора | Код<br>операции | Пример на языке<br>ассемблера |
|---------------------------------------------|-------------------------|-----------------|-------------------------------|
| Непосредственный                            | 50 xxxxxxxx 8F          | D0              | MOVL *K, R0*                  |
| Абсолютный                                  | xxxxxxx 9F              | D4              | CLRL @*A                      |
| Относительный (смещение —<br>байт)          | xx AF                   | D4              | CLRL A                        |
| Относительный (смещение —<br>слово)         | xxxx CF                 | D4              | CLRL A                        |
| Относительный<br>(смещение — длинное слово) | xxxxxxx EF              | D4              | CLRL A                        |

\* Непосредственная адресация разрешена только для операнда-источника.

Примечание. Символ A обозначает адрес, символ K обозначает константу. Формат смещения — байт, слово или длинное слово выбирается ассемблером в зависимости от значения смещения адреса A.

Рис.182 - Режимы адресации с использованием программного счётчика

#### Режим адресации с использованием регистра PC

#### Реализация

|                                          |                                                                                               |
|------------------------------------------|-----------------------------------------------------------------------------------------------|
| Непосредственный                         | Режим с автоувеличением с использованием регистра PC                                          |
| Абсолютный                               | Режим косвенной адресации с автоувеличением с использованием регистра PC                      |
| Относительный (смещение - байт)          | Режим адресации со смещением, заданным в формате байта, с использованием регистра PC          |
| Относительный (смещение - слово)         | Режим адресации со смещением, заданным в формате слова, с использованием регистра PC          |
| Относительный (смещение - длинное слово) | Режим адресации со смещением, заданным в формате длинного слова, с использованием регистра PC |

На Рис.183 приводится полный список режимов адресации с использованием программного счётчика, реализованных на ЭВМ семейства VAX.

| Режим адресации                               | Байт специ-<br>фикатора | Код<br>операции | Пример на языке<br>ассемблера |
|-----------------------------------------------|-------------------------|-----------------|-------------------------------|
| Непосредственный                              | 50 xxxxxxxx 8F          | D0              | MOVL *K, R0*                  |
| Абсолютный                                    | xxxxxxx 9F              | D4              | CLRL @*A                      |
| Относительный (смещение — байт)               | xx AF                   | D4              | CLRL A                        |
| Относительный косвенный (смещение —<br>байт)  | xx BF                   | D4              | CLRL @A                       |
| Относительный (смещение — слово)              | xxxx CF                 | D4              | CLRL A                        |
| Относительный косвенный<br>(смещение — слово) | xxxx DF                 | D4              | CLRL @A                       |
| Относительный (смещение —<br>длинное слово)   | xxxxxxx EF              | D4              | CLRL A                        |
| Относительный косвенный<br>(смещение — байт)  | xxxxxxx FF              | D4              | CLRL @A                       |

\* Непосредственная адресация разрешена только для операнда-источника.

Примечание. Символ A обозначает адрес, символ K — число. Формат смещения — байт, слово, или длинное слово выбирается ассемблером в зависимости от значения смещения адреса A.

Рис.183 - Режимы адресации с использованием программного счётчика в ЭВМ семейства VAX

## Инструкции VAX-11

Как уже упоминалось выше, системы VAX поддерживали очень большое количество инструкций, в общей сложности несколько сотен. Многие инструкции существовали в

двух- и трёхоперандных версиях, например, команда сложения содержимого двух регистров в двухоперандной моде затирает исходное содержимое регистра-получателя, а в трёхоперандной моде результат  $C = A + B$  помещается в третий регистр-получатель (C).

Ниже, на Рис.184, приведён список набора инструкций VAX-11 в максимально сжатом виде. К сожалению, не удалось найти источник с более качественным изображением.

|                                                           |                                                    |
|-----------------------------------------------------------|----------------------------------------------------|
| <b>Integer and Floating Point Logical Instructions</b>    |                                                    |
| MOV-                                                      | Move(B,W,L,F,D,Q) <sup>†</sup>                     |
| MNEG-                                                     | Move Negated(B,W,L,F,D)                            |
| MCOM-                                                     | Move Complemented(B,W,L)                           |
| MOVZ-                                                     | Move Zero-Extended(BW,BL,WL)                       |
| CLR-                                                      | Clear(B,W,L=F,Q=D)                                 |
| CVT-                                                      | Convert(B,W,L,F,D)(B,W,L,F,D)                      |
| CVTR-L                                                    | Convert Rounded(F,D) to Longword                   |
| CMP-                                                      | Compare(B,W,L,F,D)                                 |
| TST-                                                      | Test(B,W,L,F,D)                                    |
| BIS-2                                                     | Bit Set(B,W,L)2-Operand                            |
| BIS-3                                                     | Bit Set(B,W,L)3-Operand                            |
| BIC-2                                                     | Bit Clear(B,W,L)2-Operand                          |
| BIC-3                                                     | Bit Clear(B,W,L)3-Operand                          |
| BIT-                                                      | Bit Test(B,W,L)                                    |
| XOR-2                                                     | Exclusive OR(B,W,L)2-Operand                       |
| XOR-3                                                     | Exclusive OR(B,W,L)3-Operand                       |
| ROTL                                                      | Rotate Longword                                    |
| PUSHL                                                     | Push Longword                                      |
| <b>Integer and Floating Point Arithmetic Instructions</b> |                                                    |
| INC-                                                      | Increment(B,W,L)                                   |
| DEC-                                                      | Decrement(B,W,L)                                   |
| ASH-                                                      | Arithmetic Shift(L,Q)                              |
| ADD-2                                                     | Add(B,W,L,F,D)2-Operand                            |
| ADD-3                                                     | Add(B,W,L,F,D)3-Operand                            |
| ADWC                                                      | Add with Carry                                     |
| ADAWI                                                     | Add Aligned Word Interlocked                       |
| <b>Branch on Bit Instructions</b>                         |                                                    |
| BLB-                                                      | Branch on Low B(S,Cl)                              |
| BB-                                                       | Branch on Bit(S,Cl)                                |
| BBS-                                                      | Branch on Bit Set and(S,Cl)Bit                     |
| BBC                                                       | Branch on Bit Clear and(Set,Clear)Bit              |
| BBSI                                                      | Branch on Bit Set and Set Bit Interlocked          |
| BBCI                                                      | Branch on Bit Clear and Clear Bit Interlocked      |
| <b>Queue Instructions</b>                                 |                                                    |
| INSQUE                                                    | Insert Entry in Queue                              |
| REMQUE                                                    | Remove Entry from Queue                            |
| <b>Address Manipulation Instructions</b>                  |                                                    |
| MOVA-                                                     | Move Address(B,W,L=F,Q=D)                          |
| PUSHA-                                                    | Push Address(B,W,L=F,Q=D)on Stack                  |
| <b>Processor State Instructions</b>                       |                                                    |
| PUSHR                                                     | Push Registers on Stack                            |
| POPR                                                      | Pop Registers from Stack                           |
| MOVPSL                                                    | Move from Processor Status Longword                |
| BISPSW                                                    | Bit Set Processor Status Word                      |
| BICPSW                                                    | Bit Clear Processor Status Word                    |
| <b>Unconditional Branch and Jump Instructions</b>         |                                                    |
| BR-                                                       | Branch with(B,W)Displacement                       |
| JMP                                                       | Jump                                               |
| <b>Branch on Condition Code</b>                           |                                                    |
| BLSS                                                      | Less Than                                          |
| BLSSU                                                     | Less Than Unsigned                                 |
| (BCS)                                                     | (Carry Set)                                        |
| BLEQ                                                      | Less Than or Equal                                 |
| BLEQU                                                     | Less Than or Equal Unsigned                        |
| BEQ                                                       | Equal                                              |
| (BEQU)                                                    | (Equal Unsigned)                                   |
| BNEQ                                                      | Not Equal                                          |
| (BNEQU)                                                   | (Not Equal Unsigned)                               |
| BCTR                                                      | Greater Than                                       |
| BCTRU                                                     | Greater Than Unsigned                              |
| BGEQ                                                      | Greater Than or Equal                              |
| BGEQU                                                     | Greater Than or Equal Unsigned                     |
| (BCC)                                                     | (Carry Clear)                                      |
| BVS                                                       | Overflow Set                                       |
| BVC                                                       | Overflow Clear                                     |
| <b>Loop and Case Branch</b>                               |                                                    |
| ACB-                                                      | Add, Compare and Branch(B,W,L,F,D)                 |
| AOBLEQ                                                    | Add One and Branch Less Than or Equal              |
| AOBLSS                                                    | Add One and Branch Less Than                       |
| SOBGEQ                                                    | Subtract One and Branch Greater Than or Equal      |
| SOBGTR                                                    | Subtract One and Branch Greater Than               |
| CASE-                                                     | Case on(B,W,L)                                     |
| <b>Privileged Processor Register Control Instructions</b> |                                                    |
| SVPCTX                                                    | Save Process Context                               |
| LDPCTX                                                    | Load Process Context                               |
| MTPR                                                      | Move to Process Register                           |
| MTPR                                                      | Move from Processor Register                       |
| <b>Special Function Instructions</b>                      |                                                    |
| CBC                                                       | Cyclic Redundancy Check                            |
| BPT                                                       | Breakpoint Fault                                   |
| XPC                                                       | Extended Function Call                             |
| NOP                                                       | No Operation                                       |
| HALT                                                      | Halt                                               |
| <b>Subroutine Call and Return Instructions</b>            |                                                    |
| BSB                                                       | Branch to Subroutine with(B,W,) Displacement       |
| JSB                                                       | Jump to Subroutine                                 |
| RSB                                                       | Return from Subroutine                             |
| <b>Procedure Call and Return Instructions</b>             |                                                    |
| CALLG                                                     | Call Procedure with General Argument List          |
| CALLS                                                     | Call Procedure with Stack Argument List            |
| RET                                                       | Return from Procedure                              |
| <b>Access Mode Instructions</b>                           |                                                    |
| CHM                                                       | Change Mode to (Kernel,Executive,Supervisor, User) |
| <b>Index Instruction</b>                                  |                                                    |
| INDEX                                                     | Compute Index                                      |
| <b>Packed Decimal Instructions</b>                        |                                                    |
| MOVP                                                      | Move Packed                                        |
| CMPP3                                                     | Compare Packed 3-Operand                           |
| CMPP4                                                     | Compare Packed 4-Operand                           |
| ASHP                                                      | Arithmetic Shift Round and Packed                  |
| ADDP4                                                     | Add Packed 4-Operand                               |
| ADDP6                                                     | Add Packed 6-Operand                               |
| SUBP4                                                     | Subtract Packed 4-Operand                          |
| SUBP6                                                     | Subtract Packed 6-Operand                          |
| MULP                                                      | Multiply Packed                                    |
| DIVP                                                      | Divide Packed                                      |
| CVTLP                                                     | Convert Long to Packed                             |
| CVTPL                                                     | Convert Packed to Long                             |
| CVTPT                                                     | Convert Packed to Trailing                         |
| CVTTP                                                     | Convert Trailing to Packed                         |
| CVTPS                                                     | Convert Packed to Separate                         |
| CVTSP                                                     | Convert Separate to Packed                         |
| EDTPC                                                     | Edit Packed to Character String                    |
| <b>Character String Instructions</b>                      |                                                    |
| MOVCS                                                     | Move Character 3-Operand                           |
| MOVCS                                                     | Move Character 5-Operand                           |
| MOVTC                                                     | Move Translated Characters                         |
| MOVTUC                                                    | Move Translated Unit Character                     |
| CMPC3                                                     | Compare Characters 3-Operand                       |
| CMPC5                                                     | Compare Characters 5-Operand                       |
| LOCC                                                      | Locate Character                                   |
| SKPC                                                      | Skip Character                                     |
| SCANC                                                     | Scan Characters                                    |
| SPANC                                                     | Span Characters                                    |
| MATCHC                                                    | Match Characters                                   |
| <b>Variable-Length Bit Field Instructions</b>             |                                                    |
| EXTV                                                      | Extract Field                                      |
| EXTZV                                                     | Extract Zero-Extended Field                        |
| INSV                                                      | Insert Field                                       |
| <b>Integer and Floating Point Arithmetic Instructions</b> |                                                    |
| SUB-2                                                     | Subtract(B,W,L,F,D)2-Operand                       |
| SUB-3                                                     | Subtract(B,W,L,F,D)3-Operand                       |
| SBWC                                                      | Subtract with Carry                                |
| MUL-2                                                     | Multiply(B,W,L,F,D)2-Operand                       |
| MUL-3                                                     | Multiply(B,W,L,F,D)3-Operand                       |
| EMUL                                                      | Extended Multiply                                  |
| DIV-2                                                     | Divide(B,W,L,F,D)2-Operand                         |
| DIV-3                                                     | Divide(B,W,L,F,D)3-Operand                         |
| EDIV                                                      | Extended Divide                                    |
| EMOD-                                                     | Extended Modulus(F,D)                              |
| POLY-                                                     | Polynomial Evaluation(F,D)                         |
| CMFV                                                      | Compare Field                                      |
| CMFZV                                                     | Compare Zero-Extended Field                        |
| FFS                                                       | Find First Set                                     |
| FFC                                                       | Find First Clear                                   |

Рис.184 - Набор инструкций VAX-11

## Типы и форматы данных

Количество типов данных, поддерживаемых в VAX-11, существенно превосходит количество типов данных в архитектуре PDP-11, но форматы данных (представление в ОЗУ) у этих двух архитектур совпадают.

Типы данных VAX-11 делятся на пять основных классов:

1. Целые типы данных: *байт* 8 бит, *слово* 16 бит, *длинное слово* 32 бита и *квадраслово* 64 бита.
2. Данные с плавающей точкой (float): одинарная точность 32 бита и двойная точность 64 бита. Эти типы данных нормализованы, имеют 8 битовую экспоненту со знаком, и 25- или 57-битовую дробную часть со знаком, причём старший значащий разряд дробной части скрыт<sup>1</sup>.
3. Переменные битовые поля: биты от 0 до 32, расположенные произвольно относительно адресуемых границ байтов. Битовое поле задаётся тремя операндами: адресом байта, начальной позицией бита Р относительно нулевого бита этого байта и размером поля S поля. Поле может интерпретироваться как знаковое или беззнаковое.
4. Тип данных *символьная строка*, от 0 до  $2^{16}-1=65535$  подряд расположенных байт. Строка задаётся двумя операндами – начальный адрес строки и длина. Хотя тип данных называется «символьная строка», никакой специальной интерпретации значений байт в строке не производится.
5. Тип данных *десятичная строка*, от 0 до 31 десятичной цифры. Определяется двумя операндами: длина (в цифрах) и начальным адресом. Основной тип данных *упакованный десятичный*, хранит в каждом байте две десятичных цифры, за исключением байта, содержащего наименее значимую цифру – в нём содержится одна десятичная цифра и знак. Поддерживаются два десятичных типа символов ASCII: ведущий отдельный знак (leading separate sign) и конечный встроенный знак (trailing embedded sign). Ведущий отдельный знак может быть «+», «-» или пустым (<blank>, эквивалентно «+») ASCII символом, за которым следует от 0 до 31 десятичной цифры. Завершающий встроенный знак – это десятичная строка от 0 до 31 байта длиной, которые являются десятичными ASCII цифрами, за исключением байта, содержащего наименее значимую цифру, в которой хранится одна десятичная цифра и знак числа.

| Data type                 | Size                      | Range (decimal)                                            |                 |
|---------------------------|---------------------------|------------------------------------------------------------|-----------------|
|                           |                           | Signed                                                     | Unsigned        |
| Integer                   |                           |                                                            |                 |
| Byte                      | 8 bits                    | -128 to +127                                               | 0 to 255        |
| Word                      | 16 bits                   | -32768 to +32767                                           | 0 to 65535      |
| Longword                  | 32 bits                   | $-2^{31}$ to $+2^{31}-1$                                   | 0 to $2^{32}-1$ |
| Quadword                  | 64 bits                   | $-2^{63}$ to $+2^{63}-1$                                   | 0 to $2^{64}-1$ |
| Floating point            |                           | $\pm 2.9 \times 10^{-3}$ to $1.7 \times 10^{38}$           |                 |
| Floating                  | 32 bits                   | approximately seven decimal digits precision               |                 |
| Double Floating           | 64 bits                   | approximately sixteen decimal digits precision             |                 |
| Packed decimal string     | 0 to 16 bytes (31 digits) | numeric, two digits per byte sign in low half of last byte |                 |
| Character string          | 0 to 65535 bytes          | one character per byte                                     |                 |
| Variable-length bit field | 0 to 32 bits              | dependent on interpretation                                |                 |

Рис.185 - Типы данных VAX-11

<sup>1</sup> В представлении float VAX-11 имеет точность в дробной части на один (скрытый) разряд точнее, чем IEEE-754. В своё время эти форматы конкурировали на рынке, но в итоге победил менее точный и менее логично организованный стандарт IEEE-754, и в поздних моделях VAX и в процессоре Alpha DEC вынуждена была использовать этот формат представления float, чтобы быть совместимым с международным стандартом.



Все типы данных могут храниться в памяти по произвольным границам байтов, нет требований к выравниванию. Поля могут начинаться с произвольной битовой границы.

Атрибуты и символическое представление основных типов данных приведены на Рис.185 и Рис.186.

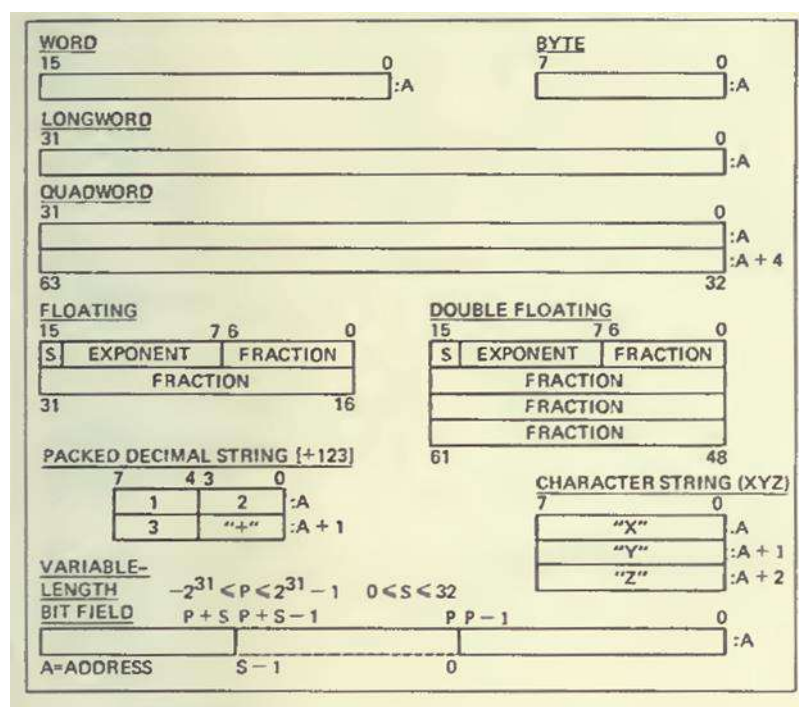


Рис.186 - Форматы данных VAX-11

## Прерываемые инструкции

Когда поступает запрос на прерывание, процессор может быть занят выполнением очередной инструкции, и обычно, перед тем как передать управление процедуре обработки прерывания, процессор завершает выполнение текущей инструкции. Однако сложные «длинные» инструкции, такие как инструкции обработки символьных строк, представляют потенциальную проблему. Такие инструкции могут выполняться довольно продолжительное время, а многие высокоприоритетные события не могут ждать, пока завершится выполнение таких команд (даже на вычислительных системах VAX пересылка 65536 байтов может занять значительное время).

Для решения проблемы инструкции с потенциально большим временем выполнения разрабатывались так, что они могут прерываться до завершения своего выполнения. Такие инструкции называются *прерываемыми инструкциями*. При возврате управления прерванной программе, выполнение прерванной инструкции продолжается с той точки, на которой оно было приостановлено (в отличие от PDP-11, где прерываемая инструкция каждый раз пыталась выполниться с начала).

При прерывании прерываемой инструкции во время её выполнения необходимо сохранить промежуточные результаты, так чтобы её выполнение возобновилось с той точки, где она была прервана. Если было бы необходимо начинать выполнение инструкции каждый раз с начала, то серии прерываний могли бы задержать завершение инструкции на неопределённое время. Поскольку прерываемые инструкции во время выполнения используют регистры R0...R5 для хранения промежуточных результатов, то восстановление после прерывания является быстрым и эффективным. К прерываемым инструкциям относятся инструкции семейства POLY, а также инструкции обработки символьной информации и упакованных десятичных чисел.

## Защита и привилегии

В любой многопользовательской системе для исключения случайного или преднамеренного влияния пользователей друг на друга должны быть реализованы различные механизмы защиты. В то же время, для системных процессов необходимо иметь возможность обходить защиту и другие ограничения, так чтобы они могли выполнять такие действия, как загрузка и выгрузка процессов пользователей, или позволять процессам пользователей взаимодействовать между собой.

### Режимы доступа

Для упрощения разделения привилегий, которые может иметь программа, на ЭВМ семейства VAX имеются четыре уровня привилегий, действие которых проявляется при выполнении программы в ОЗУ системы. Эти уровни привилегий называются *режимами доступа*, поскольку они предназначены для контроля за тем, к какого рода операциям программа имеет доступ. Ниже перечислены режимы доступа в порядке снижения уровня привилегий:

- режим ядра,
- режим управления,
- режим супервизора,
- режим пользователя.

В длинных словах смещения страницы имеются некоторые дополнительные биты, используемые для защиты памяти. Отдельные страницы могут использоваться беспрепятственно, другие могут быть открыты для чтения, но защищены от записи, а к третьим доступ может полностью запрещён. Такая организация защиты распространяется на все режимы доступа, так что для различных режимов могут существовать и различные типы защиты. Таким образом, для любой страницы возможно установить биты защиты так, чтобы программы, выполняющиеся в режиме пользователя, не имели к ней доступа, программы, выполняющиеся в режиме супервизора, имели доступ по чтению, а программы, выполняющиеся в режимах управления и ядра, имели доступ по чтению и записи. Режимы доступа образуют строгую иерархию. Поэтому программа, выполняющаяся в старших режимах, имеет доступ как у любой привилегии младших режимов доступа (не меньше). Если программа в режиме пользователя имеет доступ к странице по чтению, то и все программы в старших режимах доступа могут иметь такой доступ. Привилегия доступа по записи подразумевает наличие привилегии доступа на чтение. На Рис.187 показано, как используются биты 27 - 30 элемента таблицы страниц для задания прав доступа к странице для различных режимов доступа. В таблице нет ни одной записи, которая бы позволила программам, выполняющимся в младших режимах доступа, иметь большее право доступа, чем программам, выполняющимся в старших режимах.

Кроме того, что режимы доступа влияют на доступ к памяти, они также влияют и на использование инструкций. Например, *резервные инструкции* для чтения и записи содержимого регистра управления процессора и для загрузки или сохранения контекста процесса могут выполняться только в режиме ядра. Любая попытка выполнить эти инструкции в любом другом режиме вызовет *прерывание по резервной инструкции*. Резервные инструкции являются инструкциями, которые для своего использования требуют специальных привилегий.

| Код  | Режимы          |            |             |              |
|------|-----------------|------------|-------------|--------------|
|      | Ядра            | Управления | Супервизора | Пользователя |
| 0000 | нет             | нет        | нет         | нет          |
| 0001 | не используется |            |             |              |
| 0011 | Чт/Зап          | только Чт  | нет         | нет          |
| 0100 | только Чт       | нет        | нет         | нет          |
| 0101 | Чт/Зап          | Чт/Зап     | Чт/Зап      | Чт/Зап       |
| 0110 | Чт/Зап          | Чт/Зап     | нет         | нет          |
| 0111 | Чт/Зап          | только Чт  | нет         | нет          |
| 1000 | Чт/Зап          | Чт/Зап     | Чт/Зап      | нет          |
| 1001 | Чт/Зап          | Чт/Зап     | только Чт   | нет          |
| 1010 | Чт/Зап          | только Чт  | только Чт   | нет          |
| 1011 | только Чт       | только Чт  | только Чт   | нет          |
| 1100 | Чт/Зап          | Чт/Зап     | Чт/Зап      | только Чт    |
| 1101 | Чт/Зап          | Чт/Зап     | только Чт   | только Чт    |
| 1110 | Чт/Зап          | только Чт  | только Чт   | только Чт    |
| 1111 | только Чт       | только Чт  | только Чт   | только Чт    |

Обозначения: Чт/Зап - разрешен доступ по Чтению/Записи  
только Чт - разрешен доступ только по Чтению  
нет - все виды доступа запрещены

Рис.187 - Коды защиты памяти по доступу

Операционная система VAX/VMS устроена так, что операции разного рода выполняются в различных режимах доступа. Следующий список даёт представление об использовании режимов доступа.

| Режим        | Применение                                                                                        |
|--------------|---------------------------------------------------------------------------------------------------|
| Пользователя | Программы пользователей, утилиты, компиляторы, отладчики                                          |
| Супервизора  | Интерпретатор командного языка                                                                    |
| Управления   | Вызов системных сервисных подпрограмм и функций подсистемы RMS                                    |
| Ядра         | Управление страницами виртуальной памяти, планирование процессов, физический уровень ввода-вывода |

## Регистры процессора

Кроме 16 регистров общего назначения процессора (R0, R1, ...) на ЭВМ семейства VAX имеется ряд регистров специального назначения. Эти регистры служат для управления внутренней работой ЭВМ семейства VAX. В их число входят такие регистры, как указатели таблиц страниц виртуальной памяти. Большинство из этих регистров доступны только программам, выполняющимся в режиме ядра.

Один регистр должен быть упомянут специально, он называется *длинное слово состояния процессора* PSL (PSW в PDP-11). Этот регистр содержит поля битов, в которые заносится информация различного рода о состоянии выполняющегося процесса. Например, два бита определяют уровень доступа процесса 0, 1, 2 или 3 для режима ядра, управления, супервизора или пользователя соответственно. Ещё 2 бита сохраняют информацию о предыдущем режиме, данная информация используется инструкциями PROBE. Ещё 4 бита содержат коды условий – биты N, Z, V и C.

При выполнении программы биты регистра PSL изменяются. Например, любая инструкция, оказывающая влияние на коды условий, будет заносить информацию в поле битов кодов условий. Чтобы изменить режим доступа, необходимо занести в регистр PSL соответствующую информацию. И, конечно, для этого требуются некоторые привилегии. Однако можно прочитать содержимое регистра PSL с помощью инструкции MOVPSL. По инструкции MOVPSL содержимое регистра PSL копируется в указанное длинное слово. Ин-



струкцию MOVPSL можно использовать для определения текущего или предыдущего режима доступа, а также для получения некоторой другой информации о состоянии процессора. Ниже, на Рис.188, показано назначение отдельных битов регистра PSL.

| Биты    | Назначение                                                                                                                                                                 |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0       | Бит C                                                                                                                                                                      |
| 1       | Бит V                                                                                                                                                                      |
| 2       | Бит Z                                                                                                                                                                      |
| 3       | Бит N                                                                                                                                                                      |
| 4       | Бит трассировки, может использоваться для эмуляции пошагового выполнения программы при отладке                                                                             |
| 5       | Бит разрешения прерывания по переполнению целого числа                                                                                                                     |
| 6       | Бит разрешения прерывания по потере значимости числа с плавающей точкой                                                                                                    |
| 7       | Бит разрешения прерывания по переполнению десятичного числа                                                                                                                |
| 8 – 15  | Должны содержать 0                                                                                                                                                         |
| 16 – 20 | Уровень приоритета прерывания                                                                                                                                              |
| 21      | Должен содержать 0                                                                                                                                                         |
| 22 – 23 | Предыдущий режим работы процессора                                                                                                                                         |
| 24 – 25 | Текущий режим работы процессора                                                                                                                                            |
| 26      | Флаг стека прерываний                                                                                                                                                      |
| 27      | Флаг выполнения первой части инструкции; некоторые инструкции, занимающие при выполнении много времени, могут быть прерваны на середине (признак неоконченного выполнения) |
| 28 – 29 | Должны содержать 0                                                                                                                                                         |
| 30      | Бит задержки трассировки                                                                                                                                                   |
| 31      | Бит режима совместимости, если он установлен, то процессор будет выполнять инструкции ЭВМ серии PDP-11                                                                     |

Рис.188 - Длинное слово состояния процессора PSL

Младшие 16 битов (биты 0-15) не содержат привилегированной информации и поэтому могут быть модифицированы программой пользователя (за исключением битов 8-15, которые не используются и всегда должны иметь значение 0). Младшие 16 битов иногда называются *словом состояния процессора PSW*. Младшие 8 битов PSW могут модифицироваться; установка и сброс этих битов осуществляются с помощью инструкций BISPSW или BICPSW. Например, инструкция

```
BISPSW    #^X20
```

устанавливает бит разрешения прерывания по переполнению целого числа. Если затем какая-либо инструкция программы установит при выполнении бит V, то прерывание по переполнению целого числа приведёт к прекращению выполнения программы и будет выдано сообщение об ошибке. Установка этого бита может помочь при определении корректности целочисленных вычислений, но может и создать проблему для каких-либо программ, в которых используются арифметические операции над целыми числами без знака.

Для ускорения обмена между программами с различными привилегиями и помощи операционной системе в организации защиты в системе имеется пять различных стеков. Этими стеками являются: стек режима пользователя, стек режима супервизора, стек режима управления, стек режима ядра и стек прерываний. Подразумевается, что соответственно имеется пять регистров указателей стека. При работе в одном из режимов доступа, всякий раз, когда инструкция обращается к регистру SP явным или не явным образом, используется соответствующий режиму указатель стека. Выбор первых четырёх указателей стека определяется текущим режимом доступа. Указатель стека прерываний используется, если установлен бит 26 регистра PSL. Обычно программа может получить доступ только к своему собственному указателю стека – стеку режима пользователя. Тем не менее имеются две инструкции MTPR и MFPR (переслать в регистр процессора и переслать из регистра процессора), которые позволяют программам, выполняющимся в режиме ядра, получить доступ ко всем пяти регистрам-указателям стека.

Эти инструкции также используются для доступа к регистрам базы и длины, которые определяют таблицы страниц виртуальных адресов для пространства P0, пространства P1 и пространства системы. Кроме того, существует ещё целый ряд регистров, к которым можно получить доступ с помощью этих инструкций.

## Управление памятью, виртуальная память

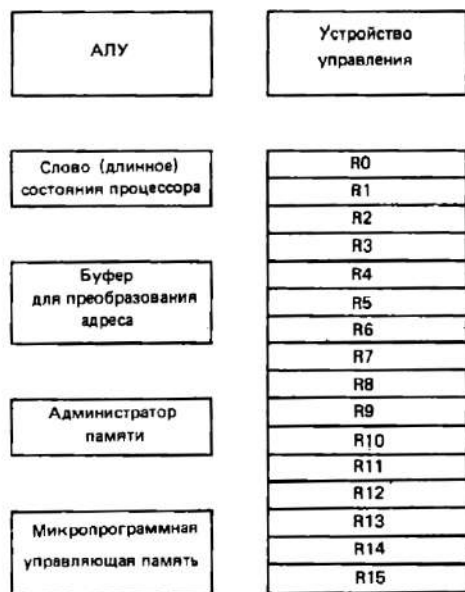


Рис.189 - Центральный процессор VAX-11

На Рис.189 представлена блок-схема центрального процессора VAX-11. В отличие от своего предшественника PDP-11, все модели VAX имели поддержку работы с виртуальной памятью. По сравнению с PDP-11 в схеме CPU VAX-11 присутствует три новых блока: буфер для преобразования адреса, администратор памяти и опциональная микропрограммная память управления.

Блок буфера преобразования адресов является компонентом системы виртуальной памяти VAX-11.

Блок *администратора памяти* образуется из аппаратных и программных модулей. Его основные функции:

1. Отображение или трансляция адресов виртуальной памяти в физические и обеспечение того, чтобы для каждой пользовательской программы это было максимально «прозрачно», чтобы каждая программа «думала», что она работает не в виртуальном адресном пространстве, а в большой физической памяти, с непрерывными адресами, начинающимися с нуля;
2. Распределение свободных сегментов физической памяти между программами на равноправной основе;
3. Обеспечение защиты конфиденциальности каждого процесса.

## Организация виртуальной памяти

Полное 32-х разрядное виртуальное адресное пространство делится на четыре региона, по 1 ГБайт каждый (Рис.190). Номер региона определяют два старших бита счётчика команд PC (31...30).

| Адрес<br>(в шестнадцатеричном<br>представлении) | Содержимое                |
|-------------------------------------------------|---------------------------|
| 0000 0000<br>:<br>3FFF FFFF                     | Регион P0<br>(программа)  |
| 4000 0000<br>:<br>7FFF FFFF                     | Регион P1<br>(управление) |
| 8000 0000<br>:<br>BFFF FFFF                     | Системный регион          |
| C000 0000<br>:<br>FFFF FFFF                     | Зарезервировано           |

Рис.190 - Адресное пространство виртуальной памяти

Регион P0 предназначен для программ, у которых значение адреса изменяется в сторону увеличения, т.е., в сторону больших адресов.

Регион P1 обычно отводится для программных структур, у которых адреса растут в сторону меньших адресов, например, стек, который обычно растёт «сверху вниз».

Системный регион предназначен для программ операционной системы.

Четвёртый регион зарезервирован для будущего использования, т.е. получается, что четырёхгигабайтное виртуальное адресное пространство поделено пополам: нижние 2 ГБайт — пользовательское пространство, старшие 2 ГБайт — системное. Интересно, что абсолютно аналогично было распределено вир-

туальное адресное пространство в 32-х разрядных версиях Windows NT, что неудивительно, т.к. у истоков разработки ОС VAX/VMS и Windows NT стояли одни и те же разработчики и архитекторы систем.

Для отображения виртуальных адресов в физический использовался ставший каноническим механизм страничного преобразования адресов. Всё виртуальное адресное пространство ( $2^{32}=4$  Гбайт) делилось на страницы относительно небольшого размера 512 байт ( $2^9$ ). В отличие от PDP-11, где страница, отображённая на физическую память, могла иметь длину от 64 байт до 8 Кбайт, в VAX-11 размер страницы был фиксированным.

Каждый регион в виртуальном адресном пространстве разбит на  $2^{21}=2$ млн. страниц, номер (адрес) страницы определяется разрядами адреса A29...9 (Рис.191).

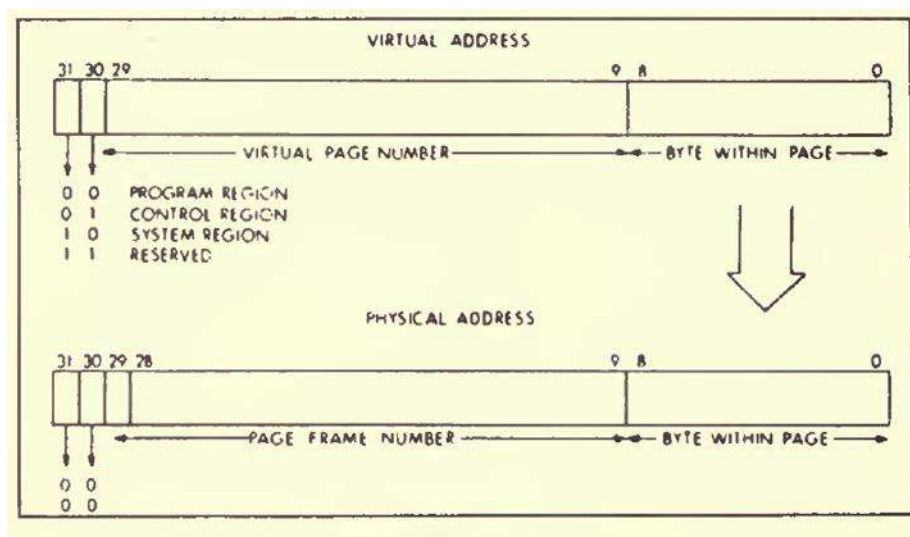


Рис.191 - Виртуальное и физическое адресные пространства VAX-11

При разработке архитектуры компьютера очень важно выбрать оптимальный размер страницы памяти: чем меньше страница, тем более экономно будет расходоваться физическое адресное пространство (меньше внутренняя фрагментация<sup>1</sup>). Но при этом, из-за увеличения количества страниц, возрастают сопутствующие накладные расходы. Для описания страниц в памяти заводится специальная структура, т.н. дескрипторная таблица, записи в которой описывают параметры каждой страницы, режимы доступа к ней и т.п. Размер записи в дескрипторной таблице не зависит от размера страницы, но общий объём памяти, занимаемый дескрипторными таблицами, естественно, будет больше в случае увеличения количества страниц.

Последнее утверждение может быть проанализировано с математической точки зрения. Пусть средний размер процесса будет составлять  $s$  байт, а размер страницы —  $p$  байт. Кроме этого предположим, что на каждую страничную запись требуется  $e$  байт (дескрипторная таблица). Тогда приблизительное количество страниц, необходимое каждому процессу, будет равно  $s/p$ , что займет  $se/p$  байт пространства таблицы страниц. Из-за внутренней фрагментации неиспользуемое пространство памяти в последней странице процесса будет равно  $p/2$ . Таким образом, общие издержки на таблицу страниц и внутреннюю фрагментацию будут получены за счет суммирования этих двух величин:

$$\text{Издержки} = se/p + p/2.$$

Первое слагаемое (размер таблицы страниц) будет иметь большее значение при меньшем размере страницы. Второе слагаемое (внутренняя фрагментация) будет иметь

<sup>1</sup> В среднем половина последней страницы останется незаполненной. Оставшееся неиспользуемое пространство на этой странице тратится впустую. Эти потери называются **внутренней фрагментацией**.

бóльшее значение при большем размере страницы. Оптимальный вариант находится где-то посередине. Если взять первую производную по переменной  $p$  и приравнять ее к нулю, то мы получим уравнение

$$-se/p^2 + 1/2 = 0.$$

Из этого уравнения можно вывести формулу, дающую оптимальный размер страницы (с учетом только потерь памяти на фрагментацию и на размер таблицы страниц).

Результат будет следующим:

$$p = \sqrt{2se}$$

Для  $s = 1$  Мбайт и  $e = 8$  байт на каждую запись в таблице страниц оптимальный размер страницы будет 4 Кбайт. Различные архитектуры используют размер страницы от 512 байт до 64 Кбайт. В ранних архитектурах использовался размер 1 Кбайт, но сейчас чаще всего встречается размер страницы 4 Кбайт и больше.

Почему архитекторы VAX-11 не стали делать страницу меньше 512 байт, чтобы управлять распределением физической памяти с меньшими потерями на внутреннюю фрагментацию? Ответ лежит в технологии обмена страницами ОЗУ ↔ Диск (paging), который производится под управлением ОС. Минимальная единица обмена данными с жёстким диском – сектор, размер которого у дисков составлял тогда стандартные 512 байт. Поэтому делать страницу меньшего размера не имело практического смысла – скорость обмена ОЗУ ↔ Диск от этого бы не возросла – даже если бы вы «подкачивали» страницу с диска, например, размером в 256 байт, это бы всё равно привело к чтению полного 512-байтного сектора, и никакого выигрыша по скорости обмена не было бы.

## **АЛГОРИТМЫ УПРАВЛЕНИЯ СТРАНИЦАМИ**

ОС использует специальные алгоритмы для выбора страницы, выгружаемой на внешнее дисковое устройство (paging), при возникновении прерывания по отсутствию страницы в памяти (с диска «подкачивается» нужная страница, и размещается на месте выгружаемой). Когда принимается решение о том, какая страница должна быть выгружена, важно выбрать страницу, которая вряд ли потребуется сразу. Неудачный подбор страниц для выгрузки может привести к тому, что при выполнении каждой инструкции будет происходить прерывание по отсутствию страницы. Эта ситуация может отразиться на выполнении программы; выполнение будет настолько медленным, как если бы внешнее дисковое устройство выступало в роли основной памяти.

Идеальным выбором страницы, предназначенной для выгрузки, могла быть страница памяти, которая не понадобится ещё долгое время. К сожалению, это невозможно предсказать в масштабе реального времени. Поэтому алгоритмы базируются на эвристических методах. Одним из таких алгоритмов является алгоритм выбора дольше всего не использовавшейся страницы. Предположение строится на том, что если страница давно не использовалась, то вряд ли она потребуется в ближайшее время. Могут быть случаи, когда это предположение не оправдывается, но в среднем посылка правильная. Однако этот алгоритм весьма дорогой как с точки зрения технических средств, так и программного обеспечения. Для определения того, какая страница давно не использовалась, может потребоваться, чтобы каждый регистр страницы имел таймер, автоматически устанавливающийся при очередном обращении к странице. Кроме того, должен осуществляться поиск регистра страницы с наиболее ранней установкой таймера.

Один простой алгоритм для выбора выгружаемой страницы состоит в том, что производится случайный выбор страницы. Другой простой алгоритм основан на последовательном выборе страниц. На практике простой алгоритм может работать ненамного хуже более сложного и дорогого алгоритма. (Следует отметить, что простые алгоритмы могут

быть улучшены, если повысить вероятность предпочтительного выбора страниц программ с меньшим приоритетом).

Если бы MMU VAX-11 использовал механизм, подобный тому, что был реализован в менеджере памяти PDP-11 – регистровая пара на каждую страницу (всего  $8 \times 2 = 16$  регистров) – это привело бы к необходимости иметь регистровый файл невероятных размеров:

$$2^{32} (4 \text{ Гбайт}) / 2^9 (512 \text{ байт}) = 2^{23} (8 \text{ млн})$$

Понятно, что единственным местом, где можно хранить такие объёмы – это ОЗУ. Но тут возникает проблема «яйца и курицы» – чтобы обратиться к ОЗУ, виртуальный адрес должен подвергнуться преобразованию по таблице страниц, которая хранится... в ОЗУ. Очевидно, что процессор должен иметь возможность обращаться в ОЗУ по физическим адресам, минуя преобразование адресов. Для доступа к регистрам базы и длины, которые определяют таблицы страниц виртуальных адресов для пространства P0, пространства P1 и пространства системы используются привилегированные инструкции MTPR и MFPR.

### **БУФЕР ТРАНСЛЯЦИИ**

Для того чтобы ускорить преобразование виртуальных адресов в реальные адреса, применяется быстродействующее устройство памяти, называемое *буфером трансляции*. Буфер трансляции предназначен для хранения наиболее часто используемых адресов. Работа этого быстродействующего устройства памяти для процесса трансляции адресов аналогична работе обычной кэш-памяти. Также, как и кэш-память, буфер трансляции является весьма дорогой частью аппаратуры ЭВМ и может быть включен или отсутствовать в различных моделях ЭВМ семейства VAX. Однако, подобно кэш-памяти, наличие буфера трансляции *прозрачно* для программиста, т.е. является для него скрытым. Программист не может сказать, имеет ли ЭВМ кэш-память или буфер трансляции, наличие этих устройств отражается только на увеличении скорости выполнения программы.

Особенностью внутреннего устройства, присущей в различной форме как кэш-памяти, так и буферу трансляции является *ассоциативная память*. Обычная память с произвольной выборкой информации имеет дешифратор адреса, который преобразует адрес так, чтобы можно было выбрать одну из N ячеек памяти. Например, память объёмом 4 млн. длинных слов могла бы иметь дешифратор адреса, который позволил бы выбрать одну ячейку памяти из 4 млн. ячеек. Дешифратор работает так просто потому, что число ячеек памяти в точности соответствует числу возможных доступных адресов.

Напротив, ассоциативная память имеет относительно небольшое число ячеек, которые соответствуют адресам из значительно большего адресного пространства. Это означает, что адресов намного больше, чем ячеек этой памяти. Как следствие, некоторые адреса не соответствуют какой-либо ячейке. Иначе говоря, не существует полного отображения адресов на ячейки памяти.

Это такая же ситуация, которая возникает и с буфером трансляции, поскольку ограниченный объём буфера позволяет вместить только наиболее часто используемые адреса. Если бы буфер трансляции был реализован программно, алгоритм поиска требовал бы включения многочисленных сравнений. Такой поиск был бы медленным и буфер трансляции не оправдал бы своего назначения. Для решения данной проблемы буфер трансляции адреса страницы реализуется аппаратно с помощью большого числа параллельно работающих аппаратных схем сравнения, обеспечивая высокую скорость доступа к хранимым данным. Буфер (или память), реализованный с помощью параллельных схем сравнения для выполнения очень быстрого поиска, называется *ассоциативной памятью*. Методы проектирования ассоциативной памяти применяются при разработке разнообразных частей больших вычислительных машин, включая в том числе буферы трансляции адреса страницы и кэш-память.

## Программное обеспечение

VAX-11 с момента «рождения» этой архитектуры (которую иногда называют расширением PDP-11) был оснащён качественным и мощным программным обеспечением, как системным – в первую очередь «родная» DEC-ая операционная система VAX/VMS<sup>1</sup> (VAX Virtual Memory System), мощная многопользовательская ОС с элементами realtime, с развитым командным языком (DCL – DEC Command Language<sup>2</sup>), качественными трансляторами со всех популярных языков программирования, мощными текстовыми редакторами процессорами. Компания DEC активно разрабатывала сетевую поддержку в VMS – Интернет тогда был фактически в зачаточном состоянии и DEC активно продвигала свою сетевую инфраструктуру – DECnet – которая сильно конкурировала с IBM-кой SNA. Интересно, что DEC стояла у истоков ещё одной технологии – Ethernet<sup>3</sup>. В 1984 году DEC создала свой первый 10 Мбит/с Ethernet. Совместив DECnet и терминальные серверы на основе Ethernet, DEC создала архитектуру сетевого хранения данных, которая позволила им конкурировать с IBM. Ethernet практически полностью вытеснил технологию Token ring от IBM и стал основой для сегодняшних локальных сетей.

## Файловая система

Файловая система в VMS имеет иерархическую организацию каталогов (папок). В отличие от unix-подобных ОС, в синтаксисе VMS каталог явно отличим от файла. Полное имя файла в сетевой иерархии выглядело так:

```
NODE"password"::device:[directory.subdirectory]filename.type;ver
```

Самая левая часть – имя (адрес) узла, как он «видится» в сети DECnet<sup>4</sup>, дальше, после двойного двоеточия – логическое имя устройства, отделённое двоеточием от квадратных скобок, в которых располагалась явно выделенная структура каталогов, перечисленных через точку, дальше идёт имя, расширение и версия (подобно версиям в файловой системе RSX Files-11), но в VMS под неё отведено два байта, а не один байт, как в RSX. Также, как в RSX-11, версия файла увеличивается на единицу при каждом обновлении файла, вместо перезаписи исходного файла.

В первых версиях файловой системы VMS (ODS) длина имени файла и расширения была ограничена 39 символами ASCII в верхнем регистре, а глубина иерархии каталогов была ограничена восемью уровнями. В последующих версиях был расширен допустимый набор символов, включая lowercase и Unicode, увеличена длина имени и расширения.

## Безопасность файлов: защита и ACL

Безопасность в файловой системе определяется двумя механизмами, контроль доступа, базирующийся на UIC<sup>5</sup> и контроль на основе ACL<sup>6</sup>. UIC основывается на владельце файла и UIC или пользователе, доступующемуся к файлу. Доступ определяется четырьмя группами разрешений:

---

<sup>1</sup> Первая свободная (free) альтернативная ОС для VAX – NetBSD/vax (порт NetBSD) – была анонсирована в 1977 году и поставлялась до 30 сентября 2000г.

<sup>2</sup> Основными разработчиками DCL были разработчики интерпретатора команд для ОС RT-11, перешедшие в команду VAX/VMS. Язык сильно отличался по синтаксису от исходного интерпретатора команд в RSX-11 (MCR).

<sup>3</sup> Самый популярный тип фрейма Ethernet, в который обычно упаковывается протокол TCP/IP, называется Ethernet-II или DIX (DEC-Intel-Xerox).

<sup>4</sup> В 1980-ые в ИЯФ СО РАН функционировал самый большой в СССР DECNet кластер.

<sup>5</sup> UIC – user identification code, код идентификации пользователя.

<sup>6</sup> Access Control List или ACL — список управления доступом, который определяет, кто или что может получать доступ к объекту (программе, процессу или файлу), и какие именно операции разрешено или запрещено выполнять субъекту (пользователю, группе пользователей).

- System
- Owner
- Group
- World

**Рис.192 - Формат отображения защиты файлов; не предоставленные разрешения не отображаются**

и четырьмя битами разрешений<sup>1</sup>:

- Read
- Write
- Execute
- Delete

«Системный» уровень доступа применим к любому пользователю, у которого код группы в UIC меньше или равен SYSGEN параметру MAXSYSGROUP (обычно это 8 или 10 в восьмеричном коде) (например, пользователь SYSTEM); «владелец» и «группа» применимы к владельцу файла и к его группе, к которой он принадлежит, «мир» (“world”) относится ко всем остальным пользователям. Существует также пятый бит разрешения, «control», который используется для определения доступа для изменения метаданных файла, таких как защита. Эта группа не может быть установлена явно; она всегда имеет значение «System» и «Владелец», и никогда не может быть «Группа» или «Мир».

На контроль доступа, базирующийся на UIC, влияют также четыре системных привилегии, которые позволяют пользователю, обладающему ими, переопределять управление доступом:

- BYPASS: пользователь неявно имеет полный (RWED) доступ ко всем файлам, независимо от защиты файлов;
- READALL: пользователь неявно имеет доступ на чтение (R) ко всем файлам;
- SYSPRV: пользователь может достигаться к файлам, основываясь на защите System;
- GRPPRV: пользователь может достигаться к файлам, основываясь на защите System если его группа UIC совпадает с группой файлов.

ACLs позволяют назначить назначить пользователю или группе дополнительные привилегии, основанные на пользователе или группе; например, UIC веб-сервера может получить доступ на чтение всех файлов в определённой директории. ACLs могут быть отмечены как *наследуемые* (*inherited*), в этом случае ACL директории применяется ко всем файлам уровнем ниже. Списки доступа (ACLs) модифицируются командами EDIT/ACL, и формируются в виде пар идентификатор/доступ (identifier/access). Пример записи ACL:

| System | Owner | Group | World |
|--------|-------|-------|-------|
| RWED   | RWED  | RWED  | RWED  |

(IDENTIFIER=HTTP\$SERVER,ACCESS=READ+EXECUTE)

что позволяет пользователю HTTP\$SERVER читать и исполнять файлы.

## Развитие архитектур DEC

В начале 1980-х DEC была на вершине финансового могущества, в значительной мере благодаря стабильно растущим объемам поставок VAX'ов. Однако уже тогда в компании существовало понимание, что дальнейший рост производительности машин с CISC-

<sup>1</sup> Очень напоминает юниксовские триады: rwx (чтение, запись, исполнение) для owner-group-other.



архитектурой (а VAX был своего рода апофеозом CISC подхода) имеет естественные пределы, в которые создатели CISC-архитектур практически уже упёрлись. Инженеры DEC создавали систему команд для VAX в то время, когда преобладало программирование на ассемблере, отчасти из-за инженерной инерции («мы всегда так делали»), а отчасти потому, что рудиментарные высокоуровневые компиляторы в то время генерировали машинный код, который проигрывал составленному вручную лаконичному коду ассемблера. Инструкции VAX поддерживали большое количество удобных для программиста режимов адресации и включали в себя отдельные машинные инструкции, выполнявшие сложные операции, такие как вставка/удаление очереди и вычисление полинома.

Огромное количество действий, логически оформляемых программистом как отдельная операция, ложилось ровно на одну команду. Не нужно было делать всякие предварительные загрузки и последующие выгрузки, укладки в конкретные регистры (за редкими несущественными исключениями). Инженеры VAX были в восторге от разработки аппаратного обеспечения, облегчавшего жизнь программистов. Микрокод позволил легко добавлять новые инструкции к системе команд — и 99-разрядные микропрограммное управление VAX раздулось до 4096 слов. Но, спустя некоторое время, несколько инженеров в DEC обнаружили, что около 20% инструкций CISC у VAX занимают около 80% микрокода, но составляют всего 0,2% общего времени выполнения программы. Стало очевидно, что архитектура CISC не такая уж замечательная.

Со временем пришло понимание, что и VAX-у придётся уйти с рынка, как это уже произошло с PDP-11. В то время многие компании начали обращать внимание на перспективные разработки в среде RISC, и DEC явно не желала остаться в стороне. С 1982 по 1985 в составе компании активно работали несколько подразделений, занимавшихся исследованиями в области RISC:

- **Titan**, высокоскоростной дизайн от Западной исследовательской лаборатории (Western Research Laboratory, DECwest) в Пало-Альто (Калифорния), под руководством Фореста Бэскетта (Forest Baskett), с 1982;
- **SAFE** (Streamline Architecture for Fast Execution), под руководством Алана Коттока (Alan Kotok) и Дэвида Орбита (David Orbits), с 1983;
- **HR-32** (Hudson RISC 32-bit), при фабрике DEC в Хадсоне (Массачусетс), под руководством Ричарда Витека (Richard Witek) и Дэниела Добберпуля (Daniel Dobberpuhl), с 1984;
- **CASCADE** Дэвида Катлера (David Cutler) в Сиэттле (Вашингтон), с 1984.

В 1985, с подачи Катлера насчёт создания «корпоративного RISC-плана», все 4 проекта были объединены в один, **PRISM** (Parallel Instruction Set Machine), и совместными усилиями к августу 1985 была подготовлена первая спецификация на новый RISC-процессор. Стоит упомянуть, что к тому времени DEC уже принимала активное участие в разработке процессора **MIPS R3000** и даже инициировала создание консорциума Advanced Computing Environment для продвижения этой архитектуры на рынке.

Поэтому нет ничего удивительного в том, что разрабатываемый процессор унаследовал много черт, присущих архитектуре MIPS, но в то же время отличия были очевидны. Все инструкции имели **фиксированную длину** в 32 бита, из них верхние 6 и нижние 5 были собственно кодом инструкции, а остальные 21 предназначались для непосредственно данных либо их адресации. Были определены 64 основных 32-битных целочисленных регистра (MIPS предусматривала 32), а также 16 дополнительных 64-битных векторных регистров, и 3 контрольных регистра для векторных операций: два 7-битных (vector length и vector count), один 64-битный (vector mask). Регистр состояния процессора отсутствовал, поэтому результат сравнения двух скалярных операндов помещался в целочисленный регистр, а результат сравнения двух векторных операндов — в vector mask. Встроенный блок вычислений



с плавающей запятой не был предусмотрен. Также присутствовал набор **специальных инструкций** (Epicode, или extended processor instruction code), реализованных программно посредством загружаемого микрокода, для облегчения решения специальных задач, присутствующих определённой среде или операционной системе, и не предусмотренных стандартным набором команд; впоследствии, эта функция была реализована в архитектуре Alpha под названием **PALcode** (Privileged Architecture Library code).

К 1988 проект **PRISM** был всё ещё в стадии разработки, когда высший менеджмент DEC закрыл его, посчитав дальнейшее финансирование нецелесообразным. Протестуя против этого, Катлер уволился и ушёл в Microsoft, где возглавил подразделение по разработке Windows NT.

В начале 1989 DEC представила свои первые рабочие станции с RISC-процессорами, DECstation 3100 на основе 32-битного **MIPS R2000** с частотой в 16МГц, и DECstation 2100 на основе того же процессора, но с частотой в 12МГц. Обе машины работали под юникс-подобной ОС Ultrix и стоили довольно умеренно (около 8 тыс. долл. США (1990) за DECstation 2100).

В 1989 морально стареющая архитектура VAX уже с трудом могла конкурировать с RISC-архитектурами 2-го поколения, такими как **MIPS** и **SPARC**, и было вполне очевидно, что следующее поколение RISC оставит VAX немного шансов на выживание. В середине 1989 перед инженерами DEC была поставлена задача создания конкурентноспособной RISC-архитектуры с перспективным потенциалом, но минимально несовместимой с VAX, чтобы облегчить перенос на неё VAX/VMS и всей сопутствующей программной базы. Также было определено, что новая архитектура должна изначально быть 64-битной, так как конкуренты уже были в процессе перехода на 64 бита. Была создана рабочая группа, в которой главными архитекторами были Ричард Витек и Ричард Сайтс (Richard Sites).

Первое официальное упоминание об архитектуре **Alpha** датируется 25 февраля 1992, на конференции в Токио. Одновременно в небольшом обзоре (для USENET-конференции comp.arch) были перечислены ключевые моменты новой архитектуры; там же было оговорено, что «Alpha» является кодовым именем нового процессора, а рыночное название будет представлено позже. Процессор имел чистую **64-битную RISC-архитектуру с инструкциями фиксированной длины** (по 32 бита), с **32 целочисленными** 64-битными регистрами, работал с 43-битными адресами виртуальной памяти (с расширением до 64 бит в перспективе), использовал, как и VAX, **little-endian** порядок следования байт (то есть, когда младший байт регистра занимает при записи младший адрес в памяти; традиционно поддерживался Intel, в противовес к big-endian, изначально внедрённому Motorola, и используемому в большинстве процессорных архитектур, где младший байт регистра занимает при записи старший адрес в памяти). Математический сопроцессор был изначально интегрирован в ядро, вместе с **32 вещественными** 64-битными регистрами, причём использовался **произвольный доступ** к ним, в отличие от примитивного стекового, используемого сопроцессорами Intel x87. Срок службы новой архитектуры был определен, как минимум, в 25 лет.

Поддерживаемый набор инструкций был максимально упрощён с целью максимального облегчения конвейеризации, и состоял из 5 групп:

- *инструкции для целочисленных вычислений;*
- *инструкции для вещественных (с плавающей запятой) вычислений;*
- *инструкции обработки условных и безусловных переходов, а также сравнения;*
- *инструкции загрузки и сохранения данных;*
- *инструкции PALcode.*

Интересно отметить, что инструкции целочисленного деления отсутствовали, так как являлись наиболее сложными и плохо поддающимися конвейеризации, а поэтому эмулировались программно.

Архитектура Alpha была «настоящей» RISC (в отличие от современных процессоров архитектуры x86, которые RISC только изнутри). Концептуально разница между RISC (Reduced Instruction Set Computing) и CISC (Complex Instruction Set Computing) состояла (и до сих пор состоит) в нескольких моментах:

| Характеристика   | CISC                                                | RISC                                                                 |
|------------------|-----------------------------------------------------|----------------------------------------------------------------------|
| Длина инструкции | Произвольная, в зависимости от характера инструкции | Фиксированная, вне зависимости от характера инструкции               |
| Набор инструкций | Широкий, адаптированный для удобства программиста   | Сбалансированный, адаптированный для удобства выполнения процессором |
| Доступ к памяти  | Разрешен для инструкций разного рода                | Разрешен только для инструкций загрузки/сохранения                   |

Процессор предполагалось запустить в производство на очень высокой для того времени частоте - **150МГц**, и довести до **200МГц** при тех же конструкторских нормах. Это оказалось возможным как благодаря удачной архитектуре, так и благодаря отказу разработчиков от использования систем автоматического проектирования и выполнению всех работ по схемотехнике исключительно вручную.

Проект вступил в производственную стадию и вскоре был реорганизован в регулярное подразделение DEC.

Трудами отдела маркетинга DEC новая архитектура получила название AXP (или Alpha AXP), хотя до сих пор неизвестно, аббревиатурой чего было это недоразумение. Возможно, что и ничего; в прошлом DEC имела проблемы с торговой маркой VAX, на которую претендовала компания по производству пылесосов, и тогда дело дошло до суда. Кроме всего прочего мотивировалось, что рекламный слоган этой компании ("*Nothing sucks like a Vax!*" - "*Ничто не сосёт так, как Vax!*") дискредитирует продукцию DEC. Так что вскоре в компьютерном мире появилась шутка, что AXP - это "*Almost Exactly PRISM*", то есть «почти точно PRISM».

Первый процессор из семейства Alpha получил название **21064** (21 указывал, что Alpha – это архитектура XXI века, 0 – процессорное поколение, 64 – разрядность в битах), и кодовое наименование **EV4** (EV является [предположительно] аббревиатурой “Extended VAX”, а цифра 4 – поколением техпроцесса, CMOS4; CMOS, в свою очередь, есть аббревиатура “Complementary Metal Oxide Semiconductor”). Стоит отметить, что прототип EV4 был готов ещё в 1991, по CMOS3 техпроцессу, а поэтому с уменьшенными кэшами и без блока вычислений с плавающей запятой; тем не менее, он сыграл свою роль в отладке архитектуры и программной среды. EV4 был представлен в ноябре 1992, и поступил в производство на передовом для того времени 3-слойном 0,75μ техпроцессе (впоследствии, производство было переведено на оптическую модификацию CMOS4 – CMOS4S, 0,675μ). Был рассчитан на напряжение питания в 3,3В, выпускался с тактовыми частотами от **150МГц** до **200МГц** (TDP от 21Вт до 27Вт). Состоял из 1,68 млн. транзисторов, и имел площадь ядра в 233 кв.мм. Поддерживал многопроцессорность, как один из ключевых моментов архитектуры. Форм-фактор: PGA-431 (Pin Grid Array).

Кэш L1 был интегрирован: 8Кб для инструкций (I-cache, instruction cache), с прямым отображением (direct-mapped), а также 8Кб для данных (D-cache, data cache), с прямым отображением и сквозной записью (write-through). Задержки при чтении из D-cache составляли 3 такта. Каждая линия I-cache состояла из 32 байт инструкций, 21-битного тега, 8-битного поля истории переходов, и нескольких вспомогательных полей; каждая линия D-cache состояла из 32 байт данных и 21-битного тега. Кэш L2 (B-cache, backup cache) был рекомендуемой опцией, набирался на микросхемах синхронной или асинхронной статической памяти, работал в режиме обратной записи (write-back) с прямым отображением и

упреждающим чтением (read-ahead), мог иметь размер до 16Мб (обычно от 512Кб до 2Мб); каждая линия состояла из 32 байт данных или инструкций с 1-битным long-word parity или 7-битным long-word ECC, максимум 17-битного тега с 1 битом чётности, и 3 битов состояния с 1 битом четности. Скорость чтения/записи В-cache программировалась, в тактах процессора. Разрядность системной шины данных составляла 64 или 128 бит (программируемо, с 1-битным long-word parity или 7-битным long-word ECC), и она мультиплексировалась с шиной данных В-cache, переключаясь при необходимости; разрядность системной шины адреса составляла 34 бита. Организационно В-cache был включающим (inclusive) D-cache, то есть поддерживал копию содержимого последнего. При выгрузке данных из В-cache в память использовался механизм victim write. Операции чтения/записи в В-cache являлись прерогативой исключительно процессора, для системной логики было разрешено только чтение файла тегов (В-tag, что жизненно необходимо особенно в многопроцессорных системах, для обеспечения согласованности (когерентности) кэшей).

Процессор имел один целочисленный конвейер (Е-box, 7 стадий), и один вещественный конвейер (F-box, 10 стадий). Декодер и планировщик инструкций (I-box) был в состоянии подавать 2 команды за такт в порядке очереди (in-order) на исполняющие устройства, то есть на Е-box, F-box и блок загрузки/сохранения (А-box). Контроллер кэшей и системной шины (С-box) работал в тесном взаимодействии с А-box и управлял как интегрированными I-cache и D-cache, так и внешним В-cache. Также присутствовал блок предсказания переходов (branch prediction unit), с таблицей переходов на 4096 записей (branch history table), по 2 бита каждая. I-TLB имел 8 записей для 8Кб-страниц, и 4 записи для 4Мб-страниц, все с полной ассоциативностью (fully-associative). D-TLB имел 32 записи, с полной ассоциативностью.

Несмотря на отличную производительность, EV4 был довольно дорог для значительной части потенциальных покупателей, и поэтому в сентябре 1993 был анонсирован его бюджетный собрат, **21066 (LCA4, или LCA4S)**. Этот процессор был основан на ядре 21064, но с дополнительно **интегрированными контроллерами памяти и шины PCI**, а также некоторыми второстепенными функциями. Однако, разрядность системной шины данных была урезана до 64 бит, что негативно сказалось на производительности. LCA4 производился по 0,675μ CMOS4S техпроцессу, и его площадь оказалась ещё меньше, чем у EV4 (209 кв.мм по сравнению с 234 кв.мм.), а тактовые частоты были занижены (от **100МГц** до **166МГц**), в значительной мере из-за потенциальной угрозы перегрева в плохо вентилируемых настольных системных блоках того времени, и чтобы не создавать дополнительной конкуренции EV4. Количество транзисторов в ядре составило 1,75 млн., а напряжение питания – 3,3В. Дизайн этого процессора был лицензирован Mitsubishi, и последняя также производила LCA4 (включая и **200МГц** версию).

Дальнейшее подробное изложение истории процессора Alpha опущено, в виду большого объёма текста. Полностью с материалом «Alpha: история в фактах и комментариях» можно ознакомиться по ссылке: <https://fcenter.ru/online/hardarticles/processors/12737#01>

## **Эпилог**

На момент написания статьи (апрель 2005) Alpha-системы всё ещё продавались, в основном через HP и Microway. Последняя даже предлагала относительно недорогие рабочие станции с 21164A и AlphaPC 164LX под Linux (за 2 тыс. долл. США в стандартной комплектации). Довольно значительное количество списанных (но все ещё работоспособных) рабочих станций и серверов, а также отдельных комплектующих, доступно через «онлайн-барахолки»; большинство этих систем предназначалось для работы с Windows NT, и на многие из них нельзя установить ни Digital UNIX, ни OpenVMS, а на некоторые даже \*BSD (системы, не поддерживающие SRM console), хотя сохраняется возможность установки Linux из-под ARC/AlphaBIOS. Если вы имеете намерение приобрести Alpha-

систему, выясните этот вопрос перед покупкой, чтоб не иметь впоследствии лишних проблем.

Согласно статистике, к июню 2001 только DEC и Compaq продали около 800 тыс. рабочих станций и серверов на Alpha. Точное количество систем, собранных и проданных другими компаниями, неизвестно, но эта цифра определённо превышает 500 тыс.

**Многие утверждают, что архитектура Alpha умерла своей смертью. Надеюсь, после прочтения этой статьи у вас не останется сомнений, что её похоронили, причём заживо. Потому что так было выгодно.**

История знает немало случаев, когда товар с худшими характеристиками вытеснял сопоставимый товар с лучшими техническими показателями. Возможно, первый товар стоил существенно дешевле второго. Также возможно, что второй товар продвигался на рынке существенно пассивнее первого. Или лицензионные отчисления были несопоставимы. Всё возможно. Не исключено, что маркетологи некоторых товаров, осознавая их ущербность, продвигают их на рынке наиболее агрессивно, понимая, что иначе их очередная зарплата может оказаться последней. Одно очевидно: для успеха на рынке технические показатели того или иного продукта имеют далеко не самое первостепенное значение.

## Наследие DEC

Яркая сорокалетняя история DEC явила миру множество интересных изобретений, великолепных инновационных архитектур, передовых технологий. К сожалению, в основном из-за многочисленных стратегических просчётов руководства компании, большей частью в конце 1980-х и начале 1990-х, DEC вошла в полосу неудач, которую компания так и не смогла преодолеть, даже после полной реорганизации структуры компании и смены руководства – «чёрная» полоса продолжилась, и в январе 1998 года DEC (бывшая когда-то номером два в мире после IBM) была приобретена компанией Compaq. Любопытные цифры – к маю 1998 года Compaq уже адаптировала в свои структуры основные подразделения DEC, включая 38 тыс. сотрудников (по сравнению с 32 тыс. у самой Compaq до поглощения), хотя значительная их часть всё же была сокращена в самом ближайшем будущем.

Что осталось от DEC сегодня? Где-то до сих пор работают даже раритетные PDP-11/LSI-11, например, в системах управления атомных электростанций в Канаде, в станках с ЧПУ на производстве ИЯФ СО РАН. Под VAX до недавнего времени поддерживался порт системы OpenBSD<sup>1</sup>, для Alpha разработка и поддержка OpenBSD/alpha продолжается (release is OpenBSD 6.7) до сих пор.

В своё время DEC, избавляясь от непрофильных активов, в июле 1994 за 400 млн. долл. США продала компании Quantum подразделение по разработке и производству дисковых и ленточных накопителей (Storage Business Unit). Quantum довольно долго был заметным игроком на рынке дисковых накопителей, но постепенно уступила более мощным и активным игрокам (Seagate и WD). Ленточное направление, также доставшееся Quantum от DEC, тоже было весьма конкурентноспособным – технология DLT довольно долгое время опережала основного конкурента – технологию LTO. Но история DLT закончилась в 2007 году, на кассетах ёмкостью 800GB<sup>2</sup>.

В августе 1994 за 100 млн. долл. США было продано Oracle подразделение баз данных (Database Software Unit), и тогда же за 140 млн. долл. США была продана 7,8% доля в итальянской Olivetti. В ноябре 1997 была подписана сделка о продаже Cabletron за 430 млн. долл. США подразделения сетевых решений (Network Product Business Unit).

---

<sup>1</sup> Поддержка порта OpenBSD/vax была прекращена после релиза 5.9. На сегодня текущий релиз OpenBSD 6.7 (19 мая 2020), «скорость» выхода версий – два релиза в год.

<sup>2</sup> У тогдашней LTO-4 кассеты ёмкость была ровно те же 800GB. Сегодняшние кассеты LTO-8 имеют ёмкость 12,8TB.

В программном обеспечении DEC в своё время заметно отметилась в разработке X Windows System<sup>1</sup>. Использовался DECwindows MOTIF для VMS, который базировался на спецификации MIT для X Window System.

В терминальных клиентах (Putty, Bitvise) в определении типа можно увидеть название vt100 – текстовый компьютерный терминал производства DEC, набор команд которого стал стандартом de facto для эмуляторов терминала.

### ***Жизнь после исчезновения***

Незадолго до конца DEC, а также вскоре после этого, многие ведущие инженеры, которым DEC была обязана своим могуществом, ушли к другим работодателям: Деррик Мейер (Derrick Meyer) ушёл в AMD создавать K7; туда же, но архитектором K8, ушел Джеймс Келлер (James Keller); Дэниел Лейбхольц (Daniel Leibholz) ушёл в Sun разрабатывать UltraSPARC V; также уволился Ричард Сайтс, один из ведущих разработчиков архитектуры Alpha на протяжении всего времени её существования. В этом плане Intel повезло меньше: доставшаяся ей от DEC архитектура StrongARM осталась почти без разработчиков, так как никто из ведущих инженеров, проектировавших StrongARM-110, а именно Дэниел Добберпуль, Ричард Витек, Грегори Хеппнер (Gregory Hoerpner) и Лайэм Мэдден (Liam Madden), не пожелал перейти на работу в Intel, а команда разработчиков StrongARM второго поколения, работавшая под руководством Витека в Остине, уволилась в полном составе, так что Intel пришлось разрабатывать ядро в буквальном смысле с нуля, усилиями своих инженеров, до этого разрабатывавших i960.

---

<sup>1</sup> **X Window System** — оконная система, обеспечивающая стандартные инструменты и протоколы для построения графического интерфейса пользователя.

## ГЛАВА 7. ЭВМ НА ОСНОВЕ ПРОЦЕССОРОВ INTEL

Изучая историю вычислительных машин, конечно же нельзя не упомянуть архитектуры, основанные на процессорах Intel и совместимых с ними устройствах<sup>1</sup>.

В 1971 году появился 4-х битный процессор 4004 от Intel<sup>2</sup>, впоследствии замененный 8-битным чипом i8008. Более удачным и массовым стал i8080, третье поколение, «внук» первого интеловского микропроцессора.

Кристалл i4004 появился после того, как небольших размеров японская компания с названием Nippon Calculating Machine, Ltd. (впоследствии Busicom Corp.) заказала всего 12 микросхем у Intel. Компании эти микросхемы были нужны для своих калькуляторов, а логический дизайн чипов был разработан сотрудником компании-заказчика). В то время для каждого устройства разрабатывался собственный набор микросхем, выполнявших узкоспециализированные функции.

При выполнении заказа Маршиан Эдвард Хофф предложил снизить число микросхем для нового устройства японской компании, введя использование центрального процессора. Именно он, по задумке инженера, должен был стать центром обработки данных и выполнения арифметических и логических функций. Процессор должен был заменить собой сразу несколько микросхем. Руководства обеих компаний одобрили эту идею. Осенью 1969 года Хофф при помощи Стэнли Мэйзора предложил новую архитектуру микросхем, число которых было сокращено всего до 4. Часть предложенных элементов — 4-х разрядный центральный процессор, ПЗУ и ОЗУ.

Сам процессор смог разработать Федерико Фаджин, физик из Италии, который стал главным проектировщиком семейства MCS-4 в Intel. Именно он, благодаря знанию технологии МОП, смог создать процессор, реализовав идею Хоффа. Кстати, первая в мире коммерческая микросхема, где использовалась технология кремниевых затворов, была разработана тоже им. Она носила название Fairchild 3708.

Фаджин, будучи сотрудником Intel, смог создать новый метод проектирования систем произвольной логики. Ему в его работе помогал Масатоси Сима, работавший в то время инженером в Busicom. Фаджин и Сима разработали впоследствии микропроцессор Zilog Z80, который, кстати, производится и сейчас<sup>3</sup>.

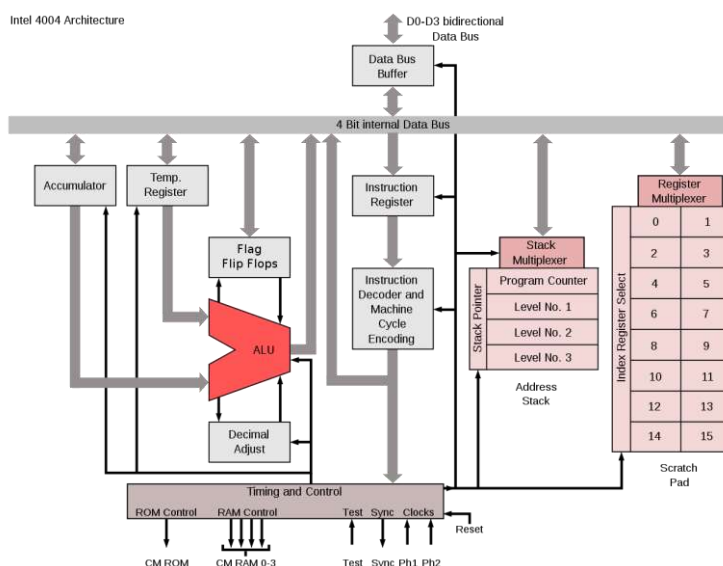


Рис.193 – Архитектура Intel 4004

<sup>1</sup> <https://habr.com/ru/post/399473/>

<sup>2</sup> Официально термин *микропроцессор* связывают как раз с первым интеловским i4004 (MCS-4). Американская компания Texas Instruments пыталась оспорить первенство Intel, утверждая, что именно она выпустила первый в мире микропроцессор, хотя признавала, что своё изделие микропроцессором она не называла.

<sup>3</sup> Дата на момент написания данного текста – ноябрь 2016.

Главное случилось 15 ноября 1971 года. Это официальная дата появления первого микропроцессора от Intel, чипа 4004, построенного по гарвардской архитектуре. Его стоимость на то время составила 200 долларов. Всего на одном кристалле были реализованы практически все функции процессора большой ЭВМ. Его анонсировали в ноябре 1971 года в журнале Electronic News.

Некоторые характеристики i4004:

- Дата появления: 15 ноября 1971 года
- Количество транзисторов: 2300
- Площадь кристалла: 12 мм<sup>2</sup>
- Техпроцесс: 10 мкм (P-channel silicon pie MOS technology)
- Тактовая частота: 740 кГц (конкретно от 500 до 740 кГц, clock period 1,35...2 мкс)
- Разрядность регистров: 4 бит
- Количество регистров: 16 (16 четырёхбитных могут быть использованы как 8 восьмибитных)
- Количество портов: 16 четырёхбитных входных и 16 четырёхбитных выходных
- Разрядность шины данных: 4 бита
- Разрядность шины адреса: 12 бит
- Гарвардская архитектура
- Стек: внутренний 3-уровневый
- Память команд (ПЗУ/ROM): 4 килобайта (32768 бит)
- Объём адресуемой памяти (ОЗУ/RAM): 640 байт (5120 бит)
- Количество инструкций: 46 (из которых 41 — 8-разрядные и 5 — 16-разрядные)
- Цикл инструкций: 10,8 микросекунд
- Напряжение питания: -15 В (pMOS)
- Рабочая температура: от 0 до +70С
- Условия хранения и эксплуатации: от -40 до +85С
- Разъём: DIP16 (микросхема непосредственно впаивалась в печатную плату либо устанавливалась в специальный слот)
- Корпус: 16-контактный DIP (1 вид пластиковый или 3 вида керамического, например, C4004 (белая керамика с серыми полосками), S4004 (белая керамика), D4004 (чёрно-серая керамика), P4004 (чёрный пластик))
- Тип поставки: отдельно и в комплектах MCS-4 (ROM, RAM, I/O, CPU)

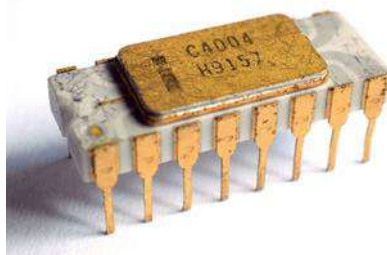


Рис.194 – Внешний вид микропроцессора i4004

### **Архитектура микроЭВМ на базе i8080**

В 1974 году Intel выпускает свой второй 8-ми разрядный микропроцессор i8080, который был по сути усовершенствованной версией их первого восьмиразрядного i8008. Кстати, первым в мире 8-битным процессором был вовсе не i8008, а TMS 1795 от Texas Instruments.

Новый процессор 8080 выпускался по новейшей тогда 6-микронной NMOS технологии, что позволило разместить на кристалле 4758 транзисторов<sup>1</sup>. Процессор, хотя и был построен на архитектуре Intel 8008, но имел множество отличий от своего предшественника, благодаря которым и получил большую популярность. В новом процессоре по сравнению с предшественником, была достаточно развитая система команд: 16 команд передачи данных, 31 команда для их обработки, 28 команд для перехода (с прямой адресацией), 5 команд управления. В микропроцессоре i8080 не было команд умножения и деления, и обычно их реализовывали программно, хотя Intel предлагала отдельный внешний арифметический сопроцессор. Благодаря 16-разрядной адресной шине процессор позволял производить адресацию 64 Кбайт памяти, которая не разделялась на память команд и данных (неймановская архитектура). Хотя процессор был 8-разрядным и содержал семь 8-битных регистров (A, B, C, D, E, H, L), он имел ограниченные возможности обработки 16-разрядных чисел, для чего регистры объединялись в пары BC, DE, HL. В новом процессоре использовался стек во внешней памяти (в i8008 он был внутренним).

### Регистровый файл i8080

|                      |       |
|----------------------|-------|
| A                    | флаги |
| B                    | C     |
| D                    | E     |
| H                    | L     |
| PC (счётчик команд)  |       |
| SP (указатель стека) |       |

Пара регистров A + флаги (первая строка таблицы) образуют 16 разрядный регистр, именуемый PSW (статусный регистр), причём A занимает старший байт слова, а флаги — младший. Аналогично регистры B и C образуют 16-разрядный регистр BC, регистры D и E — регистр DE, регистры H и L — регистр HL.

### Регистр флагов

|    |   |   |    |   |   |   |    |
|----|---|---|----|---|---|---|----|
| D7 |   |   |    |   |   |   | D0 |
| S  | Z | x | AC | x | P | x | C  |

S — знак

Z — нуль

x — значение не определено

AC — вспомогательный перенос

P — чётность

C — перенос

### Назначение регистров

|                   |                                                                                                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A                 | — аккумулятор. Все арифметические и логические операции производятся только между A и другими регистрами или между A и байтом непосредственных данных.              |
| B, C, D, E, H и L | — 8-разрядные регистры общего назначения.                                                                                                                           |
| HL                | — регистровая пара, состоящая из двух 8-разрядных регистров (H — старший регистр, L — младший), используется для косвенно-регистрационной адресации 64Кбайт памяти. |

<sup>1</sup> Сравните с количеством транзисторов в современных процессорах — разница больше, чем шесть десятичных порядков.



|    |                                                                                                                                                                                                                              |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DE | – часто используемая регистровая пара, поскольку имеется команда обмена содержимым между парами HL и DE.                                                                                                                     |
| PC | – счётчик команд, содержит адрес очередной исполняемой команды.                                                                                                                                                              |
| SP | – указатель стека автоматически инкрементируется на 2 при записи пары регистров в стек (отдельный 8-разрядный регистр в стек записать нельзя, только парами) и декрементируется при извлечении из регистровой пары из стека. |
| F  | – регистр флагов. Непосредственно недоступен программисту, но его в составе PSW можно сохранить в стеке, а потом извлечь в другую регистровую пару, если нужно специально установить или проверить нужные флаги.             |

Так как прямоадресуемое адресное пространство составляет 64Кбайт, то полный адрес, соответственно, занимает 2 байта. Команды процессора i8080 бывают одно-, двух- и трёхбайтными. В первом байте всегда содержится код операции.

Кроме прямоадресуемого ОЗУ объёмом 64Кбайт, i8080 имел отдельное адресное пространство ввода/вывода (I/O), объёмом до  $2^8=256$  байт. При обращении к пространству I/O адрес (байт) дублировался на линиях A0-A7 и A8-A15<sup>1</sup>.

Существует некоторая путаница в обозначениях именно этого процессора. Первоначальный вариант i8080 имел 48-выводный планарный корпус с шагом выводов 1/20 дюйма, максимальную тактовую частоту 2 МГц и содержал как минимум одну довольно серьёзную ошибку, которая теоретически могла привести процессор в состояние, из которого он выводился только сигналом сброс. Исправленный вариант 8080A, выпущенный через полгода, имел корпус DIP-40 с шагом выводов 1/10 дюйма, максимальную тактовую частоту до 2,5 МГц, и вышеупомянутая ошибка была в нём исправлена. Большинство авторов, используя обозначение 8080, имеют в виду в действительности 8080A.

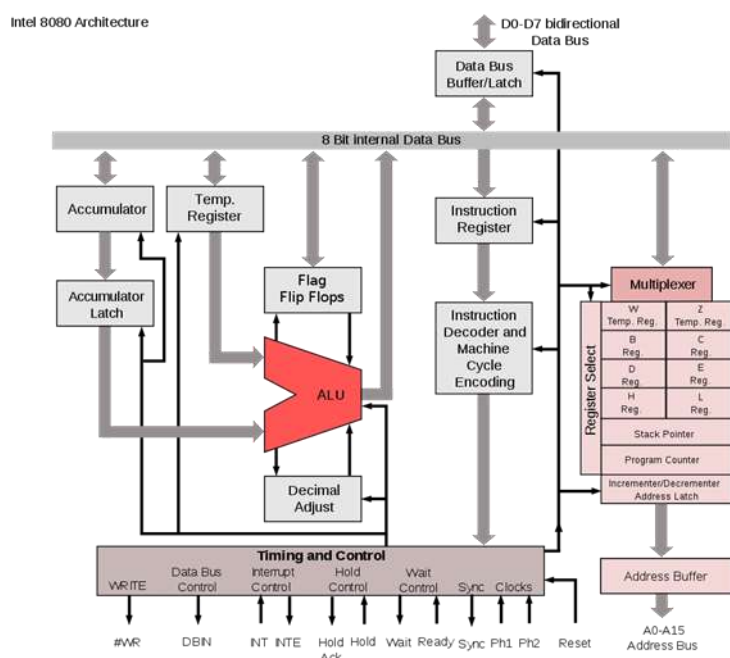


Рис.195 - Архитектура микропроцессора Intel 8080

На базе микропроцессора Intel 8080 фирмой MITS был выпущен «первый в мире микрокомпьютерный комплект, который может соперничать с промышленными образцами» (персональный компьютер) Altair-8800, который пользовался невероятно большой по тем

<sup>1</sup> Подразумевалось, что адресные входы устройств ввода-вывода необходимо распределять поровну между двумя половинами адресной шины, для сокращения токовой нагрузки на довольно маломощные драйверы-передатчики адресной шины процессора.

временам популярностью (MITS даже не успевала вовремя обрабатывать заказы). Для удешевления Altair-8800 выпускался в том числе в варианте некоего «полуфабриката», набора узлов и деталей, которые пользователь должен был самостоятельно спаять и собрать, чтобы получить работающий компьютер.

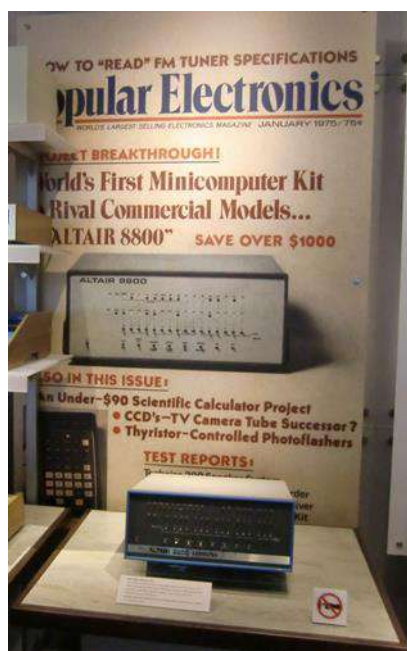


Рис.196 – Altair-8800 в музее истории вычислительной техники (Маунтин-Вью, Калифорния).

Помимо Altair-8800, микропроцессор Intel 8080 также применялся, например, в устройствах управления уличным освещением и светофорами, а также в большом количестве другого оборудования. Для Altair-8800 (и совместимых компьютеров с шиной S-100) в своё время был разработан Altair BASIC — интерпретатор языка программирования BASIC<sup>1</sup>. Altair BASIC представлял собой новую веху в истории программ — первый язык программирования для первого персонального компьютера и первый программный продукт компании Microsoft (старое название Micro-Soft). Говорят, что это был единственный продукт, в написании кода которого лично принимал участие Билл Гейтс.

## Система команд i8080/85

Система команд микропроцессора i8080 не отличалась ни особой стройностью, ни красивой логичной организацией, ни регулярностью в кодах. Если говорить о таком свойстве системы команд, как ортогональность, то очевидно, что это был последний критерий, которым руководствовались разработчики архитектуры при разработке системы команд процессора i8080. Такой подход переключался впоследствии и в архитектуру x86, где команды могли занимать от одного до пятнадцати байт, имели крайне нерегулярную структуру, сложные команды в итоге декодировались в большое количество микроопераций, плохо рапараллеливались и конвейеризовались. Забегая вперёд, можно отметить ещё один аспект, где интеловская архитектура зарекомендовала себя не самым лучшим образом — в области аппаратной поддержки виртуализации.

На момент, когда технология виртуализации стала массово распространяться на микропроцессорных платформах, интеловская x86 оказалась едва ли не худшей из всех

---

<sup>1</sup> **Бейсик (BASIC)**, сокращение от англ. *Beginner's All-purpose Symbolic Instruction Code* — универсальный код символических инструкций для начинающих) — семейство высокоуровневых языков программирования.

платформ по поддержке виртуализации, у неё насчитывалось больше всех «плохих» инструкций<sup>1</sup> с точки зрения формальных критериев.

**Коды регистров и пар регистров, используемые в командах МП**

| Регистры |         |     |                 | Пары регистров |               |               |         |
|----------|---------|-----|-----------------|----------------|---------------|---------------|---------|
| Код      | Имя (r) | Код | Имя (r)         | Код (RP)       | Имя пары (rp) | Регистры пары |         |
|          |         |     |                 |                |               | старший       | младший |
| 000      | B       | 100 | H               | 00             | B             | B             | C       |
| 001      | C       | 101 | L               | 01             | D             | D             | E       |
| 010      | D       | 110 | M (память)      | 10             | H             | H             | L       |
| 011      | E       | 111 | A (аккумулятор) | 11             | PSW           | A             | PSW     |

**Назначение разрядов регистра признаков - PWS (processor status word)**

| S                                                      | Z                                       | 0 | AC                                                                 | 0 | P                                                                     | 1 | C                                        |
|--------------------------------------------------------|-----------------------------------------|---|--------------------------------------------------------------------|---|-----------------------------------------------------------------------|---|------------------------------------------|
| Знак (sign). S=1 если результат операции отрицательный | Ноль (zero) Z=0, если результат нулевой |   | Перенос из 3-го разряда (дополнительный перенос – auxiliary carry) |   | Четность (parity): P=1, если результат содержит четное число "единиц" |   | Перенос (carry) при получении результата |

**Форматы команд и способы адресации.** В МП i8080 используются 11 форматов команд (Рис.197), коды операций (КОП) которых имеют различную длину (2, 5, 6 или 8 бит) и часто состоят из двух частей. В зависимости от способа адресации команды могут быть одно-, двух- или трёхбайтовыми.

В командах используются четыре способа адресации:

- регистровая (MOV r1, r2; ADD r; PCHL; ...);
- косвенно-регистровая (MOV M, r; ADD M; PUSH; POP; ...);
- непосредственная (MVI r, data 8; ADI data 8; ...);
- прямая (LDA addr; SHLD addr; IN port; ...).

**Список команд.** Команды МП i8080 приведены на Рис.198, Рис.199 и Рис.200. Трёхбайтовые поля адресации источника и приемника информации кодируются в машинных командах символами SSS и DDD соответственно. В мнемонических изображениях двух-адресных команд приемник указывается на первом месте, а источник — на втором<sup>2</sup>.

В описаниях команд для обозначения содержимого регистра или ячейки памяти используется запись вида: (r1), (r), (H), (M) и т. п.

**Коды условий, используемые в командах условных переходов**

| Код (CCC) | Мнемоника (cc) | Условие              | Код (CCC) | Мнемоника (cc) | Условие          |
|-----------|----------------|----------------------|-----------|----------------|------------------|
| 000       | NZ             | Не ноль (Z=0)        | 001       | Z              | Ноль (Z = 1)     |
| 010       | NC             | Нет переноса (C = 0) | 011       | C              | Перенос (C = 1)  |
| 100       | PO             | Нечетность (P = 0)   | 101       | PE             | Четность (P = 1) |
| 110       | P              | Плюс (S = 0)         | 111       | M              | Минус (S = 1)    |

<sup>1</sup> Попек и Гольдберг определили ограничительные условия, при которых компьютерная архитектура может быть виртуализирована, сформулировав их в виде трех критериев виртуализации. «Плохие» инструкции с точки зрения сформулированных ими критериев затрудняли реализацию виртуальной машины на конкретной платформе.

<sup>2</sup> В DEC-ой нотации позиции источника и приёмника расположены наоборот, по сравнению с интеловской. Например, команда в LSI-11 MOV R1, R5 пересылает содержимое регистра R1 в регистр R5 (слева-направо), а в Intel-ой интерпретации это означает пересылку содержимого R5 в R1 (справа-налево).

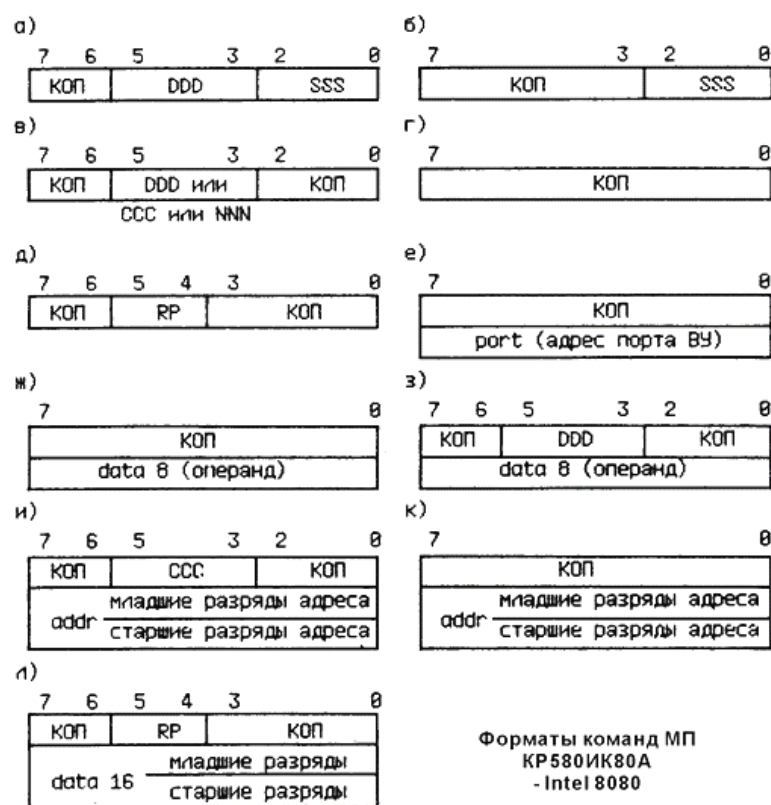


Рис.197 - Форматы команд микропроцессора i8080

| Мнемоника       | Двоичный код | Формат | Число тактов | Название и описание            |
|-----------------|--------------|--------|--------------|--------------------------------|
| MOV r1, r2      | 01DDDSSS     | а      | 5            | (r2) ⇒ (r1) Пересылка          |
| MOV r, M        | 01DDD110     | а      | 7            | (M) ⇒ (r)                      |
| MOV M, r        | 01110SSS     | а      | 7            | (r) ⇒ (M)                      |
| MVI r, data 8   | 00DDD110     | з      | 7            | data ⇒ (r)                     |
| MVI M, data 8   | 00110110     | з      | 10           | data ⇒ (M)                     |
| LXI rp, data 16 | 00RP0001     | л      | 10           | data ⇒ (rp) Загрузка           |
| LDA addr        | 00111010     | к      | 13           | (addr) ⇒ (A)                   |
| LHLD addr       | 00101010     | к      | 16           | (addr) ⇒ (L), (addr + 1) ⇒ (H) |
| LDAX rp         | 00RP1010     | д      | 7            | (rp) ⇒ (A)                     |
| XCHG            | 11101011     | г      | 4            | (H) ⇔ (D), (L) ⇔ (E) Обмен     |
| STA addr        | 00110010     | к      | 13           | (A) ⇒ (addr) Запись            |
| SHLD addr       | 00100010     | к      | 16           | (L) ⇒ (addr), (H) ⇒ (addr + 1) |
| STAX rp         | 00RP0010     | д      | 7            | (A) ⇒ (rp)                     |

Рис.198 – Список команд пересылки микропроцессора i8080

| Мнемоника  | Двоичный код | Формат | Число тактов | Название и описание                                                                                                                              | Устанавливаемые признаки |
|------------|--------------|--------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| ADD r      | 1000SSS      | б      | 4            | Сложение<br>$(A) + (r) \Rightarrow (A)$                                                                                                          | S, Z, P,<br>C, AC        |
| ADD M      | 10000110     | б      | 7            | $(A) + (M) \Rightarrow (A)$                                                                                                                      |                          |
| ADI data 8 | 11000110     | ж      | 7            | $(A) + \text{data 8} \Rightarrow (A)$                                                                                                            |                          |
| ADC r      | 10001SSS     | б      | 4            | Сложение с переносом<br>$(A) + (r) + (C) \Rightarrow (A)$                                                                                        | S, Z, P,<br>C, AC        |
| ADC M      | 10001110     | б      | 7            | $(A) + (M) + (C) \Rightarrow (A)$                                                                                                                |                          |
| ACI data 8 | 11001110     | ж      | 7            | $(A) + \text{data 8} + (C) \Rightarrow (A)$                                                                                                      |                          |
| SUB r      | 10010SSS     | б      | 4            | Вычитание<br>$(A) - (r) \Rightarrow (A)$                                                                                                         | S, Z, P,<br>C, AC        |
| SUB M      | 10010110     | б      | 7            | $(A) - (M) \Rightarrow (A)$                                                                                                                      |                          |
| SUI data 8 | 11010110     | ж      | 7            | $(A) - \text{data 8} \Rightarrow (A)$                                                                                                            |                          |
| SBB r      | 10011SSS     | б      | 4            | Вычитание с заемом<br>$(A) - (r) - (C) \Rightarrow (A)$                                                                                          | S, Z, P,<br>C, AC        |
| SBB M      | 10011110     | б      | 7            | $(A) - (M) - (C) \Rightarrow (A)$                                                                                                                |                          |
| SBI data 8 | 11011110     | ж      | 7            | $(A) - \text{data 8} - (C) \Rightarrow (A)$                                                                                                      |                          |
| ANA r      | 10100SSS     | б      | 4            | Логическое умножение<br>$(A) \text{ AND } (r) \Rightarrow (A)$                                                                                   | S, Z, P,<br>C, AC        |
| ANA M      | 10100110     | б      | 7            | $(A) \text{ AND } (M) \Rightarrow (A)$                                                                                                           |                          |
| ANI data 8 | 11100110     | ж      | 7            | $(A) \text{ AND data 8} \Rightarrow (A)$                                                                                                         |                          |
| XRA r      | 10101SSS     | б      | 4            | Исключающее ИЛИ<br>$(A) \text{ XOR } (r) \Rightarrow (A)$                                                                                        | S, Z, P,<br>C, AC        |
| XRA M      | 10101110     | б      | 7            | $(A) \text{ XOR } (M) \Rightarrow (A)$                                                                                                           |                          |
| XRI data 8 | 11101110     | ж      | 7            | $(A) \text{ XOR data 8} \Rightarrow (A)$                                                                                                         |                          |
| ORA r      | 10110SSS     | б      | 4            | ИЛИ<br>$(A) \text{ OR } (r) \Rightarrow (A)$                                                                                                     | S, Z, P,<br>C=0,<br>AC=0 |
| ORA M      | 10110110     | б      | 7            | $(A) \text{ OR } (M) \Rightarrow (A)$                                                                                                            |                          |
| ORI data 8 | 11110110     | ж      | 7            | $(A) \text{ OR data 8} \Rightarrow (A)$                                                                                                          |                          |
| CMP r      | 10111SSS     | б      | 4            | Сравнение<br>$(A) - (r)$                                                                                                                         | S, Z, P,<br>C, AC        |
| CMP M      | 10111110     | б      | 7            | $(A) - (M)$                                                                                                                                      |                          |
| CPI data 8 | 11111110     | б      | 7            | $(A) - \text{data 8}$                                                                                                                            |                          |
| INR r      | 00DDD100     | в      | 5            | Инкремент<br>$(r) + 1 \Rightarrow (r)$                                                                                                           | S, Z, P,<br>AC           |
| INR M      | 00110100     | в      | 10           | $(M) + 1 \Rightarrow (M)$                                                                                                                        |                          |
| INX rp     | 00RP0011     | д      | 5            | Декремент<br>$(rp) + 1 \Rightarrow (rp)$                                                                                                         | —                        |
| DCR r      | 00DDD101     | в      | 5            | $(r) - 1 \Rightarrow (r)$                                                                                                                        | S, Z, P,<br>AC           |
| DCR M      | 00110101     | в      | 10           | $(M) - 1 \Rightarrow (M)$                                                                                                                        |                          |
| DCX rp     | 00RP1011     | д      | 5            | $(rp) - 1 \Rightarrow (rp)$                                                                                                                      | —                        |
| DAD rp     | 00RP1001     | д      | 10           | Сложение содержимых регистровых пар<br>$(H, L) + (rp) \Rightarrow (H, L)$                                                                        | C                        |
| RLC        | 00000111     | г      | 4            | Циклический сдвиг влево<br>Все биты A сдвигаются на один разряд влево. Старший разряд A переходит в его нулевой разряд и регистр признака C      | C                        |
| RRC        | 00001111     | г      | 4            | Циклический сдвиг вправо<br>Выполняется аналогично RLC. Старший разряд A и C приобретают значение младшего разряда A                             | C                        |
| RAL        | 00010111     | г      | 4            | Арифметический сдвиг влево через перенос<br>Все биты A сдвигаются на один разряд влево; старший разряд A переходит в C, а C — в младший разряд A | C                        |
| RAR        | 00011111     | г      | 4            | Арифметический сдвиг вправо через перенос<br>Выполняется аналогично RAL, но сдвиг вправо                                                         | C                        |
| DAA        | 00100111     | г      | 4            | Десятичная коррекция аккумулятора<br>Преобразование содержимого A в двоично-десятичный код                                                       | S, Z, P,<br>C, AC        |
| CMA        | 00101111     | г      | 4            | Инвертирование A<br>Получение обратного кода (инверсии) содержимого A                                                                            | —                        |
| STC        | 00110111     | г      | 4            | Установка переноса в 1<br>$1 \Rightarrow C$                                                                                                      | C=1                      |
| CMC        | 00111111     | г      | 4            | Инвертирование переноса<br>Инвертирование содержимого регистра C                                                                                 | C                        |

Рис.199 – Список арифметических и логических команд МП i8080



| Мнемоника | Двоичный код | Формат | Число тактов | Название и описание                                                                                                                                                                                                                          |
|-----------|--------------|--------|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| JMP addr  | 11000011     | к      | 10           | Безусловный переход к команде, адрес которой определяется вторым и третьим байтами данной команды                                                                                                                                            |
| PCHL      | 11101001     | г      | 5            | Косвенный переход по адресу, указанному в регистрах H, L                                                                                                                                                                                     |
| Jcc addr  | 11CCCC010    | и      | 10           | Условные переходы: если условие (cc из табл. 2) истинно, то переход к команде, адрес которой определяется вторым и третьим байтами данной команды; иначе выполняется команда, расположенная вслед за данной                                  |
| CALL addr | 11001101     | к      | 17           | Обращение к подпрограмме: содержимое счетчика команд заносится в стек по адресу, на который указывает SP; содержимое SP уменьшается на 2 и выполняется переход к команде, адрес которой определяется вторым и третьим байтами данной команды |
| Ccc addr  | 11CCCC100    | и      | 11/17        | Условное обращение к подпрограмме: если условие (cc из табл. 2) истинно, то выполняется команда CALL; иначе выполняется команда, расположенная вслед за данной                                                                               |
| RET       | 11001001     | г      | 10           | Возврат из подпрограммы: переход к команде, адрес которой записан в верхней паре ячеек стека, и увеличение содержимого SP на 2                                                                                                               |
| Rcc       | 11CCCC000    | в      | 5/11         | Условный возврат из подпрограммы: если условие (cc из табл. 2) истинно, то выполняется команда RET; иначе выполняется команда, расположенная вслед за данной                                                                                 |
| RST n     | 11NNN111     | в      | 11           | Повторный запуск МП с адреса 8·NNN (0, 8, 16, ..., 56)                                                                                                                                                                                       |
| RUSH gr   | 11RP0101     | д      | 11           | Запись в стек содержимого пары регистров (gr)                                                                                                                                                                                                |
| POP gr    | 11RP0001     | д      | 10           | Считывание данных из стека в пару регистров (gr); при выдаче PSW происходит установка S, Z, P, C и AC                                                                                                                                        |
| XTHL      | 11100011     | г      | 18           | Обмен содержимым верхней пары ячеек стека и пары регистров H, L                                                                                                                                                                              |
| SPHL      | 11111001     | г      | 5            | Пересылка содержимого пары регистров H, L в указатель стека SP                                                                                                                                                                               |
| IN port   | 11011011     | е      | 10           | Ввод в аккумулятор данных из адресуемого порта                                                                                                                                                                                               |
| OUT port  | 11010011     | е      | 10           | Вывод данных из аккумулятора в адресуемый порт                                                                                                                                                                                               |
| EI        | 11111011     | г      | 4            | Разрешение прерывания                                                                                                                                                                                                                        |
| DI        | 11110011     | г      | 4            | Запрещение прерывания                                                                                                                                                                                                                        |
| HLT       | 01101110     | г      | 7            | Останов                                                                                                                                                                                                                                      |
| NOP       | 00000000     | г      | 4            | Нет операции                                                                                                                                                                                                                                 |

Рис.200 – Список команд управления, ввода-вывода и работы со стеком

Признаки результата (S, Z, P, C и AC) устанавливаются лишь при выполнении большинства арифметических и логических команд, а также команды POP PSW. В таблице на Рис.199 для каждой из команд дан перечень устанавливаемых признаков (в командах INX, DCX и CMA признаки не устанавливаются).

В командах условного перехода, условного обращения к подпрограмме и условного возврата из подпрограммы используются коды условий (CCC) из таблицы с кодами условий. Мнемонические обозначения этих команд состоят из символов J, C или R и соответствующих символов (cc) из той же таблицы (например, JNC, CP, RM).

Число тактов, необходимых для исполнения команд условного обращения к подпрограмме и условного возврата из подпрограммы, зависит от того, выполнено (знаменатель дроби) или не выполнено (числитель дроби) условие, указанное в команде.

## Ядро системы на базе процессора i8080, шина

Нужно отметить, что разработчиками i8080A были выбраны некоторые довольно странные решения для дизайна микропроцессора. Например, количества выводов в корпусе DIP-40, в который упаковывался i8080A (40), было недостаточно, чтобы каждый сигнал вывести на свой индивидуальный пин. Довольно неудобно было реализовано питание процессора – i8080A требовал три отдельных источника напряжения: +5В, -5В и +12В. Т.е., в итоге четыре вывода (три напряжения питания плюс «земля») – 10% от общего количества – занято линиями питания. Тем не менее 16 линий шины адреса и 8 линий шины данных в

i8080 были выведены на индивидуальные пины, не используя технику мультиплексирования (мультиплексирование сэкономило бы восемь проводников – 16 вместо  $16 + 8 = 24$ ). Да, немультимплексированные шины адреса/данных заметно упрощали схемотехнику подключения ОЗУ и периферийных устройств к шине, но при этом, из-за нехватки выводов в корпусе, приходилось дооснащать процессор дополнительным внешним дешифратором (демультиплексором) сигналов, чтобы реализовать полноценную шину управления.

Ещё одной, не очень «симпатичной» схемотехнической особенностью i8080A, была схема тактирования – требовалось заводить в процессор тактовые сигналы по двум отдельным линиям, с импульсами определённой скважности и определённым образом сфазированными друг относительно друга, при этом амплитуда тактирующих импульсов должна была составлять +12В.

Для преодоления этих схемотехнических сложностей Intel выпустила две дополнительных специализированных микросхемы: контроллер 8228/8238 (технология ТТЛШ), который по сути содержал два независимых компонента – системный контроллер (дешифратор управляющих сигналов) и драйвер шины данных (двунаправленный восьмиразрядный приёмопередатчик, который увеличивает нагрузочную способность шины данных до стандартных для TTL уровней значений<sup>1</sup>) и генератор тактовых импульсов 8224. В следующей «реинкарнации» своего восьмиразрядного микропроцессора – i8085 – Intel фактически объединил три этих компонента – процессор 8080A, контроллер 8228, генератор тактовых импульсов 8224 – на одном кристалле, попутно избавившись от необходимости иметь три разных номинала напряжения питания (оставив только стандартные для TTL +5В) и добавив немаскируемый вход запроса прерывания NMI. Вычислительное ядро на основе 8080A + 8228 + 8224 представлено на Рис.201 ниже:

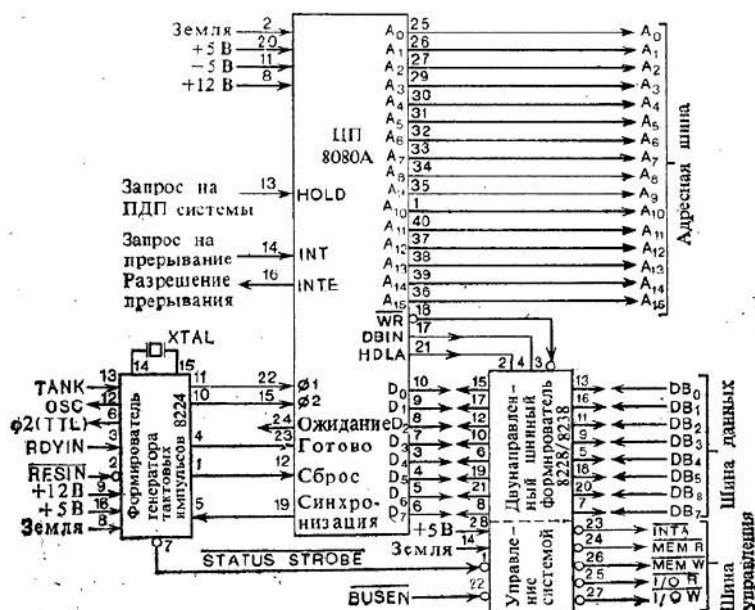


Рис.201 – Организация вычислительного ядра на базе i8080A

Каждая команда в микропроцессоре выполняется на протяжении командного цикла. Командный цикл состоит из цикла выборки команды и цикла выполнения команды (Рис.202).

<sup>1</sup> Исходно i8080 был выполнен по pMOS технологии, которая была совместима с TTL по уровням напряжения, но была гораздо «слабее» по нагрузочной способности и не могла обеспечить стандартные десять TTL нагрузок по втекающему току (16mA).



Рис.202 – Пример командного цикла i8080

Продолжительность цикла выборки команды зависит от формата команды (количества байт в машинном коде команды). Команды занимают от одного до трёх байт в программной памяти. Многобайтные команды хранятся в соседних ячейках памяти. Для выборки однобайтной команды (например, сложения содержимого аккумулятора А и регистра В – ADD B) необходимо одно обращение к памяти, для выборки трехбайтной команды (например, вызова подпрограммы по адресу ADDR – CALL ADDR) – три обращения. Продолжительность цикла исполнения команды зависит от способа адресации операндов. Так, при выполнении команд с регистровой адресацией не нужно дополнительное обращение к памяти для чтения операнда, в командах с косвенной адресацией такое обращение необходимо. В связи с этим продолжительность командного цикла в i8080 различна для различных команд и определяется количеством обращений к памяти или к внешнему устройству. Интервал, на протяжении которого осуществляется одно обращение процессора к памяти или к внешнему устройству, определяется как **машинный цикл М**. В итоге, командный цикл процессора состоит из некоторого количества машинных циклов (в зависимости от типа команды). В приведенном на Рис.202 примере цикл выборки состоит из двух машинных циклов (M1 и M2), а цикл исполнения – из одного машинного цикла (M3). В команде может быть от одного (для однобайтных команд с регистровой адресацией) до пяти (для трехбайтных сложных команд) машинных циклов.

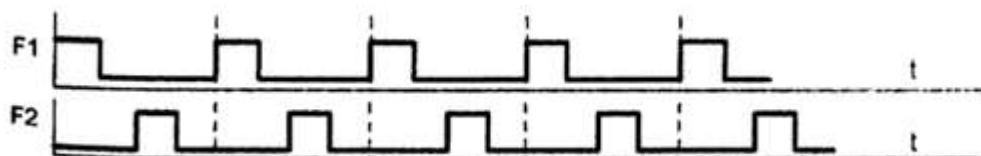


Рис.203 – Последовательности тактовых импульсов

Машинный цикл, в свою очередь, разбивается на некоторое количество машинных тактов Т, на протяжении каждого из которых выполняется элементарное действие (микрооперация) в процессоре. Количество тактов в цикле определяется кодом команды и лежит в пределах от 3 до 5. Продолжительность такта задается периодом импульсов синхронизации и определяется как интервал времени между фронтами двух соседних импульсов последовательности F1 (Рис.203). Таким образом, командный цикл МП i8080 состоит из некоторого количества машинных циклов, а каждый машинный цикл – из определенного количества тактов, на протяжении которых выполняются те или иные элементарные действия в процессоре. В зависимости от действий, выполняемых микропроцессором, различают следующие типы машинных циклов:

1. ВЫБОРКА (чтение первого байта команды);
2. ЧТЕНИЕ ПАМЯТИ (чтение второго и третьего байтов команды, чтение операнда);
3. ЗАПИСЬ В ПАМЯТЬ;
4. ЧТЕНИЕ СТЕКА;
5. ЗАПИСЬ В СТЕК;
6. ВВОД данных из внешнего устройства;



7. ВЫВОД данных на внешнее устройство;
8. ПЕРЕРЫВАНИЕ;
9. ОСТАНОВ;
10. ПЕРЕРЫВАНИЕ ПРИ ОСТАНОВЕ.

Каждый машинный цикл процессора идентифицируется (начинается) байтом, называемым байтом состояния. Байт состояния выдается на шину данных в начале каждого машинного цикла и сопровождается одновременной выдачей сигнала SYNC на одноимённый контакт микропроцессора. Байт состояния содержит информацию о последующих действиях процессора. Этот байт запоминается по сигналу SYNC в контроллере 8228/38, который в итоге формирует управляющие сигналы (Рис.201), для которых не хватило контактов в корпусе микросхемы (и ещё раз можно вспомнить, что разработчики процессора могли сэкономить как минимум 8 выводов, используя мультиплексирование шины адреса и шины данных).

Временная диаграмма машинного цикла ВЫБОРКА (чтение памяти) для процессора i8080 показана на Рис.204, а цикл ЗАПИСЬ В ПАМЯТЬ – на Рис.205. Такты отсчитываются по передним фронтам последовательности F1, а микрооперации в каждом такте определяются передним фронтом последовательности F2. При тактовой частоте 2 МГц длительность такта очевидно равна 0,5 мкс.

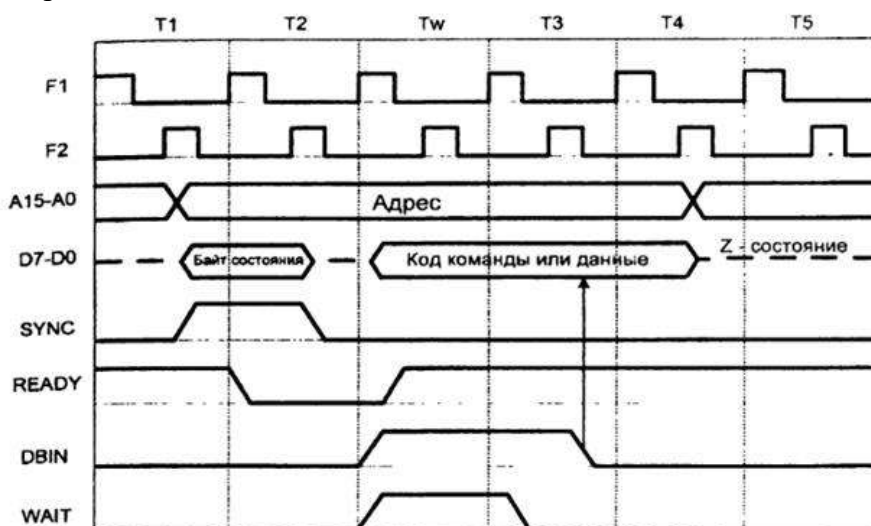


Рис.204 – Цикл ВЫБОРКА (чтение памяти)

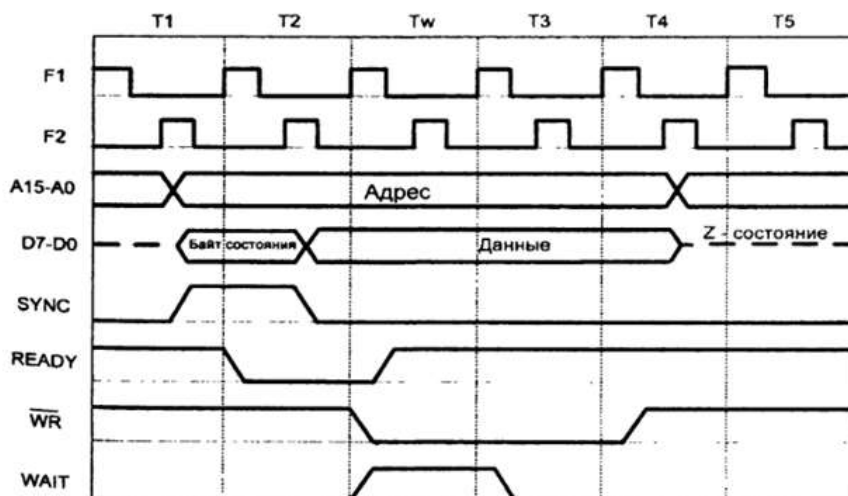


Рис.205 – Цикл ЗАПИСЬ в память

Сигналы на линиях шин A15-A0 (D7-D0) изображены на Рис.204 на одной временной диаграмме в виде линий – L- и H- уровень одновременно. Таким образом, на диаграмме указан лишь тип информации на шине, например, присутствие на ней адреса или данных. Пунктирной линией обозначено высокоимпедансное состояние линий шин (высокоимпедансное Z-состояние).

В первом машинном такте T1 на шину адреса (линии A15-A0) выдается адрес – содержимое указателя команд IP, если выполняется цикл ВЫБОРКА, или содержимое регистра-указателя адреса – если выполняется цикл ЧТЕНИЕ ПАМЯТИ. Вместе с тем на шину данных (линии D7-D0) выдается *байт состояния*, а также формируется сигнал SYNC на одноименном выводе i8080.

Во втором такте T2 заканчивается поступление байта состояния и сигнала SYNC, продолжительность которых равняется одному такту. В машинном цикле ВЫБОРКА содержимое IP увеличивается для адресации следующего байта команды или следующей команды. В этом же такте устройство управления микропроцессора производит анализ сигналов на входах READY и HOLD, а также контроль выполнения команды останова HLT. Если память или внешнее устройство не готовы к обмену (READY = 0) вследствие того, что поступил запрос DMA (HOLD = 1) или выполняется команда останова HLT, то обмен данными осуществляться не может, и процессор переходит в один из режимов – ожидание, захват шин или останов. В этих режимах осуществляется ожидание сигнала на протяжении нескольких тактов ожидания Tw, количество которых определяется внешними сигналами. На Рис.204 в такте T2 сигнал READY равняется логическому нулю, а в такте Tw - логической единице.

В такте T3 в зависимости от типа машинного цикла осуществляется обращение к памяти, стеку или внешнему устройству. В результате в процессор вводится (см. Рис.204) или из него выводится (см. Рис.205) байт команды, адреса или данных. В зависимости от типа команды машинный цикл может содержать такты T4 и T5 (например, если для выполнения команды нужна обработка операндов). В последнем такте команды (T3, T4 или T5) анализируется наличие сигнала запроса прерывания INT. Если прерывание разрешено, то процессор переходит к машинному циклу ПРЕРЫВАНИЕ.

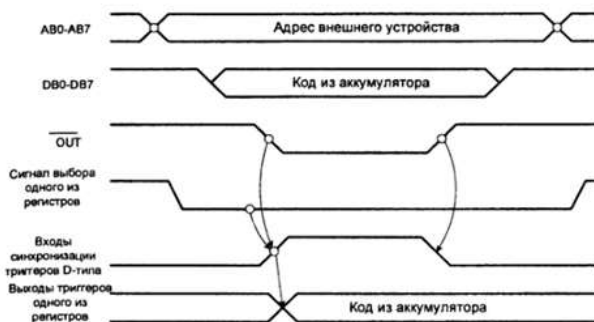


Рис.206 - Временная диаграмма работы порта вывода

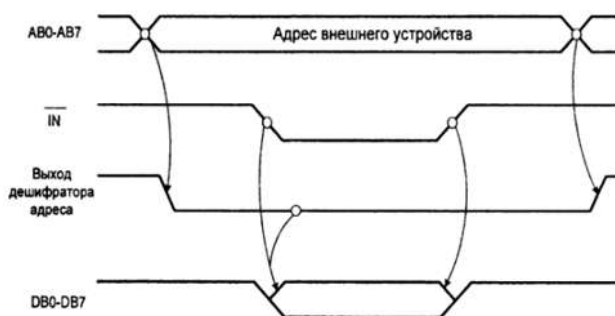


Рис.207 - Временная диаграмма работы порта ввода

Шина i8080, если не принимать во внимание сигнал READY, работает синхронно, каждый цикл привязан к основной тактовой частоте. Сигнал READY вносит элемент асинхронности, «растягивая» цикл на время неготовности устройства, вставляя такты ожидания, фактически останавливая процессор на время, пока устройство будет держать входной сигнал READY = 0. Просто, где-то даже примитивно и очень ненадёжно – некорректно работающее устройство может «навечно» остановить всю систему<sup>1</sup>.

Неопытность разработчиков процессора i8080 проявилась как минимум в ещё одном компоненте системы – при подаче сигнала сброс (RST) процессор начинает выполнение программы с нулевой ячейки памяти, что для машин с неймановской архитектурой довольно неудобно – обычно нижние адреса памяти заняты структурами операционной системы (например, таблицами векторов прерываний, ядром ОС, которые размещались в «нижней» памяти) и должны были обеспечивать доступ на чтение-запись (RAM/ОЗУ). Например, в одной из лучших операционных систем для микропроцессора i8080 (и его последователей – i8085, Zilog Z80) CP/M<sup>2</sup> первые 16Кбайт памяти обязательно должны были быть ОЗУ/RAM (а не ПЗУ/ROM). В результате разработчикам машин на базе i8080 пришлось дополнительно дорабатывать в системе аппаратные компоненты, которые реализовывали следующий алгоритм (один из возможных) при сбросе процессора: при подаче сигнала СБРОС (RST) в нулевые адреса вместо ОЗУ подставляется ПЗУ (т.н. shadow ROM), в котором была реализована программа загрузки ядра ОС с дискового устройства. После загрузки и настройки ОС, ПЗУ отключалось и «уступало место» ОЗУ, которое размещалось в положенных ему «нижних» адресах, начиная с нулевого.

## Подключение периферийных устройств

Важнейшим фактором, способствовавшим быстрому развитию микрокомпьютеров на базе интеловских i8080/85, было решение производителя выпустить большую номенклатуру периферийных программируемых контроллеров. В результате, для подключения требуемого периферийного устройства – клавиатуры, диска, дисплея и т.п. – можно было задействовать готовый, специализированный контроллер нужного типа, без необходимости дорогостоящей разработки цифровых схем с использованием ИС малой и средней степени интеграции. Периферийные контроллеры имели унифицированное стандартизованное подключение к шине процессора, в результате разработка аппаратной составляющей при создании микроЭВМ резко упрощалась, центр тяжести кардинально смещался в область программного обеспечения – для работы в составе системы стандартно-однозначно включённый в схему периферийный контроллер всего лишь требовалось инициализировать и запрограммировать на нужный режим, что обычно достигалось записью в его управляющий регистр нескольких «магических» байт (обычно хорошо документированных). Такая программируемая периферия резко снизила порог вхождения на уровне «железа» машины, сильно удешевляя аппаратную составляющую – совершенно работоспособную принципиальную, даже весьма сложную, схему микроЭВМ, при использовании «строительных кубиков» от Intel и справочных руководств по ним, мог в результате создать разработчик даже не очень высокой квалификации. Главным действующим лицом становился тот, кто пишет для всего этого программы.

Номенклатура периферийных контроллеров и дополнительных схем, выпускаемых Intel, была довольно обширна, вот несколько примеров таких микросхем:

---

<sup>1</sup> Из-за схемотехнических особенностей i8080 имел ограничение по тактовой частоте снизу – тактовая частота не могла быть ниже несколько сотен кГц. Следовательно, если периферийное устройство выставляло неготовность (READY = 0) на шине на достаточно длительный период – порядка нескольких десятков микросекунд – система на базе i8080 просто переставала работать.

<sup>2</sup> CP/M (Control Program/Monitor либо Control Programs for Microcomputers) — операционная система, первоначально предназначенная для 8-разрядных микрокомпьютеров. Написана в 1973 году программистом Гари Килдаллом на языке программирования PL/M. Стала первой операционной системой общего назначения для микрокомпьютеров, в которой были предусмотрены операции с дисковыми магнитными накопителями.

- 8216/8226 — шинные формирователи (8226 — инвертирующий) с большой нагрузочной способностью;
- 8224 — генератор синхросигналов (Рис.201);
- 8228/8238 — схемы системного контроллера и формирователя шины данных, обеспечивающие демультиплексирование передаваемых процессором сигналов управления шиной (Рис.201);
- 8231 — арифметический сопроцессор;
- 8232 — процессор чисел с плавающей запятой (32 и 64 разряда);
- 8251 — микросхема последовательного интерфейса;
- 8253 — 3-канальный таймер/счётчик;
- 8255 — микросхема трёхканального параллельного интерфейса;
- 8256 — микросхема многофункционального периферийного адаптера (5 таймеров, два параллельных порта, последовательный интерфейс, контроллер прерываний);
- 8257/37 — контроллер прямого доступа в память
- 8259 — контроллер прерываний;
- 8271 — контроллер накопителя на гибких магнитных дисках;
- 8275 — контроллер дисплея;
- 8355 — микросхема интерфейса с периферией (с 16 Кбайт ПЗУ).

Для работы в составе микроЭВМ на базе замечательного микропроцессора Zilog Z80<sup>1</sup> можно было использовать как интеловские периферийный контроллеры, так и разработанные компанией Zilog специально для Z80. Некоторые категории контроллеров в системах на базе Z80 были не нужны – например, можно было обойтись без контроллера прерываний (i8259), без внешнего системного контроллера i8228/38 (он был встроен в процессор, примерно также, как Intel сделала в своём i8085). Z80, будучи бинарно совместимым с i8080, благодаря своим преимуществам уже через несколько лет вытеснил 8080 с рынка, став одним из самых популярных 8-разрядных процессоров и вероятно самым массовым (считая однокристалльные контроллеры с системой команд Z80). Если 8085 широко применялся в контроллерах и производился до конца 1990-ых, то 8080, найдя применение лишь в микрокомпьютерах первой волны, уже в начале 1980-ых годов в большинстве стран был снят с производства. Как пример, одно из интересных свойств Z80: в несколько раз большее, чем обычно у 8-разрядных процессоров, число регистров (РОН) позволяло строить на Z80 микроконтроллеры вообще без ОЗУ, используя «альтернативные» регистры для оперативного хранения данных (потому на Z80 легко было писать тесты ОЗУ, а как примеры изделий на Z80 без ОЗУ, можно упомянуть часы с развитым функционалом и контроллеры клавиатуры).

Несмотря на существенно более продвинутый Z80 по сравнению с i8080, дальнейшее рассмотрение развития интеловских архитектур в курсе будет основываться всё же на i8080<sup>2</sup> – как неоднократно показывала жизнь, далеко не всегда побеждает лучший. И если в восьмиразрядной «Вселенной» Z80 несомненно был одним из лучших, если не лучшим, то последующие процессоры от Zilog с увеличенной разрядностью – 16, 32 – не смогли составить серьёзной конкуренции x86 линейке от Intel, даже таким весьма неуклюжим процессорам, как i8086 и i80286.

<sup>1</sup> **Zilog Z80** — 8-разрядный микропроцессор, разработанный и производившийся фирмой Zilog с 1976 года. Z80 вместе с 6502 были доминирующими ЦПУ на рынке 8-разрядных компьютеров с середины 1970-х до середины 1980-х годов. После ухода из Intel Федерико Фаджин основал компанию Zilog. К проекту присоединились Ральф Уингерманн (инженер из Intel) и Масатоши Шима (японский инженер, один из создателей Intel 4004). Команда Фаджина сразу же принялась за разработку нового процессора, взяв за основу Intel 8080, выпущенный незадолго перед этим. Новый CPU создавался бинарно совместимым с 8080, так что код для этого процессора мог работать на новом процессоре, в том числе и операционная система CP/M.

<sup>2</sup> Достаточно взглянуть на материнскую плату первого IBM PC – почти без исключения там были использованы восьмиразрядные периферийные контроллеры от Intel эпохи i8080. Одно явное исключение – микросхема UART 8250 от National Semiconductor, т.к. «родная» интеловская 8251 была слишком универсальной (могла работать, например, в т.ч. в синхронном режиме) и потому оказалась избыточной в реальной жизни.

Предложенная Intel схема обработки прерывания подразумевала преобразование номера входа, на котором возник запрос на прерывание, в адрес (или номер) вектора прерывания, своего рода «географическая» адресация. К сожалению (здесь снова сказался недостаток опыта разработчиков Intel) входы запросов прерывания имели активный высокий уровень и чисто схемотехнически на них невозможно было реализовать «монтажное ИЛИ», т.е., количество источников прерывания не могло превышать количество входов IRQ контроллера Intel 8259. По сути эта микросхема работала как мультиплексор, объединяя до восьми источников прерывания в один источник, который заводился на вход INT микропроцессора i8080. Восемь возможных источников прерывания в реальных системах, как оказалось – это очень немного (например, в первой IBM PC, где использовался i8259A, под системные нужды было занято пять из восьми входов IRQ). Разработчики IBM PC/AT учли этот момент и добавили второй контроллер i8259A, допускавший каскадное включение, в результате количество входов IRQ увеличилось почти вдвое:  $8 + 8 - 1$  (занят под каскадирование) = 15. Сравните с подходом в DEC-их архитектурах того же периода (середина 1970-хх), например, LSI-11: там единственный вход запроса прерывания никак не ограничивал количество источников прерываний.

Первоначальная версия контроллера i8259 была предназначена только для процессоров i8080 и i8085. По сигналу подтверждения прерывания INTE от процессора она выдавала на шину три байта, первый соответствовал коду операции «Вызов подпрограммы» (CALL, значение 0xCD в шестнадцатеричной системе), затем два байта адреса вызова этой подпрограммы, которые нужно было записать в соответствующие каждому источнику прерывания внутренние регистры контроллера (во время его начальной инициализации). Для процессоров семейства x86 была выпущена микросхема 8259A, так как механизм передачи управления изменился. Теперь на шину выдавались последовательно два байта, первый был «холостым», его значение микропроцессор игнорировал, во втором байте был не номер вектора прерывания, а адрес, по которому следовало передавать исполнение программы, он брался из таблицы векторов прерываний. У процессоров 8086, 80186 и 8088 она занимала самый нижний килобайт адресного пространства памяти. У более поздних моделей, начиная с i80286, таблица векторов прерываний могла быть перемещена в любое место памяти.

Использование контроллера i8259 очень облегчало жизнь разработчикам «железа» для систем на базе i8080/85, но накладывало серьёзные ограничения на возможное количество источников прерывания – их могло быть не более восьми. Для увеличения количества входов можно было каскадировать контроллеры, но это было дорогое, плохо масштабируемое решение.

Для работы, как и большинство других периферийных контроллеров от Intel, i8259 необходимо было инициализировать – задать приоритеты входов IRQ (фиксированный или rotating приоритет), соответствие адреса вектора номеру входа IRQ, битовую маску прерываний, работу по фронту или по уровню<sup>1</sup> сигнала запроса.

Подобная «географическая» адресация была реализована Intel в ещё одном их очень популярном программируемом четырёхканальном контроллере прямого доступа к памяти (DMA) i8257/37. Источников захвата шины в системе могло быть не больше, чем количество входов DRQ в микросхеме – всего четыре. С точки зрения аппаратной реализации использование этого контроллера значительно сокращало затраты на схемотехнику, но, также, как и с контроллером прерываний, накладывало ограничение на количество устройств, работавших по прямому доступу. Опять же, во всё той же IBM PC/AT, проблему недостаточного количества входов DRQ решили точно также, как с i8259 – просто добавили второй экземпляр контроллера, но с небольшой вариацией – его адресную шину сдвинули на разряд влево (линия A0 всегда была равна нулю), в результате чего второй контроллер DMA мог обращаться к памяти только по чётным адресам, по словной, а не байтовой, границе, и поэтому в документации он обозначался как «словный» DMA.

---

<sup>1</sup> Это функционал был добавлен в версии микросхемы i8259A/B.



Точно также, как и любой другой контроллер, для работы в реальной системе i8257/37 должен быть инициализирован (запрограммирован). Для работы конкретного любого канала в управляющие регистры контроллера необходимо было занести три параметра:

1. Стартовый (базовый) адрес в ОЗУ, начиная с которого контроллер будет последовательно читать/писать байты.
2. Счётчик – количество пересылаемых по DMA байт. Работает на вычитание, при антипереполнении может сгенерировать запрос на прерывание.
3. Направление передачи – MEMW, MEMR, I/OW, I/OR.

Использование DMA контроллера i8257 сэкономило разработчикам-схемотехникам ещё больше микросхем малой и средней степени интеграции, чем, например, при использовании контроллера прерываний i8259. Логика работы DMA сложна, требует довольно значительного аппаратного обеспечения – счётчиков, мультиплексоров, шинных драйверов и т.п. Если взглянуть на реализованную на микросхемах малой и средней степени интеграции логику DMA в DEC-ких периферийных адаптерах, то можно было отметить, что эта часть схемы занимала там до трети/половины площади платы одинарного размера. Intel, интегрировав эту логику в свой i8257, по сути очень сильно разгрузила разработчиков схемотехники, освободив для них эту площадь под другие полезные компоненты. В случае с i8257 всё общения источника, работающего по прямому доступу в память, фактически сводилось к простейшему общению с контроллером DMA по двум линиям: запроса DRQ и подтверждения DACK, что требовало минимальной аппаратной поддержки в виде пары триггеров. В очередной раз отметим перенос «центра тяжести» разработки микроЭВМ с интеловской архитектурой с «железа» на программное обеспечение. Но при этом существовало ограничение на четыре входа (в отличие, от, опять же, DEC-кого подхода, где количество источников захвата шины не ограничивалось количеством входов у какого-либо централизованного контроллера DMA – такового в архитектуре LSI/PDP-11 просто не было), которое можно было обойти только установкой второго, третьего и т.д. экземпляра контроллера DMA.

Ещё из удачных и активно использовавшихся программируемых контроллеров Intel можно упомянуть универсальный, многорежимный контроллер параллельного интерфейса i8255 – он имел три 8-разрядных канала ввода-вывода, 8-разрядный канал для подключения к системной шине данных, два адресных входа. Третий канал ввода-вывода мог быть разделен на два канала по 4 бита. Также, конечно же, достоин упоминания очень симпатичный трёхканальный программируемый таймер от Intel – i8253/54: он мог работать в режиме счётчика интервалов, счётчика внешних событий (подсчитывая количество импульсов), программируемого делителя частоты, одновибратора.

Как упоминалось выше, инженеры IBM в своей первой IBM PC активно использовали некоторые интеловские контроллеры, в частности i8259A, i8237 (развитие i8257), i8253, i8255.

Не удостоился попадания в IBM PC ещё один довольно популярный программируемый контроллер от Intel i8251 – универсальный синхронно/асинхронный приёмо/передатчик, упакованный в корпус DIP-28. IBM отказалась от его использования, т.к. интеловский кристалл не имел встроенного генератора с нужной сеткой частот (baudrate generator) и был более сложным и дорогим, по сравнению с конкурирующим изделием 8250 от National Semiconductor (не путать, здесь 8250 – это не Intel, хотя названия микросхем очень похожи). Микросхема i8251, например, могла работать не только в самом востребованном режиме UART (реализуя стандарт RS-232-C), но и в т.н. синхронном режиме, в котором, в отличие от асинхронного режима, в паузах постоянно передаётся «пустой» заполняющий символ, что поддерживает приёмник и передатчик в синхронизме. При этом возможно достижение гораздо более высоких скоростей передачи, чем в асинхронной моде. Но в реальной жизни

оказалось, что синхронная мода гораздо менее востребована, чем UART-режим. Позже IBM всё-таки реализовала свой собственный синхронный коммуникационный адаптер, но он так и не стал популярным.

С i8251 связана любопытная история – как упоминалось выше, кристалл был упакован в один из стандартных для индустрии тех лет 28-выводной корпус DIP-28. Схемотехника контроллера была такой, что разработчикам не хватило условно двадцать девятого вывода для реализации сигнала «Сброс» (“Reset”), который гарантировано переводит микросхему в исходное предопределённое состояние, после чего её можно было бы программировать/инициализировать. Использовать следующий по размерам сорокавыводной корпус DIP-40 было бы нерационально, и Intel в итоге так и выпускала i8251 без аппаратного сигнала «Сброс».

На практике это означало, что после подачи питания i8251 оказывался в неопределённом состоянии и обычно не реагировал на начальные команды инициализации/программирования. В наборе инструкций, которыми программировался контроллер, была команда программного сброса, но, чтобы её выполнить, i8251 должен был находиться в состоянии приёма и исполнения команд. Замкнутый круг. Исходная доступная документация хранила по этому поводу молчание. Эмпирически был найден способ привести i8251 «в чувство» – оказалось, что если послать в качестве команды три (или пять?) не нулей (например, 0x01 или 0xFF), то контроллер гарантированно оказывался в состоянии приёма и интерпретации команд и начинал правильно реагировать на команду «Сброс» (0x05) – сбрасывался и переходил в начальную точку процедуры инициализации, по которой далее нужно было отправить в контроллер три-пять команд (байт), программируя его в нужный режим и настраивая параметры приёмопередатчика.

Интересный момент из жизни: несколько лет спустя, после того, как на практике пришлось столкнуться с проблемой начальной инициализации i8251, довелось увидеть реальные исходные тексты операционной системы CP/M, в которых автор – Гари Арлен Килдалл, – в комментариях в подпрограмме инициализации i8251, особо не стесняясь в выражениях, написал всё, что он думает о разработчиках Intel, «сэкономивших» на пине «Сброс» для этого контроллера. И его «Эврика!» – необходимость записи пяти не нулей в командный регистр контроллера – сопровождалась несколькими восклицательными знаками. Видимо, и у Килдалла были проблемы с документацией. ☺

## Программное обеспечение

Упомянутый выше Гари Килдалл<sup>1</sup> первым в мире создал программное обеспечение для микропроцессоров, позволившее использовать МП не только во встроенных системах (например, в калькуляторе), а как основу для универсальных компьютеров – язык программирования, операционную систему, ставшую стандартом де-факто для микропроцессорных систем (не только интеловских).

## История создания<sup>2</sup>

В 1972 году молодой человек по имени Гари Килдалл преподавал информатику в аспирантуре военно-морского колледжа в Монтерее, Калифорния. Однажды он увидел на доске объявлений рекламу первого микропроцессора от Intel – четырехбитного i4004. Гари решил приобрести для себя этот микропроцессор и начал писать для него различные программы. Вскоре он посетил компанию Intel и стал немного подрабатывать консультантом в одном из её небольших подразделений. Работая один день в неделю на Intel в свободное от учёбы время, Гари написал первый язык программирования для микропроцессора Intel

---

<sup>1</sup> В русскоязычной литературе можно встретить различные варианты написания имени и фамилии этого человека, например, Гэри Килделл. На языке оригинала его имя пишется как Gary Kildall.

<sup>2</sup> Эта часть текста, посвящённая ОС CP/M, взята с некоторыми сокращениями и исправлениями отсюда: [Как это было: операционная система CP/M](#)



Рис.208 – Gary Kildall

8008. Он назвал свой язык PL/M - Programming Language for Microcomputers (язык программирования для микрокомпьютеров). В качестве части оплаты Intel подарила Килдаллу Intellec-8 – систему разработки, которая была постепенно модернизирована Intel для работы с процессором i8080, дополнена монитором и устройством для чтения перфолент. В 1973 году Гари удалось достать первый дисковод фирмы Shugart. Друг Килдалла John Torode по его просьбе изготовил контроллер для этого устройства.

Таким образом удалось подключить дисковод к его Intellec-8, что превратило эту систему в настоящий микрокомпьютер. Гибкие диски давали огромное преимущество перед перфолентами, но было необходимо программное обеспечение,

которое позволяло бы системе работать с дисководом. И Гари Килдалл создал такое ПО. Используя свой язык **PL/M** он написал первую операционную систему для микропроцессоров. Он назвал ее **CP/M** (Control Program for Microcomputers).

Гэри закончил писать CP/M в 1974 году, и сразу же предложил приобрести её Intel за 20.000 долларов. Но компания Intel ответила, что не заинтересована в приобретении дисковой операционной системы. Хотя, нужно заметить, язык PL/M Intel всё-таки купила.

В 1976 году Гари и его супруга Dorothy McEwen создали фирму под названием "Intergalactic Digital Research" (которая позже была сокращена до Digital Research Inc или DRI) для продаж CP/M. Гари занимался программированием, а Дороти непосредственно делами фирмы. В середине 70-х рынок микрокомпьютеров начал расти, и CP/M оказалась в нужное время в нужном месте. Большинство производителей компьютеров не хотело самостоятельно разрабатывать операционные системы, и они рекомендовали пользователям приобретать уже готовые варианты, одним из которых была CP/M. На протяжении 70-х Digital Research выпустила 2 значительные версии CP/M. Первая версия CP/M. 1.4 использовалась не так широко, но вторая версия CP/M 2.2 стала промышленным стандартом. Если вы покупали компьютер в 1970-ые на базе процессора Intel 8080 или Zilog Z80, он поставлялся с операционной системой CP/M. К концу 70-х фирмой Digital Research было продано более четверти миллиона копий операционной системы, которая работала более чем на 3000 типов микрокомпьютеров.

Интересный факт: обозначение дисковых устройств в виде буквы с двоеточием (например, A:) было придумано Гари Килдаллом, и было впоследствии беззастенчиво скопировано в MS-DOS, как и практически все команды командной строки.

## Разновидности CP/M

Популярность и число микрокомпьютеров стремительно росли, они использовались для решения множества различных задач. Это, в свою очередь, повышало требования к выполняемым программам и, в первую очередь, к операционной системе.

Исходно ОС CP/M является однопользовательской операционной системой и может одновременно управлять выполнением только одной программы, но Digital Research разработала несколько разновидностей CP/M, которые предоставляли пользователям больше возможностей.

Вот их краткое описание вариантов CP/M:

– **MP/M, MP/M-II** (Multiprogramming Monitor for Microcomputers)

Это многопользовательская, многозадачная операционная система для микроЭВМ, созданных на базе микропроцессоров Intel 8080, Intel 8085 и Zilog Z80. Операционная си-



стема МР/М-II является второй версией МР/М со значительно меньшей требуемой оперативной памятью и новыми расширенными возможностями. Требования к оборудованию те же, что и у СР/М, однако с некоторыми отличиями, а именно:

- наличие как минимум 48 Кбайт оперативной памяти
- наличие прерываний от таймера

Кроме этого, в МР/М-II имеются средства расширения возможностей оборудования до 16 печатающих устройств и 16 устройств ввода-вывода.

#### – СР/М-86

Эта операционная система разработана на базе СР/М и предназначена для функционирования на микрокомпьютерах, созданных на базе процессоров Intel 8086 и Intel 8088 (например, на IBM PC). СР/М-86 в основном аналогична СР/М, поскольку она также является однопользовательской операционной системой, поддерживает в основном те же команды и фактически тем же способом организует взаимодействие с пользователем. Однако ОС СР/М-86 предназначена для работы с микропроцессорами другого типа и обладает существенно большими возможностями по взаимодействию с аппаратурой.

#### – Concurrent СР/М-86

Операционная система Concurrent СР/М-86 создана на базе ОС СР/М-86 и включает многие характерные возможности ОС СР/М версии 3.0. Эта операционная система отличается от ОС СР/М-86 тем, что она поддерживает параллельный режим обработки, т.е. позволяет пользователю выполнять несколько программ одновременно (многозадачность). Это достигается за счет использования так называемой «виртуальной консоли». Виртуальная консоль создается самой операционной системой после того, как пользователь запустил программу. С виртуальной консолью обращаются как с реальным физическим устройством, однако управление им полностью осуществляется операционной системой.

#### – МР/М-86

Операционная система МР/М-86 является многопользовательской, многозадачной операционной системой, предназначенной для функционирования на микро-ЭВМ, собранных на основе микропроцессора Intel 8086 или Intel 8088. В целом МР/М-86 разработана как дополнение к ОС СР/М-86 и обладает в основном теми же характеристиками что и МР/М-II.

#### – СР/NET

Операционная система СР/NET предоставляет средства, с помощью которых несколько микрокомпьютеров, оснащенных ОС СР/М, ОС МР/М или МР/М-II, можно объединять в локальную сеть с тем, чтобы иметь возможность совместного использования ресурсов каждой системы. К таким совместно используемым ресурсам относятся дисковые файлы, устройства печати, консольные терминалы и различные программы баз данных. Сеть обычно состоит из главной вычислительной системы, оснащенной ОС МР/М или ОС МР/М-II и вспомогательных систем, оснащенных ОС СР/М. Главная вычислительная система функционирует в качестве базовой ЭВМ, управляющей разделяемыми ресурсами, к которым могут обращаться вспомогательные системы, включенные в сеть. С помощью ОС СР/NET можно создавать вычислительные сети различного типа, а несколько вспомога-

тельных систем можно соединить с отдельной главной системой. ОС CP/NET также позволяет передавать сообщения между главной и вспомогательными системами, а при организации электронной почты обеспечивает пересылку корреспонденции между всеми главной системой и вспомогательными.

Существовало несколько версий операционной системы CP/M, адаптированных для работы на базе различных микропроцессоров: CP/M-68k для Motorola 68000, CP/M-8000 (CP/M-8k) для процессора Zilog Z-8000 и др. Хочется заметить, что в Советском Союзе CP/M версии 2.2 была «неофициально» переведена на русский язык. Данная система получила название Микро-ДОС и была адаптирована для работы на советских компьютерах типа «Корвет», «Вектор», а также "Robotron" производства ГДР.

Под ОС CP/M было разработано большое количество прикладного программного обеспечения: трансляторы популярных языков программирования, текстовые редакторы, системы управления базами данных, электронные таблицы, игры и т.п.

## Архитектура, файловая система

Минимальные системные требования для запуска 8-битной версии CP/M следующие:

- микропроцессор Intel 8080, Zilog Z80 или аналог
- минимум 16 Кбайт ОЗУ
- минимум 1 дисковый накопитель
- терминал

Первоначальный загрузчик операционной системы должен находиться в BIOS или же на диске (дискете). Далее он загружает основную часть операционной системы. Операционная система CP/M, находясь в памяти, логически делится на 4 основные части:

- BIOS (basic input/output system)
- BDOS (basic disk operation system)
- CCP (console command processor)
- TPA (transient program area)

BIOS обеспечивает базовые операции ввода-вывода: доступ к дискам, а также к периферийному оборудованию. BIOS является аппаратно-зависимой частью. Под каждую модель (конфигурацию) микрокомпьютера необходим свой БИОС.

BDOS обеспечивает выполнение дисковых операций, контролируя один или более дисковых накопителей с независимыми файловыми системами. BDOS позволяет выполнять следующие примитивные дисковые операции:

- поиск (ищет файл по имени)
- открытие (открывает файл для дальнейших операций с ним)
- закрытие (закрывает файл)
- переименование (изменяет имя файла)
- чтение (считывает запись из открытого файла)
- запись (записывает запись в открытый файл)
- выбор (выбирает логический диск)

CCP является интерпретатором командной строки. Обеспечивая «диалог» пользователя и операционной системы, CCP считывает и выполняет команды, которые пользователь вводит в командной строке.

ТРА – это участок памяти, в котором операционная система хранит загруженные программы и их данные. К примеру, если пользователь загрузил текстовый редактор ED, то сам редактор, а также файл, с которым мы работаем, во время работы хранятся в ТРА.

### **Файловая система**

Имя файла на диске состоит из следующих частей: имя файла, до 8 символов, затем следует точка, затем расширение файла, до 3 символов. Пример:

MYFILE.TXT

Недопустимые символы в имени файла: < > . , ; : = ? \* [ ] % | ( ) / \

Расширение обычно указывало на тип файла, на то, какие именно данные он содержит.

Размер файла определялся количеством 128-байтовых записей, которые непосредственно соответствовали секторам на 8" дискетах. Поэтому не было точного способа узнать, сколько именно байт на диске занимает файл.

Времена модификации файла (timestamps) изначально не поддерживались, хотя некоторые более поздние варианты CP/M добавили эту особенность. ОС CP/M 2.2 не поддерживала никаких подкаталогов в структуре диска, но обеспечивала 16 т.н. «пользовательских областей», чтобы организовать хранение файлов на диске.

Наличие пользовательских областей приближало однопользовательскую ОС CP/M по функционалу к многопользовательской MP/M. Команда «USER номер» переключала пользователей на номера от 0 до 15. Пользователь с номером 0 был пользователем по умолчанию. Если человек за компьютером изменил пользователя с номера 0 на 1, то документы, сохраненные для пользователя 0, будут недоступны пользователю 1.

### **Команды CP/M**

В операционной системе CP/M существует два типа команд: встроенные команды и транзитные команды. Встроенные команды являются частью операционной системы и постоянно хранятся вместе с ней в одной общей области памяти. Транзитные программы команд – это команды, которые хранятся на диске в виде командных (.COM для CP/M или .CMD для CP/M-86) файлов, т. е. для использования какой-либо транзитной команды необходимо наличие соответствующего файла, реализующее транзитную команду.

### **Встроенные команды**

Операционная система CP/M содержит секцию, которая называется «процессор команд консоли» (ССР). Секция ССР является подпрограммой ОС CP/M, которая проверяет введенные с клавиатуры консоли группы символов на соответствие командам ОС CP/M. Секция ССР включает в себя набор встроенных или «резидентных» команд, для обращения к которым не требуется доступа к диску. Вызов этих встроенных команд осуществляется с помощью ввода в качестве ответа на промпт ОС CP/M (A >, B > и т. д.) имени команды. Поскольку эти команды не содержатся в файлах, они доступны для пользования независимо от того, какое дисковое устройство является текущим, то есть, в отличие от транзитных программ, для выполнения встроенных команд обращения к диску не требуется. Ниже представлено краткое описание каждой встроенной команды ОС CP/M.

#### **- DIR**

Команда DIR является, по-видимому, наиболее часто используемой командой ОС CP/M. Название DIR является сокращением английского слова directory (каталог). Команда

DIR используется для отображения или печати справочных данных всех файлов, содержащихся на указанном диске

#### **- REN**

Название REN является сокращением английского слова rename (переименовать). Команда REN используется для изменения имени файлов, хранящихся на обозначенном диске.

#### **- ERA**

Название ERA является сокращением английского слова erase (стереть). Команда ERA используется для удаления файлов, хранящихся на обозначенном диске.

#### **- TYPE**

Команда TYPE используется для просмотра на консоли содержимого файлов.

#### **- USER**

Команда USER предоставляет различным пользователям ОС CP/M средства обращения к своим файлам, хранящимся на одном и том же физическом диске. В ОС CP/M с одним и тем же диском могут работать до 15 пользователей, причем рабочие области диска каждого пользователя (совокупность его личных файлов) не пересекаются. Команда USER имеется только в версии 2.0 и более поздних версиях ОС CP/M

#### **- SAVE**

Команда SAVE используется для того, чтобы взять содержимое указанного числа страниц из системной области оперативной памяти и сохранить его в файле на обозначенном диске. Эта команда обычно используется только системными программистами.

### ***Транзитные команды***

Транзитные команды (утилиты) отличаются от встроенных команд тем, что они временно, только на время их выполнения, загружаются в оперативную память. Т.е., транзитные программы хранятся в исполняемых файлах, расположенных на дисковых носителях. Для того чтобы пользоваться транзитной командой, её следует хранить на диске, установленном в одном из устройств системы.

Далее описаны только те команды, которые всегда находятся на системном диске ОС CP/M. Хочется отметить, что на системном диске могут находиться и другие, нестандартные (сторонние) программы-утилиты (которые фирмой Digital Research не поставлялись).

Ниже представлено краткое описание стандартных транзитных команд. Эти команды имеются во всех версиях (1.3, 1.4, 2.0 и 2.2) ОС CP/M, если не оговорено особо.

#### **- STAT**

Название STAT означает Status или Statistics - состояние или статистика. Команда STAT предоставляет пользователю ОС CP/M различные средства для назначения статуса и характерных свойств файлам, дисковому носителю и другим устройствам. Команда широко используется для получения размеров файла, объема неиспользуемого дискового пространства (в килобайтах), получения различных атрибутов файла.

## **- PIP**

Название PIP<sup>1</sup> является сокращением от английского Peripheral Interchange Program (программа обмена данными между периферийными устройствами). Команда PIP используется для копирования файлов с какого-либо устройства на устройство того же или другого типа, например, с диска на диск, с диска на печатающее устройство, с консоли на диск, с диска на модем и т. д.

## **- ED**

Название ED является сокращением английского слова EDiTор (редактор). Команда ED позволяет пользователю ОС CP/M запустить текстовый редактор, который может создавать и редактировать текстовые файлы.

## **- SUBMIT**

Команда SUBMIT предоставляет возможность автоматического запуска указанной последовательности команд, предварительно подготовленных пользователем в отдельном файле.

- **XSUB** (имеется только в версии 2.0 и более поздних).

Название XSUB является сокращением от английского Extended SUBMIT (расширенная программа SUBMIT). Программа XSUB входит в состав программ команды SUBMIT. Она позволяет вводить ответы на промпты программы, уже запущенной на исполнение программой SUBMIT.

## **- ASM**

Название ASM является сокращением английского слова Assembler. Программа ASM «ассемблирует» файлы с программой на ассемблере, созданные в редакторе ED или любом другом текстовом редакторе, в так называемые HEX-файлы (шестнадцатеричные), которые, в свою очередь, можно по команде LOAD преобразовать в файлы с машинным или объектным кодом. Программа ASM используется программистами на ассемблере при создании собственных транзитных программ.

## **- LOAD**

Команда LOAD используется для преобразования шестнадцатеричного (".HEX") файла в файл, содержащий машинный или объектный код программы.

## **- DDT**

Название DDT означает Dynamic Debugging Tool (инструментальное средство для динамической отладки программ). Это утилита, которая предоставляет пользователю ОС CP/M метод определения и устранения ошибок в программах, представленных в машинном объектном коде или шестнадцатеричном формате, в целях проверки секции базовой системы ввода-вывода (BIOS) и базовой дисковой операционной системы (BDOS).

---

<sup>1</sup> Любопытный факт: в DEC-кой операционной системе RSX-11M (середина 1970-хх) была утилита точно с таким же названием и функционалом.

## **- DUMP**

Команда DUMP используется для вывода в шестнадцатеричном формате содержимого файла с объектным/машинным кодом на экран дисплея или печатающее устройство.

## **- FORMAT**

Команда FORMAT служит для инициализации диска. Эта форматирующая программа подготавливает гибкий диск к записи стандартных, совместимых с ОС CP/M файлов. Кроме того, форматирующую команду можно включить в систему, содержащую жесткий диск типа «Винчестер» (жёсткий диск). Программа FORMAT является не стандартной и может иметь несущественные различия в работе, в зависимости от поставщика версии операционной системы CP/M. На различных ЭВМ назначение и принципы действия программы FORMAT остаются примерно одинаковыми, в то время как детали физической организации дорожек и секторов меняются от системы к системе. Таким образом, большинство форматов являются зависимыми от технического обеспечения микрокомпьютера.

## **- SYSGEN**

Название SYSGEN является сокращением от английского System Generation (генерация системы). Команда SYSGEN предоставляет пользователю средства для генерирования операционной системы CP/M или копирования ее с одного диска на другой.

## **- MOVCPM**

Название MOVCPM является сокращением от английского "Move CP/M" (переместить CP/M). Команду MOVCPM применяют для модификации операционной системы CP/M с тем, чтобы она либо использовала всю доступную память ЭВМ, либо оставляла часть памяти под специальные системно-независимые программы пользователей.

Команды управляющих символов. ОС CP/M предоставляет пользователю несколько специальных команд, так называемые «команды управляющих символов». Эти команды дают пользователю ОС CP/M возможность управлять и изменять способ ввода командных строк и возможность реагировать на них. Эти команды называются «команды управляющих символов», поскольку для их выполнения необходимо одновременно нажать на клавиатуре клавишу "Control" (Ctrl) и клавишу, соответствующую указанному символу. Ниже приведен список таких команд:

**CTRL/C** – реинициализация системы

**CTRL/M** – возврат каретки

**CTRL/J** – перевод строки

**CTRL/H** – возврат курсора назад на одну позицию

**CTRL/U** – удалить текущую командную строку

**CTRL/X** – удалить и стереть текущую командную строку

**CTRL/E** – переход текущей командной строки на следующую строку

**CTRL/R** – перевести (обновить) отредактированную командную строку на следующей строке

**CTRL/S** – остановить/продолжить перемещение текста на экране консоли

**CTRL/P** – вывести на печать любой текст, распечатываемый на консоли

## Дальнейшая судьба CP/M

22 августа 1980 года представители IBM прибыли на встречу с главой Digital Research для обсуждения вопроса о приобретении CP/M. Но в ходе переговоров стороны не достигли соглашения, т.к. Гари Килдалл отказался подписывать односторонний договор о соблюдении секретности. Послы из IBM ушли с пустыми руками...

После потери сделки с IBM Гари и Дороти поняли, что управлять DRI нужно более профессионально. Гари отошел от управления Digital Research и сосредоточился на различных экспериментальных и научно-исследовательских проектах.

Одним из них был язык программирования LOGO. Килдалл надеялся, что LOGO как образовательный вариант языка LISP, вытеснит Microsoft BASIC из образовательной сферы. Но этого не произошло. Следующим проектом Гари Килдалла стала разработка собственного пользовательского графического интерфейса для CP/M, который получил название Graphical Environment Manager (GEM) Desktop. Параллельно Гари работал над новыми версиями CP/M. Так появились FLEX OS, Concurrent DOS/386, DOS-Plus. Но это уже не могло вернуть CP/M былую популярность. Время было упущено...

Digital Research была куплена компанией Novell в 1991 году. Эта сделка принесла Гари миллионы. Права на интеллектуальную собственность Digital Research перешли к компании Novell, которая на основе CP/M разрабатывала свою версию DOS – DR-DOS (multi-user DR-DOS).

В 1996 году Novell продала права на DR-DOS компании Caldera<sup>1</sup>, которая планировала использовать систему DR-DOS (модифицированную версию CP/M) в собственном программном обеспечении и выпускала ее под другим именем - Caldera OpenDOS.

В 2001 году на базе подразделения компании Caldera была создана компания Lineo, которая унаследовала все права на CP/M и DR-DOS.

19 октября 2001 компания Lineo отправила владельцу сайта любителей ОС CP/M Гэби Чодри уведомление о том, что владельцы и посетители сайта получают право на пространство и изменение исходного кода CP/M в некоммерческих целях.

Таким образом, CP/M теперь отвечает стандартам open-source, так как каждый может изменять и распространять систему согласно собственным интересам. Проект графического интерфейса GEM также обрел статус open-source. Его дальнейшая разработка выполняется любителями. GUI получил название OpenGEM и доступен по адресу: <http://gem.shaneland.co.uk>.

Хочется отметить, что самой мощной и современной ОС, построенной на базе CP/M, на сегодняшний день является ОС REAL/32. Спустя некоторое время после покупки DRI Novell передала права на развитие и распространение операционной системы фирме IMS (Intelligent Micro Software, Великобритания). Начиная с версии 7.51, выпущенной в апреле 1995 года, ОС распространяется под маркой REAL/32. Теперь REAL/32 это:

- 32-битное ядро реального времени
- многозадачная многопользовательская ОС
- поддержка до 64 пользователей, до 82 последовательных и 3 параллельных портов для соединения терминалов, принтеров и/или дополнительных устройств
- до 8 программ (DOS/WINDOWS сессий), работающих одновременно у каждого пользователя
- шлюз в Novell NetWare для поддержки до 16 одновременных NetWare сессий

и многое другое...

---

<sup>1</sup> Caldera в то время возглавлял Рэй Нуурда (Ray Noorda), бывший глава Novell.

## **Вместо заключения**

Выше была изложена удивительная история создания, головокружительного успеха и падения популярности операционной системы CP/M. Её создал по сути один человек – Гари Килдалл. Человек, который мог бы стать Биллом Гейтсом. Ведь не трудно себе представить, какого успеха достигла бы Digital Research, если бы в 1980 году ей всё же удалось заключить сделку с IBM, и вместо MS-DOS эпоха персональных компьютеров началась бы с CP/M-86.

Гениальный программист, талантливый учитель, отец двоих детей, Гари умер в 1994 году, в возрасте 52 лет, так, наверное, и не раскрыв до конца весь свой талант. Гари ушел, но он, и его труд, навсегда останутся в нашей памяти.

И поскольку появлению на свет MS-DOS и своим успехам на старте эпохи PC Microsoft, мягко говоря, обязана Гари Килдаллу, то каждый раз, когда мы загружаем Windows, будь то 98, XP, Vista или Windows 10, мы будем знать и помнить, что где-то там, внутри этих окон, есть маленький «кусочек» CP/M... (скопирую-ка я вот этот файл с диска C: на диск D:).

Интересный факт: и всё-таки CP/M жива! Даже в 21 веке она находит применение. В наше время компанией IMSAI выпускается компьютер "IMSAI SERIES TWO", работающий под этой операционной системой!

## **Архитектура компьютеров на базе Intel 8086/88**

Формально первым персональным компьютером от IBM был представленный в сентябре 1975 года IBM 5100, первый «портативный» компьютер, весом «всего-то» 25 кг.

5100 технически не был микрокомпьютером – он использовал разработанный в компании IBM процессор PALM, «распределённый» по всей материнской плате, а не содержащийся в одном микрочипе. Но с точки зрения конечного пользователя разница была не большой – он явно подходил под определение персонального компьютера. Это была самодостаточная, полная по Тьюрингу, программируемая машина размером не более чемодана, с ленточным приводом для загрузки и сохранения программ, клавиатурой, 5" экраном и памятью в 16 Кб.

IBM 5100 отличался от первой волны PC (в основном восьмиразрядных) ценой и рекламируемой областью применения. Цена начиналась от \$10 000 и легко могла забраться за отметку в \$20 000. IBM продвигала машину как серьёзный инструмент для полевых инженеров в удалённых местах, в которых нет возможности получить доступ к большим машинам IBM, а не устройство для развлечения, обучения, взлома или офисной работы.

Акценты изменились с двумя следующими итерациями концепции, моделями 5110 и 5120, которые рекламировали как системы, подходящие для офиса, с приложением для бухгалтерии, базой данных и даже текстовым процессором. Но цены оставались на высоком уровне, при том, что для реальной работы в офисе этот компьютер необходимо было подсоединять к отдельно стоящему массиву дисков, по размерам превосходившему сам компьютер, в результате чего вся система становилась больше похожей на миникомпьютер, чем на PC.

В феврале 1978 года IBM начала работу над первым микрокомпьютером – и это опять не был IBM PC. Это была машина System/23 Datamaster. Она вновь была разработана для работы в офисе, и построена на микропроцессоре Intel 8085. Она была большой и тяжёлой (вес ~43 кг) и стоила порядка \$10 000, что вкуче с её ориентацией на бизнес и другие варианты использования радикально отличало её от компьютеров вроде Apple II. Но технически это был микрокомпьютер. IBM была огромной компанией с легендарно запутанной бюрократией, что означало, что проектам для завершения иногда требовалось невероятно много времени. Несмотря на то, что проект Datamaster опередил PC на два года, он не выходил вплоть до июля 1981, как раз вовремя, чтобы его славу затмил анонс выхода IBM PC в следующем месяце (12 августа 1981).



Машина, которая станет известной, как настоящий IBM PC, (внезапно) берёт своё начало в Atari. Видимо, почувствовав вкус успеха на волне взрыва видеоигр, организованного игрой «Space Invaders» и приставкой Atari VCS, а также выпуском собственных PC, Atari 400 и 800, компания в июле 1980 сделала предложение председателю правления IBM Фрэнку Кэри: если IBM хочет сделать свой PC, Atari может снизить до разработки такого для неё.

Кэри не был таким уж ограниченным любителем мейнфреймов, как его пытаются представить<sup>1</sup>. Он ратовал за разработку малых систем – даже если под «малыми» в IBM часто понимали нечто совсем иное, чем в остальном мире. Кэри принёс это предложение к директору по системам ввода данным IBM, Биллу Лоу, работавшему в городе Бока-Ратон во Флориде. Лоу представил предложение комитету управляющих, объявивших его «тупейшим из всего, что им доводилось слышать». И действительно, IBM и Atari составляли вместе весьма и весьма странную пару. Но при этом все понимали, что Лоу выступает по личной просьбе председателя правления, а такие предложения запросто не отменяются, если вы, конечно, заботитесь о вашей карьере. Так что они велели Лоу собрать команду для составления детального предложения, как именно IBM могли бы самостоятельно сделать PC, и представить его через месяц.

Лоу собрал команду из 12 или 13 человек (источники расходятся в точной цифре) для создания черновика предложения. Нарушая традицию компании, он специально не стал собирать большую команду и вводить формальную структуру управления, надеясь создать какую-то «хакерскую магию», которая и породила PC. Его менеджер проектов Дон Эстридж сказал: «Если вы соревнуетесь с людьми, начинавшими в гараже, вам надо начинать в гараже».

В истории появления PC от IBM много интересных моментов – обычно IBM, абсолютно доминировавшая в те годы в компьютерной индустрии, разрабатывала свои ЭВМ полностью самостоятельно – можно сказать, от кремния до операционных систем и прикладного софта<sup>2</sup>. Но в случае с IBM PC корпорация пошла по необычному для неё пути: был использован «чужой» процессор, «чужие» периферийные контроллеры и, пожалуй, самое невероятное – не собственная операционная система (MS-DOS/PC-DOS). На разработку IBM PC были отведены очень сжатые сроки – всего год, и это была одна из существенных причин, по которой IBM использовала сторонние разработки.

Ещё одним необычным для IBM при разработке PC стала открытая архитектура машины – за очень небольшую, практически символическую, плату любой желающий мог разрабатывать собственные платы расширения или даже полностью «клонировать» компьютер. Это решение привело к взрывному росту альтернативных, программно и аппаратно совместимых с IBM PC «клонов», по сути, это породило целую индустрию IBM PC compatible компьютеров.

Можно сказать, что IBM сделала компьютерный конструктор, набор взаимозаменяемых компонентов, которые покупатель мог собрать в любой комбинации, подходящей для его потребностей и кошелька. Сразу после запуска у него был выбор между цветной видеокартой, способной рисовать графику и игры, или монохромной картой, способной выдавать 80 столбцов текстовых символов. Можно было выбирать объём встроенной памяти в промежутке от 16 Кб до 256 Кб, один или два флоппи-дисководы, или кассетный привод, и т.п. В результате, когда все сторонние компании включились в эту игру, и IBM расширила продуктовую линейку, можно было просто утонуть в вариантах выбора.

Среди интересных моментов, которые инженеры IBM внесли в исходную архитектуру системы PC, можно отметить знаменитую функцию **мягкой (тёплой) перезагрузки** с помощью клавиатуры, т.н. салют из трёх пальцев. Эта функция впервые была разработана

---

<sup>1</sup> Политику IBM в те годы можно сформулировать примерно следующим образом: выгоднее продать одну систему (читай, мейнфрейм) за 1 млн. \$, чем 1000 систем по 1000 \$. Микрокомпьютеры IBM не воспринимала, как что-то, достойное их внимания.

<sup>2</sup> Хотя в выше упомянутой системе от IBM System/23 Datamaster уже использовался «иностраный» i8085.

Дэвидом Брэдли, главным инженером проекта IBM PC и разработчиком машин ROM-BIOS. Первоначально он использовал для этого комбинацию Ctrl+Alt+Esc, но потом решил, что можно случайно нажать клавиши в левой части клавиатуры и перезагрузить компьютер. Поэтому главный программист проекта предложил использовать комбинацию клавиш Ctrl+Alt+Del в качестве меры безопасности. На клавиатуре IBM PC такую комбинацию невозможно нажать одной рукой. По легенде идея «мягкой перезагрузки» возникла из-за того, что инженерам IBM надоело постоянно «нырять» под стол и перезагружать часто зависавший компьютер нажатием кнопки «Reset» (холодный рестарт).

Выпустив в августе 1981 года в очень необычном для себя стиле совершенно нехарактерный для корпорации продукт, IBM сдержанно-оптимистично оценивала предполагаемое количество проданных PC в 50 тыс. до конца 1981 года. Но реальность оказалась во много раз лучше самых оптимистичных прогнозов – до конца года было продано свыше 250 тыс. IBM PC. Образно говоря, успев вскочить на подножку последнего вагона, IBM сумела практически с первой попытки вырваться в лидеры рынка персональных компьютеров, несмотря на использование в системе не самого лучшего процессора в отрасли и откровенно сырую и довольно примитивную операционную систему.

В то время почти все PC использовали один из двух процессоров – либо MOS 6502, либо Zilog Z80. Оба они появились в относительно небольших, начинающих компаниях, и каждый из них позаимствовал основной набор инструкций и большую часть схемы у другого, более дорогого CPU, производимого крупной компанией – у Motorola 6800 и Intel 8080, соответственно. (Если говорить об этической стороне дела, то оба процессора в основном разрабатывали инженеры, также работавшие и над созданием исходных процессоров, послуживших для них источником вдохновения).

К моменту начала работ над грядущим PC IBM определилась с разрядностью процессора – он точно не должен был быть восьмиразрядным, иначе корпорация рисковала оказаться всего лишь одной из многих, и очередной восьмиразрядный PC на широко распространённом процессоре точно не смог бы завоевать мир, а на меньшее IBM была не согласна – они были «царями горы», и затеряться своему PC среди очень похожих, принципиально различающихся между собой архитектур, да ещё, скорее всего, работающему на массовой, de-facto стандартной в индустрии ОС типа CP/M – просто не могли себе позволить<sup>1</sup>.

В итоге первый «настоящий» персональный компьютер IBM PC 5150 по архитектуре не имел ничего общего со своими предшественниками линейки 51XX – он был построен на базе процессора Intel 8088, «младшем брате» вышедшего годом ранее процессоре Intel 8086<sup>2</sup>, с урезанной до 8 бит шиной данных. Но, тем не менее эту эпоху принято называть x86, т.к. все последующие, программно-совместимые с i8086 процессоры Intel допентиумной эпохи в названии содержали именно цифры 86: 80186, 80286, 80386 и 80486.

Выбор шестнадцатиразрядных микропроцессоров в то время был не очень велик. Можно столкнуться с мнением, что выбор интеловского i8088 был едва ли не худшим из возможных альтернатив. Известно, что IBM рассматривала как минимум три варианта процессора: великолепную Motorola 68000, довольно неуклюжий Intel 8088<sup>3</sup> и откровенно неудачный TMS9900 от Texas Instruments, имевший всего 16-ти разрядное логическое адресное пространство, как и основные 8-разрядные процессоры того времени. Основным (по сути единственным) преимуществом i8088 перед TMS9900/9980 было 20-разрядное адрес-

---

<sup>1</sup> Формально IBM PC не был первым в мире шестнадцатиразрядным PC. Таковым считается PC от Texas Instruments 99/4, основанный на собственном 16-ти разрядном процессоре TI TMS9900.

<sup>2</sup> Процессор i8086 был выпущен 8 июня 1978 года (объявлен в апреле 1978), процессор i8088 – 1 июля 1979 года.

<sup>3</sup> В 1975 году Intel начала работу над 32-битным микропроцессором iAPX432, который в конце концов увидел свет только в 1982 году. Практически одновременно с этим началась работа над более простым 16-битным микропроцессором (8086), который должен был стать временным решением и существовать до момента выхода iAPX432 на рынок.

ное пространство – целый 1 Мегабайт против 64 Кбайт! На подходе был ещё 16-ти разрядный процессор от Zilog – Z8000, но к моменту начала работ над IBM PC он был ещё не готов.

А красивая, изящная, стройная Motorola 68000, заведомо превосходящая по характеристикам i8086/88, уступила при выборе по одной простой причине – да, i8088 был неидеальным, но, по крайней мере, он был готов к производству в нужных количествах, а 68К не был. Скрупулёзный процесс оценки качества IBM требовал, чтобы производитель предоставил тысячи образцов выпущенного продукта для каждой новой запчасти, чтобы IBM могла проводить на них интенсивные «живые» тесты. В IBM контролем качества занимались сотни инженеров, но на это уходит много времени. В первой половине 1978 года у Intel уже были изготовлены образцы i8088, а 68К не был полностью готов к выходу даже к концу 1978 года. К несчастью для Motorola<sup>1</sup>, группа IBM из Бока-Ратон хотела выводить на рынок новый IBM PC как можно скорее. А у Motorola на тот момент было только два полностью готовых 16-битных МП на выбор. В соревновании двух неидеальных чипов кристалл от Intel оказался менее неидеальным, чем чип от TI.

## Архитектура микропроцессора i8086/88

IBM выбрала для своего PC менее производительный процессор i8088 с восьмиразрядной шиной данных (через которую процессор общается с «внешним миром»), вместо более раннего i8086, по одной существенной причине – в конце 1970-хх на рынке существовало огромное количество восьмиразрядных периферийных контроллеров, разработанных для МП i8080/8085, но практически не было их шестнадцатиразрядных эквивалентов. Именно поэтому, выбрав стратегию массового использования сторонних компонент при разработке архитектуры, IBM заложилась на массово доступную периферию предыдущей 8-разрядной эпохи, идеально согласовывавшуюся с разрядностью внешней шины i8088. Внутренняя архитектура и система команд процессоров i8086 и i8088 были идентичными<sup>2</sup>, поэтому ниже будут приведены рисунки и схемы как на основе i8088, так и на i8086 – в этом смысле они эквивалентны.

МП был выполнен по nMOS 3 мкм технологии, и содержал 29000 транзисторов. Упакован процессор был во всё тот же стандартный корпус DIP-40:

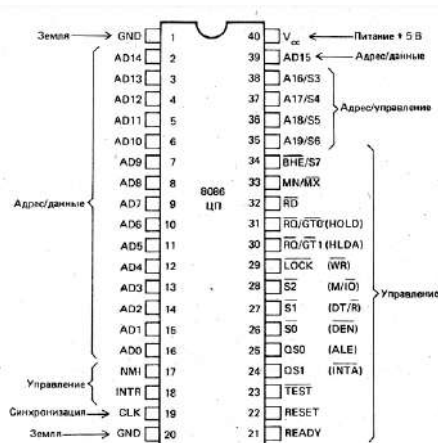


Рис.209 – Разводка контактов корпуса микропроцессора 8086

<sup>1</sup> Motorola 68000 стала основой для конкурента IBM PC – персонального компьютера Apple Macintosh.

<sup>2</sup> Кроме разницы в ширине внешней шины данных, было ещё одно небольшое различие в «железе» процессоров i8086 и i8088 – у первого из них глубина очереди команд составляла шесть байт, у второго – четыре. Тонким программированием на ассемблере можно было обнаружить эту разницу. Эпоха CPUID ещё не наступила.

В отличие от откровенной небрежности с разводкой контактов в процессоре i8080, в i8086 линии адреса и данных мультиплексированы. Было бы очень странно отдать в немультимплексированном варианте  $16 + 20 = 36$  линий из сорока доступных на адрес/данные. В i8088 мультиплексировались, соответственно, только восемь младших разрядов AD.

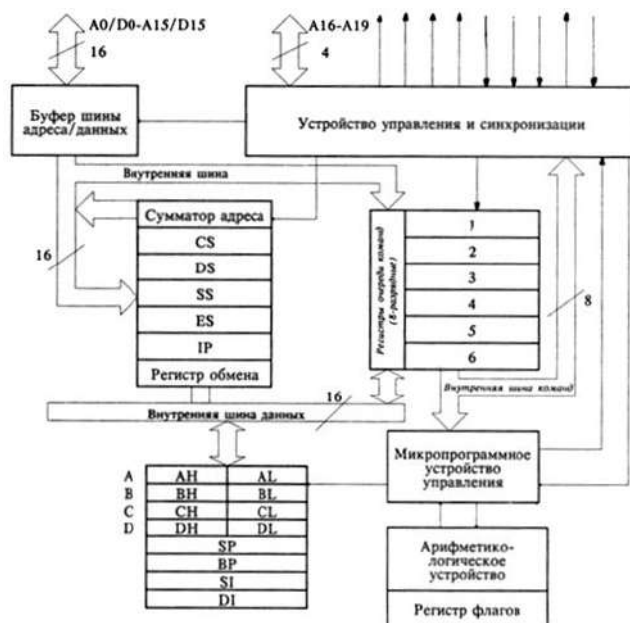


Рис.210 – Внутренняя структура микропроцессора 8086

Процессор i8086 представлял собой, по сути, глубокую модернизацию процессора Intel 8080/85, но полной бинарной совместимости между ними достигнуто не было, да и разработчики и не ставили перед собой такую цель.

Всего в процессоре i8086 имеется 14 16-разрядных регистров: 8 регистров общего назначения (AX, BX, CX, DX), 2 индексных регистра (SI, DI), 2 регистра-указателя (BP, SP), 4 сегментных регистра (CS, SS, DS, ES), программный счётчик или указатель команды (IP) и регистр флагов (FLAGS, включает в себя 9 флагов). При этом регистры данных (AX, BX, CX, DX) допускают адресацию не только целых регистров, но и отдельно их младшей (регистры AL, BL, CL, DL) и старшей половины (регистры AH, BH, CH, DH), что не только позволило использовать новое на тот момент 16-разрядное ПО, но и сохранило обратную совместимость с 8-битными программами (правда, их необходимо было, по крайней мере, перекомпилировать).

| Регистры общего назначения                           |    |                                   |
|------------------------------------------------------|----|-----------------------------------|
| AH                                                   | AL | AX (primary accumulator)          |
| BH                                                   | BL | BX (base, accumulator)            |
| CH                                                   | CL | CX (counter, accumulator)         |
| DH                                                   | DL | DX (accumulator, other functions) |
| Индексные регистры                                   |    |                                   |
| SI                                                   |    | Source Index                      |
| DI                                                   |    | Destination Index                 |
| Указательные регистры                                |    |                                   |
| BP                                                   |    | Base Pointer                      |
| SP                                                   |    | Stack Pointer                     |
| Регистр состояния                                    |    |                                   |
| 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0 (bit position) |    |                                   |
| - - - - O D I T S Z - A - P - C                      |    | Флаги                             |
| Сегментные регистры                                  |    |                                   |
| CS                                                   |    | Code Segment                      |
| DS                                                   |    | Data Segment                      |
| ES                                                   |    | Extra Segment                     |
| SS                                                   |    | Stack Segment                     |
| Указатель команды                                    |    |                                   |
| IP                                                   |    | Instruction Pointer               |

Рис.211 – Регистры процессора 8086/88

Поскольку регистры и АЛУ процессора 8086/88 – 16-битные, это накладывает ограничение на операции вычисления адреса и максимальный размер адресуемой памяти. Для прямого вычисления адресов, превышающих 16 бит, понадобились бы расширение разрядности регистров и АЛУ, а также потребовалась бы серьёзная переработка системы команд, что, в свою очередь, привело бы к увеличению числа транзисторов, усложнению и удорожанию процессора.

Тем не менее, снижающиеся цены на память делали процессор выгодным для тех применений, где требовалась обработка больших объёмов данных. Расширение адресного пространства было одним из требований к новому процессору. В результате была выбрана компромиссная схема: исполнительный блок (EU) процессора оставлен 16-разрядным, а расширение адресного пространства сделано в блоке интерфейса шины (BIU) процессора путём реализации сегментной адресации памяти, увеличивающей разрядность шины адреса до 20 бит.

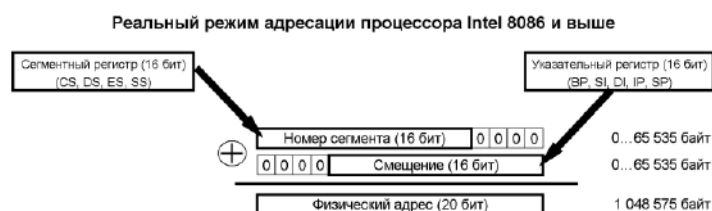
Сегментная адресация памяти использует тот факт, что обращения к памяти со стороны процессора легко можно разделить на обращения к коду программы, обрабатываемым данным и стеку. Обращения к разным типам содержимого памяти отображаются на независимые области памяти в расширенном адресном пространстве — *сегменты*. В процессоре 8086/88 для такого отображения адресов используются четыре 16-битных сегментных регистра:

- CS** (Code Segment) — сегмент кода
- DS** (Data Segment) — сегмент данных
- ES** (Extra Segment) — дополнительный сегмент
- SS** (Stack Segment) — сегмент стека

Каждый сегментный регистр определяет адрес начала сегмента в памяти, при этом сегменты могут совпадать или пересекаться. По умолчанию регистр CS используется при выборке инструкций, регистр SS при выполнении операций со стеком, регистры DS и ES при обращении к данным. В случае обращения к данным сегментный регистр по умолчанию может быть изменён путём добавления перед кодом инструкции специального префикса замены сегмента.

При этом, в пределах сегмента используются 16-битные исполнительные адреса, хранящиеся в счётчике команд, указателе стека, либо вычисляемые в соответствии с видом адресации, заданном в коде инструкции. Фактически, процессору всегда доступны четыре области памяти размером 64 Кбайт каждая. Если такой объём оказывается недостаточен, в программу приходится включать логику управления сегментными регистрами. Эта логика может существенно замедлять обращение к памяти, поэтому компиляторы с языков высокого уровня для платформы x86 позволяют указывать модель памяти в соответствии с требуемыми объёмами кода и данных.

Физический 20-битный адрес, позволяющий адресовать до  $2^{20} = 1$  Мбайт памяти, получается путём сложения исполнительного адреса и значения сегментного регистра, умноженного на 16. Из-за наличия умножения на 16 сегмент всегда начинается на границе блока в 16 байт, называемого *параграфом*. Это может приводить к потере некоторого количества памяти, если размер используемых данных сегмента не кратен параграфу. В частности, это стало одной из причин, по которой разработчики отказались от реализации 24-битного адреса в 8086, так как размер параграфа при этом возрастал до 256 байт.



**Рис.212 – Формирование физического адреса**

Возможное переполнение при вычислении физического адреса в процессоре i8086/88 игнорируется. Например, инструкция безусловного перехода по адресу 0x0010 при значении сегмента кода 0xFFFF выполнит переход по физическому адресу 0x0000<sup>1</sup>.

## Система команд i8086/88

В продолжении традиций своих процессоров предыдущего поколения, система команд i8086/88 получилась не очень стройной, довольно плохо организованной и весьма запутанной. Красота и логичность никогда не были коньком Intel, это просматривается практически во всех поколениях их процессоров.

Обозначения:  $n_T$  — число тактов, требуемое для выполнения команды;  $E$  — число тактов, затрачиваемое на вычисление исполнительного адреса ЕА операнда, находящегося в памяти;  $r$  (или  $r1, r2$ ) — регистр ЦП;  $seg$  — сегментный регистр ЦП;  $a$  (или  $A$ ) — аккумулятор  $AL$  или  $AX$ ;  $mem$  — ячейка памяти;  $data$  — данные, непосредственно представленные в команде ( $data L, data H$  — младший и старший байты данных);  $port$  — имя или номер порта ввода — вывода;  $disp$  — константа смещения ( $disp L, disp H$  — младший и старший байты константы смещения);  $label$  — метка (адрес метки или ее имя);  $name$  — имя подпрограммы (или ее начальный адрес);  $addr$  — адрес ( $addr L, addr H$  — младший и старший байты адреса);  $type$  — тип или уровень прерывания;  $mod$  — режим адресации;  $reg$  — код регистра;  $r/m$  — регистр/память (поле в постбайте команды);  $w = 0$  — размерность регистра (памяти или данных) 8 разрядов,  $w = 1$  — размерность регистра (памяти или данных) 16 разрядов;  $d = 0$  — пересылка из регистра,  $d = 1$  — пересылка в регистр;  $sw = 01$  — 16-разрядная константа,  $sw = 11$  — 8-разрядная константа;  $v = 0$  — счетчик=1,  $v = 1$  — счетчик=( $CX$ ).

| Группа команд    | № п.п. | Мнемоника     | $n_T$ | Байты формата команды |               | Описание команд    |
|------------------|--------|---------------|-------|-----------------------|---------------|--------------------|
|                  |        |               |       | нечетный              | четный        |                    |
| Пересылки данных | 1      | MOV r, r      | 2     | 1 0 0 0 1 0 d w       | mod reg r/m   | Pr←Pr              |
|                  | 2      | MOV r, mem    | 8+E   |                       |               | Pr←П               |
|                  | 3      | MOV mem, r    | 9+E   |                       |               | П←Pr               |
|                  | 4      | MOV mem, data | 10+E  | 1 1 0 0 0 1 1 w       | mod 0 0 0 r/m | П←Д                |
|                  | 5      | MOV r, data   | 4     | 1 0 1 1 w reg         | data H (w=1)  | Pr←Д               |
|                  | 6      | MOV a, mem    | 10    | 1 0 1 0 0 0 d w       | addr L        | A←П                |
|                  | 7      | MOV mem, a    | 10    | addr H                |               | П←A                |
|                  | 8      | MOV seg, r    | 2     | 1 0 0 0 1 1 d 0       | mod 0 seg r/m | Сегментный Pr←Pr   |
|                  | 9      | MOV seg, mem  | 8+E   |                       |               | Сегментный Pr←П    |
|                  | 10     | MOV r, seg    | 2     |                       |               | Pr←сегментный Pr   |
|                  | 11     | MOV mem, seg  | 9+E   |                       |               | П←сегментный Pr    |
|                  | 12     | PUSH r        | 10    | 1 1 1 1 1 1 1 1       | mod 1 1 0 r/m | Стек←Pr            |
|                  | 13     | PUSH mem      | 16+E  |                       |               | Стек←П             |
|                  | 14     | PUSH r        | 10    | 0 1 0 1 0 reg         |               | Стек←Pr            |
|                  | 15     | PUSH seg      | 10    | 0 0 0 seg 1 1 0       |               | Стек←сегментный Pr |
|                  | 16     | POP r         | 8     | 1 0 0 0 1 1 1 1       | mod 0 0 0 r/m | Pr←стек            |
|                  | 17     | POP mem       | 17+E  |                       |               | П←стек             |
|                  | 18     | POP r         | 8     | 0 1 0 1 1 reg         |               | Pr←стек            |
|                  | 19     | POP seg       | 8     | 0 0 0 seg 1 1 1       |               | Сегментный Pr←стек |
|                  | 20     | XCHG r, mem   | 17+E  | 1 0 0 0 0 1 1 w       | mod reg r/m   | Pr←П               |
|                  | 21     | XCHG r, r     | 4     | 1 0 0 1 0 reg         |               | Pr←Pr              |
|                  | 22     | XCHG AX, r    | 3     | 1 1 1 0 0 1 0 w       | port          | A←Pr               |
|                  | 23     | IN port       | 10    | 1 1 1 0 0 1 0 w       |               | A←порт             |
|                  | 24     | IN            | 8     | 1 1 1 0 1 1 0 w       |               | A←(DX)             |
|                  | 25     | OUT port      | 10    | 1 1 1 0 0 1 1 w       | port          | Порт←A             |
|                  | 26     | OUT           | 8     | 1 1 1 0 1 1 1 w       |               | (DX)←A             |
|                  | 27     | LEA r, mem    | 2+E   | 1 0 0 0 1 1 0 1       | mod reg r/m   | Pr←EA              |
|                  | 28     | LDS r, mem    | 16+E  | 1 1 0 0 0 1 0 1       | mod reg r/m   | Pr и DS←П          |
|                  | 29     | LES r, mem    | 16+E  | 1 1 0 0 0 1 0 0       | mod reg r/m   | Pr и ES←П          |
|                  | 30     | LAHF          | 4     | 1 0 0 1 1 1 1 1       |               | AH←FL              |
|                  | 31     | SAHF          | 4     | 1 0 0 1 1 1 1 0       |               | FL←AH              |
|                  | 32     | PUSHF         | 10    | 1 0 0 1 1 1 0 0       |               | Стек←F             |
|                  | 33     | POPF          | 8     | 1 0 0 1 1 1 0 1       |               | F←стек             |

**Рис.213 – Группа команд пересылки данных**

<sup>1</sup> В процессоре i80286 была допущена ошибка(?), позволявшая МП достигаться к памяти выше 1 Мбайта из реальной моды, не переводя процессор в защищенный режим. Более подробно об этом будет изложено дальше, в разделе, посвященном машинам на базе i80286 (IBM PC/AT).

| Группа команд           | № п.п. | Мнемонкод     | P <sub>r</sub> | Байты формата команды    |               | Описание команды                                                                              |
|-------------------------|--------|---------------|----------------|--------------------------|---------------|-----------------------------------------------------------------------------------------------|
|                         |        |               |                | нечетный                 | четный        |                                                                                               |
| Арифметические действия | 34     | ADD r1, r2    | 3              | 000000 d w               | mod reg r/m   | Pr←Pr+Pr                                                                                      |
|                         | 35     | ADD r, mem    | 9+E            |                          |               | Pr←Pr+П                                                                                       |
|                         | 36     | ADD mem, r    | 16+E           |                          |               | П←Pr+Pr                                                                                       |
|                         | 37     | ADD r, data   | 4              | 100000 s w               | mod 000 r/m   | Pr←Pr+Д                                                                                       |
|                         | 38     | ADD mem, data | 17+E           | data L                   | data H(sw=01) | П←П+Д                                                                                         |
|                         | 39     | ADD a, data   | 4              | 0000010 w                | data L        | A←A+Д                                                                                         |
|                         | 40     | ADC r1, r2    | 3              | 000100 d w               | mod reg r/m   | Pr←Pr+Pr+CF                                                                                   |
|                         | 41     | ADC r, mem    | 9+E            |                          |               | Pr←Pr+П+CF                                                                                    |
|                         | 42     | ADC mem, r    | 16+E           |                          |               | П←П+Pr+CF                                                                                     |
|                         | 43     | ADC r, data   | 4              | 100000 s w               | mod 010 r/m   | Pr←Pr+Д+CF                                                                                    |
|                         | 44     | ADC mem, data | 17+E           | data L                   | data H(sw=01) | П←П+Д+CF                                                                                      |
|                         | 45     | ADC a, data   | 4              | 0001010 w                | data L        | A←A+Д+CF                                                                                      |
|                         | 46     | INC r         | 2              | 1111111 w                | mod 000 r/m   | Pr←Pr+1                                                                                       |
|                         | 47     | INC mem       | 15+E           |                          |               | П←П+1                                                                                         |
|                         | 48     | INC r         | 2              | 010000 reg               |               | Pr←Pr(8)+1                                                                                    |
|                         | 49     | SUB r1, r2    | 3              | 001010 d w               | mod reg r/m   | Pr←Pr-Pr                                                                                      |
|                         | 50     | SUB r, mem    | 9+E            |                          |               | Pr←Pr-П                                                                                       |
|                         | 51     | SUB mem, r    | 16+E           |                          |               | П←П-Pr                                                                                        |
|                         | 52     | SUB r, data   | 4              | 100000 s w               | mod 101 r/m   | Pr←Pr-Д                                                                                       |
|                         | 53     | SUB mem, data | 17+E           | data L                   | data H(sw=01) | П←П-Д                                                                                         |
|                         | 54     | SUB a, data   | 4              | 0010110 w                | data L        | A←A-Д                                                                                         |
|                         | 55     | SBB r1, r2    | 3              | 000110 d w               | mod reg r/m   | Pr←Pr-Pr-CF                                                                                   |
|                         | 56     | SBB r, mem    | 9+E            |                          |               | Pr←Pr-П-CF                                                                                    |
|                         | 57     | SBB mem, r    | 16+E           |                          |               | П←П-Pr-CF                                                                                     |
|                         | 58     | SBB r, data   | 4              | 100000 s w               | mod 011 r/m   | Pr←Pr-Д-CF                                                                                    |
|                         | 59     | SBB mem, data | 17+E           | data L                   | data H(sw=01) | П←П-Д-CF                                                                                      |
|                         | 60     | SBB a, data   | 4              | 0001110 w                | data L        | A←A-Д                                                                                         |
|                         | 61     | DEC r         | 2              | 1111111 w                | mod 001 r/m   | Pr←Pr-1                                                                                       |
|                         | 62     | DEC mem       | 155+E          |                          |               | П←П-1                                                                                         |
|                         | 63     | DEC r         | 2              | 01001 reg                |               | Pr←Pr(8)-1                                                                                    |
|                         | 64     | NEG r         | 3              | 1111011 w                | mod 011 r/m   | Pr←0-Pr                                                                                       |
|                         | 65     | NEG mem       | 16+E           |                          |               | П←0-П                                                                                         |
|                         | 66     | CMP r1, r2    | 3              | 001110 d w               | mod reg r/m   | Pr←Pr                                                                                         |
|                         | 67     | CMP r, mem    | 9+E            |                          |               | Pr←П                                                                                          |
|                         | 68     | CMP mem, r    | 16+E           |                          |               | П←Pr                                                                                          |
|                         | 69     | CMP r, data   | 4              | 100000 s w               | mod 111 r/m   | Pr←Д                                                                                          |
|                         | 70     | CMP mem, data | 17+E           | data L                   | data H(w=1)   | П←Д                                                                                           |
|                         | 71     | CMP a, data   | 4              | 0011110 w                | data L        | A←Д                                                                                           |
|                         | 72     | MUL src       | 71+E<br>124+E  | data H(w=1)<br>1111011 w | mod 100 r/m   | AX←AL источник (при w=0)<br>DX, AX←AX источник (при w=1)                                      |
|                         | 73     | IMUL src      | 90+E<br>144+E  | 1111011 w                | mod 100 r/m   | AX←AL источник (при w=0)<br>DX, AX←AX источник (при w=1)<br>со знаком                         |
|                         | 74     | DIV src       | 90+E<br>155+E  | 1111011 w                | mod 110 r/m   | AL←AX/источник, AH остаток (при w=0)<br>AX←D, AX/источник, DX остаток (при w=1)               |
|                         | 75     | IDIV src      | 112+E<br>177+E | 1111011 w                | mod 111 r/m   | AL←AX/источник, AH остаток (при w=0)<br>AX←DX, AX/источник, DX остаток (при w=1)<br>со знаком |
|                         | 76     | DAA           | 4              | 00100111                 |               | AL←скорректированное (сложение, 2—10)                                                         |
|                         | 77     | DAS           | 4              | 00101111                 |               | AL←скорректированное (вычитание, 2—10)                                                        |
|                         | 78     | AAA           | 4              | 00110111                 |               | AL←скорректированное (сложение, ASCII)                                                        |
|                         | 79     | AAS           | 4              | 00111111                 |               | AL←скорректированное (вычитание, ASCII)                                                       |
|                         | 80     | AAM           | 83             | 11010100                 | 00001010      | AX←скорректированное (AX умножение)                                                           |
|                         | 81     | AAD           | 60             | 11010101                 | 00001010      | AX←скорректированное (AX деление)                                                             |
|                         | 82     | CBW           | 5              | 10011000                 |               | AH←знак AL                                                                                    |
|                         | 83     | CWD           | 5              | 10011001                 |               | DX←знак AX                                                                                    |

Рис.214 – Группа арифметических команд

| Группа команд | № п.п. | Мнемонкод   | P <sub>r</sub> | Байты формата команды |             | Описание команды                 |
|---------------|--------|-------------|----------------|-----------------------|-------------|----------------------------------|
|               |        |             |                | нечетный              | четный      |                                  |
| Сдвиги        | 84     | SHL/SAL r   | 2              | 110100 v w            | mod *00 r/m | Логический влево (Pr)            |
|               | 85     | SHL/SAL mem | 15+E           |                       |             | Логический влево (П)             |
|               | 86     | SHR r       | 2              | 110100 v w            | mod 101 r/m | Логический вправо (Pr)           |
|               | 87     | SHR mem     | 15+E           |                       |             | Логический вправо (П)            |
|               | 88     | SAR r       | 2              | 110100 v w            | mod 111 r/m | Арифметический вправо (Pr)       |
|               | 89     | SAR mem     | 15+E           |                       |             | Арифметический вправо (П)        |
|               | 90     | ROL r       | 8              | 110100 v w            | mod 000 r/m | Циклический влево (Pr)           |
|               | 91     | ROL mem     | 20+E           |                       |             | Циклический влево (П)            |
|               | 92     | ROR r       | 8              | 110100 v w            | mod 001 r/m | Циклический вправо (Pr)          |
|               | 93     | ROR mem     | 20+E           |                       |             | Циклический вправо (П)           |
|               | 94     | RCL r       | 8              | 110100 v w            | mod 010 r/m | Циклический через CF влево (Pr)  |
|               | 95     | RCL mem     | 20+E           |                       |             | Циклический через CF влево (П)   |
|               | 96     | RCR r       | 8              | 110100 v w            | mod 011 r/m | Циклический через CF вправо (Pr) |
|               | 97     | RCR mem     | 20+E           |                       |             | Циклический через CF вправо (П)  |

Рис.215 – Группа команд сдвигов

| Группа команд       | № п.п. | Мнемокод      | n <sub>r</sub> | Байты формата команды |             | Описание команды                       |
|---------------------|--------|---------------|----------------|-----------------------|-------------|----------------------------------------|
|                     |        |               |                | нечетный              | четный      |                                        |
| Логические действия | 98     | AND r1, r2    | 3              | 001000 d w            | mod reg r/m | Pr←Pr∧Pr                               |
|                     | 99     | AND r, mem    | 9+E            |                       |             | Pr←Pr∧П                                |
|                     | 100    | AND mem, r    | 16+E           |                       |             | П←П∧Pr                                 |
|                     | 101    | AND r, data   | 4              | 1000000 w             | mod 100 r/m | Pr←Pr∧D                                |
|                     | 102    | AND mem, data | 17+E           | data L                | data H(w=1) | П←П∧D                                  |
|                     | 103    | AND a, data   | 4              | 0010010 w             | data L      | A←A∧D                                  |
|                     | 104    | OR r1, r2     | 3              | 000010 d w            | mod reg r/m | Pr←Pr∨Pr                               |
|                     | 105    | OR r, mem     | 9+E            |                       |             | Pr←Pr∨П                                |
|                     | 106    | OR mem, r     | 16+E           |                       |             | П←П∨Pr                                 |
|                     | 107    | OR r, data    | 4              | 1000000 w             | mod 001 r/m | Pr←Pr∨D                                |
|                     | 108    | OR mem, data  | 17+E           | data L                | data H(w=1) | П←П∨D                                  |
|                     | 109    | OR a, data    | 4              | 0000110 w             | data L      | A←A∨D                                  |
|                     | 110    | XOR r1, r2    | 3              | 001100 d w            | mod reg r/m | Pr←Pr⊕Pr                               |
|                     | 111    | XOR r, mem    | 9+E            |                       |             | Pr←Pr⊕П                                |
|                     | 112    | XOR mem, r    | 16+E           |                       |             | П←П⊕Pr                                 |
|                     | 113    | XOR r, data   | 4              | 1000000 w             | mod 110 r/m | Pr←Pr⊕D                                |
|                     | 114    | XOR mem, data | 17+E           | data L                | data H(w=1) | П←П⊕D                                  |
|                     | 115    | XOR a, data   | 4              | 0011010 w             | data L      | A←A⊕D                                  |
|                     | 116    | TEST r1, r2   | 3              | 1000010 w             | mod reg r/m | Pr∧Pr Результат определяется по флагам |
|                     | 117    | TEST r, mem   | 9+E            |                       |             | Pr∧П                                   |
|                     | 118    | TEST r, data  | 4              | 1111011 w             | mod 000 r/m | Pr∧D                                   |
|                     | 119    | TEST mem data | 10+E           | data L                | data H(w=1) | П∧D                                    |
|                     | 120    | TEST a, data  | 4              | 1010100 w             | data L      | A∧D                                    |
|                     | 121    | NOT r         | 3              | 1111011 w             | mod 010 r/m | Pr←¬Pr                                 |
|                     | 122    | NOT mem       | 16+E           |                       |             | П←¬П                                   |

Рис.216 – Группа логических операций

| Группа команд   | № п.п. | Мнемокод      | n <sub>r</sub> | Байты формата команды |        | Описание команды                     |
|-----------------|--------|---------------|----------------|-----------------------|--------|--------------------------------------|
|                 |        |               |                | нечетный              | четный |                                      |
| Обработка строк | 123    | MOVS          | 17             | 10100101              |        | П[DI]←П[SI]                          |
|                 | 124    | CMPS          | 22             | 1010011 w             |        | П[SI]←П[DI] (см. флаги)              |
|                 | 125    | SCAS          | 15             | 1010111 w             |        | A←П[DI]                              |
|                 | 126    | LODS          | 12             | 1010110 w             |        | A←П[SI]                              |
|                 | 127    | STOS          | 10             | 1010101 w             |        | П[DI]←A                              |
|                 | 128    | REPNE/REPZ    | 6              | 1111001 z             |        | CX←CX−1, повторить, пока CX≠0 и ZF=0 |
|                 | 129    | REP/REPE/PERZ | 6              |                       |        | Повторить, пока CX≠0 и ZF=1          |

Рис.217 – Группа команд работы со строками

| Группа команд        | № п.п. | Мнемокод  | n <sub>r</sub> | Байты формата команды |             | Описание команды                                |
|----------------------|--------|-----------|----------------|-----------------------|-------------|-------------------------------------------------|
|                      |        |           |                | нечетный              | четный      |                                                 |
| Безусловные переходы | 130    | JMP label | 7              | 11101001              | disp L      | IP←IP+смещение                                  |
|                      | 131    | JMP label | 2              | 11101011              | disp L      | IP←IP+смещение, расширенное со знаком до 16 бит |
|                      | 132    | JMP label | 7+E            | 11111111              | mod 100 r/m | IP←(EA)                                         |
|                      | 133    | JMP label | 7              | 11101010              | addr L      | IP←адрес перехода                               |
|                      | 134    | JMP label | 16+E           | 11111111              | seg L       | CS←адрес сегмента                               |
|                      | 135    | CALL name | 11             | 11101000              | disp L      | IP←(EA); CS (EA+2)                              |
|                      | 136    | CALL name | 13+E           | 11111111              | disp L      | SP←SP−2; SP−1; SP←←IP                           |
|                      | 137    | CALL name | 20             | 10011010              | addr L      | IP←IP+смещение                                  |
|                      |        |           |                | addr H                | seg L       | SP←SP−2; SP−1; SP←CS                            |
|                      |        |           |                | seg H                 |             | SP←SP−2; SP−1, SP←IP                            |
|                      | 138    | RET       | 8              | 11000011              | disp L      | IP←адрес перехода; CS←адрес сегмента            |
|                      | 139    | RET       | 18             | 11000010              | data L      | IP←(SP+1, SP); SP←←SP+2                         |
|                      | 140    | RET       | 18             | 11001011              | data L      | IP←(SP+1, SP); SP←←SP+D                         |
|                      | 141    | RET       | 18             | 111001010             | data L      | IP←(SP+1, SP); SP←←SP+2                         |

Рис.218 – Группа команд безусловных переходов



| Группа команд     | № п.п. | Мнемокод            | Лг  | Байты формата команды |        | Описание команды          |
|-------------------|--------|---------------------|-----|-----------------------|--------|---------------------------|
|                   |        |                     |     | нечетный              | четный |                           |
| Условные переходы | 142    | JZ/JE label         | 8/4 | 11000011              | d1sp   | По равенству              |
|                   | 143    | JNZ/JNE label       | 8/4 | 01110101              | d1sp   | По неравенству            |
|                   | 144    | JL/JNGE label       | 8/4 | 01111100              | d1sp   | По меньше (со знаком)     |
|                   | 145    | JNL/JGE label       | 8/4 | 01111101              | d1sp   | По не меньше (со знаком)  |
|                   | 146    | JLE/JNG label       | 8/4 | 01111110              | d1sp   | По не больше (со знаком)  |
|                   | 147    | JNLE/JG label       | 8/4 | 01111111              | d1sp   | По больше (со знаком)     |
|                   | 148    | JB/JNAE label       | 8/4 | 01110010              | d1sp   | По меньше (без знака)     |
|                   | 149    | JNB/JNA label       | 8/4 | 01110011              | d1sp   | По не меньше (без знака)  |
|                   | 150    | JBE/JNA label       | 8/4 | 01110110              | disp   | По не больше (без знака)  |
|                   | 151    | JNBE/JA label       | 8/4 | 01110111              | disp   | По больше (без знака)     |
|                   | 152    | JP/JPE label        | 8/4 | 01111010              | d1sp   | По четному паритету       |
|                   | 153    | JNP/JPO label       | 8/4 | 01111011              | d1sp   | По нечетному паритету     |
|                   | 154    | JO label            | 8/4 | 01110000              | d1sp   | По переполнению           |
|                   | 155    | JNO label           | 8/4 | 01110001              | d1sp   | По непереполнению         |
|                   | 156    | JS label            | 8/4 | 01111000              | d1sp   | По минусу                 |
|                   | 157    | JNS label           | 8/4 | 01111001              | d1sp   | По плюсу                  |
|                   | 158    | LOOP label          | 9   | 11100010              | d1sp   | IP←IP+смещение, если CX≠0 |
|                   | 159    | LOOPNZ/LOOPNE label | 11  | 1110000z              |        | IP←IP+смещение, если Z=0  |
|                   | 160    | LOOPZ/LOOPE label   | 11  | 11100011              | disp   | IP←IP+смещение, если Z=1  |
|                   | 161    | JCXZ label          | 9   |                       | disp   | IP←IP+смещение, если CX=0 |

Рис.219 – Группа команд условных переходов

Ниже, в таблице на Рис.220, приведены флаги процессора, с описанием команд, влияющих на них.

| Группы команд                                                                                                      | Команды           | Флаги состояния  |    |    |    |    |    |
|--------------------------------------------------------------------------------------------------------------------|-------------------|------------------|----|----|----|----|----|
|                                                                                                                    |                   | OF               | CF | AF | SF | ZF | PF |
| Сложение и вычитание                                                                                               | ADD ADC SUB SBC   | +                | +  | +  | +  | +  | +  |
|                                                                                                                    | CMP NEG CMPS SCAS | +                | +  | +  | +  | +  | +  |
|                                                                                                                    | INC DEC           | +                | —  | +  | +  | +  | +  |
| Умножение и деление<br>Десятичная коррекция                                                                        | MUL IMUL          | +                | +  | ?  | ?  | ?  | ?  |
|                                                                                                                    | DIV IDIV          | ?                | ?  | ?  | ?  | ?  | ?  |
|                                                                                                                    | DAA DAS           | ?                | +  | +  | +  | +  | +  |
|                                                                                                                    | AAA AAS           | ?                | +  | +  | ?  | ?  | ?  |
|                                                                                                                    | AAM AAD           | ?                | ?  | ?  | +  | +  | +  |
| Логические операции                                                                                                | AND OR XOR TEST   | 0                | 0  | ?  | +  | +  | +  |
| Сдвиги:<br>одиночный многобайтный<br>одиночный многобайтный<br>Восстановление флагов<br>Управление флагом переноса | SHL SHR           | +                | +  | ?  | +  | +  | +  |
|                                                                                                                    | SHL SHR           | ?                | +  | ?  | +  | +  | +  |
|                                                                                                                    | SAR               | 0                | +  | ?  | +  | +  | +  |
|                                                                                                                    | ROL ROR RCL RCR   | +                | +  | —  | —  | —  | —  |
|                                                                                                                    | ROL ROR RCL RCR   | ?                | +  | —  | —  | —  | —  |
|                                                                                                                    | POPF IRET         | +                | +  | +  | +  | +  | +  |
|                                                                                                                    | SAHF              | —                | +  | +  | +  | +  | +  |
|                                                                                                                    | STC               | —                | 1  | —  | —  | —  | —  |
|                                                                                                                    | CLC               | —                | 0  | —  | —  | —  | —  |
|                                                                                                                    | CMC               | —                | *  | —  | —  | —  | —  |
| Группы команд                                                                                                      | Команды           | Флаги управления |    |    |    |    |    |
|                                                                                                                    |                   | DF               | IF | TF |    |    |    |
| Восстановление флагов                                                                                              | POPF IRET         | *                | +  | +  |    |    |    |
| Прерывание<br>Управление флагами                                                                                   | INT INTO          | —                | 0  | 0  |    |    |    |
|                                                                                                                    | STD CLD           | 1<br>0           | —  | —  |    |    |    |
| Управление флагами                                                                                                 | STI               | —                | 1  | —  |    |    |    |
|                                                                                                                    | CLI               | —                | 0  | —  |    |    |    |

Примечание. + — влияет на флаг, 0 — сбрасывает в «0», 1 — устанавливает в «1», ? — не определено, \* — инвертирует, — — не влияет.

Рис.220 – Флаги процессора

Как нетрудно убедиться, в системе команд i8086 не было даже намёка на ортогональность и хоть на какую-то упорядоченность в кодах операций.

## Подключение периферийных устройств, шина

Как уже было отмечено выше, в отличие от многих существовавших тогда на рынке персональных машин, в архитектуре IBM PC инженеры IBM использовали исключительно широкодоступные элементы<sup>1</sup>, ни одного специализированного кристалла под эту машину исходно разработано не было. IBM PC состоял из горизонтального корпуса с размещённой в нём основной (материнской) платой (Рис.221) с приблизительно 45 микросхемами малой и средней степени интеграции, 10 СБИС — микропроцессором i8088, опциональным арифметическим сопроцессором i8087, контроллером прямого доступа к памяти i8237, контроллером прерываний i8259A, таймером i8253, контроллером параллельного порта i8255A<sup>2</sup>, микросхемой постоянного запоминающего устройства с BIOS, четырьмя такими же микросхемами с интерпретатором BASIC, и от 9 до 36 микросхем динамического ОЗУ 4116. В правом заднем углу корпуса находился блок питания с импульсным преобразователем, а в правой передней части было отведено место для размещения одного или двух дисководов на гибких магнитных дисках форм-фактора 5 1/4 дюйма.

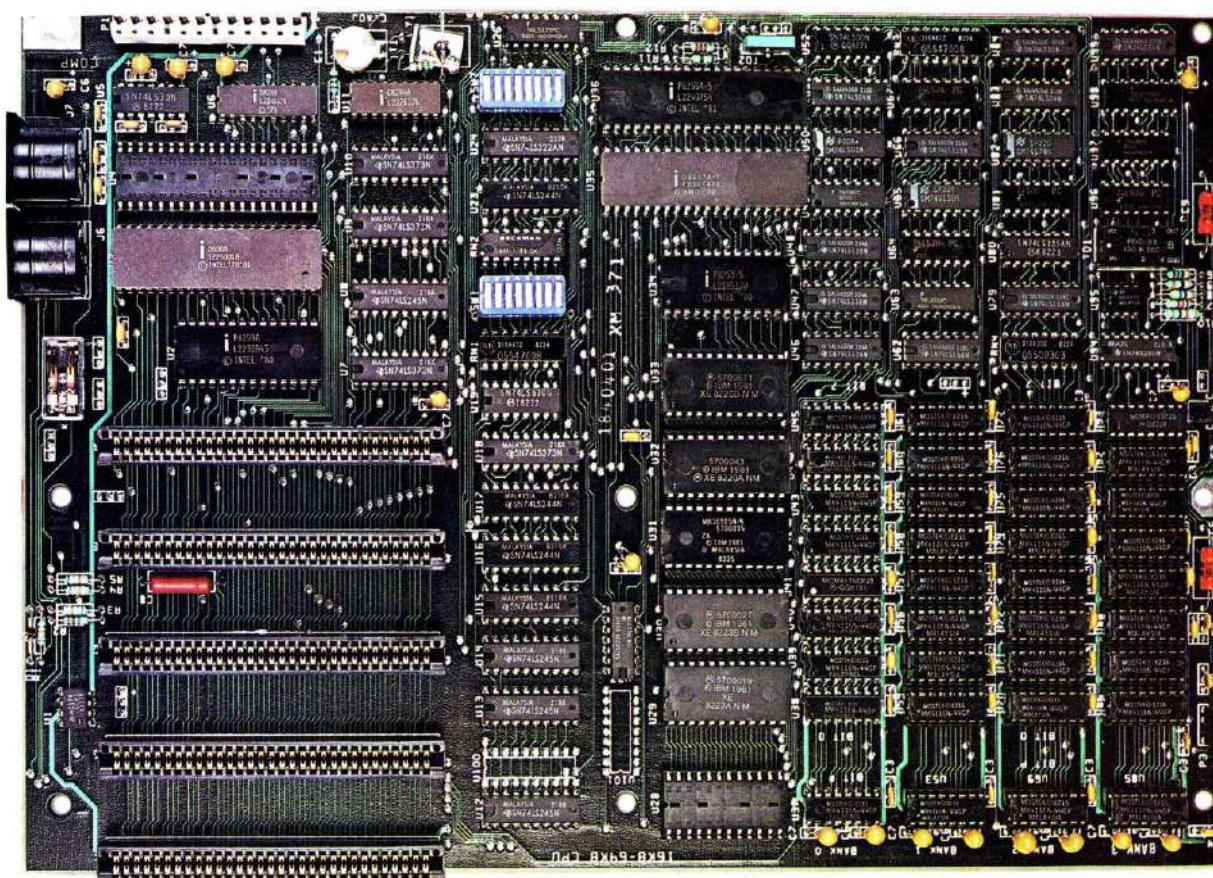


Рис.221 – Материнская плата первой IBM PC

Материнская плата модели 5150 имела 5 гнезд расширения, обычно одно из них было занято видеоадаптером и ещё одно — контроллером гибких дисков. Для подключения принтера нужно было приобрести адаптер параллельного порта, а для подключения модема — адаптер последовательного порта. Предлагались и другие платы, в частности, для расширения оперативной памяти сверх 64 килобайт на материнской плате.

<sup>1</sup> К примеру, компания Commodore приобрела MOS Technology, автора знаменитого МП 6502, которая производила специализированный чип для одноименного PC Commodore, что затрудняло создание его клонов.

<sup>2</sup> Все, кроме UART, хорошо знакомые 8-разрядные программируемые контроллеры из предыдущей эпохи i8080 — часто продвинутые модификации исходных оригинальных устройств, например, i8259A вместо исходного i8259.

Для гнезд (слотов) расширения IBM выбрала ту же шину, что и в своей предыдущей модели System/23 Datamaster, построенной на i8085, с теми же разъёмами. Шина получила название ISA (от англ. *Industry Standard Architecture*, ISA bus, произносится как ай-эс-эй), первая версия была 8-разрядной и была выполнена в виде 62-контактного разъёма, работала на частоте около 8МГц. Линии адреса (20 контактов) и данных (8 контактов) были разделены. В 1984 году, с появлением процессора i80286, шина ISA была доработана – в разъёмы было добавлено 36 контактов (всего их стало 98), была удвоена разрядность передаваемых данных – до 16 бит, также было добавлено 4 адресных линии, что дало возможность адресовывать до  $2^{24} = 16$  Мбайт физического ОЗУ.

ISA образца 1984 года была обратно совместима с платами расширения предыдущего поколения – логически, электрически и механически. Можно было вставлять 16-разрядный адаптер в 8-разрядный ISA разъём и наоборот – 8-разрядный адаптер в 16-разрядный слот. И в том, и в другом случае шина данных работала с разрядностью 8 разрядов, а адресная была ограничена 20-ю разрядами.

Так как использованные в IBM PC контроллеры прерывания и прямого доступа реализовывали «географическую» адресацию, по сути преобразовывая номер входа запроса в адрес (номер вектора) прерывания, без возможности организации «монтажного ИЛИ», то количество источников прерывания и прямого доступа прямо ограничивалось количеством входов в соответствующих контроллерах – 8 для прерываний (i8259A), 4 для DMA (i8237). В PC с максимальной конфигурацией<sup>1</sup> под системные нужды было занято до пяти входов прерывания, оставляя для пяти слотов расширения максимально всего лишь до трёх входов запроса прерывания. Та же проблема – малое количество входов запроса прямого доступа. Например, в IBM PC/XT (контроллер DMA i8237A) в системе могло быть занято до трёх линий (каналов) запроса прямого доступа из доступных четырёх:

| Номер канала | Для чего применяется                                                                           |
|--------------|------------------------------------------------------------------------------------------------|
| 0            | Обновление содержимого динамической памяти компьютера.<br>Этот канал имеет наивысший приоритет |
| 1            | Не используется                                                                                |
| 2            | Адаптер накопителя на гибком магнитном диске НГМД                                              |
| 3            | Адаптер накопителя на магнитном диске НМД.<br>Этот канал имеет низший приоритет                |

Рис.222 – Каналы DMA в IBM PC/XT

Для IBM PC-совместимых компьютеров 1 Мбайт был пределом памяти, который мог адресовать процессор i8086/88 ( $2^{20}$ ). Первый мегабайт был разделен на 16 областей по 64 Кбайт. Первые десять областей ( $64 \times 10 = 640$  Кбайт) были отведены для операционной системы и пользовательских программ (при этом сама операционная система могла обращаться ко всему мегабайту памяти) и назывались *основная память*<sup>2</sup>. Оставшиеся 6 областей объёмом 384 Кбайт (Upper Memory Area, UMA, верхняя память) резервировались для системных нужд и дополнительных устройств.

<sup>1</sup> Даже в максимальной конфигурации в первом IBM PC не была предусмотрена опция установки жёсткого диска, только один или два флоппи-дисководы. НЖМД появились только в следующей итерации IBM PC/XT.

<sup>2</sup> **Основная область памяти (Основная память, Conventional memory)** занимает первые 640 Кбайт оперативной памяти в IBM PC-совместимых компьютерах. В эту область загружается таблица векторов прерываний (занимает 1 Кбайт), некоторые данные из BIOS (например, буфер клавиатуры), различные 16-битные программы DOS. Для них 640 Кбайт являются барьером. Из каких соображений инженерами IBM была выбрана именно эта цифра для границы основной памяти – не очень понятно. Единственное, что можно отметить по этому поводу: 640 Кбайт – это сумма двух степеней двойки:  $2^{19} + 2^{17}$ .

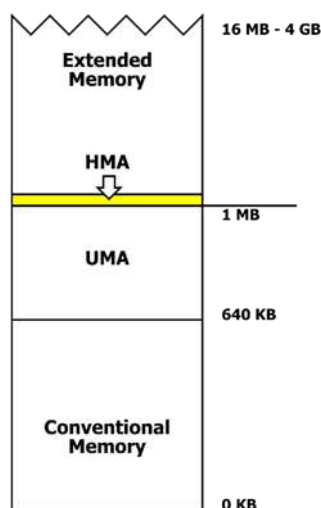


Рис.223 – Области памяти IBM PC (/AT)

В первых компьютерах здесь размещались BIOS (в самых старших адресах, 64 Кбайт непосредственно ниже границы первого мегабайта) и его расширения, память видеоадаптера и драйверы устройств. В существующих реализациях IBM PC часть верхней памяти использовалась для CGA/EGA<sup>1</sup>, другая — оставалась неиспользованной. В начале 1980-х годов память в 640 Кбайт была вполне достаточна для нужд обычного пользователя<sup>2</sup>, однако потребность в ОЗУ новых приложений росла быстрее, что привело к необходимости использовать резервные области верхней памяти различными способами, обходящими запрет на доступ прикладных программ к верхней памяти. В результате это привело к тому, что в 1 Мбайт доступной памяти, в области между 640 Кбайт и 1 Мбайт, появились зарезервированные «дыры», которые использовались различными аппаратными устройствами. Устранение подобных «дыр» было технически труднореализуемо и не поддерживалось бы существующими версиями DOS и прикладных программ. Позднее был разработан универсальный метод, позволяющий обращаться к участкам верхней памяти (Upper memory blocks, UMB).

Одним из самых мощных стимулов к поиску дополнительных ресурсов памяти, были программы определённых категорий – СУБД, электронные таблицы, САПР, игры и т.п., имевшие относительно простые алгоритмы обработки и, соответственно, сравнительно компактный код, но обрабатывавшие при этом большие объёмы данных. Для решения этой проблемы одним из вариантов была предложенная в 1984 спецификация *расширенной* памяти — аппаратно-программной системы, предоставляющей доступ DOS-приложениям к памяти, недоступной через адресное пространство основной памяти. *Расширенная память* адресуется странично через «окно», находящееся в верхней зарезервированной области памяти (UMA, в тех самых «дырах» в области между 640 Кбайт и 1 Мбайт). Спецификация расширенной памяти (Expanded Memory Specification, EMS) была разработана в 1984 году совместно Lotus Software, Intel и Microsoft, поэтому часто можно было встретить сокращение LIM EMS. С конца 1980-х до середины 1990-х EMS активно использовалась в играх и коммерческих приложениях, однако с приходом спецификаций дополнительной памяти (XMS) стала использоваться значительно реже.

Для обеспечения обратной совместимости с существующими приложениями ограничение в 640 Кбайт сохранилось в последующих реализациях стандарта IBM PC, даже после того, как архитектура Intel 8086/8088 была заменена на более современный Intel 80286 (IBM PC/AT), позволявший адресовать уже 16 Мбайт (2<sup>24</sup>) памяти в защищённом режиме.

<sup>1</sup> CGA/EGA – стандарты ранних графических карт для IBM PC.

<sup>2</sup> По легенде именно эта цифра была в своё время озвучена в известной цитате от Билла Гейтса: «640K ought to be enough for anybody».



Данное ограничение было вызвано тем, что процессор i80286 мог работать как в новом защищённом, так и в старом реальном режиме, обеспечивающим совместимость со старой архитектурой: в реальном режиме невозможно адресовать память с разрядностью выше 20 бит (или  $2^{20} = 1$  Мбайт). Последующие процессоры, совместимые с архитектурой IBM PC, поддерживали этот способ распределения памяти и в дальнейшем. Даже на современных компьютерах до сих пор существует зарезервированный диапазон памяти между 640 и 1024 Кбайтами памяти, хотя для большинства современных операционных систем этот диапазон фактически незаметен, так как в них доступ к ОЗУ реализуется через механизм виртуальной памяти, который скрывает распределение физической памяти, обеспечивая независимость доступа к ОЗУ от её распределения в реальном адресном пространстве.

Также, как и в процессорах предшественниках – i8080/85 – в МП i8086/88 было реализовано отдельное 16-разрядное адресное пространство ввода-вывода (I/O space). И вот в этой части инженеры IBM при разработке архитектуры PC приняли ещё одно довольно странное решение – в IBM PC было задействовано только 10 младших адресных разрядов пространства I/O из 16-ти, что позволяло адресовать до  $2^{10} = 1024$  отдельных регистров устройств ввода-вывода. Странность (и снова напрашивается определение небрежность) заключалась в том, что старшие 6 разрядов вообще не дешифровались, т.е. все регистры присутствовали в адресном пространстве I/O в  $2^6 = 64$  экземплярах. Экономия на схемах дешифрации? Цена вопроса – одна шестивходовая схема И. Интересно, что в следующем поколении PC/XT адресное пространство I/O было расширено вдвое, до 11 разрядов, а в PC/AT – до 12 и даже до 13 разрядов. Но самые старшие разряды адреса в пространстве ввода-вывода и в более новых PC так и не дешифровались.

## Второе поколение IBM PC/XT

Развивая довольно неожиданный, в первую очередь, для себя, успех своего первого IBM PC 5150, 8 марта 1983 года IBM представляет второе поколение IBM PC/XT или IBM 5160<sup>1</sup> (сокращение от eXtended Technology). Процессор (i8088) и основные элементы архитектуры машины остались неизменными, по сравнению с оригинальным IBM PC. В отличие от оригинального IBM PC PC/XT включает установленный в системный блок MFM<sup>2</sup>-жесткий диск с интерфейсом ST-412, объёмом 10 Мбайт (в более поздних модификациях — 20 Мбайт), ОЗУ ёмкостью 128 (базовая модификация) или 256 Кбайт с недокументированной возможностью последующего расширения до 640 Кбайт, путём замены микросхем памяти на материнской плате или расширения при помощи карт памяти, устанавливаемых в один из восьми (вместо пяти у модели 5150) разъемов шины ISA. Дисководы для гибких дисков стали двухсторонними, и их ёмкость удвоилась по сравнению с предыдущей моделью. К IBM PC/XT можно было добавить второй дисковод 5¼" и жесткий диск. Базовая конфигурация оснащалась монохромным (чёрно-белый) MDA или CGA видеоадаптером с максимум отображения 16 цветов в текстовом режиме и также с 16 цветами в графическом режиме. В более поздних модификациях использовался уже видеоадаптер EGA<sup>3</sup>. Клавиатура имела 83 клавиши: в отличие от современной, на ней отсутствовали отдельные функциональные клавиши, блок управления курсором, светодиодные индикаторы, русская раскладка. В моделях IBM PC-XT 286 поставлялась клавиатура с 84 клавишами: добавилась клавиша SysRq (системный запрос или запрос системы) и LED-индикаторы. Мощность

<sup>1</sup> Выпускался до апреля 1987 года.

<sup>2</sup> **MFM-кодирование** (иногда в литературе встречается название модифицированный код Миллера в квадрате, МЧМ-кодирование; Modified Frequency Modulation) — способ кодирования данных, один из способов физического кодирования (линейного кодирования, канального кодирования, цифрового кодирования, манипуляции сигнала). Служит для передачи цифровых данных от передатчика к приемнику по последовательному интерфейсу. Является двуполярным двухуровневым (сигнал может принимать два значения, соответствующие низкому уровню и высокому уровню) кодом, в котором каждый информационный бит кодируется комбинацией из двух битов.

<sup>3</sup> Видеоадаптер EGA позволял использовать 16 цветов при разрешении 640×350 пикселей.

блока питания в модели IBM PC-XT была увеличена в 2 раза – до 130 Ватт в сравнении с предыдущей моделью компьютера IBM PC.

В качестве базовой ОС использовалась ОС PC-DOS 2.0 и встроенный IBM BASIC.

Были у IBM и откровенные неудачи в линии их PC – в конце 1983 года IBM приняла первую попытку выйти на рынок домашних компьютеров, 1 ноября был анонсирован новый **IBM PCjr** (PC junior, IBM 4860). В связи с некоторыми конструктивными недостатками, неполной совместимостью с IBM PC и ошибками в маркетинговой политике, модель не имела коммерческого успеха. Журнал PC World внёс IBM PCjr в «список 25 худших технических продуктов всех времён». Компьютер был анонсирован 1 ноября 1983 года, а в марте 1985 года его производство было прекращено.

## Программное обеспечение

Анонс IBM PC 5150 состоялся 12 августа 1981. В большой спешке, в последний момент, буквально к моменту анонса, для этой машины была выпущена откровенно сырая, с огромным количеством ошибок и недоделок, операционная система PC DOS – по сути, переименованная версия MS-DOS, которая могла быть установлена только на оригинальные IBM PC, тогда как для клонов – это тоже было весьма необычно для IBM – Microsoft получила право предустанавливать свою MS-DOS. Ещё одним заметным отличием клонов от оригинала было отсутствие у них «вшитого» BASIC-а – при попытке обратиться по известному адресу в ПЗУ для запуска интерпретатора, на экран выводилось сообщение, что BASIC присутствует только в оригинальной IBM PC.

Далее по тексту будет приведена одна из версий истории появления MS-DOS (PC DOS), изложенная на основе статей в «Мир ПК №9 2004» ([Удивительная история рождения MS-DOS](#)) и [Гари Килдалл - автор первой операционной системы для микрокомпьютеров](#).

История MS-DOS началась в 1980 году, когда, исходя из текущей рыночной ситуации, корпорация IBM решает построить собственный 16-разрядный персональный компьютер, подобный популярному Apple (процессор MOS 6502), со схожим программным обеспечением. И для этого, разумеется, нужна была операционная система. Однако на тот момент существовавшие на рынке версии CP/M были исключительно 8-разрядными. Параллельно с этим корпорация IBM задумывается и о наборе ПО для проектируемого ПК. Зная об успешных разработках Microsoft (которая, к слову, к тому времени не имела «за душой» ни одной операционной системы собственной разработки, специализируясь на разработках в области языков программирования – несколько версий Бейсика, Кобол и Фортрана для различных компьютерных систем), IBM встречается с Биллом Гейтсом, попросив того высказать свои соображения в отношении будущего ПК. Гейтс, конечно же, предлагает «Голубому гиганту» приобрести у него BASIC. А на вопрос об операционной системе советует обратиться к Гари Килдаллу — автору бывшей уже к тому времени стандартом де-факто ОС CP/M.

Поэтому, на следующий день после визита в Сиэтл в штаб-квартиру Microsoft, представители IBM отправились в Монтерей, чтобы встретиться с Килдаллом.

Он впоследствии говорил, что его об этом не предупредили. Говорили, что Килдалл, который обожал летать на самолётах, находился в самолете в тот день, когда прибыли люди из IBM. Но Килдалл говорил, что это было совсем не так: он был в командировке до середины того дня, и поэтому люди из IBM встретились с Дороти, женой Гари. По словам Килдалла, они, по своему обыкновению, «набросились на нее с предложениями, которые могли испугать кого угодно». Гари говорил, что они хотели письменного подтверждения, что могут использовать любые идеи, исходящие от Digital Research, компании Килдалла. Когда Килдалл, возвратясь из своей поездки, узнал, что IBM и его жена достигли предварительного согласия, он решил прервать переговоры до тех пор, пока чета Килдалл не возвратится из запланированного ими ранее недельного круиза по Карибскому морю, хотя надо признать, что подписание соглашения с IBM его полностью устраивало. Но, в конце концов, итоговая сделка так и не состоялась.

В итоге, воспользовавшись задержками с Килдаллом, Гейтс не упускает свой шанс: он отправляет в IBM письмо, в котором убеждает компанию использовать в создаваемом компьютере 16-разрядный микропроцессор Intel 8088, а не 8-разрядный. В дальнейшем это письмо позволит ему склонить чашу весов в выборе IBM на свою сторону, так как это предполагает, что CP/M Гари Килдалла изначально разработана для 8-разрядных машин, 16-разрядная CP/M-86 ещё не готова, а ОС от Microsoft якобы будет сразу «заточена» под 16-разрядные компьютеры. Разумеется, Гейтс обещает поставить IBM нужную ей операционную систему в должные сроки.

Примерно в это время Пол Аллен, соратник Гейтса и со-основатель Microsoft, приобретает операционную систему QDOS<sup>1</sup> Тима Патерсона (Tim Paterson), никому не известного программиста из фирмы Seattle Computer Products (SCP). QDOS оказывается невероятно похожей на CP/M Гари Килдалла, и это неудивительно, т.к. исходно Патерсон разрабатывал свою «быструю и грязную ОС» (QDOS) в качестве быстрой альтернативы CP/M-86, выпуск которой сильно задерживался<sup>2</sup>. QDOS должна была сыграть роль недостающего звена в цепи продаж компьютеров на базе i8086<sup>3</sup>. Далее, после быстрого сиюминутного решения в виде QDOS, планировалась тщательная отладка (так никогда, к сожалению, и не реализованная) готовой DOS, которая затем должна была явиться в двух версиях: одно- и многопользовательской.

Патерсон выбрал верную стратегию для разработки своей DOS. В первую очередь была поставлена цель максимально облегчить написание программного обеспечения сторонними программистами в окружении DOS, а также упростить процесс переноса на другие платформы. Дело в том, что программы для 8-разрядной CP/M не могли непосредственно исполняться 16-разрядным i8086, их следовало дорабатывать с учетом другой схемы адресации данных, поддерживаемой новым процессором. Поскольку CP/M в то время доминировала на рынке операционных систем на микрокомпьютерах, под неё уже было написано большое количество ПО, которое требовалось с минимальными затратами перенести в среду DOS. Для этого Тим решил добиться совместимости с уже существующей 8-разрядной CP/M на уровне API. Это значительно облегчило программирование в DOS: все выглядело так, будто вы пишете под старой доброй CP/M.

Желая наделить свою операционную систему максимальной эффективностью и быстродействием, Патерсон пишет QDOS на чистом ассемблере. В период с апреля по июль 1980 г. Тим тратит почти половину своего времени на создание QDOS. И уже в августе QDOS 0.11 начала продаваться в комплекте с компьютерами производства SCP на основе Intel 8086.

В декабре 1980 г. Патерсон выпускает новую версию QDOS под маркой 86-DOS 0.33. Именно ее исходные тексты (порядка 4 тыс. ассемблерных строк) в последующем и приобретает Microsoft (в лице Пола Алена).

В итоге Microsoft заключает с SCP сделку: Гейтс единовременно выплачивает Брокку (владельцу SCP) сумму в 10 тыс. долл. за право продажи 86-DOS и по 15 тыс. долл. за каждого OEM-партнера (Original Equipment Manufacturer, изготовитель комплексного оборудования), согласившегося купить 86-DOS. При этом, разумеется, Билл не раскрывает (это было условием сделки) того, что уже имеющимся партнером выступает сама IBM. В итоге удовлетворенные стороны подписывают необходимые бумаги: Брок получает возмещенные 25 тыс. долл., Microsoft — 86-DOS.

В апреле 1981 г. Тим Патерсон покидает Seattle Computer Products (успев выпустить, кстати, 86-DOS 1.0), а в мае по приглашению Гейтса устраивается на работу в Microsoft. В этом Билл был заинтересован, как никто другой: ведь необходимо избежать возможных

---

<sup>1</sup> Есть легенда, что сам автор расшифровывал QDOS как “Quick and Dirty OS” – «быстрая и грязная ОС», намекая на не очень серьезную разработку «на коленке».

<sup>2</sup> Исходно выход CP/M-86 планировался на конец 1979 года, но ОС не была выпущена даже в апреле 1980 г.

<sup>3</sup> Seattle Computer Products в ноябре 1979 начала продажи своих компьютеров на базе Altair и процессора 8086 с установленным интерпретатором Бейсика от Microsoft.

скандалов вокруг авторских прав на 86-DOS, а также облегчить труд Microsoft по адаптации этой ОС под запросы IBM. До невозможности удивлённый и даже рассерженный тем фактом, что заказчиком операционной системы выступает IBM, Патерсон всё же продолжает работу над 86-DOS под крылом заботливого Гейтса.

В самом напряжённом темпе, чрезвычайно эффективно используя отпущенное время, Патерсон перерабатывает 86-DOS под требования IBM. Разумеется, сроки поджимают — Тим не успевает до конца выполнить все поставленные условия. В июле он передает то, что имелось на тот день, — местами прилично недоработанную MS-DOS 1.0. «Голубой гигант» изменяет название операционной системы на PC-DOS 1.0 и начинает её поставки вместе со своим IBM PC.

Гейтс приобрел исходные тексты 86-DOS 0.33, хотя в апреле 1981 г. Патерсон выпустил уже 86-DOS 1.0. Понятно, что MS-DOS 1.0, полученная в результате переработки Патерсоном 86-DOS 0.33, уже не могла служить камнем преткновения для выяснения, кому же принадлежат авторские права на DOS — Броксу (Seattle Computer Products) или Гейтсу (Microsoft).

Когда Гари Килдалл увидел PC-DOS, то высказался примерно так: «Я был сильно поражен потрясающей схожестью между PC-DOS и CP/M. Я был вне себя от злости на IBM и Microsoft. Все, кто хоть немного разбирался в компьютерах, могли убедиться, что даже команды этих операционных систем были одинаковыми. Это было просто невероятно!»

Возмущенный Килдалл выразил свое недоумение по поводу того, что такой гигант, как IBM, умудрился сделать подобное. Однако всё-таки не решился предпринимать какие-либо судебные меры в отношении корпорации — как из-за своего склада характера, так и по причине поступившего от IBM предложения. Компания посулила в качестве компенсации включить только что (наконец-то) выпущенную Килдаллом 16-разрядную версию ОС (CP/M-86) в состав ПО, поставляемого с IBM PC. Но все оказалось ещё хуже, чем хотелось несчастному Килдаллу.

Так, при приобретении IBM PC, покупателю предлагалось выбрать из трех вариантов ОС: PC-DOS, CP/M-86 и P-System UCSD (ОС с интегрированным языком Паскаль). Благодаря тому, что PC-DOS была самой дешевой (в шесть раз дешевле, чем стоила CP/M-86), выбор потребителя пал именно на неё — в итоге наметилась тенденция комплектовать все последующие компьютеры IBM PC только PC-DOS.

Что касается коммерческого видения DOS, до определенной поры для Microsoft эта ОС представлялась исключительно как вотчина IBM — Гейтс уделял много времени рынку языков программирования. Однако уже в 1982 г. Microsoft решает выйти на коммерческую арену операционных систем, заинтересовав ею и своих OEM-партнеров. Тогда же она проводит политику перехода с CP/M-версий своих продуктов (тех же языков программирования) исключительно на их DOS-реализации. Подобный шаг значительно ослабил лидирующие позиции Digital Research в монополии де-факто на операционные системы для микрокомпьютеров.

Необходимо понимать, почему Microsoft удалось пойти собственным путем вразрез с планами IBM, начав собственную кампанию по продаже MS-DOS. Сразу после получения вожделенной MS-DOS 1.0 для своего персонального компьютера IBM, конечно же, проводит её ревизию и, обнаружив порядка 300 ошибок, переписывает неверные участки кода, попутно переименовав систему в PC-DOS. По причине этой вынужденной модификации первоначальной версии MS-DOS и IBM, и Microsoft обладают авторскими правами на данную операционную систему.

Гейтсу, уверенному в будущем MS-DOS, удалось убедить IBM не заключать с ним исключительного лицензионного соглашения по передаче всех прав на DOS, а также уговорить корпорацию продавать ПК отдельно от операционной системы. По условиям контракта Microsoft получала некоторый процент с каждого проданного IBM PC. Забавный момент — нынешние менеджеры изучают детали этого договора в качестве примера катастрофической ошибки со стороны IBM.



Правильная маркетинговая политика Microsoft, вкуче с успехом IBM PC, послужили созданию прочного 15-летнего фундамента для DOS. В течение этого срока, вплоть до прихода на рынок Windows 95, DOS непрестанно развивалась, совершенствовалась и впитывала лучшие разработки третьих фирм, оставаясь при этом доминирующей ОС в нише персональных компьютеров. Учитывая плановую стратегию перехода из 16-разрядного мира DOS в сферу 32-разрядных Windows 95 и Windows NT, как относительно перспективных операционных сред, Microsoft постепенно прекратила обновления и реализацию MS-DOS, остановившись на версии 8<sup>1</sup>.

Первая версия MS-DOS (PC DOS) 1.0 была очень примитивной, например, файловая система не имела иерархической организации, была «плоской». Не существовало никакого разграничения прав, по типу пользовательских областей в CP/M. Иерархия в файловой системе появилась только в MS-DOS 2.0, но MS-DOS так и осталась довольно примитивной ОС, по сути, файловым монитором: однозадачная, работающая в реальной моде, с доступом к ОЗУ с максимальным объёмом 1 Мбайт, без механизмов разграничения прав в памяти и в ФС. Существовали альтернативные DOS-совместимые проекты, в которых были добавлены некоторые из отсутствовавших в оригинальной DOS возможности. Например, DR-DOS<sup>2</sup> от Digital Research, в некоторых ревизиях которой присутствовал механизм разграничения прав на файловой системе.

В конце 1980-х, в начале 1990-х DR-DOS довольно успешно конкурировал с MS-DOS. Например, в DR-DOS гораздо раньше, чем в ОС от Microsoft, была реализована возможность загрузки ОС в память «выше» первого мегабайта, без перевода процессора в защищённый режим (используя ошибку(?) в процессоре i80286), экономя на этом до 10% дефицитного ресурса – объёма основной (conventional) памяти. Microsoft скопировала эту идею в своей версии MS-DOS v5.

Но Microsoft не просто заимствовала чужие идеи из конкурирующих разработок, но и не гнушалась прибегать к откровенно грязным и нечестным методам конкуренции. DR-DOS, по определению, вынужденно должна была быть максимально совместимой и неотличимой от «первоисточника», т.е. MS-DOS. Но в какой-то момент, в 1991 году, вдруг «внезапно» выяснилось, что Windows 3.1, при попытке её загрузки из DR-DOS, отказывается загружаться и выдаёт сообщение о несовместимости Windows и DR-DOS. Внимательное изучение кода привело исследователей к интересному выводу: никакой несовместимости по API на самом деле нет, просто загрузчик Windows, проанализировав версию DOS и обнаружив DR-DOS «на месте» MS-DOS, принудительно завершал загрузку. Этот факт был подтверждён быстро выпущенным компанией Digital Research патчем, который устранял «несовместимость», по сути подменяя ответ на запрос версии, чтобы Windows считала, что запускается из «правильной» MS-DOS, а не из «несовместимой» DR-DOS. Никаких модификаций API при этом не было выполнено, что подтверждает сознательный «саботаж» со стороны Microsoft.

Ещё одна любопытная разработка, превращавшая однозадачную MS-DOS в многозадачную среду: DESQview, разработанная фирмой Quarterdeck многооконная многозадачная среда, являвшаяся надстройкой над MS-DOS, и реализующая так называемую «кооперативную» многозадачность в однозадачной MS-DOS. Программа позволяла запускать некоторое количество DOS-программ одновременно, при этом вывод происходил в несколько текстовых окон на экране монитора, подобно работе дисплея VT220. Первая версия DESQview появилась в июле 1985 года, на 4 месяца раньше первой версии Microsoft Windows, но не имела никаких средств графики.

---

<sup>1</sup> Последняя автономная версия MS DOS имела версию 6.22, версии 7 и выше были уже по сути платформой (загрузчиком) для запуска Windows: Windows 95 – MS DOS v7, Windows 98 – v7.1, Windows Me – v8. Хотя версии 7.x можно было использовать и в автономном режиме, без Windows.

<sup>2</sup> DR-DOS (первая версия вышла в 1988 г.) прошла в итоге длинный путь, с неоднократной сменой названия и владельцев: Digital Research – DR-DOS v3.41, v5, v6; Novell – Novell DOS 6, 7; Caldera – OpenDOS 7.01, 7.02; Lineo – DR-DOS 7.02, 7.03; DeviceLogics – DR-DOS 7.03, 8.

По ссылке [Хронология операционных систем DOS](#) приведены подробные данные обо всех выпускавшихся версиях DOS, оригинальных и совместимых.

Прикладное программное обеспечение на MS-DOS было представлено очень мощное и разнообразное: текстовые редакторы на любой вкус, трансляторы практически для всех существовавших тогда популярных языков программирования, электронные таблицы от разных производителей, игры, файловые оболочки, даже текстовые браузеры и т.п. Знаменитые интегрированные среды разработки от Borland, совмещающие редактор с подсветкой синтаксиса, транслятор, отладчик – Turbo C, Turbo Pascal и др. в первых версиях были разработаны именно для работы в среде MS-DOS.

Из ОС под i8086, не совместимых с MS-DOS, можно отметить внутреннюю разработку в Microsoft Unix-подобной операционной системы Xenix. Но эта ОС получила довольно ограниченное распространение, причём уже только на машинах с процессорами следующих поколений – i80286/386 (Xenix286, Xenix386).

## **Архитектура компьютеров на базе Intel 80286**

Intel, разрабатывая процессор 8086, рассматривала его как простое временное решение, призванное заполнить нишу до выхода на рынок 32-разрядного iAPX432<sup>1</sup>. В результате iAPX432 (трёхкристальный МП) появился на рынке только в 1982 году, четыре года спустя после «временного» i8086, оказался очень дорогим и медленным решением, и дальнейшая его разработка была прекращена.

Воодушевившись успехом своих первых 16-разрядных процессоров, во второй половине 1982 года Intel выпускает процессор следующего поколения – i80186. 186-ой фактически был усовершенствованным вариантом i8086, с той же архитектурой и немного расширенной системой команд. Процессоры Intel 8086 и 8088 требовали большого количества микросхем поддержки, и Intel решила разработать микропроцессор, уже содержащий на кристалле все необходимые модули. Новый процессор включал в себя множество компонентов, ранее выпускавшихся в виде отдельных микросхем (порядка 10 штук), это бы позволило резко сократить общее количество микросхем в компьютере, а, следовательно, и уменьшить его стоимость. На кристалл с CPU были добавлены:

- Два контроллера прямого доступа к памяти (DMA) со схемами прерываний
- Дешифраторы адреса (программируемые схемы выбора кристалла)
- Трёхканальный программируемый таймер/счётчик
- Генератор синхронизации
- Программируемый контроллер прерываний

По сути i80186 превратился из просто процессора в однокристальный контроллер, с довольно большим количеством периферии «на борту». Разработка оказалась весьма удачной, Intel (и ряд других компаний, таких как AMD, Siemens AG, Fujitsu) выпускала его с 1982 по 2007 год, по разным технологиям (nMOS, CMOS), с различными, всё более тонкими, проектными нормами (до 3 мкм) и тактовыми частотами от 6 до 25 МГц. Устройство массово использовалось во встраиваемых системах – в качестве контроллера скоростных модемов, принтеров, плоттеров и т.п. Но в качестве центрального процессора для PC i80186 никогда не применялся. Следующим интеловским процессором, после i8086/88, исполнявшим роль CPU в IBM PC/AT следующего поколения, был Intel 80286, довольно противоречивый и откровенно промежуточный по концепции процессор, последний 16-разрядный процессор в линейке x86.

---

<sup>1</sup> Одной из главных особенностей iAPX432 была аппаратная поддержка чрезвычайно популярного и модного в те годы языка программирования Ада.

## Третье поколение IBM PC/AT

В 1984 году IBM выпускает третье поколение своих PC: **IBM PC/AT** или **IBM 5170** (сокращение от Advanced Technology). PC/AT выпускались до 1990 года.

### Процессор и системная шина

Основой PC/AT стал процессор Intel 80286, и с ним был связан вынужденный переход на шину AT bus: 16-разрядная шина данных, 24-битная шина адреса, что позволяло адресовать ОЗУ объёмом до 16 Мбайт (по сравнению с 1 Мбайт оригинальной модели IBM PC). При этом обеспечивалась совместимость с платами старого образца (шина данных — 8 бит, шина адреса — 20 бит).

Подобно своему предшественнику IBM PC, IBM PC/AT поддерживает дополнительный чип математического сопроцессора Intel 80287 для более быстрого выполнения операций с числами в формате с плавающей точкой.

### BIOS

На материнской плате PC/AT появилась батарейка для питания микросхемы CMOS, в памяти которой, ёмкостью 50 байт, находилось содержимое часов реального времени. Первоначально IBM PC не имел такой памяти и пользователю, чтобы избежать установленной по умолчанию даты 01/01/1980, приходилось либо вручную устанавливать дату и время самостоятельно, при каждой загрузке системы при помощи команд Time и Date, либо устанавливать в слоты расширения карту с часами реального времени.

Находящаяся на диске программа настройки BIOS заняла место DIP-переключателей, конфигурирующих IBM PC и IBM PC/XT.

Большинство клонов IBM PC/AT имело программу настройки BIOS, размещённую в ПЗУ, которое работало быстрее, чем диск.

### Клавиатура

Изначально с IBM PC/AT поставлялась 84-клавишная клавиатура: 84-й кнопкой стала SysRq (System Request), цифровой блок на ней был отделён от основной алфавитно-цифровой группы, а над ним расположили светодиодные индикаторы режимов клавиатуры. Несмотря на то, что в клавиатурах использовался тот же разъём DIN-5, что и в более ранних моделях, предназначенных для компьютеров IBM PC и IBM PC/XT, клавиатуры были аппаратно несовместимы. В новой клавиатуре использовался двунаправленный интерфейс (клавиатуры IBM PC и IBM PC/XT использовали однонаправленный) и генерировались другие скан-коды. Двунаправленный интерфейс позволил компьютеру включать индикаторы на клавиатуре, производить сброс контроллера i8042 клавиатуры, устанавливать скорость обмена и т.д. Выпускались также клавиатуры, оснащённые переключателем, позволявшим подключать их как к компьютерам PC/AT, так и более ранних серий. Позднее для IBM PC/AT стали выпускаться расширенные клавиатуры с 101-106 (для разных языковых версий) клавишами, имевшие 12 функциональных клавиш и отдельный блок управления курсором. Эти раскладки с небольшими изменениями стали наиболее популярными в настольных компьютерах.

### Дисковые накопители

Для хранения данных в PC/AT применялись 5¼" дисководы с поддержкой дискет ёмкостью 1.2 Мбайта (15 секторов по 512 байт, 80 дорожек, две стороны), тогда как прошлое поколение IBM PC обеспечивало объём не более 360 кбайт. Вскоре на IBM PC/AT стали доступны и дисководы 3½".

Жёсткий диск ёмкостью 20 Мбайт был вдвое быстрее (задержка 40 мс), чем у предшествующей модели, IBM PC/XT, но ранние модели жёстких дисков, произведённые Computer Memories (CMI), имели 25-30% выходов из строя в течение года эксплуатации.

Частично это происходило из-за ошибки в алгоритме FAT, которым пользовалась MS-DOS 3.0 во время парковки головок чтения/записи в момент выключения компьютера.

## **Видеоадаптер и монитор**

Ранние IBM PC/AT (1984—1986 годов выпуска) оснащались CGA, монохромным MDA, EGA или VGA видеоадаптером. Модели, выпущенные позднее (после 1987 года), оснащались только VGA видеоадаптером, обратно совместимым с режимами EGA и CGA. Опционально для профессиональной работы с графикой можно было заказать видеоадаптер PGC (Professional Graphics Controller) стоимостью \$4290, способный выводить до 256 цветов на экран монитора с разрешением 640x480 пикселей, а заодно поддерживающий 2D и 3D ускорение для CAD-приложений.

IBM PC/AT был оборудован блокировкой с цилиндрическим ключом, которая могла использоваться, чтобы предотвратить доступ к компьютеру, отключая клавиатуру.

## **Операционная система**

Для поддержки всех нововведений, включая поддержку сети в ядре, пришлось серьёзно доработать операционную систему, которая вышла под названием PC-DOS 3.0. В это же время появилась операционная система OS/2.

## **Архитектура микропроцессора i80286**

В феврале 1982 года Intel выпускает новый процессор i80286, разрабатывавшийся параллельно с 80186/80188. В отличие от 186-го, это был классический CPU, без сопутствующей периферии на кристалле. Процессор выпускался по 1,5 мкм технологии, обладал резко увеличенным по сравнению с 8086 числом транзисторов (134000 штук против 29000 у 86-го) и возможностью адресации физической памяти до  $2^{24} = 16$  Мбайт. Первые 286-ые работали с тактовой частотой 6 МГц, которая, как и в случае оригинального 8086, была впоследствии удвоена и даже утроена, добравшись в итоге до величин в 20 МГц и даже 25 МГц (у клонов от конкурентов, н-р, AMD). 286-ой сразу же в два раза превысил по эффективности i8086 (в пересчёте на такт), осуществив таким образом удвоение эффективности работы для архитектуры x86, что впоследствии Intel-у больше не удавалось ни разу. На протяжении почти десятилетия 286-е<sup>1</sup> процессоры стали синонимом IBM PC, процессор был основой нового IBM PC/AT. По оценкам Intel, за 6 лет на 286-х процессорах в мире было выпущено около 15 миллионов ПК.

В процессоре i80286 был добавлен так называемый защищённый режим работы, при работе в котором в CPU включался механизм виртуальной памяти, с разграничениями доступа к ОЗУ (в этом режиме пользовательская программа уже не могла читать/модифицировать память другого процесса, как это было возможно в i8086/88). Объём адресуемого физического ОЗУ был увеличен до 24 Мбайт. Процессор имел 6-байтовую очередь, как и i8086, шина адреса и данных были не мультиплексированы<sup>2</sup>.

---

<sup>1</sup> 80286 и его клоны выпускались не только Intel, но и многими другими производителями (AMD, IBM, Siemens AG и др.), с февраля 1982 по начало 1990-х.

<sup>2</sup> Для размещения всех выводов процессора пришлось упаковывать 286-ой в новый оригинальный корпус (использовалось несколько вариантов, например, PLCC 68-pin). В старом добром DIP-40 разместить все выводы было бы уже невозможно.

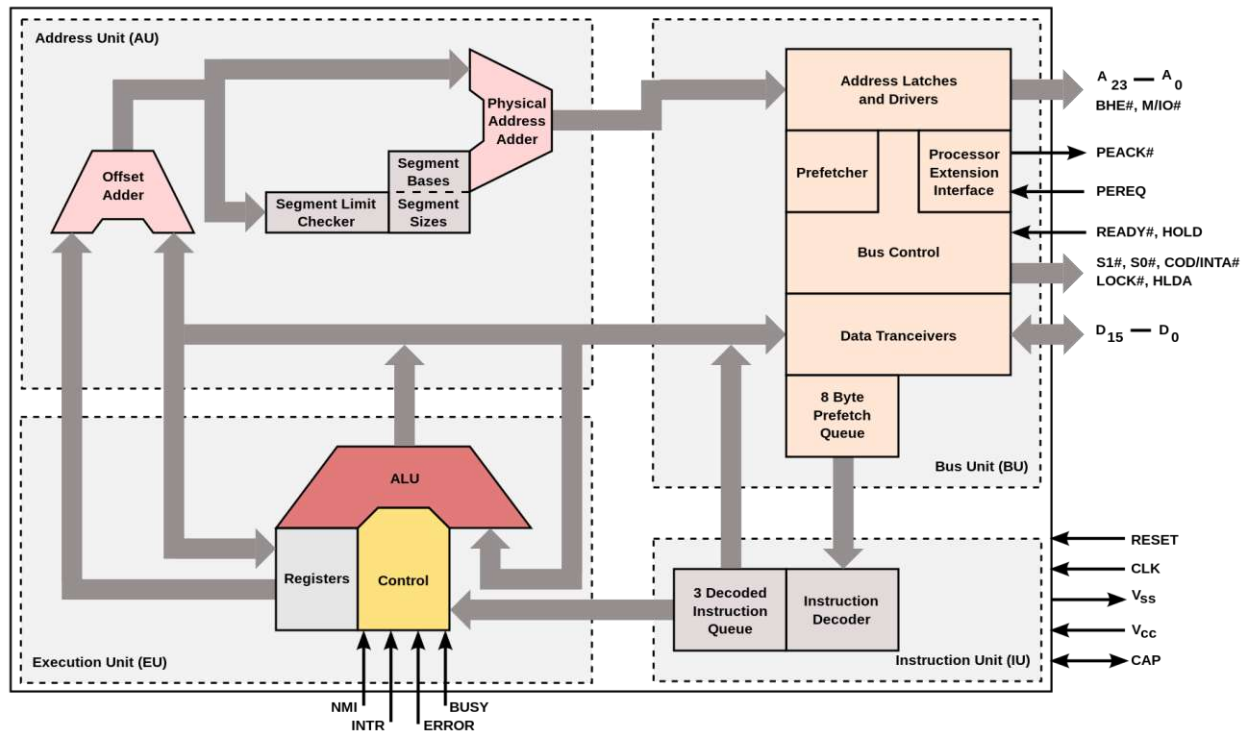


Рис.224 – Архитектура i80286

## Регистры

К 14 регистрам процессора i8086 было добавлено несколько новых регистров – пять новых «видимых» и шесть «невидимых», программно недоступных – необходимых для реализации защищённого режима и других функций:

- **MSW** – 16-битный регистр слова состояния машины. Если в MSW флаг PE=1, то процессор переключается в защищённый режим.
- **TR** – 16-битный регистр задачи, содержит селектор сегмента состояния задачи, используется при работе в многозадачном режиме.
- **GDTR** – 40-разрядный регистр, определяет размер и положение глобальной дескрипторной таблицы в ОЗУ.
- **LDTR** – 16-разрядный регистр, определяет базовый адрес локальной дескрипторной таблицы.
- **IDTR<sup>1</sup>** – 40-разрядный регистр определяет начало и размер таблицы векторов прерываний;
- 6 «невидимых» регистров расширения сегментных регистров, связанных с CS, DS, ES, SS, GDTR, IDTR.

Используя преобразование виртуального адреса по дескрипторным таблицам при работе в защищённом режиме, теоретически для i80286 было возможно создать программу размером до  $2^{29} + 2^{29} = 2^{30} = 1$  Гбайт! Процессор при этом оставался 16-разрядным.

<sup>1</sup> В процессоре 80286 появилась возможность размещать таблицу векторов прерываний в произвольном месте оперативной памяти, а не обязательно в самом начале адресного пространства, как в процессоре 8086. Для этого используется специальный регистр IDTR, по структуре аналогичный сорокаразрядному регистру GDTR. Он определяет начало и размер таблицы векторов прерываний. Имеются так же специальные команды его чтения и записи.



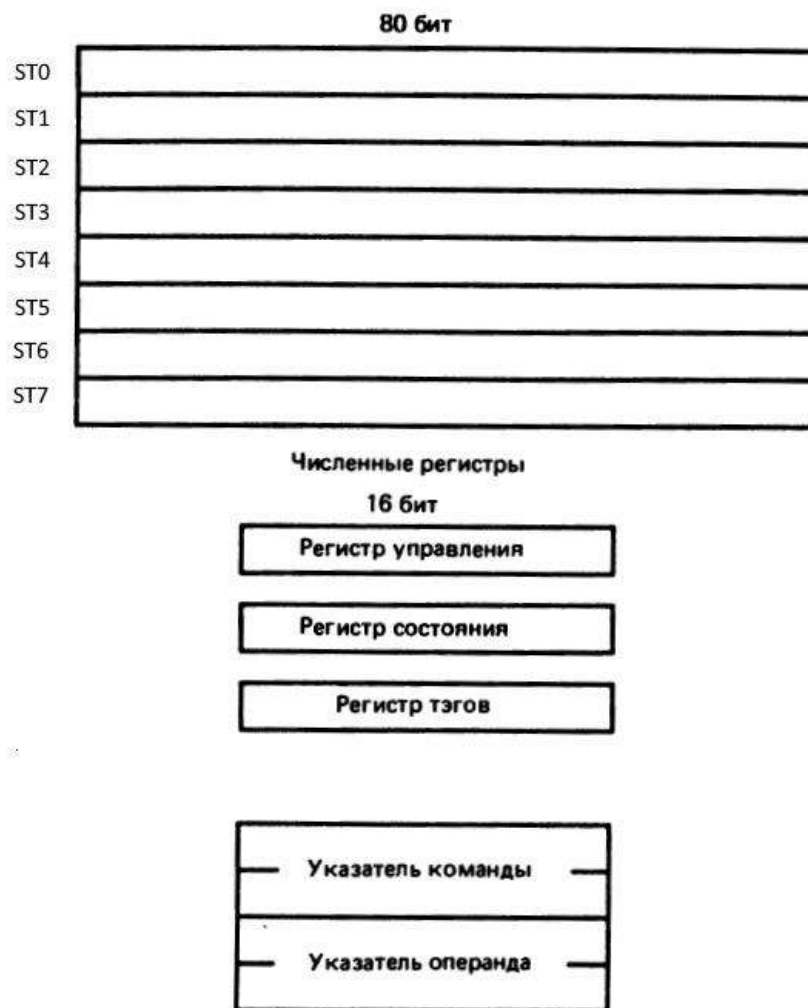


Рис.226 – Регистры арифметического сопроцессора i80287

### **Численные регистры**

Численные регистры используются как стек. Регистр состояния в поле ST содержит номер численного регистра, являющегося вершиной стека. При выполнении команд в качестве операнда могут выступать численные регистры. В этом случае номер указанного в команде регистра прибавляется к содержимому поля ST регистра состояния и таким образом определяется используемый регистр. Большинство команд после выполнения увеличивают поле ST регистра состояния, как бы записывая результаты своей работы в стек численных регистров.

Возможно использовать регистры как массив, но в этом случае необходимо заботиться о постоянстве поля ST регистра состояния, так как в противном случае номера численных регистров будут изменяться.

### **Регистр тэгов**

Этот регистр разделен на восемь двухбитовых полей, которые обозначены на Рис.227 как TAG0...TAG7. Каждое поле относится к своему численному регистру.

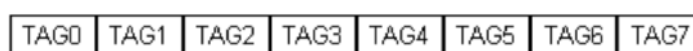


Рис.227 – Формат регистра тэгов

Поля регистра тегов классифицируют содержимое «своего» численного регистра, в соответствии с Рис.228:

| Поле | Описание                                                                           |
|------|------------------------------------------------------------------------------------|
| 00   | Регистр содержит действительное ненулевое число                                    |
| 01   | В регистре находится нуль                                                          |
| 10   | Регистр содержит недействительное число - нечисло, бесконечность, неопределенность |
| 11   | Пустой неинициализированный регистр                                                |

**Рис.228 – Поля регистра тэгов**

Например, если все регистры сопроцессора были пустые, а затем в стек численных регистров было занесено одно действительное ненулевое значение, содержимое регистра тэгов будет 0x3FFF.

### ***Регистр управления***

Регистр управления для сопроцессора i8087 показан на Рис.229.

|       |    |       |     |     |     |    |    |    |    |    |    |
|-------|----|-------|-----|-----|-----|----|----|----|----|----|----|
| 15-13 | 12 | 11-10 | 9-8 | 7   | 6   | 5  | 4  | 3  | 2  | 1  | 0  |
| XXXXX | IC | RC    | PC  | IEM | XXX | PM | UM | OM | ZM | DM | IM |

**Рис.229 - Формат регистра управления для сопроцессора i8087**

Регистр управления сопроцессоров 80287/80387 и сопроцессора, входящего в состав более современных процессоров, имеет аналогичный формат, за исключением того, что бит 7 в нем не используется Рис.230.

|       |    |       |     |     |    |    |    |    |    |    |
|-------|----|-------|-----|-----|----|----|----|----|----|----|
| 15-13 | 12 | 11-10 | 9-8 | 7-6 | 5  | 4  | 3  | 2  | 1  | 0  |
| XXXXX | IC | RC    | PC  | XXX | PM | UM | OM | ZM | DM | IM |

**Рис.230 - Формат регистра управления для i80287/387 и более современных сопроцессоров**

Биты 0...5 - маски особых случаев. Особые случаи могут возникнуть при выполнении команд сопроцессора, например, при делении на нуль, переполнении и так далее.

Если все биты масок особых случаев равны нулю, особый случай вызывает прерывание центрального процессора INT 0x10 (обратите внимание, что этот же вектор прерывания используется BIOS для работы с дисплейным адаптером). Если же особые случаи замаскированы установкой соответствующих бит в единичное состояние, прерывание не вырабатывается, а в качестве результата возвращается особое значение – бесконечность, нечисло и так далее. Ниже, на Рис.231, приведена таблица масок особых случаев:



|       |                                                                                                                                                            |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Маска | Особый случай                                                                                                                                              |
| IM    | Недействительная операция                                                                                                                                  |
| DM    | Денормализованный результат                                                                                                                                |
| ZM    | Деление на нуль                                                                                                                                            |
| OM    | Переполнение                                                                                                                                               |
| UM    | Антипереполнение                                                                                                                                           |
| PM    | Особый случай при неточном результате                                                                                                                      |
| IEM   | Маскирование одновременно всех особых случаев вне зависимости от установки бит 0-5 регистра управления. Этот бит действителен только для сопроцессора 8087 |

**Рис.231 – Таблица масок особых случаев**

Поле PC регистра управления управляет точностью вычислений в сопроцессоре, в соответствии с Рис.232:

| Поле | Описание                                                                                      |
|------|-----------------------------------------------------------------------------------------------|
| 00   | Использование расширенной точности. Этот режим устанавливается при инициализации сопроцессора |
| 10   | Округление результата до двойной точности                                                     |
| 11   | Округление результата до одинарной точности                                                   |

**Рис.232 – Управление точностью арифметических вычислений**

Искусственное ухудшение точности вычислений не приводит к ускорению работы программы. Режимы с пониженной точностью предназначены для эмуляции процессоров, использующих двойную и одинарную точность, соответственно.

Двухбитовое поле RC регистра управления задает режим округления при выполнении операций с вещественными числами, в соответствии с Рис.233:

| Поле | Описание                                                                                 |
|------|------------------------------------------------------------------------------------------|
| 00   | округление к ближайшему числу, этот режим устанавливается при инициализации сопроцессора |
| 01   | округление в направлении к отрицательной бесконечности                                   |
| 10   | округление в направлении к положительной бесконечности                                   |
| 11   | округление в направлении к нулю                                                          |

**Рис.233 – Режимы округления вещественных чисел**

Для наибольшего уменьшения ошибок вычислений целесообразно использовать режим округления в направлении к ближайшему числу. Режим округления в направлении к нулю используется при моделировании целочисленной арифметики.

Остальные два режима округления используют в интервальной арифметике. Для получения наиболее точного результата каждая команда (операция) выполняется два раза – первый раз с округлением в направлении к отрицательной бесконечности, второй раз – в направлении к положительной бесконечности. Точный результат лежит между полученными значениями. Заметьте, что здесь речь идет только об отдельных операциях, но не о том, чтобы выполнить всю программу вычислений вначале с одним режимом округления, а затем с другим.

Поле IC регистра управления предназначено для управления бесконечностью и может иметь два значения:

- 0 – проективный режим;
- 1 – аффинный режим

В проективном режиме существует только одна бесконечность, она не имеет знака. В аффинном режиме имеется две бесконечности – положительная и отрицательная.

Аффинный режим допускает выполнение многих операций с бесконечностями – сложение, умножение и т.д.

### **Регистр состояния**

Поля регистра состояния сопроцессора i8087 приведены на Рис.234.

|    |    |       |    |    |    |    |    |    |    |    |    |    |    |
|----|----|-------|----|----|----|----|----|----|----|----|----|----|----|
| 15 | 14 | 13-11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| B  | C3 | ST    | C2 | C1 | C0 | IR | XX | PE | UE | OE | ZE | DE | IE |

**Рис.234 - Формат регистра состояния сопроцессора i8087**

Регистр состояния сопроцессоров i80287/i80387 и сопроцессоров, входящих в состав более современных процессоров, имеет немного другой формат, он приведён на Рис.235.

|    |    |       |    |    |    |    |    |    |    |    |    |    |    |
|----|----|-------|----|----|----|----|----|----|----|----|----|----|----|
| 15 | 14 | 13-11 | 10 | 9  | 8  | 7  | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| B  | C3 | ST    | C2 | C1 | C0 | ES | XX | PE | UE | OE | ZE | DE | IE |

**Рис.235 - Формат регистра состояния современных сопроцессоров**

В обоих форматах биты 0-5 – флаги особых случаев. Они устанавливаются всегда при возникновении особых случаев, даже замаскированных установкой в 1 соответствующих битов регистра управления.

Ниже на Рис.236 приведена таблица флагов особых случаев:

| Флажок | Особый случай               |
|--------|-----------------------------|
| IE     | Недействительная операция   |
| DE     | Денормализованный результат |
| ZE     | Деление на ноль             |
| OE     | Переполнение                |
| UE     | Антипереполнение            |
| PE     | Неточный результат          |

**Рис.236 – Таблица флагов особых случаев**

Для того, чтобы сбросить установившийся флажок, программа должна явным образом установить его в нуль, выполнив команду записи в регистр состояния.

Назначение бита 7 регистра состояния различно для сопроцессора i8087 и сопроцессоров i80287/i80387. Для сопроцессора i8087 этот бит обозначается IR и содержит флаг запроса прерывания при возникновении незамаскированного особого случая. В этом случае флаг устанавливается в 1. Сопроцессоры i80287/i80387 используют бит 7 в качестве флага суммарной ошибки, который устанавливается в 1 при возникновении незамаскированного особого случая.

Биты C0, C1, C2, C3 – это коды условий. Они определяются по результату выполнения команд сравнения и команды нахождения остатка.

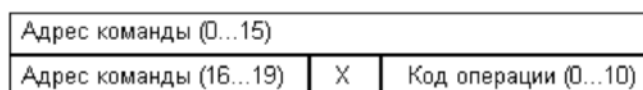
Поле ST занимает три бита 11...13 и содержит номер численного регистра, являющегося вершиной стека численных регистров.

Бит В – бит занятости. Он устанавливается в 1, когда процессор выполняет команду или когда происходит прерывание от сопроцессора. Если сопроцессор свободен, бит занятости установлен в 0.

### ***Регистры указателя команды и указателя операнда***

Регистры указателя команды и указателя операнда предназначены для обработки особых случаев, возникающих при выполнении команд в сопроцессоре.

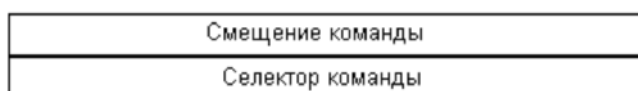
В сопроцессоре i8087 указатель команды содержит 20-разрядный адрес команды, вызвавшей особый случай и код выполняемой в этот момент операции. Адрес команды здесь указывается без учета предшествующих команде префиксов (Рис.237).



**Рис.237 - Указание адреса команды в сопроцессоре i8087**

Сопроцессоры i80287/i80387 в реальном режиме работы имеют такой же формат регистра указателя команд, однако этот указатель показывает на первый префикс команды, вызвавшей особый случай.

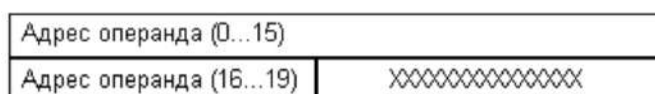
В защищенном режиме адрес состоит из селектора и смещения. Формат указателя команды для защищенного режима представлен на Рис.238.



**Рис.238 - Формат указателя команды для защищенного режима**

Код операции здесь отсутствует, но его легко получить, пользуясь адресом команды. Если при возникновении особого случая использовался операнд, находящийся в оперативной памяти, его адрес записывается в регистр указателя операнда. Ниже приведены форматы этого регистра для реального и защищенного режимов работы.

Формат указателя операнда для реального режима представлен на Рис.239.



**Рис.239 - Формат указателя операнда для реального режима**

Формат указателя операнда для защищенного режима представлен на Рис.240.

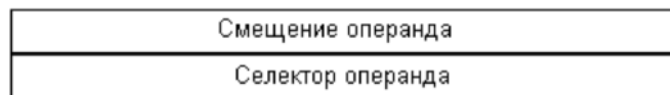


Рис.240 - Формат указателя операнда для защищенного режима

## Инструкции

Процессор имел тот же набор инструкций, что и процессор Intel 80186, в который было добавлено 16 новых команд (*LGDT*, *LIDT*, *LLDT*, *LMSW*, *LTR*, *SGDT*, *SIDT*, *SLDT*, *SMSW*, *STR*, *ARPL*, *CLTS*, *LAR*, *LSL*, *VERR*, *VERW*), необходимых для работы со средствами управления памятью. Команда *PUSH* теперь могла сохранять в стеке константы. Инструкции в Intel 80286 выполняются в среднем за 4.5 такта.

## Организация памяти

Шина адреса процессора i80286 разрядностью 24 бита позволяет адресовать до 16 Мбайт физической памяти, но в реальном режиме по-прежнему был доступен только 1 Мбайт (640 Кбайт обычной памяти и 384 Кбайт расширенной), начинающийся с нулевого адреса. С программной точки зрения память так же, как и в i8086, организуется в виде сегментов, но управление сегментацией имеет существенные различия для реального и защищённого режимов.

В реальном режиме по адресации памяти была декларировалась полная совместимость с процессором i8086, который своей 20-битной адресной шиной адресует пространство физической памяти в 1 Мбайт. По сути в реальном режиме работы i80286 был более быстрой и эффективной версией i8086 – с тем же набором регистров, с той же системой команд, с тем же объёмом адресуемой памяти 1 Мбайт.

Но разработчиками процессора i80286 была допущена ошибка, из-за которой, **работая в реальном режиме**, при возникновении переполнения (переноса) в 20-ый разряд адреса, указатель переходил не в нулевые адреса, как в случае с i8086, а активировал старшую адресную линию A20 (нумерация с нуля), позволяя достигать к 64 Кбайт-16 байт непосредственно выше первого мегабайта (во втором мегабайте). Если, например,  $Seg=0xFFFF$  и  $EA=0xFFFF$ , то физический адрес, вычисленный по формуле  $PA=16 \times Seg + EA=0x10FFEF$ , процессором i8086 трактуется как  $0x0FFEF$  — адрес, принадлежащий первому мегабайту. Но в процессоре 80286 адресный выход A20 в этом примере устанавливался в единичное значение, что соответствовало адресу ячейки из второго мегабайта физической памяти. Для обеспечения полной программной совместимости с i8086 в схему PC/AT был введён специальный вентиль Gate A20, принудительно обнуляющий бит A20 системной шины адреса.

Не оценив исходно потенциальной выгоды от этой ошибки, разработчики архитектуры PC/AT «узаконили» управление вентилем Gate A20 через программно-управляемый бит контроллера клавиатуры(!) 8042. Когда оперативная память подешевела, а «аппетит» программного обеспечения вырос, в эту небольшую область (64К-16 байт) стали помещать некоторые резидентные программы или даже часть (практически всю полностью, на самом деле) операционной системы, а для ускорения управления вентилем появились более быстрые способы (Gate A20 Fast Control).

В отличие от i8086 процессор i80286 имеет средства контроля за переходом через границу сегмента, работающие и в реальном режиме. При попытке адресации к слову, имеющему смещение  $0xFFFF$  (его старший байт выходит за границу сегмента), или выполнения инструкции, все байты которой не умещаются в данном сегменте, процессор вырабатывает exception (исключение) типа 13 (0Dh) — Segment Overrun Exception. При попытке выполнения инструкции *ESCAPE* с операндом памяти, не умещающимся в сегменте, вырабатывается исключение типа 9 — Processor Extension Segment Overrun Interrupt.

Одними из первых пользу ошибки с адресной линией A20 оценили разработчики операционной системы DR-DOS – они реализовали загрузку практически всего кода ОС в эти «бесплатные» почти 64К памяти, не переключая при этом процессор в защищённый режим. Тем самым в нижней, обычной памяти освобождалось до 64Кбайт ОЗУ, а это целых 10% от 640Кбайт – заметная цифра.

Эту способность загружать код ОС в верхнюю область памяти (UMA, строка DOS=High в config.sys) тут же скопировала Microsoft в своей MS-DOS 5.0.

В конце-концов, ошибка с A20 в процессоре i80286 была признана очень полезной и в последующих поколениях своих процессоров компания Intel её сохранила и не стала исправлять, обеспечив совместимость своих процессоров не только по системе команд, но и по ошибкам ☺.

В защищённом режиме работают все режимы адресации, допустимые для i8086 и реального режима i80286. Отличия касаются определения сегментов:

- сегментные регистры CS, DS, SS и ES хранят не сами базовые адреса сегментов, а селекторы, по которым из таблицы, хранящейся в ОЗУ, извлекаются дескрипторы сегментов;
- дескриптор описывает базовый адрес, размер сегмента (1 — 64 Кбайт) и его атрибуты;
- базовый адрес сегмента имеет разрядность 24 бита, что и обеспечивает адресацию 16 Мбайт физической памяти.

В защищённом режиме, также, как и в реальном, существуют понятия логического и физического адреса. Логический адрес в защищённом режиме (иногда используется термин «виртуальный адрес») состоит из двух 16-разрядных компонент – селектора и смещения. Селектор записывается в те же сегментные регистры, что и сегментный адрес в реальном режиме. Однако преобразование логического адреса в физический выполняется не простым сложением со сдвигом сегмента влево на четыре разряда, а при помощи специальных таблиц преобразования адресов.

В первом приближении можно считать, что для процессора i80286 селектор является индексом в таблице, содержащей базовые 24-разрядные физические адреса сегментов. В процессе преобразования логического адреса в физический процессор прибавляет к базовому 24-разрядному адресу 16-разрядное смещение, т.е. вторую компоненту логического адреса.

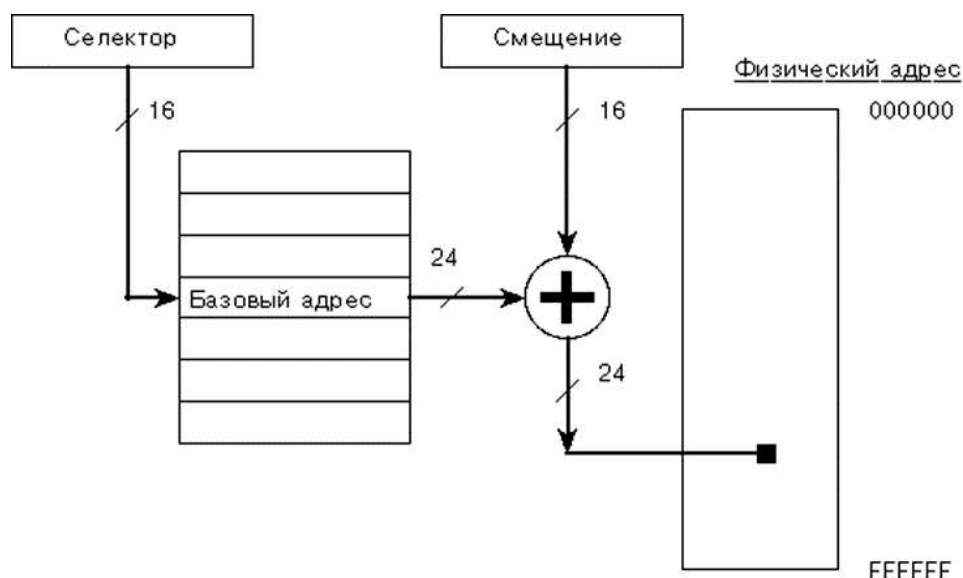


Рис.241 - Упрощённая схема преобразования логического адреса в физический в защищённом режиме

Такая схема формирования физического адреса позволяет непосредственно адресовать 16 Мбайт памяти с помощью 16-разрядных компонент логического адреса.

Необходимо отметить, что селектор – это не сегментный адрес. Это индекс, с помощью которого процессор извлекает из специальной дескрипторной таблицы 24-разрядный базовый адрес сегмента. В реальном режиме мы имеем дело с сегментным адресом и смещением, а в защищённом – с селектором и смещением.

На самом деле не все 16 бит селектора используются для индексации по таблице базовых адресов. В качестве индекса выступают *старшие* 13 бит. Два младших бита (бит 0 и бит 1) используются системой защиты памяти. Бит 2 позволяет выбирать для преобразования адреса один из двух типов таблиц преобразования адресов. Полный формат селектора адреса показан на Рис.242:



Рис.242 – Формат селектора адреса

На этом рисунке два младших бита обозначены как RPL (Requested Privilege Level). Это поле является запрошенным программой уровнем привилегий. Поле TI (Table Indicator) состоит из одного бита. Если этот бит равен нулю, для преобразования адреса используется так называемая *глобальная таблица дескрипторов* GDT (Global Descriptor Table), в противном случае – *локальная таблица дескрипторов* LDT (Local Descriptor Table).

Таблица дескрипторов – это просто таблица преобразования адресов, содержащая базовые 24-разрядные физические адреса сегментов и некоторую другую информацию. То есть каждый элемент таблицы дескрипторов (дескриптор) содержит 24-разрядный базовый адрес сегмента и дополнительную информацию, описывающую сегмент.

Таблица GDT – единственная в системе. Обычно в ней находятся описания сегментов операционной системы. Таблиц LDT может быть много. Эти таблицы содержат описания сегментов программ, работающих под управлением операционной системы, т.е. отдельных задач. В каждый данный момент времени процессор может использовать только одну таблицу LDT.

Процессор имеет два регистра, предназначенных для адресации используемых в настоящий момент таблиц GDT и LDT. Регистр GDTR описывает расположение и размер таблицы GDT, а регистр LDTR содержит ссылку на использующуюся в настоящее время таблицу LDT.

На Рис.243 показана уточнённая схема преобразования адресов в защищённом режиме. Из рисунка видно, что регистры процессора GDTR и LDTR определяют расположение в памяти таблиц GDT и LDT соответственно. Таблицы GDT и LDT содержат дескрипторы, описывающие сегменты памяти. В этих дескрипторах, помимо другой информации (заштрихованная область) содержится 24-разрядный базовый адрес сегмента.

Старшие 13 битов селектора (индекс) выбирают элемент из таблицы GDT или LDT в зависимости от состояния бита TI селектора.

Извлечённый из таблицы дескрипторов базовый адрес сегмента складывается процессором для получения 24-разрядного физического адреса.

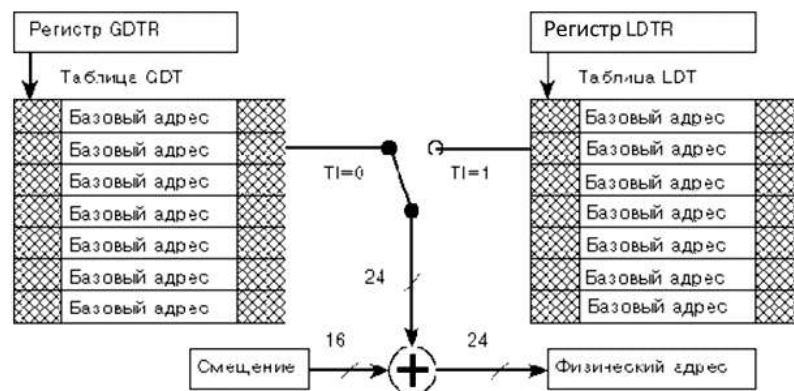


Рис.243 – Уточнённая схема преобразования адресов

Селектор 0000h адресует самый первый дескриптор в глобальной таблице дескрипторов GDT. Поле RPL для этого дескриптора равно 0, поле TI также равно 0. Селектор 0008h указывает на второй элемент таблицы GDT, а селектор 0014h указывает на третий дескриптор в локальной таблице дескрипторов LDT, т.к. поле TI в нём равно 1.

Регистр GDTR, указывающий расположение в физической памяти и размер глобальной таблицы дескрипторов GDT, является ключевым в схеме адресации защищённого режима.

Формат регистра GDTR процессора i80286 приведён ниже, на Рис.244:

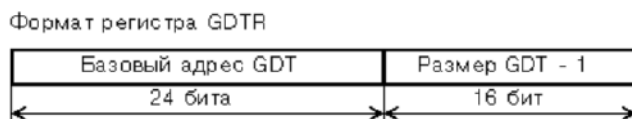


Рис.244 – Формат регистра GDTR процессора i80286

Из рисунка видно, что регистр GDTR занимает 5 байт, из которых старшие 3 байта содержат 24-разрядный физический адрес таблицы GDT, а младшие два байта – длину таблицы GDT минус 1.

Длина GDT, уменьшенная на единицу, называется пределом таблицы GDT (GDT limit). Она используется для проверки правильности задаваемых программой селекторов. Поле индекса селектора должно содержать ссылки только на существующие элементы таблицы GDT, в противном случае произойдет прерывание (исключение). Зная размер GDT, процессор блокирует использование селекторов со значениями поля индекса, выходящее за рамки разрешённых для таблицы GDT. Аналогичный механизм используется и для проверки селекторов, ссылающихся на LDT.

Несмотря на то что размер регистра GDTR составляет 5 байт, в качестве операнда для команды LGDT используется адрес области памяти размером 6 байт. Здесь нет никакой ошибки, процессор i80286 использует только 5 байт из этой области, так как физический адрес содержит 24 разряда. Процессоры i80386 и i80486 в 32-разрядном режиме используют все 6 байтов, загружая в 6-байтный регистр GDTR 32-битовый физический адрес таблицы GDT и её 16-битовый предел.

Таблицы GDT и LDT представляют собой массивы дескрипторов – описателей сегментов. Кроме дескрипторов, описывающих сегменты памяти, таблица GDT может содержать специальные типы дескрипторов - вентили вызова (*call gate*), задач (*task gate*) и ловушек (*trap gate*).

Вентили определяют точки входа в соответствующие процедуры. Например, вентиль вызова описывает адрес подпрограммы, вызываемой, к примеру, по команде CALL. При вызове подпрограммы через вентиль в качестве операнда для команды CALL используется селектор, адресующий соответствующий дескриптор в таблице GDT (или в таблице LDT).

Ниже, на Рис.245 приведён формат дескриптора сегмента для процессора i80286:

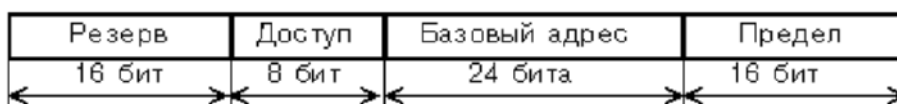


Рис.245 - Дескриптор сегмента для процессора i80286

Длина дескриптора составляет 8 байт. Он состоит из следующих полей:

- поле базового адреса длиной 24 бита содержит физический адрес сегмента, описываемого данным дескриптором;
- поле предела содержит размер сегмента в байтах, уменьшенный на единицу;
- поле доступа описывает тип сегмента (сегмент кода, сегмент данных и др.);
- зарезервированное поле длиной 16 бит для процессора i80286 должно содержать нули, это поле используется процессорами i80386 и i80486 (там, в частности, хранится старший байт 32-разрядного базового адреса сегмента<sup>1</sup>).

В реальном режиме начало сегмента в памяти определяется сегментным адресом, длина сегмента составляет 64 килобайта. В защищённом режиме можно задавать сегменты меньшего размера (процессоры i80386 и i80486 допускают также существование сегментов с размером, значительно превышающим 64 килобайта). При этом процессор автоматически отслеживает попытки программы обратиться за пределы сегмента, заданные в дескрипторе. При обнаружении такой попытки выполнение программы прерывается.

Ограничение размера сегментов и контроль за попытками адресации памяти вне пределов сегментов сильно повышает надёжность системы. Программа не может разрушить чужие сегменты, в частности, сегменты операционной системы.

Поле доступа, занимающее в дескрипторе один байт (байт доступа) служит для классификации дескрипторов. На Рис.246 приведены форматы поля доступа для трёх типов дескрипторов – дескрипторов сегментов кода, сегментов данных и системных:

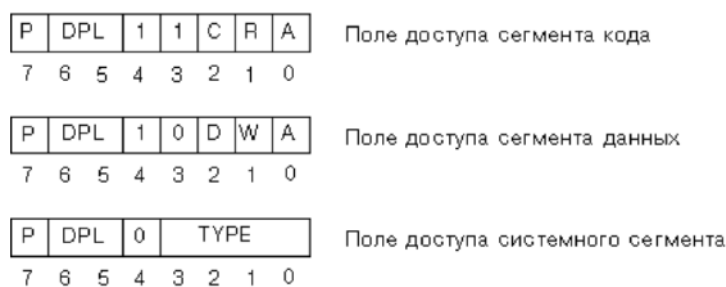


Рис.246 - Форматы поля доступа дескриптора

Поле доступа дескриптора сегментов кода содержит битовое поле R, называемое битом разрешения чтения сегмента. Если этот бит установлен в 1, программа может считывать содержимое сегмента кода. В противном случае процессор может только выполнять этот код (execute only).

Программа не может модифицировать сегмент кода. Это означает невозможность создания самомодифицирующихся программ для защищённого режима (распространённая,

<sup>1</sup> Очередной свидетельством удивительной «стройности» интеловской архитектуры – старший байт 32-разрядного адреса располагается не в подряд идущих разрядах за младшими тремя байтами. А всего-то надо было, имея в виду будущий переход к 32-разрядным архитектурам, зарезервировать место под старший байт базового адреса сегмента на месте поля доступа.



но порочная практика среди программистов, создающих программы для MS-DOS). Впрочем, есть обходной путь. Для сегмента кода можно создать ещё один, алиасный (псевдоним) дескриптор, в котором этот сегмент будет отмечен как сегмент данных. Если для этого сегмента будет разрешена запись, ничто, кроме здравого смысла, не мешает вам модифицировать код программы во время её выполнения.

Бит С называется битом подчинения.

Биты Р и А предназначены для организации виртуальной памяти. Бит Р называется битом присутствия (presence) сегмента в памяти. Для тех сегментов, которые находятся в физической памяти, этот бит должен быть установлен в 1.

Любая попытка программы обратиться к сегменту памяти, в дескрипторе которого бит Р установлен в 0, приведёт к прерыванию по отсутствию сегмента в ОЗУ.

Бит А называется битом обращения к сегменту и для всех обычных программ должен быть установлен в 0.

Поле доступа дескриптора сегмента данных имеет битовые поля W и D. Поле W называется битом разрешения записи в сегмент. Если этот бит установлен в 1, наряду с чтением возможна и запись в данный сегмент. В противном случае, при попытке записи в этот сегмент, выполнение программы будет прервано.

Поле D задаёт направление расширения сегмента. Обычный сегмент данных расширяется в область старших адресов (расширение вверх). Если же в сегменте расположен стек, расширение происходит в обратном направлении – в область младших адресов (расширение вниз). Для сегментов, в которых организуются стеки, необходимо устанавливать поле D равным 1.

Дескрипторы системных сегментов содержат поле TYPE, определяющее тип системного сегмента. В таблице на Рис.247 приведены возможные для этого поля значения:

| Поле TYPE | Тип сегмента                                   |
|-----------|------------------------------------------------|
| 0         | Запрещённое значение                           |
| 1         | Доступный TSS для процессора i80286            |
| 2         | Сегмент LDT                                    |
| 3         | Занятый TSS для процессора i80286              |
| 4         | Вентиль вызова для процессора i80286           |
| 5         | Вентиль задачи для процессоров i80286 и i80386 |
| 6         | Вентиль прерывания для процессора i80286       |
| 7         | Вентиль исключения для процессора i80286       |
| 8         | Запрещённое значение                           |
| 9         | Доступный TSS для процессора i80386            |
| A         | Зарезервировано                                |
| B         | Занятый TSS для процессора i80386              |
| C         | Вентиль вызова для процессора i80386           |
| D         | Зарезервировано                                |
| E         | Вентиль прерывания для процессора i80386       |
| F         | Вентиль ловушки для процессора i80386          |

**Рис.247 – Типы системных сегментов**

Резюмируя – перед переводом процессора в защищённый режим необходимо сформировать в памяти таблицу GDT, и загрузить в регистр GDTR её адрес и предел при помощи команды LGDT.

В отличие от регистра GDTR, регистр LDTR имеет только 16 разрядов. Он содержит не адрес и размер таблицы LDT, а селектор дескриптора, описывающего таблицу LDT. Это системный дескриптор, который должен находиться в таблице GDT и иметь в поле TYPE значение 2. Дескриптор сегмента LDT содержит базовый адрес и предел таблицы LDT.

Простейшие системы защищённого режима могут совсем не использовать таблицу LDT. Практическая польза от применения таблиц LDT появляется только в мультизадачных системах. В этом случае назначение каждой задаче собственной LDT позволяет полностью изолировать адресные пространства отдельных задач.

Краткие итоги по описанию организации работы с памятью микропроцессора i80286:

- В защищённом режиме применена новая схема преобразования логического адреса в физический, сильно отличающаяся от используемой в реальном режиме. Эта схема даёт, в частности, возможность адресовать непосредственно до 16 мегабайт физической памяти.
- Для преобразования адреса процессор i80286 использует таблицы дескрипторов, в которых хранятся базовые адреса сегментов, размеры сегментов и другая информация.
- Одновременно процессор может использовать две таблицы дескрипторов – локальную (LDT) и глобальную (GDT). Используемая таблица определяется полем TI селектора.
- Расположение и размер таблицы GDT должны быть загружены в специальный регистр процессора – GDTR. Это можно сделать командой LGDT.
- В таблице GDT могут находиться дескрипторы сегментов кода, данных и системные дескрипторы. В частности, там может находиться дескриптор, описывающий таблицу локальных дескрипторов LDT (если эта таблица используется).
- Перед переключением в защищённый режим программа должна подготовить таблицу GDT и загрузить её адрес в регистр GDTR.
- Длина индекса в селекторе равна 13 разрядам, а размер сегмента равен  $2^{16}$  байтам. Таким образом, каждая задача, используя свою LDT, получает адресное пространство  $2^{29}$  байт, а в сумме с областью, определяемой GDT, эта величина составляет  $2^{30}$  байт или 1 Гбайт. Неплохо для 16-разрядного процессора.

## Защита в процессоре i80286

Как уже несколько раз упоминалось выше, МП i80286 мог работать в двух режимах (модах): реальном, в котором он фактически был более производительной версией i8086, с тем же набором регистров, немного расширенной системой команд<sup>1</sup>, с тем же ОЗУ размером до 1 Мбайт. Чтобы сделать i80286 в реальной моде практически полностью неотличимым от i8086, инженеры IBM заботливо добавили в схему PC/AT Gate A20, который принудительно занулял адресную линию A20, нейтрализуя (оказавшуюся очень полезной в итоге) ошибку разработчиков процессора i80286.

Работающая в реальной моде операционная система MS-DOS, использующая реальный режим работы процессора, абсолютно никак не защищена от прикладных программ. Подсистема управления памятью MS-DOS основана на разбиении всей памяти на участки, в начале которых помещаются блоки управления памятью MCB (Memory Control Block). При запуске программы MS-DOS выделяет ей необходимое количество блоков и загружает соответствующим образом сегментные регистры.

В итоге, просуществовав на рынке более полутора десятков лет, MS-DOS так и осталась операционной системой реального режима, с возможностью адресовывать максимум 1 Мбайт ОЗУ, несмотря на то, что интеловские процессоры, начиная с 286-го, поддерживали защищённый режим работы, в котором имели возможность адресовывать гигабайты физической памяти. При работе по MS-DOS доступ к ОЗУ выше 1 Мбайта предоставляла не операционная система, а прикладные программы, используя технологии т.н. «расширителей DOS<sup>2</sup>».

---

<sup>1</sup> Для реальной моды процессора i80286 была немного усовершенствованна система команд: добавлены инструкции PUSHB и POPB, команда умножения допускала непосредственный операнд, расширены возможности команд сдвигов и др.

<sup>2</sup> **Расширитель DOS** (*DOS extender*) — технология, позволяющая программам для операционных систем семейства DOS работать в защищённом режиме процессора. Существуют 16- и 32-битные расширители DOS:

Если программа не изменяет содержимое сегментных регистров, она будет работать только с теми сегментами памяти, которые выделены ей операционной системой. Однако ничто не может помешать любой программе загрузить в сегментные регистры любое значение, в том числе адреса сегментов самой операционной системы.

Поэтому, строго говоря, MS-DOS фактически не имеет сколько-нибудь надёжного механизма управления памятью – всё основано на выполнении программами «джентльменского соглашения» о неразрушении операционной системы.

Для повышения надёжности работы всей системы в целом необходимо запретить обычным пользовательским программам модифицировать области памяти, принадлежащие операционной системе. Если система работает одновременно с несколькими пользователями (многопользовательская операционная система), необходимо также закрыть программам пользователя доступ к памяти, распределённой другим пользователям. Иначе говоря, необходимо разделить адресные пространства операционной системы от пользовательских программ с одной стороны, и адресные пространства отдельных пользователей друг от друга, с другой стороны.

Такие операционные системы, как WINDOWS и OS/2, являются однопользовательскими, но многозадачными системами. Пользователь может запустить одновременно несколько программ. В этом случае было бы желательно разделить адресные пространства этих программ с целью исключения их влияния друг на друга (и, разумеется, на операционную систему). Аварийное завершение одной запущенной задачи не должно вызывать аварийного завершения остальных задач или, тем более, всей операционной системы.

Кроме того, необходимо разрешить модулям операционной системы доступ к памяти всех запущенных программ для выполнения эффективного управления работой всей системы.

Отсюда с одной стороны, вытекает идея наделения программ операционной системы некоторыми привилегиями по сравнению с пользовательскими программами, с другой стороны – идея наделения отдельных участков памяти средствами защиты от доступа к ним со стороны непривилегированных программ.

Многие мини-ЭВМ и большие ЭВМ реализуют схему «супервизор-пользователь». Операционная система работает в режиме «супервизор» и ей доступны все ресурсы компьютера – вся память, любые команды процессора и т.д. Запущенные программы работают в режиме «пользователь» и имеют доступ только к тем участкам памяти, которые выделены им операционной системой. Кроме того, эти программы ограничены в использовании некоторых команд процессора (например, команд ввода/вывода).

Для предотвращения доступа программам, работающим в режиме «пользователь» к чужим блокам памяти, используются так называемые «ключи» (это просто целые числа). Отдельным блокам памяти операционная система назначает свои «ключи». Кроме того, запущенные программы также снабжаются «ключами». Программа может иметь доступ только к таким блокам памяти, к которым подходит «ключ», имеющийся в распоряжении программы.

Такая схема не обеспечивает полного изолирования адресных пространств отдельных задач. Адресное пространство здесь одно, но для обычных программ разрешён доступ только к своему участку памяти в соответствии с имеющимся ключом. Однако в системе может работать несколько программ с одинаковым ключом памяти!

### ***Кольца защиты***

Процессор i80286 использует более гибкую и надёжную схему защиты операционной системы и программ друг от друга.

---

16-битные расширители предназначены для работы на процессорах Intel 80286 и выше, 32-битные — Intel 80386 и выше.

В этой схеме используются привилегии четырёх уровней – от 0 до 3. Самые большие привилегии соответствуют уровню Ring 0. Обычно такими привилегиями обладает ядро операционной системы. Минимальные привилегии у пользовательских программ – уровень Ring 3.

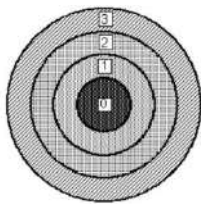


Рис.248 – Кольца защиты

Уровни привилегий часто называют кольцами защиты, см. Рис.248. Как пример, можно использовать такое распределение привилегий:

- Кольцо 0 – ядро операционной системы, системные драйверы.
- Кольцо 1 – программы обслуживания аппаратуры, драйверы, программы, работающие с портами ввода/вывода компьютера.
- Кольцо 2 – системы управления базами данных, расширения операционной системы.
- Кольцо 3 – прикладные программы, запускаемые пользователем.

Несложные системы могут использовать не все кольца, а только некоторые или даже одно. Например, можно расположить все программы операционной системы в кольце 0, а пользовательские программы – в кольце 3. Это, по сути вариант описанной выше схемы «супервизор-пользователь». Простейшие системы можно полностью реализовать в нулевом кольце.

Любой селектор имеет поле RPL (биты 0 и 1). В регистре CS хранится селектор текущего выполняемого сегмента кода. Этому селектору соответствует дескриптор в таблице GDT или LDT. В дескрипторе, в байте доступа располагается поле DPL (биты 5 и 6). Указанные поля участвуют в механизме защиты памяти.

Когда операционная система подготавливает программу для запуска, она формирует в GDT или LDT дескриптор, описывающий сегмент кода программы. В этом дескрипторе в поле DPL бита доступа проставляется номер кольца, в котором будет работать данная программа – текущий уровень привилегий CPL (Current Privilege Level). То есть возможности программы определяются содержимым поля DPL в байте доступа.

Текущий уровень привилегий копируется в поле RPL селектора сегмента кода, загруженного в регистре CS. Программа всегда может проанализировать свой текущий уровень привилегий исходя из значения поля RPL в регистре CS. Однако она не может изменить свой уровень привилегий простой заменой содержимого поля RPL в сегменте CS.

Итак, программа получает от операционной системы текущий уровень привилегий CPL, который она может проанализировать на основании содержимого регистра CS.

Дескрипторы, описывающие сегменты данных, содержат поле уровня привилегий дескриптора DPL (Descriptor Privilege Level). Поле DPL содержит минимальные привилегии, которые нужны для доступа к сегменту данных.

Перед тем, как обратиться к сегменту данных, программа должна загрузить в один из сегментных регистров селектор, соответствующий нужному сегменту данных. В селекторе необходимо указать поле уровня запрашиваемых привилегий RPL (Requested Privilege Level).

Программе будет предоставлен доступ к сегменту только в том случае, когда уровень привилегий дескриптора запрашиваемого сегмента DPL больше или равен значению  $\max(\text{CPL}, \text{RPL})$ , т.е. наибольшему из значений текущего уровня привилегий CPL и уровня запрашиваемых привилегий RPL:

$$\text{DPL} \geq \max(\text{CPL}, \text{RPL})$$

Если программа попытается получить доступ к более привилегированному, чем она сама сегменту памяти, её выполнение будет прервано.

### ***Границы сегментов***

Поле TYPE дескриптора определяет способ, которым можно использовать тот или иной сегмент. Разделение сегментов на типы позволяет защититься от случайного или преднамеренного использования сегментов не по назначению.

Сегмент кода может быть закрыт для чтения установкой бита R в байте доступа. Такие сегменты можно только выполнять, но нельзя читать. В сегмент кода нельзя записывать какие-либо данные. В регистр CS можно загружать только такие селекторы, которые относятся к сегментам кода.

Сегменты данных могут быть закрыты для записи установкой бита W в байте доступа. Сегменту данных нельзя передать управление, загрузив его селектор в регистр CS.

Дескрипторы некоторых типов, например, описывающих расположение таблицы LDT или сегмента состояния задачи, нельзя использовать для чтения или записи, даже если программа выполняется в нулевом приоритетном кольце. В случае такой необходимости следует создать дополнительные (алиасные) дескрипторы, в которых эти же сегменты описаны как сегменты данных.

### ***Границы сегментов***

Программы реального режима работают всегда с сегментами размером 64 Кбайт. Если программа состоит из нескольких сегментов, то некоторые сегменты могут перекрываться. Существует потенциальная опасность, что в результате программной ошибки (например, выхода индекса массива за допустимые пределы) произойдёт запись в другой сегмент.

Процессор i80286 позволяет создавать сегменты любого размера в пределах 64 килобайт (процессоры i80386 и i80486 могут работать с сегментами размером 4 Гбайт). Кроме того, он следит за тем, чтобы при адресации памяти не происходил выход за границы сегмента.

Границы сегмента задаются полем предела в дескрипторе сегмента, значение этого поля должно быть равно размеру сегмента в байтах минус единица.

Интерпретация поля предела зависит от состояния бита D поля доступа. Для сегментов стеков необходимо устанавливать поле D равным 1. В этом случае попытка записи в переполненный стек вызовет прерывание программы. В ответ на это прерывание операционная система может, например, выделить дополнительную память для стека.

В реальном режиме переполнение стека никак не контролируется и может привести к разрушению самой программы или операционной системы.

### ***Привилегированные и чувствительные команды***

Для обеспечения надёжности работы операционной системы необходимо ограничить использование обычными программами некоторых команд процессора. Например, обычные программы не должны иметь доступа к командам загрузки системных регистров GDTR и LDTR.

В таблице на Рис.249 приведён список привилегированных команд, которые могут выполняться только в нулевом приоритетном кольце, т.е. только теми программами, которые имеют наибольшие привилегии (CPL=0).

| Команда | Выполняемые функции                                    |
|---------|--------------------------------------------------------|
| LGDT    | Загрузка регистра глобальной таблицы дескрипторов GDTR |
| LLDT    | Загрузка регистра локальной таблицы дескрипторов LDTR  |
| LIDT    | Загрузка регистра таблицы дескрипторов прерываний IDTR |
| LTR     | Загрузка регистра задачи TR                            |
| LMSW    | Загрузка слова состояния машины MSW                    |
| CLTS    | Сброс флага переключения задачи                        |
| HLT     | Останов процессора                                     |

**Рис.249 - Привилегированные команды процессора i80286**

Существуют команды, которые должны выполняться не только в нулевом кольце, но использование которых должно быть запрещено для программ, имеющих уровень привилегий ниже некоторого, заданного для всей системы в целом. Кроме того, функции некоторых команд было бы желательно модифицировать в соответствии с привилегиями выполняющей их программы. Такие команды называются чувствительными.

Кроме привилегированных команд, необходимо ограничить обычные программы в использовании команд ввода/вывода: IN, OUT, INS, OUTS. С помощью этих команд программа может, например, перепрограммировать контроллеры прерываний и прямого доступа к памяти, что в свою очередь откроет путь к непосредственной модификации памяти по физическим адресам.

Также не следует разрешать обычным программам использовать команды, которые могут заблокировать прерывания: CLI, STI, LOCK. В мультизадачной среде обычные программы не должны иметь возможность отключать механизм разделения времени между задачами.

Регистр FLAGS флагов процессора i80286 в разрядах 12-13 содержит двухбитовое поле IOPL (Input/Output Privilege Level). Это поле определяет наименее привилегированное кольцо, в котором разрешено использовать команды ввода/вывода.

Программы, работающие не в нулевом кольце, не могут сами модифицировать поле IOPL регистра FLAGS. При выполнении команд, загружающих регистр флажков (IRET и POPF) не в нулевом кольце, поле IOPL не модифицируется. Аналогично, без изменения остаётся флаг разрешения прерываний IF.

### ***Межсегментная передача управления***

В реальном режиме передача управления выполняется с помощью команд JMP, CALL, INT, RET, IRET, а также при возникновении прерываний. Внутрисегментная передача управления выполняется командами JMP, CALL, RET, а межсегментная передача управления – командами JMP, CALL, INT, RET, IRET и в случае возникновения прерываний.

В защищённом режиме всё происходит аналогично. При внутрисегментной передаче управления в регистр IP заносится новое значение, а регистр CS не модифицируется. Межсегментная передача подразумевает одновременное изменение регистров CS и IP, а также в некоторых случаях и регистра флагов FLAGS (прерывания и команды RET, IRET).

Операционная система MS-DOS, работающая в реальном режиме, позволяет любой программе вызывать любые подпрограммы. Единственное условие для успешного вызова подпрограммы – знание адреса подпрограммы (сегмента и смещения) и формата передаваемых регистров.

При проектировании защищённых систем необходимо предотвратить возможность непосредственного вызова произвольных привилегированных подпрограмм менее привилегированными. С другой стороны, необходимо обеспечить существование механизма вызова непривилегированной программой модулей операционной системы (привилегированных) для получения обслуживания. Например, для того чтобы узнать текущее время или

выполнить обращение к магнитному диску непривилегированная программа должна вызывать соответствующие модули операционной системы.

Процессор i80286 содержит все средства, необходимые для организации с одной стороны, защиты модулей операционной системы от несанкционированного вызова прикладными программами и, с другой стороны, для обслуживания модулями операционной системы вызовов прикладных программ.

## **Команды CALL и JMP**

При использовании команд CALL или JMP для внутрисегментной передачи управления в качестве операнда для команд необходимо указывать смещение модуля в текущем сегменте, которому будет передано управление. При передаче управления процессор проверяет смещение на предмет выхода за пределы текущего сегмента кода. В случае использования этих команд для межсегментной передачи управления существуют две возможности.

Во-первых, программист может указать в качестве операнда селектор дескриптора вызываемого сегмента кода.

Во-вторых, программист может указать в качестве операнда селектор, которому соответствует дескриптор специального типа – вентиль вызова. Вентиль вызова содержит логический (а не физический, как дескриптор сегмента) адрес вызываемого модуля.

Первый способ называется прямым вызовом, второй – вызовом через вентиль (шлюз, gate) вызова.

В случае прямого вызова операнд команды вызова – селектор дескриптора вызываемого или, другими словами, целевого сегмента.

На выполнение прямого вызова целевого сегмента или прямого перехода к целевому сегменту влияет бит подчинения C, который располагается в бите 2 байта доступа дескриптора целевого сегмента.

Если бит C установлен в 0, целевой сегмент называется несогласованным. Несогласованный сегмент может быть вызван только такой программой, которая имеет большие или такие же привилегии, что и целевой сегмент. Т.е. должно выполняться условие  $CPL \leq DPL$ .

Обычная программа, выполняющаяся в кольце 3, не может вызывать несогласованный сегмент (или передавать управление несогласованному сегменту), находящемуся в кольцах 0, 1 или 2. Этот механизм блокирует несанкционированный вызов модулей операционной системы программами пользователя.

Однако должен существовать способ безопасного вызова модулей операционной системы в заранее оговоренных точках для того, чтобы программы могли получать обслуживание от привилегированных модулей ядра операционной системы. Прямой вызов несогласованного сегмента в этом случае не подходит, так как ядро располагается в нулевом кольце.

Если программный сегмент, располагающийся в ядре операционной системы, должен вызываться как самой операционной системой в нулевом кольце, так и программами пользователя в менее привилегированных кольцах, можно применить согласованный сегмент.

В согласованном сегменте кода бит подчинения C байта доступа установлен в 1. Согласованный сегмент можно вызывать из программ, находящихся в любом кольце. Но в любом случае при вызове этот сегмент будет выполняться с привилегиями вызывающей программы. Если согласованный сегмент вызывается из кольца 0, он будет выполняться с уровнем привилегий 0, если из кольца 3 – с уровнем привилегий 3.

Другой способ выполнить межсегментную передачу управления – использовать передачу управления через вентиль вызова.

В этом случае команды CALL или JMP адресуются к дескриптору с типом вентиля вызова. Формат этого дескриптора показан на Рис.250:



Рис.250 – Дескриптор вентилей вызова

Селектор и смещение представляют собой адрес вызова или передачи управления в целевом сегменте. Поле счётчика слов используется при передаче параметров вызываемому модулю.

Поле DPL вентилей вызова указывает минимальный уровень привилегий, необходимый для получения доступа через данный вентиль. Операционная система может подготовить для программ пользователя вентили, доступные из кольца 3 и обеспечивающие вызов процедур обслуживания, располагающихся в ядре операционной системы в кольце 0. Однако программы 3 кольца смогут обращаться только к таким вентилям вызова, которые содержат в поле DPL значение 3. При этом управление через вентиль вызова передаётся только в ту точку, которая описана в данном вентиле и является безопасной с точки зрения операционной системы. Программа пользователя не сможет передать управление в середину модуля или в середину машинной команды.

Но есть ещё проблема с передачей параметров. Обычно передача параметров выполняется через стек. Но для обеспечения защиты программы нулевого кольца используют отдельный стек (вообще говоря, для каждого кольца используется свой отдельный стек).

При вызове сегмента через вентиль вызова процессор копирует из стека вызывающей программы в стек целевого сегмента такое количество 16-битовых слов, которое указано в поле 5-разрядного счётчика слов вентилей вызова. Таким способом вызываемому модулю можно передать до 31 параметра.

Вентиль вызова – идеальное средство для предоставления обычным программам доступа к сервисам операционной системы. С одной стороны, вентиль вызова, формируемый операционной системой, позволяет непривилегированным программам передавать управление привилегированным кодовым сегментам для получения необходимого обслуживания (сервиса). С другой стороны, операционная система имеет возможность держать под своим контролем использование вентилей вызова, задавая в поле DPL вентилей минимальный уровень привилегий, которыми должна обладать программа, передающая управление через этот вентиль, а также безопасный адрес входа в целевом сегменте.

## Команды RET и IRET

Команды RET и IRET предназначены для возврата из подпрограмм и процедур обработки прерываний, соответственно.

Команда RET может возвращать управление в пределах одного сегмента (внутрисегментная форма) или в другой сегмент (межсегментная форма).

При выполнении внутрисегментной формы команды RET процессор выполняет проверку превышения границы текущего сегмента, не позволяя передавать управление за его пределы.

Когда выполняется межсегментная команда RET или команда IRET, процессор проверяет привилегии селектора сегмента программы, выбираемого из стека (то есть селектора сегмента, которому будет передано управление при выполнении команд RET или IRET).



Процессор может выполнить возврат только в менее привилегированный сегмент или в сегмент, обладающий такими же привилегиями, что и тот, из которого выполняется возврат.

## **Виртуальная память в процессоре i80286**

Процессор i80286 может непосредственно адресовать до 16 Мбайт физической памяти, однако реально компьютеры тех лет редко имели оперативную память такого размера. Обычный размер оперативной памяти для IBM PC/AT (Extended Memory) составлял 2-4 Мбайт.

Механизм виртуальной памяти, реализованный в процессоре i80286, позволяет организовать память большого размера (например, 16 Мбайт) с использованием относительно небольшой физической оперативной памяти и дисковой памяти.

Основная идея виртуальной памяти заключается в том, чтобы хранить (и обновлять) содержимое большой виртуальной памяти в её копии на диске, подкачивая отдельные участки виртуальной памяти в реальную оперативную память по необходимости.

Можно, например, хранить все используемые программой сегменты на диске, а в физическую память записывать только те сегменты, которые необходимы для выполнения программы в настоящий момент. Процесс «подкачки» сегментов с диска в память и их выгрузки после изменения на диск называется свопингом (swapping).

Операционная система должна вести учёт сегментов и знать, какие сегменты находятся в памяти, а какие – на диске. Процессор i80286 может оказать ей в этом существенную помощь. Два поля дескриптора – бит присутствия сегмента в памяти P и бит обращения к сегменту памяти A – предназначены для аппаратной реализации учёта сегментов.

Бит P должен быть установлен в 1 для тех сегментов, которые находятся в физической памяти. Сегменты, временно отсутствующие в памяти и находящиеся на диске, помечаются в дескрипторе битом P=0.

Установкой и сбросом бита P занимается операционная система. А вот проверка этого бита – работа для процессора. Когда программа обращается к отсутствующему в физической памяти сегменту (загрузкой селектора в сегментный регистр), выполнение программы прерывается и управление передаётся операционной системе. Та, в свою очередь, подкачивает нужный сегмент в оперативную память и устанавливает для него бит P в 1, после чего работа программы возобновляется.

Время от времени операционная система должна находить сегменты, к которым было обращение, и в случае их изменения сбрасывать на диск. Кроме того, если для закладки нового сегмента в физической памяти недостаточно свободного места, можно выгрузить самые старые сегменты из физической памяти на диск и на их место загрузить новый сегмент.

Процессор может оказать помощь операционной системе в определении тех сегментов, к которым было обращение. Для этих сегментов бит обращения A устанавливается процессором в 1. Сбросить бит обращения можно только из программы, поэтому такая работа возлагается на саму операционную систему.

Реально описанная выше схема реализации виртуальной памяти со свопингом сегментов используется в операционной системе OS/2 версий от 1.0 до 1.3 включительно.

Однако у этой схемы есть один очень серьёзный недостаток. Так как все сегменты имеют разные размеры, и все они подкачиваются по очереди (и по несколько штук) в одну область физической памяти, возможно возникновение фрагментации физической памяти. Например, был удалён сегмент размером в 10 Кбайт, и образовался свободный участок физической памяти такого же размера. Новый сегмент, который требуется загрузить в память, имеет размер 15 Кбайт, и он не поместится на место старого.

Операционная система может выполнить перемещение сегментов в физической памяти, изменив соответствующим образом 24-битовые базовые адреса сегментов в таблицах дескрипторов. После перемещения сегментов можно объединить все свободные участки памяти в один и использовать этот участок для загрузки нового сегмента. Так как программы

защищённого режима не знают физических адресов памяти (они работают только с селекторами), перемещение сегментов никак не отразится на их работе<sup>1</sup>.

Но на перемещение сегментов требуется время, и кроме того, часто нужен не весь сегмент, а только его небольшой участок. Процессор i80286 не предлагает никакого решения проблемы фрагментации физической памяти при свопинге сегментов, и не позволяет загружать сегмент по частям.

Все эти возможности имеются только в процессорах i80386 и i80486, в которых был добавлен механизм страничного преобразования памяти, в дополнение к сохранённому (для совместимости) сегментному механизму. Только эти процессоры (и новее) позволяют эффективно реализовать полноценный механизм виртуальной памяти.

## Система команд i80286

В целях экономии места, в этом разделе не будет приводиться весь список команд процессора i80286. Базовый набор инструкций i8086 был расширен 16-ю новыми командами, также была модифицирована работа части «старых» инструкций. Процессор стал гораздо эффективнее в расходе тактов, теперь инструкции в нём выполнялись в среднем за 4,5 такта вместо 10-15 тактов в i8086/88, в результате работавший на частоте 12,5 МГц 286-ой более чем в шесть раз превосходил по производительности 5 МГц 86-ой. Такого прироста производительности при переходе к следующим поколениям процессоров, в пересчёте на мегагерц, Intel-у никогда больше достичь не удалось.

Ниже будет приведено описание только для некоторых модифицированных и новых дополнительных инструкций процессора i80286. Напомним, что к 14 доступным регистрам процессора 8086 в 286-ом было добавлено пять новых программно доступных регистров и шесть «скрытых»:

- MSW – 16-битный регистр слова состояния машины. Если в MSW флаг PE=1, то процессор переключается в защищённый режим.
- TR – 16-битный регистр задачи, содержит селектор сегмента состояния задачи, используется при работе в многозадачном режиме.
- GDTR – 40-разрядный регистр, определяет размер и положение глобальной дескрипторной таблицы в ОЗУ.
- LDTR – 16-разрядный регистр, определяет базовый адрес локальной дескрипторной таблицы.
- IDTR – 40-разрядный регистр определяет начало и размер таблицы векторов прерываний;
- 6 «невидимых» регистров расширения сегментных регистров, связанных с CS, DS, ES, SS, GDTR, IDTR.

## Модифицированные инструкции

**IMUL** – умножение знаковое, в команде стало возможным использование непосредственных операндов.

---

<sup>1</sup> Но не может не отразиться на работе подсистемы DMA – сегмент нельзя перемещать в памяти, если на момент перемещения есть активные операции прямого доступа к памяти, работающие с этим же участком ОЗУ. Отслеживать эту возможную коллизию должна, конечно же, операционная система, т.к. именно она иницирует регистры контроллеров DMA, назначая им базовые адреса и счётчики.

| IMUL            |           | Умножение знаковое<br>Signed multiplication |          |                |                |               |                |                   |
|-----------------|-----------|---------------------------------------------|----------|----------------|----------------|---------------|----------------|-------------------|
| Переп<br>О<br>Р | Напр<br>D | Прер<br>I                                   | Шаг<br>T | Знак<br>S<br>H | Нуль<br>Z<br>H | Всп<br>A<br>H | Четн<br>P<br>H | Перенос<br>C<br>P |

| Код      | Команда              | Такты                         |                              |                               |                                    | Краткое описание                         |                                             |  |
|----------|----------------------|-------------------------------|------------------------------|-------------------------------|------------------------------------|------------------------------------------|---------------------------------------------|--|
| F6 /5    | IMUL r/m8            | <u>486</u><br>13-18/<br>13-18 | <u>386</u><br>9-14/<br>12-17 | <u>286</u><br>13/<br>16       | <u>8086</u><br>80-98/<br>86-104+IA | AL на r/m-байт, рез. в AX                |                                             |  |
| F7 /5    | IMUL r/m16           | 13-26/<br>13-26               | 9-22/<br>12-25               | 21/<br>24                     | 128-154/<br>134-160+IA             | AX на r/m-слово, рез. в DX:AX            |                                             |  |
| F7 /5    | IMUL r/m32           | 12-42/<br>13-42               | 9-38/<br>12-41               |                               |                                    | EAX на r/m-двойное слово, рез. в EDX:EAX |                                             |  |
| 0F AF /r | IMUL r16,r/m16       |                               |                              | <u>486</u><br>13-26/<br>13-26 | <u>386</u><br>9-22/<br>12-25       | <u>286</u>                               | r16 на r/m16, рез. в r16                    |  |
| 0F AF /r | IMUL r32,r/m32       |                               |                              | 13-42/<br>13-42               | 9-38/<br>12-41                     |                                          | r32 на r/m32, рез. в r32                    |  |
| 6B /r ib | IMUL r16,r/m16,imm8  |                               |                              | 13-26/<br>13-26               | 9-14/<br>12-17                     | 21/24                                    | r/m16 на знакорасширенный imm8, рез. в r16  |  |
| 6B /r ib | IMUL r32,r/m32,imm8  |                               |                              | 13-42/<br>13-42               | 9-14/<br>12-17                     |                                          | r/m32 на знакорасширенный imm8, рез. в r32  |  |
| 6B /r ib | IMUL r16,imm8        |                               |                              | 13-26/<br>13-26               | 9-14/<br>12-17                     | 21/24                                    | r16 на знакорасширенный imm8, рез. в r16    |  |
| 6B /r ib | IMUL r32,imm8        |                               |                              | 13-42/<br>13-42               | 9-14/<br>12-17                     |                                          | r32 на знакорасширенный imm8, рез. в r32    |  |
| 69 /r iw | IMUL r16,r/m16,imm16 |                               |                              | 13-26/<br>13-26               | 9-22/<br>12-25                     | 21/24                                    | r/m16 на знакорасширенный imm16, рез. в r16 |  |
| 69 /r id | IMUL r32,r/m32,imm32 |                               |                              | 13-42/<br>13-42               | 9-38/<br>12-41                     |                                          | r/m32 на знакорасширенный imm32, рез. в r32 |  |
| 69 /r iw | IMUL r16,imm16       |                               |                              | 13-26/<br>13-26               | 9-22/<br>12-25                     |                                          | r16 на знакорасширенный imm16, рез. в r16   |  |
| 69 /r id | IMUL r32,imm32       |                               |                              | 13-42/<br>13-42               | 9-38/<br>12-41                     |                                          | r32 на знакорасширенный imm32, рез. в r32   |  |

Выполняется знаковое умножение. Действия зависят от размера и количества операндов. Например, если задан один операнд:

- операнд байт умножается на AL, результат остается в AX. AH содержит 7 старших битов произведения. Флаги Переноса CF и Переполнения OF получают значение 0, если все биты AH равны знаковому биту AL (т.е. результат полностью разместился в AL), иначе значение 1;
- операнд слово умножается на AX, результат остается в DX:AX. DX содержит 15 старших битов произведения. Флаги Переноса CF и Переполнения OF получают значение 0, если все биты DX равны знаковому биту AX (т.е. результат полностью разместился в AX), иначе значение 1;
- операнд двойное слово умножается на EAX, результат остается в EDX:EAX. EDX содержит 31 старший бит произведения. Флаги Переноса CF и Переполнения OF получают значение 0, если все биты EDX равны знаковому биту EAX (т.е. результат полностью разместился в EAX), иначе значение 1.

Действие команды в других случаях, в том числе, когда указано 2 или 3 операнда, приведены в графе «Краткое описание».

Если после выполнения команды HE выполнено ни одно из следующих условий:

| Формат операндов | Условие для равенства нулю CF и OF         |
|------------------|--------------------------------------------|
| r/m8             | AX = знаковому расширению AL до 16 бит     |
| r/m16            | DX:AX = знаковому расширению AX до 32 бит  |
| r/m32            | EDX:EAX=знаковому расширению EAX до 64 бит |
| r16,r/m16        | Результат имеет размер точно r16           |
| r32,r/m32        | Результат имеет размер точно r32           |
| r16,r/m16,imm16  | Результат имеет размер точно r16           |
| r32,r/m32,imm32  | Результат имеет размер точно r32,          |

то флаги Переполнения OF и Переноса CF получают значение 1, в противном случае они получают значение 0.

**PUSHA** (работа со стеком) – сохранение всех регистров общего назначения в стеке.

**PUSHA** Сохранение регистров общего назначения в стеке

**PUSHAD** Push all general registers

PUSHA — только для 80286/i386/i486

PUSHAD — только для i386 и i486

| Переп<br>О | Напр<br>D | Прер<br>I | Шаг<br>T | Знак<br>S | Нуль<br>Z | Всп<br>A | Четн<br>P | Перенос<br>C |
|------------|-----------|-----------|----------|-----------|-----------|----------|-----------|--------------|
|------------|-----------|-----------|----------|-----------|-----------|----------|-----------|--------------|

| Код | Команда | Такты            |                  |                  |  | Краткое описание                                  |  |  |
|-----|---------|------------------|------------------|------------------|--|---------------------------------------------------|--|--|
| 60  | PUSHA   | <u>486</u><br>11 | <u>386</u><br>18 | <u>286</u><br>17 |  | Регистры общего назначения (слова) в стек         |  |  |
| 60  | PUSHAD  | 11               | 18               |                  |  | Регистры общего назначения (двойные слова) в стек |  |  |

Команда PUSH(A/D) с охраняет в стеке 8 регистров: (E)AX, (E)CX, (E)DX, (E)BX, (E)SP, (E)BP, (E)SI и (E)DI. Сохраненное значение (E)SP такое же, как и до выполнения PUSH(A/D).

**POPA** (работа со стеком) – восстановление всех регистров общего назначения из стека.

|              |           |                                                     |          |           |                                                 |          |           |              |  |
|--------------|-----------|-----------------------------------------------------|----------|-----------|-------------------------------------------------|----------|-----------|--------------|--|
| <b>POPA</b>  |           | Восстановление регистров общего назначения из стека |          |           |                                                 |          |           |              |  |
| <b>POPAD</b> |           | Pop all general registers                           |          |           |                                                 |          |           |              |  |
|              |           | POPA — только для 80286/i386/i486                   |          |           |                                                 |          |           |              |  |
|              |           | POPAD — только для i386 и i486                      |          |           |                                                 |          |           |              |  |
| Переп<br>О   | Напр<br>D | Прер<br>I                                           | Шаг<br>T | Знак<br>S | Нуль<br>Z                                       | Всп<br>A | Четн<br>P | Перенос<br>C |  |
| Код          | Команда   | Такты                                               |          |           | Краткое описание                                |          |           |              |  |
|              |           | 486                                                 | 386      | 286       |                                                 |          |           |              |  |
| 61           | POPA      | 9                                                   | 24       | 19        | Регистры общего назначения (слова) из стека     |          |           |              |  |
| 61           | POPAD     | 9                                                   | 24       |           | Регистры общего назначения (дв. слова) из стека |          |           |              |  |

Команда POP(A/D) восстанавливает из стека значения 7 регистров — (E)DI, (E)SI, (E)BP, (E)BX, (E)DX, (E)CX и (E)AX. Восьмой регистр — (E)SP — не восстанавливается и соответствующее значение в стеке отбрасывается. Действие POP(A/D) обратно команде PUSH(A/D), сохраняющей 8 регистров.

## Дополнительные инструкции

**LGDT** (адресация таблиц дескрипторов) – загрузка регистра GDTR

|             |             |                                       |          |           |                      |          |           |              |  |
|-------------|-------------|---------------------------------------|----------|-----------|----------------------|----------|-----------|--------------|--|
| <b>LGDT</b> |             | Загрузка регистра GDTR                |          |           |                      |          |           |              |  |
|             |             | Load global descriptor table register |          |           |                      |          |           |              |  |
|             |             | Только для 80286/i386/i486            |          |           |                      |          |           |              |  |
| Переп<br>О  | Напр<br>D   | Прер<br>I                             | Шаг<br>T | Знак<br>S | Нуль<br>Z            | Всп<br>A | Четн<br>P | Перенос<br>C |  |
| Код         | Команда     | Такты                                 |          |           | Краткое описание     |          |           |              |  |
|             |             | 486                                   | 386      | 286       |                      |          |           |              |  |
| 0F 01 /2    | LGDT m16&32 | 11                                    | 11       | 12        | Запись m16&32 в GDTR |          |           |              |  |

Эта команда загружает из 6-байтового операнда в памяти 32 или 24 битовый линейный адрес и 16 битовый предельный размер глобальной таблицы дескрипторов в регистр GDTR (-1). Если аргумент команды адрес 6-байтового операнда в памяти является 16-битовым адресом, то загружается 24-битовый линейный адрес (в соответствии с архитектурой 80286), а если 32, то 32-битовый. При этом оставшиеся старшие 8 бит не используются (нули).

LGDT используется в программном обеспечении операционных систем, а не в прикладных программах. Только LGDT и LIDT непосредственно работают с линейным адресом в защищенном режиме.

**LIDT** (управление прерываниями) – загрузка регистра IDTR

|             |             |                                          |          |           |                  |          |           |              |  |
|-------------|-------------|------------------------------------------|----------|-----------|------------------|----------|-----------|--------------|--|
| <b>LIDT</b> |             | Загрузка регистра IDTR                   |          |           |                  |          |           |              |  |
|             |             | Load interrupt descriptor table register |          |           |                  |          |           |              |  |
|             |             | Только для 80286/i386/i486               |          |           |                  |          |           |              |  |
| Переп<br>О  | Напр<br>D   | Прер<br>I                                | Шаг<br>T | Знак<br>S | Нуль<br>Z        | Всп<br>A | Четн<br>P | Перенос<br>C |  |
| Код         | Команда     | Такты                                    |          |           | Краткое описание |          |           |              |  |
|             |             | 486                                      | 386      | 286       |                  |          |           |              |  |
| 0F 01 /3    | LIDT m16&32 | 11                                       | 11       | 12        | IDTR:=m16&32     |          |           |              |  |

Эта команда загружает из 6 байтового операнда в памяти 32- или 24-битовый линейный адрес и 16-битовый предельный размер таблицы дескрипторов прерываний в регистр IDTR (-1). Если аргумент команды адрес 6-байтового операнда в памяти является 16-битовым адресом, то загружается 24-битовый линейный адрес (в соответствии с архитектурой 80286), а если 32, то 32-битовый. При этом остающиеся старшие 8 бит не используются (нули).

LIDT используется в программном обеспечении операционных систем, а не в прикладных программах. Только LIDT и LGDT непосредственно работают с линейным адресом в защищенном режиме.

**LLDT** (адресация таблиц дескрипторов) – загрузка регистра LDTR

## LLDT

Загрузка регистра LDTR

Load local descriptor table register

Только для защищенного режима 80286/i386/i486

|            |           |           |          |           |           |          |           |              |
|------------|-----------|-----------|----------|-----------|-----------|----------|-----------|--------------|
| Переп<br>О | Напр<br>D | Прер<br>I | Шаг<br>T | Знак<br>S | Нуль<br>Z | Всп<br>A | Четн<br>P | Перенос<br>C |
|------------|-----------|-----------|----------|-----------|-----------|----------|-----------|--------------|

| Код      | Команда    | Такты               | Краткое описание |                     |                     |  |  |  |
|----------|------------|---------------------|------------------|---------------------|---------------------|--|--|--|
| 0F 00 /2 | LLDT r/m16 | <u>486</u><br>11/11 | <u>386</u><br>20 | <u>286</u><br>17/19 | Запись r/m16 в LDTR |  |  |  |

Команда загружает значение в регистр локальной таблицы дескрипторов LDTR. Операнд команды должен содержать селектор глобальной таблицы дескрипторов (GDT), которая станет локальной. В результате в LDTR загружается адрес таблицы. Команда не меняет значения ни сегментных регистров, ни поля LDT в сегменте состояния задачи.

Операндом команды может быть число 0. Тогда локальная таблица дескрипторов считается неподключенной и любые команды работы с ней приводят к генерации исключения #GP.

LLDT используется в программном обеспечении операционных систем, а не в прикладных программах.

## LMSW (управление процессором) – загрузка слова состояния

### LMSW

Загрузка слова состояния

Load machine status word

Только для защищенного режима 80286/i386/i486

|            |           |           |          |           |           |          |           |              |
|------------|-----------|-----------|----------|-----------|-----------|----------|-----------|--------------|
| Переп<br>О | Напр<br>D | Прер<br>I | Шаг<br>T | Знак<br>S | Нуль<br>Z | Всп<br>A | Четн<br>P | Перенос<br>C |
|------------|-----------|-----------|----------|-----------|-----------|----------|-----------|--------------|

| Код      | Команда    | Такты               | Краткое описание    |                   |            |  |  |  |
|----------|------------|---------------------|---------------------|-------------------|------------|--|--|--|
| 0F 01 /6 | LMSW r/m16 | <u>486</u><br>13/13 | <u>386</u><br>11/14 | <u>286</u><br>3/6 | MSW:=r/m16 |  |  |  |

LMSW загружает в слово состояния процессора (часть регистра CR0) значение операнда. Эта команда может быть использована для переключения в защищенный режим. В этом случае она должна предваряться внутрисегментным переходом для очистки очереди команд. Однако с помощью LMSW нельзя переключиться обратно в реальный режим<sup>1</sup>.

LMSW используется в программном обеспечении операционных систем, а не в прикладных программах.

## LTR (многозадачность) – загрузка регистра TR

### LTR

Загрузка регистра TR

Load task register

Только для защищенного режима 80286/i386/i486

|            |           |           |          |           |           |          |           |              |
|------------|-----------|-----------|----------|-----------|-----------|----------|-----------|--------------|
| Переп<br>О | Напр<br>D | Прер<br>I | Шаг<br>T | Знак<br>S | Нуль<br>Z | Всп<br>A | Четн<br>P | Перенос<br>C |
|------------|-----------|-----------|----------|-----------|-----------|----------|-----------|--------------|

| Код      | Команда   | Такты               | Краткое описание       |                     |                   |  |  |  |
|----------|-----------|---------------------|------------------------|---------------------|-------------------|--|--|--|
| 0F 00 /3 | LTR r/m16 | <u>486</u><br>20/20 | <u>386</u><br>pm=23/27 | <u>286</u><br>17/19 | Запись r/m16 в TR |  |  |  |

Команда LTR загружает в регистр задачи (TR task register) селектор сегмента состояния задачи (TSS task state segment), указанный в операнде. При этом сегмент состояния задачи помечается занятым. Переключение задач не происходит.

LTR используется в программном обеспечении операционных систем, а не в прикладных программах.

## SGDT (адресация таблиц дескрипторов) – запись регистра GDTR

### SGDT

Запись регистра GDTR

Store global descriptor table

Только для защищенного режима 80286/386/486

|            |           |           |          |           |           |          |           |              |
|------------|-----------|-----------|----------|-----------|-----------|----------|-----------|--------------|
| Переп<br>О | Напр<br>D | Прер<br>I | Шаг<br>T | Знак<br>S | Нуль<br>Z | Всп<br>A | Четн<br>P | Перенос<br>C |
|------------|-----------|-----------|----------|-----------|-----------|----------|-----------|--------------|

| Код      | Команда | Такты            | Краткое описание |                  |                 |  |  |  |
|----------|---------|------------------|------------------|------------------|-----------------|--|--|--|
| 0F 01 /0 | SGDT m  | <u>486</u><br>10 | <u>386</u><br>9  | <u>286</u><br>11 | Запись GDTR в m |  |  |  |

<sup>1</sup> Об этой особенности процессора i80286 будет подробнее рассказано дальше, в разделе «Программное обеспечение».

Команда SGDT записывает 6 байт значения регистра глобальной таблицы дескрипторов по адресу, указанному в операнде. Первые 2 байта задают размер таблицы (-1). Если процессор находится в 16-разрядном режиме, то следующие 3 байта задают 24-битовый адрес таблицы и последний старший байт равен 0. В случае 32-битовых слов все 4 байта задают линейный адрес таблицы.

SGDT используется в программном обеспечении операционной системы, но не в прикладных программах.

### SIDT (управление прерываниями) – запись регистра IDTR

| SIDT       |           | Запись регистра IDTR<br>Store interrupt descriptor table<br>Только для защищенного режима 80286/i386/i486 |          |                  |                 |                  |                 |              |
|------------|-----------|-----------------------------------------------------------------------------------------------------------|----------|------------------|-----------------|------------------|-----------------|--------------|
| Переп<br>О | Напр<br>D | Прер<br>I                                                                                                 | Шаг<br>T | Знак<br>S        | Нуль<br>Z       | Всп<br>A         | Четн<br>P       | Перенос<br>C |
| Код        |           | Команда                                                                                                   |          | Такты            |                 | Краткое описание |                 |              |
| 0F 01 /1   |           | SIDT m                                                                                                    |          | <u>486</u><br>10 | <u>386</u><br>9 | <u>286</u><br>12 | Запись IDTR в m |              |

Команда SIDT записывает 6 байт значение регистра дескриптора таблицы прерываний по адресу, указанному в операнде. Первые 2 байта получают значение размера таблицы (поле Limit регистра IDTR). Если процессор находится в 16-разрядном режиме, то следующие 3 байта получают 24-битовый адрес таблицы и последний (старший) байт не определен. В случае 32-разрядного режима все 4 байта получают 32-битовый линейный адрес таблицы.

SIDT используется в программном обеспечении операционных систем, а не в прикладных программах.

### SLDT (адресация таблиц дескрипторов) – запись регистра LDTR

| SLDT       |           | Запись регистра LDTR<br>Store local descriptor table register<br>Только для защищенного режима 80286/i386/i486 |          |                   |                      |                   |                 |              |
|------------|-----------|----------------------------------------------------------------------------------------------------------------|----------|-------------------|----------------------|-------------------|-----------------|--------------|
| Переп<br>О | Напр<br>D | Прер<br>I                                                                                                      | Шаг<br>T | Знак<br>S         | Нуль<br>Z            | Всп<br>A          | Четн<br>P       | Перенос<br>C |
| Код        |           | Команда                                                                                                        |          | Такты             |                      | Краткое описание  |                 |              |
| 0F 00 /0   |           | SLDT r/m16                                                                                                     |          | <u>486</u><br>2/3 | <u>386</u><br>pm=2/2 | <u>286</u><br>2/3 | Запись LDTR в m |              |

Команда SLDT записывает значение регистра локальной таблицы дескрипторов (LDTR) в двухбайтовый операнд. LDTR является селектором, указывающим на глобальную таблицу дескрипторов.

SLDT используется в программном обеспечении операционных систем, а не в прикладных программах.

### SMSW (управление процессором) – запись слова состояния

| SMSW       |           | Запись слова состояния<br>Store machine status word<br>Только для защищенного режима 80286/i386/i486 |          |                   |                           |                   |            |              |
|------------|-----------|------------------------------------------------------------------------------------------------------|----------|-------------------|---------------------------|-------------------|------------|--------------|
| Переп<br>О | Напр<br>D | Прер<br>I                                                                                            | Шаг<br>T | Знак<br>S         | Нуль<br>Z                 | Всп<br>A          | Четн<br>P  | Перенос<br>C |
| Код        |           | Команда                                                                                              |          | Такты             |                           | Краткое описание  |            |              |
| 0F 01 /4   |           | SMSW r/m16                                                                                           |          | <u>486</u><br>2/3 | <u>386</u><br>2/3, pm=2/2 | <u>286</u><br>2/3 | r/m16:=MSW |              |

Команда SMSW записывает слово состояния (machine status word часть регистра CR0) в операнд.

### STR (многозадачность) – запись регистра TR



## STR

Запись регистра TR

Store task register

Только для защищенного режима 80286/i386/i486

|       |      |      |     |      |      |     |      |         |
|-------|------|------|-----|------|------|-----|------|---------|
| Переп | Напр | Прер | Шаг | Знак | Нуль | Всп | Четн | Перенос |
| O     | D    | I    | T   | S    | Z    | A   | P    | C       |

| Код      | Команда   | Такты             |                        |                   | Краткое описание |
|----------|-----------|-------------------|------------------------|-------------------|------------------|
| 0F 00 /1 | STR r/m16 | <u>486</u><br>2/3 | <u>386</u><br>pm=23/27 | <u>286</u><br>2/3 | r/m16:=TR        |

Команда STR записывает значение регистра задачи в двух байтовый операнд.

STR используется в программном обеспечении операционных систем, а не в прикладных программах.

**ARPL (проверка параметров адресов) – настройка поля RPL селектора**

## ARPL

Настройка поля RPL селектора

Adjust RPL field of selector

Только для защищенного режима 80286/i386/i486

|       |      |      |     |      |      |     |      |         |
|-------|------|------|-----|------|------|-----|------|---------|
| Переп | Напр | Прер | Шаг | Знак | Нуль | Всп | Четн | Перенос |
| O     | D    | I    | T   | S    | Z    | A   | P    | C       |

| Код  | Команда        | Такты             |                        |                        | Краткое описание                                         |
|------|----------------|-------------------|------------------------|------------------------|----------------------------------------------------------|
| 63/r | ARPL r/m16,r16 | <u>486</u><br>9/9 | <u>386</u><br>pm=20/21 | <u>286</u><br>pm=10/11 | Устанавливает RPL r/m-слова не больше RPL регистра-слова |

Команда ARPL имеет два операнда. Первый – 16-битовая переменная в памяти или словный регистр, содержащий значение некоторого селектора, обычно процедуры. Второй операнд – словный регистр, обычно содержащий значение селектора CS вызывающей программы. Если поле RPL ("requested privilege level" запрошенный уровень привилегии, два младших бита) первого операнда меньше поля RPL второго операнда, то флаг нуля устанавливается в единицу и значение поля RPL первого операнда увеличивается до значения RPL второго операнда. Иначе флаг нуля устанавливается в ноль и первый операнд не изменяется.

ARPL используется в программах операционной системы, а не в прикладных программах. Команда используется, чтобы гарантировать, что селектор подпрограммы не будет иметь больший уровень привилегий, чем разрешенный для вызывающей процедуры.

**CLTS (Команды управления операциями с плавающей точкой) – присвоение 0 флагу Переключения задач**

## CLTS

Присвоение 0 флагу Переключения задач

Clear task switched flag

Только для защищенного режима 80286/386/i486

|       |      |      |     |      |      |     |      |         |
|-------|------|------|-----|------|------|-----|------|---------|
| Переп | Напр | Прер | Шаг | Знак | Нуль | Всп | Четн | Перенос |
| O     | D    | I    | T   | S    | Z    | A   | P    | C       |

TS:=0 (TS в CR0, не из регистра флагов)

| Код   | Команда | Такты           |                 |                 | Краткое описание |
|-------|---------|-----------------|-----------------|-----------------|------------------|
| 0F 06 | CLTS    | <u>486</u><br>7 | <u>386</u><br>5 | <u>286</u><br>2 | TS:=0            |

CLTS присваивает значение 0 флагу Переключения задач (TS) в регистре CR0. Этот флаг получает значение 1 всякий раз, когда происходит переключение задач. Флаг TS используется для управления расширениями процессора (сопроцессорами) следующим образом:

- если флаг TS имеет значение 1, то каждое выполнение команды ESC прерывается: в реальном режиме генерируется прерывание 7, а в защищенном и виртуальном режиме v86 генерируется исключительная ситуация #NM;
- если флаги TS и MP имеют значение 1, то каждое выполнение команды WAIT прерывается аналогично команде ESC.

Таким образом, если переключение задачи произошло после начала выполнения команды ESC, то контекст сопроцессора может нуждаться в сохранении перед следующим выполнением ESC. Обработчик прерывания 7 или исключительной ситуации #NM сохраняет этот контекст и присваивает флагу TS значение 0.

CLTS используется в программном обеспечении операционных систем, а не в прикладных программах. Это привилегированная команда, выполняется только на нулевом уровне привилегий.

## LAR (проверка параметров адресов) – загрузка байта прав доступа к сегменту

| <b>LAR</b> |               | Загрузка байта прав доступа к сегменту<br>Load access rights byte<br>Только для защищенного режима 80286/i386/i486 |                        |                     |                          |     |      |         |
|------------|---------------|--------------------------------------------------------------------------------------------------------------------|------------------------|---------------------|--------------------------|-----|------|---------|
| Переп      | Напр          | Прер                                                                                                               | Шаг                    | Знак                | Нуль                     | Всп | Четн | Перенос |
| O          | D             | I                                                                                                                  | T                      | S                   | Z<br>P                   | A   | P    | C       |
| Код        | Команда       | Такты                                                                                                              |                        |                     | Краткое описание         |     |      |         |
| 0F 02/r    | LAR r16,r/m16 | <u>486</u><br>11/11                                                                                                | <u>386</u><br>pm=15/16 | <u>286</u><br>14/16 | r16:= r/m16 AND ff00     |     |      |         |
| 0F 02/r    | LAR r32,r/m32 | 11/11                                                                                                              | pm=15/16               |                     | r32:= r/m32 AND 00ffff00 |     |      |         |

Если селектор, указанный во втором операнде, правильный и сегмент доступен при данном уровне приоритета, то происходит следующее: LAR загружает во второй операнд младшее двойное слово дескриптора, чей селектор указан во втором операнде; первый операнд загружается старшим двойным словом дескриптора, маскированным числом 0x00FzFF00, где z означает, что эти 4 бита остаются неопределёнными; флаг ZF устанавливается в 1. Иначе (при неверном селекторе) флаг ZF устанавливается в 0.

Если операнды 16 битовые, то загружаются младшие 16-разрядные слова, а не двойные слова, как для 32-битовых операндов.

Для команды LAR доступны любые сегменты кода и данных. Более полная информация о доступных сегментах и дескрипторах приведена в документации Intel.

## LSL (проверка параметров адресов) – загрузка размера сегмента

| <b>LSL</b> |               | Загрузка размера сегмента<br>Load segment limit<br>Только для защищенного режима 80286/i386/i486 |                        |                     |                          |     |      |         |
|------------|---------------|--------------------------------------------------------------------------------------------------|------------------------|---------------------|--------------------------|-----|------|---------|
| Переп      | Напр          | Прер                                                                                             | Шаг                    | Знак                | Нуль                     | Всп | Четн | Перенос |
| O          | D             | I                                                                                                | T                      | S                   | Z<br>P                   | A   | P    | C       |
| Код        | Команда       | Такты                                                                                            |                        |                     | Краткое описание         |     |      |         |
| 0F 03 /r   | LSL r16,r/m16 | <u>486</u><br>10/10                                                                              | <u>386</u><br>pm=21/22 | <u>286</u><br>14/16 | r16:= размер в байтах    |     |      |         |
| 0F 03 /r   | LSL r32,r/m32 | 10/10                                                                                            | pm=21/22               |                     | r32:= размер в байтах    |     |      |         |
| 0F 03 /r   | LSL r16,r/m16 | 10/10                                                                                            | pm=25/26               | 14/16               | r16:= размер в страницах |     |      |         |
| 0F 03 /r   | LSL r32,r/m32 | 10/10                                                                                            | pm=26/26               |                     | r32:= размер в страницах |     |      |         |

Если сегмент, соответствующий селектору во втором операнде, доступен команде LSL при текущем уровне привилегий, то эта команда загружает его размер в первый операнд и устанавливает флаг ZF в 1. В противном случае первый операнд не изменяется и ZF устанавливается в 0. Размер сегмента задается в байтах, причем если в дескрипторе сегмента размер задан в страницах (page), то результат будет все равно в байтах (выполняется сдвиг 20 битового числа на 12 разрядов влево и установка единиц операцией OR числом 0x00000FFF).

Типами сегментов, доступными команде LSL при текущем уровне привилегий, являются: сегменты данных, сегменты кода, сегменты состояния задач (TSS) и сегменты таблиц локальных дескрипторов.

В качестве второго операнда можно использовать сегментные регистры.

## VERR/VERW (проверка параметров адресов) – проверка доступности сегмента для чтения/записи

| <b>VERR<br/>VERW</b> |             | Проверка доступности сегмента для чтения/записи<br>Verify a segment for reading or writing<br>Только для защищенного режима 80286/i386/i486 |                        |                     |                                  |     |      |         |
|----------------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------|------------------------|---------------------|----------------------------------|-----|------|---------|
| Переп                | Напр        | Прер                                                                                                                                        | Шаг                    | Знак                | Нуль                             | Всп | Четн | Перенос |
| O                    | D           | I                                                                                                                                           | T                      | S                   | Z<br>P                           | A   | P    | C       |
| Код                  | Команда     | Такты                                                                                                                                       |                        |                     | Краткое описание                 |     |      |         |
| 0F 00 /4             | VERR r/m 16 | <u>486</u><br>11/11                                                                                                                         | <u>386</u><br>pm=10/11 | <u>286</u><br>14/16 | ZF:=1, если сегмент можно читать |     |      |         |
| 0F 00 /5             | VERW r/m 16 | 11/11                                                                                                                                       | pm=15/16               | 14/16               | ZF:=1, если сегмент можно писать |     |      |         |



Команда VERR проверяет, разрешено ли читать (команда VERW, соответственно, писать) в сегмент, селектор которого указан в операнде. Если текущий уровень привилегии выше или такой же, как указано в дескрипторе сегмента, то операция чтения/записи разрешена. Если сегмент доступен для чтения/записи, то флаг ZF устанавливается в 1, иначе в 0. Чтобы сегмент был доступен, должны выполняться следующие условия:

- селектор должен быть допустимым, то есть указывать на какой-нибудь дескриптор таблиц GDT или LDT;
- сегмент должен быть сегментом кода или данных, но не сегментом состояния, локальной таблицей дескрипторов или шлюзом;
- в дескрипторе сегмента должен быть установлен флаг доступности для чтения или для записи соответственно команде VERR или VERW;
- если кодовый сегмент доступен для чтения, то уровень привилегий дескриптора (DPL) может быть любым. Иначе DPL должен иметь меньший или равный уровень привилегий, чем текущий и указанный в поле RPL селектора.

Неверное значение селектора не приводит к исключительной ситуации.

### ***Воздействие команд Intel 8086/8088 на флаги***

Символы означают, что флаг после операции (таблица на Рис.251):

P – отражает результат выполнения

пробел – не изменяется

H – не определен

0 – равен нулю

1 – равен единице

| Команда                    |                      | Флаги состояния |    |    |    |    |    |
|----------------------------|----------------------|-----------------|----|----|----|----|----|
|                            |                      | OF              | CF | AF | SF | ZF | PF |
| Арифметика, сравнения      |                      |                 |    |    |    |    |    |
|                            | ADD ADC SUB SBB      | P               | P  | P  | P  | P  | P  |
|                            | CMP CMPS SCAS NEG    | P               | P  | P  | P  | P  | P  |
|                            | INC DEC              | P               |    | P  | P  | P  | P  |
| Умножение, деление         |                      |                 |    |    |    |    |    |
|                            | MUL IMUL             | P               | P  | H  | H  | H  | H  |
|                            | DIV IDIV             | H               | H  | H  | H  | H  | H  |
| Десятичная коррекция       |                      |                 |    |    |    |    |    |
|                            | DAA DAS              | H               | P  | P  | P  | P  | P  |
|                            | AAA AAS              | H               | P  | P  | H  | H  | H  |
|                            | AAM AAD              | H               | H  | H  | P  | P  | P  |
| Логика                     |                      |                 |    |    |    |    |    |
|                            | AND OR XOR TEST      | 0               | 0  | H  | P  | P  | P  |
| Сдвиги                     |                      |                 |    |    |    |    |    |
|                            | на 1 SHL SHR         | P               | P  | H  | P  | P  | P  |
|                            | на n SHL SHR         | H               | P  | H  | P  | P  | P  |
|                            | на 1 ROL ROR RCL RCR | P               | P  |    |    |    |    |
|                            | на n ROL ROR RCL RCR | H               | P  |    |    |    |    |
| Восстановление флагов      |                      |                 |    |    |    |    |    |
|                            | POPF IRET            | P               | P  | P  | P  | P  | P  |
|                            | SAHF                 |                 | P  | P  |    |    |    |
| Управление флагом переноса |                      |                 |    |    |    |    |    |
|                            | STC                  |                 | 1  |    |    |    |    |
|                            | CLC                  |                 | 0  |    |    |    |    |
|                            | CMC                  |                 | P  |    |    |    |    |
| Команда                    |                      |                 |    |    |    |    |    |
|                            |                      | DF              | IF | TF |    |    |    |
| Восстановление флагов      |                      |                 |    |    |    |    |    |
|                            | POPF IRET            | P               | P  | P  |    |    |    |
| Прерывания                 |                      |                 |    |    |    |    |    |
|                            | INT INTO             |                 | 0  | 0  |    |    |    |
| Управление флагами         |                      |                 |    |    |    |    |    |
|                            | STD                  | 1               |    |    |    |    |    |
|                            | CLD                  | 0               |    |    |    |    |    |
|                            | STI                  |                 | 1  |    |    |    |    |
|                            | CLI                  |                 | 0  |    |    |    |    |

**Рис.251 – Воздействие команд на флаги**

Остальные команды Intel 8086/8088 на флаги не действуют.

### ***Воздействие дополнительных команд на флаги***

Символы означают, что флаг после операции (таблица на Рис.252):

P – отражает результат выполнения

пробел – не изменяется

H – не определен

0 – равен нулю

1 – равен единице

| Команда                        | Флаги состояния |    |    |    |    |    |
|--------------------------------|-----------------|----|----|----|----|----|
|                                | OF              | CF | AF | SF | ZF | PF |
| CMPD SCASD IRETD               | P               | P  | P  | P  | P  | P  |
| CMPXCHG POPFD XADD             | P               | P  | P  | P  | P  | P  |
| SHLD SHRD                      | H               | P  | H  | P  | P  | P  |
| ARPL BSF BSR LAR LSL VERR VERW |                 |    |    |    | P  |    |
| BT BTC BTR BTS                 |                 | P  |    |    |    |    |
| Флаги управления               |                 |    |    |    |    |    |
|                                | DF              | IF | TF | TS |    |    |
| CLTS                           |                 |    |    | 0  |    |    |

Рис.252 - Воздействие дополнительных команд на флаги

Остальные дополнительные команды процессора i80286 на флаги не действуют.

## Ядро системы на базе процессора i80286

Основой архитектуры IBM PC/AT стал микропроцессор i80286, шинный интерфейс которого представлен ниже на Рис.253:

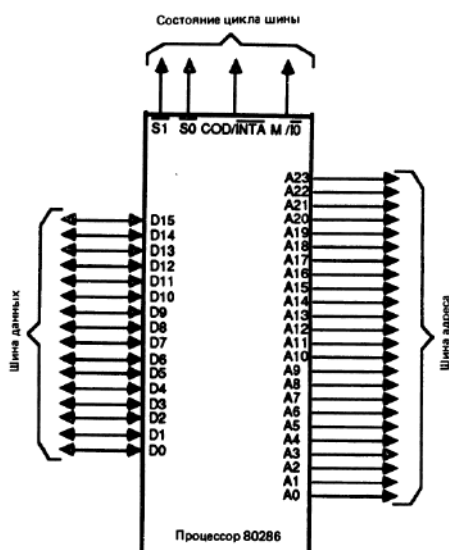


Рис.253 – Интерфейс шины процессора i80286

Шины адреса и данных, как уже упоминалось выше, у этого процессора были не мультиплексированы. Четыре линии состояния S1, S0, COD/INTA и M/IO отображают тип цикла шины и служат линиями управления шиной.

В таблице на Рис.254 показано, как интерпретировать линии состояния цикла шины. Сигналы S0 и S1 показывают, что делает процессор: принимает данные (чтение или ввод), передаёт (запись или вывод) или что-то другое (прерывание, останов или отключение). Когда процессор передаёт или принимает, сигнал M/IO показывает, производится операция с памятью или с устройством ввода-вывода. Когда процессор считывает из памяти, сигнал COD/INTA показывает, производит он выборку команды (кода) или считывает данные.

Сигналы S0 и S1 работают в инверсной логике, что отмечено символом  $\overline{HE}$  на рисунке.

| Операция               | $\overline{S1}$ | $\overline{S0}$ | $M/\overline{IO}$ | $COD/\overline{INTA}$ | Цикл шины                         |
|------------------------|-----------------|-----------------|-------------------|-----------------------|-----------------------------------|
|                        |                 |                 | 0                 | 1                     | Ввод                              |
| Процессор<br>принимает | 0               | 1               | 1                 | 0                     | Считывание из памяти<br>(данные)  |
|                        |                 |                 |                   | 1                     | Считывание из памяти<br>(команды) |
| Процессор<br>выдает    | 1               | 0               | 0                 | 1                     | Вывод                             |
|                        |                 |                 | 1                 | 0                     | Запись в память                   |
|                        |                 |                 | 0                 | 0                     | Подтверждение<br>прерывания       |
| Остальные              | 0               | 0               | 1                 | 0                     | Останов или отключение *          |

\*  $A_1 = 1$  для останова,  $A_1 = 0$  для отключения

Рис.254 – Коды состояния цикла шины

Ниже на Рис.255 приведена реальная схема вычислительного ядра на базе i80286, с использованием внешнего синхрогенератора i82284 и шинного контроллера i82288. Схема практически неотличима от вычислительного ядра на связке образца почти десятилетней давности: i8080 + i8228 + i8224, но в варианте для 286-го были использованы модифицированные версии контроллеров «обвязки». Но на материнской плате IBM PC/AT можно увидеть и старых знакомых, использовавшихся в предыдущих поколениях PC: например, контроллер прерываний i8259A и контроллер прямого доступа i8237A. Правда, количество этих устройств в PC/AT удвоилось по сравнению с PC и PC/XT, и включены были эти устройства по каскадной схеме.

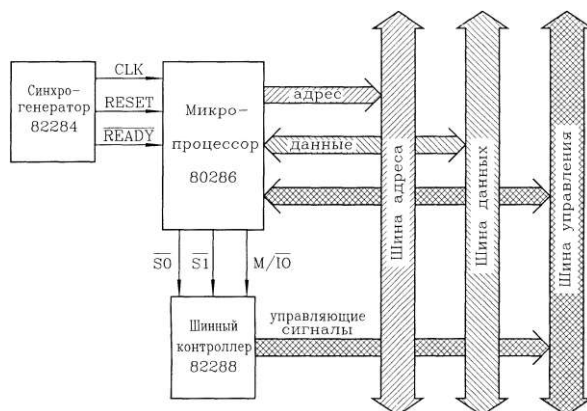


Рис.255 – Реализация ядра системы на базе i80286

Синхрогенератор генерирует тактовый сигнал CLK для синхронизации внутреннего функционирования процессора и других микросхем. Сигнал RESET производит сброс процессора в начальное состояние. Сигнал READY, также формируется с помощью синхрогенератора, предназначен для удлинения циклов шины при работе с медленными периферийными устройствами. На адресную шину, состоящую из 24 линий, микропроцессор i80286 выставляет адрес байта или слова, которые будут пересылаться по шине данных в процессор или из него. Кроме того, шина адреса используется микропроцессором для указания адресов (номеров) периферийных портов, с которыми производится обмен данными.

Шина данных состоит из 16 линий, по которым возможна передача как отдельных байтов, так и двухбайтовых слов. При пересылке байтов возможна передача отдельно как по старшим 8 линиям, так и по младшим. Шина данных двунаправленная, т.к. передача байтов и слов может производиться как в микропроцессор, так и из него. Шина управления формируется сигналами, во-первых, поступающими непосредственно от микропроцессора, во-вторых, сигналами, сформированными системным контроллером, и, в-третьих, сигналами, идущими к микропроцессору от других микросхем и периферийных адаптеров. Микропроцессор использует системный контроллер для формирования управляющих сигналов,

определяющих правила пересылки данных по шине, с помощью описанных выше кодов (Рис.254). Он выставляет три сигнала S0, S1, M/IO (выводы 5, 4 и 65), которые определяют тип цикла шины (подтверждение прерывания, чтение порта ввода/вывода, запись в порт ввода/вывода, останов, чтение памяти, запись в память). На основании значений этих сигналов системный контроллер формирует управляющие сигналы, определяющие последовательность процессов данного типа цикла шины.

Ниже, на Рис.256 приведена более подробная, чем на Рис.255 схема подключения шинного системного контроллера 82288 к МП i80286. Необходимо отметить ещё несколько дополнительных элементов на схеме Рис.256: 24-разрядный адресный регистр-защёлка и двунаправленный 16-разрядный буферный приёмопередатчик для шины данных. Эти элементы служат для увеличения нагрузочной способности линий адреса<sup>1</sup> и данных, т.к. нагрузочная способность самого процессора очень небольшая, всего порядка 1ма.

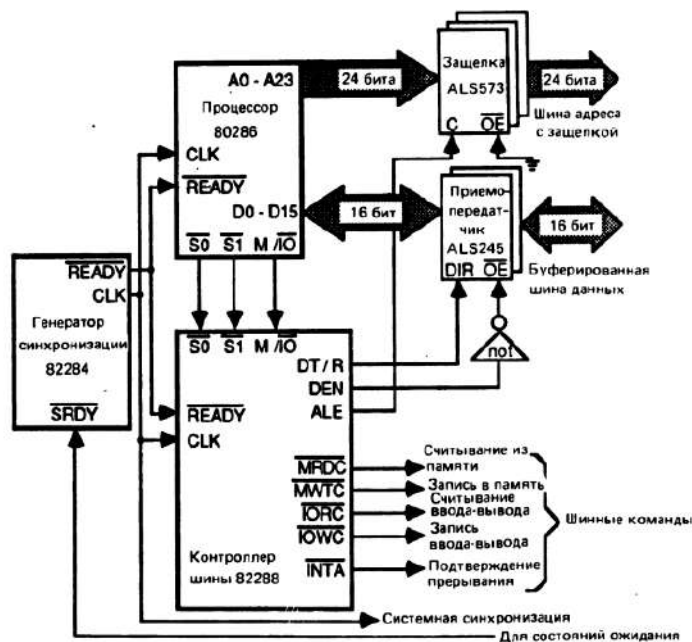


Рис.256 – Подключение контроллера шины 82288

## Временная диаграмма циклов шины

В системах на базе i80286, при изучении временных характеристик шины процессора, используются три основных понятия: *такты системной синхронизации, циклы процессора и циклы шины*.

Основным временным сигналом в системе является сигнал системной синхронизации CLK. При частоте синхронизации 16 МГц такт составляет 62.5 нс.

Два такта системной синхронизации образуют цикл процессора, первый такт называется фаза 1, второй – фаза 2 (Рис.257), т.е., процессор i80286 работает на частоте в два раза ниже частоты системной синхронизации. Например, при работе процессора на частоте 8 МГц частота системной синхронизации равна 16 МГц, а цикл процессора составляет 125 нс.

<sup>1</sup> Адресный регистр, конечно же, ещё используется и для запоминания, фиксации/защёлкивания содержимого адресных линий, иначе в этом месте схемы можно было бы использовать простой однонаправленный параллельный буфер-усилитель.

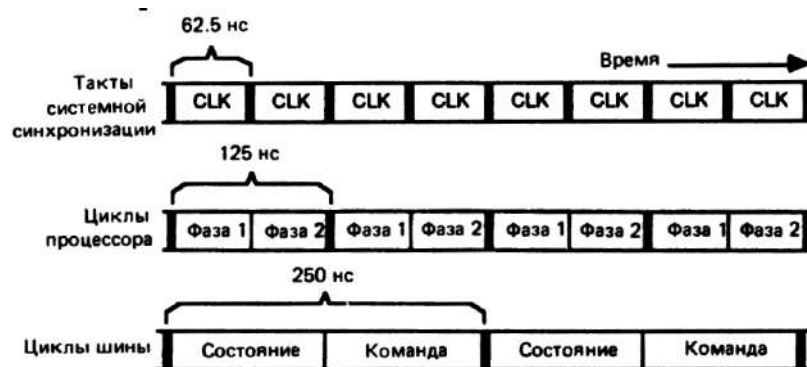


Рис.257 – Временная диаграмма шины

Каждый цикл шины включает в себя два действия и поэтому занимает, как минимум, два цикла процессора. В первом цикле процессор выдаёт состояние шины и адресную информацию, поэтому он называется *цикл состояния*. На основе цикла состояния подключённые к шине устройства должны понять, как им реагировать на обращение процессора. В следующем цикле процессора, называемом *командным циклом*, процессор и адресуемое устройство проводят собственно передачу данных.

Некоторые устройства могут не успеть отреагировать к моменту окончания командной части цикла шины. В таких ситуациях используется сигнал READY, который должно сгенерировать медленное устройство. «Увидев» READY, процессор (точнее, шинный контроллер) расширяет командный цикл шины дополнительными циклами *ожидания*<sup>1</sup>, до тех пор, пока устройство не снимет сигнал READY (Рис.258).



Рис.258 – Расширение цикла шины с помощью состояний ожидания: а – обычный цикл шины, б – цикл шины, расширенный на два состояния ожидания

### Генерирование импульсов синхронизации

На Рис.259 показана более подробная (по сравнению с Рис.255) схема подключения генератора синхронизации к процессору:

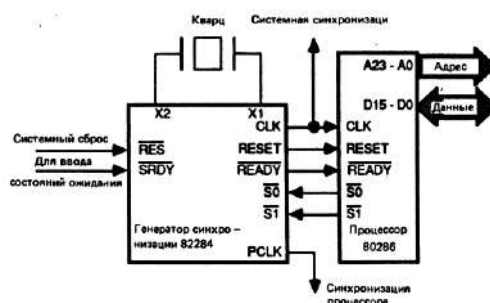


Рис.259 – Подключение генератора синхронизации к процессору

Кроме системной синхронизации CLK генератор формирует импульсы, соответствующие циклам процессора PCLK для обслуживания остальной части схемы. Чтобы сигнал

<sup>1</sup> Вставка циклов ожидания фактически превращает шину из синхронной в асинхронную.

PCLK правильно отражал один цикл процессора, а не фазу 2 одного цикла и фазу 1 следующего, необходимо синхронизировать генератор 82284 и процессор. Для этого все сигналы системного сброса подаются в микросхему 82284, а затем поступают в процессор. Эти сигналы и обеспечивают синхронизацию двух микросхем (корректное фазирование).

### Конвейеризация шины

В процессоре i80286 был использован способ повышения производительности при работе с относительно медленной памятью, называющийся *конвейеризация шины*<sup>1</sup>.

Ранее, при обсуждении цикла шины, предполагалось, что процессор выдаёт адрес на шину адреса в начале цикла состояния цикла шины и сохраняет его в течении всего цикла шины. Также предполагалось, что данные помещаются на шину в командной части и сохраняются на ней до конца цикла шины. Но если адрес выдать на шину немного раньше начала цикла шины (сдвиг), а данные не требовать даже немного спустя после начала командной части цикла, то устройствам на реакцию будет доступно больше времени.

На Рис.260 показано, что в действительности происходит в трёх частях шины процессора (линиях состояния, шине адреса и шине данных). Частота процессора в примере 8 МГц. Видно, что адрес опережает цикл шины на один такт системной синхронизации (62.5 нс). На такой же интервал данные «залезают» в следующий цикл шины. Здесь как раз используется дополнительная разрешающая способность по времени, обеспечиваемая двойной частотой системной синхронизации.

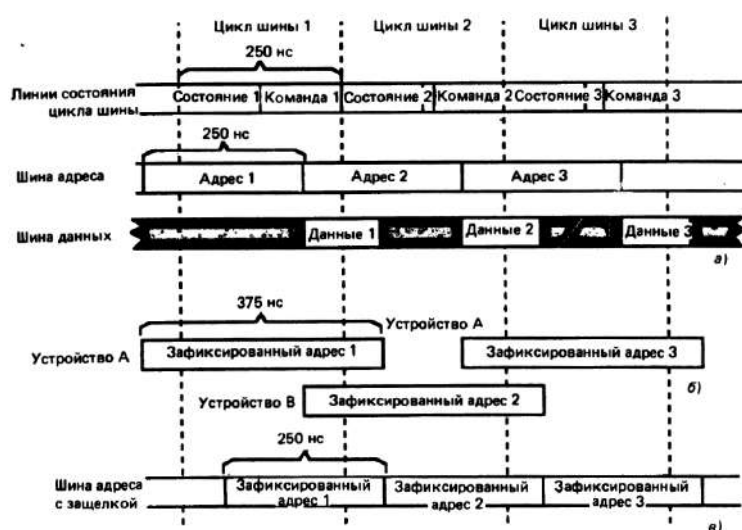


Рис.260 – Временная диаграмма конвейерной шины

Из временной диаграммы на Рис.260 очевидно, что часть схемы дешифратора адреса для устройств (памяти), работающих с конвейеризованной шиной, должна быть модифицирована, т.к. устройство, спроектированное в расчёте на то, что адрес будет сохраняться на адресных линиях неизменным в течении всего цикла шины, не «поймёт» шутки, если адрес на шине исчезнет где-нибудь в середине цикла. Решением будет запоминание, «защёлкивание» адреса в параллельном регистре-защёлке, который должен быть добавлен в адресную часть схемы каждого устройства.

Несмотря на то, что линии адреса и данных в шине AT bus не мультиплексированы и разделены пространственно, при использовании конвейерного режима приходится *защёлкивать* адрес в регистрах-защёлках, чтобы гарантированность неизменность адреса в течении всего цикла. В случае мультиплексированных шин, в которых линии адреса и данных совмещались, адресный регистр-защёлка — это обязательный компонент схемы адресного дешифратора в устройствах.

<sup>1</sup> Это приводит к некоторому усложнению аппаратной части системы, в области дешифрации адреса.

## Организация системы шин L, S, X и M в компьютерах PC/AT

PC/XT была построена на базе единственной шины (которая так и называлась – «Общая шина»). В PC/AT было реализовано несколько (четыре) шин, каждая из которых из которых обслуживала только свой определённый тип устройств – память, периферийные контроллеры и т.п. Необходимо подчеркнуть особо, что эти четыре типа шин работали на одной единой тактовой частоте, и разделение шин осуществлялось фактически только на «электрическом» уровне, с помощью двунаправленных буферных приёмопередатчиков. Т.е. с изменением тактовой частоты процессора синхронно изменялась частота работы для всех шин.

Этот факт из биографии PC/AT довольно часто приводил к неприятным ситуациям: при повышении тактовой частоты процессора (например, путём его замены на более высокочастотный) некоторые периферийные адаптеры могли работать со сбоями или отказывались работать совсем. Пример из жизни: безупречно работавшие в PC/AT с тактовой частотой процессора 12 МГц сетевые Ethernet адаптеры одной известной фирмы, при переходе на процессор 80286 с тактовой частотой 16 МГц начинали очень сильно сбоить, а в системе с процессором с тактовой частотой 20 МГц не работали вовсе. И это несмотря на то, в шине присутствовали механизмы для работы с медленными устройствами – сигнал READY, при получении которого процессор растягивал циклы шины, вставляя циклы ожидания.

Настоящее разделение на несколько шин, каждая со своей тактовой частотой, состоялось только в машинах на базе процессора следующего поколения – i80386, где локальная шина процессора, шина памяти, ISA шина и др. работали каждая на своей частоте. Начиная именно с 386-х для шины ISA была зафиксирована тактовая частота – около 7.8 МГц, в результате чего проблем с медленными адаптерами, подобным описанным выше, в этих системах не возникало, несмотря на то, что процессор при этом работал на гораздо более высоких частотах, в десятки мегагерц.

Основных шин в PC/AT-286 было четыре:

- L-шина – локальная шина процессора
- S-шина – основная, системная шина, ISA)
- M-шина – шина памяти
- X-шина – шина системных периферийных контроллеров

На Рис.261 представлена структурная схема классической IBM PC/AT первых поколений, на которой отмечены перечисленные выше шины:

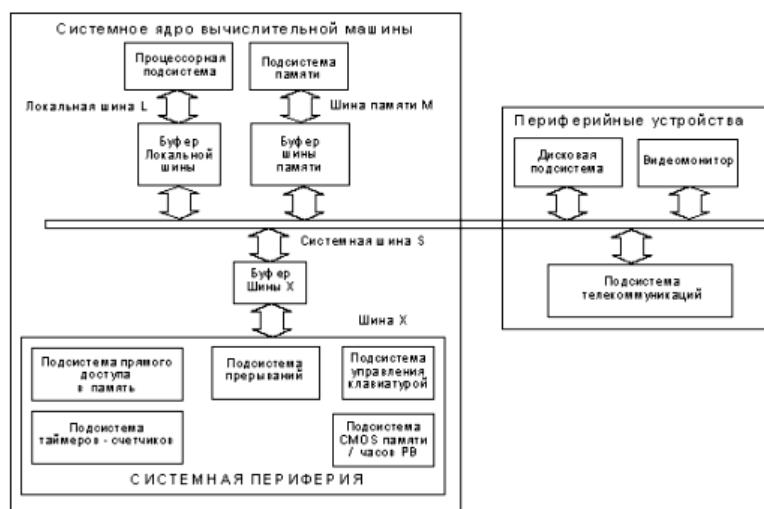


Рис.261 - Классическая архитектура IBM PC/AT-286

В более поздних моделях IBM PC/AT-286 основная оперативная память была выделена в отдельную подсистему и доступ к ней осуществлялся не через системную шину, а параллельно с доступом к системной шине. Это реализовывалось с помощью т.н. интегрированного контроллера системной шины (Рис.262). Суммарная задержка передачи данных в этом случае сокращалась примерно до 20 нс, а максимальная возможная тактовая частота повышалась до 25 МГц.

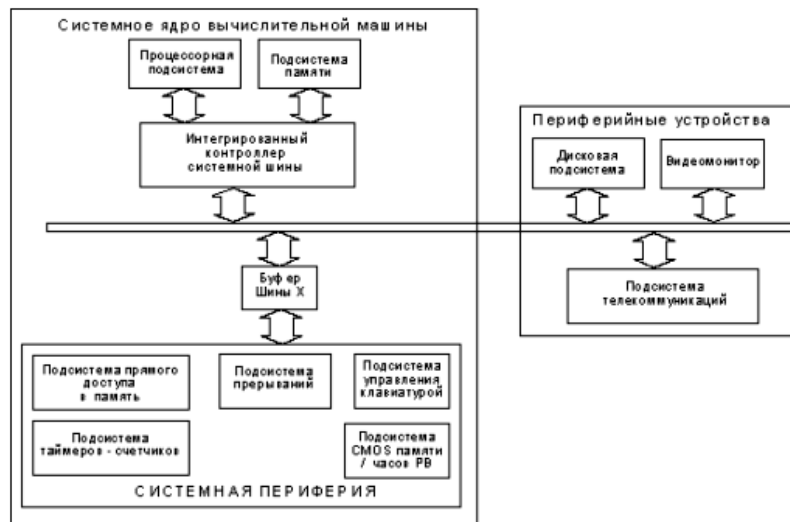


Рис.262 - Архитектура IBM PC/AT-286 поздних моделей

## Подключение периферийных устройств

Для PC/AT инженеры IBM продолжили традицию использования широкодоступных универсальных периферийных контроллеров от сторонних разработчиков (почти всегда от Intel), не став разрабатывать и для AT-архитектуры никаких узкоспециализированных схем. Со временем, через несколько поколений процессоров, индустрия придёт к объединению стандартного набора периферийных контроллеров на одном-двух кристаллах, этот подход приведёт к появлению специализированных схем «обвязки» процессора, получивших название *чипсет*.

На упрощённой структурной схеме IBM PC/AT ранней версии (Рис.263) можно отметить как хорошо знакомые по прошлым поколениям PC периферийные контроллеры, так и модифицированные версии, адаптированные под процессор 80286:

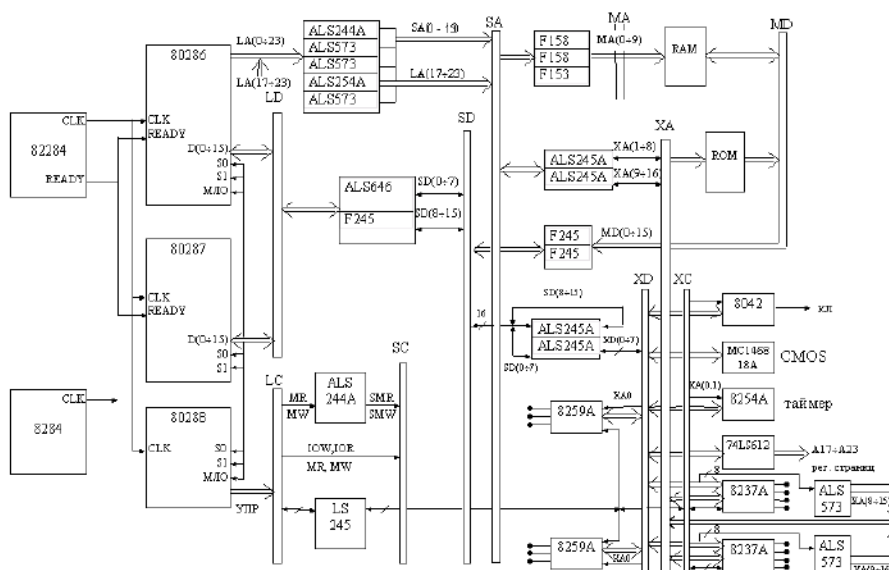


Рис.263 - Структурная схема системной платы IBM PC/AT



Контроллеры в составе PC/AT, присутствовавшие уже в двух предыдущих поколениях PC – контроллер прерываний i8259A, контроллер прямого доступа i8237A, контроллер таймера 8254A. Модифицированные под процессор 80286 – тактовый генератор i82284 и шинный контроллер i80288/82288, описание функционирования которых было приведено выше.

Из нововведений в аппаратной части AT нужно отметить микросхему Motorola MC146818A RTC – комбинация часов реального времени и энергонезависимой CMOS памяти небольшого объёма (50 байт). Без RTC в PC первых двух поколений – PC и PC/XT – пользователю приходилось вводить дату и время при каждой загрузке машины. В энергонезависимой памяти хранилась конфигурация машины, которую стало возможным модифицировать программно, из BIOS (в PC и PC/XT конфигурация менялась с помощью блока переключателей). И, как уже упоминалось выше, интерфейс с клавиатурой в AT стал двуправленным, что дало возможность управлять индикаторами на клавиатуре.

Для работы с прерываниями от устройств в PC/AT было использовано два контроллера i8259A, включенных каскадом. Ниже на схеме (Рис.264) приведена схема подключения одиночного контроллера прерываний к процессору i80286 – простой унифицированный интерфейс подключения устройства к шине системы, с минимумом дополнительных аппаратных компонентов.

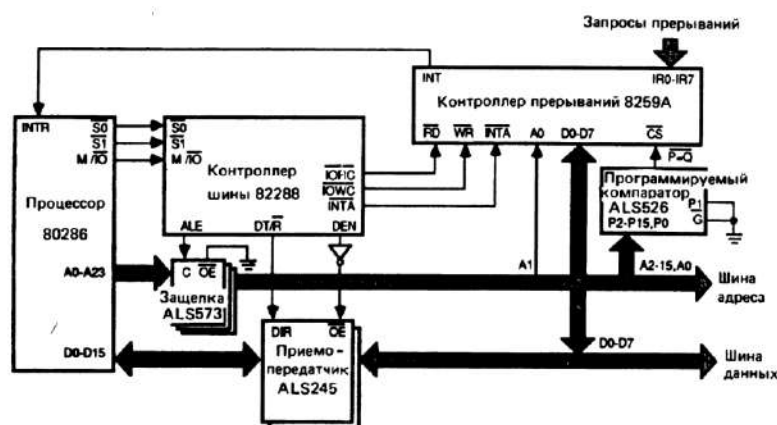


Рис.264 – Подключение контроллера прерываний 8259A к процессору 80286

В PC/AT были существенно расширены доступные ресурсы системы, в первую очередь, конечно, объём адресуемой памяти, что было «заслугой» процессора. А вот другой дефицитный ресурс – количество входов запросов прерывания – можно было увеличить только внешними по отношению к процессору средствами. Из 8 доступных входов IRQ в IBM PC/XT под системные нужды было занято пять, оставляя доступными только три входа. И разработчики PC/AT решили эту проблему, используя возможности той же самой микросхемы i8259A (сохранив тем самым обратную совместимость), включив второй дополнительный контроллер прерываний каскадно.

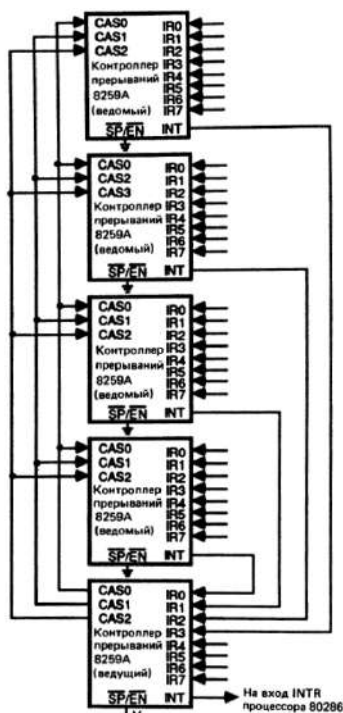


Рис.265 – Каскадное включение контроллеров прерываний

Чтобы контроллеры прерываний действовали согласованно, каждый из них должен знать свою роль – ведомый он или ведущий. Это определяется сигналом на входе SP/EN. Кроме того, при инициализации микросхемы во внутреннем регистре ведущего контроллера фиксируется, какие входы подсоединены к ведомым контроллерам, а какие непосредственно к устройствам. Во внутреннем регистре каждого ведомого контроллера, соответственно, хранится номер входа IR ведущего контроллера, к которому этот ведомый контроллер подсоединён.

Если устройство запрашивает прерывание через ведомый контроллер, то он формирует реальный сигнал INTR, идущий на центральный процессор, через ведущий контроллер. В итоге 80286 отреагирует двумя циклами шины подтверждения прерывания INTA.

В течении первого цикла шины INTA ведущий контроллер подтверждает запрос прерывания ведомого контроллера, выдавая его номер входа IR на линии каскадирования CAS. Эти линии соединяют ведущий контроллер со всеми ведомыми. Такое подтверждение необходимо потому, что запросить прерывание могут одновременно несколько ведомых контроллеров.

Во втором цикле шины INTA выбранный ведомый контроллер помещает номер прерывания прерывающего устройства на шину данных, чтобы его считал процессор. После этого процессор реагирует вызовом соответствующей процедуры обработки прерывания, как и в случае единственного контроллера.

Если сигнал запроса прерывания поступает на вход немаскируемого прерывания NMI, процессор не запрашивает у устройства информацию о типе прерывания, так как причиной NMI-запроса по логике могла стать только катастрофическая ситуация (например, пропадание питания), и у процессора в этом случае всего одна задача – спасти наиболее важные результаты в безопасное место. Подобная ситуация в таблице соответствует прерыванию второго типа. Кроме прерываний от внешних устройств, возможны и внутренние прерывания, возникающие в самом процессоре. Такие сигналы могут, генерироваться, например, при разнообразных ошибках во время выполнения программ, командой программного прерывания INT, а также для организации пошагового режима работы процессора (см. таблицу Рис.267):

На Рис.265 приведён пример каскадного включения пяти контроллеров прерываний типа i8259A, позволяющего обслуживать до 36 входов запросов прерывания – по 8 с каждого *ведомого* (slave) контроллера и 4 – с *ведущего* (master).

Теоретически, при двухуровневом каскадировании, используя девять контроллеров – один *ведущий* и восемь *ведомых* – можно получить возможность обслуживать до  $8 \times 8 = 64$  входов запросов прерываний, но инженеры IBM так далеко не зашли, они добавили в схему только одну i8259A, в результате чего в AT стало возможным обслуживать до 15 входов IRQ (Рис.266).

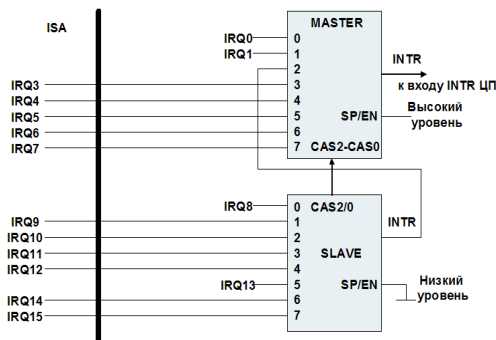


Рис.266 – Каскадное включение i8259A в IBM PC/AT

| Номер | Реальный режим                                  | Защищённый режим                              |
|-------|-------------------------------------------------|-----------------------------------------------|
| 0     | Особый случай ошибки деления                    | Особый случай ошибки деления                  |
| 1     | Прерывание пошаговой работы                     | Прерывание пошаговой работы                   |
| 2     | Немаскируемое прерывание                        | Немаскируемое прерывание                      |
| 3     | Контрольный останов                             | Контрольный останов                           |
| 4     | Особый случай переполнения                      | Особый случай переполнения                    |
| 5     | Особый случай превышения диапазона              | Особый случай превышения диапазона            |
| 6     | Особый случай недействительного кода операции   | Особый случай недействительного кода операции |
| 7     | Особый случай отсутствия сопроцессора           | Особый случай отсутствия сопроцессора         |
| 8     | Особый случай слишком малой IDT                 | Особый случай слишком малой IDT               |
| 9     | Особый случай превышения сегмента сопроцессором | Превышение сегмента сопроцессором             |
| 10    | Зарезервировано                                 | Недействительный TSS                          |
| 11    | Зарезервировано                                 | Отсутствие сегмента                           |
| 12    | Зарезервировано                                 | Особый случай стека                           |
| 13    | Особый случай превышения сегмента               | Особый случай защиты                          |
| 14    | Зарезервировано                                 | Зарезервировано                               |
| 15    | Зарезервировано                                 | Зарезервировано                               |
| 16    | Особый случай сопроцессора                      | Особый случай сопроцессора                    |
| -     |                                                 |                                               |
| -     | Зарезервированы                                 | Зарезервированы                               |
| -     |                                                 |                                               |
| 31    |                                                 |                                               |

Рис.267 – Таблица прерываний i80286

## Приоритеты обработки прерываний

Безусловным приоритетом для процессора является сигнал запроса прерывания, поступающий на вход NMI. Но что произойдёт, если на вход INTR одновременно поступают запросы сразу от нескольких устройств и флаг IF при этом установлен в единицу? Процессор может отреагировать на запросы только по очереди, в самом простом случае каждому устройству, входящему в систему, присваивается свой уникальный уровень приоритета – например, входу с бóльшим номером назначается более высокий приоритет, и если, к примеру, два прерывания возникнут одновременно, контроллер отдаст предпочтение сигналу от устройства с более высоким приоритетом, а второе прерывание запомнит и выдаст его на вход INTR только после окончания процедуры обработки первого прерывания.

Что происходит в случаях, если возникает более приоритетное прерывание, а в это время уже происходит обработка менее приоритетного прерывания? Поведение зависит от состояния флага разрешения прерываний IF. Если  $IF = 1$ , текущая процедура обработки останавливается, и запускается процедура обработки прерывания с более высоким приоритетом. В том же случае, когда  $IF = 0$ , процессор не реагирует на новое событие, игнорируя его приоритет, даже если он выше приоритета обрабатываемого прерывания.

Подобно тому, как это было реализовано для схемы обработки прерываний, в PC/AT инженеры IBM увеличили количество входов запросов DMA в подсистеме прямого доступа к памяти<sup>1</sup>. В PC/AT переключал контроллер DMA i8237A, который уже использовался в двух предыдущих поколениях PC, структурная схема контроллера представлена на Рис.268. Количество входов DRQ (Dma ReQuest) в этой микросхеме – всего четыре, из которых под системные нужды (в PC/XT) было занято три.

<sup>1</sup> Несмотря на название, интеловские контроллеры DMA могли обмениваться данными не только с памятью, но и с регистрами устройств в пространстве ввода-вывода (I/O).

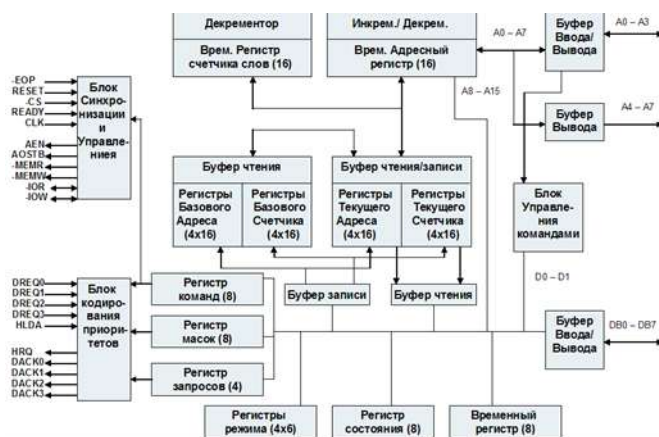


Рис.268 - Структура контроллера DMA Intel 8237A

Используя возможности микросхемы i8237A, инженеры добавили в систему ещё один контроллер, включив его каскадно с первым, обеспечив при этом обратную совместимость с контроллером DMA в IBM PC/XT.

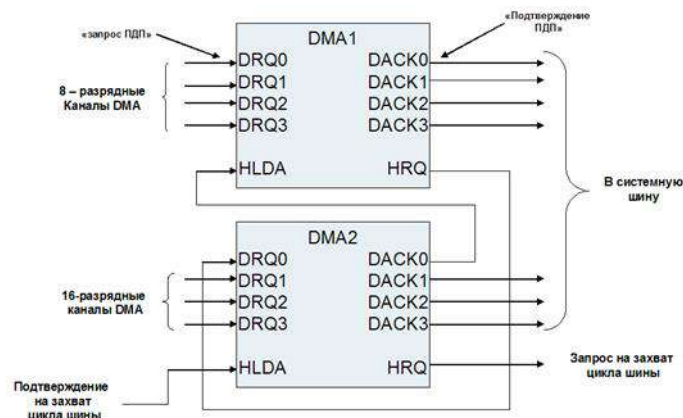


Рис.269 - Каскадное включение контроллеров в IBM PC/AT

Интересно, что второй (по схеме *ведущий*) контроллер был включён в PC/AT в *словном режиме*, что достигалось включением его адресных линий со сдвигом на один разряд влево, т.е. этот контроллер (нижний на схеме Рис.269) выполнял обмены всегда по чётным адресам, 16-разрядными словами, а не байтами, как байтовый контроллер.

В активном цикле обслуживание подсистемы DMA возможно в одном из четырёх режимов:

- режим одиночной передачи (Single Transfer Mode);
- режим передачи блока (Block Transfer Mode);
- режим передачи по требованию (Demand Transfer Mode);
- каскадный режим (Cascade Mode).

Для формирования 24-разрядного адреса используется регистр страницы, который определяет старшие биты адреса. Младшие 16 бит задаются регистром базы соответствующего канала контроллера. Такая схема обеспечивает передачу данных в пределах адресного пространства 16 Мбайт.

Для 8-битных каналов DMA регистр страницы определяет биты 16-23 физического адреса, а регистр базы - биты 0-15:

|                                  |                              |
|----------------------------------|------------------------------|
| 23 – 16                          | 15 – 0                       |
| <b>разряды регистра страницы</b> | <b>разряды регистра базы</b> |

Для 16-битных каналов DMA используется только 7 бит регистра страницы, определяющих биты 17-23 физического адреса. При этом регистр базы канала задает биты 1-16. Бит A0 всегда выставляется в 0, чтобы гарантировать выровненную передачу слова памяти:

|                                  |                              |          |
|----------------------------------|------------------------------|----------|
| 23 – 17                          | 16 – 1                       | 0        |
| <b>разряды регистра страницы</b> | <b>разряды регистра базы</b> | <b>0</b> |

Все дополнительные линии: адреса, входов запроса прерываний, входов запроса DMA – были выведены на вторую часть разъёма шины ISA16, тем самым была обеспечена полная обратная совместимость с 8-разрядной версией ISA (XT bus). В итоге в системе допускалась любая комбинация разрядности шина-адаптер. Естественно, что максимальные ресурсы – 24 разряда ОЗУ, семь дополнительных входов IRQ и три дополнительных DRQ были доступны только в единственном сочетании: 16-разрядный ISA-слот, 16-разрядный адаптер. Во всех остальных комбинациях всё работало как на 8-разрядной ISA-шине: 20 разрядов ОЗУ, пять доступных входов IRQ, три входа DRQ.

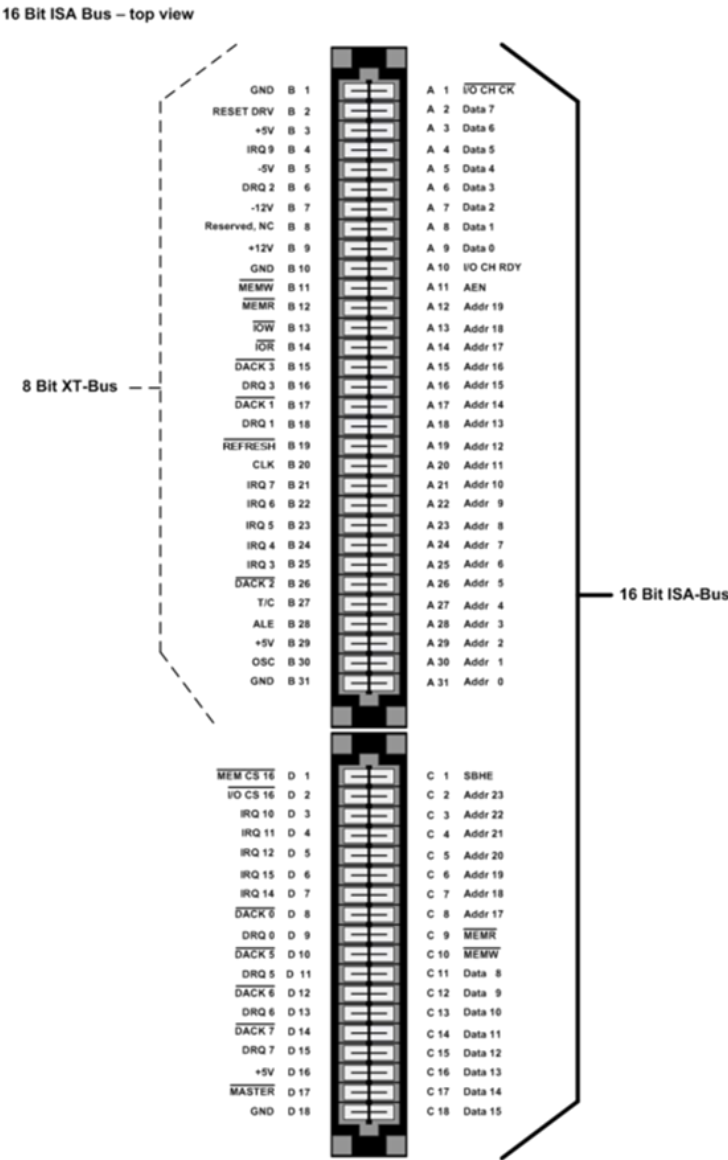


Рис.270 – 16-битная шина ISA

Коротко резюмируя – усовершенствованная в 1984 (PC/AT) году шина ISA<sup>1</sup> была улучшена по нескольким параметрам: в ней была удвоена разрядность данных (что повлекло удвоение пропускной способности) и добавлены четыре разряда адреса; кроме того, увеличилось число линий запросов прерываний и запросов прямого доступа к памяти (DMA). Также в 16-разрядной шине ISA любое подключенное к ней устройство могло выступать в роли задатчика, то есть инициировать операцию обмена данными (в 8-разрядной шине задатчиками были только процессор и контроллер DMA). Для подключения 16-разрядных устройств используются разъёмы, состоящие из двух частей: полностью совместимой с 8-разрядной XT-шиной 62-контактной частью и новой 36-контактной (Рис.270).

## Программное обеспечение

С появлением на массовом рынке процессора класса i80286 в индустрии возникла и на долгие годы сохранилась ситуация неадекватности и серьёзного несоответствия уровня программного обеспечения (в первую очередь – операционных систем) возможностям новых процессоров и компьютеров на их основе. И этот разрыв всё увеличивался, и стал особенно заметен, когда в 1985-ом году Intel выпустила свой первый 32-разрядный процессор i80386, который чаще всего работал в составе очередного поколения IBM PC под управлением 16-разрядной MS-DOS!

Альтернативные ОС, использующие все или, по крайней мере, основную массу возможностей стремительно развивающихся мощных микропроцессоров, были или нишевыми решениями, не подходящими на роль системы для массового пользователя, такие как Xenix-286/386 – unix-подобные ОС от Microsoft. Или так и не сумевшие серьёзно потеснить MS-DOS объективно гораздо более продвинутые ОС от IBM OS/2<sup>2</sup> первых версий 1.x, с функциями графического интерфейса, чего в базовой MS-DOS так никогда и не появилось, графика от Microsoft на PC – это была совершенно другая ОС Windows.

Из ОС, которые пользовались защищённым режимом процессора 80286, можно вспомнить серверную специализированную Netware-286, которая была разработана тогдашним недавним стартапом – компанией Novell. Система Netware-286 не была предназначена для работы на ней конечного пользователя, это был узко специализированный сетевой файловый и принт-сервер, удачно использовавший только набирающую тогда обороты технологию передачи данных для локальных сетей Ethernet.

Главной причиной невостребованности защищённой моды процессора 80286 была несовместимость с защищённым режимом программ, написанных для процессора 8086, в первую очередь из-за кардинально различных методов работы с памятью этих процессоров – селектор-смещение вместо сегмент-смещение.

Ещё одна проблемой являлось разделение сегментов кода и данных. В защищённом режиме нормально функционирующая программа не может модифицировать сегменты кода или выполнять код, помещённый в сегмент данных, а в программах для 8086 такие приёмы довольно широко практиковались.

---

<sup>1</sup> Впервые шина ISA появилась на компьютерах IBM PC в 1981 году. Это была 8-разрядная шина с частотой до 8 МГц и скоростью передачи данных до 4 Мбайт/с (передача каждого байта требовала минимум двух тактов шины). Шина имела 5 доступных линий запроса прерывания IRQ3 — IRQ7. Разъём состоял из 62 контактов, из которых 8 использовалось для данных, 20 — для адреса, остальные — для управляющих сигналов, а также подачи напряжений питания (GND, +5 В, -5 В, +12 В и -12 В).

<sup>2</sup> 1 августа 1984 года IBM объявила о выпуске нового поколения персональных компьютеров — IBM PC/AT — и приступила к разработке новой операционной системы для этих машин. Новая ОС должна была преодолеть ограничение DOS на 640 Кб памяти для прикладных программ и реализовать поддержку режима многозадачности. Параллельно с разработкой Windows корпорация Microsoft вместе с IBM включилась в активную работу по созданию системы, получившей наименование OS/2. Однако в начале 1990-х годов пути двух гигантов IT-индустрии разошлись. Microsoft независимо от IBM начинает разработку Windows 3.0. IBM, независимо от Microsoft, разворачивает работу над облегчённой версией OS/2, которая требовала бы меньше ресурсов, чем OS/2 1.2.

Некоторые проблемы возникали уже за пределами архитектуры центрального процессора, например, разработчики BIOS IBM PC проигнорировали рекомендации Intel в использовании зарезервированных номеров прерываний. И оказалось, что арифметический сопроцессор 80287 при возникновении ошибки генерирует прерывание 0x10, совпавшее с номером прерывания управления дисплеем, выбранном в IBM PC.

Все эти проблемы требовали значительной модификации существующих программ. Разработчики операционных систем были поставлены перед выбором — создавать относительно простую систему, работающую *только* в защищённом режиме, но несовместимую с большинством ранее написанных программ под MS-DOS, либо создавать систему, содержащую компоненты для работы и в реальном, и в защищённом режимах и переключающуюся между ними по мере необходимости, обеспечивая таким образом совместимость с ранее написанными программами.

В итоге для большей части IBM PC/AT-286 компьютеров возобладали вторая точка зрения — массовой системы, работающей только в защищённой моде, но не совместимой с прикладным программным обеспечением для MS-DOS, на рынке так и не появилось. Пользователи упорно держались за 16-разрядную ОС, которая могла адресовать только 1 Мбайт памяти, каковой она в итоге навсегда и осталась. Но как минимум обидно, имея «на борту» 2-4 Мбайта ОЗУ, продолжать втискиваться в ставшие уже очень тесными 640 Кбайт, эксплуатируя 286-ой просто как более быстрый 8086.

Но реализация второго подхода — ОС, переключающаяся между защищённым и реальным режимом, которая обеспечивала бы и совместимость с MS-DOS, и использование всех ресурсов 80286 (до 16 Мбайт физической памяти и до 1 Гбайта виртуальной) исходно была обречена на неудачу, благодаря решению разработчиков процессора Intel 80286. Ими было принято довольно странное решение — было сделано так, что команда переключения режимов работы процессора LMSW могла переключить CPU только из реального режима в защищённый. Но вот обратно, из защищённой моды в реальную, программный переход предусмотрен не был, это была дорога в один конец. Исходно 286-ой мог перейти обратно в реальный режим работы только... подачей сигнала «Сброс», который полностью инициализировал систему! Фантастическая нелепость. Интересно, что разработчики 286-го даже гордились этой особенностью процессора, аргументируя это примерно так: «Чтобы вашу защищённую дверь не взломали, подобрав к ней ключи, мы ликвидировали саму дверь, сделав невозможным обратный программный переход в реальный режим из защищённого<sup>1</sup>».

## Начальный сброс и переход в защищённый режим

По высокому уровню сигнала на входе RESET 286-ой процессор прекращает выполнение инструкций и перестает управлять локальной шиной. После аппаратного сброса процессор переходит к выполнению команды, считанной по физическому адресу 0xFFFF0h. Сброс (и только сброс!) переводит процессор из защищённого в реальный режим и устанавливает значения некоторых регистров. Только после сброса и до первой команды межсегментного перехода или вызова на шине адреса в реальном режиме биты A[20:23] в циклах выборки команд имеют единичное значения. Из этого следует, что по крайней мере на начальный период времени после сигнала RESET компьютер должен иметь образ BIOS в адресах 0xFFFFF0-0xFFFFF, в то время как PC на 8086/88 ROM BIOS располагалась под границей 0x0FFFFF. Перемещение BIOS из первого мегабайта памяти в старшие адреса «навсегда» невозможно, поскольку векторы прерывания, ссылающиеся на сервисы BIOS, в реальном режиме могут адресоваться только к памяти в диапазоне адресов 0x0...0x0FFFF

---

<sup>1</sup> Видимо опасаясь присутствия возможных ошибок в системе защиты процессора, в результате которых непривилегированный пользователь мог бы всё-таки заставить 286-ой перейти в реальную моду без сброса процессора, Intel и реализовала «одностороннюю» команду LMSW. В поздних поколениях 80286 Intel, похоже всё-таки осознав неудачность такого решения, добавила в процессор возможность программного перехода в реальный режим. Но это уже произошло на излёте эры 286-х, когда бал правили 32-разрядные 386-ые, в которых подобных странных решений не было изначально.



(0x0...0x10FFEF при открытом вентиле Gate A20). Таким образом, у PC/AT на процессоре 80286 (и старше) ROM BIOS отображается по крайней мере на две области памяти, расположенные под верхними границами первого и последнего мегабайтов физической памяти. Перевод процессора в защищённый режим осуществляется загрузкой в MSW слова с единичным значением бита PE (Protect Enable). Перед этим в памяти должны быть проинициализированы необходимые таблицы дескрипторов IDT и GDT, а в базовые регистры IDT и GDT должны быть занесены их физические адреса (24-битные) и размер. После выполнения инструкции LMSW, устанавливающий бит PE, сразу же должна выполняться команда внутрисегментного перехода JMP, для *очистки очереди инструкций*, декодированных в реальном режиме. Для установки регистров процессора в начальное состояние защищённого режима, предполагаемое системное ПО выполняет инструкцию JMP со ссылкой на селектор начального TSS системы. При этом загрузится регистр задачи, регистры LDT, регистры сегментов и общие регистры. Регистр задачи TR должен указывать на действительный TSS, поскольку операция переключения задач повлечёт сохранение текущего состояния задач.

Сложно, громоздко, но, по крайней мере, достижимо исходно чисто программными средствами.

## Переход из защищённого режима в реальный

Осознав проблему “one way ticket”, при разработке архитектуры IBM PC/AT инженеры IBM были вынуждены изобретать откровенные костыли для её преодоления. Воздействовать на разработчика процессора, по крайней мере, исходно, они не могли<sup>1</sup>, пришлось (вынужденно) искать обходные пути, т.к. «вечный» переход в защищённый режим был бы очень неприятной и неудобной особенностью системы PC/AT.

В результате был придуман весьма многословный, медлительный и тяжеловесный способ, состоявший из довольно большого количества шагов и не очень очевидных манипуляций с оборудованием и памятью – инженеры научились выполнять т.н. «мягкий» сброс процессора, не сбрасывая при этом всю систему. Для инициирования сигнала «Сброс» только для процессора в многострадальный регистр контроллера клавиатуры необходимо было послать «волшебную» комбинацию бит:

```
mov    ax, 0FEh      : команда сброса
out    64h, ax
```

Перед выдачей команды сброса процессора необходимо запомнить содержимое регистра SP, так как после передачи управления по адресу, записанному в области данных BIOS 0x0040:0x0067, регистр SS:SP будет указывать на стек BIOS. Кроме того, в энергонезависимой памяти (в микросхеме, скомбинированной с часами реального времени) нужно было в свободной ячейке 50-байтового CMOS ОЗУ поставить флаг-признак, который PC проверит при рестарте (некий признак «мягкого» сброса).

После выдачи команды сброса необходимо было подождать, когда произойдёт реальный сброс процессора, а это занимало несколько **миллисекунд**. Ожидание можно было выполнить, например, выдавая в цикле команду HLT.

Вот фрагмент программы, возвращающий процессор в реальный режим:

```
; Запоминаем содержимое указателя стека, так как после
; сброса процессора оно будет потеряно
```

```
mov    [real_sp],sp
```

---

<sup>1</sup> Здесь в очередной раз сказалась необычная для IBM история создания PC, когда, используя сторонний процессор, они вынуждены были бороться с ошибками и недостатками «чужого» устройства, вместо того, чтобы сразу сделать «как надо» в собственном процессоре.



```
mov    al,SHUT_DOWN
out    STATUS_PORT,al
```

```
wait_reset:
    hlt
    jmp    wait_reset
```

Для закрытия линии A20 можно воспользоваться следующей процедурой:

```

;-----
; Процедура закрывает адресную линию A20
;-----

```

Следующая последовательность команд размаскирует все прерывания:

```

mov    ax,000dh      ; разрешаем немаскируемые прерывания
out    CMOS_PORT,al

in     al,INT_MASK_PORT ; разрешаем маскируемые прерывания
and    al,0
out    INT_MASK_PORT,al
sti

```

Перезагрузившись по такой схеме «мягкого» сброса, процессор (работавший уже в реальном режиме) начинал выполнять процедуру начальной инициализации POST в BIOS<sup>1</sup>, где в самом начале проверял флаг в CMOS, по которому он определял, был ли сброс «холодным» (питание, кнопка «Сброс») или «мягким» (через статусный регистр контроллера клавиатуры). Во втором случае опускались все процедуры проверок и инициализации периферийных контроллеров (чтобы не нарушить работу системы, работавшей до этого в за-

317

щищённой моде), и система возвращалась в реальную моду (MS-DOS), в которой выполняла вызов процедуры (например, обращение в BIOS), ради которой и производился этот переход из защищённого режима в реальный.

При такой схеме перехода процессор реально сбрасывался, при этом процедура от подачи сброса до выполнения первой инструкции была весьма длительной, порядка в нескольких миллисекунд. И в течении этого времени процессор был «глух и нем», не выполнял никаких команд, не реагировал на прерывания и т.п. И это приводило к весьма неприятным последствиям при работе программного обеспечения, использовавшего переключение режимов работы процессора.

Например, знаменитые интегрированные среды разработки компании Borland – Turbo C, Turbo Pascal и др. – умели работать в защищённой моде процессора 286, используя технологию *расширителей* DOS (DOS Extender). Но это были прикладные программы, которым вынуждено приходилось исполнять роль операционной системы защищённого режима, т.к. ОС реального режима MS-DOS так и не научилась работать с x86 процессорами в любом режиме, отличном от реального.

Расширители DOS не заменяли полностью операционную систему, они всего лишь давали возможность программам достигать к увеличенному объёму ОЗУ, поэтому за базовыми сервисами ОС, типа вывода символов на экран, ввода символа с клавиатуры и т.п. программы вынуждены были обращаться к «настоящей» ОС MS-DOS. Но каждое такое обращение было очень и очень медленным (ведь каждое обращение требовало сброса процессора, а это небыстро). Например, если, работая в текстовом редакторе борландовской среды разработки, нажать любую клавишу на клавиатуре, то было заметно, насколько тормозит ввод-вывод символов на экран. Ведь для приёма каждого символа с клавиатуры и вывода его на экран приходилось каждый раз переключаться из защищённого режима в реальный и возвращаться обратно. В итоге машина, процессор которой работал на частотах порядка десятка мегагерц, не успевала обслуживать прерывания от клавиатуры, идущие с максимальной частотой 30 Гц, теряя до половины символов! Следствие плохо продуманного дизайна процессора 80286.

Microsoft в конце-концов всё-таки разработала пользовательскую операционную систему, которая умела использовать защищённый режим работы 286-го – Windows 3.0. Но, во-первых, это была не MS-DOS, и не полноценная самостоятельная ОС, а фактически надстройка над MS-DOS, которая здесь по сути выступала в роли загрузчика. И, во-вторых, произошло это много лет спустя после появления 80286 – 22 мая 1990, когда уже год на рынке присутствовал i80486. Есть такая поговорка: «Дорога ложка к обеду».

## **Другие ОС, работавшие в защищённом режиме 286-го**

В январе 1985 компания Digital Research анонсировала Concurrent DOS 286 — операционную систему, созданную совместно с Intel. Продукт должен был использовать только защищённый режим 80286, обеспечивая пользователям все преимущества этого режима по обеспечению многопользовательского, многозадачного выполнения программ, при одновременной поддержке эмуляции 8086. Заявленные возможности были работоспособны на использовавшемся в начале разработки прототипе процессора степпинга В-1, но в мае Digital Research обнаружила проблемы с эмуляцией на серийном процессоре степпинга С-1, которые не позволяли Concurrent DOS 286 запускать программы для 8086 в защищённом режиме. Выпуск Concurrent DOS 286 был задержан, а Intel начала разработку новой версии чипа. В августе, после интенсивного тестирования образцов 80286 степпинга Е-1, Digital Research подтвердила, что Intel исправила все документированные ошибки, но заявила, что остаются недокументированные проблемы с производительностью на пре-релизной версии Concurrent DOS 286. Intel заявила, что выбранный Digital Research подход к эмуляции программ для 8086 в защищённом режиме отличается от начальных спецификаций. Тем не менее Intel внесла незначительные изменения в микрокод процессоров степпинга Е-2, которые позволили Digital Research выполнять эмуляцию значительно быстрее.

Компания IBM в 1986 году выбрала Concurrent DOS 286 (переименовав её в IBM 4680 OS) операционной системой для компьютера IBM 4680, использованного в системе розничной торговли в качестве POS-терминала. Те же ограничения затронули версию 1.0 FlexOS 286, операционную систему наследницу Concurrent DOS 286, разработанную Digital Research в 1986 и представленную в январе 1987. (Эта система позже была использована IBM как основа IBM 4690 OS).

Из-за описанных выше проблем Билл Гейтс окрестил 80286 «чипом с мёртвым мозгом», так как было очевидно, что новая операционная система Microsoft Windows не сможет на 80286 эффективно выполнять несколько приложений MS-DOS одновременно. Возможно, это стало причиной раскола между Microsoft и IBM, так как IBM настаивала, чтобы OS/2, изначально совместная разработка IBM и Microsoft, поддерживала защищённый режим 80286.

## Прикладное ПО

С прикладным программным обеспечением под 286-ой был полный порядок, если порядком считать доступные системе ресурсы не более 1 Мбайт ОЗУ. Для поддержки PC/AT Microsoft выпустила третью версию MS-DOS v3.0, но это была всё та же DOS, со всеми её недостатками и ограничениями, при работе в которой, для утилизации доступных ресурсов процессора, приходилось использовать откровенные костыльные сторонние решения типа LIM EMS, DOS Extender и т.п. Откровенно неудачный, плохо продуманный процессор, вынуждено породивший большое количество странного, неуклюжего, компромиссного ПО, хотя количество выпущенных только IBM PC/AT на базе 286-го и исчислялось довольно солидной цифрой ~15 млн. штук.

## Архитектура компьютеров на базе Intel 80386/486

Выпустив 13 октября 1985 года процессор 80386, Intel перешла, образно говоря, в гораздо более «тяжёлую» весовую категорию, открыв этим устройством эру 32-разрядных x86 процессоров. Так и не «взлетевший» проект 32-разрядного iAPX32, разрабатывавшегося в Intel с 1975 года, был благополучно забыт, и в очередной раз сработало известное правило: «побеждает не лучший, а наиболее приспособленный». Архитектура, которую сама Intel позиционировала как нечто проходное, временное, успешно продолжила завоевание мира, уже в третьем своём поколении.

Наиболее серьёзные изменения в процессоре 386, по сравнению с 286-м: три режима работы (реальный, защищённый, виртуальный<sup>1</sup>), 26 новых инструкций, 32-разрядная шина адреса, позволявшая адресовать до  $2^{32} = 4$  Гбайт физической памяти. В новом процессоре была использована 32-битная плоская модель памяти, отличная от сегментной, использовавшейся до этого. Эта модель будет использоваться во всех процессорах архитектуры x86 вплоть до появления 64-битных технологий AMD64 и EM64T.

Такой серьёзный скачок, подобный переходу от 16-разрядных 8086/88/286 к 32-разрядному 386-му в дальнейшем в истории произошёл только ещё один раз, с анонсом 64-битной архитектуры x86-64. Что любопытно, автором 64-разрядной версии x86-архитектуры в этот раз была не Intel, а их «вечно второй» конкурент – компания AMD. Intel в то время, совместно с компанией Hewlett-Packard, разрабатывала «правильный» 64-разрядный процессор<sup>2</sup>, который, как они рассчитывали, наконец-то должен был прийти на смену «временной» x86-архитектуре.

---

<sup>1</sup> Если быть точнее, виртуальный v86 был скорее подрежимом защищённого режима.

<sup>2</sup> Речь идёт о процессоре Itanium — микропроцессоре с архитектурой IA-64 для серверов и рабочих станций, разработанном совместно компаниями Intel и Hewlett-Packard. Впервые был представлен 29 мая 2001 года.

## Поколение PC на основе процессоров i80386/486

Первым компьютером, использующим процессор i386, стал Compaq Deskpro 386. На тот момент Compaq стала первой «сторонней» компанией в истории, которая внесла существенные изменения в платформу PC. До того момента новые компьютеры первой всегда выпускала IBM. Она могла оказаться первой и на этот раз, но у IBM был долгосрочный контракт на использование 286-х процессоров, и в компании предпочли уделить 16-битной платформе ещё некоторое время. Как показала история, этот шаг стал довольно большой ошибкой. Deskpro 386 отлично продавались, поэтому к моменту запуска первых компьютеров IBM на базе 386-го процессора (семь месяцев спустя после выхода Deskpro 386) компания уже утратила свои лидирующие позиции, в лидерах оказалась как раз компания Compaq, которая в итоге сумела заметно «перекроить» весь рынок десктопов, а в дальнейшем – и x86-серверов. Возросла конкуренция, и влияние IBM на рынке PC стало уже не столь существенным.

Compaq не была начинающим выскочкой к 1986 году. Их ноутбук Compaq Portable имел невероятный успех в начале 1980-х, Compaq была любимцем отрасли, и Deskpro 386 укрепил позиции Compaq как отраслевого новатора.

Compaq Deskpro 386 не был чем-то запредельно революционным, но как оказалось, необходимости в этом и не было. Адекватное программное обеспечение, которое выявило бы слабости Compaq386, ещё не существовало. По большей части это был клон компьютеров на базе 286-го, с минимально необходимым количеством изменений, чтобы заставить работать на этой материнской плате процессор 386. Первоначально Deskpro 386 оснащался процессором i80386 с частотой 16 МГц. По скорости он был примерно в три раза быстрее, чем IBM PC/AT на базе 6 МГц 286-х, и в два раза быстрее, чем самые быстрые из доступных 286-х систем того времени.

Intel выпустила 80386 в октябре 1985 года, в требуемых объёмах процессор стал доступен к июню 1986 года.

По словам тогдашнего президента Compaq Рода Каниона (Rod Canion), если IBM не выпустит компьютер с процессором 386 в течение шести месяцев, то Deskpro 386 станет отраслевым стандартом. IBM потребовалось семь месяцев, чтобы ответить своей моделью PS/2<sup>1</sup> 80, а Deskpro 386 действительно стал стандартом.

## Цена Compaq Deskpro 386

Цена была, конечно же, немаловажным фактором в битве за лидерство на рынке PC. Базовая цена Deskpro 386 составляла 6499\$ за систему с 1 МБ ОЗУ, MS-DOS 3.1, одним гибким дисководом объёмом 1.2 МБ и жёстким диском объёмом 40 МБ с временем поиска 30 мс. Цена не включает монитор или видеокарту. Базовая CGA/монохромная видеокарта стоила 199\$, а EGA – 599\$, цветной монитор стоил 799\$, полная система с видеокартой и монитором стоила почти 8 000\$. В долларах 2017-го года это было бы почти 18 000\$.

И тогда, и по современным меркам это была дорогая система, но она всего на 23% превосходила по цене IBM PC/AT, будучи при этом вдвое производительнее.

## Спецификация Compaq Deskpro 386

Compaq Deskpro 386 имел 32-битный слот расширения для оперативной памяти (могло быть использовано до 14 мегабайт ОЗУ), четыре 16-битных слота и три 8-битных слота. Все слоты, кроме 32-битного слота, были разъёмами шины ISA. Deskpro 386 поставлялся с 32-битной платой памяти с 1 мегабайтом оперативной памяти с завода. Пять слотов были доступны для пользовательских плат расширения.

---

<sup>1</sup> История попытки реванша IBM на рынке персональных компьютеров, с выпуском семейства IBM PS/2, будет приведена ниже.

Наличие 8- и 16-разрядных слотов замедлило работу системы в целом, но до настоящих 32-разрядных операционных систем оставалось ещё несколько лет. По сути, это была быстрая для своего времени 16-битная система.

Deskpro 386 имел гнездо для математического сопроцессора Intel 80287 с тактовой частотой 8 МГц. Это не было ошибкой, т.к. i80387 на момент выхода Deskpro386 ещё не был доступен. Компромиссное решение с медленным сопроцессором i80287 было всё же быстрее, чем без сопроцессора вообще. Как только Intel 80387 появился на рынке, Compaq пересмотрела дизайн материнской платы, чтобы получить возможность использования на выбор математического сопроцессора 287 или 387.

Машина имела четыре 5,25-дюймовых отсека для хранения, каждый из которых мог вместить 5,25-дюймовый дисковод гибких дисков, 40-мегабайтный стриммер (магнитофон) для резервного копирования или жёсткий диск. Распространённые заводские конфигурации обычно включали 1.25 Мбайт 5,25-дюймовый флоппи-дисковод и жёсткий диск объемом 40, 70 или 130 Мбайт.

## **Наследие Compaq Deskpro 386**

Compaq Deskpro 386 по сути вынудил IBM выпустить свой компьютер на базе 386. IBM сопротивлялась выпуску PC на базе 80386, потому что они конкурировали бы с мини-компьютерами IBM 4300-й серии, а сама с собой IBM конкурировать не хотела, следуя своей внутрикорпоративной политике.

Compaq использовала это в своей рекламе, позиционируя Deskpro 386 как альтернативу рабочим станциям и мини-компьютерам, рекламируя его практически аналогичную производительность наряду с возможностью запуска стандартного программного обеспечения для DOS, такого как электронные таблицы Lotus 1-2-3. IBM рассматривала 386-й как конфликт интересов. Но для Compaq создание самого быстрого и мощного ПК с технологией 1986 года было возможностью выйти в лидеры. Compaq правильно сделала ставку на то, что ПК с достаточной мощностью, чтобы конкурировать с профессиональными рабочими станциями и обладавшие при этом обратной совместимостью с MS-DOS, приобретут популярность. В руководстве пользователя Deskpro386 Compaq правильно предсказала, что высококачественное мощное программное обеспечение, ранее не практиковавшееся для PC, вскоре станет широко доступным. В 1987 году SCO выпустила XENIX 386, порт System V Unix, который мог работать на таких системах, как Deskpro 386, что позволяло ему функционировать как высокопроизводительная Unix рабочая станция или сервер.

В конечном счете именно Compaq Deskpro 386 дал Microsoft и Intel возможность перейти от низкопроизводительного настольного компьютера к рабочим станциям среднего уровня и даже к доступным настольным высокопроизводительным вычислениям.

Процессор следующего (четвёртого) поколения Intel 80486 закрепил и развил успех, достигнутый первым 32-разрядным x86 микропроцессором 80386. Вместо большого количества несовместимых между собой рабочих станций от различных производителей, построенных на проприетарных закрытых аппаратно-программных платформах, индустрия выбрала общий знаменатель – 32-разрядную интеловскую архитектуру – доступную, открытую, унифицированную. Которая в скором времени успешно вторглась не только на рынок производительных рабочих станций, но и на серверный рынок. Не забыв при этом и нижний сегмент – дешёвые ПК, ноутбуки. Где платформе x86 не удалось завоевать сколь-нибудь значительную долю рынка – это мир мобильных устройств. В этой же области, кстати, так и не удалось серьёзно закрепиться другому участнику конгломерации Wintel – компании Microsoft. Сегодня количество выпускаемых процессоров архитектуры ARM, основы подавляющего большинства мобильных устройств – смартфонов, планшетов и т.п. – в несколько раз превосходит объёмы выпуска процессоров x86, а поддержка мобильной

Windows была прекращена в декабре 2019 года, и тем самым было признано её полное поражение перед iOS и Android<sup>1</sup>.

## **Конкуренция на рынке систем на базе 80386**

Несколько конкурентов предложили свои собственные компьютеры на базе 386 в течение нескольких месяцев после выпуска Deskpro 386. Но возможность быть первой дало Compaq свои преимущества. Одним из первых её клиентов был Microsoft. Разработчики Microsoft использовали Deskpro 386s, потому что они были самыми быстрыми компьютерами на рынке. В результате в конце 1980-х и начале 1990-х годов продукты Microsoft, такие как Windows 3.0, очень хорошо работали на оборудовании Compaq. Поэтому, если вы хотели запустить Windows, было бы неплохо купить для этого именно Compaq.

IBM последовала за ней в 1987 году со своей линейкой PS/2, которая включала проприетарную шину расширения под названием Micro Channel, лицензию на клонирование IBM продавала уже за очень приличную цену, в отличие от открытой идеологии PC первых поколений. Compaq ответила тем, что совместно с другими производителями клонов разработала открытый конкурирующий стандарт под названием EISA. EISA была не так хороша, как Microchannel, но цена победила возможности. EISA и будущие открытые стандарты, такие как VESA и PCI, действительно победили, и в конечном итоге даже IBM вернулась на этот рынок, используя эти открытые решения.

## **Если вы коллекционер 386-ых**

Ранние 386-ые машины очень ценятся у коллекционеров. Они появляются на аукционах не очень часто, т.к. в своё время не продавались в огромных количествах из-за достаточно высокой цены. Многие из них были выведены из эксплуатации и «отправлены в отставку» из-за проблемы 2000-го года, независимо от того, действительно ли у систем были проблемы с Y2K или нет. Многие ИТ-специалисты использовали Y2K как предлог для избавления от стареющих, но продолжавших надёжно и стабильно работать, систем.

До появления компьютеров на базе 386-х, проприетарные Unix-рабочие станции и мини-компьютеры доминировали на рынке высокопроизводительных вычислений. Наличие доступных массовых компьютеров на 386-х сделало возможным взрывное развитие Linux и Windows NT. Такие громкие имена, как Digital Equipment Corporation, Sun Microsystems и даже IBM, со временем исчезли в результате появления Intel 80386. Deskpro 386 был первым из этой волны. Современная рабочая станция PC является духовным потомком Compaq Deskpro 386.

## **Попытка IBM вернуться на рынок PC, семейство IBM PS/2**

С многомесячным опозданием, в 1987 году, через полтора года после появления процессора i80386, IBM всё же решает вернуться на рынок x86 ПК, который она когда-то и создала. Засидевшись на старте 32-разрядной эры, IBM утратила лидерство на этом направлении, уступив его компании Compaq. Летом 1987-го состоялся анонс новой линейки IBM PC-совместимых систем Personal System/2 (PS/2), которая оказала довольно серьёзное влияние на рынок персональных компьютеров.

К 1987-у году положение IBM на ею же созданном рынке персональных компьютеров было не слишком выигрышным. К тому времени целая масса компаний выпускала клоны IBM PC – компьютеры, архитектурно близкие к решениям IBM, которые к тому же позволяли запускать ПО от IBM. Одной из главных причин такого «произвола» являлось отсутствие правовых ограничений на создание BIOS, сопоставимого по функционалу с оригинальным – тем, что применялся в IBM PC. Логично, что производители «клонов» откусили у IBM весьма серьёзную долю рынка: если в 1983-м году компания контролировала порядка 76 его процентов, то в 1987-м – уже только 26.

---

<sup>1</sup> К слову: технически Android – это реализация Java-машины на Linux.

В IBM разработали план по возвращению доли рынка и, следовательно, прибыли. Компания решила выпустить новую серию компьютеров – названную PS/2 – с проприетарной шиной расширения MCA<sup>1</sup>, фирменной операционной системой и новым BIOS-ом, за использование которого компаниям – «специалистам по клонированию», пришлось бы платить большие лицензионные отчисления. Таким образом IBM рассчитывала навязать высокотехнологичным конкурентам свои правила игры, но... не удалось: несмотря на то что PS/2 оказали сильное влияние на индустрию персональных компьютеров, IBM так и не смогла вернуть себе неоспоримое лидерство на этом рынке.

## **Атака клонов**

Свой первый PC компания IBM выпустила в 1981-м. Проблема заключалась в том, что этот компьютер был собран из широко распространённых стандартных комплектующих, которые мог приобрести кто угодно – в том числе и конкуренты IBM. Впрочем, компания реализовала в своих компьютерах ряд разработок, которые все же смогли сделать их уникальными. Первая такая разработка – это BIOS, системное программное обеспечение, которое контролировало аппаратные компоненты и процесс работы компьютера. Второй компонент уникальности – дисковая операционная система, разработанная Microsoft.

В контракте на поставку PC-DOS (а именно так эта ОС и называлась) компании IBM был пункт, который позволял Microsoft продавать данную операционную систему и другим производителям компьютеров. Поскольку в Microsoft, как и в любой другой крупной американской компании, работают закоренелые прагматики, буквально сразу же операционная система, сменившая название на MS-DOS, стала доступна конкурентам.

Поначалу это не было серьёзной проблемой: обладатели конкурирующих компьютеров с MS-DOS не могли легально использовать полный набор выпущенного IBM программного обеспечения и аппаратных дополнений. Влияние и прибыли IBM пошли вверх, а другие компании, выпускавшие PC-совместимые компьютеры, задумались о новых – более эффективных и актуальных – методах «клонирования». Прежде всего им предстояло реконструировать код BIOS, используя методологию «чистой комнаты» (способ копирования дизайна аппаратного или программного обеспечения с помощью так называемой обратной инженерии). Это было необходимо для предотвращения нарушений авторских прав IBM, за которыми наверняка бы последовали судебные иски.

## ***Первый «клон»: MPC 1600***

Долго ждать не пришлось: первый «клон» был выпущен в июне 1982-м году компанией Columbia Data Products. В 1983-м появились аналогичные решения от Dynalistic и Compaq, а дальше и вовсе наступило время «ужаса» IBM: компании вроде Phoenix Technologies разработали аналоги BIOS для IBM PC-совместимых компьютеров, которые могли лицензировать все желающие производители ПК. Этот момент можно считать переломным: IBM начала терять влияние на рынке персональных компьютеров.

Конечно, процесс «потери власти» был достаточно долгим – в первые годы рынок компьютеров не мог существовать без IBM, своего главного инноватора. Так, в моделях IBM PC XT (1983-й год) и PC AT (1984-й) было реализовано немало интересных технических решений, которые, впрочем, были быстро скопированы конкурентами.

Звание «главного инноватора» у IBM не раз пытались отобрать, и кое-кому это всё же удалось – а именно той самой компании Compaq с её моделью DeskPro 386, в которой

---

<sup>1</sup> **MCA** (Micro Channel Architecture, архитектура Micro Channel) — системная шина, представленная в 1987 году корпорацией IBM вместе с новым семейством персональных компьютеров IBM PS/2. Отличается от существовавшей на тот момент, и предлагаемой к замене, архитектуры Intel ISA более совершенными возможностями, предоставляемыми разработчикам вычислительных систем. К наиболее интересным из них можно отнести: использование программируемых ключей (Programmable Option Select), механизма многопользовательского доступа к шине (Multi-Device Interface), аппарата уравнивания прерываний совместного использования и протокола Master.

первым был использован процессор Intel 80386 и которая, по большинству параметров, превосходила решения IBM того времени. «Голубой гигант» серьезно задумался и приготовился к ответным действиям – чтобы вернуть себе славное имя, влияние и деньги.

Этим самым ответным действием и стали IBM PS/2. Выпуск этих компьютеров, состоявшийся в апреле 1987-го года, сопровождался мощной рекламной кампанией с выступлениями бывших участников хитового шоу MASH TV, а также новым слоганом: «PS/2 It!». Наблюдатели, издавшие и более мощные машины с более приятными ценниками, не особо впечатлились. Всем было понятно, что с помощью PS/2 IBM попытается поставить подножку производителям PC-совместимых компьютеров. И отчасти ей это удалось: в новинках был реализован ряд наработок, задержавший «специалистов по клонированию» на пару лет и дал IBM возможность частично наверстать упущенное.

## Четыре первые модели

В ходе презентации, состоявшейся в апреле 1987-го года, IBM продемонстрировала четыре модели: Model 30, Model 50, Model 60 и Model 80. Они заметно различались по стоимости и характеристикам: так, Model 30 (фактически, наследник IBM PC XT) была оснащена 8-МГц процессором Intel 8086, 640 Кбайт оперативной памяти и жёстким диском на 20 Мбайт. Стоимость этой новинки составляла 2 295\$ (на сегодня<sup>1</sup> это более 4 500\$ – если учитывать инфляцию).

Самая мощная конфигурация – Model 80 – оценивалась уже в 10 995\$ (более 22 000 долларов в наши дни). Она располагала процессором Intel 386 с частотой 20 МГц, 2 Мб оперативной памяти и жёстким диском ёмкостью 115 Мб.

Операционная система PC-DOS 3.3 не входила в комплект поставки ни одной из моделей – она предлагалась за дополнительные 120\$ долларов (242 доллара сегодня).

Ниже, на Рис.271, приведена таблица, в которой содержатся характеристики всех моделей IBM PS/2, анонсированных в апреле 1987-го года:

| All models include integrated display support, 256-color graphics capability, clock/calendar, and ports for serial, parallel and pointing devices. All systems use a common IBM enhanced keyboard and accept any IBM Personal System/2 monochrome or color display. All models accept the 200MB IBM 3363 Optical Disk Drive option. |                      |                                    |                                    |                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|------------------------------------|------------------------------------|-------------------------------------|
|                                                                                                                                                                                                                                                                                                                                     | Model 30             | Model 50                           | Model 60                           | Model 80                            |
| <b>Microprocessor</b>                                                                                                                                                                                                                                                                                                               | 8086                 | 80286                              | 80286                              | 80386                               |
| <b>Potential system throughput<sup>1</sup></b>                                                                                                                                                                                                                                                                                      | Up to 2½ times PC XT | Up to 2 times Personal Computer AT | Up to 2 times Personal Computer AT | Up to 3½ times Personal Computer AT |
| <b>Standard Memory</b>                                                                                                                                                                                                                                                                                                              | 640KB                | 1MB                                | 1MB                                | Up to 2MB                           |
| <b>Expandable to</b>                                                                                                                                                                                                                                                                                                                |                      | 7MB                                | 15MB                               | 16MB                                |
| <b>Diskette size and capacity</b>                                                                                                                                                                                                                                                                                                   | 3.5 inch 720KB       | 3.5 inch 1.44MB                    | 3.5 inch 1.44MB                    | 3.5 inch 1.44MB                     |
| <b>Fixed disk<sup>2</sup></b>                                                                                                                                                                                                                                                                                                       | 20MB                 | 20MB                               | 44, 70MB                           | 44, 70, 115MB                       |
| <b>Additional Options</b>                                                                                                                                                                                                                                                                                                           |                      |                                    | 44, 70, 115MB                      | 44, 70, 115MB                       |
| <b>Maximum configuration<sup>3</sup></b>                                                                                                                                                                                                                                                                                            | 20MB                 | 20MB                               | 185MB                              | 230MB                               |
| <b>Expansion slots<sup>4</sup></b>                                                                                                                                                                                                                                                                                                  | 3                    | 3                                  | 7                                  | 7                                   |
| <b>Operating system(s)</b>                                                                                                                                                                                                                                                                                                          | PC DOS 3.3           | PC DOS 3.3 and Operating System/2  | PC DOS 3.3 and Operating System/2  | PC DOS 3.3 and Operating System/2   |

1. Based on the testing described in the IBM Personal System/2 Performance Guide. Your results may vary. 2. Model 30 also comes in a diskette-based configuration. 3. Models with 44MB fixed disk expandable to 88MB. 4. Model 30 accepts most IBM PC and IBM PC XT option cards. Models 50, 60 and 80 accept new IBM Micro Channel option cards.

Рис.271 – четыре первых модели семейства PS/2

В каждой модели была реализована по меньшей мере одна функция, которая ранее не встречалась ни в продуктах IBM, ни в любом другом персональном компьютере на

<sup>1</sup> Текст написан в июле 2012 года, к 25-летию выхода семейства PS/2 ([IBM PS/2: легенда отмечает 25-летие](#))



рынке. Далее будут рассмотрены особенности семейства IBM PS/2 и дана оценка влиянию, которое эти машины оказали на компьютерную индустрию.

### ***Интегрированная система ввода/вывода, новый стандарт памяти***

Со времён первого IBM PC, выпущенного в 1981-м году, компания IBM предпочитала наделять персональные компьютеры минимальным необходимым набором функций. Пользователям же предлагалось самостоятельно наращивать функционал системы с помощью плат расширения, подключавшихся к соответствующим слотам. Правда, очень уж широких возможностей для персонализации компьютеров их обладателям IBM не предоставляла: скажем, в IBM PC образца 1981-го года таких слотов было пять, и четыре из них были заняты графической картой, контроллером дисководов, последовательным портом и платой для подключения принтера.

В компьютерах PS/2 IBM решила интегрировать большинство основных функций ввода/вывода в материнскую плату. В каждой машине этой серии были встроенные последовательный и параллельный порты, разъём для подключения мыши, видеоадаптер и контроллер дисководов; таким образом освобождались слоты расширения, которые пользователь мог использовать по своему усмотрению.

В компьютерах серии PS/2 было ещё несколько интересных технических решений. Например, чип 16550 UART, который обеспечивал более высокую скорость последовательного подключения (полезно при использовании модема), а также 72-пиновые слоты для установки модулей оперативной памяти формата RAM SIMM. Оба этих решения стали стандартами для компьютерной индустрии.

### ***Порты для подключения мыши и клавиатуры***

Каждый компьютер серии PS/2 был оснащен портами нового типа для подключения мыши и клавиатуры, каждый из которых использовал 6-контактный коннектор формата mini-DIN. IBM позиционировала мышь как периферийное устройство, которое сыграет ключевую роль в управлении компьютером – в планах компании был выпуск новой мультиоконной операционной системы с графическим интерфейсом, которая могла бы конкурировать с Macintosh от Apple.



**Рис.272 - Разъёмы PS/2 для клавиатуры (сиреневый, сверху) и мыши (зелёный, снизу)**

Сегодня уже практически невозможно встретить новые компьютеры, оснащённые разъёмами PS/2 для мыши и клавиатуры, этот функционал теперь реализуется использованием шины и разъёмов стандарта USB, но лет десять-пятнадцать назад зелёно-сиреневые разъёмы PS/2 были штатной принадлежностью многих ПК (Рис.272).

### ***Новые дисководы***

Компьютеры IBM PS/2 поставлялись с 3.5-дюймовыми дискетами нового типа, разработанными компанией Sony. Ранее эти носители уже использовались в Macintosh.

Младшая модель, PS/2 Model 30, комплектовалась дискетой двойной плотности, которая вмещала 720 Кбайт данных. Со всеми же остальными компьютерами данной серии поставлялись дискеты нового формата: с высокой плотностью записи, ёмкостью в 1 440 Кбайт. Дискеты такого объема стали стандартом на 20 следующих лет.

Использование 3.5-дюймовых дискет было в диковинку для PC-индустрии. Более того, до этого момента IBM сама использовала исключительно накопители в формфакторе 5.25 дюйма и казалась убежденным его сторонником. Так или иначе, поначалу недовольных столь резкой сменой формата было куда больше, чем довольных: у многих пользователей были огромные библиотеки программного обеспечения, записанные именно на привычные 5.25-дюймовые дискеты.

Впрочем, IBM не оставила таких пользователей в беде: специально для них предлагался внешний дисковод, работающий с дискетами в формфакторе 5.25 дюйма. Однако вскоре стандартом стали именно 3,5 дюйма: «специалисты по клонированию» начали поставлять такие дискеты со своими компьютерами, а разработчики программного обеспечения стали предлагать свои продукты на носителях обоих формфакторов, дабы угодить и «староверам», и идущим в ногу со временем.

## ***VGA и MCGA***

Одним из наиболее значимых для истории технических решений, реализованных в IBM PS/2, стал стандарт Video Graphics Array (VGA).

Он позволял отображать на дисплее компьютера картинку с разрешением 640 x 480 точек и палитрой в 16 цветов, а также 320 x 200 пикселей и 256-цветной палитрой. Такие параметры можно было назвать настоящим прорывом для рынка PC-совместимых компьютеров того времени. Также VGA был совместим с более ранними стандартами от IBM, а именно Enhanced Graphics Adapter (EGA) и Color Graphics Adapter (CGA).

Именно в PS/2 был впервые реализован порт, который мы сегодня называем VGA – 15-контактный разъём типа D-sub, он также стал стандартом для отрасли.

При этом в младшем компьютере серии – Model 30 – использовался более простой графический адаптер MCGA, который мог выводить изображение с разрешением в 320 x 200 точек и палитрой в 256 цветов, однако в режиме VGA (640 x 480 пикселей) он вообще был неспособен вывести цветную картинку – только монохромную. Кроме того, он не поддерживал стандарт Enhanced Graphics Adapter (EGA). Вскоре MCGA использовать перестали – конкуренты вообще не посмотрели в сторону этого решения, да и сама IBM задействовала его лишь в нескольких дешёвых версиях PS/2.

## ***Микроканальная архитектура***

Одним из самых важных аппаратных улучшений, реализованных в PS/2, была новая шина расширения Micro Channel Architecture, упоминавшаяся выше. Слоты MCA для плат расширения функциональности имелись практически во всех компьютерах PS/2 за исключением Model 30, в которой было предусмотрено три слота формата ISA (он использовался в оригинальном IBM PC образца 1982-года). ISA поддерживался почти всеми производителями PC-совместимых компьютеров, так что в машину компании А можно было без проблем установить плату компании В, в том числе и IBM. Последнюю эта ситуация по понятным причинам категорически не устраивала, и для серии PS/2 было решено разработать новую шину, конструкцию которой можно было лицензировать, а использование – контролировать, остановив таким образом поток «клонов». Ведь их производителям пришлось бы выплачивать IBM внушительные суммы за лицензию.

К середине 80-х ISA уже была медленным и устаревшим решением, тогда как MCA можно было назвать передовым: за счет увеличения ширины шины данных с 16 до 32 бит (впрочем, поддерживался и 16-битный режим) и повышения частоты передачи с 8 до 10 МГц. Теоретическая пропускная способность MCA достигала 66 МБайт/с (на практике — максимум 40 МБайт/с, что всё равно в несколько раз больше пропускной способности ISA). Устройства наконец-то могли общаться друг с другом напрямую, минуя центральный процессор. С такими улучшениями MCA могла бы стать индустриальным стандартом, но IBM сама всё испортила, не став развивать рынок периферии для новой шины, более того, тщательно тормозила этот процесс — сторонние производители должны были получать специализированный ID для каждого устройства, за право выпуска устройств под MCA нужно было платить лицензионные отчисления и роялти, при том, что IBM, не желая раскрыть технические данные новой шины, её никому не лицензировала.

MCA также предоставляла ограниченную форму Plug-and-Play функциональности: каждая плата расширения имела уникальный 16-битный идентификационный номер, который позволял IBM PS/2 автоматически определять и настраивать их.

В теории такой подход был гораздо проще и удобнее предлагавшегося в ISA, но на практике всё оказалось несколько сложнее. Устаревшие версии IBM Reference Disk (утилиты, которая позволяла менять настройки системы) не знали идентификаторов новых плат, что заставляло IBM регулярно обновлять данное ПО. При этом получать новые версии могли далеко не все пользователи – не будем забывать, что взлёт (а затем и падение) IBM PS/2 произошёл задолго до масштабного проникновения Интернета...

Да и производители PC-клонов приняли стандарт MCA без особых восторгов: буквально через год после его внедрения в моделях IBM консорциум из девяти компаний, выпускающих персональные компьютеры, представил свой собственный стандарт – EISA<sup>1</sup>. По сути он являлся расширенной версией ISA с пропускной способностью в 32 бита, за использование которой требовалось выплачивать минимальные лицензионные отчисления. EISA появился в ряде компьютеров, однако до начала 90-х годов, когда Intel представила PCI, в компьютеростроении всё же доминировала 16-битная ISA. Подробнее об устройстве шины EISA, имевшей довольно оригинальную концепцию и совместимую механически, электрически и логически с ISA, будет изложено дальше по тексту.

## OS/2

MCA популяризации IBM PS/2 не очень поспособствовал, однако у компьютеров этой серии был ещё один серьёзный козырь – OS/2, новая операционная система с поддержкой оконного графического интерфейса. В IBM рассчитывали, что эта программная платформа позволит в полной мере реализовать потенциал 386-го процессора в Model 80, используя в качестве основного средства управления компьютером мышь, а также сможет успешно конкурировать с Apple и её Macintosh.

Как стало понятно позже, IBM здесь допустила (очередную) серьёзную ошибку: она начала разработку OS/2 совместно с компанией Microsoft, создателем PC-DOS.

Microsoft в то время непрерывно росла и богатели: компания продавала лицензии на право использования MS-DOS всем производителям PC-клонов, и создание новой, общей с IBM проприетарной ОС для компьютеров в её стратегические планы не входило. Тем более что Microsoft в то время уже всю тестировала собственный продукт, который многое почерпнул из первых версий OS/2, – Windows 2.0. В итоге IBM начала разработку собственного варианта OS/2 (в нём «Голубой гигант» планировал самостоятельно реализовать все запланированные инновации), который был выведен на рынок в конце 1988-го – спустя полтора года после коммерческого запуска Windows 2.0. Вот и получилось, что все сливки с рынка ПК, нуждавшегося в новой ОС с графическим интерфейсом, сняла Microsoft, а IBM осталась ни с чем. Между тем по рынку прошло сразу несколько волн слухов о теориях некоего заговора, который отбросил начавшую выкарабкиваться из кризиса IBM в прошлое и вознёс до небес Microsoft.

## Конец эпохи IBM

Что касается конкретно PS/2, то эти компьютеры первое время продавались довольно неплохо (к январю 1988-го года было реализовано порядка 1.5 млн. экземпляров), однако стоили они относительно дорого – и поэтому в конечном итоге проиграли войну за потребителя машинам, предлагаемым производителями PC-«клонов». Тем более что все интересные технические решения, применённые в PS/2, со временем появлялись и в моделях конкурентов. В итоге в конце 80-х годов продажи компьютеров серии PS/2 резко упали, и эти устройства, как ни странно это звучит, превратились для IBM в бремя.

---

<sup>1</sup> EISA (Extended Industry Standard Architecture «расширенная ISA») — шина для IBM-совместимых компьютеров. Была анонсирована в конце 1988 консорциумом из девяти основных производителей IBM-совместимых компьютеров (Compaq, Hewlett-Packard, Epson, NEC, Olivetti, AST Research, Tandy, Wyse и Zenith Data Systems) как ответ на введение фирмой IBM новой скоростной (по сравнению с устаревающей ISA), но закрытой проприетарной шины MCA в компьютерах серии PS/2.

В 1990-м году стало совершенно очевидным, что IBM не является ни лидером рынка PC, ни его главным инноватором. А в 1994-м IBM проиграла ещё и в цифрах: крупнейшим поставщиком компьютеров на рынке США стала компания Compaq.

Однако всё это не помешало IBM оставаться на рынке персональных компьютеров до 2004-го года, когда она продала свое подразделение, занимавшееся разработкой и производством таких устройств, китайской компании Lenovo. В девяностых и нулевых IBM удалось привнести на рынок целый ряд интересных решений (в области графических стандартов и портативных компьютеров, в частности, линейки ThinkPad), однако стать инициатором новых технических революций, как в случае PS/2, IBM уже так и не смогла.

## Архитектура микропроцессоров i80386/486

Первый процессор семейства 386 имел тактовую частоту 16 МГц (в реальности, из-за проблем с производством, 386-ой дебютировал на частоте 12 МГц). После выпуска процессоров семейства 386SX<sup>1</sup> процессоры этой серии получили индекс “DX” — **D**’ouble-word **eX**ternal, что указывало на его 32-разрядную внешнюю шину. Процессор позиционировался как производительное решение для настольных систем. Производился процессор по CHMOS IV технологии и потреблял 400 мА, что было значительно меньше, чем у Intel 8086. 16 февраля 1987 года была анонсирована модель с частотой 20 МГц; 4 апреля 1988 года с частотой 25 МГц и 10 апреля 1989 года с частотой 33 МГц. Процессор выпускался в корпусах PQFP-132 (такие процессоры имели литеру «NG» в начале названия, например, NG80386DX25) или в керамическом PGA-132 (такие процессоры имели литеру «A» в начале названия, например, A80386DX25). На кристалле процессора располагалось примерно 275 тыс. транзисторов, размещенных на подложке, площадью приблизительно в 104 мм<sup>2</sup>. Процессор изготавливался по технологии сначала 1.5 мкм, затем 1 мкм. Процессор 80386 не содержал никакого кэша для команд или данных, общий кэш первого уровня располагался непосредственно на материнской плате и был опцией. Процессор не содержал блока операций с плавающей точкой, для этого использовался отдельный сопроцессор i80387, который был выпущен несколько позднее, из-за чего самый первый Compaq386 имел гнездо для сопроцессора предыдущего поколения, 80287, работавшего асинхронно на более низкой частоте и имевшего более низкую производительность на такт. Процессор не умножал частоту и работал всегда на частоте шины, от 12 до 33 МГц.

Для компании Intel процессор 80386 был прорывным сразу в нескольких направлениях. Кроме очевидного перехода на архитектуры x86 на 32 разряда и всех сопутствующих этому улучшений, это был первый в истории процессор x86, лицензию на производство которого не получил ни один из производителей процессоров-клонов. Когда в своё время IBM выбрала Intel поставщиком процессоров для IBM PC, одним из условий сделки было лицензирование процессора нескольким производителям. IBM не хотела зависеть от одного поставщика. Intel раздала довольно много лицензий, и клоны процессоров с 8088 по 80286 выпускались множеством компаний, от AMD до Siemens. Несмотря на то, что разработчиком процессора была Intel, и основные затраты на создание процессора также несла Intel, доля компании на рынке была далеко не самой большой. В старых компьютерах IBM, например, гораздо чаще можно найти процессор AMD, чем процессор Intel. Это сказывалось на финансовом состоянии компании, и руководству необходимо было что-то предпринять. Так и было принято решение не лицензировать 386-ой процессор. Теперь производители процессоров-клонов должны были сами вкладывать деньги и время в разработку своих чипов. Многие компании отказались от этого рынка, какое-то время производя ускоренные 286-ые (Harris выпустила 286-ой процессор, работавший на частоте 25 МГц, в то время, как

---

<sup>1</sup> Процессоры семейства SX позиционировались как решения для настольных ПК начального уровня и портативных ПК. Семейство SX отличалось от семейства DX тем, что у него разрядность внешней шины данных составляла 16 бит, а разрядность внешней шины адреса — 24 бит, в результате чего процессор мог адресовать только 16 Мбайт физической памяти, что делало вычислительные системы, построенные на его основе, аппаратно совместимыми с предыдущими процессорами Intel 80286.

Intel остановилась на 12.5 МГц), а некоторые решили всё же не уходить с рынка, и разрабатывали свой дизайн. Однако теперь у Intel появилось преимущество во времени, какое-то время 32-разрядный x86 процессор был только у Intel (это была солидная фора, так, например, AMD Am386, из-за судебных тяжб с Intel, вышел лишь в 1991 году, почти через 6 лет после дебюта 80386 и более, чем через год после Intel 80486!).

Из-за требований IBM иметь нескольких поставщиков процессора, в Intel серьёзно опасались, что IBM вообще откажется от использования 80386, но игра стоила свеч, ведь к тому времени IBM PC-совместимые компьютеры производились огромным количеством третьих фирм. И производители PC-клонов сыграли на руку Intel. Как уже рассказывалось выше, воспользовавшись задержкой со стороны IBM, фирма Compaq выпустила первый в мире 32-разрядный IBM PC-совместимый компьютер<sup>1</sup>. Теперь тон на рынке стала задавать не IBM, а Intel. Так что 386 — действительно очень важный для самой компании процессор. Революционный во многих смыслах. Революционный настолько, что поставщик комплектующих стал диктовать свою политику производителям конечных устройств.

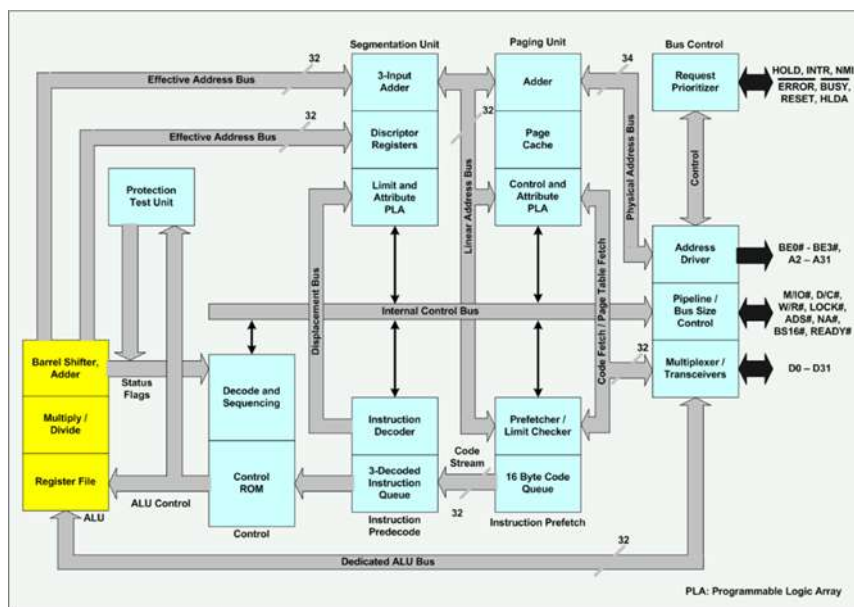


Рис.273 - Архитектура микропроцессора Intel 80386DX

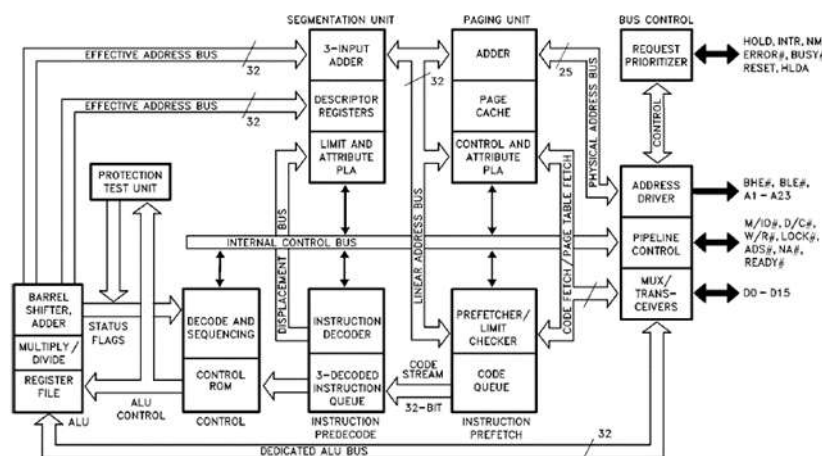


Рис.274 - Архитектура микропроцессора Intel 80386SX

Для того, чтобы обеспечить совместимость с предыдущими микропроцессорами и с огромной библиотекой DOS-программ, Intel 80386 был разработан таким образом, чтобы

<sup>1</sup> Для IBM это был первый звоночек, довольно скоро компания не просто потеряет лидерство на рынке PC, но и полностью уйдёт с него, не сумев вернуться на этот рынок, выпустив семейство ПК PS/2.

быть как можно больше похожим на i8086 и i80286. Как и его предшественники, Intel 80386 мог работать в реальном режиме, с ограничением по адресуемой памяти в 1 Мбайт. В этом режиме он загружал и без проблем выполнял все программы, разработанные на микропроцессорах предыдущих поколений. Из реального режима Intel 80386 мог быть переведён в защищённый режим, где он функционировал подобно 80286, за исключением возможности доступа к большему объёму ОЗУ. В этом режиме в распоряжении программиста было больше памяти, чем было доступно на 286-ом, и он мог более гибко манипулировать ею, потому что было возможным изменять размеры сегмента до 4 Гбайт (в 286-ом размер сегмента был максимум 64 Кбайт). В противоположность Intel 80286, i80386 мог переходить из одного режима в другой без перезагрузки процессора, чисто программными средствами, не требуя никаких малочевидных манипуляций с CMOS-памятью, «тёплых» сбросов и т.п. странных процедур.

В 386-ом появился поддержка т.н. виртуального v86 (Virtual 8086 mode). Важно понимать, что «виртуальный режим», несмотря на похожесть названия, является не «третьим режимом работы процессора» (то есть реальным, защищённым и виртуальным), а лишь подрежимом работы задачи в многозадачном окружении защищённого режима.

Виртуальный режим предназначен для одновременного выполнения программ реального режима (например, программы для DOS) под многозадачной операционной системой защищённого режима. Выполнение в виртуальном режиме практически идентично реальному, за несколькими исключениями, обусловленными тем, что виртуальная задача выполняется в защищённом режиме:

- виртуальная задача не может выполнять привилегированные команды, потому что имеет низший уровень привилегий
- все прерывания и исключения обрабатываются операционной системой защищённого режима (которая, впрочем, может инициировать обработчик прерывания виртуальной задачи), вместе с тем в задаче виртуального режима можно использовать:
  - страничное преобразование, например, для:
    - расширения памяти, путём включения страниц в неиспользуемое адресное пространство
    - эмуляции расширений с переключением банков (например, EMS-памяти)
    - виртуальной развёртки или свёртки буферов внешних устройств (видеопамять, аппаратная EMS-память)
- эмуляцию внешних устройств через эмуляцию портов ввода-вывода
- отладку
- при выполнении нескольких задач виртуального режима, каждая из них может выполняться совершенно отдельно друг от друга, чего принципиально невозможно достигнуть в реальном режиме

Этот режим позволял процессору разбивать память на множество виртуальных машин, каждая из которых работала так, как будто она была отдельным компьютером на процессоре 8086. Каждая из этих виртуальных машин могла запускать свою собственную программу, которая была полностью изолирована от всех остальных. Это означало, что вы можете одновременно выполнять несколько DOS-программ на одном компьютере. Без такой аппаратной поддержки в архитектуре Intel 80386 такая система была очень сложной и капризной (а на 286-ом – практически нереализуемой, из-за особенностей процессора). Intel 80386 делал реализацию программного обеспечения многозадачного режима почти тривиальной, потому что вся самая тяжелая работа выполнялась на уровне «железа» процессора. Готовые DOS-программы выполнялись на Intel 80386 в многозадачной среде без всяких доработок.

Режим виртуального 8086 поддерживается и в последующих 32-битных процессорах x86, вплоть до режима совместимости в x86-64.

Intel 80386 был великолепным микропроцессором, но очень дорогим, по сравнению с процессорами предыдущих поколений – сразу после презентации Intel 80386 его стоимость превышала 500\$, тогда как 8086 стоил в некоторых случаях даже меньше 10\$. То есть вы могли купить целый компьютер за ту же цену, что и один 80386. Частично такой высокой цене способствовала Intel, которая не продавала лицензию на производство 80386 другим компаниям (за исключением IBM).

Intel 80386 имел, пожалуй, наиболее болезненную историю своего внедрения по сравнению с другими микропроцессорами. Вскоре после его создания были обнаружены ошибки, связанные с его работой при выполнении 32-битных математических операций. Эта проблема не была выявлена в первых PC-совместимых компьютерах, реализованных на этом чипе, потому что в DOS используются только 16-битные операции. Ошибки всплыли только после того, когда 80386 стал активно использоваться в 32-битных режимах. Ошибки были быстро обнаружены и исправлены. Микросхемы, выпущенные после апреля 1987 года, не имеют описанных проблем. По решению фирмы все доработанные чипы маркировались двойным символом сигма. Некоторые, но не все, ранее выпущенные микропроцессоры, были промаркированы «только для 16-битных операций». Это говорило о том, что данные микросхемы могут работать со всеми 16-битными версиями DOS (вплоть до версии 3.3), а также с OS/2 (версии 1.0 и 1.1), которые не используют 32-битную математику.

## Основные характеристики Intel 80386

Ниже перечислены основные характеристики процессора Intel 80386:

- 32-х битная шина данных.
- Адресация до 4 ГБайт физической памяти (теоретически)
- EIP, ES1, EDI, ESP – по 32 бита
- G1 – 16 бит
- GDT и IDT – по 32 бита
- CR1, CR2 и CR3 – по 32 бита
- механизмы аппаратной отладки
- тактовая частота от 25МГц до 40МГц (последние версии)
- конвейер 6 ступеней
- защищенный режим:  $2^{46}$  виртуальное адресное пространство, 64 Тбайт
- реальный режим
- полная совместимость программ с 80286
- модификации процессора:
  - Intel 80386SL
  - Intel 80386EX
  - Intel 80386DX
- страничная адресация памяти
- 386 требует присутствия кэша L1 (внешний аппаратный кэш, для работы необходим отдельный контроллер)



## Процессор Intel

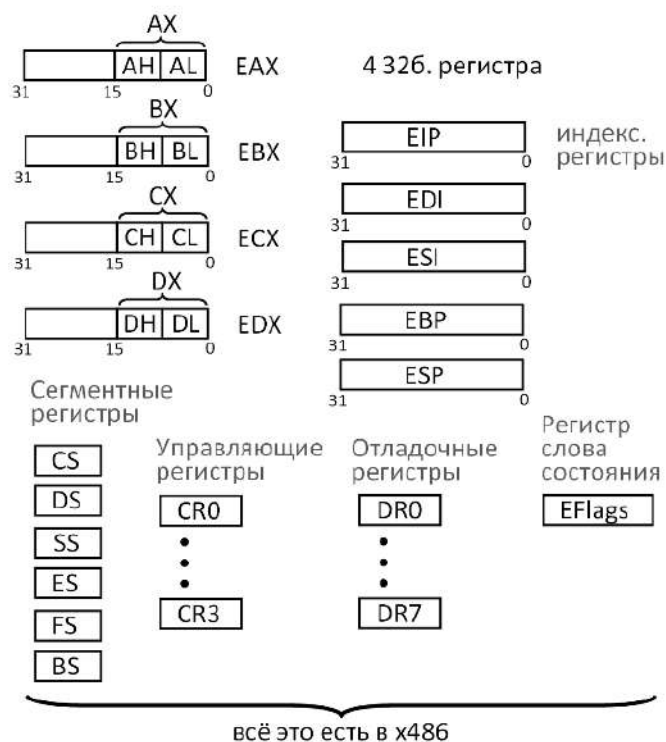


Рис.275 – Регистры 386-го

## Интерфейс процессоров 386DX и 386SX

Процессоры 386DX выпускались в корпусах PGA и PQFP (назначение выводов приведено в таблице на Рис.276). Их интерфейс отличается от 8086 применением отдельных шин адреса и данных, конвейерной адресации, а также составом и назначением управляющих сигналов. По сравнению с интерфейсом 80286 появилась возможность динамического управления разрядностью шины данных и конвейеризацией адреса, изменился состав управляющих сигналов, назначение которых стало более отчетливым.

| Сигнал | 386DX-PQFP | 386DX-PGA | 386SX | Сигнал | 386DX-PQFP                     | 386DX-PGA                                         | 386SX                                  |
|--------|------------|-----------|-------|--------|--------------------------------|---------------------------------------------------|----------------------------------------|
| A01    | —          | —         | 18    | D27    | 113                            | N5                                                | —                                      |
| A02    | 67         | C4        | 51    | D28    | 112                            | M6                                                | —                                      |
| A03    | 68         | A3        | 52    | D29    | 109                            | P4                                                | —                                      |
| A04    | 69         | B3        | 53    | D30    | 108                            | P3                                                | —                                      |
| A05    | 70         | B2        | 54    | D31    | 107                            | M5                                                | —                                      |
| A06    | 71         | C3        | 55    | ADS#   | 27                             | E14                                               | 16                                     |
| A07    | 72         | C2        | 56    | BE0#   | 31                             | E12                                               | —                                      |
| A08    | 74         | C1        | 58    | BE1#   | 32                             | C13                                               | —                                      |
| A09    | 75         | D3        | 59    | BE2#   | 33                             | B13                                               | —                                      |
| A10    | 76         | D2        | 60    | BE3#   | 38                             | A13                                               | —                                      |
| A11    | 77         | D1        | 61    | BHE#   | —                              | —                                                 | 19                                     |
| A12    | 78         | E3        | 62    | BLE#   | —                              | —                                                 | 17                                     |
| A13    | 79         | E2        | 64    | BS16#  | 29                             | C14                                               | —                                      |
| A14    | 81         | E1        | 65    | BUSY#  | 46                             | B9                                                | 34                                     |
| A15    | 82         | F1        | 66    | CLK2   | 24                             | F12                                               | 15                                     |
| A16    | 84         | G1        | 70    | D/C#   | 41                             | A11                                               | 24                                     |
| A17    | 86         | H1        | 72    | ERROR# | 47                             | A8                                                | 36                                     |
| A18    | 87         | H2        | 73    | FLT#   | 54                             | —                                                 | 28                                     |
| A19    | 88         | H3        | 74    | HLDA   | 8                              | M14                                               | 3                                      |
| A20    | 89         | J1        | 75    | HOLD   | 28                             | D14                                               | 4                                      |
| A21    | 93         | K1        | 76    | INTR   | 53                             | B7                                                | 40                                     |
| A22    | 94         | K2        | 79    | LOCK#  | 42                             | C10                                               | 26                                     |
| A23    | 95         | L1        | 80    | M/IO#  | 40                             | A12                                               | 23                                     |
| A24    | 96         | L2        | —     | N/C    | 36, 37, 39, 59, 60, 61, 62, 63 | A4, B4, B6, B12, C6, C7, E13, F13, 44, 45, 46, 47 | 20, 27, 29, 30, 31, 43, 44, 45, 46, 47 |
| A25    | 97         | K3        | —     | —      | —                              | —                                                 | —                                      |
| A26    | 98         | M1        | —     | —      | —                              | —                                                 | —                                      |
| A27    | 100        | N1        | —     | —      | —                              | —                                                 | —                                      |
| A28    | 101        | L3        | —     | —      | —                              | —                                                 | —                                      |
| A29    | 102        | M2        | —     | NA#    | 30                             | D13                                               | 6                                      |
| A30    | 103        | P1        | —     | NMI    | 52                             | B8                                                | 38                                     |
| A31    | 104        | N2        | —     | PEREQ  | 50                             | C8                                                | 37                                     |
| D00    | 20         | H12       | 1     | READY# | 26                             | G13                                               | 7                                      |
| D01    | 19         | H13       | 100   | RESET  | 45                             | C9                                                | 33                                     |
| D02    | 18         | H14       | 99    | W/R#   | 43                             | B10                                               | 25                                     |

Рис.276 - Расположение выводов процессоров 386



| Сигнал       | I/O | Назначение                                                                                                                                                                                                                            |
|--------------|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CLK2         | I   | Clock — внешний сигнал синхронизации процессора                                                                                                                                                                                       |
| D[31:0] *    | I/O | Data — сигналы шины данных                                                                                                                                                                                                            |
| A[31:2] *    | O   | Address — сигналы шины адреса                                                                                                                                                                                                         |
| BE[0:3] *    | O   | Byte Enable — сигналы, указывающие на используемые байты шины данных в данном цикле шины                                                                                                                                              |
| BS16# *      | I   | Bus Size 16 — сигнал, по которому внешние устройства могут сообщить о разрядности своей шины данных, равной 16 бит                                                                                                                    |
| BLE#, BHE# * | O   | Byte Low Enable, Byte High Enable — аналоги BE0# и BE1#                                                                                                                                                                               |
| W/R#         | O   | Write/Read — сигнал, определяющий тип шинного цикла (запись/чтение)                                                                                                                                                                   |
| M/IO#        | O   | Memory/ IO# Select — высокий уровень указывает на обращение к памяти (или останов), низкий — на обращение к портам ввода-вывода или подтверждение прерывания                                                                          |
| D/C#         | O   | Data/Control# — сигнал, определяющий тип шинного цикла (высокий уровень при передаче данных памяти или ввода-вывода, низкий — при выборке кода, в цикле подтверждения прерывания или при останове)                                    |
| ADS#         | O   | Address Status — сигнал идентификации адресного цикла, во время которого действительны сигналы W/R#, D/C#, M/IO#, BE[0:3] и A[31:2]                                                                                                   |
| NA#          | I   | Next Address — запрос следующего адреса при конвейерной адресации                                                                                                                                                                     |
| READY#       | I   | Bus Ready — сигнал завершения шинного цикла. Шинный цикл расширяется на неограниченное время до появления сигнала готовности (низкий уровень)                                                                                         |
| LOCK#        | O   | Bus Lock — сигнал монополизации управления шиной, вырабатывается на время выполнения инструкции по префиксу LOCK и автоматически при выполнении инструкций XCHG с памятью, подтверждения прерывания и доступа к таблицам дескрипторов |
| INTR         | I   | Interrupt Request — сигнал запроса (высоким уровнем) маскируемого прерывания                                                                                                                                                          |

| Сигнал       | I/O | Назначение                                                                                                                                                                                                                         |
|--------------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NMI          | I   | Non Maskable Interrupt — сигнал, высокий уровень которого вызывает немаскируемое прерывание NMI                                                                                                                                    |
| RESET        | I   | Сигнал аппаратного сброса (высоким уровнем)                                                                                                                                                                                        |
| HOLD HLDA    | O   | Bus Hold Request и Hold Acknowledge — запрос управления локальной шиной от другого контроллера и подтверждение предоставления                                                                                                      |
| PEREQ        | I   | Processor Extension Operand Request — запрос на передачу операнда сопроцессором                                                                                                                                                    |
| BUSY# ERROR# | I   | Processor Extension Busy и Error — сигналы состояния сопроцессора. Активность сигнала BUSY# останавливает процессор, а сигнала ERROR# вызывает исключения при выполнении инструкций WAIT и некоторых инструкций ESCAPE             |
| FLT#         | I   | Float — сигнал, принудительно переводящий все выходы в высокоимпедансное состояние для обеспечения возможности подключения тестового оборудования к плате без извлечения процессора. Имеется только у процессоров с корпусами PQFP |

Примечание. У процессора 386SX разрядность шины данных 16 бит — имеются только сигналы D[15:0], разрядность шины адреса 24 бит — A[23:1], сигналы BE[0:3] и BS16# отсутствуют. У процессора 386DX отсутствуют сигналы BLE# и BHE#.

Рис.277 - Назначение сигналов процессора i80386

**Синхронизация процессора** осуществляется внешним сигналом CLK2, частота которого делится пополам внутренним делителем для получения сигнала внутренней синхронизации. Каждому периоду CLK2 соответствует одна из двух фаз внутренней синхронизации (фаза 1 или фаза 2). Максимальная пропускная способность локальной шины процессора — одно двойное слово за каждые два цикла внутренней частоты PCLK.

**Шина данных** допускает как 16-, так и 32-разрядный режим обмена, в зависимости от состояния входного сигнала BS16#. При чтении в 16-разрядном режиме данные на линиях D[16:31] игнорируются, при записи процессор управляет всей шиной данных независимо от сигнала BS16#.

**Шина адреса** позволяет адресовать 4 ГБайт физической памяти (0x00000000-0xFFFFFFFF) в защищённом режиме и 1 МБайт из области младших адресов в реальном режиме. Из этого правила для реального режима есть два исключения:

- линии A[20:31] после сигнала RESET во время выборки кода находятся в единичном состоянии до первой инструкции межсегментного перехода;
- линия A20 может принимать единичное значение, когда сочетание значения сегментного регистра и эффективного адреса приводят к переходу через границу 0x0FFFFFFF. Для обеспечения полной совместимости с 8086 применяют программно управляемый вентиль (Gate A20), принудительно обнуляющий бит A20 системной шины адреса.

При выполнении инструкций ввода-вывода процессор адресуется к области в 64 КБайт пространства ввода-вывода (0x00000000...0x0000FFFF). При обмене с сопроцессором в циклах ввода-вывода процессор выставляет адреса 0x800000F8...0x800000FF, что упрощает дешифрацию адреса внешними схемами.

Линии A[31:2] идентифицируют адрес с точностью до двойного слова, а в пределах этого слова сигналы BE[0:3] непосредственно указывают, какие байты используются в данном цикле. BE0# относится к D[0:7], BE1# — к D[8:15], BE2# — к D[16:23] и BE3# — к D[24:31]. Число активных линий BEi определяет, сколько байт (1, 2, 3 или 4) передается в данном цикле, при этом все байты могут быть только смежными. Если в текущем цикле записи оба сигнала BE0# и BE1# неактивны, выводимые данные старших байт автоматически дублируются на линиях младших (D[24:31] на D[8:15], D[16:23] на D[0:7]). Если адресованное слово (или двойное слово) пересекает границу двойного слова, процессор после пересылки уместившейся младшей части автоматически генерирует дополнительный цикл обмена с другими значениями A[31:2] и BE[0:3].

Сигнал **BS16#** вводится внешней схемой, если текущий цикл адресуется к 16-битному устройству, подключенному к младшей половине шины данных. Если в данном цикле предполагалась передача данных и двух старших байт, процессор автоматически вводит дополнительный цикл для передачи этих байт по младшей половине шины данных.

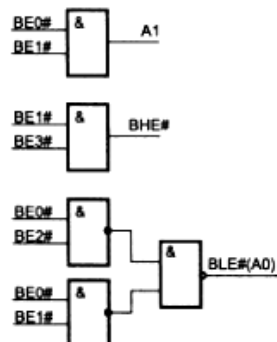


Рис.278 - Формирование сигналов адресации для 16-разрядных шин

Для связи с системной шиной, имеющей возможность подключения 8-, 16- и 32-битных устройств (шины ISA и EISA) и полную шину адреса, включающую сигналы A0, A1 и SBHE#, недостающие сигналы генерируются из сигналов BE[0:3]# по логике, приведенной на Рис.278. Для 8-битных устройств, как и при использовании 80286 или 8086, необходим «диагональный» буфер данных между линиями D[8:15] процессора и Data[0:7] системной шины.

Тип цикла локальной шины определяется сигналами **M/IO#**, **D/C#**, **W/R#** и **LOCK#**, комбинация которых действительна во время активности сигнала ADS# (таблица на Рис.279).

Сигналы управления шиной ADS#, READY# и NA# задают временные границы циклов и управляют конвейерной адресацией. Сигнал ADS# отмечает начало каждого цикла, во время его действия производится идентификация типа цикла и фиксируется соответствующий ему адрес (линии A[31:2] и BE[0:3]#). Сигнал READY# должен вводиться внешней схемой в каждом цикле, отмечая готовность к завершению текущего цикла. Сигнал NA# используется для запроса следующего адреса при конвейерной адресации, когда внешние схемы уже готовы воспринять новый адрес и сигналы идентификации. Если процессор уже имеет внутренний запрос на следующий шинный цикл, он выдаст адрес и идентификаторы следующего цикла до завершения текущего.

| M/IO# | D/C# | W/R# | Тип цикла                                                                                                     |
|-------|------|------|---------------------------------------------------------------------------------------------------------------|
| 0     | 0    | 0    | Подтверждение прерывания (INTA#)                                                                              |
| 0     | 1    | 0    | Чтение порта (IORD#)                                                                                          |
| 0     | 1    | 1    | Запись в порт (IOWR#)                                                                                         |
| 1     | 0    | 1    | Если BE2# = 0 — останов (Halt), если BE0# = 0 — отключение (Shutdown); при этом A[2:31]=0, а остальные BEi=1. |
| 1     | 0    | 0    | Выборка кода (MEMRD#)                                                                                         |
| 1     | 1    | 0    | Чтение памяти (MEMRD#)                                                                                        |
| 1     | 1    | 1    | Запись в память (MEMWR#)                                                                                      |

Рис.279 - Декодирование шинных циклов процессора 386

**Сигналы арбитража локальной шины** предназначены для передачи управления локальной шиной другому устройству по его запросу — переходу в состояние лог. «1» линии HOLD. Когда процессор сочтёт возможным передачу управления (в незаблокированных циклах сразу по завершении текущего шинного цикла), он выставит сигнал подтверждения HLDA. В таком состоянии процессор переводит в высокоимпедансное состояние все двунаправленные и выходные линии (кроме самого сигнала HLDA). Этим состоянием можно пользоваться и для отладки аппаратных средств.

**Сигналы интерфейса с сопроцессором** практически те же, что и у процессора 80286, это позволяет наряду с сопроцессором i80387 использовать сопроцессор i80287. Сигнал **PEREQ**, является запросом сопроцессора на обмен данными, в ответ на который процессор выполняет передачи в циклах ввода-вывода. Сигнал **BUSY#** вырабатывается сопроцессором во время выполнения вычислительных инструкций. Сигнал **ERROR#** вызывает исключение с номером 16 при инструкциях, адресованных сопроцессору, но в PC он, как правило, не используется.

Высокий уровень сигналов **NMI** и **INTR** вызывает немаскируемое и маскируемое прерывания. Сигнал **RESET** сбрасывает процессор в исходное состояние, при его действии никакие другие входные сигналы, включая **HOLD**, не воспринимаются. Если во время спада сигнала **RESET** удерживается низкий уровень на линии **BUSY#**, процессор выполнит внутренний тест BIST (Built-In Self Test).

**Конвейерная адресация** (Pipelined Addressing), применяемая в процессоре, предусматривает возможность начала (фазы идентификации и адресации) нового цикла, не дожидаясь завершения физического обмена данными предыдущего цикла. Конвейерная адресация запрашивается внешними схемами по сигналу **NA#**. Если этот сигнал неактивен, конвейерная адресация не используется и значение сигналов адреса и идентификации будет действительно на все время цикла (в отличие от конвейерной шины 80286). При использовании конвейерной адресации для совместимости с шинами (например, ISA) и устройствами, подразумевающими действительность адреса на всё время цикла, сигналы шины адреса процессора должны защёлкиваться в регистрах. Временные диаграммы шинных циклов процессора приведены на Рис.280.

Конвейерный режим не повышает пропускную способность локальной шины процессора, а только позволяет уменьшить число тактов ожидания, вводимых медленными устройствами за счёт более раннего начала выполнения дешифрации адреса устройством памяти или ввода-вывода. Конвейерный режим в основном используется контроллерами памяти, позволяя им эффективно использовать быстрый страничный режим и чередование банков.

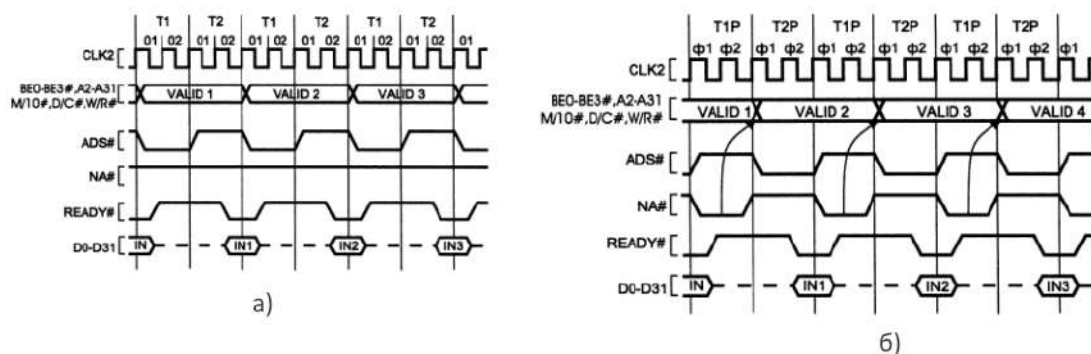


Рис.280 - Шинные циклы 80386: а) неконвейерные; б) конвейерные.

Формирование управляющих сигналов системной шины **IOR#**, **IOW#**, **MEMR#**, **MEMWR#**, **INTA#** и **ALE** выполняется внешними микросхемами чипсета системной платы.

**Управление временной диаграммой** подразумевает задержку (Command delay) и растяжку (Command extension) командных циклов. **Растяжка** при необходимости удлиняет командный цикл обращения с помощью задержки сигнала **READY#**. **Задержка** предназначена для сдвига начала командного импульса системной шины относительно соответствующего ему сигнала **ADS#** и осуществляется внешними микросхемами чипсета. Управление задержкой и растяжкой циклов в зависимости от типа и адреса обращения возлагается на микросхемы чипсета системной платы, а для задания длительности циклов системной шины и времени восстановления (Recovery Time) для различных операций в машинах класса AT обычно имеются соответствующие опции BIOS SETUP.

**Отличия процессора 386SX от 386DX** обусловлены сокращением разрядности шины данных и адреса.

- Шина данных имеет разрядность 16 бит, динамическое изменение разрядности 16/32 (сигнал BS16#) исключено, и 32-битные обмены требуют, как минимум, двух циклов.
- Из четырех сигналов разрешения байт остаются только два младших, переименованных в BLE# и BHE#. Их назначение сохранилось (не путать с сигналом BHE# процессора 80286, который имеет несколько иное функциональное назначение). В связи с этим в шине адреса появился сигнал A1.
- Разрядность шины адреса сокращена до 24 бит, адрес выводится линиями A[23:1], BHE# и BLE#. Поскольку старшие 8 бит адреса просто отсечены, попытка адресации по физическому адресу, превышающему 16 Мб, приводит к циклическому повторению первых 16 Мб адресного пространства. При обмене с сопроцессором в циклах ввода-вывода процессор выставляет адреса 0x8000F8...0x8000FF.

В остальном сигналы процессоров 386SX и 386DX совпадают.

### **Четвёртое поколение x86 процессоров i80486**

Выпущенный 10 апреля 1989 года, процессор Intel 80486 был фактически доработанной, отъюстированной версией 386-го – на кристалл основного процессора был добавлен математический сопроцессор<sup>1</sup>, кэш первого уровня, появились вычислительные конвейеры, множители тактовой частоты шины (в более поздних моделях). Кэш – общий для данных и инструкций – имел объём 8 Кбайт, позднее – 16 Кбайт, и был организован в 4-канальную наборно-ассоциативную структуру, содержащую 128 наборов по четыре строки размером по 16 байт каждая, и работал на уровне физических адресов памяти. В первых 486-ых кэш-память работала только со *сквозной записью* (write-through), то есть операции записи не кэшировались. Более поздние модели имели возможность включить кэш в режим с *отложенной (обратной) записью* (write-back). Также процессоры i486 работали с внешним кэшем второго уровня (L2), который располагался на материнской плате. Его объём на то время составлял от 256 Кбайт до 512 Кбайт.

Наличие кэш-памяти значительно усложнило кристалл. Процессор i80486 содержал почти 1.2 млн. транзисторов и около половины из них приходилось именно на кэш-память. Сложность чипа стала причиной его высокого энергопотребления и тепловыделения. Так, в системах, использовавших i486, впервые стало применяться активное охлаждение. Вдобавок к этому сложность процессора стала причиной увеличения брака при производстве, из-за чего заметно выросла себестоимость устройства.

Также в процессорах i486 появились вычислительные конвейеры, суть работы которых заключается в разделении обработки компьютерной инструкции на последовательность независимых стадий с сохранением результатов в конце каждой стадии. Что-то подобное было реализовано еще в Zilog Z8000. Конвейер i486 состоял из пяти ступеней: выборка, декодирование, декодирование адресов операндов, выполнение команды, запись результата выполнения инструкции. Появление конвейеров не только увеличило быстродействие, но и в какой-то степени упростило процессорную архитектуру. Также стоит отметить, что появление конвейеров благоприятно сказалось на разгонном потенциале CPU.

Что касается сопроцессора (FPU, Floating-Point Unit, модуль операций с плавающей запятой), то в i486 он был встроен в кристалл основного процессора. Многие модели 486SX, без FPU, на самом деле представляли собой отбраковку полноценных 486DX.

---

<sup>1</sup> Intel выпускала также модели 486SX без встроенного FPU.

Коэффициенты умножения появились не в самых первых процессорах i486. Первые модели 486SX и 486DX обходились без него и работали на частоте системной шины. Поддержка множителя появилась лишь в кристалле 486DX2. При частоте системной шины 33 МГц тактовая частота самого процессора составляла 66 МГц. То есть множитель равнялся двум. В 486DX4 коэффициент умножения был увеличен до трех. Вместе с введением множителей появился оверклокинг.

Как и в случае с третьим поколением, изначально Intel вывела на рынок только две модели: 486SX (Рис.281) и 486DX (Рис.282).

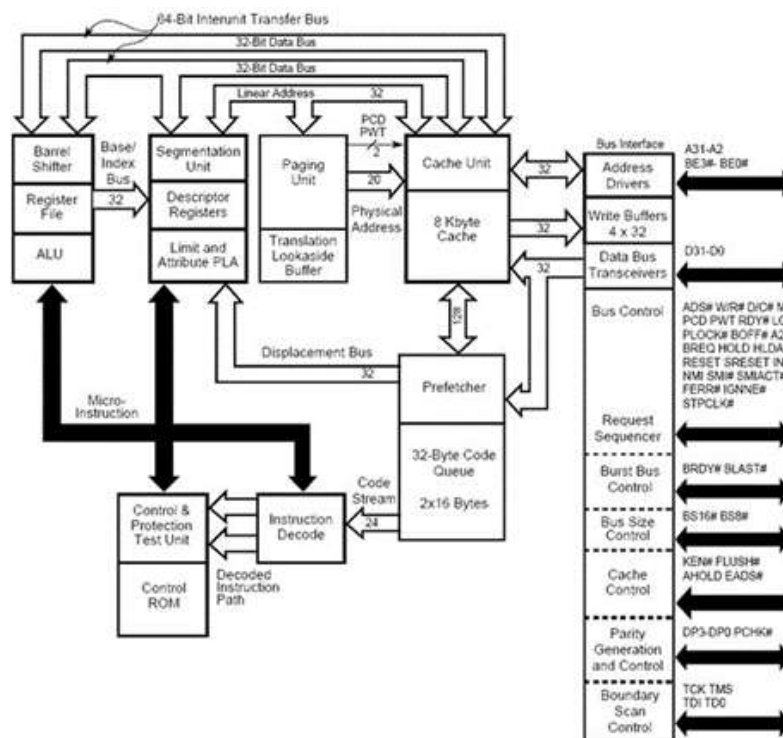


Рис.281 - Архитектура микропроцессора Intel 80486SX

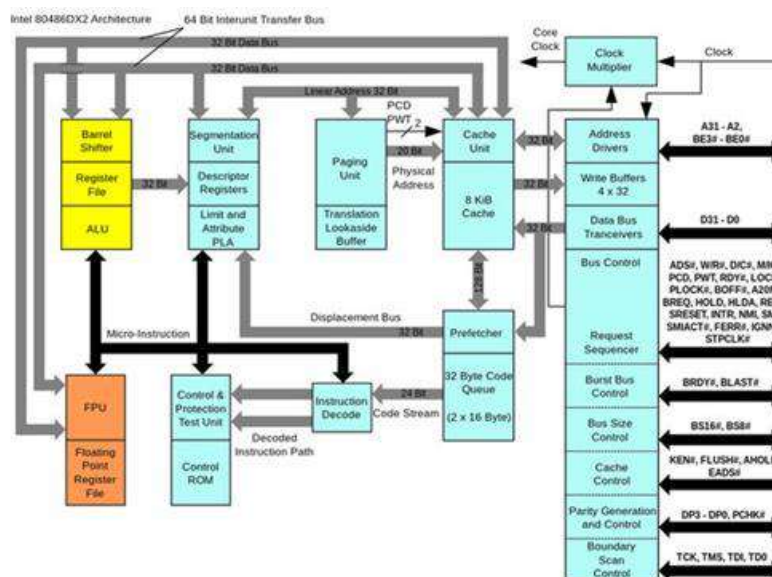


Рис.282 - Архитектура микропроцессора Intel 80486DX2/DX4

Как отмечалось выше, единственным различием между этими процессорами стало отсутствие встроенного FPU. В остальном кристаллы были полностью идентичны. Чипы производились по 1 мкм техпроцессу, а чуть позже по 0.8-мкм технормам. Тактовая частота

обеих моделей варьировалась от 25 до 50 МГц. Максимальное энергопотребление достигало отметки в 5 Вт.

Несмотря на то, что Intel всё активнее защищала свои разработки патентами, на рынке появилось немало клонов i486. Производством копий занимались AMD, Cyrix, IBM, Texas Instruments и другие производители.

Основу линейки клонов AMD Am486 составляли модели Am486SX и Am486DX. Процессоры производились по более тонкому 0.7-мкм техпроцессу, а затем в соответствии с 0.5-мкм и даже 0.35-мкм технологическими нормами. Технически Am486SX и Am486DX были полными аналогами кристаллов Intel. Процессоры, не использовавшие множители, имели частоты от 25 до 40 МГц, а кристаллы с коэффициентом умножения работали на тактовой частоте от 66 до 100 МГц.

В 1995 году AMD представила самый быстрый i486-совместимый процессор под названием Am5x86. Кристалл производился по 350-нм техпроцессу и имел 1.6 млн транзисторов. Объем кэш-памяти 1-го уровня был увеличен до 16 Кбайт, а коэффициент умножения частоты был равен 4. Процессор работал с шиной с частотой 33 МГц, то есть тактовая частота самого кристалла составляла 133 МГц. По производительности Am5x86 был сопоставим с процессором Pentium с частотой 75 МГц. Am5x86 был топовым решением AMD до выхода новых процессоров поколения K5.

Клоны i486, производимые другими компаниями, ничем особенным не отличались. Они архитектурно копировали оригинал и, естественно, имели такую же производительность.

## Различия между Intel 486DX и Intel 386

Intel 486DX, 486DX2 и 486DX4 представляют собой кристалл, содержащий центральный процессор, математический сопроцессор и контроллер кэша. Полностью совместимые на уровне предпроцессора с процессорами Intel 386, тем не менее, они имеют следующие отличия:

- процессоры Intel 486, в отличие от Intel ULP486GX, который имеет поддержку только 16-битной шины данных, обеспечивают динамическое изменение размера используемой шины для поддержки 8-, 16- и 32-битных транзакций. Intel 386 поддерживают только два размера ширины шины, 16 и 32 бита, и не требуют внешней логики для организации смены ширины шины.
- Процессоры Intel 486 имеют режим укороченной передачи, который позволяет за одну транзакцию передать по шине четыре 32-битных слова из внешней памяти в кэш, используя всего пять циклов. Intel 386 для передачи того же объема данных требуется минимум восемь циклов.
- Процессор Intel 486 имеет сигнал BREQ, используемый для поддержки мультипроцессорных систем.
- Шина процессора Intel 486 значительно мощнее шины процессора Intel 386. Новые возможности в виде умножения частоты шины, проверки чётности (отсутствует в ULP486SX и ULP486GX), укороченный цикл передачи данных, кэшируемые циклы, в том числе кэшируемый цикл без проверки данных, поддержка транзакций по 8-битной шине.
- Для поддержки кэша на кристалле введены новые управляющие регистры (CD и NW), добавлены новые выводы для шины, новые типы циклов обмена по шине.
- Набор инструкций математического процессора Intel 387 не только поддерживался в полном объеме, но и был расширен. Во время выполнения команды по обработке данных с плавающей точкой не выполняются никакие циклы ввода-вывода. Не задействовано прерывание 9, используется прерывание 13.

- Процессор Intel 486 поддерживает новые режимы выявления ошибок, что гарантирует совместимость с DOS. Эти новые режимы требуют нового бита в управляющем регистре 0 (NE).
- Набор инструкций не претерпел существенных изменений, но были добавлены дополнительные инструкции для работы со внутренней кэш-памятью (INVD, INVLPG, WBINVD), одна инструкция (BSWAP) для обеспечения совместимости с процессорами Motorola (big endian), две инструкции для атомарных операций с памятью: CMPXCHG (для сравнения с обменом — новое значение записывалось только если старое совпадало с заданным, старое запоминалось) и XADD (инструкция для сложения двух операндов с помещением результата во второй операнд, а не в первый, как в ADD). Инструкция CPUID позволяла впервые в семействе x86 напрямую получить детальную информацию о версии и свойствах процессора. Помимо этого, к набору инструкций добавилось 75 инструкций FPU.
- В управляющем регистре 3 назначены два новых бита, отвечающих за кэширование текущего каталога страниц.
- Добавлены новые возможности защиты страниц, требующие нового бита в управляющем регистре 0.
- Добавлены новые возможности проверки выравнивания, требующие нового бита в регистре флагов и управляющем регистре 0.
- Заменён алгоритм для TLB на алгоритм псевдо-LRU (PLRU), подобно используемому в кэше на кристалле.
- Для тестирования кэша на кристалле добавлены три новых тестовых регистра: TR5, TR6 и TR7. Повышена стабильность работы TLB.
- Добавлены новые флаги в регистре флагов (EFLAGS) и в других управляющих регистрах (CR0, CR3).
- Очередь предварительной выборки увеличена с 16 до 32 байт. Для гарантированного правильного выполнения новых инструкций всегда выполняются переходы после модификации кода.
- После сброса в старший байт ID записывается значение <04>.

## Блоки и реализация интерфейсов

Интерфейс процессоров 486 похож на интерфейс 386-х, но имеет некоторые новшества: пакетный режим передачи, механизм аннулирования строк и очистки кэша, расширенный арбитраж, контроль паритета данных и возможность 8-разрядного обмена. Для более поздних версий процессора, поддерживающих политику обратной записи первичного кэша, и процессоров со средствами SMM и управления энергопотреблением имеются дополнительные сигналы.

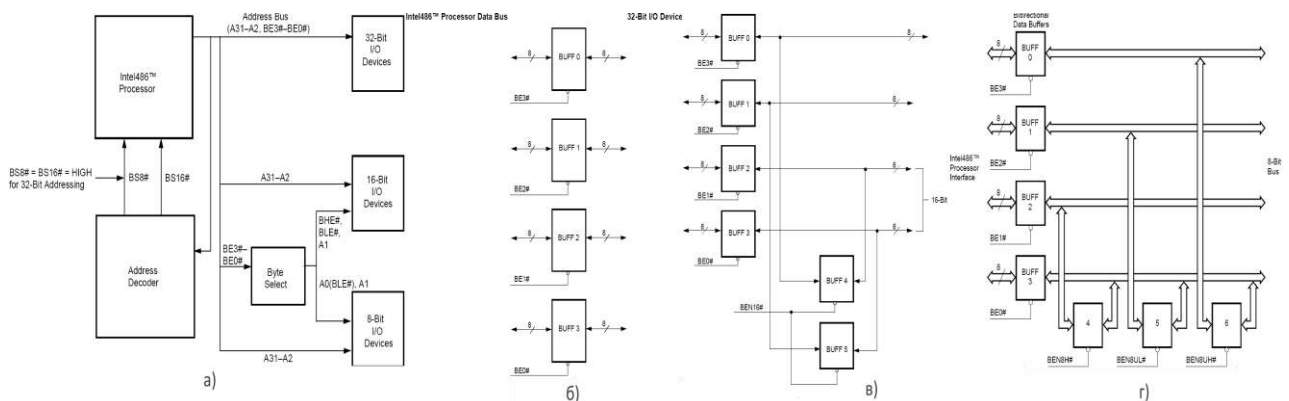


Рис.283 – а) организация интерфейса с устройствами ввода-вывода; б) организация интерфейса с 32-битными устройствами ввода-вывода; в) организация интерфейса с переменным размером шины данных: 16 бит; г) организация интерфейса с 8-битными устройствами ввода-вывода

**Синхронизация процессора** осуществляется внешним сигналом CLK, все временные параметры отсчитываются относительно его положительного перепада. Внешняя тактовая частота процессором уже не делится пополам (как у 386-го), а может даже умножаться на коэффициент, который у разных моделей процессоров может принимать значение 1, 2, 2.5 и 3. При этом интерфейс внешней шины процессора всегда работает на внешней частоте (Bus Frequency), а частота тактирования вычислительного ядра (Core Frequency) может повышаться в несколько раз. Стандартными значениями внешней частоты являются 25, 33.33, 40 и 50 МГц. Коэффициент умножения задается уровнем логического сигнала на входе CLKMUL. Для процессоров Am5x86 высокий уровень (или неподключенный контакт) задает K=4, низкий — K=3. Для процессоров Intel DX4 и AMD DX4 высокий уровень (или неподключенный контакт) задает K=3, низкий — K=2, для процессоров DX2 низкий уровень задает K=2. Эти сигналы используются только в процессорах DX2 и DX4, и на системной плате обычно имеется джампер (или переключатель), задающий его потенциал. Установкой джампера в соответствующее положение можно выбрать требуемый коэффициент умножения, но, естественно, только в пределах возможностей данного процессора: DX2 не заставить умножать частоту на 3.

**Шина адреса** позволяет адресовать 4 Гбайт физической памяти (00000000...0xFFFFFFFF) в защищенном режиме и 1 Мбайт из области младших адресов в реальном режиме. Сигналом A20M# можно включить принудительное обнуление линии A20 (как для внутреннего кэша, так и для внешних операций). Таким образом эмулируется циклический возврат (wrap) адреса процессора 8086/88 (сигнал воспринимается только в реальном режиме). При выполнении инструкций ввода-вывода процессор адресуется к области в 64 Кб пространства ввода вывода (0x00000000...0x0000FFFF).

Линии A[31:2] идентифицируют адрес с точностью до двойного слова, а в пределах этого слова сигналы BE[0:3] непосредственно указывают, какие байты используются в данном цикле. Линии A[31:4], определяющие адрес строки внутреннего кэша, в циклах аннулирования работают на ввод.

**Шина данных D[31:0]** допускает как 32-разрядный режим обмена, так и 16- и 8-разрядный, в зависимости от состояния входных сигналов BS16# и BS8#. Сигналы BS16# или BS8# вводятся внешней схемой, если текущий цикл адресуется к 16- или 8-битному устройству, подключенному к младшей части шины данных, BS8# имеет более высокий приоритет. Для связи с системной шиной, имеющей возможность подключения 8-, 16- и 32-битных устройств (шины ISA и EISA) и полную шину адреса, включающую сигналы AO, A1 и SBHE#, недостающие сигналы генерируются из сигналов BE[0:3]# по логике, приведенной на Рис.283. 8-битные устройства, наконец, могут подключаться к шине данных без «диагонального» буфера.

Каждый байт шины данных имеет **бит паритета DP[3:0]**. Схемы паритета генерируют корректные контрольные биты в циклах записи, а в циклах чтения в случае ошибки паритета только вырабатывается сигнал ошибки на выходе PCHK#, никак не влияющий на работу процессора. Он может использоваться внешними схемами по усмотрению разработчика системной платы. Контроль чётности (число единиц в байте вместе с битом паритета должно быть чётным) выполняется для всех байт, участвующих в конкретном цикле.

Тип цикла локальной шины определяется сигналами M/IO#, D/C#, W/R# и LOCK# во время активности сигнала ADS# (таблица на Рис.284). Декодирование отличается от такового в 386-м положением специального цикла, который перемещен из комбинации 101 в 001.

Для управления шинными циклами процессор имеет входные сигналы готовности RDY# и BRDY#. Они вырабатываются внешними схемами по их готовности к завершению шинного цикла (как и для процессора 386). Сигнал BRDY# вырабатывает устройства, поддерживающие пакетный режим обмена. На возможность пакетного продолжения начатого



цикла процессор указывает выходным сигналом **BLAST#** (при его низком уровне нет перспектив пакетного продолжения).

| M/IO# | D/C# | W/R# | Тип Цикла                                                             |
|-------|------|------|-----------------------------------------------------------------------|
| 0     | 0    | 0    | Подтверждение прерывания (INTA#)                                      |
| 0     | 0    | 1    | Если BE2# = 0 - останов (Halt), если BE0# = 0 - отключение (Shutdown) |
| 0     | 1    | 0    | Чтение порта (IORD#)                                                  |
| 0     | 1    | 1    | Запись в порт (IOWR#)                                                 |
| 1     | 0    | 0    | Выборка кода (MEMRD#)                                                 |
| 1     | 1    | 0    | Чтение памяти (MEMRD#)                                                |
| 1     | 1    | 1    | Запись в память (MEMWR#)                                              |

Рис.284 - Декодирование шинных циклов процессора 486

**Сигналы арбитража** локальной шины по сравнению с предыдущими процессорами дополнены выходной линией **BREQ**, управляемой процессором в любом состоянии. Этот сигнал указывает внешнему арбитру на то, что процессор имеет внутренний запрос на использование шины. Сигнал устанавливается одновременно со стробом **ADS#**. Если процессор в данный момент не управляет шиной (по сигналу **HOLD**, **AHOLD** или **BOFF**), то сигнал появится в тот момент, когда процессор выставил бы сигнал **ADS#**, если бы он управлял шиной. Сигнал **BREQ** обеспечивает эффективный арбитраж в многопроцессорных системах.

Для запроса передачи управления локальной шиной другому устройству используется сигнал **HOLD**. Когда процессор сочтёт возможным передачу управления (в незаблокированных циклах — сразу по завершении текущего шинного цикла), он выставит сигнал подтверждения **HLDA**. Кроме того, имеется сигнал безусловного отключения процессора от шины — **BOFF#**. По этому сигналу процессор отдаёт управление шиной в следующем же такте, а текущий цикл может быть прерван. По окончании действия сигнала **BOFF#** процессор рестартует прерванный цикл, снова введя такт адресации и идентификации со стробом **ADS#**. Для внутренних узлов процессора это выглядит как введение тактов ожидания.

Сигнал **AHOLD** предназначен для захвата внешним контроллером только шины адреса для аннулирования строк внутренней кэш-памяти. По этому сигналу процессор в ближайшем такте переключает линии **A[31:4]** в режим ввода (адрес, выводимый процессором, воспринимается внешними схемами только в такте идентификации). Внешний контроллер устанавливает на шине адрес аннулируемой строки (в сущности, это адрес памяти, по которому производится запись со стороны внешнего контроллера). Аннулирование строки производится при наличии строба внешнего адреса **EADS#**.

К **интерфейсу кэш-памяти** относятся входные сигналы **KEN#** (разрешение кэширования памяти по текущему адресу), **FLUSH#** (аннулирование всех строк внутреннего кэша с предварительной выгрузкой модифицированных строк в случае применения обратной записи) и выходные сигналы **PWT** и **PCD**, управляющие внешним кэшированием. Процессоры, работающие в режиме обратной записи кэша, используют дополнительные сигналы **INV**, **HITM#**, **CACHE#** и **WB/WT#**.

Первые процессоры семейства 486-х в первичном кэше обеспечивали только политику сквозной записи (Write Through). Для них не было необходимости в реализации пакетного режима при записи. Запись внешним контроллером в кэшируемую память приводит только к аннулированию строки кэша, если ячейка, к которой обращается внешний контроллер, представлена и во внутреннем кэше. Для политики обратной записи интерфейс усложняется — необходимо обеспечение возможности выгрузки модифицированных строк кэша в основную память, если к памяти, отображаемой этими строками, обращается внешний контроллер шины. С этими различиями связаны понятия стандартного и расширенного режима шины процессора 486.

**Стандартный режим шины** предназначен для работы первичного кэша с политикой сквозной записи, что полностью совместимо с интерфейсом первых процессоров 486 с WT-кэшем. Его основные отличия следующие:

- на сигнал FLUSH# процессор не отвечает специальным циклом подтверждения;
- по сигналу FLUSH# процессор аннулирует все строки внутреннего кэша за 15-20 тактов CLK;
- сигналы, специфичные для WB-кэша (BLEN#, EWBE#, WB/WT#, INV), игнорируются;
- сигнал EADS# воспринимается в любой момент времени.

**Расширенный режим шины** предназначен для работы первичного кэша с политикой обратной записи, что полностью совместимо с интерфейсом процессоров 486 с WB-кэшем. Его основные отличия следующие:

- по сигналу FLUSH# процессор выполняет обратные записи модифицированных строк кэша, после чего отвечает специальным циклом подтверждения;
- обратная запись модифицированных строк кэша, выполняемая по сигналу FLUSH# и инструкции WBINVD, может занимать около 2000 тактов CLK, система должна наблюдать за шиной, ожидая специального цикла подтверждения;
- сигналы BLEN#, EWBE#, WB/WT#, INV воспринимаются процессором;
- сигнал WB/WT# воспринимается в каждом цикле обращения к памяти, позволяя определять политику записи для каждой строки отдельно;
- сигнал EADS# воспринимается только в состоянии HOLD, AHOLD или BOFF#;
- сигнал PLOCK# не активен (постоянный высокий уровень).

Выбор режима шины осуществляется процессором по состоянию линии WB/WT# в момент окончания сигнала RESET, низкому уровню соответствует стандартный режим шины (сквозная запись). Сигнал внутри процессора резистором подтягивается к низкому уровню, так что на системной плате, не поддерживающей режим обратной записи, у которой контакт В-13 (Т-1 в сокетах 2 и 3) свободен, процессор всегда будет работать в стандартном режиме.

**Сброс и аппаратные прерывания** процессора по интерфейсу аналогичны предыдущим процессорам. Сигналы **NMI** и **INTR** высоким уровнем вызывают немаскируемые и маскируемые прерывания. Сигнал **RESET** сбрасывает процессор в исходное состояние, при его действии никакие другие входные сигналы, включая и HOLD, не воспринимаются. Если во время спада сигнала RESET удерживается высокий уровень на линии AHOLD, процессор выполнит внутренний тест BIST (Built-In Self Test). Дополнительный сигнал **SRESET** переводит процессор в исходное состояние, но с сохранением состояния кэша, буферов записи, регистров FPU и некоторых бит других регистров.

Встроенный математический **сопроцессор** не требует дополнительных сигналов для организации обмена данными, как это было в прежних процессорах. Однако для обеспечения совместимости с архитектурой IBM PC, в которой векторы прерываний, зарезервированные Intel под исключения процессора и сопроцессора, традиционно используются для иных целей, процессоры 486DX имеют пару сигналов FERR# и IGNNE#. Сигнал ошибки сопроцессора **FERR#** аналогичен сигналу ERROR# сопроцессора 387 — он вырабатывается в случае возникновения немаскированного исключения. Этот сигнал в PC заводится на вход запроса IRQ13 вторичного контроллера прерываний. Входной сигнал IGNNE# заставляет сопроцессор игнорировать ошибки (не вырабатывать внутренние исключения), при этом ошибка индицируется сигналом FERR# и может обрабатываться принятым в PC способом.

Процессор 486SX не может работать ни с каким внешним сопроцессором (нет внешнего интерфейса). При установке «сoproцессора» 487SX существующий «основной» процессор полностью отключается.

Изначально все выпускавшиеся микропроцессоры Intel 486 оснащались работающим сопроцессором, эти процессоры получили имя Intel486DX. Позже, в 1991 году, Intel решает выпустить процессоры с отключённым сопроцессором, и эти процессоры получили наименование Intel486SX.

## Организация памяти в 80386/486

По сравнению с 286-ым, в микропроцессоре i80386 было сделано несколько серьёзных изменений в части управления и организации памяти:

1. Была увеличена разрядность адреса, сегменты переменной длины могли иметь длину 64 Кбайт (режим, совместимый с 286-ым) или 4 Гбайт;
2. Кроме блока сегментного преобразования виртуального (логического) адреса был добавлен блок (unit) страничного преобразования. Теперь виртуальный адрес, прежде чем попасть на адресную шину, подвергался трехступенчатому преобразованию: виртуальный (логический) → линейный (сегментное преобразование) → физический (страничное преобразование).

Каждая задача в 80386 могла иметь максимум до  $2^{14} - 1 = 16,383$  сегментов, с размером до 4 Гбайт каждый, предоставляя почти до  $2^{36} = 64$  Тбайт виртуальной памяти каждой задаче. Общая схема сегментно-страничной организации памяти приведена на Рис.285 ниже:

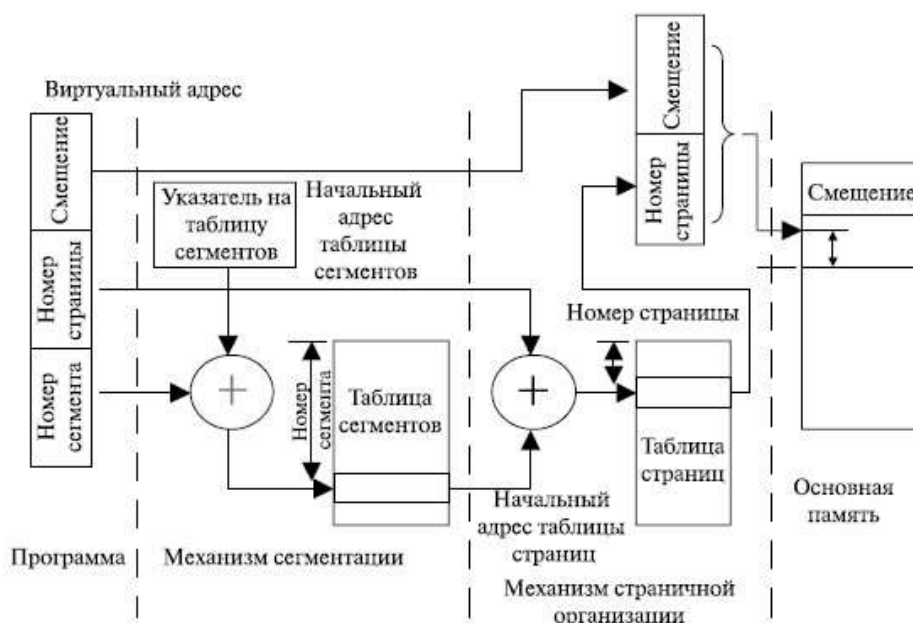


Рис.285 - Сегментно-страничная организация памяти

В блоке сегментного преобразования, также, как в 80286, была реализована четырёх-уровневая система защиты (кольца защиты), от Ring0 до Ring3.

Модуль Bus interface unit (BIU) также обеспечивал конвейеризацию адресной шины, необходимую разрядность шины данных, и генерировал сигналы Byte Enable для каждого байта на шине данных.

Напомним, что в 80386 было реализовано три режима работы: реальная мода, защищённая мода и мода виртуального 8086 (v86) (иногда третий, v86, относят к подрежиму защищённой моды).

В реальном режиме 386-ой был максимально похож на обычный 8086:

- При сбросе процессора или подаче напряжения питания 80386 инициализировался для работы в реальной моде.
- В реальном режиме процессор имел ту же архитектуру, что и 8086, но имел возможность достигаться к 32-разрядному набору регистров 386-го.
- Механизм адресации физической памяти, размер доступного ОЗУ, обработка прерываний были идентичны реальному режиму процессора 80286.
- Все инструкции 386-го, доступные в реальном режиме, были 16-битными, совпадающими с набором инструкций 8086.
- Для использования 32-битных регистров и манипуляций с модами адресации, необходимо было применять переопределяющий префикс 'E'.
- Размер сегмента в 80386 в реальном режиме составляет 64 Кбайт, поэтому эффективные адреса должны были находиться в диапазоне до 0x0000FFFF.
- Основное предназначение реальной моды в 80386 – преднастройка процессора для перехода в защищённый режим.

### Адресация памяти в реальном режиме работы 80386/80486

В реальной моде 386 работал с памятью практически неотличимо от 8086, с оговоркой об ошибке, которую в своё время допустили разработчики 80286 – возможность, не переключаясь в защищённую моду, достигаться к первым 64 Кбайт минус 16 Байт во втором мегабайте ОЗУ. Эту полезную ошибку Intel сохранила во всех последующих своих процессорах. Коротко повторим описание работы с памятью для реальной моды:

- Предел ОЗУ 1 Мбайт (с оговоркой про ошибку выше), т.е. используются только адресные линии A0-A19.
- В реальной моде линейный адрес совпадает с физическим, так как страничное преобразование не разрешено.
- Точно также, совместимо с 8086 и с 80286-ым в реальной моде, физический адрес вычисляется суммированием сдвинутого влево на четыре разряда (умноженного на 16) содержимого сегментного регистра и содержимого регистра смещения.
- Все адреса в реальной моде не могут быть больше  $2^{16} = 64$  Кбайт и могут читаться, писаться и исполняться (executed).
- При попытке достигаться к данным за пределами сегмента, 80386 генерирует исключение с номером 13.
- В реальном режиме сегменты могут перекрываться, это означает, что если определённый сегмент не использует все 64 Кбайт, он может переиспользовать верхушку (top) неиспользуемой части предыдущего сегмента.
- Эта техника позволяет минимизировать объём физической памяти, требуемой для программы.

### Адресация памяти в защищённом режиме работы 80386/486

Как уже упоминалось выше, в защищённом режиме процессоры i80386/486 используют трёхступенчатую схему преобразования адреса.

Программы работают с логическим адресом, состоящим из селектора и смещения (аналогично процессору i80286). Селектор полностью аналогичен используемому в процессоре i80286. Компонента смещения является 32-разрядной, т.е. допустимый размер сегмента значительно превышает 64 Кбайт.

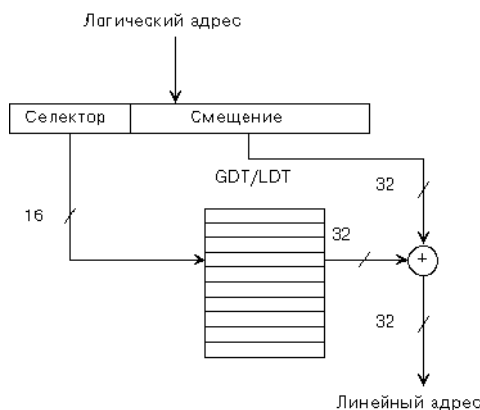
Уровень *логического* адреса – это первая ступень в схеме преобразования адресов.

Вторая ступень – получение из *логического* адреса 32-разрядного *линейного* адреса. Линейный адрес берётся из глобальной или локальной таблицы дескрипторов (GDT или

LDT) в зависимости от соответствующего бита селектора (бит 2). Механизм получения линейного адреса напоминает механизм получения 24-разрядного физического адреса в процессоре i80286. Однако линейный адрес не выводится непосредственно на адресную шину памяти, то есть он не является физическим адресом.

Для получения из линейного адреса физического адреса используется третья ступень – механизм *страничной* адресации. С помощью этого механизма 20 старших бит линейного адреса используются для выбора блока памяти размером 4 КБайт. Такой блок называется страницей физической памяти. Оставшиеся 12 бит линейного адреса представляют собой смещение внутри страницы.

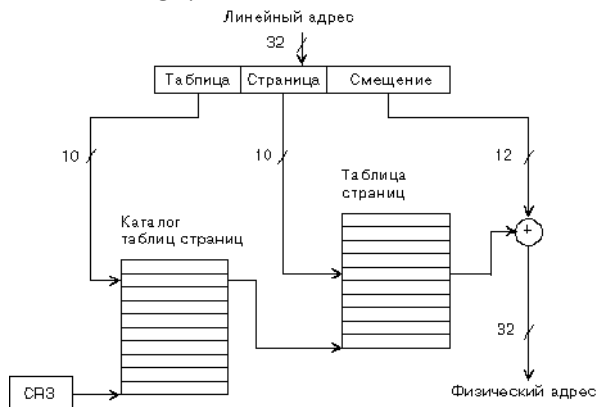
Процесс преобразования логического адреса в линейный иллюстрируется на Рис.286.



**Рис.286 - Преобразование логического адреса в линейный**

Значение из поля индекса селектора используется в качестве индекса в таблице LDT/GDT для выборки 32-разрядного базового адреса. Этот базовый адрес складывается со второй компонентой логического адреса – 32-разрядным смещением. В результате получается 32-разрядный линейный адрес.

На следующем этапе происходит страничное преобразование линейного адреса в физический, как показано на Рис.287.



**Рис.287 - Преобразование линейного адреса в физический**

Процесс вычисления адреса страницы часто называют трансляцией страниц. Старшие 10 бит линейного адреса используются как индекс в таблице, называемой каталогом таблиц страниц. Расположение каталога таблиц страниц в физической памяти определяется содержимым системного регистра процессора CR3.

Каталог таблиц страниц содержит дескрипторы таблиц страниц, определяющие физический адрес таблиц страниц. В каталоге таблиц страниц всего может быть до 1024 дескрипторов. Самих же каталогов может быть сколько угодно, но в каждый момент времени используется только один – тот, на который указывает регистр CR3.

Следующие 10 бит линейного адреса предназначены для индексации таблицы страниц, выбранной с помощью старших 10 бит адреса. Таблица страниц содержит 1024 дескриптора, определяющих физические адреса страниц памяти. Размер одной страницы составляет 4 Кбайт ( $2^{12} = 4096$  байт). Младшие 12 бит линейного адреса указывают смещение к адресуемому байту внутри страницы.

На Рис.288 представлен формат дескриптора таблицы страниц.

|                               |    |      |     |   |   |     |   |   |   |
|-------------------------------|----|------|-----|---|---|-----|---|---|---|
| 31                            | 12 | 11-9 | 8-7 | 6 | 5 | 4-3 | 2 | 1 | 0 |
| Адрес каталога таблиц страниц |    | AVL  | 0   | D | A | 0   | U | W | P |

**Рис.288 - Дескриптор таблицы страниц**

Для представления старших 20 битов физического адреса таблицы страниц в дескрипторе используются биты 12-31. Младшие 12 битов адреса таблицы всегда равны нулю, таким образом, таблица страниц должна быть выровнена в памяти на границу 4096 байт (на границу страницы).

Формат дескриптора страницы представлен на Рис.289.

|                       |    |      |     |   |   |     |   |   |   |
|-----------------------|----|------|-----|---|---|-----|---|---|---|
| 31                    | 12 | 11-9 | 8-7 | 6 | 5 | 4-3 | 2 | 1 | 0 |
| Адрес таблицы страниц |    | AVL  | 0   | D | A | 0   | U | W | P |

**Рис.289 - Дескриптор страницы**

Биты 12-31 в дескрипторе страниц указывают старшие 20 бит физического адреса страницы. Младшие 12 бит адреса страницы всегда равны нулю.

Назначение бит 0-11 одинаково и для дескриптора таблицы страниц, и для дескриптора страницы. В таблице на Рис.290 приведено описание этих бит.

Как видно, что для процессора i80386 добавилось ещё два типа таблиц, содержащих дескрипторы и, соответственно, два типа дескрипторов – дескриптор таблиц страниц и дескриптор страниц.

Для использования механизма трансляции страниц операционная система должна установить в 1 старший бит системного регистра CR0. Если этот бит не установлен в 1, физический адрес будет равен линейному, содержимое регистра адреса каталога таблиц страниц CR3 при этом для преобразования адреса использоваться не будет.

Форматы дескрипторов, располагающихся в таблицах GDT, LDT и IDT претерпели изменения по сравнению с используемыми в процессоре i80286. Например, вместо 24-битового физического адреса в дескрипторах должен находиться 32-битовый линейный адрес.

| Номер бита | Назначение                                                                                                                                                                                                                                                     |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0 (P)      | Бит присутствия в памяти. Установлен в 1, если определяемая данным дескриптором таблица страниц находится в оперативной памяти. Этот бит используется для организации виртуальной памяти.                                                                      |
| 1 (W)      | Разрешение записи. Если бит установлен в 1, то запись в страницы разрешена. Бит используется для организации защиты от записи на уровне страниц.                                                                                                               |
| 2 (U)      | Пользователь/супервизор. Используется для разграничения доступа к страницам операционной системы (страницы супервизора) и страницам программ пользователя. Значение бита, равное 0, соответствует страницам супервизора, 1 - страницам программы пользователя. |
| 3-4        | Эти биты зарезервированы и должны быть установлены в 0 для совместимости со следующими моделями процессора.                                                                                                                                                    |
| 5 (A)      | Бит доступа. Он устанавливается процессором перед выполнением операций чтения страницы или записи в страницу.                                                                                                                                                  |
| 6 (D)      | Бит мусора. Устанавливается, если была выполнена запись в каталог или страницу.                                                                                                                                                                                |
| 7-8        | Эти биты зарезервированы и должны быть установлены в 0 для совместимости со следующими моделями процессора.                                                                                                                                                    |
| 9-12 (AVL) | Эти биты доступны для использования операционной системой (AVL - Available for use).                                                                                                                                                                           |

Рис.290 - Биты 0-12 дескрипторов таблиц страниц и страниц

Как упоминалось ранее, в дескрипторах процессора i80286 было зарезервировано два байта, i80386 эти байты задействовал (Рис.291).

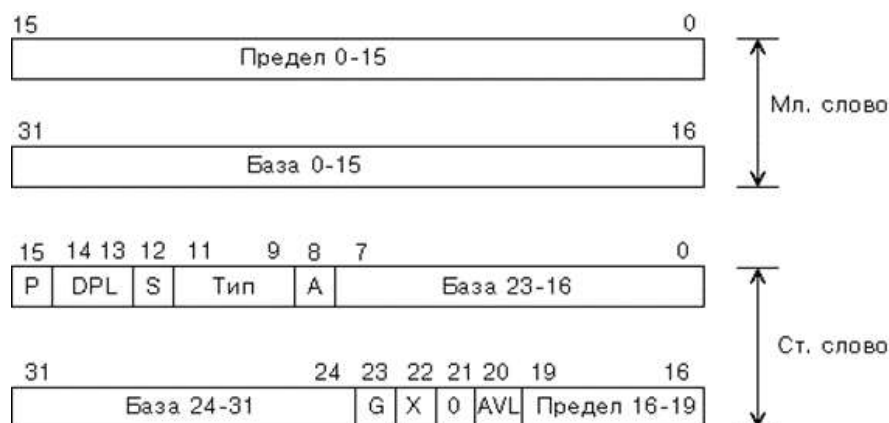


Рис.291 - Дескрипторы для процессора i80386

Процессор i80386 использует 32-разрядный базовый адрес сегмента и 20-разрядное поле предела. В зарезервированном для процессора i80286 поле в битах 24-31 находится старший байт 32-разрядного базового адреса сегмента. Биты 16-19 используются для хранения старших четырёх битов предела.

В процессоре i80286 поле предела указывало размер сегмента в байтах. Для процессора i80486 интерпретация поля предела зависит от установки бита G – бита *гранулярности*. Если бит G установлен в 1, поле предела содержит размер сегмента в *страницах* (размером 4096 байт). Если бит G сброшен, размер сегмента вычисляется в *байтах*.

Бит гранулярности G также находится в поле, которое в процессоре i80286 было отмечено как зарезервированное.



Поле, обозначенное на Рис.291 как X, указывает разрядность выполняемых команд, принятых по умолчанию. Если этот бит установлен в 1, используются 32-разрядные команды, если сброшен в 0 – 16-разрядные.

Бит AVL предназначен для использования системным программным обеспечением.

Бит S – признак системного сегмента. Если этот бит сброшен в 0, то сегмент системный.

Назначение остальных полей дескриптора аналогично используемому в процессоре i80286. Ниже (Рис.292) эта таблица приведена ещё раз.

| Поле TYPE | Тип сегмента                                   |
|-----------|------------------------------------------------|
| 0         | Запрещённое значение                           |
| 1         | Доступный TSS для процессора i80286            |
| 2         | Сегмент LDT                                    |
| 3         | Занятый TSS для процессора i80286              |
| 4         | Вентиль вызова для процессора i80286           |
| 5         | Вентиль задачи для процессоров i80286 и i80386 |
| 6         | Вентиль прерывания для процессора i80286       |
| 7         | Вентиль исключения для процессора i80286       |
| 8         | Запрещённое значение                           |
| 9         | Доступный TSS для процессора i80386            |
| A         | Зарезервировано                                |
| B         | Занятый TSS для процессора i80386              |
| C         | Вентиль вызова для процессора i80386           |
| D         | Зарезервировано                                |
| E         | Вентиль прерывания для процессора i80386       |
| F         | Вентиль ловушки для процессора i80386          |

Рис.292 - Типы системных сегментов

## Обработка прерываний

Процессор i80386 в защищённом режиме обрабатывает прерывания точно также, как и процессор i80286. Есть отличия в формате дескрипторов, располагающихся в дескрипторной таблице прерываний IDT. Эти отличия заключаются в использовании других значений в поле типа (см. таблицу на Рис.292) и в том, что два байта, зарезервированные в вентилях прерывания и исключения процессора i80286, используются процессором i80386 для хранения адресных битов 16-31 32-разрядного смещения.

## Мультизадачность

В процессоре i80386 получила дальнейшее развитие аппаратная поддержка мультизадачности, впервые введённая в процессоре i80286.

На Рис.293 представлен формат сегмента TSS для процессора i80386.

Из рисунка видно, что в TSS предусмотрены поля для хранения сегментных регистров GS, FS, DS, SS, CS, ES (процессор i80386 имеет два новых сегментных регистра – GS и FS). Имеется поле для хранения содержимого регистра LDTR, указывающего на локальную таблицу дескрипторов, распределённую данной задаче.

Для хранения содержимого 32-разрядных регистров используются поля TSS, обозначенные на рисунке как EDI, ESI, EBP, ESP, EBX, EDX, ECX, EAX, EFLAGS, EIP.



Поле CR3 хранит содержимое системного регистра CR3. Этот регистр является указателем на каталог таблиц страниц (физический адрес). Таким образом, каждая задача может иметь свой собственный каталог таблиц страниц, что позволяет выполнить изоляцию задач не только на уровне сегментов, но и на уровне страниц.

TSS процессора i80386 содержит указатели на стеки для второго, первого и нулевого приоритетных колец. Это поля SS2:ESP2, SS1:ESP1, SS0:ESP0.

Поле LINK используется для ссылки на TSS вызвавшей задачи при вложенном вызове задач, аналогично тому как это было в процессоре i80286.

Бит T используется для отладки. Если он установлен в 1, при переключении на задачу возникает отладочное исключение, которое может быть использовано системным отладчиком.

|                         |        |   |
|-------------------------|--------|---|
| База карты ввода-вывода | 0      | T |
| 0                       | LDTR   |   |
| 0                       | GS     |   |
| 0                       | FS     |   |
| 0                       | DS     |   |
| 0                       | SS     |   |
| 0                       | CS     |   |
| 0                       | ES     |   |
|                         | EDI    |   |
|                         | ESI    |   |
|                         | EBP    |   |
|                         | ESP    |   |
|                         | EBX    |   |
|                         | EDX    |   |
|                         | ECX    |   |
|                         | EAX    |   |
|                         | EFLAGS |   |
|                         | EIP    |   |
|                         | CR3    |   |
| 0                       | SS2    |   |
|                         | ESP2   |   |
| 0                       | SS1    |   |
|                         | ESP1   |   |
| 0                       | SS0    |   |
|                         | ESP0   |   |
| 0                       | LINK   |   |

Рис.293 - Сегмент TSS процессора i80386

Есть ещё одно очень интересное новшество, введённое в процессор i80386 – битовая карта ввода/вывода.

Как упоминалось ранее – для обеспечения безопасной работы системы необходимо ограничить доступ пользовательским программам ко всем или, по крайней мере к некоторым, портам ввода/вывода. Злонамеренная программа, имеющая доступ к портам контроллера прямого доступа к памяти, может выполнить с помощью этого контроллера чтение или запись информации по любым физическим адресам, обходя таким способом межпрограммную защиту в процессоре. Процессор i80286 хранит в регистре флагов уровень привилегий IOPR, на котором разрешено выполнять команды ввода/вывода. С помощью этого механизма можно запретить непривилегированным программам выполнять команды ввода/вывода.

Однако такой способ защиты не слишком удобен. Некоторые порты ввода/вывода не только безопасны для использования, но и весьма полезны для обычных программ (например, порт системного динамика или принтера).

Битовая карта ввода/вывода процессора i80386 позволяет для каждой задачи определить порты, которые эта задача может использовать. То есть операционная система имеет

возможность санкционировать любую задачу для использования любого набора адресов портов ввода/вывода. Если задача попытается обратиться к несанкционированному порту ввода/вывода, произойдёт исключение.

Сегмент TSS содержит поле, обозначенное на Рис.293 как *база карты ввода/вывода*. Оно служит для указания расположения битовой карты ввода/вывода задачи, использующей данный TSS.

Поле базы карты ввода/вывода указывает 16-разрядное смещение начала битовой карты ввода/вывода относительно TSS. Предел TSS должен определяться с учётом карты ввода/вывода.

Каждый бит в карте ввода/вывода соответствует адресу байта порта ввода/вывода. После битовой карты должен располагаться байт 0x0FF.

При выполнении 16- или 32-разрядных операций ввода/вывода процессор проверяет все биты (2 или 4 бита), соответствующие адресу порта. Если проверяемый бит установлен в 1, происходит исключение.

Для тех программ, которые являются привилегированными, если уровень привилегий меньше или равен уровню IOPL, процессор не выполняет проверку битовой карты ввода/вывода.

Для того чтобы полностью запретить задаче обращаться к портам ввода/вывода, достаточно установить базу карты ввода/вывода большей или равной пределу TSS. В этом случае любая команда ввода/вывода приведёт к генерации исключения.

## Режим виртуального процессора i8086

Процессор i80386 содержит средства для работы в так называемом (под)режиме виртуального процессора i8086, называемого также для краткости режимом v86 или просто виртуальным режимом. Отметим, что до разработки фирмой Intel процессора i80836 термин «виртуальный режим» иногда использовался в литературе для обозначения защищённого режима работы процессора i80286.

В режим v86 процессор может перейти из защищённого режима, если установить в регистре флагов EFLAGS бит виртуального режима (VM-бит). Номер бита VM в регистре EFLAGS – 17.

Когда процессор i80386 находится в виртуальном режиме, его поведение во многом напоминает поведение старого доброго процессора i8086. В частности, для адресации памяти используется схема *сегмент:смещение*, размер сегмента составляет 64 КБайт, а максимальный размер адресуемой в этом режиме памяти – 1 МБайт.

Виртуальный режим предназначен для работы программ, ориентированных на процессор i8086 (или i8088). Но виртуальный режим – это не реальный режим процессора i8086, в его работе имеются существенные отличия, например, процессор, работая в режиме v86, фактически продолжает использовать схему преобразования адресов памяти и средства мультизадачности защищённого режима.

В виртуальном режиме используется механизм трансляции страниц памяти. Это позволяет в мультизадачной операционной системе создавать несколько задач, работающих в виртуальном режиме, и каждая из этих задач может иметь собственное адресное пространство, размером в 1 МБайт.

Все задачи виртуального режима обычно выполняются в третьем, наименее привилегированном кольце защиты. Когда в такой задаче возникает прерывание, процессор автоматически переключается из виртуального режима в защищённый. Поэтому все прерывания отображаются в операционную систему, работающую в защищённом режиме.

Обработчики прерываний защищённого режима могут моделировать функции соответствующих прерываний реального режима, что необходимо для правильной работы программ, ориентированных на реальный режим операционной системы MS-DOS.

## Виртуальные машины

Обычно мультизадачные операционные системы предоставляют для выполнения i8086-программ «виртуальную машину i8086». Эти виртуальные машины реализуются при помощи задач виртуального (под)режима v86.

Операционная система может виртуализовать ресурсы компьютера – память, порты ввода/вывода, систему обработки прерываний. При этом программа, ориентированная на реальный режим MS-DOS, получает в своё распоряжение ресурсы, которые она воспринимает как физические.

Виртуализация памяти заключается в преобразовании адреса формата реального режима *сегмент:смещение* в линейный, а затем в физический с использованием схемы преобразования адресов защищённого режима. В этом случае программе предоставляется 1 Мбайт памяти, которая может быть расположена в любом месте физической памяти компьютера и может подвергаться свопингу (swap) на диск для реализации виртуальной оперативной памяти. С точки зрения программы, работающей в виртуальной машине, адреса отображаются на этот 1 Мбайт памяти так, как будто бы это была физическая память компьютера.

Виртуализация ввода/вывода основана на том, что такие команды, как IN, OUT, INS, OUTS, CLI, STI чувствительны к текущему уровню IOPL. Так как обычно виртуальная машина работает в непривилегированном третьем кольце защиты, выдача этих команд программой в виртуальном режиме приводит к исключению. Операционная система, перехватив исключение, может эмулировать действие этих команд безопасным для себя образом, но это, однако, может привести к существенному замедлению работы виртуальной машины (обработка исключения на каждую операцию ввода/вывода – это достаточно ресурсоёмкая процедура).

Виртуализация прерываний основана на выходе процессора из виртуального режима и возврате в защищённый при возникновении прерываний. При возникновении прерывания управление получает обработчик защищённого режима, установленный операционной системой. Операционная система может эмулировать функции соответствующего обработчика реального режима, например, функции обработчика прерывания INT 21h MS-DOS или BIOS.

## Переключение в защищённый и реальный режимы

Процессоры i80386 и i80486 могут легко переключаться из реального режима в защищённый и обратно с помощью команды MOV, в отличие от процессора i80286, в котором переход из защищённого режима в реальный был возможен только путём сброса процессора. Младший бит PE системного регистра CR0 определяет текущий режим работы процессора. Если этот бит установлен в 1, процессор работает в защищённом режиме, а если в 0 – в реальном.

Для переключения процессора из реального режима в защищённый можно использовать, например, такую последовательность команд:

```
mov ax, cr0
or ax, 1
mov cr0, ax
```

Для совместимости с процессором i80286 оставлена возможность переключения в защищённый режим с помощью команды LMSW.

Для возврата в реальный режим необходимо сбросить бит PE:

```
mov ax, cr0
and ax, 0xfffe
```

mov cr0, ax

Таким образом, появился более красивый и удобный способ возврата в реальный режим, чем выполнение аппаратного сброса или перевод процессора в состояние отключения (конечно, вы по-прежнему можете пользоваться старым способом перевода процессора в реальный режим, если вам это зачем-то нужно).

Перед переключением в реальный режим из защищённого программа должна выполнить следующие действия:

- обеспечить равенство линейных адресов физическим;
- отключить трансляцию страниц, сбросив бит PG в регистре CR0;
- загрузить ноль в регистр CR3 для сброса кэш-памяти страниц;
- передать управление сегменту кода с пределом 64 килобайта;
- загрузить в сегментные регистры SS, DS, ES, FS, GS селекторы дескрипторов, подготовленных для адресации памяти в реальном режиме и содержащих соответствующие реальному режиму значения;
- запретить маскируемые и немаскируемые прерывания;
- сбросить бит PE, переключив процессор в реальный режим;
- выполнить команду длинного (межсегментного) перехода, для очистки внутренней очереди команд процессора;
- настроить систему прерываний для работы в реальном режиме;
- разрешить прерывания;
- загрузить в сегментные регистры значения, необходимые для работы в реальном режиме.

Как видно, процедура возврата в реальный режим сильно упростилась и ускорилась по сравнению с тем, как это было для процессора i80286. Однако большинство программ, переключившись в защищённый режим, никогда больше не возвращаются обратно. Они либо всё время работают в защищённом режиме, либо переключаются в режим виртуального процессора 8086.

## Описание моделей памяти в защищённом режиме

В защищенном режиме процессора 386 появилась возможность работать с четырьмя новыми моделями памяти (недоступными в реальном режиме).

### *Плоская или линейная модель памяти (flat memory)*

Поскольку адресация в защищённом режиме стала 32-битной (но с сохранением возможности создания 16-битных сегментов, для обратной совместимости с 80286), она позволила впервые со времени появления 8086-го забыть о сегментации, а точнее, об ограничении размера сегмента  $2^{16} = 64$  Кбайт.

До появления i386 программы и операционные системы использовали несколько разных моделей организации памяти (крохотная — *tiny*, малая — *small*, большая — *large*, огромная — *huge*), различающихся по организации в памяти сегментов кода, стека и данных. 32-битный адрес позволил использовать вместо них одну простую плоскую (*flat*), по сути безсегментную, модель — 32-битный вариант крохотной модели, в которой все сегменты задачи находятся в одном адресном пространстве. Плоская модель обеспечивает размер такого «общего» сегмента до почти 4 Гбайт, которых по тем временам (середина 1980-х) хватало для любой мыслимой задачи.

Но плоская модель имеет и недостатки:

- в ней появляются проблемы перемещаемости машинного кода, которые ранее легко решались сегментацией; обеспечение перемещаемости легло на плечи операционных систем, с новыми форматами дискового образа программы.
- плоская модель практически сводит на нет управление памятью в защищённом режиме (ограничение доступа и поддержка виртуальной памяти), которое до i386 могло выполняться только на уровне сегментов. Только появление (добавление) новой модели управления памятью — страничного преобразования — обеспечило плоской модели её сегодняшнюю популярность.

Плоская модель вошла в обиход столь широко, что современные программисты зачастую и не подозревают, что программы обращаются в память через сегменты.

### ***Сегментированная модель памяти (segmented memory)***

Подобно тому, как это делается в реальном режиме, в защищённом режиме память может делиться на отдельные пространства адресов – сегменты. При этом программный код, данные и стек размещаются в отдельных сегментах памяти. Программы в 32-разрядном режиме могут использовать до 16383 сегментов разного размера, каждый из которых может иметь размер  $2^{32}$  байт (4 Гбайт). Однако в отличие от реального режима, преобразование логических адресов в физические в сегментированной модели памяти защищённого режима выполняется значительно сложнее. По-прежнему логический адрес формируется при помощи сегментных регистров и регистров, в которых хранятся смещения. Однако сегментные регистры теперь хранят не адрес сегмента, а селектор. В результате каждая задача может иметь размер в те самые невероятные 64 Тбайта.

### ***Страничная модель памяти (paging)***

По сути является надстройкой над сегментированной или плоской моделью памяти. В этом режиме память рассматривается как совокупность блоков фиксированного размера (страниц) размером 4 Кбайт. Начиная с пятого поколения процессоров, появилась возможность увеличения размера страницы до 4 Мбайт. Страничная модель памяти гораздо проще организует работу виртуальной памяти, по сравнению с сегментной моделью. Благодаря виртуальной памяти программы могут использовать для работы объём памяти больший, чем объём физической памяти. Суть виртуальной памяти заключается в том, что страницы могут выгружаться из физической оперативной памяти на диск (в файл обмена, файл подкачки или swp-файл) и по мере необходимости подкачиваться с него обратно. Через страничное преобразование i386 может адресовать до 4 Гбайт физической памяти и до 64 Тбайт виртуальной памяти. Виртуальная память включается в настройках операционной системы. Разбиение на страницы выполняется на аппаратном уровне и программисту знать подробностей, как правило, не требуется. На программном уровне страничный механизм включается установкой специального бита (PG) в регистре CR0 при помощи привилегированной команды.

### ***Модель памяти в режиме v86***

Эта модель уже была описана выше, повторим здесь её характеристики. С точки зрения программиста эта модель памяти работает точно также как в обычном реальном режиме. Т.е. память делится на сегменты по 64 Кбайт, ячейки внутри которых адресуются с помощью двух слов, записываемых в виде *сегмент:смещение*, максимальная адресуемая память 1 Мбайт и пр. Однако в режиме v86 выполняются все проверки защиты защищённого режима, из-за чего в некоторых случаях не будут работать некоторые инструкции, которые работали в «чистом» защищённом режиме. Особенно это касается инструкций ввода-вывода IN, OUT, (REP) INS, (REP) OUTS и инструкций обработки прерываний: INT n, PUSFF, POPF, STI, CLI и IRET.

## Система команд i80386/486

Далее, в целях экономии места, будет приведено только краткое описание отличий в наборе инструкций (системе команд) процессоров i80386/486.

### Процессор i80386

Процессор i80386 был совместим с программным обеспечением для процессоров более ранних моделей – i8086/88, i80286. Главным отличием 386-го от моделей предыдущих поколений была его 32-разрядная архитектура.

Набор инструкций i386 был расширен, в основном, за счёт добавления 32-битных вариантов существующих инструкций (образованных 32-битными префиксами), включая те, у которых 32-битные варианты имеют другие мнемоники (*pushad/popad*, *pushfd/popfd*, *cwd\*/cdq*, *movsd/cmpps/scasd/lodsd/insd/outsd*, *iretd*), а также команды *mov* для новых регистров. Другие новые инструкции:

- установка байта по условию (*set\**);
- команды переходов (*jmp*, *j\**) с относительным 16-битным смещением;
- команды загрузки полного адреса для новых сегментных регистров (*lfs*, *lgs*);
- логические команды (*and*, *or*, *xor*) при работе с 16-разрядными непосредственными данными получили возможность использовать знаковые расширения байта, что позволяет сократить такие команды на байт, когда данные помещаются в байт (эта возможность впервые была описана в фирменной документации на 80386, но, вполне возможно, имела и в более ранних процессорах).

Средняя длина инструкций составляла 3.2 байта.

### Процессор i80486

Процессор i80486 практически полностью совместим с более ранними моделями – i8086, i80286, i80386, но в тоже время обладает существенно более высоким быстродействием. Наиболее часто встречающиеся в программах команды выполняются за один машинный цикл (а когда-то для этого требовалось несколько циклов, иногда даже более десятка). На кристалле процессора находится 8-килобайтный кэш оперативной памяти, обеспечивающий высокую производительность даже при использовании относительно медленной памяти.

Кроме того, процессор i80486 содержит встроенный 32-разрядный арифметический сопроцессор, значительно увеличивающий скорость выполнения арифметических команд.

С точки зрения прикладного программиста процессор i80486 практически не отличается от процессора i80386 (за исключением более высокой производительности). Все отличия касаются только системного уровня. Как правило, большинство программистов никогда не работают напрямую с системными регистрами процессоров i80386 и i80486, оставляя эту работу операционной системе или драйверам расширенной памяти, поэтому подробности в этой части, для экономии места, будут опущены.

В процессоре i80486 стали определены некоторые, зарезервированные ранее в процессоре i80386, биты регистров CR0, CR3, EFLAGS, появились некоторые новшества в таблицах страниц, добавилось новое исключение и несколько новых команд.

- В регистре CR0 стали определены пять новых битов – NE, WP, AM, NW, CD.
- В регистре CR3 определены два новых бита – PCD и PWT.
- В регистре EFLAGS появился новый флаг AC (бит 18). Этот бит вместе с битом AM регистра CR0 контролирует проверку выравнивания объектов в памяти.

Форматы системных регистров CR0 и CR3 для процессора i80486, а также регистра EFLAGS приведены ниже на Рис.294.

| Номер бита   | Назначение                                                                                   | Регистр    | Назначение                                                                                                                                                                                                                    |
|--------------|----------------------------------------------------------------------------------------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0 - CF       | Флаг переноса                                                                                | CR0        | Регистр состояния процессора                                                                                                                                                                                                  |
| 1 - 1        | Зарезервировано и равно 1                                                                    | CR1        | Зарезервирован                                                                                                                                                                                                                |
| 2 - PF       | Флаг чётности                                                                                | CR2        | Линейный адрес отказа страницы                                                                                                                                                                                                |
| 3 - 0        | Зарезервировано и равно 0                                                                    | CR3        | Базовый адрес каталога страницы                                                                                                                                                                                               |
| 4 - AF       | Флаг вспомогательного переноса                                                               | в)         |                                                                                                                                                                                                                               |
| 5 - 0        | Зарезервировано и равно 0                                                                    |            |                                                                                                                                                                                                                               |
| 6 - ZF       | Флаг нуля                                                                                    |            |                                                                                                                                                                                                                               |
| 7 - SF       | Флаг знака                                                                                   |            |                                                                                                                                                                                                                               |
| 8 - TF       | Флаг ловушки                                                                                 |            |                                                                                                                                                                                                                               |
| 9 - IF       | Флаг разрешения прерываний                                                                   |            |                                                                                                                                                                                                                               |
| 10 - DF      | Флаг направления                                                                             |            |                                                                                                                                                                                                                               |
| 11 - OF      | Флаг переполнения                                                                            |            |                                                                                                                                                                                                                               |
| 12-13 - IOPL | Уровень привилегий ввода/вывода                                                              |            |                                                                                                                                                                                                                               |
| 14 - NT      | Флаг вложенной задачи                                                                        |            |                                                                                                                                                                                                                               |
| 15 - 0       | Зарезервировано и равно 0                                                                    |            |                                                                                                                                                                                                                               |
| 16 - RF      | Флаг возобновления (только i80386 и i80486)                                                  |            |                                                                                                                                                                                                                               |
| 17 - VM      | Флаг режима виртуального процессора 8086 (только i80386 и i80486)                            |            |                                                                                                                                                                                                                               |
| 18 - AC      | Флаг проверки выравнивания (только i80486)                                                   |            |                                                                                                                                                                                                                               |
| 19-31 - 0    | Зарезервировано и равно 0                                                                    |            |                                                                                                                                                                                                                               |
| а)           |                                                                                              |            |                                                                                                                                                                                                                               |
| Номер бита   | Назначение                                                                                   | Номер бита | Назначение                                                                                                                                                                                                                    |
| 0-2          | Зарезервировано                                                                              | 0 - PE     | Включение защищённого режима работы процессора.                                                                                                                                                                               |
| 3 - PWT      | Прозрачность записи на уровне страниц. Используется для управления записью во внешний кэш.   | 1 - MP     | Присутствие сопроцессора.                                                                                                                                                                                                     |
| 4 - PCD      | Запрещение кэширования на уровне страниц. Используется для управления работой внешнего кэша. | 2 - EM     | Эмуляция сопроцессора.                                                                                                                                                                                                        |
| 5-31 - PBDR  | Базовый регистр каталога страниц                                                             | 3 - TS     | Переключение задачи.                                                                                                                                                                                                          |
| б)           |                                                                                              | 4 - ET     | Тип сопроцессора - i80287 или i80387.                                                                                                                                                                                         |
|              |                                                                                              | 5-14       | Зарезервировано                                                                                                                                                                                                               |
|              |                                                                                              | 15 - PG    | Включение механизма трансляции страниц                                                                                                                                                                                        |
|              |                                                                                              | г)         |                                                                                                                                                                                                                               |
|              |                                                                                              | Номер бита | Назначение                                                                                                                                                                                                                    |
|              |                                                                                              | 0 - PE     | Включение защищённого режима работы процессора.                                                                                                                                                                               |
|              |                                                                                              | 1 - MP     | Присутствие сопроцессора.                                                                                                                                                                                                     |
|              |                                                                                              | 2 - EM     | Эмуляция сопроцессора.                                                                                                                                                                                                        |
|              |                                                                                              | 3 - TS     | Переключение задачи.                                                                                                                                                                                                          |
|              |                                                                                              | 4 - ET     | Тип сопроцессора - i80287 или i80387.                                                                                                                                                                                         |
|              |                                                                                              | 5 - NE     | Числовая ошибка. Разрешает обработку ошибок при операциях с плавающей точкой.                                                                                                                                                 |
|              |                                                                                              | 6-15       | Зарезервировано                                                                                                                                                                                                               |
|              |                                                                                              | 16 - WP    | Защита записи. При установке этого бита страницы пользователя защищены от записи в режиме супервизора.                                                                                                                        |
|              |                                                                                              | 17         | Зарезервировано                                                                                                                                                                                                               |
|              |                                                                                              | 18 - AM    | Бит маски выравнивания. Этот бит разрешает или запрещает контроль выравнивания операндов команд в памяти. Контроль выравнивания разрешён только для программ, работающих в третьем кольце, при условии что установлен бит AM. |
|              |                                                                                              | 19-28      | Зарезервировано                                                                                                                                                                                                               |
|              |                                                                                              | 29 - NW    | Разрешение сквозной записи. Используется в механизме управления кэшированием.                                                                                                                                                 |
|              |                                                                                              | 30 - CD    | Запрещение кэширования. Если этот бит установлен, внутреннее кэширование запрещено.                                                                                                                                           |
|              |                                                                                              | 31 - PG    | Включение механизма трансляции страниц                                                                                                                                                                                        |
|              |                                                                                              | д)         |                                                                                                                                                                                                                               |

Рис.294 – Регистры процессоров i80386/486: а) регистр EFLAGS; б) регистр CR3 процессора i80486; в) управляющие регистры процессора i80386; г) регистр CR0 процессора i80386; д) регистр CR0 процессора i80486.

Новое исключение касается проверки выравнивания и имеет номер 17. Оно возникает при попытке обращения к данным, не выровненным в памяти. Для того, чтобы при обращении к не выровненному операнду в памяти произошло исключение контроля выравнивания, должны быть установлены флаг AC в регистре EFLAGS и бит AM в управляющем регистре CR0.

Исключение контроля выравнивания генерируется только в программах, работающих в третьем, непривилегированном кольце.

Добавились три команды, предназначенные для использования прикладными программами – BSWAP, XADD, CMPXCHG, а также три новые системные команды, управляющие кэшем и TLB – INVD, WBINVD, INVLPG. Новые команды облегчают и ускоряют работу с семафорами, что очень важно для мультизадачных операционных систем.

Есть также некоторые изменения в формате команды MOV, используемой для доступа к тестовым регистрам. Появились новые регистры для работы с кэшем.

В целом можно отметить, что архитектура процессора i80486 не претерпела революционных изменений по сравнению с процессором i80386. Поэтому в большинстве случаев прикладной программист может считать, что процессор i80486 – это быстрый вариант процессора i80386.

## Построение вычислительной системы

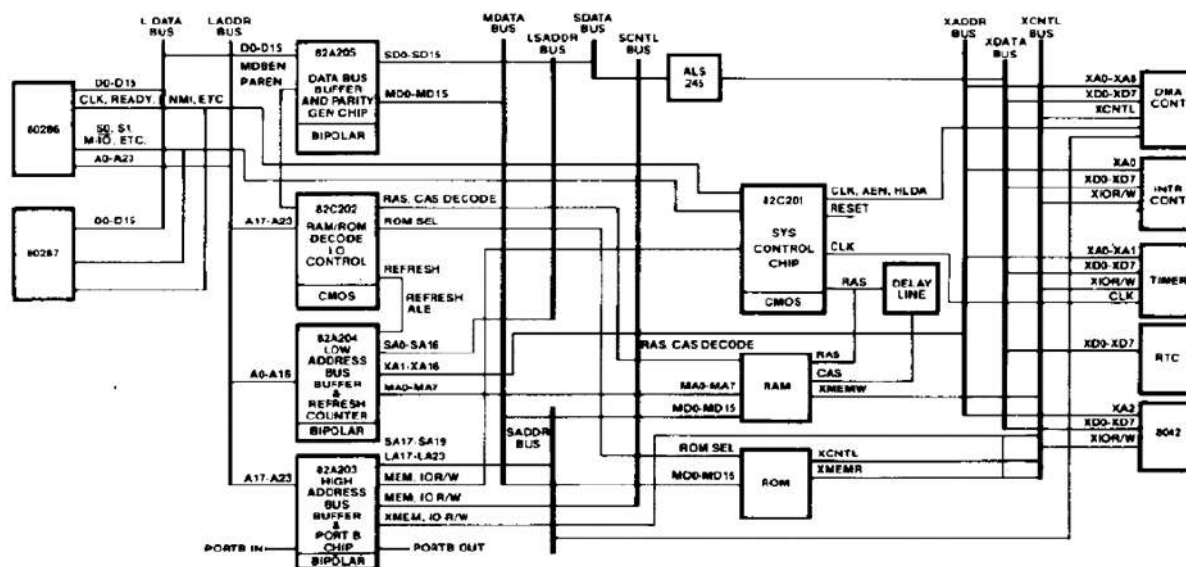
Если первые IBM PC/PC XT строились на базе универсальных, широкодоступных компонентов, то уже начиная с систем на базе i80286 разработчики компонентов для ПК стали разрабатывать первые специализированные наборы микросхем, «обвязку» процессора, обеспечивающую взаимодействие CPU с окружающим его «миром», то, что получило общее название *chipset*.

Формальное определение: чипсёт (chipset) — набор микросхем, спроектированных для совместной работы с целью выполнения набора заданных функций. В компьютерах чипсет, размещаемый на материнской плате, выполняет функцию связующего компонента (моста), обеспечивающего взаимодействие центрального процессора с различными типами памяти, устройствами ввода-вывода, контроллерами и адаптерами ПУ, как непосредственно через себя (и имея некоторые из них в своём составе), так и через другие контроллеры и адаптеры, с помощью многоуровневой системы шин, так как ЦП, как правило, не может взаимодействовать с ними напрямую.

Чипсет определяет функциональность системной платы. Он включает в себя интерфейс шины процессора и определяет в конечном счете тип и быстродействие используемого процессора. Определяет во многом тип, объём, быстродействие и вид поддерживаемой памяти, рабочие частоты различных шин, их разрядность и тип, поддержку плат расширения, их количество и тип и т. д. Таким образом, этот набор микросхем относится к числу наиболее важных компонентов системы, во многом определяя её быстродействие, расширяемость, стабильность работы при различных настройках и условиях, модернизируемость, сферу применения и т. д. Являясь по сути основой платформы/системной платы, чипсеты встречаются и в других устройствах, например, в сотовых телефонах и сетевых медиаплеерах.

Первой чипсет – набор назывался CS8220 – для IBM PC/AT-совместимых систем предложила компания *Chips & Technologies*. Это был 5-кристальный набор, выполненный с применением разных технологий: два малопотребляющих CMOS кристалла 82C201 и 82C202, и три чипа по технологии маломощной TTL Schottky – 82A203, 82A204 и 82A205. Микросхемы 82C201 и 82C202 выполняли функции интеловского тактового генератора i82284, шинного контролера i82288, тактового генератора i8254A, заменяя в итоге примерно 30 микросхем малой и средней степени интеграции.

Микросхемы 82A203, 82A204 и 82A205 (ТТЛ Шотки) включали большинство требуемых для схемы РС/АТ-286 буферных схем и драйверов, обеспечивая вытекающий ток до -3.3 мА и втекающий до 24 мА. Ниже, на Рис.295, приведена блок-схема РС/АТ-286, построенная на чипсете CS8220:



**Рис.295 – Блок-схема РС/АТ**



Переход от десятков универсальных микросхем «обвязки» малой и средней интеграции к нескольким специализированным кристаллам чипсета позволяло сильно экономить площадь материнской платы, сократить энергопотребление, повысить надёжность системы в целом. Начиная с систем на базе i386, наличие чипсета стало обязательной частью архитектуры PC-совместимых компьютеров.

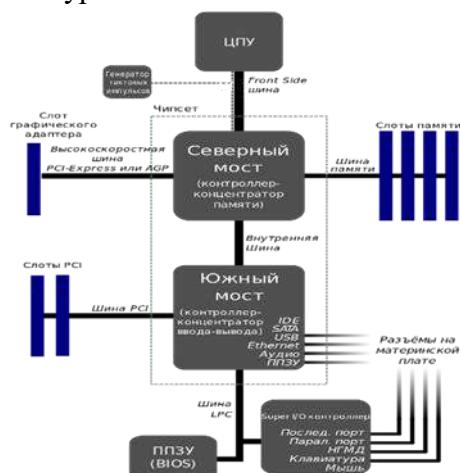


Рис.296 - Схематическое изображение традиционного чипсета материнской платы

В результате довольно скоротечной эволюции чипсет практически от всех производителей стал строиться по двухкристалльной схеме – микросхеме *северного* и *южного* моста, обобщённая схема приведена на Рис.296:

Термины *северный* и *южный* для обозначения мостов были выбраны, если можно так сказать, по географическому принципу – т.к. вверху на картах у нас располагается север, внизу – юг.

На Рис.297 ниже отображены функции чипсета:

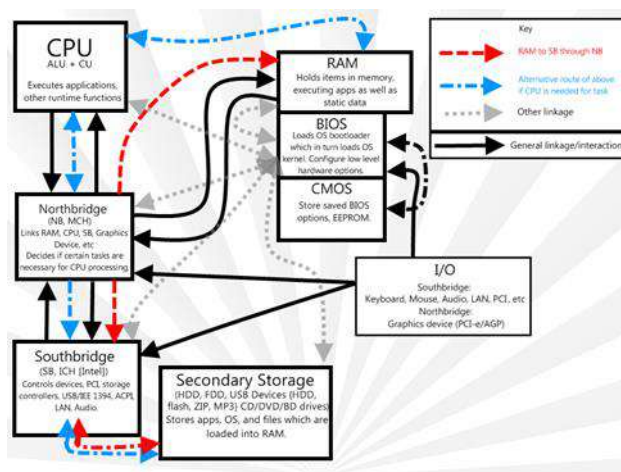


Рис.297 – Функции чипсета

Существуют и чипсеты, заметно отличающиеся от традиционной схемы. Например, у процессоров для разъёма LGA 1156 функциональность северного моста (соединение с видеокартой и памятью) полностью встроена в сам процессор, и, следовательно, чипсет для LGA 1156 состоит из одного единственного южного моста, соединённого с процессором через шину DMI.

Для своих процессоров i80286 и i80386SX Intel лицензировала чипсеты ROACH фирмы ZyMOS. В списке ранних чипсетов Intel входили:

- 82350 EISA
- 82350DT EISA
- 82310 MCA
- 82340SX PC AT
- 82340DX PC AT
- 82320 MCA
- 82360SL — чипсет для мобильных процессоров 80386SL и 80486SL. Включал в себя контроллеры DMA, PIC, COM- и LPT-порты и логику управления энергопотреблением.

Для i486 Intel выпускала чипсет собственной разработки, набор микросхем серии 420, три модификации: 420TX, 420ZX и 420EX<sup>1</sup>. Но всерьёз, полноценно, в гонку чипсетов для собственных процессоров Intel включилась только начиная с процессора Pentium (P5).

Список основных производителей чипсетов для архитектуры x86:

1. Intel
2. NVidia
3. ATI/AMD
4. VIA
5. SiS

## Подключение периферийных устройств.

На первых ПК на базе 386-го в качестве шины для подключения плат расширения была доступна только шина ISA, обычно 2-3 восьмиразрядных слота и 3-4 16-разрядных. Пропускная способность даже ISA16 была существенно ниже максимально возможных скоростей обмена на 32-разрядной шине данных процессора 386DX. Шина для работы с памятью была 32-разрядной и работала на другой частоте, чем ISA. В своё время работа всех шин на одной единой тактовой частоте была серьёзной проблемой для систем на базе i80286, где частота ISA-шины напрямую была связана с тактовой частотой процессора и памяти.

Производители систем полностью осознавали проблему – периферийная шина ISA была совершенно неадекватна процессору 386DX, что, в частности, побудило Intel выпустить «облегчённую» версию 386SX, у которого разрядность адресной шины и шины данных просто были выполнены сделаны такими же, как у шины ISA16. Но как-то нужно было решать проблемы и для полновесного 386DX.

Одной из попыток, описанных ранее, была архитектура PS/2, в которой для систем на базе 386-го использовалась проприетарная MCA-шина, выше были приведены её основные характеристики, повторим их кратко ещё раз:

- Разрядность данных – 32 бита;
- Пропускная способность – до 66 Мбайт/сек (теоретически), до 40 Мбайт/сек на практике;
- Ограниченный механизм Plug-and-Play;
- Устройства могли общаться друг с другом напрямую, минуя центральный процессор (режим multi master bus).

## Шина EISA

Новые компьютеры с шиной MCA оказались значительно дороже аналогов с ISA. Вдобавок, начинали пользоваться всё большей популярностью системы от других производителей: Dell, Research Machines, Olivetti. Многим производителям компьютеров основательно поднадоела политика IBM, в итоге они объединились и начали работать над альтернативным стандартом.

Альянс *AST Research, Compaq, Epson, Hewlett-Packard, NEC, Olivetti, Tandy, Wyse* и *Zenith Data Systems* шутливо назвали «Бандой девяти» (“Gang of Nine”). Результаты их труда обозначились уже в 1988 году, когда партнеры представили 32-битную шину *Extended Industry Standard Architecture (EISA)*. Она обладала практически всеми преимуществами MCA, но конструкционно и электрически была совместима с классической ISA, обеспечивая совместимость с уже выпущенными 8- и 16-битными адаптерами. Лицензия на шину EISA, особенно в сравнении с MCA, стоила недорого.

---

<sup>1</sup> Развитие 400-ой серии интеловского чипсета была продолжено для обеспечения поддержки процессоров следующих поколений P5 (Pentium) и P6 (Pentium Pro, Pentium II, Pentium III, Celeron и Xeon).

Для обеспечения обратной совместимости с ISA-платами разработчиками было придумано довольно остроумное решение – контактные площадки (ламели) в разъёме EISA размещались с шагом вдвое более частым, чем в разъёме ISA, при этом дополнительные ламели в EISA были размещены в шахматном порядке во втором ряду, выше ламелей в разъёме ISA (выделено красным прямоугольником на Рис.298 ниже).



**Рис.298 – Графический адаптер EISA**

Если адаптер ISA8/16 вставлялся в слот EISA, то он контактировал только с нижним рядом ламелей в слоте и шинная логика материнской платы работала с таким адаптером неотличимо, как если бы он был вставлен в обычный ISA-разъём. И обратная ситуация – если EISA-адаптер вставлялся в разъём ISA, он также контактировал со слотом только нижним рядом своих контактов и работал как адаптер ISA16. И только когда EISA-адаптер вставлялся в EISA-слот, плата расширения контактировала обоими рядами контактных площадок с каждой стороны и работала в полноценном 32-разрядном режиме.

Ниже на Рис.299 приведена схема распиновки 32-разрядного EISA слота. Необходимо отметить две новых сигнальных линии – EX32 и EX16, которые определяли, что bus slave поддерживает, соответственно, 32- или 16-разрядный цикл EISA. Если ни один из этих сигналов не был получен в начале цикла шины, выполнялся цикл ISA.

32 Bit EISA Bus – top view

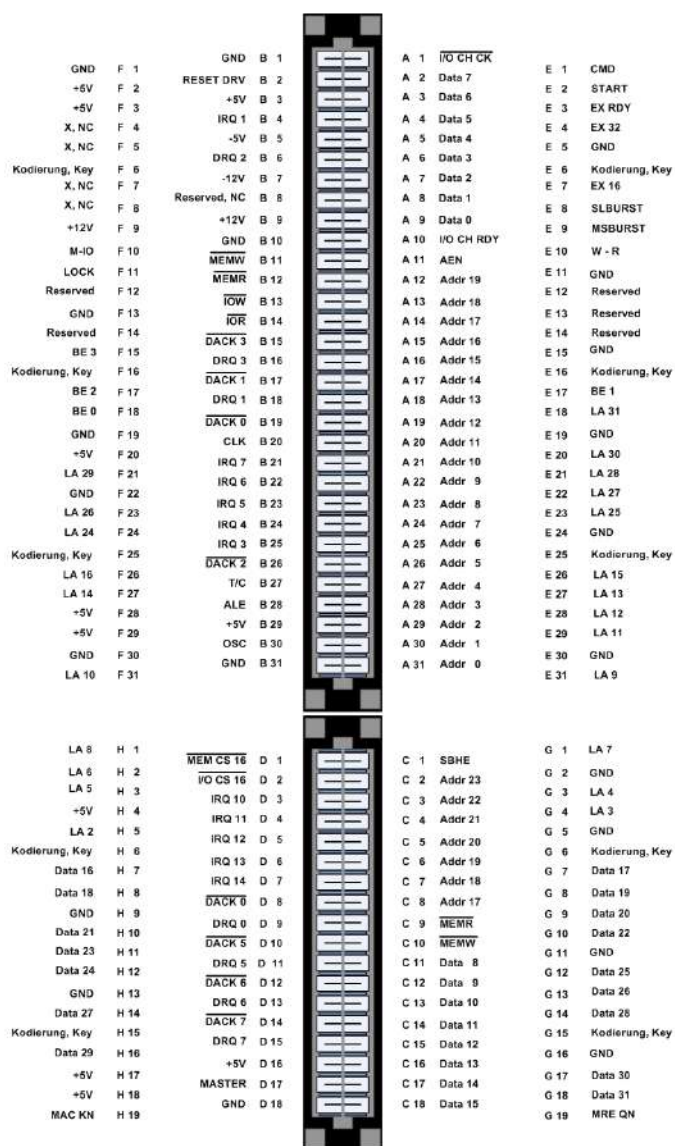


Рис.299 – 32-разрядный разъём EISA, вид сверху

Основные характеристики шины EISA:

- 32-разрядная передача данных;
- максимальная пропускная способность — до 33 МБ/сек;
- 32-разрядная адресация памяти позволяла адресовать до 4 Гбайт физического ОЗУ;
- поддержка multiply bus master;
- возможность задания двухуровневого прерывания как для edge-triggered прерывания, так и для level-triggered, что позволяло нескольким устройствам разделять один вход запроса прерывания;
- автонастройка плат расширения;

Важной особенностью шины являлась возможность для любого bus master обращаться к любому устройству памяти или периферийному устройству, даже если они имели разную разрядность шин. Говоря о полной обратной совместимости с ISA, следует отметить, что ISA-карты, естественно, не поддерживали разделение прерываний, даже будучи

вставленными в EISA-коннектор. Что касается поддержки multiply bus master, то она представляла собой улучшенную и дополненную версию таковой для ISA. На шине было реализовано четыре уровня приоритета:

1. схемы регенерации обновления памяти;
2. DMA;
3. процессор;
4. адаптеры шины.

«За порядком» следил арбитр шины EISA — периферийный контроллер (ISP — Integrated System Peripheral). Кроме этого, присутствовало ещё одно устройство — Intel Bus Master Interface Chip (BMIC), которое следило за тем, чтобы master «не засиживался» на шине, не захватывая её надолго. Через определённое количество тактов «засидевшийся» master «снимался» с шины устройством BMIC, и это событие генерировало немаскируемое прерывание (NMI).

Казалось бы, EISA должна была стать идеальным решением для ПК на базе интеловских 386/486-х процессоров, её предпочли большинство производителей PC, даже несмотря на то, что формально, в технических аспектах, шина MCA была более совершенным решением. Тем более, что полноценная поддержка EISA в чипсетах разных производителей появилась практически одновременно с появлением EISA на рынке. Вот так, к примеру (Рис.300), выглядела схема типовой конфигурации ПК на базе 486-го, с использованием шины EISA, а так (Рис.301) был организован контроллер шины EISA.

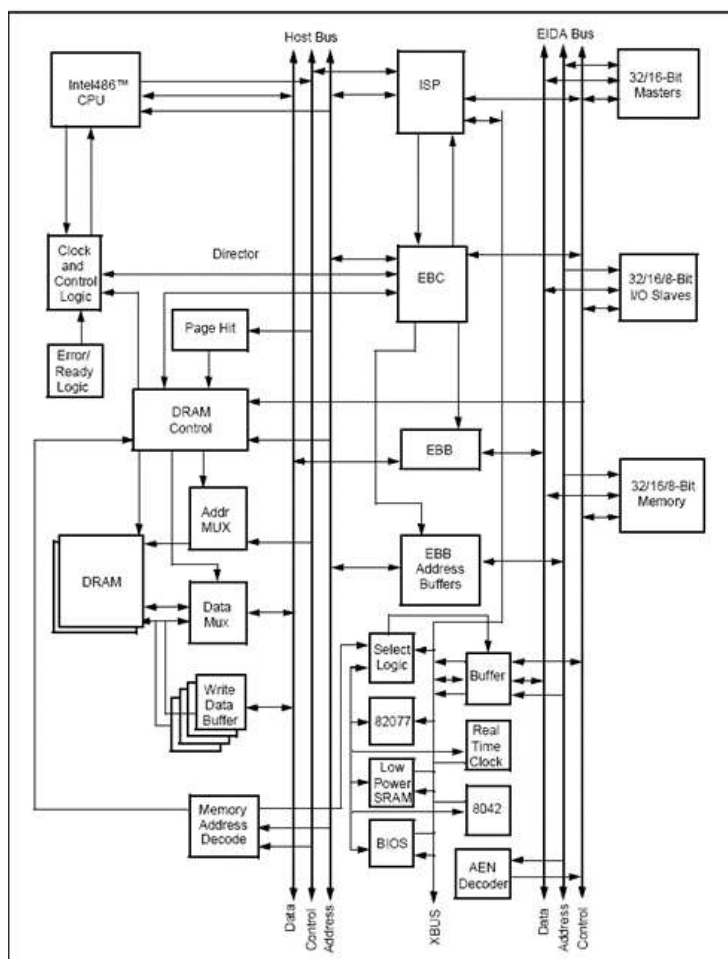


Рис.300 - Построение типовой конфигурации с использованием шины EISA

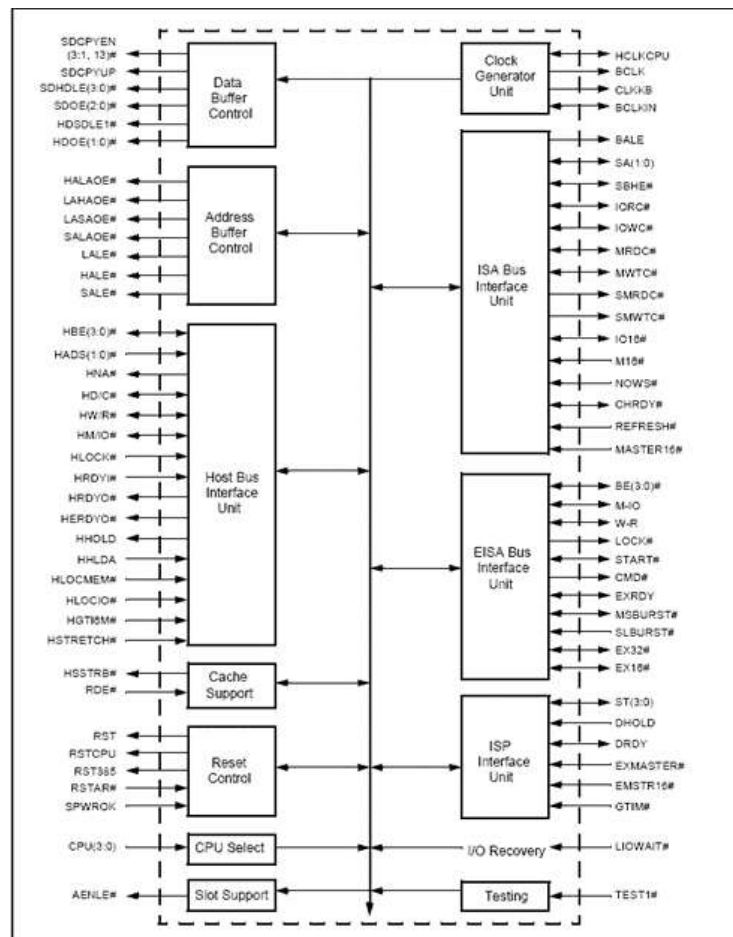


Рис.301 - Организация контроллера шины EISA

Стугбила MCA в соревновании с EISA скорее политика, чем техника – огромные лицензионные отчисления, которые IBM хотела получать за использование её микроканальной архитектуры, оттолкнули основную массу производителей клонов IBM PC. Хотя способствовала поражению MCA и техническая сторона – полное отсутствие совместимости с ISA. Понимая, что в реальной жизни обойтись в ПК только MCA-шиной не удастся, IBM во всех своих PS/2 машинах добавляла некоторое количество ISA-слотов. А машины, построенные на основе EISA, могли содержать только разъёмы EISA, в которые можно было вставлять в любой комбинации адаптеры трёх стандартов: ISA8/ISA16/EISA.

Ну, ко всему прочему, в «соревновании шин» неявно поприисутствовала и «рука» Intel. В стремлении освободить рынок для своих новых процессоров 80386 и 80486, Intel выпускала EISA-чипсеты, не поддерживающие 286 процессор, в то время, как шина MCA прекрасно работала и на компьютерах с 286-ми. Таким образом, перспективная разработка IBM так и осталась перспективной разработкой, но и шина EISA не стала хитом: к тому времени, когда потребности компьютеров среднего уровня окончательно переросли возможности шины ISA, разработчики перешли, минуя EISA, к локальной шине VLB (1992 год) и к замечательной интеловской PCI (1991 год).

В итоге, даже в начале 1990-х, спустя более пяти лет после выхода 386-го, более года с момента объявления 486-го и более двух лет с появления чипсетов с поддержкой EISA, эта шина так и не стала доминирующим мейнстримом в архитектуре PC-386/486. Одолевшая MCA, но так и не сумевшая полностью вытеснить, казалось бы, давно безнадёжно устаревшую медлительную ISA.

В итоге всё получилось довольно забавно – в битве двух 32-разрядных конкурентов – MCA против EISA – победил 16-разрядный, старенький и неказистый соперник. Для замещения которого собственно и были разработаны участники состязания ☺. В массовых ПК среднего класса образца начала 1990-х уже редко можно было встретить EISA, резко



упали продажи машин семейства PS/2 на основе MCA, а ISA по-прежнему оставался живее всех живых. Окончательного «свержения» этой шины пришлось ждать ещё несколько лет, когда на периферии воцарилась параллельная шина PCI, продержавшаяся на рынке более десятилетия.

### Локальная шина VESA Local Bus

Ещё одна попытка оснастить 32-разрядные процессоры адекватной периферийной шиной была предпринята группой разработчиков ассоциации VESA<sup>1</sup>. Их разработка **VESA local bus** (VL-Bus или VLB), по существу, являлась расширением внутренней локальной шины микропроцессора Intel 80486 (для 386-х шины VLB не было), обычно для связи с видеоадаптером и реже – с контроллером HDD. Реальная скорость передачи данных по VLB — 80 Мбайт/с (теоретически достижимая — 132 Мбайт/с).

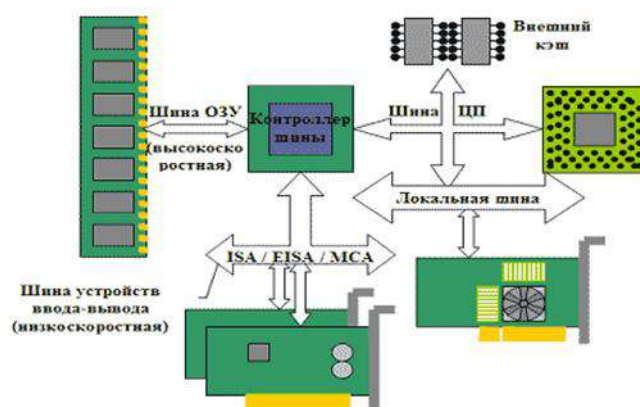


Рис.302 – Шины ПК на базе i80486

VLB была разработана в 1992г. Главной целью её разработки была дешёвая альтернатива шинам MicroChannel и EISA, пригодная для использования в массовых настольных компьютерах, и с этой ролью шина VLB довольно успешно справилась. Было выпущено большое количество плат контроллеров, использовавших эту шину, на основе выпущенных ранее микросхем, работавших до этого с шиной ISA. Даже при 16-битной архитектуре мог быть получен более чем 4-кратный выигрыш по тактовой частоте. С появлением шины PCI и процессоров Intel Pentium необходимость в её использовании исчезла, и на материнские платы для Pentium слоты VLB почти не ставили. Существовали материнские платы для процессоров 80486, Pentium и NexGen, имевшие сразу три типа слотов: VLB, ISA и PCI, которые назывались VIP-компьютерами, по первым символам названий шин.

Типичное сочетание слотов в VIP-компьютерах: 1 VESA – обычно под видеоадаптер, 2-3 слота PCI, 2-3 слота ISA.

Слот VLB был расширением шины ISA. Поэтому карты для шины ISA могли вставляться в слот VLB и нормально работать. Это делало разъем довольно длинным, и из-за этого аббревиатура VLB в шутку расшифровывалась как Very Long Bus («Очень Длинная Шина»). Дополнительная часть VLB разъёма была окрашена в светло-коричневый цвет (Рис.303), для неё использовался тот же самый 116-контактный разъём, что и для MicroChannel (контакты в нём размещались с меньшим шагом, чем в ISA). Физический разъём (слот, форм-фактор) шины PCI практически совпадает с дополнительной частью разъёма VLB, но расположен у заднего края системной платы и имеет другие назначения выводов.

<sup>1</sup> **Video Electronics Standards Association (VESA)** — ассоциация стандартизации видеоэлектроники, основанная в 1989 году компанией NEC Home Electronics и восемью другими производителями видеоадаптеров. Первоначальной задачей ставилось создание стандарта SVGA (800x600 пикселей) для видеодисплеев. После этого, VESA продолжила создавать стандарты, в основном относящиеся к функционированию видео периферии в IBM-совместимых компьютерах.



Рис.303 - Слоты VLB (красная стрелка) и ISA (синяя стрелка) на материнской плате

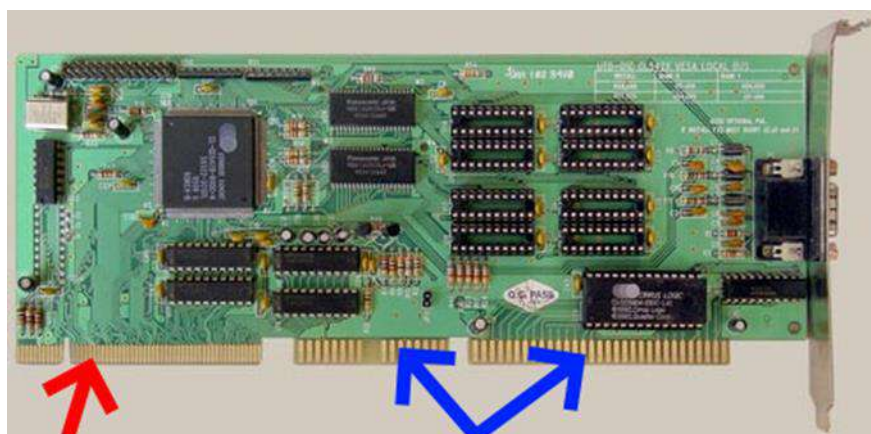


Рис.304 - VLB SVGA-карта: VLB-слот (красная стрелка), ISA-слот (синие стрелки)

VLB была расширением шины ISA только для процессоров Intel 80486 и использовала его технические особенности. По сути, на контакты дополнительного слота выходили физические линии системной шины (процессор-память). Таким образом, процессор мог напрямую обращаться к буферам и памяти контроллеров, работающих на VLB. Для процессора это выглядело как дополнительные модули обычной памяти (общее адресное пространство). Таким образом, процессор работал с устройством на тех же скоростях, что и с памятью, пересылая данные по 32-разрядной шине (в то время как ISA использовала фиксированную тактовую частоту ~8 МГц и 16-битную шину), что и обеспечивало высокое быстродействие.

В случае процессоров Pentium и NexGen функциональность шины VLB реализовывалась с помощью дополнительных мостов в чипсете, что приводило к катастрофическому падению производительности.

К основным недостаткам шины VLB следует отнести плохую совместимость и слабую нагрузочную способность VLB. Действительно, поскольку VLB работает на тактовой частоте процессора, контроллеры, подключаемые к шине VLB, должны были работать на этой частоте. На разных ПК частота процессора могла отличаться. Слабая нагрузочная способность VLB в основном была связана с тем, что данная шина фактически являлась продолжением контактов собственной шины процессора ПК. Каждое подключаемое устройство является дополнительной нагрузкой и искажает форму передаваемых процессором сигналов (их фронтов). Именно поэтому число подключаемых к локальной шине VLB



устройств ограничено: стандарт VLB предусматривает подключение к данной шине не более трёх устройств, обычно это одно или два устройства<sup>1</sup>, как правило, это был видеоконтроллер и контроллер жёстких дисков. Платы VLB, за редким исключением, не могли работать, будучи установленными в «чистый» слот ISA.

## **Шина PCI**

Альтернативный вариант локальной шины был разработан фирмой Intel – шина PCI (Peripheral Component Interconnect). Эта шина образует 32-битный канал между процессором и контроллерами периферийных устройств. Для PCI частота ограничена 33 МГц, что позволяет передавать данные по шине, как и в случае использования шины VESA, со скоростью до 132 Мбайт/с. Таким образом, шина PCI имеет практически такие же скоростные свойства, что и шина VLB при частоте процессора 33 МГц. Однако число контроллеров не ограничивается тремя, как для VLB, а может достигать десяти устройств. Стандарт шины PCI поддерживает спецификации VESA для BIOS и видеоадаптеров.

Стандарт на шину PCI определяет:

- физические параметры (например, разъёмы и разводку сигнальных линий);
- электрические параметры (например, напряжения);
- логическую модель (например, типы циклов шины, адресацию на шине).

Развитием стандарта PCI занимается организация PCI Special Interest Group.

Интерфейс широко применялся в бытовых компьютерах в период 1993 – 2005 годов. Затем он был вытеснен более новым стандартом PCI Express (последовательный), частично совместимым с PCI по программной модели, и, для ряда применений, различными вариантами шины USB (тоже последовательная).

Стандарт PCI предусматривает конфигурирование устройств, подключаемых к компьютеру, программным способом, что соответствует концепции plug-and-play (включил и работай). При этом в момент обнаружения нового устройства персональный компьютер без перезагрузки и выхода из текущего приложения должен установить параметры, необходимые для работы устройств в составе системы: номера прерываний IRQ, номера каналов прямого доступа DMA и т. д. Для реализации этой возможности необходимо, чтобы аппаратно-программное обеспечение ПК (BIOS и ОС) поддерживало plug-and-play.

Шина PCI архитектурно заметно сложнее, и не была так сильно привязана к определённому типу центрального процессора. Компания Intel выпустила ряд специализированных микросхем (чипсет) для шины PCI, сильно упростив тем самым её реализацию. Это позволило шине PCI полностью и довольно быстро, вытеснить шину VLB из архитектуры ПК.

Весной 1991 года компания Intel завершает разработку первой макетной версии шины PCI. Перед инженерами была поставлена задача разработать недорогое и производительное решение, которое позволило бы реализовать возможности процессоров 486, Pentium и Pentium Pro. Кроме того, было необходимо учесть ошибки, допущенные VESA при проектировании шины VLB (электрическая нагрузка не позволяла подключать более 3 плат расширения), а также реализовать автоконфигурирование устройств по примеру протокола Autoconfig для компьютеров Amiga. Были учтены и маркетинговые ошибки MCA, приведшие к противостоянию с EISA «банды девяти».

В 1992 году появляется первая версия шины PCI, Intel объявляет, что стандарт шины будет открытым, и создаёт PCI Special Interest Group. Благодаря этому любой заинтересо-

---

<sup>1</sup> В реальности три одновременно подключённых VESA-адаптера были крайне капризной и очень ненадёжно работающей комбинацией.

ванный разработчик получает возможность создавать устройства для шины PCI без необходимости приобретения лицензии. Первая версия шины имела тактовую частоту 33 МГц, могла быть 32- или 64-битной, а устройства могли работать с сигналами в 5 В или 3,3 В. Теоретически пропускная способность шины 133 Мбайт/с, однако в реальности пропускная способность составляла около 80 Мбайт/с.

В середине 1993 года компания Intel выходит из ассоциации VESA и начинает предпринимать активные шаги по продвижению шины PCI на рынке. Ответом на критику со стороны специалистов из конференций Usenet и конкурирующих компаний (характеристики шины были во многом аналогичны, например, Zorro III, публиковались статьи об ошибочном дизайне шины) стала PCI 2.0.

В 1995 году появляется версия PCI 2.1 (ещё одно название — «параллельная шина PCI»), которая обеспечила передачу данных по шине с частотой 66 МГц и максимальную скорость передачи в 533 Мбайт/с (для 64-битного варианта с частотой 66 МГц). Кроме того, эта шина уже была поддержана на уровне ОС Windows 95 (технология Plug and Play). Версия шины PCI 2.1 оказалась настолько популярной, что вскоре уже она была перенесена на платформы с процессорами Alpha, MIPS, PowerPC, SPARC и др.

В 1997 году, в связи с развитием компьютерной графики и разработкой шины AGP, шина PCI перестала удовлетворять новым, повышенным требованиям к видеокартам и перестала использоваться для установки видеокарт.

В конце 2000-х — начале 2010-х интерфейс PCI постепенно был вытеснен последовательными интерфейсами PCI Express и USB. На бытовых материнских платах уменьшалось количество разъемов PCI, их устанавливается не более 1-2, вместо 3-4 или более, применявшихся в начале 2000-х. На некоторых материнских платах (особенно, компактных форм-факторов mATX и т. п.) PCI-разъём не устанавливается вовсе.

При разработке стандарта PCI было принято решение размещать микросхемы на другой стороне печатной платы адаптеров, по сравнению с ISA. Это дало возможность размещать на материнской плате т.н. совмещённые разъёмы, когда в одну и ту же позицию можно было установить или PCI- или ISA-адаптер (не одновременно), тем самым экономя место на плате.

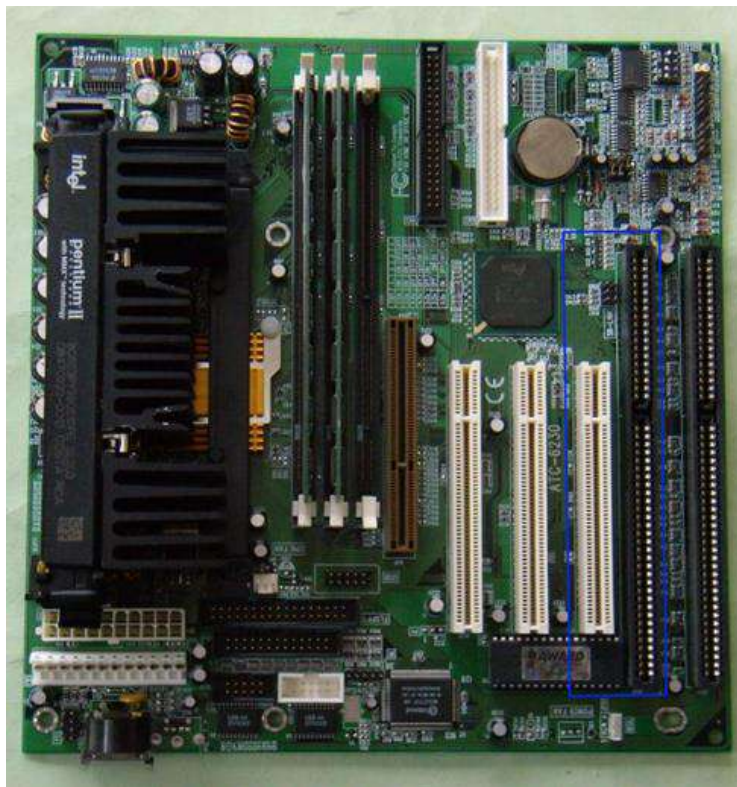


Рис.305 – Материнская плата с 3 PCI слотами и 2 слотами ISA. Синим прямоугольником выделен совмещённый слот (возможные сочетания: 3 PCI + 1 ISA или 2 PCI + 2 ISA).

## Архитектура

Первоначально в шине PCI было 32 проводника адреса/данных, работавших на частоте 33 МГц. Позже появились версии с 64 проводниками (используется дополнительная колодка разъёма) и частотой 66 МГц.

Шина децентрализована, нет главного устройства, любое устройство может стать инициатором транзакции. Для выбора инициатора используется арбитраж с отдельно реализованной логикой арбитра. Арбитраж «скрытый», не отбирает времени — выбор нового инициатора происходит во время транзакции, исполняемой предыдущим инициатором.

Транзакция состоит из 1 или 2 циклов адреса (2 цикла адреса используются для передачи 64-битных адресов, поддерживаются не всеми устройствами, дают поддержку DMA для памяти объёмом более 4 Гбайт) и одного или нескольких циклов данных. Транзакция со многими циклами данных называется «пакетной» (burst), понимается как чтение/запись подряд идущих адресов и даёт более высокую скорость — один цикл адреса на несколько, а не на каждый цикл данных, и отсутствие простоев (на переходные процессы в проводниках) между транзакциями.

Специальные типы транзакций используются для обращений к конфигурационному пространству устройства.

«Пакетная» транзакция может быть временно приостановлена обоими устройствами из-за отсутствия данных в буфере или из-за его переполнения.

Поддерживаются «расщеплённые» транзакции, когда целевое устройство отвечает состоянием «нахожусь в процессе» и инициатор должен освободить шину для других устройств, захватить её снова через арбитраж и повторить транзакцию. Это делается, пока целевое устройство не ответит «сделано». Используется для сопряжения шин с разными скоростями (например, PCI и Front Side Bus процессора) и для предотвращения тупиковых ситуаций в сценарии с несколькими межшинными мостами.

Была реализована богатая поддержка межшинных мостов и режимов кэширования, таких, как:

- posted write — данные записи немедленно принимаются мостом, и мост сразу отвечает «сделано», уже после этого пытаясь провести операцию записи на ведомой шине.
- write combining — несколько запросов на posted write, идущих подряд по адресам, соединяются в мосте в одну «взрывную» транзакцию на ведомой шине.
- prefetching — используется при транзакциях чтения, означает выборку сразу большого диапазона адресов одной «взрывной» транзакцией в кэш моста, дальнейшие обращения исполняются самим мостом без операций на ведомой шине.

Прерывания поддерживаются либо как Message Signaled Interrupts (новое), либо классическим способом с использованием проводников INTA-D#. Проводники прерываний работают независимо от всей остальной шины, возможно разделение одного проводника запроса прерывания несколькими устройствами.

## Конфигурирование

PCI-устройства с точки зрения пользователя работают как самонастраиваемые (Plug and Play). После старта компьютера системное программное обеспечение обследует конфигурационное пространство PCI каждого устройства, подключённого к шине, и распределяет ресурсы.

Каждое устройство может затребовать до шести диапазонов в адресном пространстве памяти PCI или в адресном пространстве ввода-вывода PCI.

Кроме того, устройства могут иметь ПЗУ, содержащее исполняемый код для процессоров x86 или PA-RISC, Open Firmware (системное ПО компьютеров на базе SPARC и PowerPC) или драйвер EFI.

Настройка прерываний осуществляется также системным программным обеспечением (в отличие от шины ISA, где настройка прерываний осуществлялась переключателями на карте). Запрос на прерывание на шине PCI передаётся с помощью изменения уровня сигнала на одной из линий IRQ, поэтому имеется возможность работы нескольких устройств с одной линией запроса прерывания; обычно системное ПО пытается выделить каждому устройству отдельное прерывание для увеличения производительности.

## Спецификация

- частота шины — 33.33 или 66.66 МГц, передача синхронная;
- разрядность шины — 32 или 64 бита, шина мультиплексированная (адрес и данные передаются по одним и тем же линиям);
- пиковая пропускная способность для 32-разрядного варианта, работающего на частоте 33.33 МГц — 133 Мбайт/с;
- адресное пространство памяти — 32 бита (4 байта);
- адресное пространство портов ввода-вывода — 32 бита (4 байта);
- конфигурационное адресное пространство (для одной функции) — 256 байт;
- напряжение — 3.3 или 5 В.

Стандарт PCI активно развивался и расширялся, росла тактовая частота, увеличилась разрядность шины. На рынке одновременно сосуществовало несколько версий PCI, не все они были совместимы между собой, чаще всего из-за различных напряжений питания, или имели ограниченную совместимость. Ниже приведена сводная таблица различных версий стандарта PCI (Рис.306), сводная таблица совместимости (Рис.307) и картинка с типами PCI слотов (Рис.308).

| Стандарт            | Макс. скорость, Мб/сек | Тип слота     | Тип карты                   |
|---------------------|------------------------|---------------|-----------------------------|
| PCI 1.x-2.0         | 133                    | 32-бита, 5В   | 32-бита, 5В                 |
| PCI 2.1-2.3 33МГц   | 133                    | 32-бита, 5В   | 32-бита, 5В/универсальный   |
| PCI 2.2-2.3 66МГц   | 266                    | 32-бита, 3.3В | 32-бита, 3.3В/универсальный |
| PCI64 33МГц (v 2.1) | 266                    | 64-бита, 5В   | 64-бита, 5В/универсальный   |
| PCI64 33МГц (v 2.2) | 266                    | 64-бита, 3.3В | 64-бита, 3.3В/универсальный |
| PCI64 66МГц         | 533                    | 64 бита, 3.3В | 64-бита, 3.3В/универсальный |
| PCI-X 1.0           | 1024                   | 64 бита, 3.3В | 64-бита, 3.3В/универсальный |
| PCI-X 1.0           | 4096                   | 64 бита, 3.3В | 64-бита, 3.3В               |

Рис.306 - Сводная таблица конструктивов карт и слотов в зависимости от версии стандарта

| Слоты              | Карты          |                        |                           |                                                     |                                                     |              |                                                     |                                                     |
|--------------------|----------------|------------------------|---------------------------|-----------------------------------------------------|-----------------------------------------------------|--------------|-----------------------------------------------------|-----------------------------------------------------|
|                    | PCI 2.0/2.1 5В | PCI 2.1 универсальный  | PCI 2.2/2.3 универсальный | PCI64 5В (33МГц)                                    | PCI64 универсальный                                 | PCI64 3.3В   | PCI-X/3.3В                                          | PCI-X универсальный                                 |
| PCI 2.0            | Совместимы     | Совместимы             | Несовместимы              | Ограниченно совместимы с потерей производительности | Ограниченно совместимы с потерей производительности | Несовместимы |                                                     |                                                     |
| PCI 2.1            | Совместимы     | Совместимы             | Ограниченно совместимы    | Ограниченно совместимы с потерей производительности | Ограниченно совместимы с потерей производительности | Несовместимы |                                                     |                                                     |
| PCI 2.2            | Совместимы     |                        |                           | Ограниченно совместимы с потерей производительности | Ограниченно совместимы с потерей производительности | Несовместимы | Несовместимы                                        | Ограниченно совместимы с потерей производительности |
| PCI 2.3            | Несовместимы   | Ограниченно совместимы | Совместимы                | Несовместимы                                        | Ограниченно совместимы с потерей производительности | Несовместимы | Несовместимы                                        | Ограниченно совместимы с потерей производительности |
| PCI64 64.5В(33МГц) | Совместимы     | Совместимы             | Ограниченно совместимы    | Совместимы                                          | Ограниченно совместимы с потерей производительности | Несовместимы | Несовместимы                                        | Ограниченно совместимы с потерей производительности |
| PCI64 3.3В         | Несовместимы   | Ограниченно совместимы | Совместимы                | Несовместимы                                        | Совместимы                                          | Совместимы   | Ограниченно совместимы с потерей производительности | Ограниченно совместимы с потерей производительности |
| PCI-X              | Несовместимы   | Ограниченно совместимы | Совместимы                | Несовместимы                                        | Совместимы                                          |              |                                                     |                                                     |

Рис.307 - Сводная таблица совместимости карт и слотов в зависимости от версии и конструктива

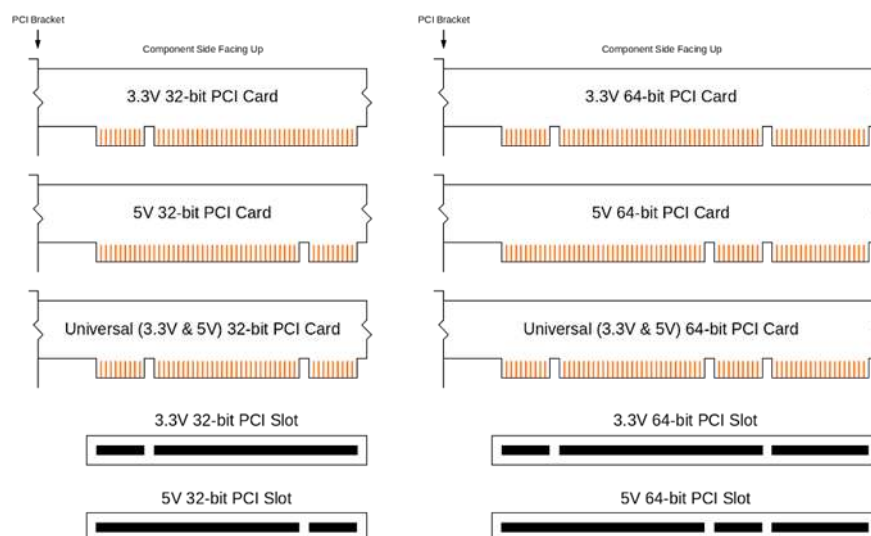


Рис.308 - Типы PCI-слотов

Описание версий PCI новее исходного канонического варианта 32 бита/33 МГц будет приведено дальше, в разделах, посвящённых архитектурам ПК на Pentium.

## Программное обеспечение

Появление на рынке 32-разрядных x86 процессоров в лице сначала i80386, а чуть позже – i80486, открыло возможности для разработки надёжных производительных операционных систем и прикладного ПО под них. Исправив идеологические ошибки вроде невозможности перехода процессора i80286 из защищённой моды в реальную, расширив адресуемое физическое адресное пространство, значительно улучшив аппаратную поддержку мультизадачных систем, включая реализацию режима виртуального v86, Intel предоставила мощную аппаратную 32-разрядную платформу разработчикам software и ОС. Но, как и в случае с i80286, системное программное обеспечение, утилизирующее все возможности новых процессоров, сильно запаздывало. Даже аппаратная возможность, которой очень не хватало 286-м процессорам – при соответствующей поддержке ОС – запускать одновременно несколько виртуальных копий MS-DOS, оказалась реально воплощённой в операционные системы многие годы спустя после появления на рынке первых интеловских 32-разрядных процессоров. 32-разрядная OS/2 v2.0, продвигавшаяся IBM под слоганом «лучшая DOS, чем MS-DOS, и лучшая Windows, чем Windows», была объявлена весной 1992 года, а первая версия Windows NT, получившая номер версии сразу 3.1 (чистый маркетинг), появилась годом спустя, в 1993 году. Ещё раз напомним даты выхода интеловских 32-разрядных процессоров: октябрь 1985 года – i80386, апрель 1989-го – i80486.

Альтернативные ОС, для работы которых нужен был процессор не ниже 80836 – Novell Netware 386 – 1989 год, но это не пользовательская система, а узко специализированный файл-сервер. Unix-подобная ОС Xenix-386 от Microsoft – примерно 1988 год, но эта система никогда не продавалась конечным пользователям и не конкурировала с пользовательскими MS-DOS, Windows, OS/2. Линус Торвалдс в 1991 году только собирался создать совместимое с UNIX ядро операционной системы для архитектуры Intel 80386. Взлёт либука был ещё впереди.

В результате 32-разрядные 386/486 на начало 1990-х всё ещё не имели массовой пользовательской операционной системы, соответствующей их мощной 32-разрядной архитектуре.

## Немного теории заговора

В Интернете можно наткнуться на мнение, что Билл Гейтс, не будучи приглашённым на презентацию первого IBM PC 5150 (это факт), затаил обиду на «небожителей» из IBM и сумел потом, менее чем через десять лет, отомстить обидчикам. Его MS-DOS, а потом и



Windows, имели тотальное превосходство на пользовательском рынке x86 компьютеров, хотя по объективным характеристикам OS/2 от IBM (которая вначале разрабатывалась ими совместно с Microsoft) превосходила ОС от Microsoft. Примерно в начале 1990-х IBM потеряла лидирующие и определяющие позиции на рынке PC, который она же и породила, фактически признав своё поражение как на рынке «железа» (на это сильно повлияла относительная неудача с семейством PS/2), так и на рынке операционных систем для архитектуры x86. Несколько лет спустя некогда всемогущая IBM полностью ушла с рынка компьютеров архитектуры x86 – сначала с десктопного направления, потом – с мобильного (ноутбучное направление было продано китайской Lenovo), и, наконец, с серверного, продав его всё той же Lenovo. И уже очень давно никто не называет x86 ПК как IBM PC – вначале, после ухода IBM, их стало принято называть Wintel (WINdows INTEL, отмечая ведущую роль в архитектуре разработчика процессора и ОС Windows), сейчас чаще называют нейтрально – x86, отмечая и имеющиеся великолепные альтернативы для интеловских процессоров, и получившие широчайшее распространение операционные системы, совершенно не похожие на Windows, в первую очередь – Linux.

## История «развода» IBM и Microsoft

К 1990-му году IBM и Microsoft продолжали совместную работу над ОС OS/2, но Microsoft, не сильно афишируясь, параллельно вела работу ещё над двумя проектами. Один из них в итоге появился на рынке в виде 16-разрядной Windows 3.0. Второй проект, с рабочим названием “NT OS/2”, разработка которого началась в ноябре 1988 года командой Дэйва Катлера (работавшего ранее над ОС VAX/VMS, и незадолго до этого перешедшего из DEC в Microsoft), в результате превратился в ОС Windows NT<sup>1</sup>.

Выпущенная 22 мая 1990 года Windows 3.0 была 16-разрядной графической надстройкой над MS-DOS, которая работала фактически загрузчиком Windows. Windows 3.0 имела значительно обновлённый пользовательский интерфейс, а также технические усовершенствования, позволявшие использовать возможности процессоров Intel 80286 и 80386 по управлению памятью. «Окна» не были самостоятельной операционной системой, в частности, файловая система в них была файловой системой FAT-16 «нижележащей» MS-DOS.

Текстовые программы, созданные для MS-DOS, могли быть запущены в окне (эта возможность была ранее доступна в более ограниченном виде в Windows/386 2.1). Это делало систему пригодной к использованию в качестве простой многозадачной базы для старых программ; однако, для домашних компьютеров эта функция не имела большого значения, так как большинство игр и развлекательных приложений по-прежнему требовали непосредственного доступа к DOS.

Написана Windows 3.0 (и вообще все версии Windows 3.x, вплоть до последней версии третьей линейки «Windows для рабочих групп 3.11») с использованием т.н. «невывесняющей» многозадачности<sup>2</sup>, работала довольно неустойчиво, часто «зависала». Любопытные цифры: дистрибутив Windows 3.0 занимал всего пять 5.25" дискет ёмкостью 1.2 Мбайт.

Несмотря на все свои недостатки, на не очень надёжную работу, всего лишь 16-разрядный режим работы, Windows 3.0 была пользовалась большим успехом, пожалуй, ставший неожиданным даже для самой Microsoft<sup>3</sup>. IBM очень ревниво отнеслась к удаче ОС, которая фактически «втихую» разрабатывалась их партнёром.

---

<sup>1</sup> Интересно, что NT разных версий работала на семи(!) разных аппаратных платформах, и первая рабочая реализация планировалась не под x86, а под интеловский RISC-процессор i860.

<sup>2</sup> Невывесняющая многозадачность (Non-preemptive multitasking) – это способ планирования процессов, при котором активный процесс выполняется до тех пор, пока он сам, по собственной инициативе, не отдаст управление планировщику операционной системы для того, чтобы тот выбрал из очереди другой, готовый к выполнению процесс.

<sup>3</sup> Windows 3.x продемонстрировала удивительное для столь несовершенной системы долголетие: Microsoft прекратила выдачу лицензий на её использование только 1 ноября 2008 года.

Кроме того, Microsoft решила добавить во второй проект, NT OS/2, программный интерфейс (API), совместимый с вызовами Windows API. Это решение вызвало серьёзные трения между компаниями-партнёрами, которые закончились разрывом совместной работы. IBM продолжила разработку OS/2, двигаясь в сторону ранее объявленной версии 3 самостоятельно, а Microsoft стала работать над системой, которая была в результате выпущена под названием Windows NT. Хотя эта система не сразу стала популярной, как это было с DOS, Windows 3.x или Windows 9x, с точки зрения маркетинга Windows NT оказалась существенно более удачной, чем OS/2.

В качестве программных интерфейсов ОС NT изначально планировались API OS/2 и затем POSIX, поддержка Windows API была добавлена в последнюю очередь. Кроме того, в качестве аппаратной платформы для NT изначально планировались Intel i860 и затем MIPS, поддержка Intel x86 также была добавлена позднее. Затем в процессе эволюции этой ОС исчезла поддержка обоих изначально запланированных программных интерфейсов и обеих изначально запланированных аппаратных платформ. Для i860 не было ни одной стабильной версии этой ОС, хотя именно от кодового названия этого процессора, *N10* (N Ten), происходит название самой ОС NT. Ныне Microsoft расшифровывает аббревиатуру NT как *New Technology*<sup>1</sup>. А в качестве альтернативы POSIX-подсистеме Microsoft стала предлагать пакет Сервисы Microsoft Windows для UNIX. В обновлении «Anniversary Update» для Windows 10 за июль 2016 года предложена бета-версия подсистемы Windows Subsystem for Linux для выполнения бинарных исполняемых файлов для ядра Linux без применения виртуализации. В качестве базовой системы предлагается образ Ubuntu, в версии за 11 апреля 2017 добавлены также образы SUSE и Fedora.

Переносимость NT на разные платформы была одной из её первоочередных задач. Именно поэтому разработка этой ОС изначально велась для процессора i860, хотя двоичная совместимость с OS/2, бывшая одним из условий проекта NT OS/2, в любом случае требовала бы создания версии NT для x86 либо включения в неё эмуляции этой платформы. Количество платформ, для которых существовали релизные версии ОС семейства Windows NT, впечатляет: кроме названных MIPS и Intel x86, сюда входят PowerPC, DEC Alpha, Itanium и AMD x86-64. Независимыми производителями компьютерных систем были также разработаны версии Windows NT для архитектур Clipper и SPARC; однако эти версии не были выпущены как самостоятельные программные продукты. В качестве ОС высокой переносимости при разработке NT были взяты за пример ОС Unix и Mach.

Как уже упоминалось выше, для разработки ОС NT фирма Microsoft пригласила группу специалистов из компании DEC во главе с Дэвидом Катлером, обладающую опытом создания многозадачных операционных систем, таких как VAX/VMS и RSX-11. Некоторое сходство, отмеченное между внутренними архитектурами Windows NT и ОС семейства VMS, дало основания обвинить вновь принятых сотрудников Microsoft в краже интеллектуальной собственности DEC. Возникший конфликт был разрешён мирным путём: DEC признала собственность Microsoft на технологии, лежащие в основе Windows NT, а Microsoft создавала и поддерживала версию Windows NT для архитектуры DEC Alpha.

Несмотря на общие корни, совместимость Windows NT и OS/2 уменьшалась с каждым новым выпуском этой ОС. Поддержка API OS/2 2.0, хотя планировалась в NT, так и не была завершена; в Windows NT 4.0 была удалена поддержка файловой системы HPFS, а в Windows XP была удалена подсистема поддержки программ для OS/2 1.x.

В 1995 году состоялся окончательный «развод» бывших партнёров, месть Билла Гейтса вошла в завершающую стадию – успешный старт продаж Windows 95 в очередной раз заметно потеснил OS/2 на рынке ПК. Microsoft брала с производителей PC отчисления за каждую предустановленную копию Windows 95, в размере 37\$ за лицензию. IBM, пытаясь хоть как-то спасти сжимающийся на глазах рынок OS/2, предложила Microsoft, как было

---

<sup>1</sup> Есть альтернативное шутливое объяснение появления аббревиатуры **Windows NT**: намёк на команду разработчиков под руководством Дэйва Катлера, работавших в DEC-е над ОС VAX/VMS – если к каждому из ASCII-символов аббревиатуры **VMS** добавить единицу, получится **WNT** ☺.

когда-то давно, в истории с PC-DOS и CP/M, устанавливать на их ПК две ОС на выбор пользователя: Windows 95 и их OS/2. Microsoft согласилась, но, как старому партнёру, предложила IBM, производителю ПК, платить отчисления за каждую копию Windows 95... на восемь долларов больше, чем платил любой другой производитель ПК, т.е. 45\$. Такого оскорбления IBM не перенесла, и окончательно разорвала все отношения с бывшим партнёром.

### ***Microsoft раскрыла исторические аспекты разработки Windows***

На самом деле, кроме шутильной трансформации VMS → WNT, между VAX/VMS и Windows NT действительно существовала реальная связь. В апреле 2016 года Microsoft опубликовала очень любопытную информацию. Опубликованная статья называлась *“The engineer’s engineer: Computer industry luminaries salute Dave Cutler’s five-decade-long quest for quality”* и посвящена заслуженному инженеру Microsoft и первому архитектору Windows NT Дэйву Катлеру, который заложил основу всех версий Windows, начиная с 1993 года.

Известно, что Windows NT разрабатывалась как серверная операционная система и она прошла достаточно долгий путь эволюции, перед тем как предстать во всей своей красе перед домашними пользователями в виде Windows 2000 и Windows XP. Не многие знают, что, на самом деле, всё семейство ОС Windows NT, начиная от самых первых ее версий и заканчивая Windows 10, основано на одной из первых полностью 32-битных ОС фирмы Digital Equipment Corporation под названием VAX/VMS ([Microsoft раскрыла исторические аспекты разработки Windows](#)).

Ниже приведены наиболее значительные отрывки из вышеуказанной статьи с комментариями.

В 1976 г. компания Digital Equipment Corporation (DEC) (или просто Digital) была на волне успеха и считалась преуспевающей компьютерной фирмой. [Digital не только специализировалась на разработке своей ОС, но выпускала большое количество «железа», компьютеров]. Совместно с другими компаниями, такими как Data General, Prime Computer и прочими IT-компаниями, которые располагались вдоль т. н. шоссе 128 (Massachusetts Route 128), DEC формировала компьютерный ландшафт того времени, выпуская очень популярные в то время миникомпьютеры.

В то же время уже известный на тот момент инженер Гордон Белл (Gordon Bell), который работал в Digital на довольно высокой должности, был обеспокоен падением динамики продаж флагманского 16-битного миникомпьютера компании, PDP-11, работавшего под ОС RSX-11. Другое беспокойство менеджеров DEC вызывал факт разработки IBM первого [микро-]компьютера.

Белл решил использовать свое видение будущего, поставив на создание нового 32-битного миникомпьютера под названием VAX с новой ОС [VMS], которая будет обеспечивать обратную совместимость с приложениями для миникомпьютера PDP-11. Тогда Белл обратился к Дэвиду Катлеру, которому, на тот момент, было 33 года. Он считался перспективным инженером Digital, к тому же, имевшим опыт разработки RSX-11. Белл попросил Катлера возглавить проект разработки VAX [Virtual Address Extension].



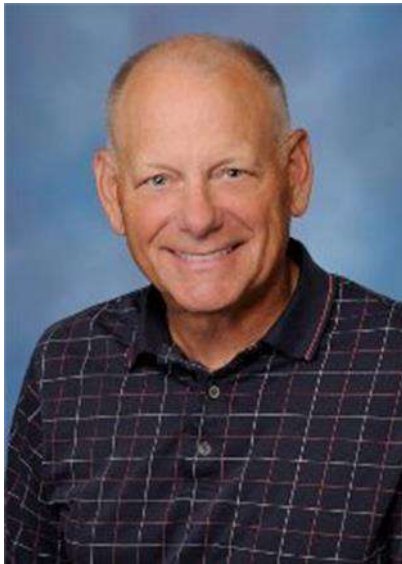


Рис.309 – Дэйв Катлер

Два года спустя, первые экземпляры миникомпьютеров VAX (суперминиЭВМ) увидели свет на производстве Digital. При этом VAX сразу же закрепил свою позицию лидера на этом рынке. Тогда Белл назвал Катлера *«лучшим разработчиком операционных систем в мире»*.

В 1982 г. Катлер встретился с Беллом и анонсировал свой уход из DEC, что стало для Белла полной неожиданностью. В свою очередь, руководство DEC не собиралось просто так расставаться со своим самым талантливым инженером, хотя, подозревало, что его недовольство относится к растущей бюрократии в руководстве Digital. Катлеру сделали встречное предложение, от которого он не смог отказаться. Так была создана дочерняя компания DECWest.

Офис новой компании расположился на расстоянии 3 тыс. миль от штаб-квартиры Digital: в Бельвью, штат Вашингтон, т. е. недалеко от Сиэтла и Microsoft. Молодой инженер Len Kawell стал одним из первых талантливых специалистов этой фирмы. Специалисты собрались разработать ОС реального времени (real-time) для VAX, но в результате был создан и новый компьютер MicroVAX, а также real-time ОС для него под названием VAXeln. Катлер взял на себя функции разработки нового железа, а Len Kawell отвечал за создание ПО.

Kawell говорил, что у Катлера есть уникальное видение функций железа (hardware), что отличало его от других инженеров того времени. Он мыслил на уровне машинных инструкций и операций с регистрами микропроцессора. Тем не менее, в 1988 г. Катлер всё-таки покинул Digital, а новый компьютер MicroVAX и ОС для него так и не увидели свет. После этого Катлер ушел в Microsoft, познакомившись с Биллом Гейтсом ещё за пять лет до этого. Для него уже был готов проект новой ОС, которой суждено было произвести настоящий переворот в области микрокомпьютеров.

После переговоров с Биллом Гейтсом в 1988 году, Катлер принял решение о переходе в Microsoft для создания настоящей 32-битной ОС для микрокомпьютеров. К нему также присоединилась команда инженеров из предыдущего проекта: пять специалистов по разработке ОС и шесть специалистов в hardware.

Команда Катлера приступила к разработке новой ОС для Microsoft в апреле 1989-го, работа велась более десяти часов в сутки, иногда без выходных. Основной задачей при разработке Windows NT стала портируемость будущей ОС, т. е. возможность ее работы не только на микропроцессорах Intel архитектуры x86, но также и на RISC-архитектурах, которые были популярны в то время и обеспечивали большую производительность по сравнению с x86. Одновременно с этим, специалисты Microsoft уже сотрудничали с компанией IBM в новом проекте по разработке ОС под названием OS/2. Но у OS/2 были явные недостатки, во-первых она была 16-битной, а во вторых не была портируемой и могла работать только на микропроцессорах фирмы Intel.

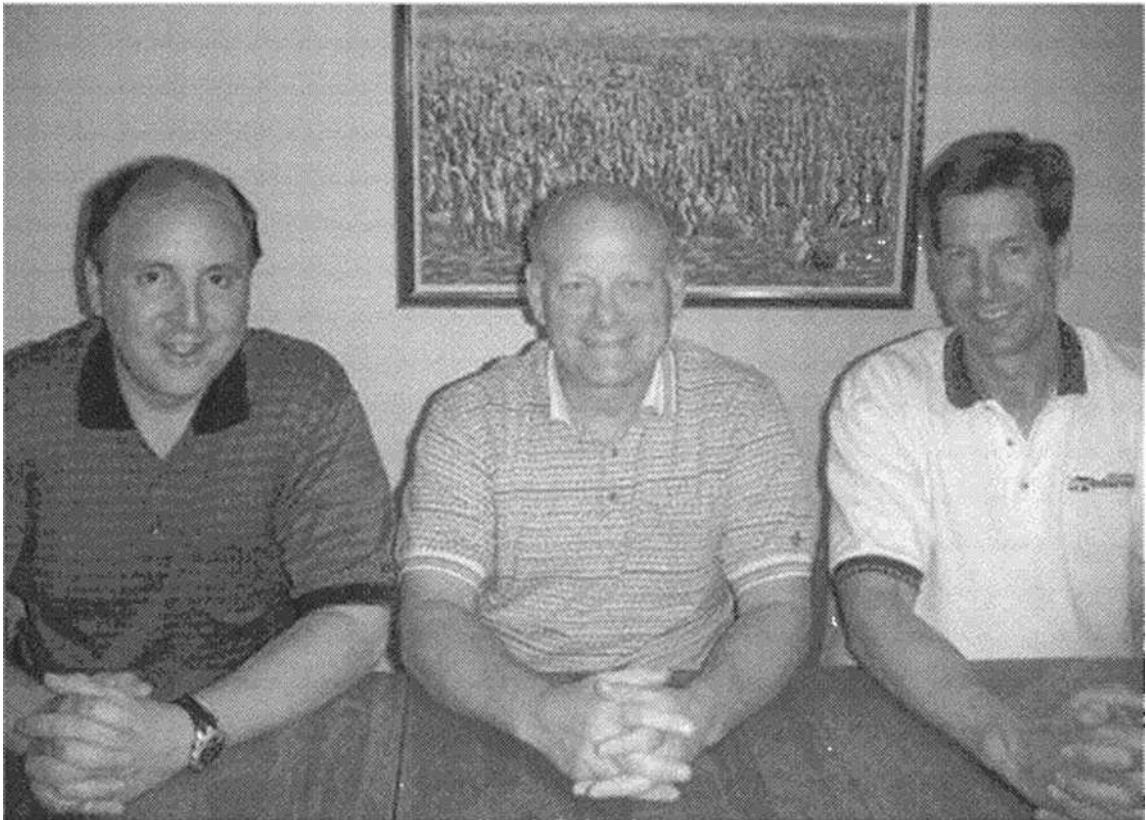


Рис.310 - Катлер с Соломоном (слева) и Марком Руссиновичем (справа), авторами известной книги «Windows Internals».

Цели разработки Windows NT были сложными и амбициозными. Команда Катлера должна была обеспечить выполнение следующих требований:

- Поддержка различных микропроцессорных архитектур [на уровне исходных текстов], включая, MIPS, Alpha, PowerPC, и x64.
- Поддержка запуска приложений других ОС [модель подсистем], включая, UNIX POSIX, OS/2 и Win32.
- ОС должна быть безопасной и соответствовать стандарту безопасности C2 (trusted computer certification).
- Поддержка нескольких одновременно работающих микропроцессоров (мультипроцессорная архитектура) и подлинная многозадачность. Обе эти функции считались уникальными на тот момент для сегмента ПК.

Одной из самых сложных задач, которую предстояло решить разработчикам, была задача тестирования разрабатываемой ОС. На раннем этапе команда не обладала всеми необходимыми ресурсами для разработки нужного набора тестов. Вместо этого, они сделали выбор в пользу динамической системы проверки стрессоустойчивости, что создавало высокую нагрузку на систему в целом. Каждую ночь разработчики запускали стресс-тесты новой ОС на сотнях машин. Утром авторы Windows приезжали в офис и производили сортировку ошибок для последующего их анализа.

Интересно отметить, что первоначально в качестве основной подсистемы Windows рассматривалась не Win32, а OS/2, но позже Microsoft отказалась от этой идеи. Отчасти это было связано и с быстро набравшей популярность Windows 3.x, продажи которой составили 16 миллионов копий за полгода. В результате 16-битное Windows API было взято за основу и расширено до 32-х бит. Для новой ОС также была написана своя 32-разрядная графическая подсистема. Позже разработчики были вынуждены адаптировать Windows NT на RISC

микропроцессор MIPS R3000 вместо заявленного изначально Intel i860 XR. Еще одной проблемой, которую необходимо решить, была совместимость новой ОС с устаревшими приложениями MS-DOS и 16-битной Windows.

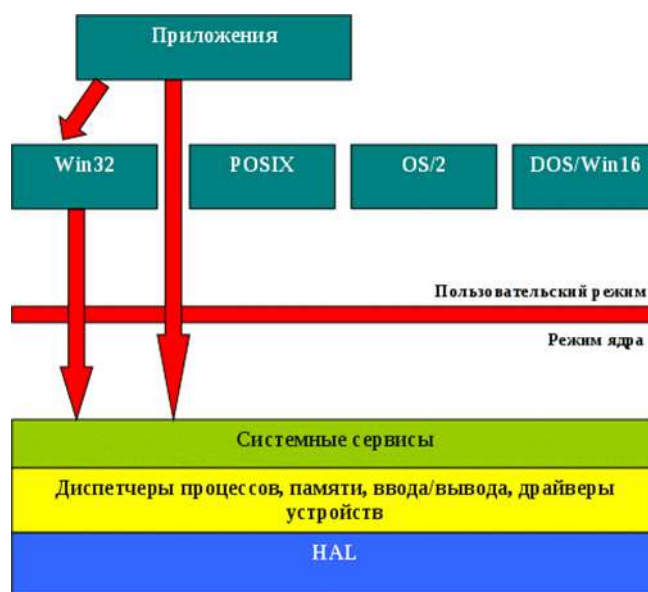


Рис.311 - Архитектура Windows NT

Windows NT 3.1<sup>1</sup> увидела свет 27 июля 1993 года. В 1996 Катлер оставил руководство всем проектом Windows NT и сконцентрировался на руководстве команды разработки ядра до 2006 года. В марте 2005 года Катлер закончил часть работы по портированию Windows NT на платформу AMD64, когда были выпущены первые 64-битные версии для рабочих станций и серверов.

## Архитектура компьютеров на базе Intel Pentium поколений P5...P6/M...NetBurst

Начав всего с двух разных моделей Pentium пятого поколения, Intel в дальнейшем резко увеличила количество типов, линеек и всевозможных модификаций выпускаемых ею процессоров, в течении нескольких лет даже производя одновременно процессоры двух разных поколений – с ядром P6 и на основе архитектуры NetBurst (которую автор в тексте иногда обозначает не самым распространённым вариантом названия – поколение P7, просто для сокращения длины).

В виду просто невероятного разнообразия интеловских (и не только) процессоров и чипсетов, на которых строились компьютеры «стандартной архитектуры», для сокращения объёмов материала в последующем тексте приведены описания только для ключевых (с точки зрения автора) процессоров, появление которых приводило к существенным изменениям в архитектуре всего компьютера.

Например, следуя этой логике, опущены описания процессоров с микроархитектурой NetBurst новее Prescott-2M (в тексте только упоминается, что Pentium 4F, начиная со степпинга D0, стали поддерживать 64-битные расширения EM64T), таких как Pentium D, Pentium Extreme Edition, Pentium M и др.

<sup>1</sup> Присвоив первой версии Windows NT сразу номер 3.1, Microsoft пыталась убедить пользователей, что это зрелая и надёжная система, имеющая солидную историю. Действительно, чисто психологически, к ранним версиям программного обеспечения пользователи обычно относятся настороженно, обоснованно полагая, что первые версии очень «сырые» и содержат массу ошибок и недоработок.

Автор считает, что действительно серьёзные изменения в архитектуре интеловских микропроцессоров после Pentium 4, повлекшие за собой значительные изменения в архитектуре компьютеров, произошли начиная с поколения Core<sup>1</sup>. Даже появление чуть раньше 64-битного расширения EMТ64 в процессорах не повлекло за собой значительных качественных изменений в архитектуре компьютеров.

Поддержка двухпроцессорного режима, появившаяся, например, в том же Pentium D, мало отличалась в смысле организации компьютера от двухсокетного варианта материнской платы на Pentium II/III – Pentium D фактически был построен на двух отдельных кристаллах (каждый условно Pentium 4), упакованных в один корпус, т.е., он ещё не был «настоящим» двухъядерным процессором (в доказательство этого утверждения – для сохранения когерентности кэш-ядер требовалось пересылать синхронизируемые данные через фронтальную шину FSB, точно также, как это делалось в двухсокетных вариантах с обычными одноядерными процессорами).

*Интересно, что эти «предреволюционные» процессоры и их «революционные» последователи использовали один и тот же тип процессорного разъёма LGA775. Всего под этот очень популярный сокет-долгожитель выпускалось несколько десятков моделей процессоров, основные семейства кратко перечислены ниже:*

- Intel Xeon — от 2 до 4 ядер
- Core 2 Quad — 4 ядра
- Core 2 Duo — 2 ядра
- Pentium Dual-Core — 2 ядра
- Pentium — от 1 до 2 ядер
- Celeron — от 1 до 2 ядер

LGA775 представлял собой разъём с подпружиненными или мягкими контактами, к которым с помощью специального держателя с захватом и рычага прижимается процессор, не имеющий штырьковых контактов. Такое подключение «наоборот», когда штыревые контакты переместились с процессора на плату, в гнездо, а на нижней стороне процессора были размещены куполообразные позолоченные контактные площадки, выглядело довольно необычно, и было поначалу воспринято пользователями довольно настороженно.

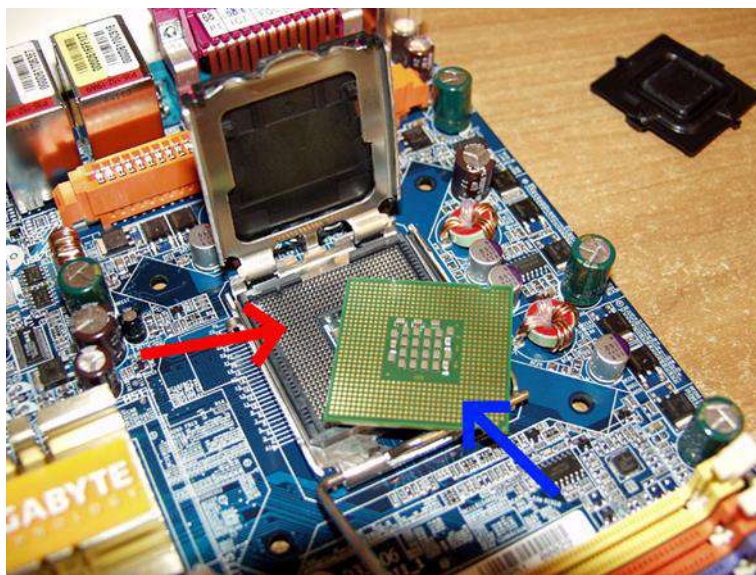


Рис.312 – Сокет LGA775 («игольчатое ложе», красная стрелка); процессор, повернутый контактными площадками вверх (синяя стрелка).

<sup>1</sup> Intel не вполне согласна с этой точкой зрения, и начала отсчёт новых поколений с микроархитектуры Nehalem (назвав его «первым поколением»).



С появлением Socket LGA775, Intel впервые отказалась от соответствия маркировки тактовой частоте процессоров, оставив лишь производственный номер. С одной стороны, разобраться в многообразии процессоров стало немного сложнее, с другой же такая маркировка привнесла свои удобства.

Ключевым моментом в истории развития компьютерных архитектур на базе интеловских процессоров, с точки зрения автора, стал практически *полный* переход от параллельных периферийных шин/интерфейсов к последовательной технологии передачи данных, фактически к пакетной коммутации. Раньше всего этот переход произошёл с появлением USB, что, собственно, и отражает название стандарта – Universal Serial Bus. Переход от параллельных ATA/IDE к SATA, SCSI к SAS, и, самое существенное – переход от бывшей до этого основной периферийной (параллельной) шиной PCI к последовательной PCI Express, и всё это в сочетании с выпуском истинно многоядерных процессоров – вот точка отсчёта, начиная с которой можно говорить о новой архитектуре *компьютера*.

В этом разделе будут приведены описания архитектур, начиная с компьютеров на базе процессоров поколения P5, и заканчивая процессорами где-то в последней четверти жёлтой и розовой части Рис.313, до зелёной части с Core. Согласно приведённой в [Список микропроцессоров Intel](#) классификации, в рассмотрение будут включены процессоры до п.11 включительно (кроме последних моделей, с п.11.8 и далее), п.12.1.

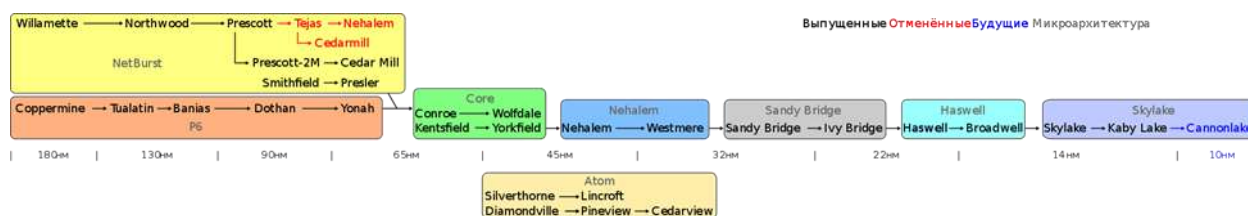


Рис.313 - Хронология процессорных микроархитектур Intel от NetBurst и Pentium M до Skylake

Следующий раздел начнётся с «зелёного» п.15 и продолжится до п.23, с полным исключением п.16 (Atom). Все разделы с Intel Core будут по возможности максимально сокращены, так как принципиальных *качественных* архитектурных различий между разными линейками процессоров семейств Core нет, в основном менялись количественные характеристики процессоров – рабочие частоты, количество ядер, объём кэша, количество каналов памяти, количество поддерживаемых шин и интерфейсов и т.п.

## PC на основе процессоров Pentium поколений P5...P7

В марте 1993 года Intel представляет на рынке свой процессор пятого поколения. Чуть раньше, в октябре 1992 года Intel объявила, что грядущие процессоры пятого поколения, ранее носившие кодовое имя P5, будут называться *Pentium*, а не 586, как предполагали многие. Это было вызвано тем, что многие фирмы, производящие процессоры, активно освоили производство «клонов» (и не только) процессоров i386 и i486. Intel собиралась зарегистрировать в качестве торговой марки название «586», чтобы больше никто не смог заниматься производством процессоров с таким названием, однако оказалось, что регистрировать цифры в качестве торговой марки нельзя, поэтому было принято решение назвать новые процессоры «Pentium» (за основу было взято др.-греч. πέντε «пять»<sup>1</sup>), что также указывало на поколение данного процессора. 22 марта 1993 года состоялась презентация нового микропроцессора, через несколько месяцев появились и первые компьютеры на основе Pentium.

Интересно, что в этом процессоре впервые в истории Intel тактовая частота была ниже, чем у процессоров предыдущего поколения – 60 или 66 МГц. Также это были первые и последние Pentium-ы, имевшие напряжение питания 5 вольт.

<sup>1</sup> Есть забавная конспирологическая версия, что Pentium был назван так по фамилии одного из со-разработчиков процессора – Владимира Пентковского.

В отличие от ситуации с появлением первых 32-разрядных интеловских процессоров 386/486, индустрия ПК была гораздо лучше готова к поколению процессоров Pentium: получил развитие чипсет от Intel 400-ой серии, обеспечивавший поддержку всей линейки процессоров адекватным 32-разрядному процессору набором шин и интерфейсов, включая память различных типов (FPM, EDO, SDRAM), шины ISA, EISA, PCI, чуть позже – AGP и USB, а на серверных платформах и на мощных рабочих станциях – шину SCSI<sup>1</sup>. Существовали ревизии 400-го чипсета, поддерживавшие работу в двух-, четырёх- и даже восьмипроцессорных<sup>2</sup> конфигурациях (эпоха настоящих многоядерных процессоров началась с поколения Core 2, после не самого удачного Pentium 4).

Намного лучше обстояли дела и с программным обеспечением, на рынке присутствовало несколько уже вполне зрелых 32-разрядных пользовательских ОС: OS/2, Windows NT, стремительно набиравший обороты Linux и др.

Необходимо отметить, что Intel, кроме мейнстрима в виде Pentium, стала выпускать также облегчённые удешевленные версии процессоров под названием Celeron, и «тяжёлые», дорогие серверные процессоры Xeon<sup>3</sup>, а также мобильные экономичные Pentium-M.

Стратегически Pentium – по разрядности, объёму доступного ОЗУ и пр. – отличался от предшественника – процессора i80486, гораздо меньше, чем в своё время 386-ой отличался от 286-го, и поэтому рынок быстро отреагировал появлением ПК на базе 586/686-го, отработав до этого необходимые технологии на первых 32-разрядных 386/486-ых. Материнские платы с «обвязкой» в виде стандартных (спасибо, Intel) чипсетов выпускались десятками производителей, среди которых всё большую роль играли китайские (тайваньские) производители: ASUS, Gigabyte, MSI и др. Со временем практически всё производство PC переместится в юго-восточную Азию.

Ниже приведена фотография (Рис.314) материнской платы Socket 7 ASUS TXP4, типичного представителя класса десктоп, со стандартным для своего времени набором интерфейсов/шин, построенной на базе процессорного разъёма Socket 7 и интеловского чипсета i430TX, рассчитанных на процессоры Pentium/Pentium MMX (P5). В набор интерфейсов этой платы входили:

- Три разъёма шины ISA<sup>4</sup>;
- Четыре разъёма PCI (один совмещён с ISA);
- Два слота подключения памяти SDRAMM, четыре – SIMM;
- Два разъёма IDE (параллельный 16-разрядный интерфейс для подключения жёстких дисков, до двух на каждый разъём);
- Разъём для подключения флоппи-дисков (FDD);
- Набор разъёмов для подключения периферии – USB, два COM-порта, параллельный LPT, DIN-разъём, для подключения PS/2 мыши и клавиатуры.

---

<sup>1</sup> SCSI (Small Computer System Interface, произносится «скази») — представляет собой набор стандартов для физического подключения и передачи данных между компьютерами и периферийными устройствами. SCSI-стандарты определяют команды, протоколы и электрические и оптические интерфейсы. Разработан для объединения на одной шине различных по своему назначению устройств, таких, как жёсткие диски, накопители на магнитооптических дисках, приводы CD, DVD, стримеры, сканеры, принтеры и т. д. Исходно был параллельным, 8- или 16-разрядным, впоследствии перешёл на последовательный вариант – SAS (Serial Attached SCSI), с сохранением логической и программной совместимости с параллельной версией.

<sup>2</sup> Конфигурацию с восемью процессорами (и даже более) поддерживали, например, процессоры Intel Pentium III Xeon.

<sup>3</sup> Первый представитель этого семейства базировался на архитектуре Pentium II.

<sup>4</sup> Год выпуска Pentium MMX – 1997-ой, а ISA-шина всё ещё востребована.



Рис.314 - Материнская плата Socket 7 ASUS TXP4

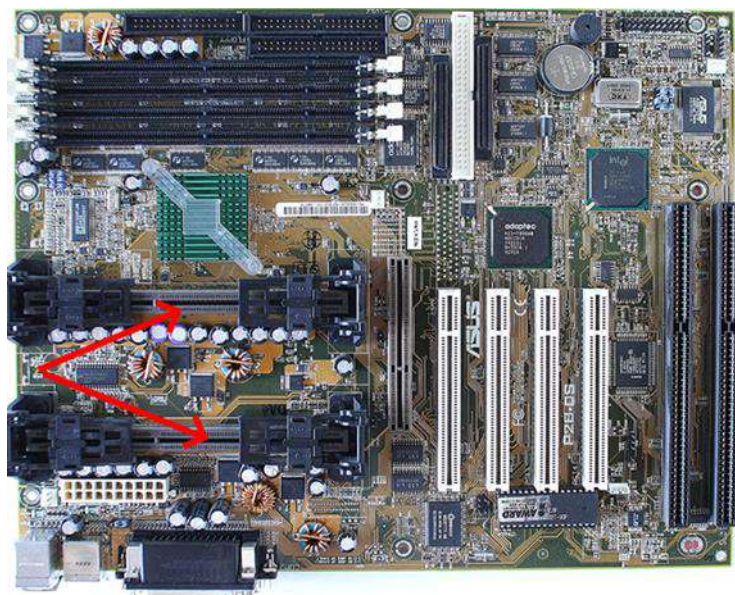
Intel, сумев обеспечить необходимые объёмы выпуска новых процессоров, стала главным драйвером рынка PC. Её процессоры работали уже не только в десктопных компьютерах или в ноутбуках, но и стали активно выходить на серверный рынок, рынок высокопроизводительных рабочих станций. И если раньше словосочетание «сервер на основе процессора x86» вызывало разве что скептическую улыбку, то теперь, с появлением «тяжёлой артиллерии» в виде семейства Xeon, способного, в том числе, работать в многопроцессорных конфигурациях, термин Intel-сервер постепенно становился всё более привычным, трансформировавшись со временем в общепринятый «сервер *стандартной архитектуры*».

Ниже приведена фотография (Рис.315) двухслотовой материнской платы ASUS P2B-DS, рассчитанной на установку до двух процессоров Pentium II/III в формфакторе Slot 1 (подробности далее), чипсет Intel i440BX. Набор шин и интерфейсов на материнской плате:

- Два процессорных разъёма Slot 1 (отмечены красными стрелками на фото);
- Два разъёма шины ISA (1998 год!);
- Четыре разъёма классического PCI (32 разряда/33 МГц);
- Разъём ATA-33;
- Два 68-контактных разъёма параллельной шины Ultra2 SCSI;
- Четыре слота для DIMM памяти;
- Разъём AGP;

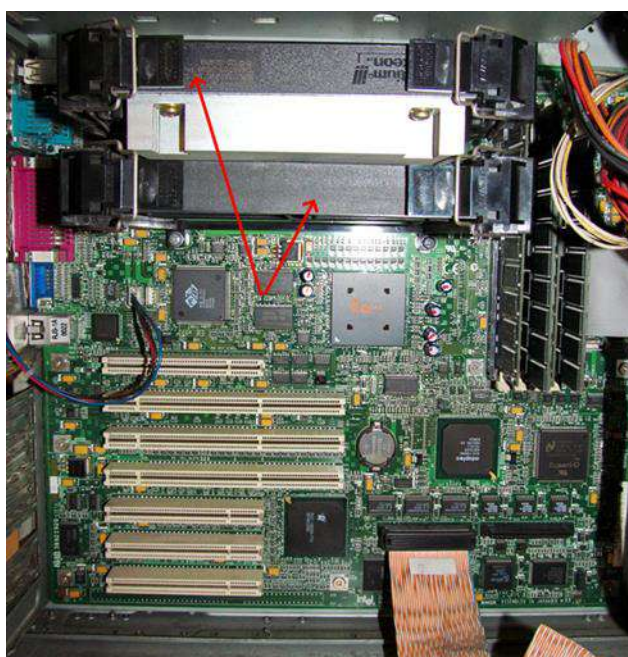
Двухпроцессорная материнская плата и наличие на ней сложного и дорогого периферийного интерфейса SCSI позволяло относить P2B-DS к классу серверных (хотя, наверное, правильнее было бы отнести её к рабочим станциям – всё-таки поддерживаемые процессоры – Pentium II/III – формально не относятся к серверным).





**Рис.315 – Материнская плата Dual Slot 1 ASUS P2B-DS**

Пример мощной серверной материнской платы Intel SBT2 Server Board приведён на Рис.316. На этой плате возможна была установка до двух процессоров Intel III Xeon Slot 2 с фронтальной шиной (FSB), работающей на 133 МГц; серверный чипсет ServerWorks ServerSet III LE.



**Рис.316 – Серверная материнская плата Intel SBT2 server board**

Набор шин и интерфейсов, присутствующий на плате SBT2, позволяет однозначно отнести её к полноценному серверному решению. Обратите внимание – на материнской плате отсутствуют характерные для десктопов тех лет разъёмы AGP (есть только простой интегрированный видеоадаптер ATI Rage II, с видеопамятью 4 Мбайт):

- Два процессорных разъёма Slot 2 (отмечены красными стрелками на фото);
- Три разъёма PCI полной длины;
- Три разъёма PCI-64 полной длины;
- Разъём ATA-33;



- Разъём Ultra2 Wide SCSI;
- Разъём Ultra Wide SCSI;
- Четыре слота памяти DIMM 168-pin, SDRAM PC133, с коррекцией ошибок;
- Интегрированный видеоадаптер ATI Rage II, 4 Мбайт видеопамяти;
- Интегрированный Ethernet-адаптер Intel.

Наконец-то на плате мы не видим ни одного разъёма ISA, зато на ней присутствуют полноразмерные PCI, включая 64-битный вариант – такие версии PCI были совершенно не характерны для десктопных плат. И, конечно же, нужно отметить разъём Ultra2 Wide SCSI – этот интерфейс тоже один из признаков «настоящего» сервера, на десктопных платах интегрированный SCSI практически не встречался, присутствуя только на платах класса рабочая станция и выше.

Пример типичной компактной материнской платы ACORP 4PE800 на основе процессора Pentium 4 Socket 478 и чипсета Intel 845PE приведён на фото (Рис.317).



**Рис.317 – Материнская плата ACORP 4PE800**

Характеристики платы 4PE800:

- Гнездо процессора Socket478
- Чипсет Intel i845PE + ICH4;
- Частота шины 400, 533, 800 МГц (частота 800 МГц не документированна);
- Поддержка Hyper Threading;
- Поддержка Pentium 4, Celeron;
- AGP 4x/2x (v 2.0), карты с Vi/o 3.3V не поддерживаются;
- Количество разъемов PCI 5 (v2.2);
- Два разъёма DDR DIMM;
- Тип поддерживаемой памяти PC3200 (DDR400), PC2700 (DDR333), PC2100 (DDR266), PC1600 (DDR200) (поддержка памяти PC3200 не документированна);
- Максимальный объём оперативной памяти до 2 Гбайт;
- Поддержка UDMA/33/66/100, 2 канала с возможностью подключения до четырёх устройств;
- Звук – 6-канальный AC97 кодек;
- Клавиатура/мышь PS/2;
- Порты: 1xPS/2 клавиатура, 1xPS/2 мышь, 1xLPT, 2xCOM, 2xUSB 2.0, 1x Game/MIDI, аудиоразъемы (line-out, line-in, mic-in);

## Архитектура микропроцессоров Pentium поколений P5...P7

Следует отметить, что в процессорах пятого (по интеловской классификации) и следующих поколений всё больший процент площади кристалла процессора стали занимать схемы, не относящиеся непосредственно к вычислительным функциям – кэш, логика внеочередного выполнения команд (появилась в шестом поколении процессоров), конвейеры и т.п.

### Основные отличия от процессора 486

- Суперскалярная архитектура<sup>1</sup>. Благодаря использованию суперскалярной архитектуры процессор может выполнять 2 команды за 1 такт. Такая возможность существует благодаря наличию двух конвейеров — U и V. U-конвейер — основной, выполняет все операции над целыми и вещественными числами; V-конвейер — вспомогательный, выполняет только простые операции над целыми и частично над вещественными. Чтобы старые программы (для 486) в полной мере использовали возможности такой архитектуры, необходимо было их перекомпилировать. Pentium — первый CISC-процессор, использующий многоконвейерную архитектуру.
- 64-битная шина данных позволяет процессору Pentium за один шинный цикл обмениваться вдвое большим объёмом данных с оперативной памятью, чем 486-му (при одинаковой тактовой частоте).
- Механизм предсказания адресов ветвления. Применяется для сокращения времени простоя конвейеров, вызванного задержками выборки команд при изменении счётчика адреса во время выполнения команд ветвления. Для этого в процессоре используется буфер адреса ветвления BTB (Branch Target Buffer), использующий алгоритмы предсказания адресов ветвления.
- Раздельное кэширование программного кода и данных, сократившее число промахов кэша при выборке инструкций и операндов по сравнению с i80486. В процессорах Pentium используется кэш-память первого уровня (кэш L1) объёмом 16 Кб, разделённая на 2 сегмента: 8 Кбайт для данных и 8 Кбайт для инструкций. Для сокращения времени доступа и снижения стоимости реализации оба сегмента являются 2-канальными множественно-ассоциативными, в отличие от 4-канального кэша 80486.
- Улучшенный блок вычислений с плавающей точкой (FPU). Некоторые инструкции ускорились на порядок, например, FMUL, скорость выполнения которой увеличилась в 15 раз. Процессор также может выполнять инструкцию FXCH ST(x) параллельно с обычными инструкциями (арифметическими или загрузки/выгрузки регистров).
- Четырёхвходовые адресные сумматоры. Позволяют уменьшить латентность вычисления адреса по сравнению с i80486. В процессоре Pentium можно за один такт вычислить эффективный адрес при использовании базово-индексной адресации с масштабированием и смещением. i80486 имеет трёхвходовый адресный сумматор, поэтому в нём вычисление такого адреса занимает два такта.
- Микрокод может использовать оба конвейера, в результате чего инструкции с префиксом повторения, такие как REP MOVSW, выполняют одну итерацию за такт, в то время как 80486 требуется три такта на итерацию.
- Более быстрый полностью аппаратный умножитель в несколько раз сокращает (и делает более предсказуемым) время выполнения инструкций MUL и IMUL по сравнению с

---

<sup>1</sup> Суперскалярный процессор (superscalar processor) — процессор, поддерживающий так называемый параллелизм на уровне инструкций (то есть, процессор, способный выполнять несколько инструкций одновременно) за счёт включения в состав его вычислительного ядра нескольких функциональных узлов (таких как ALU, FPU, умножитель (integer multiplier), сдвигающее устройство (integer shifter) и другие устройства). Планирование исполнения потока инструкций осуществляется динамически вычислительным ядром (но не статически, компилятором).

80486. Время выполнения уменьшается с 13—42 тактов до 10—11 для 32-битных операндов.

- Виртуализация прерываний, позволяющая ускорить режим виртуального 8086.

В таблице на Рис.318 приведены характеристики ядер процессоров Pentium первого поколения (P5).

|                                   | P5                                | P54C                | P54CS                                          | P55C          |
|-----------------------------------|-----------------------------------|---------------------|------------------------------------------------|---------------|
| Дата анонса первой модели         | 23 марта 1993                     | 7 марта 1994        | 27 марта 1995                                  | 8 января 1997 |
| Тактовые частоты, МГц             | 60, 66                            | 75, 90, 100         | 120, 133, 150, 166, 200                        | 166, 200, 233 |
| Частота системной шины (FSB), МГц | 60, 66                            | 50, 60, 66          | 60, 66                                         | 66            |
| Кэш L1, Кбайт                     | 8 (для данных)+8 (для инструкций) |                     |                                                | 16+16         |
| Кэш L2, Кбайт                     | внешний до 2 Мбайт                |                     |                                                |               |
| Напряжение питания, В             | 5                                 | 3,3                 |                                                | 2,8/3,3       |
| Количество транзисторов, млн      | 3,1                               | 3,2                 | 3,3                                            | 4,5           |
| Площадь кристалла, мм²            | 294                               | 148                 | 90                                             | 141           |
| Максимальное тепловыделение, Вт   | 16                                | 10,1                | 15,5                                           | 16            |
| Техпроцесс, нм                    | 800                               | 600                 | 350                                            | 280           |
| Разъём                            | Socket 4                          | Socket 5, Socket 7  | Socket 5, Socket 7 (150—200 — только Socket 7) | Socket 7      |
| Корпус                            | 273-контактный PGA                | 296-контактный CPGA | 296-контактный CPGA/PPGA                       |               |
| Адресуемая память                 | 4 Гбайт                           |                     |                                                |               |
| Разрядность регистров             | 32                                |                     |                                                |               |
| Разрядность внешней шины          | 64                                |                     |                                                |               |
| Разрядность шины адреса           | 32                                |                     |                                                |               |

Рис.318 – Характеристики ядер процессоров поколения P5

Интересная ситуация в своё время произошла с выпуском модификации процессора формально всё того же пятого поколения Pentium MMX<sup>1</sup> – процессор включал в себя устройство MMX с конвейерной обработкой команд, кэш L1 был увеличен до 32 Кбайт (16 Кбайт для данных и 16 Кбайт для инструкций, количество set-ов (way) увеличено с 2-way до 4-way). Содержал 57 новых команд по параллельной обработке целочисленных данных, был введён тип данных 64 бита. Были доступны модели с тактовыми частотами 166, 200 и 233 МГц. Только за счёт изменения размера и конфигурации кэша некоторые программы, без какой-либо перекомпиляции или оптимизации, начинали работать на MMX-процессоре существенно – до полутора раз – быстрее, чем на не-MMX процессоре с такой же тактовой частотой.

Pentium MMX состоял из 4.5 млн транзисторов и производился по усовершенствованной 280-нанометровой технологии с использованием кремниевых полупроводников CMOS, работал на пониженном напряжении 2.8 В<sup>2</sup>. Максимальный потребляемый ток равен 6.5 А, тепловыделение равно 17 Вт (для Pentium 233 MMX). Площадь кристалла у процессоров Pentium MMX равна 141 мм². Процессоры выпускались в 296-контактном корпусе типа CPGA или PPGA для Socket 7.

<sup>1</sup> В базовое ядро P55C был добавлен новый набор инструкций, названный MMX (MultiMedia eXtension), существенно увеличивающий (от 10 до 60 %, в зависимости от оптимизации) производительность компьютера в мультимедиа-приложениях.

<sup>2</sup> Другое напряжение питания ядра процессоров с MMX потребовало доработки материнских плат, которые должны были обеспечить новое напряжение питания 2.8 вольта. Такие платы получали логотип “MMX Ready”, при этом процессорное гнездо было заменено с Socket 5 на Socket 7, отличавшиеся ровно одним выводом для подачи раздельного питания для ядра и для периферийных схем.

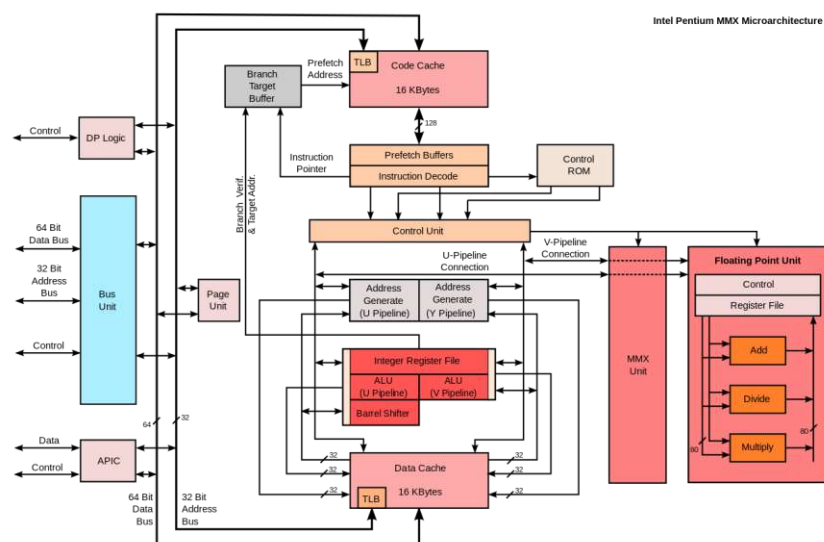


Рис.319 - Микроархитектура Intel Pentium MMX

## Pentium Pro

Продолжая выпуск и модернизацию поколения Pentium P5, Intel параллельно разрабатывала процессор шестого поколения, который был анонсирован в ноябре 1995 года, получив название Pentium Pro. Изначально планировалось полностью заменить всю линейку P5 на Pentium Pro, но он оказался очень дорогим, сложным в производстве, и позиционировался в итоге как high-end процессор для серверов и рабочих станций. Мог работать в многопроцессорной конфигурации (до четырёх процессоров).

Были выпущены модели с тактовой частотой 150, 166, 180 и 200 МГц. Несмотря на схожее название ядра (Pentium), оно сильно отличалось от всех предыдущих ядер. Помимо абсолютно новой суперскалярной суперконвейерной архитектуры, разработанной независимо от процессоров пятого поколения, этот процессор отличало применение технологии динамического исполнения (изменения порядка исполнения инструкций) и двойной независимой шины (Dual independent bus), благодаря чему были сняты ограничения на пропускную способность памяти.

Процессор был заключён в 387-контактный корпус типа SPGA, устанавливаемый в Socket 8, который механически не был совместим ни с одним ранее выпускавшимся разъёмом. Socket 8 разрабатывался с учетом того, чтобы объединить до 4 процессоров в многопроцессорную структуру MPS. В одном корпусе размещалось сразу два кристалла — само ядро процессора и кэш-память L2. Кроме того, была добавлена ещё одна шина, которая соединяла процессор с кэшем L2. В результате всего этого был впервые применён кэш L2, работающий на частоте ядра.

Кэш первого уровня составлял 16 Кбайт, а кэш L2 первоначально имел размер 256 Кбайт (также выпускались процессоры Pentium Pro 166 и 200 с кэшем L2 объёмом 512 Кбайт), однако 18 августа 1997 года был анонсирован процессор Pentium Pro 200 с 1 Мбайт кэш-памяти L2 на борту (два кристалла кэш-памяти L2 по 512 Кбайт). Pentium Pro был очень недешёвым устройством (Pentium Pro 200 с 1 Мбайт L2 первоначально стоил 2675\$) ввиду того, что теперь в корпусе находилась очень дорогая по себестоимости кэш-память L2. Разрядность шины адреса составляла 36 бит.

Pentium Pro не поддерживал технологию MMX. При 32-битных вычислениях и многозадачности он значительно превосходил по производительности Pentium, но при этом в 16-битных приложениях довольно существенно ему уступал.

## Технические характеристики ядра P6

- Дата анонса первой модели: 1 ноября 1995 года

- Тактовые частоты (МГц): 150, 166, 180, 200
- Частота системной шины (FSB) (МГц): 60, 66
- Размер кэша L1 (Кбайт): 8 (для данных) + 8 (для инструкций)
- Напряжение питания: 3.1 или 3.3 В
- Количество транзисторов в ядре (млн.): 5.5
- Площадь кристалла ядра (кв. мм): 195 или 306
- Адресуемая память: 64 Гбайт
- Разрядность регистров: 32
- Разрядность внешней шины данных: 64
- Разрядность шины адреса: 36
- Размер кэша L2(Кбайт): 256 Кбайт (для Pentium Pro 166 также 512Кб, для Pentium Pro 200 также 512Кб и 1Мб)
- Максимальное тепловыделение (Вт): 47
- Техпроцесс (мкм): 0.5, 0.35
- Разъём: Socket 8
- Корпус: 387-контактный SPGA

Ниже на Рис.320 приведён список всех выпускавшихся моделей Pentium Pro

| Модель          | Частота | Кэш первого уровня | Кэш второго уровня | Технология              | Частота FSB | Напряжение | TDP    | Сокет         | Дата выпуска |
|-----------------|---------|--------------------|--------------------|-------------------------|-------------|------------|--------|---------------|--------------|
| Pentium Pro 150 | 150 МГц | 8+8 КБ             | 256 КБ             | 0,60/0,60 $\mu\text{m}$ | 60 МГц      | 3,1v       | 29,2 W | Socket 8 PGA  | 1.11.1995    |
| Pentium Pro 166 | 166 МГц | 8+8 КБ             | 512 КБ             | 0,35/0,35 $\mu\text{m}$ | 66 МГц      | 3,3v       | 35 W   | Socket 8 PGA  | 1.11.1995    |
| Pentium Pro 180 | 180 МГц | 8+8 КБ             | 256 КБ             | 0,35/0,50 $\mu\text{m}$ | 60 МГц      | 3,3v       | 31,7 W | Socket 8 PGA  | 1.11.1995    |
| Pentium Pro 200 | 200 МГц | 8+8 КБ             | 256 КБ             | 0,35/0,50 $\mu\text{m}$ | 66 МГц      | 3,3v       | 35 W   | Socket 8 PGA  | 1.11.1995    |
| Pentium Pro 200 | 200 МГц | 8+8 КБ             | 512 КБ             | 0,35/0,35 $\mu\text{m}$ | 66 МГц      | 3,3v       | 37,9 W | Socket 8 PGA  | 1.11.1995    |
| Pentium Pro 200 | 200 МГц | 8+8 КБ             | 1024 КБ            | 0,35/0,35 $\mu\text{m}$ | 66 МГц      | 3,3v       | 47 W   | Socket 8 PPGA | 18.08.1997   |

**Рис.320 – Список микропроцессоров Pentium Pro**

Отказавшись от планов полностью заменить процессором Pentium Pro пятое поколение своих процессоров – PPro был очень дорогим и плохо (медленно) работал с 16-битными приложениями, уступая в скорости P5 – Intel на основе архитектуры P6 разработала новые микропроцессоры Pentium II, Pentium III, Celeron и Xeon. В отличие от x86-совместимых процессоров предыдущих поколений с CISC-ядром, процессоры архитектуры P6 имеют RISC-ядро, исполняющее сложные инструкции x86 не напрямую, а предварительно декодируя их в более простые внутренние микрооперации.

## Pentium II

Следующим, после Pentium Pro, микропроцессором шестого поколения стал Pentium II. Торговая марка Pentium уже была очень популярной и Intel решила сохранить это наименование для последующих своих процессоров. Intel Pentium II был анонсирован 7 мая 1997 года. Ядро Pentium II представляет собой модифицированное ядро P6 (впервые использованное в процессорах Pentium Pro). Основными отличиями от предшественника являются увеличенный с 16 до 32 Кбайт кэш первого уровня и наличие блока SIMD-инструкций MMX (появившихся немногим ранее в Pentium MMX), была повышена производительность



при работе с 16-разрядными приложениями. В системах, построенных на базе процессора Pentium II, повсеместное применение нашли память SDRAM и шина AGP.

К тому времени уже практически стало правилом – для каждого своего следующего процессора Intel использовала новый сокет/корпус/разъём. Не стал исключением и P-II – для него впервые был использован довольно громоздкий катридж типа SECC/SECC2, предназначенный для установки в 242-контактный щелевой Slot 1.

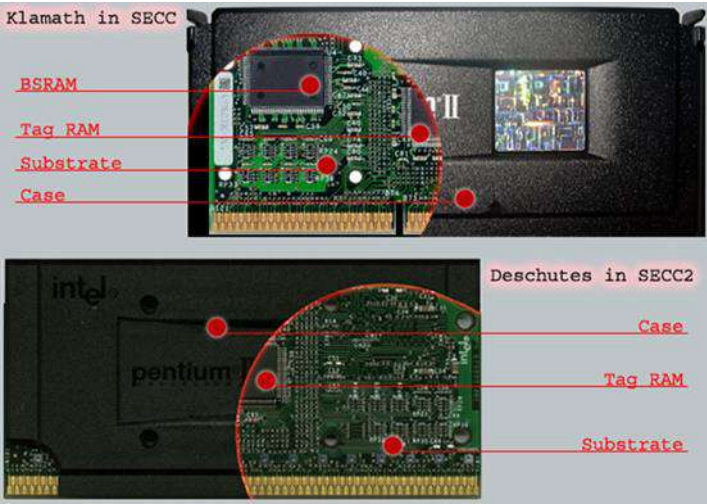


Рис.321 - Pentium-II: корпуса SECC и SECC2

Картридж содержал процессорную плату («субстрат») с установленной на ней микросхемой процессора, а также двумя или четырьмя микросхемами кэш-памяти типа BSRAM и tag-RAM. Кэш-память второго уровня работает на *половине* частоты ядра, в отличие от Pentium Pro. В корпусе типа SECC выпускались все процессоры на ядре Klamath, ранние модели на ядре Deschutes с частотами 266—333 МГц и часть более поздних моделей на этом ядре.

Основным отличием корпуса типа SECC2 от SECC является отсутствие теплораспределительной пластины. Кулер, установленный на процессор в корпусе типа SECC2, контактирует непосредственно с микросхемой процессора. В корпусе типа SECC2 выпускались часть поздних моделей Pentium II на ядре Deschutes с частотами 350—450 МГц.

Существовал также вариант Pentium II OverDrive в корпусе PGA (устанавливался в гнездовой разъём Socket 8) с полноскоростным кэшем второго уровня, предназначенный для замены Pentium Pro.

**Модели**

Первые процессоры Pentium II (Klamath) были предназначены для рынка настольных персональных компьютеров и производились по 350 нм технологии. Дальнейшим развитием семейства десктопных Pentium II стало 250 нм ядро Deschutes. Через некоторое время вышли процессоры Mobile Pentium II, предназначенный для установки в ноутбуки, и Xeon, ориентированный на высокопроизводительные системы и серверы. На базе ядра Deschutes выпускались также процессоры Celeron (Covington), предназначенные для использования в недорогих компьютерах. Они представляли собой Pentium II, лишённый картриджа и кэша L2. Ниже, на Рис.322, приведены основные характеристики процессоров Pentium II для настольных компьютеров.

| Кодовое имя ядра            | Klamath    |              |                 |                | Deschutes      |                 |     |     |     |
|-----------------------------|------------|--------------|-----------------|----------------|----------------|-----------------|-----|-----|-----|
| Тактовая частота ядра (МГц) | 233        | 266          | 300             | 266            | 300            | 333             | 350 | 400 | 450 |
| Анонсирован                 | 7 мая 1997 | 14 июля 1997 | 1 сентября 1997 | 26 января 1998 | 15 апреля 1998 | 24 августа 1998 |     |     |     |
| Цена, долл. США             | 636        | 775          | 1981            | —              | —              | 772             | 621 | 824 | 669 |

Рис.322 - Процессоры Pentium II для настольных компьютеров (desktop)

## Celeron

15 апреля 1998 года был анонсирован первый процессор Celeron — большое семейство недорогих офисных x86-совместимых процессоров компании Intel. Процессоры Celeron изначально позиционировались как низкобюджетные версии старших моделей и предназначались для расширения доли рынка компании Intel за счёт недорогих компьютеров для дома и офиса. Одной из причин невысокой цены является их более низкая, по сравнению со старшими моделями, производительность, что достигается двумя основными методами: искусственным снижением частоты шины процессора и блокировкой или урезанием части кэш-памяти второго уровня (L2).

Первый процессор семейства Celeron был анонсирован 15 апреля 1998 года и был построен на основе Pentium II. Позже вышли процессоры, основанные на Pentium III, Pentium 4, Pentium M, Pentium D, Core 2 Duo и Core i3, i5, i7.

До выпуска Celeron процессоры Intel активно вытеснялись с рынка low-end такими конкурентами, как AMD с процессором K6 и IDT с процессором WinChip. Оба процессора были предназначены для уже устаревающей платформы Socket 7. Конкурировать с ними тогда мог только Pentium MMX, в то время уже позиционировавшийся как процессор для low-end рынка. Но производительности Pentium MMX уже начинало не хватать, и компания Intel решила выпустить процессор, построенный на архитектуре Pentium II и при этом с ценой, привлекательной для построения малобюджетной системы. В результате Intel удалось отвоевать большую долю рынка. Процессор, как и Pentium II, выпускался для Slot 1, но использовал корпус типа SEPP, в котором нет верхней пластиковой крышки.

В первой попытке выпустить дешёвый процессор Intel допустила тактическую ошибку — первые процессоры семейства Celeron были основаны на ядре *Covington*, которое представляло собой ядро *Deschutes* без кэша L2. Отсутствие кэша L2 привело к тому, что процессор был заметно менее производительным, чем даже Pentium MMX, и это при том, что частота Celeron была выше. В результате получилось так, что на смену старому процессору пришёл процессор с новой архитектурой, но при этом заметно более медленный. Всё это вынудило Intel быстро выпустить новое ядро — *Mendocino* (Celeron 300A). На ядре *Covington* было выпущено всего две модели с частотами 266 и 300 МГц. Вместе с тем, эти процессоры стали настоящей находкой для оверклокеров, большинство из них разгонялись до 400 и 450 МГц и более. При этом производительность разогнанных процессоров в 3D-играх была не на много меньше чем у Pentium II с той же частотой, а стоили они в несколько раз дешевле.

Интересный факт: официально, по документации, Celeron-ы не могли работать в двухпроцессорных конфигурациях, но путём несложной аппаратной доработки была возможность заставить работать *Mendocino* (и некоторые следующие модификации) в двухпроцессорных системах.

Всего было выпущено несколько десятков моделей семейства Celeron, основанных на ядре Pentium II и новее. Нумерация версий Celeron далеко не всегда напрямую коррелировала с названием базовой модели процессора, например, Celeron на ядре Coppermine-128, относящийся к семейству Pentium III Celeron, именовался Celeron II.

Ниже приведён неполный список моделей Celeron, выпускавшихся на основе разных ядер Pentium:

1. Covington
2. Mendocino
3. Coppermine-128
4. Tualatin
5. Willamette-128
6. Northwood-128
7. Prescott-256
8. Cedar Mill

9. Conroe-L
10. Allendale
11. Wolfdale
12. Sandy Bridge
13. Ivy Bridge
14. Braswell

## **Pentium II Xeon**

Intel Pentium II Xeon – другой, высокопроизводительный и дорогой фланг линейки процессоров Intel на основе ядра Pentium II. Выпускался Intel с июня 1998 года. Всего в линейку входило пять модификаций процессоров, последние из которых (Pentium II Xeon 450 (L2 кэш – 1 Мбайт), Pentium II Xeon 450 (L2 кэш – 2 Мбайт) были представлены в январе 1999 года. В отличие от «десктопной» версии Pentium II, Xeon имел кэш второго уровня, работавший на полной, а не на половинной, частоте ядра. Процессор был упакован в картридж SEC, но он был значительно крупнее, чем аналогичный для Pentium II, и использовал другой разъём.

Intel Pentium II Xeon был анонсирован в 1998 году, сразу после анонса Pentium II на частоте FSB 100 МГц и процессора Intel Celeron. К тому времени, Intel уже год как занималась заменой процессоров Intel Pro на Pentium II. Основное различие между ними заключалось в скорости и местоположении кэша второго уровня в ЦП. Максимальная частота тактирования для Pentium Pro была 200 МГц (ограничение быстродействия кэша L2, расположенного на том же чипе, что и ядро процессора). Pentium II был первым процессором Intel, поставляемым в картридже, в виде печатной платы с ядром процессора, а также двумя или четырьмя микросхемами кэша L2, работавшего на половинной частоте.

В то время как Pentium Pro был довольно быстро заменен Pentium II на рынке настольных ПК, рынок серверов по-прежнему требовал процессор класса Pentium Pro. Для этого было несколько причин. Основной проблемой Pentium II в серверных системах было исходное ограничение адресуемой основной памяти до 512 Мбайт, что для серверных применений часто было уже недостаточно. Другая причина, по которой Pentium Pro по-прежнему был очень привлекателен для серверов, заключалась в наличии у него кэша второго уровня, работавшего на полной частоте процессора, в результате чего Pentium Pro был быстрее, чем любой Pentium II с частотами до 350 МГц. Были доступны варианты Pentium Pro с объёмом кэша до 1 Мбайт, тогда как Pentium II мог предложить максимум 512 Кбайт L2.

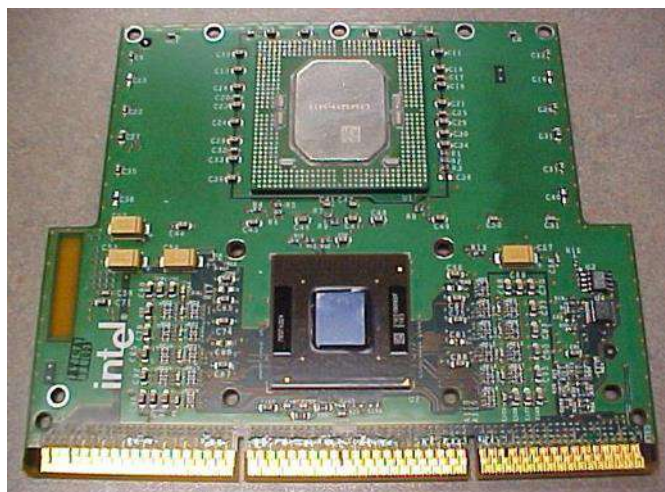
И, наконец: Pentium Pro мог работать как в двух-, так и в четырёхsocketном варианте, в то время как Pentium II мог работать максимум в двухsocketной версии. И, в итоге, для полноценной замены Pentium Pro в «тяжёлых» системах, Intel выпустила процессор Intel Pentium II Xeon.

## **Архитектура Pentium II Xeon**

В процессоре было использовано ядро “Deschutes” (Pentium II, первый Celeron). Главная отличительная черта Xeon – кэш L2. В то время как Celeron либо не имел в своем составе кэш L2 или же он был сильно урезан в объёме, а Pentium II мог предложить лишь 512 Кбайт кэша L2 на половинной частоте, Xeon поставлялся в версиях с 512 Кбайт, 1 Мбайт и 2 Мбайт кэша L2, работавшего на полной частоте.

Чипы памяти для L2-кэша Intel производила самостоятельно, в отличие от чипов, установленных в кэше Pentium II. Объём одного кристалла был 512 Кбайт, поэтому в вариантах 1 Мбайт и 2 Мбайт было необходимо устанавливать 2 и 4 таких чипа, соответственно.





**Рис.323 – Pentium II 450-512**

Таким образом, более быстрый кэш L2 был основной причиной увеличения производительности Intel Pentium II Xeon. Важным моментом была возможность работы процессоров в четырёх- и даже восьмисокетных конфигурациях, в сочетании с чипсетом i450NX и специальным кластерным контроллером. Предел кэшируемой памяти достигал величины  $2^{36} = 64$  Гбайт (при использовании 36-битной шины памяти и PSE36 режима).

Частота фронтальной шины (FSB) была 100 МГц, что обеспечивало пропускную способность при работе с памятью до 800 Мбайт/с.

Кроме всего прочего, Xeon, за счёт функционала, встроенного в чипсеты i440GX и i450NX, мог работать с памятью с коррекцией ошибок (ECC).

## **Pentium III**

26 февраля 1999 года был анонсирован Intel Pentium III — x86-совместимый микропроцессор архитектуры Intel P6. Ядро Pentium III представляет собой модифицированное ядро Deschutes (которое ранее использовалось в процессорах Pentium II). По сравнению с предшественником был расширен набор команд (добавлен набор инструкций SSE) и оптимизирована работа с памятью. Это позволило повысить производительность как в новых приложениях, использующих расширения SSE, так и в существующих (за счёт возросшей скорости работы с памятью). Также был введён 64-битный серийный номер, уникальный для каждого процессора.

С выпуском P-III Intel изменила своему правилу – исходно процессор упаковывался в тот же корпус, что и его предшественник P-II, но дальше Intel «исправилась», кардинально изменив тип корпуса с катриджа на гнездовой. Всего процессоры Pentium III для настольных компьютеров выпускались в трёх вариантах корпусов: SECC2, FCPGA и FCPGA2.

Pentium III в корпусе SECC2 представляет собой картридж, содержащий процессорную плату («субстрат») с установленным на ней ядром процессора (во всех модификациях), а также микросхемами кэш-памяти BSRAM и tag-RAM (в процессорах, основанных на ядре Katmai). Маркировка находится на картридже. Процессор предназначен для установки в 242-контактный щелевой разъём Slot 1. В процессорах, основанных на ядре Katmai, кэш-память второго уровня работает на половине частоты ядра, а в процессорах на ядре Coppermine — на частоте ядра.

Pentium III в корпусе FCPGA представляют собой подложку из органического материала зелёного цвета с установленным на ней открытым кристаллом на лицевой стороне и контактами на обратной. Также на обратной стороне корпуса (между контактами) расположено несколько SMD-элементов. Маркировка нанесена на наклейку, расположенную под кристаллом. Кристалл защищён от сколов специальным покрытием синего цвета, снижающим его хрупкость. Однако, несмотря на наличие этого покрытия, при неаккуратной установке радиатора (особенно неопытными пользователями) кристалл мог получить трещины

и сколы (процессоры, получившие такие повреждения, на жаргоне назывались «коло-тыми»). В некоторых случаях процессор, даже получивший существенные повреждения кристалла (сколы до 2—3 мм с углов), продолжал работать без сбоев или с редкими сбоями.

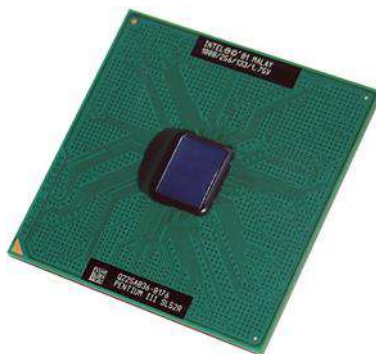


Рис.324 - Pentium III 1 ГГц (FCPGA)

Процессор предназначен для установки в 370-контактный гнездовой разъём Socket 370. В корпусе FCPGA выпускались процессоры на ядре Coppermine.

Корпус FCPGA2 отличался от FCPGA наличием теплораспределителя (металлическая крышка, закрывающая кристалл процессора), защищающего кристалл процессора от сколов (однако его наличие снижает эффективность охлаждения). Маркировка нанесена на наклейки, расположенные сверху и снизу от теплораспределителя. В корпусе FCPGA2 выпускались процессоры на ядре Tualatin, а также процессоры на поздней версии ядра Coppermine (известной как Coppermine-T).

### **Особенности архитектуры**

Приведённые ниже сведения об особенностях архитектуры P6 частично повторяют то, что уже было приведено в тексте ранее.

Первые процессоры архитектуры P6 в момент выхода значительно отличались от существующих процессоров. Процессор Pentium Pro отличало применение технологии динамического исполнения (изменения порядка исполнения инструкций), а также архитектура двойной независимой шины (Dual Independent Bus), благодаря чему были сняты многие ограничения на пропускную способность памяти, характерные для предшественников и конкурентов. Тактовая частота первого процессора архитектуры P6 составляла 150 МГц, а последние представители этой архитектуры имели тактовую частоту 1.4 ГГц. Процессоры архитектуры P6 имели 36-разрядную шину адреса, что позволило им адресовать до 64 Гбайт памяти (при этом линейное адресное пространство процесса ограничено 4 Гб (PAE)).

### **Суперскалярный механизм исполнения инструкций с изменением их последовательности**

Принципиальным отличием архитектуры P6 от предшественников является RISC-ядро, работающее не с инструкциями x86, а с более простыми внутренними микрооперациями. Это позволяет снять множество ограничений набора команд x86, таких как нерегулярное кодирование команд, переменная длина операндов и операции целочисленных пересылок регистр-память. Кроме того, микрооперации выполняются не в той последовательности, которая предусмотрена программой, а в оптимальной с точки зрения производительности, а применение трёхконвейерной обработки позволяет исполнять несколько инструкций за один такт (по сути это и есть определение суперскалярности).

### **Суперконвейеризация**

Процессоры архитектуры P6 имеют конвейер глубиной 12 стадий. Это позволяет достигать более высоких тактовых частот по сравнению с процессорами, имеющими более

короткий конвейер при одинаковой технологии производства. Так, например, максимальная тактовая частота процессоров AMD K6 на ядре (глубина конвейера — 6 стадий, 180 нм технология) составляет 550 МГц, а процессоры Pentium III на ядре Coppermine способны работать на частоте, превышающей 1000 МГц.

Для того, чтобы предотвратить ситуацию ожидания исполнения инструкции (и, следовательно, простоя конвейера), от результатов которого зависит выполнение или невыполнение условного перехода, в процессорах архитектуры P6 используется предсказание ветвлений. Для этого в процессорах архитектуры P6 применяется сочетание статического и динамического предсказания: двухуровневый адаптивный исторический алгоритм (Bimodal branch prediction) применяется в том случае, если буфер предсказания ветвлений содержит историю переходов, в противном случае применяется статический алгоритм.

### Двойная независимая шина

С целью увеличения пропускной способности подсистемы памяти, в процессорах архитектуры P6 применяется двойная независимая шина. В отличие от предшествующих процессоров, системная шина которых была общей для нескольких устройств, процессоры архитектуры P6 имеют две отдельные шины: Back side bus, соединяющую процессор с кэш-памятью второго уровня, и Front side bus, соединяющую процессор с северным мостом набора микросхем.

### Модели

Первые процессоры Pentium III (Katmai) предназначались для настольных компьютеров и производились по 250 нм технологии. Дальнейшим развитием семейства настольных Pentium III стало 180 нм ядро Coppermine, а последним ядром, использованным в процессорах семейства Pentium III, стало 130 нм ядро Tualatin.

На базе ядра Katmai выпускался также процессор Xeon (ядро Tanner), на базе ядра Coppermine — Xeon (Cascades) и Celeron (Coppermine-128), на базе ядра Tualatin — Celeron (Tualatin-256). Ниже, на Рис.325, приведены основные технические характеристики процессоров Pentium III на ядрах разных поколений, а на Рис.326 — функциональная схема процессора Pentium III на ядре Coppermine.

|                                                          | Katmai                                                                                                               | Coppermine                                              |                                                                | Tualatin                                                             |                 |                  |
|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|----------------------------------------------------------------|----------------------------------------------------------------------|-----------------|------------------|
|                                                          | Настольный                                                                                                           |                                                         | Мобильный                                                      | Настольный                                                           | Серверный       | Мобильный        |
| Тактовая частота                                         |                                                                                                                      |                                                         |                                                                |                                                                      |                 |                  |
| Частота ядра, МГц                                        | 450—600                                                                                                              | 500—1133                                                | 500—1133                                                       | 400—1000                                                             | 1000—1400       | 1133, 1266, 1400 |
| Частота FSB, МГц                                         |                                                                                                                      | 100, 133                                                |                                                                | 100                                                                  | 133             | 100, 133         |
| Характеристики ядра                                      |                                                                                                                      |                                                         |                                                                |                                                                      |                 |                  |
| Набор инструкций                                         | IA-32, MMX, SSE                                                                                                      |                                                         |                                                                |                                                                      |                 |                  |
| Разрядность регистров                                    | 32 бит (целочисленные), 80 бит (вещественночисленные), 64 бит (MMX), 128 бит (SSE)                                   |                                                         |                                                                |                                                                      |                 |                  |
| Глубина конвейера                                        | (целочисленный) 12 — 17 стадий (в зависимости от типа исполняемой инструкции), вещественночисленный: 25 стадий       |                                                         |                                                                |                                                                      |                 |                  |
| Разрядность ША                                           | 36 бит                                                                                                               |                                                         |                                                                |                                                                      |                 |                  |
| Разрядность ШД                                           | 64 бит                                                                                                               |                                                         |                                                                |                                                                      |                 |                  |
| Аппаратная предвыборка длин                              | нет                                                                                                                  |                                                         |                                                                |                                                                      | есть            |                  |
| Количество транзисторов, млн                             | 9,5                                                                                                                  |                                                         | 28                                                             |                                                                      | 44              |                  |
| Кэш L1                                                   |                                                                                                                      |                                                         |                                                                |                                                                      |                 |                  |
| Кэш данных                                               | 16 Кб, 4-канальный наборно-ассоциативный, длина строки — 32 байта, двухпортовый                                      |                                                         |                                                                |                                                                      |                 |                  |
| Кэш инструкций                                           | 16 Кб, 4-канальный наборно-ассоциативный, длина строки — 32 байта                                                    |                                                         |                                                                |                                                                      |                 |                  |
| Кэш L2                                                   |                                                                                                                      |                                                         |                                                                |                                                                      |                 |                  |
| Объем, Кб                                                | 512                                                                                                                  |                                                         | 256                                                            |                                                                      | 512             |                  |
| Частота                                                  | 1/2 частоты ядра                                                                                                     |                                                         |                                                                | частота ядра                                                         |                 |                  |
| Разрядность ESB                                          | 64 бит + 8 бит ECC                                                                                                   |                                                         |                                                                | 256 бит + 32 бит ECC                                                 |                 |                  |
| Организация                                              | Объединенный, наборно-ассоциативный, непоколебимый, с контролем и исправлением ошибок (ECC), длина строки — 32 байта |                                                         |                                                                |                                                                      |                 |                  |
| Ассоциативность                                          | 4-канальный                                                                                                          |                                                         |                                                                | 8-канальный                                                          |                 |                  |
| Интерфейсы                                               |                                                                                                                      |                                                         |                                                                |                                                                      |                 |                  |
| Разъем                                                   | Slot 1                                                                                                               | Socket 370                                              | Socket 465, SMD                                                | Socket 370                                                           | Socket 478, SMD |                  |
| Корпус                                                   | OLGA в картридже SECC2                                                                                               | FCPGA, FCPGA2                                           | BGA2, mBGA2                                                    | FCPGA2                                                               | mFCPGA, mFCPGA  |                  |
| Шина                                                     | AGTL + (сигнальный уровень — 1,5 В)                                                                                  |                                                         |                                                                | AGTL, сигнальный уровень — 1,25 В)                                   |                 |                  |
| Технологические, электрические и тепловые характеристики |                                                                                                                      |                                                         |                                                                |                                                                      |                 |                  |
| Технология производства                                  | 250 нм, КМОП (эпитаксиальный, алюминиевые соединения)                                                                | 180 нм, КМОП (шестиступенчатый, алюминиевые соединения) |                                                                | 130 нм, КМОП (шестиступенчатый, медные соединения, Low-K диэлектрик) |                 |                  |
| Площадь кристалла, мм²                                   | 126                                                                                                                  | 106 (rev. A2)<br>105 (rev. B0)<br>90 (rev. C0)          | 106 (rev. A2)<br>100 (rev. B0)<br>90 (rev. C0)<br>85 (rev. D0) | 80                                                                   |                 |                  |
| Напряжение ядра, В                                       | 2,0 — 2,09                                                                                                           | 1,65 — 1,7                                              | 1,6 — 1,75                                                     | 0,975 — 1,7                                                          | 1,475 — 1,5     | 1,40 — 1,5       |
| Напряжение кэша L2, В                                    | 3,3                                                                                                                  |                                                         |                                                                | напряжение ядра                                                      |                 |                  |
| Напряжение цепей I/O, В                                  |                                                                                                                      |                                                         |                                                                | 3,3                                                                  |                 |                  |
| Максимальное тепловыделение, Вт                          | 34,5                                                                                                                 | 26,1                                                    | 37,5                                                           | 34,0                                                                 | 32,2            | 22               |

Рис.325 – Технические характеристики Pentium III на разных ядрах

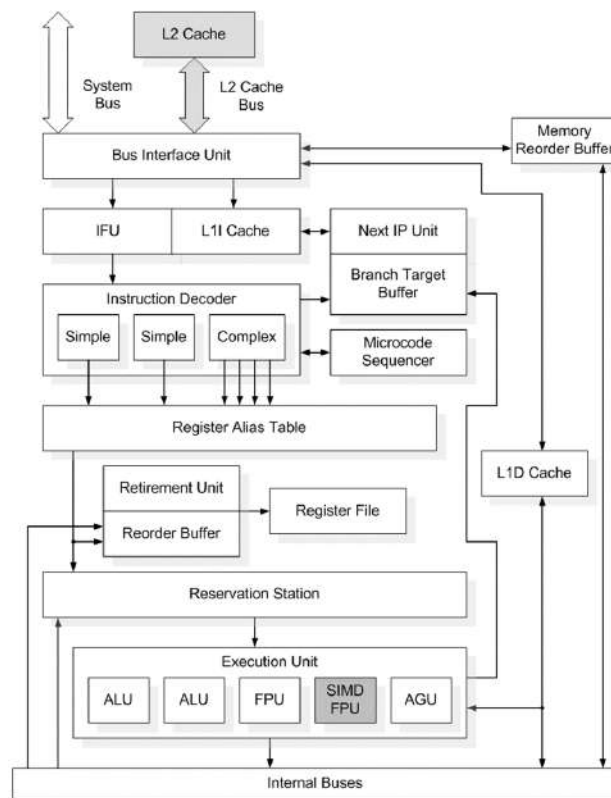


Рис.326 - Функциональная схема процессора Pentium III на ядре Coppermine

### Положение на рынке и сравнение с конкурентами

Pentium III являлся флагманским процессором компании Intel для настольных компьютеров с момента выхода в феврале 1999 года и до появления на рынке процессора Pentium 4 в ноябре 2000 года. После выхода процессора Pentium 4 выпускались процессоры Pentium III на ядре Tualatin, однако широкого распространения они не получили. Параллельно с Pentium III существовали следующие x86-процессоры:

- Intel Celeron (Mendocino). Предназначался для рынка недорогих настольных компьютеров. В момент своего появления, младший в семействе, Celeron 300A при работе в разогнанном режиме на 100 МГц шине позволял получить производительность, сравнимую с самым производительным процессором Pentium II 450. Даже после выпуска процессора Pentium III на ядре Katmai, за счёт полноскоростного кэша, разогнанные Celeron 300A обеспечивали сравнимую производительность в приложениях, не поддерживающих расширения SSE<sup>1</sup>.
- Intel Celeron (Coppermine-128). Предназначался для рынка недорогих настольных компьютеров. Уступал как процессорам Pentium III на ядре Coppermine, так и конкуренту — AMD Duron — в основном за счёт использования медленной системной шины (66/100 МГц против 100/133 МГц у Coppermine и 200 МГц у AMD

<sup>1</sup> SSE (Streaming SIMD Extensions, потоковое SIMD-расширение процессора) — это SIMD (Single Instruction, Multiple Data, одна инструкция — множество данных) набор инструкций, разработанный Intel и впервые представленный в процессорах серии Pentium III как ответ на аналогичный набор инструкций 3DNow! от AMD, который был представлен годом раньше. Первоначально названием этих инструкций было KNI — Katmai New Instructions (Katmai — название первой версии ядра процессора Pentium III).

Технология SSE позволяла преодолеть две основные проблемы MMX: при использовании MMX невозможно было одновременно использовать инструкции сопроцессора, так как его регистры были общими с регистрами MMX, и возможность MMX работать только с целыми числами.

SSE включает в архитектуру процессора восемь 128-битных регистров и набор инструкций, работающих со скалярными и упакованными типами данных.

Duron). Уменьшенный до 128 Кбайт кэш второго уровня также не позволял процессорам Celeron приблизиться к конкурентам.

- Intel Celeron (Tualatin). В отличие от процессоров Pentium III на ядре Tualatin, Celeron, представлявший собой Pentium III с 256 Кбайт кэш-памяти второго уровня и внешней частотой 100 МГц, получил широкое распространение, в частности, среди оверклокеров: так, например, процессоры Celeron 1.0A и 1.1A легко разгонялись до 1333 и 1466 МГц соответственно путём увеличения частоты шины до 133 МГц. Уступал процессорам Pentium III и Pentium III-S за счёт менее скоростной системной шины, увеличенной латентности кэш-памяти второго уровня и меньшего её объёма (в случае с Pentium III-S), во многих задачах также уступал своим конкурентам — AMD Athlon XP и Duron.
- Intel Pentium 4. Флагманский процессор компании Intel с ноября 2000 года. Серьёзно уступал как Pentium III, так и AMD Athlon XP на равных частотах, однако за счёт архитектуры NetBurst имел значительно более высокий частотный потенциал, позволявший получить преимущество в производительности.
- AMD K6-III. Конкурировал с процессорами Pentium III на ядре Katmai. Значительно уступал в подавляющем большинстве задач (за исключением офисных приложений, а также игр, оптимизированных под расширения 3DNow!) из-за устаревшей архитектуры и инфраструктуры. Устанавливался в материнские платы с разъёмом Super Socket 7.
- AMD Athlon (K7). Конкурировал с процессорами Pentium III на ядрах Katmai и Coppermine. Во многих задачах опережал Pentium III, в некоторых — уступал им из-за отсутствия поддержки расширений SSE и менее быстродействующей кэш-памяти второго уровня.
- AMD Athlon (Thunderbird). Конкурировал с процессорами Pentium III на ядре Coppermine. В некоторых задачах опережал Pentium III за счёт архитектурных преимуществ, в некоторых — уступал им из-за отсутствия поддержки расширений SSE и 64-битной шины кэш-памяти (против 256-битной у Coppermine).
- AMD Duron (Spitfire). Предназначался для рынка недорогих настольных компьютеров. Конкурировал с процессорами Intel Celeron на ядре Coppermine, значительно опережая их в большинстве задач, приближаясь к значительно более дорогим Pentium III, работавшим на немного меньшей частоте, а в некоторых задачах опережал их.
- VIA C3. Предназначался для компьютеров с низким энергопотреблением, имел крайне низкую производительность и уступал всем конкурирующим процессорам.
- Transmeta Crusoe. Предназначался для использования в портативных компьютерах. Имел очень низкое энергопотребление, по производительности отставал от равночастотного Pentium III.

### **«Битва за гигагерц»**

К концу 1999 года тактовые частоты процессоров, выпускаемых компаниями Intel и AMD, вплотную приблизились к отметке 1 ГГц. С точки зрения рекламных возможностей, первенство в покорении этой частоты означало серьёзное превосходство над конкурентом, поэтому Intel и AMD прикладывали значительные усилия для преодоления гигагерцового рубежа.

Процессоры Intel Pentium III на тот момент выпускались по 180 нм технологии и имели интегрированный кэш второго уровня, работающий на частоте ядра. Но на частотах, близких к 1 ГГц, интегрированный кэш работал нестабильно.

Процессоры AMD Athlon выпускались по 180 нм технологии и имели внешний кэш, работающий максимум на половине частоты процессора. На частотах, близких к 1 ГГц, использовались большие делители, что позволяло наращивать тактовую частоту собственно процессора.

Это предопределило исход противостояния: 6 марта 2000 года компанией AMD был представлен процессор Athlon, работающий на тактовой частоте 1 ГГц. Кэш-память второго уровня в этом процессоре работала на частоте 333 МГц. Процессор появился в продаже сразу после анонса.

8 марта 2000 года был анонсирован процессор Intel Pentium III 1 ГГц. При этом были пропущены более медленные модели: 850, 866 и 933 МГц, анонсированные позже, 20 и 24 марта. Процессор с тактовой частотой 1 ГГц появился в продаже со значительной задержкой, а анонсированный в июне Pentium III (Coppermine) с частотой 1.13 ГГц был отозван из-за нестабильной работы. Выпуск моделей, надёжно работающих на частотах 1.1 и 1.13 ГГц, оказался возможен лишь в 2001 году после обновления ядра Coppermine (ревизия D0).

### ***Интересные факты***

- Первым суперкомпьютером на базе процессоров Pentium III, вошедшим в список TOP500, стал кластер HpcLine производства Fujitsu Siemens Computers, установленный в Университете Падерборна (Германия). В июне 1999 года он занимал 455 место, а в ноябре — 451-е.
- Дольше всех в списках TOP500 продержался кластер Magi Cluster PIII 933 МГц производства NEC, установленный в Исследовательском центре Цукубы (Япония). Построенный в 2001 году и занявший 39 место в ноябрьском списке 2001 года, он исчез из списков в ноябре 2004 года.
- Наибольшее количество суперкомпьютеров на базе процессоров Pentium III — 38 — присутствует в списке TOP500 за июнь 2002 года.
- Процессор Pentium III 1 ГГц на ядре Coppermine cC0 принял участие в известном видеоролике, снятом в 2001 году Томасом Пабстом и демонстрирующем эффективность термозащиты процессоров. После снятия кулера с работающего процессора, система с процессором Pentium III зависла, однако процессор был своевременно отключён, в то время как AMD Athlon и Athlon MP получили необратимые термические повреждения.
- Процессоры Pentium III 733 МГц на ядре Coppermine с уменьшенной до 128 Кбайт кэш-памятью второго уровня использовались компанией Microsoft в приставке Xbox. В отличие от процессоров Celeron на ядре Coppermine-128, также имеющих 128 Кбайт кэша, данные процессоры имеют 8-канальный ассоциативный кэш второго уровня (Celeron имеет 4-канальный ассоциативный кэш).
- Процессоры Pentium III имели серийные номера, доступные по команде CPUID, кроме того, существовал скрипт, позволяющий читать этот номер дистанционно. Из-за угрозы электронной слежки было предложено запретить Pentium III в Евросоюзе. В Pentium M и Pentium 4 Intel отказался от программно читаемых серийных номеров.

### **Pentium III Xeon**

Intel Pentium III Xeon — 32-битный микропроцессор шестого поколения архитектуры x86, выпущенный компанией Intel 17 марта 1999 года. Предназначался для серверных систем.

Практически повторяет Intel Pentium II Xeon и включает новые возможности: 70 дополнительных инструкций SSE (Streaming SIMD Extensions, именовавшихся также MMX2), ориентированных на поддержку мультимедиа; введены уникальные серийные номера для улучшения сетевой безопасности.



Первый процессор из линейки Pentium III Xeon был построен на базе Pentium III Katmai, ядро называлось Tanner. Содержал 512, 1024 или 2048 Кбайт полноскоростной кэш второго уровня. Кэш первого уровня – 16 Кбайт данные + 16 Кбайт инструкции. Выпускался с рабочими частотами 500 и 550 МГц с применением 0.25 мкм техпроцесса, 9.5 млн транзисторов на кристалле. Системная шина работала на частоте 100 МГц. Выпускался в конструктиве S.E.C.C для Slot 2. Был предназначен для использования в двух-, четырех-, восьмипроцессорных (и более) серверах и рабочих станциях. Как и Pentium II Xeon, третий Xeon монтировался на специальной процессорной плате, которая устанавливалась в тот же разъем Slot 2.

В октябре того же года вышло новое ядро Cascades (аналог Coppermine). С переходом Pentium III на новое ядро 25 октября 1999 года появилась и модификация процессора Xeon с новым ядром Cascades. По сути, это было модернизированное ядро Coppermine. Процессор имел от 256 Кбайт до 2048 Кбайт кэш памяти второго уровня, работал на частотах системной шины 100 и 133 МГц (в зависимости от версии). Выпускались процессоры с частотами от 600 до 900 МГц. Процессоры с частотой 900 МГц из первых партий перегревались и их поставки были временно приостановлены. Как и предшественник, Xeon Cascades был рассчитан на установку в разъем Slot 2. Выпускался по тех.процессу 0.18 мкм, содержал 28.1 млн. транзисторов. Мог работать в двух-, четырех- и восьмипроцессорных конфигурациях.

На базе ядра Tualatin процессоры Xeon не выпускались. Их место занял Pentium III-S. Процессоры Xeon поддерживали наборы команд MMX и SSE.

На этом линия процессоров Pentium III Xeon закончилась. Следующими серверными процессорами от Intel стали Xeon (серверный аналог Pentium 4, март 2001), Pentium III-S (последний серверный процессор с ядром P6, июнь 2001) и Xeon MP (вариант Xeon для четырёхпроцессорных систем, март 2002).

### **Характеристики**

Процессоры Pentium III Xeon были специально предназначены для работы в серверах и рабочих станциях среднего и более высокого уровня с учётом масштабируемости, постоянной доступности и высокой управляемости. Процессор архитектурно совместим с предшествующими кристаллами, в нем используются технологии динамического исполнения и двойной независимой шины DIB, свойственные микроархитектуре P6. Высокая тактовая частота кристалла и системной шины, а также кэш-память большого объёма обеспечивали запас мощности, необходимый для сложных вычислений и интенсивного ввода-вывода. Кэш-память объемом 2 Мбайт, в которой реализованы технологии ATC и ASB, имеет увеличенную ширину шины обмена данными с процессором и обеспечивает меньшее время ожидания.

Встроенная поддержка 4-процессорной конфигурации может быть эффективно расширена до 8-процессорной и более при помощи специализированных наборов микросхем и технологий кластеризации. Благодаря этому можно было создавать недорогие 2-, 4- и 8-процессорные симметричные конфигурации (SMP) и значительно повысить производительность многозадачных ОС и многопоточковых приложений. Архитектура Intel Extended Server Memory Architecture обеспечивает расширенную поддержку 36-разрядности процессора (новый режим PSE-36) совместно с 36-разрядной кэш-памятью и наборами микросхем, поддерживающих 64 Гбайт памяти.

Pentium III Xeon обладали важными особенностями, которые позволяли расширить возможности серверной платформы по управлению и защите процессора и его окружения. Процессоры были заключены в картридж SC330, что позволяло производителям серверов использовать существующие компоненты платформ и обеспечить их быстрое продвижение на рынок. Встроенные в картридж схемы регулирования питающих напряжений и поддержки средств управления увеличивали надёжность систем, а также обеспечивали удалён-



ный контроль и диагностику. Специальный термодатчик, непрерывно отслеживающий температуру ядра и использующий характеристики управляемости платформы, обеспечивал активное управление тепловым режимом и тепловую защиту.

Код с исправлением ошибок (ЕСС) использовался со всеми транзакциями с данными на шинах кэш-памяти второго уровня и системной памяти. Однобитные ошибки исправлялись автоматически, а при любых двухбитных ошибках выдавался сигнал предупреждения. Все ошибки регистрировались, и система может проследить частоту возникновения ошибок, что позволяло определить неисправные компоненты. Полный контроль функциональной избыточности повышает целостность приложений с критичными данными, он осуществлялся проверкой совпадения данных на выходе для двух процессоров и выдаче предупреждающего сигнала при их несовпадении.

Процессор Pentium III Xeon имел несколько дополнительных функций управления. Шина системного управления (SMBus) обеспечивала эффективное взаимодействие между термодатчиком процессора, ПЗУ идентификации процессора, перезаписываемым ПЗУ для данных OEM-производителя и остальной системой.

## **Pentium 4**

20 ноября 2000 года Intel представила новый одноядерный x86-совместимый микропроцессор Pentium 4. В его основе лежала принципиально новая по сравнению с предшественниками архитектура седьмого поколения (по классификации Intel) — NetBurst. Помимо различных вариантов Pentium 4, к процессорам архитектуры NetBurst относятся двухъядерные процессоры Pentium D, а также некоторые процессоры Xeon, предназначенные для серверов. Кроме того, часть процессоров Celeron для систем нижнего ценового уровня представляет собой Pentium 4 с уменьшенным объёмом кэша второго уровня.

Производство процессоров Pentium 4 было начато в 2000 году. С середины 2005 года началось их постепенное вытеснение в нижнюю ценовую категорию двухъядерными процессорами Pentium D. 27 июля 2006 года появились первые процессоры семейства Core 2 Duo, пришедшие на смену процессорам архитектуры NetBurst, а 8 августа 2007 года компания Intel объявила о запуске программы по снятию с производства всех процессоров архитектуры NetBurst.

### ***Особенности архитектуры***

Суперскалярная гиперконвейерная микроархитектура архитектура NetBurst (рабочее наименование — P68), лежащая в основе процессоров Pentium 4, разрабатывалась компанией Intel, в первую очередь, с целью достижения высоких тактовых частот процессоров. На P6 всё никак не удавалось приблизиться к «магическому» 1 ГГц, а Pentium 4 с ходу удалось взять планку 1.3 ГГц. NetBurst не является развитием P6, а представляет собой принципиально новую по сравнению с предшественниками архитектуру. Характерными особенностями архитектуры NetBurst являются гиперконвейеризация и применение кэша последовательностей микроопераций вместо традиционного кэша инструкций. АЛУ процессоров архитектуры NetBurst также имеет существенные отличия от АЛУ процессоров других архитектур.

#### **Гиперконвейеризация**

Процессоры Pentium 4 на ядрах Willamette и Northwood имеют конвейер глубиной 20 стадий, а процессоры на ядрах Prescott и Cedar Mill — 31 стадию (без учёта стадий декодирования инструкций: в связи с применением кэша последовательностей микроопераций, декодер вынесен за пределы конвейера). Это позволяет процессорам Pentium 4 достигать более высоких тактовых частот по сравнению с процессорами, имеющими более короткий конвейер при одинаковой технологии производства. Так, например, максимальная тактовая частота процессоров Pentium III на ядре Coppermine (180 нм технология) составляет 1333

МГц, а процессоры Pentium 4 на ядре Willamette способны работать на частоте, превышающей 2000 МГц.

Основными недостатками длинного конвейера являются уменьшение удельной производительности по сравнению с коротким конвейером (за один такт выполняется меньшее количество инструкций), а также серьёзные потери производительности при некорректном выполнении инструкций (например, при неверно предсказанном условном переходе или кэш-промахе). Можно сказать, что чем длиннее конвейер, тем больше он «не любит» команды ветвлений и переходов, и тем дороже обходятся неверно предсказанные переходы. NetBurst архитектура демонстрировала максимально возможную производительность на длинных линейных участках кода (например, при потоковом кодировании), резко теряя в производительности при большом количестве ветвлений и переходов.

Для минимизации влияния неверно предсказанных переходов, в процессорах архитектуры NetBurst используются увеличенный по сравнению с предшественниками буфер предсказания ветвлений (branch target buffer) и новый алгоритм предсказания ветвлений, что позволило достичь высокой точности предсказания (около 94 %) в процессорах на ядре Willamette. В последующих ядрах механизм предсказания ветвлений подвергался модернизациям, повышавшим точность предсказания.

### **Кэш последовательностей микроопераций (Execution Trace Cache)**

Процессоры архитектуры NetBurst, как и большинство современных x86-совместимых процессоров, являются CISC-процессорами с RISC-ядром: перед исполнением сложные инструкции x86 преобразуются в более простой набор внутренних инструкций (микроопераций), что позволяет повысить скорость обработки команд. Однако, вследствие того, что инструкции x86 имеют переменную длину и не имеют фиксированного формата, их декодирование связано с существенными временными затратами.

В связи с этим, при разработке архитектуры NetBurst было принято решение отказаться от традиционной кэш-памяти инструкций первого уровня, хранящей команды x86, в пользу кэша последовательностей микроопераций, хранящего последовательности микроопераций в соответствии с предполагаемым порядком их исполнения. Такая организация кэш-памяти позволила также снизить временные затраты на выполнение условных переходов и на выборку инструкций.

### **АЛУ и механизм ускоренного выполнения целочисленных операций (Rapid Execution Engine)**

Так как основной идеей разработки архитектуры NetBurst было повышение производительности за счёт достижения высоких тактовых частот, возникла необходимость увеличения темпа выполнения основных целочисленных операций. Для достижения этой цели АЛУ процессоров архитектуры NetBurst разделено на несколько блоков: «медленное АЛУ», способное выполнять большое количество целочисленных операций, и два «быстрых АЛУ», выполняющих только простейшие целочисленные операции (например, сложение). Выполнение операций на «быстрых АЛУ» происходит последовательно в три этапа: сначала вычисляются младшие разряды результата, затем старшие, после чего могут быть получены флаги.

«Быстрые АЛУ», обслуживающие их планировщики, а также регистровый файл синхронизируются по половине такта процессора, таким образом, эффективная частота их работы вдвое превышает частоту ядра. Эти блоки образуют механизм ускоренного выполнения целочисленных операций.

В процессорах на ядрах Willamette и Northwood «быстрые АЛУ» способны выполнять лишь те операции, которые обрабатывают операнды в направлении от младших разрядов к старшим. При этом результат вычисления младших разрядов может быть получен через половину такта. Таким образом, эффективная задержка составляет половину такта. В

процессорах на ядрах Willamette и Northwood отсутствуют блоки целочисленного умножения и сдвига, и данные операции выполняются другими блоками (в частности, блоком инструкций MMX).

В процессорах на ядрах Prescott и Cedar Mill присутствует блок целочисленного умножения, а «быстрые АЛУ» способны выполнять операции сдвига. Эффективная задержка операций, исполняемых «быстрыми АЛУ», возросла по сравнению с процессорами на ядре Northwood и составляет один такт.

### **Система повторного исполнения микроопераций (Replay System)**

Основной задачей планировщиков микроопераций является определение готовности микроопераций к исполнению и передача их на конвейер. Вследствие большого числа стадий конвейера, планировщики вынуждены отправлять микрооперации на исполнительные блоки до того, как завершится выполнение предыдущих микроопераций. Это обеспечивает оптимальную загрузку исполнительных блоков процессора и позволяет избежать потери производительности в том случае, если данные, необходимые для выполнения микрооперации, находятся в кэш-памяти первого уровня, регистровом файле, или могут быть переданы минуя регистровый файл.

При определении готовности новых микроопераций к передаче на исполнительные блоки, планировщику необходимо определить время выполнения тех предыдущих микроопераций, результатом которых являются данные, необходимые для выполнения новых микроопераций. В том случае, если время выполнения заранее не определено, планировщик для его определения использует наименьшее время её выполнения.

Если оценка времени, необходимого для получения данных, оказалась верной, микрооперация выполняется успешно. В том случае, если данные не были получены вовремя, проверка корректности результата заканчивается неудачей. При этом микрооперация, результат выполнения которой оказался некорректен, ставится в специальную очередь (replay queue), а затем вновь направляется планировщиком на исполнение.

Несмотря на то, что повторное исполнение микроопераций приводит к значительным потерям производительности, применение данного механизма позволяет в случае ошибочного исполнения микроопераций избежать остановки и сброса всего конвейера, что приводило бы к более серьёзным потерям производительности.

### **Модели**

Процессор с кодовым именем Willamette впервые появился в официальных планах компании Intel в октябре 1998 года, хотя его разработка началась вскоре после завершения работ над процессором Pentium Pro, вышедшим в конце 1995 года, а название «Willamette» упоминалось в анонсах 1996 года. Необходимость в проектировании нового процессора архитектуры IA-32 появилась в связи со сложностями, возникшими при разработке 64-битного процессора Merced, которому в соответствии с планами компании Intel была отведена роль преемника процессоров архитектуры P6: разработка, осуществлявшаяся с 1994 года, сильно затянулась, а производительность Merced при выполнении инструкций x86 оказалась неудовлетворительной по сравнению с процессорами, для замены которых он предназначался.

Предполагалось, что Willamette выйдет во второй половине 1998 года, однако, в результате многочисленных задержек анонс был перенесён на конец 2000 года. В феврале 2000 года на форуме разработчиков Intel (IDF Spring 2000) был продемонстрирован компьютер, основой которого служил инженерный образец процессора Willamette, получившего наименование «Pentium 4», работающий на частоте 1.5 ГГц.

Первые серийные процессоры Pentium 4 на ядре Willamette, анонсированные 20 ноября 2000 года, производились по 180 нм технологии. Дальнейшим развитием семейства Pentium 4 стали процессоры на ядре Northwood, производившиеся по 130 нм технологии. 2

февраля 2004 года были представлены первые процессоры на ядре Prescott (90 нм), а последним ядром, использовавшимся в процессорах Pentium 4, стало ядро Cedar Mill (65 нм). На базе ядер Northwood и Prescott выпускались также мобильные процессоры Pentium 4 и Pentium 4-M, представлявшие собой Pentium 4 с пониженным энергопотреблением. На базе всех ядер, перечисленных выше, выпускались также процессоры Celeron, предназначенные для бюджетных компьютеров, представлявшие собой Pentium 4 с уменьшенным объёмом кэш-памяти второго уровня и пониженной частотой системной шины.

Ниже, на Рис.327, представлены технические характеристики различных моделей процессоров Pentium 4.

|                                                          | Willamette                                                                                                                     | Northwood                                                        |                   | Gallopia          | Prescott                                                                                               |                     | Prescott 2M                                                              | Cedar Mill |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|-------------------|-------------------|--------------------------------------------------------------------------------------------------------|---------------------|--------------------------------------------------------------------------|------------|
|                                                          | Настольный                                                                                                                     | Настольный                                                       | Мобильный         | Настольный        | Мобильный                                                                                              |                     | Настольный                                                               |            |
| Тактовая частота                                         |                                                                                                                                |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Частота ядра, ГГц                                        | 1.3—2                                                                                                                          | 1.6—3.4                                                          | 1.8—3.2           | 3.2—3.466         | 2.4—3.6                                                                                                | 2.8—3.333           | 2.8—3.6                                                                  | 3—3.6      |
| Частота FSB, МГц                                         | 400                                                                                                                            | 400, 533, 600                                                    | 400, 533          | 800, 1066         |                                                                                                        | 533, 600, 1066 (EE) |                                                                          | 800        |
| Характеристики ядра                                      |                                                                                                                                |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Набор инструкций                                         | IA-32, MMX, SSE, SSE2                                                                                                          |                                                                  |                   |                   | IA-32, EM64T (некоторые модели), MMX, SSE, SSE2, SSE3                                                  |                     |                                                                          |            |
| Разрядность регистров                                    | 32/64 бит (целочисленные), 80 бит (высказывания/плавающие), 64 бит (MMX), 128 бит (SSE)                                        |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Глубина конвейера                                        | 20 стадий (без учёта декодера инструкций)                                                                                      |                                                                  |                   |                   | 31 стадия (без учёта декодера инструкций)                                                              |                     |                                                                          |            |
| Разрядность ША                                           | 36 бит                                                                                                                         |                                                                  |                   |                   | 40 бит                                                                                                 |                     |                                                                          |            |
| Разрядность ШД                                           |                                                                                                                                |                                                                  |                   |                   | 64 бит                                                                                                 |                     |                                                                          |            |
| Аппаратная предвыборка данных                            |                                                                                                                                |                                                                  |                   |                   | есть                                                                                                   |                     |                                                                          |            |
| Количество транзисторов, млн                             | 42                                                                                                                             | 55                                                               |                   | 178               | 125                                                                                                    |                     |                                                                          | 186        |
| Кэш L1                                                   |                                                                                                                                |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Кэш данных                                               | 8 Кбайт, 4-канальный наборно-ассоциативный, длина строки — 64 байта, двухпортовый со сквозной записью                          |                                                                  |                   |                   | 16 Кбайт, 8-канальный наборно-ассоциативный, длина строки — 64 байта, двухпортовый со сквозной записью |                     |                                                                          |            |
| Кэш инструкций                                           | Кэш последовательностей микроопераций, 12 000 микроопераций, 8-канальный наборно-ассоциативный, длина строки — 8 микроопераций |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Кэш L2                                                   |                                                                                                                                |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Объём, Мбайт                                             | ¼                                                                                                                              | ¼                                                                |                   |                   | 1                                                                                                      |                     | 2                                                                        |            |
| Частота                                                  | частота ядра                                                                                                                   |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Разрядность BSB                                          | 256 бит + 32 бит ECC                                                                                                           |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Организация                                              | Объединённый, наборно-ассоциативный, небуферизованный, с контролем и исправлением ошибок (ECC), длина строки — 64 байта        |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Ассоциативность                                          | 8-канальный                                                                                                                    |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Кэш L3                                                   |                                                                                                                                |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Объём, Мбайт                                             |                                                                                                                                |                                                                  |                   |                   | 2                                                                                                      |                     |                                                                          |            |
| Ассоциативность                                          | нет                                                                                                                            |                                                                  |                   |                   | 8-канальный                                                                                            |                     |                                                                          |            |
| Длина строки                                             |                                                                                                                                |                                                                  |                   |                   | 64 байта                                                                                               |                     |                                                                          |            |
| Интерфейсы                                               |                                                                                                                                |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Разъём                                                   | Socket 423, Socket 478                                                                                                         | Socket 478                                                       | Socket 478        | Socket 478        | Socket 478, Socket 775                                                                                 | Socket 478          | Socket 775                                                               |            |
| Корпус                                                   | FCPGA2, FC-mPGA2                                                                                                               | FC-mPGA2                                                         | FC-mPGA, FC-mPGA2 | FC-mPGA, FC-mPGA2 | FC-mPGA2, FC-LGA4                                                                                      | FC-mPGA2, FC-mPGA4  | FC-LGA4                                                                  |            |
| Шина                                                     | AGTL+ (оптимальный уровень равен напряжению ядра)                                                                              |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Технологические, электрические и тепловые характеристики |                                                                                                                                |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Технология производства                                  | 130 нм КМОП (полупроводниковый, алюминиевые соединения)                                                                        | 130 нм КМОП (целостисловый), медные соединения, Low-K диэлектрик |                   |                   | 90 нм КМОП (самостисловый), медные соединения, Low-K, растущий транзистор                              |                     | 65 нм КМОП (восьмисловый), медные соединения, Low-K, растущий транзистор |            |
| Площадь кристалла, мм²                                   | 217                                                                                                                            | 146 (rev. B0)<br>131 (rev. C1, D1, M0)                           |                   |                   | 112                                                                                                    |                     | 135                                                                      | 81         |
| Напряжение ядра, В                                       | 1.7—1.75                                                                                                                       | 1.475—1.65                                                       | 1.3—1.55          | 1.4—1.55          | 1.4—1.425                                                                                              | 1.325               | 1.4—1.425                                                                | 1.2—1.3375 |
| Напряжение цепей I/O                                     | напряжение ядра                                                                                                                |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Напряжение кэш L2                                        |                                                                                                                                |                                                                  |                   |                   |                                                                                                        |                     |                                                                          |            |
| Максимальное тепловыделение, Вт                          | 100                                                                                                                            | 134                                                              | 48.78             | 125.59            | 151.13                                                                                                 | 112                 | 148.18                                                                   | 118.75     |

Рис.327 – Технические характеристики процессоров Pentium 4 на разных ядрах

## Положение на рынке

Процессор Pentium 4 являлся флагманским процессором компании Intel для настольных компьютеров с момента выхода в ноябре 2000 года и до появления на рынке двухъядерных процессоров Pentium D в мае 2005 года. В момент своего выхода процессоры Pentium 4 занимали верхнюю ценовую нишу, а после выхода процессоров Pentium D — среднюю. Pentium 4 продвигался компанией Intel не как универсальный процессор, а как мощный мультимедийный процессор, позволяющий получить максимальную производительность в существующих играх, звуковых и видеоредакторах, а также при работе в сети Internet.

Процессоры Pentium 4 Extreme Edition являлись «имиджевыми» процессорами, а оптовая цена на эти процессоры в момент анонса составляла 999 \$.

Несмотря на то, что в течение года после анонса Pentium 4 основу продаж компании Intel по-прежнему составляли процессоры Pentium III (это было связано с крайне высокой стоимостью систем на базе Pentium 4 в сочетании с памятью типа RDRAM, альтернативы которой не было до выхода набора микросхем (чипсета) Intel 845 осенью 2001 года). Впоследствии, благодаря агрессивной рекламной и маркетинговой политике компании Intel (в том числе, предоставление скидок производителям компьютеров и торговым сетям за ис-

пользование и продажу исключительно продукции Intel, а также выплаты за отказ от использования продукции конкурентов) в сочетании с не всегда удачной маркетинговой политикой основного конкурента — компании AMD — процессоры Pentium 4 стали популярны среди пользователей. Этому также способствовала более высокая тактовая частота процессоров Pentium 4 (в частности, из-за высокой тактовой частоты процессоров конкурента, а также популярности «мифа о мегагерцах», компания AMD была вынуждена ввести рейтинг производительности процессоров Athlon XP, нередко вводивший неопытных пользователей в заблуждение). Тем не менее, компании AMD удалось серьёзно потеснить Intel на рынке микропроцессоров благодаря удачным продуктам — ранним Athlon XP и Athlon 64, превосходившим процессоры Pentium 4 в производительности и имеющим меньшую стоимость. Так, с 2000 по 2001 год компании AMD удалось увеличить свою долю на рынке процессоров архитектуры x86 с 18% до 22% (доля Intel при этом сократилась с 82.2% до 78.7%), а после решения проблем, возникших у AMD в 2002 году, когда её доля на рынке сократилась до 14%, с 2003 по 2006 — до 26% (доля Intel — около 73%).

### **Сравнение с конкурентами**

Параллельно с процессорами семейства Pentium 4 существовали следующие x86-процессоры:

- Intel Pentium III-S (Tualatin). Предназначались для рабочих станций и серверов. Несмотря на меньшую тактовую частоту, по производительности превосходили процессоры Pentium 4 на ядре Willamette в большинстве задач. Кроме того, в отличие от Pentium 4, процессоры Pentium III-S могли работать в двухпроцессорной конфигурации. Также компанией Intel выпускались процессоры Pentium III на ядре Tualatin, отличавшиеся от Pentium III-S меньшим объёмом кэш-памяти второго уровня. Оба этих процессора не получили широкого распространения: они были представлены позже, чем Pentium 4, бывшие в то время флагманскими процессорами компании Intel, и стоили значительно дороже, чем Pentium 4, имеющие сравнимую производительность.
- Intel Celeron (Tualatin). Представляли собой Pentium III с уменьшенным размером кэш-памяти второго уровня (256 Кбайт против 512 Кбайт у Pentium III) и уменьшенной частотой системной шины, предназначались для недорогих систем и в целом уступали процессорам Pentium 4 за счёт меньшей тактовой частоты (старшая модель Celeron работала на частоте 1.4 ГГц, в то время, как младшая модель Pentium 4 — на 1.3 ГГц) и небольшой пропускной способности памяти (в системах на процессорах Celeron обычно использовалась память PC133 SDRAM, а процессоры Pentium 4 чаще всего работали с памятью типа RDRAM или DDR SDRAM) и системной шины (100 МГц против 400 МГц). Производительность разогнанных Celeron была сравнима с производительностью равночастотных Pentium 4 при более низкой цене.
- Intel Celeron (Willamette-128 и Northwood-128), Celeron D (Prescott-256 и Cedar Mill-512). Представляли собой Pentium 4 с уменьшенными частотой системной шины и размером кэш-памяти второго уровня, предназначались для недорогих систем и всегда уступали процессорам Pentium 4. В некоторых задачах Celeron на ядре Willamette-128 уступали также и предшественникам (Celeron на ядре Tualatin) со значительно более низкими частотами.
- Intel Pentium M и Celeron M. Являлись дальнейшим развитием процессоров Pentium III. Предназначались для мобильных компьютеров, обладали низким энергопотреблением и тепловыделением. Pentium M опережал как большинство мобильных Pentium 4М, так и некоторые настольные процессоры Pentium 4, обладая при этом значительно меньшей тактовой частотой и тепловыделением.

Процессор Celeron M имел близкую к Pentium M производительность, незначительно отставая от него.

- Intel Pentium D (Presler, Smithfield). Двухъядерные процессоры, представлявшие собой два ядра Prescott (процессоры на ядре Smithfield) или Cedar Mill (Presler), находящиеся либо на одном кристалле (Smithfield), либо в одном корпусе (Presler). Опережали равночастотные Pentium 4 в большинстве задач. Однако процессоры Pentium 4 имели большую тактовую частоту, чем Pentium D (старшая модель Pentium D на ядре Smithfield работала на частоте 3.2 ГГц, а старшая модель Pentium 4 — на 3.8 ГГц), что позволяло им опережать двухъядерные процессоры в задачах, не оптимизированных под многопоточность.
- AMD Athlon (Thunderbird). Конкурировали с процессорами Pentium 4 на ядре Willamette. В задачах, использующих дополнительные наборы инструкций SSE и SSE2, требующих высокой пропускной способности памяти, а также в приложениях, оптимизированных под архитектуру NetBurst (приложения, работающие с потоковыми данными), процессоры Athlon уступали процессорам Pentium 4, однако в офисных и бизнес-приложениях, задачах трёхмерного моделирования, а также в математических расчётах, процессоры Athlon показывали более высокую производительность.
- AMD Athlon XP. Конкурировали в основном с процессорами Pentium 4 на ядре Northwood. В названиях моделей этих процессоров фигурировала не тактовая частота, а рейтинг, показывающий производительность процессоров Athlon XP относительно Pentium 4. «Равнорейтинговые» Athlon XP уступали процессорам Pentium 4 в приложениях, оптимизированных под архитектуру NetBurst, требовавших наличие поддержки инструкций SSE2 или высокой пропускной способности памяти, однако значительно опережали их в вычислениях с плавающей запятой и неоптимизированных приложениях. Старшие Pentium 4 опережали конкурента в большинстве приложений.
- AMD Athlon 64. Конкурировали в основном с процессорами Pentium 4 на ядре Prescott. Опережали их в ряде задач (например, офисные приложения, научные расчёты или игры) за счёт меньших задержек при работе с памятью (за счёт встроенного контроллера памяти) и более эффективного математического сопроцессора, уступали процессорам Pentium 4 в задачах, оптимизированных под архитектуру NetBurst или имеющих поддержку многопоточности (например, кодирование видео).
- AMD Athlon 64 FX. Конкурировали с процессорами Pentium 4 Extreme Edition. Как и в случае с Athlon 64 и Pentium 4, Athlon 64 FX опережали конкурентов за счёт архитектурных особенностей, интегрированного контроллера памяти или более эффективного математического сопроцессора, уступая им в задачах, оптимизированных под архитектуру NetBurst или имеющих поддержку многопоточности.
- AMD Duron (Morgan и Applebred). Были нацелены на рынок недорогих процессоров и конкурировали с процессорами Celeron, в целом уступая процессорам Pentium 4, однако в некоторых приложениях, которые не были оптимизированы под архитектуру NetBurst и не использовали набор инструкций SSE2, могли опережать Pentium 4, имеющие значительно более высокие тактовые частоты.
- VIA C3 (Nehemiah) и VIA Eden. Предназначались для компьютеров с низким энергопотреблением и ноутбуков (C3 и Eden-N) и для интегрирования в системные платы (Eden), имели низкую производительность и уступали конкурирующим процессорам.
- VIA C7. Также, как и процессоры VIA C3, предназначались для компьютеров с низким энергопотреблением и ноутбуков. Серьёзно уступали конкурентам и

могли опережать их только в задачах шифрования (за счёт наличия аппаратной поддержки).

- Transmeta Efficeon. Предназначались для ноутбуков, имели низкое энергопотребление и тепловыделение. Уступали в большинстве задач мобильным процессорам AMD и Intel, опережая мобильные процессоры VIA.

Работавшие на высокой частоте процессоры Pentium 4 отличались большим энергопотреблением и, как следствие, тепловыделением. Максимальная тактовая частота серийных процессоров Pentium 4 составила 3.8 ГГц, при этом типичное тепловыделение превысило 100 Вт, а максимальное — 150 Вт. Однако при этом процессоры Pentium 4 были лучше защищены от перегрева, чем конкурирующие процессоры. Работа Thermal Monitor — технологии термозащиты процессоров Pentium 4 (а также последующих процессоров Intel) — основана на механизме модуляции тактового сигнала (clock modulation), позволяющем регулировать эффективную частоту работы ядра с помощью введения холостых циклов — периодического отключения подачи тактового сигнала на функциональные блоки процессора («пропуск тактов», «троттлинг»). При достижении порогового значения температуры кристалла, зависящего от модели процессора, автоматически включается механизм модуляции тактового сигнала, эффективная частота снижается (при этом определить её снижение можно либо по замедлению работы системы, либо с помощью специального программного обеспечения, так как фактическая частота остаётся неизменной), а рост температуры замедляется. В том случае, если температура всё же достигает максимально допустимой, происходит отключение системы. Кроме того, поздние процессоры Pentium 4 (начиная с ядра Prescott ревизии E0), предназначенные для установки в разъём Socket 775, обладали поддержкой технологии Thermal Monitor 2, позволяющей снижать температуру путём снижения фактической тактовой частоты (с помощью уменьшения множителя) и напряжения ядра.

Наглядным примером эффективности термозащиты процессоров Pentium 4 стал эксперимент, проведённый в 2001 году Томасом Пабстом. Целью этого эксперимента являлось сравнение эффективности термозащиты процессоров Athlon 1.4 ГГц, Athlon MP 1.2 ГГц, Pentium III 1 ГГц и Pentium 4 2 ГГц на ядре Willamette. После снятия кулеров с работающих процессоров, процессоры Athlon MP и Athlon получили необратимые термические повреждения, а система на Pentium III зависла, в то время как система с процессором Pentium 4 лишь замедлила скорость работы. Несмотря на то, что ситуация с полным отказом системы охлаждения (например, в случае разрушения крепления радиатора), смоделированная в экспериментах, маловероятна, а в случае возникновения приводит к более серьёзным последствиям (например, к разрушению плат расширения или системной платы в результате падения на них радиатора) вне зависимости от модели процессора, результаты эксперимента Томаса Пабста отрицательно повлияли на популярность конкурирующих процессоров AMD, а мнение об их ненадёжности было широко распространено даже после выхода процессоров Athlon 64, имеющих более эффективную по сравнению с предшественником систему защиты от перегрева. К тому же температуры процессоров Intel в данном эксперименте, равные 29 и 37 по Цельсию, вызывают сомнение — ведь это рабочие температуры процессоров Intel при нулевой загрузке ЦПУ, и при наличии штатной системы охлаждения. В эксперименте Томаса Пабста были показаны в гипертрофированном виде имеющие место достоинства процессоров Intel и недостатки процессоров AMD, касающиеся тепловой защиты. Возможно, это была рекламная акция в пользу новых процессоров Intel, особенно учитывая отношение потребителей к первым процессорам Pentium 4 из-за их высокой цены и относительно низкой производительности.

Из-за особенностей архитектуры NetBurst, позволявших процессорам работать на высокой частоте, процессоры Pentium 4 пользовались популярностью среди оверклокеров. Так, например, процессоры на ядре Cedar Mill были способны работать на частотах, превы-



шавших 7 ГГц, с использованием экстремального охлаждения (обычно использовался стакан с жидким азотом), а младшие процессоры на ядре Northwood со штатной частотой системной шины 100 МГц надёжно работали при частоте системной шины 133 МГц и выше.

### ***Некоторые особенности Pentium 4***

По некоторым характеристикам Pentium 4 уступал своим предшественникам, например, этот процессор принципиально мог работать только в однопроцессорном варианте, абсолютно не поддерживая SMP. Чтобы избежать ситуации, случившейся с некоторыми Celeron-ами на базе ядра P-III, которые, как оказалось, можно было заставить работать в двухпроцессорной конфигурации (это была официально не документированная возможность<sup>1</sup>), в P-4 логика, поддерживающая многопроцессорный режим, была полностью «вырезана» из кристалла, и уже никакие умельцы в принципе не могли заставить работать P-4 в двухпроцессорной конфигурации.

Из технологий, разработанных Intel для процессоров на микроархитектуре NetBurst, следует отметить широко рекламировавшуюся **Hyper-Threading** (гиперпоточность, официальное название — *hyper-threading technology*, *HTT* или *HT*). HTT реализует идею «одновременной многопоточности» (*simultaneous multithreading*, *SMT*) и является развитием технологии суперпоточности (*super-threading*), появившейся в процессорах Intel Xeon в феврале 2002. В ноябре 2002 года HT была добавлена в процессоры Pentium 4. После включения HT один физический процессор (**одно** физическое ядро) определяется операционной системой как два отдельных процессора (**два** логических ядра). При определённых рабочих нагрузках использование HT позволяет увеличить производительность процессора. Суть технологии: передача «полезной работы» (*useful work*) бездействующим исполнительным устройствам (*execution units*).

Процессор, поддерживающий технологию hyper-threading, может хранить состояние сразу двух потоков и при этом содержит по одному набору регистров и по одному контроллеру прерываний (APIC) на *каждый* из двух *логических* процессоров, но имеет *один* комплект исполнительных устройств, что отличает HT от настоящего двухъядерного процессора.

Технологию HTT Intel не стала реализовывать в процессорах следующих поколений, серии Core 2 («Core 2 Duo», «Core 2 Quad»). В процессорах Core i3, Core i7 и некоторых Core i5 была реализована сходная по своим принципам технология, сохранившая то же название hyper-threading.

### **64-битное расширение архитектуры x86**

К 2000-му году стало понятно, что классическая архитектура x86 уже вплотную подобралась к своим 32-разрядным пределам. Казавшееся когда-то огромным адресное пространство  $2^{32} = 4$  Гбайт всё чаще оказывалось недостаточным, особенно если учитывать особенности распределения физического адресного пространства в x86 — даже в теории 32-разрядной ОС все 4 Гбайт были в принципе не доступны, в лучшем случае доступный объём мог составлять величину максимум 3,0...3,3 Гбайт.

64-битные системы на то время не были чем-то невероятным и экзотическим, на рынке давно присутствовали 64-разрядные микропроцессоры других производителей: R4000 (MIPS Technologies, 1991 год), Alpha (DEC, 1992 год), UltraSPARC (Sun, 1995 год). Все эти процессоры были поддержаны соответствующими 64-разрядными операционными системами и прикладным программным обеспечением.

Intel, чтобы не оказаться последней в этой гонке, в 1994 объявляет о планах по созданию совместно с Hewlett-Packard 64-разрядной архитектуры IA-64, которую они тогда

---

<sup>1</sup> Умельцы нашли способ заставить работать Celeron в двухпроцессорной конфигурации: для этого нужно было: а) изолировать один вывод на каждом процессоре; б) пробросить дополнительный проводник между определёнными выводами процессоров.

планировали на замену IA-32 (32-битный x86) и PA-RISC (микропроцессор, разработанный Hewlett-Packard). Датой выхода были обозначены 1998-1999 года.

Но с разработкой IA-64 не задалось – только в 2001 году, сорвав все первоначальные сроки, Intel наконец запускает линейку 64-битных процессоров, которые были рассчитаны на высокопроизводительные серверы, назвав процессор Itanium. Проект не соответствует ожиданиям из-за многочисленных задержек при выпуске IA-64 на рынок. NetBSD становится первой операционной системой, которая была запущена на процессоре Intel Itanium после его выхода. Кроме того, Microsoft также выпускает Windows XP 64-Bit Edition для архитектуры IA-64 семейства Itanium, хотя в ней сохраняется возможность запускать 32-битные приложения при помощи прослойки WoW64.

Компания AMD, осознав факт, что Intel видит в качестве 64-разрядного будущего иную, не x86, архитектуру, решает идти условно эволюционным путём, подобным тому, что проделали в своё время в Intel, при переходе от 16-разрядного i80286 к 32-разрядному i80386. В 1999 году AMD публично объявляет о своём наборе 64-битных расширений для IA-32, который был назван x86-64 (позже переименован в AMD64).

## Архитектура AMD64

Архитектура AMD x86-64 простое, но в то же время мощное обратно-совместимое расширение стандартной промышленной архитектуры x86. Она добавляет 64-битное адресное пространство и расширяет регистровые ресурсы для достижения большей производительности для перекомпилированных 64-битных программ, обеспечивая поддержку устаревшего 16-битного и 32-битного кода приложений и операционных систем без их модификации или перекомпиляции.

Необходимость 64-битной x86 архитектуры определяется приложениями, которым требовалось всё большее адресное пространство. Это, в первую очередь, высокопроизводительные серверы, системы управления базами данных, САД-системы и, конечно, игры. Такие приложения получают преимущество от 64-битного адресного пространства и увеличения количества регистров. Малое количество регистров, доступное в устаревшей 32-битной x86 архитектуре, ограничивает производительность в вычислительных задачах.

В архитектуре AMD64 были введены расширенные (64 бита) регистры общего назначения и увеличено их количество с 8 до 16. Также было добавлено 8 новых 128-битных XMM регистров. Для доступа к расширенным регистрам использовался префикс REX. На Рис.328 приведён набор регистров, доступных в AMD64.

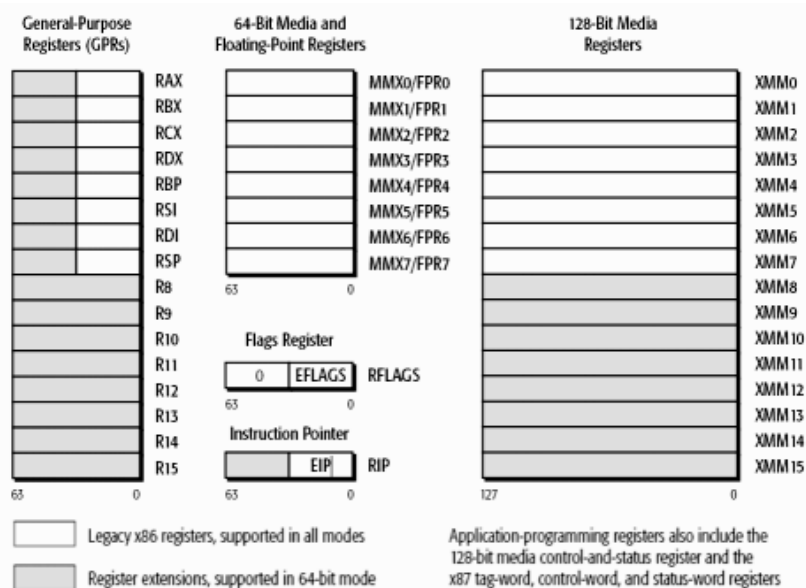


Рис.328 – Набор регистров x86-64

- 16 целочисленных 64-битных регистра общего назначения (RAX, RBX, RCX, RDX, RBP, RSI, RDI, RSP, R8 — R15);
- 8 80-битных регистров с плавающей точкой (ST0 — ST7);
- 8 64-битных регистров Multimedia Extensions (MM0 — MM7, имеют общее пространство с регистрами ST0 — ST7);
- 16 128-битных регистров SSE (XMM0 — XMM15);
- 64-битный указатель RIP и 64-битный регистр флагов RFLAGS.

Для адресации новых регистров для команд введены так называемые «префиксы расширения регистра», для которых был выбран диапазон кодов 0x40-0x4F, использующихся для команд INC <регистр> и DEC <регистр> в 32-битных режимах. Команды INC и DEC в 64-битном режиме должны кодироваться в более общей, двухбайтовой форме.

### Режимы работы

Процессорные архитектуры AMD64 поддерживают два режима работы: Long mode («длинный» режим) и Legacy mode («унаследованный», режим совместимости с 32-битным x86). Ниже (Рис.329) приведена таблица, в которой эти режимы описаны:

| Operating Mode |                    | Operating System Required | Application Recompile Required | Defaults            |                     | Register Extensions | Typical          |
|----------------|--------------------|---------------------------|--------------------------------|---------------------|---------------------|---------------------|------------------|
|                |                    |                           |                                | Address Size (bits) | Operand Size (bits) |                     | GPR Width (bits) |
| Long Mode      | 64-Bit Mode        | New 64-bit OS             | yes                            | 64                  | 32                  | yes                 | 64               |
|                | Compatibility Mode |                           | no                             | 32                  | no                  | 32                  |                  |
|                |                    |                           |                                | 16                  |                     | 16                  | 16               |
| Legacy Mode    | Protected Mode     | Legacy 32-bit OS          | no                             | 32                  | 32                  | no                  | 32               |
|                |                    |                           |                                | 16                  | 16                  |                     |                  |
|                | Virtual-8086 Mode  | 16                        |                                | 16                  | 16                  |                     |                  |
|                | Real Mode          | Legacy 16-bit OS          |                                |                     |                     |                     |                  |

Рис.329 – Режимы работы процессора

Таблица на Рис.330 содержит сравнение ресурсов регистров и стека, доступных приложению в различных режимах. Левые колонки показывают ресурсы устаревшей x86 архитектуры, которые доступны только для совместимости. Правые колонки показывают ресурсы доступные в 64-битном режиме. Серым цветом выделено отличие между режимами.

| Register or Stack                | Legacy and Compatibility Modes         |        |             | 64-Bit Mode                                    |        |             |
|----------------------------------|----------------------------------------|--------|-------------|------------------------------------------------|--------|-------------|
|                                  | Name                                   | Number | Size (bits) | Name                                           | Number | Size (bits) |
| General-Purpose Registers (GPRs) | EAX, EBX, ECX, EDX, EBP, ESI, EDI, ESP | 8      | 32          | RAX, RBX, RCX, RDX, RBP, RSI, RDI, RSP, R8-R15 | 16     | 64          |
| 128-Bit XMM Registers            | XMM0-XMM7                              | 8      | 128         | XMM0-XMM15                                     | 16     | 128         |
| 64-Bit MMX Registers             | MMX0-MMX7                              | 8      | 64          | MMX0-MMX7                                      | 8      | 64          |
| x87 Registers                    | FPR0-FPR7                              | 8      | 80          | FPR0-FPR7                                      | 8      | 80          |
| Instruction Pointer              | EIP                                    | 1      | 32          | RIP                                            | 1      | 64          |
| Flags                            | EFLAGS                                 | 1      | 32          | RFLAGS                                         | 1      | 64          |
| Stack                            | —                                      |        | 16 or 32    | —                                              |        | 64          |

Рис.330 - Регистры и стек, доступные в различных режимах

## **Long mode**

«Длинный» режим — «родной» для процессоров AMD64. Этот режим даёт возможность воспользоваться всеми дополнительными преимуществами, предоставляемыми архитектурой AMD64. Для использования этого режима необходима 64-битная операционная система, например, Windows Server 2003/2003R2/2008/2008R2/2012/2016/2019, Windows XP Professional x64 Edition, Windows Vista x64, Windows 7/8/8.1/10 x64 или 64-битные варианты UNIX-подобных систем GNU/Linux, FreeBSD, OpenBSD, NetBSD (чистые 64-битные сборки, однако, есть возможность запуска 32-битных приложений), Solaris (смешанная 32/64 сборка с разными ядрами для 32- и 64-битных процессоров), Mac OS X (смешанная 32/64 сборка с 32-битным ядром, начиная с версии 10.4.7).

Этот режим позволяет выполнять 64-битные программы; также (для обратной совместимости) предоставляется поддержка выполнения 32-битного кода, например, 32-битных приложений, хотя 32-битные программы не смогут использовать 64-битные системные библиотеки, и наоборот. Чтобы справиться с этой проблемой, большинство 64-разрядных операционных систем предоставляют два набора необходимых системных файлов: один — для родных 64-битных приложений, и другой — для 32-битных программ. (Этой же методикой пользовались ранние 32-битные системы — например, Windows 95 — для выполнения 16-битных программ.)

В «длинном» режиме упразднён ряд «рудиментов» архитектуры x86, таких, как режим виртуального 8086, сегментированная модель памяти (однако, осталась возможность использования сегментов FS и GS, что полезно для быстрого нахождения важных данных потока при переключении задач), аппаратная мультизадачность, а также ряд команд, как реализующих упразднённые возможности, так и работающие с BCD-числами, которые в новых программах практически не использовались. Среди особенностей «длинного» режима следует отметить тот факт, что он активируется установкой флага CR0.PG, который используется для включения страничного MMU (при условии, что такое переключение разрешено (EFER.LME=1), в противном случае просто произойдёт включение MMU в «унаследованном» режиме). Таким образом, невозможно исполнение 64-битного кода с запрещённым страничным преобразованием. Это создаёт определённые трудности в программировании, поскольку при переключении из «длинного» в «унаследованный» режим и обратно (например, для вызова функций BIOS или DOS, монитором виртуальной машины, и т. д.) требуется двойной сброс MMU, для чего код переключения должен находиться в тождественно отображённой странице.

## **Legacy mode**

Данный «унаследованный» режим позволяет процессору AMD64 выполнять инструкции, рассчитанные для процессоров x86, и предоставляет полную совместимость с 32-битным кодом и операционными системами. В этом режиме процессор ведёт себя точно так же, как x86-процессор, например, Athlon или Pentium III, и дополнительные функции, предоставляемые архитектурой AMD64 (например, дополнительные регистры), в нём недоступны. В этом режиме 64-битные программы и операционные системы работать не будут.

### **Адресное пространство**

Хотя 64-битный процессор теоретически может адресовать 16 экзбайт памяти ( $2^{64}$ ), Win64 в настоящий момент поддерживает 16 терабайт ( $2^{44}$ ). Этому есть несколько причин. Текущие процессоры могут обеспечивать доступ лишь к 1 терабайту ( $2^{40}$ ) физической памяти. Архитектура (но не аппаратная часть) может расширить это пространство до 4 петабайт ( $2^{52}$ ). Однако в этом случае необходимо огромное количество памяти для страничных таблиц, отображающих память. Помимо перечисленных ограничений, объем памяти, который доступен в той или иной версии 64-битной операционной системы Windows зависит также от коммерческих соображений компании Microsoft.

## **AMD64 – а что же Intel?**

На работу в Intel, продолжавшую увлечённо разрабатывать, совместно с Hewlett-Packard, «единственно верную» 64-битную архитектуру Merced/Itanium, в какой-то момент перешли все разработчики этой архитектуры из HP. Просто каждая компания решила сосредоточиться на том, что она умеет лучше всего: Intel – на процессорах, HP – на серверах на их основе.

Но Itanium оказался очень дорогим долгостроем, так и не оправдавшим полностью возлагавшиеся на него надежды. Основой архитектуры IA-64 была концепция, которая до этого не использовалась ни в одном массовом процессоре, т.н. EPIC. «Родного» 64-битного программного обеспечения в необходимом количестве для Itanium разработано в итоге так и не было, а как эмулятор системы команд IA-32 он уступал в производительности настоящим x86, будучи при этом существенно более дорогим устройством. Интересно, что какое-то время процессор Itanium-2 удерживал мировой рекорд по количеству элементов на кристалле – около 1,75 млрд транзисторов. И, очень важно, что IA-64 всегда позиционировалась, как процессор для «тяжёлых» решений – серверы, рабочие станции. На массовые десктопы и, тем более, ноутбуки, этот монстр никогда и не рассматривался. А ведь 64-битная x86-64 сейчас мейнстрим и на десктопах, и в серверах, и на ноутбуках.

Осознав, что стратегическая ставка исключительно на IA-64 оказалась ошибочной, Intel была вынуждена признать и принять подход AMD, разработав собственную, бинарно практически полностью совместимую с AMD64 технологию под названием EM64T. Здесь двум основным игрокам на поле x86 удалось договориться, но Intel-у впервые пришлось сыграть «вторым номером», лицензируя у стороннего производителя решения для архитектуры, которую они сами когда-то и породили. В немного другой области – аппаратной поддержке виртуализации, Intel и AMD имеют собственные, бинарно не совместимые решения, благо, что оба варианта были поддержаны основными игроками на рынке средств виртуализации.

Первенство AMD в 64-битных x86 архитектурах зафиксировано в названии многих дистрибутивов операционных систем, например, Windows и Linux. Дистрибутивы так и называются – amd64. Некоторых такое название сбивает с толку, и они считают, что эта версия не будет работать на их процессорах Intel, часто ошибочно пытаясь на 64-разрядную интеловскую платформу установить дистрибутив ia64, который не имеет к архитектуре x86 ни малейшего отношения (это версия для Itanium).

Набор команд x86-64 в настоящее время поддерживается:

- AMD с 2001 года — процессорами Z-серии (например, AMD Z-03), C-серии (например, AMD C-60), G-серии (например, AMD T56N), E-серии (например, AMD E-450), E1, E2, A4, A6, A8, A10, FX, Athlon 64, Athlon 64 FX, Athlon 64 X2, Athlon II, Phenom, Phenom II, Turion 64, Turion 64 X2, Turion II, Opteron, FX, Ryzen, Ryzen Threadripper последними моделями Sempron;
- Intel (с незначительными упрощениями) под названием «Intel 64» (ранее известные как «EM64T» и «IA-32e») с 2003 года в моделях процессоров Pentium 4, а также в Pentium D, Pentium Extreme Edition, Celeron D, Celeron G-серии, Celeron V-серии, Pentium Dual-Core, Pentium T-серии, Pentium P-серии, Pentium G-серии, Pentium B-серии, Core 2 Duo, Core 2 Quad, Core 2 Extreme, Core i3, Core i5, Core i7, Core i9, Atom (далеко не всеми, но большинством последних) и Xeon;
- VIA — процессорами Nano.

В частности, можно отметить, что поддержка EM64T была добавлена и в самый массовый на тот момент интеловский десктопный процессор Pentium 4 (начиная с некоторых моделей Prescott).

## Организация памяти в P5...P7

В архитектуре P5 и далее Intel сохранила трёхступенчатую схему преобразования логических адресов в физические: логический адрес(1) преобразуется в линейный(2), а тот, в свою очередь в физический(3), используя механизм страничного преобразования. В MMU были сделаны некоторые модификации, например, появилась возможность изменения размера страницы – Page Size Extension (PSE), страница теперь могла иметь размер отличный от умолчательных 4КБ. Эта опция была заложена ещё в первых Pentium-ах, но была задокументирована только с выходом процессора Pentium Pro (P6). Наличие расширения PSE можно проверить с помощью инструкции CPUID.

Главной мотивацией увеличения размера страницы было сокращение накладных расходов на обслуживание каталогов страниц и таблиц страниц. К примеру, прикладная программа запрашивает блок памяти размером 1 МБ. Для выполнения этого запроса операционная система, поддерживающая разбиение на страницы и работающая на предыдущих моделях процессоров x86, должна будет выделить 256 страниц по 4 КБ каждая. Для обслуживания каталогов страниц и таблиц страниц ей потребуются дополнительная память в размере 1 КБ памяти (накладные расходы, 4 байта на каждую из 256 страниц в таблице страниц).

При доступе к этой памяти размером 1 МБ каждая из 256-страничных записей будет кэшироваться в резервном буфере трансляции (TLB). Загромождение TLB большим количеством записей, возможно, является одним из самых больших недостатков малого размера страницы (вместо одной записи в каталоге, описывающей одну большую страницу, мы имеем большое количество записей, описывающих множество мелких страниц). Большое количество записей в TLB приводит к её быстрому заполнению (переполнению кэша), что серьёзно сказывается на общей производительности при активном доступе к памяти.

Ниже, на Рис.331, представлены действия, которые выполняет MMU при преобразовании виртуального адреса в физический. Блок управления памятью ЦП (MMU) хранит кэш недавно использованных сопоставлений из таблицы страниц операционной системы в резервном буфере трансляции (TLB, ассоциативный кэш).

Когда виртуальный адрес необходимо преобразовать в физический, сначала выполняется поиск в TLB. Если совпадение найдено (TLB hit), возвращается физический адрес и доступ к памяти продолжается. Однако, если совпадения нет (TLB miss), модуль управления памятью или обработчик промахов TLB операционной системы обычно просматривает сопоставление адресов в таблице страниц (page table), чтобы узнать, существует ли такое сопоставление (обход страниц, page walk). Если сопоставление существует, оно записывается обратно в кэш (TLB) (это необходимо сделать, поскольку оборудование в системе виртуальной памяти обращается к памяти через TLB), и инструкция, вызвавшая сбой (промах), перезапускается (это может происходить параллельно). Последующая трансляция обнаружит попадание в TLB (hit), и доступ к памяти будет возобновлён.

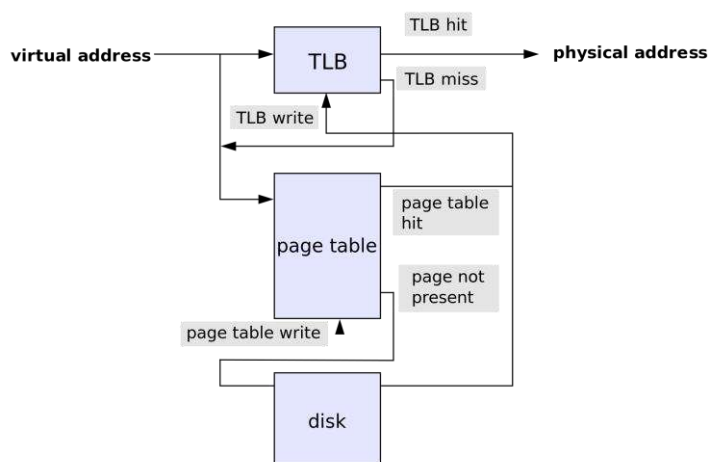


Рис.331 - Действия, выполняемые при преобразовании виртуального адреса в физический.

PSE позволяет увеличить размер страницы до 4 МБ. Описанный выше запрос в 1 МБ можно было легко выполнить с помощью всего одной страницы размером 4 МБ и для этого потребовалась бы только одна запись в TLB. Однако недостатком использования страниц большого размера является внутренняя фрагментация, в данном примере теряется 3 МБ.

Чтобы система и программы, запущенные в ней, могли использовать страницы большого размера, технология 4 МБ страниц (hugepages) должна быть соответствующим образом активирована, а приложение настроено на использование страниц такого размера.

Число записей в одной таблице страниц (page table) ограничено и зависит от размера записи и размера страницы. В x86 используется многоуровневая организация таблиц, обычно 2 или 3 уровня (или даже 4 уровня для 64-разрядных архитектур).

В случае двух уровней используется каталог страниц, в котором хранятся записи, указывающие на физические адреса таблиц страниц. В таблицах содержатся записи, указывающие на страницы данных. Старшие биты виртуального адреса указывают на номер записи в каталоге, средние — номер записи в таблице, младшие (адрес внутри страницы) попадают в физический адрес без трансляции.

Исторически 32-разрядные x86 использовали 32-битные PTE (Page Table Entry), 32-битные виртуальные адреса, 4 КБ страницы, 1024 записи в таблице, двухуровневые таблицы. Старшие 10 битов виртуального адреса — номер записи в каталоге, следующие 10 — номер записи в таблице, младшие 12 — адрес внутри 4 КБ страницы (Рис.332).

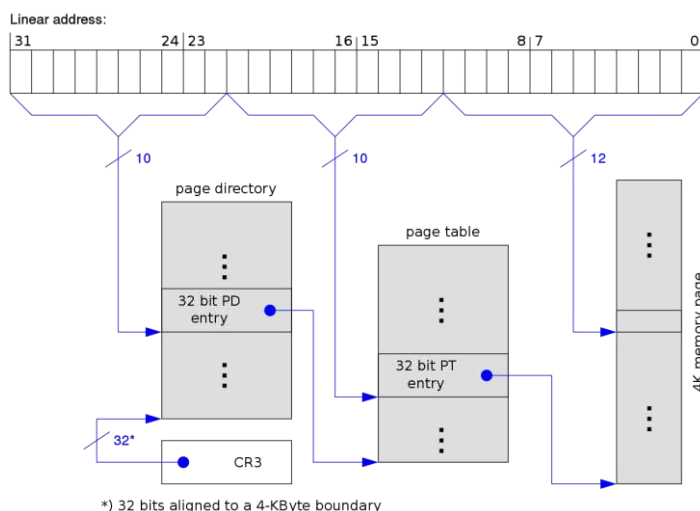


Рис.332 - Двухуровневая структура таблицы страниц в архитектуре x86 (без PAE или PSE)

При использовании трёхуровневой организации добавляется надкаталог, хранящий записи, указывающие на несколько каталогов (Рис.333).



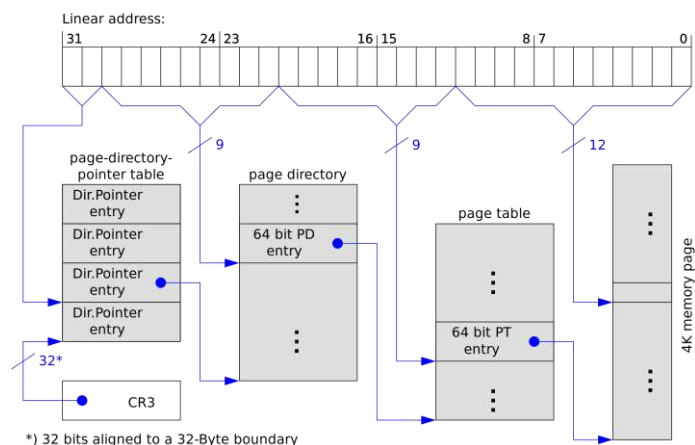


Рис.333 - Трёхуровневая структура таблицы страниц в архитектуре x86 (с PAE, без PSE).

Процессор x86 в режиме PAE (Physical Address Extension) и в режиме x86-64 (long mode) использует 64-битные PTE (из них реально задействованы не все биты физического адреса, от 36 в PAE до 48 в некоторых x86-64), 32-битные виртуальные адреса, 4 КБ страницы, 512 записей в таблице, трёхуровневые таблицы с четырьмя каталогами и четырьмя записями в надкаталоге. Старшие 2 бита виртуального адреса определяют номер записи в надкаталоге, следующие 9 — в каталоге, следующие 9 — в таблице. Физический адрес каталога или же надкаталога загружен в один из управляющих регистров процессора.

При использовании PAE вместо 4 МБ страниц используются страницы размером 2 МБ. Всего в архитектуре x86-64 возможно использование страницы размером 4 КБ, 2 МБ, 4 МБ и (в некоторых AMD64) 1 ГБ.

## Отказ страницы (page fault)

Если обращение к памяти не может быть оттранслировано через TLB, то микрокод процессора обращается к таблицам страниц и пытается загрузить PTE оттуда в TLB. Если и после такой попытки сохранились проблемы, то процессор исполняет специальное прерывание (исключение), называемое «отказ страницы» (page fault). Обработчик этого прерывания находится в подсистеме виртуальной памяти ядра ОС.

Отказ страницы (page fault) — разновидность аппаратного исключения, возникающего в компьютере с виртуальной памятью на основе подкачки страниц в момент обращения к странице памяти, которая не включена блоком управления памятью в виртуальное адресное пространство процесса. Обращение к этой странице может быть логически допустимым, но может потребовать её добавления в таблицу страниц и, возможно, загрузки её содержимого из вторичного хранилища (жёсткого диска) в оперативную память. Обнаружение отказа страницы выполняет аппаратное обеспечение (блок управления памятью, MMU), а его обработку выполняет программное обеспечение, обычно являющееся частью ядра операционной системы. При обработке отказа страницы операционная система пытается поместить требуемую страницу в оперативную память и сделать доступной процессу, либо завершает программу в случае недопустимого обращения к памяти.

Несмотря на название, отказы при обращении к допустимым страницам являются нормальной частью функционирования любой операционной системы, использующей виртуальную память, например, OpenVMS, Microsoft Windows и UNIX-подобных системы. Фактически, они являются просто сигналами, необходимыми операционной системе для выделения памяти работающим программам.

### Типы отказа страницы

#### Легкий или программный

Отказ страницы, который в Linux-подобных системах называется *лёгким* (*minor*), а в Windows — *программным* (*soft*), означает, что требуемая страница либо уже находится в оперативной памяти, но не отмечена в блоке управления памятью как загруженная, либо она вновь выделена и ещё ни разу не использовалась. Обработчик исключения в операционной системе должен только добавить запись в таблицу страниц для требуемой страницы и указать, что она загружена в память. Загрузка страницы с диска не требуется. Отказ происходит в следующих случаях:

- страница присутствует в памяти, но включена в рабочее множество другого процесса (например, если несколько процессов взаимодействуют через разделяемую память)
- страница находится в промежуточном состоянии, потому что она либо была исключена из рабочих множеств всех процессов и ожидает записи на диск, либо была загружена ранее (например, компонентом Prefetcher)
- процесс обращается к вновь выделенной странице в первый раз

Поскольку обработка таких отказов не сопровождается задержкой для обращения к диску, они обрабатываются быстрее *значительных* или *аппаратных* отказов.

### **Значительный или аппаратный**

Отказ страницы, который в Linux-подобных системах называется *значительным* (*major*), а в Windows — *аппаратным* (*hard*), является основным механизмом, используемым операционной системой для выделения программам памяти по их запросу. Операционная система откладывает загрузку частей программы с диска до тех пор, пока программа не попытается получить доступ к ним и тем самым сгенерирует отказ страницы. Если страница не загружена в память на момент отказа, тогда отказ называется значительным или аппаратным. Обработчик отказа страницы в операционной системе должен найти свободное место в оперативной памяти: свободную либо занятую страницу. Занятая страница может принадлежать другому процессу. В этом случае операционная система должна выгрузить данные этой страницы на диск (если они не были выгружены ранее) и пометить эту страницу в таблице страниц процесса как отсутствующую в памяти. Как только свободное место становится доступным, операционная система может загрузить данные для новой страницы в память, добавить её физический адрес в таблицу страниц исходного процесса и пометить страницу, как находящуюся в памяти. Необходимость обращения к диску делает обработку таких отказов гораздо более медленной по сравнению с *лёгкими*.

### **Обращение к недействительной странице**

Если отказ страницы возникает при обращении к адресу, не принадлежащему виртуальному адресному пространству процесса, то есть страницы в памяти, соответствующей этому адресу, не может быть в принципе, тогда этот отказ называется отказом недействительной страницы (*invalid page fault*). Обработчик отказа страницы в операционной системе в этом случае, как правило, передает ошибку сегментации исходному процессу, показывая, что обращение было недействительным. Это приводит к аварийному завершению блока кода, выполнившего недействительное обращение. Примером недействительного обращения является разыменование нулевого указателя. Нулевой указатель — это указатель, не ссылающийся ни на какой объект. Обычно он представляется как указатель на адрес 0. Большинство операционных систем не включают страницу, содержащую нулевой адрес, в адресное пространство процесса, поэтому попытки чтения или записи памяти через нулевой указатель приводят к отказу недействительной страницы.

### **Причины отказа страницы в архитектуре x86**

- не существует таблицы, отображающей данный регион,
- PTE не имеет взведённого флага «страница отображена»,
- попытка обратиться из пользовательского режима к странице «только для ядра»,
- попытка записи в страницу «только для чтения»,
- попытка исполнения кода из страницы «исполнение запрещено».

Обработчик отказов в ядре может загрузить нужную страницу из файла или же из области подкачки (свопинг), может создать доступную на запись копию страницы «только для чтения», а может возбудить исключительную ситуацию (в терминах UNIX — сигнал SIGSEGV) в данном процессе.

Каждый процесс имеет свой собственный набор таблиц страниц. Регистр «каталог страниц» перегружается при каждом переключении контекста процесса. Также необходимо сбросить ту часть TLB, которая относится к данному процессу.

В большинстве случаев ядро ОС помещается в то же адресное пространство, что и процессы, для него резервируются верхние 1-2 гигабайта 32-битного адресного пространства каждого процесса. Это делается с целью избежать переключения таблиц страниц при входе в ядро и выходе из него. Страницы ядра помечаются как недоступные для кода режима пользователя.

Память региона ядра часто совершенно одинакова для всех процессов, однако некоторые подрегионы региона ядра (например, область Windows, где находится подсистема графики и драйвер видео) могут быть различными для разных групп процессов (сессий).

Так как память ядра одинакова у всех процессов, соответствующие ей TLB не нужно перегружать после переключения процесса. Для этой оптимизации x86 поддерживает флаг «глобальный» у PTE.

## Система команд

Поколение P5 и далее поддерживало весь набор команд процессоров предыдущих поколений, собственно, этот набор уже был синонимом интеловской архитектуры, которую стало принято обозначать как x86.

При разработке новых поколений процессоров Intel активно расширяла набор поддерживаемых процессором инструкций. Иногда это происходило внутри одного поколения, например, процессоры Pentium и Pentium MMX формально относились к одному и тому же пятому поколению P5, но система команд для второго из них была расширена 57 инструкциями класса MMX.

Часто увеличение набора поддерживаемых инструкций сопровождалось расширением аппаратной части процессора, например, появлением новых регистров. Собственно, новые инструкции обычно и добавлялись к основной системе команд для получения возможности работать с новым «железом».

Ниже будут перечислены некоторые основные расширения, которые в разное время были добавлены в базовую x86 архитектуру.

### PAE

В более поздних 32-разрядных процессорах (начиная с Pentium Pro) появилось PAE (Physical Address Extension) — расширение адресов физической памяти до 36 бит (возможность адресации 64 Гбайт ОЗУ). Это изменение не затронуло разрядности задач — они остались 32-битными.

### MMX

Дополнительный «мультимедийный» (Multi-Media eXtensions) набор инструкций, выполняющих по несколько характерных для процессов кодирования/декодирования потоковых аудио/видеоданных действий за одну машинную инструкцию (SIMD). Впервые появился в процессорах Pentium MMX. Обеспечивает только целочисленные вычисления.

## **SSE**

SSE (Streaming SIMD Extensions — потоковое SIMD-расширение) — SIMD (Single Instruction, Multiple Data — «одна инструкция — множество данных») набор инструкций, разработанный Intel и впервые представленный в процессорах серии Pentium III. Поддерживает вычисления с плавающей точкой. SSE состоит из восьми 128-битных регистров (от xmm0 до xmm7). Каждый регистр определяет четыре последовательных значения с плавающей точкой одинарной точности. SSE включает в себя инструкции, которые производят операции со скалярными и упакованными типами данных.

## **SSE2**

Улучшенное расширение SSE. Появилось в процессорах Pentium 4. Производит потоковые вычисления с вещественными числами двойной точности (2 числа по 64 бита в одном регистре SSE). Кроме того, добавлены инструкции, аналогичные расширению MMX, работающие с регистрами SSE (16 байт, 8 слов, 4 двойных слова или 2 учетверённых слова в одном регистре). SSE2 включает в себя ряд команд управления кэшем, предназначенных для минимизации «загрязнения» кэша при обработке неопределённых потоков информации.

## **SSE3**

Продолжение SSE и SSE2, появилось в процессорах Prescott (Pentium 4). Набор SSE3 содержит 13 инструкций: FISTTP (x87), MOVSLDUP (SSE), MOVSHDUP (SSE), MOVDDUP (SSE2), LDDQU (SSE/SSE2), ADDSUBPD (SSE), ADDSUBPD (SSE2), HADDPS (SSE), HSUBPS (SSE), HADDPD (SSE2), HSUBPD (SSE2), MONITOR (нет аналога в SSE3 для AMD), MWAIT (нет аналога в SSE3 для AMD). Наиболее заметное изменение — возможность «горизонтальной» работы с регистрами. Если говорить более конкретно, добавлены команды сложения и вычитания нескольких значений, хранящихся в одном регистре. Эти команды упростили ряд DSP и 3D-операций. Существует также новая команда для преобразования значений с плавающей точкой в целые без необходимости вносить изменения в глобальном режиме округления.

## **SSSE3**

Дополнение к SSE3 для работы с упакованными целыми. Новыми в SSSE3, по сравнению с SSE3, являются 16 уникальных команд, работающих с упакованными целыми. Каждая из них может работать как с 64-битными (MMX), так и с 128-битными (XMM) регистрами, поэтому Intel в своих материалах ссылается на 32 новые команды.

Категории новых инструкций: работа со знаком, сдвиги, перемешивание байт, умножения, горизонтальные сложения и вычитания целых.

## **SSE4**

Новый набор команд Intel, впервые реализованный в процессорах серии Penryn.

SSE4 состоит из 54 инструкций, 47 из них относят к SSE4.1 (они есть только в процессорах Penryn). Полный набор команд (SSE4.1 и SSE4.2, то есть 47 + оставшиеся 7 команд) доступен в процессорах Nehalem. Ни одна из SSE4 инструкций не работает с 64-битными mmx регистрами, только со 128-битными регистрами xmm0-15. 32-битных процессоров с SSE4 не выпускалось.

Добавлены инструкции, ускоряющие компенсацию движения в видеокодеках, быстрое чтение из USWC памяти, множество инструкций для упрощения векторизации программ компиляторами. Кроме того, в SSE4.2 добавлены инструкции обработки строк 8/16 битных символов, вычисления CRC32, rorq. Впервые в SSE4 регистр xmm0 стал использоваться как неявный аргумент для некоторых инструкций.

Новые инструкции SSE4.1 включают ускорение видео, работу с векторными примитивами, вставки/извлечения, скалярное умножение векторов, смешивания, проверки бит, округления, чтение WC-памяти.

Новые инструкции SSE4.2 включают обработку строк, подсчёт CRC32, подсчёт популяции единичных бит, работу с векторными примитивами.

### **SSE5**

Новое расширение x86 инструкций от AMD, названное SSE5. Было анонсировано в 2007 году, но так и не было реализовано как изначально предполагалось. Позднее было заменено XOR, FMA, F16C.

### **AVX**

Следующий набор расширений от Intel. Поддерживается обработка чисел с плавающей запятой упакованных в 256-битные «слова». Для них вводится поддержка тех же команд, что и в семействе SSE. 128-битные регистры SSE XMM0 — XMM15 расширяются до 256-битных YMM0-YMM15

Intel Post 32 nm processor extensions — новый набор инструкций Intel, позволяющий конвертировать числа с половинной точностью в числа с одинарной и двойной, аппаратно получать истинно случайные числа и обращаться к регистрам FS/GS.

### **AVX2**

Дальнейшее развитие AVX. Целочисленные команды SSE начинают работать с 256-битными AVX регистрами.

### **AES**

Расширение системы команд AES — реализация в микропроцессоре поддержки шифрования стандарта AES.

### **3DNow!**

Набор инструкций для потоковой обработки вещественных чисел одинарной точности. Поддерживается процессорами AMD начиная с K6-2. Процессорами Intel не поддерживается.

Инструкции 3DNow! используют регистры MMX в качестве операндов (в один регистр помещается два числа одинарной точности), поэтому, в отличие от SSE, при переключении задач не требуется отдельно сохранять контекст 3DNow!.

### **64-битный режим<sup>1</sup>**

К началу 2000-х годов стало очевидно, что 32-битное адресное пространство архитектуры x86 ограничивает производительность приложений, работающих с большими объёмами данных. 32-разрядное адресное пространство позволяет процессору осуществлять непосредственную адресацию лишь 4 Гбайт данных. Этого может оказаться недостаточным для некоторых приложений, связанных, например, с обработкой видео или обслуживанием баз данных.

Для решения этой проблемы Intel, совместно с Hewlett Packard, разработали новую архитектуру IA-64 — основу семейства процессоров Itanium. Для обеспечения обратной совместимости со старыми приложениями, использующими 32-разрядный код, в IA-64 был предусмотрен режим эмуляции. Однако на практике данный режим работы оказался чрезвычайно медленным.

Компания AMD предложила альтернативное решение проблемы увеличения разрядности процессора. Вместо того, чтобы изобретать совершенно новую систему команд, было

---

<sup>1</sup> Более подробно об этом расширении было рассказано выше, в предыдущем разделе «64-битное расширение архитектуры x86».

предложено ввести 64-разрядное расширение к уже существующей 32-разрядной архитектуре x86. Первоначально новая архитектура называлась x86-64, позже она была переименована в AMD64. Первоначально новый набор инструкций поддерживался процессорами семейств Opteron, Athlon 64 и Turion 64 компании AMD. Успех процессоров, использующих технологию AMD64, наряду с вялым интересом к архитектуре IA-64, побудили Intel лицензировать набор инструкций AMD64. При этом был добавлен ряд специфических инструкций, не присутствовавших в изначальном наборе AMD64. Новая версия архитектуры получила название EM64T.

В литературе и названиях версий своих программных продуктов компании Microsoft и Sun используют объединённое именование AMD64/EM64T, когда речь заходит о 64-разрядных версиях их операционных систем Windows и Solaris соответственно. В то же время, поставщики программ для операционных систем семейства Linux, BSD используют метки «x86-64» или «amd64», а в Mac OS X используется метка «x86\_64», если необходимо подчеркнуть, что данное ПО использует 64-разрядные инструкции.

### **Аппаратная виртуализация**

Аппаратная виртуализация — виртуализация с поддержкой специальной процессорной архитектуры<sup>1</sup>. В отличие от программной виртуализации, с помощью данной техники возможно использование изолированных гостевых систем, управляемых гипервизором напрямую.

Гостевая система не зависит от архитектуры хостовой платформы и реализации платформы виртуализации.

Аппаратная виртуализация обеспечивает производительность, сравнимую с производительностью неvirtуализованной машины, что дает виртуализации возможность практического использования и влечет её широкое распространение. Наиболее распространены технологии виртуализации Intel-VT и AMD-V.

В Intel VT (Intel Virtualization Technology) реализована виртуализация режима реальной адресации (режим совместимости с 8086). Соответствующая аппаратная виртуализация ввода-вывода — VT-d (кодовое название — Vanderpool). Часто обозначается аббревиатурой VMX (Virtual Machine eXtension).

AMD-V часто обозначается аббревиатурой SVM (Secure Virtual Machines). Кодовое название — Pacifica. Соответствующая технология виртуализации ввода-вывода — IOMMU. AMD-V проще и эффективнее, чем Intel VT. Поддержка AMD-V появилась в Xen 3.3.

## **Построение вычислительной системы**

Начиная с поколения P5 и далее, выход каждого нового процессора обычно сопровождался одновременным выпуском набора микросхем поддержки (чипсета) от производителя процессора, на основе которого строились соответствующие платформы — десктопы, серверы, ноутбуки и т.п. Некоторая заминка случилась разве что при выходе на рынок процессора Pentium 4, с первыми версиями которого Intel предлагала использовать очень дорогую память типа Rambus (RDRAM) и поддерживающие её чипсеты i820- и i850-ой серий, так и не сумевшие в итоге занять на рынке сколь-нибудь заметную долю. В результате несколько позже Intel выпустила удачный чипсет i845 под P-4 для работы с обычной памятью типа SDRAM и DDR.

### **Чипсеты для процессоров поколения P5**

Всего для поколения P5, с марта 1993 по февраль 1997, Intel выпускала семь семейств чипсетов серии 430. Ниже, в таблице на Рис.334, приведены их основные характеристики.

---

<sup>1</sup> Более подробно об этой технологии будет рассказано дальше, в разделе «Аппаратная поддержка виртуализации в x86 архитектурах».

| Chipset               | Code Name                     | Part Numbers                         | South Bridge                                   | Release Date  | Processors | FSB          | SMP | Memory types  | Max. memory | Max. cacheable                                      | Parity/ECC | L2 Cache Type   | PCI support | AGP support |
|-----------------------|-------------------------------|--------------------------------------|------------------------------------------------|---------------|------------|--------------|-----|---------------|-------------|-----------------------------------------------------|------------|-----------------|-------------|-------------|
| 430LX                 | Mercury <sup>[16]</sup>       | 82434LX (PCMC)<br>2x 82433LX (LBX)   | SIO (ISA)<br>PCEB/ESC (EISA)                   | March 1993    | P60/66     | 60/66 MHz    | No  |               | 192 MB      | 192 MB                                              |            |                 |             |             |
| 430NX                 | Neptune <sup>[17]</sup>       | 82434NX (PCMC)<br>2x 82433NX (LBX)   | SIO (ISA)<br>SIO.A (DP ISA)<br>PCEB/ESC (EISA) | March 1994    |            |              | Yes | FPM           | 512 MB      | 512 MB                                              | Parity     | Async.          | 2.0         |             |
| 430FX                 | Triton <sup>[18][19]</sup>    | 82437FX/JX (TSC)<br>2x 82438FX (TDP) | PIIX                                           | January 1995  |            | 50/60/66 MHz | No  |               | 128 MB      | 64 MB                                               | Neither    |                 |             | No          |
| 430MX                 | Mobile Triton                 | 82437MX                              | MPIIX                                          | October 1995  |            |              |     |               |             |                                                     |            |                 |             |             |
| 430HX                 | Triton II <sup>[19][20]</sup> | 82439HX/JHX (TXC)                    | PIIX3                                          | February 1996 | P75+       |              | Yes | FPM/EDO       | 512 MB      | 64 MB<br>512 MB (w/ 11-bit tag RAM) <sup>[21]</sup> | Both       | Async. / Pburst | 2.1         |             |
| 430VX                 | Triton II <sup>[19][22]</sup> | 82437VX (TVX)<br>2x 82438VX (TDX)    |                                                |               |            | 60/66 MHz    | No  |               | 128 MB      | 64 MB                                               | Neither    |                 |             |             |
| 430TX <sup>[23]</sup> |                               | 82439TX (MTXC)                       | PIIX4                                          | February 1997 |            |              |     | FPM/EDO/SDRAM | 256 MB      |                                                     |            |                 |             |             |

Рис.334 – Чипсеты для процессоров P5

## Схема компьютера на базе P5 и чипсета i430

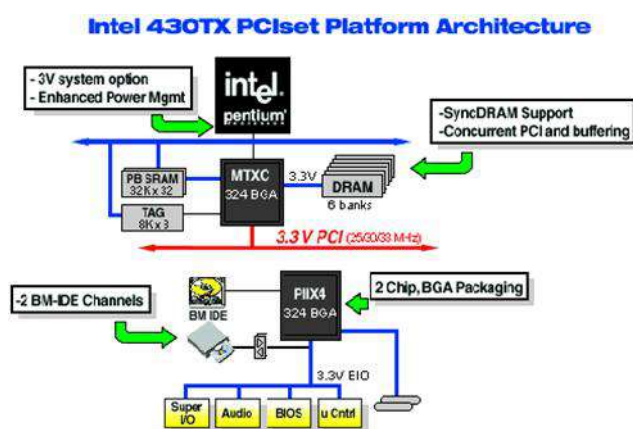


Рис.335 – PC на базе Pentium и чипсета 430TX

Одна из ставшей канонической схема PC на базе процессоров Pentium первого поколения (пятого поколения x86), с использованием чипсетов 430-ой серии, представлена на Рис.335. Всего в этом семействе было выпущено около семи версий чипсета, от i430LX Mercury (март 1993, поддержка первого Pentium-a), до i430TX Mobile Triton II (февраль 1997), в двух-четырёхкристальном варианте.

Кроме большого количество вариантов чипсетов, Intel стала выпускать много разнообразных процессоров, разделяя их как по линейкам: Celeron, Pentium,

Хеон, так и по тактовым частотам, а также регулярно расширяя набор поддерживаемых инструкций в более новых процессорах.

Интересный момент: чипсет i430TX был последним в линейке i430, но, если сравнить его основные характеристики с выпущенным годом ранее i430NX – по объёму поддерживаемого ОЗУ и объёму кэшируемой памяти – то по ним более «свежий» Triton II уступал предшественнику (ОЗУ до 512 Мбайт в НХ, до 384 Мбайт в ТХ, в НХ кэшируется вся память, в ТХ – только первые 64 Мбайт). Из улучшений была только поддержка нового типа памяти (SDRAM) и поддержка USB, которой в 430NX не было. Казалось бы, странное решение – более новое устройство уступает по функционалу более старому. Но и в этот раз, снова, как было в случае с неподдерживаемой шиной MCA для 286-х, в Intel-е политические соображения были поставлены выше технических: в это же время Intel продвигала на рынок Pentium II и хотела форсировать переход своих потенциальных покупателей от первых пен-тиумов на новый P-II, сознательно ухудшая характеристики ПК на базе первых Pentium-ов.



## Чипсеты для процессоров поколения P6 400-ой серии

Этим поколением чипсетов серий i440-i450 обеспечивалась поддержка микропроцессоров Pentium Pro, Pentium II, Pentium III, Celeron и Xeon. Пришли на смену чипсетам, обеспечивающим поддержку процессоров поколения Pentium P5 (i430). В свою очередь, были сменены чипсетами 800 серии.

Всего было выпущено около пятнадцати семейств чипсетов серий 440-450.

| Chipset  | Code Name | Part numbers                       | South Bridge                           | Release Date  | Processors <sup>(a)</sup>      | FSB        | SMP              | Type              | Memory Max.                                     | Bank | Parity/ECC | PCI support           | AGP support |
|----------|-----------|------------------------------------|----------------------------------------|---------------|--------------------------------|------------|------------------|-------------------|-------------------------------------------------|------|------------|-----------------------|-------------|
| 450KX    | Mars      | 82451KX, 82452KX, 82453KX, 82454KX | SIO, SIO.A, PIIX (ISA) PCEB/ESC (EISA) | November 1995 | Pentium Pro                    | 60/66 MHz  | Yes              | FPM               | 1 GB                                            |      |            | 2.0                   |             |
| 450GX    | Orion     | 82451GX, 82452GX, 82453GX, 82454GX | SIO.A (ISA) PCEB/ESC (EISA)            |               |                                |            | Yes (up to four) |                   | 8 GB                                            |      | Both       |                       | No          |
| 440FX    | Natoma    | 82441FX, 82442FX                   | PIIX3 (ISA) PCEB/ESC (EISA)            | May 1996      | Pentium Pro, Pentium II        |            | Yes              | FPM / EDO / BEDO  | 1 GB                                            |      |            | 2.1                   |             |
| 440LX    | Balboa    | 82443LX                            | PIIX4                                  | August 1997   | Pentium II, Celeron            | 66 MHz     |                  | FPM / EDO / SDRAM | 1 GB EDO / 512 MB SDRAM <sup>(b)(c)</sup>       | 4    |            | 2.1                   |             |
| 440EX    | N/A       | 82443EX                            |                                        |               |                                |            | No               |                   | 256 MB                                          | 2    | Neither    |                       | AGP 2x      |
| 440BX    | Seattle   | 82443BX                            |                                        | April 1998    | Pentium II/III, Celeron        | 66/100 MHz | Yes              | EDO / SDRAM       | 512 MB (non-ECC) / 1 GB (ECC) <sup>(d)(e)</sup> |      |            | 2.1 (64-bit optional) |             |
| 440GX    |           | 82443GX                            |                                        |               |                                |            |                  | SDRAM             | 2 GB                                            | 4    | Both       | 2.1                   |             |
| 450NX    |           | 82451NX, 82452NX, 82453NX, 82454NX | PIIX4E                                 | June 1998     | Pentium II/III, Xeon           | 66/100 MHz | Yes (up to four) | FPM / EDO         | 8 GB                                            |      |            | 2.1 (64-bit optional) | No          |
| 440ZX-66 | N/A       | 82443ZX                            |                                        | November 1998 | Celeron, Pentium II/III        | 66 MHz     |                  |                   | 512 MB                                          |      |            | 2.1                   |             |
| 440ZX    |           |                                    |                                        |               |                                |            |                  |                   |                                                 |      |            |                       |             |
| 440ZX-M  |           | 82443ZX-M                          | PIIX4M                                 |               | Pentium III, mobile Celeron    | 66/100 MHz | No               | SDRAM             | 256 MB                                          | 2    | Neither    | 2.1                   | AGP 2x      |
| 440MX    | Banister  | 82443MX                            | Same chip                              |               | Pentium II/III, mobile Celeron |            |                  |                   | 512 MB                                          |      |            | 2.2                   | No          |

Рис.336 – Чипсеты для Pentium Pro/II/III

| Chipset | Part Number | ATA support | USB support           | CMOS/clock | ISA support | LPC support | Power management |
|---------|-------------|-------------|-----------------------|------------|-------------|-------------|------------------|
| ESC     | 82374EB/SB  | None        | None                  |            | Yes         |             | SMM              |
| PCEB    | 82375EB/SB  |             |                       |            |             |             |                  |
| SIO     | 82378IB/ZB  |             |                       |            |             |             |                  |
| SIO.A   | 82379AB     |             |                       |            |             |             |                  |
| MPIIX   | 82371MX     | PIO         | 1 Controller, 2 Ports | No         | Yes         | No          | SMM              |
| PIIX    | 82371FB     | PIO/WDMA    |                       |            |             |             |                  |
| PIIX3   | 82371SB     |             |                       |            |             |             |                  |
| PIIX4   | 82371AB     | PIO/UDMA 33 |                       |            |             |             |                  |
| PIIX4E  | 82371EB     |             |                       | Yes        |             |             |                  |
| PIIX4M  | 82371MB     |             |                       |            |             |             |                  |

Рис.337 – Южные мосты чипсетов серии 4xx

## Чипсеты для процессоров поколения P6 и NetBurst 800-ой серии

В данной серии чипсетов Intel впервые реализовала «хабовую архитектуру» с использованием северного (концентратор контроллера памяти Memory Controller Hub — MCH; системы с интегрированной графикой вместо стандартного MCH используют концентратор контроллера графической памяти Graphics Memory Controller Hub — GMCH) и южного моста (концентратор контроллера ввода-вывода I/O Controller Hub — ICH).

Системы с интегрированной графикой реализовали последовательно инновации Intel в развитии видеопроцессоров Intel Graphics Technology, Intel Extreme Graphics и Intel Extreme Graphics 2, которые в 900-й серии получили новую реализацию, Intel Graphics Media Accelerator.

Эволюционно 800-ая серия сменили чипсеты Intel для процессоров P6, и затем, в свою очередь, была сменена чипсетами 900-ой серии.

| Chipset | Code name | Part numbers | South bridge         | Release date   | Processors              | FSB            | SMP | Memory types                       | Max. memory | Memory banks | Parity or ECC | PCI                        | Ext. AGP/speed | IGP |            |  |  |  |  |  |  |
|---------|-----------|--------------|----------------------|----------------|-------------------------|----------------|-----|------------------------------------|-------------|--------------|---------------|----------------------------|----------------|-----|------------|--|--|--|--|--|--|
| 810     | Whitney   | 82810        | ICH/ICH0             | April 1999     | Celeron, Pentium II/III | 66/100 MHz     | No  | EDO/PC100 SDRAM                    | 512 MB      | 4            | Neither       | v2.2/33 MHz                | No             | Yes |            |  |  |  |  |  |  |
| 810E    |           | 82810E       | ICH                  | September 1999 |                         | 66/100/133 MHz |     | PC100/133 SDRAM                    |             |              |               |                            |                |     |            |  |  |  |  |  |  |
| 810E2   |           |              | ICH2                 |                |                         |                |     |                                    |             |              |               |                            |                |     |            |  |  |  |  |  |  |
| 815     |           | 82815        | ICH                  | June 2000      |                         |                |     |                                    |             |              |               |                            |                |     |            |  |  |  |  |  |  |
| 815E    | ICH2      |              | Celeron, Pentium III |                | No                      |                |     |                                    |             |              |               |                            | Yes/AGP 4x     | No  |            |  |  |  |  |  |  |
| 815G    | 82815G    | ICH/ICH0     |                      |                |                         |                |     |                                    |             |              |               |                            | September 2001 |     | No         |  |  |  |  |  |  |
| 815EG   |           | ICH2         |                      |                |                         |                |     |                                    |             |              |               |                            |                |     |            |  |  |  |  |  |  |
| 815P    | 82815EP   | ICH/ICH0     |                      |                |                         |                |     |                                    |             |              |               |                            | March 2001     |     |            |  |  |  |  |  |  |
| 815EP   |           | ICH2         |                      |                |                         |                |     |                                    |             |              |               |                            | November 2000  |     | Yes/AGP 4x |  |  |  |  |  |  |
| 820     | Camino    | 82820        | ICH                  | November 1999  | Celeron, Pentium II/III | 66/100/133 MHz | Yes | PC800 RDRAM/PC100 SDRAM (with MTH) | 1 GB        | 2            | Both          | v2.2/33 MHz + PCI-X/66 MHz | Yes/AGP 4x     | No  |            |  |  |  |  |  |  |
| 820E    |           |              | ICH2                 | June 2000      | Pentium III, Xeon       |                |     |                                    |             |              |               |                            |                |     |            |  |  |  |  |  |  |
| 840     |           | 82840        | ICH                  | October 1999   | Pentium III, Xeon       |                |     | Dual-Channel PC800 RDRAM           | 4 GB        | 2x4          |               |                            |                |     |            |  |  |  |  |  |  |

Рис.338 – Чипсеты для Pentium II/III

| Chipset | Code name | Part numbers | South bridge | Release date | Processors                         | FSB         | SMP | Memory types | Max. memory | Memory banks | Parity or ECC | PCI         | Ext. AGP/speed | IGP |
|---------|-----------|--------------|--------------|--------------|------------------------------------|-------------|-----|--------------|-------------|--------------|---------------|-------------|----------------|-----|
| 815EM   | Almador   | 82815EM      | ICH2-M       | October 2000 | Mobile Celeron, Mobile Pentium III | 100 MHz     | No  | PC100 SDRAM  | 512 MB      | 2            | Neither       | v2.2/33 MHz | Yes/AGP 4x     | Yes |
| 830M    |           | 82830M       | ICH3-M       | July 2001    | Celeron, Pentium III-M             | 100/133 MHz | No  | PC133 SDRAM  | 1 GB        | 2            | Neither       | v2.2/33 MHz | Yes/AGP 4x     | Yes |
| 830MP   |           | 82830MP      |              |              |                                    |             |     |              |             |              |               |             | No             | No  |
| 830MG   |           | 82830MG      |              |              |                                    |             |     |              |             |              |               |             | No             | Yes |

Рис.339 – Мобильные чипсеты Pentium III

| Chipset       | Code name     | Part numbers   | South bridge       | Release date   | Socket                   | Processor brands                                                                                           | FSB [ MT/s ]              | SMP | Memory types                            | Max. memory [ GiB ]                | Parity/ECC              | Graphics                                                                                                                                | TDP                                                                                              |  |  |
|---------------|---------------|----------------|--------------------|----------------|--------------------------|------------------------------------------------------------------------------------------------------------|---------------------------|-----|-----------------------------------------|------------------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|--|--|
| 860           | Cousa         | 82860 (MCH)    | ICH2               | May 2001       | Socket 603<br>Socket 604 | Xeon                                                                                                       | 400 MT/s<br>(100 MHz GDR) | Yes | PC800/600 RDRAM                         | 4 (w 2 repeaters)                  | Yes/Yes                 | AGP 4x                                                                                                                                  |                                                                                                  |  |  |
| 845           | Brookdale     | 82845 (MCH)    |                    | January 2002   | Socket 423               | Celeron, Pentium 4                                                                                         |                           | No  | DDR 200/266 SDR 133                     | 2 (DDR) 3 (SDR)                    |                         |                                                                                                                                         |                                                                                                  |  |  |
| 850           | Tehama        | 82850 (MCH)    |                    | November 2000  | Socket 478               |                                                                                                            |                           |     | PC800/600 RDRAM                         |                                    |                         |                                                                                                                                         |                                                                                                  |  |  |
| 850E          | Tehama-E      | 82850E (MCH)   |                    |                |                          |                                                                                                            |                           |     | PC1066/800/600 RDRAM                    |                                    |                         |                                                                                                                                         |                                                                                                  |  |  |
| 845E          | Brookdale-E   | 82845E (MCH)   | ICH4               | May 2002       | Socket 478               | Celeron, Celeron D, Pentium 4                                                                              | 400/533 MT/s              | No  | DDR 200/266 SDR 133                     |                                    | No/No                   | Intel Extreme Graphics AGP 4x                                                                                                           | 5.8 W                                                                                            |  |  |
| 845G          | Brookdale-G   | 82845G (GMCH)  |                    |                |                          |                                                                                                            | DDR 200/266 SDR 133       |     |                                         | Intel Extreme Graphics no AGP slot |                         | 5.1 W (SDRAM) 5.7 W (DDR2)                                                                                                              |                                                                                                  |  |  |
| 845GV         | Brookdale-GV  | 82845GV (GMCH) |                    | October 2002   | Socket 478 LGA 775       |                                                                                                            | DDR 200/266 SDR 133       |     |                                         | Intel Extreme Graphics AGP 4x      |                         | 5.1 W (SDRAM) 5.8 W (DDR2)                                                                                                              |                                                                                                  |  |  |
| 845GL         | Brookdale-GL  | 82845GL (GMCH) |                    | May 2002       | Socket 478               |                                                                                                            | DDR 200/266 SDR 133       |     |                                         | Intel Extreme Graphics AGP 4x      |                         | 6.3 W                                                                                                                                   |                                                                                                  |  |  |
| 845GE         | Brookdale-GE  | 82845GE (GMCH) | ICH5/ICH5R         | October 2002   | Socket 478 LGA 775       | Celeron, Celeron D, Pentium 4                                                                              | 400/533 MT/s              | No  | DDR 200/266/333                         |                                    | No/No                   | AGP 4x                                                                                                                                  | 5.8 W                                                                                            |  |  |
| 845PE         | Brookdale-PE  | 82845PE (MCH)  |                    |                | Socket 478               |                                                                                                            |                           |     | AGP 8x                                  | 8.1 W                              |                         |                                                                                                                                         |                                                                                                  |  |  |
| 848P          | Breeds Hill   | 82848P (MCH)   |                    | August 2003    | Socket 478 LGA 775       | Pentium 4, Pentium 4 EE, Pentium D, Celeron, Celeron D                                                     | 400/533/800 MT/s          |     | DDR-400 (single-channel)                |                                    |                         | AGP 8x                                                                                                                                  | 10.3 W                                                                                           |  |  |
| 865P          | Springdale-P  | 82865P         |                    | May 2003       |                          | Pentium 4, Celeron D                                                                                       | 400/533 MT/s              |     | DDR-333                                 |                                    |                         | AGP 8x                                                                                                                                  | 11.3 W                                                                                           |  |  |
| 865PE         | Springdale-PE | 82865PE        |                    |                |                          | Pentium 4, Pentium 4 EE, Pentium D, Pentium Extreme Edition, Celeron, Celeron D, Core 2, Pentium Dual Core | 400/533/800 MT/s          |     | DDR-400                                 | 4                                  |                         | Intel Extreme Graphics 2 AGP 8x                                                                                                         | 12.9 W                                                                                           |  |  |
| 865G          | Springdale    | 82865G (GMCH)  |                    |                |                          |                                                                                                            |                           |     |                                         |                                    |                         | Intel Extreme Graphics 2 no AGP slot                                                                                                    |                                                                                                  |  |  |
| 865GV         | Springdale-GV | 82865GV (GMCH) | September 2003     |                |                          |                                                                                                            |                           |     |                                         |                                    |                         |                                                                                                                                         |                                                                                                  |  |  |
| 875P          | Canterwood    | 82875P (MCH)   | ICH6/ICH5R/6300ESB | April 2003     |                          | Pentium 4, Pentium 4 EE, Pentium D, Pentium Extreme Edition, Celeron, Celeron D, Xeon                      | 400/533/800 MT/s          | No  | DDR-400                                 | 4                                  | Yes/Yes unbuffered only | AGP 8x                                                                                                                                  | 12.1 W                                                                                           |  |  |
| E7205         | Granite Bay   | E7205 (MCH)    | ICH4               | Nov 2002       | Socket 604               | Pentium 4                                                                                                  | 400/533 MT/s              |     |                                         |                                    |                         |                                                                                                                                         | Yes/Yes                                                                                          |  |  |
| E7500         | Plumas        | E7500 (MCH)    | ICH3-S             | Feb 2002       |                          | Xeon                                                                                                       | 400 MT/s                  |     | DDR-266                                 | 16                                 |                         | PCI-X                                                                                                                                   |                                                                                                  |  |  |
| E7501         | Plumas 533    | E7501 (MCH)    | ICH4               | Dec 2002       |                          | Xeon, Pentium M (2M)                                                                                       | 400/533 MT/s              |     |                                         |                                    |                         | AGP 8x                                                                                                                                  |                                                                                                  |  |  |
| E7505         | Placer        | E7505 (MCH)    |                    | Nov 2002       |                          | Xeon                                                                                                       | 400/533 MT/s              |     |                                         |                                    |                         |                                                                                                                                         |                                                                                                  |  |  |
| E7221 [10831] | Copper River  | E7221 (MCH)    | ICH6/ICH6R         | September 2004 | LGA 775                  | Pentium 4, Pentium 4 HT                                                                                    | 533/800 MT/s              | No  | DDR 333/400 DDR2 400/533 (dual-channel) | 4                                  | Yes/Yes unbuffered only | • Integrated graphics engine (5VGA)<br>• PCI Express x8 (1.0a), or<br>• PCI-X (with PCIe bridge) not specifically dedicated to graphics |                                                                                                  |  |  |
| E7230 [10832] | Mukilteo      | E7230 (MCH)    | ICH7/ICH7R         | July 2005      | LGA 775                  | Pentium D, Pentium 4, Pentium 4 HT, Pentium 4 EE, Celeron D                                                | 533/800/1066 MT/s         |     | DDR2 400/533/666 (dual-channel)         | 6                                  |                         | Yes/Yes unbuffered only                                                                                                                 | • PCI Express x8 (1.0a), or<br>• PCI-X (with PCIe bridge) not specifically dedicated to graphics |  |  |

Рис.340 – Чипсеты Pentium 4

| Chipset  | Code name    | Part numbers    | South bridge | Release date | Processors                      | FSB      | SMP | Memory types    | Max. memory | Parity/ECC | PCI Type    | Graphics                                    | TDP   |
|----------|--------------|-----------------|--------------|--------------|---------------------------------|----------|-----|-----------------|-------------|------------|-------------|---------------------------------------------|-------|
| 845MZ    | Brookdale-MZ | 82845 (MCH)     | ICH3-M       | March 2002   | Mobile Celeron, Pentium 4-M     | 400 MT/s |     | DDR 200         | 1 GB        |            |             | AGP 4x                                      |       |
| 845MP    | Brookdale-M  | 82845 (MCH)     | ICH3-M       | March 2002   | Mobile Celeron, Pentium 4-M     | 400 MT/s |     | DDR 200         | 1 GB        |            |             | AGP 4x                                      |       |
| 852GM    | Montara-GM   | 82852GM (GMCH)  | ICH4-M       | Q2 '04       | Pentium 4-M, Celeron, Celeron M | 400 MT/s |     | DDR 200/266     | 2 GB        | No/No      | v2.2/33 MHz | Integrated 3D-bit 3D Core @ 133 MHz         | 3.2 W |
| 852GMV   | Montara-GM   | 82852GMV (GMCH) | ICH4-M       | Q2 '04       | Pentium 4-M, Celeron, Celeron M | 400 MT/s |     | DDR 200/266     | 2 GB        | No/No      | v2.2/33 MHz | AGP 1x/2x/4x                                | 5.7 W |
| 852PM    | Montara-GM   | 82852PM (MCH)   | ICH4-M       | Q4 '03       | Pentium 4-M, Celeron, Celeron D | 400 MT/s | No  | DDR 200/266/333 | 2 GB        | No/No      | v2.2/33 MHz | AGP 1x/2x/4x                                | 5.7 W |
| 852GME   | Montara-GM   | 82852GME (GMCH) | ICH4-M       | March 2005   | Celeron M ULV                   | 400 MT/s |     | DDR 266/333     | 2 GB        | No/No      | v2.2/33 MHz | Integrated Extreme Graphics 2 graphics core | 3.2 W |
| 854 [36] | Montara-GM   | 82854 (GMCH)    | ICH4-M       | March 2005   | Celeron M ULV                   | 400 MT/s |     | DDR 200/266     | 2 GB        | No/No      | v2.2/33 MHz | AGP 1x/2x/4x                                | 4.3 W |
| 855GM    | Montara-GM   | 82855GM (GMCH)  | ICH4-M       | March 2005   | Pentium M, Celeron M            | 400 MT/s |     | DDR 200/266/333 | 2 GB        | No/No      | v2.2/33 MHz | AGP 2x/4x                                   | 5.7 W |
| 855GME   | Montara-GM   | 82855GME (MCH)  | ICH4-M       | March 2005   | Pentium M, Celeron M            | 400 MT/s |     | DDR 200/266/333 | 2 GB        | No/No      | v2.2/33 MHz | AGP 2x/4x                                   | 5.7 W |
| 855PM    | Odern        | 82855PM (MCH)   | ICH4-M       | March 2005   | Pentium M, Celeron M            | 400 MT/s |     | DDR 200/266/333 | 2 GB        | No/No      | v2.2/33 MHz | AGP 2x/4x                                   | 5.7 W |

Рис.341 – Мобильные чипсеты Pentium 4-M/Pentium M/Celeron M

| Chipset | Part Number | ATA            | SATA                     | RAID Level   | USB         |                                                                | PCI         |         |
|---------|-------------|----------------|--------------------------|--------------|-------------|----------------------------------------------------------------|-------------|---------|
|         |             |                |                          |              | v1.1        | v2.0                                                           | Rev 2.2     | Rev 2.3 |
| ICH     | 82801AA     | UDMA 66/33     | No                       | No           | 2 ports     | N/A                                                            | 6 PCI slots | N/A     |
| ICH0    | 82801AB     | UDMA 33        |                          |              |             |                                                                | 4 PCI slots |         |
| ICH2-M  | 82801BAM    |                |                          |              |             |                                                                | 2 PCI slots |         |
| ICH2    | 82801BA     |                |                          |              |             |                                                                | 6 PCI slots |         |
| ICH3-M  | 82801CAM    |                |                          |              |             |                                                                | 2 PCI slots |         |
| ICH3-S  | 82801CA     |                |                          |              | 6 PCI slots |                                                                |             |         |
| ICH4-M  | 82801DBM    | UDMA 100/66/33 |                          |              | 4 ports     | 3 PCI slots                                                    |             |         |
| ICH4    | 82801DB     |                |                          |              | 6 ports     | 6 PCI slots                                                    |             |         |
| ICH5-M  | 82801EBM    |                |                          |              | 4 ports     | 4 PCI slots                                                    |             |         |
| ICH5    | 82801EB     |                |                          |              | 8 ports     | N/A                                                            | 6 PCI slots |         |
| ICH5R   | 82801ER     |                |                          |              | 6 ports     |                                                                |             |         |
| 6300ESB | 6300ESB     |                | SATA 1.5 Gbit/s, 2 ports | RAID0, RAID1 | 4 ports     | Rev 2.2 4 PCI slots,<br>Rev 1.0 2 PCI-X slots + 2 PCI-X device |             |         |

Рис.342 – Южные мосты чипсетов серии 8xx

## Чипсеты серии 9xx и серий 3/4

Чипсеты Intel 900 серии пришли на смену 800 серии и принесли за собой значительные изменения для улучшения производительности компьютеров. Их частота системной шины была увеличена до 800—1066 МГц, был внедрён двухканальный контроллер памяти нового стандарта DDR2. Несколько позже под выпуск первых двухъядерных процессоров Pentium D на базе микроархитектуры NetBurst специально была разработана 940-ая серия. Процессоры микроархитектуры Core 2 обзавелись поддержкой 960-ой серии чипсетов. Также с выходом этих чипсетов закончилась эпоха AGP и началось распространение шины PCI Express x16 в качестве основы для графической составляющей, пришедшей ему на смену.

- Do not support SMP
- Support (-R and -DH) variants for South Bridges

| Chipset               | Code Name     | Part numbers   | South Bridge | Release Date   | Supported Processors                        | FSB [MT/s]                                                 | Memory                    |                  | Parity / ECC | Graphics         |                         | TDP [W] |      |      |      |
|-----------------------|---------------|----------------|--------------|----------------|---------------------------------------------|------------------------------------------------------------|---------------------------|------------------|--------------|------------------|-------------------------|---------|------|------|------|
|                       |               |                |              |                |                                             |                                                            | types                     | max. [GB]        |              | PCIe             | integrated core         |         |      |      |      |
| <a href="#">910GL</a> | Grantsdale-GL | 82910GL (GMCH) | ICH6/ICH6R   | September 2004 | Pentium 4, Celeron, Celeron D               | 533                                                        | DDR 333/400               | 2                | No/No        | N/A              | <a href="#">GMA 900</a> | 16.3    |      |      |      |
| <a href="#">915GL</a> |               | 82915GL (GMCH) |              | March 2005     |                                             |                                                            |                           | 4                |              |                  |                         |         |      |      |      |
| <a href="#">915PL</a> | Grantsdale-PL | 82915PL (MCH)  |              |                |                                             |                                                            |                           | 2                |              |                  |                         |         |      |      |      |
| <a href="#">915P</a>  | Grantsdale    | 82915P (MCH)   |              | June 2004      | Pentium 4, Celeron D                        | 533/800                                                    | DDR 333/400, DDR2 400/533 | 4                | No/No        | x16              | N/A                     |         |      |      |      |
| <a href="#">915G</a>  | Grantsdale-G  | 82915G (GMCH)  |              |                |                                             |                                                            |                           |                  |              |                  |                         |         |      |      |      |
| <a href="#">915GV</a> | Grantsdale-GV | 82915GV (GMCH) |              |                |                                             |                                                            |                           |                  |              |                  |                         |         |      |      |      |
| 925X                  | Alderwood     | 82925X (MCH)   |              | November 2004  | Pentium 4, Pentium 4 EE                     | 800                                                        |                           | DDR2 400/533     |              | 4 <sup>[1]</sup> | Yes/Yes                 |         |      | N/A  | 12.3 |
| 925XE                 | Alderwood-XE  | 82925XE (MCH)  |              |                |                                             | 800/1066                                                   |                           |                  |              |                  |                         |         |      |      | 13.3 |
| <a href="#">945PL</a> | Lakeport-PL   | 82945PL (MCH)  | ICH7         | March 2006     | Pentium 4, Pentium D, Celeron D, Core 2 Duo | 533/800                                                    |                           | 2 <sup>[1]</sup> | No/No        | x16              | N/A                     | 15.2    |      |      |      |
| <a href="#">945P</a>  | Lakeport      | 82945P (MCH)   | ICH7/ICH7R   | May 2005       | Pentium 4, Pentium D, Celeron D, Core 2 Duo | 533/800/1066                                               | DDR2 400/533/667          | 4 <sup>[1]</sup> |              |                  |                         | GMA 950 | 22.2 |      |      |
| <a href="#">945G</a>  | Lakeport-G    | 82945G (GMCH)  |              |                |                                             |                                                            |                           |                  |              |                  |                         |         |      |      |      |
| <a href="#">955X</a>  | Lakeport-X    | 82955X (MCH)   |              | April 2005     |                                             | Pentium 4, Pentium 4 EE, Pentium D, Pentium XE, Core 2 Duo | 800/1066                  | DDR2 533/667     | 8            | Yes/Yes          |                         |         | N/A  | 13.5 |      |

[1] Remapping of PCIe/APIC memory ranges not supported, some physical memory might not be accessible (e.g. limited to 3.5 GB or similar).

Рис.343 – Чипсеты для процессоров Pentium 4/Pentium D/Pentium EE



| Chipset                | Code Name | Part numbers    | South Bridge | Release Date | Supported Processors | FSB                   | Memory Types          | Max. Memory | Parity/ECC | Graphics           | TDP   |
|------------------------|-----------|-----------------|--------------|--------------|----------------------|-----------------------|-----------------------|-------------|------------|--------------------|-------|
| <a href="#">910GML</a> | Alviso-GM | 82910GML (GMCH) | ICH6-M       | January 2005 | Celeron M            | 400 MT/s              | DDR 333/400, DDR2 400 | 2 GB        | No/No      | Integrated GMA 900 | 6 W   |
| <a href="#">915GMS</a> |           | 82915GMS (GMCH) |              |              | Pentium M, Celeron M |                       | DDR2 400              |             |            |                    | 4.8 W |
| <a href="#">915GM</a>  |           | 82915GM (GMCH)  |              |              | 400/533 MT/s         | DDR 333, DDR2 400/533 | 6 W                   |             |            |                    |       |
| <a href="#">915PM</a>  | Alviso    | 82915PM (MCH)   |              |              |                      |                       | PCI Express ×16       |             |            | 5.5 W              |       |

Рис.344 – Мобильные чипсеты для процессоров Pentium M/Celeron M

| Chipset                | Code name | Part numbers    | South bridge | Release date | Processors supported (official)                               | FSB [MT/s]       | Memory         |           | Graphics           |                 | TDP [W] |  |
|------------------------|-----------|-----------------|--------------|--------------|---------------------------------------------------------------|------------------|----------------|-----------|--------------------|-----------------|---------|--|
|                        |           |                 |              |              |                                                               |                  | types          | max. [GB] | graphics core      | 3D Render       |         |  |
| <a href="#">940GML</a> | Calistoga | 82940GML (GMCH) | ICH7-M       | January 2006 | Celeron M                                                     | 533              | DDR2 400/533   | 2         | Integrated GMA 950 | Max. 166 MHz    | 7       |  |
| <a href="#">943GML</a> |           | 82943GML (GMCH) |              |              | Celeron M, Core Solo, Pentium Dual-Core <sup>[2]</sup>        |                  |                |           |                    | Max. 200 MHz    |         |  |
| <a href="#">945GSE</a> |           | 82945GSE (GMCH) |              | Q1'06        | Intel Atom                                                    | 533/667          |                |           |                    |                 |         |  |
| <a href="#">945GMS</a> |           | 82945GMS (GMCH) |              |              |                                                               |                  |                |           |                    |                 |         |  |
| <a href="#">945GME</a> |           | 82945GME (GMCH) |              | January 2006 | Core 2 Duo, Core Duo, Pentium Dual-Core, Core Solo, Celeron M | DDR2 400/533/667 | 4 <sup>1</sup> |           |                    | Max. 250 MHz    | 7       |  |
| <a href="#">945PM</a>  |           | 82945PM (MCH)   |              |              |                                                               |                  |                |           |                    | PCI Express ×16 |         |  |

<sup>[1]</sup> Remapping of PCIe/APIC memory ranges not supported, some physical memory might not be accessible (e.g. limited to 3.5 GB or similar).

Рис.345 – Мобильные чипсеты Core/Core 2

| Chipset               | Code name | Part numbers             | South bridge | Release date | Lithography | Processors supported (official)                                    | FSB [MT/s]   | Memory                      |                  | Graphics               |              | TDP [W]             |   |
|-----------------------|-----------|--------------------------|--------------|--------------|-------------|--------------------------------------------------------------------|--------------|-----------------------------|------------------|------------------------|--------------|---------------------|---|
|                       |           |                          |              |              |             |                                                                    |              | types.                      | max. [GB]        | graphics core          | 3D Render    |                     |   |
| <a href="#">GL960</a> | Crestline | 82960GL (GMCH)           | ICH8-M       | May 2007     | ?? nm       | Celeron M, Pentium Dual-Core                                       | 533          | DDR2 533/667                | 3/5 <sup>1</sup> | Integrated GMA X3100   | Max. 400 MHz | 13.5                |   |
| <a href="#">GM965</a> |           | 82965GM (GMCH)           |              |              |             | Core 2 Duo                                                         | 533/667/800  |                             | 4/8 <sup>2</sup> |                        | Max. 500 MHz |                     |   |
| <a href="#">PM965</a> |           | 82965PM (MCH)            |              |              |             | PCIe ×16                                                           |              |                             | 8                |                        |              |                     |   |
| <a href="#">GL40</a>  | Cantiga   | 82GL40 (GMCH)            | ICH9-M       | Sep 2008     | 65 nm       | Core 2 Duo, Celeron, Celeron M, <a href="#">Pentium Dual-Core</a>  | 667/800      | DDR2 667/800, DDR3 800/1066 | 4                | Integrated GMA X4500HD | Max. 400 MHz | 12                  |   |
| <a href="#">GS40</a>  |           | 82GS40 (GMCH)            |              |              |             | Core 2 Duo, Celeron, Celeron M?, <a href="#">Pentium Dual-Core</a> |              |                             | 4                |                        |              |                     |   |
| <a href="#">GS45</a>  |           | 82GS45 (GMCH) (For CULV) |              |              |             | Core 2 Solo, Core 2 Duo, Core 2 Extreme, Celeron M                 | 800/1066     |                             | 8                |                        | Max. 533 MHz | 7/8/12 <sup>3</sup> |   |
| <a href="#">GM45</a>  |           | 82GM45 (GMCH)            |              |              |             | Core 2 Duo, Core 2 Extreme, Celeron M                              | 667/800/1066 |                             |                  |                        | 12           |                     |   |
| <a href="#">PM45</a>  |           | 82PM45 (MCH)             |              |              |             | Core 2 Duo, Core 2 Extreme                                         |              |                             |                  |                        | PCIe ×16     |                     | 7 |

• <sup>1</sup> Unofficially this chipset support 5GB.

• <sup>2</sup> Officially only 4GB is supported. Unofficially many laptops with this chipset support 8GB.

• <sup>3</sup> Low power mode, HD playback mode and Full performance mode respectively.

Рис.346 - Мобильные чипсеты Core 2

All Core 2 Duo chipsets support the Pentium Dual-Core and Celeron processors based on the Core architecture. Support for all Itanium-based processors was officially dropped starting with the Bearlake chipset family. However, some motherboards still support the older processors<sup>[24]</sup>.

| Chipset                    | Code Name                 | Part numbers                 | South Bridge        | Release Date           | Processors                                                           | Lithography | Intel VT-d support <sup>[2]</sup> | FSB (MT/s)                          | Memory                         |                                                                   | Parity/ECC | PCIe                                            | iGP                         |            |                    |            |
|----------------------------|---------------------------|------------------------------|---------------------|------------------------|----------------------------------------------------------------------|-------------|-----------------------------------|-------------------------------------|--------------------------------|-------------------------------------------------------------------|------------|-------------------------------------------------|-----------------------------|------------|--------------------|------------|
|                            |                           |                              |                     |                        |                                                                      |             |                                   |                                     | types                          | max.                                                              |            |                                                 |                             |            |                    |            |
| <b>945GC<sup>[1]</sup></b> | Lakeport-DC               | 82945GC (MCH)                | ICH7/ICH7R/ICH7-DH  | 2006-02 <sup>[2]</sup> | Pentium 4, Pentium D, Celeron D, Core 2 Duo, Pentium Dual-Core, Atom | 130 nm      |                                   | 533-800 (last rev. 1066)            | DDR2 400/533/667               | 2 GB (some boards supports 4 GB shrink to 3.27 GB) <sup>[3]</sup> | No/No      | 1x16                                            | 945G                        |            |                    |            |
| <b>945GZ</b>               | Lakeport-GZ               | 82945GZ (MCH)                | ICH7                | 2006-08                | Pentium 4, Pentium D, Celeron D, Core 2 Duo, Pentium Dual-Core       |             |                                   |                                     | DDR2 400/533                   | 4 GB (shrink to 3.27 GB due to chipset limitation [4])            |            | some motherboards have 1x16 PCIe <sup>[5]</sup> | 4 from (Ch <sup>[6]</sup> ) |            |                    |            |
| <b>949PLG</b>              | Lakeport-PL               | 82949PL (MCH)                | ICH7/ICH7R          | 2006-07                |                                                                      |             |                                   |                                     | DDR2 400/533                   | 4 GB                                                              |            | 1x16                                            | N/A                         |            |                    |            |
| <b>949GZG</b>              | Lakeport-GZ               | 82949GZ (MCH)                |                     |                        |                                                                      |             |                                   |                                     | DDR2 533/667                   | 4 GB                                                              |            | 1x16                                            | 945G 3000                   |            |                    |            |
| <b>P965G</b>               | Prodeuster <sup>[7]</sup> | 82P965G (MCH)                | ICH8/ICH8R/ICH8-DH  | 2006-08                | Pentium Dual-Core, Core 2 Quad, Core 2 Duo                           | 130 nm      | No                                | 800-1066/1066                       | DDR2 667/800/1000              | 8 GB                                                              | No/No      | 1x16, 1x4                                       | N/A                         |            |                    |            |
| <b>Q955G</b>               | Prodeuster-GS             | 82Q955G (MCH)                |                     |                        | Pentium Dual-Core, Core 2 Duo                                        |             |                                   |                                     | DDR2 667/800/1000              | 8 GB                                                              |            | 1x16                                            | 945G X3000                  |            |                    |            |
| <b>Q955G</b>               | Prodeuster-GS             | 82Q955G (MCH)                |                     |                        |                                                                      |             |                                   |                                     | DDR2 667/800/1000              | 8 GB                                                              |            | 1x16                                            | 945G 3000                   |            |                    |            |
| <b>Q955G</b>               | Prodeuster-GS             | 82Q955G (MCH)                |                     |                        |                                                                      |             |                                   |                                     | DDR2 667/800/1000              | 8 GB                                                              |            | 1x16                                            | 945G 3000                   |            |                    |            |
| <b>975XG</b>               | Glenwood                  | 82975XG (MCH) <sup>[8]</sup> | ICH7/ICH7R/ICH7-DH  | 2006-11                | Pentium D, Core 2 Quad, Core 2 Duo, Pentium Dual-Core                | 90 nm       | Yes <sup>3</sup>                  | 800-1066/1066                       | DDR2 667/800/1000 <sup>3</sup> | 4 GB                                                              | Yes/Yes    | 1x16 <sup>3</sup> , 2x4                         | N/A                         |            |                    |            |
| <b>P31G</b>                | Bearlake (P)              | 82P31G (MCH)                 | ICH7                | 2007-03                | Pentium Dual-Core, Core 2 Duo, Core 2 Quad                           |             |                                   |                                     | DDR2 667/800                   | 4 GB                                                              |            | 1x16 rev. 1.1                                   | 945G 3100                   |            |                    |            |
| <b>G31G</b>                | Bearlake (G)              | 82G31G (MCH)                 |                     |                        |                                                                      |             |                                   |                                     | DDR2 667/800                   | 8 GB                                                              |            | 1x16 rev. 1.1                                   | 945G 3100                   |            |                    |            |
| <b>G33G</b>                | Bearlake (G+)             | 82G33G (MCH)                 |                     |                        |                                                                      |             |                                   |                                     | DDR2 800/1066                  | 4 GB                                                              |            | 1x16, 1x4 rev. 1.1                              | N/A                         |            |                    |            |
| <b>P35G</b>                | Bearlake (P+)             | 82P35G (MCH)                 | ICH8/ICH8R/ICH8-DH  | 2007-03                | Pentium Dual-Core, Core 2 Duo, Core 2 Quad                           |             | Yes <sup>3</sup>                  | 800-1066/1066                       | DDR2 667/800                   | 8 GB                                                              | No/No      | 1x16 rev. 1.1                                   | 945G X3000                  |            |                    |            |
| <b>G35G</b>                | Bearlake                  | 82G35G (MCH)                 |                     |                        |                                                                      |             |                                   |                                     | DDR2 667/800                   | 8 GB                                                              |            | 1x16 rev. 1.1                                   | 945G X3000                  |            |                    |            |
| <b>Q33G</b>                | Bearlake (Q?)             | 82Q33G (MCH)                 |                     |                        |                                                                      |             |                                   |                                     | DDR2 667/800                   | 8 GB                                                              |            | 1x16 rev. 1.1                                   | 945G X3000                  |            |                    |            |
| <b>Q35G</b>                | Bearlake (Q)              | 82Q35G (MCH)                 |                     |                        |                                                                      |             |                                   |                                     | DDR2 667/800/1000              | 16 GB                                                             |            | 1x16 rev. 1.1                                   | 945G X3000                  |            |                    |            |
| <b>E41G</b>                | Exagolate (G)             | 82E41G (MCH)                 | ICH7                | 2008-08                | Core 2 Duo, Core 2 Quad                                              | 65 nm       | No                                | 800-1066/1033 (P43 unofficial 1000) | DDR3                           | 4 GB                                                              | No/No      | 1x16 rev. 1.1                                   | 945G X4800                  |            |                    |            |
| <b>E43</b>                 | Exagolate (G)             | 82E43G (MCH)                 |                     |                        |                                                                      |             |                                   |                                     | ICH10D                         | DDR3                                                              |            | 8 GB                                            | 1x16 rev. 1.1               | 945G X4800 |                    |            |
| <b>P41G</b>                | Exagolate (P)             | 82P41G (MCH)                 |                     |                        |                                                                      |             |                                   |                                     | ICH10/ICH10R                   | 2008-08                                                           |            | Core 2 Duo, Core 2 Quad                         | DDR3 800/1066/1333          | 16 GB      | 1x16, 2x4 rev. 2.0 | N/A        |
| <b>P45G</b>                | Exagolate (P+)            | 82P45G (MCH)                 |                     |                        |                                                                      |             |                                   |                                     |                                |                                                                   |            |                                                 | DDR2 800/1066/1066+         | 16 GB      | 1x16, 2x4 rev. 2.0 | 945G X4800 |
| <b>Q43G</b>                | Exagolate (Q)             | 82Q43G (MCH)                 | ICH10/ICH10R/ICH10D | 2008-08                | Core 2 Duo, Core 2 Quad                                              | 65 nm       | Yes <sup>3</sup>                  | 800-1066/1333                       | DDR3 800/1066                  | 16 GB                                                             | No/No      | 1x16 rev. 2.0                                   | 945G X4800                  |            |                    |            |
| <b>Q45G</b>                | Exagolate (Q+)            | 82Q45G (MCH)                 |                     |                        |                                                                      |             |                                   |                                     | DDR3 800/1066                  | 16 GB                                                             |            | 1x16 rev. 2.0                                   | 945G X4800                  |            |                    |            |
| <b>Q47G</b>                | Exagolate (Q)             | 82Q47G (MCH)                 |                     |                        |                                                                      |             |                                   |                                     | ICH10/ICH10R/ICH10D            | DDR3 800/1066                                                     |            | 16 GB                                           | 1x16 rev. 2.0               | 945G X4800 |                    |            |
| <b>Q49G</b>                | Exagolate (Q)             | 82Q49G (MCH)                 |                     |                        |                                                                      |             |                                   |                                     | ICH10/ICH10R/ICH10D            | DDR3 800/1066                                                     |            | 16 GB                                           | 1x16 rev. 2.0               | 945G X4800 |                    |            |
| <b>X38G</b>                | Bearlake (X)              | 82X38G (MCH)                 | ICH8/ICH8R/ICH8-DH  | 2007-09 <sup>[9]</sup> | Core 2 Duo, Core 2 Quad, Core 2 Extreme                              | 90 nm       | Yes <sup>3</sup>                  | 800-1066/1333/1600                  | DDR3 800/1066/1333             | 16 GB                                                             | No/No      | 2x16 rev. 2.0                                   | N/A                         |            |                    |            |
| <b>X48G</b>                | Bearlake (X)              | 82X48G (MCH)                 |                     | 2008-03                |                                                                      |             |                                   |                                     | DDR2 800/1000/1200             | 16 GB                                                             |            | 2x16 rev. 2.0                                   | N/A                         |            |                    |            |

<sup>[27]</sup> Remapping of PCE/ATC memory ranges not supported. some physical memory might not be accessible (e.g. limited to 3.5 GB or similar). Operational configuration is 4 banks - 2x 2 GB dual rank modules or 4x 1 GB single rank modules - depends on number of motherboard DDR2 slots.

Рис.347 – Чипсеты Core 2

| Chipset   | Part Number | Parallel ATA   | Serial ATA          |           | AHCI Support        | RAID Levels  | USB                 | TDP [W]             |     |
|-----------|-------------|----------------|---------------------|-----------|---------------------|--------------|---------------------|---------------------|-----|
|           |             |                | 3.0Gbit/s           | 1.5Gbit/s |                     |              | v2.0                |                     |     |
| ICH6-M    | 82801FBM    | UDMA 100/66/33 | N/A                 | 2 ports   | Yes                 | None         | 4 ports             | 3.8                 |     |
| ICH6      | 82801FB     |                |                     | 4 ports   | No                  | None         | 8 ports             |                     |     |
| ICH6R     | 82801FR     |                |                     | 4 ports   | Yes                 | 0, 1, Matrix | 8 ports             |                     |     |
| ICH7-M    | 82801GBM    |                | 4 ports             | N/A       | 2 ports             | Yes          | No                  | 4 ports             | 3.3 |
| ICH7-M DH | 82801GHM    |                |                     |           | Yes                 | 0, 1, Matrix | 8 ports             |                     |     |
| ICH7      | 82801GB     |                |                     |           |                     | No           |                     | None                |     |
| ICH7DH    | 82801GDH    |                |                     |           |                     | Yes          |                     | 0, 1, Matrix        |     |
| ICH7R     | 82801GR     |                |                     |           |                     |              |                     | 0, 1, 5, 10, Matrix |     |
| ICH8M     | 82801HM     | 3 ports        |                     |           | None                | 10 ports     | 2.4                 |                     |     |
| ICH8M-E   | 82801HEM    |                |                     |           | 0, 1, Matrix        |              |                     |                     |     |
| ICH8      | 82801HB     | 4 ports        |                     |           | None                |              | 3.7                 |                     |     |
| ICH8R     | 82801HR     | 6 ports        |                     |           | 0, 1, 5, 10, Matrix |              |                     |                     |     |
| ICH8DH    | 82801HH     |                |                     |           |                     |              |                     |                     |     |
| ICH8DO    | 82801HO     |                |                     |           |                     |              |                     |                     |     |
| ICH9M     | 82801IBM    | No             | 4 ports             |           |                     |              | None                | 8 ports             | 2.5 |
| ICH9M-E   | 82801IEM    |                |                     |           |                     |              | 0, 1, Matrix        | 12 ports            | 4.3 |
| ICH9      | 82801IB     |                |                     |           |                     |              | None                |                     |     |
| ICH9R     | 82801IR     |                |                     |           |                     |              | 0, 1, 5, 10, Matrix |                     |     |
| ICH9DH    | 82801IH     |                |                     |           |                     |              |                     |                     |     |
| ICH9DO    | 82801IO     |                |                     |           |                     |              |                     |                     |     |
| ICH10     | 82801JB     |                |                     | 6 ports   |                     | None         |                     |                     |     |
| ICH10D    | 82801JH     |                |                     |           |                     |              |                     |                     |     |
| ICH10R    | 82801JR     |                | 0, 1, 5, 10, Matrix |           |                     |              |                     |                     |     |
| ICH10DO   | 82801JO     |                |                     |           |                     |              |                     |                     |     |

Рис.348 – Южные мосты серий 9xx и 3/4

Приведённые выше обширные (далеко неполные) списки выпускавшихся чипсетов закрепляют факт, что Intel со временем (конец 1990-х – начало 2000-х) была уже не просто производителем отдельных центральных процессоров, но также выпускала совершенно невероятное количество вариантов наборов микросхем поддержки, для построения вычислительных платформ на их основе. Можно было найти чипсет от Intel практически под любую задачу: от экономичных мобильных решений (ноутбуки) до мощных четырёх- или восьми-процессорных серверных платформ на основе Pentium Xeon.

Такой подход, когда производитель CPU в обязательном порядке выпускал комплекты микросхем поддержки под каждый свой процессор, привёл через некоторое время к фактическому исчезновению рынка сторонних производителей чипсетов. В 2010-году с этого рынка ушёл последний альтернативный производитель чипсетов для процессоров Intel и AMD – калифорнийская Nvidia (кстати, территориально ближайший сосед Intel, обе компании находятся в г.Санта-Клара), выпускавшая когда-то великолепные чипсеты под процессоры обоих производителей. Теперь на этом «поле» присутствовали только чипсеты от производителей CPU x86, которых в начале 2000-х осталось практически только два: Intel и AMD. Существовали нишевые решения, например, чипсеты от Hewlett Packard, реализующий поддержку до восьми Xeon-ов в самых мощных серверных платформах, но это была очень незначительная доля от общего количества выпускаемых чипсетов, здесь можно вспомнить, что только весьма удачных интеловских i440BX было произведено свыше 100 млн. штук.

## Шина AGP

Начиная с процессоров Pentium II в десктопных компьютерах стала активно использоваться ещё одна новая шина – AGP, которая несколько раз упоминалась в тексте ранее.

В системах поколения P5 фактическим стандартом стало сочетание двух периферийных параллельных шин: 16-битной ISA и 32-битной PCI. Поддержка шины EISA была убрана из поздних чипсетов (хотя и присутствовала в предпоследнем чипсете 430-ой серии HX), USB только набирала обороты, её поддержка имела только в двух чипсетах, и только самых первых версий (1.0 и 1.1).

При переходе на более производительную архитектуру P6 первыми периферийными устройствами, которым стало не хватать пропускной способности шины данных поколения P5, были, конечно же, видеоадаптеры. Росло количество точек (пикселей), глубина цвета, количество кадров в секунду, технологии 3D – и вот уже PCI, с её максимально возможными 133 Мбайт/с (на всех) становится узким местом. А ведь были ещё и другие претенденты на эту же полосу пропускания – жёсткие диски, у которых скорости обмена к тому времени выросли до нескольких десятков Мбайт/с, где-то на подходе уже был гигабитный Ethernet (а это сразу 125 Мбайт/с). Но главным и первым, конечно, нужно было обеспечить высокую скорость обмена данными для видеоадаптеров.

Вероятно, были варианты перехода на шину PCI-X, с увеличенной разрядностью и скоростью обмена по сравнению с первой классической шиной PCI 32 бита/33 МГц. В максимальной версии шина PCI-X могла работать со скоростью передачи 64-битных данных до 4 Гбайт/с. Но это направление на массовых десктопах не получило распространения – оказалось слишком дорогим, и нашло применение только в серверном сегменте.

В результате на роль палочки-выручалочки пришла новая, разработанная Intel, 32-разрядная AGP, узкоспециализированная шина, предназначенная для работы только с видеоадаптером. Немного напоминает ситуацию с шиной VLB в своё время – эдакое сиюминутное, быстрое решение, которое, впрочем, продержалось на рынке довольно долго – с 1996 по 2004 год, когда AGP была вытеснена новой революционной версией шины PCI Express.

Более подробное описание **AGP** (Accelerated Graphics Port, ускоренный графический порт): специализированная 32-разрядная системная шина для видеокарты, разработанная в 1996 году компанией Intel. Появилась одновременно с чипсетами для процессора Intel

Pentium MMX; у сторонних производителей появилась в чипсетах MVP3, MVP5 с Super Socket 7. Основной задачей разработчиков было увеличение производительности и уменьшение стоимости видеокарты, за счёт уменьшения количества встроенной видеопамати. По замыслу Intel, большие объёмы видеопамати для AGP-карт были бы не нужны, поскольку технология предусматривала высокоскоростной доступ к общей памяти. Её отличия от предшественницы, шины PCI (AGP была разработана на базе стандарта PCI R2.1):

- работа на тактовой частоте 66 МГц;
- увеличенная пропускная способность (устранено мультиплексирование линий адреса и данных);
- режим работы с памятью DMA и DME;
- разделение запросов на операцию и передачу данных;
- возможность использования видеокарт с бóльшим энергопотреблением, нежели PCI.

Вот так выглядел слот AGP на материнской плате (Рис.349) – он всегда располагался ближе всех остальных разъёмов к процессору.

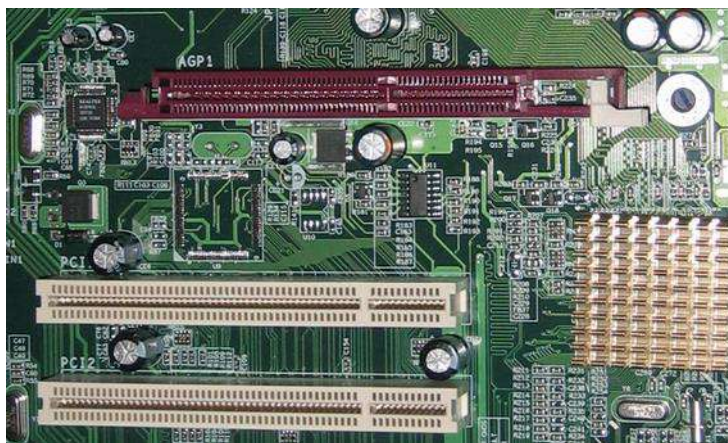


Рис.349 - Слот AGP (фиолетовый) и два слота PCI (белые)

### **Модификации AGP**

Первая версия (спецификация AGP 1.0) AGP 1x используется редко, поскольку не обеспечивает необходимой скорости работы с памятью в режиме DME.

Сразу же при проектировании была добавлена возможность посылать 2 блока данных за один такт — это **AGP 2x**.

#### **AGP 4x**

В 1998 году вышла вторая версия (спецификация AGP 2.0) — AGP 4x, которая могла пересылать уже 4 блока за один такт и обладала пропускной способностью около 1 ГБ/с. Уровень напряжения вместо обычных 3.3В был понижен до 1.5 В.

#### **AGP 8x**

Шина AGP 8x (спецификация AGP 3.0) передаёт уже 8 блоков за один такт, таким образом, пропускная способность шины достигает 2 ГБайт/с. Также в стандарте была заложена возможность использования двух видеокарт (аналогично AMD CrossFireX, Nvidia SLI), однако эта возможность не была использована производителями. Современные видеокарты требуют большой мощности питания, более 40 Вт, которую шина AGP дать не может, так появилась спецификация AGP Pro с дополнительными шинами питания на разъёме.



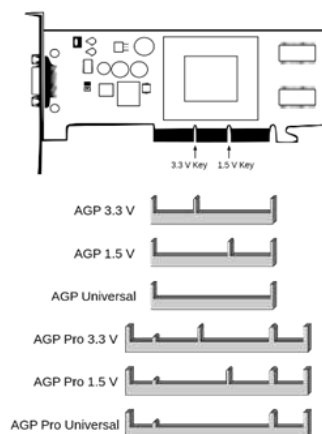


Рис.350 – Версии AGP

Вот так выглядит обобщённая архитектура компьютера с шиной AGP:



Рис.351 – Архитектура компьютера с шиной AGP

## Схемы компьютеров на базе P6 и чипсетов i440/450

Ниже будут приведены несколько структурных схем компьютеров на основе процессоров шестого поколения и чипсетов 440/450-серий.

Ранее, разделяя свои продукты, как правило, все одного поколения, лишь тактовыми частотами, Intel оставляла возможность постепенного перевода компьютера из одного класса в другой. Например, приличная рабочая станция на базе i486DX2-66 вполне могла превратиться в сервер, путем наращивания памяти, объёма кэша и установки производительного по тем временам SCSI адаптера для шины VLB, или наоборот, быть низведена в ранг офисного PC, путём замены процессора на i486DX-33 и удаления кэша вообще, чем и достигалась трёхкратная разница в производительности. Но во времена Pentium/Celeron/Xeon разделение проводилось уже не только по процессору, но и по поддерживаемому его чипсету. По этому критерию можно было выделить несколько категорий:

- **Basic PC** — основной уровень — семейство Celeron, чипсет — *i440EX*. Позже выйдет его модифицированный вариант, со встроенной i740 видеокартой.
- **Workstation PC** — рабочая станция — Pentium II и процессорная шина Slot 1, теперь уже в 100 МГц варианте. Мир AGP и PC100 SDRAM. Чипсеты i440LX и постепенно вытесняющий его i440BX.

- **Entry, Midrange Server** — сервер базового уровня — процессор Xeon. Картридж Slot 2. Чипсет i440GX — серверный вариант i440BX.
- **High Server** — мощный сервер — Xeon, но, как правило, с большим объемом L2 кэша — 1-2 Мбайт, чипсет i450NX.

На Рис.352 представлена схема компьютера на базе самого популярного чипсета i440BX AGPset. Существовали материнские платы на i440BX с двумя процессорными разъёмами, т.к. этот чипсет поддерживал работу в двухпроцессорной SMP конфигурации.

Чипсет i440BX AGPset состоит из двух микросхем: 82443BX и 82371AB/EB. Микросхема 82443BX – Host Bridge/Controller PCI AGP (PAC) в 492-pin Ball Grid Array (BGA). Микросхема 82371AB/EB – PCI-to-ISA/IDE Xcelerator (PIIX4/PIIX4E) в 324-pin Ball Grid Array (BGA).

По тогдашним современным критериям существенным недостатком i440BX являлось отсутствие поддержки протокола UltraDMA/66, поддерживаемого всеми основными производителями жёстких дисков (IBM, Quantum, Western Digital, Seagate, Fujitsu, Maxtor и т. д.). Однако эту проблему производители материнских плат решали установкой дополнительного UltraDMA/66-контроллера, реализуемого с помощью специализированных микросхем, например, High Point HPT 366. Именно с помощью такой микросхемы реализована данная возможность в некоторых материнских платах фирмы Abit, а также в ряде изделий других фирм, выпускающих подобные платы.

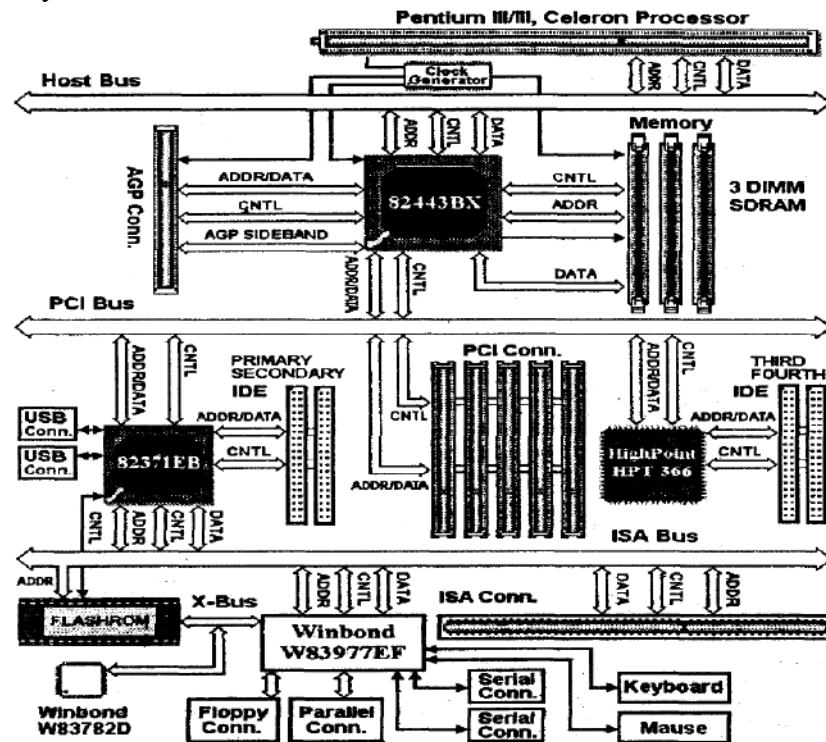


Рис.352 - Структура компьютера с чипсетом i440BX AGPset, контроллером UltraDMA/66 (High Point HPT 366) и микросхемой аппаратного мониторинга (Winbond W83782D) и ввода/вывода (Winbond W83977EF).

Далее на Рис.353 представлена структура компьютера на чипсете для рабочих станций среднего и высокого уровня i440GX AGPset. Как и многие другие, этот чипсет состоит из двух микросхем, модификации уже известного нам по TX, LX и BX чипа PIIX4 (PCI ISA IDE Xcelerator 4) и, собственно, самого GX (82443GX Host Bridge/Controller). Как и его предшественники, построен он по модной тогда архитектуре QPA (Quad Port Architecture) и представляет собою коммутатор между четырьмя интерфейсами: AGP, PCI, FSB и шиной памяти.

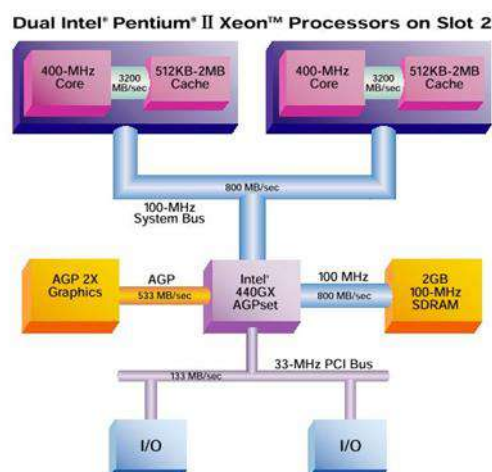


Рис.353 – Структура двухпроцессорного компьютера с чипсетом i440GX AGPset.

Немного подробнее по схеме Рис.353:

- FSB: Slot 1 и Slot 2 процессоры с внешней частотой (64-битной) шины 100 МГц. Поддержка отложенных транзакций и SMP для двух процессоров. Режим 66 МГц больше не поддерживается (частота PCI всегда равна частоте FSB/3).
- AGP: 1x (66 МГц) и 2x (133 МГц) режимы, возможно только одно AGP устройство.
- PCI: спецификация версии 2.1 (32 бит, 5.5 и 3.3 вольта, 33 МГц), и все возможности известные еще со времён чипсета i430TX.
- MEMBUS: До 8 банков 100 МГц SDRAM памяти (4 разъёма 16-, 64-, 128- и еще не выпускаемых на тот момент серийно 256-Мбит чипы на одно- и двухбанковых модулях DIMM). Итого – возможность установки 16 Мбайт...2 Гбайт памяти с поддержкой коррекции ошибок (ECC). Первое серьезное отличие от LX и VX чипсетов, если не считать поддержки необходимых для Slot 2 уровней логических сигналов.

Структура самого мощного серверного варианта на базе чипсета i450NX PCIset приведена на Рис.354 ниже.

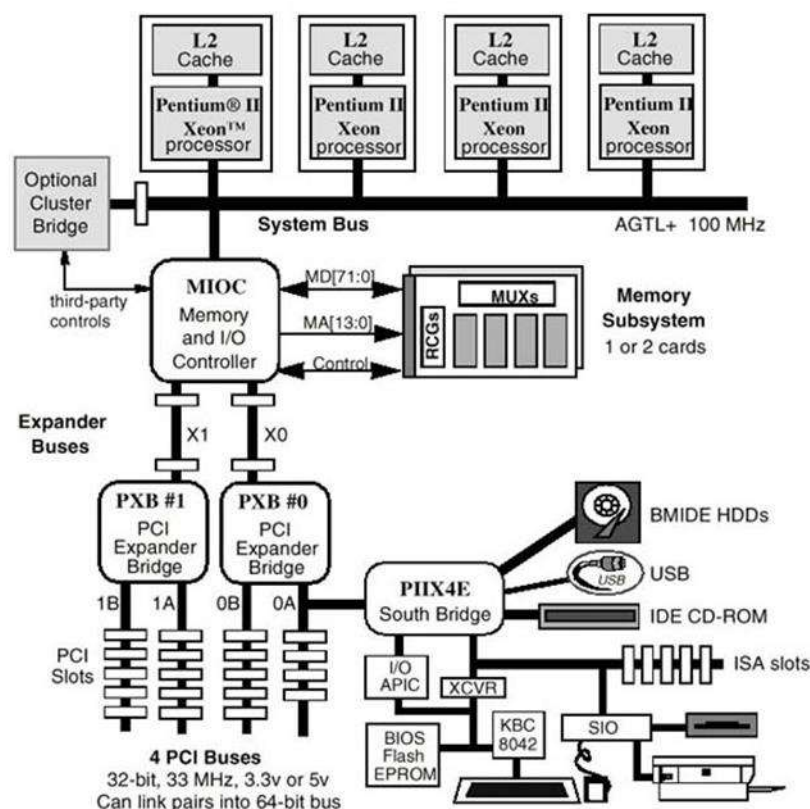


Рис.354 - Структура четырёхпроцессорного компьютера с чипсетом i450NX PCIset

Если i440GX был по сути модификацией чипсета BX, то i450NX PCIset был действительно новым набором, имеющим собственную оригинальную архитектуру, нацеленную на достижение максимальной производительности в системе с несколькими процессорами. Первое, что бросается в глаза — это отсутствие слова AGP в названии чипсета. Вряд ли сервер стоимостью \$100000 будет использоваться в качестве персональной графической станции. Основные характеристики чипсета NX:

- Поддержка до четырёх (с применением специальных расширителей — до восьми) Slot 2 процессоров
- Поддержка полной 36 битной адресации
- До 8 Гбайт памяти, DIMM 50...60 нс, 3.3 вольт, 16- и 64- мегабитные чипы, память устанавливается блоками по 64 Мбайта
- Чередувание (interleave) обращений к памяти, с четырьмя банками
- До четырёх шин PCI 32 бита, 33 МГц или до двух PCI 64 бит
- Хорошая зависимость производительности в зависимости от числа процессоров (производительность возрастает до 3.5 раз при четырёх процессорах)
- Чередувание обращений bus-master карт к памяти
- До 6 masters и всего до 10 карт на каждой PCI шине
- Коррекция ошибок (ECC) на шине памяти и FSB
- Диагностика ошибок (Parity) по всем сигналам PCI
- Предвыборка данных при чтении из памяти

За подобные возможности, естественно, приходилось расплачиваться ценою, рассеиваемой чипсетом мощностью и количеством кристаллов, из которых он состоит. Рассеивается 30 Ватт тепла (только чипсет, не считая памяти и процессоров). Набор состоит из чипов четырех типов:

- 82451NX Memory and I/O Bridge Controller (MIOC) — самый «главный» чип, коммутатор
- 82454NX PCI Expander Bridge (PXB) — контроллер шин PCI (две 32 бит или одна 64 бита)
- 82452NX RAS/CAS Generator (RCG) — генератор сигналов управления памятью
- 82453NX Data Path Multiplexor (MUX) — мультиплексор данных памяти

Причём, первый из них как правило один (возможны конфигурации с двумя наборами на одной плате, для поддержки 8 процессоров, но они требуют специальных заказных чипов), контроллеров шин может быть два, а генераторов сигналов и мультиплексоров до четырех, в зависимости от количества поддерживаемой памяти. Реальная скорость считывания из памяти с 4-х кратным чередованием 1.067 Гбайт/сек, что несколько превышает предельную пропускную способность FSB Xeon. Сделано это не зря, четыре PCI шины способны одновременно затребовать до 400 Мбайт/сек. Возможна установка на одну из PCI шин RPH4, для обеспечения совместимости с PC архитектурой и периферией.

## Схемы компьютеров на базе процессоров P6 и NetBurst и чипсетов 800-ой серии

### Чипсеты i810 и i810E

Чипсет i810 был разработан на основе i440BX AGPset. Предназначен для компьютеров начального уровня (Basic PC, Value PC). Оптимизирован для процессоров Celeron, хотя может быть использован и с более мощными процессорами типа Pentium II.

Структура чипсета i810 базируется на использовании хабовой архитектуры (Accelerated Hub Architecture). i810 — это первый чипсет, в котором компания Intel реализовала данную архитектуру. Такая архитектура позволяет минимизировать загрузку процессора при операциях обмена, однако потребовалось коренным образом изменить традиционную структуру чипсетов, состоящую обычно из двух микросхем — North Bridge и South Bridge, соединенных ставшей уже традиционной шиной PCI с частотой 33 МГц и с пропускной способностью до 133 Мбайт/с.

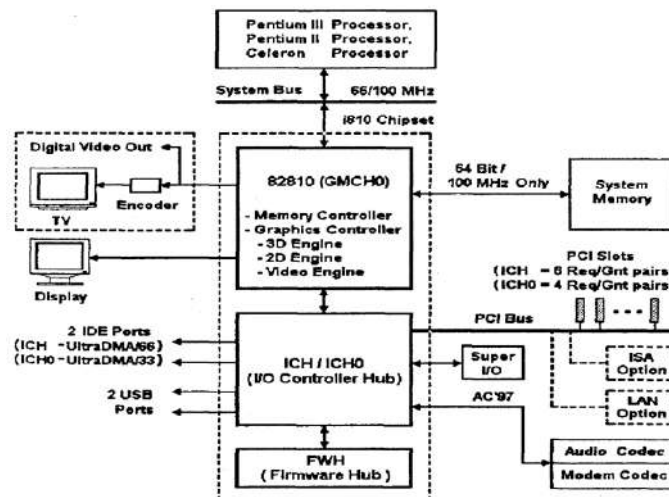


Рис.355 - Структура компьютера с чипсетом i810 Chipset

В качестве основной шины для соединения своих компонентов — хабов в чипсете i810 применяется не традиционная, сравнительно медленная шина PCI, а специально разработанная, закрытая шина, обеспечивающая пропускную способность до 266 Мбайт/с. В соответствии с современными тенденциями, сформулированными, в частности, в спецификациях PC99 и PC2000, в архитектуре чипсета i810 вообще не была предусмотрена шина ISA.

Состоит чипсет i810 из трёх микросхем: 82810 Graphics and Memory Controller Hub (GMCH) в 421-pin Ball Grid Array (BGA), 82801 Integrated I/O Controller Hub (ICH) в 241-pin BGA, 82802 Firmware Hub (FWH) в 32-pin PLCC или 40-pin TSOP.

### Чипсеты i815 и i815E

Чипсеты i815 и i815E также построены на основе использования хабовой архитектуры (Accelerated Hub Architecture) и предназначены для высокопроизводительных компьютеров с процессорами типа Pentium II/III и Celeron с разъемами Slot 1 и Socket 370 и частотой шины FSB 66/100/133 МГц.

На Рис.356 приведена структурная схема компьютера на основе чипсета i815E. Встроенный в чипсет контроллер памяти поддерживает 64-битный интерфейс памяти SDRAM, объём памяти – от 32 до 512 Мбайт.

Встроенные средства поддерживают: AGP 2.0 с поддержкой режимов AGP 1X/2X/4X, интегрированную графику на основе i752; PCI Rev.2.0, до 6 устройств PCI; 2 (i815) или 4 (i815E) порта USB; 2 порта IDE либо с UltraDMA/33/66 (i815), либо с UltraDMA/33/66/100 (i815E); интерфейс LPC (Low Pin Count); контроллер LAN (i815E); AC'97 audio с 2 (i815) или с 6 (i815E) каналами; ACPI; Hardware monitoring и другие функции и устройства.

Чипсет i815 состоит из трёх микросхем: 82815 Graphics and Memory Controller Hub (GMCH), 82801AA I/O Controller Hub (ICH), 82802 Firmware Hub (FWH). Чипсет i815E состоит из трёх микросхем: 82815 Graphics and Memory Controller Hub (GMCH), 82801BA I/O Controller Hub (ICH2), 82802 Firmware Hub (FWH).

Данные чипсеты, рассчитанные на память SDRAM (PC100/PC133), фактически являлись достойными преемниками чипсетов i440BX и i440ZX, хотя фирма Intel когда-то планировала на эту роль ранее выпущенные чипсеты i820 и i820E с памятью Rambus.

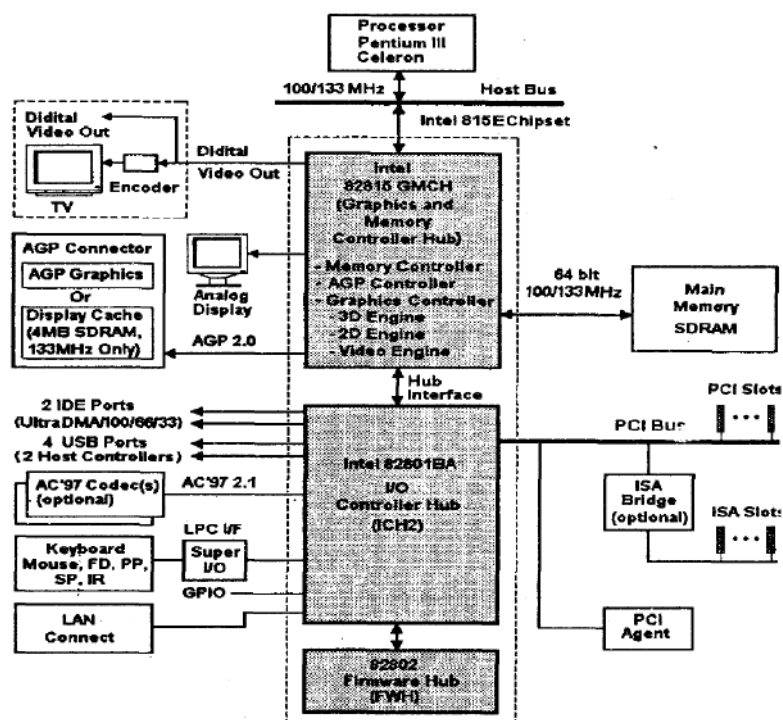


Рис.356 - Структура компьютера с i815E Chipset

### Чипсет i845

Первые Pentium 4 работали с памятью Rambus (RDRAM), для поддержки которой Intel выпускала чипсеты i820- и i850-ой серий. Но в начале 2000-х стало понятно, что эта

память не приживётся на рынке, в первую очередь из-за очень высокой стоимости, а системы с RDRAM на базе Pentium 4, соответственно, будут очень плохо продаваться. Поэтому Intel начинает разработку другого чипсета, ориентированного на обычную память SDRAM, а также на набирающую популярность DDR SDRAM. И вот в сентябре 2001 года был официально представлен чипсет i845 (кодовое название “Brookdale”). Это не была переделка i850 или i815, это был совершенно новый чипсет.

Как и все современные чипсеты Intel того времени, i845 был построен по хабовой архитектуре. Чипсет состоит из четырех чипов: системного контроллера MCH 82845, хаба ввода-вывода ICH2 82801BA, Flash-хаба FWH и контроллера устаревших интерфейсов Super I/O. Все чипы, кроме первого, унифицированы и используются в других чипсетах Intel. ICH2 содержит контроллеры PCI, ATA/ATAPI, USB, LPC, AMR/CNR, а FWH содержит системный BIOS и генератор случайных чисел. В дальнейших модификациях версия хаба ввода-вывода выросла до ICH4. Именно такие варианты структуры компьютера на основе чипсета i845GL (интегрированное видео) и i845PE (AGP) приведены далее на Рис.357 и Рис.358, соответственно.

Центром нового чипсета является хаб MCH 82845. По сути, он является мостом (коммутатором, QPA (Quad Port Architecture)) между четырьмя интерфейсами, обеспечивая обмен данными между процессором (FSB), памятью, AGP-контроллером видеокарты и периферийными устройствами (точнее, хабом ICH2/4).

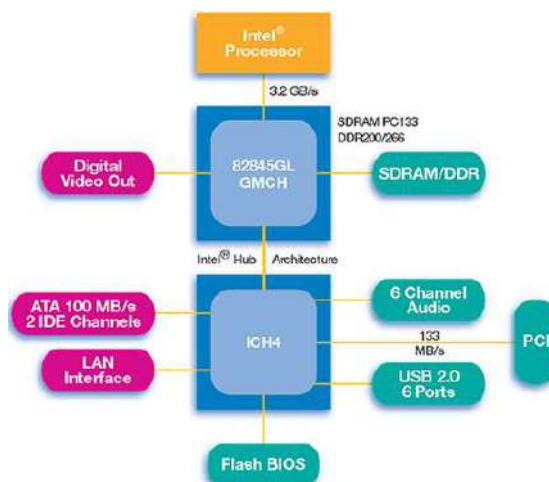
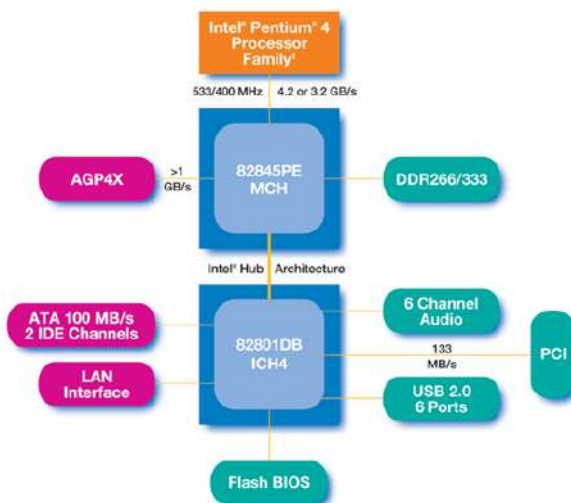


Рис.357 – Структура компьютера с чипсетом i845GL



<sup>1</sup> Validated with Intel® Pentium® 4 processor in the 478-pin package

Рис.358 - Структура компьютера с чипсетом i845PE



### Чипсеты Intel 865PE/G/P (Springdale) и Intel 875P (Canterwood)

Логичным шагом в популяризации новых топовых процессоров Pentium 4 с частотой шины 800 МГц (для его поддержки предназначался дорогой и производительный двухканальный DDR-чипсет i875P (Canterwood) и заодно технологи Hyper-Threading, стал выпуск относительно недорогой линейки чипсетов i865 (бывшая серия Springdale), обладающей практически теми же функциями, что и 875-ый чипсет, ориентированный на применение в рабочих станциях.

Чипсет Intel 875 отличался от i865 поддержкой двух дополнительных технологий:

- Памяти с ECC
- PAT<sup>1</sup>

Центром нового чипсета i865/875 был хаб MCH 82865/82875, который коммутировал уже не четыре, а шесть шин (Рис.359):

- Шина процессора (FSB)
- AGP 8x
- Два канала шины памяти
- Шина CSA (Communications Streaming Architecture, для чипов Intel Gigabit Ethernet)
- Шина обмена с периферийным хабом ICH5



Рис.359 - Структура компьютера с чипсетом i865 (слева) и i875 (справа)

Структура компьютера на основе чипсета i875 (Рис.359, справа) фактически ничем не отличается от схемы на основе чипсета i865.

Как было принято у Intel, чипсет с суффиксом G в названии означал версию с интегрированной графикой. Из архитектурных новшеств – поддержка шины CSA, предназначенной в первую очередь для подключения скоростных Ethernet адаптеров (гигабитных)

<sup>1</sup> Суть технологии PAT состоит в ускорении прохождения данных через MCH (не задействуются лишние в данном случае синхронизационные буфера внутри чипсета, данные передаются напрямую) в режиме работы FSB/памяти 800/400.

GbE). Для сокращения задержек и увеличения пропускной способности CSA была подключена «поближе» к памяти и процессору, в северный мост.

Ещё одной важной новинкой стала появившаяся в чипсете «родная» (native) поддержка SATA-дисков, пока только максимум двух (параллельных PATA-дисков можно было подключить до четырёх штук) и с некоторыми ограничениями. Примерно через год, в июле 2004г., с анонсом архитектуры Core 2, пал последний оплот параллельных шин в архитектуре PC – долгожитель PCI, параллельная периферийная шина образца 1992 года. Более высокочастотные и более «широкие» параллельные версии PCI и PCI-X так и не прижились в машинах нижнего и среднего ценового диапазона, присутствуя в основном в топовых серверных архитектурах, а базового 32-битного 33 МГц PCI с его максимальной скоростью (на всех) 133 Мбайт/с было давно уже недостаточно. Отсюда и откровенные «костыли» в виде шины CSA, для поддержки гигабитного Ethernet, который, будучи подключённым к обычному PCI, «съел» бы почти всю его полосу пропускания.

Южный мост (ICH5) существовал в двух версиях: в варианте с суффиксом R (ICH5R) в хаб был интегрирована лицензированная у компании Adaptec технология построения дисковых массивов RAID (только самых простых, уровня 0 или 1). В ICH5 также была встроена поддержка шестиканального 5.1 звука, что позволило разработчикам недорогих ПК на их основе отказаться от внешних звуковых карт.

Также следует отметить увеличенное количество портов USB 2.0, по сравнению с предшествующими чипсетами (845) – с шести до восьми. USB занял доминирующее положение в ПК, заменяя россыпь устаревших интерфейсов предыдущих поколений – LPT, COM, PS/2 и др.

### **Чипсеты Intel 925X/915P/915G Express**

Для поддержки работы Pentium 4 с новой памятью стандарта DDR2 Intel выпускает новые чипсеты 900-ой серии: 925X/915P/915G Express. Архитектурных различий между i925X и i915P нет, а i915G, что очевидно следует из его названия — это i915P со встроенной графикой (Рис.360).

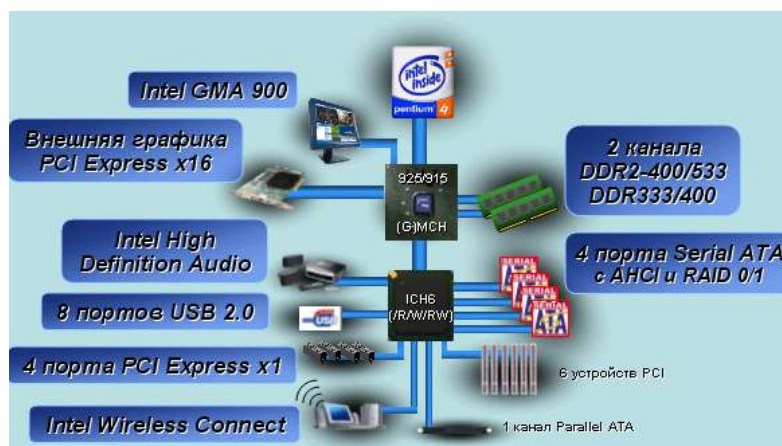


Рис.360 - Структура компьютера с чипсетом i915/i925.

Основные характеристики чипсетов i915/i925 и систем на их основе:

- **Поддержка процессоров с частотой шины 533/800 МГц.** Поддержка 533 МГц – только чипсеты серий i915, топовый i925X не поддерживает устаревшие модели Pentium 4 и, тем более, новые Celeron D). Как бы неявно подразумевается, что новые чипсеты будут работать только с новыми процессорами под Socket 775, но, т.к. шинный интерфейс не изменился, теоретически было возможно создание плат на i9xx под Socket 478. Есть поддержка Hyper-Threading.
- **Двухканальный контроллер памяти DDR333/400 и DDR2-400/533.** Здесь, опять-таки, вступает в силу разведение продуктов по рыночным нишам: топовый

i925X официально со старой DDR работать не будет — никаких компромиссов с прошлым. При выборе частоты функционирования памяти понижающих коэффициентов также не будет: DDR400/DDR2-400/533 при FSB 800 МГц (1:1 и 4:3) и DDR333/400 при FSB 533 МГц (5:4 и 3:2). Воспользоваться DDR2 удастся только совместно с процессорами на 800-мегагерцовой шине. Количественные параметры подсистемы памяти прежние: до 4 модулей/4 Гбайт, в i925X есть поддержка ECC и PAT, у i915 — нет.

- **Шина для внешних видеоускорителей PCI Express x16.** От AGP в этом чипсете Intel отказалась полностью, т.к. совмещать работу с обоими интерфейсами чипсета было бы очень сложно (слишком разные электрические параметры), а отдельный блок выводов для AGP 8x — непозволительная роскошь в условиях, когда MCH и так имеет более 1200(!) пинов.
- **Встроенная графика i915G.** Третье поколение современной встроенной графики Intel носит официальное название GMA (Graphics Media Accelerator) 900. GMA 900 отличается от своего предшественника в i865G повышенной частотой ядра (333 МГц против 266), числом конвейеров (4 против 1), аппаратной поддержкой DirectX 9 (против 7.1) и OpenGL 1.4 (против 1.3), поддержка более высоких частот обновления экрана (до 85 Гц в разрешении QXGA (2048x1536) и двух независимых изображений вместо одного в предыдущих поколениях встроенной графики Intel.
- **Южный мост ICH6(R/W/RW) и соединение с северным по шине DMI,** являющейся электрически измененной версией PCI Express x4 (~2 Гбайт/с). Существовало четыре варианта ICH6: минимальный, с функцией RAID, с функцией организации беспроводной сети (Wi-Fi) и комбинация двух последних.
- **4 порта PCI Express x1.** Конечно же, отнюдь не случайно полное название новых чипсетов выглядит как Intel 9xxx Express. Приход PCI-E<sup>1</sup> на место PCI — это событие, которое по своей важности перекрывает даже начало перехода на DDR2, не то что какую-то смену сокета.
- **Matrix Storage — поддержка устройств Serial ATA с RAID и AHCI (RAID только в версиях ICH6R/RW).** Теперь поддерживаются 4 порта SATA на 4 устройства с возможностью создания RAID 0 или 1 из любых двух дисков при независимом создании RAID 0 или 1 из двух других дисков. Однако со времен ICH5 произошли не только количественные перемены. Весной 2003 года на рынке практически не было жестких дисков с полноценным контроллером SATA (как правило, просто использовали интерфейсный мост PATA-SATA на управляющей плате винчестера). Поэтому реализация таких функций, как NCQ (Native Command Queuing, конвейеризация команд) и прочих из набора возможностей SATA AHCI (Advanced Host Controller Interface), была невозможна. В 2004 году уже были доступны винчестеры, реализующие эту базовую функциональность, так что новое поколение южных мостов Intel предлагает поддержку AHCI<sup>2</sup>.
- **High Definition Audio — новый стандарт для встроенного звука.** Основой стандарта HDA является поддержка 24-разрядного 8-канального (7.1) звука при частоте дискретизации 192 кГц — спецификация AC'97 определяла 20-разрядный 6-канальный звук с частотой дискретизации 96 кГц. Схема функционирования со времён AC'97 принципиально не изменилась.
- **Wireless Connect — организация беспроводной сети.** В паре с южным мостом ICH6W приобретаемая отдельно карта расширения реализует функциональность беспроводного сетевого контроллера стандарта IEEE 802.11b/g (Wi-Fi).

<sup>1</sup> Подробное описание шины PCI Express будет приведено в следующем разделе, посвящённом архитектурам процессоров Core и новее, для которых PCI-E (PCIe) станет основным стандартом, придя на замену PCI, всерьёз и надолго, к 2020 году добравшийся уже до версии PCI-E 5.0.

<sup>2</sup> Встроенный в Windows XP SATA-драйвер поддержки технологии AHCI не имеет.

- **Поддержка прочей периферии южным мостом.** Поддержка 8 портов USB 2.0. Несмотря на наличие «свежих» PCI Express и Serial ATA, была сохранена поддержка части старых периферийных интерфейсов – с ICH6 по-прежнему могут работать до 6 устройств PCI Bus Master, но вот число каналов Parallel ATA было урезано до одного (до двух устройств). Интерфейса, подобного CSA в чипсетах i865/i875 серий, предназначенного для подключения Gigabit Ethernet адаптера к северному мосту, в серии i9xxx Intel не реализовала, в ICH6 присутствовал лишь MAC-контроллер Fast Ethernet (10/100 Мбит/с), что было вполне логично – PCI-E x1 с большим запасом справляется с гигабитным потоком.

## Программное обеспечение

К моменту выпуска первых Pentium-ов на рынке уже присутствовало достаточно много качественного системного и прикладного программного обеспечения под 32-разрядную архитектуру x86. Программная индустрия сумела в целом наверстать существовавшее ранее долгие годы значительное запаздывание выхода на рынок операционных систем, соответствующих архитектуре и возможностям новых процессоров, относительно момента появления новых процессоров на рынке. Больше не повторялось ситуации, когда от выпуска процессора (i80386, 1985 год) до появления массовой полноценной 32-разрядной ОС пользователям пришлось ждать долгих восемь лет (Windows NT, 1993 год).

Microsoft в середине 1990-х присутствовала на рынке операционных систем для ПК сразу двумя семействами своих продуктов: 32-битной Windows NT (3.1/3.50/3.51/4.0/2000/XP<sup>1</sup>...) и 16-битной линейкой Windows 3.0/3.1/3.11 WfWG/95/98/Me.

ОС Windows NT изначально позиционировалась как система для мощных рабочих станций и серверов, исходно без поддержки продвинутой графики, тогда как 16-разрядная линейка рассматривалась больше как офисная, домашняя или игровая платформа. Главным её преимуществом были достаточно скромные требования к оборудованию, в первую очередь, к объёму ОЗУ компьютера. Ведь даже самая первая версия NT 3.1 в качестве минимального объёма для установки системы хотела видеть минимум 16 Мбайт ОЗУ – весьма солидная по тем временам цифра.

Несмотря на очевидные ограничения 16-разрядного режима, 16-разрядная Windows задержалась на рынке довольно надолго, едва не застав начало эпохи x86-64. Но, выпустив в 2000-ом году две знаковые версии своих ОС – ставшую последней в 16-битном семействе Windows Me, и первую версию Windows 2000 с поддержкой службы каталога AD, Microsoft поставила точку в этой истории внутренней конкуренции самой с собой. NT к тому времени уже умела быстро работать с графикой, включая полную поддержку 3D (API DirectX), и у 16-разрядной линейки просто уже не оставалось аргументом, оправдывающих её существование.

«Агония», тем не менее, была довольно активной – с августа 1995 года, даты выхода Windows 95, по 2000-ый, Microsoft выпустила пять(!) ревизий системы: Windows 95, Windows 95 OSR2, Windows 98, Windows 98 OSR2, Windows Me, фактически каждый год по новой версии. Если считать за точку отсчёта выпуск 16-разрядной Windows 3.0 (май 1990 года), то эта линейка ОС продержалась на рынке без малого 20 лет!

Хорошей, надёжной версией NT оказалась выпущенная в 1996 году WinNT версии 4.0 – уже тогда произошло разделение на версии для рабочих станций и серверов, включая поддержку многопроцессорных платформ. Но называть по-разному пользовательские и серверные версии Microsoft начала только с выходом WinXP и Windows Server 2003 – если пользовательские версии после «именной» Windows Vista просто нумеровались подряд: 7, 8, 8.1, «вечная» 10 (с пропуском «неудачной» цифры 9 в названии, чтобы у пользователей

<sup>1</sup> Windows XP, анонсированная 24 августа 2001 года, упомянута здесь потому, что попадает в рассматриваемый временной интервал выпуска процессоров поколений P5...P7.

не возникало ассоциаций со старыми «девятками», линейкой Windows 9x)<sup>1</sup>, то серверы маркировались по году выпуска версии, с последующим добавлением суффикса R2 через два года после выхода исходной версии: Windows Server 2003, Windows Server 2003R2, Windows Server 2008, Windows Server 2008R2 и т.д. Выход Windows 2000, с поддержкой службы каталога AD, закрепил присутствие Microsoft на корпоративном рынке.

Переход Microsoft с 32-разрядной платформы на AMD64/EMT64 произошёл значительно спокойнее и быстрее: после объявления расширения x86-64 архитектуры в 2003-2004 году, практически сразу появились 64-разрядные версии Windows, а уже начиная с 2010 года Microsoft объявила о выпуске серверных версий Windows только под 64-разрядные платформы<sup>2</sup>. Для пользовательских версий Windows 10 поддержка 32-разрядных платформ сохранена до 2023 года.

В мире, кроме Windows, конечно же существовали и другие операционные системы, работавшие на x86. Долгое время основным конкурентом на пользовательском рынке была IBM OS/2, всегда превосходившая по техническим параметрам ОС семейства Windows. Но откровенно провальная маркетинговая политика IBM в отношении этой ОС, а также не всегда честные методы, использовавшиеся Microsoft в борьбе со своим бывшим партнёром, привели сначала к продаже OS/2 сторонней компании, с довольно быстрым последующим её полным уходом с рынка. Фанаты системы пытались убедить IBM открыть исходные тексты OS/2, переведя её в категорию Open Source. Но IBM на этот шаг не пошла, возможно по причине того, что в исходных текстах OS/2 присутствует немало copyright от Microsoft, например, одним из авторов основной файловой системы HPFS (High Performance File System) в этой ОС был сотрудник Microsoft, а эта компания никогда не была горячим сторонником Open Source.

Но, начиная с 1991 года, очень быстро развивалась другая открытая операционная система – проект Линуса Торвальдса, Linux. Это крупнейший в истории проект с открытым исходным кодом, стремительно выросший с нуля до весьма и весьма солидных объёмов<sup>3</sup>. Например, Linux абсолютно доминирует на рынке высокопроизводительных вычислений, в списке Top500 самых производительных компьютеров практически все системы работают на том или ином диалекте Linux-a. Windows ушла с этого рынка (HPC) полностью.

Операционная система Linux в середине 1990-х бурно развивалась, разработчики ОС, напрямую конкурируя с Microsoft, старались максимально оперативно реагировать на появление на рынке нового оборудования, пытаясь не уступать по количеству и качеству поддержки самого разнообразного «железа» проприетарной закрытой Windows, имевшей большие предпочтения у производителей этого самого оборудования. Хотя часто реализация поддержки в Linux сталкивалась с очень серьёзными объективными трудностями, когда идеология Open Source входила в принципиальный конфликт с некоторыми производителями оборудования, не желавшими открывать полностью спецификации своего «железа» и/или исходные тексты драйверов под него. В первую очередь это касалось основных производителей видеокарт Nvidia, ATI, одного из ключевых производителей SCSI-оборудования компании Adaptec, полупроводникового гиганта Broadcom<sup>4</sup> и др.

Необходимо всё же отметить, что и сегодня в Linux-е гораздо хуже, чем в Windows, поддерживаются некоторые классы оборудования, в первую очередь – продвинутые видеоадаптеры, хотя определённые подвижки в сторону исправления этой ситуации в последние годы всё же происходят. Но подавляющее количество мощного, «тяжёлого» программного

---

<sup>1</sup> Несмотря на все заверения Microsoft, что десктопная версия Windows «навечно» останется с версией 10, в июне 2021 года было объявлено о выходе Windows версии 11.

<sup>2</sup> Первой 64-only версией был Windows Server 2008R2.

<sup>3</sup> К слову – компания RedHat, производитель коммерческих дистрибутивов Linux номер один в мире, впервые в истории ИТ-индустрии сумела достичь миллиардных оборотов, работая в парадигме Open Source.

<sup>4</sup> Строго говоря, в середине 1990-х Broadcom ещё не существовала. Компания, бывшая когда-то подразделением Hewlett Packard, стала самостоятельным юридическим лицом под названием Avago Technologies только в 2005 году.

обеспечения, работающего с 3D-графикой – САПРы, игры и т.п. – выпускаются сегодня по-прежнему только под платформу Windows.

Из заметных альтернативных «течений» в сфере ОС в те годы, кроме Linux, можно отметить BSD Unix-подобные операционные системы: очень осторожную и довольно консервативную OpenBSD<sup>1</sup>, быстро и динамично развивающуюся FreeBSD, очень популярную в России. Но эти системы были скорее серверного класса, на пользовательские десктопы из не-Windows ОС всерьёз можно было рассматривать всё-таки только Linux.

Существовала ещё одна линейка ОС, весьма успешно конкурировавшая с Windows как раз на рынке десктопов, чрезвычайно популярная в США – Mac OS, от давнего конкурента Microsoft, компании Apple. Но в описываемый период времени MAC-и работали на другом процессоре, PowerPC, созданным в своё время конгломератом разработчиков из трёх компаний: IBM, Motorola и Apple как альтернатива Pentium-у. Apple, с выпуском интеловских Core 2, о которых речь пойдёт в следующем разделе, в 2004 году перешла на платформу Intel, сильно расстроив этим многих своих поклонников. Кстати, сейчас грядёт новая смена платформы Apple – на этот раз планируется уход уже с интеловской платформы на ARM (Cortex) разработки Apple, выпускаемого по 5 нм проектным нормам. Этот переход (начало его, можно сказать, уже состоялось во второй половине 2020г.), спровоцирует ещё более глубокий кризис, в котором Intel пребывает последние несколько лет. Семь лет компания не может наладить выпуск достаточного количества процессоров по проектным нормам 7 нм. Её на серверном поле обыграла «вечно» вторая AMD, со своими великолепными EPYC, причём обыграла серьёзно, если можно так сказать – с «разгромным счётом». Подробности об этом – в следующем разделе.

Конечно же, необходимо упомянуть здесь ещё одну довольно популярную операционную систему для платформы x86 тех лет – Novell Netware. Как уже говорилось выше, это была не пользовательская ОС, а специализированный файл- и принт-сервер, написанный с использованием техники невытесняющей многозадачности (подобно Windows 3.0). В этой ОС раньше других появилась массовая и недорогая служба каталога NDS/eDir, во многом в ответ на которую появилась аналогичная служба AD (Active Directory) в Windows 2000 (на шесть лет позже, чем в Netware). Netware была отличным быстрым файл-сервером, но из-за невытесняющей многозадачности, «плоской» модели памяти, в которой программные модули не были так хорошо изолированы друг от друга, как в Windows или Linux, была откровенно неважным сервером приложений. В начале 2000-х Novell перенесла свои основные службы с проприетарной Netware на Linux SuSE, но, из-за откровенно бездарной маркетинговой политики и серьёзных просчётов руководства, практически полностью потеряла рынок локальных сетей, на котором ещё в середине 1990-х имела долю около 70%, а в 2010 году прекратила существование и сама компания Novell.

Коротко резюмируя по программному обеспечению для интеловской платформы можно сказать: в рассматриваемый в этом разделе период – 1993...2004 года – рынок программного обеспечения для архитектуры x86 приближался, но, на взгляд автора, пока ещё не достиг своей зрелости. Да, продержавшаяся на рынке рекордные почти 13 лет Windows XP (анонс – осень 2001 год, снята с поддержки – апрель 2014) может считаться серьёзным аргументом в пользу показателя зрелости этой операционной системы и рынка ОС для x86 в целом, но по факту, для более качественной и полноценной поддержки 64-битной архитектуры x86-64 компания Microsoft всё же была вынуждена разрабатывать новое, сильно обновлённое ядро<sup>2</sup>, ставшее в 2010 году основой для очень и очень приличного воплощения

<sup>1</sup> Пример консервативного подхода в OpenBSD: технология SATA была объявлена в январе 2003 года, а полноценно заработавшая поддержка SATA в OpenBSD появилась только осенью 2004 года, после неоднократных обращений к разработчикам.

<sup>2</sup> Появившаяся довольно быстро Windows XP 64-bit была по сути комбинацией 64-bit ядра Windows Server 2003 и графического интерфейса Windows XP. Из-за сильно отличающейся модели работы с памятью под XP 64 не было полноценной поддержки 3D-графики (в виде драйверов), в результате рабочие станции на базе этой ОС сильно уступали по графическим возможностям системам на основе 32-разрядных версий XP. По этой же причине на XP 64 не работали продвинутые 3D-игры, например.



в виде версии Windows 7 (промежуточная Windows Vista оказалась в целом всё-таки не самой успешной системой). Интересно, что Microsoft именно в версии Windows 7 сумела сделать единое с сервером Windows Server 2008 ядро, сведя воедино две ветви разработки, существовавшие до того времени параллельно – отдельно для рабочих станций, отдельно для серверов.

## **Архитектура компьютеров на базе поколений процессоров Intel Core 2 и далее**

В этом разделе будут приведены описания архитектур компьютеров на основе поколений процессоров, начиная с Intel Core 2 Duo и заканчивая последними на сегодня (2020 год) процессорами семейства Skylake. Началом появления действительно новой архитектуры, довольно значительно отличающейся от архитектуры предыдущих поколений (как уже было написано выше) автор считает 2006 год, когда Intel выпустила «настоящие» многоядерные процессоры<sup>1</sup>, завершила переход платформы на последовательные периферийные шины, сделала стандартом кэш третьего уровня L3 на кристалле процессора (Nehalem), отказалась от продолжения развития архитектуры NetBurst. Этим моментом стал выход на рынок поколения Core 2.

### **PC на основе процессоров поколений Intel Core 2, Nehalem, Sandy Bridge...**

Практически с момента выхода оказавшегося очень удачным семейства Core 2, Intel стала быстро наращивать своё присутствие во всех сегментах рынка, активно тесня конкурентов – её процессоры стали доминировать в ноутбуках, включая ультрапортативные версии на сверхнизковольтных процессорах (ULV), в десктопах, от недорогих до продвинутых, уровня рабочих станций, и в мощных серверах на базе процессоров Xeon, суперкомпьютеры на основе которых стали регулярно появляться в первых строках самых производительных архитектур в списке Top500.

Далее будет приведено несколько примеров типичных материнских плат PC на основе процессоров поколений Intel Core и новее. Как уже неоднократно упоминалось выше, различия в архитектурах компьютеров после поколения Core 2 носили большей частью количественный характер, сохраняя основные черты исходной архитектуры. Даже перенос контроллера памяти из северного моста на процессорный кристалл не принципиально менял структуру компьютера в целом, хотя само по себе изменения в многопроцессорных системах вроде перехода с SMP на ccNUMA – это весьма значительное изменение.

Intel выпускала просто огромное количество вариаций и модификаций своих процессоров, но конечно же, без соответствующей поддержки производителей материнских плат всё это разнообразие CPU не имело бы большого смысла. Признаком успешности семейства Core 2 стало просто невообразимое количество моделей «материнок», выпускавшихся на их основе. Например, на странице ([Intel Core 2 Duo E6850 3.0GHz Compatible Motherboards](#)), где приведены списки совместимых с Intel Core 2 Duo E6850 3.0GHz Motherboards (всего лишь одна модель процессора), присутствует более 30(!) производителей, а у одного из них (самого крупного, ASUS) количество вариантов материнских плат с поддержкой этого процессора зашкаливает за 100(!).

На Рис.361, приведена фотография одной из популярных десктопных материнских плат из этого списка – ASUS P5K SE форм-фактора ATX (младшая модель из серии P5K). Её основные характеристики:

---

<sup>1</sup> Если быть точным, то совсем уж «настоящие» многоядерные процессоры (четыре ядра и больше) появились в следующем после Core 2 поколении Nehalem, которое Intel назвала «Поколение 1». Четырёхпроцессорные Kentsfield и Yorkfield (оба Core 2) состояли из двух кристаллов по 2 ядра в каждом, тогда как все 4 ядра Bloomfield (Nehalem) находились на одном кристалле.



- Процессор
  - Сокет: LGA775
  - Поддерживаемые процессоры: Intel Celeron 420,430,440/Core 2 Duo/Core 2 Extreme/Core 2 Quad/Pentium 4/Pentium D/Pentium Extreme
  - Системная шина: 800 МГц – 1333 МГц
  - Поддержка Hyper-Threading: есть
  - Поддержка многоядерных процессоров: есть
- Чипсет: северный мост Intel P35 Express, южный мост ICH9
- Память: DDR2 DIMM, 667 – 1066 МГц, количество слотов – 4, поддержка двухканального режима, максимальный объём – 8 Гбайт
- Дисковые контроллеры: IDE, 1 шт. UltraDMA 133; SATA 3Gb/s, 4 шт.
- Слоты расширения: 1xPCI-E x16, 3xPCI-E x1, 2xPCI
- Звук: 7.1CH, HDA, на основе Realtek ALC 883
- Сеть: Ethernet 1 Гбит/с
- Наличие интерфейсов: 12 USB, выход S/PDIF, 1xCOM, 1xeSATA, Ethernet, PS/2 (клавиатура)



**Рис.361 - Материнская плата ASUS P5K SE**

В P5K SE использовался сокет LGA775, о котором уже упоминалось ранее. Этот сокет был одним из самых «долгоиграющих» на рынке, был по сути переходным, поддерживая процессоры нескольких поколений, включая даже «старый» NetBurst Pentium 4 на ядре Prescott. От этого подхода – один сокет, поддерживающий несколько поколений процессоров – Intel уже вскоре отказалась. Например, уже в линейке Intel Core 2 Duo процессоры одного поколения Wolfdale/Yorkfield выпускались для трёх версий сокета: LGA775 (настольные), S479 (мобильные), LGA771 (Xeon), а в линейке процессоров Nehalem ядра Bloomfield и Lynnfield использовали разные сокеты: LGA1366 и LGA1156, соответственно.

Отметим, что на P5K SE присутствовала, казалось бы, уже давно устаревшая шина PCI, но количество разъемов PCI было сокращено до минимума – их было всего два. Поддержка одного IDE/PATA интерфейса и дополнительного порта e-SATA реализована не в чипсете, а средствами дополнительного внешнего контроллера. Впечатляет 12 разъемов USB 2.0 – а это ведь low-end решение.

Серверная часть спектра выпускаемых Intel процессоров новых поколений Core Xeon также весьма солидно была поддержана производителями материнских плат. Core Xeon-ы выпускались под свой socket LGA771, но, интересный момент – процессоры LGA771 могли без особых проблем работать на большинстве материнских платах с socket LGA775, будучи механически, электрически и логически совместимыми с ним. Понятно, что обратное – установка процессора LGA775 в socket LGA771 – было невозможно.

Ниже приведено фотография (Рис.362) и основные характеристики серверной<sup>1</sup> двух-socketной материнской платы на основе LGA771, от компании SuperMicro.



**Рис.362 – Серверная материнская плата SuperMicro X7DAE**

- Чипсет: Intel 5000X + 6702PXH-V + 6321ESB (ESB-2)
- Гнездо процессора: Socket LGA771 x2
- Макс. кол-во процессоров на материнской плате: 2
- Поддержка поколений ядер процессоров: Quad-Core Intel® Xeon® Processor 5400/5300 sequence (Harpertown/Clovertown processor) up to 3.16 GHz; Dual-Core Intel® Xeon® Processor 5200/5100/5000 sequence (Wolfdale/Woodcrest/Dempsey processor) up to 3.40 GHz;
- Частота шины: 1333, 1066, 667 МГц
- Тип поддерживаемой памяти: FB-DIMM (Fully Buffered DIMM) DDR667 (PC2-5300), DDR533 (PC2-4200) с поддержкой ECC
- Количество разъемов FBD DDRII: 8, четырёхканальный контроллер памяти, модули памяти устанавливаются только парами одинакового объёма.
- Максимальные поддерживаемые стандарты памяти: PC2-5300 (DDR2 667 МГц), PC2-4200 (DDR2 533 МГц)
- Максимальный объём оперативной памяти: 32 Гбайт, поддержка ECC
- Дисковая система: RAID-контроллер, встроен в чипсет, возможно построение RAID массивов уровней 0, 1, 10, 5 из Serial ATA устройств

<sup>1</sup> Формально производитель позиционирует платформу как рабочую станцию: «Industry First Xeon Dual-Core Greencreek Workstation MainBoard».

- SATA-II: 6 каналов
- Поддержка UDMA/100: 1 канал с возможностью подключения 2xIDE устройств или одной карты CF.
- Сеть 2x1 Гбит/с интегрированный в чипсет контроллер + интерфейс физического уровня Intel 82563EB
- 6xUSB 2.0
- Порты 1xLPT, 1xCOM порты
- PS/2 клавиатура + PS/2 мышь
- Количество разъемов PCI Express: 2 слота 16x работают в режиме 16-4
- Количество разъемов PCI-X/PCI-64: 3 (2 разъема PCI-X 64 бита/133 МГц, 1 разъем PCI-X 64 бита/100 МГц)
- Количество разъемов PCI: 1 слот
- Звук: 6-канальный AC'97 кодек

Жизненный цикл серверных платформ всегда более длительный, чем у десктопов и ноутбуков, поэтому на серверных материнских платах часто присутствует довольно консервативный набор интерфейсов. На примере X7DAE (2006 год): мы видим разъем старого «классического» PCI (1992 год), разъемы PS/2 (1987 год) и даже совсем уж «древние» LPT/COM-порты (1981 год).

Далее, на Рис.363, приведена фотография десктопной материнской платы ASUS P7P55 LX форм-фактора ATX, бывшей в своё время одной из лучших в своём классе. Плата была построена на платформе Nehalem (1-е поколение по нумерации Intel, следующее после Core).



Рис.363 – Материнская плата ASUS P7P55 LX

Основные характеристики P7P55 LX:

- Процессор:
  - Сокет: LGA1156
  - Поддерживаемые типы процессоров: Intel серии Core i7 8xx, Core i5 6xx/7xx, Core i3-5xx, Pentium G6xx0
  - Поддерживаемые ядра: Lynnfield, Clarkdale
  - Частота шины: 2500 МГц
  - Поддержка Hyper-Threading: есть
- Чипсет: Intel P55, поддержка CrossFire X (две видеокарты)

- Память:
  - Тип поддерживаемой памяти: DDR3. Максимальная поддерживаемая пропускная способность памяти указана в описании процессора
  - Количество разъемов DDR3: 4 (двухканальный контроллер памяти, встроенный в процессор). Поддерживается Extreme Memory Profile (XMP).
- Дисковая подсистема:
  - RAID-контроллер: встроен в чипсет, возможно построение RAID массивов уровней 0, 1, 10, 5 из SATA устройств
  - SATA-II: 6 каналов
  - Поддержка UDMA/133: 1 канал с возможностью подключения до 2х устройств (внешний контроллер VIA VT6415, т.е., не средствами чипсета)
- Сеть: Ethernet 1 Гбит/с. Сетевой контроллер Realtek RTL8112L
- Интерфейсы, разъёмы:
  - USB: 14 x USB 2.0
  - Клавиатура/мышь: PS/2 клавиатура + PS/2 мышь
- Слоты расширения: 2x16x PCI-E 2.0 (поддержка режима CrossFire X); 2x1x PCI-E 2.0; 3xPCI
- Звук: 7.1 HDA кодек, внешний контроллер VIA VT1708S (не средствами чипсета)

Если сравнить эту плату с платой на платформе предыдущего поколения (Core, ASUS P5K SE), то станет очевидным то, о чём уже несколько раз говорилось выше: принципиально архитектура компьютера на основе Nehalem практически не изменилась. Да, из схемы исчез северный мост, т.к. контроллер памяти фактически переместился на кристалл процессора (что позволило значительно сократить задержки при доступе к памяти), но общая структура компьютера в целом осталась той же. Ещё из архитектурных изменений – появился второй разъём 16x PCI-E, с возможностью работы двух видеоадаптеров в паре (режим CrossFire X), но, опять же, общая структура системы при этом сохранялась. Основные изменения в системах на основе Nehalem – количественные: PCI-E из версии 1.0 стал v2.0 (вдвое более высокая скорость обмена), количество разъемов 16x PCI-E удвоилось. Объёмы поддерживаемой памяти увеличились, конкретная цифра зависела от модели процессора (а не чипсета, как было раньше), но была не меньше 16 Гбайт (ASUS P5K SE – 8 Гбайт максимум). Версия памяти стала DDR3 вместо DDR2, больше стало SATA-разъемов, выше частота шины процессора. Ну, и, как положено, количество контактов в процессорном сокете тоже выросло – 1156 против 775 ☺.

Одним из важнейших достоинств архитектуры Nehalem стала возможность создания процессора под любой класс задач путём комбинирования стандартных узлов CPU, варьируя количество ядер, объёмы и канальность кэша, количество каналов работы с памятью, набор расширений в системе команд и т.п., не разрабатывая процессор под каждую задачу с нуля. Топовые серверные процессоры микроархитектуры Nehalem относились к семейству Intel Xeon Westmere EX (формально это поколение относили к Nehalem-C), старшая модель E7-8870 имела 10 ядер, 20 потоков, 30 МБ L3; четырёхканальный контроллер DDR3 800/1066/1333 ECC памяти, поддерживающий до 4096 Гбайт для E7-88xx серии с максимальной скоростью до 1066 МГц; 3200 МГц 6.4 GT/s QPI, TDP 130 Вт, работала на частоте 2.4 ГГц (Turbo Boost до 2.8 ГГц), поддерживала работу в восьмисокетной конфигурации. Ниже приведена фотография двухсокетной серверной материнской платы для процессоров классом ниже, Xeon 5500/5600.



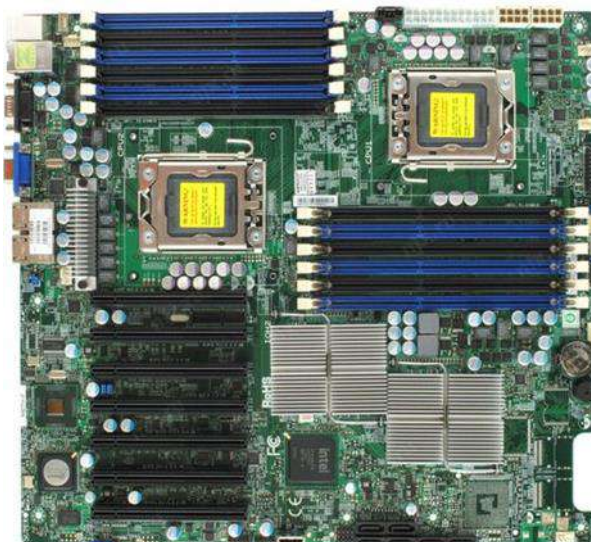


Рис.364 - Серверная материнская плата SuperMicro X8DTH-iF

Основные характеристики:

- Чипсет: Intel 5520 + 82801IJR (ICH10R)
- Гнездо процессора: Socket LGA1366 x2
- Поддержка процессоров
  - Макс. кол-во процессоров на материнской плате: 2
  - Поддержка типов процессоров: Intel серии Xeon X55xx, Xeon E55xx, Xeon L55xx, Xeon X56xx, Xeon E56xx, Xeon L56xx
  - Поддержка поколений ядер процессоров: Nehalem-EP, Westmere
  - Частота шины: 6400, 5860, 4800 МГц
  - Поддержка Hyper Threading: есть
- Поддержка памяти
  - Тип поддерживаемой памяти: Registered DDR3, DDR3 ECC. Максимальная поддерживаемая пропускная способность памяти указана в описании процессора (192 Гбайт)
  - Количество разъемов Registered DDR3: 12 (по 6 на каждый процессор)
  - Количество разъемов DDR3: 12 (по 6 на каждый процессор)
  - Максимальные поддерживаемые стандарты памяти: зависит от модели процессора
- Дисковая система
  - RAID-контроллер: встроен в чипсет, возможно построение RAID массивов уровней 0, 1, 10, 5 из Serial ATA устройств
  - SATA-II: 6 каналов
  - Разъем для подключения FDD: Нет
- Сеть
  - Intel® 82576 Dual-Port Gigabit Ethernet Controller
  - 1x Realtek RTL8201N PHY (dedicated IPMI)
- Слоты расширения: 7 слотов 16x PCI-E 2.0, работающих в режиме 8x
- Графика: интегрированный Matrox G200eW

На этой модели уже нет старых интерфейсов даже поколения «классической» PCI, разработчики платформы разместили на плате только разъёмы шины 16x PCI-E (7 слотов), работающих в режиме 8x. Кстати, на десктопах и серверах было принято использовать разные сочетания режимов работы PCI-E: обычно это было 1-2 слота 16x и 3-4 слота 1x на десктопах и несколько 4x-8x-16x слотов на серверных платформах. Правда, в последнее

время на десктопах и в ноутбуках набирает всё бóльшую популярность стандарт подключения компактных компьютерных плат расширения M.2 NVMe, в котором используется 4x линии PCI Express 3.0.

В качестве примера систем на интеловских процессорах с микроархитектурой Skylake приведена материнская плата *Asus ROG Rampage VI Extreme Encore* на чипсете Intel X299, поддерживающая процессоры Intel Core X 7-го, 9-го и 10-го поколений. Это изделие относится к категории Hi End Desktop (HEDT), основной признак принадлежности к которой – бескомпромиссная, экстремальная производительность десктопного компьютера, востребованная, например, в играх. Системы класса HEDT собираются из самых производительных компонентов, стоящих в разы дороже элементов массовых ПК – топовых процессоров, продвинутых материнских плат, быстрых твердотельных (SSD) дисков, быстрого ОЗУ большого объёма (десятки и даже сотни Гбайт) и т.п., когда цена не является определяющим фактором при выборе. Хотя приведённая в качестве примера плата является не рекордной по стоимости – её цена на начало 2020 года была «всего» около 40 тыс. рублей.

Фотография (Рис.365) и таблица (Рис.366) с характеристиками материнской платы были взяты отсюда: [Обзор материнской платы Asus ROG Rampage VI Extreme Encore на чипсете Intel X299](#) – очень подробный и интересный обзор.



Рис.365 - Материнская плата Asus ROG Rampage VI Extreme Encore

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Поддерживаемые процессоры  | Intel Core X 7-го, 9-го и 10-го поколений                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Процессорный разъем        | LGA 2066                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Чипсет                     | Intel X299                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Память                     | 8 × DDR4, до 256 Гб, до DDR4-4266 (XMP), четыре канала                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Аудиоподсистема            | 1 × Realtek ALC1220 (перемаркирован в S1220) (7.1) + ЦАП ESS ES9218                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Сетевые контроллеры        | 1 × Intel WGI219-AT Ethernet 1 Гбит/с<br>1 × Aquantia AQC107 (Ethernet 10 Гбит/с)<br>1 × Intel Dual Band Wireless AX200NGW/CNV (Wi-Fi 602 11a/b/g/n/ac/ax (2.4/5 ГГц) + Bluetooth 5.0)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Слоты расширения           | 3 × PCI Express 3.0 x16 (режимы x16, x16+x16 (SLI/CrossFire), x16+x16+x8 (CrossFire));<br>1 × PCI Express 3.0 x4                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Разъемы для накопителей    | 8 × SATA 6 Гбит/с (X299)<br>2 × M.2 (через DIMM 2) (CPU, PCI-E 3.0 x4 для устройств формата 2242/2260/2280/22110)<br>1 × M.2 (X299, PCI-E 3.0 x4/SATA для устройств формата 2242/2260/2280)<br>1 × M.2 (X299, PCI-E 3.0 x4 для устройств формата 2242/2260/2280)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| USB-порты                  | 2 × USB 2.0: 1 внутренний разъем на 2 порта (X299)<br>1 × USB 2.0: 1 порт Type-A (черный) на задней панели (X299)<br>4 × USB 3.2 Gen1: 4 порта Type-A (синие) на задней панели (X299)<br>4 × USB 3.2 Gen1: 4 порта Type-A (синие) на задней панели (ASMedia)<br>4 × USB 3.2 Gen1: 2 внутренних разъема на 4 порта (ASMedia)<br>1 × USB 3.2 Gen2x2: 1 порт Type-C на задней панели (ASMedia)<br>2 × USB 3.2 Gen2: 2 внутренних разъема Type-C (ASMedia)<br>2 × USB 3.2 Gen2: 1 порт Type-A (красный) и 1 порт Type-C на задней панели (ASMedia)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Разъемы на задней панели   | 2 × USB 3.2 Gen2 (Type-C)<br>1 × USB 3.2 Gen2x2 (Type-C)<br>1 × USB 3.2 Gen2 (Type-A)<br>8 × USB 3.2 Gen1 (Type-A)<br>2 × RJ-45<br>5 аудиоразъемов типа миниджек<br>1 × S/PDIF (оптический, выход)<br>2 антенных разъема<br>кнопка сброса CMOS<br>кнопка перепрошивки BIOS — Flashback                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Прочие внутренние элементы | 24-контактный разъем питания ATX<br>2 8-контактных разъема питания EPS12V<br>1 6-контактный разъем питания EPS12V<br>1 4-контактный разъем питания «Molex»<br>1 слот M.2 (E-key), занят адаптером беспроводных сетей<br>2 разъема для подключения портов USB 3.2 Gen2 Type-C<br>2 разъема для подключения 4 портов USB 3.2 Gen1<br>1 разъем для подключения 2 портов USB 2.0<br>8 разъемов для подключения 4-контактных вентиляторов (поддержка помп ЖСО)<br>2 разъема для подключения неадресуемой RGB-ленты<br>2 разъема для подключения адресуемой ARGB-ленты<br>1 разъем аудио для передней панели корпуса<br>1 разъем NODE<br>2 разъема для подключения управления с передней панели корпуса<br>1 разъем включения режима LN2<br>1 переключатель Slow Mode<br>1 переключатель Full Speed Mode<br>1 переключатель Pause<br>1 кнопка Safe_boot<br>1 кнопка RelJoy<br>1 кнопка переключения BIOS<br>1 кнопка включения питания Power<br>1 кнопка перезагрузки FlexKey (по умолчанию Reset)<br>точки для измерения напряжений |
| Форм-фактор                | E-ATX (305×275 мм)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Ожидаемая розничная цена   | 45 000 руб.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

Рис.366 – Перечень функциональных особенностей Asus ROG Rampage VI Extreme Encore

Отметим набор поддерживаемых шин и интерфейсов: почти все или самых последних версий и стандартов, или максимум на одно поколение ниже. Суммарно по количеству портов ввода-вывода tandem X299+Core X предоставляет:

- 28/44/48<sup>1</sup> PCI-E 3.0<sup>2</sup> линий для видеокарт или иной периферии с PCI-E интерфейсом (от процессора);
- суммарно 14 портов USB, внутри которых до 10 портов USB 3.2 Gen1 (Gen2 не поддерживается), до 14 портов USB 2.0 (от чипсета);
- 8 портов SATA 6Гбит/с (от чипсета);
- 24 порта PCI-E 3.0 (от чипсета).

Конечно же, нельзя не отметить поддержку четырёхканальной памяти объёмом до 256 Гбайт! Как говаривал один персонаж из известной детской сказки: «Это просто праздник какой-то!» ☺

<sup>1</sup> Процессоры Intel Core i9-7xxxX/9xxxX/10xxxX (совместимые с сокетом LGA2066 и поддерживаемые чипсетом X299) имеют 28 (Core 78xxX) или 44 (Core 79xxX/99xxX) или 48 (Core 10xxxX) линий ввода-вывода (включая PCI-E 3.0).

<sup>2</sup> Версию PCI-E 4.0 на сегодня (середина 2020 года) реализовала в «железе» только компания AMD. К середине 2021 года поддержка PCI-E 4.0 появилась и в новейших интеловских продуктах.



## Архитектура процессоров Intel Core 2...Skylake

В сентябре 2006 года на конференции Intel Developer Forum компания Intel анонсировала экстенсивную стратегию разработки своих микропроцессоров, названную «Тик-Так» (англ. tick-tock).

Цикл разработки делится на две стадии — «тик» и «так»:

- «Тик» означает уменьшение технологического процесса на основе существующей микроархитектуры.
- «Так» означает выпуск микропроцессоров с новой микроархитектурой на основе существующего технологического процесса<sup>1</sup>.

По планам Intel, каждая часть цикла должна была занимать примерно год.

В Интернете можно найти множество красивых картинок, иллюстрирующих интеловский «Тик-Так». Формально первым элементом в стратегии были 65 нм микроархитектуры Intel P6 и NetBurst (Cedar Mill, Presler, Yonah), а последним на сегодня – 10-е поколение Intel Core X. Но найти рисунок требуемого качества со всеми поколениями архитектур не удалось, поэтому ниже приведён самый симпатичный, на взгляд автора, но слегка урезанный вариант, начинающийся с шага «Так» Nehalem и заканчивающийся 9-ым поколением Coffe Lake Refresh (Рис.367), т.е., в соответствии с таблицей на Рис.368, в нём отсутствует три шага «Тик-Так-Тик» в начале и один шаг «Оптимизация» в конце.



Рис.367 – Стратегия разработки микропроцессоров компании Intel

Первые несколько лет всё работало как было заявлено производителем – в обещанном темпе менялась микроархитектура и техпроцессы, росла производительность новых процессоров. Но по мере снижения размеров элемента в новых техпроцессах Intel столкнулась с рядом серьёзных трудностей. 14-нанометровый техпроцесс удалось разработать и реализовать достаточно успешно: цикл «тик» пришёлся на архитектуру Broadwell, слегка усовершенствованную наследницу Haswell, но использующую более тонкие технологические нормы. Следующий цикл, «так», пришёлся на разработку и выпуск микроархитектуры Skylake, использующей тот же техпроцесс, что и Broadwell (Рис.368).

<sup>1</sup> В некоторых русскоязычных источниках приходилось встречать описание стратегии «Тик-Так» со стадиями, переставленными наоборот – будьте бдительны!

| Change (step)                                        | Fabrication process     | Micro-architecture                       | Code names for step                             | Intel Generation Desktop | Intel Generation Xeon | Intel Microcode shortcut(s) Desktop/WS <sup>[7][8]</sup>  | Intel Microcode shortcut(s) Xeon/Server | Release date                              | Processors                    |                               |                                |                             |                              |                                                           |                                                                |               |
|------------------------------------------------------|-------------------------|------------------------------------------|-------------------------------------------------|--------------------------|-----------------------|-----------------------------------------------------------|-----------------------------------------|-------------------------------------------|-------------------------------|-------------------------------|--------------------------------|-----------------------------|------------------------------|-----------------------------------------------------------|----------------------------------------------------------------|---------------|
|                                                      |                         |                                          |                                                 |                          |                       |                                                           |                                         |                                           | 8P/4P Server                  | 4P/2P Server/WS               | Embedded Xeon                  | 1P Xeon                     | Enthusiast/WS                | Desktop                                                   | Mobile                                                         |               |
| Tick (new fabrication process)                       | 65 nm                   | P6, NetBurst                             | Presler, Cedar Mill, Yonah                      |                          |                       |                                                           |                                         | 1995-11-1 (P6), 2000-11-20 (Netburst)     |                               |                               |                                |                             | Presler                      | Cedar Mill                                                | Yonah                                                          |               |
| Tock (new micro-architecture)                        |                         | Core                                     | Merom <sup>[8]</sup>                            |                          |                       |                                                           |                                         | 2006-07-27 <sup>[19][11]</sup>            | Tigerion                      | Woodcrest Clovertown          |                                |                             | Kentsfield                   | Conroe                                                    | Merom                                                          |               |
| Tick                                                 | 45 nm                   |                                          | Penryn                                          |                          |                       |                                                           |                                         | 2007-11-11 <sup>[12]</sup>                | Dunnington                    | Harpertown                    |                                |                             | Yorkfield                    | Wolfdale                                                  | Penryn                                                         |               |
| Tock                                                 |                         | Nehalem                                  | Nehalem                                         | 1                        | N/A                   | NHM <sup>[13]</sup>                                       |                                         | 2008-11-17 <sup>[14]</sup>                | Beckton                       | Gainestown                    |                                | Lynnfield                   | Bloomfield                   | Lynnfield                                                 | Clarkfield                                                     |               |
| Tick                                                 | 32 nm                   |                                          | Westmere                                        | 1                        | N/A                   | WSM <sup>[13]</sup>                                       |                                         | 2010-01-04 <sup>[15][18]</sup>            | Westmere-EX                   | Westmere-EP                   |                                |                             | Gulftown                     | Clarkdale                                                 | Arrandale                                                      |               |
| Tock                                                 |                         | Sandy Bridge                             | Sandy Bridge                                    | 2                        | 1                     | SNB                                                       | JKT (Jaketown)                          | 2011-01-06 <sup>[17]</sup>                | (Skipped) <sup>[18]</sup>     | Sandy Bridge-EP               | Gladden                        | Sandy Bridge                | Sandy Bridge-E               | Sandy Bridge                                              | Sandy Bridge-M                                                 |               |
| Tick                                                 |                         |                                          | Ivy Bridge                                      | 3                        | 2                     | IVB                                                       | IVT (Ivytown)                           | 2012-04-29                                | Ivy Bridge-EX <sup>[20]</sup> | Ivy Bridge-EP <sup>[20]</sup> | Gladden                        | Ivy Bridge                  | Ivy Bridge-E <sup>[21]</sup> | Ivy Bridge                                                | Ivy Bridge-M                                                   |               |
| Tock                                                 |                         | Haswell                                  | Haswell                                         | 4                        | 3                     | HSW, CRW (Crystal Well) with Iris Pro <sup>[22][23]</sup> | HSX                                     | 2013-06-02                                | Haswell-EX                    | Haswell-EP                    | N/A                            | Haswell-DT                  | Haswell-E                    | Haswell-DT <sup>[24]</sup>                                | Haswell-MB (notebooks) Haswell-LP (ultrabooks) <sup>[24]</sup> |               |
| Refresh                                              |                         |                                          | Haswell Refresh, Devil's Canyon <sup>[25]</sup> | 4                        | N/A                   | HSW, CRW (Crystal Well) with Iris Pro <sup>[22][23]</sup> |                                         | 2014-05-11, 2014-06-02                    | No server version released    |                               |                                |                             |                              | Devil's Canyon                                            | No mobile version released                                     |               |
| Tick                                                 | 14 nm <sup>[19]</sup>   |                                          | Broadwell <sup>[26]</sup>                       | 5                        | 4                     | BDW                                                       | BDX                                     | 2014-09-05                                | Broadwell-EX <sup>[27]</sup>  | Broadwell-Ep <sup>[27]</sup>  | Broadwell-DE                   | Broadwell-DT                | Broadwell-E                  | Broadwell-DT                                              | Broadwell-H Broadwell-U Broadwell-Y                            |               |
| Tock                                                 | Skylake <sup>[26]</sup> | Skylake <sup>[28]</sup>                  | 6                                               | 5                        | SKL                   | SKX                                                       | 2015-08-05 <sup>[28]</sup>              | Skylake-SP                                |                               | Skylake-DE                    | Skylake-DT/H                   | Skylake-X                   | Skylake                      | Skylake-H Skylake-U Skylake-Y                             |                                                                |               |
|                                                      |                         | Kaby Lake <sup>[32]</sup>                | 7                                               | 6                        | KBL                   |                                                           | 2017-01-03 <sup>[33]</sup>              | Only 1P server (Xeon E3) version released |                               |                               | Kaby Lake-DT/H cores: 4 (4/8)  | Kaby Lake-X <sup>[34]</sup> | Kaby Lake                    | Kaby Lake-H Kaby Lake-U Kaby Lake-Y                       |                                                                |               |
|                                                      |                         | Kaby Lake R <sup>[35][36]</sup>          | 8                                               | N/A                      | KBL-R                 |                                                           | 2017-08-21 <sup>[36]</sup>              | Only mobile version released              |                               |                               |                                |                             |                              |                                                           | Kaby Lake R                                                    |               |
| Optimizations (refreshes) <sup>[4][29][30][31]</sup> |                         | Coffee Lake                              | 8, g <sup>[37]</sup>                            | E-Zxxx                   | CFL                   |                                                           | 2017-10-05 <sup>[38]</sup>              | Cascade Lake-SP Cooper Lake (Q3'19)       |                               | Cascade Lake-AP               | Coffee Lake-DT/H cores: 6 (12) | Skylake-X Refresh           | Coffee Lake Coffee Lake-R    | Coffee Lake-H Coffee Lake-U Coffee Lake-H Refresh         |                                                                |               |
|                                                      |                         | Whiskey Lake, Amber Lake <sup>[39]</sup> | 8                                               | N/A                      | WHL AMIL              |                                                           | 2018-08-28 <sup>[39]</sup>              | Only mobile version released              |                               |                               |                                |                             |                              |                                                           | Whiskey Lake-U Amber Lake-Y                                    |               |
|                                                      |                         | Comet Lake <sup>[40]</sup>               | 10                                              | N/A                      | CML                   |                                                           | 2019-08-21 <sup>[40]</sup>              | No server version released                |                               |                               |                                |                             | Comet Lake-S                 | Comet Lake-U <sup>[41]</sup> Comet Lake-Y <sup>[41]</sup> |                                                                |               |
| Process                                              | 10 nm <sup>[42]</sup>   | Sunny Cove                               | Cannon Lake                                     | 8                        | N/A                   | CNL                                                       |                                         | 2018-05-16 <sup>[43]</sup>                | Only mobile version released  |                               |                                |                             |                              |                                                           |                                                                | Cannon Lake-U |
| Architecture                                         |                         |                                          | Ice Lake <sup>[44]</sup>                        | 10                       |                       | ICL                                                       |                                         | 2019-08-01 <sup>[45]</sup>                | Ice Lake-SP (1H'20)           |                               |                                |                             |                              |                                                           | Ice Lake-U Ice Lake-Y                                          |               |
| Optimization <sup>[31]</sup>                         |                         |                                          | Tiger Lake <sup>[31]</sup>                      |                          |                       |                                                           |                                         |                                           |                               |                               |                                |                             |                              |                                                           |                                                                |               |
| Process                                              | 7 nm <sup>[42]</sup>    |                                          | Sapphire Rapids <sup>[46]</sup>                 |                          |                       |                                                           |                                         |                                           |                               |                               |                                |                             |                              |                                                           |                                                                |               |
| Architecture                                         |                         |                                          | Granite Rapids                                  |                          |                       |                                                           |                                         |                                           |                               |                               |                                |                             |                              |                                                           |                                                                |               |
| Optimization                                         |                         |                                          |                                                 |                          |                       |                                                           |                                         |                                           |                               |                               |                                |                             |                              |                                                           |                                                                |               |
| Process                                              | 5 nm <sup>[42]</sup>    |                                          |                                                 |                          |                       |                                                           |                                         |                                           |                               |                               |                                |                             |                              |                                                           |                                                                |               |
| Architecture                                         |                         |                                          |                                                 |                          |                       |                                                           |                                         |                                           |                               |                               |                                |                             |                              |                                                           |                                                                |               |
| Optimization                                         |                         |                                          |                                                 |                          |                       |                                                           |                                         |                                           |                               |                               |                                |                             |                              |                                                           |                                                                |               |

Рис.368 – Intel «Тик-Так»

Но в следующем цикле Intel, что называется, «заклинило». Изначально новый «тик» должен был означать внедрение 10-нанометрового техпроцесса и выпуск процессоров Skymont, позднее переименованных в Cannonlake. Появиться они должны были во втором квартале 2016 года. Но столь тонкий техпроцесс вызвал массу проблем и, как следствие, серьёзные задержки и проволочки в реализации «Тик-Так» стратегии. В итоге компания решила сделать «тик» на 14-нанометровом процессе, выпустив серию Kaby Lake.

С пропуском шага, с «внеочередным» появлением процессоров Intel Core седьмого поколения (Kaby Lake) многим стало понятно, что до этого дня вполне успешно работавшая стратегия «Тик-Так», которой Интел следовал всё это время, дала сбой. Обещание уменьшить технологический процесс с 14 до 10 нм так и осталось обещанием, началась долгая эпоха «Так-а» Skylake, во время которой случился Kaby Lake (седьмое поколение), внезапный Coffee Lake (восьмое) с незначительным изменением техпроцесса с 14 нм до 14 нм+ и

даже Coffee Lake Refresh<sup>1</sup> (девятое). В итоге мы имеем несколько процессоров разных поколений, которые созданы на основе одной микроархитектуры Skylake, с одной стороны. И уверения Интела о том, что каждый новый процессор — лучше прежнего, с другой.

Краткое сравнение трёх поколений упомянутых выше — чем же они действительно отличались друг от друга<sup>2</sup>.

## **Kaby Lake**

Появление процессоров в рознице состоялось в начале 2017 года. Что же нового появилось в этом семействе относительно его предшественника? Прежде всего, это новое графическое ядро — Intel UHD 630. Плюс поддержка технологии памяти Intel Optane (3D Xpoint), а также новый чипсет 200-ой серии (6-ое поколение, Skylake, работало с 100-ой серией). И на этом из действительно интересных новшеств всё.

## **Coffe Lake**

8-ое поколение с кодовым названием Coffee Lake было выпущено в конце 2017 года. В процессорах этого поколения добавили ядер и пропорционально кэша третьего уровня, подняли Turbo Boost на 200 МГц, добавили поддержку DDR4-2666 (до этого было максимум DDR4-2400), но вырезали поддержку DDR3. Графическое ядро осталось прежним, но ему добавили 50 MHz частоты. За все повышения частот пришлось расплатиться увеличением теплопакета до 95 ватт. Ну и, конечно, новый чипсет 300-ой серии. Последнее было совсем не обязательно, так как достаточно скоро специалисты смогли запустить это семейство на чипсетах 100-ой серии, хотя представители Интел заявляли, что это невозможно из-за особенностей построения цепей питания. Позднее, правда, компания Интел официально признала, что была не права. Так что же нового в 8-ом семействе? По факту больше похоже на обычный рефреш с добавлением ядер и частот.

## **Coffee Lake Refresh**

Очередной рефреш! В четвёртом квартале 2018 года были выпущены процессоры Coffee Lake 9-го поколения, оснащённые аппаратными средствами защиты от некоторых уязвимостей класса Meltdown/Spectre. Аппаратные изменения, внесённые в новые чипы, защищают от Meltdown V3 и L1 Terminal Fault (L1TF Foreshadow). Изменения в программном обеспечении и микрокоде защищают от атак Spectre V2, Meltdown V3a и V4. Защита от Spectre V1 по-прежнему будет осуществляться исправлениями на уровне операционной системы. Появление исправлений на уровне кристалла должно уменьшить влияние программных патчей на производительность процессоров. Но всю эту радость с защитами Интел реализовала только в процессорах для массового сегмента рынка: i5-9600k, i7-9700k, i9-9900k. Всем остальным, включая серверные решения, аппаратных защит не досталось. Впервые в истории потребительских процессоров Intel процессоры Coffee Lake Refresh поддерживают до 128 Гбайт оперативной памяти (в 10-ом поколении эта цифра удвоилась, до 256 Гбайт). И всё, больше по сути никаких изменений. В итоге имеем два года рефрешей, игры с ядрами и частотами, плюс набор мелких улучшений.

В результате, в 2016 Intel официально объявила об отказе от стратегии «тик-так» и переходе вместо неё на стратегию из трёх этапов: «техпроцесс-архитектура-оптимизация». На вторую половину 2020 года мы видим, что Intel подзадержалась на этой стадии, первоначальные обещания сдвинуться с «пробуксовки» к 2018 году оказались не выполненными.

Но компания Intel медленно, но всё же переводит свою линейку процессоров на 10-нанометровый техпроцесс. Мобильные CPU Ice Lake уже можно купить<sup>3</sup>, недавно Intel

---

<sup>1</sup> До этого подобная «пробуксовка» уже имела место в четвёртом поколении — вместо «Тик» Broadwell был реализован шаг «Оптимизация» Haswell Refresh.

<sup>2</sup> Основной текст по сравнению взят по ссылке [Так-Так-Так и никакого Тика. Чем отличаются процессоры Intel Core разных поколений на основе одной архитектуры](#)

<sup>3</sup> Осень 2019 года.

представила архитектуру Tremont, на которой будут построены 10-нанометровые «атомные» процессоры.

С большим опозданием (август 2020 года) Intel анонсировала 10-нанометровые серверные CPU Intel Xeon Scalable (*Ice Lake-SP*). А вот с настольными процессорами никакой ясности нет, хотя Intel заверила, что они всё же будут выпущены.

Также представитель Intel заявил, что компания намерена вернуться к своему старому графику крупных обновлений раз в 2-2.5 года, к прежней стратегии «тик-так». Для этого Intel уже сейчас работает над пятинанометровым техпроцессом, но конкретики относительно данного направления нет. Сейчас компания сконцентрировала все свои силы на увеличении производства 10-нанометровой продукции.

## Немного истории

Если поизучать хронологию интеловских процессоров (например, здесь: [Список микропроцессоров Intel](#)), то можно обратить внимание на несколько моментов: во-первых, как уже отмечалось выше, начиная с процессоров под LGA775, Intel существенно изменила принципы маркировки своих МП – после сумбура и хаоса в именовании процессоров поколений Netburst и поздних P6 (как вам, к примеру, процессор Intel Core, который был создан НЕ на основе микроархитектуры... Intel Core ☺), приведя обозначения к более-менее стройному виду, начиная с Nehalem (1-ое поколение). В 3-ем поколении (Ivy Bridge) процессоры под маркой Pentium (торговая марка присутствовала на рынке с марта 1993) были «разжалованы» и переведены в нижний сегмент (low-end), где они, по классификации Intel, расположились чуть выше Atom и Celeron. Главным брендом Intel в процессорах стало семейство Core, к которому в шестом поколении добавились топовые Core i9 к уже существовавшим до этого Core M/i3/i5/i7.

Во-вторых, после того, как контроллер памяти и кэш L3 в поколении Nehalem были размещены на кристалле процессора, это потребовало смены сокета – LGA775 уже не хватало, чтобы обслуживать заметно увеличившееся число контактов процессора. После «гонки мегагерц», закончившейся на отменённом Tejas (архитектура Netburst), началась гонка сокетов – почти без исключения, каждый следующий процессор требовал другой тип гнезда. Например, казалось бы, на первый взгляд, ближайшие родственники LGA1150 и LGA1151 (или LGA1155 и LGA1156) – не взаимозаменяемы. Времена, когда процессоры LGA771 прекрасно работали при установке в LGA775, ушли безвозвратно.

В-третьих, упёршись в объективные физические пределы при наращивании тактовых частот, Intel (как и все остальные производители процессоров) начали гонку за количеством ядер в процессоре. Интересный факт – начиная с 2010 года, каждый следующий серверный Хеоп имел более низкую производительность в пересчёте на одно ядро, по сравнению с предшественниками (с одним небольшим исключением в 2014 году). Если экстремальные Netburst процессоры добивались до частот ~5 ГГц, то гораздо более поздние архитектуры работали уже где-то ближе к 3 ГГц (и даже ниже). Но надо отдать должное упорству компании – сегодняшние (2020-21 года) процессоры имеют рабочие тактовые частоты в диапазоне 4...5 ГГц. А рекорд по количеству ядер в серверных универсальных процессорах x86 на сегодня уверенно захватила компания AMD, выпустив процессоры Ерус<sup>1</sup>.

## Core

Микроархитектура Intel Core является многоядерной микропроцессорной архитектурой, представленной фирмой Intel в первом квартале 2006 года. Микроархитектура Intel Core основана на обновлённой версии ядра Yonah и может рассматриваться в качестве последней итерации микроархитектуры Intel P6, которая ведёт свою историю с Pentium Pro, представленного в 1995 году. Чрезмерно высокое энергопотребление и завышенные требо-

---

<sup>1</sup> Топовый Ерус второго поколения имеет 64 ядра/128 потоков выполнения.



вания к охлаждению процессоров, основанных на микроархитектуре NetBurst, и, в результате, неспособность эффективно увеличивать тактовую частоту, а также другие узкие места, такие, как неэффективность конвейера, являются главными причинами, почему Intel отказалась от микроархитектуры NetBurst. Микроархитектура Intel Core была разработана командой Intel Israel (IDC), которая ранее разработала мобильный процессор Pentium M.

Микроархитектура Intel Core обеспечивает высокую производительность, энерго-сбережение и быстродействие в многозадачных средах. Она имеет несколько ядер и аппаратную поддержку виртуализации (Intel VT), а также Intel 64 и SSE3.

Первые процессоры, использовавшие эту архитектуру, входили в семейство Core 2 (Рис.369) и вышли под кодовыми названиями Merom, Conroe и Woodcrest. Merom предназначался для мобильных компьютеров, Conroe — для настольных систем, а Woodcrest — для серверов и рабочих станций. Хотя их архитектуры идентичны, эти три линии процессоров отличаются используемым разъёмом, типом шины и потреблением энергии. Часть процессоров основанных на микроархитектуре Core имеет марку Pentium Dual-Core, а процессоры низшего ценового сегмента — марку Celeron. Процессоры для серверов и рабочих станций продаются под маркой Xeon, а для пользователей настольных и мобильных ПК — как Core 2. Несмотря на своё название, процессоры, продаваемые как Intel Core, фактически не используют микроархитектуру Core.

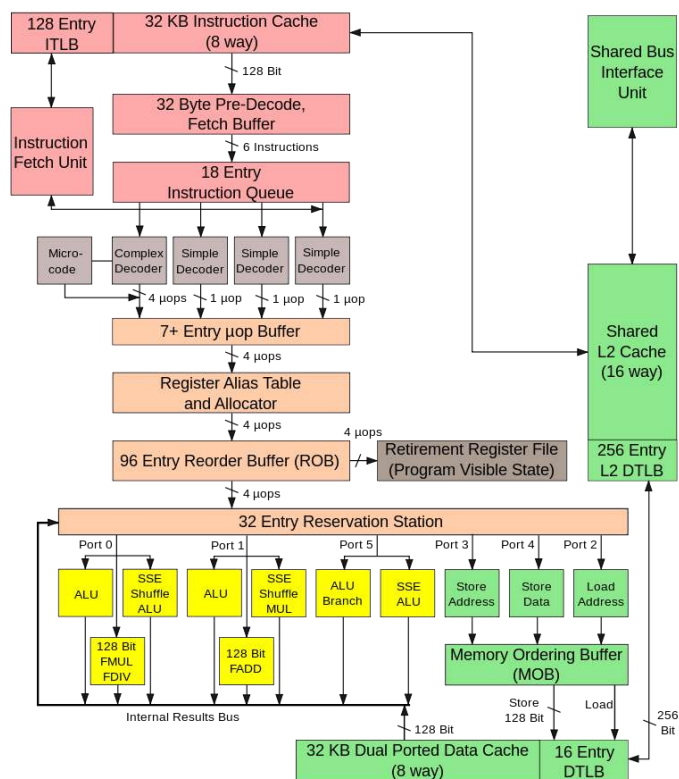


Рис.369 - Микроархитектура Intel Core 2

## Технологии

### Wide Dynamic Execution

Микроархитектура Intel Core была проектирована с нуля, но по философии микроархитектуры Pentium M. Длина исполнительного конвейера составляет 14 ступеней — менее половины от длины конвейера в предыдущем поколении Prescott (31 ступень), является ключевой особенностью технологии *динамического исполнения команд*.

Каждое ядро микропроцессора может получать, обрабатывать, исполнять и отбрасывать до четырёх полных команд одновременно. Это значительно повышает производительность по сравнению с конкурирующими процессорными технологиями P6, P-M (Banias,

Dothan, and Yonah) и NetBurst, поддерживающими одновременную обработку только трёх команд максимум.

### Advanced Smart Cache

Новая архитектура оптимизирована под двухъядерную архитектуру процессора. Основной кэш первого уровня L1 связан с общей для обоих ядер динамически распределяемой кэш-памятью второго уровня L2 (данные, содержащиеся в L1, обязательно содержатся и в L2, *инклюзивный кэш*) для достижения максимальной производительности на ватт потребляемой мощности и улучшения масштабируемости.

### Macro Fusion

Ещё одной новой технологией, включённой в микроархитектуру Intel Core при проектировании, является *Технология макро-слияния* (Macro Fusion), позволяющая объединять некоторые распространенные инструкции x86 в одну команду для исполнения. В предыдущих версиях процессорной микроархитектуры каждая инструкция декодировалась независимо от остальных. При использовании Macro Fusion некоторые пары инструкций (например, инструкция сравнения и условного перехода) при декодировании могут объединяться в одну микроинструкцию (*micro-op*), которая в дальнейшем будет выполняться именно как одна микроинструкция. Для эффективного поддержания этой технологии в архитектуре Intel Core используются расширенные блоки ALU, которые способны поддержать выполнение таких слитых микроинструкций.

### Nehalem

**Nehalem** — микроархитектура процессоров компании Intel, представленная в 4 квартале 2008 года, для ядра Bloomfield (сокет) LGA1366 и для ядра Lynnfield (сокет LGA1156), соответственно. Микропроцессоры выпускаются под торговой маркой Core i7 и Core i5.

### Архитектура

Архитектура Nehalem (Рис.370) построена на базе Core, но содержит некоторые кардинальные изменения:

- Встроенный на кристалл процессора контроллер памяти<sup>1</sup>, поддерживающий 2 или 3 канала DDR3 SDRAM или 4 канала FB-DIMM.
- Новая шина QPI, пришедшая на смену шине FSB (только в процессорах для LGA1366; процессоры для LGA1156 используют шину DMI). Более подробно о шинах QPI и DMI будет рассказано в разделе «Построение вычислительной системы».
- Возможность выпуска процессоров со встроенным графическим процессором (в бюджетных решениях на базе двухъядерных процессоров).
- В отличие от Kentsfield и Yorkfield, которые состояли из двух кристаллов по 2 ядра в каждом, все 4 ядра Bloomfield находятся на одном кристалле.
- На кристалл процессора был добавлен кэш 3-го уровня.
- Добавлена поддержка Hyper-threading (организация двух логических ядер из одного физического).

---

<sup>1</sup> Стоит отметить, что и здесь Intel была не первой – ещё в 2001 году компания AMD анонсировала свою архитектуру Hammer со встроенным в процессорный кристалл контроллером памяти. AMD немного не повезло, что спустя всего пару лет (анонс – второй квартал 2003 года) индустрия стала массово переходить на стандарт памяти DDR2, работу с которой встроенный контроллер памяти не поддерживал. В результате компании пришлось выпускать внеочередную итерацию процессора с контроллером, поддерживающим DDR2.

Процессоры Nehalem содержат порядка 731 млн транзисторов, что на 10% меньше, чем у процессоров Yorkfield. Но площадь кристалла значительно увеличилась по сравнению с предшественником — с 214 до 263 мм<sup>2</sup>.

Первые процессоры Nehalem были основаны на том же 45-нм техпроцессе, что и Penryn.

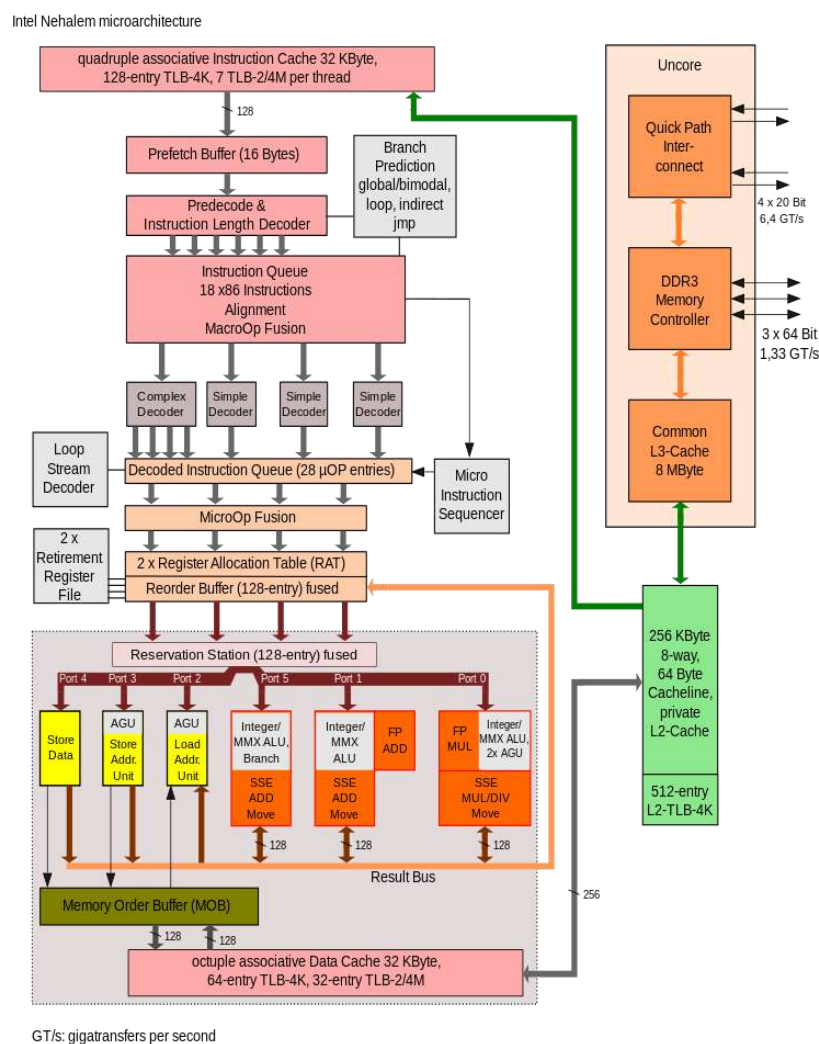


Рис.370 - Микроархитектура Intel Nehalem в 4-ядерной реализации.

## Будущее архитектуры

В соответствии с заявлениями компании Intel, процессоры Nehalem должны были производиться до конца 2009 года. В начале 2010 компания планировала показать первые Westmere-процессоры, основанные на 32-нм технологическом процессе («Тик»).

## Westmere

Решения на основе микроархитектуры **Westmere** производились с соблюдением норм 32-нм техпроцесса. Это должно было снизить как стоимость изготовления процессоров, так и потребляемую мощность. Была осуществлена доработка решений, впервые применённых в микроархитектуре Nehalem. Благодаря более тонкому техпроцессу площадь кристаллов была меньше, что позволило увеличить количество ядер.

Первый представитель новой микроархитектуры — Clarkdale, который обладает двумя ядрами и интегрированным графическим ядром, производимым по 45-нм техпро-



цессу, что избавляло от интегрированной графики в системной логике. Процессоры на основе дизайна Clarkdale используют сокет LGA1156, но для поддержки интегрированной графики потребовалась разработка специальных наборов системной логики, в которую входили чипсеты Intel H55, Intel H57 и Intel Q57. Это решение заменило процессоры на основе Wolfdale (Core 2 Duo). Продукты на основе дизайна ядер Clarkdale поступили в продажу 7 января 2010 года.

Потом в серийное производство пошёл флагманский дизайн процессора данной архитектуры — Gulftown, он имел шесть ядер/двенадцать потоков исполнения (НТ), 12 МБайт общего кэша третьего уровня, использовал системную шину QuickPath Interconnect, но несмотря на это, его энергопотребление не превышало 130 Вт. Он работал на сокете LGA1366 и наборе системной логики Intel X58 Express. Фактически этот дизайн представлял собой полтора чипа с дизайном Bloomfield (Core i7) на одной подложке, произведенной с соблюдением норм 32-нм техпроцесса. Этот вариант дизайна был первым, который перешагнул за психологическую отметку — один миллиард транзисторов. Всего на чипе располагалось около 1.17 млрд транзисторов, однако за счет 32-нм техпроцесса его площадь осталась в разумных пределах — 248 мм<sup>2</sup>. На основе этого дизайна вышел процессор Core i7 980X Extreme Edition с частотой 3333 МГц и Core i7 970 с частотой 3200 МГц (3 квартал 2010 года), а также Core i7 990X Extreme Edition с частотой 3466 МГц (1 квартал 2011 года). Процессоры на основе дизайна ядер Gulftown поступили в открытую продажу 16 марта 2010 года.

## **Sandy Bridge**

**Sandy Bridge** — микроархитектура центральных процессоров, основанная на 32-нм технологическом процессе, содержащая встроенный видеоускоритель. Была анонсирована 3 января 2011 года. Процессоры этой архитектуры продавались на рынке под торговыми марками Core i7/Core i5/Core i3/Pentium/Celeron.

Первый дизайн ядер на основе этой архитектуры представлял сочетание CPU с частотой до 3.5 ГГц, обладающего 2...4 ядрами и высокопроизводительного GPU с частотой до 1.35 ГГц (Intel HD Graphics 2000, для K серии — HD Graphics 3000) (Рис.371). Также в чип был интегрирован северный мост набора системной логики (контроллер PCI Express 2.0 и двухканальный контроллер памяти стандарта DDR3 SDRAM с частотой до 1333 МГц). Каждое ядро имеет по 256 КБайт кэша второго уровня и до 8 МБайт объединённого кэша третьего уровня. Процессор, графика, кэш-память и контроллеры выполнены на единой кремниевой подложке площадью 216 мм<sup>2</sup>. Энергопотребление данного дизайна не выходило за пределы 130 Вт для топовых моделей.

## **Нововведения**

В микроархитектуру Sandy Bridge была добавлена поддержка новых SIMD инструкций для работы с векторными вычислениями Advanced Vector Extensions, AVX, которые дополнили расширения SSE (новый набор, оставаясь обратно совместимым с SSE, увеличивает разрядность регистров в два раза — до 256 бит, а также даёт в распоряжение программистов дополнительные трёх- и четырёхоперандные команды). Intel обещала, что использование AVX будет способно поднять скорость работы некоторых алгоритмов до 90%.

Это была первая архитектура, в которую Intel встроила технологию Quick Sync, предназначенную для ускорения кодирования и декодирования видеоконтента. Реализована в виде специализированных аппаратных модулей в составе графического ядра.

Поддерживаются технологии Advanced Encryption Standard (AES) и Virtualization Machine Extensions (VMX).

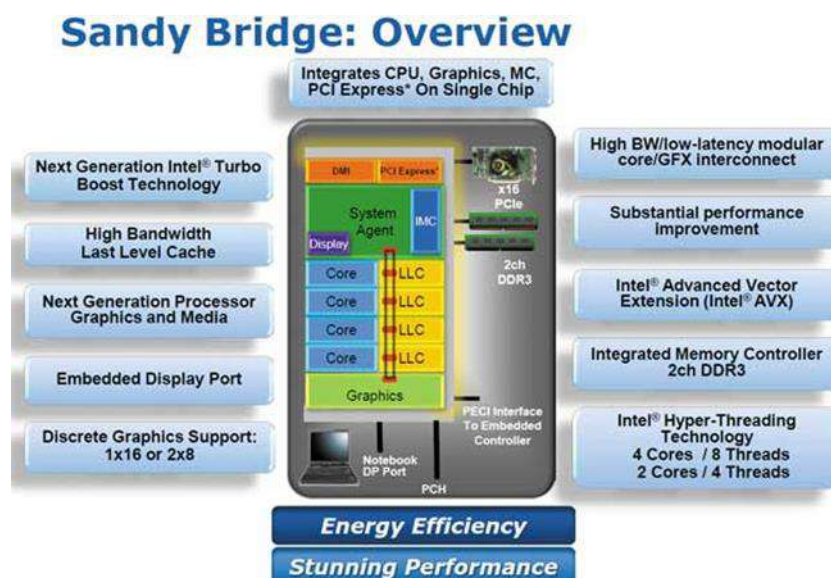


Рис.371 - Модульная структура процессора Sandy Bridge

Для процессоров с микроархитектурой Sandy Bridge Intel в очередной раз сменила тип сокета – теперь это были LGA1155 в десктопных моделях, LGA2011 и LGA1356 – в серверных.

### Процессорные ядра

- Добавлен кэш инструкций нулевого уровня (L0), содержащий до 1536 декодированных микроопераций для экономии энергии и улучшения пропускной способности инструкций. Блок предварительной выборки может отключать декодер инструкций, если он обнаруживает уже декодированную инструкцию в кэше. Также, как и в процессорах с микроархитектурой Nehalem, ядра имеют по 32 Кбайт кэша данных первого уровня и такое же количество кэша инструкций.
- Оптимизирована и улучшена точность предсказателя переходов.
- Поддержка улучшенных векторных расширений (AVX) — 256-битного набора инструкций к SSE.

Поскольку набор инструкций AVX предполагает применение 256-бит регистров YMM вместо 128-бит XMM, как это было в предыдущем поколении процессоров, модернизации подвергся и исполнительный кластер. Инженеры объединили имеющиеся тракты данных внутренней шины исполнительных блоков, и теперь ядро способно за один такт производить до восьми 256-битных операций и до шестнадцати 128-битных.

### Структура

Структуру чипа Sandy Bridge можно условно разделить на следующие основные элементы:

- процессорные ядра (Рис.372);
- графическое ядро;
- кэш-память L3;
- «Системный агент» (System Agent).

Все перечисленные выше элементы объединены с помощью 256-битной межкомпонентной кольцевой шины, выполненной на основе новой версии технологии QPI.

The diagram illustrates the Intel Xeon Phi 7200 architecture, showing the flow of instructions and data through various stages and units. Key components include:

- Instruction Path:** 32k L1 Instruction Cache → Pre decode → Instruction Queue → Decoders.
- Branch Prediction:** Branch Pred unit.
- Cache:** 1.5k uOP cache.
- Execution Stages:** Allocate/Rename/Retire (highlighted with Zeroing Idioms) → Scheduler.
- Scheduler Ports:** Port 0, Port 1, Port 5, Port 2, Port 3, Port 4.
- Execution Units:**
  - Port 0: ALU, VI MUL, VI Shuffle, DIV, AVX FP MUL, AVX FP Blend.
  - Port 1: ALU, VI ADD, VI Shuffle, AVX FP ADD.
  - Port 5: ALU, JMP, AVX/FP Shuf, AVX/FP Bod, AVX FP Blend.
  - Port 2: Load, Store Address.
  - Port 3: Load, Store Address.
  - Port 4: Store Data.
- Memory Hierarchy:** L2 Data Cache (MLC) ↔ 32k L1 Data Cache (via Fill Buffers) ↔ Memory Control (48 bytes/cycle).
- Out-of-order Execution:** Indicated by the Scheduler and the flow from the Memory Control unit.

Шина состоит из четырёх колец:

- ### Основные преимущества кольцевой топологии шины:

- Производительность кольцевой шины достигает 96 Гбайт/с секунду на соединение при тактовой частоте 3 ГГц, что фактически в четыре раза превышает показатели процессоров Intel предыдущего поколения. При этом система автоматически определяет кратчайший путь передачи данных, обеспечивая минимальную латентность.

Использование кольцевой шины позволило заметно снизить латентность кэша L3. Так, в процессорах предыдущего поколения (Westmere) латентность (задержки) доступа к кэшу L3 составляет 36 циклов, а в процессорах Sandy Bridge — 26-31 цикл. Кроме того, теперь L3-кэш работает на частоте ядра (в процессорах Westmere частота работы кэша L3 не соответствовала частоте ядра).

Кэш L3 в микроархитектуре Sandy Bridge остался полностью инклюзивным (включающим) по отношению к кэшам L2 (как и в микроархитектуре Nehalem).

Для того чтобы эффективно управлять работой всех доменов многофункционального процессора, разработчики добавили на кристалл новый модуль, именуемый системным агентом (uncore). Он содержал в себе все контроллеры ввода-вывода данных, а именно контроллер памяти DDR3, шины PCI Express, интерфейсов DMI и FDI для связи с микросхемой PCN, т.е., по сути системный агент выполнял роль северного моста в прежних архитектурах. Помимо того, в системном агенте присутствовал аппаратный контроллер питания, обеспечивающий работу технологии TurboBoost 2.0. Отметим, что Intel в новом поколении процессоров переименовала кэш-память второго и третьего уровней в MLC (Middle Level Cache) и LLC (Last Level Cache) соответственно.

По ссылке [https://ru.wikipedia.org/wiki/Sandy\\_Bridge](https://ru.wikipedia.org/wiki/Sandy_Bridge) можно найти более подробную информацию по выпускавшимся CPU с микроархитектурой Sandy Bridge. Стоит отметить просто невероятное количество версий и модификаций процессоров, различающихся сочетанием рабочих частот, количеством ядер/потоков, объёмом кэша, количеством каналов памяти, потребляемой мощностью, возможностями разгона и т.д. Все эти вариации были основаны на одной и той же микроархитектуре. Просто для примера: топовый Core i7 Extreme мог работать (режим Turbo) на частотах до 4 ГГц, имел кэш L3 объёмом 15 Мбайт, поддерживал PCIe v3.0 (основные процессоры поддерживали PCIe v2.0).

Поколения Sandy Bridge и следующего за ним Ivy Bridge – эпоха зрелости и расцвета процессорных архитектур от Intel, период их практически монопольного доминирования на «поле» x86. Доля присутствия AMD на рынке, выпустившей в это время несколько неудачных поколений своих процессоров (например, Bulldozer), сократилась до нескольких процентов, а в некоторых сегментах (высокопроизводительные десктоп процессоры) упала почти до нуля. Ничто не предвещало ситуации, когда через несколько лет AMD сумела воспрянуть и вернуться на, казалось бы, навсегда потерянный для неё рынок. Вернуться, не просто догнав Intel, но и сильно опередив её по ключевым параметрам и характеристикам своих новых процессоров.

## Ivy Bridge

**Ivy Bridge** – кодовое название 22-нм версии микроархитектуры Sandy Bridge третьего поколения процессоров Intel Core; этап «тик» (миниатюризации технологического процесса), чётко, пока без запинки «тикающей» стратегии интеловской стратегии «тик-так», переход с 32-нанометрового Sandy Bridge на 22-нанометровый процесс.

Релиз первых процессоров на данной архитектуре состоялся 23 апреля 2012 года, поступление в продажу — 29 апреля.

### *Особенности архитектуры*

- Переход на 22-нм техпроцесс (улучшение производительности и снижение энергопотребления).
- 16 графических исполнительных блоков (EU, Execution Units).
- Увеличение IPC (количества инструкций, выполняемых за такт), дополнение системы команд (Instruction Set Architecture) четырьмя инструкциями ускоренного доступа к базовым регистрам FS и GS, ускорение строковых инструкций REP MOVSB/STOSB, ускорение преобразования чисел с плавающей точкой из 16-битного формата в 32-битный формат.
- Кольцевая шина Ring Interconnect (более производительная чем QPI), объединяющая процессорные ядра, графическое ядро и системного агента (System Agent) через общий кэш последнего уровня (LLC, L3).
- Обратная совместимость с сокетом второго поколения процессоров Sandy Bridge.
- Новый 2- или 4-канальный контроллер DDR3, поддерживающий память до DDR3-2800 МТ/с, и DDR3L (низковольтная).
- Встроенный контроллер PCI Express 3.0 (кроме процессоров i3).

- Встроенная поддержка USB 3.0 (4 порта) в чипсетах 7 серии.
- Встроенная поддержка интерфейса Thunderbolt.
- Чипсет Panther Point с новым интерфейсом FDI, рассчитанным на одновременное подключение до трёх дисплеев
- Улучшенные технологии энергосбережения (конфигурируемое TDP, режим пониженного энергопотребления)
- Добавлен высокоскоростной и высококачественный аппаратный генератор случайных чисел с поддержкой стандартов ANSI X9.82, NIST SP 800-90 и NIST FIPS 140-2/3 сертификации уровня 2.
- Добавлена новая инструкция RDRAND для работы с генератором случайных чисел, возвращающая случайное число в 16-, 32- или 64-битный регистр.
- Добавлен новый режим защиты в режиме супервизора (SMEP, Supervisor Mode Execution Prevention) предотвращающий исполнение кода из пользовательских страниц.

### Интегрированное видео

- Интегрированное GPU доработано до соответствия требованиям API DirectX 11 с поддержкой стандарта HDMI 1.4a и подключения до трёх мониторов; было использовано два варианта графического ядра:
  - HD Graphics 2500 с частотами 650/1050 МГц в i3/i5 (за счет оптимизаций и специального кэша возможен паритет с HD Graphics 3000).
  - HD Graphics 4000 с частотами 650/1150 МГц (высокопроизводительное решение, ориентировано главным образом на ноутбуки, где использование дискретной графики наносит серьёзный удар по мобильности, в десктопных же процессорах можно было получить лишь в составе редких специальных предложений (например, процессор i3-3225 с TDP 55 Вт, который можно с успехом использовать в НТПС) либо как часть дорогих CPU, i7/i5-3570k).
- Поддержка нового поколения технологии Intel Quick Sync (примерно на 75% быстрее применяемого в Sandy Bridge) — ускорение кодирования и декодирования видео (в том числе и Full HD) средствами интегрированного GPU. Ориентировочная производительность IGP Ivy Bridge в тесте 3DMark 2006 — порядка 5000—6000 баллов.

Процессоры семейства используют новый чипсет Panther Point и совместимы с сокетом LGA1155. Владельцам материнских плат на чипсетах Cougar Point (P67, Z77, H67, Z68, H61) для поддержки новых процессоров Ivy Bridge было необходимо обновить BIOS материнской платы. Однако процессоры Ivy Bridge не поддерживаются чипсетами Q65, Q67 и B65.

Пиковая рассеиваемая мощность процессоров Ivy Bridge не превышала 77 Вт.

### Haswell

**Haswell** — кодовое название микроархитектуры четвёртого поколения процессоров Intel Core, которая является третьим значительным изменением микроархитектуры Core согласно стратегии разработки микропроцессоров «Тик-так» компании Intel вслед за «тиком» Ivy Bridge без изменения техпроцесса 22 нм, использующего транзисторы с трёхмерной структурой затвора.

Для процессоров Haswell предназначено новое семейство чипсетов Intel 8-й серии, включающее в себя модели H81, B85, H87, Q85, Q87 и Z87 для поддержки процессорного разъёма LGA1150 (версия Core i7 Extreme работала на сокете LGA2011-v3), которые осу-

щественно поддерживают до 6 портов USB 3.0 (а также функция I/O Port Flexibility, позволяющую задавать, какие именно USB-порты будут функционировать как 3.0) и до 6 шин SATA III (при этом отсутствует «чистый» SATA II). Также, оптимизирована работа с SSD, использованы улучшенные технологии Rapid Storage и Intel vPro, внедрена поддержка 4-поточного чтения через последовательный интерфейс, понижено энергопотребление, внесены другие улучшения.

Релиз первых процессоров на данной архитектуре состоялся 1-4 июня 2013 года на COMPUTEX 2013.

### **Особенности архитектуры**

- Конструктивное исполнение LGA1150 (Socket H3).
- Базовое количество ядер – 2 или 4.
- Полностью новый дизайн кэша.
- Улучшенные механизмы энергосбережения.
- Поддержка технологии Thunderbolt.
- Интегрированный векторный сопроцессор.
- Добавлены инструкции AVX 2; FMA (Fused Multiply Add); битовые инструкции BMI1 и BMI2.
- Расширение команд TSX (Transactional Synchronization Extensions) для аппаратной поддержки транзакционной памяти (кроме процессоров с индексом K). В августе 2014 была обнаружена неправильная работа инструкций TSX, и Intel выпустила микрокод, отключающий набор TSX в процессорах моделей Haswell и Haswell-E.
- Память eDRAM объёмом 64 Мбайт (по некоторым сведениям — 128 МБ) как отдельный кристалл, но в общей упаковке — только в процессорах для BGA (сокеты BGA1364), например, Core i7-4770R.
- Энергопотребление на 30% ниже по сравнению с аналогами из линейки Sandy Bridge; в некоторых режимах — до 20(!) раз ниже.

В чипе реализована возможность одновременной работы с четырьмя операндами, позволяющая за одну инструкцию совершать сразу две операции умножения и сложения либо вычитания.

Процессоры, построенные на архитектуре Haswell, имеют дополнительный интегрированный регулятор напряжений (VRM, FIVR), выполненный в виде отдельного кристалла под общей теплораспределительной крышкой. FIVR имеет размеры около 13×8 мм и изготовлен по 90-нм процессу.

### **Процессоры**

С появлением Haswell корпорация Intel разделяет ассортимент на две группы:

- настольные и мобильные версии;
- специальные версии для ультрабуков.

#### **Настольные версии**

Процессоры четвёртого поколения Intel Core для настольных компьютеров получили заметные улучшения в области автоматического и ручного разгона: повышены коэффициенты для Turbo-режима, памяти и графического ядра, расширены лимиты на питание и напряжение. Также была добавлена возможность разгона по частоте системной шины. В результате, по заявлению Intel, процессоры среднего звена Haswell в области разгона соответствуют по своим возможностям моделям Ivy Bridge для энтузиастов (X79 для LGA2011).

Все процессоры используют интерфейс DMI 2.0 PCIe 3.0.



Главное отличие процессоров Core i5 от Core i7 заключается в отсутствии Hyper-Threading (кроме Core i5 4570T и 4570TE) и уменьшенном объёме кэш-памяти L3.

Настольные процессоры серии R оснащены графикой Iris Pro 5200 (GT3e), другие настольные процессоры Core i5 и Core i7 и Core i3 4330 - 4370 имеют графическое ядро Intel HD 4600 Graphics (GT2), более младшие Core i3 – Intel HD 4400 Graphics, а Pentium и Celeron – Intel HD Graphics.

Большинство процессоров предназначено для установки в сокет LGA1150. Исключение составляют процессоры серии R, предназначенные для сокета BGA1364. Процессоры серии Haswell-E устанавливаются в сокет LGA 2011-v3.

### Серверные версии

Все серверные процессоры предназначены для сокета LGA1150, работают с шиной DMI 2.0 PCIe 3.0 и поддерживают до двух каналов DDR3-1333/1600 (ECC).

### Мобильные версии

Мобильные версии процессоров доступны в двух- или четырёхъядерных конфигурациях, но комплектуются контроллером памяти, поддерживающим только DDR3L либо LPDDR3 DIMM.

Platform Controller Hub (PCH) интегрирован в процессор, что несколько освобождает пространство на материнской плате.

## Broadwell

**Broadwell** (ранее также называлась **Rockwell**) — кодовое название процессорной микроархитектуры пятого поколения, разработанной Intel. Микроархитектура Broadwell представляет собой перенос микроархитектуры Haswell на техпроцесс 14 нм («тик»). Процессоры на базе Broadwell получили коммерческое название «Процессоры Intel(R) Core™ 5-го поколения». В отличие от предыдущих переходов на новые микроархитектуры, Broadwell не заменил весь диапазон версий Haswell, в частности, на её основе не выпускались дешёвые процессоры для настольных компьютеров.

Процессоры вариантов Broadwell H и C использовались с чипсетами Intel 9-й серии (Z97, H97 и HM97), сохранив обратную совместимость с некоторыми чипсетами Intel 8-й серии.

По состоянию на октябрь 2014 года было известно о 7 моделях процессоров с Broadwell, было доступно несколько моделей для ноутбуков.

Процессоры Broadwell станут первыми настоящими SoC<sup>1</sup> по мнению Bright Side Of News.

### Дизайн и варианты

Broadwell выпускался в трёх основных вариантах:

- Чипы с BGA-корпусом (не используют сокет, а распаиваются непосредственно на материнской плате):
  - *Broadwell-Y*: Система на кристалле (СнК, SoC); тепловыделение не более 4.5 Вт и 3.5 Вт, для планшетных компьютеров и некоторых ультрабуков. В качестве графического решения используют GT2; поддерживают до 8 Гбайт оперативной памяти LPDDR3-1600. Выпуск первых чипов произошёл в конце 2014 года, процессоры получили обозначение Intel Core M. В

---

<sup>1</sup> Система на кристалле (СнК), однокристальная система (System-on-a-Chip, SoC) — в микроэлектронике — электронная схема, выполняющая функции целого устройства (например, компьютера) и размещённая на одной интегральной схеме.



этой серии процессоров был запрещён набор инструкций TSX, из-за обнаруженной ошибки, которую невозможно было исправить обновлением микрокода.

- *Broadwell-U*: СнК (SoC); два класса тепловыделения (TDP) – 15 Вт для конфигураций 2+2 и 2+3 (два ядра с GPU GT2 или GT3) и 28 Вт для конфигураций 2+3 для двухъядерных процессоров с GT3. Предназначены для использования с чипсетом PCH-LP в интеловских ультрабуках и компактных настольных компьютерах NUC. Поддерживают либо до 16 Гбайт ОЗУ DDR3 или LPDDR3, с максимумом DDR3-1600 и LPDDR3-1867. Конфигурация 2+2 была запланирована на Q4 2014, конфигурация 2+3 – на Q1 2015. Для моделей Broadwell-U с интегрированным 5x00 GPU размер чипа 82 мм<sup>2</sup>, количество транзисторов около 1.3 млрд, модели с GPU 6100 и 6200 размер кристалла 133 мм<sup>2</sup>, количество транзисторов около 1.9 млрд.
- *Broadwell-H*: варианты с тепловыделением до 37 Вт или до 47 Вт, для плат с чипсетами HM86, HM87, QM87 и HM97 для систем «всё-в-одном», плат размера Mini-ITX и других компактных форматов. 2 или 4 ядра (4 или 8 потоков соответственно), графическое решение GT3e или GT2. Поддержка до 32 Гбайт ОЗУ DDR3-1600. Выход ожидается во втором квартале 2015 года.
- **Десктопная версия с разъёмом LGA1150 2-го поколения:**
  - *Broadwell-DT*: четырёхъядерная разблокированная десктопная версия с интегрированной графикой GT3e (Iris Pro 6200) и 128 Мбайт кэша eDRAM L4, TDP 65 Вт. Была анонсирована как обратно совместимая с материнскими платами LGA1150, спроектированных для процессоров Haswell.
- **Сокет LGA 2011-1:**
  - *Broadwell-EX*: Платформа Brickland для серверов mission-critical. Шина Intel QuickPath Interconnect (QPI) должна быть обновлена до версии 1.1, предоставляя возможность бесшовного масштабирования системы до восьми сокетов. Максимальная скорость памяти DDR3-1600 и DDR4-1866. До 24 ядер/48 потоков исполнения, до 60 Мбайт кэша L3, 32 линии PCI Express 3.0, TDP 115–165 Вт.
- **Сокет LGA 2011-v3:**
  - *Broadwell-EP*: маркируется как Xeon E5-2600 v4 etc., использует чипсет C610 Wellsburg. До 22 ядер/44 потоков, до 55 Мбайт кэша, 40 линий PCI Express 3.0, TDP 55–160 Вт. Максимальная поддерживаемая скорость памяти – четырёхканальная DDR4-2400.
  - *Broadwell-E*: платформа HEDT для энтузиастов. Объявлена на Computex 2016, была выпущена в июле этого же года. Серия состояла из четырёх процессоров: the 6800K, 6850K, 6900K и 10-ядерный 6950X, с тактовыми частотами от 3 до 4 ГГц, 25 Мбайт кэша L3.

Broadwell с решением GT3e представлял собой многокристальную сборку (Multi-Chip Package, MCP), в состав которой входил процессор и отдельный чип L4 памяти, либо чип встроенного южного моста.

По заявлениям производителя, быстродействие графической подсистемы по сравнению с подсистемой Haswell должно вырасти на 40%.

### **Расширение инструкций**

Уже традиционно, в Broadwell было добавлено несколько расширений в систему команд:

- *Intel ADX*: инструкции ADOX и ADCX для увеличения производительности при работе с целыми числами произвольной точности.

- *RDSEED* для генерации 16-, 32- или 64-битных случайных чисел из a thermal noise entropy stream, в соответствии с NIST SP 800-90B и 800-90C.
- Инструкция *PREFETCHW*, которая может быть использована с помощью встроенной функции `_mm_prefetch` с подсказкой `_MM_HINT_ET0`.
- *Supervisor Mode Access Prevention (SMAP)* – опционально запрещает доступ из памяти пространства ядра к памяти пространства пользователя, функция, направленная на то, чтобы усложнить использование ошибок программного обеспечения.
- *Transactional Synchronization Extensions (TSX)*: Этот набор инструкций был повторно введён во все версии Broadwell, за исключением Broadwell-Y, из-за ошибки, которая не могла быть исправлена обновлением микрокода в Broadwell-Y и во всех версиях Haswell (за исключением варианта Haswell-EX, где эта ошибка была исправлена в новом степпинге CPU). *Похоже, опечатка: фактически, из всех процессоров Broadwell i3, i5 и i7, только четыре из них поддерживали инструкции TSX (i7 5650U и 5600U, i5 5350U and 5300U); на сайте Intel даже не уточнено, что i5 5200U вообще не поддерживает инструкции TSX instructions.*

## Skylake

**Skylake** — кодовое название шестого поколения микроархитектуры центральных процессоров Intel Core, которая является четвёртым значительным изменением микроархитектуры Core согласно стратегии разработки микропроцессоров «Тик-Так» компании Intel вслед за «тиком» Broadwell, без изменения технологического процесса 14 нм. Первая микроархитектура в «озёрной» серии, была объявлена в августе 2015 года. На середину 2020 года пока это был последний «тик», в соответствии с таблицей на Рис.368, все следующие поколения – с 7 по 10 (Kaby Lake, Coffee Lake, Cannon Lake, Whiskey Lake, Comet Lake и Rocket Lake) – попали в категорию «Оптимизация».

Skylake стала последней платформой Intel, на которой Microsoft официально поддерживает версии Windows более ранние, чем Windows 10. Хотя силами энтузиастов были созданы модификации, которые позволяют Windows 8.1 и более ранним версиям работать и получать обновления на более поздних платформах.

4 марта 2019 года Intel объявила об окончании жизненного цикла и прекращении поддержки процессоров Skylake LGA1151.

Были обещаны к выпуску следующие серии чипов:

- Skylake-S (LGA1151) — для настольных ПК;
- Skylake-U (BGA1356) — для мобильных устройств (ультрабуки, тонкие и лёгкие ноутбуки);
- Skylake-H (BGA1440) — высокопроизводительные ноутбуки;
- Skylake-Y (BGA1515) — безвентиляторные устройства, планшеты и гибридные гаджеты;
- Skylake-X (LGA2066) — высокопроизводительные настольные ПК;
- Skylake-EP, Skylake-EX (LGA3647) — серверные процессоры;
- Skylake-SP (LGA3647) — масштабируемые высокопроизводительные серверы.

Первые процессоры архитектуры Skylake Core i7-6700K и Core i5-6600K были выпущены в августе 2015 года вместе с сопутствующим выпуском чипсета Z170. Между 30 августа и 5 сентября 2015 года Intel выпустили чипы Core i7-6700/6700T, Core i5-6600, 6500, 6400, 6600T, 6500T и 6400T с системной логикой H170 и B150. Чипсет H110 был запланирован к выпуску на период между 27 сентября и 3 октября 2015 года, а логика Q170 и Q150 была запланирована на октябрь или ноябрь 2015 года.

Выпуск процессоров Skylake для ноутбуков был запланирован на октябрь 2015 года.

Младшие версии настольных и мобильных систем получают корпус BGA. Старшие, высокопроизводительные процессоры перейдут на очередной новый процессорный разъем LGA1151 с набором логики Z170/H170/B150/Q150/Q170.

В сентябре 2015 года Intel официально представила процессоры семейства Skylake, состоящее из 46(!) моделей, не считая двух уже выпущенных Skylake-K.

### Возможности архитектуры

Подобно своему предшественнику, серии Broadwell, Skylake была доступна в пяти вариантах, которые различались по суффиксу в названии: “S” (SKL-S), “X” (SKL-X), “H” (SKL-H), “U” (SKL-U) и “Y” (SKL-Y). SKL-S and SKL-X включают варианты “K” и “X” с поддержкой оверклокинга с разблокированными множителями. Варианты H, U и Y упакованы в корпус BGA (ball grid array), варианты S и X упакованы в новый LGA1151 (LGA2066 для Skylake X). Skylake используется в сочетании с чипсетом Intel 100 серии, также известного как Sunrise Point.

Основные изменения в архитектуре Skylake по сравнению с Haswell включают удаление полностью интегрированного регулятора напряжения (FIVR), появившегося в Haswell, с заменой его на дискретный Platform Controller Hub (PCH). Интерфейс Direct Media Interface (DMI) 2.0 был заменён DMI версии 3.0, что увеличило скорость до 8 GT/s.

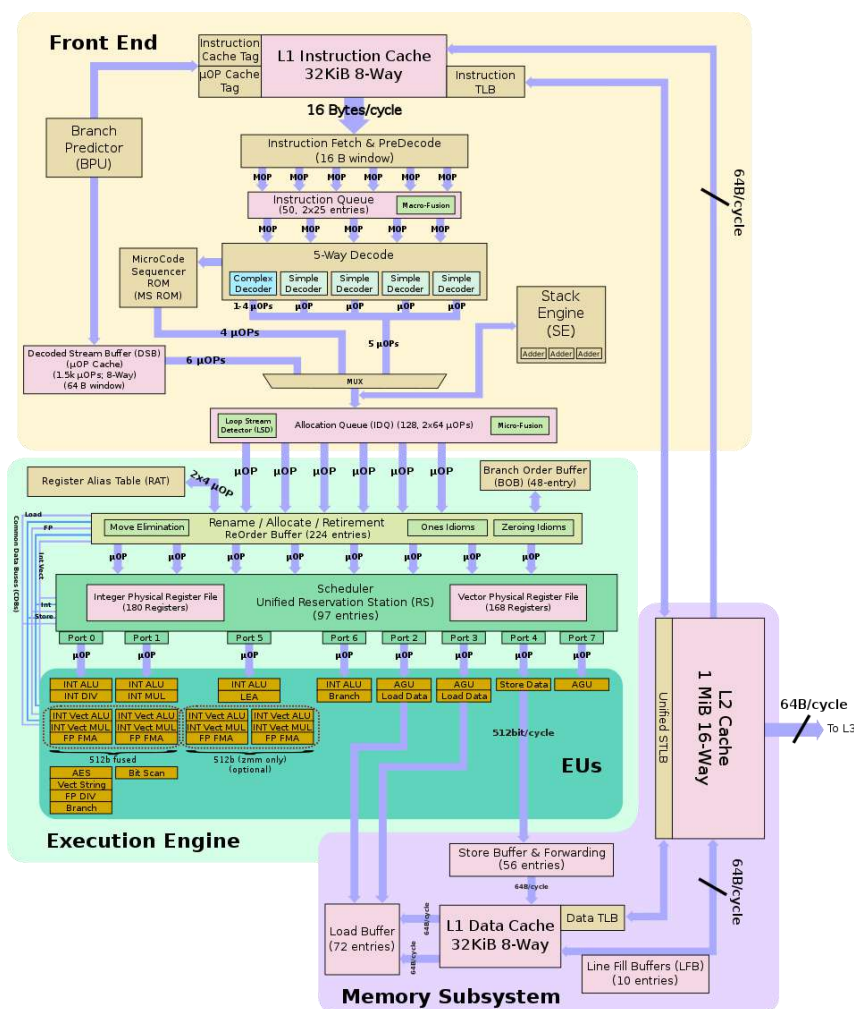


Рис.373 - Микроархитектура Intel Skylake (сервер, одно ядро).

Варианты Skylake U и Y поддерживают один слот DIMM на канал, тогда как варианты H и S поддерживают два слота DIMM на канал. Выпуск и начало продаж Skylake пришлись на время перехода индустрии от памяти стандарта DDR3 SDRAM к DDR4. Архитек-

тура Skylake сохранила обратную совместимость, позволяя работать с обоими типами памяти: и DDR3, и DDR4. С поддержкой микроархитектурой двух стандартов памяти, также был анонсирован новый тип SO-DIMM модулей, названный UniDIMM, способный содержать чипы памяти DDR3 или DDR4.

Некоторые Р варианты Skylake имели урезанный графический процессор (12 разрешённых исполнительных устройств вместо полных 24 execution units). В противоположность этому CPU Ivy Bridge с суффиксом Р имели полностью отключённую встроенную графику.

Другие расширения включали поддержку Thunderbolt 3.0, SATA Express, Iris Pro graphics с Direct3D уровня 12\_1, с L4 eDRAM кэшем объёмом до 128 Мбайт на некоторых SKU. Линейка процессоров Skylake полностью прекратила поддержку VGA, поддерживая до пяти мониторов, подключаемых по интерфейсам HDMI 1.4, DisplayPort 1.2 или Embedded DisplayPort (eDP). Стандарт HDMI 2.0 (4K@60 Hz) поддерживался только на материнских платах, оборудованных интеловским контроллером Alpine Ridge Thunderbolt.

Набор инструкций Skylake был расширен наборами Intel MPX (Memory Protection Extensions) и Intel SGX (Software Guard Extensions). Будущие варианты Xeon также получили Advanced Vector Extensions v3.2 (“AVX-512F”).

Согласно прогнозам, ноутбуки на базе Skylake должны были использовать беспроводную технологию Rezenze для зарядки и другие беспроводные технологии для связи с периферийными устройствами. Многие крупные производители ПК согласились использовать эту технологию в ноутбуках на базе Skylake; однако по состоянию на 2019 год ни один ноутбук с этой технологией не был выпущен.

Вариант интегрированного GPU в Skylake поддерживал стандарты Windows DirectX 12 Feature Level 12\_1, OpenGL 4.6 с последними обновлениями драйверов Windows 10 (OpenGL 4.5 на Linux) and OpenCL 2.0, а также некоторые современные стандарты аппаратного кодирования/декодирования, такие как VP9 (GPU ускорял только процесс декодирования), VP8 и HEVC (аппаратно ускоренное 8-битное кодирование/декодирование и ускоренное средствами GPU 10-битное декодирование).

Intel также реализовала возможность разблокировки (для разгона) мобильных версий процессоров Skylake.

В отличие от предыдущих поколений, Xeon E3 на основе Skylake официально больше не работали с десктопными чипсетами 100-й и 200-й серии, поддерживающими тот же самый сокет, и требовали для работы чипсет C232 или C236. *Тем не менее, на 2019 год группе энтузиастов удалось обойти данное ограничение и сделать программу, которая снимает подобные ограничения путем отключения части функционала подпрограммы Intel ME.*

### **Известные проблемы**

Короткие циклы со специфическим сочетанием инструкций могли вызвать непредсказуемое поведение системы на CPU с Hyper Threading. Для исправления проблемы были выпущены обновления микрокода.

Skylake подвержен атакам класса Spectre. In fact, it is more vulnerable than other processors because it uses indirect branch speculation not just on indirect branches but also when the return prediction stack underflows.

Задержка для инструкции spinlock PAUSE была значительно увеличена (с обычных 10 циклов до 141 цикла), что могло вызвать проблемы с производительностью со старыми программами или библиотеками, использующими инструкцию pause. В документации Intel отметила увеличение задержки как свойство, которое улучшает энергоэффективность процессора.

## Архитектура

- Улучшенный front-end интерфейс, увеличена глубина out-of-order буферов, улучшена работа исполнительных блоков (юнитов), исполнительных юнитов стало больше (был добавлен третий векторный целочисленный ALU (VALU)) для пяти ALU в целом, большая пропускная способность загрузки/сохранения, улучшена технология Hyper-Threading (более широкий retirement), ускорение AES-GCM и AES-CBC на 17% и 33%, соответственно.
- 14 нм технологический процесс.
- Сокет LGA1151 для массовых десктопных процессоров и сокет LGA2066 для процессоров «X-серии» для энтузиастов игровых/рабочих станций.
- Чипсет 100-й серии (Sunrise Point).
- «X-серия» использовала чипсет серии X299.
- Thermal design power (TDP) до 95 Вт (LGA 1151); до 165 Вт (LGA 2066).
- Поддержка двух стандартов памяти DDR3L SDRAM и DDR4 SDRAM в mainstream вариантах, используя custom UniDIMM SO-DIMM форм-фактор с возможностью иметь до 64 Гбайт ОЗУ на LGA1151 вариантах. Обычная DDR3 память также поддерживалась некоторыми производителями материнских плат, хотя Intel официально DDR3 не поддерживала.
- Поддержка 16 линий PCI Express 3.0 lanes из процессора CPU, 20 линий PCI Express 3.0 из PCH (LGA 1151), 44 линий PCI Express 3.0 для Skylake-X.
- Поддержка интерфейса Thunderbolt 3 (Alpine Ridge).
- От 64 до 128 Мбайт L4 eDRAM кэша на некоторых SKUs.
- До четырёх ядер в базовой стандартной конфигурации, и до 18 ядер для X-серии.
- AVX-512: F, CD, VL, BW, и DQ для некоторых будущих вариантов Xeon, но не для Xeon E3.
- Intel MPX (Memory Protection Extensions).
- Intel SGX (Software Guard Extensions).
- Intel Speed Shift.
- Интегрированный Gen9 GPU поддерживает Direct3D 12 уровня 12\_1.
- Полностью исправлено функционирование ускорителя HEVC Main/8bit кодирования/декодирования. Гибридное/частичное ускорение HEVC Main10/10bit декодирования. Ускорение кодирования JPEG для разрешения до 16,000×16,000 пиксел. Частичное ускорение VP9 кодирования/декодирования.
- Размер кэша L1 остался неизменным – 32 КБайт (KiB) кэш инструкций и 32 КБайт (KiB) кэш данных на каждое ядро.
- Усовершенствована технология Intel Processor Trace: точная синхронизация пакетов CYS (режим с точностью до цикла) и поддержка фильтрации IP (Instruction Pointer) адресов.

## Топология mesh

Для связи компонентов чипа, то есть вычислительных ядер, кэш-памяти L3, графического ядра и системного агента (контроллеров памяти, PCI Express, питания и дисплея), в архитектурах, начиная с Sandy Bridge, использовалась кольцевая шина (ring bus). За основу была взята скоростная шина QPI (Quick Path Interconnect, пропускная способность до 6.4 Гбайт/с на частоте 3.2 ГГц), впервые реализованная в чипах Nehalem Lynnfield (Core i7 9xxx для Socket LGA1366), адресованных энтузиастам. Кольцевая шина была необходима для более эффективного обслуживания увеличивающегося количества ядер на кристалле. Разработчики Intel использовали три варианта дизайна процессоров (в зависимости от максимального числа ядер на кристалле) с тремя вариантами кольцевой шины. В самом сложном случае процессор внутри разделялся на два кластера, каждый из которых обслуживался

двумя кольцевыми шинами. Между собой шины соединялись двунаправленными коммутаторами с буферизацией (на Рис.374 обозначены серым цветом).

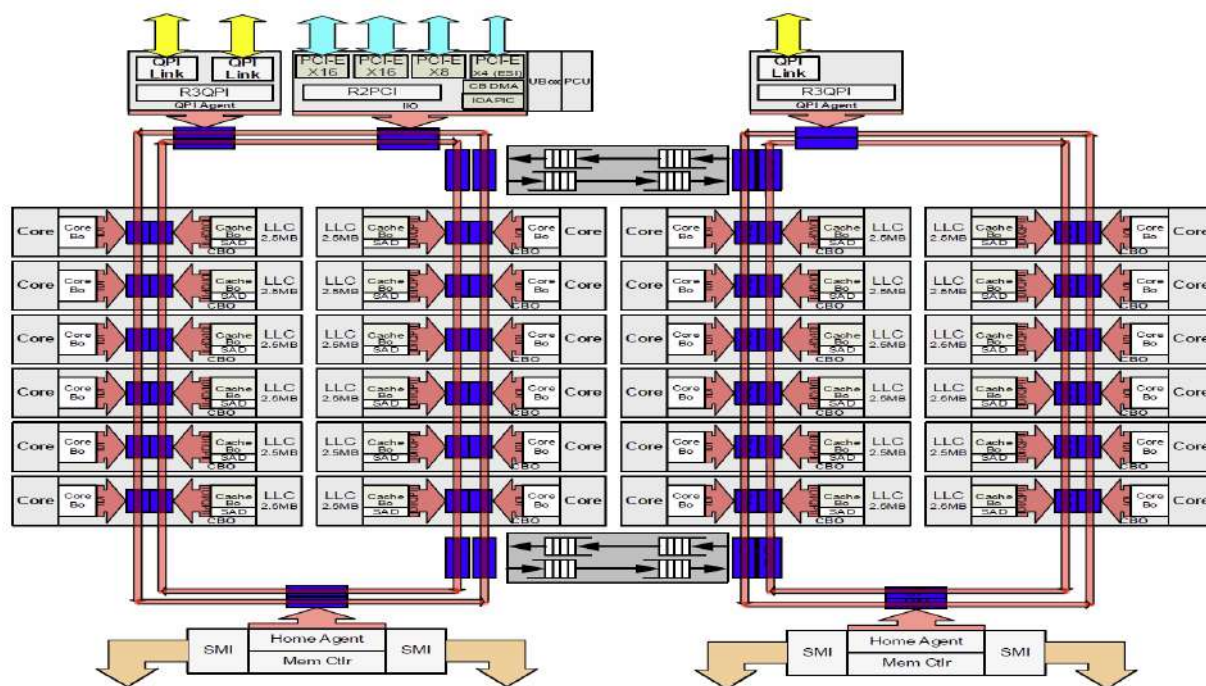


Рис.374 – Intel® Xeon® Processor E7 v4 Product Family

В марте 2017 года компания Intel показала изображение нового 28-ядерного кристалла процессора Skylake-SP и тогда многие обратили внимание, что расположение ядер и интерфейсов в этом процессоре претерпели значительные изменения.

По мере роста числа ядер кольцевая шина стала препятствием на пути увеличения пропускной способности и снижения задержек. Она становилась слишком энергоэффективной, затрудняя масштабирование в сторону увеличения скорости обмена данными при увеличении количества пользователей шины. Из-за особенностей топологии, одна связка (кластер) способна взаимодействовать с ограниченным числом ядер (10-12). Рекордсменом по количеству ядер с топологией кольцевой шины был процессор Broadwell-EP (Intel® Xeon® E7-8890 v4) – 24 физических ядра.

Решая проблему ограничения при масштабировании ring bus, в процессорах Skylake-SP (-X) разработчики Intel решили применить иную структуру для связи ядер друг с другом – ранее успешно опробованную в архитектуре Intel® Xeon® Phi (Knights Landing) т.н. ячеистую сеть (mesh) (Рис.375), в которой каждое ядро (в Intel® Xeon® Phi их количество могло быть более 70) поддержано собственным коммутатором и возможностью координатной пересылки данных фактически напрямую любому другому ядру в процессоре.

Кэш L3, контроллеры памяти и ввода/вывода также были интегрированы в mesh сетку. В результате с такой топологией оказалось намного проще наращивать количество ядер процессора, сохраняя при этом высокую пропускную способность и низкие задержки.

При переходе на топологию сетки Intel пришлось также изменить расположение контроллеров памяти. Раньше они располагались ниже ядер, теперь они оказались вынесены в правую и левую часть кристалла. Все это обеспечивало более эффективную работу с памятью (меньшие задержки, Рис.375).

Сеть состоит из горизонтальных и вертикальных межсоединений между ядрами, кэшем и контроллерами ввода-вывода. Возможность «ступенчатого» движения данных через ядра позволяет осуществлять гораздо более сложную и более эффективную маршрути-



зацию. Intel утверждает, что 2D-сеть имеет более низкое напряжение и частоту, чем у кольцевой шины, но при этом обеспечивает более высокую пропускную способность и более низкую задержку<sup>1</sup>.

Каждое ядро в такой архитектуре имеет свой коммутатор с буфером и связано с любым другим ядром в составе процессора только через два узла – исходящий и входящий. Это позволяет ячеистой сети работать на относительно небольших частотах и существенно снизить общее энергопотребление интерфейса без ухудшения пропускной способности и увеличения задержек. К тому же подобная структура коммуникаций очень хорошо масштабируется, позволяя в будущем наращивать количество ядер на кристалле без заметного увеличения энергетических затрат на внутреннюю транспортировку данных.

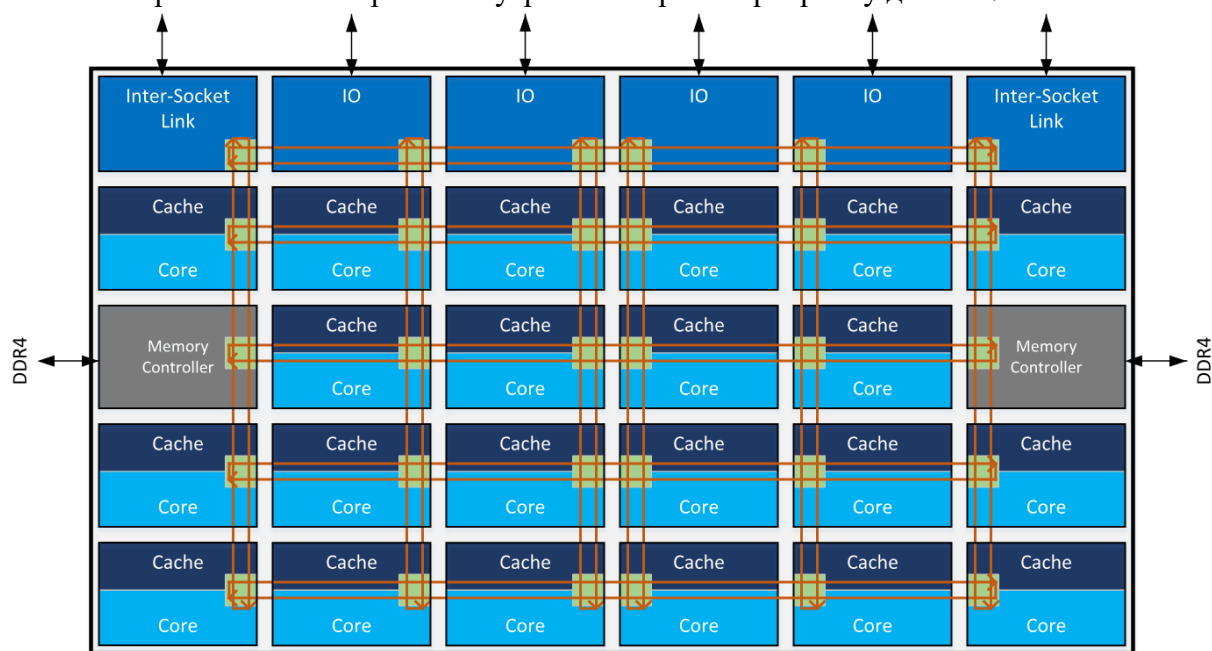


Рис.375 – Структура full mesh сети в Skylake-SP (-X).

## Kaby Lake, Kaby Lake Refresh

**Kaby Lake** — кодовое название семейства процессоров Intel Core 7-го поколения, которая является усовершенствованным «таким» Skylake с незначительным изменением техпроцесса 14-нм до 14-нм+. Появление процессоров в рознице состоялось в начале 2017 года.

### Особенности

- 14-нм технологический процесс 14FF+
- Конструктивное исполнение LGA1151
- Базовое количество ядер — 2 или 4
- Чипсет 200-й серии (Union Point); поддерживаются чипсеты 100-й серии (может потребоваться обновление BIOS'a)
- Требования по теплоотводу (TDP) до 95 Вт (LGA1151)
- Поддержка DDR3L SDRAM и DDR4 SDRAM, ограниченная поддержка UniDIMM SO-DIMM
- Поддержка PCI Express 3.0
- Поддержка USB 3.1, в отличие от Skylake, где требуется наличие дополнительных контроллеров на материнской плате для работы USB 3.1 портов

<sup>1</sup> В шине (по определению) полоса пропускания делится на всех, коммутатор – соединение точка-точка, каждый линк работает со скоростью потребителя, ни с кем не деля полосу пропускания.



- Поддержка Thunderbolt 3 с возможностью подключения до двух мониторов с разрешением 4K, либо один с разрешением 5K
- Опциональная память eDRAM 4-го уровня от 64 до 128 МБайт
- Новая графическая архитектура для повышения производительности при работе с трёхмерной графикой и воспроизведения 4K видео
- Поддержка технологии памяти Intel Optane (3D Xpoint)
- Поддержка формата видеосжатия HEVC (H.265) с глубиной цвета 10 бит и VP9 с глубиной цвета 10 бит и ускорением декодирования
- Нативная поддержка технологии защиты медиаконтента HDCP 2.2
- Официальная совместимость только с Microsoft Windows 10 и отсутствие таковой с предыдущими операционными системами Microsoft

## Бренды

Интел выпускает процессоры семейства Kaby Lake под следующими брендами (Рис.376):

| Семейство    | Сегмент рынка                        | Технологические особенности |    |     |      |     |     |
|--------------|--------------------------------------|-----------------------------|----|-----|------|-----|-----|
|              |                                      | Ядра                        | HT | AVX | AVX2 | TBT | ECC |
| Celeron      | Начальный уровень, Бюджетные         | 2                           | X  | X   | X    | X   | X   |
| Pentium      | Бюджетные (Мобильные)                | 2                           | ✓  | X   | X    | X   | X   |
|              | Бюджетные (Настольные)               |                             | ✓  | X   | X    | X   | ✓   |
| Pentium Gold | Бюджетные (Мобильные)                | 2                           | ✓  | X   | X    | X   | X   |
|              | Бюджетные (Настольные)               |                             | ✓  | X   | X    | X   | ✓   |
| Core i3      | Низкопроизводительные                | 2                           | ✓  | ✓   | ✓    | X   | X   |
|              | Низкопроизводительные (Встраиваемые) |                             | ✓  | ✓   | ✓    | X   | ✓   |
| Core i5      | Средней производительности           | 2                           | ✓  | ✓   | ✓    | ✓   | X   |
|              | Средней производительности (KBL-R)   | 4                           | X  | ✓   | ✓    | ✓   | X   |
| Core i7      | Высокопроизводительные               | 2                           | ✓  | ✓   | ✓    | ✓   | X   |
|              |                                      | 4                           | ✓  | ✓   | ✓    | ✓   | X   |
| Xeon E3      | Рабочие станции/Сервера              | 4                           | ✓  | ✓   | ✓    | ✓   | ✓   |

Рис.376 – Процессоры семейства Kaby Lake

## Процессоры 8-го поколения

Процессоры 8-го поколения семейства **Kaby Lake Refresh** имеют четыре физических ядра и поддерживают восемь вычислительных потоков благодаря технологии Hyper-Threading. Turbo-частоты немного увеличились, а базовые — наоборот, уменьшились (для удержания TDP на уровне 15 Вт). Поддерживается более скоростная память DDR4-2400 и LPDDR3-2133. Модели U-серии выпускаются по 14-нанометровому техпроцессу.

## Процессоры Amber Lake-Y

28 августа 2018 Intel анонсировала обновлённую линейку мобильных процессоров с пониженным энергопотреблением под кодовым названием Amber Lake. Эти процессоры используют разъём BGA1515.

## Coffee Lake

**Coffee Lake** — кодовое название семейства процессоров восьмого поколения Intel Core. Согласно стратегии разработки микропроцессоров «Тик-так» компании Intel, вслед за

«тиком» Broadwell последовал «так» в виде Skylake, Kaby Lake и Coffee Lake с незначительным изменением техпроцесса с 14-нм до 14-нм+. Основным отличием семейства процессоров является увеличение до шести количества ядер в настольных (Coffee Lake-S) и мобильных (Coffee Lake-H) вариантах процессора. Тепловой пакет (TDP) для настольных процессоров составляет 95 Вт, мобильных — до 45 Вт, а «ультрабучной» категории Coffee Lake-U — 28 Вт. Настольные процессоры имеют встроенную графику Intel UHD Graphics 630 с eDRAM и поддержкой DP 1.2 на HDMI 2.0 и HDCP 2.2.

Производительность процессоров Coffee Lake на 15 процентов больше по сравнению с процессорами Kaby Lake. Компания Intel сообщает о 30-процентном приросте производительности процессоров Coffee Lake в тесте SYSmark 2014 в сравнении с процессорами Skylake U-серии с TDP 15 Вт.

Чипы официально анонсированы 24 сентября 2017 года и стали доступны для покупки, начиная с 5 октября 2017 года. Из-за повышенных требований к системе питания гнезда LGA1151 процессоры Coffee Lake официально совместимы только с материнскими платами на чипсете 300-й серии. Однако нескольким энтузиастам из Китая удалось путем модификации микрокода BIOS запустить процессоры 7-го поколения на плате с чипсетом Z370 и наоборот, 4-ядерные процессоры 8-го поколения на плате с чипом Z170.

### ***Ключевые отличия от семейства Kaby Lake***

Микроархитектура новых CPU практически не претерпела изменений.

- Увеличена производительность многопоточных вычислений: до 4 процессорных ядер в моделях процессоров i3; До 6 ядер в моделях i5 и i7;
- Увеличен размер кэша L3 в соответствии с количеством ядер;
- Увеличены частоты в турборежиме в моделях процессоров i5 и i7 на 200 МГц;
- Увеличена тактовая частота встроенной графики на 50 МГц;
- Добавлена поддержка памяти DDR4 до 2666 МГц (для процессоров i5 и i7) и 2400 МГц (для i3 процессора); официально память DDR3 больше не поддерживается;
- Тепловая мощность (TDP) до 95 Вт (в LGA1151), при активации турборежима доходящая до 145 Вт.

Тесты IPC (Instructions per cycle) не показывают никакой разницы в сравнении со Skylake или Kaby Lake.

21 августа 2017 года Intel заявила, что восьмое поколение процессоров Core будет базироваться на нескольких микроархитектурах, включая Kaby Lake, Coffee Lake и Cannonlake.

### ***Coffee Lake Refresh***

В четвертом квартале 2018 года были выпущены процессоры Coffee Lake 9-го поколения, которые оснащены аппаратными средствами защиты от некоторых уязвимостей Meltdown/Spectre.

Впервые в истории потребительских процессоров Intel эти процессоры поддерживают до 128 Гбайт оперативной памяти.

### ***Неофициальная поддержка DDR3 памяти***

Несмотря на отсутствие официальной поддержки DDR3 памяти, процессоры Coffee Lake, как и его предшественники вполне успешно могут работать с данным устаревшим типом памяти при использовании специализированных материнских плат.

## Whiskey Lake

**Whiskey Lake** — кодовое название семейства процессоров восьмого поколения Intel Core. Данное семейство использует оптимизированный технологический процесс 14 нм и является эволюцией микроархитектуры Skylake, вслед за Kaby Lake Refresh и Coffee Lake. О доступности мобильных процессоров этого поколения было объявлено 28 августа 2018 года. Чипы Whiskey Lake получили первые аппаратные исправления против уязвимостей Meltdown и Foreshadow.

## Comet Lake

**Comet Lake** — кодовое название семейства процессоров десятого поколения Intel Core. Данное семейство использует оптимизированный технологический процесс 14 нм и является эволюцией микроархитектуры Skylake, вслед за Kaby Lake Refresh, Coffee Lake и Whiskey Lake. О доступности мобильных процессоров этого поколения было объявлено 21 августа 2019 года.

### Особенности

- Макс. до 10 ядер/20 потоков;
- Макс. до 5.3 ГГц;
- Макс. до 20 Мбайт кэш L3;
- Макс. TDP до 125 Вт
- Поддержка DDR4-2666, LPDDR4x-2933;
- Встроенный контроллер беспроводных сетей Intel Wi-Fi 6 AX201 со скоростью передачи данных более 1 Гбит/с.

## Cannon Lake

**Cannon Lake** — кодовое имя микроархитектуры 8-го поколения процессоров фирмы Intel. Чипы на этой микроархитектуре изготавливаются на технологическом процессе 10 нанометров. Микроархитектура Cannon Lake — преемница микроархитектуры Coffee Lake-Refresh, чипы которой изготавливались по норме 14 нанометров. Согласно этапам разработки процессоров Intel — это так этап «тик», на котором техпроцесс не оптимизируется, а полностью меняется: с 14 нм на 10нм.

Первоначально ожидалось, что Cannon Lake появятся в 2016 году, но позже из-за проблем с наладкой производственных линий на технологии 10 нм выпуск был отложен до конца 2017. В очень ограниченном количестве первые чипы Cannon Lake были изготовлены в начале 2018 года: их получили партнёры компании.

Во второй половине 2018 года процессоры были-таки выпущены ограниченной партией (двухъядерный с Hyper-Threading i3-8121U<sup>1</sup>), устанавливались в некоторые партии ноутбука Lenovo IdeaPad 330.

## Подробности о CPU Rocket Lake-S, неизвестные модели Cannon Lake

Весной 2020 года в Сети появились интересные сведения о будущих процессорах Intel и о CPU, производство которых вроде бы прекращено.

Впервые появилось публичное упоминание о процессоре *Rocket Lake-S* — это настольная платформа, которая будет выпущена после *Comet Lake-S*. *Rocket Lake-S* является «обратным переносом» (backport) неизвестной новой микроархитектуры CPU (*Sunny* или *Willow Cove*), то есть она будет производиться по старому 14 нм техпроцессу.

Intel уже тестирует первые образцы CPU *Rocket Lake-S*, количество ядер в процессоре — восемь, т.е., с поддержкой Hyper-Threading мы получим 16 потоков. Но прежде Intel

---

<sup>1</sup> 10-нм процессор Core i3-8121U имеет два ядра, лишён встроенной графики; поставляется в очень ограниченных количествах.

должна (обещала) выпустить новые процессоры *Comet Lake-S*. Мобильные *Comet Lake-H* уже вышли, так что ждать осталось недолго.

Процессоры *Rocket Lake-S* – это будущее, но в базе данных тестов процессоров можно встретить некоторые записи, указывающие на процессоры, которых быть как бы не должно. В своё время CPU Cannon Lake стали пробным шаром Intel с 10 нм техпроцессом. Но проблемы с производством были весьма существенными, что вынудило Intel много раз переносить запуск 10 нм техпроцесса и выпуск этой линейки CPU.

Мобильные процессоры *Cannon Lake* вышли весьма ограниченными партиями. Более того, встроенная графика (Gen10) у них была всегда отключена. Вероятно, из-за ошибок при производстве кристаллов. Intel легко распрощалась с процессорами *Cannon Lake*, сконцентрировавшись на мобильных *Ice Lake*.

Между тем в базе данных тестов появились записи с процессором *Cannon Lake-Y* (*CannonLake-Y 2+2*), который оснащён двумя физическими ядрами и работающей графикой Gen10. Процессор фигурирует под названием Core m3-8114Y, но официально он так и не вышел. Процессор в тестах обошел Core m3-7Y30 (*Kaby Lake-Y*), по крайней мере, по производительности CPU.

Вероятно, у Intel имеются не только образцы младших процессоров *Cannon Lake*, но и старших *Cannon Lake-H*, содержащих до восьми ядер. Линейка H ориентирована на игровые ноутбуки и мобильные рабочие станции, как и *Comet Lake-H*. Максимальная частота Boost образца составила 4,2 ГГц. В базе данных есть и шестиядерные модели.

У Intel, вероятно, были большие планы по процессорам *Cannon Lake*, но их пришлось похоронить. Возможно, с 2021 года Intel может к ним вернуться. Пока что стратегия Intel весьма запутана. Например, к 10-му поколению Core относятся две разные микроархитектуры и два разных техпроцесса. В 2021 году ожидаются процессоры *Rocket Lake-S*, первые процессоры *Tiger Lake* должны появиться уже в конце 2020 года, как и *Ice Lake Xeon*. Но что выйдет в реальности, пока не очень понятно.

|                | Техпроцесс | Архитектура CPU   | Архитектура GPU | Дата выхода |
|----------------|------------|-------------------|-----------------|-------------|
| Coffee Lake-S  | 14 нм      | Skylake           | Gen9.5          | 2018        |
| Skylake-X      | 14 нм      | Skylake           | -               | 2018        |
| Cascade Lake-X | 14 нм      | Skylake           | -               | 2019        |
| Comet Lake-S   | 14 нм      | Skylake           | Gen9.5          | 2019        |
| Rocket Lake-S  | 14 нм      | Sunny/Willow Cove | Gen12           | 2021        |
| Tiger Lake     | 10 нм      | Willow Cove       | Gen12           | 2021/22     |
| Alder Lake     | 10 нм      | -                 | -               | 2022        |
| Meteor Lake    | 7 нм       | -                 | -               | 2022/23     |

Рис.377 - Процессоры Intel для настольных ПК, настоящее время и ближайшие планы.

### ***Rocket Lake-S – уже совсем скоро?***

В некоторых новостных лентах в конце июля 2020 года можно было встретить новости с примерно таким заголовком: «8 ядер и 5,0 ГГц. Процессоры Intel Rocket Lake-S постепенно прибавляют в частоте».

Содержимое новости было следующим: компания Intel готовит линейку настольных процессоров *Rocket Lake-S*, которые придут на смену нынешним *Comet Lake-S*. Ранее представители перспективной серии уже появлялись в бенчмарках, но их частоты не впечатляли:

4,1 и 4,3 ГГц — это не то, что ждёшь от Intel даже при сохранении 14-нанометрового техпроцесса. Но вот появился новый тест, и он выглядит уже гораздо лучше: потенциальный 8-ядерный *Rocket Lake-S* продемонстрировал частоту 5,0 ГГц. При этом базовая частота составила 3,4 ГГц (таблица на Рис.378).

| System Information    |                                               |
|-----------------------|-----------------------------------------------|
| Operating System      | Microsoft Windows 10 Enterprise (64-bit)      |
| Model                 | Intel Corporation Rocket Lake Client Platform |
| Motherboard           | Intel Corporation RocketLake S UDIMM 6L RVP   |
| Processor Information |                                               |
| Name                  | Intel 0000                                    |
| Topology              | 1 Processor, 8 Cores, 16 Threads              |
| Identifier            | GenuineIntel Family 6 Model 167 Stepping 0    |
| Base Frequency        | 3.41 GHz                                      |
| Maximum Frequency     | 4.98 GHz                                      |
| Package               |                                               |
| Codename              |                                               |
| L1 Instruction Cache  | 32.0 KB x 8                                   |
| L1 Data Cache         | 48.0 KB x 8                                   |
| L2 Cache              | 512 KB x 8                                    |
| L3 Cache              | 16.0 MB x 1                                   |

Рис.378 – Таблица результатов тестирования процессора Rocket Lake-S

Интересно, что в сравнении с 8-ядерным Core i7-10700K семейства *Comet Lake-S*, новинка немного превзошла нынешнюю модель в однопоточном тесте, но значительно уступила в многопоточном. Впрочем, надо сделать скидку на то, что тестировался предсерийный образец — производительность товарной версии должна быть выше.

Пока нет никакой конкретики относительно даты выпуска *Rocket Lake-S*. Intel ничего не сказала об этих процессорах во время оглашения финансовых итогов за второй квартал, во время пресс-брифинга о них тоже никто не упомянул. Если принять во внимание, что CPU *Alder Lake-S* должны выйти во второй половине 2021 года, то премьера *Rocket Lake-S* должна состояться или в конце этого года (2020), или в начале следующего (2021). В отличие от 10-нанометровых *Alder Lake-S*, *Rocket Lake-S* будут производиться по нормам 14-нанометрового техпроцесса, что на сегодня уже воспринимается как катастрофа. Буквально только что (август 2020 года) мелькали сообщения, что TSMC, производящая CPU EPYC для AMD по 7 нм техпроцессу, нашла первого потребителя для своего новейшего 2 нм (двухнанометрового!) техпроцесса. На этом фоне многолетнее топтание Intel в районе 14 нм выглядит уже совсем безнадежно. Так кардинально и так надолго в технологиях компания в своей истории ещё никогда не отставала, наоборот, она всегда была технологическим лидером мировой полупроводниковой индустрии.

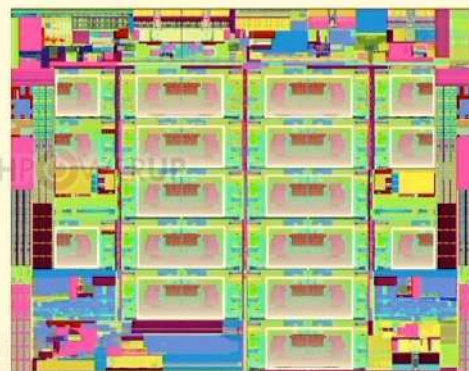
### 28-ядерный процессор Intel Xeon Scalable (Ice Lake-SP) — больше не секрет

Новость от 18 августа 2020 года: компания Intel посвятила вступительную презентацию на виртуальной конференции Hot Chips 32 рассказу о процессоре Xeon Scalable следующего поколения, известному под кодовым наименованием *Ice Lake-SP*. Этот процессор, рассчитанный на выпуск по 10-нанометровому техпроцессу, представляет собой первое использование ядра *Sunny Cove* в процессорах корпоративного сегмента. Переход на *Sunny Cove* обеспечивает прирост показателя IPC по сравнению с ядром *Skylake*, которое Intel использует в микроархитектуре процессоров Xeon с 2015 года. Хотя реализация *Sunny Cove* для Xeon мало отличается от реализации для мобильных процессоров *Ice Lake-U* 10-го поколения, она учитывает иерархию кэш-памяти и топологию кристаллов, принятую для процессоров Intel корпоративного сегмента.



## Ice Lake SP CPU: SoC (all but the cores)

- New infrastructure architecture
- SoC architecture
- Memory Hierarchy and I/Os
- Performance and Power enhancements



Die picture of a 28C Ice Lake-SP die

Рис.379 – Кристалл Ice Lake-SP (28 ядер)

В кристалле *Ice Lake-SP*, представленном на Hot Chips 32, насчитывается 28 ядер. Ядро сконфигурировано с той же кэш-памятью L1D размером 48 КБайт, что и его реализация для клиентского сегмента, но с гораздо большей выделенной кэш-памятью L2 размером 1280 КБайт. Ядро также получило второй объединенный модуль умножения/сложения (FMA-512), которого нет в клиентской реализации, и несколько новых наборов команд, эксклюзивных для корпоративного сегмента, включая AVX-512 VPMADD52, Vector-AES, Vector Carry-less Multiply, GFNI, SHA-NI, Vector POPCNT, Bit Shuffle и Vector BMI.

## Sunny Cove Core Microarchitecture

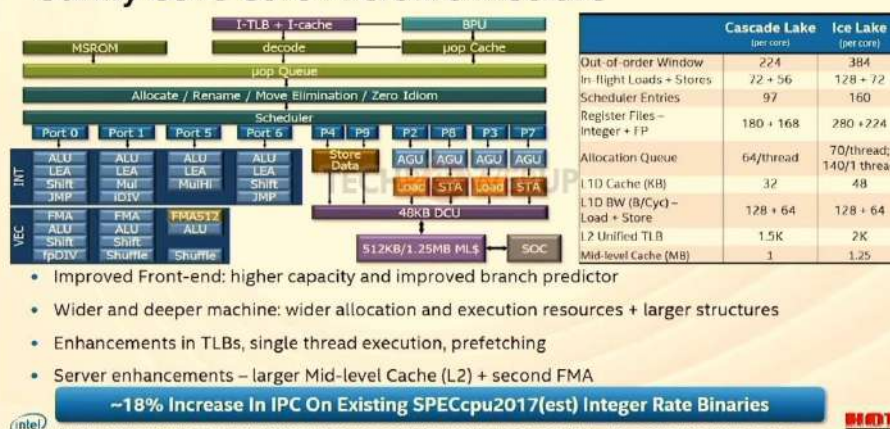
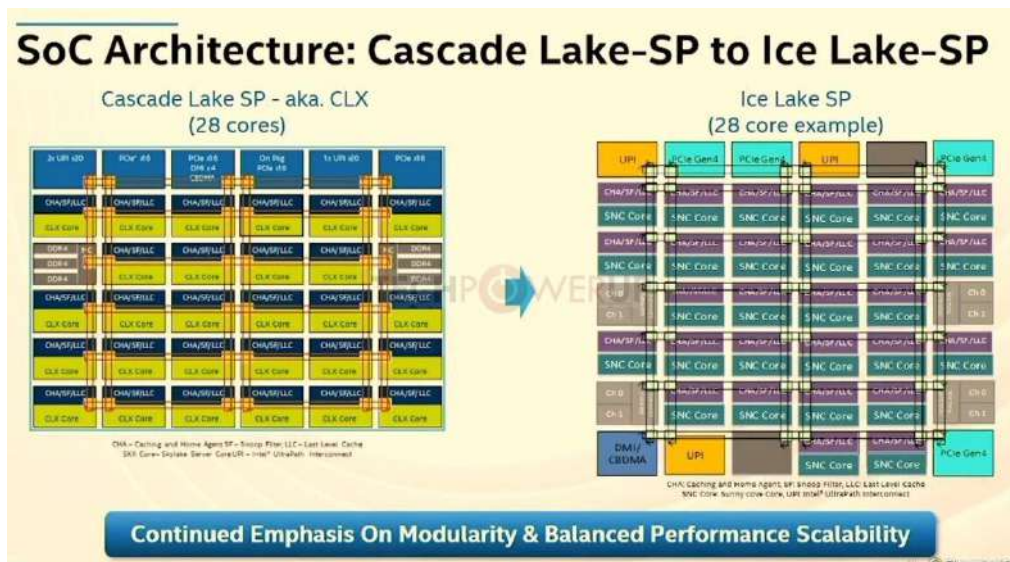


Рис.380 – Микроархитектура Sunny Cove Core

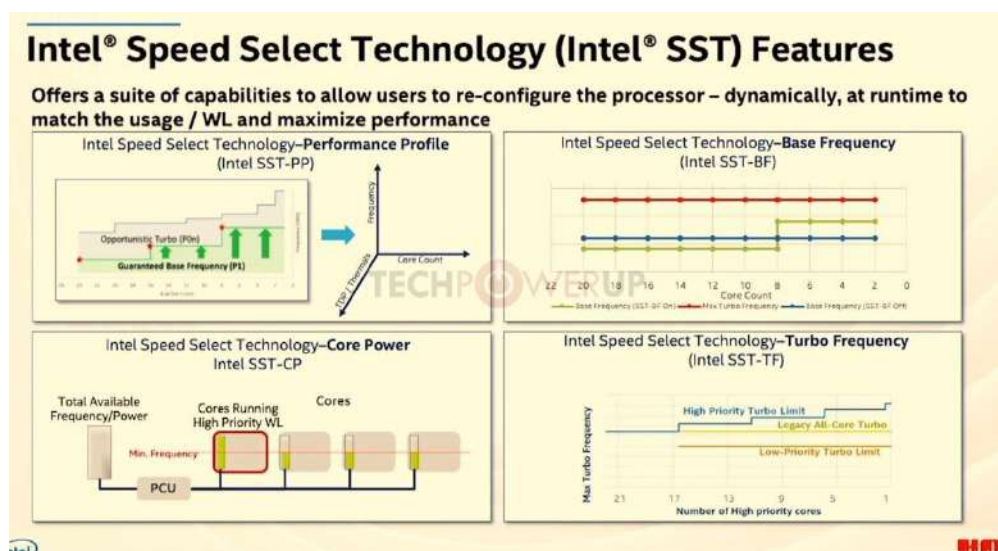
Помимо более высокого IPC ядер *Sunny Cove* плюсом *Ice Lake-SP* является улучшенный и модернизированный ввод/вывод. Так, 28-ядерный процессор имеет восьмиканальный интерфейс DDR4, построенный из четырех независимых двухканальных контроллеров памяти, тогда как кристалл *Cascade Lake-SP* располагает двумя трехканальными контроллерами. Каждый контроллер памяти *Ice Lake-SP* оснащен аппаратными средствами для полного шифрования памяти по алгоритму AES-XTS со 128-битным ключом.



**Рис.381 – Архитектура SoC: Cascade Lake-SP → Ice Lake-SP**

Процессор оснащен четырьмя независимыми элементами корневого комплекса PCIe Gen4. В нем реализована виртуализация ввода-вывода и другие возможности, востребованные в корпоративном сегменте. Кристалл также имеет три соединения UPI для связи между сокетами или кристаллами. Основным внутренним интерфейсом *Ice Lake-SP* является усовершенствованное соединение ячеистой сети второго поколения, которое характеризуется уменьшенными задержками.

Интересным новшеством является технология Intel Speed Select (Рис.382). Она позволяет администраторам отключать ядра процессора и тратить бюджет TDP на повышение частоты оставшихся ядер. Эта же функция используется для отключения ядер с целью снижения энергопотребления сервера.



**Рис.382 – Возможности технологии Intel Speed Select (SST)**

Ожидается, что Intel выпустит процессоры *Xeon Scalable Ice Lake-SP* в текущем полугодии (второе полугодие 2020г.).

Уточнение: процессоры *Xeon Scalable Ice Lake-SP* Intel формально представила в ноябре 2020, в продаже они появятся лишь в следующем году (2021г.). Ниже, на Рис.383, приведена представленная в ноябре 28-ядерная версия Ice Lake-SP, а буквально только что (декабрь 2020) появились слухи, что серверные CPU Ice Lake-SP могут содержать больше 32 ядер.



# Ice Lake-SP

## Shared distributed Last Level Cache (LLC)

## Integrating PCIe Gen4 controllers

- New IO virtualization design and improved P2P BW

## 3 independently clocked UPI links

## 4 memory controllers with enhanced per channel schedulers

## Intel Optane Persistent Memory 200 Series (Barlow Pass)

## Infrastructure and performance optimizations across the SoC

- Improved efficiency and performance scalability

## Strong security emphasis

- Intel® Total Memory Encryption (TME): DRAM encryption using AES-XTS 128bit
- Intel® Software Guard Extensions (SGX): application isolation w/ up to 1TB enclave

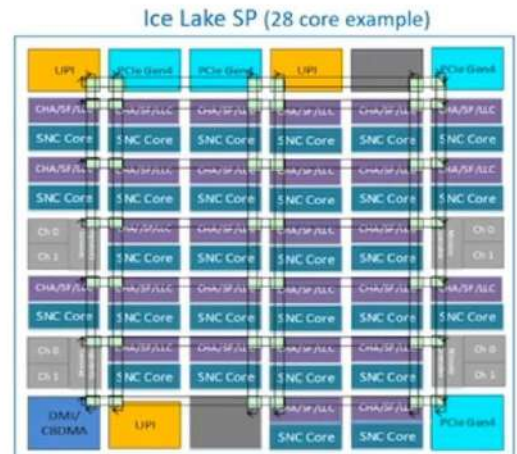


Рис.383 – Серверные Ice Lake-SP

## Финальные характеристики флагманского процессора Core i7-1185G7

Новость от 28 августа 2020 года: официальная премьера мобильных процессоров Intel *Ice Lake* состоится уже на следующей неделе, но уже сейчас можно оценить полные характеристики флагмана линейки Core i7-1185G7.

Core i7-1185G7 — это топовая четырехъядерная модель на архитектуре *Willow Cove* и с поддержкой многопоточности. Она имеет 5 МБайт кэш-памяти второго и 12 МБайт кэш-памяти третьего уровня. В *Ice Lake* впервые дебютируют встроенные GPU на архитектуре Xe. Конкретно в Core i7-1185G7 используется графический процессор DG1 на архитектуре Xe-LP (Low Performance) с 96 исполнительными блоками и 768 ядрами. Частота GPU составляет 1,55 ГГц.

| System Information    |                                               |
|-----------------------|-----------------------------------------------|
| System Information    |                                               |
| Operating System      | Microsoft Windows 10 Enterprise (64-bit)      |
| Model                 | Intel Corporation Tiger Lake Client Platform  |
| Motherboard           | Intel Corporation TigerLake U DDR4 SODIMM RVP |
| Processor Information |                                               |
| Name                  | 11th Gen Intel Core i7-1185G7                 |
| Topology              | 1 Processor, 4 Cores, 8 Threads               |
| Identifier            | GenuineIntel Family 6 Model 140 Stepping 1    |
| Base Frequency        | 2.99 GHz                                      |
| Maximum Frequency     | 4.78 GHz                                      |
| Package               |                                               |
| Codename              | Tiger Lake-U                                  |
| L1 Instruction Cache  | 32.0 KB x 4                                   |
| L1 Data Cache         | 48.0 KB x 4                                   |
| L2 Cache              | 1.25 MB x 4                                   |
| L3 Cache              | 12.0 MB x 1                                   |
| Memory Information    |                                               |
| Size                  | 16.77 GB                                      |
| Type                  |                                               |
| OpenCL Information    |                                               |
| Platform Vendor       | Intel(R) Corporation                          |
| Platform Name         | Intel(R) OpenCL HD Graphics                   |
| Device Vendor         | Intel(R) Corporation                          |
| Device Name           | Intel(R) Gen12 Desktop Graphics Controller    |
| Compute Units         | 96                                            |
| Maximum Frequency     | 1.55 GHz                                      |
| Device Memory         | 3.06 GB                                       |

Рис.384 - Таблица результатов тестирования процессора Core i7-1185G7

Базовая частота CPU составляет 3,0 ГГц, максимальная — 4.8 ГГц. Это, соответственно, на 7% и 2% больше, чем у «второго по значимости» процессора линейки — Core

i7-1165G7. Базовое значение TDP процессора — 15 Вт, но оно может расширяться до 28 Вт (PL1) при наличии адекватной системы охлаждения.

В начале сентября 2020 года Intel представила анонсированные ранее в августе мобильные процессоры *Tiger Lake-U*. Они сохранили 10-нанометровый техпроцесс, но он был заметно улучшен — в процессоре осталось максимум четыре ядра, но рабочие частоты существенно выросли. К тому же они основаны на новых архитектурах, как процессорной, так и графической. Это первые действительно крутые процессоры Intel за долгие годы, тесты *Tiger Lake* могут заставить AMD беспокоиться<sup>1</sup>.

В середине сентября Сети появились первые тесты, которые позволяют сложить представление о топовом CPU линейки — Core i7-1185G7<sup>2</sup>.

Необходимо отметить, что, как и многие другие современные мобильные процессоры, *Tiger Lake* в каждом конкретном ноутбуке могут обеспечивать существенно отличающуюся производительность из-за разных лимитов мощности. Ниже будут представлены результаты для Core i7-1185G7 с TDP 15 Вт, 28 и даже 34 Вт, и по производительности это разные CPU. Также стоит отметить, что тестирования проводились на эталонном ноутбуке, предоставленном самой Intel, так как серийных моделей с *Tiger Lake* на рынке ещё нет.

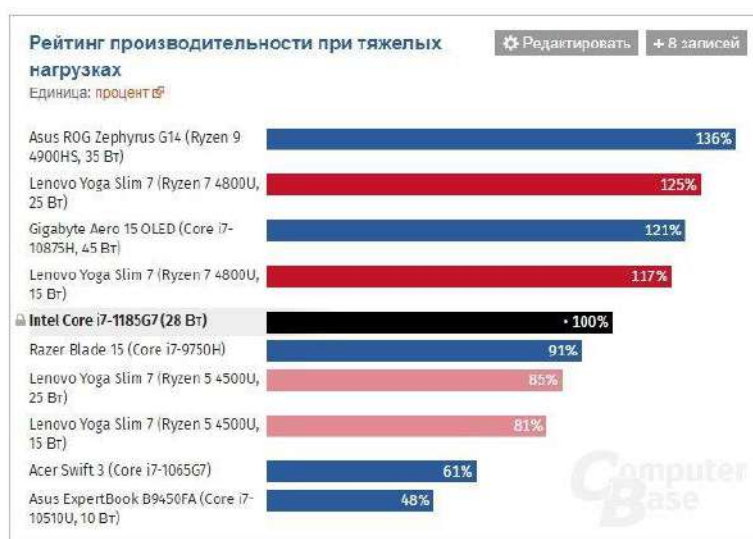


Рис.385 – Производительность при тяжёлых нагрузках.

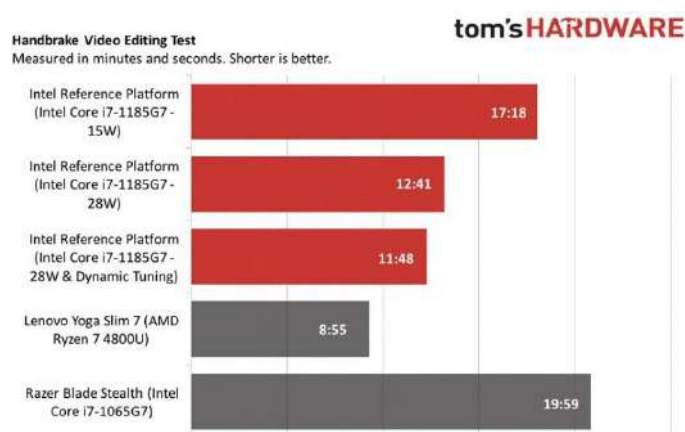


Рис.386 – Тест редактирования видео.

<sup>1</sup> Это первые действительно крутые процессоры Intel за долгие годы. Тесты *Tiger Lake* могут заставить AMD беспокоиться.

<sup>2</sup> Напомним — он содержит четыре ядра с частотами 1,2-4,8 ГГц и GPU Xe с 96 вычислительными блоками.

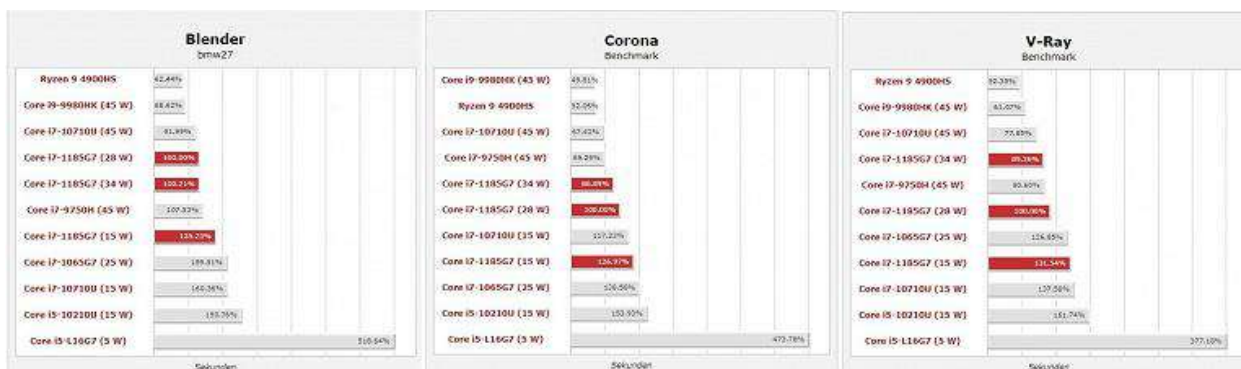


Рис.387 – Blender, Corona и V-Ray - тесты рендеринга на основе соответствующих приложений для рабочих станций. Как правило, они хорошо масштабируются в зависимости от числа доступных ядер.

Можно однозначно констатировать, что на этот раз у Intel получилось. *Tiger Lake* — это действительно именно такое новое поколение процессоров, которое и должно выходить на рынок. При своих четырёх физических ядрах Core i7-1185G7 на 15-20% отстаёт от восьмиядерного Ryzen 7 4800U, но на 15-20% опережает шестиядерный Ryzen 5 4500U. Правда, последний не поддерживает SMT, но потребителя это не особо волнует — важен результат. При этом Core i7-1185G7 умудряется обходить даже высокопроизводительный шестиядерный Core i7-9750H, но это скорее проблема последнего, чем достижение первого. Также можно отметить, что Core i7-1185G7 быстрее предшественника в лице Core i7-1065G7 на внушительные 65%!

Однако всё это — результаты для Core i7-1185G7 в режиме TDP 28 Вт. А вот если такой CPU ограничить 15 Вт, а это явно будет встречаться часто, производительность ощутимо падает. В таком режиме новый CPU уже лишь незначительно опережает Core i7-1065G7 (с TDP 25 Вт) и порой даже отстаёт и от него, и от 15-ваттного Core i7-10710U. Таким образом, покупая ноутбук с Core i7-1185G7 либо иным процессором Tiger Lake-U, нужно подходить к выбору модели внимательно, если есть желание получить максимум производительности. Иначе проще взять что-то с Ryzen.

Что же касается производительности GPU, в 28-ваттном режиме топовое ядро Xe показывает себя великолепно, обходя Vega 8 в Ryzen 7 4800U на 20-40% в зависимости от игры (хотя есть игры, где разницы почти нет). Но при снижении лимита мощности до 15 Вт преимущество Xe исчезает и Vega 8 немного вырывается вперёд. Впрочем, сравнение с тем же Core i7-1065G7 показывает двукратный прирост в 15-ваттном режиме, а в 28-ваттном режиме новинка быстрее уже в три раза! Учитывая, что Ice Lake — это предыдущее поколение мобильных процессоров Intel, которое имеет тот же техпроцесс, это феноменальный результат.

Также один из источников отмечает, что в режиме 28 Вт Core i7-1185G7 на самом деле потребляет до 36 Вт, тогда как восьмиядерный Ryzen 7 4800U — 30 Вт. Но в каждом конкретном ноутбуке лимиты мощности и возможности системы охлаждения всё равно будут разные, так что тут можно пожуричь новинку Intel лишь условно.

Учитывая столь впечатляющие результаты *Tiger Lake-U*, грядущие настольные процессоры *Rocket Lake* с той же архитектурой, но на 14-нанометровом техпроцессе, вполне могут составить конкуренцию решениям AMD, но, конечно, лишь при правильной ценовой политике.

## Intel Core 2, Nehalem, Sandy Bridge... — а что же AMD?

Битва AMD и Intel в сфере процессоров для ПК и серверов насчитывает не один десяток лет. В начале 2000-х накал борьбы между корпорациями достиг предела, когда на ринг вышли два тяжеловеса в лице Athlon XP и Pentium 4<sup>1</sup>.

<sup>1</sup> На основе материала из статьи [История противостояния AMD и Intel: от Athlon XP и Pentium 4 до Ryzen 2000 и Core 9-го поколения](#).

Athlon XP являлся улучшенной версией Athlon, построенном на прежней архитектуре K7. В названии процессора был применен так называемый Pг-рейтинг, задействованный в предыдущих CPU от трехбуквенной корпорации, например, в K5 PR100, только с приставкой “+”. Это давало пользователю понять, что же он на самом деле покупает, то есть, какому именно «Пентиуму» соответствует данный процессор.

Так Athlon 1700+ был по производительности равен Pentium 4, работающему на частоте 1700 МГц, хотя сам 1700-й Athlon функционировал на 1466 МГц. Удивительно, но когда-то равночастотные процессоры от AMD опережали изделия от Intel с их «кукурузными» мегагерцами. Кстати, при разгоне Athlon-ов, кроме частоты в BIOS, также изменялся и их рейтинг.

Неизвестно, сколько всего Intel потратила на рекламу Pentium 4, но понятно, что маркетинговый отдел компании не сидел без дела. Реклама «четвёртых пней» буквально заполонила медийное пространство, на ТВ и в журналах, везде мы видели этот сине-жёлтый логотип.

Благодаря обширной рекламной компании массовый потребитель фактически знал только про Pentium. Нередко можно было услышать «а какой у тебя пентиум?». Ведь в голову рядового пользователя, для которого компьютер сродни холодильнику или стиральной машине, и не придёт мысль о том, что существуют какие-либо другие процессоры.

Pentium-ы сами по себе стоили недёшево, а заветные 3 ГГц модели могли разорить даже самого ярого любителя новинок железа. Так как тогда все были помешаны на мегагерцах, то архитектура NetBurst казалась вполне себе прогрессивной, позволяющей наращивать производительность исключительно за счет прироста частоты, т.к. другие методы (архитектурные) были тогда не особо популярны и не очень хорошо известны.

А вот у AMD уже была готова революционно-новая архитектура под названием K8. Она принесла с собой 64-битные вычисления с адресацией огромных массивов памяти. Появление подобной архитектуры так или иначе должно было произойти, поскольку ограничение прежними процессорами поддержки в максимум 4 Гбайт оперативной памяти сильно сказывалось на дальнейшем развитии компьютерной индустрии.

Во множестве приложений и тестов, особенно в играх, Athlon 64 опережал Pentium 4. Еще одним звоночком для Intel стал выход Athlon 64 X2<sup>1</sup>, ознаменовавших начало первой «ядерной» войны. Об успехе AMD говорят и тогдашние высокие цены, установленные на двухъядерных монстров. В первый и в последний раз компания смогла противопоставить конкуренту действительно высокопроизводительный процессор высшего сегмента.

Intel понимала, что медлить нельзя и необходимо дать хоть какой-то отпор. В спешке была выпущена довольно неуклюжая «склейка» из двух Pentium 4, именовавшаяся Pentium D. Частоты нового процессора были существенно снижены, что не лучшим образом сказалось на производительности, вместе с тем базовое тепловыделение составляло внушительные для того времени 130 Вт.

Понятно, что ни о какой внятной конкуренции с Athlon 64 X2 речи и не шло. Доля AMD на рынке подобралась к невероятным 50%, казалось, ещё немного и Intel будет скинута с пьедестала.

Кардинальным переломом в ситуации стал выход процессоров на микроархитектуре Core 2<sup>2</sup>. Неэффективный NetBurst отправился на свалку истории, вместо него инженеры взяли Pentium-M и адаптировали его для настольных ПК. До этого существовал способ установки Pentium-M на обычные материнские платы через специальный переходник, и при этом в тестах он сильно обгонял Pentium 4.

Двухъядерные процессоры Core 2 сразу вышли в топ по производительности, обогнав Athlon 64 X2, в связи с чем AMD пришлось снижать цену на последние. Новинки очень

---

<sup>1</sup> Athlon 64 X2 компании AMD является первым двухъядерным процессором для настольных компьютеров. AMD официально начала поставки Athlon 64 X2 на выставке Computex 1 июня 2005 года.

<sup>2</sup> Многоядерная микроархитектура Intel Core была представлена компанией Intel в 1-ом квартале 2006 года.

хорошо разгонялись, и порой, купив недорогой процессор, можно было достичь частот самой быстрой модели, а то и выше. Все это обусловило тотальное превосходство Intel.

Но прогресс не стоял на месте, и на CES 2007 был представлен первый четырехъядерный CPU Core 2 Quad Q6600, обогнавший свое время. Да, это была «склейка» из двух Core 2 Duo, но зато быстрых и энергоэффективных. В многопоточных задачах, коих на тот момент было не так много, Q6600 демонстрировал феноменальные результаты. Это было будущее десктопных процессоров.

AMD терпела поражения по всем фронтам, но всё-таки смогла ответить выпуском первого настоящего четырехъядерного процессора без всяких склеек. Однако, его выход задерживался, и компании пришлось пойти на крайние меры, результатом которых стал выход платформы AMD Quad FX, в состав которой входили два двухъядерных Athlon 64 FX. И, наконец, после многочисленных задержек, на ринг таки вышел процессор Phenom первой генерации. К сожалению, его производительность не впечатляла, он больше походил на четырехъядерный Athlon 64. Лишь ценообразование позволяло AMD удерживаться на плаву.

Тем временем Intel уже вовсю штурмовала 45 нм техпроцесс. Новая микроархитектура Penryn выжала максимум из сокета LGA775. В целом всё стандартно – быстрее, выше, сильнее. Разогнанные экземпляры Penryn камня на камне не оставляли от аналогов от AMD.

Но случилось то, чего все так долго ждали – вышел Phenom II, который смог на равных противостоять последней итерации Core 2. Производительность значительно возросла, а главное появилась возможность разблокировать ядра процессора. Так, при помощи специальных утилит, без особой сложности и при определенном везении, из двухъядерного Phenom II можно было получить четырехъядерный.

Но практически в это же время Intel полностью меняет свой плацдарм, отправив в отставку Core 2 вместе с LGA775, и заменяя всё это на абсолютно новую архитектуру Nehalem<sup>1</sup>. Со времён Pentium-M происходили лишь *количественные* изменения, теперь настало время *качественных*. Внесено множество изменений и улучшений, сформировавших образ нынешних процессоров Intel.

Среди них: добавление встроенного на кристалл процессора контроллера памяти, кэша третьего уровня, возвращение технологии Hyper-Threading и объединение всех ядер в одну упаковку. Была определена «классовая» иерархия, в целом сохранившаяся до настоящего времени. Высокопроизводительные десктопные процессоры вынесены в отдельную платформу со своим сокетом, берущую многое у серверных собратьев. Для Intel началась золотая пора.

AMD пошла по схожему пути, с нуля разработав инновационный процессор FX, построенный на архитектуре Bulldozer<sup>2</sup>. Впервые обычному пользователю стали доступны сразу восемь ядер. Первые тесты мгновенно выявили несостоятельность подхода компании в проектировании FX-ов, заключавшегося в том, что несколько его ядер использовали некие общие ресурсы.

В 2015 году два покупателя процессоров AMD FX поколения Bulldozer обратились с коллективным иском к компании, обвиняя её в ложном информировании потребителей о характеристиках восьмиядерных моделей семейства. По словам истцов, коими выступили Тони Дикки (Tony Dickey) и Пол Пармер (Paul Parmer), в реальности архитектура процессоров Bulldozer не позволяла заявленным «восьми ядрам» действовать независимо, и по уровню быстродействия они уступали гипотетическим процессорам с реальными восемью ядрами. При этом истцы утверждали, что AMD в своих рекламных материалах делала упор

---

<sup>1</sup> Микроархитектура Nehalem была представлена в 4-ом квартале 2008 года,

<sup>2</sup> **Bulldozer** — кодовое название процессорной архитектуры процессоров AMD64 (в рамках системы команд x86) от компании AMD, семейства 15h, изготавливаемых по 32-нм технологии и предназначенных для серверов и высокопроизводительных ПК. Серийный выпуск процессоров архитектуры Bulldozer начался в сентябре 2011 года.

на способность её восьмиядерных процессоров превосходить по быстродействию модели конкурента с меньшим количеством ядер.

Окончательный гвоздь в крышку гроба AMD забила микроархитектура Sandy Bridge<sup>1</sup> в лице легендарных Core i5-2500K и Core i7-2600K, сделавшими ненужными даже недавно вышедшие топовые шестиядерные процессоры на платформе LGA1366. Высокая однопоточная производительность и высокие разгонные частоты сделали свое дело, в игровых приложениях ни один процессор от AMD не мог и близко дотянуться до указанной парочки. Слегка улучшенная архитектура Vishera не спасла положение AMD.

Пока конкурент пребывал в нокдауне, Intel расслабилась, не предлагая в новых процессорах ничего *кардинально* нового. Даже наоборот, если в Sandy Bridge под крышкой CPU использовался металлический припой, то в Core 3000-й серии он был заменен на менее эффективную термопасту. Это позволило компании убить сразу двух зайцев: во-первых, снизить издержки, а во-вторых, намеренно ограничить срок службы чипов, так как высохшая после истечения гарантии термопаста приводила к увеличению температуры и повышенному износу процессора.

Год спустя на сцену вышел Haswell<sup>2</sup>, повлекший за собой смену сокета. Многие недолюбливают Intel за то, что она слишком часто заставляет владельцев менять материнскую плату при апгрейде. Платформы AMD в этом свете выглядят долгожителями, что безусловно высоко ценится владельцами продуктов компании.

Мэйнстримовые CPU на LGA1150 не предложили ничего кардинально нового, а вот в HEDT сегменте наметились значительные изменения. Платформа LGA2011-3 принесла поддержку памяти стандарта DDR4 и 8-ядерного 16-поточного Core i7-5960X. Скорость вычислений относительно Core i7-4790K фактически удвоилась, был задан новый стандарт производительности процессоров для энтузиастов. Казалось, что AMD никогда не угнаться за Intel.

Спустя два года грянула очередная смена сокета вместе с обновлением архитектуры. Что же получили пользователи? Те же четыре ядра, что и восемь лет назад. Единственным утешением служило то, что кулеры менять не приходилось, так как большинство моделей, подходящих для LGA1155, вставляли и на LGA1150, и на LGA1151.

Да, с каждым поколением производительность интеловских процессоров увеличивалась, но незначительно. Так разрыв между разогнанными Core i7-2600K и Core i7-7700K равен приблизительно 30% в пользу последнего. И это за шесть лет. Кстати сказать, Core i5-2500 быстрее разогнанного Core 2 Quad Q6600 как минимум в два раза, а ведь временной промежуток между их выпусками составлял всего четыре года.

Монополия Intel рухнула 22 февраля 2017 года с выходом долгожданных процессоров Ryzen первого поколения. Гениальная по своей простоте компоновка кристаллов на архитектуре Zen привела к созданию недорогих и достаточно быстрых решений, занявших места во всех сегментах рынка, от начального до экстремального.

Процессоры Ryzen для сокета AM4 стали по-настоящему народными. Ещё бы, пользователи получили фактически аналоги HEDT процессоров Intel, наподобие 5820K и 5960X, только за втрое меньшую цену. Кроме того, Ryzen не требовались дорогие материнские платы, а разгон был доступен не только на старших чипсетах. Ryzen Threadripper, позиционировавшиеся в качестве решений для рабочих станций, также нашли своего покупателя, во многом благодаря хорошему соотношению цена-производительность. В дальнейшем цены на первое поколение только падали, что делало их еще более выгодным приобретением на фоне невнятной ценовой политики Intel.

Очередная «ядерная» война вновь была развязана благодаря AMD. Именно так, ведь Intel всё же пришлось снять розовые очки и вступить в жёсткую конкурентную борьбу, от которой выигрывают потребители. И результат не заставил себя долго ждать, теперь уже

---

<sup>1</sup> Микроархитектура **Sandy Bridge** была анонсирована 3 января 2011 года.

<sup>2</sup> Релиз первых процессоров на архитектуре **Haswell** состоялся 1-4 июня 2013 года на COMPUTEX 2013.



Intel начинают отражать атаку AMD. Со стороны Intel последовало кардинальное обновление модельного ряда с изменением как качественных, так и количественных характеристик, конкретнее говоря, было увеличено количество ядер и потоков при сохранении той же отпускной цены.

Но AMD и не думала сбавлять темп. В начале 2018 года в ход пошли Ryzen второго поколения. Ничего сверхсущественного они не привнесли, но обострили конкуренцию с Core восьмой серии. К сожалению, для компании Intel, в это время случился дефицит её процессоров, и цены соответствующим образом взлетели. На этом фоне рыночная доля AMD начала расти стремительными темпами.

Желая спасти свое положение, Intel в срочном порядке выпустила Core 9-го поколения. Надо признать, что в игровых приложениях новинки проявили себя в лучшем свете, так как непревзойдённая однопоточная производительность всё же достаточно сильно на это влияла. Ryzen 2000-серии заметно отставали в неоптимизированных игровых приложениях, но при задействовании высоких разрешений разница значительно нивелировалась. При многопоточной нагрузке Ryzen из той же ценовой категории значительно обходили процессоры Core.

Впереди нас ждет конференция Computex 2019<sup>1</sup>, на которой в планах у AMD анонс Ryzen третьего поколения, построенных на архитектуре Zen 2. Некоторые производители материнских плат уже предоставили новые версии BIOS для их поддержки чипсетами 300-й и 400-й серий. Если слухи насчет характеристик новых процессоров подтвердятся, то уже в начальном сегменте мы получим то, что оппонент (Intel) сейчас предлагает в высоком.

Исходя из последнего роадама, Intel не планирует избавляться от архитектуры Core, а будет лишь вносить в нее улучшения. Таким образом, если AMD и дальше продолжит выпускать новые чипы, то у нее будут все шансы обойти Intel также и в играх, а не только в профессиональных задачах.

## Серверные процессоры AMD EPYC<sup>2</sup>

Выше речь шла только о новых процессорах AMD для десктопов и ноутбуках. Но не менее интересные изменения произошли в эти годы на другом рынке – рынке серверных платформ. AMD сумела представить решения – процессоры EPYC – которые превзошли топовые решения от Intel – и по количеству ядер, и по технологическим проектным нормам, и по общей производительности CPU. Причём превзошли значительно, иногда в 1.5-2 раза.

На сегодня на более консервативном серверном рынке x86-процессоров компания Intel практически полностью доминирует, имея долю свыше 93-95%, у AMD на этом рынке пока очень небольшой процент и, даже имея превосходные EPYC, которые были с энтузиазмом встречены крупнейшими потребителями продукции AMD, типа Google, Oracle, Dell, Microsoft, Hewlett Packard и др., компания надеется расширить своё присутствие в серверных платформах до 15-20% только примерно к 2023/24 году.

В прошлый раз AMD завоёвывала рынок серверных процессоров 64-битными процессорами Opteron в уже очень далеком 2004 году. С тех времен доля AMD на этом рынке скатывалась практически до нуля, но уже первое поколение процессоров Epyc на основе микроархитектуры Zen 1, позволило им набрать некоторое количество клиентов, хотя до Intel оставалось ещё очень далеко. Анонс первого поколения процессоров Epyc в июле 2017 года начал новую страницу компании на этом рынке. Уже первые решения линейки предлагали большее количество вычислительных ядер, повышенную пропускную способность памяти и больше возможностей для подключения периферии, по сравнению с тогдашними конкурентами от Intel.

---

<sup>1</sup> Статья была написана 25 апреля 2019 года.

<sup>2</sup> Этот раздел написан на основе статьи [Обзор линейки серверных процессоров AMD Epyc 7002](#)



Но многие индустриальные игроки ждали чего-то ещё более конкурентоспособного, и наконец-то дождались — второе поколение Ерус решило многие проблемы первого, перешло на самый совершенный техпроцесс, благодаря хитроумной компоновке обеспечив максимальное количество ядер (для x86-совместимых решений), а также предложило отличные возможности по поддержке оперативной памяти и внешних устройств, подключаемых по шине PCI Express. Второе поколение Ерус, известное под кодовым именем «Rome», предлагает ещё больше производительности вместе с некоторыми новыми возможностями.

Новые серверные процессоры были неплохо приняты индустрией при всей её консервативности и инерционности, было выпущено большое количество аппаратных решений с применением Ерус, они были поддержаны программно, в том числе и самыми популярными облачными платформами: Microsoft Azure, Amazon Web Services, Tencent Cloud, Baidu, Oracle Cloud и другими. Но серверные решения — это не самая стремительно меняющаяся индустрия, и для того, чтобы ещё больше усилить продвижение Ерус в массы, было необходимо ещё улучшать возможности этих процессоров. Чем AMD и занималась прошедшие два года, работая над вторым поколением семейства серверных процессоров Ерус.

Новые серверные процессоры AMD обеспечивают высочайшую производительность в широком спектре задач, имея до 64 ядер на процессор. Ерус 7002 обеспечивают до двух раз большую производительность по сравнению с предыдущим поколением серверных процессоров компании, и на 25%-50% меньшую совокупную стоимость владения, по сравнению с конкурирующими продуктами.

Наиболее впечатляющим стал прирост количества ядер и многопоточной производительности — новинки оказались более чем вдвое быстрее первого поколения Ерус, что важно в том числе и потому, что с их помощью можно использовать одноsocketные серверы там, где раньше использовались два процессора. И всё это великолепие — в том же самом сокете и с небольшим ростом энергопотребления и тепловыделения. Новые CPU можно устанавливать в платформы первого поколения, хотя для поддержки части функциональности потребуется обновить BIOS системной платы, предназначенной для установки Ерус 7001. Но так как для серверных процессоров такой апгрейд не слишком распространен, то чаще всего будут приобретаться платформы второго поколения, раскрывающие все возможности Ерус 7002, такие как поддержка PCIe 4.0 с вдвое большей пропускной способностью, полезной для скоростных Ethernet-адаптеров и SSD-накопителей, например.

### ***Техпроцесс и микроархитектурные улучшения***

Новые процессоры Ерус 7002 стали первыми по многим показателям. В том числе, это первые 64-ядерные x86-совместимые процессоры, первые x86-совместимые, созданные с применением 7 нм техпроцесса, первые процессоры с поддержкой шины PCI Express 4.0, первые процессоры с поддержкой памяти стандарта DDR4-3200 и т. д. и т. п.

В свое время компания AMD сделала серьезную ставку на максимум инноваций: обязательный переход на 7 нм техпроцесс, многочисленные улучшения архитектуры, устраняющие основные недостатки и применение совершенно новых компоновочных решений. Все эти пункты сработали отлично, один только самый современный техпроцесс позволил получить вдвое большую плотность размещения транзисторов и вдвое меньшее потребление энергии при той же производительности, а заодно и прирост частоты примерно на четверть.

Следует отметить, что своего полупроводникового производства у AMD нет, в своё время компания продала все свои фабрики, сосредоточившись на разработке дизайна процессоров. Так же относительно недавно поступил ещё один крупный производитель процессоров — IBM, продавшая свои фабрики по производству микросхем, и с тех пор размещающая заказы на производство собственных процессоров Power у сторонних производителей.



Рис.388 – Производительность на единицу мощности в зависимости от проектных норм.

Инвестиции в разработку 7 нм решений для AMD оправдались с лихвой, что особенно хорошо заметно на фоне проблем главного конкурента с освоением примерно аналогичного по возможностям техпроцесса. Даже несмотря на то, что у TSMC<sup>1</sup> и Intel очень разные «нанометры», и картинка выше (Рис.388) слегка преувеличивает превосходство 7 нм над 10 нм, ранее преимущество всегда было за имеющей собственное производство компанией Intel, но теперь, за счет собственных инвестиций и совместной работы с тайваньской компанией TSMC, а также с учетом проблем конкурента с их полупроводниковым производством, AMD не просто сравнялась с соперником, но и вышла вперед — такого ранее ещё не бывало!

Почему применяемый техпроцесс вообще столь важен? Да хотя бы уже потому, что он позволяет обеспечить более низкую себестоимость, а вместе с ней и снижение цены на продукцию. По мнению промышленных аналитиков, современные 7 нм процессоры Ерус с многокристальной чиплетной компоновкой достигают уровня выхода годных кристаллов около 90%, в то время как Intel довольствуется более чем вдвое меньшей долей годной продукции. С учетом разницы в техпроцессах (14 нм у Intel и 7 нм у AMD на TSMC), каждый процессор первой обходится раза в полтора дороже даже при том, что второй приходится платить сторонним производителям: TSMC и GlobalFoundries<sup>2</sup>. Эти примерные прикидки недвусмысленно говорят о том, что ставка AMD оправдалась.

Однако новой технологией производства дело не ограничилось, в AMD решили исправить одну из явных проблем первого поколения архитектуры Zen — относительно низкое количество исполняемых инструкций за такт (IPC). Во многом именно за счёт этого конкурент имел преимущество над решениями AMD в некоторых задачах из разных сфер применения. И в Zen 2 инженеры смогли добиться прироста в скорости вычислений при той же частоте на 15%, а если говорить о приросте в многопоточных вычислениях, то в типичных серверных задачах новый Ерус быстрее старого при прочих равных уже на 23%, и это — без учета удвоенного количества вычислительных ядер и бóльшей рабочей частоты!

В Zen 2 сделали массу микроархитектурных улучшений, по сравнению с Zen 1. Если вкратце, то для увеличения производительности в новой микроархитектуре улучшили предсказание переходов (появился новый предсказатель переходов TAGE), слегка увеличили

<sup>1</sup> **TSMC** (аббревиатура *Taiwan Semiconductor Manufacturing Company*) — тайваньская компания, занимающаяся изучением и производством полупроводниковых изделий. Основана в 1987 году правительством Китайской республики и частными инвесторами. Согласно информации TrendForce на конец 2017 года, TSMC — крупнейший контрактный производитель полупроводниковых микросхем с долей рынка 55,9 %. На втором месте находится GlobalFoundries (9,4 %), на третьем — United Microelectronics Corporation (8,5 %).

<sup>2</sup> **GlobalFoundries (GF)** — американский производитель полупроводниковых изделий, штаб-квартира г.Санта Клара, Калифорния. Была образована в результате продажи производственного подразделения Advanced Micro Devices (AMD). Эмират Абу-Даби является владельцем компании через свою дочернюю компанию *Advanced Technology Investment Company (ATIC)*.

целочисленную производительность, увеличив буферы и улучшив планировщики, оптимизировали работу кэш-памяти первого уровня, практически удвоив её пропускную способность, удвоили ёмкость L3-кэша и т. д. Кроме этого, в Zen 2 были добавлены некоторые новые инструкции.

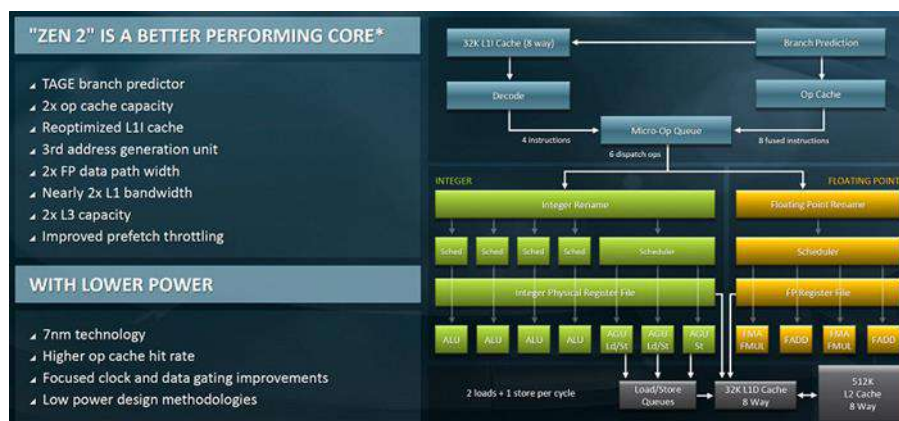


Рис.389 – Улучшения, внесённые в микроархитектуру Zen 2.

Но всё же самое важное изменение в Zen 2 — увеличение ширины блока операций с плавающей запятой с 128 до 256 бит. Благодаря этому улучшению, все процессоры архитектуры Zen 2 исполняют 256-битные AVX2-инструкции вдвое быстрее, по сравнению с первым поколением. То есть, в Zen 2 появилась поддержка выполнения двух AVX-256 инструкций за такт, что позволило AMD заявить о двукратном росте FP-производительности. Причем, в отличие от решений Intel, второе поколение Ерус не снижает частоты при выполнении AVX2 слишком сильно, а просто действует в рамках ограничений по энергопотреблению, установленных платформой.

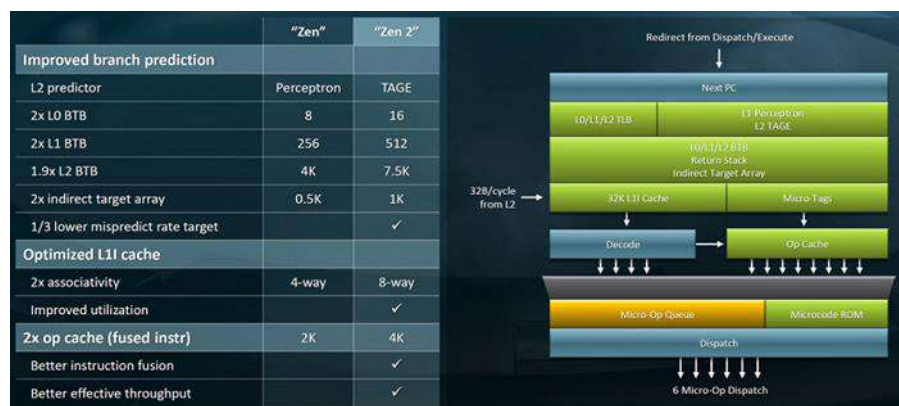


Рис.390 – Сравнение характеристик Zen и Zen 2.

Отметим также удвоенный объем кэш-памяти для декодированных микроопераций, что может снизить простои исполнительных блоков конвейера, а также улучшенное предсказание переходов при помощи нового TAGE-предсказателя и увеличенный объем буферов ветвлений первого и второго уровней. Эти изменения призваны снизить вероятность появления ошибок предсказания и увеличивают эффективность прогнозирования ветвления кода, повышая общую производительность.

В новых вычислительных ядрах появился третий блок генерации адресов (AGU), улучшающий доступ исполнительным устройствам к данным. Ширина шины кэш-памяти была увеличена вдвое, а объём кэш-памяти третьего уровня удвоен — его объём достиг 32 МБ на каждый чиплет. Это помогает ускорить обращение исполнительных устройств к данным. Были увеличены размеры очередей планировщиков и размер регистрового файла, повышающие эффективность исполнения многопоточного кода.

Дополнительное преимущество второе поколение Ерус получило при оптимизации энергоэффективности в виде улучшенного управления питанием, позволяющим получить максимально возможную турбо-частоту при разном количестве активных вычислительных ядер. То есть, как и в настольных Ryzen, даже фабричные частоты выжимают из CPU практически всю возможную производительность. Если говорить о конкретных цифрах, то при восьми активных ядрах тактовая частота топовой модели Ерус 7742 составляет 3.4 ГГц, при 16 падает до 3.33 ГГц, и плавно снижается до 3.2 ГГц для всех 64-х ядер.

Отметим, что средняя однопоточная производительность Ерус 7002 в широком круге задач выросла даже несколько больше, чем на 15%, о которых заявила AMD, судя по некоторым тестам. И очень похоже, что с такими характеристиками и возможностями, решения AMD будут успешно сражаться не только на рынке настольных ПК, но и на рынке высокопроизводительных вычислений, где до сих пор царил Intel Xeon.

### ***Чиплетная компоновка***

Но всё же самое главное, чем бьёт конкурента новый серверный процессор AMD — инновационное компоновочное решение с использованием так называемых чиплетов — отдельных кристаллов, связанных быстрой шиной. Уже в первом поколении Ерус использовался не единый кристалл, а четыре отдельных, включающих в себя вычислительные ядра, контроллеры памяти и системы ввода-вывода, и все они были объединены быстрой шиной. Такой подход позволил обойти ограничения на размер единого кристалла и снизить себестоимость производства многоядерных CPU, ведь выход годных у маленьких кристаллов выше. Заодно повысилась масштабируемость, так как количество отдельных кристаллов, содержащих по несколько ядер, можно было варьировать в более широких пределах.

Но во втором поколении Ерус инженеры компании пошли еще дальше, применив второе поколение архитектуры AMD Infinity, оптимизированной для многоядерных вычислений. В первом поколении Ерус одним из спорных моментов была повышенная сложность решения: 32-ядерные процессоры содержали по четыре кристалла с 8 ядрами, каждый из которых имел по два канала памяти, а в двухпроцессорной конфигурации дела были еще хуже, ведь она приводила к сложностям при доступе к памяти из ядер в разных процессорах. Из-за этих проблем большое количество приложений показывало недостаточно высокую производительность даже при сравнительно большом количестве CPU ядер.

Во втором поколении Ерус проблему решили при помощи центрального чиплета ввода-вывода, в котором содержатся все необходимые для работы контроллеры. Полная версия чипа состоит из восьми чиплетов Core Complex Die (CCD) и одного ядра ввода-вывода I/O Die (IOD). Все CCD соединены с центральным хабом при помощи высокоскоростных каналов Infinity Fabric (IF), и при их помощи получают данные из памяти и внешних PCIe-устройств, а также от соседних вычислительных ядер. Т.е., таким решением AMD проделала обратную процедуру, по сравнению с той, что когда-то была выполнена примерно полтора десятилетия назад — контроллер памяти был перемещён обратно с кристалла процессора на внешний чип (когда-то это был северный мост).

Каждый из CCD-чиплетов содержит по паре четырехъядерных блоков Core Complex (CCX), которые также включают и 16 МБ L3-кэша. Получается, что топовый 64-ядерный Ерус состоит из 8 чиплетов CCD и 16 блоков CCX, которые обмениваются данными друг с другом при помощи центрального IOD-чиплета.

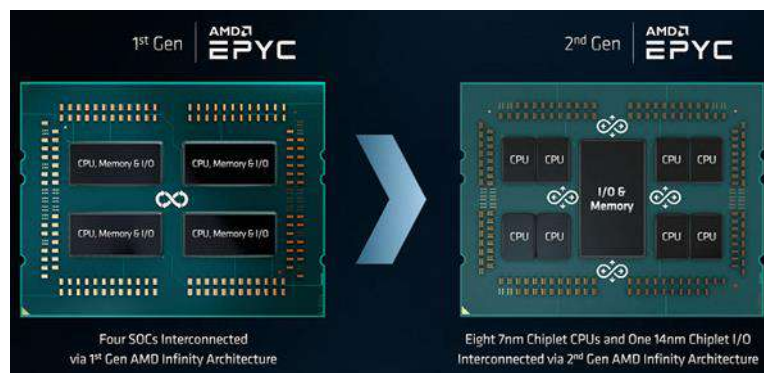


Рис.391 – Изменения в компоновке в поколении Zen 2.

При этом разные чиплеты используют оптимальный техпроцесс для их производства: CPU-чиплеты производятся на фабриках TSMC с использованием техпроцесса 7 нм, а чиплет ввода-вывода — на GlobalFoundries с применением технологии 14 нм. Кристалл с вычислительными ядрами и кэш-памятью использует самый совершенный техпроцесс для того, чтобы снизить размер кристалла, максимизировать производительность при минимальном энергопотреблении, а чиплет с контроллерами памяти и PCIe не нуждается в столь радикальных мерах и вполне обходится обкатанным и проверенным более «толстым» техпроцессом. AMD называет такую упаковку гибридной мультиядерной системой-на-чипе (SoC).

Это полезно в том числе и потому, что схемы ввода-вывода сложнее производить на более тонких техпроцессах, и их перенос на давно и хорошо отлаженную технологию производства упрощает и удешевляет производство, ускоряя вывод решений на рынок. В результате такого подхода AMD получила значительно преимущество, производя сравнительно небольшие CCD-кристаллы по нормам 7 нм с хорошим уровнем выхода годных.



Рис.392 – Иллюстрация разного подхода в компоновке традиционной монолитной архитектуры, 1-го и 2-го поколений EPYC.

Такой подход позволяет улучшить задержки при передаче данных, обеспечив гибкую и унифицированную архитектуру доступа к памяти. По сравнению с первым поколением ещё более гибким стало масштабирование по количеству вычислительных ядер, ушла необходимость наличия подсистем ввода-вывода и контроллеров памяти в каждом из кристаллов, а главное — единый центральный чиплет ввода-вывода улучшил показатели неравномерного доступа к памяти (NUMA) при межкристальном взаимодействии.

Во втором поколении серверных процессоров Ерус было снижено количество типов узлов удалённой памяти NUMA. Если в первом поколении каждое ядро имело три возможных времени доступа к памяти, физически присоединенной к разным кристаллам процессора (к контроллерам памяти рассматриваемого кристалла, контроллеров в соседних кристаллах и контроллеров во втором чипе), то во втором поколении Ерус варианта лишь два: контроллеры памяти в текущем чиплете ввода-вывода и в соседнем. Но самое «короткое» время в Ерус 2 стало больше, чем было в первом поколении, т.к. память была «отодвинута» дальше от ядра процессора.



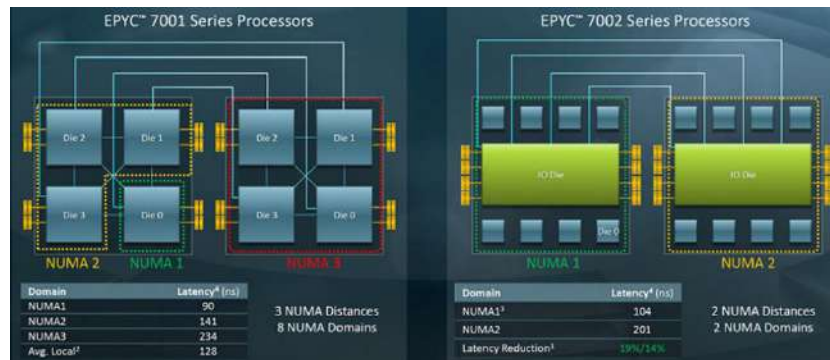


Рис.393 – Изменения в топологии NUMA во втором поколении EPYC.

Соответственно, время доступа в первом поколении Ерус может быть 90, 141 или 234 нс, а во втором — или 104 или 201 нс. И в среднем, задержки доступа к памяти при двухчиповой схеме были снижены на 14%-19%. Это улучшение очень важно, так как производительность в большинстве современных задач очень сильно зависит от работы подсистемы памяти, включая эффективность кэширования данных.

Чиплетная компоновка сработала отлично, этот шаг был фактически обязателен для дальнейшего увеличения количества ядер, и иная схема была бы куда менее выгодной. Конечно, монолитный кристалл обеспечил бы куда меньшие задержки как доступа к памяти, так и между вычислительными ядрами, но тогда вряд ли удалось бы увеличить количество ядер до 64 штук — для примера можно посмотреть на решения конкурента.

Есть один неприятный момент в схеме, принятой AMD. Если требуется доступ к данным в кэше, который принадлежит не к тому же ССХ, но находящемуся в том же кристалле CCD, то он будет таким же медленным (относительно), как и доступ к данным кэша вообще из другого кристалла. В таком случае, данные всегда будут проходить по шине IF в чиплет ввода-вывода и обратно — уже к нужному ядру.

Это не так страшно в реальности, так как каждое вычислительное ядро в составе ССХ имеет по 4 МБ L3-кэша, что заметно больше, чем у конкурирующих процессоров Intel, и блоки предвыборки данных имеют куда больший объем для того, чтобы загрузить все необходимые данные. Хотя некоторые задачи, вроде приложений баз данных, могут пострадать, а относительно медленный обмен данными с центральным чиплетом снижает скорость синхронизации. И в некоторых тестах 28-ядерный Intel Xeon 8280 именно поэтому оказывался быстрее 32-ядерного Ерус 7601 из предыдущего поколения.

Возможно, есть и другие подобные задачи, но в большинстве случаев 16 МБайт L3-кэша для каждого из четырех ядер в составе ССХ должно быть вполне достаточно. Большой объем L3-кэша в Ерус 7742 дает значительно меньшую задержку доступа при объеме данных между 4 и 16 МБайт, по сравнению с аналогичным Ерус из предыдущего поколения, а также L3-кэш новых Ерус весьма быстр, по сравнению с решениями конкурента в Intel Xeon Platinum 8280, что подтверждают и синтетические тесты.

Сама по себе шина Infinity Fabric в Ерус второго поколения была ускорена, её ширина выросла вдвое — с 256 до 512 бит. И задержки при пересылке данных между ядрами действительно заметно улучшились. Разные ядра процессора обмениваются данными на 25% – 33% быстрее, и скорость обмена между ядрами в пределах одного блока ССХ получается даже лучше, чем у конкурента с кольцевой шиной. Ускорение Infinity Fabric проявляется не только при пересылке данных между ядрами. Каждый блок ССХ имеет собственную кэш-память третьего уровня в 16 МБайт, и обращения через Infinity Fabric происходят, когда ядра одного блока ССХ нуждаются в данных, находящихся в L3-кэше соседнего блока, не говоря уже о других чиплетах. Так что ускорение Infinity Fabric положительно сказалось на производительности в широком круге задач при активном доступе к данным.

Сама подсистема кэш-памяти в новых процессорах мало изменилась, кэш-память первого и второго уровней сохранила свой объем и организацию, а вот кэш третьего уровня

был удвоен (16 МБайт на каждые четыре ядра) за счет перехода на 7 нм техпроцесс, который позволил увеличить транзисторный бюджет для чиплетов. Увеличение объема L3-кэша стало в том числе причиной того, что в новых процессорах (и Ерус и Ryzen) контроллеры памяти теперь расположены не рядом с вычислительными ядрами, а в отдельном чиплете ввода-вывода. Кэширование большого объема данных нужно для того, чтобы снизить задержки, когда вычислительные ядра простаивают в ожидании получения данных из памяти.

Рост объема кэш-памяти традиционно сопровождается некоторым увеличением её задержек, но рост латентности L3-кэша в случае перехода от Zen 1 к Zen 2 оказался довольно небольшим. Задержки L1- и L2-кэша остались на прежнем уровне из-за отсутствия особых изменений. А вот L1-кэш стал быстрее, так как теперь он способен обслуживать два 256-битных чтения и одну 256-битную запись за такт, что вдвое больше, чем у Ерус первого поколения. И если скорость работы L1- и L2-кэша в новых процессорах архитектуры Zen 2 сопоставимы с параметрами кэш-памяти процессоров конкурента, то L3-кэш обеспечивает даже меньшие задержки по сравнению с кэшем процессоров Intel. Впрочем, не всё так просто, и алгоритмы работы L3-кэша в процессорах разных производителей отличаются, как и их практическая эффективность.

А вот показатели задержек доступа к памяти во всех Zen 2 дают некоторый повод для беспокойства — по этим параметрам новинки оказываются даже несколько хуже предшественников, проигрывая по латентности памяти процессорам конкурента. Все дело в той самой чиплетной компоновке, которая разделила вычислительные ядра и контроллеры памяти. Чиплеты с вычислительными ядрами и L3-кэшем отделены от вынесенных в чиплет ввода-вывода контроллера памяти, контроллера шины PCI Express и других элементов. Между памятью и всеми процессорными ядрами появилось еще одно звено в виде шины Infinity Fabric. И хотя AMD утверждает, что она аналогична по характеристикам шине, связывающей пару CCX блоков внутри чиплета, вряд ли это не сказывается на задержках, возникающих при доступе к данным.

Но насколько хуже стала работа с памятью в новых серверных процессорах AMD? Увеличение задержек во всех процессорах Zen 2 по сравнению с процессорами прошлого поколения достигает 10%, да и реальная пропускная способность при записи в память несколько снизилась. Отделение контроллера памяти от вычислительных ядер не могло привести к другому результату, ведь именно для ускорения доступа к ней лет 15 назад перенесли контроллер памяти из чипсета в CPU. В итоге, ПСП при чтении у новых Ерус действительно достаточно высока, а вот по скорости записи они уступают конкурентам от Intel. Это тем более неприятно, так как по скорости работы с памятью первые Ерус и так проигрывали решениям конкурента, а теперь ситуация в некоторых задачах может даже усугубиться.

Но все же новая организация доступа к памяти — это правильное решение. Ведь основное преимущество второго поколения Ерус перед первым состоит в том, что под него гораздо проще оптимизировать программное обеспечение. Каждый процессор (в двухпроцессорной конфигурации) имеет лишь два возможных значения задержки доступа к памяти, так как каждое ядро имеет одинаковый «короткий» путь ко всем каналам «своей» памяти и одинаковый «длинный» путь к памяти соседнего процессора. А в первом поколении Ерус было две области NUMA на каждый CPU, так как память в них присоединена к разным кристаллам. Так что в двухпроцессорной системе Ерус 7002 будет работать в традиционной NUMA-конфигурации, которую много лет знают программисты. И хотя в некоторых случаях доступ к памяти у Ерус 7001 и получается быстрее, топология первого поколения излишне сложна, и во многих других случаях задержки памяти возрастают, что сложно предсказать и оптимизировать в ПО. Конфигурация памяти Ерус 7002 с точки зрения ПО выглядит гораздо проще, что сократит время, требующееся для его оптимизации.

Основными задачами при разработке микроархитектуры Zen 2 было увеличение пропускной способности внутричиповых соединений, улучшенные возможности по присоединению внешних устройств (большое количество каналов PCIe 4.0), а также улучшенное



масштабирование (возможность выпуска продуктов с разным количеством вычислительных ядер и каналов памяти). Процессоры Ерус 7002 совместимы с существующими платформами при межсокетном соединении на скорости в 10.7 GT/s, но на втором поколении платформ эта скорость вырастет до 18 GT/s, и таких соединений между процессорными разъемами может быть до четырёх, что в результате дает пропускную способность до 202 ГБайт/с.

О внутреннем содержании чиплета ввода-вывода AMD рассказывает довольно мало. Во всех моделях Ерус он идентичен, поддерживая 128 линий PCIe 4.0 и 8 каналов памяти DDR4-3200 с коррекцией ошибок. Поддерживаются модули емкостью до 256 ГБ и рекомендуется равномерно заполнять все каналы одинаковым объемом и типом модулей, хотя в теории можно использовать даже один модуль памяти на всю систему, правда смысла в этом нет никакого. Средняя задержка доступа к памяти для восьми каналов в рамках одного CPU составляет чуть больше 100 нс, а конкретные значения времени доступа зависят от частоты памяти и типа модулей. При использовании двух модулей на канал, максимальная скорость снижается с 3200 до 2933 или даже до 2666 МГц при наборе модулями большого объема.

Но при всех своих ограничениях и оговорках, улучшенная архитектура AMD Infinity обеспечила довольно высокие показатели по пиковой пропускной способности и объёму памяти, а также характеристикам подсистемы ввода-вывода. Так, второе поколение Ерус поддерживает до 4 ТБайт памяти стандарта DDR4-3200 при 8 каналах на каждый разъем, с пиковой ПСП до 204 ГБайт/с на процессор. То есть, максимальная ПСП на двухпроцессорный сервер для Ерус 7002 получается до 410 ГБайт/с, тогда как у Ерус 7001 она была 340 ГБайт/с, а у конкурирующих процессоров от Intel (Xeon Cascade Lake SP) — лишь 282 ГБайт/с.

### ***Другие технологии и новинки***

С поддержкой шины PCI Express изменилось немного, кроме поддерживаемой версии. Для ввода-вывода новым процессорам доступны 128 линий PCIe 4.0 на каждый разъем, с максимальной пропускной способностью в 512 ГБ/с. Модели Ерус 7002 стали первыми x86-совместимыми процессорами с такой поддержкой, когда все восемь x16 каналов на каждый CPU поддерживают удвоенную скорость передачи данных. 16-канальные соединения PCIe 4.0 можно разделить на несколько устройств, требующих меньшую пропускную способность.

Но хотя на каждый CPU есть по 128 линий PCIe 4.0, для двухсокетной системы это количество не увеличится, так как по 64 линии от каждого из CPU забирает связующая их Infinity Fabric (есть возможность получить 192 линии, забрав часть у шины, связывающей процессоры — с соответствующими последствиями). Линии процессора разбиты на восемь групп по 16 штук, и каждая из них поддерживает разделение до x1, но с общим числом слотов на группу не выше восьми. Половина групп поддерживает переключение восьми линий PCIe в режим SATA3, и в общем получается поддержка до 32 штук SATA- или NVMe-накопителей.

На серверном рынке весьма важно обеспечение безопасности для всех клиентов, и тут AMD имеет явное преимущество перед конкурентом, в том числе если говорить о наметивших угрозах Spectre, Meltdown, Foreshadow и прочих. Если первое поколение Ерус требовало обновления прошивки и поддержки защиты со стороны ОС, то второе поколение уже имеет в том числе и аппаратные элементы защиты от всех версий Spectre, известных на момент разработки Ерус 2.

Важное обновление касается расширения возможностей шифрования оперативной памяти по алгоритму AES-128, которое практически не сказывается на производительности. Ерус 7002 имеет поддержку второго поколения технологии безопасной виртуализации Secure Encrypted Virtualization 2 (SEV2) и технологии шифрования памяти Secure Memory Encryption (SME). Для этого в чипы Ерус внедрён выделенный 32-битный микроконтроллер

«AMD Secure Processor» в виде ARM Cortex-A5, который управляется собственной прошивкой и ОС и обеспечивает криптографическую функциональность.

Это выделенное ARM-ядро управляет криптографическими ключами и является невидимым для x86-ядер. При работе SME, позволяющей обезопаситься от атак неавторизованного доступа к памяти, вся память шифруется при помощи единого ключа прозрачно для пользовательских приложений, а технология SEV2 позволяет выбирать активный криптографический ключ для каждой виртуальной машины. Это используется для защиты виртуальных машин друг от друга, для чего используется отдельный криптографический ключ для главного гипервизора и по ключу для каждой виртуальной машины или их групп, изолируя гипервизор от гостевых виртуальных машин.

Поддержка этих технологий уже имеется в большом количестве серверных ОС, и отличие Ерус 7002 от первого поколения в значительно большем количестве поддерживаемых гостевых виртуальных машин (и одновременно используемых криптографических ключей, соответственно) — технология SEV2 обеспечивает шифрование для 509 уникальных виртуальных машин и совместима с существующей технологией виртуализации AMD-V. Особенностью реализации является прозрачность для аппаратных средств, обращающихся к памяти — все шифрование и дешифрование происходит «на лету».

Интересно, что на возможностях серверных процессоров, связанных с обеспечением безопасности, сказалась активная работа AMD над заказными продуктами, включая решения для игровых консолей. Компания применяет опыт, полученный при разработке SoC для игровых консолей, в том числе и при создании серверных процессоров. В частности, второе поколение Ерус стало более защищенным благодаря разработке чипов для игровых консолей Microsoft Xbox One и Sony PlayStation 4. Эти компании настояли, чтобы игры запускались в изолированном программном окружении, которое защищалось бы от пиратов при помощи аппаратного шифрования.

Ниже, на Рис.394 приведено несколько результатов сравнения на некоторых тестах Ерус 2-го поколения и конкурирующей платформы Intel Xeon Platinum 8280L по информации от AMD.

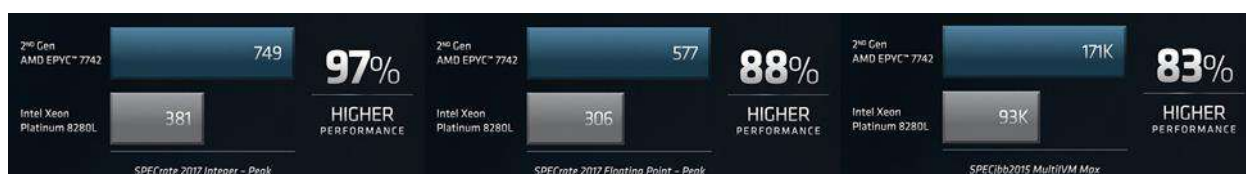


Рис.394 – Сравнение производительности Ерус 2 и Хеоп 8208L.

Результаты серьёзные, даже если AMD где-то слегка преувеличила, подобные приросты впечатляют. На Рис.395 – опять же по информации от AMD – прирост производительности на платформе Ерус 2 в различных приложениях.



Рис.395 – Прирост производительности Ерус 2 в различных приложениях.

В среднем, компания AMD оценивает преимущество над конкурентом где-то в 1.8-2.0 раза (есть задачи с 50% превосходства, но есть и удвоенная производительность) при сниженной совокупной стоимости владения на 25%-50%.

В следующем поколении Zen – Zen 3 (“Milan”) изменения в архитектуре были не столь значительными, как при переходе Zen → Zen 2. Ниже будет приведен только один

рисуюнок (Рис.396), на котором отражены эти изменения – по-другому был скомпонован L3-кэш – вместо 16 Мбайт на каждые четыре ядра в Milan появилось 32+ Мбайт на восемь ядер.

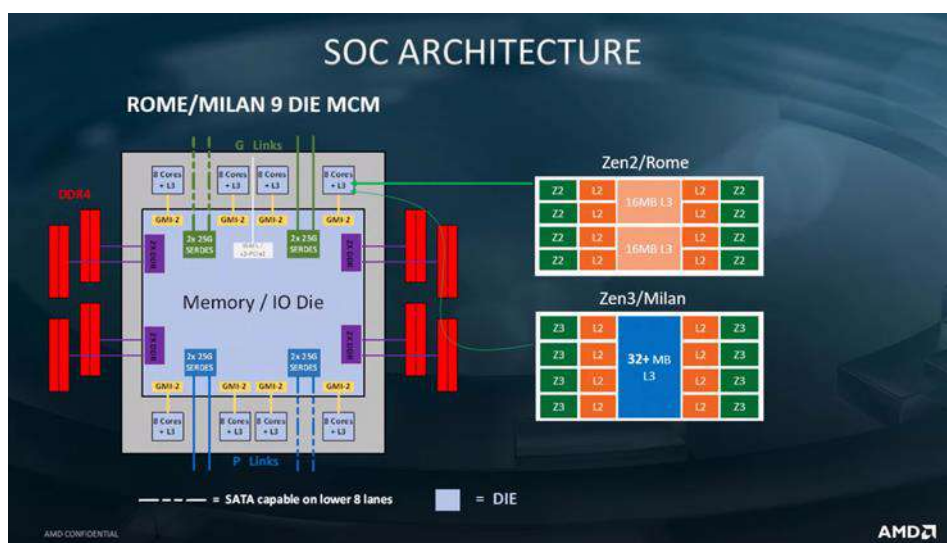


Рис.396 – Система на кристалле (SoC) Zen 3 “Milan”.

## Система команд

Базовая система команд, которую мы знаем, как x86(-64), осталась той же, несмотря на то, что ей давно, и очень многие, предрекали неизбежную кончину, полное поражение и капитуляцию перед RISC-архитектурами. И несмотря на то, что x86-архитектуру в своё время чуть было не «похоронила» сама её компания-создатель, сделавшая ставку на совершенно новый 64-битный Itanium с архитектурой EPIC, но вовремя спохватившаяся и вынужденная сработать вторым номером, лицензировав 64-битное расширение x86-64 у главного своего конкурента, компании AMD. Ядро всех современных x86-процессоров, и от Intel, и от AMD, уже давно внутри имеют RISC-архитектуру, представляя систему команд x86 только для «внешнего» пользователя.

И, невзирая на широко распространённое (и во многом верное) мнение, что x86 – это архаичный, запутанный, нелогичный и довольно сумбурный набор инструкций, громоздкая и переусложнённая архитектура, предсказатели раз за разом ошибаются, называя каждый раз всё новые даты смерти этой «архаики». Которой, к слову, уже стукнуло 40 лет, если вести отсчёт с даты появления на рынке первого IBM PC (август 1981 года), и даже более, если за точку отсчёта брать дату выхода первого x86-процессора (i8086, июнь 1978г.).

Если взглянуть на набор поддерживаемых инструкций современных процессоров от Intel и AMD, то набор команд x86 может затеряться среди длинного списка расширений, которыми производители процессоров регулярно и активно расширяли систему команд своих CPU на протяжении нескольких десятилетий. Тем не менее, архитектура x86, для совместимости с триллионами строк кода и сотнями миллионов устройств, выпущенных на этих процессорах, и сегодня живет всех живых. И до совсем недавнего времени было не очень понятно, кто мог бы всерьёз потеснить этого «динозавра» в ближайшее время на его поле – ноутбуки, десктопы, высокопроизводительные рабочие станции, серверы, вплоть до самых производительных. Но события буквально последних месяцев как бы намекают нам, что «убийцей» x86 вполне может стать ARM в своих современных реинкарнациях – тут и переориентация Apple (пока только ноутбуки) на эту архитектуру, и Top1 в суперкомпьютерных архитектурах 2020/21 года.

Ранее уже отмечалось, что, по мнению автора, архитектура x86 проиграла полностью, и похоже бесповоротно, без шансов на реванш, рынок мобильных устройств, где сегодня безоговорочно правит ARM. Неоднократные попытки Intel пробиться на явно упущенный в своё время стратегами компании мобильный рынок, оказались безуспешными,

ни одно из её энергоэффективных решений так и не сумело занять сколь-нибудь заметную часть на рынке мобильных устройств. Количество выпущенных ARM процессоров многократно превосходит количество выпущенных Intel-ом процессоров x86 – миллиарды против сотен миллионов.

На этом же рынке потерпел поражение и основной стратегический партнёр Intel – компания Microsoft. В конце 2019-го года она объявила о полном сворачивании работ на мобильном направлении, прекратив поддержку Windows Mobile. Внедряясь на мобильный рынок, Microsoft даже попутно прикупила бывшую долгие годы номером один на рынке мобильных телефонов компанию Nokia, но это не помогло некогда непобедимой связке Wintel стать лидерами этого направления.

Расширение системы команд не всегда проходило гладко и безболезненно, например, после обнаружения ошибок в работе команды TSX, которые невозможно было исправить обновлением микрокода, Intel запрещала (исключала) эту команду, модифицировав BIOS, затем возвращала её в процессоры следующего степпинга, но не всегда и не во все модели. В общем, довольно привычный сумбур и хаос в исполнении Intel ☺.

Ниже приведён список наборов команд для одного из современных семейств Intel Skylake. Обратите внимание, что даже внутри одного семейства поддерживаемый набор инструкций может варьироваться:

*x86-64, Intel 64, SSE, SSE2, SSE3, SSSE3, SSE4, SSE4.1, SSE4.2, ADX, AVX, AVX2, AVX-512 (SkyLake-SP, SkyLake-W & SkyLake-X), MPX, TXT, TSX, SGX, VT-x, VT-d*

## Построение вычислительной системы

Практика обязательного выпуска чипсета под каждый новый процессор была продолжена и закреплена выходом новых серий поддерживающих наборов микросхем. Описание части чипсетов под процессоры Core 2 (965, G31/P35/Q45/X48 и др.) уже были приведены ранее, в таблицах на Рис.345, Рис.346 Рис.347, Рис.348, поэтому далее будут представлены данные по чипсетам для процессоров поколения Nehalem и новее.

## Чипсеты серий 5/6/7/8/9

Начиная с микроархитектуры Nehalem контроллер памяти был перенесён на кристалл процессора. Для high-end процессоров Nehalem чипсет X58 IOH<sup>1</sup> действует как мост между шинами QPI и периферийной PCI Express и между шиной DMI и южным мостом ICH10. Для процессоров Nehalem массового и low-end уровня встроенный контроллер памяти (IMC) представляет собой по сути целый северный мост (некоторые даже имеют встроенный GPU), а PCH (Platform Controller Hub) функционирует как южный мост.

Ниже не указан набор микросхем 3450 (он относится к категории «Наборы микросхем Хеон»), который совместим с процессорами Nehalem массового и high-end уровня, но не претендует на совместимость с ядрами Core iX. С процессором Core i5 или i3 наборы микросхем серии 3400 позволяют использовать функции ECC для небуферизованной памяти ECC.

Чипсеты серии Cougar Point Intel 6 со степпингом B2 были отозваны из-за аппаратной ошибки, из-за которой интерфейс Serial ATA со скоростью 3 Гбит/с со временем деградировал, вплоть до полного отказа. Следующий степпинг B3 набор микросхем 6-й серии эту ошибку исправил. Чипсет Z68, который поддерживает разгон процессора и использование встроенной графики, не имеет этой аппаратной ошибки, в отличие от всех остальных чипсетов со степпингом B2. Z68 также добавил поддержку прозрачного кэширования данных с жесткого диска на твердотельные накопители (до 64 ГБайт), технологию, называемую Smart Response Technology.

---

<sup>1</sup> Северный мост чипсета **X58** называется IOH (Input/Output Hub), а не как он назывался в прошлых моделях X48, X38, P45 — MCH (Memory Controller Hub).

| Chipset | Code Name | sSpec Number              | Part numbers  | Release Date | Bus Interface | Link Speed <sup>[c]</sup> | PCI Express lanes      | PCI | SATA<br>3 Gbit/s | USB<br>v2.0 | FDI support | TDP   |  |  |  |  |  |  |  |  |
|---------|-----------|---------------------------|---------------|--------------|---------------|---------------------------|------------------------|-----|------------------|-------------|-------------|-------|--|--|--|--|--|--|--|--|
| H55     | Ibex Peak | SLGZX(B3)                 | BD82H55 (PCH) | Jan 2010     | DMI           | 2 GB/s                    | 6 PCIe 2.0 at 2.5 GT/s | Yes | 6 ports          | 12 ports    | Yes         | 5.2 W |  |  |  |  |  |  |  |  |
| P55     |           | SLH24 (B3),<br>SLGWV (B2) | BD82P55 (PCH) | Sep 2009     |               |                           | 8 PCIe 2.0 at 2.5 GT/s |     |                  | 14 ports    | No          | 4.7 W |  |  |  |  |  |  |  |  |
| H57     |           | SLGZL(B3)                 | BD82H57 (PCH) | Jan 2010     |               |                           |                        |     |                  |             | Yes         | 5.2 W |  |  |  |  |  |  |  |  |
| Q57     |           | SLGZW(B3)                 | BD82Q57 (PCH) |              |               |                           |                        |     |                  |             |             | 5.1 W |  |  |  |  |  |  |  |  |

Рис.397 - Чипсеты поддержки CPU на сокете LGA1156 (Lynnfield и Clarkdale).

| Chipset           | Code name     | sSpec number                       | Part numbers  | Release date      | Bus interface | Link speed <sup>[c]</sup> | PCI Express lanes | PCI | SATA     |          | USB          |               | FDI support | TDP   |
|-------------------|---------------|------------------------------------|---------------|-------------------|---------------|---------------------------|-------------------|-----|----------|----------|--------------|---------------|-------------|-------|
|                   |               |                                    |               |                   |               |                           |                   |     | 6 Gbit/s | 3 Gbit/s | v3.2 Gen 1x1 | v2.0          |             |       |
| H61 <sup>Ⓔ1</sup> | Cougar Point  | SLH83(B2)<br>SLJ4B(B3)             | BD82H61 (PCH) | February 20, 2011 | DMI 2.0       | 4 GB/s                    | 6 PCIe 2.0        | No  | None     | 4 ports  | None         | 10 ports      | Yes         | 6.1 W |
| B65 <sup>Ⓔ1</sup> |               | SLH98(B2)<br>SLJ4A(B3)             | BD82B65 (PCH) | February 25, 2011 |               |                           | 8 PCIe 2.0        | Yes | 1 port   | 5 ports  |              | 12 ports      |             |       |
| Q65 <sup>Ⓔ1</sup> |               | SLH99(B2)<br>SLJ4E(B3)             | BD82Q65 (PCH) | Q2 2011           |               |                           |                   | No  | 2 ports  | 4 ports  |              | 14 ports      |             |       |
| P67 <sup>Ⓔ1</sup> |               | SLH84(B2) (Recalled)<br>SLJ4C (B3) | BD82P67 (PCH) | January 9, 2011   |               |                           |                   |     |          |          |              |               |             |       |
| H67 <sup>Ⓔ1</sup> |               | SLH82(B2) (Recalled)<br>SLJ49 (B3) | BD82H67 (PCH) |                   |               |                           |                   |     |          |          |              |               |             |       |
| Q67 <sup>Ⓔ1</sup> |               | SLH85(B2)<br>SLJ4D(B3)             | BD82Q67 (PCH) | February 20, 2011 |               |                           | Yes               |     |          |          |              |               |             |       |
| Z68 <sup>Ⓔ1</sup> |               | SLJ4F(B3)                          | BD82Z68 (PCH) | May 11, 2011      |               |                           | No                |     |          |          |              |               |             |       |
| B75 <sup>Ⓔ2</sup> | Panther Point | SLJ85(C1)                          | BD82B75 (PCH) | May 13, 2012      |               |                           |                   |     | 1 port   | 5 ports  | 4 ports      | 8 ports       | Yes         | 6.7 W |
| Q75 <sup>Ⓔ2</sup> |               | SLJ84(C1)                          | BD82Q75 (PCH) |                   |               |                           |                   |     |          |          |              | April 8, 2012 |             |       |
| Z75 <sup>Ⓔ2</sup> |               | SLJ87(C1)                          | BD82Z75 (PCH) | No                |               |                           |                   |     |          |          |              |               |             |       |
| H77 <sup>Ⓔ2</sup> |               | SLJ88(C1)                          | BD82H77 (PCH) |                   |               |                           |                   |     |          |          |              | Yes           |             |       |
| Q77 <sup>Ⓔ2</sup> |               | SLJ83(C1)                          | BD82Q77 (PCH) | May 13, 2012      |               |                           |                   |     |          |          | No           |               |             |       |
| Z77 <sup>Ⓔ2</sup> |               | SLJC7(C1)                          | BD82Z77 (PCH) | April 8, 2012     |               |                           |                   |     |          |          |              |               |             |       |

Рис.398 – Чипсеты поддержки CPU на сокете LGA1155 (Sandy Bridge и Ivy Bridge). Линии PCIe 2.0, выходящие из PCH в этой серии работают на скорости 5 GT/s, в отличие от предыдущих кристаллов для LGA1156.

Чипсеты, поддерживающие CPU на сокете LGA1150, приведены ниже, в таблице на Рис.399. Процессоры Haswell и Haswell Refresh также поддерживаются всеми перечисленными чипсетами, но для поддержки Haswell Refresh материнскими платами с 8-ой серий Lynx Point требуется обновление BIOS. Процессоры Broadwell поддерживаются только чипсетами 9-ой серии, которую обычно называют Wildcat Point.

Степпинг C1 чипсета Lynx Point содержал ошибку – система могла потерять связь с USB-устройствами, включёнными в порты USB 3.0, обеспечиваемыми чипсетом, если система переходила в состояние сна S3.

| Chipset                                                                                 | Code name     | sSpec number                        | Part numbers  | Release date | Bus Interface | Link Speed <sup>[c]</sup> | PCI Express lanes | Intel VT-d support | PCI | SATA     |          | USB          |         | FDI support | TDP   | PCIe M.2 support |  |
|-----------------------------------------------------------------------------------------|---------------|-------------------------------------|---------------|--------------|---------------|---------------------------|-------------------|--------------------|-----|----------|----------|--------------|---------|-------------|-------|------------------|--|
|                                                                                         |               |                                     |               |              |               |                           |                   |                    |     | 6 Gbit/s | 3 Gbit/s | v3.2 Gen 1x1 | v2.0    |             |       |                  |  |
| H81  | Lynx Point    | SR13B(C1)<br>SR177(C2)              | DH82H81 (PCH) | June 2013    | DMI 2.0       | 4 GB/s                    | 6 PCIe 2.0        | No                 | No  | 2 ports  |          | 2 ports      | 8 ports | Yes         | 4.1 W | No               |  |
| B85  |               | SR13C(C1)<br>SR178(C2)              | DH82B85 (PCH) |              |               |                           | 8 PCIe 2.0        |                    |     | 4 ports  | 2 ports  | 4 ports      |         |             |       |                  |  |
| Q85  |               | SR138(C1)<br>SR174(C2)              | DH82Q85 (PCH) |              |               |                           |                   |                    |     | 4 ports  |          |              |         |             |       |                  |  |
| Q87  |               | SR137(C1)<br>SR173(C2)<br>SR19E(C2) | DH82Q87 (PCH) |              |               |                           | Yes               |                    |     |          |          |              |         |             |       |                  |  |
| H87  |               | SR139(C1)<br>SR175(C2)              | DH82H87 (PCH) |              |               |                           |                   |                    |     |          |          |              |         |             |       |                  |  |
| Z87  |               | SR13A(C1)<br>SR176(C2)              | DH82Z87 (PCH) |              |               |                           |                   |                    |     |          |          |              |         |             |       |                  |  |
| Z97  | Wildcat Point | SR1JJ(A0)                           | DH82Z97 (PCH) | May 2014     |               |                           |                   | No                 |     | 6 ports  | None     | 6 ports      |         |             |       | Yes              |  |
| H97  |               | SR1JK(A0)                           | DH82H97 (PCH) |              |               |                           |                   |                    |     |          |          |              |         |             |       |                  |  |

Рис.399 – Чипсеты поддержки CPU на LGA1150.



| Chipset                                   | Code Name  | sSpec Number                                              | Part numbers                  | Release Date      | Socket      | Bus Interface | Link Speed <sup>(1)</sup> | PCI Express lanes                                | Intel VT-d support | PCI      | SATA     |          | USB          |          | FDI support | TDP    |
|-------------------------------------------|------------|-----------------------------------------------------------|-------------------------------|-------------------|-------------|---------------|---------------------------|--------------------------------------------------|--------------------|----------|----------|----------|--------------|----------|-------------|--------|
|                                           |            |                                                           |                               |                   |             |               |                           |                                                  |                    |          | 6 Gbit/s | 3 Gbit/s | v3.2 Gen 1x1 | v2.0     |             |        |
| X58 <sup>(2)</sup> (I/O hub) <sup>1</sup> | Tylersburg | SLGBT (B2),<br>SLGMX (B3),<br>SLH3M (C2)                  | AC82X58 (ICH)                 | November 2008     | LGA 1366    | QPI           | Up to 12.8 GB/s           | 36 PCIe 2.0 at 5 GT/s (IOH),<br>6 PCIe 1.1 (ICH) | Yes                | Yes      | None     | 6 ports  | None         | 12 ports | No          | 28.6 W |
| X79 <sup>(3)</sup>                        | Patsburg   | SLJHW (C0), <sup>(57)</sup><br>SLJN7 (C1) <sup>(58)</sup> | BD82X79 (PCH) <sup>(59)</sup> | November 14, 2011 | LGA 2011    | DMI 2.0       | 4 GB/s                    | 8 PCIe 2.0                                       |                    | No       | 2 ports  | 4 ports  |              | 14 ports |             | 7.8 W  |
| X99 <sup>(4)</sup>                        | Wellsburg  | SLKDE (B1),<br>SLKM9 (B1)                                 | DH82031PCH (PCH)              | August 29, 2014   | LGA 2011-v3 |               |                           |                                                  |                    | 10 ports | None     | 6 ports  |              | 8 ports  |             | 6.5 W  |

**Рис.400 – Чипсеты, поддерживающие односокетные конфигурации процессоров на LGA1366, LGA2011 и LGA2011-v3.**

| Chipset             | Code Name       | sSpec Number                      | Part numbers   | Release Date      | Process support | Bus interface | Link Speed <sup>(1)</sup> | PCI Express lanes | Intel VT-d support | SATA    |          | USB          |          | TDP      | FDI support |            |          |          |         |
|---------------------|-----------------|-----------------------------------|----------------|-------------------|-----------------|---------------|---------------------------|-------------------|--------------------|---------|----------|--------------|----------|----------|-------------|------------|----------|----------|---------|
|                     |                 |                                   |                |                   |                 |               |                           |                   |                    | 6 Gb/s  | 3 Gb/s   | v3.2 Gen 1x1 | v2.0     |          |             |            |          |          |         |
| PM55 <sup>(2)</sup> | Ibex Peak-M     | SLGVN(B2),<br>SLH23(B3),<br>SLGWP | BD82PM55 (PCH) | September 2009    | 45 nm, 32 nm    | DMI           | 2 GB/s                    | 8 PCIe 2.0        | Yes                | None    | 6 ports  | None         | 14 ports | 3.5 W    | No          |            |          |          |         |
| HM55 <sup>(2)</sup> |                 | SLQ2S(B3)                         | BD82HM55 (PCH) | January 2010      |                 |               |                           | 6 PCIe 2.0        |                    |         |          |              |          |          |             | 6 ports    | 12 ports |          |         |
| HM57 <sup>(2)</sup> |                 | SLGZR(B3)                         | BD82HM57 (PCH) |                   |                 |               |                           | 8 PCIe 2.0        |                    |         |          |              |          |          |             |            |          | 14 ports |         |
| QM57 <sup>(2)</sup> |                 | SLQ2Q(B3)                         | BD82QM57 (PCH) |                   |                 |               |                           |                   |                    |         |          |              |          |          |             |            |          |          | 3.4 W   |
| QS57 <sup>(2)</sup> |                 | SLGZV(B3)                         | BD82QS57 (PCH) |                   |                 |               |                           |                   |                    |         |          |              |          |          |             |            |          |          |         |
| HM65 <sup>(2)</sup> | Cougar Point-M  | SLH9D(B2) (Recalled)<br>SLJ4P(B3) | BD82HM65 (PCH) | January 9, 2011   | 32 nm           | DMI 2.0       | 4 GB/s                    | No                | 2 ports            | 4 ports | None     | 12 ports     | 3.9 W    | Yes      |             |            |          |          |         |
| HM67 <sup>(2)</sup> |                 | SLH9C(B2) (Recalled)<br>SLJ4N(B3) | BD82HM67 (PCH) |                   |                 |               |                           |                   |                    |         |          |              |          |          | 14 ports    |            |          |          |         |
| UM67 <sup>(2)</sup> |                 | SLH9U(B2)<br>SLJ4L(B3)            | BD82UM67 (PCH) |                   |                 |               |                           |                   |                    |         |          |              |          |          |             |            |          |          |         |
| QM67 <sup>(2)</sup> |                 | SLH9B(B2)<br>SLJ4M(B3)            | BD82QM67 (PCH) | February 20, 2011 |                 |               |                           | Yes               |                    |         |          | 3.9 W        |          |          |             |            |          |          |         |
| QS67 <sup>(2)</sup> |                 | SLH9A(B2)<br>SLJ4K(B3)            | BD82QS67 (PCH) |                   |                 |               |                           |                   |                    |         |          |              | 3.4 W    |          |             |            |          |          |         |
| NM70                |                 | Panther Point-M                   | SLJ7A(C1)      | BD82NM70 (PCH)    |                 |               |                           | August 2012       |                    |         |          | 22 nm        | DMI 2.0  |          | 4 GB/s      | 4 PCIe 2.0 | ?        | 1 port   | 3 ports |
| HM70 <sup>(2)</sup> | SLJ7N(C1)       |                                   | BD82HM70 (PCH) | 8 PCIe 2.0        | 4 ports         | 6 ports       |                           |                   |                    |         |          |              |          |          |             |            |          |          |         |
| HM75 <sup>(2)</sup> | SLJ8F(C1)       |                                   | BD82HM75 (PCH) |                   | 2 ports         | 4 ports       | 12 ports                  |                   |                    |         |          |              |          |          |             |            |          |          |         |
| HM76 <sup>(2)</sup> | SLJ8E(C1)       |                                   | BD82HM76 (PCH) | 4 PCIe 2.0        |                 | 1 port        | 3 ports                   | 8 ports           |                    |         |          |              |          |          |             |            |          |          |         |
| UM77 <sup>(2)</sup> | SLJ8D(C1)       |                                   | BD82UM77 (PCH) |                   | April 8, 2012   |               | 4 ports                   | 5 ports           |                    |         |          |              |          |          |             |            |          |          |         |
| HM77 <sup>(2)</sup> | SLJ8C(C1)       |                                   | BD82HM77 (PCH) | 4.1 W             |                 |               |                           |                   |                    |         |          |              |          |          |             |            |          |          |         |
| QM77 <sup>(2)</sup> | SLJ8A(C1)       |                                   | BD82QM77 (PCH) |                   |                 | 10 ports      |                           |                   |                    |         |          |              |          |          |             |            |          |          |         |
| QS77 <sup>(2)</sup> | SLJ8B(C1)       |                                   | BD82QS77 (PCH) | 3.0 to 3.6 W      |                 |               |                           |                   |                    |         |          |              |          |          |             |            |          |          |         |
| HM86 <sup>(2)</sup> | Lynx Point-M    | SR13J(C1)<br>SR17E(C2)            | DH82HM86 (PCH) | June 2013         |                 |               | 8 PCIe 2.0                | Yes               | 4 ports            | 2 ports | 5 ports  | 8 ports      | 2.7 W    |          |             |            |          |          |         |
| QM87 <sup>(2)</sup> |                 | SR13G(C1)<br>SR17C(C2)            | DH82QM87 (PCH) |                   |                 |               |                           |                   |                    |         | 6 ports  |              |          |          |             |            |          |          |         |
| HM87 <sup>(2)</sup> |                 | SR13H(C1)<br>SR17D(C2)            | DH82HM87 (PCH) |                   |                 |               |                           |                   |                    |         |          |              |          |          |             |            |          |          |         |
| HM97                |                 | SR1JN(A0)                         | DH82HM97 (PCH) |                   |                 |               |                           |                   |                    |         | May 2014 |              |          | 10 ports |             |            |          |          |         |
|                     | Wildcat Point-M |                                   |                |                   |                 |               |                           | 2                 |                    |         |          |              |          |          |             |            |          |          |         |

**Рис.401 – Мобильные чипсеты. Все серии Core-i имеют встроенный южный мост.**

## Чипсеты серий 100/200/300

Все чипсеты этих серий поддерживают технологию VT-d и не поддерживают PCI.

### LGA 1151 rev 1

Чипсеты 100-ой серии (Sunrise Point), для процессоров Skylake, использующих сокет LGA1151 socket, были выпущены в третьем квартале 2015 года.

Чипсеты 200-ой серии (Union Point) были представлены вместе с процессорами Kaby Lake, которые также используют сокет LGA1151; были выпущены в первом квартале 2017 года.

| Chipset | Code Name     | sSpec Number    | Part numbers   | Release Date  | Bus Interface | Link Speed <sup>[c]</sup> | PCI Express lanes | Intel Optane Memory support | SATA 6 Gbit/s | SATAe    | PCIe M.2    | USB ports    |              |          | TDP |          |          |
|---------|---------------|-----------------|----------------|---------------|---------------|---------------------------|-------------------|-----------------------------|---------------|----------|-------------|--------------|--------------|----------|-----|----------|----------|
|         |               |                 |                |               |               |                           |                   |                             |               |          |             | v3.2 Gen 1x1 | v3.2 Gen 2x1 | Total    |     |          |          |
| H110    | Sunrise Point | SR2CA(D1) SR286 | GL82H110 (PCH) | Sep. 27, 2015 | DMI 2.0       | 2.0 GB/s                  | 6 PCIe 2.0        | No                          | 4             | None     | None        | Up to 4      | None         | Up to 10 | 6 W |          |          |
| B150    |               | SR2C7(D1) SR283 | GL82B150 (PCH) | Sep. 1, 2015  | DMI 3.0       | 3.93 GB/s                 | 8 PCIe 3.0        |                             | 6             | Up to 1  |             | Up to 2      |              | Up to 6  |     | Up to 12 |          |
| Q150    |               | SR2C6(D1) SR282 | GL82Q150 (PCH) | H2 2015       |               |                           | 10 PCIe 3.0       |                             |               |          |             |              |              | Up to 8  |     | Up to 14 |          |
| H170    |               | SR2C8(D1) SR284 | GL82H170 (PCH) | Sep. 1, 2015  |               |                           | 16 PCIe 3.0       |                             |               |          | Up to 2     |              |              | Up to 2  |     |          |          |
| Q170    |               | SR2C5(D1) SR281 | GL82Q170 (PCH) | Oct. 2015     |               |                           | 20 PCIe 3.0       |                             |               |          | Up to 3     |              |              | Up to 3  |     |          | Up to 10 |
| Z170    |               | SR2C9(D1) SR285 | GL82Z170 (PCH) | Aug. 2015     |               |                           | Jan. 3, 2017      |                             |               |          | 24 PCIe 3.0 |              |              | Yes      |     |          | Up to 3  |
| B250    | SR2WC(A0)     | GL82B250        | Union Point    | 14 PCIe 3.0   | Up to 1       | Up to 1                   |                   | Up to 6                     | Up to 8       | Up to 14 |             |              |              |          |     |          |          |
| Q250    | SR2WD(A0)     | GL82Q250        |                |               |               |                           |                   |                             |               |          |             |              |              |          |     |          |          |
| H270    | SR2WA(A0)     | GL82H270        |                |               |               |                           |                   |                             |               |          |             |              |              |          |     |          |          |
| Q270    | SR2WE(A0)     | GL82Q270        |                |               |               |                           |                   |                             |               |          |             |              |              |          |     |          |          |
| Z270    | SR2WB(A0)     | GL82Z270        |                |               |               |                           |                   |                             |               |          |             |              |              |          |     |          |          |

Рис.402 - Чипсеты поддержки CPU на LGA1151 rev 1.

## LGA 1151 rev 2

Хотя процессоры Coffee Lake использовали тот же сокет, что и процессоры Skylake и Kaby Lake, эта ревизия LGA1151 электрически несовместима с 100-ой и 200-ой сериями чипсетов.

300-ая серия чипсетов была представлена с процессорами Coffee Lake, которые использовали сокет LGA1151; модель для энтузиастов была выпущена в последнем квартале 2017 года, остальная часть линейки была выпущена в 2018 году.

| Chipset              | Code Name    | sSpec Number                        | Part numbers  | Release Date    | Bus Interface | Link Speed <sup>[c]</sup> | PCI Express lanes | Intel Optane Memory support | SATA 6 Gbit/s | SATAe   | PCIe M.2 | USB ports       |              |          | TDP |          |         |  |  |
|----------------------|--------------|-------------------------------------|---------------|-----------------|---------------|---------------------------|-------------------|-----------------------------|---------------|---------|----------|-----------------|--------------|----------|-----|----------|---------|--|--|
|                      |              |                                     |               |                 |               |                           |                   |                             |               |         |          | v3.2 Gen 1x1    | v3.1 Gen 2x1 | Total    |     |          |         |  |  |
| <a href="#">Z370</a> | Cannon Point | SR3MD(A0)                           | GL82Z370      | October 5, 2017 | DMI 3.0       | 3.93 GB/s                 | 24 PCIe 3.0       | Yes                         | 6             | up to 3 | up to 3  | Up to 10        | None         | Up to 14 | 6 W |          |         |  |  |
| <a href="#">H310</a> |              | SR409(B0)<br>SRCXT(B0)<br>SRCXY(B0) | ?             | April 3, 2018   | DMI 2.0       | 2.0 GB/s                  | 6 PCIe 2.0        | No                          | 4             | None    | None     | Up to 4         |              | Up to 10 |     |          |         |  |  |
| <a href="#">B360</a> |              | SR406(B0)                           |               | Dec. 14, 2018   | DMI 3.0       | 3.93 GB/s                 | 12 PCIe 3.0       | Yes                         | 6             | Up to 1 | Up to 1  | Up to 6         |              | Up to 12 |     |          |         |  |  |
| <a href="#">B365</a> | Union Point  | SREVJ(A0)                           |               |                 |               |                           | 20 PCIe 3.0       |                             |               |         |          | Up to 2         | None         | Up to 14 |     |          |         |  |  |
| <a href="#">H370</a> | Cannon Point | SR405(B0)                           | April 3, 2018 | 24 PCIe 3.0     |               |                           |                   |                             |               |         |          | Up to 8         |              |          |     |          |         |  |  |
| <a href="#">Q370</a> |              | SR404(B0)                           | Q2 2018       |                 |               |                           |                   |                             |               |         |          | Up to 4         |              |          |     |          |         |  |  |
| <a href="#">Z390</a> |              | SR406(B0)                           | FH82Z390      |                 |               |                           |                   |                             |               |         |          | October 8, 2018 |              | Up to 3  |     | Up to 10 | Up to 6 |  |  |

Рис.403 - Чипсеты поддержки CPU на LGA1151 rev 2.

## Чипсеты под процессоры Xeon

Чипсеты C232 и C242 не поддерживают интегрированные в процессор GPU, из-за отсутствия у них поддержки FDI. Официально эти чипсеты поддерживают только процессоры Xeon, но некоторые материнские платы поддерживают также пользовательские десктопные процессоры (Core 6/7-го поколений для серий C230 series, Core 8/9-го поколения для серий C240 и их Pentium/Celeron производные).

| Chipset | Code Name     | sSpec Number | Part numbers   | Release Date      | Bus Interface | Link Speed <sup>[c]</sup> | PCI Express lanes | SATA 6 Gbit/s | SATAe   | PCIe M.2 | USB ports |      | TDP |
|---------|---------------|--------------|----------------|-------------------|---------------|---------------------------|-------------------|---------------|---------|----------|-----------|------|-----|
|         |               |              |                |                   |               |                           |                   |               |         |          | v3.0      | v2.0 |     |
| C232    | Sunrise Point | SR2CB(D1)    | GL82C232 (PCH) | September 1, 2015 | DMI 3.0       | 3.93 GB/s                 | 8 PCIe 3.0        | 6             | Up to 3 | Up to 3  | Up to 6   | 6    | 6 W |
| C236    |               | SR2CC(D1)    | GL82C236 (PCH) |                   |               |                           | 20 PCIe 3.0       | 8             |         |          | Up to 10  | 4    |     |
| C242    | Coffee Lake   | SR40C(B0)    | FH82C242       | November 2018     | DMI 3.0       | 3.93 GB/s                 | 10 PCIe 3.0       | 6             | ?       | ?        | ?         | ?    |     |
| C246    | Coffee Lake   | SR40A(B0)    | FH82C246       | July 2018         |               |                           | 24 PCIe 3.0       | 8             |         |          | ?         | ?    |     |

Рис.404 – Чипсеты для Xeon.

## LGA2066

Чипсеты, поддерживающие сокет LGA2066 для процессоров Skylake-X и процессоров Kaby Lake-X.



Чипсет C621 также поддерживает сокет FCLGA3647 для процессоров Skylake-SP, а также Cascade Lake-W и Cascade Lake-SP.

| Chipset | Code Name   | sSpec Number                                     | Part numbers | Release Date  | Bus Interface | Link Speed <sup>[c]</sup> | PCI Express lanes | SATA<br>6 Gbit/s | SATAe | PCIe M.2 | USB ports<br>v3.0 v2.0 | TDP    |
|---------|-------------|--------------------------------------------------|--------------|---------------|---------------|---------------------------|-------------------|------------------|-------|----------|------------------------|--------|
| X299    | Basin Falls | SR2Z2(A0)                                        | GL82X299     | May 30, 2017  | DMI 3.0       | 3.93 GB/s                 | 24 PCIe 3.0       | Up to 8          | ?     | ?        |                        | 6 W    |
| C422    | Kaby Lake   | SR2WG(A0)                                        | GL82C422     |               |               |                           |                   |                  |       |          |                        | 6 W    |
| C621    | Lewisburg   | SR36S(B1)<br>SR354(S0)<br>SR3HE(B2)<br>SR3HL(S1) | EY82C621X    | July 11, 2017 | UPI           | ?                         | 20 PCIe 3.0       | Up to 14         | ?     | ?        | Up to 10 Up to 14      | 15 W   |
| C622    |             | SR36X(S0)<br>SR3HK(S1)                           | EY82C622     |               |               |                           |                   |                  |       |          |                        | 17 W   |
| C624    |             | SR36Y(S0)<br>SR3HM(S1)                           | EY82C624     |               |               |                           |                   |                  |       |          |                        | 19 W   |
| C625    |             | SR36W(B1)<br>SR3HL(B2)                           | EY82C625     |               |               |                           |                   |                  |       |          |                        | 21 W   |
| C626    |             | SR36V(B1)<br>SR3HH(B2)                           | EY82C626     |               |               |                           |                   |                  |       |          |                        | 23 W   |
| C627    |             | SR36U(B1)<br>SR3HG(B2)                           | EY82C627     |               |               |                           |                   |                  |       |          |                        | 28.6 W |
| C628    |             | SR36T(B1)<br>SR3HF(B2)                           | EY82C628     |               |               |                           |                   |                  |       |          |                        | 26.3 W |

Рис.405 - Чипсеты поддержки CPU на LGA2066.

| Chipset | Code Name | sSpec Number       | Part numbers    | Release Date      | Bus Interface | Link Speed <sup>[c]</sup> | PCI Express lanes | SATA           | SATAe | PCIe M.2 | USB ports |         |          | TDP         |             |         |          |          |             |         |         |         |             |         |
|---------|-----------|--------------------|-----------------|-------------------|---------------|---------------------------|-------------------|----------------|-------|----------|-----------|---------|----------|-------------|-------------|---------|----------|----------|-------------|---------|---------|---------|-------------|---------|
|         |           |                    |                 |                   |               |                           |                   | 6 Gbit/s ports |       |          | v3.2      |         | v2.0     |             |             |         |          |          |             |         |         |         |             |         |
|         |           |                    |                 |                   |               |                           |                   |                |       |          | Gen 1x1   | Gen 2x1 |          |             |             |         |          |          |             |         |         |         |             |         |
| HM170   | Skylake   | SR2C4(D1)<br>SR27Z | GL82HM170 (PCH) | September 1, 2015 | DMI 3.0       | 3.93 GB/s                 | 16 PCIe 3.0       | Up to 4        | ?     | Up to 2  | Up to 8   | None    | Up to 14 | 2.6 W       |             |         |          |          |             |         |         |         |             |         |
| QM170   |           | SR2C3(D1)<br>SR27Y | GL82QM170 (PCH) |                   |               |                           |                   |                |       |          |           |         |          |             | 20 PCIe 3.0 | Up to 8 | Up to 3  | Up to 10 |             |         |         |         |             |         |
| CM236   |           | SR2CE(D1)          | GL82CM236 (PCH) |                   |               |                           |                   |                |       |          |           |         |          |             |             |         |          |          | ?           | ?       | ?       | ?       |             |         |
| QMS180  |           | SR2NH(D1)          | GLQMS180 (PCH)  |                   |               |                           |                   |                |       |          |           |         |          |             |             |         |          |          |             |         |         |         | ?           | ?       |
| HM175   |           | SR30W(D1)          | GL82HM175 (PCH) | January 3, 2017   |               |                           | 16 PCIe 3.0       | Up to 4        |       | Up to 2  | Up to 8   |         | 2.6 W    |             |             |         |          |          |             |         |         |         |             |         |
| QM175   |           | SR30V(D1)          | GL82QM175 (PCH) |                   |               |                           |                   |                |       |          |           |         |          | 20 PCIe 3.0 | Up to 8     | Up to 3 | Up to 10 | 3.67 W   |             |         |         |         |             |         |
| CM238   |           | SR30U(D1)          | GL82CM238 (PCH) |                   |               |                           |                   |                |       |          |           |         |          |             |             |         |          |          | 16 PCIe 3.0 | Up to 4 | Up to 2 | Up to 8 |             |         |
| HM370   |           | SR40B(B0)          | FH82HM370 (PCH) |                   |               |                           |                   |                |       |          |           |         |          |             |             |         |          |          |             |         |         |         | 20 PCIe 3.0 | Up to 8 |
| QM370   | SR40D(B0) | FH82QM370 (PCH)    | Q2 2016         | 24 PCIe 3.0       | Up to 8       | Up to 4                   | Up to 6           |                |       |          |           |         |          |             |             |         |          |          |             |         |         |         |             |         |
| CM246   | SR40E(B0) | FH82CM246 (PCH)    |                 |                   |               |                           |                   |                |       |          |           |         |          |             |             |         |          |          |             |         |         |         |             |         |

Рис.406 – Мобильные чипсеты.

## Чипсеты серии 400

### LGA1200

Сокет LGA1200 совместим с десктопными процессорами Comet Lake. Подобно своим предшественникам, LGA1200 имеет то же количество контактов – 1200, как следует из названия. Под «капотом» LGA1200 модифицированная версия LGA1151, его предшественника и на 2020 год это был последний CPU сокет для интеловских процессоров. Дополнительные 49 контактов используются для улучшения подачи питания и обеспечения поддержки возможных обновлений с функциями ввода-вывода.

| Chipset              | Code Name  | sSpec Number | Part numbers | Release Date | Bus interface | Link Speed <sup>[c]</sup> | PCI Express lanes | Intel Optane Memory support | SATA 6 Gbit/s | SATAe | PCIe M.2 | USB ports |              |              | TDP |
|----------------------|------------|--------------|--------------|--------------|---------------|---------------------------|-------------------|-----------------------------|---------------|-------|----------|-----------|--------------|--------------|-----|
|                      |            |              |              |              |               |                           |                   |                             |               |       |          | v2.0      | v3.2 Gen 1x1 | v3.2 Gen 2x1 |     |
| <a href="#">H410</a> | Comet Lake | SRH1D(A0)    | ?            | Q2'20        | DMI 3.0       | 3.93 GB/s                 | 6 PCIe 3.0        | No                          | 4             | ?     | ?        | Up to 10  | Up to 4      | None         | 6 W |
| <a href="#">B460</a> |            | SRH1C(A0)    |              |              |               |                           | 16 PCIe 3.0       | Yes                         |               |       |          | Up to 12  | Up to 8      | None         |     |
| <a href="#">H470</a> |            | SRH14(A0)    |              |              |               |                           | 20 PCIe 3.0       | Yes                         | 6             |       |          | Up to 14  | Up to 10     | Up to 4      |     |
| <a href="#">Q470</a> |            | SRH1A(A0)    |              |              |               |                           | Yes               |                             |               |       |          |           |              |              |     |
| <a href="#">Z490</a> |            | SRH13(A0)    |              |              |               |                           | Yes               |                             | Up to 6       |       |          |           |              |              |     |
| <a href="#">W480</a> |            | SRH19(A0)    |              |              |               |                           | Yes               | 8                           | Up to 8       |       |          |           |              |              |     |

Рис.407 - Чипсеты поддержки CPU на LGA1200.

По приведённым выше спискам микросхем поддержки (также далеко не полным), можно понять, что с чипсетами под современные интеловские процессоры дела обстоят очень неплохо. Выпускались чипсеты, поддерживавшие несколько поколений процессоров, но практиковался и выпуск чипсетов под конкретную модель, одновременно с выходом нового CPU, оптимизированных под новый процессор. Ситуаций «есть процессор, но нет адекватного поддерживающего его набора микросхем», как иногда бывало в прошлом, больше никогда не возникало. Микросхемы поддержки были доступны для всего спектра решений от Intel, для любых платформ.

## Шина PCI Express

Начиная с поколения Core, основной компьютерной шиной в архитектуре x86 компьютеров стала PCI Express (PCI-E, PCIe), сменившая, наконец-то, старые параллельные PCI/PCI-X и узкоспециализированную AGP.

Роль универсального интерфейса для подключения самой разнообразной периферии уверенно заняла шина USB, добравшаяся на сегодня до версии USB 3.2 (несколько ревизий) и до внушительных скоростей уровня 20 Гбит/с.

Несмотря на похожие наименования, шины PCI и PCI Express существенно различаются на физическом уровне, т.к. по сути PCI-E шиной не является, реализуя соединение типа «точка-точка», используя *программную* модель шины PCI и высокопроизводительный физический протокол, основанный на последовательной передаче данных, являясь, фактически пакетной сетью передачи данных с топологией звезда.

Протокол параллельной передачи данных, используемый в PCI, накладывал ограничения на ширину полосы пропускания и частоту работы шины; последовательная передача данных, примененная в PCI Express, обеспечивает возможность масштабирования (в спецификациях описываются реализации PCI Express 1x, 2x, 4x, 8x, 16x и 32x). На данный момент актуальной является версия шины с индексом 4.0<sup>1</sup>.

Ниже, на Рис.408 (рисунок уже приводился ранее) приведена фотография материнской платы с разъёмами PCI-E разной «разрядности».

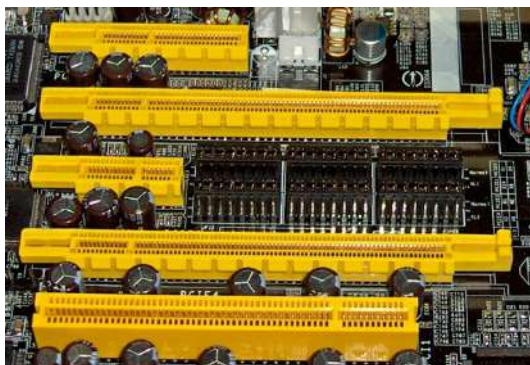


Рис.408 - (сверху вниз): x4 PCI Express, x16 PCI Express, x1 PCI Express, x16 PCI Express, стандартный 32-разрядный слот PCI

Устройства PCI Express взаимодействуют между собой через среду, образованную коммутаторами, при этом каждое устройство напрямую связано соединением типа точка-точка с коммутатором.

Кроме того, шиной PCI Express поддерживается:

---

<sup>1</sup> Компания AMD, с выпуском процессоров Ryzen 3000 (осень 2019), вывела на рынок платформы с версией PCI Express 4.0, у Intel решений с PCIe v4.0 появились только в 2021 году. В мае 2019 года была принята окончательная спецификация стандарта PCI Express 5.0, скорость передачи данных по шине PCI Express в котором составит 32 GT/s. Финальная спецификация стандарта PCI Express 6.0 планируется к публикации в 2021 году. Предполагаемая скорость передачи данных составит 32 Гбайт/с для 4 линий и 128 Гбайт/с для 16 линий.

- горячая замена карт;
- гарантированная полоса пропускания (QoS);
- управление энергопотреблением;
- контроль целостности передаваемых данных.

Шина PCI Express нацелена на использование только в качестве локальной шины. Так как программная модель PCI Express во многом унаследована от PCI, то существующие системы и контроллеры могут быть доработаны для использования шины PCI Express заменой только физического уровня, без кардинальной переработки программного обеспечения.

### ***Описание протокола***

Для подключения устройства PCI Express используется двунаправленное последовательное соединение типа точка-точка, называемое линией (lane — полоса, ряд).

Соединение (link — связь, соединение) между двумя устройствами PCI Express состоит из одной (x1) или нескольких (x2, x4, x8, x16 и x32) двунаправленных последовательных линий. Каждое устройство должно поддерживать соединение, по крайней мере, с одной линией (x1).

На электрическом уровне каждое соединение использует низковольтную дифференциальную передачу сигнала (LVDS), приём и передача информации производится каждым устройством PCI Express по отдельным двум проводникам, таким образом, в простейшем случае устройство подключается к коммутатору PCI Express всего лишь четырьмя проводниками.

Использование подобного подхода имеет следующие преимущества:

- карта PCI Express помещается и корректно работает в любом слоте той же или большей пропускной способности (например, карта x1 будет работать в слотах x4 и x16);
- слот большего физического размера может использовать не все линии (например, к слоту x16 можно подвести проводники передачи информации, соответствующие x1 или x8, и всё это будет нормально функционировать; однако при этом необходимо подключить все проводники питания и заземления, необходимые для слота x16).

В обоих случаях на шине PCI Express будет использоваться максимальное количество линий, доступных как для карты, так и для слота. Однако это не позволяет устройству работать в слоте, предназначенном для карт с меньшей пропускной способностью шины PCI Express. Например, карта x4 физически не поместится в стандартный слот x1, несмотря на то, что она могла бы работать в слоте x1 с использованием только одной линии. На некоторых материнских платах можно встретить нестандартные слоты x1 и x4, у которых отсутствует крайняя перегородка, таким образом, в них можно устанавливать карты большей длины, чем разъём. При этом не обеспечивается питание и заземление выступающей части карты, что может привести к различным проблемам.

Во всех высокоскоростных последовательных протоколах (например, гигабитный Ethernet) информация о синхронизации встроена в передаваемый сигнал. На физическом уровне PCI Express использует метод канального кодирования 8b/10b (8 полезных бит из десяти, избыточность — 20%) для устранения постоянной составляющей в передаваемом сигнале и для встраивания информации о синхронизации в поток данных. Начиная с версии PCI Express 3.0 используется более экономное кодирование 128b/130b с избыточностью 1.5%.

### ***Пропускная способность***

PCIe является полнодуплексным протоколом, т.е., потоки приёма и передачи имеют независимые каналы и одинаковые максимальные скорости. Скорость компьютерных шин принято выражать в гигабайтах в секунду. За 1 транзакцию передаётся одно кодовое слово. Для расчёта пропускной способности 1 линии шины необходимо учесть кодировку 8b/10b (англ. 8b/10b encoding) (в каждом 10 битах 8 информационных) (для PCI-E 3.0 и выше — 128b/130b (128b/130b encoding)). Например, пропускная способность линии PCIe 1.0 составляет:

$$2.5 \text{ ГТ/с} \times 8/10 \text{ бит/Т} = 2 \text{ Гбит/с} = 2 \times 10^9 \text{ бит/с} = 250 \text{ МБайт/с}$$

Несмотря на то, что стандарт допускает x32 линий на порт, такие решения физически достаточно громоздки для прямой реализации, и выпускаются только в проприетарных разъёмах.

| Год выпуска            | Версия PCI Express | Кодирование      | Скорость передачи одной линии | Пропускная способность на x линий |               |               |               |                |
|------------------------|--------------------|------------------|-------------------------------|-----------------------------------|---------------|---------------|---------------|----------------|
|                        |                    |                  |                               | x1                                | x2            | x4            | x8            | x16            |
| 2002                   | 1.0                | 8b/10b           | 2,5 ГТ/с                      | 250 Мбайт/с                       | 0,50 Гбайт/с  | 1,0 Гбайт/с   | 2,0 Гбайт/с   | 4,0 Гбайт/с    |
| 2007                   | 2.0                | 8b/10b           | 5 ГТ/с                        | 500 Мбайт/с                       | 1,0 Гбайт/с   | 2,0 Гбайт/с   | 4,0 Гбайт/с   | 8,0 Гбайт/с    |
| 2010                   | 3.0                | 128b/130b        | 8 ГТ/с                        | 984,6 Мбайт/с                     | 1,97 Гбайт/с  | 3,94 Гбайт/с  | 7,88 Гбайт/с  | 15,8 Гбайт/с   |
| 2017                   | 4.0                | 128b/130b        | 16 ГТ/с                       | 1,969 Гбайт/с                     | 3,94 Гбайт/с  | 7,88 Гбайт/с  | 15,75 Гбайт/с | 31,5 Гбайт/с   |
| 2019 <sup>[6][7]</sup> | 5.0                | 128b/130b        | 32 ГТ/с                       | 3,938 Гбайт/с                     | 7,88 Гбайт/с  | 15,75 Гбайт/с | 31,51 Гбайт/с | 63,02 Гбайт/с  |
| 2021 <sup>[8]</sup>    | 6.0                | 128b/130b, PAM-4 | 64 ГТ/с                       | 7,877 Гбайт/с                     | 15,75 Гбайт/с | 31,51 Гбайт/с | 63,02 Гбайт/с | 126,03 Гбайт/с |

Рис.409 - Пропускная способность PCI Express, Гбайт/с

Начиная с поколения Sandy Bridge, основным «поставщиком» линий (lanes) PCI Express в системе является процессор. В предыдущем поколении (Nehalem) из северного моста на процессорный кристалл перебрался только контроллер памяти, и интерфейс PCIe обеспечивался северным мостом (например, ЮН x58, связанный с процессором шиной QPI, по сути был большим контроллером PCIe 2.0). Сейчас обычно линии PCI-E обеспечиваются частично процессором, частично чипсетом. Сегодняшний рекордсмен по количеству lanes на чип – AMD Ерус v2 – имеет 128 линий PCIe 4.0. Более чем внушительная цифра. В топовых интеловских десктопных процессорах Core серии X десятого поколения эта цифра поскромнее – 48 lanes.

## Шина QPI

**Intel QuickPath Interconnect (QuickPath, сокр. QPI, ранее Common System Interface, CSI)** — последовательная кэш-когерентная<sup>1</sup> шина типа точка-точка, разработанная компанией Intel для соединения процессоров в многопроцессорных системах и для передачи данных между процессором и чипсетом. QPI создавалась в ответ на разработанную ранее консорциумом во главе с компанией AMD шину HyperTransport.

Шина QuickPath была создана для замены применявшейся ранее шины *Front Side Bus*, которая осуществляла связь между центральным процессором и северным мостом материнской платы. Первые процессоры с интерфейсом QuickPath были выпущены на рынок в 2008 году. По состоянию на начало 2010 года, внешний интерфейс QuickPath использовался только в сериях процессоров Xeon и Core i7 с ядром Nehalem для разъёма LGA1366, а также в следующем поколении Itanium (ядро Tukwila). При этом чипсеты для разъёма LGA1366 используют шину DMI для связи между северным и южным мостом. Процессоры для разъёма LGA1156 не имеют внешнего интерфейса QuickPath, поскольку чипсеты для

<sup>1</sup> **Когерентность кэша (cache coherence)** — свойство кэш-ей, означающее целостность данных, хранящихся в локальных кэшах для разделяемого ресурса. Когерентность кэш-ей — частный случай когерентности памяти.

данного разъёма поддерживают только однопроцессорную конфигурацию, а функциональность северного моста встроена в сам процессор (и, следовательно, для связи процессора с аналогом южного моста используется шина DMI). Однако внутри процессора LGA1156 связь между ядрами и встроенным контроллером PCIe осуществляется через встроенную шину QuickPath.

Каждое соединение шины QuickPath состоит из пары односторонних каналов, каждый из которых физически реализован как 20 дифференциальных пар проводов. Данные передаются в виде пакетов (дейтаграмм).

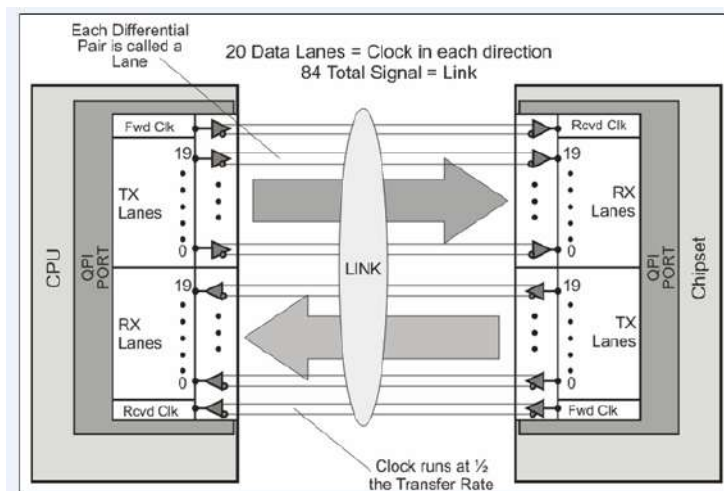


Рис.410 – Физический уровень шины QPI.

Пропускная способность одного канала составляет от 4.8 до 6.4 GT/s (гигатранзакций в секунду). Одна передача содержит 16 бит полезной нагрузки, следовательно, теоретическая суммарная пропускная способность одного соединения (в двух направлениях) — от 19.2 до 25.6 гигабайт в секунду (то есть от 9.6 до 12.8 гигабайт/с в каждую сторону); при этом один процессор может иметь несколько соединений, например, как представлено на Рис.411 – каждый процессор имеет два QPI-соединения – одно для связи CPU между собой, второе – для связи с ИОН (северный мост).

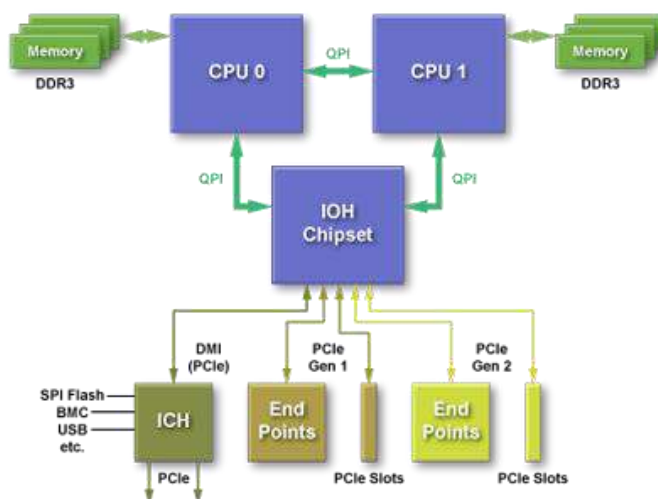


Рис.411 – Шины QPI: процессор-процессор, процессор-ИОН; шина DMI: ИОН-ИОН.

Функции, выполняемые QPI, логически сгруппированы в четыре различных уровня, как показано на Рис.412. Для Intel® QuickPath архитектуры используются три формата, которые идентифицируют степень детализации информации, в обмене между уровнями: *phits*, *пакеты*, *кадры*.



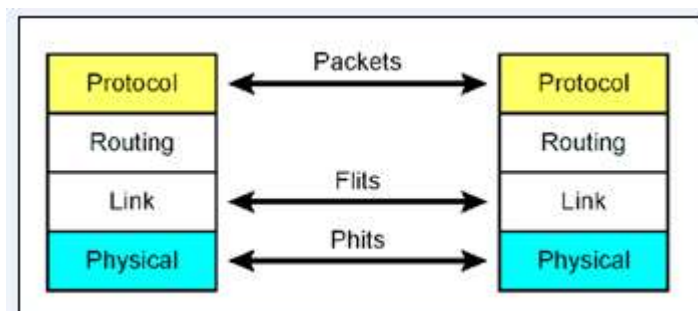


Рис.412 - Архитектурные уровни шины Intel® QuickPath

Модульный подход обеспечивает расширение возможностей шины, которые будут необходимы для удовлетворения всё возрастающих потребностей компонентов системы (например, описание физического уровня – сегодня он обеспечивает определенный диапазон эксплуатационной скорости и особенности подключенных к сети каналов, но это не ограничивает QPI в развитии в сторону увеличения скорости, использования новых каналов коммуникаций, или новых типов используемых сообщений).

## Шина DMI

**Direct Media Interface (DMI)** — последовательная шина, разработанная компанией Intel, для соединения южного моста (ICH) материнской платы с северным (MCH или GMCH) (Рис.411) или с центральным процессором в случае интегрированного контроллера памяти. Впервые DMI использована в чипсетах семейства Intel 915 с южным мостом ICH6 в 2004 году. Серверные чипсеты используют похожий интерфейс, называемый *Enterprise Southbridge Interface* (ESI). Позже интерфейс DMI получил два обновления.

### *DMI первого поколения*

Пропускная способность шины DMI первого поколения составляет 2 ГБайт/с, что значительно выше, чем пропускная способность шины Hub Link (266 МБ/с), которая используется для связи между северным и южным мостами в чипсетах Intel 815/845/848/850/865/875. Вместе с этим суммарная полоса пропускания 2 ГБайт/с (по 1 ГБайт/с в каждом направлении) делится между всеми устройствами, подключенными к южному мосту (например, PCI Express x1, PCI, HD Audio, жёсткие диски), так что фактическая доступная скорость будет ниже.

В материнских платах для процессоров с разъёмом LGA1156 (то есть для Core i3, Core i5 и некоторых серий Core i7 и Xeon) и со встроенным контроллером памяти DMI используется для подсоединения чипсета (PCH) непосредственно к процессору. (Процессоры серии Core i7 для LGA1366 подсоединяются к чипсету через шину QPI).

DMI является собственной разработкой Intel. В 2009 году Intel отказалась лицензировать шину DMI фирме Nvidia. Поскольку поддержка DMI встроена в процессоры с ядрами Lynfield и Clarkdale для разъёма LGA1156 и используется для подсоединения к чипсету, Nvidia фактически потеряла право производить чипсеты для большей части новых процессоров Intel.

### *DMI 2.0*

В 2011 году было представлено второе поколение интерфейса, DMI 2.0, в котором скорость передачи данных была увеличена в 2 раза, до 2 ГБайт/с в каждую сторону по DMI 2.0 на базе 4 линий. Данный вариант использовался для соединения центральных процессоров Intel 2011-2015 годов выпуска с микросхемой Platform Controller Hub (PCH), частично заменившей набор из южного и северного мостов.

### *DMI 3.0*

DMI 3.0 был представлен в августе 2015. В третьем поколении скорость обменов была увеличена до 8 GT/s на каждой линии. Интерфейс с 4 линиями позволяет передавать данные со скоростью до 3.93 ГБайт/с между процессором и РСН. Используется в процессорах с микроархитектурой Skylake (варианты с 2 чипами) и чипсетах Intel серии 100, например, Z170.

## М.2

**М.2** (ранее известный как **Next Generation Form Factor** и **NGFF**) — спецификация компактных компьютерных карт расширения и их разъёмов. Был создан в качестве замены формату mSATA и Mini PCI-E, использовавшему физический разъём и размеры модулей Mini PCI-E. Стандарт М.2 допускает более разнообразные размеры модулей, как по ширине, так и по длине. Формат М.2 часто используется для реализации производственных твердотельных накопителей (на базе флэш-памяти, SSD), особенно при использовании в компактных устройствах, таких как ультрабуки и планшеты.

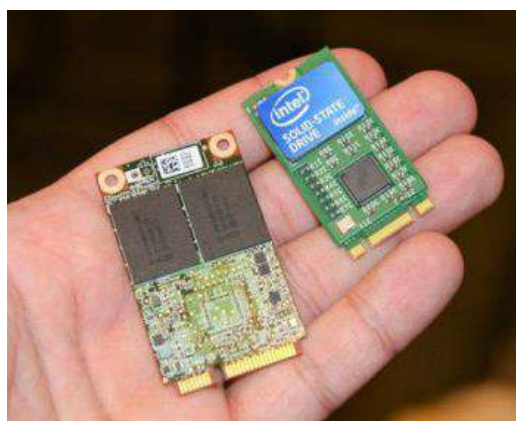


Рис.413 - Сравнение размеров твердотельного флэш-накопителя (SSD) формата mSATA (слева) и формата М.2 (справа; размер 2242, ключ В и М).

Интерфейсы, выведенные на разъём М.2, являются надмножеством интерфейса SATA Express. Фактически, М.2 является более компактной реализацией SATA Express (предоставляет поддержку шин PCI Express 3.0 и SATA 3.0), дополненной внутренним интерфейсом USB 3.0. Платы М.2 могут иметь различные ключевые вырезы для обозначения конкретного варианта используемого интерфейса.

### *Возможности*

Карты расширения М.2 могут предоставлять различный функционал, например: Wi-Fi, Bluetooth, спутниковая навигация, NFC-радиосвязь, цифровое радио, Wireless Gigabit Alliance (WiGig), Wireless WAN (WWAN). В виде модулей М.2 часто изготавливают быстрые и компактные твердотельные флэш-накопители (SSD). На разъём М.2 выводятся шины PCI Express 3.0, Serial ATA 3.0 и USB 3.0 (включая обратную совместимость с USB 2.0). Спецификация SATA 3.2 по состоянию на август 2013 определила формат «SATA М.2» для носителей информации.

В варианте М.2 реализован PCI Express 4x (4 линии) и один порт SATA 3.0 со скоростью до 6 Гбит/с, поэтому в форм-факторе М.2 могут быть реализованы как устройства PCI Express, так и накопители SATA. Используется стандартный PCI Express без каких-либо дополнительных слоёв абстракции. Группа PCI-SIG выпустила спецификацию М.2 версии 1.0 в декабре 2013 года.

Для карт расширения М.2 доступно три варианта реализации логического интерфейса и набора команд, по аналогии со стандартом SATA Express:

#### **«Legacy SATA»**



Используется для SSD с SATA интерфейсом и скоростями до 6.0 Гбит/с (SATA 3.0).

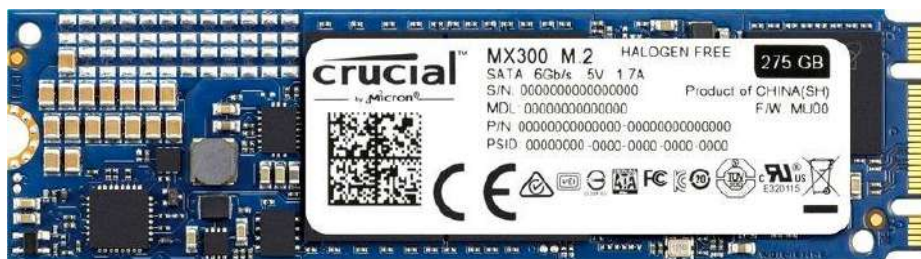


Рис.414 – M.2 SATA (M+V ключ).

### «SATA Express» с использованием AHCI

Используется для SSD с интерфейсом PCI Express и драйвером AHCI (для совместимости с большим количеством операционных систем). Из-за использования AHCI производительность будет несколько ниже оптимальной (получаемой с NVMe), так как AHCI был разработан для взаимодействия с более медленными накопителями с медленным последовательным доступом (например, HDD), а не для SSD с быстрым произвольным доступом.

### «SATA Express» с использованием NVMe

Используется для SSD с интерфейсом PCI Express и высокопроизводительным драйвером NVMe, созданным для работы с быстрыми флэш-накопителями. NVMe был разработан с учётом низких задержек и параллелизма SSD с интерфейсом PCI Express. NVMe лучше использует параллелизм в управляющем компьютере и программном обеспечении, требует меньше стадий при передаче данных, предоставляет более глубокую очередь команд и более эффективную обработку прерываний.



Рис.415 - PCI-Express (M ключ).

SSD-диски могут иметь следующие варианты ключей:



Рис.416 – Варианты ключей SSD-дисков.

## Схемы компьютеров на базе процессоров поколения Nehalem

### Серверные платформы

Базовая схема двухсокетного сервера на основе четырёхъядерного процессора Intel® Xeon® 5500 (Nehalem EP, с поддержкой HyperThreading, 2 потока на ядро) приведена ниже на Рис.417. Некоторые новшества, появившиеся в этом поколении архитектуры:

- Новая подсистема памяти (встроенный в процессор контроллер памяти).
  - DDR3, 1333 МГц, регистровая или небуферизованная;
  - Шесть каналов на два CPU: 2 x 3 канала;
  - Максимальный объём ОЗУ 144 Гбайт (18 модулей по 8 Гбайт);
  - Режим NUMA;
- Шина QPI, используемая для связи CPU-CPU и CPU-чипсет (Intel 5520 Node Manager).
  - Скорость 6.4 GT/s (25.6 Гбайт/с);
  - Задержки уменьшены на 40%;
- Технология управления питанием Intel Intelligent Power.
  - Режим Turbo Boost, прирост:
    - +133 МГц
    - +266 МГц
    - +399 МГц
  - Число переходных состояний: до 15
  - Время перехода: 2 нс
  - Потребление в режиме простоя: 10 Вт
- Микроархитектура
  - До 8 Мбайт кэш L3
  - SSE4.2
  - Виртуализация
    - VT-x2
    - EPT
    - PVID
- Улучшенная подсистема ввода/вывода.

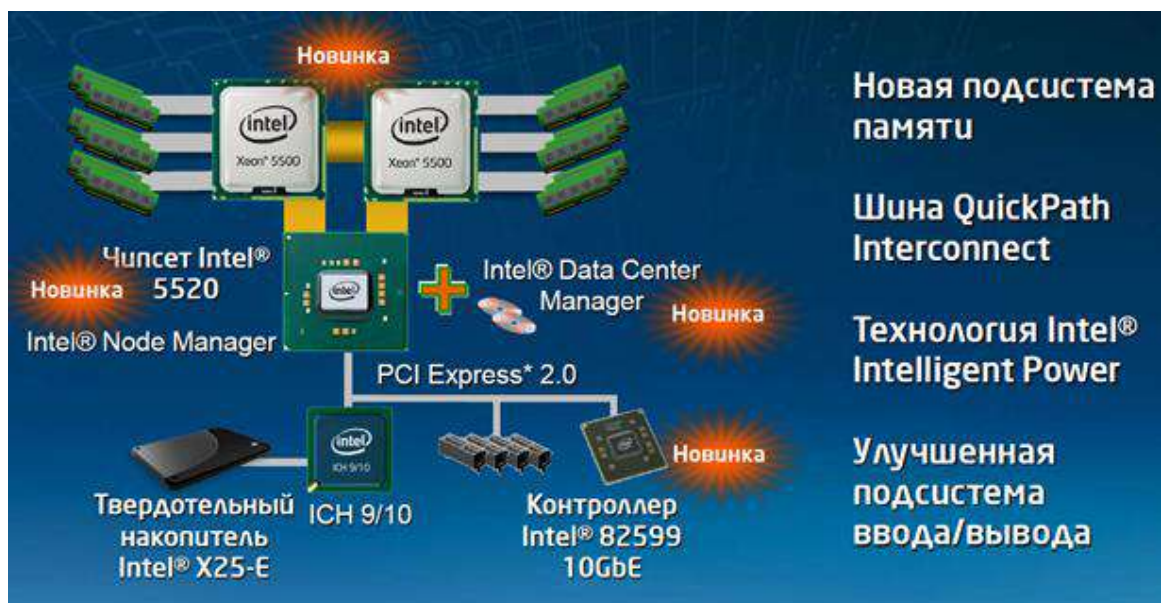


Рис.417 – Двухсокетная система на базе процессоров Intel Xeon 5500.

Поскольку контроллер памяти расположен на кристалле процессора, то двухпроцессорная система будет относиться уже к категории NUMA<sup>1</sup>, а не SMP<sup>1</sup>/UMA<sup>2</sup> – к «своей» памяти процессор доступен с меньшими задержками, чем к памяти, обслуживаемой контроллером второго процессора, к которой он доступен через шину QPI. Разница во времени доступа к «своей»/«чужой» памяти весьма значительная, и программное обеспечение (операционная система, в первую очередь) должно уметь правильно работать с NUMA, оптимально располагая в памяти данные/код так, чтобы каждый процессор работал в основном со «своей» быстрой памятью, а не с медленной «чужой».

В Nehalem поколении линии PCIe обеспечивал не процессор (как это было реализовано в следующем поколении – Sandy Bridge), а северный мост (Рис.417). В этой версии серверной платформы в качестве линка между северным и южным мостами использовался PCIe 2.0.

В 2010 Intel делает шаг «тик» в своей «тик-так» стратегии – переводит микроархитектуру Nehalem на новый технологический процесс: 45 нм → 32 нм, выпустив шестиядерные процессоры Xeon 5600 (Westmere-EP) (до 6 ядер, до двух сокетов, трёхканальный контроллер памяти, до 288 Гбайт ОЗУ).

Также были выпущены Xeon 7500 по 45 нм проектным нормам – до 8 ядер, до 4-8 (в теории – до 256!) сокетов, четырёхканальный контроллер памяти, до 1 Тбайт памяти на 4 сокета, до 2-х Тбайт памяти на 8 сокетов.

Единая архитектура компьютеров на основе этих поколений процессоров представлена на Рис.418 ниже. Из заметных изменений, по сравнению с поколением 5500 – для линка между северным и южным мостами используется шина DMI, а в Xeon 7500 линк между CPU и I/O Hub реализован на шине QPI.



Рис.418 – Архитектура систем на базе Nehalem 2008-2010 года.

<sup>1</sup> **Симметричная multiprocessing** (*Symmetric Multiprocessing*, сокращённо **SMP**) — архитектура многопроцессорных компьютеров, в которой два или более одинаковых процессора сравнимой производительности подключаются единообразно к общей памяти (и периферийным устройствам) и выполняют одни и те же функции (почему, собственно, система и называется симметричной). В английском языке SMP-системы носят также название *tightly coupled multiprocessors*, так как в этом классе систем процессоры тесно связаны друг с другом через общую шину и имеют равный доступ ко всем ресурсам вычислительной системы (памяти и устройствам ввода-вывода) и управляются все одной копией операционной системы.

<sup>2</sup> **Uniform Memory Access** (сокращённо **UMA** — «однородный доступ к памяти») — архитектура многопроцессорных компьютеров с общей памятью. Все процессоры в UMA-архитектуре используют физическую память одновременно. При этом время запроса к данным из памяти не зависит ни от того, какой именно процессор обращается к памяти, ни от того, какой именно чип памяти содержит нужные данные. Однако каждый процессор может использовать свой собственный кэш. Такая схема работы с памятью используется в системах с симметричной multiprocessing (SMP-машинах), поэтому термины UMA и SMP часто используются вместе как «SMP/UMA».

## Десктопные платформы

Топовые десктопные решения поколения Nehalem строились на процессорах с сокетом LGA1366 (Core i7) и на чипсете X58 Express.

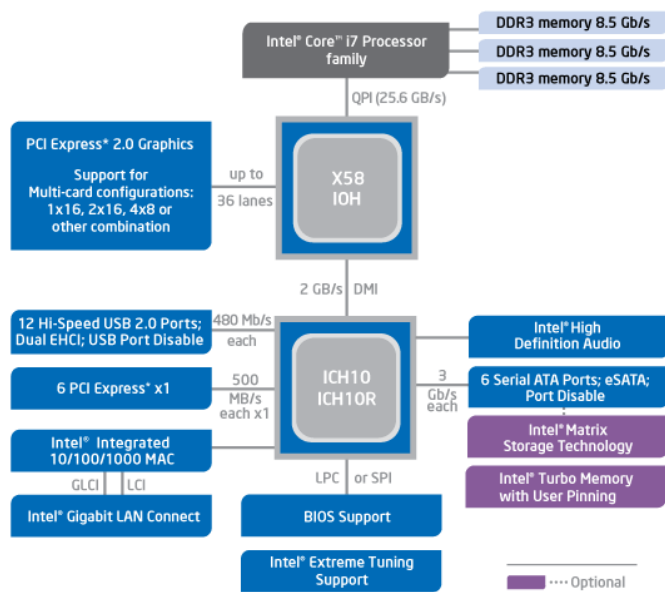


Рис.419 - Блок-схема функциональных возможностей Intel X58 Express

Основные возможности чипсета X58 и систем на его основе:

- поддержка новых процессоров (представленных на момент анонса статьи семействами Core i7 и Core i7 Extreme Edition), основанных на микроархитектуре Nehalem, при подключении к этим процессорам по шине QPI;
- 2 графических интерфейса PCI Express 2.0 x16, с возможностью разбить каждый на два графических интерфейса с половинной скоростью или даже на четыре со скоростью  $\frac{1}{4}$ ;
- дополнительный интерфейс PCI Express 2.0 x 4, с возможностью разбить его на два интерфейса с половинной скоростью;
- шина DMI (с пропускной способностью ~2 ГБ/с) до южного моста ICH10/R.
- Заметим, что вполне традиционно вышедший первым топовый чипсет новой линейки использует старый южный мост, новый появился лишь в момент выхода чипсетов среднего уровня. Функциональность применяемых южных мостов:
  - до 6 портов PCI-E x 1 (PCI-E 1.1);
  - до 4 слотов PCI;
  - 6 портов Serial ATA II на 6 устройств SATA300 (SATA-II, второе поколение стандарта), с поддержкой режима AHCI и функций вроде NCQ (у ICH10 работоспособность этого режима гарантирована только под Windows Vista), с возможностью индивидуального отключения, с поддержкой eSATA и разветвителей портов;
  - возможность организации RAID-массива (только для ICH10R) уровней 0, 1, 0+1 (10) и 5 с функцией Matrix RAID (один набор дисков может использоваться сразу в нескольких режимах RAID — например, на двух дисках можно организовать RAID 0 и RAID 1, под каждый массив будет выделена своя часть диска);
  - 12 устройств USB 2.0 (на двух хост-контроллерах EHCI) с возможностью индивидуального отключения;



- MAC-контроллер Gigabit Ethernet и специальный интерфейс (LCI/GLCI) для подключения PNY-контроллера (i82567 для реализации Gigabit Ethernet, i82562 для реализации Fast Ethernet);
- поддержка Intel Turbo Memory (только для ICH10R);
- High Definition Audio (7.1);
- обвязка для низкоскоростной и устаревшей периферии.

## Схемы компьютеров на базе процессоров поколения Sandy Bridge

В очередной раз повторим утверждение – неотъемлемая часть новой *платформы* – чипсет. Выход в 2011 году процессоров Sandy Bridge (шаг «так», новая микроархитектура на предыдущей 32 нм технологии, после стадии «тик» Nehalem (Westmere)), был сопровождён выходом чипсетов 6-ой серии.

Новые чипсеты примерно сохранили (немного расширив) прежний уровень функциональности, но архитектурно это был по-прежнему один единственный мост с малым потреблением энергии, подключаемый к процессору по шине DMI (только теперь новой версии), а также по специальному интерфейсу FDI в случае «интегрированного» чипсета, и главной задачей которого является обеспечить функционирование основной массы периферийных устройств, подключаемых к материнской плате. Ниже представлена схема компьютеров на основе чипсета P67 (Рис.420), без поддержки интегрированной в *процессор* графики и H67 (Рис.421), с поддержкой Intel HD Graphics.

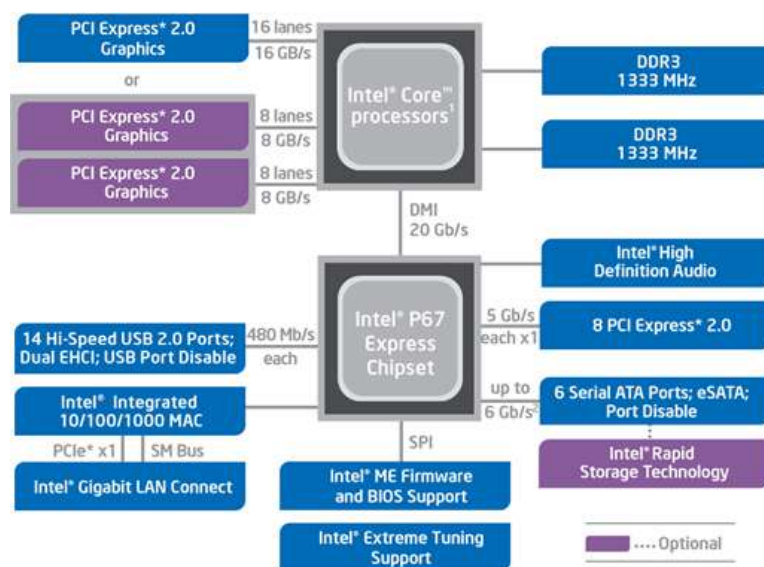


Рис.420 - Блок-схема функциональных возможностей Intel P67

Из особенностей чипсета P67 можно отметить отсутствие в нём поддержки шины PCI (тем самым де-факто признанной Intel окончательно устаревшей) и, удивительное отсутствие поддержки уже, напротив, одного из самых «свежих» на то время интерфейсов – USB 3.0, стандарт на который был принят за полтора года до выхода Sandy Bridge. Обычная практика производителей материнских плат на Sandy Bridge + 67-ой чипсет – установка дополнительных внешних контроллеров, обеспечивающих поддержку 1-2 слотов PCI и нескольких портов USB 3.0.

Основные возможности чипсета P67 и систем на его основе:

- поддержка всех новых процессоров на ядре Sandy Bridge при подключении к этим процессорам по шине DMI 2.0 (с пропускной способностью  $\approx 4$  ГБ/с);
- до 8 портов PCI-E x 1 (полноценные PCI-E 2.0);

- 2 порта Serial ATA III на 2 устройства SATA600 и 4 порта Serial ATA II на 4 устройства SATA300, с поддержкой режима AHCI и функций вроде NCQ, с возможностью индивидуального отключения, с поддержкой eSATA и разветвителей портов;
- возможность организации RAID-массива уровней 0, 1, 0+1 (10) и 5 с функцией Matrix RAID (один набор дисков может использоваться сразу в нескольких режимах RAID – например, на двух дисках можно организовать RAID 0 и RAID 1, под каждый массив будет выделена своя часть диска);
- 14 устройств USB 2.0 (на двух хост-контроллерах EHCI) с возможностью индивидуального отключения;
- MAC-контроллер Gigabit Ethernet и специальный интерфейс (LCI/GLCI) для подключения PHY-контроллера (i82579 для реализации Gigabit Ethernet, i82562 для реализации Fast Ethernet);
- High Definition Audio (7.1);
- обязанка для низкоскоростной и устаревшей периферии.

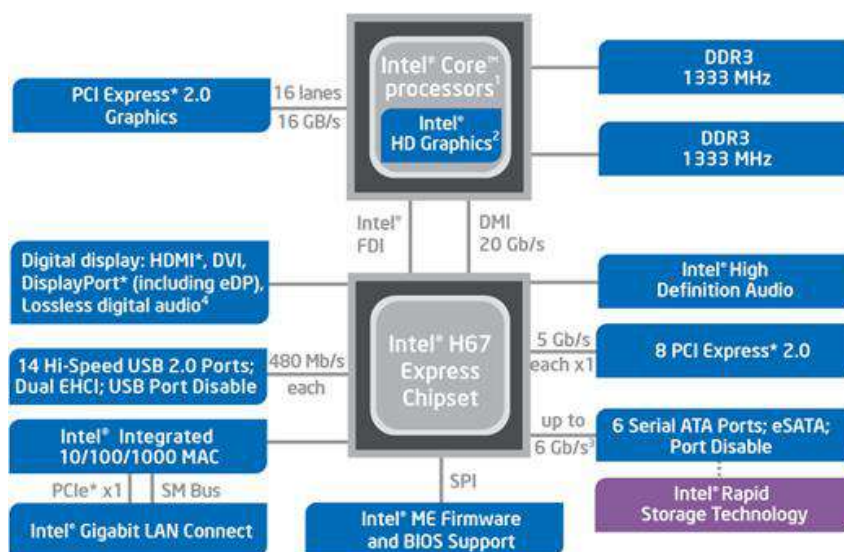


Рис.421 - Блок-схема функциональных возможностей Intel H67

Основные возможности чипсета H67 и систем на его основе:

- поддержка всех новых процессоров на ядре Sandy Bridge при подключении к этим процессорам по шине DMI 2.0 (с пропускной способностью  $\approx 4$  Гб/с);
- интерфейс FDI для получения полностью отрисованной картинки экрана от процессора и блок вывода этой картинки на устройство(-а) отображения;
- до 8 портов PCI-E x 1 (полноценные PCI-E 2.0);
- 2 порта Serial ATA III на 2 устройства SATA600 и 4 порта Serial ATA II на 4 устройства SATA300, с поддержкой режима AHCI и функций вроде NCQ, с возможностью индивидуального отключения, с поддержкой eSATA и разветвителей портов;
- возможность организации RAID-массива уровней 0, 1, 0+1 (10) и 5 с функцией Matrix RAID (один набор дисков может использоваться сразу в нескольких режимах RAID – например, на двух дисках можно организовать RAID 0 и RAID 1, под каждый массив будет выделена своя часть диска);
- 14 устройств USB 2.0 (на двух хост-контроллерах EHCI) с возможностью индивидуального отключения;

- MAC-контроллер Gigabit Ethernet и специальный интерфейс (LCI/GLCI) для подключения PHY-контроллера (i82579 для реализации Gigabit Ethernet, i82562 для реализации Fast Ethernet);
- High Definition Audio (7.1);
- обвязка для низкоскоростной и устаревшей периферии.

## Схема компьютера на базе процессора Core X и чипсета X299

Некоторое время назад буквой «X» компания Intel маркировала самые топовые системные чипсеты для ПК (X58/79/99), а также с подобным суффиксом выходили и самые производительные и дорогие процессоры типа Core i7-900-5900X (для простоты их называли Core X). Затем несколько лет назад в сегменте «интеловских» платформ случилось разделение: материнские платы и чипсеты с «X» стали относить к особому сегменту высокопроизводительных решений (Hi-End Desktop — *HEDT*), отделив их от игровых (и просто общепользовательских), отдав последним букву «Z», поэтому появились продукты уже на Z-серии (Z170, Z370, Z390), и для них вышли процессоры Core i9-серии. Это всё стало тоже топовой платформой, но для более широкого круга потребителей, в основном геймеров-энтузиастов. Решения же семейства HEDT (с «X») имели не только свои чипсеты, но и отдельный «экстремальный» сокет LGA2066<sup>1</sup>, и отличались весьма и весьма высокими ценами – и процессоры Core X стоили в разы дороже массовых процессоров, и материнские платы на чипсете X299 – самые дорогие из всего спектра десктопных «материнок» (уступая в цене разве что продвинутым серверным платформам).

В категорию решений HEDT несколько лет назад довольно агрессивно вторглась AMD со своими Ryzen Threadripper. Выход Ryzen Threadripper 3000 настолько шокировал Intel, что она пошла на совсем уж отчаянный шаг, выпустив Core X серии 10000X, назначив на эти процессоры цены даже ниже, чем у предыдущих 9000X, тем самым фактически просто «убивая» свою предыдущую линейку.

---

<sup>1</sup> **LGA2066** (Socket R4) — разъём для процессоров компании Intel, поддерживающий процессоры архитектуры Skylake-X и Kaby Lake-X без интегрированного графического ядра. Разработан в качестве замены разъёма LGA2011/2011-3 (Socket R/R3) для высокопроизводительных настольных ПК на платформе *Basin Falls* (набор системной логики X299) и однопроцессорных серверов и рабочих станций (набор логики C422), в то время как LGA3647 (Socket P) заменит LGA2011-1/2011-3 (Socket R2/R3) в многопроцессорных серверных платформах и станциях на базе Skylake-EX (Xeon «Purley»).



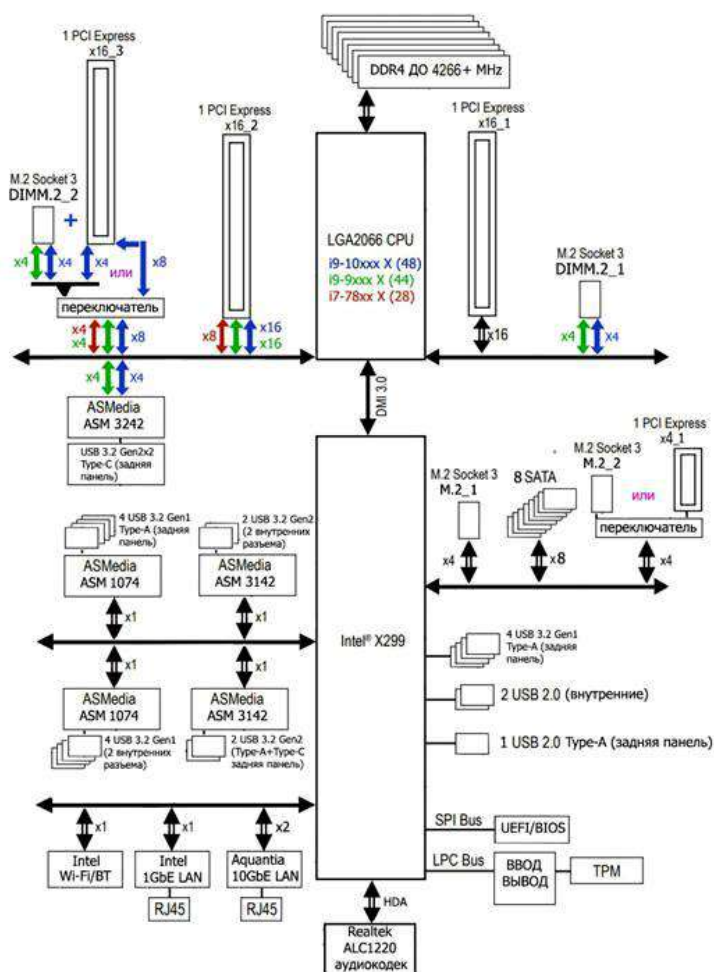


Рис.422 - Блок-схема чипсета X299

Чипсет X299 можно считать ветераном-долгожителем<sup>1</sup>, на сегодня это едва ли не единственный пример, когда один и тот же набор микросхем поддерживает процессоры нескольких (на сегодня уже трёх) поколений – Intel Core X 7x, 9x и 10x. Для серии Core X Intel не стала выпускать новый чипсет под каждый новый процессор (если откровенно, об этом уже говорилось выше – серьёзных архитектурных различий в процессорах последних поколений было не так уж и много, ну не считать же увеличение количества lanes в CPU с 44 до 48 при переходе от 9x к 10x значительным изменением), X299 был спроектирован с хорошим запасом по характеристикам и отлично сочетается с CPU всех поколений на протяжении более трёх лет.

Ранее, в таблице на Рис.366 («Перечень функциональных особенностей Asus ROG Rampage VI Extreme Encore»), уже были приведены подробные данные по этой материнской плате, основанной на чипсете X299. Ниже будут повторно приведены некоторые важнейшие характеристики системы в части поддержки шин, интерфейсов, объёмов ОЗУ. Суммарно по количеству портов ввода-вывода tandem X299+Core X предоставляет:

- 28/44/48 PCI-E 3.0 линий для видеокарт или иной периферии с PCI-E интерфейсом (от процессора);
- суммарно 14 портов USB, внутри которых до 10 портов USB 3.2 Gen1 (Gen2 не поддерживается), до 14 портов USB 2.0 (от чипсета);
- 8 портов SATA 6Гбит/с (от чипсета);

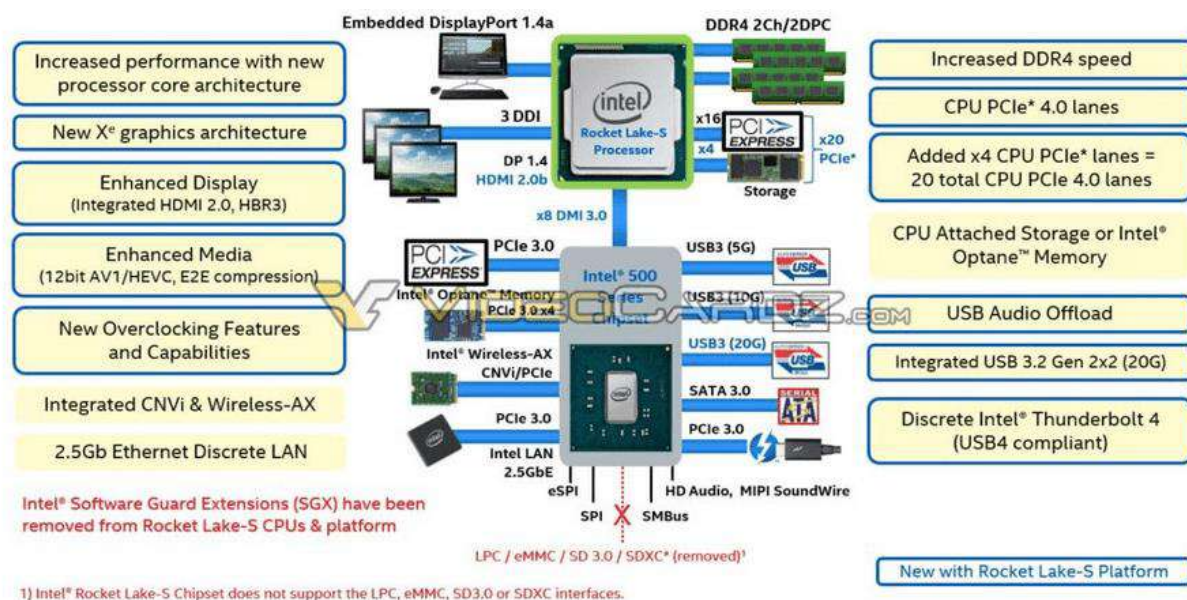
<sup>1</sup> Чипсет X299 был выпущен в июне 2017 года, одновременно с выходом процессора Intel Core i9-7900X.

- 24 порта PCI-E 3.0 (от чипсета);
- Память: 8 × DDR4, до 256 ГБайт, до DDR4-4266 (XMP), четыре канала.

## Схема компьютера на базе процессора Intel Rocket Lake-S и нового чипсета 500-ой серии

Как и предполагалось ранее, вместе с процессорами *Rocket Lake-S* компания Intel выпустит новые микросхемы системной логики 500-й серии, а производители представят новые материнские платы на их основе. Пока<sup>1</sup> неизвестно, будут ли *Rocket Lake-S* работать с платами на чипсетах Intel 400-й серии, который выйдут уже совсем скоро вместе с *Comet Lake-S*. Также остаётся загадкой, смогут ли *Comet Lake-S*, а свою очередь, работать с чипсетами Intel 500-й серии. Всё же две платформы могут довольно сильно отличаться.

Известно, что Rocket Lake-S станут носителями новой процессорной архитектуры, правда, непонятно, какой именно. Прежние слухи указывали, что это может быть адаптация под 14 нм техпроцесс архитектуры *Willow Cove*, которая также будет использоваться в 10 нм мобильных *Tiger Lake*. В любом случае радует то, что Intel наконец-то предложит в настольном сегменте что-то отличное от *Skylake*.



Поменяется встроенная графика. Процессоры *Rocket Lake-S* получат интегрированный ускоритель с архитектурой двенадцатого поколения, также известной как *Intel Xe*. Это является ещё одним подтверждением родства процессоров *Rocket Lake* с *Tiger Lake*. Новые «вставки» обеспечат поддержку интерфейсов HDMI 2.0b и DisplayPort 1.4a.

Следующей важной особенностью новой настольной платформы Intel станет поддержка интерфейса PCI Express 4.0. Процессоры *Rocket Lake-S* будут обладать двадцатью линиями PCIe 4.0, из которых шестнадцать предназначены для подключения видеокарты, а ещё четыре — для твердотельного накопителя. То есть основной GPU и основной SSD будут напрямую подключаться к процессору. Сейчас у Intel массовые настольные CPU имеют лишь 16 линий PCIe.

Чипсет 500-й серии будет подключаться к CPU посредством восьми линий DMI 3.0, вместо нынешних четырёх, что даст вдвое более высокую пропускную способность. К чипсету также смогут подключаться SSD-накопители и карты расширения, но уже посредством 24 линий PCIe 3.0. Ещё системная логика Intel 500-й серии сможет предложить поддержку Thunderbolt 4, USB 3.2 2x2 и 2.5-гигабитных сетевых контроллеров, а также встроенный

<sup>1</sup> Данные конца марта 2020 года.

беспроводной контроллер Wi-Fi 6. Также Intel уберёт из платформы *Rocket Lake-S* поддержку технологии *Software Guard Extensions (SGX)*.

Прежние слухи указывали на то, что процессоры *Rocket Lake-S* должны быть представлены уже к концу текущего (2020) года. Однако задержка *Comet Lake-S*, а также неопределённость, вызванная пандемией коронавируса, может внести некоторые коррективы в эти планы. Похоже, что раньше следующего года (2021) ждать новое поколение настольных процессоров Intel не стоит.

## Прерывания от внешних устройств в системе x86<sup>1</sup>

В этом разделе будут рассмотрены механизмы доставки прерываний от внешних устройств в x86, и эволюция этих механизмов (контроллеров), начиная от систем на основе i8086/88, до самых современных. Именно поэтому это рассмотрение было размещено в последней главе, посвящённой архитектурам ЭВМ на интеловских x86-процессорах.

Напомним определение прерывания цитатой из википедии:

***Прерывание** (interrupt) — сигнал от программного или аппаратного обеспечения, сообщающий процессору о наступлении какого-либо события, требующего немедленного внимания. Прерывание извещает процессор о наступлении высокоприоритетного события, требующего прерывания текущего кода, выполняемого процессором. Процессор отвечает приостановкой своей текущей активности, сохраняя свое состояние и выполняя функцию, называемую обработчиком прерывания (или программой обработки прерывания), которая реагирует на событие и обслуживает его, после чего возвращает управление в прерванный код.*

В зависимости от источника возникновения сигнала прерывания делятся на:

- *асинхронные, или внешние (аппаратные) — события, которые исходят от внешних аппаратных устройств (например, периферийных устройств) и могут произойти в любой произвольный момент времени: сигнал от таймера, сетевой карты или дискового накопителя, нажатие клавиши клавиатуры, движение мыши. Факт возникновения в системе такого прерывания трактуется как запрос на прерывание (англ. Interrupt request, IRQ) - устройства сообщают, что они требуют внимания со стороны ОС;*
- *синхронные, или внутренние — события в самом процессоре как результат нарушения каких-то условий при исполнении машинного кода: деление на ноль или переполнение стека, обращение к недопустимым адресам памяти или недопустимый код операции;*
- *программные (частный случай внутреннего прерывания) — инициируются исполнением специальной инструкции в коде программы. Программные прерывания, как правило, используются для обращения к функциям встроенного программного обеспечения (firmware), драйверов и операционной системы.*

Термин «ловушка» (*trap*) иногда используется как синоним термина «прерывание» или «внутреннее прерывание». Как правило, словоупотребление устанавливается в документации производителя конкретной архитектуры процессора.

Далее речь пойдёт только о работе с внешними прерываниями (IRQ) в архитектуре Intel x86.

Количество входов запроса прерывания в типичных процессорах невелико, обычно всего один/два. Второй вход обслуживал т.н. немаскируемые (NMI) прерывания, которые требовали незамедлительной и максимально оперативной реакции при возникновении запроса, например, на сигнал, сообщающий о пропадании напряжения питания.

---

<sup>1</sup> Материал раздела основан на статье [Прерывания от внешних устройств в системе x86. Часть 1. Эволюция контроллеров прерываний](#).

Для решения проблемы «один вход IRQ – несколько устройств, генерирующих запросы прерывания» Intel предложила использовать в самых ранних своих архитектурах (ещё на базе процессора i8080) программируемый контроллер прерываний (PIC).

## PIC

Первой была микросхема PIC Intel 8259. Восемь входных линий (IRQ0...7), и одна выходная, соединяющая контроллер с входом INTR процессора. Когда возникало прерывание от какого-либо устройства, 8259 активировало линию INTR, сигнализируя процессору, что какое-то устройство запрашивает прерывание. Процессор опрашивает PIC (читает регистр входов запросов прерывания), чтобы понять на каком именно выводе IRQx возникло прерывание. Появляется дополнительная задержка на этот опрос, но зато количество линий прерываний увеличивается до 8.

Первоначальная версия контроллера (8259) была предназначена только для процессоров i8080 и i8085. По сигналу подтверждения прерывания она выдавала на шину три байта, первый соответствовал коду операции «Вызов подпрограммы» (CALL) (значение 0xCD в шестнадцатеричной системе), затем два байта адреса вызова этой подпрограммы, которые нужно было записать в соответствующие каждому источнику прерывания внутренние регистры контроллера во время его инициализации. Т.е., по сути при обработке CPU не делал callback (обратный вызов), а просто выполнял команду вызова подпрограммы (CALL), которую ему таким способом «подставил» PIC.

Для процессоров семейства x86 была выпущена микросхема 8259A, так как механизм передачи управления изменился. Теперь на шину выдавались последовательно два байта, первый был «холостым», его значение микропроцессор игнорировал, во втором байте был номер вектора прерывания, а адрес, по которому следовало передавать исполнение программы, брался из таблицы векторов прерываний, размещённой в младших адресах ОЗУ, начиная с 0. У процессоров 8086, 80186 и 8088 эта таблица занимала самый нижний килобайт адресного пространства памяти. У более поздних, начиная с 80286, таблица векторов прерываний могла быть перемещена в любое место памяти.

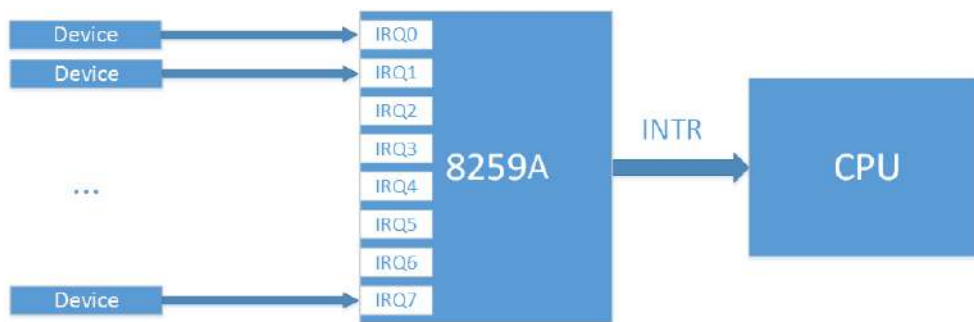


Рис.423 – Организация прерываний с использованием PIC i8259A

Выше, в разделе, посвящённом архитектурам на основе CPU i80286 (PC/AT), уже упоминался основной недостаток такого решения – из-за практической невозможности подключить ко входу IRQ более чем один источник запроса, система обладала плохой масштабируемостью – восемь входов это всё-таки не очень много.

Там же (в разделе, посвящённом PC/AT) было описано решение, которое реализовали разработчики IBM PC/AT – каскадное включение двух PIC 8259A по схеме master-slave:



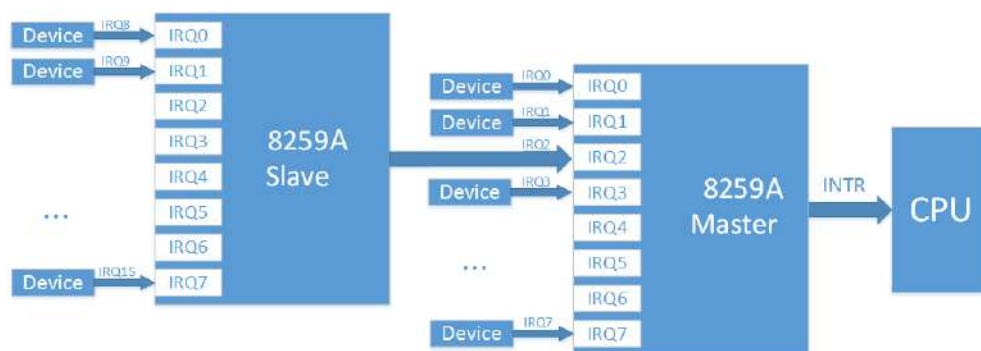


Рис.424 – Организация прерываний с использованием двух PIC i8259A

IRQ с 0 по 7 обрабатываются первым Intel 8259 PIC (master), а IRQ с 8 по 15 – вторым 8259 PIC (slave). О возникновении прерывания CPU сигнализирует только master. Если возникло прерывание на линиях 8...15, второй PIC (slave) сигнализирует о прерывании мастеру по линии IRQ 2, и тот уже в свою очередь сигнализирует CPU (INTR). Каскадное прерывание отнимает одну из 16 линий, что в итоге даёт 15 доступных прерываний для устройств.

Схема утвердилась, и именно её имеют ввиду, когда говорят о PIC (Programm Interrupt Controller). Во времена, когда основной шиной для подключения внешних устройств была шина ISA, такой системы обычно хватало. Надо было лишь следить, чтобы разные устройства не подключались на одну линию IRQ для избежания конфликтов, так как прерывания ISA не разделяемые<sup>1</sup>.

Обычная раскладка прерываний под устройства в PC/AT:

- IRQ 0 — system timer
- IRQ 1 — keyboard controller
- IRQ 2 — cascade (прерывание от slave контроллера)
- IRQ 3 — serial port COM2
- IRQ 4 — serial port COM1
- IRQ 5 — parallel port 2 and 3 or sound card
- IRQ 6 — floppy controller
- IRQ 7 — parallel port 1
- IRQ 8 — RTC timer
- IRQ 9 — ACPI
- IRQ 10 — open/SCSI/NIC
- IRQ 11 — open/SCSI/NIC
- IRQ 12 — mouse controller
- IRQ 13 — math co-processor
- IRQ 14 — ATA channel 1
- IRQ 15 — ATA channel 2

Конфигурация и работа с микросхемами 8259A осуществляется через I/O порты:

<sup>1</sup> На самом деле ISA устройства формально могут разделять линию прерываний, при условии, что они спроектированы с учётом возможности подобного разделения. Но стандарт не делает это требование обязательным, поэтому в целом подключение нескольких ISA устройств к одной линии IRQ не является возможным и безопасным.

| Чип        | Регистр | I/O port |
|------------|---------|----------|
| Master PIC | Command | 0x0020   |
| Master PIC | Data    | 0x0021   |
| Slave PIC  | Command | 0x00A0   |
| Slave PIC  | Data    | 0x00A1   |

На смену шине ISA пришла шина PCI. И количество устройств явно стало превосходить число 15, плюс в отличие от статической шины ISA, в PCI устройства могут добавляться в систему динамически. Но, к счастью, в данной шине прерывания могут быть разделяемыми (то есть к одной линии IRQ можно подсоединить несколько устройств). В итоге чтобы решить проблему нехватки линий IRQ, прерывания от всех PCI устройств решили группировать в линии PIRQ (Programmable Interrupt Request).

Допустим на PIC контроллере есть 4 линии запроса прерываний, а PCI устройств в системе 20 штук. Прерывания объединяются по 5 устройств на линию PIRQx, и линии PIRQx подключаются к контроллеру. При возникновении прерывания на линии PIRQx процессору придётся опросить все устройства подключённые к данной линии, чтобы понять от кого именно пришло прерывание, но в целом это решает задачу. Устройство, осуществляющее связывание линий прерываний PCI в линии PIRQ, часто называют PIR router.

В данном методе надо следить, чтобы линии PIRQx не подсоединялись к линиям IRQx на которых уже заведены прерывания ISA (так как это вызовет конфликты), и чтобы линии PIRQx были сбалансированы (ведь чем больше устройств мы подключили к одной линии PIRQ, тем больше устройств надо будет опрашивать процессору, чтобы понять, какое именно из этих устройств вызвало прерывание).



Рис.425 – Организация прерываний с использованием PIR router

На рисунке Рис.425 отображение (mapping) PCI device → PIR приведено абстрактно, на самом деле оно устроено несколько сложнее. В реальности каждое PCI device имеет 4 линии прерываний (INTA, INTB, INTC, INTD). У каждого PCI устройства (device) может быть до 8 функций (functions) и вот каждой функции соответствует одно прерывание INTx. Какую именно линию INTx будет «дёргать» конкретная функция устройства, определяется конфигурацией чипсета.

По сути, функции – это отдельные логические блоки. Например, в одном PCI устройстве может быть функция Smbus controller, функция SATA controller, функция LPC<sup>1</sup> bridge. Со стороны ОС каждая функция выглядит как отдельное устройство со своим конфигурационным пространством PCI Config.

<sup>1</sup> **Low Pin Count (LPC bus)** — шина, используемая в IBM PC-совместимых персональных компьютерах для подключения устройств, не требующих большой пропускной способности к ЦПУ. К таким устройствам относятся загрузочное ПЗУ и контроллеры устаревших низкопроизводительных интерфейсов передачи данных, такие, как последовательный и параллельные интерфейсы, интерфейс подключения манипулятора «мышь» и клавиатуры, НГМД, а с недавнего времени и устройств хранения криптографической информации. Обычно контроллер шины LPC расположен в южном мосте на материнской плате.

Информацию о маршрутизации (роутинге) прерываний на PIC контроллере BIOS передавал ОС с помощью таблицы \$PIR и с помощью заполнения регистров 3Ch (INT\_LN Interrupt Line (R/W)) и 3Dh (INT\_PN Interrupt Pin (RO)) конфигурационного пространства PCI для каждой функции. Содержимое строк таблицы \$PIR можно понять из [PCI BIOS Specification](#) [4.2.2. Get PCI Interrupt Routing Options].

## APIC

Описанные выше методы работали, пока не появились многопроцессорные системы. По логике своего устройства PIC может передавать прерывания только на один главный процессор. А для многопроцессорной системы очень желательно, чтобы нагрузка на процессоры от обработки прерываний была сбалансированной. Решением данной задачи стал новый интерфейс APIC (Advanced PIC).

Для каждого процессора добавляется специальный контроллер LAPIC (Local APIC) и для маршрутизации прерываний от устройств добавляется контроллер I/O APIC. Все эти контроллеры объединяются в общую шину с названием APIC (новые системы соединяются по стандартной системной шине).

Когда прерывание от устройства приходит на вывод I/O APIC, контроллер направляет прерывание в LAPIC одного из процессоров. Наличие I/O APIC позволяет сбалансировано распределять прерывания от внешних устройств между процессорами.

Первой микросхемой APIC был 82489DX, это был отдельный чип, соединяющий в себе LAPIC и I/O APIC. Для создания системы из двух процессоров нужно было три таких микросхемы. Две функционировали как LAPIC и одна как I/O APIC. Позднее функциональность LAPIC была напрямую включена в процессоры, а функциональность I/O APIC была оформлена в чип 82093AA.

I/O APIC 82093AA содержала 24 входных линии, а архитектура APIC могла поддерживать до 16 CPU. Для поддержки совместимости со старыми системами, прерывания 0...15 отвели под старые прерывания ISA. А прерывания от PCI устройств стали выводить на линии IRQ 16...23, что дало возможность избежать конфликтов прерываний от ISA и PCI устройств. Также благодаря увеличенному количеству свободных линий прерываний возможно стало также увеличить количество линий PIRQx. Следует отметить, что в реальных системах прерывания от legacy ISA всегда заводятся только на CPU0.

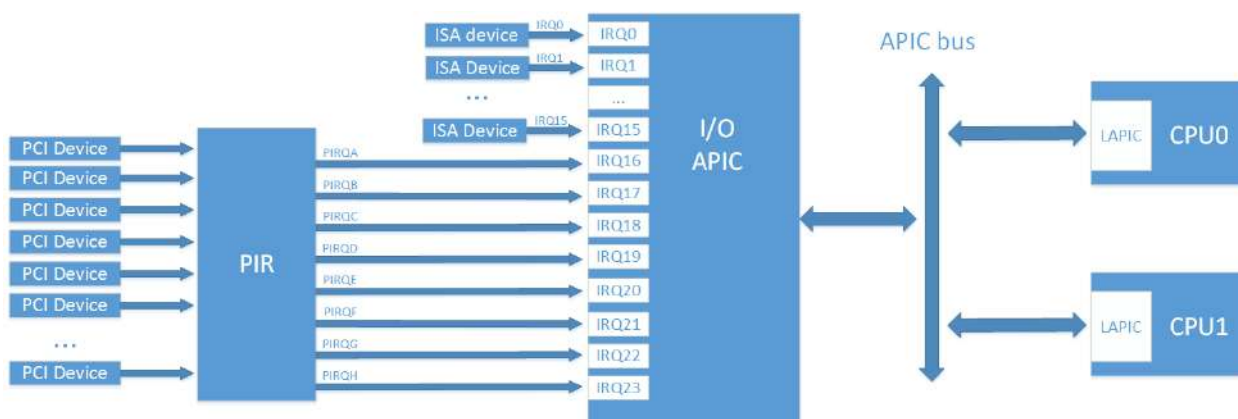


Рис.426 – Организация прерываний с использованием APIC

Программирование I/O APIC и LAPIC осуществляется через ММЮ<sup>1</sup>. Регистры LAPIC расположены обычно по адресу 0xFEE00000, регистры I/O APIC по адресу 0xFEC00000. Хотя в принципе все эти адреса возможно переконфигурировать.

<sup>1</sup> ММЮ – ввод-вывод с отображением на память использует одно и то же адресное пространство для адресации как памяти, так и устройств ввода-вывода.



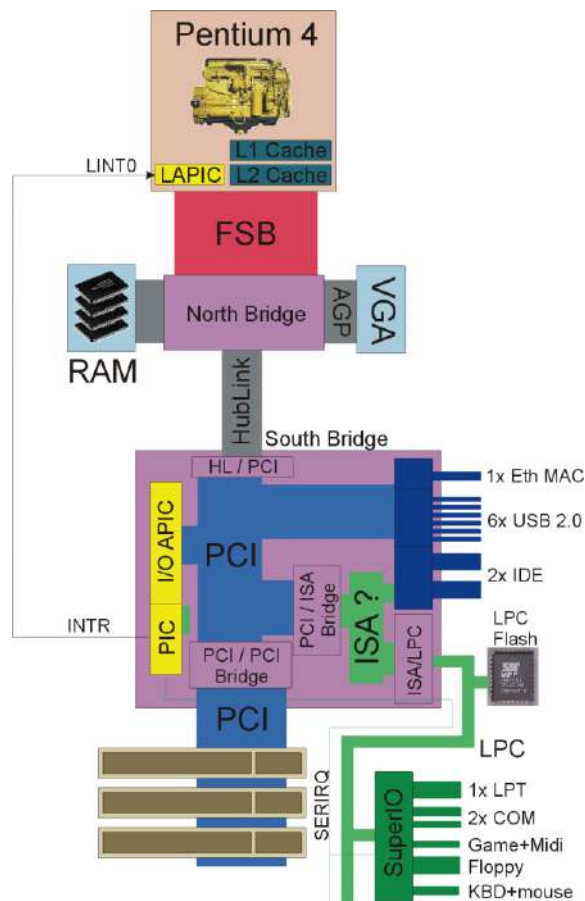


Рис.427 – Пример организации прерываний в системе на основе Pentium 4

Выше, на Рис.427, представлена общая схема организации компьютера на базе CPU Pentium 4 с использованием LAPIC.

Первоначально APIC ввода-вывода были автономными микросхемами, взаимодействующими с LAPIC ЦП через отдельную выделенную «шину APIC». Позже IO-APIC переместились в южный мост чипсета ПК (Рис.427), а в некоторых случаях были включены в автономные мосты PCI. Теперь – это очень важно – передача событий APIC IRQ в восходящем направлении (запрос к процессору) переместилась в основную полосу (“in band”), тогда как раньше запросы передавались во «внеполосном» (“out of band”) стиле. Т.е., с этого момента времени запрос прерывания стал передаваться сообщениями по системной шине, по сути являясь ещё одним «типом транзакции шины».

В дальнейшем архитектура APIC была модифицирована и новый вариант получил название xAPIC (x — extended). Была сохранена обратная совместимость с предыдущим вариантом, количество возможных CPU в системе увеличилось до 256.

Следующий виток развития архитектуры получил название x2APIC. Количество возможных CPU в системе увеличилось до  $2^{32}$ . Контроллеры могут работать в режиме совместимости с xAPIC, а могут работать в новом режиме x2APIC, где программирование LAPIC осуществляется не через MMIO, а через MSR регистры (что выполняется гораздо быстрее). Для работы этого режима необходима поддержка IOMMU<sup>1</sup>.

Следует отметить, что в системе может быть несколько контроллеров I/O APIC. Например, один на 24 прерывания в южном мосту, другой на 32 в северном. В контексте I/O APIC прерывания часто обозначаются GSI (Global System Interrupt). Соответственно, в такой системе будут GSI с номерами 0...55.

<sup>1</sup> **IOMMU** (input/output memory management unit) — блок управления памятью (MMU) для операций ввода-вывода. Подробнее об этом элементе будет рассказано в разделе, посвящённом технологии виртуальных машин.

Понять, есть ли в CPU встроенный LAPIC и какой именно архитектуры, можно битовым флагам в CPUID.

Чтобы система могла обнаружить LAPIC и I/O APIC, BIOS должен представить информацию о них системе, либо через таблицу MPtable (старый метод), либо через таблицу ACPI (таблицу MADT в данном случае). Помимо общей информации, и в MPtable и в ACPI (на этот раз в таблице DSDT) должна содержаться информация о маршрутизации прерываний, то есть информация о том, какое устройство какой линии прерываний соответствует (аналог таблицы \$PIR).

О таблице MPTable можно почитать в официальной спецификации. Раньше спецификация была на сайте Intel, сейчас её можно найти только в архиве. Спецификация ACPI сейчас расположена на сайте UEFI. Следует отметить, что с помощью ACPI можно указать роутинг прерываний и для систем без APIC (вместо использования таблицы \$PIR).

## MSI

Описанный выше вариант обработки прерываний с APIC хорош, но не лишён недостатков. Все эти параллельные линии прерываний от устройств усложняют схему, увеличивают вероятности сбоев (ошибок), плохо масштабируются. И, когда на смену шине PCI пришла PCI express, то в ней линии прерываний решили полностью убрать. Чтобы сохранить обратную совместимость, сигналы о возникновении прерываний (INTx#) эмулируются отдельными видами *сообщений*. В этой схеме логическое ИЛИ линий прерываний, которое раньше производилось физическим соединением проводов, легло на плечи PCI мостов. Однако поддержка legacy INTx прерываний — это лишь поддержка обратной совместимости с шиной PCI. На деле PCI express предложил новый метод доставки сообщений о прерываниях — MSI (Message Signaled Interrupts). В этом методе для сигнализации о прерывании устройство просто производит запись в MMIO область, отведённую под LAPIC процессора. Т.е., вместо активации линии запроса прерывания теперь прерывание инициализируется передачей сообщения по общей шине.

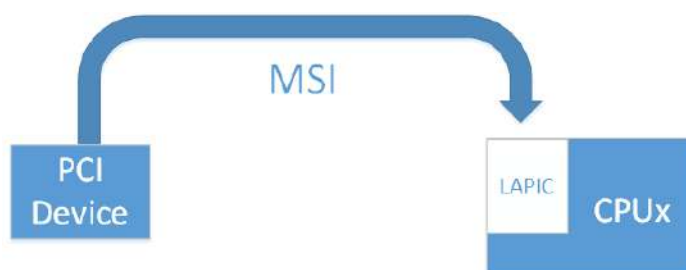


Рис.428 – Организация прерываний с использованием MSI

В результате, если раньше на одно PCI устройство (то есть на *все* его функции) выделялось всего четыре прерывания, то сейчас стало возможным адресовать до 32-х прерываний.

В случае с MSI нет никакого sharing (разделения) для линий, каждое прерывание соответствует своему устройству один к одному.

Прерывания MSI решают также ещё одну проблему. Допустим устройство проводит memopу-write транзакцию, и хочет сообщить о её завершении через прерывание. Но write транзакция может быть задержана на шине в процессе передачи (о чём устройство никак не узнает), и сигнал о прерывании придёт в процессор раньше, чем завершится запись. Таким образом CPU будет читать ещё невалидные данные. В случае, если используется MSI, информация MSI передаётся также, как и данные (по той же шине), и раньше записи она прийти в CPU просто не сможет.

Следует заметить, что прерывания MSI не могут работать без LAPIC, но использование MSI может заменить I/O APIC (упрощение аппаратного дизайна).

Впоследствии данный метод получил расширение, которое назвали MSI-X. Теперь каждое устройство может иметь до 2048 прерываний. И стало возможным указывать индивидуально каждому прерыванию, на каком процессоре оно должно выполняться. Это может быть очень полезно для высоконагруженных устройств, например, сетевых карт.

Для поддержки MSI не требуется никаких дополнительных таблиц BIOS. Но устройство должно сообщить о поддержке MSI в одной из Capabilities в своём PCI Config, а драйвер устройства должен поддерживать работу с MSI.

В статье [Прерывания от внешних устройств в системе x86. Часть 2. Опции загрузки ядра Linux](#) приведены интересные практические примеры работы с прерываниями в современных Linux-системах, при различных сочетаниях опций загрузки ядра. Во второй статье – [Прерывания от внешних устройств в системе x86. Часть 3. Настройка роутинга прерываний в чипсете на примере coreboot](#) – описано, как BIOS настраивает маршрутизацию IRQ в чипсете (на примере Coreboot).

## Программное обеспечение

Поколения Intel Core 2 однозначно можно отнести к эпохе зрелости архитектуры x86. Вовремя прекратив заранее обречённую «гонку за мегагерцами» на микроархитектуре NetBurst, компания Intel, по сути вернувшись к заброшенной было ею архитектуре P6, сумела вернуть себе с помощью поколений Core 2 тогда едва не ускользнувшее от неё лидерство, закрепив свой успех удачными поколениями Nehalem, Sandy Bridge, Ivy Bridge. Практически на десятилетие Intel стала абсолютно доминирующим поставщиком процессоров с архитектурой x86, едва ли не полностью подавив своего основного конкурента – компанию AMD.

## Microsoft, Windows

Программное обеспечение для поколений архитектур этого периода (начиная с Core) также уже совершенно обоснованно можно отнести к зрелому и вполне сформировавшемуся. Очень показателен в этом плане пример, показывающий, за какое время программная индустрия сумела обеспечить полную и адекватную поддержку для 64-битного расширения архитектуры x86, вспоминая, как долго 32-битным процессорам пришлось «ютиться» в тесном 16-битном окружении, работая под управлением ограниченных 16-разрядных ОС класса MS-DOS, Windows 3.x, Windows 95/98/Me. Переход на 64 бита состоялся практически одновременно с массовым появлением x86-64 архитектур на рынке. Уже в 2010 году Microsoft отказалась от разработки 32-разрядных версий своих Windows Server – начиная с Windows Server 2008R2 серверные платформы от Microsoft стали только 64-битными. В мае 2019 года такое же решение приняла и компания Red Hat, номер один в мире ОС Linux – начиная с линейки RHEL 8, все её будущие версии будут только 64-битными.

В самом начале эпохи поколений Core 2 Microsoft всё-таки подзадержалась на старте, продолжая эксплуатировать оказавшуюся на сегодня самой «долгоиграющей» Windows XP. Первая попытка замены XP на Windows Vista (ноябрь 2006 года) оказалась не очень удачной, рынок принял Vista довольно прохладно, в результате XP продержалась «на плаву» ещё почти восемь лет, когда в апреле 2014 года Microsoft объявила о полном прекращении её поддержки<sup>1</sup>.

Следующая версия Windows 7 оказалась намного удачнее и удачливее – она просуществовала на рынке более десяти лет – была объявлена в октябре 2009 года, расширенная поддержка прекращена 14 января 2020 года. В этой версии десктопной Windows Microsoft удалось, наконец, свести в одну ветку разработку ядра системы с ядром серверной ветки (это была Windows Server 2008R2), чему компания была очень рада и чем очень гордилась,

---

<sup>1</sup> Для многих будет неожиданностью, что поддержку «неудачной» Windows Vista корпорация Microsoft прекратила относительно недавно, 11 апреля 2017 года.

рассказывая об этом факте, правда, не очень понятном рядовому пользователю, едва ли не на каждой презентации. Видимо, разработчики ОС их поймут ☺.

Windows 7 имела полноценную 64-битную версию, которая полностью использовала все ресурсы современных процессоров, в первую очередь – значительно расширенное адресное пространство. Выпускавшаяся в своё время Windows XP 64 была на самом деле неким гибридом – 64-битное ядро сервера Windows 2003, «скрещенное» с графическим интерфейсом обычной 32-разрядной Windows XP. Из-за существенно различающихся моделей работы с памятью у десктопных и серверных систем разной разрядности, под XP 64 не было адекватной поддержки в виде драйверов и утилит, т.к. производители оборудования, из-за относительно небольшого рынка, плохо поддерживали эту версию Windows. Например, долгое время не было продвинутых драйверов для мощных 3D-видеокарт, из-за чего использование XP 64 в качестве графической рабочей станции было весьма проблематичным. Кроме версии под x86-64, существовала версия Windows XP 64-Bit Edition для работы на семействе микропроцессоров Intel Itanium в родном (native) IA-64 режиме. Но это совсем другая история, закончившаяся, впрочем, довольно быстро – в начале 2005 года дальнейшая разработка Windows XP 64-Bit Edition 2003 была прекращена.

Windows 7 оказалась удачной ревизией, на которую стало принято ссылаться, как на весомый аргумент, подтверждающий теорию чётных-нечётных версий ОС от Microsoft – было замечено, что нечётные версии – Windows XP, 7 – были довольно удачными, а чётные – Vista, и вышедшая в октябре 2012 года Windows 8 – не очень.

Это, конечно, шутка, но доля истины тут есть. К какой категории – чётная или нечётная – следует отнести версию Windows 8.1, вышедшую на рынок в октябре 2013? Формально, после «чётной» «восьмёрки», это должна была быть удачная «нечётная» версия, но, ведь мажорный релиз – цифра 8 в названии ОС – остались неизменными, так что, всё-таки, скорее это была «неудачная чётная» версия ☺.

Интересная история произошла с пользовательским интерфейсом в Windows 8 – Microsoft много и активно рекламировала грядущий «плиточный» Metro, объявив его едва ли не основным новшеством «восьмёрки». Но перед самым выходом Windows 8, буквально за две недели до даты презентации, совершенно неожиданно для всех, Microsoft объявила, что термин Metro теперь запрещён и далее не должен использоваться, от слова совсем. И то, что он обозначал ранее, теперь следует называть «Новый Windows интерфейс». Причина такого спешного отказа от уже довольно хорошо раскрученного бренда не вполне ясна и сегодня.

Но даже после переименования, «плиточный» интерфейс так и не был принят основной массой пользователей Windows 8. Они критиковали сильно изменённый интерфейс, заставляющий тратить дополнительное время на обучение работе с новой операционной системой. Главный маркетинговый директор Microsoft Тами Реллер в одном из интервью сказала, что некоторые ключевые элементы Windows 8 будут изменены при выпуске обновленной версии системы Windows 8.1. Это было воспринято некоторыми СМИ как фактическое признание неудачи компании с выпуском Windows 8. Но Windows 8.1 по инерции тоже была принята негативно, особенно после прекращения поддержки Windows XP.

Пропустив скорее из околполитических соображений версию 9<sup>1</sup>, в июле 2015 года Microsoft выпускает Windows 10, сервер на той же версии ядра – Windows Server 2016. Интересный, но не очень широко известный факт – в базовую поставку Windows 10 версии 1809 и Windows Server 2016 были добавлены клиент и сервер OpenSSH. На этом присутствие Opensource в ранее наглухо закрытой проприетарной Windows не закончилось – на эти же версии Windows можно установить чрезвычайно популярную систему «лёгкой» контейнерной виртуализации Docker (с поддержкой linux в контейнерах), который тоже был

---

<sup>1</sup> В Microsoft, видимо, решили, что номер 9 у пользователей будет ассоциироваться со старыми линейками 16-битных Windows 95/98 (их действительно часто называли «девятками»), про которые, похоже, компания хотела бы поскорее забыть.

включён в дистрибутив в качестве штатного компонента. В продолжении темы, в следующем релизе Windows Server 2019, появился ещё один мощный Opensource продукт с названием Kubernetes, предназначенный для оркестрации контейнеров (н-р, того же Docker), в среде Windows Server. Также из мира unix-систем в Windows Server 2019 были портировано множество базовых утилиты: curl, tar, bash и др.

Выпустив Windows 10, Microsoft объявила, что в дальнейшем номера настольных версий Windows больше меняться не будут, и «десятка» — это навсегда<sup>1</sup>. ОС конечно продолжает развиваться и совершенствоваться, выходят новые версии (которые Microsoft официально называет «обновлением компонентов»), но при этом меняется только номер внутренней версии, т.н. сборка. Начиная с 2017 года, каждая новая версия Windows 10 поддерживается 18 месяцев (хотя есть и «долгоиграющие» варианты, с увеличенным циклом поддержки, т.н. LTS).

На сегодня (август 2020 года) история версий Windows 10 выглядит так:

1. Windows Threshold
  - a. Threshold 1 — Initial Version (1507)
  - b. Threshold 2 — November Update (1511)
2. Windows Redstone
  - a. Redstone 1 — Anniversary Update (1607)
  - b. Redstone 2 — Creators Update (1703)
  - c. Redstone 3 — Fall Creators Update (1709)
  - d. Redstone 4 — April 2018 Update (1803)
  - e. Redstone 5 — Windows 10 October 2018 Update (1809)
  - f. May 2019 Update (1903)
  - g. November 2019 Update (1909)
  - h. Версия 20H1 или 2004
  - i. Версия 20H2

Windows 10, несмотря на не всегда джентльменские методы внедрения (например, в первых версиях практиковалось принудительное обновление с Windows 7/8/8.1, без возможности для пользователя отказаться от апгрейда), оказалась в целом неплохой и добротной системой, и была достаточно хорошо принята пользователями, в отличие от довольно прохладного приёма предыдущих Windows 8/8.1. Уже в апреле 2017 года, менее чем через два года после выхода первой версии, согласно статистическим данным сайта W3Schools, Windows 10 заняла первое место в мире среди операционных систем, используемых для доступа к сети Интернет, опередив предыдущего лидера — Windows 7.

Серверные версии Windows, в отличие от десктопных, продолжали нумероваться по годам выпуска. С версии 2003 по 2012 Microsoft практиковала через два года после выпуска исходной версии сервера выпускать его обновление, добавляя в названии к году суффикс R2: Windows Server 2003/2003R2, ..., Windows 2012/2012R2. Начиная с Server 2016, суффикс R2 в названии больше не добавляется, и на сегодня есть две версии, Windows Server 2016 и 2019, для которых нет вариантов с R2.

Под Windows наработано невероятное количество ПО — как пользовательского, так и серверного класса. САПРы, текстовые процессоры, СУБД, видеоредакторы, звуковые редакторы, системы видеомонтажа, продукты для обмена сообщениями и совместной работы, системы документооборота, гипервизоры, службы облачных вычислений, системы мониторинга, 3D игры и т.п. Очень часто эти продукты занимают лидирующее положение на

---

<sup>1</sup> В течении первого года после выхода Windows 10 пользователи предыдущих лицензионных Windows 7, 8.1 могли бесплатно обновиться на «десятку». Было объявлено, что Windows 10 будет последней «коробочной» версией, все последующие версии будут распространяться исключительно в цифровом виде. Несмотря на обещания «вечной» жизни Windows 10, Microsoft в июне 2021 года объявляет о выходе Windows 11, которая выйдет на рынок осенью 2021г. Сейчас (лето 2021) уже доступны её ознакомительные версии.

рынке, либо находятся среди лидеров. Есть целые категории программного обеспечения, например, системы автоматизированного проектирования, которые присутствуют только в версиях под Windows, у которой здесь основной конкурент – MAC OS от Apple. Уровень поддержки ПО подобной категории под ОС Linux существенно уступает тому, что производители программного обеспечения предлагают для Windows.

Но есть категории ПО, в которой Windows на сегодня практически не присутствует – например, это высокопроизводительные системы (HPC – High Performance Computing), Microsoft прекратила работы в этом направлении несколько лет назад, сейчас там практически безраздельно господствует Linux. Ну и, конечно нужно ещё раз упомянуть полный уход Windows с мобильного рынка.

Приведём некоторые характеристики ОС семейства Windows:

- Максимальный размер файла – 16 Тбайт (десктопные версии), 256 Тбайт (серверные).
- Максимальный размер раздела файловой системы – 256 Тбайт, с возможностью при необходимости этот лимит увеличить, в файловой системе NTFS нет жёстких, «прибитых гвоздями» ограничений.
- Максимальный объём поддерживаемого ОЗУ (сервер 2016/2019) – 24 Тбайт.
- Максимальное число сокетов (сервер 2019) – 64.
- До 512 логических процессоров (сервер 2016).
- До 12 Тбайт на виртуальную машину.

Все версии Windows описываемой эпохи поколения Core и новее поддерживали работу приложений двух смежных разрядностей: 32-разрядные версии Windows могли запускать 16-разрядные приложения (например, утилиты MS-DOS, и саму DOS) и родные 32-разрядные; 64-битные, соответственно – 32- и 64-разрядные приложения. Запуск приложений MS-DOS на 64-битной платформе напрямую невозможен, в случае необходимости приходится использовать технику виртуализации.

## Apple, MacOS

Ещё один крупный софтверный игрок, перешедший на интеловскую платформу в 2005 году – компания Apple, со своей Mac OS X<sup>1</sup>. Многие фанаты «яблочной» компании были обеспокоены и встревожены её переходом на «попсовую» x86, опасаясь превращения предмета их обожания в ещё один очередной wintel, слабо отличимый от основной массы «писишек». Но их опасения не оправдались, Apple сумела сохранить свою самобытность и качество, не затерявшись и не растворившись на более массовом рынке «масс-культуры», где правила Intel и Microsoft. Автор в своё время был впечатлён абсолютно «гладким» и беспроблемным переходом Mac OS X на 64 бита при обновлении на MacOS очередной новой версии – после длившегося около часа процесса апгрейда и последующей перезагрузки ОС было обнаружено (внезапно, примерно через несколько дней ☺), что MAC Book теперь оказывается работает на 64-битной ОС. Замечено это было только после входа в панель управления – настолько хорошо и качественно был осуществлён переход системы 32 бита → 64 бита, что заметить это «невооружённым» взглядом оказалось практически невозможным. Все приложения – и старые 32-битные и новые 64-битные – исправно работали, никаких сбоев и ошибок не наблюдалось. Заметим, что апгрейд ОС такого типа, с переходом на другую разрядность, в исполнении Microsoft для Windows невозможен – система может обновляться только на новую версию той же разрядности.

Объявляя причину перехода на x86 на конференции WWDC<sup>2</sup> 6 июня 2005 года, Стив Джобс был краток и убедителен: в PowerPC от IBM и Freescale, ватт потраченной энергии

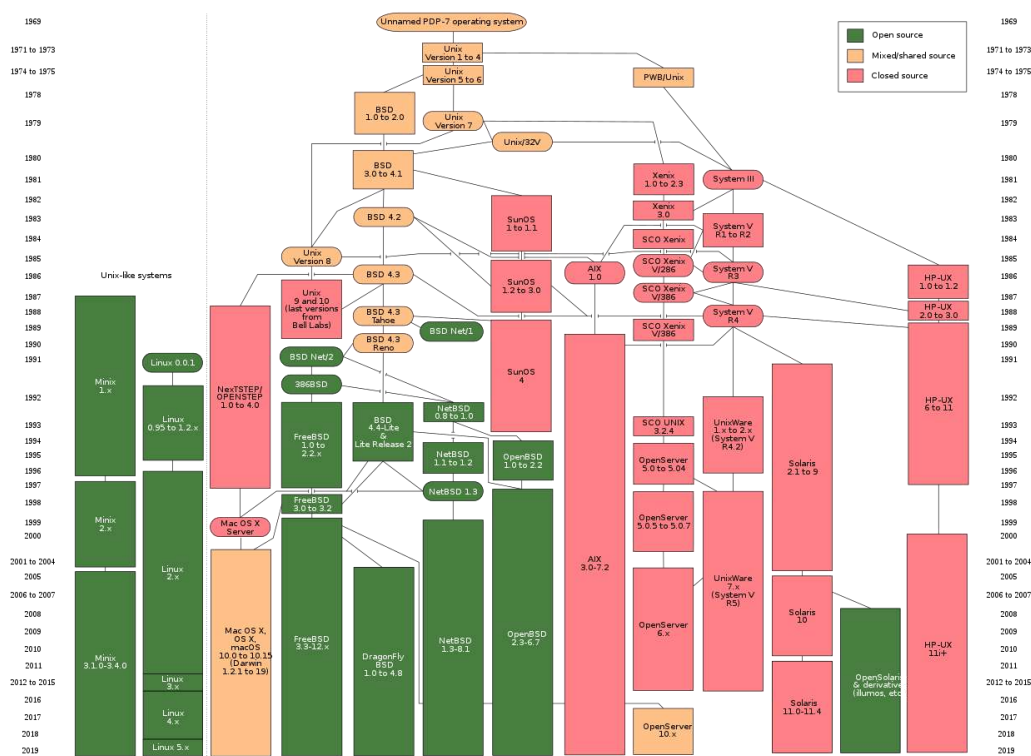
---

<sup>1</sup> До 2012 года ОС для Mac-а называлась Mac OS X, затем – просто OS X.

<sup>2</sup> **Apple Worldwide Developers Conference (WWDC)** — всемирная конференция для разработчиков на платформах Apple. Проводится ежегодно в Калифорнии, США.



Когда речь идёт об OS X, то это, в первую очередь, пользовательские операционные системы, на серверном рынке Apple была представлена довольно слабо. OS X относят к категории mixed/shared source – не полностью открытая, как, например, Linux, и не полностью закрытая, как, скажем Windows – в OS X есть открытые и закрытые (проприетарные) компоненты. В этой Unix-подобной ОС, к примеру, основная файловая система взята из BSD-like Unix-а. Ниже (Рис.429) приведена диаграмма, на которой отображены отношения между различными Unix-подобными ОС, включая предков macOS ([macOS version history](#)).



С первой macOS, получившей старший номер версии 10 (римская X) (не считая первую публичную Mac OS X Public Beta Kodiak), Apple не просто скучно прибавляла единичку в младшем номере каждой новой версии, а называла её в честь какого-либо представителя семейства кошачьих (Рис.430).

На Рис.430 нет двух последних (на август 2020 года) версий MacOS:

- 521



- **Version 11.0: “Big Sur”**: macOS *Big Sur* была анонсирована 22 июня 2020 года, на очередной конференции WWDC. Впервые с появления “Mac OS X” был изменён старший номер версии, таким образом превратив macOS в ОС версии 11.0, и было объявлено о поддержке процессоров ARM.

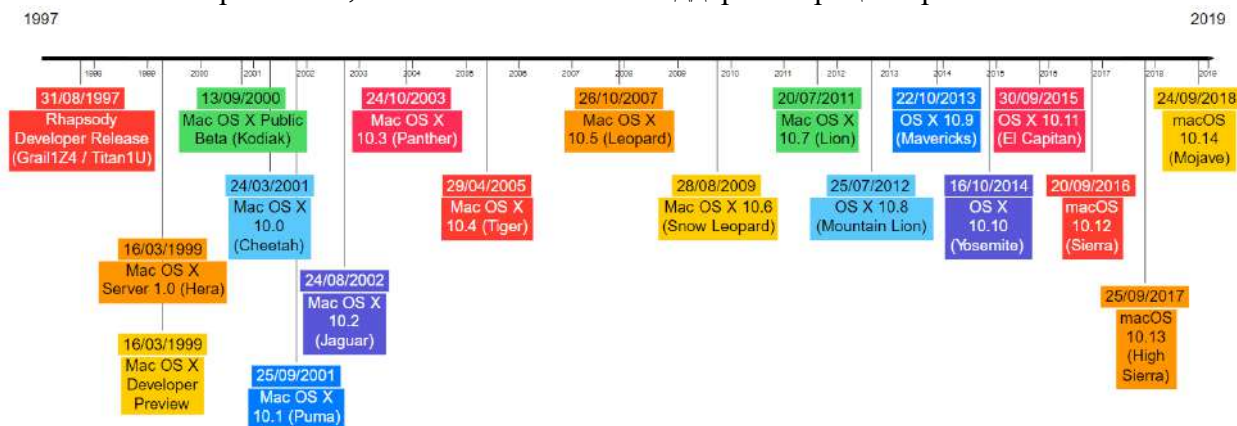


Рис.430 – Timeline версий MacOS

На той же конференции WWDC в июне 2020 года, где была анонсирована версия MacOS 11.0 “Big Sur”, Apple официально объявила о переходе на собственные процессоры на базе ARM. Генеральный директор Apple Тим Кук напомнил эпоху PowerPC, потом Mac OS X и переход на чипы Intel, прежде чем обнародовать планы по использованию собственных микросхем от Apple на базе ARM (Apple A12Z, A13Z, A14) в компьютерах Mac в будущем.

Первый Mac с чипом Apple<sup>1</sup> выйдет в конце этого года. Ожидается, что полный переход займёт два года. Новые Mac на базе Intel по-прежнему в разработке, поэтому о полном переходе на ARM-платформу пока говорить рано. Тем не менее, это важное решение для Apple — в перспективе отказаться от процессоров Intel.

Главное преимущество новой платформы — возможность для приложений iOS и iPadOS нативно работать на macOS. «Большинство приложений будут просто работать», — говорит Apple, имея в виду, что вы впервые сможете запускать на macOS рядом друг с другом и нативные приложения iOS, и нативные приложения macOS.

## Linux world

История Linux началась в 1991 году, когда финский студент Линус Торвалдс, увлёкся идеей написать совместимое с UNIX ядро операционной системы для своего персонального компьютера с процессором ставшей к тому времени широко распространённой архитектуры Intel 80386.

В своей книге “Just for fun” («Рассказ нечаянного революционера» в русскоязычном издании) Торвалдс рассказывал, как всё начиналось – ему почему-то ужасно не нравилась операционная система VAX/VMS, основная ОС для студентов в то время, и ему было откровенно лень приходить в университет, чтобы поработать за терминалом VAX-а. И Торвалдс решил написать терминальный эмулятор, чтобы иметь возможность удалённо работать с VAX-ом из общедоступного места. В очередной раз подтвердив расхожую фразу «лень – двигатель прогресса» ☺.

Пользуясь наработками проекта GNU Ричарда Столлмана, из ядра Linux, утилит и приложений GNU впервые стало возможно сделать полностью свободную операционную

<sup>1</sup> Apple не имеет собственного полупроводникового производства, это т.н. fabless-компания. Долгие годы ARM-процессоры для iPhone производились на фабриках Samsung. Процессоры A12 для Apple по 5 нм технологическим проектным нормам будет производить тайваньская TSMC (производитель чиплетов для процессоров EPYC AMD).

систему, то есть работать с компьютером и, более того, разрабатывать новое программное обеспечение, пользуясь только свободным программным обеспечением. Идеал полностью некоммерческой разработки, сформулированный Столлманом, теперь мог быть воплощён в жизнь.

В середине 1990-х, точнее в 1994 году появился первый «настоящий» (в современном понимании) дистрибутив Linux – Slackware Патрика Фолькердинга. В 1995 появилась компания Red Hat, первая в мире сумевшая достичь миллиардных оборотов на разработке и поддержке открытого программного обеспечения, сегодняшний лидер, номер один в современном мире Linux. Примерно в то же время образовалась ещё одна знаковая для Linux-сообщества компания – Debian. Её задача была примерно той же — сделать целостный дистрибутив Linux и свободного программного обеспечения GNU, однако этот проект был задуман как принципиально некоммерческий, проводимый в жизнь сообществом разработчиков, нормы взаимодействия в котором полностью соответствовали бы идеалам свободного ПО. Сообщество разработчиков Debian — международное, участники которого взаимодействуют через Internet, а нормы взаимодействия между ними определяются специальными документами — полиси (policy).

Ниже (Рис.431) приведена инфографика, на которой отмечены наиболее важные события в истории Linux, произошедшие за двадцать пять лет со «дня рождения» этой ОС (1991 + 25 = 2016 год) – [Timeline Of The Most Important Events In 25 Years Of Linux](#). Хронология в текстовом виде ([История Linux - хронология](#)) расширена до 2019 года и слегка отличается от представленной на Рис.431.

- 1991: 25 августа 1991 года 21-летний финский студент *Linus Benedict Torvalds* (Линус Торвальдс) публично представил первое ядро Linux.
- 1992: Linux не всегда был программным обеспечением из категории Open Source, первые лицензии Linux запрещали коммерческое распространение. В 1992 году ядро Linux версии 0.12 было выпущено под лицензией GNU GPL, был создан первый Linux дистрибутив.
- 1993: Над ядром Linux работают более 100 разработчиков. С их помощью ядро адаптируется к среде GNU, что создало широкий спектр типов приложений для Linux. Выпускается впервые самый старый (по состоянию на 2018 год) из существующих дистрибутивов Linux, Slackware. Позже в том же году создается проект Debian. На сегодня это самый крупный дистрибутив, созданный сообществом.
- 1994: Торвальдс считает, что все компоненты ядра полностью созрели: он выпускает версию 1.0 Linux. Проект XFree86 предоставляет графический пользовательский интерфейс (GUI). Производители коммерческих дистрибутивов Linux Red Hat и SUSE публикуют версию 1.0 своих дистрибутивов Linux.
- 1995: Linux портирован на системы DEC Alpha и Sun SPARC. В последующие годы он был перенесён на большое количество различных платформ.
- 1996: Выходит версия 2.0 ядра Linux. Эта версия теперь поддерживает многопроцессорные платформы, используя symmetric multiprocessing (SMP).
- 1998: Многие крупные компании, такие как IBM, Compaq и Oracle, заявляют о своей поддержке Linux. Компания Netscape опубликовала исходный код своего веб-браузера Netscape Communicator. Действия Netscape привлекают внимание популярной технической прессы к модели разработки Linux с открытым исходным кодом. Кроме того, группа программистов начинает разработку графического пользовательского интерфейса KDE.
- 1999: Группа разработчиков начинает работу над графической средой GNOME, призванной стать бесплатной заменой KDE, которая в то время зависела от проприетарного инструментария Qt. В течение года IBM анонсирует масштабный проект по поддержке Linux. Выпущена версия ядра Linux 2.2.

- 2000: Компания Dell объявляет, что теперь она является вторым поставщиком систем на базе Linux в мире и первым крупным производителем, предлагающим Linux на всей линейке своих продуктов.
- 2001: Выходит версия ядра 2.4 Linux.
- 2002: СМИ сообщают, что «Microsoft убила Dell Linux». Dell незаметно убрала Linux со своих настольных ПК летом 2001 года, как впоследствии обнаружила Эшли Вэнс из IDG, через шесть месяцев после того, как мы услышали, как Майкл Делл заявил о своей любви к Linux на настольных ПК в предыдущую зиму.
- 2003: Выходит версия ядра 2.6 Linux.
- 2004: Команда XFree86 разделяется и объединяется с существующим органом стандартов X-Windows, чтобы сформировать X.Org Foundation, что приводит к существенно более быстрой разработке X-сервера для Linux.
- 2005: Сообщество Novell начинает бесплатное распространение проекта openSUSE. Также проект OpenOffice.org представляет версию 2.0, которая затем начала поддерживать стандарты OASIS OpenDocument.
- 2006: Oracle выпускает собственный дистрибутив Red Hat Enterprise Linux. Novell и Microsoft объявляют о сотрудничестве для улучшения взаимодействия и взаимной патентной защиты.
- 2007: Dell начинает распространять ноутбуки с предустановленной Ubuntu.
- 2009: Рыночная капитализация Red Hat сравнялась с капитализацией Sun, что интерпретируется как символический момент для «экономики, основанной на Linux».
- 2011: Выходит версия ядра 3.0 Linux.
- 2012: Совокупный доход рынка серверов Linux превышает доход остального рынка Unix-систем.
- 2013: Android от Google на базе Linux занимает 75% рынка смартфонов по количеству отгруженных телефонов.
- 2014: Ubuntu заявляет о 22,000,000 миллионах пользователей.
- 2015: Выходит версия ядра 4.0 Linux.
- 2019: Выходит версия ядра 5.0 Linux.

На сегодняшний день (2020 год) Linux это, пожалуй, наиболее динамичная и активно развивающаяся система, широко представленная во всех спектрах приложений, от мобильных (к примеру, Андроид – это виртуальная java-машина, работающая под Linux) до сверхпроизводительных суперкомпьютеров из Top500. Есть сферы применения, где основным конкурент Linux-а (Windows) либо не присутствует совсем, либо значительно уступает в распространённости; это, как уже упоминалось выше, сфера высокопроизводительных вычислений (HPC), откуда Microsoft незаметно удалилась примерно десять лет назад, и рынок мобильных устройств, поражение на котором Microsoft фактически признала, объявив о полном прекращении поддержки и развития своей Windows Mobile в декабре 2019 года. Стоит отметить, что на мобильный рынок Microsoft пыталась пробиться давно и долго, выходя на него вначале с ОС для персональных цифровых ассистентов (PDA), предшественников смартфонов, с каждым раз переименованной версией вроде Windows 2003, Windows 2005 и т.п. Но возникновение и последующее бурное развитие рынка смартфонов Microsoft откровенно упустила<sup>1</sup> и отреагировала с запозданием и недостаточно активно. В результате, после нескольких лет довольно вялого прогресса своей мобильной ветки Windows, Microsoft «снялась с гонки», уступив iOS от Apple (фактически давшей старт гонки смартфонов) и Google Android. Количество выпущенных устройств с процессорами не от Intel и с ОС не от Microsoft на сегодня в разы превосходит количество intel-устройств, речь идёт о

---

<sup>1</sup> Как когда-то Microsoft едва не упустила появление массового Internet-а, продвигая в 1990-ые свою собственную глобальную сеть Microsoft Network (MSN).

масштабах десятки (может быть сотни) миллионов x86/Windows против миллиардов выпущенных iPhone/iPad/Android на базе ARM.



Рис.431 – Важнейшие события в 25-летней истории Linux

Автор попытался проиллюстрировать ["Линию жизни" дистрибутивов Linux](#) (Рис.432), но из-за вынужденно мелкого масштаба эта картинка практически бесполезна, подробности можно рассмотреть только при большом увеличении при просмотре по ссылке выше.



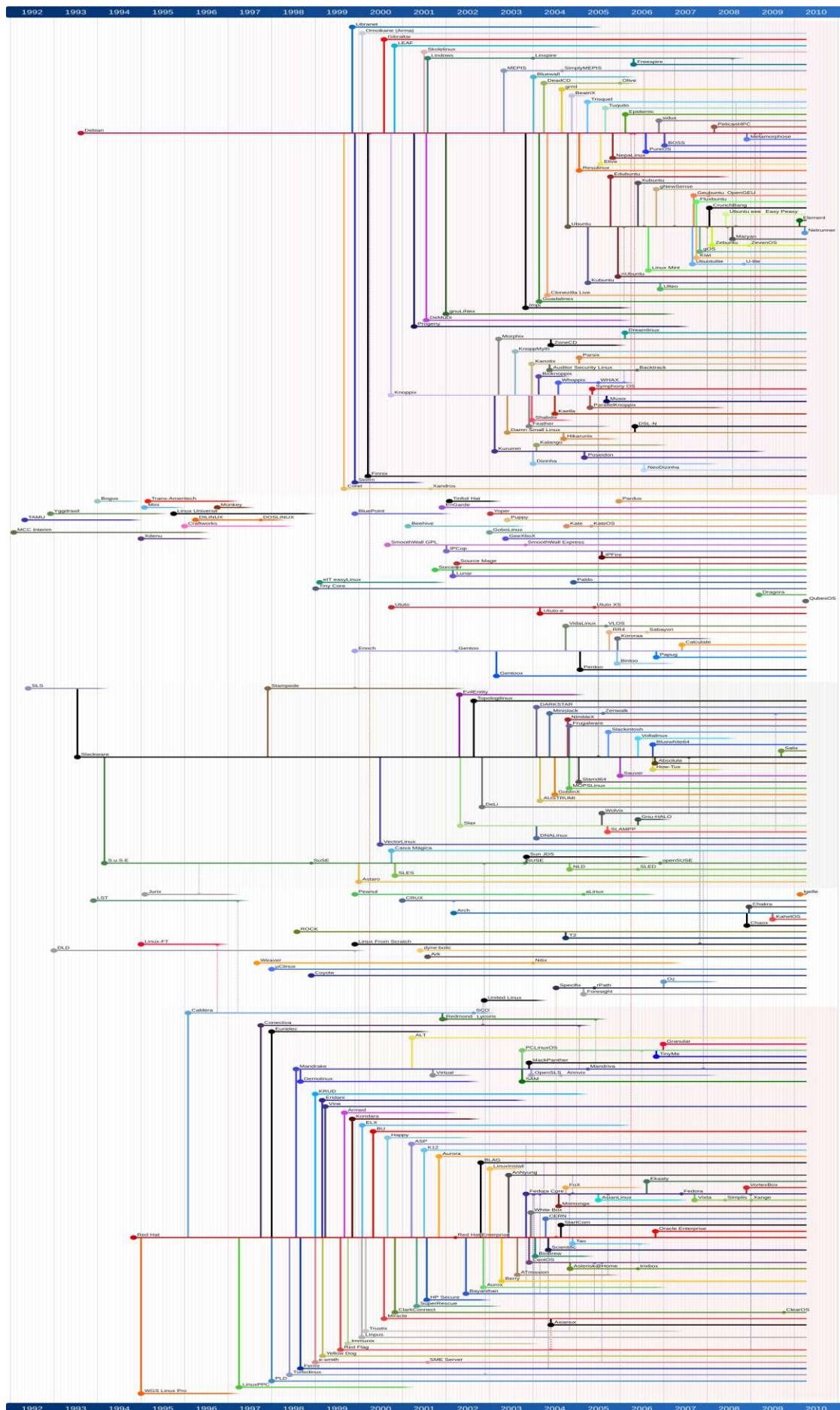


Рис.432 – «Линия жизни» дистрибутивов Linux

Где Linux сегодня пока ещё заметно проигрывает Windows и OS X (iOS) – это десктопы и ноутбуки, а вот в планшетах присутствие Linux уже гораздо более значительно (в основном благодаря Android). В сфере настольных приложений Linux всё ещё заметно уступает Windows по уровню поддержки производителей оборудования, в первую очередь – топовых видеокарт, и производителей «тяжелого» программного обеспечения класса САПР, 3D-игр и т.п. В серверной части с поддержкой high-end «железа» в Linux-е сегодня дела обстоят значительно лучше, чем это было пять-десять лет назад – мощный производительный Linux-сервер на закрытой проприетарной аппаратной платформе – это на сегодня mainstream, основа многих продвинутых знаменитых облачных платформ. Хотя, необходимо отметить, что в облачных технологиях неплохо себя проявила и зарекомендовала платформа Azure от Microsoft, здесь наблюдается если не паритет между Linux и Windows, то очень активная конкуренция. В отличие от истории с мобильным миром, Microsoft довольно успешно и вовремя вышла на рынок облачных технологий. Один из самых известных примеров их облачных продуктов – Office365, успешно конкурирующий с «облачным» офисом от Google – Google Docs.

Коротко резюмируя: работая в принципиально иной парадигме Open Source, в противоположность ОС с полностью закрытым (Windows) или частично закрытым (OS X) исходным кодом, Linux сумел, совместными усилиями сообщества и коммерческих компаний, вырасти в один из главных столпов современного ИТ-мира<sup>1</sup>. Linux сегодня – это и «облака», и мобильные устройства, и мощные серверы, и – хотя пока ещё не очень значительное – но всё более уверенное проникновение на рынок пользовательских настольных машин и ноутбуков (на примере Ubuntu), где правят Windows и iOS/OS X, это индустрия с многомиллиардными оборотами. Неплохой прогресс для студенческой разработки, начавшейся с нескольких сотен строк кода на Си ☺.

И, хотя Linux на сегодня – это самая кроссплатформенная операционная система, работающая практически на любом «железе», от утюга до «облака», можно отметить, что этап зрелости этой ОС, по мнению автора, совпал с этапом зрелости платформы именно Intel x86, который – опять же по мнению автора – пришёлся на момент выпуска процессоров Intel Core 2. В этом совпадении ничего странного и удивительного нет, если вспомнить, что Linux начинался как ОС для PC на основе i80386, это была самая первая и какое-то время единственная Linux-платформа. И развитие аппаратной части x86 естественным образом сопровождалось развитием и взрослением программного обеспечения для неё, а для многих дистрибутивов Linux платформа x86(-64) до сих пор остаётся единственной.

## BSD\*, Solaris

Из представленного выше (Рис.429) упрощённого «генеалогического дерева» ОС UNIX, автор решил отметить несколько наиболее интересных и значимых UNIX-подобных операционных систем, работающих на платформе x86: это ОС семейства BSD<sup>2</sup> и ОС Solaris<sup>3</sup>. ОС семейства BSD портирована на несколько аппаратных платформах, включая даже такие раритеты, как DEC VAX (OpenBSD), но основная платформа для операционных систем этого семейства – Intel x86(-64). Тогда как Solaris x86 – это, всё же скорее, номер два, первым для этой ОС всегда была платформа Sun SPARC.

---

<sup>1</sup> Главный критерий, по которому все эти десятки, если не сотни самых разнообразных дистрибутивов Linux, имеют право называться Linux-ом – общее, единое, открытое ядро, на основе которого и строятся самые разнообразные конечные версии дистрибутивов.

<sup>2</sup> К семейству **BSD** относятся – NetBSD, FreeBSD, OpenBSD, ClosedBSD, MirBSD, DragonFly BSD, PC-BSD, DesktopBSD, SunOS, TrueBSD, Frenzy, Ultrix и частично XNU (ядро macOS, iOS, tvOS, watchOS, CarPlay, Darwin).

<sup>3</sup> **Solaris** — операционная система, разработанная компанией Sun Microsystems для платформы SPARC, с 2010 года принадлежит вместе с активами Sun корпорации Oracle. Несмотря на то, что Solaris — операционная система с закрытым исходным кодом, большая его часть открыта и опубликована в проекте OpenSolaris. Начиная с версии 2.4 (ноябрь 2004 года), выходит в версии для процессоров SPARC и x86.

## ***BSD-like OC***

BSD (Berkeley Software Distribution) — система распространения программного обеспечения в исходных кодах, созданная для обмена опытом между учебными заведениями. Особенностью пакетов ПО BSD была специальная лицензия BSD, которую кратко можно охарактеризовать так: весь исходный код — собственность BSD, все правки — собственность их авторов.

В данный момент термин BSD чаще всего употребляется как синоним BSD-UNIX — общего названия вариантов UNIX, восходящих к дистрибутивам университета Беркли<sup>1</sup>.

### **Немного истории BSD**

В начале 1978 года аспирант Университета Калифорнии в Беркли Билл Джой начал создание дистрибутива программ Беркли. Первая магнитная лента (360 м, ценой около 50 долл.) содержала систему Pascal для UNIX и редактор ex; распространено около 30 копий. До конца 1978 года распространялся пакет 2BSD, продано около 75 копий. Приблизительно тогда же компания Interactive Systems выпускает первую коммерческую версию UNIX, а компания Whitesmiths — первый клон системы UNIX — Idris.

Начиная с версии 2.79BSD, программные дистрибутивы университета Беркли перестали содержать код AT&T UNIX. Версия UNIX 7 была первой переносимой операционной системой. Однако её производительность была хуже, чем в версии шесть. После многих доработок в январе 1982 года вышла в свет модель 2.8.1BSD.

Впоследствии вышли версии:

- 4BSD — октябрь 1980 года
- 4.1BSD — июнь 1981 года
- 4.1a, 4.1b и 4.1c (1982—1983)
- 4.2 — сентябрь 1983 года — сильно переработанная система, включавшая поддержку сетей TCP/IP, новую файловую систему и возможность использования сигналов
- 4.3BSD

Последней университетской версией была 4.4BSD (июнь 1993 года). Из нововведений эта версия включала виртуальную файловую систему, поддержку сетевой файловой системы NFS, отладчик ядра и мощную реализацию стека протоколов TCP/IP.

После суда права на BSD были переданы компании BSDI.

На базе BSD-UNIX создано множество операционных систем, преимущественно также с открытыми исходными кодами. Также многие ОС использовали наработки BSD в своём ядре, к примеру, сетевой стек TCP/IP для своей линейки Windows NT Microsoft работала на основе стека протоколов от BSD.

«Генеалогическое дерево» ОС семейства BSD (выполненное в виде «рисунка» в текстовом виде, т.н. ASCII art) слишком большое и громоздкое (более 850 строк), поэтому приводить его здесь ни полностью, ни частично не будем. Ознакомится с этим списком можно по ссылке [Генеалогическое дерево BSD](#).

---

<sup>1</sup> Университет Беркли находится на расстоянии примерно часа езды от эпицентра мировой ИТ-индустрии — Кремниевой долины, на северо-востоке от залива Сан-Франциско, и практически относится к той же «экосистеме».



На популярном сайте Distrowatch<sup>1</sup> на сегодня рейтинг посещаемости страниц с описанием дистрибутивов ОС, относящихся к семейству BSD, выглядит следующим образом (номер в скобках означает позицию в списке рейтинга со страницы [Search BSD Distributions](#)):

1. **FreeBSD** (29) – UNIX-подобная операционная система для i386, amd64<sup>2</sup>, IA-64, arm, MIPS, powerpc, ppc64, PC-98 и UltraSPARC платформ, базирующаяся на релизе U.C. Berkeley's "4.4BSD-Lite", с некоторыми расширениями "4.4BSD-Lite2". Он также косвенно основан на порте Уильяма Джолитца из Калифорнийского университета. От Berkeley "Net/2" для i386, известного как "386BSD", хотя в текущей версии от 386BSD осталось очень мало кода. FreeBSD используется компаниями, провайдерами Интернет-услуг, исследователями, компьютерными профессионалами, студентами и домашними пользователями по всему миру в своей работе, учебе и отдыхе. FreeBSD поставляется с более чем 20 000 пакетов (предварительно скомпилированное программное обеспечение, которое объединено для легкой установки), охватывающих широкий спектр областей: от серверного программного обеспечения, баз данных и веб-серверов до программного обеспечения для настольных ПК, игр, веб-браузеров и программного обеспечения для бизнеса – всё бесплатно (free) и просто в установке.
2. **GhostBSD** (46) – дружественная к пользователю настольная (десктоп) операционная система, основанная на FreeBSD. Рабочий стол по умолчанию использует MATE, но существует отдельная редакция от сообщества с Xfce. Также включает систему инициализации OpenRC, набор часто используемых программных продуктов, модель разработки с непрерывным выпуском и загружаемый динамический образ с интуитивно понятным графическим установщиком системы.
3. **NomadBSD** (54) – 64-битная live система, для загрузки с USB flash drives, основанная на FreeBSD. Используя автоматическое обнаружение и настройку оборудования, ОС настроена для использования в качестве настольной системы, которая работает «из коробки» (без установки на жёсткий диск), но также может использоваться для восстановления данных.
4. **FuryBSD** (68) – open-source, ориентированная на десктоп система, основанная на FreeBSD. Это попытка возродить дух других простых в использовании проектов на базе FreeBSD прошлого (например, PC-BSD и TrueOS), но также добавляет дополнительные удобства в виде гибридного образа для загрузки с USB/DVD. Проект предоставляет отдельные live-образы для рабочих столов Xfce и KDE Plasma. FuryBSD бесплатна, и она может свободно распространяться по лицензии BSD.
5. **DragonFly BSD** (77) – операционная система и среда, разработанная как логическое продолжение серии ОС FreeBSD-4.x. Эти операционные системы принадлежат к тому же классу, что и Linux, поскольку они основаны на идеалах UNIX и API. DragonFly – это, если можно так сказать, развилка (fork) на пути, дающая возможность на базе BSD развиваться в совершенно новом направлении по сравнению с тем, что было в серии FreeBSD-5.

---

<sup>1</sup> **DistroWatch** — информационный и новостной веб-сайт о различных дистрибутивах Linux и других свободных и открытых операционных системах (UNIX-подобных, например, OpenSolaris или BSD). Также, на нём ведётся рейтинг посещаемости страниц с описанием дистрибутивов, как примерный способ оценки их популярности. Счётчик не коррелирует с использованием дистрибутивов, с качеством дистрибутивов, и не может использоваться как инструмент для измерения рыночной доли дистрибутивов. Он просто показывает количество заходов на страницу конкретного дистрибутива на сайте DistroWatch, и ничего больше.

<sup>2</sup> Ещё раз отметим, что в мире UNIX ОС 64-битную платформу x86 принято называть amd64, по имени компании, которая первой реализовала это расширение в своих процессорах.

6. **OpenBSD (78)** – проект OpenBSD создает бесплатную многоплатформенную UNIX-подобную операционную систему на основе BSD 4.4. Его усилия делают упор на переносимость, стандартизацию, корректность, проактивную безопасность и интегрированную криптографию. В рамках проекта также разрабатывается широко используемое и популярное программное обеспечение OpenSSH (OpenBSD Secure Shell), которое обеспечивает зашифрованные сеансы связи по компьютерной сети с использованием протокола SSH. Пожалуй, самый консервативный представитель семейства BSD, позже всех реагирующий на появление нового оборудования. Например, реально работающая поддержка интерфейса SATA появилась в системе спустя более чем полтора года, после выхода SATA на массовый рынок. Главное достоинство этого дистрибутива – активно продвигаемое разработчиками – присутствие в нём настоящего «духа» UNIX и «пуленепробиваемость» ОС.
7. **pfSense (133)** – открытый, бесплатный, сильно кастомизированный дистрибутив FreeBSD, предназначенный для использования в качестве межсетевого экрана и маршрутизатора, полностью управляемых через веб-интерфейс. Помимо того, что pfSense является платформой маршрутизации, он включает длинный список связанных функций и систему пакетов, которая даёт возможность дальнейшего расширения.
8. **NetBSD (134)** – свободная, безопасная, UNIX-подобная ОС с открытым кодом (Open Source), портированная на большое количество платформ: от 64-битного AlphaServers и настольных систем, до портативных и встраиваемых устройств. Чистый дизайн и расширенные функции этого дистрибутива делают его отличным выбором как в производственной, так и в исследовательской среде, с пользовательской поддержкой, включающей полный исходный код. Многие приложения легко доступны через систему пакетов The NetBSD Packages Collection.
9. **MidnightBSD (153)** – операционная система, производная от FreeBSD. Важной целью проекта является создание простой в использовании среды рабочего стола с графическим управлением портами и конфигурацией системы с помощью GNUstep. MidnightBSD была ответвлением (fork) от бета-версии FreeBSD 6.1.
10. **BSD Router Project (158)** – это встроенный бесплатный маршрутизатор с открытым исходным кодом, основанный на FreeBSD с Quagga (программный пакет маршрутизации) и BIRD (реализация с открытым исходным кодом для маршрутизации пакетов интернет-протокола). В отличие от других встроенных сетевых инструментов, BSDRP фокусируется исключительно на маршрутизации пакетов, а не на передовых методах межсетевого экрана. Дополнительные функции могут быть добавлены в операционную систему через коллекцию портов FreeBSD.
11. **OPNsense (160)** – специализированная операционная система на основе HardenedBSD (и ответвление от pfSense), разработанная для межсетевых экранов и маршрутизаторов. Дистрибутив разработан Deciso B.V. (Нидерланды). Некоторые из функций OPNsense включают прокси-сервер с прямым кэшированием, формирование трафика (traffic shaping), обнаружение вторжений, двухфакторную аутентификацию и простую настройку клиента OpenVPN. Ориентация проекта на безопасность включает ряд уникальных функций, таких как возможность использования LibreSSL вместо OpenSSL (выбирается в графическом интерфейсе). OPNsense также включает механизм обновления, который своевременно обеспечивает важные обновления безопасности.
12. **FreeNAS (166)** – крошечная (tiny) операционная система на основе FreeBSD, которая предоставляет бесплатные сервисы сетевого хранилища (NAS) (CIFS, FTP и NFS).
13. **ClonOS (196)** – платформа с открытым исходным кодом под ключ, основанная на FreeBSD и фреймворке CBSD. ClonOS предлагает полный веб-интерфейс для

простого контроля, развертывания и управления jail во FreeBSD, контейнерами и виртуальными средами гипервизора Bhyve/Xen.

14. **FuguIta (201)** – Live CD OpenBSD с портатбельным рабочим местом, низкими требованиями к оборудованию, дополнительным программным обеспечением и частичной поддержкой японского языка. Этот live CD должен быть максимально похож на обычную OpenBSD при установке по умолчанию на жёсткий диск.
15. **XigmaNAS (203)** – (ранее NAS4Free) встраиваемый дистрибутив NAS (Network-Attached Storage) с открытым исходным кодом, основанный на FreeBSD. XigmaNAS поддерживает совместное использование в нескольких операционных системах, включая Windows, Apple и UNIX-подобные системы. XigmaNAS легко устанавливается в большинстве домашних и корпоративных сред и позволяет легко управлять большими объёмами данных и обмениваться ими по сети. XigmaNAS также включает множество различных функций потоковой передачи для совместного использования мультимедийных данных с другими устройствами в сети. XigmaNAS включает ZFS v28 (RAIDZ, RAIDZ2 и RAIDZ3), программный RAID (0,1,5), шифрование диска, отчеты SMART/электронной почты, а также поддерживает следующие протоколы: CIFS (samba), FTP, NFS, TFTP, AFP, RSYNC, Unison, iSCSI (инициатор и target), HAST, CARP, Bridge, UPnP и BitTorrent. Все они легко настраиваются из веб-интерфейса.
16. **HardenedBSD (205)** – Основанная в 2014 году Оливером Пинтером и Шоном Уэббом, HardenedBSD – это форк FreeBSD с повышенной безопасностью. Проект HardenedBSD реализует множество технологий защиты и предотвращения уязвимостей поверх FreeBSD. Проект начался с рандомизации адресного пространства (ASLR – Address Space Layout Randomization) в качестве начальной точки отсчёта, в настоящее время в дистрибутив внедряются дальнейшие методы устранения уязвимостей.

## **Solaris**

В начале 1990-х годов Sun Microsystems заменила основанную на BSD SunOS 4 на UNIX System V Release 4 (SVR4), разрабатываемую ею совместно с AT&T, а также изменила имя SunOS 5 на Solaris 2. После выхода версии 2.6 Sun Microsystems отбросила из имени «2.», и следующая версия называлась уже Solaris 7.

Фактически, Solaris — это операционная система SunOS с графической оболочкой и некоторыми дополнительными компонентами.

Начиная с версии Solaris 9 выпускалась общедоступная (в бинарном виде, то есть с закрытым исходным кодом) некоммерческая версия Solaris по лицензии CDDL. От коммерческой версии она отличалась отсутствием технической поддержки от Sun, печатной документации и некоторого объёма дополнительного проприетарного программного обеспечения (Value Added Software).

В июне 2005 года Sun Microsystems приняла решение открыть значительную часть исходного кода последней версии системы — Solaris 10 и запустить проект OpenSolaris. Стандартные бинарные сборки Solaris 10 были размещены на веб-сайте Sun Microsystems и доступны всем желающим по лицензии CDDL после регистрации на веб-сайте компании.

Разработка следующей версии — Solaris 11 (рабочее название проекта — Solaris Nevada) велась уже в сотрудничестве с сообществом разработчиков OpenSolaris. Вдобавок к исходному коду операционной системы Solaris, Sun Microsystems открыла целый ряд программного обеспечения собственной разработки для этой ОС в рамках проекта OpenSolaris.

С апреля 2010 года новый владелец интеллектуальной собственности — корпорация Oracle, изменил условия лицензирования системы Solaris 10. По новым правилам, продуктом бесплатно, в коммерческих целях, стало разрешено пользоваться только в течение 90 дней, для последующего коммерческого использования Solaris 10 и выпущенной в ноябре

2011 года версии 11 необходимо приобретение контракта на техническую поддержку операционной системы либо целиком системы от Oracle. Бинарные сборки Solaris 10 и 11 по-прежнему доступны на веб-сайте компании после регистрации, но уже по условиям особой разработческой лицензии (Oracle Technology Network Developer License), разрешающей их использование исключительно для разработки и тестирования приложений под платформу Solaris.

2 сентября 2017 года, Саймон Фиппс, бывший сотрудник Sun Microsystems, который был против приобретения компании Sun Oracle'ом, сообщил в Twitter'е, что Oracle уволил много разработчиков Solaris'a, и многие уволенные сотрудники интерпретировали это как знак того, что Oracle больше не намерен поддерживать будущую разработку платформы. В то время, как у Oracle случились массовые увольнения, продолжалась разработка Solaris 11.4, которая должна была выйти в 2018 году.

На сегодня Solaris поддерживает две основных архитектуры:

- SPARC (Sun UltraSPARC, Fujitsu SPARC64).
- 32 и 64-битные процессоры x86 производства AMD, Intel и VIA.

В Solaris 10 Sun Microsystems реализовала двоичную совместимость с Linux, что позволяет запускать Linux-приложения под Solaris на системах x86 в зонах BrandZ. Планируется достичь двоичной совместимости с FreeBSD. В Solaris SPARC эта функциональность не доступна.

В начале 2000-х самые мощные и производительные рабочие станции и многопроцессорные серверы работали на платформе SPARC/Solaris, но Intel, вступив в эпоху зрелости и расцвета (начавшуюся со всё того же поколения процессоров Core 2), получив обширную и всеобъемлющую поддержку от всех главных игроков программной индустрии, от мирового сообщества Open Source, ближе к концу 2010-х перехватила лидерство, и продолжает удерживать его до сих пор – сегодня почти весь топ (кроме многократно озвученной ранее мобильной платформы) – это x86: ноутбуки, десктопы, мощные рабочие станции и игровые компьютеры, серверы.

Хотя события относительно недавних дней (июнь 2020), когда были озвучены планы Apple по переходу на ARM – это тревожный звонок для Intel, серьёзное покушение на многолетнюю гегемонию старой (самой старой из присутствующих сегодня на рынке) архитектуры x86. Что из этого выйдет – покажут ближайшие два-три года. В своё время, пятнадцать лет назад, когда Apple перешла с PowerPC на Intel, это привело фактически к смерти тогдашнего интеловского конкурента, и полному исчезновению PPC с арены процессоров. Аппаратура тогда чего-то стоит, когда она поддержана соответствующим программным обеспечением, лишаясь такой поддержки, архитектура, сколь бы совершенна и красива она ни была, быстро приходит в упадок и уходит со сцены. Яркий пример – EPIC Itanium, 64-битный процессор от Hewlett Packard PA-RISC.

## **Краткий итог – программное обеспечение для Intel-архитектуры, современное состояние**

В 2021 году можно констатировать, что рынок софта под x86 невероятно многообразен, огромен (и в количественном и денежном выражении), насыщен качественным, мощным программным обеспечением. На выбор пользователю предлагаются десятки, точнее даже, сотни самых разнообразных операционных систем, одних только дистрибутивов Linux наберётся, наверное, более двух сотен вариантов. На сегодня Linux из всего многообразия UNIX-подобных ОС абсолютно доминирует (в количестве установленных экземпляров), далеко обогнав и проприетарные UNIX-ы, и другие проекты с открытым кодом. На упоминавшемся выше сайте Distrowatch в списке популярности (достаточно условном, рей-

тинг Distrowatch, напомним, это просто счётчик посещений страниц с описанием ОС) первая НЕ Linux операционная система – FreeBSD, расположена в самом конце третьего десятка, на 29-ом месте.

Рядовым пользователям компьютеров в первую очередь интересны не тонкости внеочередного исполнения команд в том или ином поколении процессора, или размер и количество «путей» в кэше первого или второго уровня, не длина конвейера, или версия шины PCIe. По большому счёту, пользователям не особо интересна и операционная система сама по себе, главное для него – целевое прикладное программное обеспечение и сервисы, предоставляемые ОС, с помощью которых он решает свои задачи. Это текстовые процессоры, современные браузеры, системы видеомонтажа, звуковые редакторы, системы автоматизированного проектирования, игры, наконец. В последнее время всё чаще появляется возможность работать с приложением в его обычном виде (в Windows 10 такие приложения называются классические) или в «облачном» варианте, пример – «классический» Microsoft Office с версией, совпадающей с годом выпуска или «облачный» Office365, его полный эквивалент, который работает на любой ОС, главное, чтобы браузер, с помощью которого чаще всего приходится работать с «облаками», был достаточно современным.

Заставить обычного пользователя перейти на другую версию операционной системы бывает достаточно непросто, если на его текущей, к примеру, старой-доброй Windows XP работают все ему нужные приложения, если он может зайти на все интересующие его сайты с той максимальной версией браузера, которая установлена на его XP, если на ней идут его старые любимые игры. И только если очередная итерация Microsoft Office, скажем, версия 2019 вообще откажется устанавливаться на его системе, а любимые сайты начинают массово отказывать в соединениях с браузерами типа IE 8 или Mozilla Firefox 56, или если не удастся поиграть во что-нибудь типа “World of Tanks” – вот тогда пользователь всерьёз заинтересуется и версией своей ОС, и аппаратными ресурсами своего компьютера ☺.

В программном обеспечении для серверного сегмента архитектуры x86 сегодня также можно констатировать зрелость и высокое качество присутствующих на рынке решений. В серверном сегменте на сегодня очень сильны позиции у ОС Linux, но и Windows Server там не затерялся, вполне успешно присутствуя, например, в облачных технологиях Azure от Microsoft. «Облака», по сути, это симбиоз нескольких ключевых технологий: многопроцессорной аппаратной платформы, гипервизора (виртуализация), под управлением которого работают обычные «гостевые» ОС, и продвинутые средства управления и оркестрации тысячами, десятками и даже сотнями тысяч виртуальных машин, с возможностью активировать требуемый сервис буквально в пару кликов мыши. Вместо обычной рутинной и обычно весьма продолжительной процедуры по схеме «установка ОС» – «конфигурирование сетевых настроек» – «установка и настройка необходимого программного обеспечения для получения нужного сервиса» в панели управления гипервизора выбирается готовый шаблон виртуальной машины с требуемыми параметрами – и через пару минут у вас готов, настроен и работает, например, почтовый сервер на 1000 пользователей, с правильными и согласованными настройками всех компонентов.

На сегодня хостовая ОС (ОС гипервизора, если можно так сказать) – это чаще всего или обычный Linux (гипервизор KVM от RedHat), или специально доработанный (Xen, VMware), или Windows Server с гипервизором Hyper-V (Azure).

Ещё пример «тяжеловесного» программного обеспечения для серверного сегмента: системы управления базами данных (СУБД), с практически неограниченной масштабируемостью (например, известно, что работу Facebook – а это около двух миллиардов пользователей! – в background-е поддерживает несколько сотен экземпляров (instance) СУБД MySQL).

Популярные нынче «облачные» технологии всё чаще становятся решением для организации информационно-вычислительной среды в организации, вместо классической «серверной» комнаты, набитой разнородными серверами разных поколений, купленными в

разное время, в разной комплектации, без реальной возможности масштабирования и «вширь» и «вглубь» – арендуются необходимые сервисы в «облаке»: электронная почта, система документооборота, система мониторинга, СУБД. Даже граничный межсетевой экран (firewall), с проверкой на вирусы и на спам, с обнаружением и предотвращением атак, в «облачном» исполнении сегодня – это совершенно обычное решение. И основа, бэкграунд подавляющего большинства сегодняшних мощных «облачных» сервисов, «железо» – архитектура x86. Архитектура, существующая на рынке без малого сорок лет, из которых как минимум последние три десятилетия – на лидирующих позициях – это действительно очень впечатляет.

О неотъемлемой составляющей, так сказать, бэкграунде «облачных» технологий, многомашиных кластеров и т.п. – технологиях аппаратной виртуализации – более подробно рассказано в одной из следующих глав, посвящённой архитектуре виртуальных машин. Здесь только упомянем тот факт, что компания Microsoft, всерьёз пришедшая на рынок серверной виртуализации с почти полутора-десятилетним опозданием (Hyper-V на Windows Server 2008), сумела менее, чем за десять лет не просто догнать лидера и родоначальника рынка гипервизоров на архитектуре x86 – компанию VMware – но и обогнать её по некоторым существенным параметрам и характеристикам в своих решениях.

## ГЛАВА 8. СУПЕРКОМПЬЮТЕРЫ

Автор долго выбирал, как назвать эту главу – используя англоязычную кальку «Суперкомпьютеры» или всё ближе к русскому «СуперЭВМ». Слово ЭВМ отдаёт некоторой архаикой, хотя довольно точно передаёт смысл исходного термина «компьютер» – устройство для счёта, вычислитель. В общем, хотя, на взгляд автора, термин «ЭВМ» слух особо не режет, но употребление термина «Суперкомпьютеры» в этой главе будет всё же более правильным (хотя ближе к концу раздела автор всё же чаще стал употребляться термин «суперЭВМ» ☺).

Так уж сложилась жизнь, что в области ИТ основные термины чаще всего имеют англоязычное происхождение, надо принимать это как данное. Мы уже давно не вздрагиваем от словосочетаний типа «сервер», «интерфейс», «дисплей», «флэш-память» и т.п., они стали частью русского языка. В то же время вспоминаются примеры откровенно неудачных и иногда весьма забавных попыток непременно перевести все технические термины на русский язык. Когда только появился и начал своё победное шествие по планете язык программирования Java, доводилось видеть просто восхитительный перевод английского термина, связанного с этой технологией: фраза “Java applet started” была переведена как «Ява приложеньице запущено». Смотрелось это, если честно, откровенно глуповато.

### **Определение понятия суперкомпьютер**

**Суперкомпьютер (Supercomputer, СверхЭВМ, СуперЭВМ, сверхвычислитель)** — специализированная вычислительная машина, значительно превосходящая по своим техническим параметрам и скорости вычислений большинство существующих в мире компьютеров.

Как правило, современные суперкомпьютеры представляют собой (очень) большое число высокопроизводительных серверных компьютеров, соединённых друг с другом локальной высокоскоростной магистралью для достижения максимальной производительности в рамках реализации распараллеливания вычислительной задачи.

Определение понятия «суперкомпьютер» не раз было предметом многочисленных споров и обсуждений. Чаще всего авторство термина приписывается Джорджу Майклу (George Anthony Michael) и Сиднею Фернбачу (Sidney Fernbach), в конце 60-х годов XX века работавших в Ливерморской национальной лаборатории, и компании CDC. Тем не менее, известен факт, что ещё в 1920 году газета New York World рассказывала о «супервычислениях», выполнявшихся при помощи табулятора IBM, собранного по заказу Колумбийского университета.

В общеупотребительный лексикон термин «суперкомпьютер» вошёл благодаря распространённости компьютерных систем *Сеймура Крэя*, таких как CDC 6600, CDC 7600, Cray-1, Cray-2, Cray-3 и Cray-4. Сеймур Крэй разрабатывал вычислительные машины, которые по сути становились основными вычислительными средствами правительственных, промышленных и академических научно-технических проектов США с середины 60-х годов до 1996 года. Не случайно в то время одним из популярных определений суперкомпьютера было следующее: «любой компьютер, который создал Сеймур Крэй». Сам Крэй никогда не называл свои детища суперкомпьютерами, предпочитая использовать вместо этого обычное название «компьютер».

Компьютерные системы Крэя удерживались на вершине рынка в течение 5 лет с 1985 по 1990 годы. 80-е годы XX века охарактеризовались появлением множества небольших конкурирующих компаний, занимающихся созданием высокопроизводительных компьютеров, однако к середине 90-х большинство из них оставили эту сферу деятельности, что даже заставило обозревателей заговорить о «крахе рынка суперкомпьютеров». На сегодняшний день суперкомпьютеры являются уникальными системами, создаваемыми «традиционными» игроками компьютерного рынка, такими как IBM, Hewlett-Packard, NEC и



другими, которые приобрели множество ранних компаний, вместе с их опытом и технологиями. Компания Cray по-прежнему занимает достойное место в ряду производителей суперкомпьютерной техники<sup>1</sup> (High Performance Computing).

Из-за большой гибкости самого термина до сих пор распространены довольно нечёткие представления о понятии «суперкомпьютер». Шутливая классификация Гордона Белла и Дона Нельсона, разработанная приблизительно в 1989 году, предлагала считать суперкомпьютером любой компьютер, весящий более тонны. Современные суперкомпьютеры действительно обычно весят более 1 тонны, однако далеко не каждый тяжёлый компьютер достоин чести считаться суперкомпьютером. В общем случае, суперкомпьютер — это компьютер значительно более мощный, чем доступные для большинства пользователей машины. При этом скорость технического прогресса сегодня такова, что сегодняшний лидер легко может стать завтрашним аутсайдером.

Архитектура также не может считаться признаком принадлежности к классу суперкомпьютеров. Ранние компьютеры CDC были обычными машинами, всего лишь оснащёнными быстрыми для своего времени скалярными процессорами, скорость работы которых была в несколько десятков раз выше, чем у компьютеров, предлагаемых другими компаниями.

Большинство суперкомпьютеров 1970-х оснащались векторными процессорами, а к началу и середине 1980-х небольшое число (от 4 до 16) параллельно работающих векторных процессоров практически стало стандартным суперкомпьютерным решением. Конец 1980-х и начало 1990-х годов охарактеризовались сменой магистрального направления развития суперкомпьютеров от *векторно-конвейерной* обработки к большому и сверхбольшому числу *параллельно соединённых скалярных* процессоров.

*Массово-параллельные* системы стали объединять в себе сотни, тысячи и даже десятки тысяч отдельных процессорных элементов, причём ими могли служить не только специально разработанные, но и общедоступные в свободной продаже процессоры. Большинство массово-параллельных компьютеров создавалось на основе мощных процессоров с архитектурой RISC, наподобие PowerPC или PA-RISC.

В конце 1990-х годов высокая стоимость специализированных суперкомпьютерных решений и нарастающая потребность разных слоёв общества в доступных вычислительных ресурсах привели к широкому распространению компьютерных кластеров. Эти системы характеризует использование отдельных узлов на основе дешёвых и широко доступных компьютерных комплектующих для серверов и персональных компьютеров и объединённых при помощи мощных коммуникационных систем и специализированных программно-аппаратных решений. Несмотря на кажущуюся простоту, кластеры довольно быстро заняли достаточно большой сегмент суперкомпьютерного рынка, обеспечивая высочайшую производительность при минимальной стоимости решений.

В настоящее время суперкомпьютерами принято называть компьютеры с огромной вычислительной мощностью («числодробилки» или «числогрызы» ☺). Такие машины используются для работы с приложениями, требующими наиболее интенсивных вычислений (например, прогнозирование погодно-климатических условий, моделирование ядерных испытаний и т. п.), что в том числе отличает их от серверов и мэйнфреймов (mainframe) — компьютеров с высокой общей производительностью, призванных решать типовые задачи (например, обслуживание больших баз данных или одновременная работа с множеством пользователей).

Иногда суперкомпьютеры используются для работы с одним-единственным приложением, использующим всю память и все процессоры системы; в других случаях они обеспечивают выполнение большого числа разнообразных приложений.

---

<sup>1</sup> В сентябре 2019 Cray была приобретена компанией Hewlett Packard Enterprise (HPE) за относительно небольшую сумму в \$1,4 млрд. А ещё раньше, в 2016 году HPE купила другого известного HPC-игрока, легендарную SGI (Silicon Graphics, Inc.).

Одним из первых «настоящих» суперкомпьютеров считается Cray-1, созданный в 1974 году. С помощью аппаратной поддержки векторных операций эта суперЭВМ достигала производительности в 160 миллионов операций в секунду над числами с плавающей точкой (FLOPS).

## Классификация параллельных машин

Как было отмечено выше, в начале 1990-х произошла смена генерального направления развития суперкомпьютеров от парадигмы векторно-конвейерной обработки к системам с массовым параллелизмом. Первый «настоящий» суперкомпьютер Cray-1 в базовой версии был однопроцессорным, с уникальной архитектурой процессора и компьютера в целом, очень отличающихся от архитектуры обычных компьютеров того времени. Появление Cray в середине 1970-х на тот момент казалось означало безоговорочную победу суперкомпьютеров с векторно-конвейерной архитектурой и относительно небольшим количеством высокопроизводительных ядер (процессоров) над компьютерами с массовым параллелизмом (первым в мире компьютером с массовым параллелизмом был ILLIAC IV, который, имея 64 процессора, уступал в производительности однопроцессорному Cray-1). Но повышение производительности одного ядра рано или поздно упирается в естественные физические пределы – невозможно поднять тактовую частоту кремниевого процессора выше 10-20 ГГц<sup>1</sup> – и единственным методом повышения производительности супервычислителя остаётся переход к параллельным вычислениям, когда в топовых уникальных решениях используются миллионы (это не опечатка) вычислительных ядер. Значительная часть самых производительных суперкомпьютеров, представленных сегодня в [top500](#) – это гибридные архитектуры, комбинация сотен тысяч ядер обычных универсальных CPU класса Intel Xeon, IBM Power и т.п. и сотен тысяч (миллионов) вычислительных ядер на графических процессорах (GPU). Хотя японский суперкомпьютер FUGAKU, A64FX 48C 2.2GHZ, довольно неожиданно ставший номером один<sup>2</sup> в июньском рейтинге 2020 года (и сохранивший первую позицию в ноябрьском рейтинге 2020 и июньском 2021), построен без использования GPU, на базе ARM-совместимого процессора Fujitsu A64FX (довольно неожиданная модель процессора для компьютеров из top500), с 7,630,848 ядрами!

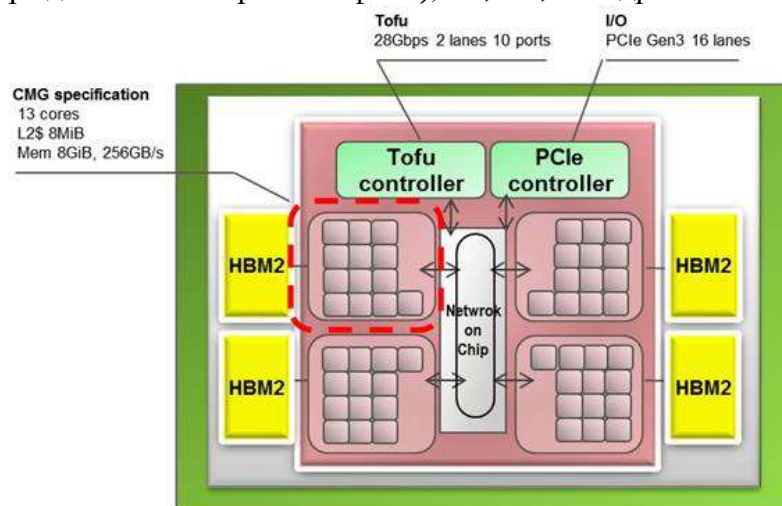


Рис.433 – Архитектура процессора Fujitsu A64FX

<sup>1</sup> Напомним в очередной раз: в начале 2000-х в роадмэпах Intel можно было найти обещания компании к 2006 году выпустить массовый процессор с тактовыми частотами до 5 ГГц, а к 2010-му – так и со всеми 10 ГГц! Интересно, чего в этом было больше – рекламы, трезвого расчёта, ошибки? Хочется думать, что на тот момент инженеры Intel искренне заблуждались, считая, что в нарушение законов физики, им всё-таки каким-то чудом удастся разогнать CPU до таких частот, не погружая процессоры, например, в жидкий азот, а используя обычное воздушное охлаждение.

<sup>2</sup> Последние несколько лет за первые места в рейтинге top500 конкурировали суперкомпьютеры из Китая и США.

*Процессор Fujitsu A64FX выпускается с использованием 7-нм техпроцесса, очевидно, что на линиях компании TSMC. Он насчитывает 8,7 млрд транзисторов. Пиковая производительность процессора для операций с двойной точностью достигает 2,7 терафлопс. Процессор без потерь на переход может выполнять операции с одинарной точностью и половинной, соответственно, в два и четыре раза быстрее. Также, за что надо благодарить тему машинного обучения, процессор A64FX оптимизирован для обработки 16- и 8-битных целочисленных значений.*

## Таксономия Флинна

В 1966 году Майкл Флинн (Michael Flynn) предложил классификацию (таксономию) архитектур ЭВМ по признакам наличия параллелизма в потоках команд и данных. Классификация (была расширена в 1972 году), базируется на понятии потока, под которым понимается последовательность элементов, команд или данных, обрабатываемая процессором. На основе числа потоков команд и потоков данных Флинн выделяет четыре класса машинных архитектур и «псевдо-машинные» SPMD/MPMD:

1. Single Instruction Stream, Single Data Stream: **SISD**.
2. Multiple Instruction Stream, Single Data Stream: **MISD**.
3. Single Instruction Stream, Multiple Data Stream: **SIMD**.
4. Multiple Instruction Stream, Multiple Data Stream: **MIMD**.
5. Single Program, Multiple Data/Multiple Programs, Multiple Data: **SPMD/MPMD**.

### SISD

**SISD** (*single instruction stream/single data stream*) – одиночный поток команд и одиночный поток данных. К этому классу относятся, прежде всего, классические последовательные машины, или иначе, машины фон-неймановского типа, например, PDP-11 или VAX 11/780. В таких машинах есть только один поток команд, все команды обрабатываются последовательно, друг за другом, и каждая команда инициирует одну операцию с одним потоком данных. Не имеет значения тот факт, что для увеличения скорости обработки команд и скорости выполнения арифметических операций может применяться конвейерная обработка - как машина CDC 6600 со скалярными функциональными устройствами, так и CDC 7600 с конвейерными попадают в этот класс.

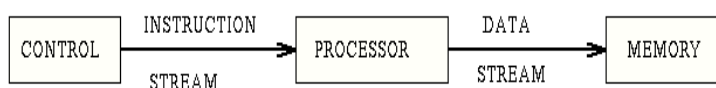


Рис.434 – SISD архитектура

### SIMD

**SIMD** (*single instruction stream/multiple data stream*) - одиночный поток команд, множественный поток данных. В архитектурах подобного рода сохраняется один поток команд, включающий, в отличие от предыдущего класса, векторные команды. Это позволяет выполнять одну арифметическую операцию сразу над многими данными – элементами век-

тора. Способ выполнения векторных операций не оговаривается, поэтому обработка элементов вектора может производиться либо процессорной матрицей, как в ILLIAC IV, либо с помощью конвейера, как, например, в машине CRAY-1.

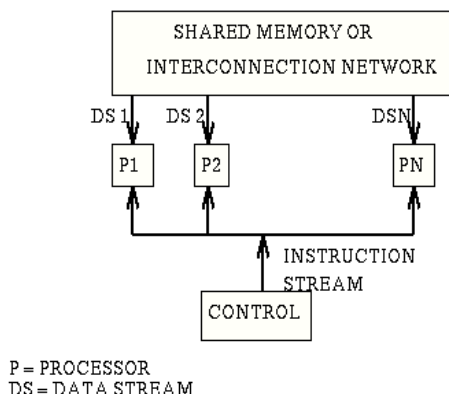


Рис.435 – SIMD архитектура

### ***SM-SIMD (shared memory SIMD)***

К этому подклассу относятся векторные процессоры. В научных вычислениях большая часть операций связана с применением какой-то одной операции к большому массиву данных. Причём эту операцию можно осуществлять над каждым элементом данных независимо друг от друга, то есть присутствовал параллелизм данных, для использования которого и были созданы векторные процессоры.

Векторные процессоры получили распространение в начале 1970-х годов, в первую очередь в суперкомпьютерах тех времен (CDC STAR-100, Cray-1). С середины 1970-х до конца 1980-х все суперкомпьютеры были векторными машинами, и под суперкомпьютером в те годы подразумевалась именно векторная машина. Векторные суперкомпьютеры до сих пор находят применение в промышленности и научных вычислениях, и они до сих пор входят в перечень продукции почти всех ведущих производителей суперкомпьютеров: NEC, Fujitsu, Hitachi, Cray. Развитие миниатюризации в вычислительной технике позволило добавить векторный способ обработки данных в современные микропроцессоры, где они представлены набором специальных команд-расширений. Выполняя их, процессор переходит в векторный режим и превращается на это время в SM-SIMD-машину.

### ***DM-SIMD (distributed memory SIMD)***

К этому подклассу относятся так называемые «матричные процессоры». Они представляют собой массив процессоров, которые контролируются одним управляющим процессором, выполняя по его команде одну операцию над своей собственной порцией данных, хранящихся в локальной памяти. Так как обмена данными между процессорами нет, не требуется никакой синхронизации, что позволяет достигать огромных скоростей вычислений и с лёгкостью расширять систему, просто увеличивая количество процессоров. Для понимания работы матричного процессора достаточно представить себе утренние телевизионные уроки по аэробике, где актёр в студии задает движения, а миллионы телезрителей повторяют их в такт одновременно по всей стране.

Так как матричные процессоры можно использовать только на ограниченном круге задач, долгое время они существовали только в виде экспериментальных, узкоспециализированных машин. Кроме того, для их производства требовалось создавать специализированные процессоры. Первой попыткой (нельзя сказать, что очень удачной) построить матричный процессор был компьютер ILLIAC IV в начале 1970-х годов, второй знаменитой попыткой были машины CM-1 и CM-2 компании Thinking Machines и суперкомпьютеры компании MasPar в начале 1980-х. Развитие миниатюризации в вычислительной технике

позволило вернуться к концепции матричных процессоров и возродить её в графических картах (GPGPU), которые используются для высокопроизводительных вычислений.

## MISD

**MISD** (*multiple instruction stream/single data stream*) – множественный поток команд и одиночный поток данных. Определение подразумевает наличие в архитектуре многих процессоров, обрабатывающих один и тот же поток данных. Параллелизм достигается выполнением разных инструкций над одним потоком данных в одно и то же время.

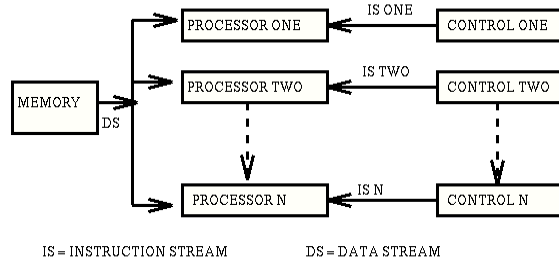


Рис.436 – MISD архитектура

В качестве примера MISD-параллелизма можно привести реализацию проверки, является ли число  $Z$  простым, пробуя разделить  $Z$  на все делители от 2 до  $Z-1$ <sup>1</sup>. Используя количество процессоров  $N = Z - 2$ , на каждый вход процессора подаётся число  $Z$ , и каждый процессор пытается разделить  $Z$  на связанный с этим процессором делитель. Таким образом, можно за один шаг проверить, является ли  $Z$  простым. Более реалистично, когда  $N < Z-2$ , и каждому процессору будет назначено подмножество делителей.

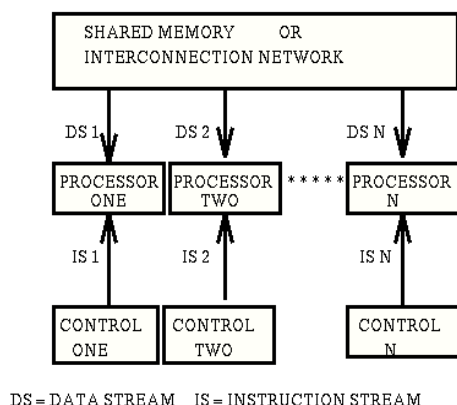
Для большинства реальных приложений MISD очень неудобно использовать, и коммерческих машин с такой конструкцией не существует. В некоторых [источниках](#) можно встретить более категорично сформулированное утверждение: «...ни Флинн, ни другие специалисты в области архитектуры компьютеров до сих пор не сумели представить убедительный пример реально существующей вычислительной системы, построенной на данном принципе. Ряд исследователей относят к данному классу конвейерные системы, однако это не нашло окончательного признания. Отсюда принято считать, что пока данный класс пуст».

## MIMD

**MIMD** (*multiple instruction stream/multiple data stream*) – множественный поток команд и множественный поток данных. Этот класс предполагает, что в вычислительной системе есть несколько устройств обработки команд, объединенных в единый комплекс и ра-

<sup>1</sup> Очевидно, что это совершенно не оптимальный алгоритм – если число нечётное, то проверять его на деление на чётные делители не имеет смысла. Здесь этот алгоритм приведён просто для иллюстрации концепции MISD.

ботающих каждое со своим потоком команд и данных. Сюда принято относить традиционные мультипроцессорные машины, многоядерные и многопоточные процессоры, а также компьютерные кластеры.



**Рис.437 – MIMD архитектура**

По принципам работы с памятью этот класс делится на подклассы.

### ***SM-MIMD (shared memory MIMD)***

В эту группу попадают многопроцессорные машины с общей памятью, многоядерные процессоры с общей памятью.

Классический и самый распространенный пример — мультипроцессоры — многопроцессорные SMP-сервера. В таких машинах память каждому процессору видна как общее адресное пространство, и процессоры обмениваются друг с другом данными по общей адресной шине через общие переменные (shared variables). Для каждого процессора доступ (время) к любому участку памяти является одинаковым.

Достоинства этого подхода: относительно легко программировать, поддержка SMP существует уже давно во всех основных операционных системах.

Недостатком данных машин является их невысокая масштабируемость: чем больше процессоров в системе, тем выше становится нагрузка на общую шину. В коммерческих вариантах таких систем максимальное число процессоров не превышает (примерно) 64.

Память, которая программисту видна как одно общее адресное пространство, может быть физически распределена по узлам системы. Такой подкласс машин называется DSM-MIMD (distributed shared memory MIMD). В этом подклассе машин у каждого процессора имеется своя локальная память, а к другим участкам памяти процессор обращается через высокоскоростное соединение. Так как доступ к разным участкам общей памяти является неодинаковым (к своему локальному участку быстрее, к другим — медленнее), то такие системы носят название NUMA (от Non-Uniform Memory Access). Так как память физически распределена, возникает трудность с тем, чтобы каждый процессор видел в памяти изменения, сделанные другими процессорами. Придумано несколько способов решения этой проблемы: через согласование кэша — ccNUMA, без согласования кэша — pccNUMA.

NUMA-системы имеют более высокую масштабируемость, позволяя создавать масштабно-параллельные вычислительные системы, где число процессоров достигает нескольких тысяч. Модель программирования в таких системах остается прежней — потоки исполнения обмениваются друг с другом данными через общие переменные.

### ***DM-MIMD (distributed memory MIMD)***

В этот подкласс попадают многопроцессорные MIMD-машины с распределённой памятью.

У каждого процессора имеется своя собственная локальная память, которая напрямую не видна другим процессорам. Каждый процессор в такой системе выполняет свою

задачу со своим набором данных в своей локальной памяти. Если процессору нужны данные из памяти другого процессора, данный процессор обменивается с другим процессором сообщениями, то есть в таких системах используется модель программирования *Message Passing*, с помощью *Parallel Virtual Machine (PVM)* или реализации *Message Passing Interface (MPI)*.

Главное преимущество DM-MIMD машин — их высокая масштабируемость, позволяющая создавать массово-параллельные системы из нескольких сотен тысяч (и даже миллионов) процессоров.

Компьютерные кластеры типа Beowulf тоже относятся к этому классу как *network of workstations*.

## SPMD и MPMD

В специализированной литературе можно встретить ещё такие подклассы MIMD-класса: **SPMD** (single program, multiple data) и **MPMD** (multiple programs, multiple data).

**SPMD** (single program, multiple data) — описывает систему, где на всех процессорах MIMD-машины выполняется только одна единственная программа, и на каждом процессоре она обрабатывает разные блоки данных. Можно сказать, что SPMD — это не аппаратная парадигма, это своего рода программный эквивалент синхронного параллелизма SIMD, в котором каждый процессор работает синхронно, как в примере на Рис.438 ниже.



Рис.438 – SIMD – парадигма синхронного параллелизма

**MPMD** (multiple programs, multiple data) — описывает систему, а) где на одном процессоре MIMD-машины работает мастер-программа, а на других подчинённая программа, работой которой руководит мастер-программа (принцип master/slave или master/worker); б) где на разных узлах MIMD-машины работают разные программы, которые по-разному обрабатывают один и тот же массив данных (принцип coupled analysis), большей частью они работают независимо друг от друга, но время от времени обмениваются данными для перехода к следующему шагу.

## Особенности

Отношение конкретных машин к конкретному классу сильно зависит от точки зрения исследователя. Так, конвейерные машины могут быть отнесены и к классу SISD (конвейер — единый процессор), и к классу SIMD (векторный поток данных с конвейерным процессором), и к классу MISD (множество процессоров конвейера обрабатывают один поток данных последовательно), и к классу MIMD — как выполнение последовательности различных команд (операций ступеней конвейера) с множественным скалярным потоком данных (вектором).

В общем, рассматривая вычислительный комплекс на разных уровнях интеграции, его можно относить к разным классам: например, высокопроизводительный массово-параллельный суперкомпьютер ASCI Blue Pacific являлся DM-MIMD-машиной, где каждый узел общался с другим с помощью передачи сообщений (*message passing*), но каждый узел в этом



суперкомпьютере, в свою очередь, представлял собой многопроцессорный SMP-сервер RS/6000, то есть являлся SM-MIMD-машиной. В то же время высокопроизводительный массово-параллельный суперкомпьютер ASCI Blue Mountain являлся системой SMP-серверов (SM-MIMD), связанных в общую машину с распределённой общей памятью (то есть DSM-MIMD).

## Основные классы современных параллельных компьютеров

Основным параметром классификации параллельных компьютеров является наличие общей (SMP) или распределенной памяти (MPP). Нечто среднее между SMP и MPP представляют собой NUMA-архитектуры, где память физически распределена, но логически общедоступна. Кластерные системы являются более дешёвым вариантом MPP. При поддержке команд обработки векторных данных говорят о векторно-конвейерных процессорах, которые, в свою очередь могут объединяться в PVP-системы с использованием общей или распределённой памяти (<https://parallel.ru/computers/classes.html>).

## Массивно-параллельные системы (MPP)

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Архитектура             | <p>Система состоит из однородных <i>вычислительных узлов</i>, включающих:</p> <ul style="list-style-type: none"> <li>• один или несколько центральных процессоров (обычно RISC),</li> <li>• локальную память (прямой доступ к памяти других узлов невозможен),</li> <li>• коммуникационный процессор или сетевой адаптер</li> <li>• иногда – жесткие диски (как в SP) и/или другие устройства В/В</li> </ul> <p>К системе могут быть добавлены специальные узлы ввода-вывода и управляющие узлы. Узлы связаны через некоторую коммуникационную среду (высокоскоростная сеть, коммутатор и т.п.)</p> |
| Примеры                 | IBM RS/6000 SP2, Intel PARAGON/ASCI Red, CRAY T3E, Hitachi SR8000, транспьютерные системы Parsytec.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Масштабируемость        | Общее число процессоров в реальных системах достигает нескольких тысяч (ASCI Red, Blue Mountain).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Операционная система    | <p>Существуют два основных варианта:</p> <ol style="list-style-type: none"> <li>1. Полноценная ОС работает только на управляющей машине (front-end), на каждом узле работает сильно урезанный вариант ОС, обеспечивающие только работу расположенной в нем ветви параллельного приложения. Пример: Cray T3E.</li> <li>2. На каждом узле работает полноценная UNIX-подобная ОС (вариант, близкий к <i>кластерному</i> подходу). Пример: IBM RS/6000 SP + ОС AIX, устанавливаемая отдельно на каждом узле.</li> </ol>                                                                                 |
| Модель программирования | Программирование в рамках модели <i>передачи сообщений</i> (MPI, PVM, BSPlib)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

## Симметричные мультипроцессорные системы (SMP)

|                         |                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Архитектура             | Система состоит из нескольких однородных процессоров и массива общей памяти (обычно из нескольких независимых блоков). Все процессоры имеют доступ к любой точке памяти с одинаковой скоростью. Процессоры подключены к памяти либо с помощью общей шины (базовые 2–4 процессорные SMP-сервера), либо с помощью crossbar-коммутатора (HP 9000). Аппаратно поддерживается когерентность кэшей. |
| Примеры                 | HP 9000 V-class, N-class; SMP-сервера и рабочие станции на базе процессоров Intel (IBM, HP, Compaq, Dell, ALR, Unisys, DG, Fujitsu и др.).                                                                                                                                                                                                                                                    |
| Масштабируемость        | Наличие общей памяти сильно упрощает взаимодействие процессоров между собой, однако накладывает сильные ограничения на их число – не более 32 в реальных системах. Для построения масштабируемых систем на базе SMP используются <i>кластерные</i> или NUMA-архитектуры.                                                                                                                      |
| Операционная система    | Вся система работает под управлением единой ОС (обычно UNIX-подобной, но для Intel-платформ поддерживается Windows NT). ОС автоматически (в процессе работы) распределяет процессы/нити по процессорам (scheduling), но иногда возможна и явная привязка.                                                                                                                                     |
| Модель программирования | Программирование в модели <i>общей памяти</i> . (POSIX threads, OpenMP). Для SMP-систем существуют сравнительно эффективные средства автоматического распараллеливания.                                                                                                                                                                                                                       |

## Системы с неоднородным доступом к памяти (NUMA)

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Архитектура             | Система состоит из однородных базовых модулей (плат), состоящих из небольшого числа процессоров и блока памяти. Модули объединены с помощью высокоскоростного коммутатора. Поддерживается единое адресное пространство, аппаратно поддерживается доступ к удаленной памяти, т.е. к памяти других модулей. При этом доступ к локальной памяти в несколько раз быстрее, чем к удаленной.<br>В случае, если аппаратно поддерживается когерентность кэшей во всей системе (обычно это так), говорят об архитектуре <b>cc-NUMA</b> (cache-coherent NUMA) |
| Примеры                 | HP HP 9000 V-class в SCA-конфигурациях, SGI Origin2000, Sun HPC 10000, IBM/Sequent NUMA-Q 2000, SNI RM600.                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Масштабируемость        | Масштабируемость NUMA-систем ограничивается объемом адресного пространства, возможностями аппаратуры поддержки когерентности кэшей и возможностями операционной системы по управлению большим числом процессоров. На настоящий момент, максимальное число процессоров в NUMA-системах составляет 256 (Origin2000).                                                                                                                                                                                                                                  |
| Операционная система    | Обычно вся система работает под управлением единой ОС, как в <b>SMP</b> . Но возможны также варианты динамического "подразделения" системы, когда отдельные "разделы" системы работают под управлением разных ОС (например, Windows NT и UNIX в NUMA-Q 2000).                                                                                                                                                                                                                                                                                       |
| Модель программирования | Аналогично <b>SMP</b> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

## Параллельные векторные системы (PVP)

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Архитектура             | Основным признаком PVP-систем является наличие специальных векторно-конвейерных процессоров, в которых предусмотрены команды однотипной обработки векторов независимых данных, эффективно выполняющиеся на конвейерных функциональных устройствах.<br>Как правило, несколько таких процессоров (1–16) работают одновременно над общей памятью (аналогично <b>SMP</b> ) в рамках многопроцессорных конфигураций. Несколько таких узлов могут быть объединены с помощью коммутатора (аналогично <b>MPP</b> ). |
| Примеры                 | NEC SX-4/SX-5, линия векторно-конвейерных компьютеров CRAY: от CRAY-1, CRAY J90/T90, CRAY SV1, CRAY X1, серия Fujitsu VPP.                                                                                                                                                                                                                                                                                                                                                                                  |
| Модель программирования | Эффективное программирование подразумевает векторизацию циклов (для достижения разумной производительности одного процессора) и их распараллеливание (для одновременной загрузки нескольких процессоров одним приложением).                                                                                                                                                                                                                                                                                 |

## Кластерные системы

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Архитектура             | Набор рабочих станций (или даже ПК) общего назначения, используется в качестве дешевого варианта <b>массивно-параллельного</b> компьютера. Для связи узлов используется одна из стандартных сетевых технологий (Fast/Gigabit Ethernet, Myrinet) на базе шинной архитектуры или коммутатора.<br><br>При объединении в кластер компьютеров разной мощности или разной архитектуры, говорят о <b>гетерогенных</b> (неоднородных) кластерах.<br><br>Узлы кластера могут одновременно использоваться в качестве пользовательских рабочих станций. В случае, когда это не нужно, узлы могут быть существенно облегчены и/или установлены в стойку. |
| Примеры                 | NT-кластер в NCSA, Beowulf-кластеры.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Операционная система    | Используются стандартные для рабочих станций ОС, чаще всего, свободно распространяемые – Linux/FreeBSD, вместе со специальными средствами поддержки параллельного программирования и распределения нагрузки.                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Модель программирования | Программирование, как правило, в рамках модели передачи сообщений (чаще всего – <b>MPI</b> ). Дешевизна подобных систем оборачивается большими накладными расходами на взаимодействие параллельных процессов между собой, что сильно сужает потенциальный класс решаемых задач.                                                                                                                                                                                                                                                                                                                                                              |

## Краткая история появления параллелизма в архитектуре ЭВМ

Идея параллельной обработки в ЭВМ появились очень давно. Изначально эти технологии внедрялись в самых передовых – а потому единичных – компьютерах своего времени. Затем, после должной отработки технологии и удешевления производства, они спустились в компьютеры среднего класса, и наконец сегодня всё это в полном объеме воплощается в массовых рабочих станциях и персональных компьютерах.

Все основные нововведения в архитектуре современных процессоров на самом деле используются ещё со времен, когда ни микропроцессоров, ни даже понятия суперкомпьютеров еще не было.

### IBM 701, IBM 704

*IBM 701 (1953), IBM 704 (1955): разрядно-параллельная память, разрядно-параллельная арифметика.*

Все самые первые компьютеры (EDSAC, EDVAC, UNIVAC) имели разрядно-последовательную память, из которой слова считывались последовательно бит за битом. Первым

коммерчески доступным компьютером, использующим разрядно-параллельную память (на CRT) и разрядно-параллельную арифметику, стал IBM 701, а наибольшую популярность получила модель IBM 704 (продано 150 экз.), в которой, помимо сказанного, была впервые применена память на ферритовых сердечниках и аппаратное АУ с плавающей точкой.

## **IBM 709**

*IBM 709 (1958): независимые процессоры ввода/вывода.*

Процессоры первых компьютеров сами управляли вводом/выводом. Однако скорость работы самого быстрого внешнего устройства, а по тем временам это была магнитная лента, была в тысячи раз меньше скорости процессора, поэтому во время операций ввода/вывода процессор фактически простаивал. В 1958 году к компьютеру IBM 704 присоединили 6 независимых процессоров ввода/вывода, которые после получения команд могли работать параллельно с основным процессором, а сам компьютер переименовали в IBM 709. Данная модель получилась удивительно удачной, так как вместе с модификациями было продано около 400 экземпляров, причем последний был выключен в 1975 году – 20 лет работы!

## **IBM STRETCH**

*IBM STRETCH (1961): опережающий просмотр вперед, расслоение памяти.*

В 1956 году IBM подписывает контракт с Лос-Аламосской научной лабораторией на разработку компьютера STRETCH, имеющего две принципиально важные особенности: опережающий просмотр вперед (lookahead) для выборки команд и расслоение памяти (interleave) на два банка для согласования низкой скорости выборки из памяти и скорости выполнения операций.

## **ATLAS**

*ATLAS (1963): конвейер команд.*

Впервые конвейерный принцип выполнения команд был использован в машине ATLAS, разработанной в Манчестерском университете. Выполнение команд разбито на 4 стадии: выборка команды, вычисление адреса операнда, выборка операнда и выполнение операции. Конвейеризация позволила уменьшить время выполнения команд с 6 мкс до 1,6 мкс. Данный компьютер оказал огромное влияние, как на архитектуру ЭВМ, так и на программное обеспечение: в нём впервые использована мультипрограммная ОС, основанная на использовании виртуальной памяти и системы прерываний.

## **CDC 6600**

*CDC 6600 (1964): независимые функциональные устройства.*

Фирма Control Data Corporation (CDC) при непосредственном участии одного из её основателей, Сеймура Р.Крэя (Seymour R.Cray) выпускает компьютер CDC-6600 – первый компьютер, в котором использовалось несколько независимых функциональных устройств. Для сравнения с сегодняшним днём приведём некоторые параметры этого компьютера:

- время такта 100нс,
- производительность 2...3 млн. операций в секунду,
- оперативная память разбита на 32 банка по 4096 60-ти разрядных слов,
- цикл памяти 1мкс,
- 10 независимых функциональных устройств.

Машина имела громадный успех на научном рынке, активно вытесняя машины компании IBM.

## CDC 7600

*CDC 7600 (1969): конвейерные независимые функциональные устройства.*

CDC выпускает компьютер CDC-7600 с восемью независимыми конвейерными функциональными устройствами – сочетание параллельной и конвейерной обработки. Основные параметры:

- такт 27,5 нс,
- 10...15 млн. опер/сек.,
- 8 конвейерных ФУ,
- 2-х уровневая память.

## ILLIAC IV

*ILLIAC IV (1974): матричные процессоры.*

Проект: 256 процессорных элементов (ПЭ) = 4 квадранта по 64ПЭ, возможность реконфигурации: 2 квадранта по 128ПЭ или 1 квадрант из 256ПЭ, такт 40нс, производительность 1 ГФлоп. Работы были начаты в 1967 году, к концу 1971 изготовлена система из 1 квадранта, в 1974г. она введена в эксплуатацию, доводка велась до 1975 года. Центральная часть: устройство управления (УУ) + матрица из 64 ПЭ.

- УУ – это простая ЭВМ с небольшой производительностью, управляющая матрицей ПЭ; все ПЭ матрицы работали в синхронном режиме, выполняя в каждый момент времени одну и ту же команду, поступившую от УУ, но над своими данными;
- ПЭ имел собственное АЛУ с полным набором команд, ОП – 2Кслова по 64 разряда, цикл памяти 350нс, каждый ПЭ имел непосредственный доступ только к своей ОП;
- сеть пересылки данных: двумерный тор со сдвигом на 1 по границе по горизонтали;

Несмотря на не слишком хорошие результаты в сравнении с заложенными в проект параметрами – стоимость оказалась в 4 раза выше, был изготовлен лишь 1 квадрант из четырёх, такт 80 нс, реальная производительность до 50 МФлоп – данный проект оказал огромное влияние на архитектуру последующих машин, построенных по схожему принципу, в частности: PEPE, BSP, ICL DAP.

## CRAY 1<sup>1</sup>

*CRAY 1 (1976): векторно-конвейерные процессоры.*

В 1972 году С.Крэй покидает CDC и основывает свою компанию Cray Research, которая в 1976г. выпускает первый векторно-конвейерный компьютер CRAY-1: время такта 12.5нс, 12 конвейерных функциональных устройств, пиковая производительность 160 миллионов операций в секунду, оперативная память до 1Мслова (слово – 64 разряда), цикл памяти 50нс. Главным новшеством является введение векторных команд, работающих с целыми массивами независимых данных и позволяющих эффективно использовать конвейерные функциональные устройства.

---

<sup>1</sup> Более подробное описание архитектур ILLIAC IV и Cray 1 будет приведено далее в соответствующих разделах.

## Использование параллельных вычислительных систем

К сожалению чудеса в жизни редко случаются. Гигантская производительность параллельных компьютеров и супер-ЭВМ с лихвой компенсируется сложностями их использования. Предположим, у вас есть программа и доступ, скажем, к 256-процессорному компьютеру. И вы, ожидаете, что ваша программа будет выполняться в 256 раз быстрее, чем на одном процессоре. Ну, или хотя бы, раз в 200 быстрее. Но такого прироста производительности в реальности достичь никогда не получится.

### Закон Амдала-Уэра и его следствия

Предположим, что в вашей программе доля операций, которые нужно выполнять последовательно, равна  $f$ , где  $0 \leq f \leq 1$  (при этом доля понимается не по статическому числу строк кода, а по числу операций в процессе выполнения). Крайние случаи в значениях  $f$  соответствуют полностью параллельным ( $f=0$ ) и полностью последовательным ( $f=1$ ) программам. Для того, чтобы оценить, какое ускорение  $S$  может быть получено на компьютере из 'p' процессоров при данном значении  $f$ , можно воспользоваться законом Амдала:

$$S \leq \frac{1}{f + (1-f)/p}$$

Если 9/10 программы выполняется параллельно, а 1/10 – по-прежнему последовательно, то ускорения более, чем в 10 раз получить в принципе невозможно вне зависимости от качества реализации параллельной части кода и числа используемых процессоров (ясно, что 10 получается только в том случае, когда время исполнения последовательной части равно 0).

Посмотрим на проблему с другой стороны: а какую же часть кода надо ускорить (а значит и предварительно исследовать), чтобы получить заданное ускорение? Ответ можно найти в следствии из закона Амдала: для того чтобы ускорить выполнение программы в  $q$  раз необходимо ускорить не менее, чем в  $q$  раз не менее, чем  $(1-1/q)$ -ю часть программы. Следовательно, если есть желание ускорить программу в 100 раз по сравнению с ее последовательным вариантом, то необходимо получить не меньшее ускорение не менее, чем на 99,99% кода!

Отсюда первый вывод – прежде, чем основательно переделывать код для его переноса на параллельный компьютер (а любой суперкомпьютер, в частности, является таковым) надо основательно подумать. Если, оценив заложенный в программе алгоритм вы поняли, что доля последовательных операций велика, то на значительное ускорение рассчитывать явно не приходится и нужно думать о кардинальной смене критичных частей алгоритма.

В ряде случаев последовательный характер алгоритма изменить не очень сложно. Допустим, что в программе есть следующий фрагмент для вычисления суммы  $n$  чисел, записанный на некоем псевдоязыке:

```
s = 0
Do i = 1, n
    s = s + a(i)
EndDo
```

По своей природе он строго последователен, так как на  $i$ -ой итерации цикла требуется результат от  $(i-1)$ -й и все итерации выполняются одна за одной. Имеем 100% последовательных операций, а значит и никакого эффекта от использования параллельных компьютеров. Вместе с тем, выход очевиден. Поскольку в большинстве реальных программ нет существенной разницы, в каком порядке складывать числа, выберем иную схему сложения. Сначала найдем сумму пар соседних элементов:  $a(1) + a(2)$ ,  $a(3) + a(4)$ ,  $a(5) + a(6)$  и т.д.

Заметим, что при такой схеме все пары можно складывать одновременно (параллельно)! Действуя аналогично на следующих шагах, получим вариант параллельного алгоритма.

Казалось бы, в данном конкретном примере все проблемы удалось разрешить. Но представьте, что доступные вам процессоры разнородны по своей производительности. Значит будет такой момент, когда кто-то из них ещё трудится, а кто-то уже все сделал и бесполезно простаивает в ожидании. Если разброс в производительности компьютеров большой, то и эффективность всей системы при равномерной загрузке процессоров будет крайне низкой. Итоговая производительность системы в целом определяется самым медленным её звеном – это один вариантов формулировки закона Амдала – если в эстафете участвуют заяц и черепаха, то общее время забега в значительной степени определяется временем прохождения своего этапа более медленной черепахой.

Но пойдём дальше и предположим, что все процессоры одинаковы. Проблемы кончились? Опять нет! Процессоры выполнили свою работу, но результат-то надо передать другому для продолжения процесса суммирования... А на передачу уходит время – и в это время процессоры опять простаивают...

Словом, заставить параллельную вычислительную систему или суперЭВМ работать с максимальной эффективностью на конкретной программе – это задача не из простых, поскольку *необходимо тщательное согласование структуры программ и алгоритмов с особенностями архитектуры параллельных вычислительных систем.*

Парадигма параллельного программирования кардинально отличается от классического последовательного подхода к программированию. На сегодняшний день имеются мощные развитые системы, библиотеки и языки параллельного программирования, что упрощает множество проблем, предоставляя пользователю механизмы для организации параллельных вычислений. Среди них можно отметить MPI, PVM, языки Cisl, NESL, ZPL, Java, а также расширение всевозможных обычных языков программирования.

Ниже, в таблице на Рис.439, представлены численные значения ускорения работы программы в зависимости от значения параметров  $f$  (доля последовательных операций) и  $p$  (количество процессорных элементов/потоков выполнения).

| Число ПЭ | Доля последовательных вычислений |      |      |       |       |
|----------|----------------------------------|------|------|-------|-------|
|          | 50%                              | 25%  | 10%  | 5%    | 2%    |
| 2        | 1,33                             | 1,60 | 1,82 | 1,90  | 1,96  |
| 8        | 1,78                             | 2,91 | 4,71 | 5,93  | 7,02  |
| 32       | 1,94                             | 3,66 | 7,80 | 12,55 | 19,75 |
| 512      | 1,99                             | 3,97 | 9,83 | 19,28 | 45,63 |
| 2048     | 2,00                             | 3,99 | 9,96 | 19,82 | 48,83 |

**Рис.439 – Ускорение программы в зависимости от доли последовательных вычислений и количества процессорных элементов**

В этой таблице особенно обескураживающим и обидным выглядит результат в самом правом нижнем углу – на 2048 процессорах и всего при 2% доле последовательных вычислений максимально достижимое ускорение будет меньше, чем в ~49 раз! Более чем в сорок раз медленнее ожидаемого (где-то в глубине души) возможного ускорения в ~2000 раз.

На Рис.440 закон Амдала представлен в виде графиков:

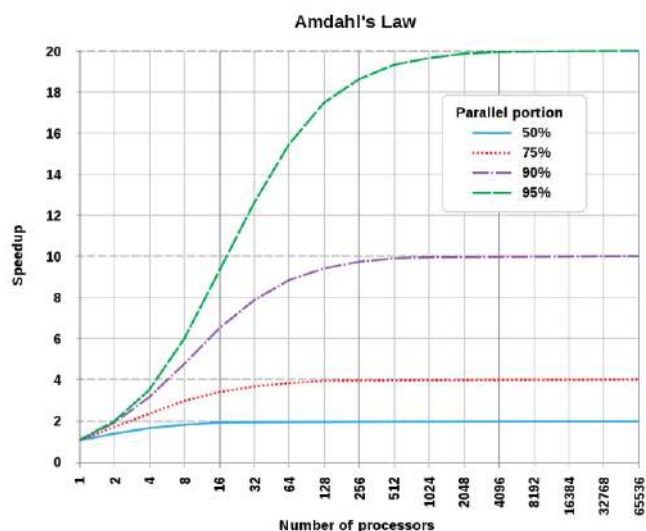


Рис.440 – Графическая иллюстрация закона Амдала

## Параллельные вычисления на GPU

Графические процессоры (graphics processing unit, GPU) — яркий пример того, как технология, исходно спроектированная для задач графической обработки, распространилась на, казалось бы, не связанную область высокопроизводительных вычислений. Современные GPU являются сердцем множества сложнейших проектов в сфере суперкомпьютеров, машинного обучения и анализа данных.

Графические процессоры за последние несколько лет сильно изменились. Помимо колоссального прироста производительности, произошло разделение устройств по типу использования. Так, в отдельное направление выделяются видеокарты для домашних игровых систем и установок виртуальной реальности. Появляются мощные узкоспециализированные вычислительные устройства: для серверных систем одним из ведущих ускорителей является NVIDIA Tesla P100, разработанный именно для промышленного использования в дата-центрах. Помимо GPU активно ведутся исследования в сфере создания нового типа процессоров, имитирующих работу головного мозга. Примером может служить однокристальная платформа Kirin 970 с собственным нейроморфным процессором для задач, связанных с нейронными сетями и распознаванием образов.

Для начала вспомним, что же такое GPU. Graphics Processing Unit — это графический процессор, широко используемый в настольных и серверных системах. Отличительной особенностью этого устройства является ориентированность на *массовые параллельные* вычисления. В отличие от графических процессоров, архитектура другого вычислительного модуля в компьютере – CPU (Central Processor Unit) – исходно была спроектирована для *последовательной* обработки данных. Если количество ядер в обычном CPU измеряется десятками, то в GPU их счёт идет на тысячи, что, конечно же, накладывает ограничения на типы выполняемых команд, однако обеспечивает высокую вычислительную производительность в задачах, включающих параллелизм.

Развитие видеопроцессоров на ранних этапах было тесно связано с нарастающей потребностью в отдельном вычислительном устройстве для обработки двух- и трёхмерной графики. До появления отдельных схем видеоконтроллеров в 1970-х годах вывод изображения осуществлялся через использование дискретной логики, что сказывалось на увеличенном энергопотреблении и больших размерах печатных плат видеокарт. Специализированные микросхемы позволили выделить разработку устройств, предназначенных для работы с графикой, в отдельное направление.

Следующим революционным событием стало появление нового класса более сложных и многофункциональных устройств — видеопроцессоров. В 1996 году компания 3dfx



Interactive выпустила чипсет Voodoo Graphics, который быстро занял до 85% рынка специализированных видеоустройств и стал лидером в области 3D графики того времени. После серии неудачных решений менеджмента компании, среди которых была покупка производителя видеокарт STB, 3dfx уступила первенство NVIDIA и ATI (позднее AMD), а в 2002 объявила о своем банкротстве. В 2021 году в «лиге» разработчиков дискретных высокопроизводительных GPU неожиданно появился новый старый игрок – компания Intel.

## Архитектура параллельных вычислений CUDA

В 2006 году NVIDIA объявила о выпуске линейки продуктов GeForce 8 series, которая положила начало новому классу устройств, предназначенных для общих вычислений на графических процессорах (**GPGPU**<sup>1</sup>). В ходе разработки NVIDIA пришла к пониманию, что большее число ядер, работающих на меньшей частоте, более эффективны для параллельных нагрузок, чем малое число более производительных ядер. Видеопроцессоры нового поколения обеспечили поддержку параллельных вычислений не только для обработки видеопотоков, но также для проблем, связанных с машинным обучением, линейной алгеброй, статистикой и другими научными или коммерческими задачами.

Компанией Nvidia была представлена технология (по сути идеология) под названием **CUDA: (Compute Unified Device Architecture)** — программно-аппаратная архитектура параллельных вычислений, которая позволяет существенно увеличить вычислительную производительность благодаря использованию графических процессоров этой компании. Вот так, в представлении Nvidia, выглядит путь к вычислениям на GPU (Рис.441):



Рис.441 – Путь к общим вычислениям на GPU

<sup>1</sup> **GPGPU** (также GPGP, GP<sup>2</sup>U, *General-purpose computing on graphics processing units*, неспециализированные вычисления на графических процессорах) — техника использования графического процессора видеокарты, предназначенного для компьютерной графики, для производства математических вычислений, которые обычно производит центральный процессор. Это стало возможным благодаря добавлению программируемых шейдерных блоков и переходу к более высокой арифметической точности растровых конвейеров, что позволяет разработчикам ПО использовать потоковые процессоры видеокарт для выполнения не графических вычислений.

CUDA SDK позволяет программистам реализовывать на специальных упрощённых диалектах языков программирования Си, С++ и Фортран алгоритмы, выполнимые на графических и тензорных процессорах Nvidia. Архитектура CUDA даёт разработчику возможность по своему усмотрению организовывать доступ к набору инструкций графического или тензорного ускорителя и управлять его памятью. Функции, ускоренные при помощи CUDA, можно вызывать из различных языков, в т.ч. Python, MATLAB и т.п.

В качестве доказательства гораздо большей эффективности и производительности вычислителей на основе GPU при решении параллельных задач Nvidia приводит графики роста производительности «классических» CPU в сравнении с ростом производительности графических ускорителей (Рис.442):



Рис.442 – Сравнение роста вычислительной производительности CPU и GPU

Причин всё большего отставания обычных универсальных CPU по производительности в параллельных вычислениях было несколько: во-первых, если посмотреть, как распределена площадь кристалла у CPU и GPU, сразу бросается в глаза, что собственно «вычислительная» компонента (ALU) у «классических» процессоров занимает меньше четверти(!) площади чипа (зелёный цвет на Рис.443), тогда как вся остальная «территория» – это схемы кэш-памяти, сложная и громоздкая логика управления и т.п. невычислительные элементы. На GPU «вычислительная» часть занимает более чем 90% площади кристалла.



Рис.443 – «К.п.д.» расходования площади кристалла для CPU и GPU

Отсюда логично следует вторая причина – гораздо большее количество вычислительных ядер, которые удаётся разместить на одном чипе GPU, в сравнении с количеством ядер в универсальном CPU. Да, каждое из существенно более простых ядер GPU довольно значительно уступает в производительности отдельному ядру CPU, но если в CPU их (ядер) 2...4...8...64 (сегодняшний рекорд AMD EPYC), то в GPU их *тысячи*. Кстати, на сегодня рекорд по количеству транзисторов на одном кристалле принадлежит как раз графическим ускорителям Nvidia – около 30 млрд. штук! Помогло им выиграть в этом соревновании и то, что GPU имеют гораздо более регулярную структуру, чем универсальные CPU.

Различия в изначальной постановке задач перед CPU и GPU привели к значительным расхождениям в архитектуре устройств — высокая частота против многоядерности. Для

графических процессоров это заложило вычислительный потенциал, который в полной мере реализуется в настоящее время. Видеопроцессоры с внушительным количеством более слабых вычислительных ядер отлично справляются с параллельными вычислениями. Центральный же процессор, исторически спроектированный для работы с последовательными задачами, остается лучшим в своей области.

## Программная архитектура

Первоначальная версия CUDA SDK была представлена 15 февраля 2007 года. В основе интерфейса программирования приложений CUDA лежит язык Си с некоторыми расширениями. Для успешной трансляции кода на этом языке в состав CUDA SDK входит собственный Си-компилятор командной строки *nvcc* компании Nvidia. Компилятор *nvcc* создан на основе открытого компилятора Open64 и предназначен для трансляции *host*-кода (главного, управляющего кода) и *device*-кода (аппаратного кода) (файлов с расширением *.cu*) в объектные файлы, пригодные для процесса сборки конечной программы или библиотеки в любой среде программирования, например, в NetBeans.

В архитектуре CUDA используется модель памяти грид (сетка), кластерное моделирование потоков и SIMD-инструкции. Применима не только для высокопроизводительных графических вычислений, но и для различных научных вычислений с использованием видеокарт nVidia. Учёные и исследователи широко используют CUDA в различных областях, включая астрофизику, вычислительную биологию и химию, моделирование динамики жидкостей, электромагнитных взаимодействий, компьютерную томографию, сейсмический анализ и многое другое. В CUDA имеется возможность подключения к приложениям, использующим OpenGL и Direct3D. CUDA — кроссплатформенное программное обеспечение для таких операционных систем, как Linux, Mac OS X и Windows. 22 марта 2010 года nVidia выпустила CUDA Toolkit 3.0, который содержал поддержку OpenCL.

## API CUDA

Команда разработчиков CUDA создала набор программных уровней для работы с GPU, схема представлена ниже, на диаграмме (Рис.444):

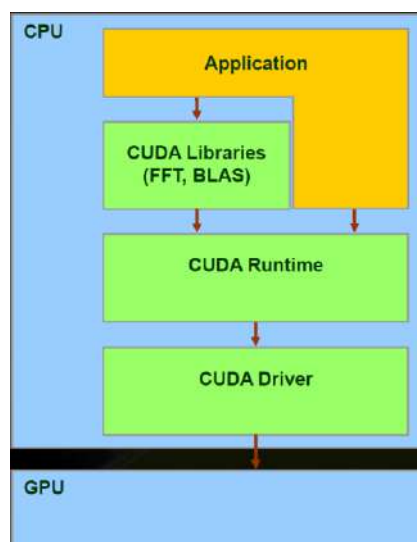


Рис.444 – API CUDA

Как можно видеть на диаграмме, CUDA обеспечивает два API:

- высокоуровневый API: CUDA Runtime API;
- низкоуровневый API: CUDA Driver API.

Поскольку высокоуровневый API реализован над низкоуровневым, каждый вызов функции уровня Runtime разбивается на более простые инструкции, которые обрабатывает Driver API. Эти два API взаимно исключают друг друга: программист может использовать один или другой API, но смешивать вызовы функций двух API не получится.

Третий программный уровень отдан библиотекам:

- **CUBLAS** – CUDA вариант BLAS (Basic Linear Algebra Subprograms), предназначенный для вычислений задач линейной алгебры и использующий прямой доступ к ресурсам GPU;
- **CUFFT** – CUDA вариант библиотеки Fast Fourier Transform для расчёта быстрого преобразования Фурье, широко используемого при обработке сигналов. Поддерживаются следующие типы преобразований: complex-complex (C2C), real-complex (R2C) и complex-real (C2R).

CUBLAS – это переведённые на язык CUDA стандартные алгоритмы линейной алгебры; на данный момент поддерживается только определённый набор основных функций CUBLAS.

CUFFT – CUDA вариант функции быстрого преобразования Фурье – широко используемой и очень важной при анализе сигналов, фильтрации и т.п. CUFFT предоставляет простой интерфейс для эффективного вычисления FFT на видеочипах производства NVIDIA без необходимости в разработке собственного варианта FFT для GPU.

## Оборудование

Платформа CUDA впервые появились на рынке с выходом чипа NVIDIA восьмого поколения G80 и стала присутствовать во всех последующих сериях графических чипов, которые используются в семействах ускорителей GeForce, Quadro и NVidia Tesla.

Первая серия оборудования, поддерживающая CUDA SDK, G8x, имела 32-битный векторный процессор одинарной точности, использующий CUDA SDK как API (CUDA поддерживает тип double языка Си, однако сейчас его точность понижена до 32-битного с плавающей запятой). Более поздние процессоры GT200 имеют поддержку 64-битной точности (только для SFU), но производительность значительно хуже, чем для 32-битной точности (из-за того, что SFU всего два на каждый потоковый мультипроцессор, а скалярных процессоров — восемь). Графический процессор организует аппаратную многопоточность, что позволяет задействовать все ресурсы графического процессора. Таким образом, открывается перспектива переложить функции физического ускорителя на графический ускоритель (пример реализации — PhysX). Также открываются широкие возможности использования графического оборудования компьютера для выполнения сложных неграфических вычислений: например, в вычислительной биологии, в физике высоких энергий (моделирование и поиск треков), и в иных отраслях науки.

По сравнению с традиционным подходом к организации вычислений общего назначения посредством возможностей графических API, у архитектуры CUDA отмечают следующие преимущества в этой области:

- Интерфейс программирования приложений CUDA (CUDA API) основан на стандартном языке программирования Си с некоторыми ограничениями. По мнению разработчиков, это должно упростить и сгладить процесс изучения архитектуры CUDA;
- Разделяемая между потоками память (shared memory) размером в 16 КБайт может быть использована под организованный пользователем кэш с более широкой полосой пропускания, чем при выборке из обычных текстур;
- Более эффективные транзакции между памятью центрального процессора и видеопамятью;

- Полная аппаратная поддержка целочисленных и битовых операций;
- Поддержка компиляции кода GPU средствами открытого проекта LLVM.

Существуют некоторые ограничения при написании программ под архитектуру CUDA: все функции, выполнимые на устройстве, не поддерживают рекурсии (но в версии CUDA Toolkit 3.1 поддерживаются указатели и рекурсия).

Ниже приведены схема (Рис.445) и внешний вид (Рис.446) компактного гибридного суперкомпьютера (комбинация CPU/GPU), который может быть использован, например, в качестве вычислительного узла многомашиного кластера.

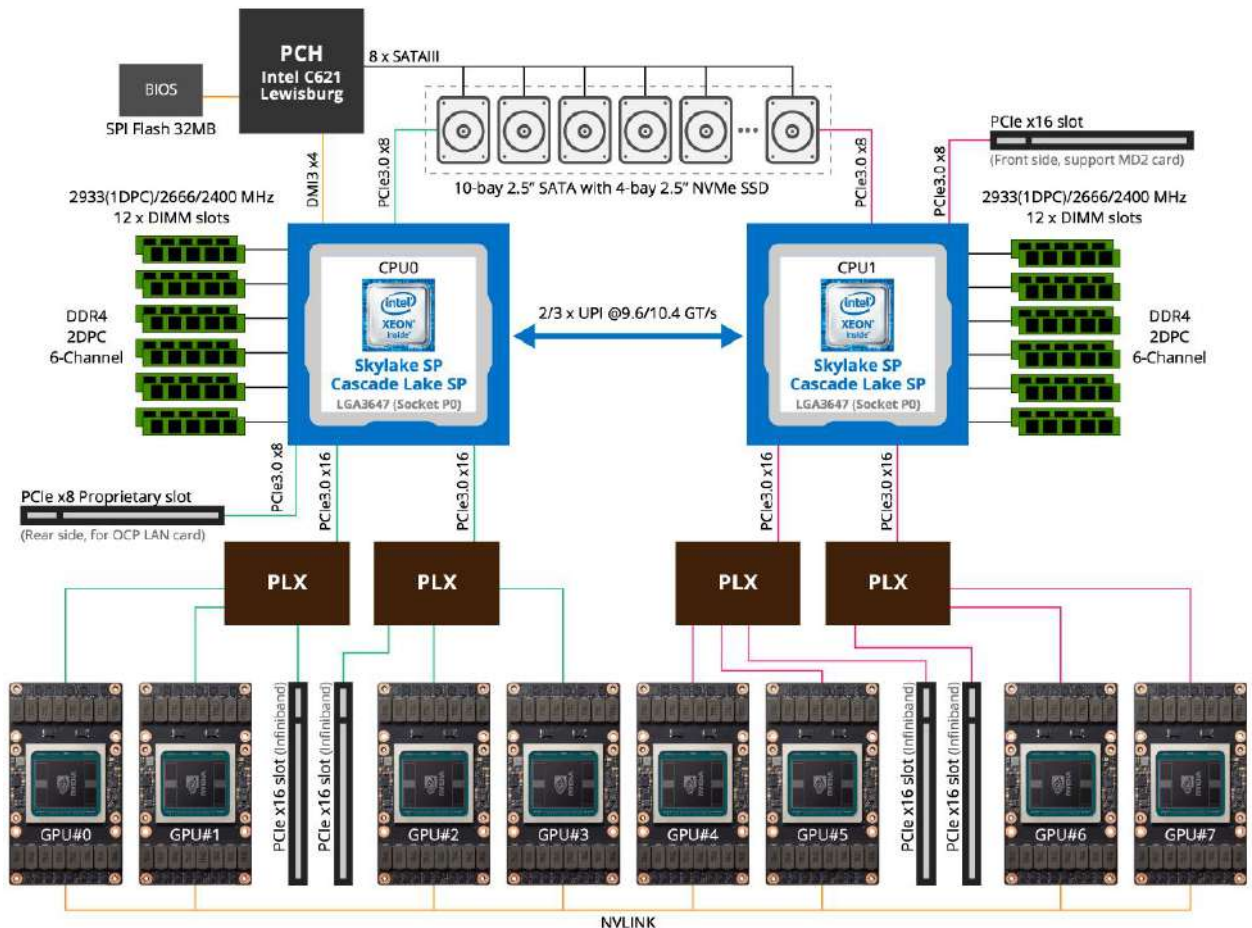


Рис.445 – Гибридный CPU/GPU сервер

Некоторые характеристики сервера:

- Процессоры: Intel® Xeon® Scalable 1 или 2 поколения
- Максимальный объем памяти: 3TB (24 DIMM x 128GB)
- Накопители: 10 x 2.5" HS HDD/SSD
- Сетевые интерфейсы: 2 x 1GbE
- Электропитание: Четыре блока питания по 2200Вт
- Форм-фактор: 4U
- До 8 модулей NVIDIA Tesla® V100 SXM2 (GPU)
- Тип сервера: GPU сервер



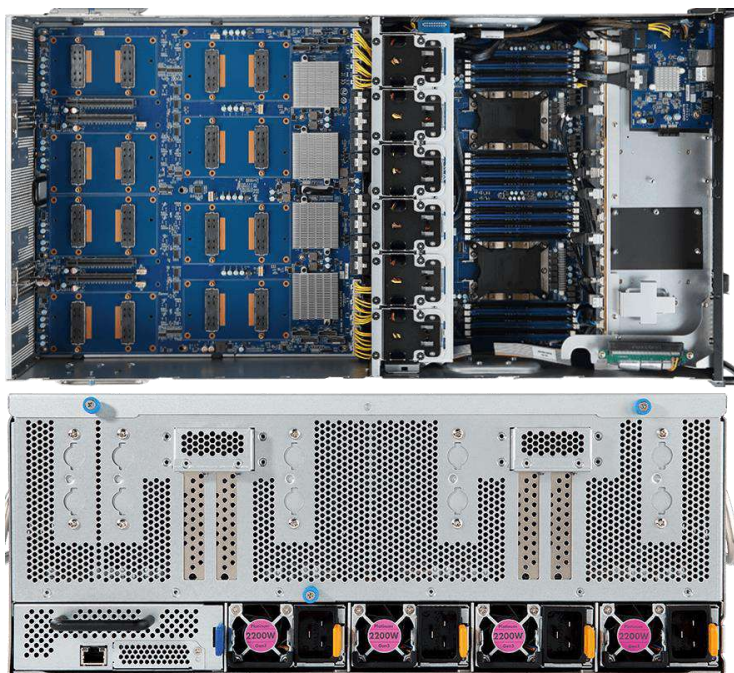


Рис.446 – GPU сервер – вид сверху (верхняя часть снимка), вид сзади (нижняя часть)

Обратите внимание на вид сзади (нижняя часть фотографии) – сплошная (сетчатая) задняя стенка, на которой вообще не предусмотрено место под разъёмы видео – это действительно счётный узел, а не графическая станция, несмотря на букву ‘G’ в названии системы.

### ***Ты помнишь, как всё начиналось – ILLIAC IV, Cray, CDC Cyber***

В этом разделе будут приведено описание нескольких знаковых компьютерных проектов, с которых, собственно и начиналось направление, известное как «суперкомпьютеры». Выше уже предлагалось называть «любой компьютер, который создал Сеймур Крэй», суперкомпьютером, но, хотя задолго до Cray-1 этот гениальный инженер, работая на тот момент в компании CDC, создал самые быстродействующие в мире «числодробилки», вполне достойные иметь приставку супер-, к примеру, такие как CDC 6600 и CDC 7600, термин «суперкомпьютер» стал общепринятым, пожалуй всё-таки начиная именно с Cray-1. Синоним «Крэй == суперкомпьютер» немного напоминает историю другого русскоязычного термина – «ксерокс», которым у нас часто обозначают светокопировальные устройства, произведённые любой фирмой, не обязательно компанией Xerox.

### **ILLIAC IV**

ILLIAC IV был первым в мире компьютером с массовым параллелизмом. Первоначально система была разработана в варианте с 256 64-битными модулями с плавающей запятой (FPU) и четырьмя центральными процессорами (CPU), способными обрабатывать в пике до 1 миллиарда операций в секунду. Из-за бюджетных ограничений в итоге был построен только один «квадрант» с 64 FPU и одним CPU. Поскольку все FPU должны были выполнять одну и ту же инструкцию в один и тот же момент времени – ADD, SUB и т. д., то ILLIAC IV по классификации Флинна относился к категории SIMD.

Идея создания компьютера с использованием массива процессоров пришла к Дэниелу Слотнику, когда он работал программистом на машине IAS в 1952 году. Формальное проектирование началось только в 1960 году, когда Слотник работал в Westinghouse Electric и сумел организовать финансирование разработки в рамках контракта US Air Force. Когда это финансирование закончилось в 1964 году, Слотник перешел в Иллинойский университет и присоединился к команде Illinois Automatic Computer (ILLIAC). При финансовой поддержке Advanced Research Projects Agency (ARPA) они начали разработку новой концепции

с 256 64-битными процессорами вместо первоначальной концепции с 1024 1-битными процессорами.

Пока машина строилась компанией Burroughs, университет начал строительство нового здания для её размещения. Когда в 1972 году был завершён первый 64-процессорный квадрант машины, он был отправлен в Исследовательский центр NASA Ames в Калифорнии. После трёх лет тщательной модификации для исправления различных недостатков ILLIAC IV был подключен к ARPANet для распределенного использования в ноябре 1975 года, став первым доступным в сети суперкомпьютером, опередившим Cray-1 почти на 12 месяцев.

Работая на половине своей проектной скорости, одноквадрантный ILLIAC IV выдавал пиковые значения 50 MFLOP, что делало его самым быстрым компьютером в мире на тот момент времени. Также он был первым большим компьютером, использующим полупроводниковую память, а также самым сложным компьютером, построенным в те годы, с более чем миллионом логических элементов. Принято считать ILLIAC IV провалом, из-за огромного перерасхода бюджета, так и не сумевшего достичь заявленной производительности. Но проект сыграл важную роль в разработке новых методов и систем для программирования параллельных систем, послужив концептом для нескольких машин, построенных в 1980-е годы.

## Описание

### *Физические характеристики*

Каждый квадрант машины имел 10 футов (~3 м) в высоту, 8 футов (~2,4 м) в глубину и 50 футов (~15 м) в длину. Рядом с квадрантом располагалась его система ввода-вывода (I/O), дисковая система которой хранила до 2,5 GiB, и могла читать и записывать данные со скоростью 1 гигабит в секунду, а также компьютер B6700, который подключался к машине через тот же 1024-битовый интерфейс, что и дисковая подсистема.

Машина состояла из ряда несущих шасси, на которых размещалось несколько небольших модулей. Большинство из них были блоками обработки (PU), которые содержали модули для одного PE (процессорный элемент), его PEM и Memory Logic Unit, который обрабатывал преобразование адресов и ввод/вывод. PU были идентичны, поэтому их можно было заменить или переупорядочить по мере необходимости.



Рис.447 – ILLIAC IV

### *Детали процессора*

В каждом CU (control unit) было от 30 до 40 000 вентиляей. У CU было шестнадцать 64-битных регистров и отдельный 64-битный «блокнот» (scratchpad) на шестьдесят четыре



слота (LDB). Присутствовало четыре аккумулятора, от AC0 до AC3, счётчик программ ILR и различные регистры управления. Система имела короткий конвейер инструкций, и также в ней был реализован опережающий просмотр инструкций (lookahead).

PE содержал около 12 000 вентиляей. Он включал в себя четыре 64-битных регистра: аккумулятор A, буфер операнда B и вторичный «блокнот» S. Четвёртый регистр R использовался для широковещательных рассылок или приёма данных от других PE. Процессорные элементы использовали сумматор с упреждением (carry-lookahead adder<sup>1</sup>), детектор старшей единицы (leading-one detector<sup>2</sup>) для логических операций и «цилиндрический сдвигатель» (barrel shifter<sup>3</sup>). 64-битное сложение занимало около 200 нс, а умножение – около 400 нс. PE были подключены к локальному банку памяти, PEM, который содержал 2048 64-битных слов. Время доступа было порядка 250 нс. PE использовали архитектуру load/store.

Набор инструкций (ISA) содержал два отдельных набора инструкций, один для CU (или модуля внутри него, ADVAST), другой для PE. Инструкции для PE не декодировались, а вместо этого отправлялись непосредственно в регистр FINST для отправки в PE для обработки. Инструкции ADVAST декодировались и поступали в конвейер обработки CU.

### Логическая структура

Каждый квадрант содержал 64 PE и один CU (Рис.448). CU имел доступ ко всей шине ввода-вывода и мог адресовать всю память машины. PE могли получить доступ только к своему собственному локальному хранилищу, PEM, состоящему из 2048 64-битных слов. И PE, и CU могли использовать операции загрузки/сохранения для доступа к дисковой системе.

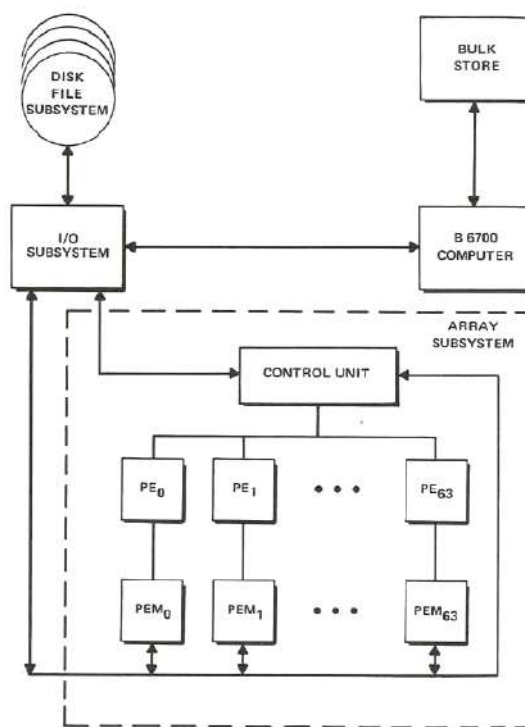


Рис.448 – Функциональная диаграмма одного квадранта ILLIAC IV

<sup>1</sup> Сумматор с упреждением (CLA) или быстрый сумматор – это тип электронного сумматора, используемый в цифровой логике. Сумматор с опережающим переносом увеличивает скорость за счет сокращения времени, необходимого для определения битов переноса.

<sup>2</sup> **Leading-one detector** – электронная схема, чаще всего встречающаяся в центральных процессорах, обычно в их арифметико-логических устройствах (ALU). Используется для определения того, является ли ведущий бит в компьютерном слове 1 или 0, что используется для выполнения базовых бинарных проверок типа IF A > 0.

<sup>3</sup> **Barrel shifter** – цифровая схема, которая может сдвигать слово данных на определенное количество битов без использования какой-либо последовательной логики, используя только комбинационную логику.

Шкафы, в которых размещались электронные схемы ILLIAC IV, были настолько большими (протяжёнными), что для прохождения сигналов от одного конца до другого требовалось 240 нс. По этой причине СУ не мог использоваться для общей координации действий. Вместо этого вся система была синхронизирована с централизованными часами, и все операции в РЕ гарантированно занимали одинаковое количество времени, независимо от обрабатываемых операндов. Таким образом, СУ мог быть уверен, что все операции завершены, не дожидаясь их результатов или кодов состояния.

Чтобы улучшить производительность операций, которые требовали, чтобы выходные данные одного РЕ использовались в качестве входа для другого РЕ, РЕ были подключены напрямую к своим соседям, а также к тем, которые находятся на расстоянии восьми шагов от него – например, РЕ1 был подключён напрямую к РЕ0 и РЕ2, а также к РЕ9 и РЕ57 (Рис.449). Соединения, расположенные в восьми шагах, обеспечивали более быструю пересылку данных, когда их требовалось переместить между более удалёнными РЕ. Каждый перенос данных перемещал 64 слова за один 125 нс такт.

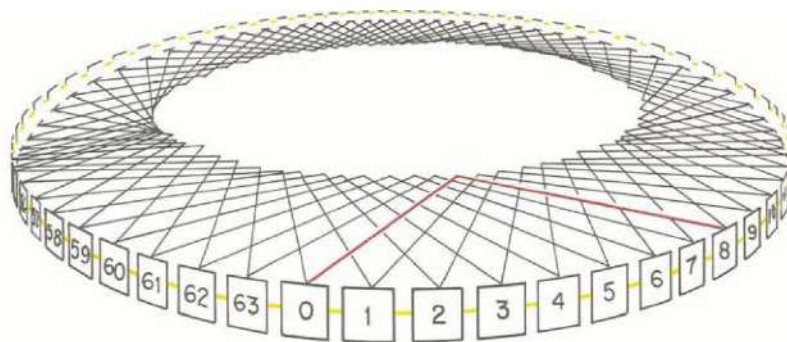


Рис.449 – «Маршрутизация» между РЕ

Система использовала одноадресный формат, в котором инструкции включали адрес одного из операндов, а другой операнд находился в аккумуляторе РЕ (регистре А). Адрес отправлялся в РЕ по отдельной «широковещательной» шине. В зависимости от инструкции, значение на шине может относиться к ячейке памяти в РЕМ РЕ, значению в одном из регистров РЕ или к числовой константе.

Поскольку каждый РЕ имел свою собственную память, в то время как формат инструкций и блоки СУ видели всё адресное пространство, система включала индексный регистр (X) для смещения базового адреса. Это позволяло, например, одному и тому же потоку команд работать с данными без выравнивания их в памяти в разных РЕ. Типичным примером может служить массив данных, загруженных в разные места в РЕМ, которые затем можно сделать единообразными, установив индекс в РЕ.

## Узкие места

Сильно перерасходовав бюджет (стоимость проекта за время его разработки выросла в несколько раз), так и не сумев достичь заявленных характеристик по производительности, ILLIAC IV выявил ещё одну серьёзную проблему, относящуюся к программной части системы. Для того, чтобы получить существенный прирост в производительности программы, работающей на параллельном компьютере, программы для массово-параллельной архитектуры необходимо было исходно разрабатывать в совершенно ином стиле, значительно отличающемся от того, который был принят для классических последовательных архитектур, а именно – используя парадигму параллельного программирования. «Умный» транслятор может распознать в программе обычный цикл, в котором, к примеру, производится поэлементное сложение двух массивов, и сумеет превратить этот цикл в одну элементарную векторную операцию сложения векторов. Но, вспоминая закон Амдала, определяющего пределы достижимой производительности для параллельных архитектур, с большим массивом

процессорных элементов, в которых общая производительность системы существенно всего зависит от самого медленного её звена – тракта последовательной части обработки, можно коротко резюмировать – для получения значительного выигрыша в производительности от распараллеливания вычислений, необходимо сразу, исходно, создавать программу в «параллельном» стиле. Просто переписать её, не затрагивая основных алгоритмов и структур, скорее всего не получится, это должна быть совершенно *другая* программа.

ILLIAC IV был первым в мире массово-параллельным компьютером, понятно, что опыта разработки программ для таких архитектур тогда ни у кого ещё не было, его приходилось нарабатывать «на ходу», по мере освоения параллельных систем. В связи с этим можно озвучить два основных вывода:

1. Машины с массовым параллелизмом имеют довольно узкую специализацию, класс задач, на которых достигаются рекорды производительности – это задачи с массовыми параллельными вычислениями. В однопоточном алгоритме проку от  $N$  процессоров будет мало, т.к. одновременно работать будет только один процессорный элемент, а остальные  $N-1$  будут простаивать. А производительность одного ПЭ в таких архитектурах обычно далеко не самая рекордная.
2. Программы для параллельных компьютеров приходится переписывать заново, адаптировать исходные последовательные алгоритмы под параллельные архитектуры, мягко говоря, нерационально. В отличие от векторных суперкомпьютеров (например, Cray), которые успешно справляются с задачами, представленными на стандартных языках последовательного типа, для эффективного программирования SIMD/MIMD-систем потребовалось введение в обиход совершенно новых парадигм (языков) параллельного программирования.

Ещё одним проблемным местом архитектуры ILLIAC IV было «бутылочное горлышко» при одновременном доступе к общей памяти нескольких CU. До 1980-го, года, когда была зафиксирована «вилка» – всё нарастающее отставание производительности памяти от производительности процессоров – было ещё довольно много времени, но с массово-параллельным ILLIAC IV эта «вилка», по сути, уже возникла, просто за счёт большого количества конкурирующих за одновременный доступ к памяти множества потребителей. Без необходимых изменений на архитектурном уровне, в первом приближения, могла возникнуть просто провальная ситуация – 64 процессорных элемента (последовательно, по очереди) получают свои операнды, пускай по паре за условный такт. Потом за один такт одновременно каждый из выполняет одну и ту же операцию (SIMD), всего – 64 операции, и дальше следует запись 64 результатов выполнения команды, на которые снова тратится 64 такта. В итоге, имеем невероятно низкий к.п.д. системы в целом: 64 арифметических операции выполнены за один такт, а «паразитные» сопутствующие загрузка/сохранение – 2 x 64 такта.

## CDC Cyber

Бывший помощник Сеймура Крэя Джим Торнтон (Jim Thornton), который помогал в своё время Крэю в создании знаменитых CDC 6600 и CDC 7600, не стал работать совместно с Крэем над следующим его проектом (в итоге, провалившимся) CDC 8600, а занялся собственным суперкомпьютерным проектом STAR-100 (STrings of binary digits that made up ARrays). Крэй, после неудачи с 8600, ушёл из CDC и в 1972 году образовал новую компанию Cray Research, первым компьютером которой стал феноменальный векторный Cray-1.

В 1974 году CDC представила архитектуру STAR. STAR – это совершенно новый 64-разрядный дизайн процессора с виртуальной памятью и инструкциями векторной обработки, добавленными для обеспечения высокой производительности при выполнении определенного класса математических задач. Векторный конвейер STAR – это конвейер из па-

мяти в память, который поддерживает длину вектора до  $2^{16} = 65536$  элементов. К сожалению, задержки (стартовое время) векторного конвейера были очень велики, поэтому пиковая скорость достигалась только при использовании очень длинных векторов. Скалярный процессор был намеренно упрощён, чтобы освободить место для векторного процессора, и был относительно медленным по сравнению с CDC 7600. Таким образом, оригинальный STAR оказался большим разочарованием, когда был выпущен (наблюдаемое в «полный рост» следствие закона Амдала). По наиболее точным оценкам, было поставлено всего три системы STAR-100.

Но, оказалось, что почти все проблемы в STAR вполне решаемы. В конце 1970-х CDC решила некоторые из этих проблем в системе Cyber 203 (STAR-100A). Новое имя было выбрано, возможно, чтобы дистанцироваться от фактического провала STAR. В Cyber 203 была сильно переработана скалярная обработка, использована конструкция слабосвязанного ввода-вывода (loosely coupled I/O), но фактически был сохранён векторный конвейер STAR. По оценкам, было поставлено (или обновлены со STAR) всего два экземпляра Cyber 203.

В 1980 году было объявлено о преемнике Cyber 203, Cyber 205 (STAR-100C). Метеорологическое бюро Великобритании в Брэкнелле, Англия, было первым его заказчиком, и они получили свой Cyber 205 в 1981 году. В Cyber 205 векторный конвейер STAR был заменён модернизированными векторными конвейерами: как в скалярных, так и в векторных модулях используются вентильные ЭСЛ-матрицы, охлаждаемые фреоном, были значительно сокращены времена доступа к памяти – 80 нс, период тактовой частоты был уменьшен в два раза по сравнению с STAR-100 – с 40 нс до 20 нс. Системы Cyber 205 выпускались с двумя или четырьмя векторными конвейерами, причём версия с четырьмя конвейерами теоретически обеспечивала до 400 64-битных MFLOP и до 800 32-битных MFLOP. На практике такие скорости были недостижимы, достичь их можно было только, разве что, при ручном программировании на ассемблере. ЭСЛ-матрицы содержали по 168 логических вентилей каждая, причем задержки по пути распространения тактовой частоты до каждого элемента настраивались вручную, путём ручной регулировки длины коаксиального кабеля. Набор инструкций можно было отнести к категории V-CISC (очень сложный набор инструкций – very complex instruction set). Специализированные наборы инструкций облегчали аппаратно-поддержанный поиск, матричную математику, были специальные инструкции, ускоряющие дешифрование.

Оригинальный Cyber 205 был переименован в Cyber 205 Series 400 в 1983 году, когда была представлена Cyber 205 Series 600. Series 600 отличалась технологией памяти и корпусом, но в остальном осталась неизменной. Был установлен один четырёхконвейерный Cyber 205, все остальные инсталляции были двухконвейерными, точное количество поставленных систем неизвестно.

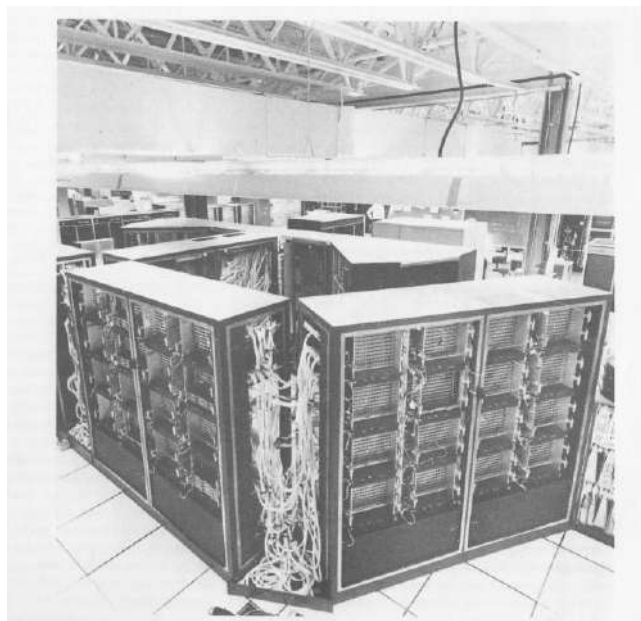


Рис.450 – Cyber 205

Архитектура Cyber 205 превратилась в ETA10, когда команда разработчиков выделилась в ETA Systems в сентябре 1983 года. Последней разработкой стала Cyber 250, выпуск которой намечался на 1987 год по цене 20 миллионов долларов; Позже он был переименован в ETA30 после того, как ETA Systems была снова поглощена CDC.

## CRAY

Приступая в 1972 году к созданию своего нового компьютера, Крэй учёл ошибки своего предыдущего неудачного проекта CDC 8600, а также плюсы и минусы конкурирующего проекта CDC STAR-100.

Во-первых, он отказался от транзисторов в пользу интегральных микросхем, которые давали такую плотность упаковки логических элементов при высокой надежности, которую невозможно было достичь с помощью дискретных транзисторов. Это позволило без значительной потери в производительности увеличить время на такт до 12,5 нс (80 МГц), вместо амбициозных 8 нс (125 МГц) в CDC 8600. Во-вторых, он отказался от многопроцессорной системы в пользу векторного процессора, как у проекта-конкурента CDC STAR-100.

Крэй учёл недостатки STAR-100. В реальной жизни компьютеру во время исполнения программы требуется выполнять как векторные, так и скалярные вычисления. STAR-100 показывал высокую скорость на векторных вычислениях, но был медленным в скалярных (и снова вспоминаем закон Амдала). Из-за этого мощь STAR-100 проявлялась только на специальных задачах, где требовалась обработка больших массивов данных. Для Cray-1 Сеймур Крэй построил процессор, который быстро выполнял и скалярные и векторные вычисления. Этого удалось добиться через создание так называемых «векторных регистров» — модулей памяти небольшого объема, которые располагались близко к процессору и работали очень быстро. Таким образом центральный процессор брал данные из регистров и записывал данные тоже в регистры, реализуя принцип работы с памятью «регистр-регистр», в то время как CDC STAR-100 использовал прежний способ работы с памятью — «load-store», то есть чтение и запись в память (которая была медленной) напрямую.

В CDC STAR-100 основная память была на ферромагнитных сердечниках, а в Cray-1 для памяти использовались полупроводники. Кроме того CDC STAR-100 строился, держа в голове совместимость с предыдущими моделями компании CDC 6600 и CDC 7600, а Cray-1 начинался с нового листа, и совместимости с предыдущими моделями не требовалось, что значительно облегчало задачу Крэю. В 1974 году первые тесты машины показали производительность 80 MFLOPS.

## Архитектура Cray-1

Память Cray-1 выполнена в виде 16 блоков, общей ёмкостью в 1 млн. 64 разрядных слов. Допускается одновременное обращение к этим блокам таким образом, что реальное время обращения становится равным 12,5 нсек, в то время как время обращения к одному блоку равно 50 нсек.

Основная память машины Cray-1, в отличие от других высокопроизводительных машин, не имеет иерархической структуры, она была столь быстра, что в такой иерархии отпала необходимость. Считается, что машина Cray-1 является наиболее быстродействующей из класса однопроцессорных систем.

В состав центрального процессора Cray-1 входят (Рис.451):

- главная память, объемом до 1048576 слов, разделенная на 16 независимых по обращению блоков, ёмкостью 64К слов каждый;
- регистровая память, состоящая из пяти групп быстрых регистров, предназначенных для хранения и преобразования адресов, для хранения и обработки векторных операндов;
- функциональные модули, в состав которых входят 12 параллельно работающих устройств, служащих для выполнения арифметических и логических операций над адресами, скалярными и векторными величинами;
- устройство, выполняющее функции управления параллельной работой модулей, блоков и устройств центрального процессора;
- 24 канала ввода-вывода, организованные в 6 групп с максимальной пропускной способностью 500000 слов в секунду (2 млн. байт в сек.).

Двенадцать функциональных устройств машины Cray-1, играющие роль арифметико-логических преобразователей, не имеют непосредственной связи с главной памятью. Также, как и в машинах семейства CDC-6000, они имеют доступ только к быстрым операционным регистрам, из которых выбираются операнды и в которые засылаются результаты после выполнения соответствующих действий.

Три группы операционных регистров, непосредственно связанные с арифметико-логическими устройствами, называются основными. К ним относятся восемь так называемых А-регистров, состоящих из 24 разрядов каждый. А-регистры связаны с двумя функциональными модулями, выполняющими сложение (вычитание) и умножение целых чисел. Эти операции используются главным образом для преобразования адресов, их базирования и индексирования. Они также используются для организации счетчиков циклов. В ряде случаев А-регистры можно использовать для выполнения арифметических операций над целыми числами.

В следующую группу основных операционных регистров входят восемь 64-разрядных S-регистров, непосредственно связанных с функциональными устройствами выполнения арифметических действий со скалярными величинами, представленными с фиксированной и плавающей запятой. Эти S-регистры аналогичны по назначению X-регистрам операндов в машине CDC-6600. Особый интерес представляют восемь 64-элементных *векторных* регистров, которые предназначены для хранения восьми операндов-векторов. Каждый такой операнд состоит из 64 компонент (элементов). В свою очередь каждая компонента представляет собой 64-разрядное слово, в котором хранится число с плавающей или фиксированной запятой. Компоненты вектора могут представлять собой также элементы некоторой таблицы. В системе команд машины предусмотрены специальные операции, в качестве операндов которых выступают многокомпонентные векторы. Не во всех задачах требуется обрабатывать векторы размерности 64. Специальный управляющий регистр центрального процессора задает требуемую размерность (число элементов). Этот регистр про-

граммно управляем, что позволяет в процессе вычислений изменять размерность обрабатываемых векторов. Кроме того, в центральном процессоре предусмотрен регистр маски, с помощью которого можно блокировать выполнение арифметико-логических действий над некоторыми компонентами вектора, т. е. осуществлять выборочные покомпонентные действия. Регистр маски по своему назначению аналогичен регистру маски машины ILLIAC IV.

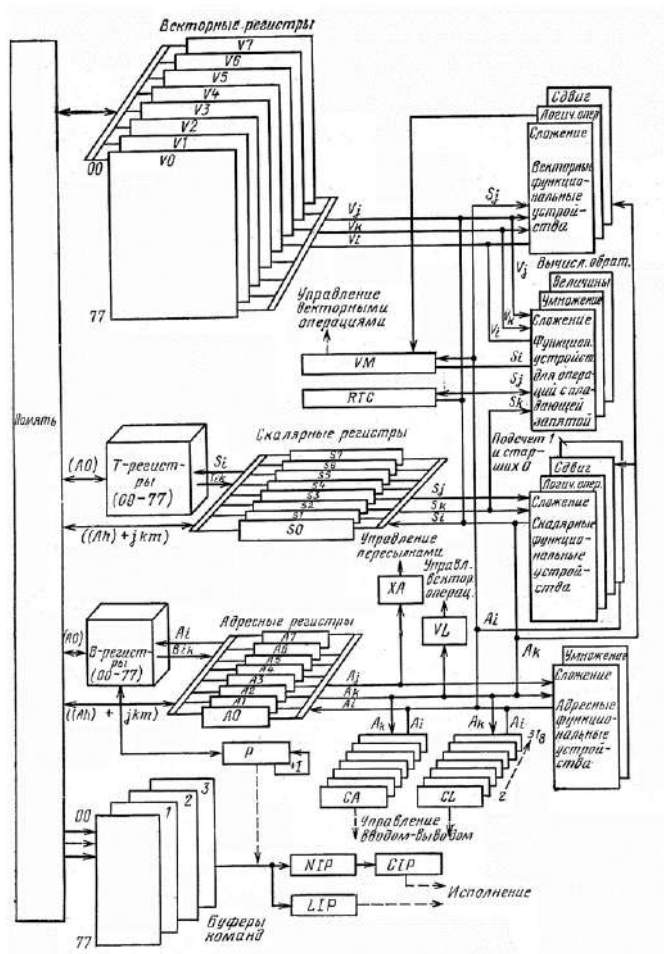


Рис.451 – Структурная схема Cray-1 (всех моделей).

В вычислительных методах линейной алгебры часто встречается процедура, состоящая в том, что строку матрицы (все элементы строки матрицы) умножают на некоторую скалярную величину и затем вычитают из элементов другой строки, с тем чтобы получить, например, нулевой коэффициент при некотором неизвестном. На этой процедуре основан метод исключения Гаусса при решении систем линейных алгебраических уравнений. Выполнение такой процедуры можно запараллелить двумя несколькими различными способами, приводящими, естественно, к одинаковому результату.

Поскольку в машине Cray-1 устройства, выполняющие операции умножения и вычитания, могут работать одновременно, то эту процедуру на ней можно запараллелить следующим образом – умножить первую компоненту первого вектора на скалярную величину, после этого приступить к выполнению операции вычитания результата из первой компоненты второго вектора, а пока происходит это вычитание, параллельно выполнить операцию умножения скаляра на вторую компоненту первого вектора. Этот механизм, позволяющий совмещать различные арифметические действия, был назван цепочкой («сцеплением»). В литературе этот принцип организации параллелизма иногда называют суперконвейерным. Сочетание конвейерного параллелизма с «покомпонентным», по мнению разработчиков машины Cray-1, составляет одну из важных особенностей и достоинств её структурной организации.



В состав регистровой памяти центрального процессора входят две группы вспомогательных буферных регистров, сокращающих число обращений к главной памяти. В первую группу входят 64 так называемых В-регистра, которые служат для накопления операндов, поступающих из А-регистров или направляемых в А-регистры из главной памяти. Во вторую группу входят 64 буферных регистра операндов, связанных с S-регистрами. Они называются Т-регистрами и служат тем же целям в отношении главной памяти, что и В-регистры. Совместно В- и Т-регистры можно рассматривать как единый буфер для хранения часто используемых операндов и их адресов.

Поскольку к регистрам В и Т можно обращаться из программы, то их можно рассматривать как промежуточную, доступную для программы память, хранящую 24-разрядные операнды (В-регистры) и 64-разрядные операнды (Т-регистры). Пересылками между памятью к В- и Т-регистрами управляют программы пользователя. Это означает, что В- и Т-регистры можно рассматривать как программно управляемые буферы. В этом состоит отличие от механизма управления буферной памятью, принятого, например, в машинах IBM-370, который реализован аппаратным путем.

Кроме того, в состав центрального процессора машины Cray-1 входит регистровая буферная память значительного объема для промежуточного хранения команд программы, исполняемой в данный момент. Эта буферная память состоит из четырех секций, каждая по 16 слов. Последовательность команд программы предварительно поступает в этот буфер. Если она содержит условный переход, то в буфере накапливаются также команды, относящиеся к последовательности, на которую возможен этот условный переход. Буфер команд является средством ускорения работы устройства управления, так как заметно минимизирует время ожидания доставки команд из главной памяти.

Система команд машины Cray-1 прямо отражает регистровую структуру центрального процессора, своеобразие связи функциональных модулей с операционными регистрами и связи их с главной памятью. Команды машины Cray-1 имеют два формата: короткие команды – 16 разрядов и длинные – 32 разряда.

Семь первых разрядов определяют код операции, затем следуют трёхразрядные поля  $i$ ,  $j$ ,  $k$ , определяющие соответственно номер регистра результата и номера регистров исходных операндов.

В одном слове машины Cray-1 может размещаться до четырёх команд короткого формата. Длинные команды могут начинаться в одном слове и продолжаться в следующем. Это позволяет плотно упаковывать команды в памяти машины и в какой-то степени ускоряет их выборку.

К главным управляющим регистрам машины относятся следующие: счётчик команд, способный адресовать каждую четвертую часть слова, регистр базы (BA), который служит для образования абсолютного адреса при обращении к памяти и одновременно является верхней границей адресного пространства программы; регистр границы адресов (LA), содержащий нижнюю границу доступных для текущей программы адресов, регистры BA и LA служат для защиты памяти, выделенной для программы, активной в мультипрограммном режиме. Тем самым в машине Cray-1 применен метод защиты по границам, используемый во многих машинах, в частности в машине Минск-32.

В составе центрального процессора имеется девятиразрядный регистр F, фиксирующий причину прерываний, а именно: прерывания по нормальному окончанию задачи, окончанию по ошибке, прерывания по вводу-выводу, по появлению ошибок в операндах, по переполнениям, меткам времени и сигналам с пульта оператора машины. Специальный регистр режимов исполнения программ позволяет блокировать некоторые группы прерываний.

Система прерываний в машине Cray-1 построена по образу и подобию таковой, принятой для машин-предшественников семейства CDC, а именно, такая же, как у CDC-7600.

Схемами управления автоматически запоминаются все управляющие и основные операционные регистры прерванной программы при появлении прерывания в область главной памяти, адрес начала которой указан в специальном управляющем регистре. После этого управление автоматически передается программам операционной системы, которые анализируют причину прерывания и при необходимости запоминают в главной памяти содержимое всех групп промежуточных и буферных регистров (B, T и V). В главной памяти для каждой задачи, идущей в мультипрограммном режиме, выделены массивы, в которых сохраняется вся информация, необходимая для продолжения прерванной программы.

Отметим, что защита памяти и система прерываний организованы так, что значительная доля работы по запоминанию информации для возврата, по организации процесса переключения с одной задачи на другую возлагается на операционную систему. Отсутствие достаточно развитых аппаратных средств динамического перераспределения памяти, упрощенный механизм преобразования программных адресов в физические (аппаратное базирование) вполне допустимы в машинах, предназначенных для решения крупных научных задач. Этот режим «научных» вычислений характерен сравнительно редким появлением ситуации, требующих переключения с решения одной задачи на другую. Это в свою очередь позволяет по-иному реализовать защиту и прерывания, не заботясь об их быстром аппаратном выполнении.

Здесь хотелось бы еще раз подчеркнуть то влияние на структурную организацию ЭВМ, которое оказывает класс решаемых задач. Интересен способ представления чисел с плавающей запятой, принятый в машине Cray-1. На мантиссу со знаком выделено 49 разрядов. На двоичный порядок выделено 15 разрядов. Тем самым цена младшего разряда  $2^{-48}$  ( $\sim 10^{-17}$ ), а диапазон представимых чисел приблизительно оценивается величиной  $10^{2500}$ . Может показаться на первый взгляд странным увеличение диапазона представимых чисел до таких огромных пределов. Однако для решения некоторых задач линейной алгебры такое расширение диапазона может оказаться необходимым.

## Программное обеспечение

Математическое обеспечение машины Cray-1, также, как и у её предшественников машин фирмы CDC 6600/7600, было «фортранно-ориентировано». Это означает, что в качестве основного входного языка был выбран Фортран, наиболее интенсивно используемый в научных расчётах. Для машины Cray-1 создан специальный оптимизирующий транслятор со стандартного Фортрана, учитывающий специфику этой векторной машины. При некоторых условиях, наложенных на программу, данный транслятор обеспечивает конвейерный параллелизм и готовит объектные программы, эффективно использующие эти возможности машины.

В состав математического обеспечения входил также макроассемблер CAL (Cray Assembler Language) и библиотека стандартных программ, составленная на этом языке и ориентированная на использование средств параллельных вычислений, присущих данной машине.

Операционная система COS (Cray Operating System) предназначена для режима *пакетной обработки* и дистанционной пакетной обработки заданий, полученных с удалённых терминалов. Операционная система рассчитана на мультипрограммную обработку одновременно до 63 активных задач. Для обеспечения работы в режиме дистанционного доступа в качестве машины-сателлита вычислительной системы Cray-1 использовался миникомпьютер “Eclipse”. С этой машиной были связаны внешние каналы центрального вычислителя с помощью специальных сопрягающих устройств. На миникомпьютер возлагались функции управления приёмом-передачей информации, управления линиями связи, т. е. функции процессора передачи данных. Фирма Cray разработала для этих целей собственный специализированный микрокомпьютер, который в последующем заменил “Eclipse”. Производительность Cray-1 сильно зависела от характера решаемых задач. Экспериментальная проверка показала, что она колебалась от 20 до 160 млн. операций в секунду. При выполнении

операций с плавающей запятой диапазон её быстродействия оценивался в пределах от 20 до 60 млн. операций в секунду.

### Анатомия первого суперкомпьютера

Одной из инженерных задумок Крея на стадии проектирования являлось использование в Cray-1 всего трёх типов интегральных схем: элементов логики, чипов оперативной памяти и регистров, что позволяло значительно повысить надежность и отказоустойчивость системы. Впрочем, общее количество микросхем в системе в итоге достигло 350 тысяч штук, и, чтобы упаковать такое количество в сравнительно небольшой корпус, Крею и его инженерам пришлось изрядно потрудиться.

В итоге они решили объединить индивидуальные двухсторонние печатные платы в модули с размерами 15х23 см. Центральный процессор Cray-1 состоял из 500 печатных плат, на каждой из которых с обеих сторон располагалось по 144 микросхемы. Всего получилось 144 000 микросхем, которые охлаждались фреоном.

Каждый модуль содержал пять слоев: на двух внешних с обеих сторон располагались микросхемы, общее количество которых доходило до 288 штук, тогда как три внутренние слоя платы предназначались исключительно для обеспечения электрических соединений между ними. Еще на этапе проектирования суперкомпьютера создатели подразумевали, что модуль будет «одноразовым»: при выходе из строя хотя бы одной микросхемы его заменяли новым работоспособным, ремонт вышедшего из строя компонента системы не предусматривался. Всего в Cray-1 было использовано 113 различных комбинаций модулей общей численностью 1662 штук.

Крэй прекрасно понимал, что одним из главных лимитирующих факторов в компьютерных системах является скорость, с которой электрический сигнал перемещается по соединительным проводам: чем они длиннее, тем больше время задержки. Решение проблемы конструктор видел в придании новому компьютеру многогранной формы (близкой к 270-градусному цилиндру), благодаря чему сокращалась длина проводов, а также увеличивалась плотность расположения компонентов системы. В довершение к этому центральный процессор Cray-1 был со всех сторон окружен чипами оперативной памяти, в результате чего время доступа к каждому из них было одинаковым.

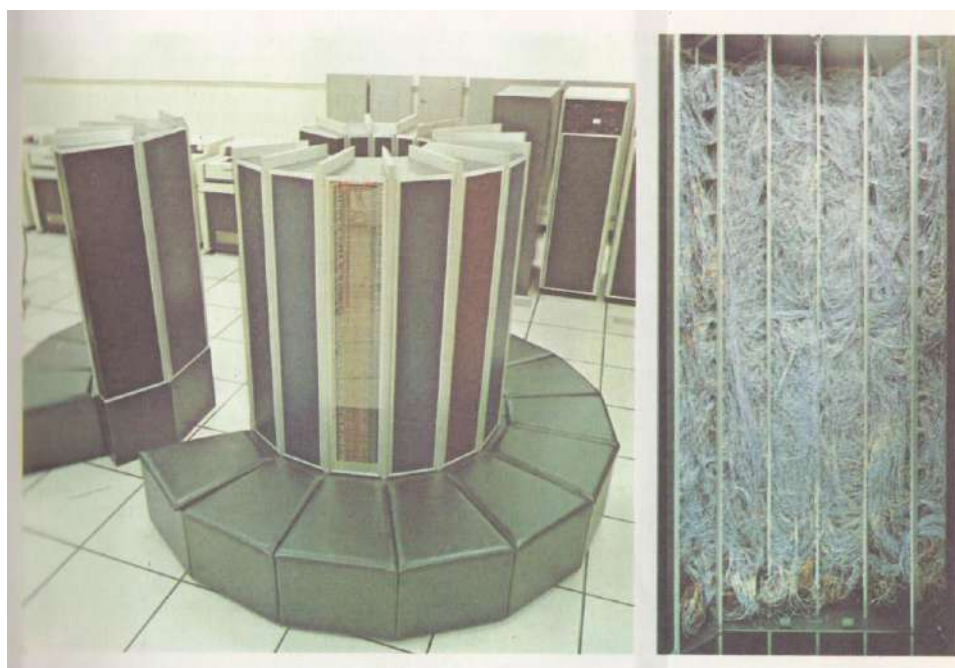


Рис.452 – Самая дорогая в мире «садовая скамейка» Cray-1: слева – внешний вид, справа – задняя панель. Обитая кожей нижняя часть перевернутого «гриба» - блоки питания и система охлаждения.

Для соединения электронных модулей были применены провода трех типов – 30, 60 и 120 см длины, время прохождения электрического сигнала в которых составляло, соответственно, одну, две и четыре наносекунды. Трудности пайки проводов в собиравшейся системе заключались в том, что провода практически не маркировались: присутствовало всего два цвета – синий и серый. По словам инженера Cray Research Ли Хибги (Lee Higbie), процедуру пайки немного облегчало то, что кабели разной длины припаивались по очереди: «Вначале мы разбирались с проводами длиной 30 см, затем – 60 и 120 см. Последних в системе всего пара штук».

Большинство сотрудников, занимавшихся сборкой первого суперкомпьютера Cray Research, составляли женщины, которых Крей нанимал при наличии у них опыта работы ткачихами. Их дисциплина и стремление к совершенству оказались незаменимыми при точном припаивании сотен километров проводов.

Из-за сверхплотной интеграции компонентов внутри суперкомпьютера Cray-1 выделял огромное количество тепла, поэтому ему необходима была серьезная система охлаждения. Инженеры Cray Research разработали соответствующую установку, использовавшую в качестве охладителя жидкий фреон. Между модулями с электронными компонентами были установлены медные пластины с тефлоновым покрытием, собиравшие тепло и отводившие его к краям стойки. Они крепились к основной «охлаждающей магистрали», состоявшей из идущих вдоль каждой из 12 панелей алюминиевых «кожухов», внутри которых располагались стальные трубки. По ним протекал жидкий фреон, охлаждавшийся в нижней части системы, где и находилась холодильная установка. Температура «магистрали» составляла 12 градусов Цельсия, тогда как в центральной части модулей с электронными компонентами она доходила до 54 градусов. В целом температура суперкомпьютера была ниже комнатной, что позволяло установить его даже в помещение, не оборудованное системой воздушного кондиционирования.

Трудности с системой охлаждения заставили сдвинуть срок изготовления первого Cray-1 на полгода: смазочное вещество, которое обычно смешивается с фреоном для обеспечения нормальной работы компрессора, протекало сквозь микроскопические щели в сварочных швах и покрывало платы слоем масла, выводя их из строя. Лишь благодаря использованию новых технологий в сварке металлоконструкций удалось должным образом загерметизировать стыковочные места трубок системы охлаждения.

Корпус находившихся в нижней части суперкомпьютера блоков питания и холодильной установки с внешней стороны был покрыт слоем мягкого наполнителя, напоминавшего своеобразные сиденья, на которых техники и программисты любили сидеть и спать по ночам. Поэтому Cray-1 в былые времена шуточно прозвали «самой дорогой в мире садовой скамейкой».

Вся электроника суперкомпьютера была упакована в многогранник с шестнадцатью гранями (четыре из которых отсутствовали для доступа к внутренностям системы) высотой 195 см и диаметром 150 см. Как и для всех последующих суперкомпьютеров Сеймура Крэя, покупатель был волен самостоятельно выбрать цвет окраски граней своего Cray-1. Вес всей системы составлял 5,25 тонны, энергопотребление – 115 кВт. Суммарная же потребляемая мощность Cray-1 вместе с системой хранения данных и охлаждающей установкой составляла 250 кВт! Неудивительно, что за одну только подачу питания на суперкомпьютер отвечал отдельный агрегат под названием «Устройство распределения энергии» (Power Distribution Unit) высотой в два метра, работавший от собственного дизельного генератора.

Архитектурное превосходство Cray-1 сделало из него непревзойденного чемпиона по быстродействию. Средняя производительность суперкомпьютера составляла 80 MFLOPS. При выполнении некоторых операций скорость работы Cray-1 повышалась до 150 MFLOPS. Пиковая же производительность составляла 250 MFLOPS. Для сравнения: популярные в то время мейнфреймы Cyber 176 и IBM 360/195 были способны лишь на 4...7 MFLOPS.

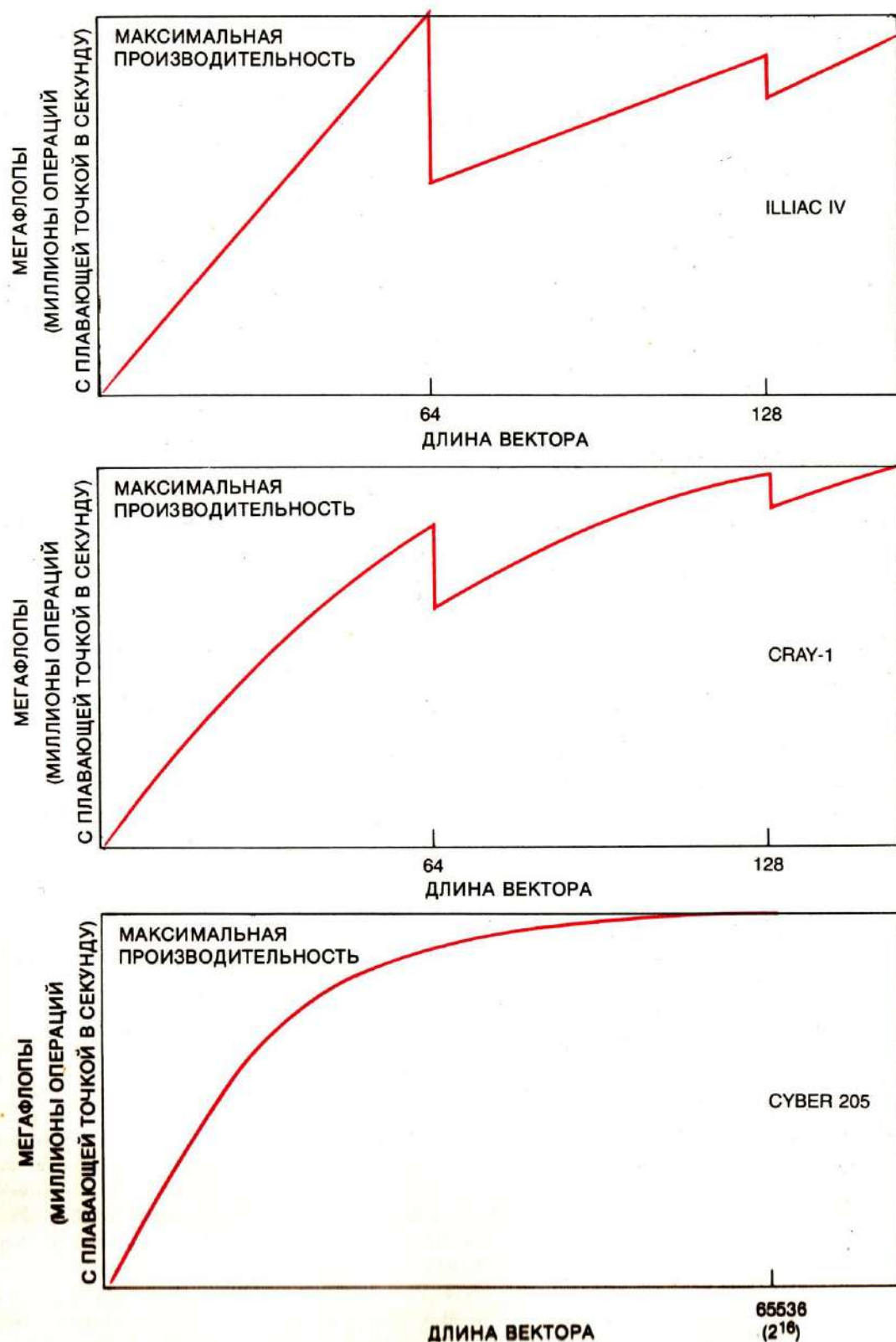


Рис.453 – Зависимость пиковой производительности некоторых суперкомпьютеров от длины вектора

В середине 1970-х CRAY-1 стал безусловным лидером по производительности на рынке суперкомпьютеров. На Рис.453 приведены графики сравнения пиковой производительности на операциях с плавающей точкой для трёх разных суперкомпьютеров того времени: ILLIAC IV, CRAY-1 и Cyber 205 в зависимости от длины обрабатываемых векторов. Совершенно понятно, почему ILLIAC IV и CRAY-1 «любят» вектора, длина которых кратна

шестидесяти четырём – у CRAY были 64-элементные векторные регистры, у ILLIAC – 64 процессорных элемента. А вот Cyber, имея довольно слабый скалярный процессор, большую задержку старта векторного конвейера, данные в который поступали из медленной памяти, а не из быстрых векторных регистров, «разгонялся» до своих предельных скоростей только на очень длинных векторах, с длинами, которые в реальной жизни практически не встречаются (и опять и снова закон Амдала). Зная особенность архитектуры Cyber, для него были написаны синтетические тесты, в которых демонстрировались те самые заявленные 400/800 MFlops, но на реальных тестах и задачах Cyber однозначно проигрывал своему более низкочастотному конкуренту. Ещё более сокрушительным было поражение 64-процессорного ILLIAC IV, который проиграл однопроцессорному CRAY-1 по производительности на реальных задачах, превзойдя конкурента в несколько раз только по общей стоимости системы.

Многие находки в архитектуре CRAY были скопированы и воспроизведены в конкурирующих архитектурах у других производителей суперкомпьютеров. В некоторых случаях такое «слепое» заимствование и подражание доходило почти до абсурда. Например, суперкомпьютер Hitachi S-810 позаимствовал у CRAY идею «сцепления» конвейеров (цепочки, суперконвейер), об этой технологии было рассказано выше – когда второй вычислительный конвейер можно было стартовать, подавая на его вход результат работы первого конвейера, не дожидаясь его полного прохождения, со сдвигом всего на один такт, в результате чего можно было получить результат выполнения двух операций – при длине конвейера  $N$  – за  $N+1$  такт, а не за  $N+N=2N$  тактов. В CRAY была возможность «сцепить» только два конвейера, и в Hitachi, скопировав суть идеи, сделали возможным устраивать «цепочки» из шести конвейеров максимум. Конечно же, зная эту особенность архитектуры, были написаны соответствующие тесты, демонстрировавшие это преимущество S-810. Шесть «сцепленных» конвейеров, конечно, лучше, чем два, только вот, как показала практика, в реальных программах такой «суперконвейер» практически никогда востребован не был. Три «сцепленных» конвейера – да, это могло пригодиться, но шесть были явным «перебором», результатом не слишком критического подхода при копировании изначальной идеи. И в итоге S-810 таки уступил CRAY в гонке реальной производительности,

## Успех на рынке

Cray-1, представленный компанией Cray Research в 1975 году, с восторгом был встречен на рынке. Восторг от машины был таким, что между Лос-Аламосской и Ливерморской лабораториями — главными потребителями сверхскоростных вычислительных машин в то время — развернулась настоящая подковёрная война. Каждая из лабораторий через своё лобби в Министерстве энергетики США набивала цену на машину и всячески мешала своему конкуренту приобрести первый экземпляр. В конце концов Лос-Аламосская национальная лаборатория выиграла и получила экземпляр с серийным номером 001 в марте 1976 года на 6 месяцев на пробу. Программисты лаборатории дали восторженные отзывы о машине, и на основании этих отзывов первым официальным заказчиком Cray-1 в 1977 году стал Национальный Центр Атмосферных исследований США, заплативший за машину с серийным номером 003 8,86 миллиона долларов (7955000 долларов за машину плюс 1 миллион за диски). Машина поступила в центр в 11 июля 1977 года и вступила в строй в декабре 1977 года, сменив стареющий и перегруженный работой CDC 7600, и проработала в Центре до 1989 года. Клиенты могли арендовать Cray-1 за сумму 210 500 долларов в месяц, 1 час работы на Cray-1 стоил 7500 долларов.

При покупке компьютера Национальный Центр Атмосферных исследований потребовал, чтобы в машину был добавлен модуль коррекции ошибок. После долгих споров Крэй согласился это сделать. Из-за этого все машины Cray-1, кроме серийного номера 001, выше на 4 дюйма и содержат на 8 модулей больше в каждой стойке. Серийный номер 001 после

работы в Лос-Аламосе использовался Cray Research по всему миру как временный компьютер, на котором мог тренироваться заказчик, дожидаясь поставки своего экземпляра. Серийный номер 001 закончил свою карьеру в Великобритании в мае 1989 года.

Экземпляр машины с серийным номером 002 остался в Cray Research, когда были обнаружены проблемы с памятью в машине, проданной в Лос-Аламос. Ливерморская национальная лаборатория приобрела 4 машины Cray-1 в 1978 году.

Конкурентами Cray-1 на рынке суперкомпьютеров выступали машины компании CDC STAR-100 и Cyber 76, TI ASC компании Texas Instruments, ILLIAC IV компании Burroughs, IBM 370/195, STARAN компании Goodyear Aerospace и Hypercube компании IMS Associates. Руководители Cray Research на основании предыдущего опыта предполагали, что им удастся продать не больше дюжины машин Cray-1. Всего же с 1975 по 1984 год была продана 61 машина Cray-1, что принесло с собой большой финансовый успех и поражение практически всех конкурентов: после выхода на рынок Cray-1 компания Texas Instruments прекратила продажи своей машины TI ASC и больше никогда не занималась суперкомпьютерами, компании CDC удалось продать только 4 машины CDC STAR-100, IBM также потеряла (на время) интерес к рынку суперкомпьютеров.

## Дальнейшая история CRAY

Натиск японских производителей суперЭВМ (Hitachi, Fujitsu, NEC) во второй половине 1970-х был весьма впечатляющим, но Сеймур Крэй наносит своевременный контрудар – в 1982 году на рынке появилась первая многопроцессорная модель семейства суперкомпьютеров CRAY X-MP<sup>1</sup>, а двумя годами позже в Ливерморской национальной физической лаборатории им. Лоуренса был установлен первый экземпляр суперЭВМ CRAY-2. Машины от Cray Research опередили конкурентов в главном – они ознакомили зарождение нового поколения ЭВМ сверхвысокой производительности, в которых векторно-конвейерный параллелизм дополнялся мультипроцессорной обработкой. Крэй применил в своих компьютерах неординарные решения проблемы увеличения производительности. Сохранив в CRAY-2 и CRAY X-MP архитектуру и структурные наработки CRAY-1, он сокрушил конкурентов сразу на двух фронтах: достиг рекордно малой длительности машинного цикла (4.1 нс) и расширил параллелизм системы за счет мультипроцессорной обработки. В итоге Cray Research сохранила за собой звание абсолютного чемпиона по производительности: CRAY-2 продемонстрировала пиковую производительность 2 GFLOPS, обогнав NEC SX-2 – самую быструю японскую суперЭВМ – в полтора раза. Для решения проблемы оптимизации машинного цикла Крей пошел дальше японцев, которые уже владели технологией ЭСЛ-БИС, позволившей в Fujitsu VP достичь длительности машинного цикла в 7.5 нс. Помимо того, что в CRAY-2 были использованы быстродействующие ЭСЛ-схемы, конструктивное решение блоков ЦП обеспечивало максимальную плотность монтажа компонентов. Для охлаждения такой уникальной системы, которая выделяла ни много, ни мало 195 кВт, была использована технология погружения модулей в карбид фтора – специальный жидкий хладагент производства американской фирмы 3M («самый дорогой в мире аквариум»).

Второе революционное решение, реализованное в суперкомпьютере Cray-2, заключалось в том, что объем оперативной памяти был доведен до 2 Гбайт. Сеймuru Крэйу удалось выполнить критерий балансировки производительности и ёмкости оперативной памяти по Флинну: «Каждому миллиону операций производительности процессора должно соответствовать не менее 1 Мбайт ёмкости оперативной памяти». Суть проблемы заключается в том, что типичные задачи гидро- и аэродинамики, ядерной физики, геологии, метеорологии и других дисциплин, решаемые с помощью суперЭВМ, требуют обработки значительного объёма данных для получения результатов приемлемой точности. Естественно, при таких объемах вычислений относительно малая ёмкость оперативной памяти вызывает

---

<sup>1</sup> Cray X-MP (Cray Y-MP) был создан на основе Cray-1 под руководством Стива Чена, ученика и помощника Сеймура Крэя.



интенсивный обмен с дисковой памятью, что в полном соответствии с законом Амдала ведет к резкому снижению производительности системы.

Всё-таки новый качественный уровень суперкомпьютера CRAY-2 определялся не столько сверхмалой длительностью машинного цикла и сверхбольшой (для того времени) ёмкостью оперативной памяти, сколько мультипроцессорной архитектурой, заимствованной у другой разработки Cray Research – семейства многопроцессорных суперЭВМ CRAY X-MP. Его три базовые модели – X-MP/1, X-MP/2 и X-MP/4 – предлагали пользователям одно-, двух- или четырёхпроцессорную конфигурацию системы с производительностью 410 MFlops на процессор. Спектр доступных вариантов расширялся за счет возможности установки памяти разного объёма (от 32 до 128 Мбайт на систему). Такой ориентированный на рынок подход к построению суперкомпьютера впоследствии принес фирме Cray Research ощутимый коммерческий эффект. Мультипроцессорная архитектура суперкомпьютеров производства CRAY была разработана с учетом достижений и недостатков многопроцессорных мэйнфреймов, в первую очередь фирмы IBM. В отличие от «классических» операционных систем IBM, которые используют для взаимодействия процессов механизм глобальных переменных и семафоров в общей памяти, мультипроцессорная архитектура CRAY предполагает обмен данными между процессорами через специальные кластерные регистры, кроме того, для обслуживания взаимодействия процессов в архитектуре CRAY были предусмотрены аппаратно-реализованные семафорные флажки, которые устанавливаются, сбрасываются и анализируются с помощью специальных команд, что также ускоряет межпроцессорный обмен и в итоге увеличивает системную производительность. В результате этих новшеств коэффициент ускорения двухпроцессорной суперЭВМ CRAY X-MP/2 по отношению к однопроцессорной CRAY X-MP/1 составлял не менее 1.86 раз.

В отличие от семейства CRAY X-MP, модели которого работают под управлением операционной системы COS (Cray Operating System), CRAY-2 комплектовалась новой операционной системой CX-COS, созданной фирмой Cray Research на базе Unix System V.

После коммерческого успеха моделей CRAY X-MP фирма Cray Research выпустила модифицированное семейство суперкомпьютеров CRAY Y-MP, обладающих большим числом процессоров (до восьми) и пониженной длительностью машинного цикла (6 нс). Старшая модель этого семейства CRAY Y-MP/832 имела пиковую производительность 2666 MFLOPS и занимала двенадцатую позицию в рейтинге Дж. Донгарра по результатам тестирования на пакете LINPACK.

Первые же пять позиций принадлежали представителям CRAY-MP C90, старшая модель которого – 16-процессорная машина CRAY Y-MP C90/16256 – имела оперативную память ёмкостью 2 Гбайт и могла демонстрировать производительность на уровне 16 GFLOPS. Все 16 процессоров и оперативная память этого компьютера размещались в одной стойке весьма скромных размеров: 2.95x2.57x2.17 м3. Подсистема ввода/вывода CRAY Y-MP C90 имела до 256 каналов с общей пропускной способностью 13.6 Гбайт/с, встроенный кремниевый диск ёмкостью 16 Гбайт и поддерживала дисковую память общей ёмкостью до 4 Тбайт. «Суперпараметры» модели CRAY Y-MP C90 эффектно дополняются развитым программным обеспечением, центральным ядром которого являлись компиляторы CF77 Fortran, Cray Standard C Compiler, Cray ADA и Pascal.

15 мая 1989 года Крэй объявил об уходе из компании Cray Research и решил продолжить работу над Cray-3 в рамках новой компании — Cray Computer Corporation, обосновавшейся в Колорадо-Спрингс. Расставание было мирным: Cray Research вложило в новую компанию Крэя 100 миллионов долларов наличными и 50 миллионов добавило имуществом. В Cray Computer Corporation Крэй работал как подрядчик, а не как штатный работник.

В конце августа 1990 года компании удалось заключить договор на продажу первого компьютера с Ливерморской национальной лабораторией. По этому договору Cray-3 должен был быть поставлен не позднее июня 1992 года с поэтапной сдачей модулей. До июня 1991 года все этапы были пройдены успешно, однако к декабрю 1991 года стало ясно, что

сдать в срок блок вычислений с плавающей запятой не удастся. В конце декабря Ливерморская лаборатория сообщила, что отказывается от контракта на Cray-3 и приобретает вместо него компьютер C90 у компании Cray Research.

В основу проекта Cray-3 была заложена идея перехода на принципиально новую элементную базу – БИС на основе *арсенида галлия*, которая теоретически позволяла обеспечить субнаносекундную длительность машинного цикла. Затея оказалась весьма рискованной, тем более, что в конце 1980-х годов в мире не существовало промышленно освоенной технологии для производства подобной элементной базы. Проект Cray-3 «затормозился» именно из-за неудовлетворительного состояния технологии разработки и производства GaAs-микросхем, а также сборок вентиляционных матриц на их основе.

С потерей своего главного клиента для компании Cray Computer Corporation наступили трудные времена. Ежемесячные расходы компании составляли 5 миллионов долларов. Список правительственных организаций, способных приобрести компьютер за 30 миллионов долларов, сократился с окончанием Холодной войны и сокращением финансирования на оборону. За неимением покупателя было решено остановить разработку 16-процессорного варианта Cray-3 и найти покупателя на 8-процессорную машину.

Несмотря на всё это, в начале 1994 года Крэй начал работу над новым компьютером — Cray-4, который был более простой и более надёжной машиной. Работа над Cray-4 шла успешнее, чем над Cray-3. Удалось достигнуть тактовой частоты 1 ГГц (время такта 1 нс), общая стоимость машины была в пять раз ниже, чем Cray-3.

В целом к середине 1994 года работа в компании наладилась, уже строились планы на машины Cray-5 и Cray-6, а Крэй подумывал о почётном уходе на пенсию. Однако компании в условиях экономического спада не удалось найти денег на оплату текущих расходов и завершение работ. 24 марта 1995 года компания объявила о своем банкротстве, истратив в общей сумме 300 миллионов долларов на разработку и поставив бесплатно на пробу в мае 1993 года компьютер Cray-3 лишь одному покупателю — Национальному центру атмосферных исследований, где он так за несколько лет и не заработал корректно и был списан через 2 дня после того, как Cray Computer Corporation обанкротилась.

До середины июля 1995 года Крэй вместе с оставшимися руководителями компании пытался найти инвестора для продолжения работ, однако выпуск модели Cray T90 компанией Cray Research поставил крест на перспективах компьютера Cray-4.

## **Seymour Roger Cray – печальный финал**

Почти через год — в июле 1996 года — Крэй организовал новую компанию под названием SRC Computers (SRC — инициалы его имени “Seymour Roger Cray”), в которой он начал разработку с нуля новой модели суперкомпьютера. Отказываясь от векторного суперкомпьютера, Крэй планировал создать многопроцессорную машину с 512 процессорами от компании Intel. Разработки только начались, когда 22 сентября 1996 Крэй попал в автомобильную катастрофу и скончался от полученных травм 2 недели спустя — 5 октября 1996 года — в возрасте 71 года. Компания SRC Computers продолжила своё существование и до сих пор занимается разработкой компьютерных систем, правда, не производит суперкомпьютеров.

## **Интересные факты**

- В 1971 году руководство компании CDC потребовало от Крэя сократить на 10% расходы своей лаборатории. В его лаборатории работало всего 40 инженеров, проект CDC 8600 требовал очень больших усилий, а сокращение расходов на 10% означало увольнение четырёх человек из команды. Чтобы не жертвовать никем, Сеймур Крэй сократил расходы за счёт себя и отказался от своей зарплаты. CDC платила ему минимальную зарплату, разрешённую по трудовому кодексу США — 1.25\$ в час.
- Выбор места для своей лаборатории в 1962 году Сеймур Крэй объяснял руководству CDC тем, что Миннеаполис, как крупный промышленный центр США, наверняка

входит в список целей при нанесении ядерного удара странами Варшавского блока, и он, дескать, старается быть подальше от эпицентра взрыва. На самом деле Чиппеуа-Фоллз был достаточно далеко от штаб-квартиры CDC, чтобы менеджеры CDC не тратились на дорогие междугородные телефонные звонки по пустякам и не слишком часто приезжали туда на автомобиле.

- При переезде в Чиппеуа-Фоллз Сеймур сам спроектировал свой дом, цокольный этаж которого был превращен в настоящее укрепленное противоядерное бомбоубежище с бассейном, в котором можно было хранить питьевую воду, и топливным баком на 10 000 галлонов, способным обеспечить дом теплом на 4 года.

## **СуперЭВМ в России**

В относительно недалёком прошлом, в 1970-80-ые годы, в России и, ранее в СССР, дела на суперкомпьютерном направлении обстояли довольно неплохо. Мы, если не были в абсолютных мировых лидерах, то по крайней мере имели в проектах и в «железе» несколько разных моделей достаточно приличных суперЭВМ, некоторые из которых имели полностью оригинальное «отечественное происхождение», а некоторые были созданы с заимствованиями идей из успешных западных разработок. Если коротко, то можно констатировать, что были времена, когда СССР/Россия на суперкомпьютерном «поле» были представлены вполне достойно.

К середине 1980-х годов в СССР существовала достаточно стройная государственная программа по суперкомпьютерам, которая, помимо финансирования довольно широкого спектра проектов, включала действия по модернизации производства элементной базы и оснащению заводов необходимым технологическим оборудованием. В случае успеха этой программы на свет должен был появиться целый ряд вполне современных суперкомпьютеров с производительностью от 100 MFLOPS до 1.2 GFLOPS. Кроме того, советская суперкомпьютерная программа предполагала значительное продвижение в области MIMD-систем. К этому времени СССР уже обладал компьютерами «Эльбрус-2», параметры которых вполне соответствовали определению «суперЭВМ». Фаворитом суперкомпьютерной программы стал коллектив разработчиков этой машины – «Институт точной механики и вычислительной техники им. Лебедева АН СССР» (ИТМиВТ), известный такими запоминающимися проектами, как «БЭСМ», «Чегет» и «Эльбрус-1». Именно в этом институте были заложены два (из четырёх) основных проекта государственной программы – «Эльбрус-3» и «Модульный конвейерный процессор» (МКП). Одно направление возглавил проект «Эльбрус-3», основанный на архитектуре свершироких команд (*Very Long Instruction Word*, VLIW). Архитектура этого типа интересна тем, что центральный процессор выбирает из памяти и запускает на исполнение сразу несколько операций, упакованных компилятором в одно командное слово. Пиковая производительность полной конфигурации системы (16 процессоров) ожидалась на уровне 10 GFLOPS, что было очень неплохо по тем временам.

Второе направление развития советских суперЭВМ предполагало достижение более скромных показателей производительности (около 1 GFLOPS на процессор) за счет применения проверенного практикой принципа векторно-конвейерной обработки, но без копирования зарубежных суперкомпьютеров. В ИТМиВТ параллельно с VLIW-системой «Эльбрус-3» стартовал проект МКП под официальным названием «Эльбрус-3Б», который предполагалось завершить созданием системы из 2...20 процессоров, обладающей суммарной пиковой производительностью 2...10 GFLOPS.

Направление разработки векторно-конвейерных суперЭВМ оригинальной структуры было выбрано и для суперкомпьютеров Единой Системы. Головной институт программы «ЕС ЭВМ» – Научно-исследовательский центр электронной вычислительной техники (НИЦЭВТ) – в 1986 году анонсировал начало проекта по созданию суперкомпьютера ЕС1191 с производительностью 1.2 GFLOPS. Хотя эта машина по своим характеристикам и уступала «Эльбрусам», но потенциально её конструктивные особенности могли оказаться весьма привлекательными для пользователей. Во-первых, оригинальное решение системы

из четырех скалярных процессоров и одного общего векторного процессора позволило значительно уменьшить занимаемый объем: вся центральная часть машины размещалась в стойке, примерно равной по габаритам распространенному мэйнфрейму ЕС1066. Во-вторых, ЕС1191 реализовывала дружественный интерфейс пользователя ЕС ЭВМ (берущий начало от ОС фирмы IBM) с применением всего разнообразия системных и прикладных программных продуктов, разработанных под эту архитектуру. И наконец, «козырной картой» ЕС1191 была воздушная система охлаждения, которая резко снижала затраты на производство и эксплуатацию машины по сравнению с жидкостным охлаждением «Эльбрусов».

В разработке суперкомпьютера ЕС1191 инженеры НИЦЭВТ отошли от практики копирования, но также параллельно в НИЦЭВТ был запущен проект разработки суперЭВМ «Электроника СС БИС», который базировался на структурных решениях и системе команд машин CRAY. В принципе, это направление хотя и предполагает постоянное отставание от прототипа, как говорится, «по определению», но практически «обречено на успех», особенно при детальном копировании оригинала. Во всяком случае Китай получил очень неплохие суперкомпьютеры VH-1 и VH-2 за счет почти «дословного» копирования машин CRAY-1 и CRAY X-MP.

Все суперкомпьютерные проекты советского периода использовали однотипную элементную базу – матричные большие интегральные схемы по технологии ЭСЛ со степенью интеграции 1500 вентилях на кристалл и быстродействием порядка 0.5 нс на вентиаль. Для середины 1980-х годов эти параметры были вполне на уровне и практически не отставали от параметров элементной базы японских суперкомпьютеров Fujitsu, NEC или Hitachi.

К сожалению, отечественная программа создания суперЭВМ, о которой говорилось выше, так и не была выполнена. «Эльбрус-3Б» и «Электроника СС БИС» существовали только в виде опытных образцов, «Эльбрус-3», после многолетнего долгостроя, так и не заработав, был тихо демонтирован, успев за годы «строительства» безнадежно устареть и морально, и физически, работы над ЕС1191 были заморожены, так и не явив миру реально работающего компьютера с этой архитектурой.

## Современное состояние

Последующий текст основан на лекции директора «Института программных систем имени А. К. Айламазяна», члена-корреспондента РАН Сергея АБРАМОВА, прочитанной им в рамках ежегодного Национального суперкомпьютерного форума (НСКФ-2018) 27 ноября 2018 года ([Суперкомпьютеры: обратные рекорды](#)).

Иллюстрация на Рис.454 показывает состояние суперкомпьютерной инфраструктуры в США, в странах Евросоюза, в России, Китае и Японии на ноябрь 2018 года.

Современный суперкомпьютер – это MIMD-система с массовым параллелизмом, с сотнями тысяч и даже миллионами процессорных ядер. Отдельные машины (узлы) в суперкомпьютере связаны высокоскоростной специализированной сетью. На вопрос о важности суперкомпьютерной отрасли ещё в 2004 году ответила Дебора Винс-Смит, президент Совета по конкурентоспособности США: «Технологии, таланты и деньги сейчас можно найти во всём мире, США сталкивается с беспрецедентной иностранной экономической конкуренцией. Страна, желающая победить в конкуренции, должна победить в вычислениях!»<sup>1</sup> (*“With technology, talent and capital now available globally, the US is facing unprecedented economic competition from abroad. The country that wants to out compete must out-compute”*). Эти слова сказал не программист, не учёный, а чиновник.

---

<sup>1</sup> Debora Wince-Smith, President of the Council on Competitiveness. 4 June 2004, «Supercharging Innovation Competitiveness: HPC Conference».

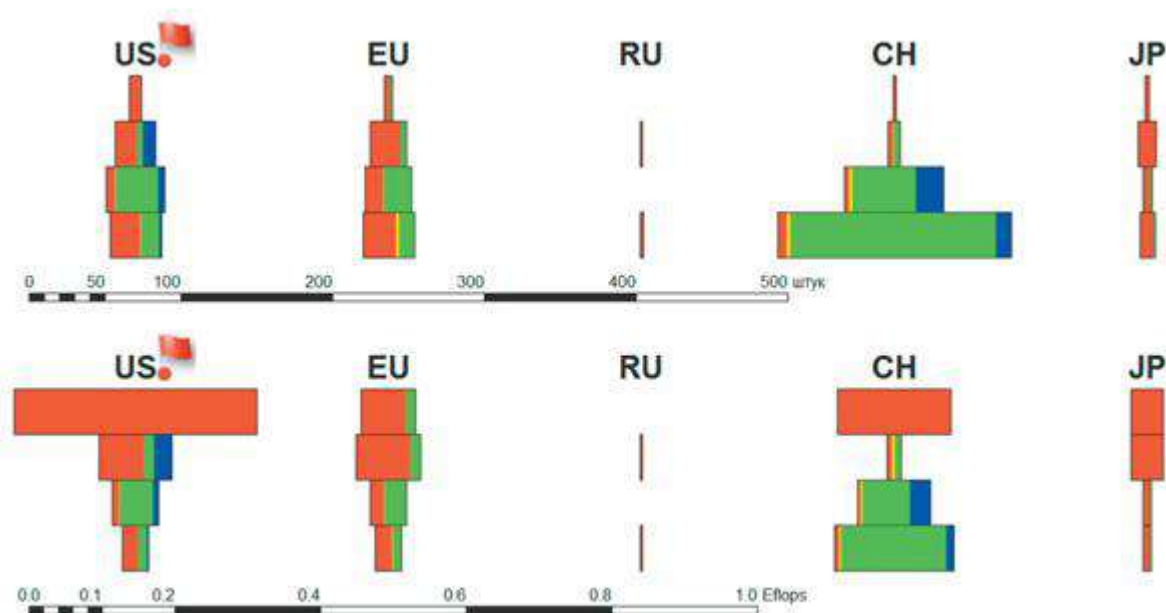


Рис.454 – Суперкомпьютерная инфраструктура США, стран Евросоюза, России, Китая и Японии на ноябрь 2018 года. На графике (вверху – количество машин, внизу – их производительность) зелёным цветом отмечены машины, которые используются в промышленности, красным – в науке и академических исследованиях, синим – для государственных и военных нужд. Верхний слой пирамидок – высший уровень суперкомпьютеров (Top1-20, 20 систем), второй слой – высокий уровень (Top21-100, 80 систем), третий слой – средний уровень (Top101-250, 150 систем), четвёртый – начальный уровень (Top251-500, 250 систем).

Нельзя говорить о цифровой экономике, если не построен её базис, где суперкомпьютерам отводится ключевое место. В передовых странах о разработке суперкомпьютерной киберинфраструктуры заботится государство. На создание новых суперкомпьютеров, стоимость которых может достигать сотен миллионов долларов США, выделяются деньги на федеральном, региональном и даже городском уровнях. По такой схеме были построены самые производительные суперкомпьютеры США и Китая.

К сожалению, у нас в стране на уровне власти пока этого понимания нет.

Для строительства экономики, основанной на знаниях, должна быть создана государственная система из мощных национальных суперкомпьютерных центров, объединённых сверхбыстрыми каналами связи в грид-систему.

Если дать нормальное определение суперкомпьютеру, то окажется, что в России на сегодня их очень мало. Что принято сегодня называть суперкомпьютером? Это система, имеющая производительность, которая соответствует производительностям машин, включённых в соответствующую данному времени редакцию списка [Top500](#) — свободно доступный массив данных или международный рейтинг суперкомпьютеров, который обновляется каждые полгода (в июне и ноябре) с 1993 года. За 25 лет (на ноябрь 2018 года) вышло 50 редакций этого списка. По этому достоверному, многоплановому, ретроспективному описанию суперкомпьютеров можно изучать параметры машин, следить за изменениями и судить о состоянии отрасли в мире.

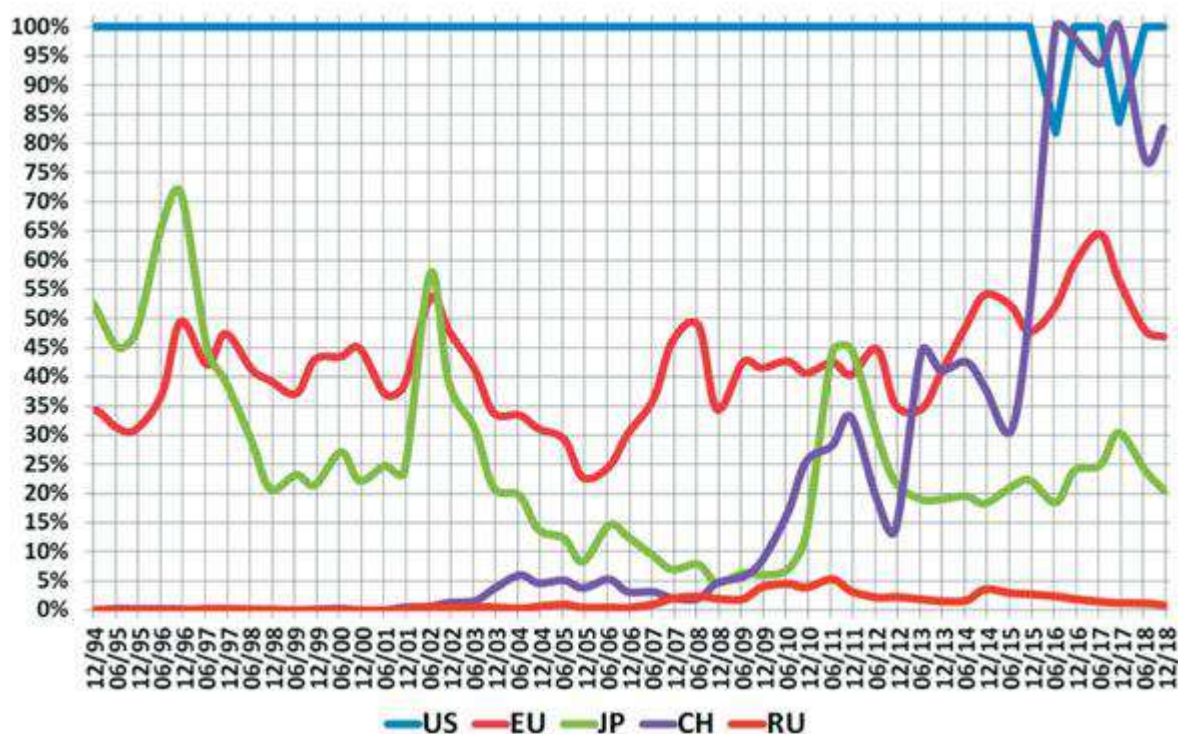


Рис.455 – Отношения сумм производительности всех суперкомпьютеров США, стран Евросоюза, России, Китая и Японии, где за 100% принято лучшее значение для этих стран.

Суперкомпьютеры, которые входят в Top500, отличаются по техническим характеристикам, а значит, и по инженерным решениям. Поэтому сравнивают их по производительности<sup>1</sup> на реальных задачах.

Машины для самых сложных задач значительно превосходят по производительности и другим техническим параметрам большинство существующих компьютеров и совершают много операций в секунду, отличаются большой памятью и высокими скоростями внутренних обменов данными.

Суперкомпьютерная отрасль в мире развивалась быстрыми темпами. Согласно закону Мура, производительность машин удваивалась каждые полтора года или в тысячу раз за одиннадцать лет по экспоненциальному закону. С помощью программы Top500 Analyzer была посчитана скорость роста производительности машин и было обнаружено, что с определённого момента она начала падать. Перелом произошёл примерно в 2008 году — экспоненциальный характер роста производительности сохранился, но уменьшилось основание экспоненты. До 2008 года производительность самой мощной системы в мире увеличивалась в 1000 раз за 11 лет: 1 Mflops ( $10^6$  flops) в 1975-м, 1 Gflops ( $10^9$  flops) в 1986-м, 1 Tflops ( $10^{12}$  flops) в 1997-м, 1 Pflops ( $10^{15}$  flops) в 2008 году. Ожидалось, что в 2019 году производительность вырастет до 1 Eflops ( $10^{18}$  flops), но, очевидно, это произойдёт с «опозданием» на три-четыре года (в 2022—2023 годах). После переломного года производительность увеличивается в 1000 раз уже не за 11 лет, а за больший срок. Например, в 2016—2017 годах производительность увеличивалась в 1000 раз за 17—19 лет. Очевидно, в период 2008—2017 годов суперкомпьютерная отрасль во всём мире столкнулась с серьёзными научно-

<sup>1</sup> Под словом «производительность» понимается максимальная реальная производительность, которую на практике система смогла продемонстрировать, решая систему линейных уравнений (так называемый тест Linpack). Слово «максимальная» связано с тем, что одни размеры систем линейных уравнений могут оказаться «более удобными» для конкретного решения, чем другие. Производительность, понимаемую таким образом, принято обозначать «Rmax».



техническими проблемами, поэтому прогнозы достижения новых рубежей производительности таковы: 1 Eflops ( $10^{18}$  flops) — примерно в 2022 году<sup>1</sup>, а 1 Zflops ( $10^{21}$  flops) — приблизительно в 2039 году.

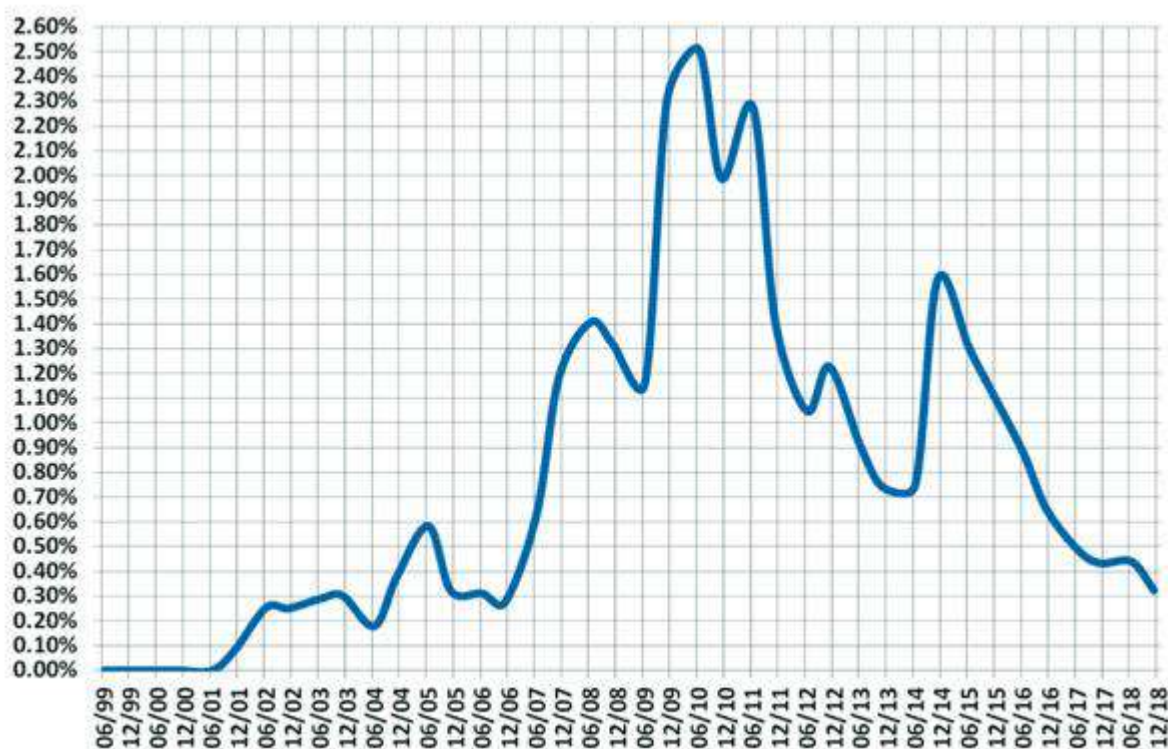


Рис.456 – Доля России в совокупной мировой вычислительной производительности суперкомпьютеров по годам.

Исследование данных Top500 показывает, что передовые страны (США, Китай, страны Евросоюза и Япония) создали гармоничную суперкомпьютерную инфраструктуру. Особенно развернулся Китай, который в последние годы успешно соревнуется с США в борьбе за лидерство (Рис.457) (занимая первые места в Top500 на протяжении пяти лет – с 2013 по 2017 год), и даже строит суперкомпьютерные центры в других странах.

<sup>1</sup> В июне 2020 года самым быстрым суперкомпьютером стал японский Fugaku («гора Фудзи»). Заявленная производительность Фугаку составляет **Rpeak 0,54 эксафлопс** в тесте FP64, используемом TOP500. До заветного эксафлопса (Eflops,  $10^{18}$ ) осталась всего двойка. Лидерство Fugaku было подтверждено в ноябре 2020 года и в июне 2021г.



|                                                                                                                                                                                           |                                                                                                                                                                                           |                                                                                                                                                                                           |                                                                                                                                                                                               |                                                                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1993</b><br><br>     | <b>1994</b><br><br>     | <b>1995</b><br><br>     | <b>1996</b><br><br>     | <b>1997</b><br><br>     |
| <b>1998</b><br><br>     | <b>1999</b><br><br>     | <b>2000</b><br><br>     | <b>2001</b><br><br>     | <b>2002</b><br><br>     |
| <b>2003</b><br><br>     | <b>2004</b><br><br>     | <b>2005</b><br><br>     | <b>2006</b><br><br>     | <b>2007</b><br><br>     |
| <b>2008</b><br><br>     | <b>2009</b><br><br>     | <b>2010</b><br><br>     | <b>2011</b><br><br>     | <b>2012</b><br><br>     |
| <b>2013</b><br><br> | <b>2014</b><br><br> | <b>2015</b><br><br> | <b>2016</b><br><br> | <b>2017</b><br><br> |
| <b>2018</b><br><br> | <b>2019</b><br><br> | <b>2020</b><br><br> | <b>2021</b><br>                                                                                          |                                                                                                                                                                                               |

Рис.457 – Top1 по странам

Евросоюз в настоящее время (ноябрь 2018) отстаёт на 50% от США по сумме производительности суперкомпьютеров в этих странах. К сожалению, российские показатели ухудшаются, а суммарная производительность машин, установленных в России, составляет всего около 5% производительности американских.

Последние годы мы бьём рекорды в отставании, всё больше отдаляясь от США, Китая, Европы и Японии. Были времена, когда мы резко сокращали разрыв, но в последние годы такой тенденции не просматривается.

Что можно сказать о технологическом уровне отечественных машин? Печально, но наши технологии отстали где-то на 8.5 лет, а когда-то отставание было 2.5 года. США, Китай, Япония и Евросоюз вносят весомый вклад в совокупную мировую производительность суперкомпьютеров: Китай — 31.1%, США — 37.64%, Япония — 7.74%, страны Евросоюза — 17.65%. У России максимальный вклад был в 2010 году — 2.5% в мировой ВВП, сейчас он составляет всего 0.32%.

Доли Китая, США, Японии и Евросоюза весьма значительны. Легко оценить устремлённость этих стран к цифровой экономике или индекс цифровизации (насколько экономика насыщена счётом), зная долю страны в мировом ВВП.

В таблице на Рис.458 показаны индексы цифровизации экономик различных стран (на ноябрь 2018 года): А — доля страны в суммарной производительности всех суперкомпьютеров мира; В — доля страны в мировом ВВП, отношение А/В — индекс цифровизации экономики.

| Страна   | Доля<br>$\Sigma R_{\max}$<br>(А), % | Доля<br>$\Sigma \text{ВВП}$<br>(В), % | Отноше-<br>ние А/В |
|----------|-------------------------------------|---------------------------------------|--------------------|
| Китай    | 31,11                               | 14,84                                 | 2,10               |
| США      | 37,64                               | 24,32                                 | 1,55               |
| Япония   | 7,74                                | 5,91                                  | 1,31               |
| Евросоюз | 17,65                               | 21,37                                 | 0,83               |
| Россия   | 0,32                                | 1,8                                   | 0,18               |

**Рис.458 – Соотношение Rmax и ВВП для некоторых стран.**

Как видно из таблицы, Китай, США, Япония, страны Евросоюза реально устремлены к цифровой экономике. Что касается России, то приходится констатировать, что в нашей стране нет никакой сбалансированной суперкомпьютерной инфраструктуры, то есть инфраструктуры, включающей системы из всех четырёх уровней производительности (высшая, высокая, средняя и начальная), используемых во всех отраслях применения (исследования, индустрия, военные и государственные нужды). Доля России в суперкомпьютерном «мире» более чем в пять раз меньше её доли в мировом ВВП. По индексу цифровизации экономики положение России хуже в 12 раз, чем у Китая, в 9 раз, чем у США, в 7 раз, чем у Японии, и в 5 раз, чем у стран Евросоюза.

*Все графики и диаграммы в данной статье построены с использованием комплекса Top500 Analyzer<sup>1</sup>.*

<sup>1</sup> Абрамов С. М. Top500 Analyzer — программа для анализа данных рейтинга Top500 // Электронный ресурс в сети интернет <https://goo.gl/Mr89Ho>.

## ГЛАВА 9. АРХИТЕКТУРА ВИРТУАЛЬНЫХ МАШИН

В этой главе будет дано описание основ технологии виртуализации, главным образом на платформе x86; будут приведены примеры нескольких современных гипервизоров, первого и второго типов, с описанием их возможностей.

В части аппаратной поддержки виртуализации два основных производителя – Intel и AMD – в своих x86-совместимых процессорах реализовали довольно серьёзно различающиеся между собой подходы. Если, лицензируя в своё время решение x86-64 у AMD, Intel практически «дословно» воспроизвела его в своей EMТ64, то с виртуализацией компания AMD всех удивила: она выпустила свою собственную технологию виртуализации, бинарно совершенно несовместимую с технологией, предложенной Intel годом ранее. Решение AMD было и совершеннее, и современнее. В качестве доказательства этого утверждения можно привести пример последующей адаптации Intel-ом технологии под названием Rapid Virtualization Indexing (ранее известной, как Nested Page Tables) от AMD, которую Intel в своём процессоре назвала Extended Page Tables (EPT). А несовместимость в технологиях виртуализации проявилась лишь в относительно небольшой части кода модуля VMM (монитор виртуальных машин), и, в итоге оказалась несущественным фактором, т.к. обе технологии виртуализации были быстро поддержаны всеми основными разработчиками программного обеспечения (гипервизоров). Думается, что без такой поддержки компания AMD быстро исчезла бы с рынка серверных процессоров, да и с десктопного, пожалуй, тоже – слишком важным стал фактор поддержки аппаратной виртуализации к тому времени. Несовместимое с Intel решение поддержки виртуализации было серьёзным риском для AMD, но риск оправдался, рынок принял оба решения.

### **Виртуализация**

Виртуализация компьютерных ресурсов интересовала разработчиков компьютеров задолго до изобретения микропроцессора, ещё во времена преобладания больших систем — мейнфреймов, ресурсы которых были очень дорогими, и их простой был элементарно невыгоден. Виртуализация позволяла повысить степень утилизации таких систем, при этом избавляя пользователей и прикладных программистов от необходимости переписывать своё ПО, так как с их точки зрения виртуальная машина была практически идентична физической. Пионером в этой области являлась фирма IBM с мейнфреймами System/360, System/370, созданными в 1960-1970-х гг.

### **Классические критерии виртуализуемости**

Критерии возможности создания эффективного монитора виртуальных машин были получены примерно в то же время, в 1960-1970-х гг. Они сформулированы в классической работе 1974 года Жеральда Попека и Роберта Голдберга (Gerald J. Popek and Robert P. Goldberg) “Formal requirements for virtualizable third generation architectures” («Формальные требования для виртуализируемых архитектур третьего поколения»).

Требования, сформулированные Попеком-Голдбергом – это набор условий, достаточных для того, чтобы компьютерная архитектура могла эффективно поддерживать виртуализацию системы. Несмотря на то, что эти требования были выведены на основе упрощающих предположений, они по-прежнему представляют собой удобный способ определения того, поддерживает ли компьютерная архитектура эффективную виртуализацию, и предоставляют руководящие принципы для проектирования виртуализированных компьютерных архитектур.

### **Определение монитора виртуальных машин (VMM)**

В статье Попека-Голдберга используется упрощённое представление «стандартной» ЭВМ, состоящей из одного центрального процессора и линейной однородной оперативной памяти. Периферийные устройства, а также средства взаимодействия с ними опускаются.

Процессор поддерживает два режима работы: режим супервизора, используемый операционной системой, и режим пользователя, в котором исполняются прикладные приложения. Память поддерживает режим сегментации, используемый для организации виртуальной памяти.

Системные виртуальные машины (подробнее описаны далее) способны виртуализировать полный набор аппаратных ресурсов, включая процессор (или процессоры), память и ресурсы хранения, а также периферийные устройства. Монитор виртуальной машины (VMM, также называемый гипервизором) – это часть программного обеспечения, которая обеспечивает абстракцию виртуальной машины. При анализе среды, созданной VMM, есть три интересных свойства:

- **Эквивалентность (Equivalence)** – программа, работающая под управлением VMM, должна демонстрировать поведение, полностью идентичное её исполнению в реальной системе, за исключением эффектов, вызванных двумя обстоятельствами: различием в количестве доступных ресурсов (например, VM может иметь меньший объём памяти) и длительностями выполнения операций (из-за возможности разделения времени исполнения с другими VM).
- **Контроль ресурсов/Безопасность/Изоляция (Resource control/Safety)** – VMM должен полностью контролировать виртуализированные ресурсы, каждая виртуальная машина должна иметь доступ только к тем ресурсам, которые были ей назначены. Она не должна иметь возможности влиять на работу как монитора, так и других VM.
- **Эффективность/Производительность (Efficiency/Performance)** – статистически преобладающая часть машинных инструкций должна выполняться процессором напрямую, без вмешательства VMM.

Требование эффективности является самым неоднозначным из трёх перечисленных требований. В случае симуляторов, основанных на интерпретации инструкций, условие эффективности не выполняется, т.к. каждая инструкция гостевой машины требует обработки симулятором. По терминологии Попека и Голдберга, VMM должен представлять все три свойства. В терминологии, используемой в справочнике Смита и Наира (2005), обычно предполагается, что VMM удовлетворяет свойствам эквивалентности и управления ресурсами, а те, которые дополнительно соответствуют свойству производительности, называются эффективными VMM.

Попек и Голдберг описывают характеристики, которыми должна обладать архитектура набора команд (ISA, instruction set architecture) физической машины для запуска VMM, обладающих вышеуказанными свойствами. Их анализ выводит такие характеристики с использованием модели «архитектур третьего поколения» (например, IBM 360, Honeywell 6000, DEC PDP-10), которая, тем не менее, является достаточно общей, чтобы её можно было распространить на современные машины.

## Классы инструкций

Состояние процессора содержит минимум три регистра: **M**, определяющий, находится ли он в режиме супервизора **s** или пользователя **u**, **P** — указатель текущей инструкции и **R** — состояние, определяющее границы используемого сегмента памяти (в простейшем случае **R** задаёт отрезок, т.е.  $R = (l, b)$ , где  $l$  — адрес начала диапазона,  $b$  — его длина).

Память **E** состоит из фиксированного числа ячеек, к которым можно обращаться по их номеру  $t$ , например,  $E[t]$ . Размер памяти и ячеек для данного рассмотрения несущественен.

При исполнении каждая инструкция  $i$  в общем случае может изменить как (**M**, **P**, **R**), так и память **E**, т.е. она является функцией преобразования:  $(M1, P1, R1, E1) \rightarrow (M2, P2, R2, E2)$ .

Считается, что для некоторых входных условий инструкция вызывает исключение ловушки (trap), если в результате её исполнения содержимое памяти не изменяется, кроме единственной ячейки  $E[0]$ , в которую помещается предыдущее состояние процессора ( $M1, P1, R1$ ). Новое состояние процессора ( $M2, P2, R2$ ) при этом копируется из  $E[1]$ . Другими словами, ловушка позволяет сохранить полное состояние программы на момент до начала исполнения её последней инструкции и передать управление обработчику, в случае обычных систем обычно работающему в режиме супервизора и призванного обеспечить дополнительные действия над состоянием системы, а затем вернуть управление в программу, восстановив состояние из  $E[0]$ .

Далее, ловушки могут иметь два признака:

1. Вызванные попыткой изменить состояние процессора (ловушка потока управления).
2. Обращения к содержимому памяти, выходящему за пределы диапазона, определённого в  $R$  (ловушка защиты памяти).

Эти признаки не взаимоисключающие, то есть результатом исполнения могут быть одновременно ловушка потока управления и защиты памяти.

Машинные инструкции рассматриваемого процессора можно классифицировать следующим образом:

- **Привилегированные** (privileged). Инструкции, исполнение которых с  $M = u$  всегда вызывает ловушку потока управления. Другими словами, такая инструкция может исполняться только в режиме супервизора, иначе она обязательно вызывает исключение.
- **Служебные** (sensitive). (Установившего русского термина для этого понятия нет. Иногда в литературе встречается перевод «чувствительные» инструкции). Класс состоит из двух подклассов:
  - Инструкции, исполнение которых закончилось без ловушки защиты памяти и вызвало изменение  $M$  и/или  $R$ . Они могут менять режим процессора из супервизора в пользовательский или обратно, или изменять положение и размер доступного сегмента памяти.
  - Инструкции, поведение которых в случаях, когда они не вызывают ловушку защиты памяти, зависят или от режима  $M$ , или от значения  $R$ .
- **Безвредные** (innocuous). Не являющиеся служебными. Самый широкий класс инструкций, не манипулирующие ничем, кроме указателя инструкций  $P$  и памяти  $E$ , поведение которых не зависит от того, в каком режиме или с каким адресом в памяти они расположены.

## Достаточное условие построения монитора VM

Соблюдение трёх сформулированных выше условий возможности построения монитора виртуальных машин даётся в следующем предложении: множество служебных инструкций является подмножеством привилегированных инструкций (Рис.459). Опуская формальное доказательство теоремы 1 из статьи Попека-Голдберга, отметим следующие обстоятельства:

- Изоляция обеспечивается размещением монитора в режиме супервизора, а VM — только в пользовательском. При этом последние не могут самовольно изменить системные ресурсы — попытка вызовет ловушку потока управления на служебной инструкции и переход в монитор, а также память из-за того, что конфигурация не допускает этого, и процессор выполнит ловушку защиты памяти.
- Эквивалентность доказывается тем, что безвредные инструкции выполняются одинаково вне зависимости от того, присутствует ли в системе монитор или нет,

а служебные всегда вызывают исключение и интерпретируются. Отметим, что даже в описанной выше простой схеме проявляется первое ослабляющее условие: даже без учёта памяти, необходимой для хранения кода и данных гипервизора, объём доступной для VM памяти будет как минимум на две ячейки меньше, чем имеется у хозяйской системы.

- Эффективность гарантируется тем, что все безвредные инструкции внутри VM исполняются напрямую, без замедления. При этом подразумевается, что их множество включает в себя «статистически преобладающее подмножество инструкций виртуального процессора».



Рис.459 - Выполнение условия виртуализуемости. Множество служебных инструкций является подмножеством привилегированных

## Ограничения применимости критерия виртуализуемости

Несмотря на простоту использованной модели и полученных из неё выводов, работа Голдберга и Попека является актуальной до сих пор. Несоблюдение описанных в ней условий вовсе не делает создание или использование виртуальных машин на некоторой архитектуре принципиально невозможным, и есть практические примеры реализаций, подтверждающие это. Однако соблюсти оптимальный баланс между тремя свойствами: изоляцией, эквивалентностью и эффективностью, — становится невозможным. Чаще всего расплачиваться приходится скоростью работы виртуальных машин из-за необходимости тщательного поиска и программного контроля за исполнением ими служебных, но не привилегированных инструкций, так как сама аппаратура не обеспечивает этого (Рис.460). Даже единственная такая инструкция, исполненная напрямую VM, угрожает стабильной работе монитора, и поэтому он вынужден сканировать весь поток гостевых инструкций.



Рис.460 - Невыполнение условия виртуализуемости. Служебные, но не привилегированные инструкции требуют реализации сложной логики в мониторе

## Наборы инструкций некоторых архитектур

В этом разделе представлены некоторые архитектуры и их отношение к требованиям виртуализации.

## ***PDP-10***

В архитектуре PDP-10 есть несколько инструкций, которые являются чувствительными (служебными) (изменяют или запрашивают режим процессора), но не являются привилегированными. Эти инструкции сохраняют или восстанавливают коды условий, содержащие биты USER или IOT:

- JSR: jump to subroutine
- JSP: jump and save program counter
- PUSHJ: push down and jump
- JRST: jump and restore

## ***System/370***

Все чувствительные инструкции в System/370 являются привилегированными: это удовлетворяет требованиям виртуализации.

## ***Motorola MC68000***

Motorola MC68000 имеет единственную непривилегированную чувствительную (служебную) инструкцию:

- MOVE from SR

Эта инструкция является чувствительной, поскольку она позволяет получить доступ ко всему регистру состояния, который включает не только коды условий, но также бит пользователя/супервизора, уровень прерывания и управление трассировкой. В большинстве более поздних членов семейства, начиная с MC68010, инструкция MOVE from SR была сделана привилегированной, и была предоставлена новая инструкция MOVE from CCR, чтобы разрешить доступ только к регистру кода условия.

## ***IA-32 (x86)***

Набор инструкций IA-32 процессора Pentium содержит 18 чувствительных (служебных) непривилегированных инструкций. Их можно разделить на две группы:

- Инструкции для чувствительных регистров: чтение или модификация чувствительных регистров и/или участков памяти, таких как регистр часов или регистры прерывания:
  - SGDT, SIDT, SLDT
  - SMSW
  - PUSHF, POPF
- Инструкции системы защиты: ссылка на систему защиты хранилища, память или систему перемещения адресов:
  - LAR, LSL, VERR, VERW
  - POP
  - PUSH
  - CALL, JMP, INT n, RET
  - STR
  - MOV (сегментные регистры)

Введение в 2005 году наборов инструкций AMD-V и Intel VT-x позволяет процессорам x86 соответствовать критериям виртуализации Попека-Голдберга.



## IA-64

Усилия, необходимые для поддержки виртуализации на архитектуре IA-64, описаны в статье 2000 года Магенхеймера и Кристиана (["vBlades: Optimized Paravirtualization for the Itanium Processor Family"](#)).

## SPARC

«Гиперпривилегированный» режим для архитектуры UltraSPARC был определен в статье [UltraSPARC Architecture 2005: One Architecture... Multiple Innovative Implementations \(DraftD0.9\)](#). В ней определяется платформа sun4v, которая является надстройкой платформы sun4u, оставаясь совместимой со спецификацией SPARC v9 Level-1.

## PowerPC

Все конфиденциальные (служебные) инструкции в наборе команд PowerPC являются привилегированными.

## Производительность на практике

Требование эффективности в определении VMM, данное Попеком и Голдбергом, касается только выполнения непривилегированных инструкций, которые должны выполняться процессором нативно (native), без интерпретации. Это то, что отличает VMM от более общего класса программного обеспечения, эмулирующего оборудование. К сожалению, даже в архитектуре, отвечающей требованиям Попека и Голдберга, производительность виртуальной машины может значительно отличаться от производительности на реальном оборудовании. Ранние эксперименты, проведенные на System/370 (которая полностью соответствует формальным требованиям теоремы 1), показали, что производительность виртуальной машины может составлять всего 21% от собственной машины в некоторых тестах. Стоимость перехвата и эмуляции привилегированных инструкций в VMM может быть значительной. Это побудило инженеров IBM внедрить ряд вспомогательных аппаратных средств, которые примерно вдвое увеличили производительность виртуальных машин System/370. Эти вспомогательные средства добавлялись в несколько этапов, и в итоге на поздних моделях System/370 было сделано более 100 таких модификаций (assists).

Одним из основных факторов развития вспомогательных аппаратных средств для System/370 была сама виртуальная память. Когда гостем была операционная система, которая сама реализовывала виртуальную память, даже непривилегированные инструкции могли получать более длительное время выполнения – накладные расходы, возникающие из-за требования доступа к таблицам трансляции, не используемым при нативном исполнении инструкций (*«таблицы теневых страниц»*).

## Виды виртуализации

### Оборудование

- Эмуляция — полная виртуализация (полная виртуализация платформы); например, QEMU или эмуляторы игровых консолей.

### Операционные системы

- Программная виртуализация
  - Динамическая трансляция; при динамической (бинарной) трансляции проблемные команды гостевой операционной системы перехватываются гипервизором.
  - Паравиртуализация: операционная система взаимодействует с программой гипервизора, который предоставляет ей гостевой API, вместо использования напрямую таких ресурсов, как таблица страниц памяти.

- Встроенная виртуализация
- Аппаратная виртуализация — виртуализация с поддержкой специальной процессорной архитектуры. В отличие от программной виртуализации, с помощью данной техники возможно использование изолированных гостевых систем, управляемых гипервизором напрямую.
- Виртуализация на уровне операционной системы: работа нескольких экземпляров пространства пользователя в рамках одной ОС (одно ядро). Примерами могут быть Docker, LXC, chroot в Linux, jail во FreeBSD.

### ***Программное обеспечение***

- Виртуализация приложений (также виртуализация рабочего окружения): работа отдельных приложений в среде, отделённой от основной ОС. Эта концепция тесно связана с переносимыми (portable) приложениями. Примерами могут быть: Citrix XenApp, Microsoft App-V.
- Виртуализация сервисов: эмуляция поведения системных компонентов, необходимых для запуска приложения в целях отладки и тестирования (Application Under Test). Вместо виртуализации компонентов целиком, эта технология виртуализует только необходимые компоненты. Примеры: SoapUI, Parasoft Virtualize.

### ***Память***

- Виртуализация памяти (memory virtualization) — объединением оперативной памяти из различных ресурсов в единый массив. Реализации: Oracle Coherence, GigaSpaces XAP.
- Виртуальная память — изоляция адресного пространства приложения от всего адресного пространства. Применяется во всех современных ОС.

### ***Хранилище данных***

- Виртуализация хранения данных, представление набора физических носителей в виде единого физического носителя.
  - Блочная виртуализация
  - Файловая виртуализация
- Распределённая файловая система — любая файловая система, которая позволяет получать доступ к файлам с нескольких устройств, с помощью компьютерной сети.
- Виртуальная файловая система (Virtual File System) — уровень абстракции поверх конкретной реализации файловой системы. Целью VFS является обеспечение единообразного доступа клиентских приложений к различным типам файловых систем.
- Гипервизор хранения данных — любая файловая система, которая позволяет получать доступ к файлам с нескольких устройств, с помощью компьютерной сети.
- Виртуальная файловая система (storage hypervisor) — программа, которая управляет виртуализацией пространства для хранения данных и может объединять различные физические пространства в единый логический массив.
- Виртуализация устройств хранения данных: виртуализация жёсткого (логический диск) или оптического диска (например, DAEMON Tools).

### ***База данных***

- Виртуализация данных (data virtualization) — представление данных в абстрактном виде, независимо от нижележащих систем управления и хранения

данных, а также их структуры. Это подход к унификации данных из нескольких источников на одном уровне, чтобы приложения, средства отчётности и конечные пользователи могли получать доступ к данным, не нуждаясь в подробных сведениях об исходных источниках, местоположениях и структурах данных.

- Виртуализация баз данных.

### ***Сеть***

- Виртуализация сети (network virtualization) — процесс объединения аппаратных и программных сетевых ресурсов в единую виртуальную сеть.
  - Внешняя, соединяющая множество сетей в одну виртуальную.
  - Внутренняя, создающая виртуальную сеть между программными контейнерами внутри одной системы.
- Виртуальная частная сеть (virtual private network) — обеспечение одного или нескольких сетевых соединений поверх другой сети.

## **Области применения виртуализации**

### ***Виртуальные машины***

Виртуальная машина — это окружение, которое представляется для «гостевой» операционной системы, как аппаратное. Однако на самом деле это программное окружение, которое эмулируется программным обеспечением хостовой системы. Эта эмуляция должна быть достаточно надёжной, чтобы драйверы гостевой системы могли стабильно работать. При использовании паравиртуализации, виртуальная машина не эмулирует аппаратное обеспечение, а, вместо этого, предлагает использовать специальный API.

Примеры применения:

- Тестовые лаборатории и обучение: тестированию в виртуальных машинах удобно подвергать приложения, влияющие на настройки операционных систем, например, устанавливаемые приложения. За счёт простоты в развёртывании виртуальных машин, они часто используются для обучения новым продуктам и технологиям.
- Распространение предустановленного программного обеспечения: многие разработчики программных продуктов создают готовые образы виртуальных машин с предустановленными продуктами и предоставляют их на бесплатной или коммерческой основе. Такие услуги предоставляют VMware VMTN или Parallels PTN.

### ***Виртуализация ресурсов***

Виртуализация ресурсов (или разделение ресурсов, partitioning) может быть представлена как разделение одного физического узла на несколько частей, каждая из которых видна для владельца в качестве отдельного сервера. Не является технологией виртуальных машин, осуществляется на уровне ядра операционной системы.

В системах с гипервизором второго типа обе операционные системы (гостевая и гипервизора) отнимают физические ресурсы, и требуют отдельного лицензирования. Виртуальные серверы, работающие на уровне ядра ОС, почти не теряют в быстродействии, что дает возможность запускать на одном физическом сервере сотни виртуальных.

Дисковое пространство или пропускной канал сети разделены на некоторое количество меньших составляющих, и потому легче используемых ресурсов того же типа.

Например, к реализации разделения ресурсов можно отнести OpenSolaris Network Virtualization and Resource Control (Проект Crossbow), позволяющий создавать несколько виртуальных сетевых интерфейсов на основе одного физического.

Агрегация, распределение или добавление множества ресурсов в большие ресурсы или объединение ресурсов. Например, симметричные мультипроцессорные системы объединяют множество процессоров; RAID и дисковые менеджеры объединяют множество дисков в один большой логический диск; RAID и сетевое оборудование использует множество каналов, объединённых так, чтобы они представлялись, как единый широкополосный канал. На мета-уровне компьютерные кластеры делают все вышеперечисленное. Иногда сюда же относят сетевые файловые системы, абстрагированные от хранилищ данных на которых они построены, например, VMware VMFS, Solaris/OpenSolaris ZFS, NetApp WAFL.

### **Виртуализация приложений**

Виртуализация приложений — процесс использования приложения, преобразованного из требующего установки в операционную систему в не требующее инсталляции (требуется только его запустить). Для виртуализации приложений программное обеспечение виртуализатора определяет при установке виртуализуемого приложения, какие требуются компоненты ОС, и эмулирует их. Таким образом, создаётся необходимая специализированная среда для конкретно этого виртуализуемого приложения и, тем самым, обеспечивается изолированность работы этого приложения. Для создания виртуального приложения виртуализуемое приложение помещается в контейнер, оформленный, как правило, в виде папки файловой системы. При запуске виртуального приложения запускается виртуализуемое приложение и контейнер, являющийся для него рабочей средой. Рабочая среда запускается и предоставляет локальные, ранее созданные ресурсы, которое включает в себя ключи реестра, файлы и другие компоненты, необходимые для запуска и работы приложения. Такая виртуальная среда работает как прослойка между приложением и операционной системой, что позволяет избежать конфликтов между приложениями. Виртуализацию приложений обеспечивают, например, программы Citrix XenApp, SoftGrid и VMware ThinApp.

Достоинства:

- изолированность исполнения приложений;
- каждый раз в первозданном виде: не загромождается реестр, нет конфигурационных файлов — очень полезно для сервера;
- меньшие ресурсозатраты по сравнению с эмуляцией полной операционной системы.

### **Виртуальная машина**

Виртуальная машина (VM, virtual machine) — программная и/или аппаратная система, эмулирующая аппаратное обеспечение некоторой платформы (target — целевая, или гостевая платформа) и исполняющая программы для target-платформы на host-платформе (host — хост-платформа, платформа-хозяин) или виртуализирующая некоторую платформу и создающая на ней среду, изолирующую программы или даже операционные системы друг от друга; также спецификация некоторой вычислительной среды (например: «виртуальная машина языка программирования Си»).

Виртуальная машина исполняет некоторый машинно-независимый код (например, байт-код<sup>1</sup>) или машинный код реального процессора. Помимо процессора, VM может эму-

---

<sup>1</sup> **Байт-код** (bytecode, также р-код, р-code от portable code) — стандартное промежуточное представление, в которое может быть преобразована автоматическими средствами исходная компьютерная программа. С технической точки зрения байт-код представляет собой машинно-независимый код низкого уровня, генерируе-

лизовать работу как отдельных компонентов аппаратного обеспечения, так и целого реального компьютера (включая BIOS, оперативную память, жёсткий диск и другие периферийные устройства). В последнем случае в VM, как и на реальный компьютер, можно устанавливать обычные операционные системы (например, Windows можно запускать в виртуальной машине под Linux или наоборот). На одном компьютере может функционировать несколько виртуальных машин (это может использоваться для имитации нескольких серверов на одном реальном сервере с целью оптимизации (консолидации) использования ресурсов сервера).

## Типы виртуальных машин

Принято различать два типа виртуальных машин:

### *Системные виртуальные машины*

Системные виртуальные машины (system virtual machines, также называемые виртуальными машинами с полной виртуализацией) обеспечивают полнофункциональную среду, полностью замещающую реальную машину. С помощью системных VM одна аппаратная платформа (хост) способна одновременно поддерживать несколько разных изолированных друг от друга гостевых систем.

«Виртуальная машина» была первоначально определена Попеком и Голдбергом (Popek и Goldberg) как *«эффективная изолированная копия реальной компьютерной машины»*, включая виртуальные машины, которые не имеют прямого соответствия какому-либо реальному оборудованию. Хост может эмулировать одновременно несколько гостевых систем, каждая из которых может эмулировать различные операционные системы и аппаратные платформы.

Концепция виртуальной машины как совокупности ресурсов, которые симулируют поведение реальной машины, появилась в Кембридже в конце 1960-х годов в виде расширения концепции виртуальной памяти вычислительной машины Atlas (Манчестер, Великобритания). Первоначальным мотивом появления концепции виртуальных машин было желание запускать несколько операционных систем на одной платформе, чтобы обеспечить разделение времени между несколькими однозадачными операционными системами. В некотором отношении системную виртуальную машину можно рассматривать как обобщение концепции виртуальной памяти, которая исторически предшествовала ей. IBM VM/CMS, первая система, допускающая полную виртуализацию, реализовала разделение времени, предоставив каждому пользователю однопользовательскую операционную систему, Conversational Monitor System (CMS). В отличие от виртуальной памяти, системная виртуальная машина дает пользователю право использовать в своём коде привилегированные инструкции. Этот подход имел определенные преимущества, такие как добавление устройств ввода/вывода, не доступных в стандартной хост-системе.

Страницы памяти в разных гостевых системах, которые имеют идентичное содержимое, могут быть совместно использованы несколькими виртуальными машинами, работающими на одном хосте, что позволяет сопоставить их с одной и той же физической страницей с помощью метода, называемого объединением одинаковых страниц ядра (KSM). Это особенно полезно для страниц, предназначенных только для чтения, таких как страницы, содержащие сегменты кода, что имеет место в случае нескольких виртуальных машин с одинаковым или похожим программным обеспечением, программными библиотеками, веб-серверами, компонентами промежуточного программного обеспечения и т.д. Гостевым операционным системам не требуется совместимость с аппаратным обеспечением хоста, что позволяет запускать разные операционные системы на одном компьютере (например,

---

мый транслятором из исходного кода. По форме байт-код похож на машинный код, но предназначен для исполнения не реальным процессором, а виртуальной машиной. В качестве виртуальной машины обычно выступает интерпретатор соответствующего языка программирования (например, байт-код Java).

Windows, Linux или устаревшие версии операционных систем, которые не могут работать с новым оборудованием).

### ***Процессные виртуальные машины***

Процессные виртуальные машины (process virtual machines) предназначены для выполнения компьютерных программ в независимой от платформы среде. Некоторые эмуляторы виртуальных машин, такие как QEMU, предназначены для эмуляции различных архитектур, и позволяют выполнять программные приложения и операционные системы, написанные для процессора с другой системой команд или другой архитектуры.

Процессная виртуальная машина, иногда называемая application VM или управляемой средой выполнения (Managed Runtime Environment, MRE), запускается как обычное приложение внутри обычной ОС хоста и поддерживает один процесс. Он создается при запуске этого процесса и уничтожается при выходе. Его цель – предоставить платформо-независимую среду программирования, которая абстрагирует детали базового оборудования или операционной системы и позволяет программе выполняться одинаково на любой аппаратной платформе.

Процессная виртуальная машина обеспечивает абстракцию языка программирования высокого уровня (по сравнению с абстракцией ISA (instruction set architecture) низкого уровня системной виртуальной машины). Виртуальные машины процессов реализуются с помощью интерпретатора; производительность, сравнимая с компилируемыми языками программирования, может быть достигнута за счет использования just-in-time (JIT<sup>1</sup>) компиляции.

Такой тип VM стал очень популярен с появлением языка программирования Java, который реализован с помощью виртуальной машины Java (JVM). В качестве других примеров можно привести виртуальную машину Parrot и .NET Framework, работающую на виртуальной машине, которая называется Common Language Runtime (CLR). Все они могут служить слоем абстракции для любого «классического» компьютерного языка. Ещё один очень хороший пример виртуализации на уровне процесса – проект Wine — свободное программное обеспечение, позволяющее пользователям UNIX-подобных операционных систем, например GNU/Linux, Apple macOS и других UNIX-подобных с поддержкой архитектур CPU x86, AMD64 (и других архитектур CPU при наличии совместимости с ними запускаемых приложений) исполнять 16-, 32- и 64- битные приложения, в том числе игры, созданные исключительно для Microsoft Windows, без наличия при этом установленной Microsoft Windows.

---

<sup>1</sup> **JIT-компиляция** (также динамическая трансляция или компиляция во время выполнения) – способ выполнения компьютерного кода, который включает компиляцию во время выполнения программы, а не до, как при обычной трансляции. Исходный код или, как правило, байт-код преобразуется в машинный код, который затем выполняется напрямую процессором. Система, реализующая JIT-компилятор, обычно непрерывно анализирует выполняемый код и определяет части кода, в которых ускорение, полученное от компиляции или перекompиляции, перевесит накладные расходы на компиляцию этого кода.

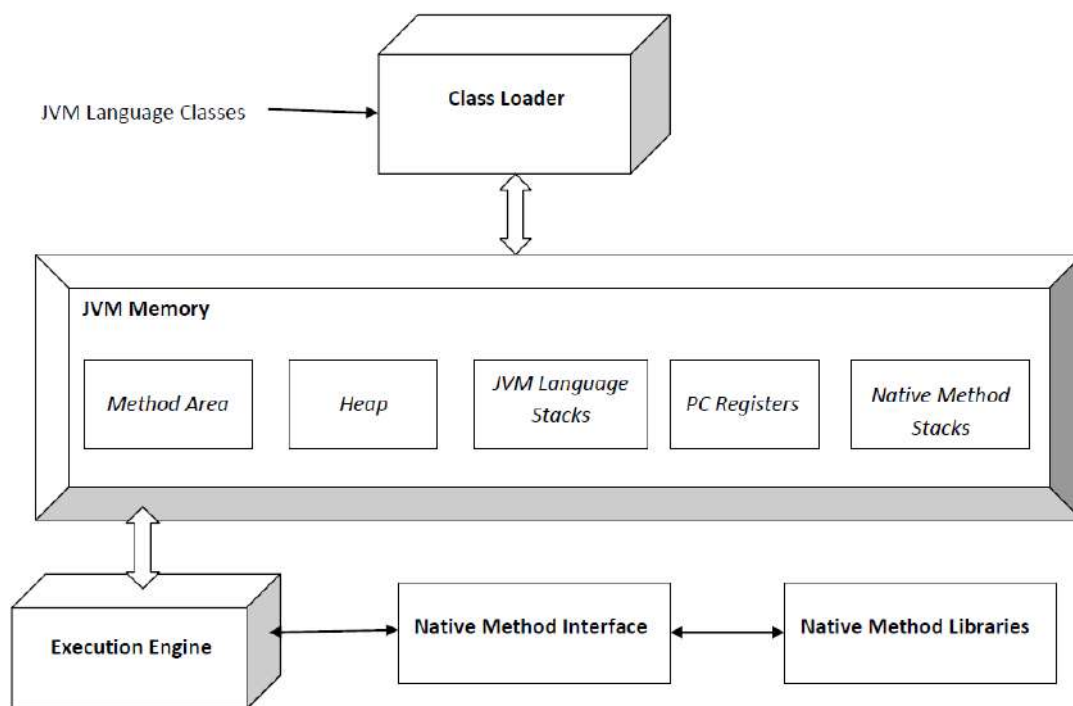


Рис.461 - Архитектура виртуальной (стековой) машины Java (JVM) на основе спецификации виртуальной машины Java SE 7 Edition

Частным случаем процессных виртуальных машин являются системы, которые абстрагируются от механизмов связи (потенциально гетерогенного) компьютерного кластера. Такая виртуальная машина состоит не из одного процесса, а из одного процесса на каждую физическую машину в кластере. Они предназначены для упрощения задачи программирования параллельных приложений, позволяя программисту сосредоточиться на алгоритмах, а не на механизмах связи, обеспечиваемых межсоединением и ОС. Они не скрывают того факта, что происходит коммуникация между разными VM, и не пытаются представить кластер как единую машину.

В отличие от других процессных виртуальных машин, эти системы не предоставляют конкретный определенный язык программирования, а встраиваются в стандартный язык; обычно такая система обеспечивает привязки для нескольких языков (например, С и Fortran). Примерами являются параллельная виртуальная машина (PVM) и интерфейс передачи сообщений (MPI). Строго говоря, они не являются виртуальными машинами, потому что приложения, запущенные поверх, по-прежнему имеют доступ ко всем службам ОС и, следовательно, не ограничиваются системной моделью.

## Гипервизор

Гипервизор<sup>1</sup> (Hypervisor; от др.-греч. ὑπέρ «над, выше, сверх» + лат. vīsiō «зрение; видение») или монитёр виртуальных машин (VMM) (в компьютерах) — программа или аппаратная схема, обеспечивающая или позволяющая одновременное, параллельное выполнение нескольких операционных систем на одном и том же хост-компьютере. Гипервизор также обеспечивает изоляцию операционных систем друг от друга, защиту и безопасность, разделение ресурсов между различными запущенными ОС и управление ресурсами.

<sup>1</sup> Термин «гипервизор» специально придуман IBM в 1972 году для того, чтобы подчеркнуть отличие от термина супервизор, которым традиционно называли ядро ОС, а точнее, менеджер ресурсов ядра в эпоху мэйнфреймов. Гипервизор является расширением и обобщением понятия супервизора: подобно тому, как супервизор в ядре ОС обеспечивает изоляцию пользовательских программ друг от друга, выделение и освобождение ресурсов для пользовательских процессов, так и гипервизор обеспечивает изоляцию и управление ресурсами для самих ОС как целого, вместе с их пользователями и процессами.



Гипервизор также может (но не обязан) предоставлять работающим под его управлением на одном хост-компьютере ОС средства связи и взаимодействия между собой (например, через обмен файлами или сетевые соединения) так, как если бы эти ОС выполнялись на разных физических компьютерах. Гипервизор сам по себе в некотором роде является минимальной операционной системой (микроядром или наноядром, в Microsoft его даже называют уровнем -1, подчёркивая, что он находится ниже в иерархии в терминах классических «колец безопасности» Ring0...Ring3). Он предоставляет запущенным под его управлением операционным системам сервис виртуальной машины, виртуализируя или эмулируя реальное (физическое) аппаратное обеспечение конкретной машины, и управляет этими виртуальными машинами, выделением и освобождением ресурсов для них. Гипервизор позволяет независимое «включение», перезагрузку, «выключение» любой из виртуальных машин с той или иной ОС. При этом операционная система, работающая в виртуальной машине под управлением гипервизора, может, но не обязана «знать», что она выполняется в виртуальной машине, а не на реальном аппаратном обеспечении.

## **История гипервизоров**

В конце 1960-х и вплоть до 1970-х годов, большинство систем виртуализации и гипервизоров работали на мэйнфреймах, разработанных компанией IBM. Использовались они для разработки процессов использования компьютера в режиме разделения времени, для тестирования новых операционных систем и идей для их усовершенствования или даже для изучения новых аппаратных концепций. Виртуализация позволила программистам разворачивать системы и устранять неисправности, не подвергая угрозам стабильность основной производственной системы, ну и к тому же она позволила уйти от разворачивания дополнительных дорогостоящих аппаратных платформ.

В середине 2000-х годов гипервизоры выходят на новый уровень, когда Unix, Linux и другие Unix-подобные операционные системы массово начали использовать технологии виртуализации. Причинами роста интереса к гипервизорам и технологиям виртуализации, во-первых, было существенное увеличение и расширение аппаратной составляющей – причина заключалась в значительно выросших аппаратных ресурсах – высокие тактовые частоты, многоядерные процессоры, значительно увеличившиеся объёмы поддерживаемой памяти (основной ресурс при запуске на хост-машине нескольких гостевых систем); во-вторых, появление аппаратной поддержки виртуализации в самых массовых тогда (2005 год) x86-архитектурах, что значительно уменьшило издержки и накладные расходы, сказывавшиеся на производительности виртуальных машин – программное обеспечение, особенно процессорно-ориентированное (активно потребляющее CPU) работало практически с той же скоростью, что и на «голом железе» (bare metal); в-третьих, значимую роль в стремительном росте интереса к технологиям виртуализации сыграла безопасность и надёжность благодаря усовершенствованию архитектуры гипервизоров; и, наконец, ещё одна важная причина — возможность запуска зависимых от ОС приложений в различных аппаратных или операционных средах. Кроме понятного и естественного присутствия поддержки виртуализации в серверных процессорах, такая поддержка быстро стала востребованной и на десктопных платформах. Intel здесь в первое время проводила довольно странную политику, долго не включая аппаратную поддержку виртуализации в своих массовых десктопных процессорах среднего уровня, тогда как AMD активировала эту поддержку сразу во всей линейке своих CPU, включая мобильные и настольные модели.

## **Типы гипервизоров**

### **Автономный гипервизор (Тип 1, X)**

Гипервизор первого типа (автономный, тонкий, исполняемый на «голом железе» — Type 1, native, bare-metal) — программа, исполняемая непосредственно на аппаратном

уровне компьютера и выполняющая функции эмуляции физического аппаратного обеспечения и управления аппаратными средствами и гостевыми ОС. То есть такой гипервизор сам по себе в некотором роде является минимальной операционной системой. Имеет свои собственные встроенные драйверы устройств, модели драйверов и планировщик и поэтому не зависит от базовой ОС. Так как автономный гипервизор работает непосредственно в окружении усечённого ядра, то он более производителен, но проигрывает в производительности виртуализации на уровне ОС и паравиртуализации. Например, кроссплатформенный гипервизор Хеп может запускать виртуальные машины в паравиртуальном режиме (зависит от ОС).

Примеры: VMware ESX, VMware Cloud, Citrix XenServer.

## **На основе базовой ОС (Тип 2, V)**

Гипервизор второго типа (хостовый, монитор виртуальных машин — hosted, Type-2, V) — специальный дополнительный программный слой, работающий (поверх) в одном кольце безопасности (Ring0) с ядром основной хостовой ОС, который в основном выполняет функции управления гостевыми ОС, а эмуляцию и управление аппаратурой берёт на себя хостовая ОС. Гостевой код может выполняться прямо (без эмуляции) на физическом процессоре, но доступ к устройствам ввода-вывода компьютера из гостевой ОС осуществляется через второй компонент — обычный процесс основной ОС — монитор уровня пользователя.

Примеры: Microsoft Virtual PC, VMware Workstation/Player, QEMU, Parallels, VirtualBox.

## **Гибридный (Тип 1+)**

Гипервизор гибридный (Hybrid, Type-1+) — объединенный вариант первых двух, в котором функции управления аппаратными средствами выполняются тонким гипервизором и специальной непривилегированной сервисной гостевой ОС, работающей под управлением тонкого гипервизора. Обычно гипервизор управляет напрямую процессором и памятью компьютера, а через сервисную ОС гостевые ОС работают с получают доступ к остальному физическому оборудованию.

Примеры: Microsoft Virtual Server, Sun Logical Domains, Xen, Citrix XenServer, Microsoft Hyper-V.

В широком смысле гипервизор — это все три перечисленные выше вида, а в узком — это гипервизор первого типа. Это нужно знать для понимания того, что именно имеется в виду, когда о некотором продукте говорится, что он — гипервизор.

Именно гипервизор первого типа считается классическим вариантом реализации архитектуры VM, как раз он был впервые реализован компанией IBM в 1960 г. в виде системы CP/CMS, которая считается прародителем сегодняшней z/VM\*.

Виртуализация систем x86 начиналась в конце 1990-х с использования второго (хостового) типа, который долгое время доминировал на рынке и тогда назывался просто «ПО виртуализации». Термин гипервизор стал использоваться применительно к x86-компьютерам примерно с 2005-2006 года в связи с началом широкого применения первого типа другими разработчиками для создания собственных продуктов, и чтобы отличить его от традиционных хостовых решений. В 2008 году тип 1 гипервизора стал уже доминирующим на рынке, и потому данный термин стал использоваться для обозначения всего ПО исполнения виртуальных машин.

Ниже, на Рис.462, представлены основные варианты управления виртуальными машинами, в соответствии с приведённым описанием типов гипервизоров.



Рис.462 - Основные варианты управления виртуальными машинами.

Примерно к 2009 году в проблематике виртуализации тема собственно гипервизоров, доминировавшая в 2007—2008 годах, отошла на второй план, уступив место вопросам управления виртуальными средами. Связано это было с выходом в 2008 году продуктов Microsoft Hyper-V и Citrix XenServer.

До выхода этих двух продуктов именно вопрос гипервизора был ключевым при выборе поставщика технологий. Ведь система на базе настоящего гипервизора была только у VMware — ESX Server. У Microsoft тогда в портфеле решений был только Virtual Server, в котором управление виртуальными машинами (VM) реализовано с использованием хостовой ОС, что для промышленного применения было не очень хорошо. XenServer уже присутствовал на рынке, но принадлежал небольшой компании, веса которой было недостаточно для серьезного присутствия на корпоративном секторе рынка. Потом это решающее конкурентное преимущество у VMware исчезло: Microsoft сделала «настоящий» супервизор, а XenServer перешел в руки Citrix, более авторитетного ИТ-игрока. Супервизорных решений стало достаточно много, к тому же они быстро перешли в разряд бесплатных (прямое следствие возросшей конкуренции), и акцент борьбы за заказчика переместился на уровень выше — в сферу управления виртуальными средами.

Сейчас, примерно с 2008 года, доминирующее положение на рынке ПО виртуализации занимают решения, реализованные на базе «настоящего» гипервизора 1-го типа. Интересно, что википедия причисляет к типу 1 почти все имеющиеся на рынке продукты — VMware ESX Server, Microsoft Hyper-V, Citrix XenServer, Oracle VM, Sun Logical Domains Hypervisor и др. Однако более внимательное изучение этого вопроса говорит о том, что «настоящий» тип 1 — это только VMware ESX, а остальные — это, скорее, гипервизоры смешанного типа.

На Рис.463 проиллюстрирована разница в подходах в реализации гипервизоров у двух главных конкурентов — VMware ESX и Microsoft Hyper-V.

VMware — это классический вариант полностью автономного гипервизора, который относится к категории архитектуры монолитного ядра. Он содержит всё необходимое для работы VM и, по сути, является автономной ОС, включающей в том числе и драйверы для работы с оборудованием. Именно на эти моменты, как на архитектурные преимущества, делает акцент VMware, подчёркивая, что такой подход обеспечивает наиболее полную изоляцию VM, а, следовательно, и более высокую надёжность системы в целом. Отмечается также независимость гипервизора от какого-то ПО других разработчиков. В целом её ESX Server состоит из собственно гипервизора, среды исполнения (ESXi) и сервисной консоли на базе ядра Linux, при этом ESXi может использоваться автономно, но в этом случае у него ограничены возможности управления системой.

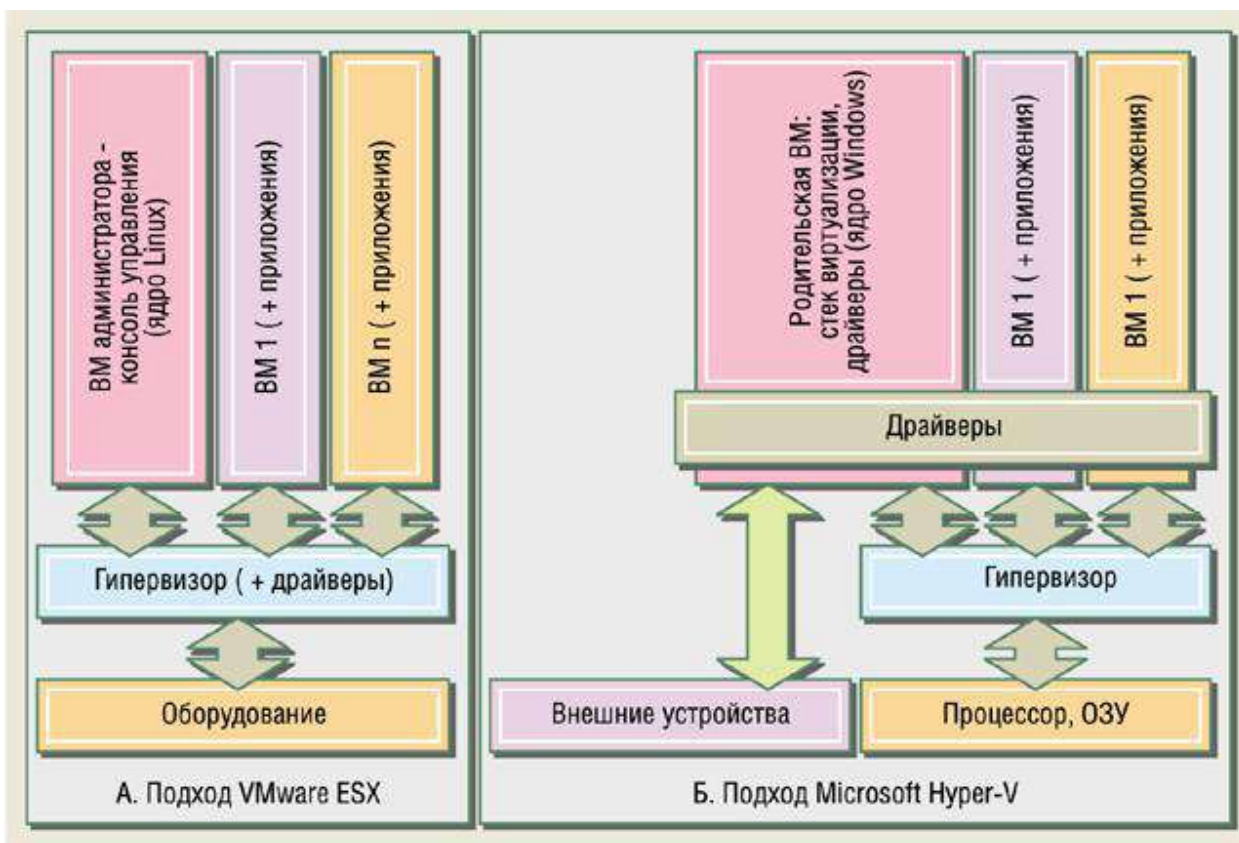


Рис.463 - Различные подходы к реализации гипервизоров.

Критики данного варианта указывают на сложность решения задачи «всё в одном» и, в частности, на то, что VMware пришлось реализовать собственную модель драйверов, доводка которой до нужного уровня является сложной многолетней задачей. Отмечается также, что вся архитектура ESX — закрытая, проприетарная (на это делает акцент даже Microsoft), хотя известно, что в основе ядра ESXi используются наработки Linux.

Microsoft придерживается иного подхода, который называет архитектурой микро-ядра, т. е. разделения функций по разным модулям и уровням. По сути — это как раз вариант гипервизора смешанного типа, когда сам гипервизор выполняет только функции управления памятью и процессором, а для взаимодействия с внешними устройствами и управления служит привилегированная родительская VM на базе ядра Windows (в случае Citrix — Linux<sup>1</sup>).

По мнению разработчиков Microsoft, такой подход более эффективен при высокой вычислительной нагрузке, когда используется только «тонкий» гипервизор, который в случае Microsoft занимает всего порядка 100 КБайт оперативной памяти (ESXi занимает существенно больше).

Для ускорения работы на уровне драйверов Microsoft использует два механизма взаимодействия прикладных VM с «родительской» машиной. В одном случае применяется специальный внутренний интерфейс VMBus, который позволяет общаться VM между собой напрямую. Он доступен для VM, реализованных на базе Windows, а также Xen, но только для тех разработчиков, с кем у Microsoft есть соответствующий уровень сотрудничества (когда-то это были Novell, Citrix, Red Hat). Для всех остальных ОС используется второй вариант полной эмуляции драйверов.

Как своё преимущество Microsoft также подчеркивает наличие в её Hyper-V проверенной модели драйверов, которая поддержана в обычном Windows Server.

<sup>1</sup> При многих технологических различиях архитектурные подходы Microsoft Hyper-V и Citrix XenServer в целом очень близки, что и определяет их более тесный альянс на рынке.

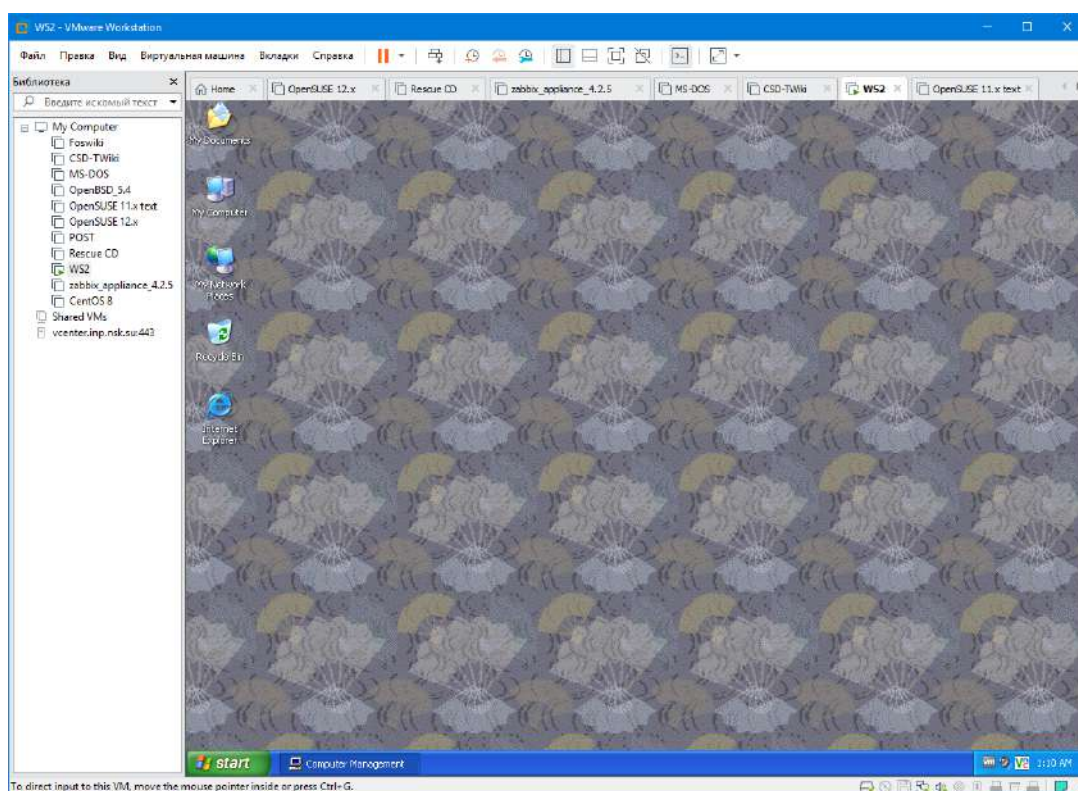


Выйдя на рынок гипервизоров намного позже компании VMware (весна 2008 года, Windows Server 2008), микрософтовский Hyper-V поначалу заметно уступал по основным характеристикам решению от VMware – по максимальным поддерживаемым объёмам ОЗУ хоста и гостевой системы, по возможному количеству виртуальных машин на хост, по максимальным размерам объектов файловой системы. Также в первой версии Hyper-V, например, не была реализована технология «живой миграции» (live migration) виртуальных машин, на основе которой VM можно переместить на другой хост без прерывания её работы. Но, с выпуском 3-й версии Hyper-V, Microsoft не только догнала лидера рынка, компанию VMware, но, по некоторым характеристикам даже сумела её превзойти.

На сегодня все основные игроки на рынке гипервизоров – VMware ESXi (VMware ESXi Free или VMware Free vSphere Hypervisor), Microsoft Hyper-V<sup>1</sup>, Citrix XenServer, Red Hat KVM – предоставляют свои базовые гипервизоры (Тип 1 и Тип 1+) бесплатно. Центр тяжести сместился в сторону управления виртуальными средами. Когда у вас на одном физическом хосте работает 2...5...10 виртуальных машин, без продвинутых средств управления виртуальной средой (как правило, это очень и очень недешёвые решения) вполне можно справиться. Но, когда речь пойдёт о десятках, сотнях, тысячах и даже сотнях тысяч «виртуалок» (суперкомпьютеры, «облачные» технологии, грид-компьютеры) – без адекватного инструментария для управления и «оркестрации» такими огромными и сложными системами обойтись будет невозможно.

## Под «капотом» у виртуальной машины

В качестве примера виртуальных машин, работающих под управлением гипервизора второго типа (хостовый) ниже приведены скриншоты нескольких разных ОС – Windows XP 32 бит (Рис.464), OpenSuSE 32 бит (Рис.465), CentOS 8 64 бит (Рис.466). Гипервизор – VMware Workstation, хостовая ОС – Windows 10 64 бит.



**Рис.464 – Виртуальная машина с ОС Windows XP, запущенная на VMware Workstation.**

<sup>1</sup> Здесь не следует путать возможные варианты гипервизора от Microsoft, на примере сервера Windows 2016: Windows Server 2016 со стандартной ролью Hyper-V и Hyper-V Server 2016. Бесплатен только Hyper-V Server 2016, работающий на «голом железе», Windows Server 2016 – в редакции Standart или Datacenter – не бесплатен.

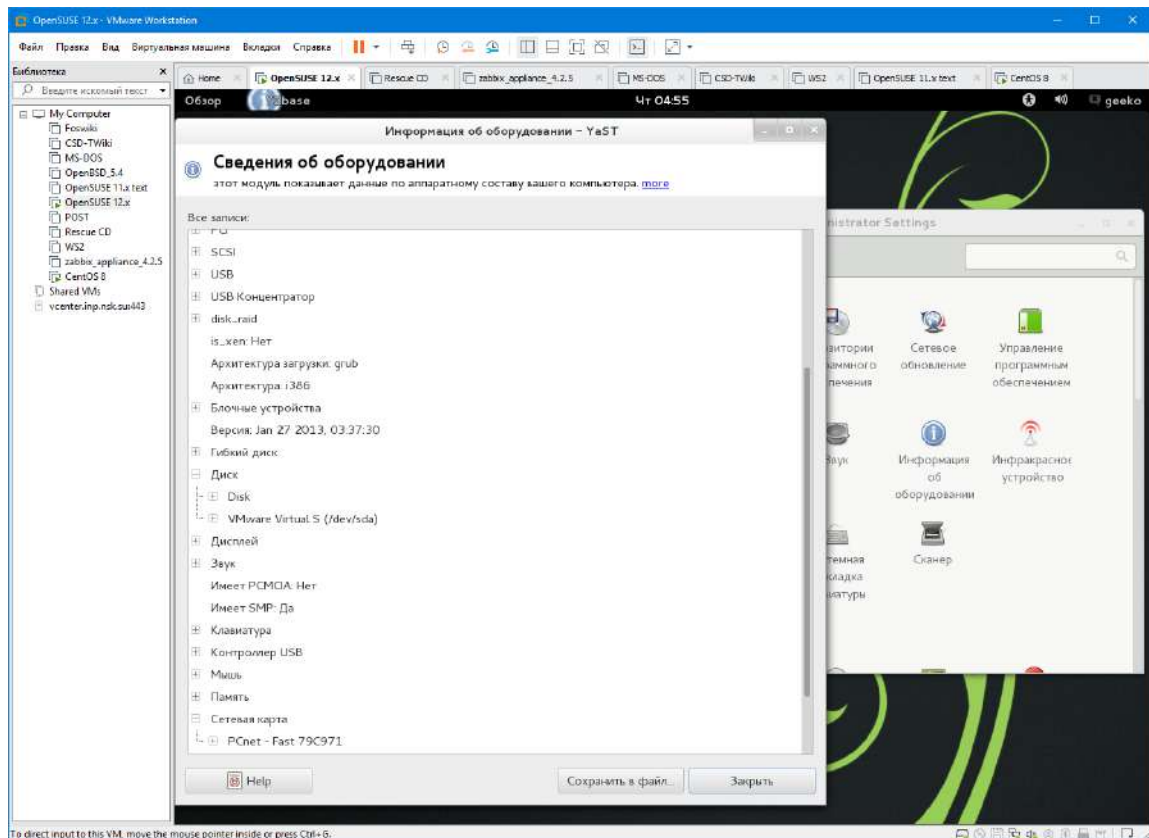


Рис.465 - Виртуальная машина с ОС OpenSUSE 12, запущенная на VMware Workstation.

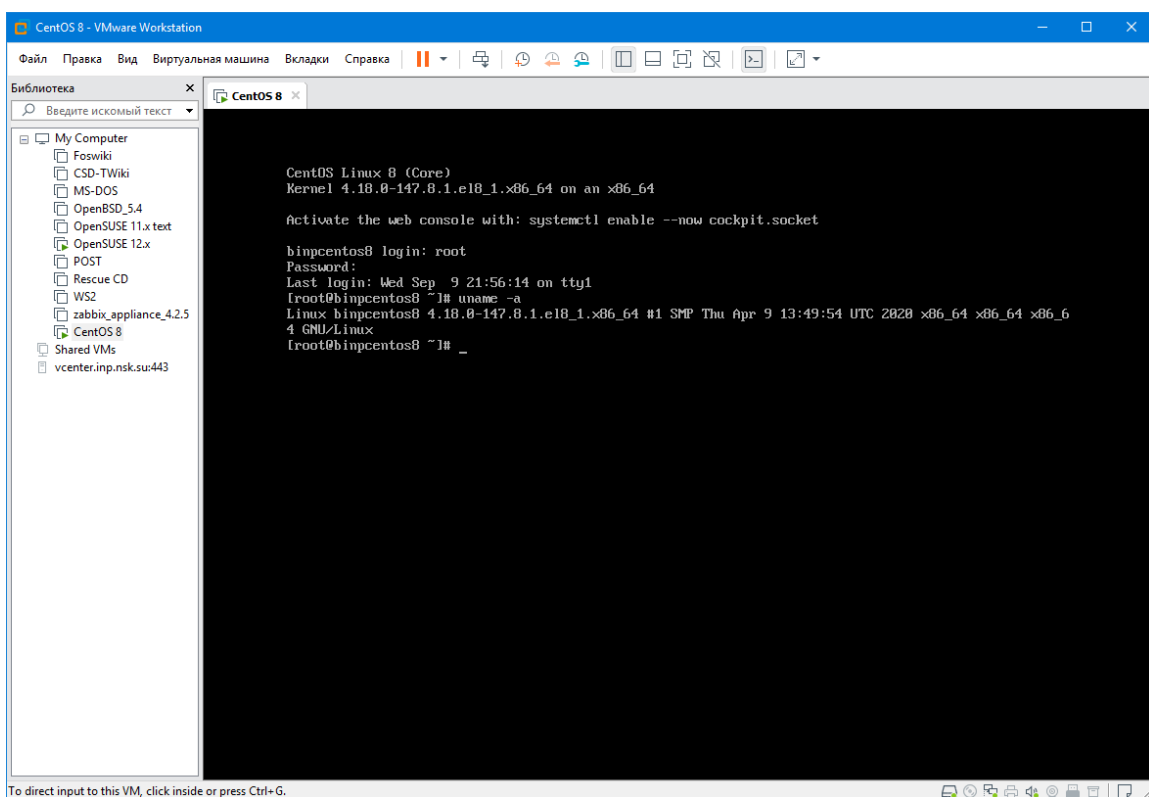


Рис.466 - Виртуальная машина с ОС CentOS 8, запущенная на VMware Workstation.

Глобальные настройки VM – объём доступного виртуальной машине ОЗУ, количество ядер CPU, объём жёсткого диска – для ОС выглядят абсолютно неотличимыми от варианта загрузки ОС на настоящей «железной» машине с такими параметрами. Но, если про-

инспектировать оборудование виртуальной машины более детально – например, с использованием «Диспетчера устройств» в Windows XP (Рис.467) – то можно заметить, что большая часть периферийных устройств имитируются/эмулируются средствами гипервизора. Честный, без имитации, в виртуальной машине только процессор (но не количество ядер/потоков в нём).

После установки ОС на VM настоятельно рекомендуется установить в гостевой ОС т.н. «гостевые инструменты» (конкретно для VMware они называются “VMware Tools”) – это набор утилит и драйверов, который улучшает взаимодействие виртуальной машины и хост-платформы, увеличивает производительность операционной системы в виртуальной машине, а также улучшает функции управления виртуальной машиной.

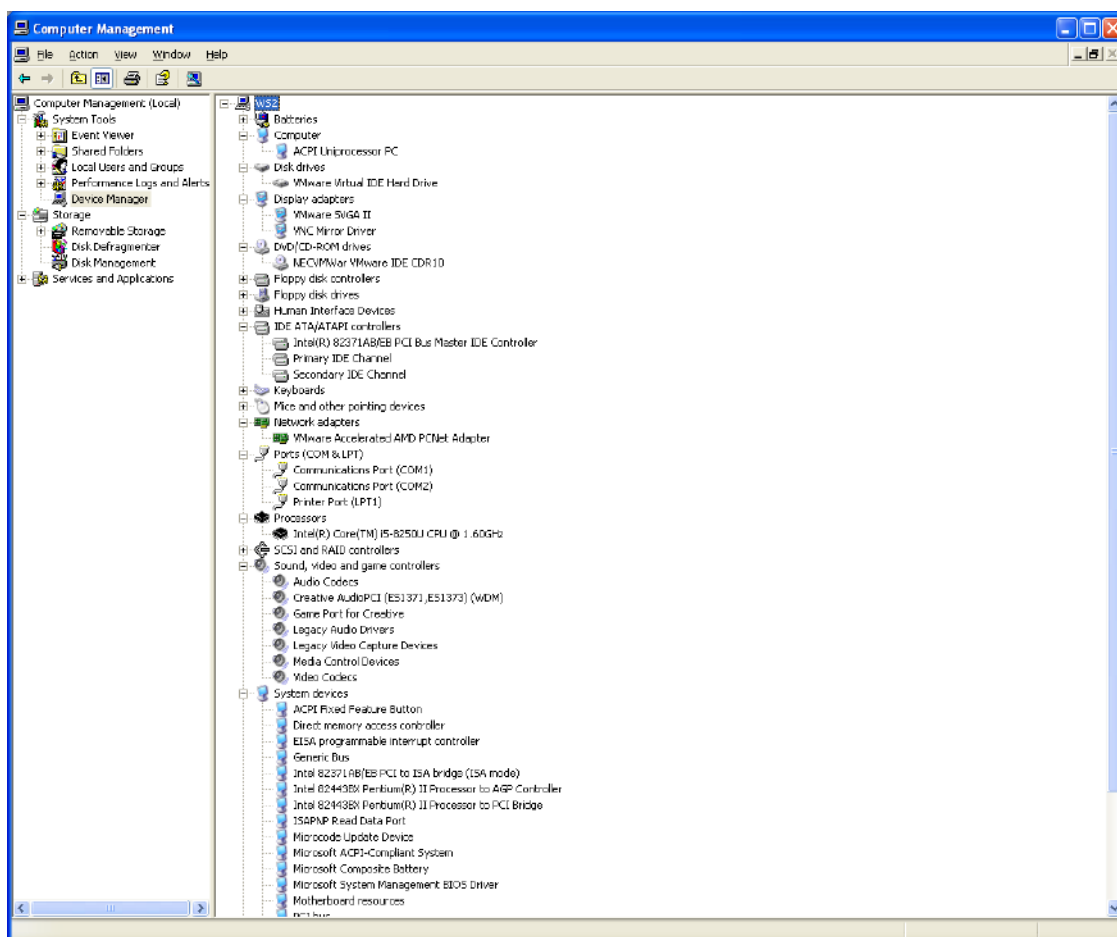


Рис.467 – Диспетчер устройств в виртуальной машине с ОС Windows XP.

Установленные в гостевую ОС VMware Tools предоставляют следующие возможности:

- Более новые и оптимизированные драйвера для гостевой ОС;
- Улучшенное быстродействие видеоадаптера и мыши;
- Расширенное управление памятью;
- Синхронизация времени.

Без установки VMware Tools гостевая система будет работать очень медленно, с медленным вводом-выводом с использованием эмуляции BIOS-а, с базовой VGA-графикой 800x600 256 цветов.

Как уже было отмечено выше, из оборудования в «Диспетчере устройств» на Рис.467 настоящий только процессор, в данном примере Intel® Core™ i5-8250U CPU @ 1.60GHz. А вот как выглядит относящаяся к процессору (тому же самому) информация в «Диспетчере



устройств» хост-системы (Рис.468): процессор – абсолютно тот же, что и в виртуальной машине, только ядер больше в восемь раз – четыре физических ядра + 4 HT.

Остальная периферия в VM или имитирует настоящее (но сильно устаревшее) оборудование, например, северный мост чипсета Intel 82443BX Pentium® II Processor, южный мост Intel® 82371AB/EB, звуковую карту Creative AudioPCI (ES1371/1373), последовательные и параллельный порты COM1/COM2/LPT или же эмулируют псевдоустройства, которых никогда в реальном «железе» не существовало: видеоадаптер “VMware SVGA II”, драйвер диска “VMware Virtual IDE Hard Drive”, сетевой адаптер “VMware Accelerated AMD PCNet Adapter” и др.

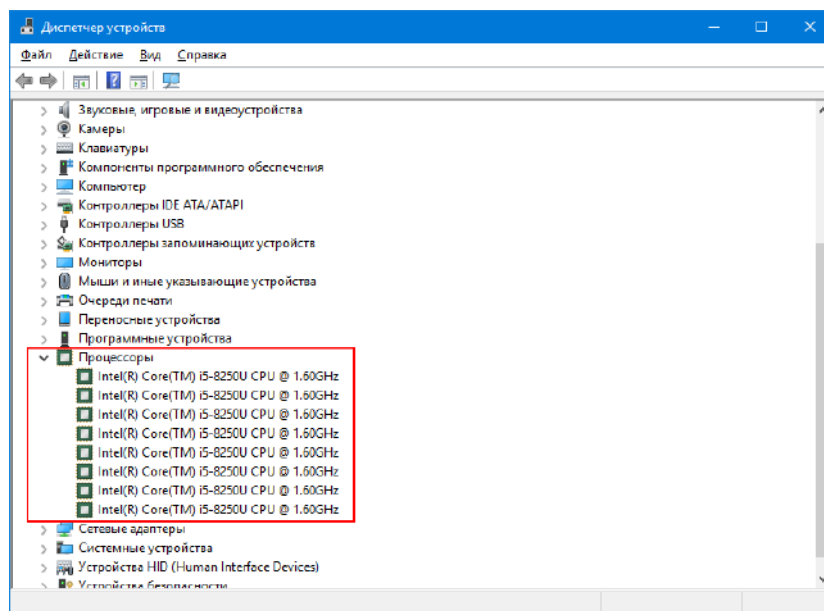


Рис.468 – CPU в хост-системе (Windows 10 64 бит).

Достаточно забавным в VM выглядит сочетание оборудования, которое никогда не могло сосуществовать в одной реальной машине, например, процессор образца 2018 года и последовательный порт из первых IBM PC 1981 года. Или очень знаменитый в своё время чипсет 82443BX 1998 года, который в принципе не мог работать с процессорами поколения Core.

## **Аппаратная поддержка виртуализации в x86 архитектурах**

Виртуализация долгое время была проблемой для архитектуры x86 из-за дефектов в архитектуре Intel 386, которые упорно в течение 20 лет переносились на новые центральные процессоры во имя обратной совместимости. Как было отмечено выше, набор инструкций IA-32 (x86) процессора Pentium содержит 18 чувствительных непривилегированных инструкций. Очевидно, что поддержка аппаратной виртуализации – до какого-то времени – не была среди главных ориентиров при разработке и развитии архитектуры x86. А ведь центральный процессор i386, являвшийся полной переработкой процессора i286, увидел свет спустя 10 лет после выхода статьи Попека и Голдберга, но разработчики архитектуры 386-го, видимо эту статью не читали или не придали ей должного значения.

Если коротко переформулировать один из критериев Попека-Голдберга («множество служебных/чувствительных инструкций является подмножеством привилегированных») – при попытке сделать в пользовательском режиме то, что вы не должны делать в этом режиме, оборудование должно вызвать системное прерывание (исключение). В отличие от IBM/370, обладающей этим свойством, у Intel 386 для 18-ти инструкций это не выполняется. При выполнении в пользовательском режиме будут проигнорированы или выполнены по-другому многие служебные инструкции 386-ой машины. Например, инструкция POPF заменяет регистр флагов, который изменяет бит, благодаря которому блокируются и

разблокируются прерывания. В пользовательском режиме этот бит просто не изменяется. Вследствие этого 386-е машины и их преемники не могли быть виртуализированы, следовательно, они не могли поддерживать гипервизор напрямую.

На самом деле ситуация была ещё хуже – вдобавок к проблемам с инструкциями, которые, выполняясь в пользовательском режиме, вызывали системные прерывания, были еще и инструкции, которые могли считывать конфиденциальное состояние, не вызывая системных прерываний. Например, на процессорах x86 выпуска до 2005 года программа путём чтения селектора своего кодового сегмента могла определять, в каком режиме она выполняется, в пользовательском или в режиме ядра. Операционная система, выполнявшая такое действие и позволявшая обнаруживать, что она в данный момент находится в пользовательском режиме, могла на основе этой информации принимать неправильные решения.

Эти проблемы были окончательно решены, когда в начале 2005 года Intel и AMD представили виртуализацию на своих центральных процессорах. На центральных процессорах Intel она называлась технологией виртуализации (Virtualization Technology (VT)), а на центральных процессорах AMD она называлась безопасной виртуальной машиной (Secure Virtual Machine (SVM)). Обе технологии были вдохновлены примером работы IBM VM/370, но при этом имеют некоторые отличия. Основной замысел заключался в создании контейнеров, в которых могли бы запускаться виртуальные машины. При запуске в контейнере гостевой операционной системы она продолжает работать в нем, пока ею не будет вызвано исключение и не будет осуществлено системное прерывание в гипервизоре, например, при выполнении инструкции ввода-вывода. Набор операций, вызывающих это системное прерывание, контролируется битовым массивом, устанавливаемым гипервизором. С таким расширением появилась возможность создания классической виртуальной машины, работающей по принципу вызова системного прерывания и эмуляции.

Тут можно отметить некоторое противоречия в датах некоторых событий – с одной стороны, было сказано, что x86-машины не могли быть виртуализированы вплоть до появления архитектурного расширения, представленного Intel/AMD в 2005 году, а с другой стороны известен факт, что VMware запустила свой первый гипервизор на x86 в 1999 году<sup>1</sup>.

Ответ заключается в том, что гипервизоры до 2005 года фактически не запускали настоящие гостевые операционные системы. Вместо этого они переписывали часть кода на лету, заменяя проблемные инструкции безопасной кодовой последовательностью, эмулирующей исходную инструкцию. Предположим, к примеру, что гостевая операционная система выполняет привилегированную инструкцию ввода-вывода или модифицирует один из привилегированных управляющих регистров центрального процессора (например, регистр CR3, содержащий указатель на каталог страниц). Важно, чтобы последствия выполнения таких инструкций ограничивались данной виртуальной машиной и не оказывали влияния на другие виртуальные машины или на сам гипервизор. Таким образом, небезопасные инструкции ввода/вывода заменялись системным прерыванием, которое после проверки безопасности выполняло эквивалентную инструкцию и возвращало результат. Поскольку происходила перезапись в памяти, этим трюком можно было воспользоваться для замены инструкций, являвшихся служебными, но не входивших в число привилегированных. Все остальные инструкции выполнялись обычным порядком. Эта технология известна как метод динамической бинарной трансляции (Рис.469).

---

<sup>1</sup> В 1990-е годы исследователи из Стэнфордского университета разработали новый гипервизор с названием **VMware**, основав компанию с таким же именем, на сегодняшний день лидера на рынке гипервизоров, средств виртуализации, управления виртуализацией и облачных технологий. VMware представила свое первое решение по виртуализации для x86 в 1999 году, это был гипервизор второго типа VMware Workstation.

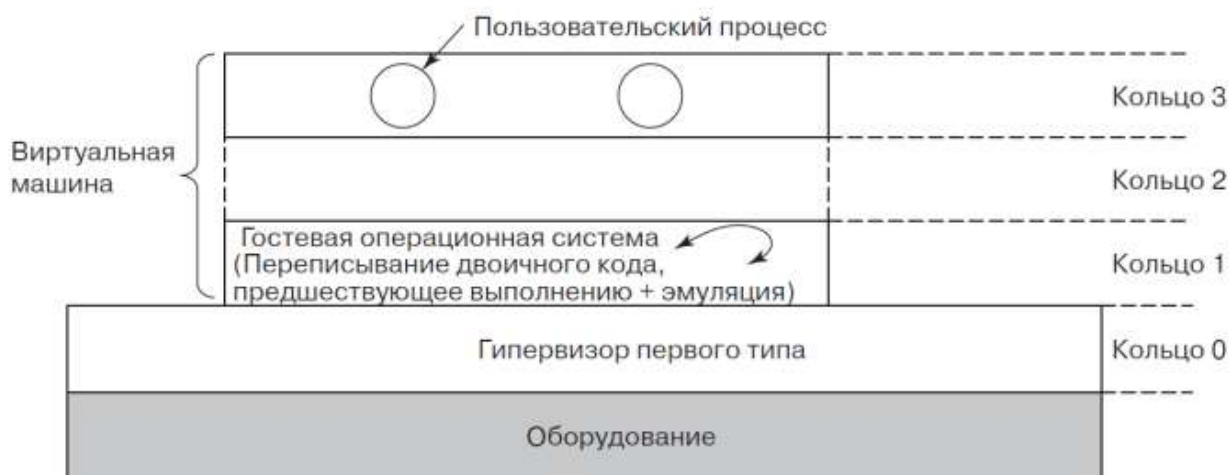


Рис.469 - Двоичный транслятор «на лету» переписывает инструкции гостевой операционной системы, работающей в кольце 1, в то время как гипервизор работает в кольце 0.

Ещё одна форма виртуализации, работающая без аппаратной поддержки, известна как *паравиртуализация*. Она сильно отличается от полной виртуализации, поскольку никогда даже не стремится представить виртуальную машину, которая бы выглядела как настоящее оборудование. Вместо этого она представляет машиноподобный программный интерфейс, который явно раскрывает факт наличия виртуальной среды. Например, он предлагает набор гипервызовов (hypercall), позволяющих гостю отправлять явные запросы к гипервизору (во многом это похоже на то, как системные вызовы предоставляют службы ядра приложениям). Гостевые системы используют гипервызовы для привилегированных служебных операций, таких как обновление таблиц страниц, но поскольку они делают это явным образом во взаимодействии с гипервизором, в целом система может получаться проще и быстрее.

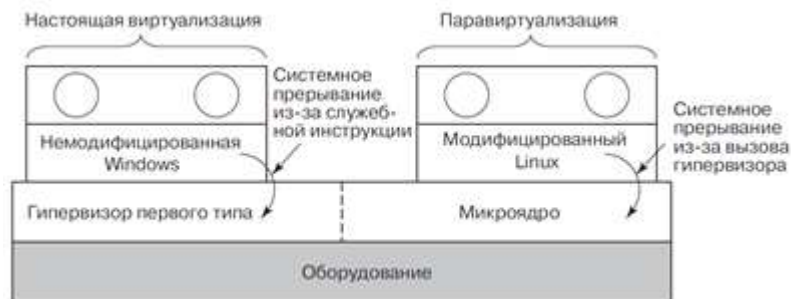


Рис.470 - Настоящая виртуализация и паравиртуализация

На Рис.470 показан гипервизор, разделенный на две части пунктирной линией. На самом деле на оборудовании выполняется только одна программа. Одна её часть (на рисунке слева) отвечает за интерпретацию перехваченных служебных инструкций, в данном случае от Windows. А другая часть (справа) просто выполняет гипервызовы (на этой части написано «Микроядро»). Если гипервизор предназначен для запуска только паравиртуализированных гостевых операционных систем, ему не нужно эмулировать служебные инструкции, и мы имеем дело, по сути, с настоящим микроядром, которое просто предоставляет базовые основные службы, такие как диспетчеризация процессов и управление MMU. Таким образом, граница между гипервизором первого типа и микроядром стирается.

Главный недостаток в технологии паравиртуализации – необходимость модификации ядра гостевой операционной системы (для этого исходные тексты ядра должны быть или открыты, или их должен модифицировать разработчик ОС). Метод паравиртуализации позволяет добиться более высокой производительности гостевой системы, чем метод динамической трансляции.

## Intel VT-x

Для аппаратной поддержки виртуализации компания Intel пошла достаточно прямым путём, попросту выпустив «минимально необходимую» заплатку к x86. Полное название «заплатки» – Intel Virtualization Technology for x86 (VT-x). Intel ввела в своих процессорах специальный «режим выполнения виртуальной машины» (Virtual Machine eXecution mode, VMX), предназначенный специально для виртуализационного ПО (Virtual Machine Manager, VMM), и определила для этого режима несколько ключевых «виртуализационных» инструкций, таких как, к примеру, «создать виртуальный компьютер» и «запустить виртуальный компьютер». Собственно, этот самый «виртуальный компьютер» в VT-x описывается специальной структурой под названием VMCS (Virtual Machine Control Structure) и по сути своей является небольшим участком физической оперативной памяти, хранящим минимально необходимые данные для запуска гостевой операционной системы, а также данные, необходимые для безопасного выхода из режима работы гостевой ОС, и некоторые настройки, относящиеся к управлению этой виртуальной машиной.

Программист практически вручную создаёт эту структуру, полностью описывающую необходимый ему виртуальный компьютер и его свойства, после чего «загружает» её в специальный аппаратный регистр «текущей виртуальной машины». Затем программист может «запустить» свежесозданную виртуальную машину специальной инструкцией.

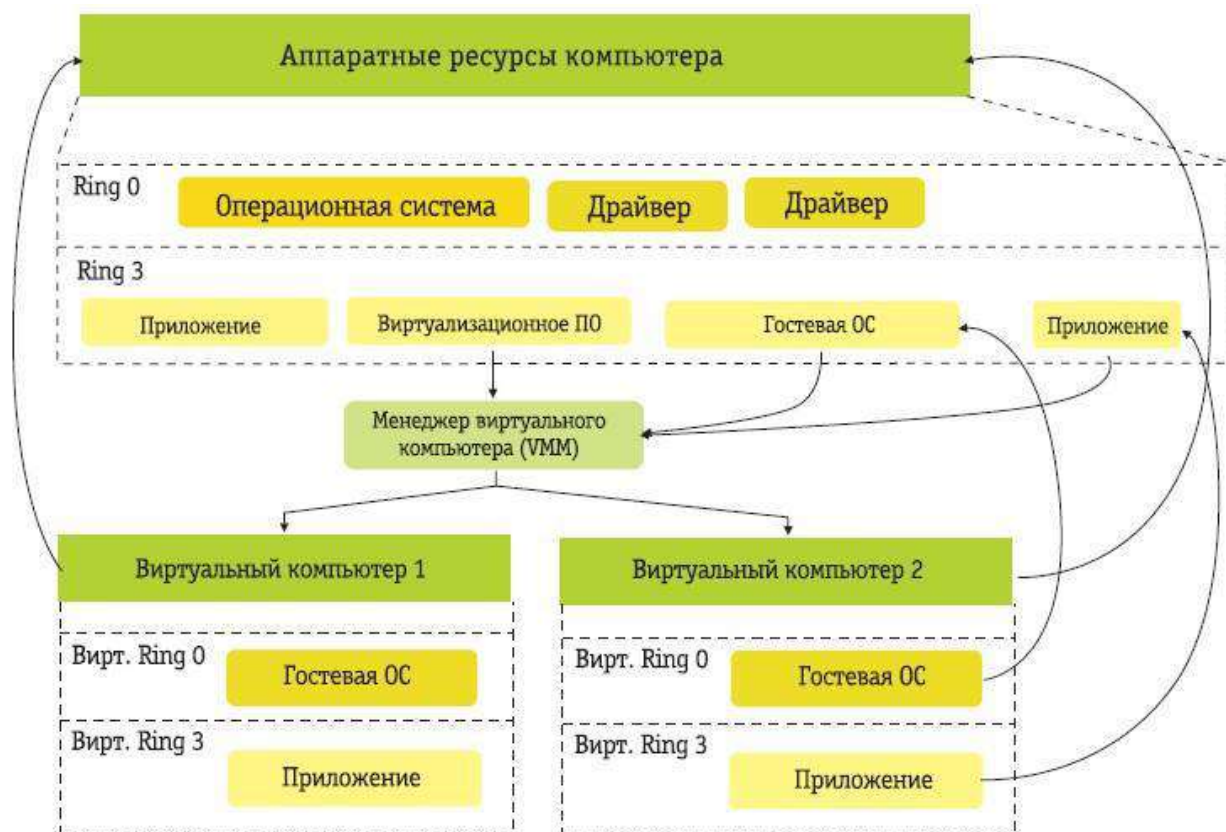


Рис.471 - Виртуализация с использованием VT-x.

Запущенная виртуальная машина работает на обычных аппаратных ресурсах компьютера (указанных VMM в описании виртуальной машины) и для запущенного на ней программного обеспечения практически ничем не отличается от обычной «физической» машины. Но в отличие от «обычного» режима, в эту виртуальную машину можно вставлять сколь угодно много «закладок», которые будут прерывать её исполнение, передавая управление обратно к VMM, который будет вручную имитировать то или иное событие, или выполнение той или иной инструкции. Какие именно события будут перебрасываться для

«ручной обработки» VMM, определяется только самим программистом, но их в любом случае даже больше, чем необходимо для реализации сколь угодно сложной виртуальной машины, вплоть до точной имитации совершенно другого процессора. К примеру, проблемная в обычном случае операция чтения/записи регистра CR3 в VT-х попросту приведёт к тому, что процессор на долю секунды приостановит гостевую ОС, вызовет VMM, который программно симитирует результат её выполнения (не выполняя настоящую инструкцию MOV to/from CR3 с аппаратным регистром, а подменив её чтением-записью обычной области оперативной памяти), после чего возобновит выполнение гостевой ОС, которая «подвоха» даже не ощутит.

При желании можно даже вручную «подкручивать» счётчик тактов процессора и показания системного таймера, чтобы у гостевой ОС не возникло ненужных сомнений, по поводу того, с чего это вдруг некоторые инструкции на данном «процессоре» выполняются так долго. Можно даже попробовать один-в-один симитировать AMD Athlon XP на каком-нибудь Intel Pentium 4 – так, что программное обеспечение «гостевой» операционной системы не будет и догадываться о подмене. Правда, если быть совсем точным, то поскольку Pentium 4 не поддерживает набор инструкций AMD 3Dnow!, а технология VT-х не позволяет перехватывать ошибки типа «неизвестная инструкция» (Invalid Opcode), то симитировать таким способом поддержку 3Dnow! на P4 невозможно. Но 3Dnow! всё равно практически не используется, а во всём остальном (скажем, во всём том, что рапортует о процессоре стандартная инструкция CPUID), имитируемый компьютер будет вести себя точно также, как AMD Athlon XP, так что подавляющее большинство ПО поддастся на «уловку».

Собственно, вот и вся технология виртуализации VT-х – мы просто переводим наш процессор в такой режим, когда он перехватывает некоторые (определенные нами) события и передаёт их в специальную программу – менеджер виртуальной машины. И никакой сложной (и медленной) бинарной трансляции!

Всего в VT-х было добавлено десять новых инструкций:

- VMXON и VMXOFF включают и выключают режим VMX.
- VMWRITE позволяет программисту записывать данные в структуру VMCS, описывающую виртуальную машину; VMREAD – аналогично позволяет читать данные из VMCS. Формат структуры VMCS официально неизвестен, и каким образом и что там, вообще говоря, сохраняется – одна Intel знает. Заметим, также, что сама структура VMCS относительно невелика по размерам (единицы килобайт) и не хранит в себе, к примеру, данных о виртуальной памяти, образующей физическую память «виртуального компьютера», – эти данные менеджер VMM поддерживает (загружает и сохраняет) для виртуальных машин самостоятельно.
- VMPTRLD позволяет выбрать текущую виртуальную машину (указатель на VMCS). VMPTRST, аналогично, позволяет сохранить указатель на текущую виртуальную машину.
- VMLAUNCH позволяет запустить выбранную виртуальную машину (описывающуюся ранее установленным указателем на корректную текущую VMCS).
- Исполнение кода работающей виртуальной машины прерывает либо наступление указанного в VMCS события (внешнего прерывания, попытки выполнить ту или иную инструкцию), либо выполнение инструкции VMCALL (если она разрешена в настройках VMCS).
- VMRESUME позволяет продолжить прерванное событием выполнение кода на виртуальной машине.
- VMCLEAR используется для инициализации пустой структуры VMCS и для перевода выбранной виртуальной машины в «остановленное» состояние (с сохранением данных VMCS).





ется в «гостевой режим» – и запущенный в этой гостевой машине код работает со свежесозданной виртуальной машиной в практически полностью автономном режиме, без какого-либо вмешательства VMM.

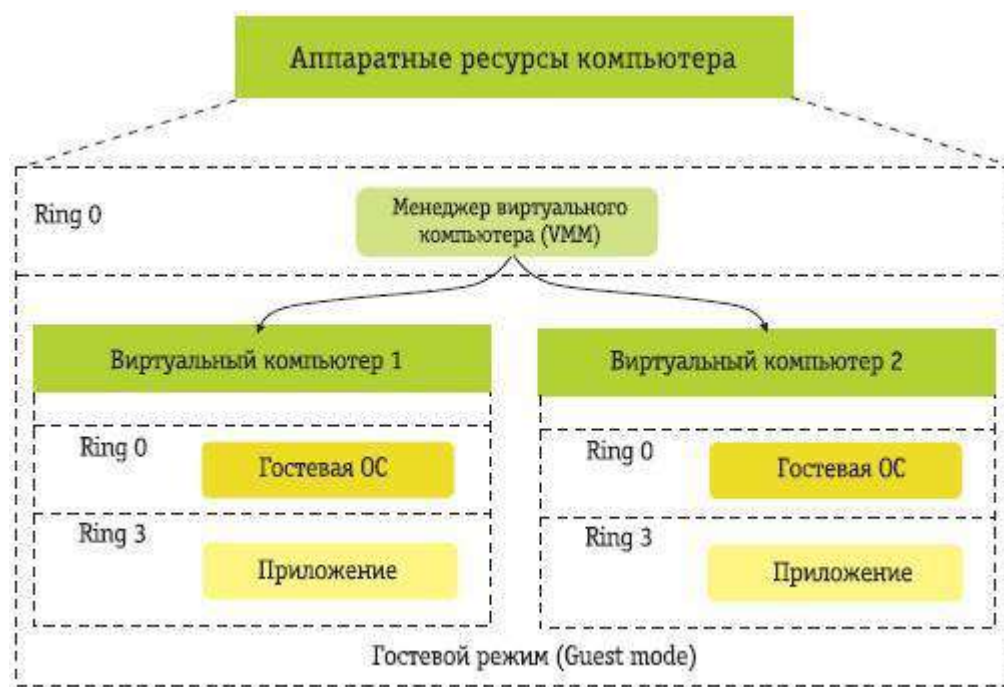


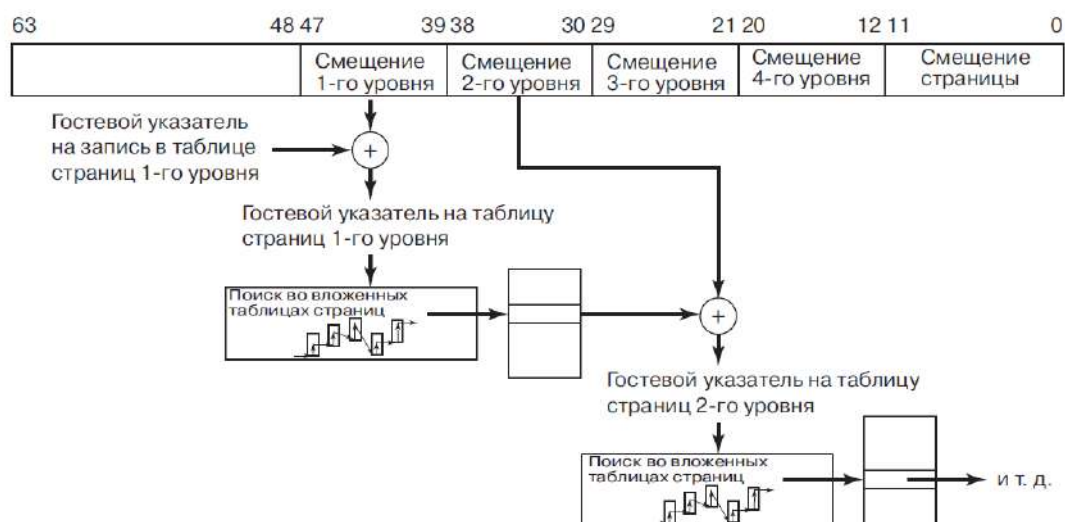
Рис.473 - Виртуализация с AMD-V.

AMD действительно виртуализует физические ресурсы, и, прежде всего, – оперативную память<sup>1</sup>. Интересно, что первое виртуализационное расширение в процессорах x86 компании Intel вообще не включало поддержку виртуализации памяти. Хотя процессоры с VT-расширениями позволили избавиться от многих узких мест, связанных с виртуализацией центрального процессора, копания в таблицах страниц обходилось дороже, чем когда-либо. Intel в следующей итерации поддержки виртуализации VT фактически воспроизвела в своём решении технологию работы с виртуализированной памятью от AMD.

Высокая стоимость обработки *теневого* таблиц (shadow page table) страниц заставила разработчиков микросхем добавить аппаратную поддержку для вложенных таблиц страниц (Рис.474). Термин вложенные таблицы страниц (*nested page tables*) использовался компанией AMD. Общее название технологии аппаратной поддержки виртуализированной памяти – **SLAT** (Second Level Address Translation), также известный как вложенный пейджинг, представляет собой технологию виртуализации с аппаратной поддержкой, которая позволяет избежать значительных накладных расходов, связанных с управляемыми программным обеспечением таблицами теневого страниц.

<sup>1</sup> В добавление к **гостевым виртуальным адресам** (guest virtual addresses) у нас теперь есть **гостевые физические адреса** (guest physical addresses) и, соответственно, **основные физические адреса** (host physical addresses), которые иногда называют **машинными физическими адресами** (machine physical addresses).





**Рис.474 - Расширенные (вложенные) таблицы страниц, включая обращение к каждому уровню гостевых таблиц страниц, подвергаются обходу при каждом обращении к гостевой физической странице**

AMD поддерживает SLAT с помощью технологии Rapid Virtualization Indexing (RVI) с момента появления своих процессоров третьего поколения Opteron (кодовое имя Barcelona, 2007 год). Реализация Intel SLAT, известной как таблица расширенных страниц (EPT), была представлена в микроархитектуре Nehalem (конец 2008 года), и присутствовала в некоторых (не во всех) процессорах Core i3, Core i5 и Core i7. Технологии похожи друг на друга и нацелены на сокращение основной части издержек путём полностью аппаратных дополнительной манипуляции с таблицами страниц, и всё это без каких-либо (очень затратных) системных прерываний.

По сути «физическая» виртуальная память, видимая гостевыми ОС в AMD-V – это просто виртуальная память второго уровня. Таблицы трансляции для которого контролирует, естественно, VMM. То есть если какая-нибудь программа, запущенная в этой гостевой ОС, скажем, обращается к памяти по адресу такому-то, то процессор, используя таблицы трансляции, контролируемые гостевой ОС, вначале преобразует этот адрес в гостевой «физический» адрес, который уже аппаратно, без участия VMM преобразуется в настоящий физический адрес.

Ещё одна интересная особенность AMD-V – это специальный защищенный режим запуска VMM. При желании на компьютере с процессором, поддерживающим данную технологию, можно сделать так, чтобы при старте компьютер на аппаратном уровне проверил цифровую подпись к VMM. Проще говоря, можно «защитить» в компьютер такую программу, что на нём принципиально невозможно будет запустить неподписанные доверенным источником модули VMM. А сами модули от этого «доверенного источника», в свою очередь могут проверить цифровую подпись у запускаемых ими операционных систем; операционные системы – проверять цифровую подпись у запускаемых приложений, и так далее, до сколь угодно высоких степеней контроля того, что именно запущено на компьютере.

Само по себе функционирование VMM у AMD очень напоминает функционирование VMM в технологии Intel: VMM с помощью специальных инструкций подготавливает специальные управляющие структуры, описывающие виртуальные компьютеры, «запускает» эти структуры на исполнение и перехватывает выбранные события, там происходящие, подменяя их «ручной» работой. Главное отличие – это объем и тип выполняемой ра-

боты. У AMD нет необходимости заниматься сложнейшим менеджментом памяти с подставными таблицами трансляции и синхронизацией этих таблиц с настоящими<sup>1</sup>; нет необходимости в перехвате связанных с этими таблицами трансляции событий. Нужно перехватывать и обрабатывать только некоторые внешние события (скажем, прерывание от таймера, чтобы, переключаясь между гостевыми операционными системами создавать иллюзию их одновременной работы; или прерывание от клавиатуры – переключаться между гостевыми ОС по нажатию определенной комбинации клавиш). Да и то, при желании в AMD-V можно попросту «напрямую» подключить выбранные устройства в гостевых операционных системах к физическим ресурсам (в VT-х, для сравнения, любое обращение гостевой ОС к портам ввода-вывода принудительно перехватывается модулем VMM).

Есть и другие особенности, позволяющие минимизировать количество ненужных переключений от гостевой ОС к VMM и обратно, переложив соответствующие проверки на аппаратное обеспечение. А если приходится всё же переключаться между гостевой ОС, VMM, и другой гостевой ОС – то, опять же по контрасту с VT-х, это переключение происходит крайне быстро, с очень ограниченной необходимостью в сохранении контекста и полным отсутствием необходимости сброса, к примеру, буфера TLB, который обычно при переключении между разными таблицами трансляции приходится полностью очищать. Естественно, что набор перехватываемых событий и возможности «эмуляции» всего и вся у AMD-V ничуть не меньше, чем у VT-х; однако настройка что перехватывать, а что нет – гораздо гибче (в VT-х многое перехватывается безусловно; в AMD-V можно отключить практически любой перехват).

Набор инструкций AMD-V столь же прост и изящен, как и само решение AMD:

- Команда VMRUN переключает выполнение на выбранную виртуальную машину.
- Из виртуальной машины управление возвращается либо по перехвату одного из указанного в настройках виртуальной машины событий, либо при вызове специальной инструкции VMSCALL (если последняя разрешена настройками).
- Информация о виртуальной машине хранится в специальной структуре данных VMCB (Virtual Machine Control Block) известного формата. VMM работает с данной структурой напрямую, вручную изменяя при необходимости соответствующие поля (в отличие от VT-х, где формат аналогичной структуры VMCS официально неизвестен и для работы с VMCS используются специальные инструкции VMREAD и VMWRITE). На всякий случай еще раз напомним, что сама эта структура относительно невелика и описывает только состояние процессора виртуальной машины, а виртуальную память и состояние виртуальных устройств этой машины VMM должна обслуживать самостоятельно.
- Самые необходимые операции по переключению контекста при переходах VMM к гостевой ОС и обратно выполняются полностью автоматически. Однако, чтобы не совершать лишних действий, сохраняется и загружается действительно только самое необходимое, и при необходимости каких-либо сложных действий или переключения на другую гостевую ОС, «дополнительные» операции сохранения состояния процессора в VMCB и обратной загрузки выполняются инструкциями VMLOAD и VMSAVE.
- Инструкция SKINIT позволяет начать загрузку процессора в «безопасном режиме», на аппаратном уровне гарантировав соответствие загрузчика (до 64 Кбайт кода) указанной в аппаратуре (в модуле TPM) цифровой подписи.

---

<sup>1</sup> Здесь имеется в виду сравнение с первой итерацией Intel VT-х, без появившейся позже в их процессорах поддержки EPT.

Всего пять инструкций против десяти в куда более сложном и менее функциональном VT-х. Правда, к пяти «базовым» для полного раскрытия возможностей AMD-V можно еще использовать «тактические» три инструкции, позволяющие дополнительно ускорить скорость работы AMD-V:

- STGI, CLGI – управляют схемой перехвата прерываний в AMD-V (включают-выключают «глобальный перехват прерываний»).
- INVLPGA – сбрасывает TLB, но не целиком, а только те записи TLB, которые относятся к конкретной гостевой ОС (или к VMM).



Рис.475 - Набор инструкций AMD-V.

Как и в случае с VT-х, для того, чтобы получить доступ к новым инструкциям, программному обеспечению нужно эти инструкции разблокировать (установить 12-й бит регистра EFER MSR, отныне известный как EFER.SVME). При желании в AMD-V можно отключить все её продвинутое функции, вплоть до отключения двойной трансляции виртуальных адресов, что позволяет максимально приблизить (хотя и не до конца) схемы использования к VT-х.

## Виртуализация ввода-вывода

Ещё одним компонентом в части аппаратной поддержки виртуализации является виртуализация ввода-вывода. Как мы могли видеть выше, в разделе «Под «капотом» у виртуальной машины», все периферийные устройства в виртуальных машинах эмулируют либо какое-то реальное аппаратное обеспечение (обычно довольно старое), либо представляются реально не существующими устройствами, наподобие «графического адаптера» “VMware SVGA II”. А, например, «стандартный» диск в виртуальной машине – это чаще всего обычный файл образа VM в файловой системе хоста.

Перехватить обращения к устройствам ввода-вывода в архитектуре x86 давно не проблема, но как быть с прямым доступом к памяти – DMA? Ведь стандартная периферия совершенно «не в курсе» о т.н. «гостевых физических адресах», bus master, захватив шину, имеет дело исключительно с реальными машинными физическими адресами.

Конечно же, весь ввод-вывод, в том числе и программирование адресных регистров DMA, делается под контролем VMM, но, в дополнение к этому, в архитектуре x86 «между» ОЗУ и устройствами ввода-вывода, располагается специальный аппаратный элемент – **ИОММУ** (input/output memory management unit) — блок управления памятью (MMU) для операций ввода-вывода (Рис.476). Так же как традиционный, процессорный блок управле-

ния памятью, который переводит виртуальные адреса, видимые процессором, в физические, этот блок занимается трансляцией виртуальных адресов, видимых аппаратным устройством, в физические адреса. Некоторые IOMMU также позволяют задавать различные ограничения операций ввода-вывода для защиты от неправильно работающих устройств или для изоляции, например, при использовании виртуализации (VT-d).

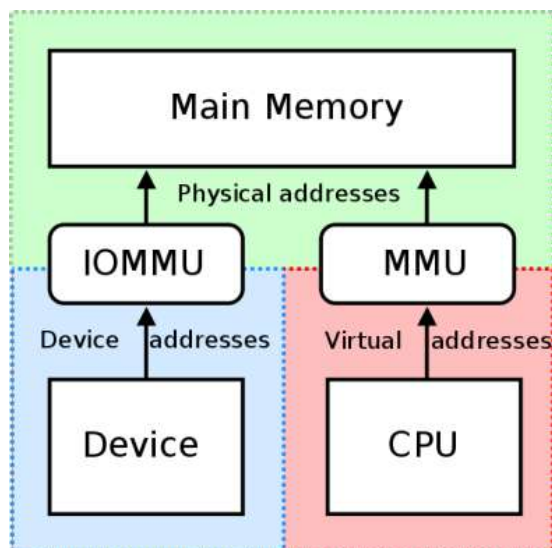


Рис.476 – Обычный MMU и I/O MMU в архитектуре физической машины.

При наличии IOMMU у аппаратуры имеется возможность проводить DMA-операции не только по физическим адресам, но и по логическим (виртуальным). Такая возможность упрощает устройства, которым больше не нужно заботиться о поддержке DMA по разрывному (с точки зрения физических адресов) региону памяти (поддержка такого DMA в драйвере влечет за собой заметные накладные расходы).

Недостатками использования IOMMU по сравнению с прямой физической адресацией памяти в DMA запросах являются:

- Некоторое ухудшение производительности из-за необходимости трансляции адресов и расходов на управление, например, проход по иерархии таблицы страниц.
- Дополнительное потребление памяти для хранения таблиц отображения. Может быть уменьшено при использовании основных таблиц трансляции адресов процессора.

IOMMU используется для прямой работы виртуализованных операционных систем с оборудованием основной системы. Наличие IOMMU для таких комбинаций позволяет повысить безопасность, производительность и упростить реализацию виртуальной машины. Примеры IOMMU для виртуализации на платформах x86/x86-64: Intel VT-d и AMD-Vi.

Как правило, обычные PC-совместимые компьютеры не имели IOMMU (кроме AGP GART). Вместо этого все основное периферийное оборудование где-то с конца 1990-х годов разрабатывалось с поддержкой chain DMA.

Тем не менее, широкое распространение гипервизоров виртуальных машин привело к включению поддержки IOMMU в «гостевые» ОС, такие, как Windows (API ядра Windows всегда поддерживал данную функцию, хотя обычно данная поддержка не реализовывалась).

Ниже, на Рис.477 приведён примеры работы с IOMMU в гипервизоре KVM (Linux):

```

[root@danilo ~]# virt-host-validate
QEMU: Checking for hardware virtualization : PASS
QEMU: Checking if device /dev/kvm exists : PASS
QEMU: Checking if device /dev/kvm is accessible : PASS
QEMU: Checking if device /dev/vhost-net exists : PASS
QEMU: Checking if device /dev/net/tun exists : PASS
QEMU: Checking for cgroup 'memory' controller support : PASS
QEMU: Checking for cgroup 'memory' controller mount-point : PASS
QEMU: Checking for cgroup 'cpu' controller support : PASS
QEMU: Checking for cgroup 'cpu' controller mount-point : PASS
QEMU: Checking for cgroup 'cpucacct' controller support : PASS
QEMU: Checking for cgroup 'cpucacct' controller mount-point : PASS
QEMU: Checking for cgroup 'cpuset' controller support : PASS
QEMU: Checking for cgroup 'cpuset' controller mount-point : PASS
QEMU: Checking for cgroup 'devices' controller support : PASS
QEMU: Checking for cgroup 'devices' controller mount-point : PASS
QEMU: Checking for cgroup 'blkio' controller support : PASS
QEMU: Checking for cgroup 'blkio' controller mount-point : PASS
QEMU: Checking for device assignment IOMMU support : PASS
QEMU: Checking if IOMMU is enabled by kernel : WARN (IOMMU appears to be disabled in kernel. Add intel_iommu=on to kernel cmdline arguments)
LXC: Checking for Linux >= 2.6.26 : PASS
LXC: Checking for namespace ipc : PASS
LXC: Checking for namespace mnt : PASS
LXC: Checking for namespace pid : PASS
LXC: Checking for namespace uts : PASS
LXC: Checking for namespace net : PASS
LXC: Checking for namespace user : PASS
LXC: Checking for cgroup 'memory' controller support : PASS
LXC: Checking for cgroup 'memory' controller mount-point : PASS
LXC: Checking for cgroup 'cpu' controller support : PASS
LXC: Checking for cgroup 'cpu' controller mount-point : PASS
LXC: Checking for cgroup 'cpucacct' controller support : PASS
LXC: Checking for cgroup 'cpucacct' controller mount-point : PASS
LXC: Checking for cgroup 'cpuset' controller support : PASS
LXC: Checking for cgroup 'cpuset' controller mount-point : PASS
LXC: Checking for cgroup 'devices' controller support : PASS
LXC: Checking for cgroup 'devices' controller mount-point : PASS
LXC: Checking for cgroup 'blkio' controller support : PASS

```

Рис.477 – Проверка настроек виртуализации хоста.

Команда `virt-host-validate` выдаёт настройки виртуализации KVM-хоста. Из Рис.477 видно, что в данном случае поддержка IOMMU не включена в ядре хостовой ОС (в данном примере CentOS 7). Для включения IOMMU необходимо добавить параметр в загрузчике, для Intel VT-d это “... `intel_iommu=on`”, обновить конфигурацию загрузчика и перезагрузить хост. Если всё в порядке и оборудование поддерживает IOMMU, после перезагрузки команда `virt-host-validate` должна выдать:

```

QEMU: Checking for device assignment IOMMU support : PASS
QEMU: Checking if IOMMU is enabled by kernel : PASS
LXC: Checking for Linux >= 2.6.26 : PASS

```

Рис.478 – Пример включённой поддержки IOMMU в KVM.

Как и обычные MMU-блоки, IOMMU использует таблицы страниц для отображения адреса памяти, которым желает воспользоваться устройство (адрес устройства), на физический адрес. В виртуальной среде гипервизор может настроить таблицы страниц таким образом, что устройство, выполняющее DMA, не сможет достигнуть к памяти, не принадлежащей виртуальной машине, от имени которой оно работает.

IOMMU-блоки при работе с устройством в виртуализированном мире предлагают различные преимущества. **Прямая передача устройства** (*device pass through*) позволяет физическому устройству быть напрямую назначенным конкретной виртуальной машине. В общем, было бы идеально, если бы адресное пространство устройства совпадало с гостевым физическим адресным пространством. Но это вряд ли может произойти, пока у вас не будет IOMMU. Блок управления памятью позволяет адресам пройти явное переназначение, тогда как и устройство, так и виртуальная машина будут пребывать в абсолютном неведении о производимом в аппаратных недрах преобразовании адресов.

**Изоляция устройств** (*device isolation*) гарантирует, что устройство, назначенное виртуальной машине, может напрямую обращаться к этой виртуальной машине, не подвергая опасности неприкосновенность других гостевых операционных систем. Иными словами, IOMMU предотвращает неконтролируемый DMA-трафик точно так же, как обычный MMU предотвращает неконтролируемое обращение к памяти из процессов. В обоих случаях обращение к неотображенным страницам влечет за собой сбой – обращение к несуществующей странице.



## Виртуализация прерываний

Для полноты картины нам нужно также виртуализировать прерывания, чтобы прерывания, выданные устройством, попадали на нужную виртуальную машину, имея правильный номер. В связи с этим современные блоки IOMMU поддерживают переназначение прерываний (*interrupt remapping*). Предположим, устройство отправило сообщение, оповещающее о прерывании с номером 1. Сначала это сообщение попадает в IOMMU, чтобы преобразоваться в новое сообщение, предназначенное для центрального процессора, на котором в данный момент работает виртуальная машина, да ещё и с номером вектора, ожидаемого этой виртуальной машиной (например, 66), для чего будет использована таблица переназначения прерываний.

И наконец, наличие IOMMU также помогает 32-разрядным устройствам с DMA иметь доступ к памяти, превышающей порог 4 Гбайт. Обычно такие устройства не могут обращаться к адресам выше 4 Гбайт, но IOMMU может легко переназначить нижние адреса устройства на любые адреса в физическом, более обширном адресном пространстве.

## Виртуализация ввода-вывода в отдельно взятом физическом устройстве

Непосредственное назначение устройства виртуальной машине плохо масштабируется. Таким способом, к примеру, с четырьмя физическими сетевыми адаптерами можно поддерживать не более четырёх виртуальных машин. Для восьми виртуальных машин нужны восемь физических сетевых карт, а при запуске 128 виртуальных машин ваш компьютер в сплетении сетевых кабелей вряд ли отыщется ☺.

Организовать программным способом совместное использование устройств несколькими гипервизорами возможно, но зачастую не оптимально, потому что уровень эмуляции (или домен устройства) вклинивается между оборудованием, драйверами и гостевыми операционными системами. Эмулируемое устройство зачастую не реализует все расширенные функции, поддерживаемые оборудованием. В идеале технология виртуализации должна была бы предложить полноценный эквивалент устройства для перехода без существенных издержек от отдельно взятого устройства к нескольким его экземплярам в каждой виртуальной машине. Виртуализация отдельно взятого устройства, вводящая каждую виртуальную машину в заблуждение, что у неё имеется исключительный доступ к её собственному устройству, существенно облегчается, если оборудование проделает эту виртуализацию за вас. На PCIe такое действие называется виртуализацией ввода-вывода в отдельно взятом физическом устройстве.

**Виртуализация ввода-вывода в отдельно взятом физическом устройстве** (*Single root I/O virtualization (SR-IOV)*) позволяет обойти привлечение гипервизора к обмену данными между драйвером и устройством. Устройства, поддерживающие SR-IOV, предоставляют независимое пространство памяти, прерывания и DMA-поток каждой использующей их виртуальной машине (Intel, 2011). Устройства показываются как несколько отдельных устройств, каждое из которых может быть сконфигурировано отдельной виртуальной машиной. Например, у каждого устройства будут отдельный регистр базового адреса и отдельное адресное пространство. Виртуальная машина отображает одну из этих областей памяти (используемую, к примеру, для конфигурации устройства) на свое адресное пространство.

SR-IOV предоставляет доступ к устройству в двух разновидностях: **физических функциях** (*Physical Functions (PF)*) и **виртуальных функциях** (*Virtual Functions (VF)*). Физические функции являются полноценными PCIe-функциями и позволяют устройству конфигурироваться любым способом, какой администратор сочтёт нужным. Гостевым операционным системам физические функции недоступны. Виртуальные функции представляют собой облегченные PCIe-функции, не предлагающие подобных вариантов конфигурирования. Они идеально подходят для виртуальных машин. В целом технология SR-IOV позволяет устройствам виртуализироваться в сотнях (или около того) виртуальных функций,

которые создают у виртуальных машин уверенность в том, что они являются единственными владельцами устройства. Например, если взять сетевой интерфейс с технологией SR-IOV, виртуальная машина может управлять своей виртуальной сетевой картой, как будто она является физической. К тому же у многих современных сетевых карт имеются отдельные (кольцевые) буферы для отправки и получения данных, выделенные этим виртуальным машинам. Например, сетевые карты Intel серии I350 имеют восемь очередей на отправку и восемь очередей на приём.

## Облачные технологии

В стремительном росте популярности облачных вычислений технологии виртуализации играют решающую роль. Существует множество вариантов облаков. Некоторые из них относятся к публичным и доступны любому, кто согласен платить за использование ресурсов, другие же являются закрытыми (частными) облаками организаций. Также разные облака предлагают разные услуги. Некоторые из них дают своим пользователям доступ к физическому оборудованию, но большинство виртуализируют свою среду. Некоторые предлагают просто машины, как виртуальные, так и физические, и больше ничего, а некоторые — готовое к использованию и способное к объединению весьма интересными способами программное обеспечение или платформы, облегчающие своим пользователям разработку новых служб. Поставщики облачных услуг обычно предлагают различные категории ресурсов, таких как «большие машины», «малые машины» и т. д.

При всех разговорах об облаках, похоже, мало кто с уверенностью может сказать, что они на самом деле собой представляют. Национальный институт стандартов и технологий (США) перечислил пять основных характеристик:

1. **Самообслуживание по требованию** (*On-demand self-service*). Пользователи должны иметь возможность получать ресурсы автоматически, без человеческого участия.
2. **Широкополосный доступ по сети** (*Broad network access*). Все эти ресурсы должны быть доступны по сети посредством стандартных механизмов, чтобы ими могли воспользоваться любые устройства.
3. **Объединение ресурсов в пул** (*Resource pooling*). Компьютерные ресурсы, принадлежащие поставщику, должны быть объединены в пул для обслуживания нескольких пользователей с возможностью динамического назначения и освобождения ресурсов. Пользователи обычно не знают точного местонахождения «своих» ресурсов и даже того, в какой стране они расположены.
4. **Быстродействующая эластичность** (*Rapid elasticity*). Должна быть предоставлена возможность эластичного получения и освобождения ресурсов, может быть, даже в автоматическом режиме, чтобы происходило незамедлительное масштабирование в соответствии с потребностями пользователей.
5. **Учтённые услуги** (*Measured service*). Поставщик должен вести учёт потребленных ресурсов тем способом, который соответствует типу заранее оговоренных облачных услуг.

Одно и то же облако может запускать разные операционные системы, возможно, на одном и том же оборудовании. В понятиях облака это называется инфраструктурой в качестве услуги (*Infrastructure As A Service (IAAS)*) в противоположность платформе в качестве услуги (*Platform As A Service (PAAS)*), когда предоставляется среда, включающая такие компоненты, как конкретная операционная система, база данных, веб-сервер и т. д., программного обеспечения в качестве услуги (*Software As A Service (SAAS)*), когда предоставляется доступ к конкретному программному продукту, например Microsoft Office 365 или Google Apps, и многим другим типам облаков в качестве услуг. Одним из примеров IAAS-



облака является Amazon EC2, основанный на гипервизоре Xen и насчитывающий сотни тысяч физических машин.

Технология виртуализации позволяет не только создавать IAAS-облака для одновременного запуска на одном и том же оборудовании нескольких разных операционных систем, но и управлять ими. О возможности выделения большего количества ресурсов, чем их фактически имеется, особенно в сочетании с дедупликацией, мы уже говорили. Теперь давайте рассмотрим другие проблемы управления: что, если машине понадобится обслуживание (или даже замена), а на ней запущено множество важных машин? Наверное, клиенты не обрадуются, если их системы утратят работоспособность по причине того, что поставщик облачных услуг хочет заменить дисковый привод.

Гипервизоры разделяют виртуальную машину и физическое оборудование. Иными словами, для виртуальной машины на самом деле неважно, на какой конкретно машине она работает. Следовательно, можно просто остановить все виртуальные машины и снова запустить их на совершенно новой машине. Но в результате этого получится весьма существенный простой. Задача состоит в том, чтобы переместить виртуальную машину с оборудования, нуждающегося в обслуживании, на новую машину вообще без остановки.

Немного более приемлемым подходом может быть не остановка виртуальной машины, а её приостановка. Во время паузы мы как можно быстрее копируем страницы памяти, используемые виртуальной машиной, на новое оборудование, проводим корректное конфигурирование в новом гипервизоре, а затем возобновляем работу виртуальной машины. Кроме памяти требуется перенести подключения к устройству хранения данных и к сети, но, если машины рядом, это можно сделать относительно быстро. Для начала можно использовать сетевую файловую систему (например, NFS), чтобы было все равно, на каком оборудовании работает ваша виртуальная машина, на серверной стойке 1 или 3. Также на новое место может быть просто перемещён IP-адрес. И всё-таки для такого переноса придется приостановить работу машины на вполне заметный период времени. Возможно, на это уйдет меньше времени, чем вы ожидали, но все же его невозможно будет не заметить.

Вместо этого современные решения виртуализации предлагают так называемую **живую миграцию** (*live migration*). Иными словами, перемещение виртуальной машины происходит без прекращения её работы. Например, в этих решениях используется технология, подобная **миграции памяти с предварительным копированием** (*pre-copy memory migration*). Это означает, что страницы памяти копируются в то время, когда исходная машина всё еще обрабатывает запросы. Большинство страниц памяти не подвергается интенсивной модификации, следовательно, их копирование возможно и безопасно. Следует помнить, что виртуальная машина при этом всё еще работает, поэтому страница может быть изменена после того, как уже скопирована. При изменении страниц памяти мы должны гарантировать копирование в место назначения самой последней (актуальной) их версии, поэтому такие страницы помечаются как измененные. Позже будут предприняты попытки повторного их копирования. Когда будет скопировано большинство страниц памяти, останется небольшое количество модифицированных (за время копирования) страниц. Делается повторная попытка скопировать эти оставшиеся страницы, по той же технологии – если страница не модифицируется, она просто копируется на целевую, если модифицируется – будет следующая итерация, пока в итоге не будут скопированы все страницы, после чего работа виртуальной машины возобновляется на новом месте. Хотя без паузы здесь всё же не обходится, она настолько коротка, что это обычно не оказывает никакого влияния на приложения (команда `ping` теряет обычно не более одного ответа, т.е., это будет всего лишь секундная пауза). Ситуация, когда простой (практически) не заметен, называется **бесшовной живой миграцией** (*seamless live migration*).

Отделение виртуальной машины и физического оборудования имеет дополнительные преимущества. В частности, уже говорилось, что машину можно приостановить. Это полезно само по себе. Если состояние приостановленной машины (например, состояние центрального процессора, страниц памяти и хранилища данных) сохраняется на диске, у

нас получается **мгновенный снимок** (*snapshot*) работающей машины. Если что-то в работе машине пойдёт не так, можно мгновенно откатиться к сохранённому snapshot-у и продолжить работу с момента, когда был сделан снимок.

Наиболее простой способ создать snapshot — это скопировать всё, включая всю файловую систему. Но копирование диска в несколько терабайт процесс небыстрый, даже если у вас сверхпроизводительный диск. К тому же приостанавливать работу машины на длительное время, пока проделывается все необходимое, нежелательно. Решение заключается в использовании технологий **копирования при записи** (*copy on write*), чтобы данные копировались только если они были модифицированы.

## ГЛАВА 10. АРХИТЕКТУРЫ ЭВМ, РАЗРАБОТАННЫХ В ИЯФ

В 1970-ые годы в Институте ядерной физики СО АН работало несколько крупных электрофизических установок – коллайдеры ВЭПП-2 и ВЭПП-4, и несколько довольно серьёзных установок масштабами поменьше. Управление большими и сложными устройствами абсолютно невозможно без компьютерного «сопровождения» – человек физически не в состоянии согласованно, точно и быстро реагировать на сигналы от многочисленных (тысяч и даже десятков тысяч) датчиков, производя в ответ управляющие воздействия на исполнительные устройства – управление большими системами возможно только при помощи управляющих компьютеров.

Вторая область, где без вычислительных машин совершенно невозможно обойтись – обработка данных. Основное предназначение коллайдеров, работающих на физику высоких энергий – генерация, ускорение встречных пучков заряженных частиц, с последующим их столкновением и регистрацией в детекторах результатов столкновения. Детекторы регистрируют огромное количество событий, из которых, путём реконструкции и специальной математической обработки, получаются (вычисляются) траектории движения продуктов столкновения, определяются типы частиц, участвовавших в процессе, вычисляются их характеристики (н-р, масса) и т.д.

Когда говорится об огромном количестве данных, которые «порождают» детекторы, в качестве примера можно привести объёмы данных, которые «производят» четыре эксперимента (детектора) на большом адронном коллайдере в ЦЕРН (LHC) – объёмы «сырых» данных с этих детекторов оцениваются в невообразимую цифру ~100 Петабайт<sup>1</sup> в год!

Совершенно очевидно, что без компьютеров работать с такими объёмами просто невозможно<sup>2</sup>.

В 1970-80-х в ИЯФе остро не хватало вычислительных мощностей (советская промышленность не могла обеспечить выпуск необходимого количества вычислительной техники) – не хватало как управляющих вычислительных машин, так и быстродействующих «арифмометров», предназначенных для сложной математической обработки данных с детекторов, моделирования и т.п.

Поэтому было принято решение о разработке собственных компьютеров, в результате чего был разработана микроЭВМ «Одрёнок»<sup>3</sup> – в основном использовавшаяся для управления, и арифметический (со)процессор АП-32 – для обработки данных. Интересно отметить, что основными разработчиками как аппаратного, так и программного обеспечения для «Одрёнка» и АП-32 были сотрудники, если можно так сказать, некомпьютерных подразделений, в основном шестой лаборатории.

### **МикроЭВМ Одрёнок**

С 1971 г. в Институте ядерной физики СО РАН для управления большими экспериментальными физическими установками широко использовались 24-разрядные ЭВМ серии Одра-1300 (производство ПНР), являвшиеся функциональными аналогами английских машин серии ICL-1900. Эти машины относились к классу универсальных ЭВМ, оснащались развитым программным обеспечением, а по своим функциональным возможностям (большое адресное пространство, средства поддержки мультипрограммирования, повышенная точность) превосходили ряд широко распространенных в то время мини-ЭВМ. В использовании ЭВМ Одра-1300 в ИЯФ СО АН СССР был накоплен большой опыт, создано обширное системное и прикладное программное обеспечение, имелся значительный контингент специалистов, знакомых с особенностями работы на них.

<sup>1</sup> Петабайт (ПБ, ПБайт) — единица измерения количества информации, равная  $10^{15}$  (квадриллион) байт.

<sup>2</sup> Бозон Хигса, об экспериментальном открытии которого было объявлено ЦЕРНом в июле 2012, был вычислен на основе отбора всего нескольких десятков событий, полученных в результате обработки сотен петабайт «сырых» данных.

<sup>3</sup> [Двадцатичетырёхразрядная ЭВМ в стандарте САМАС](#)

Развитие систем на базе ЭВМ серии Одра-1300 шло по пути децентрализации за счет введения периферийных автономных контроллеров крейтов КАМАК<sup>1</sup>, позволявших решить известную проблему сопряжения аппаратуры КАМАК (требующей высокой интерактивности взаимодействия) с универсальной ЭВМ. Однако ограниченные функциональные возможности микропроцессоров, традиционно применяемых в таких автономных контроллерах, препятствовали широкому использованию распространенных языков высокого уровня для их программирования, что, в свою очередь, ограничивало круг потенциальных пользователей.

Вследствие этого в 1981 г. было принято решение о разработке автономного контроллера крейта КАМАК, совместимого на уровне пользовательских программ с ЭВМ серии Одра-1300 и получившего условное название Одрёнок. По сути Одрёнок – это машина с системой команд Одра (ICL-1900), которая была расширена добавлением инструкций для работы с КАМАК-магистралью. За счёт такого расширения системы команд вызов команды обращения к КАМАК-модулю, сделанный на языке высокого уровня, транслировался в одну машинную инструкцию, т.е. взаимодействие с КАМАК-ом было очень эффективным и быстрым, и при написании управляющих программ можно было избежать программирования на языке ассемблера и тем более программирования в машинных кодах. Ещё одним удачным обстоятельством было то, что и Одрёнок и КАМАК имели одинаковую разрядность данных – 24 бита, т.е. для Одрёнка «общение» с КАМАК-ом было более простым и «естественным», чем, например, для шестнадцатиразрядных ЭВМ.

## Одрёнок

Одренок выполнен в КАМАК-модуле шириной 2М на 130 схемах большой и средней степени интеграции, включая процессорные секции K1804BC1<sup>2</sup>; он имеет встроенную память (64 К 24-разрядных слов) и реализует полную систему команд машин Одра-1300. Производительность на операциях с фиксированной запятой составляет около 300 000 операций в секунду, на операциях с плавающей запятой – около половины производительности ЭВМ Одра-1305 (старшей модели семейства).

Связь Одренка с ЭВМ верхнего уровня или файловой машиной сети осуществляется с помощью отдельного интерфейсного КАМАК-модуля, реализующего прямой доступ к памяти и регистрам контроллера и содержащего средства для дистанционной диагностики и отладки.

В семействе Одра-1300 последовательно проводится принцип виртуальной машины. Так, каждая пользовательская программа работает в собственном виртуальном адресном пространстве, а уровни операционной системы (ОС) и пользователя различаются аппаратно. В системе команд пользователя отсутствуют низкоуровневые операции ввода-вывода: вместо них применяются высокоуровневые команды обмена, исполнение которых влечет вызов соответствующих примитивов ОС (т.н. экстракоды ввода-вывода). Благодаря этому обстоятельству удалось достаточно просто, переписав машинно-зависимое ядро ОС, реализовать сетевую операционную систему, обеспечивающую возможность работы любых программ, ранее работавших в ЭВМ Одра. В Одренке реализована полная система команд пользовательского уровня. По составу привилегированных команд, которые обычно сильно зависят от структуры подсистемы ввода-вывода, Одренок значительно отличается от процессоров серии Одра-1300.

Одна из основных целей создания рассматриваемого контроллера – повышение эффективности взаимодействия с аппаратурой КАМАК. Для достижения этой цели исходная система команд расширена (за счет использования «резервных» кодов команд) введением операций для работы с магистралью КАМАК.

---

<sup>1</sup> **КАМАК** (CAMAC — *Computer Automated Measurement and Control*) — стандарт, определяющий организацию магистрально-модульной шины, предназначенной для связи измерительных устройств с цифровой аппаратурой обработки данных в системах сбора данных. Появился в 1970-х годах.

<sup>2</sup> Аналог bit-slice AMD2900.

## Особенности архитектуры

Основная адресуемая единица информации – 24-разрядное слово. В процессоре имеется восемь регистров общего назначения (РОН) и один 48-разрядный аккумулятор для чисел с плавающей запятой. Числа с плавающей запятой содержат 38-разрядную мантиссу в дополнительном коде и двоичный 9-разрядный порядок, что дает относительную погрешность  $1e^{-12}$  и динамический диапазон чисел  $10e^{+8}$ . Для кодирования символьной информации используется укороченный 6-разрядный байт. В одном слове помещается четыре таких байта. Программы работают с информацией во внутреннем представлении, при выводе на внешние устройства ОС программно выполняет соответствующую перекодировку в код КОИ-7.

Команда всегда занимает одно слово. Основные форматы команд показаны на Рис.479.

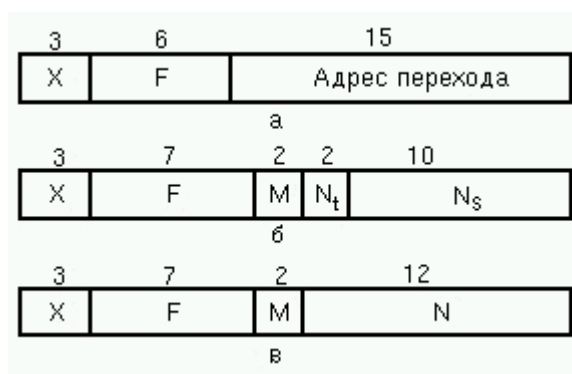


Рис.479 - Форматы команд

Поле X задает один из восьми РОН, участвующих в операции. Особенность данной ЭВМ - то, что РОН отображаются на первые восемь ячеек в адресном пространстве программы, поэтому в системе команд отсутствуют явные операции типа регистр - регистр. Они являются частным случаем операций типа регистр-память. Поле F задает код операций. Система команд включает около 120 различных типов операций. В процессоре реализованы четыре способа адресации операнда:

- прямая
- индексная со смещением
- непосредственная
- непосредственная с индексацией

При  $M = 0$  поле содержит адрес операнда (или сам операнд при непосредственной адресации). В противном случае поле X указывает на соответствующий РОН, содержимое которого будет использоваться в качестве, индексного регистра. Для получения исполнительного адреса 15 младших разрядов индексного регистра (РОН 1, 2 или 3) алгебраически суммируются с содержимым поля N. Таким образом, адресное пространство одной задачи не превышает 32 Кслов. В системе имеется возможность переключения на работу с расширенным диапазоном адресов (так называемый режим 22 АМ); в этом случае адресное пространство одной задачи составляет 4 Мслов.

В системе команд предусмотрены операции для работы с числами в формате с фиксированной запятой многократной точности, содержится обширная группа команд для работы с битовыми полями, в том числе, команды для манипуляций с 8-ми битными байтами. Система команд в значительной степени ортогональна относительно способов адресации и симметрична для основных арифметических и логических операций: есть как команды вида регистр:= регистр + память, так и команды типа память:= регистр + память.

Команды условного перехода (Рис.479, а) разделены на две группы:

- по содержимому указанного РОН
- по состоянию флагов процессора (7-флаг переполнения, С-флаг переноса).

В команде обращения к подпрограмме текущее слово состояния процессора сохраняется в РОН, указанном в команде. Имеются также четыре команды для организации циклов.

На (Рис.479, б) показан формат команд сдвига. Поле N, задает тип сдвига: циклический, арифметический, логический; поле N, определяет количество сдвигов. Возможны сдвиги одинарного или двойного слова на произвольное число разрядов. Существуют команды для нормализации числа и поиска старшего значащего бита в слове. В системе команд предусмотрены операции для преобразования из десятичного формата в двоичный и обратно, групповой пересылки блока данных из одной области памяти в другую, подсчета контрольной суммы, а также для работы с числами в формате с плавающей запятой.

Формат команд взаимодействия с магистралью КАМАК такой же как для обычных команд (Рис.479, в). В РОН содержится NAF<sup>1</sup>, а поле N(M) стандартным образом задает адрес слова данных, участвующего в обмене в КАМАК-цикле. После выполнения команды в старшие разряды РОН заносится состояние КАМАК-сигналов Q, X, I. Кроме команды выполнения одиночного NAF, предусмотрены команды группового обмена (до Q == 0) и сканирования крейта по субадресам и номерам станций, а также команда чтения состояния регистра LAM. Существенно, что все обращения к магистрали КАМАК выполняются с помощью одной машинной команды, т. е. все они являются неразрывными операциями. Это дает возможность использовать КАМАК-операции непосредственно в программе пользователя при работе в мультизадачном режиме. Эта ситуация радикально отличается от имеющей место в традиционных шестнадцатиразрядных мини-ЭВМ, к которым аппаратура КАМАК подключается как нестандартное периферийное оборудование. Для выполнения одиночного обмена с магистралью КАМАК в случае использования, например, распространенного контроллера СС-11, требуется выполнить несколько действий: записать в командный регистр контроллера крейта код функции F, занести старший байт данных в регистр старшего байта, переслать 16 младших разрядов данных. Поскольку при работе в мультизадачном режиме контроллер крейта - разделяемый ресурс, все эти действия должны выполняться внутри критического интервала при запрещенных прерываниях или на уровне ОС. Работа через ОС приводит к резкому увеличению времени, затрачиваемого на организацию одиночного обмена. Так, например, на выполнение одиночного КАМАК-цикла на ЭВМ VAX-11/780 (ОС VMS) и PDP-11/60 (ОС RSX-11M) затрачивается 1,8 и 2,2 мс соответственно. В нашем случае на вызов аналогичной подпрограммы на языке высокого уровня требуется 25 мкс, причем время в основном расходуется на передачу параметров в подпрограмму, собственно команда генерации КАМАК-цикла выполняется за 3 мкс. Максимальная скорость обмена с магистралью КАМАК в режиме передачи массива или сканирования составляет 300 000 24-разрядных слов в секунду.

## Системные средства

Для, реализации мультипрограммного режима работы в Одрёнке предусмотрены встроенные средства управления памятью и два режима работы процессора: СУПЕРВИЗОР и ЗАДАЧА. При всех обращениях к памяти в состоянии ЗАДАЧА к адресу памяти прибавляется смещение из регистра базы, как показано на Рис.480.

---

<sup>1</sup> NAF – полный адрес модуля в КАМАКе определяют N – номер позиции, A – (суб)адрес, F – функция (команда).

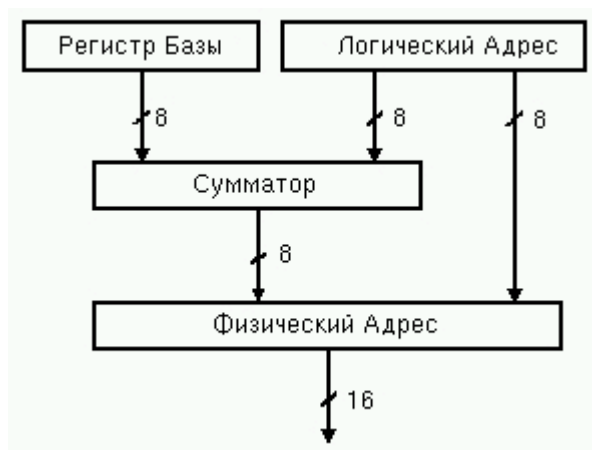


Рис.480 - Формирование физического адреса

При запуске задачи ОС заносит в регистр базы (с помощью соответствующей привилегированной команды) адрес начала ее размещения в физической памяти. Таким образом, все программы работают в собственном виртуальном адресном пространстве, а адресное пространство ОС недоступно пользовательским программам.

В системе возможны два типа прерываний: внешние от встроенного таймера и сигналов ЛАМ в крейте и внутренние, вызываемые попыткой выполнить нелегальную команду или одну из команд обращения к супервизору (так называемые экстракоды). В том и другом случае состояние процессора (включая состояние всех регистров) аппаратно сохраняется в теле задачи и управление передается в ОС по абсолютному адресу 28 для внутренних или по адресу 29 для внешних прерываний. Сохранение контекста в пространстве адресов задачи позволяет легко динамически перераспределять память, осуществлять запись образа задачи во внешнюю память с последующим восстановлением в произвольном месте физической памяти. Восстановление контекста и запуск задачи выполняются с помощью одной привилегированной команды. Для ускорения обработки внутренних прерываний команда, вызвавшая прерывание, распаковывается таким образом, чтобы при входе в ОС в регистрах содержалась вся необходимая для обработки экстракода информация.

Единственный источник внешних прерываний-сигналы запросов (ЛАМ) в крейте. Каждый источник, запроса может индивидуально маскироваться соответствующими разрядами в регистре маски ЛАМ. Этот регистр программно доступен, причем для запрета прерываний предусмотрена неразрывная команда чтения текущего состояния регистра маски с последующим его сбросом. В состоянии СУПЕРВИЗОР запрещены внешние прерывания и механизм преобразования адреса. В первой версии Одрёнка из-за ограниченной емкости ПЗУ микропрограмм не удалось реализовать на микропрограммном уровне полную систему команд виртуальной машины (не реализованы команды деления, а также преобразования типов). Упомянутый механизм экстракодов (программных прерываний) используется для программной интерпретации этих команд. Если в программе встречается такая команда, то происходит программное прерывание и управление передается в ОС, которая интерпретирует ее и возвращает результат в программу так, как если бы она была выполнена аппаратно. Это дало возможность обеспечить полную программную совместимость на уровне двоичных программ.

Единственный канал ввода-вывода Одрёнка - магистраль КАМАК. Кроме программного обмена, инициируемого машинными командами, имеется возможность прямого доступа к памяти, а также управления состоянием ЭВМ от магистрали крейта. Термин прямой доступ в данном случае довольно условный, так как на самом деле реализуется микропрограммой. Модуль связи с машиной верхнего уровня, файловой машиной сети (или модуль ПЗУ начального загрузчика) устанавливается в произвольную позицию крейта. Для выбора





величины, используемые микропрограммой: маски для выделения частей слова, начальные значения счетчиков циклов и т. д. На входную 24-разрядную шину арифметического устройства коммутируются выходы ПЗУ констант, регистра чтения из КАМАК-магистрали, выход ОЗУ и сигналы LAM. Выход устройства соединен с регистром NAF, регистром адреса памяти, регистром базы схемы управления памятью, входом памяти, а также (через шинные формирователи) подается на шины W КАМАК-магистрали.

Память содержит 24 схемы K565PY5 и управляется независимым автоматом, выполняющим, кроме функций чтения и записи, преобразование виртуальных адресов в физические, а также независимую от работы процессора "регенерацию динамической памяти.

Блок микропрограммного управления Рис.482

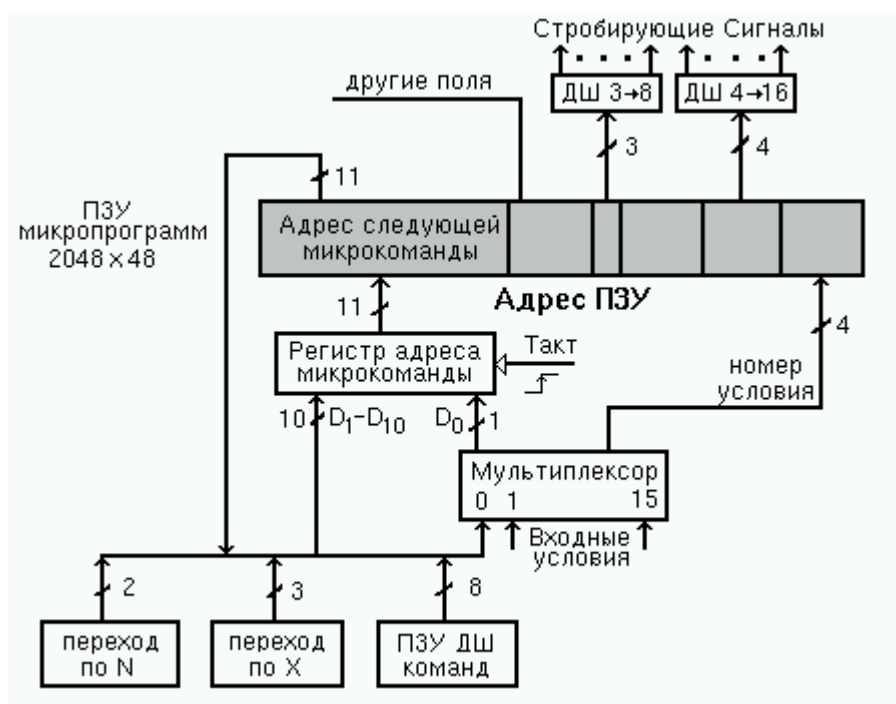


Рис.482 - Блок микропрограммного управления

сделан по простой схеме на базе регистра памяти текущего адреса микрокоманды и мультиплексора условий. Используется принудительная адресация — в каждой микрокоманде содержится полный адрес следующей. Переходы осуществляются в четный или нечетный адрес, в зависимости от состояния, выбранного (с помощью мультиплексора) в данный момент условия. Такая схема широко используется и в других разработках, произведенных в ИЯФе, она отличается простотой и малой вносимой задержкой. Микрокоманда имеет ширину 48 бит и содержит 14 полей (адрес следующей микрокоманды, номер условия перехода, адрес ПЗУ констант, код функции АЛУ и др.). Объем микропрограммы 512 слов. В новой версии Одrenка объем расширен до 2048 слов. Еще одна схема ПЗУ (K556PT5) используется в дешифраторе команд.

Блок микропрограммного управления работает с фиксированной длительностью такта 300 нс. Длительности тактов автоматов управления памятью и генератора КАМАК-цикла 100 нс. На выполнение большинства команд из подмножества, наиболее, часто используемых, требуется от 6 до 9 тактов.

## Заключение

В настоящее время<sup>1</sup> в ИЯФ СО АН СССР эксплуатируется около 90 микроЭВМ Од-рёнок. Они применяются в распределенных системах управления электрофизическими

<sup>1</sup> Данный материал основан на статье, опубликованной в журнале "Автометрия", 1986г. №4.

установками, в качестве инженерных рабочих станций, используемых при проектировании и наладке радиоэлектронной аппаратуры, а также в персональных станциях научного работника. Первоначально Одрёнок проектировался лишь как автономный контроллер крейта. Авторы не рассчитывали на его использование в качестве полноценной ЭВМ. Однако благодаря разработке сетевой ОС, позволившей сохранить все программное обеспечение, ранее применявшееся на ЭВМ Одра-1325, а также по причине довольно высокой производительности процессора микроЭВМ, вариант использования Одрёнка в научных рабочих станциях получил в ИЯФ широкое распространение.

В задачах с интенсивным использованием операций с плавающей запятой производительность Одрёнка составляет 0,3-0,5 от производительности ЭВМ Одра-1305. Мультипрограммные возможности позволяют работать на микроЭВМ нескольким пользователям одновременно. В типичной конфигурации к одной файловой машине (Одрёнок с подключенным к нему винчестерским накопителем на магнитных дисках) подсоединяется с помощью последовательных линий связи до 15 станций на базе Одрят, на каждой из которых, в свою очередь, может работать одновременно до четырех пользователей.

В настоящее время предполагается, что дальнейшее развитие систем управления экспериментальными установками в ИЯФ будет в значительной мере базироваться на применении микроЭВМ, программно совместимых с семейством Одра-1300. Несмотря на то что архитектура этих машин была разработана около 20 лет тому назад, она достаточно сбалансированна и современна.

## **Универсальный арифметический процессор АП-32**

Описание архитектуры, системы команд, характеристик арифметического процессора АП-32 приведено в этом документе с разрешения одного из авторов проекта, Андрея Геннадьевича Чертовских, сотрудника лаб.6-1 ИЯФ СО РАН. Используются материалы из проекта «Виртуальный компьютерный музей» ([Применение универсальных процессоров АП-20/АП-32 для обработки физической информации в Институте ядерной физики им. Г.И. Будкера](#), по [материалам международной конференции SORUCOM 2011](#) (12–16 сентября 2011 года)), и препринтов ИЯФ 89-175, 89-176, 90-3 и 90-64.

### **Введение**

Проблема обработки физической информации с детекторов элементарных частиц в физике высоких энергий имеет давнюю историю. В ИЯФ СО АН она стала особенно острой в 1982 — 1984г. с введением в эксплуатацию ускорителя ВЭПП-4 и магнитного детектора МД-1. В рамках программы обработки экспериментов был спроектирован спецпроцессор АП-20 с быстродействием около 10 MIPS. На его основе в 1984г. была создана высокопроизводительная система для обработки физической информации. Новое поколение детекторов с существенно улучшенными параметрами предъявляет более высокие требования и к системам обработки данных. В связи с этим в ИЯФ в 1985 г. была начата и в 1988г. завершена разработка арифметического процессора АП-32, который превосходит АП-20 по многим важным техническим характеристикам (объёму памяти, разрядности и т. д.).

АП-32 — универсальный 32-разрядный высокопроизводительный (10 MIPS) арифметический процессор с собственным блоком управления и оригинальной системой команд. Построен на отечественной элементной базе и может быть отнесён к классу супермини-ЭВМ. В АП-32 аппаратно реализованы операции сложения, умножения, деления 32-разрядных операндов в форме чисел с плавающей запятой, а также операции вычисления элементарных функций [ $\ln(x)$ ,  $\text{EXP}(x)$ ,  $\text{SIN}(x)$ ,  $\text{ARCTAN}(x)$ ,  $\text{SQRT}(x)$ ], что позволяет эффективно использовать его и для научно-технических расчетов.

Хотя АП-32 — автономный процессор, однако в качестве основного канала ввода-вывода в нём используются серийные ЭВМ семейства «Электроника» (Э-60.1, Э-100/25) и СМ. Сопряжение АП-32 с ними (Рис.483) осуществляется с помощью интерфейсов, специально разработанных для этой цели. Такая конфигурация аппаратных средств позволяет

осуществлять ввод в АП-32 обрабатываемой информации и вывод результатов из него путём обмена данными между АП-32 и ЭВМ, играющей роль машины «накачки» (ЭВМН). При этом большая вычислительная мощность АП-32 сочетается с развитым каналом ввода-вывода и удобной операционной системой.

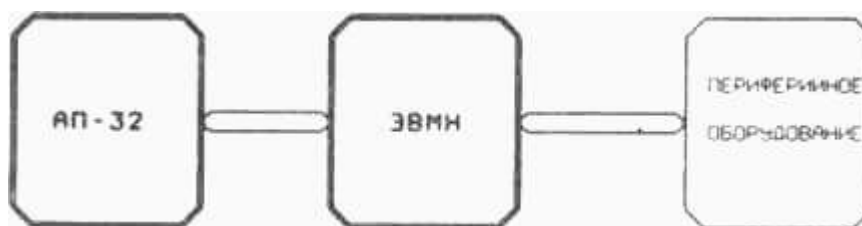


Рис.483 - Вычислительный комплекс АП — ЭВМН.

Программа обработки данных в системе команд АП-32 вводится в процессор из ЭВМН, где реализованы программные средства для её подготовки и отладки. Управление исполнением программы осуществляется самим АП-32 автономно.

В основу принципов протокола обмена, аппаратурной и программной поддержки положен подход, позволяющий рассматривать комплекс АП-ЭВМН как единую быстродействующую универсальную ЭВМ. Основные принципы следующие:

1. Процедуры разработки программ АП-32 принципиально не отличаются от аналогичных процедур на ЭВМН. Для этого используются кросс-средства ЭВМН под управлением её ОС (RT-11 или TSX-Plus).
2. Для пользователя процесс в АП-32 не отличается от процесса в ЭВМН: из проблемной программы АП-32 обеспечивается доступ к системным средствам и периферийным устройствам ЭВМН.

Представляется, что дальнейшее развитие вычислительного комплекса АП-ЭВМН может осуществляться в следующих направлениях:

- увеличение производительности (до нескольких десятков млн. оп./с) за счёт подключения к ЭВМН дополнительных процессоров АП-32 (многопроцессорный комплекс);
- применение более мощных современных моделей ЭВМ в качестве ЭВМН;
- перевод АП-32 на более совершенную элементную базу нового поколения с обеспечением программной совместимости с прототипом.

## Архитектура АП-32

### Основные принципы

Принципы, положенные в основу архитектуры процессора АП-32, имеют много общего с идеологией ЭВМ с сокращенной системой команд (RISC):

- исполнение команды за один командный цикл;
- отказ от микропрограммного управления, что, безусловно, усложняет проектирование процессора, но позволяет сократить длительность командного цикла;
- два типа памяти (данных и команд) позволяют совместить доступ к ним обоим в одном командном цикле (133 нс) (гарвардская архитектура);
- большой набор регистров общего назначения.

В качестве основного схемотехнического метода проектирования был принят модульный принцип структурной организации вычислительных систем: каждая арифметико-логическая операция осуществляется системой отдельных модулей (сопроцессоров), логически и электрически связанных между собой и центральным блоком управления шинами данных, команд, флагов и т. д.

Такой метод обладает рядом преимуществ:

- достаточно просто реализуются возможности одновременного исполнения вычислительных операций, поскольку почти каждая из них производится в отдельном модуле с автономным узлом управления;
- при наличии большого набора модулей возможна адаптация (в определённых пределах) вычислительных ресурсов системы к задачам пользователя;
- возможна модернизация модулей, как средство борьбы с моральным старением, в сторону расширения их функциональных возможностей без радикального пересмотра общей структуры процессора в целом.

Недостатком такого метода является то, что в данном случае требуется большее количество интегральных микросхем, т.к., наряду с упрощением центрального блока управления во всех модулях оказывается неизбежным дублирование многих функций и однотипных узлов. Однако, это увеличение аппаратуры происходит не столь интенсивно и не требует, на наш взгляд, таких усилий, которые были бы необходимы в случае иного подхода.

## Структурная схема АП-32

На структурной схеме (Рис.484) показаны основные Модули АП-32:

- БУ — блок управления;
- ПЦЧ — процессор целых чисел;
- ОЗУД — память данных;
- ОЗУП — память программ;
- УМН — умножитель чисел в форме с плавающей запятой;
- СУМ — сумматор чисел в форме с плавающей запятой;
- СПЭФ — сопроцессор элементарных функций;
- ИНТ — интерфейс ЭВМ «Электроника».

Со стороны ЭВМН АП-32 можно рассматривать как мощный сопроцессор с собственным блоком управления и памятью. И наоборот, со стороны АП-32, ЭВМН также выглядит как сопроцессор АП-32 (процессор канала ввода-вывода).

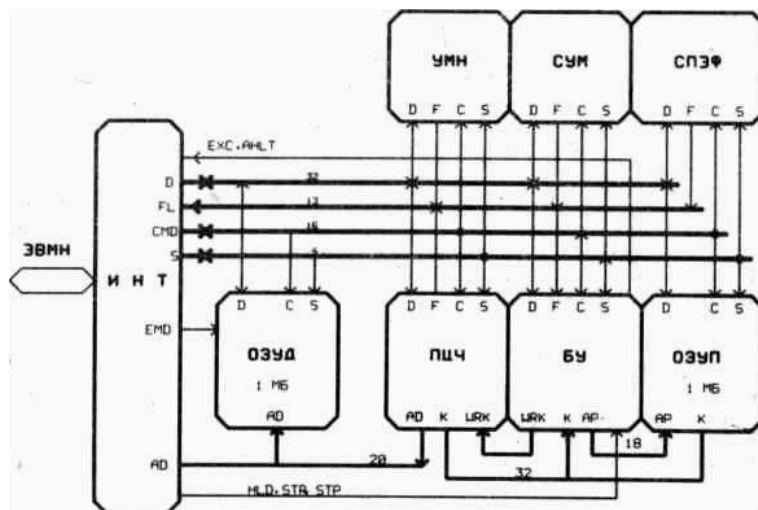


Рис.484 - Структурная схема АП-32.

Взаимодействие модулей АП-32 друг с другом, а также каждого из них с ОЗУД, обеспечивается с помощью системы шин (таблица на Рис.485) магистрали АП-32.

| № | Назначение                  | Обозначение                   | Кол-во | Функция                                                       |
|---|-----------------------------|-------------------------------|--------|---------------------------------------------------------------|
| 1 | Шина данных                 | D                             | 32     | Передача данных                                               |
| 2 | Шина команд ПЦЧ, БУ         | K                             | 32     | Передача команд для ПЦЧ, БУ                                   |
| 3 | Шина адреса памяти данных   | AD                            | 20     | Передача адреса ОЗУД от ПЦЧ или ИНТ                           |
| 4 | Шина адреса памяти программ | AP                            | 19     | Передача от БУ адреса ОЗУП                                    |
| 5 | Шина команд сопроцессоров   | CMD                           | 16     | Передача команд для УМН, СУМ, СПЗФ от БУ или ИНТ              |
| 6 | Шина флагов                 | FL                            | 13     | Передача признаков результата от ПЦЧ и сопроцессоров БУ и ИНТ |
| 7 | Шина стробирующих сигналов  | S                             | 4      | Синхросигналы командного цикла                                |
| 8 | Шина управления             | EMD, EXC, HLD, STR, STP, AHLT | 6      | Установка интерфейсом режимов работы АП-32                    |

**Рис.485 – Шины АП-32.**

С помощью сигналов управления (HLD, STR, STP) ЭВМН через интерфейс может задать несколько режимов работы АП-32. Основные из них указаны в таблице на Рис.486:

| № | Обозначение | Режим                                                                                                                                                                 |
|---|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | «АВТ»       | Пуск программы АП-32 по адресу, содержащемуся в счётчике команд БУ. Управление магистралью и модулями передано БУ, который вырабатывает сигнал WRK = 0 («подключён»). |
| 2 | «ЗАХВ»      | Останов программы АП-32. БУ отключается от магистрали и вырабатывает сигнал WRK=1 («отключён»). Управление магистралью и модулями передано ИНТ.                       |

**Рис.486 – Режимы работы АП-32.**

Установка указанных режимов осуществляется загрузкой ЭВМН соответствующего управляющего слова в регистр состояний и управления интерфейса.

В режиме «АВТ» работа модулей АП-32 полностью определяется содержимым памяти программ. Извлекаемые из ОЗУП команды (32 разряда) по шинам К поступают в БУ и ПЦЧ, где и запоминаются в регистрах команд. При этом старшая часть слова команды через БУ передается на шины CMD (для сопроцессоров УМН, СУМ, СПЭФ). Одновременно с выборкой ОЗУП в БУ инициализируется генератор командного цикла, вырабатывающий синхросигналы на шинах S.

Все модули, имеющие доступ к шинам К или CMD, дешифрируют команды одновременно. Они исполняются только в тех модулях, которым предназначены. Так, например, в БУ исполняются только команды управления программой (JUMP, CALL и т. п.), а в ПЦЧ — арифметико-логические операции над операндами в форме чисел с фиксированной запятой.

Частным случаем указанного режима является режим пошагового исполнения команд («ИНИЦ»), Установка режима и пуск БУ на одну команду производится ЭВМН. В этом случае управление модулями передается БУ на промежуток времени, равный командному циклу. Исполнив текущую команду, БУ «отвечает» интерфейсу сигналом EXC.

Режим «ЗАХВ» используется как в профилактических целях (настройка, тестирование модулей), так и при вводе данных и выводе результатов расчета из АП-32 в ЭВМН. В

этом режиме БУ блокируется и его роль исполняет интерфейс ЭВМН. Поскольку регистр команд интерфейса подключен только к шине команд CMD, то ЭВМН может управлять теми модулями (или регистрами модулей), которые имеют доступ к этой шине.

Команды обращения к ОЗУД, ОЗУП или другим модулям АП-32 в регистр команд записываются ЭВМН. С помощью регистра состояний и управления ею же инициализируется генератор синхросигналов (S) командного цикла интерфейса.

При обращении к ОЗУД адресная часть команды содержится в адресном регистре интерфейса, куда её предварительно записывает также ЭВМН. Обмен данными между ИНТ и ОЗУД осуществляется через регистр данных командами типа память-регистр. Этими же командами можно произвести обмен и между другими модулями и ИНТ. Однако в этом случае ОЗУД должна быть отключена от магистрали (сигналом EMD).

Обращение ЭВМН к регистру данных интерфейса выполняется как под управлением ЦП ЭВМН (программный канал), так и под управлением самого ИНТ (канал прямого доступа к ОЗУ ЭВМН).

Таким образом, в режиме «ЗАХВ» возможности ЭВМН и аппаратных механизмов, заложенных в интерфейс, обеспечивают обмен данными и статусной информацией между памятью ЭВМН и модулями АП-32. Это позволяет представлять их на видеотерминале в удобной для пользователя форме в любой, наперёд заданной, точке программы АП-32. Поэтому ИНТ, как показано на Рис.484, имеет доступ ко всем основным шинам магистрали АП-32.

Используя терминологию обычных ЭВМ, можно сказать, что модули ПЦЧ, БУ и ОЗУП образуют собой центральный процессор АП-32, а УМН, СУМ, СПЭФ — процессор для чисел с плавающей запятой (ППЗ).

В ПЦЧ исполняются все арифметико-логические операции над 16- и 32-разрядными числами с фиксированной запятой (в модифицированном дополнительном коде). Он содержит массив из 32-х регистров общего назначения (РОН), в дальнейшем именуемых I-регистрами, арифметико-логическое устройство, сдвигатель и другие необходимые узлы. В режиме «АВТ» этим же модулем на шине АЦ формируется исполнительный адрес ОЗУД. В режиме «ЗАХВ» шина AD отключена от ПЦЧ, а адрес ОЗУД формируется ИНТ. Признаки результата операции (флаги N, Z, V, C)<sup>1</sup> по шине FL поступают в БУ и ИНТ. Обмен данными между ОЗУД («АВТ»), ИНТ («ЗАХВ») и I-регистрами производится по шине D магистрали АП-32.

ППЗ представляет собой систему модулей-сопроцессоров, предназначенных для арифметико-логических операций над 32-разрядными данными, хранящимися в ОЗУД. Типы операций определяются самой структурой модуля, а исполнение их осуществляется под управлением собственного узла управления модуля. Число этих модулей, их типы не являются для АП-32 постоянными. Допустима, например, установка нескольких одинаковых устройств. Комбинируя модули (из имеющихся), можно создавать конфигурацию, максимально соответствующую реализуемому алгоритму вычисления.

В представленной версии АП-32 все модули (УМН, СУМ, СПЭФ) реализуют операции над числами с плавающей запятой с однократной точностью. Формат представления чисел совместим с форматом, принятым в семействе ЭВМ «Электроника» (PDP-11).

Несмотря на то, что практически каждый сопроцессор является весьма сложным устройством с собственным блоком управления, с точки зрения программиста он представляет собой некоторый набор 32-разрядных регистров. В дальнейшем мы будем называть их F-регистрами. Каждый F-регистр имеет свой адрес и, кроме того, специальное назначение. Именно то, что F-регистры специализированы, и отличает их от I-регистров. Ограничений на число регистров в одном модуле не предусматривается, однако общее количество F-регистров не должно превышать 32. Системой команд обеспечивается обмен данными F-регистров с ОЗУД и между собой. Как в автономном режиме, так и в режиме «ЗАХВ» такой

---

<sup>1</sup> Флаги NZVC повторяют флаги архитектуры PDP-11 (ЭВМН).



обмен производится командами обращения к ОЗУД. Причем во втором случае («ЗАХВ») роль ОЗУД исполняет регистр данных ИНТ.

Системой команд не предусматривается непосредственная передача содержимого I-регистра F-регистру. Такой обмен возможен только через ОЗУД. Однако обмен данными между однотипными регистрами предусмотрен и реализуется командами типа регистр-регистр (I – I, E – E).

Сопроцессоры могут быть как статического (СУМ, УМН), так и динамического (СПЭФ) типов. В сопроцессорах первого типа отсутствуют элементы запоминания промежуточных результатов, в них используются комбинационные схемы, работа которых не тактируется внутренним генератором. С того момента времени, когда в конкретный входной F-регистр будет загружен последний операнд, модуль статического типа начинает исполнять ту арифметическую (логическую) операцию, для которой он предназначен. По истечении определенного отрезка времени в его выходном F-регистре будет находиться результат операции. В сопроцессорах данного типа узел управления как таковой практически отсутствует.

В сопроцессорах динамического типа, наоборот, присутствуют элементы запоминания промежуточных результатов. Наряду с комбинационными в них используются схемы последовательностного типа, работа которых тактируется внутренним генератором. В таких устройствах наличие внутреннего блока управления, синхронизирующего работу всех узлов, является обязательным. Инициализация (запуск) блока управления, как правило, производится специальной командой. Разумеется, этому должна предшествовать процедура загрузки операндами входных F-регистров модуля.

Таким образом, инициализация сопроцессоров может осуществляться либо только загрузкой входных F-регистров (модули статического типа), либо загрузкой и последующим запуском специальной командой (модули динамического типа). Причём источниками операндов, как это предусмотрено системой команд АП-32, могут быть либо выходные F-регистры самих сопроцессоров (регистр-регистр), либо ОЗУД (память-регистр).

Команды (все сопроцессоры дешифрируют их одновременно) поступают по шине CMD и представляют собой старшее полуслово (16 разрядов) команды К (32 разряда). При обмене данными между F-registрами такой команды половинного формата (регистр-регистр) достаточно. В случае же обмена с ОЗУД необходим полный формат команды с адресной частью. Поскольку адрес ОЗУД формируется ПЦЧ, то полный формат команды передается ему, а сопроцессорам только её старшая половина. В ней кодируются операция обращения к ОЗУД и адрес F-регистра (источника, приемника). В режиме «ЗАХВ», когда БУ АП-32 блокируется (управление передается ЭВМН), источником команд (CMD), адреса ОЗУД (AD) и данных (D) является ИНТ.

Для регистрации признаков результата операции (знак, нуль и т. п.) в сопроцессорах имеются флаговые одноразрядные регистры, выходы которых шиной FL соединяются с БУ и регистром флагов интерфейса.

Флаговые регистры могут быть неадресуемые, а могут наравне с F-registрами иметь адрес. Неадресуемые флаги используются непосредственно командами условного перехода, по одной команде на конкретный флаг. Состояние адресуемого флага нужно предварительно установить (в соответствии с признаком результата операции), для чего используется специальная команда. Затем уже можно применять команду перехода, условием которого является это состояние.

При чтении выходного регистра модуля, содержимое которого оказалось результатом ошибочной операции, сопроцессором будет установлен флаг FA, что приведет к останову АП-32. В этом случае БУ вырабатывает сигнал АНЛТ, с помощью которого в ИНТ формируется запрос на прерывание ЭВМН с фиксацией в регистре статуса ИНТ информации о причине останова.

Структура памяти АП-32 отличается от классической. Как известно, классическая организация памяти предполагает наличие адресного регистра единственного типа. В современных ЭВМ его роль обычно выполняют универсальные аккумуляторы.

В АП-32 конструктивно и логически память разделена на две области: 1) команд (ОЗУП); 2) данных (ОЗУД). Такая структура позволяет разделить общий поток чисел на два отдельных потока.

Каждая область имеет свой адресный регистр. Роль адресных регистров ОЗУД играют I-регистры ПЦЧ с достаточно универсальными логическими возможностями формирования адреса. Адресным регистром области команд служит счётчик команд в БУ, функции которого резко ограничены. Это дало возможность при проектировании этого регистра применить быстродействующие микросхемы и добиться малой длительности командного цикла. При обращении к ОЗУД извлечение операнда из этой области и выборка команды из области команд (ОЗУП) производится практически одновременно. Следует отметить, что платы ОЗУП и ОЗУД не взаимозаменяемы и являются отдельными модулями.

## Потоки информации в АП-32

Своеобразие архитектуры АП-32 состоит в том, что в зависимости от режимов работы, направление основных потоков информации (данных, команд, адресов ОЗУД) в нём меняются.

На Рис.487 изображена схема потоков информации в режиме «АВТ» («ИНИЦ»). Источником команд являются БУ и ОЗУП.

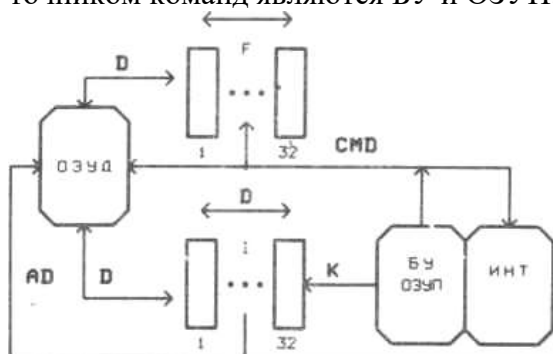


Рис.487 - Потоки информации в АП-32 в режиме «АВТ».

Команды полного формата (K) поступают из ОЗУП только в ПЦЧ и БУ. На шину CMD БУ выводится только половинный формат (старшее полуслово).

Адрес (AD) ОЗУД формируется I-регистрами в командах обращения (LD ST) к ОЗУД.

Потоки данных распределяются между ОЗУД и I-, F-регистрами. Помимо упомянутых (ОЗУД-I, ОЗУД-F) существуют и два других потока: I-I, F-F.

В режиме «АВТ» функции ИНТ резко ограничены. В этом случае в нём исполняются всего две команды для передачи в ЭВМН статусной информации (через регистр статуса и управления). Эта информация используется для синхронизации процессов в АП-32 с процессами в ЭВМН.

В режиме «ЗАХВ» направление основных потоков информации в АП-32 меняется (Рис.488).

Так как БУ в данном режиме блокирован и отключён от магистрали АП-32, то источником команд (CMD) оказывается ИНТ, а не ОЗУП. Поскольку для адресации ОЗУД используется регистр адреса в ИНТ, а для ОЗУП — счётчик команд в БУ, то для обмена данными между памятью, I-, F-регистрами и ИНТ достаточно, как уже упоминалось, команды половинного формата.

Когда обмен производится между I-, F-регистрами и ИНТ, то ОЗУД отключается от магистрали АП-32 сигналом EMD. В модулях схемы дешифрации команд идентифицируют тот или

Рис.488 - Потоки информации в АП-32 в режиме «ЗАХВ».

иной тип регистра по старшему разряду команды.

В случае обмена ИНТ с ОЗУД ( $EMD = 0$ ) источником потока адресов (AD) является ИНТ. Он же является источником (приёмником) и потока данных (D). Аналогично производится обмен и с ОЗУП. И в том, и в другом случае регистр адреса ИНТ и счётчик команд БУ работают в инкрементном режиме.

## **Система команд АП-32**

### **Основные принципы**

При разработке системы команд АП-32 основная цель состояла в том, чтобы получить удовлетворительный компромисс между простотой и богатством её функциональных возможностей. Кроме того, необходимо было учитывать, что быстродействие процессора в значительной степени определяется длительностью командного цикла.

Ниже сформулированы основные принципы построения системы команд АП-32, заключающиеся в следующем.

1. В системе команд АП-32 нет многотактных команд. Все команды одноктактные и однословные (32 разряда), что позволило упростить процедуру выборки их из ОЗУП и сократить неизбежные потери времени на задержках электронных схем.
2. Разновидности временных диаграмм командного цикла АП-32 сведены к минимуму — использовано всего два типа. Это упростило схему генераторов состояний БУ и ИНТ, реализующих временные диаграммы, и сократило потери времени на инициализацию их работы.
3. Поле кода операции команды состоит из ограниченного количества разрядов (6). Причем команды кодируются как двоичным (3 разряда), так и позиционным кодом. Такая кодировка позволила сократить фазу дешифрации команды. Малый размер поля кода операции, естественно, ограничивает список команд. Однако, система команд АП-32, как показал опыт, вполне достаточна для реализации языков программирования высокого уровня.
4. Для достижения высокого быстродействия исполнения в некоторых командах выделены поля, разряды которых имеют смысл управляющих сигналов. Это означает, что уже на фазе дешифрации команды сигналы этих разрядов подаются непосредственно на соответствующие операционные схемы модулей, а в фазе исполнения они начинают играть роль управляющих.
5. Системой команд все модули, независимо от их функционального назначения, интерпретируются как регистры. Поэтому различные арифметико-логические операции, особенно операции над данными с плавающей запятой, дополнительно к коду операции кодируются ещё и адресами соответствующих F-регистров и иницируются командами типа загрузки/выгрузки (LD/ST). Такой подход позволил создать компактную систему команд, пригодную для достаточно широкой номенклатуры модулей.

### **Командный цикл**

Генератор состояний БУ АП-32 вырабатывает два вида временных диаграмм командного цикла. На Рис.489 представлена диаграмма, соответствующая командам обращения к памяти данных (LD/ST).

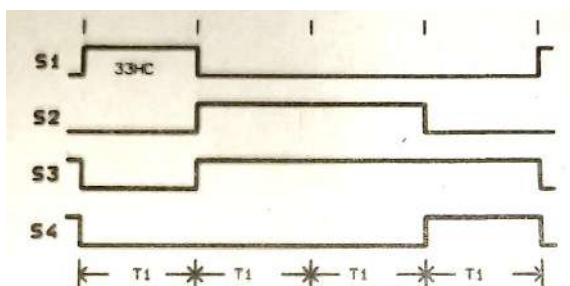


Рис.489 - Командный цикл АП-32 типа LD/ST.

Командный цикл состоит из 4 фаз однократной длительности  $T_1$ , практически полностью зависящей от быстродействия памяти. Управляющие сигналы  $S_1, S_2, S_3, S_4$  вырабатываются либо БУ (режим «АВТ»), либо ИНТ (режим «ЗАХВ») и передаются на сквозные полосковые линии (S) магистрали АП-32.

Диаграмма всех остальных команд по форме аналогична предыдущей и отличается от неё только тем, что короче на одну фазу  $T_1$ .

## Форматы команд

Основная информация о команде содержится в 6-разрядном коде операции. Это поле подвергается дешифрации всеми модулями уже на фазе выборки команды из памяти программ. Оно представлено на Рис.490.

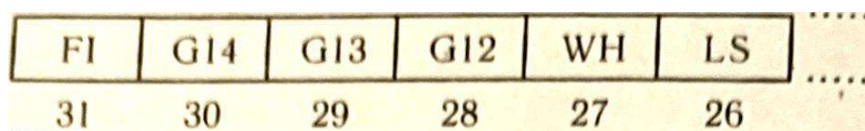


Рис.490 – Код операции команды.

Все команды делятся на 8 групп. Номер группы кодируется двоичными разрядами  $G_{14}, G_{13}, G_{12}$  (таблица на Рис.491). Разрядом  $F_1$  обозначен тип регистра ( $F_1 = 0$  – F-регистр,  $F_1 = 1$  – I-регистр). Разряд  $WH$  кодирует тип данных ( $WH = 0$  – слово,  $A_{УН}=1$  – полуслово). Разряд  $LS$  указывает направление обмена данными:

| LS | Направление    |
|----|----------------|
| 0  | ОЗУД → регистр |
| 1  | регистр → ОЗУД |

| $G_{14}$ | $G_{13}$ | $G_{12}$ | Команды                                                             |
|----------|----------|----------|---------------------------------------------------------------------|
| 0        | 0        | 0        | Управление программой, условные переходы.                           |
| 0        | 0        | 1        | Управление программой.                                              |
| 0        | 1        | 0        | Непосредственная загрузка I-регистров                               |
| 0        | 1        | 1        | Управление флагами ПЦЧ.                                             |
| 1        | 0        | 0        | Обмен данными между I-, F-регистрами и ОЗУД с прямой адресацией.    |
| 1        | 0        | 1        | Обмен данными между I-, F-регистрами и ОЗУД с индексной адресацией. |
| 1        | 1        | 0        | Арифметико-логические операции.                                     |
| 1        | 1        | 1        | Команды действия.                                                   |

Рис.491 – Группы команд АП-32.

Каждая группа содержит из различного количества команд. Так, например, в группе 010 всего одна команда, в то время как в 110 количество команд весьма велико. В таблице на Рис.492 показано, в каких модулях АП-32 исполняются указанные группы команд.

| Группа | В каких модулях исполняется |
|--------|-----------------------------|
| 000    | БУ                          |
| 000    | БУ                          |
| 010    | ПЩЧ                         |
| 011    | ПСЧ                         |
| 100    | ОЗУД, ПЩЧ, ППЗ              |
| 101    | ОЗУД, ПЩЧ, ППЗ              |
| 110    | ПЩЧ, ППЗ                    |
| 111    | ППЗ                         |

Рис.492 – Соответствие групп команд модулям, в которых они исполняются.

### Группа 000

К этой группе относятся команды управления программой (HLT, RET, WAIT, CALL) и команды безусловного и условного переходов (JMP, JCN). Дополнительно к полю кода операций, который для всех этих команд одинаков, используются и другие поля (C1, J), а именно:

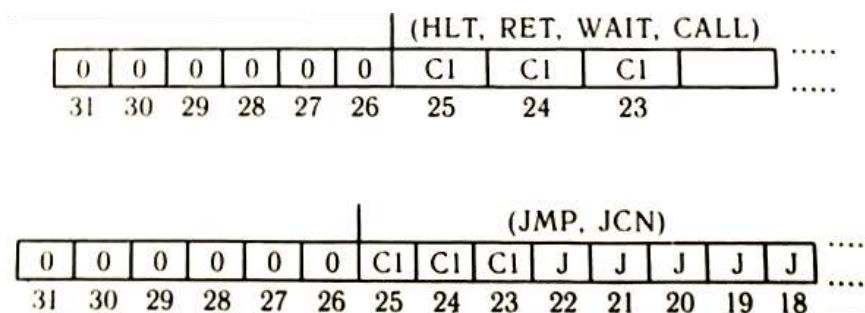


Рис.493 – Команды группы 000.

Разрядами поля C1 кодируется конкретный тип команды, а полем J — условия перехода.

### Группа 001

В этой группе всего одна команда NOP (нет операций).

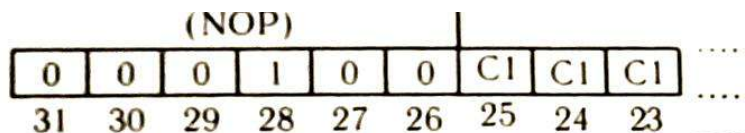


Рис.494 – Команда группы 001 (NOP).

### Группа 010

Она тоже состоит из одной команды LXI. Производит загрузку I-регистра непосредственно 19-разрядным беззнаковым операндом (операнд в поле команды).

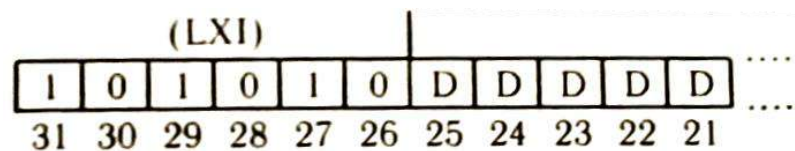


Рис.495 – Команда группы 010 (LXI).

Поле DD — код адреса приемника.

### Группа 011

Команды установки/сброса флагов ПЦЧ. Дополнительно к полю кода операции используется ещё 5-разрядное поле N, Z, V, C и SC.

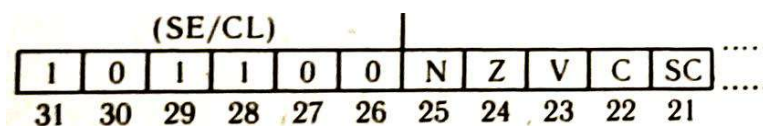


Рис.496 – Команды группы 011.

### Группа 100

Команды обращения к ОЗУД (метод прямой адресации).

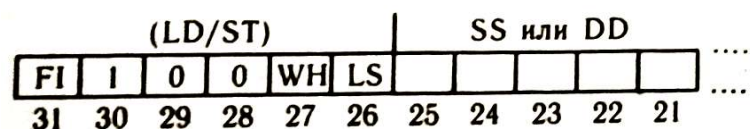


Рис.497 – Команды группы 100.

Осуществляют загрузку/выгрузку ОЗУД содержимым I-, F-регистров. Адрес памяти содержится в адресном поле команды. Тип регистров, тип данных, направление обмена кодируются разрядами FI, WH, LS. Поле SS (DD) — код регистра-источника (приёмника).

### Группа 101

Содержание команд аналогично командам предыдущей группы, однако метод адресации ОЗУД – индексный.

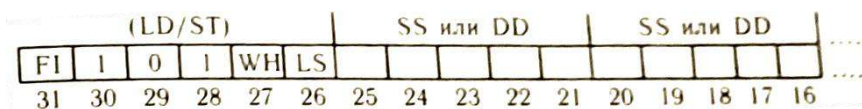


Рис.498 – Команды группы 101.

Добавлено поле адреса регистра, содержащего базовый адрес ОЗУД.

### Группа 110

Команды типа регистр-регистр для выполнения арифметико-логических операций над данными обоих форматов.

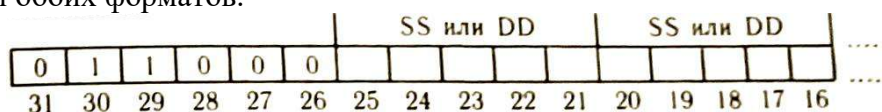
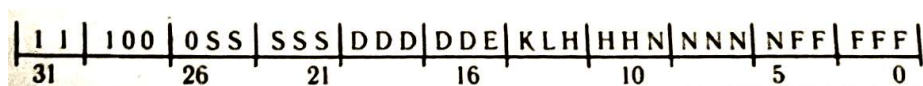


Рис.499 – Команды группы 110.

Представленный формат команды соответствует случаю, когда  $FI = 0$ ,  $WH = 0$ ,  $LS = 0$  (команды типа MVF).

Таковыми командами осуществляется обмен данными между F-регистрами, доступными для записи и чтения. Поскольку вид операции над содержимым F-регистров кодируется их адресами, то для выполнения предполагаемых арифметико-логических действий достаточно команды 16-разрядного формата. Эти команды исполняются как в режиме «АВТ», так и в режиме «ЗАХВ».

Операции над данными I-регистров осуществляются командами 32-разрядного формата (команды типа FUNC). В этом случае  $FI = 1$ , а вид конкретной операции закодирован в младшей части слова команды. Эти команды исполняются только в режиме «АВТ».



Здесь:

FFF — код арифметико-логической операции;

HHH — код операции сдвига;

NNN — код числа сдвигов;

L — источник кода числа сдвигов

( $L = 0$  — число сдвигов задано полем NNN,

$L = 1$  — число сдвигов задано в регистре-источнике);

K — код вида операции

( $K = 0$ ,  $DD \leftarrow SS.OP.DD$ ;

$K = 1$ ,  $DD \leftarrow DD.OP.SS$ );

E — код результата операции

( $E = 0$  — результат записывается в приемник,

$E = 1$  — результат не записывается);

SS — код адреса I-регистра-источника;

DD — код адреса I-регистра-приёмника.

## Группа 111

Команды этой группы, называемые командами действия, используются, как правило, для получения информации о текущем состоянии модулей (установка адресуемых флагов), для инициации (или блокировки) узлов управления и других аналогичных действий.



Рис.500 – Команды группы 111.

Здесь поле RR — адрес псевдорегистра модуля, NN — поле «расширения» кода адреса псевдорегистра. Предусмотрено в качестве резерва.

## Технические характеристики АП-32

Магистраль АП-32 состоит из трех двуслойных печатных плат, с помощью которых контакты трёх разъемов модуля соединяются с сигнальными линиями. Все шины выполнены печатными проводниками (0,5 мм) в виде полосковых линий с волновым сопротивлением около 50-60 Ом. Оба конца линий подключаются к согласующим сопротивлениям (100 Ом,  $-2B$ ), которые обеспечивают уровень «логического нуля» ( $-1,65 B$ ). Общая длина



магистральной составляет 382,5 мм. В неё вмонтировано три ряда разъемов по 18 штук в каждом, что обеспечивает возможность подключения до 18 печатных плат.

Процессор АП-32 собран на 14 четырехслойных печатных платах (из них 8 заняты под ОЗУ) размером 350х400 мм. Были использованы микросхемы отечественного производства серий 1800, 1500, 500. Уровни электрических сигналов соответствуют стандарту ЭСЛ.

Память собрана на БИС ОЗУ статического типа серии К132РУ6А (16К х 1).

## Технические параметры АП-32

Производительность – около 10 млн. коротких операций в сек.

Разрядность слова данных и команды – 32

Среднее время выполнения операций над 32-разрядными операндами:

- сложение с фиксированной запятой – 0,10 мкс
- сложение с плавающей запятой (однократ. точн.) – 0,10 мкс
- умножение с плавающей запятой (однократ. точн.) – 0,20 мкс
- короткие операции – 0,10 мкс
- деление с плавающей запятой – 4,0 мкс
- вычисление элементарных функций – 4,0—13,0 мкс

Оперативная память:

- объём – 2 Мбайт
- время доступа – 0,10 мкс

Потребляемая мощность – 300 Вт

Размер корпуса (без источника питания) – 500х500х600 мм

## Сравнительная оценка быстродействия АП-32

Ниже приведены характеристики быстродействия АП-32 и некоторых универсальных ЭВМ на основе тестов «решето Эратосфена» (таблица на Рис.501) и Ветстоуна (таблица на Рис.502, параметр ТЕСТА N = 500, однократная точность).

| ЭВМ            | Время, с | Язык |
|----------------|----------|------|
| АП-32          | 0,50     | F-4  |
| ЕС-1061        | 1,77     | F-4  |
| VAX-11/780     | 2,3      | F-77 |
| БЭСМ-6         | 2,6      | F-4  |
| ЭЛЕКТРОНИКА-79 | 3,3      | F-4  |

Рис.501 – «Решето Эратосфена» на разных архитектурах.

| ЭВМ            | Время, с | Язык         |
|----------------|----------|--------------|
| VAX 8800       | 10,04    | F-77         |
| АП-32          | 20,0     | F-4          |
| ЕС-1061        | 53,0     | F-4          |
| ЕС-1061        | 33,0     | F-77, ОПТ(3) |
| ЭЛЕКТРОНИКА-79 | 78,6     | F-77         |

Рис.502 – Тест Whetstone на разных архитектурах.

## Система команд АП-32

Из соображений экономии полное описание системы команд АП-32, временные диаграммы, подробные структурные схемы процессора и т.п. в этом документе автор приво-

дить не стал. Интересующиеся подробностями могут отослать на сайт ИЯФ СО РАН, в каталог с архивом препринтов (к сожалению, старые препринты присутствуют там только в виде сканов-картинок):

1. Г.А. Аксенов, А.В. Кислицын, Ю.И. Мерзляков, В.Я. Сазанский, И.А. Ткаченко, А.Г. Чертовских. [Универсальный арифметический процессор АП-32. 1. Архитектура, система команд, технические характеристики.](#)
2. Г.А. Аксенов, В.Я. Сазанский, И.Б. Стенина, А.Г. Чертовских. [Универсальный арифметический процессор АП-32. 2. Центральный процессор, память.](#)
3. Г.А. Аксенов, Ю.И. Мерзляков, В.Я. Сазанский, А.Г. Чертовских. [Универсальный арифметический процессор АП-32. 3. Канал управления и ввода-вывода, драйвер операционной системы.](#)
4. Г.А. Аксенов, А.В. Кислицын, В.Я. Сазанский, А.Г. Чертовских. [Универсальный арифметический процессор АП-32. 4. Арифметика с плавающей запятой, сопроцессор элементарных функций.](#)
5. Г.А. Аксенов, В.Н. Козлов, Ю.И. Мерзляков, В.Я. Сазанский, И.А. Ткаченко. [Программное обеспечение процессора АП-32. 1. Комплекс программного обеспечения, средства разработки программ, операционная среда программ АП-32.](#)

## Канал ввода/вывода и управления АП-32

АП-32 является автономным арифметическим процессором со своим блоком управления, памятью и оригинальной системой команд. Аппаратура АП-32 ориентирована главным образом на вычислительный процесс, поэтому процессор не имеет собственных устройств ввода/вывода, пульта управления и его функционирование возможно только в комплексе с другой ЭВМ, исполняющей относительно него функции управления режимами работы и ввода/вывода (ЭВМ «накачки», ЭВМН)<sup>1</sup>.

В качестве интеллектуального контроллера периферийного оборудования и мониторинговой подсистемы такого комплекса используется промышленная ЭВМ семейств «Электроника» (или СМ ЭВМ), информационная и командная связь которой с АП-32 на аппаратном уровне осуществляется с помощью интерфейса ИНТ60-АП32. Этот комплекс по способу связей можно определить, как прямосвязанный многомашинный вычислительный комплекс (ММВК). ИНТ60-АП32 обеспечивает связь машин по каналу прямого управления («процессор – процессор»), а также выступает в роли адаптера «канал Q-bus – магистраль АП-32» для обмена информацией.

Главную задачу создания такого комплекса можно обозначить как одновременное достижение следующих целей:

1. большой вычислительной мощности при решении счётных задач;
2. широких возможностей обмена информацией с периферией (высокая скорость обмена и полная номенклатура внешних устройств, включая сети ЭВМ);
3. эффективного сервиса на уровнях оператора и программы пользователя;
4. удобства проведения регламентных работ и контроля надёжности аппаратуры АП-32.

Для решения этой задачи выработан ряд принципов взаимодействия упомянутых ЭВМ в рамках этого комплекса.

1. В комплекс входят ЭВМ с разными характеристиками, выполняющими различные функции. Решая каждая свою задачу, эти машины находятся в определенной соподчинённости, что позволяет назвать комплекс сателлитным, т. е. ЭВМ делятся на главную

---

<sup>1</sup> Собственно, именно поэтому АП-32 везде обозначается как процессор, а не полноценная ЭВМ.

и вспомогательную. Реализуются два режима взаимодействия ЭВМ, когда роль спутниковой (подчинённой) ЭВМ выполняет та или другая машина, в зависимости от цели. Например, при регламентных работах на АП-32 он выполняет роль спутникового процессора (управление процессом осуществляет ЭВМН), а при нормальной эксплуатации основная обработка информации проходит в АП-32 (ЭВМН при этом работает как процессор ввода/вывода).

2. Взаимодействие ЭВМ обеспечивается при помощи иерархической аппаратно-программной системы. Нижний уровень системы представляет аппаратный протокол взаимодействия, выше – несколько программных уровней.
3. На аппаратном уровне взаимодействия роль главной ЭВМ всегда выполняет ЭВМН, управляющая информационным и командным каналами связи. Работающий АП-32 может участвовать во взаимодействии, только исполняя некоторые команды, приводящие к его останову и прерыванию ЭВМН. Информационный и командный каналы связи ЭВМ не могут работать одновременно. Это означает, что обмен информацией можно проводить только с остановленным АП-32, что вызвано стремлением к упрощению протокола магистрали АП-32 ради повышения его быстродействия.
4. Нижним уровнем программной иерархии является драйвер операционной системы ЭВМН, реализующий аппаратный протокол связи ЭВМ (управление информационным и командным каналами ИНТ60-АП32) и программный протокол взаимодействия.
5. Для работы в режиме «ЭВМН – главный процессор, АП-32 – спутниковый» достаточно перечисленных уровней. Выше в иерархии находится произвольная основная программа ЭВМН, управляющая деятельностью АП-32. АП-32 может исполнять также произвольную программу или работать с отключенным блоком управления, как пассивный набор аппаратных модулей.
6. Для реализации режима «АП-32 – главный процессор, ЭВМН – процессор ввода/вывода» требуется ещё один программный уровень взаимодействия, основанного на информационной связи произвольной программы АП-32 с системной библиотекой и специальной программой ЭВМН.

Исходя из основных целей и опираясь на эти принципы, можно сформулировать ряд конкретных задач, решаемых аппаратными средствами (интерфейс ИНТ60-АП32) и программными (драйвер АП-32).

1. Реализация информационного канала связи – ввод/вывод информации в память АП-32. К памяти относятся регистровая память (I-, F-регистры), память программ и данных.
2. Реализация командного канала связи (управление режимами работы АП-32): пуск, останов, пошаговое исполнение команд, задание интервала времени работы АП-32, организация режимов наладки и контроля аппаратуры АП-32.
3. Извлечение служебной информации о состоянии АП-32, которая используется программами обеспечения связи (по завершению операций по обоим каналам), диагностика и обработка аппаратных и арифметических ошибок АП-32.
4. Выполнение базового набора законченных функций, требующих интенсивной работы с аппаратурой и взаимодействия с программой АП-32. Сюда относятся функции отладки программ (точки останова в программе АП-32), автоматическое определение объёма памяти АП-32, отработка протокола передачи блока данных АП-32 по его останову и некоторые другие.

## **Интерфейс ИНТ60-АП32**

На аппаратные средства интерфейса ИНТ60-АП32, рассчитанного на работу с 16-, 18- и 22-разрядной шиной «Q-bus» (общая шина ЭВМ «Электроники») возложены следующие задачи:

1. пуск и останов процессора АП-32, пошаговое исполнение программы, т. е. управление режимами работы комплекса;

2. организация информационного обмена ЭВМН с модулями процессора как по инициативе ЭВМН, так и по инициативе АП-32;
3. управление модулями АП-32 от ЭВМН (в этом случае процессор выступает только в роли набора специализированных модулей, исполняющих специфические функции в зависимости от назначения каждого).

Поскольку АП-32 имеет высокую производительность, большой объём памяти, то скорость информационного обмена данными с устройствами ввода-вывода оказывает значительное влияние на общую производительность комплекса. Поэтому с целью её увеличения возникает необходимость организации обмена между ИНТ60-АП32 и ЭВМН не только по программному каналу ЭВМН, но и по каналу прямого доступа (КПД/DMA) к её памяти.

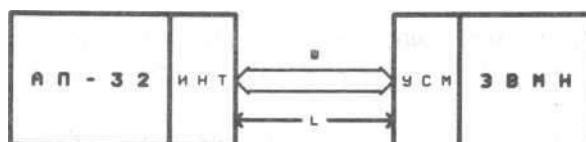


Рис.503 - Канал В/В и управления (аппаратный уровень).

## Структура ИНТ60-АП32

На Рис.504 изображена упрощённая функциональная схема интерфейса. Он состоит из следующих основных узлов:

1. схемы управления магистралью ЭВМН;
2. схемы управления магистралью АП-32;
3. массива регистров;
4. автомата канала прямого доступа к памяти ЭВМН.

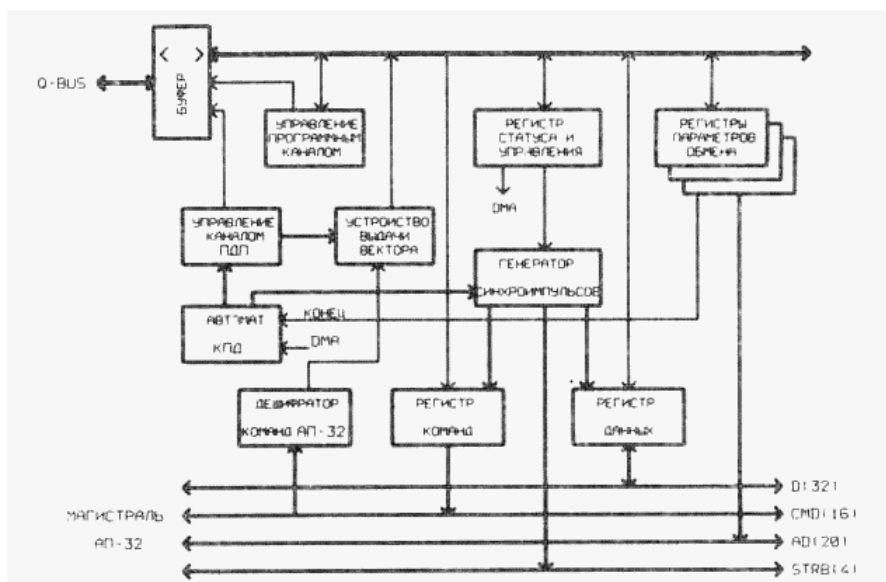


Рис.504 – Структура ИНТ60-АП32.

Схема управления ОШ ЭВМ содержит схему обмена по программному каналу, устройство выдачи вектора прерывания, дешифратор адресов регистров, генератор временной диаграммы прямого доступа к памяти.

Схема управления магистралью АП-32 содержит устройство синхронизации АП-32, шинные формирователи и дешифратор команд АП-32.

В режиме обмена по каналу прямого доступа к памяти ЭВМН управление всеми узлами ИНТ берет на себя автомат КПД. Параметры обмена предварительно должны быть

загружены в соответствующие регистры интерфейса со стороны ЭВМН (по программному каналу).

Схема управления магистралью ЭВМН является стандартной (в части обмена по программному каналу) и обычна для большинства периферийных устройств семейства «Электроника».

Доступ ЭВМН к регистрам и памяти АП-32 осуществляется схемой управления магистралью АП-32 и производится следующим образом.

1. В регистр команд ИНТ (описание регистров см. ниже) загружается 16-разрядная команда ввода/вывода из системы команд АП-32 (тип команды определяется модулем, к которому планируется произвести обращение).
2. В регистры адреса интерфейса записывается адрес ОЗУД (в случае, если обращение идет к памяти данных).
3. В регистр состояния и управления заносится управляющее слово, которое инициирует устройство синхронизации. В результате, интерфейсом генерируется командный цикл АП-32, содержимое регистра команд и регистра адреса ОЗУД поступают на соответствующие шины АП-32.

Команда считается исполненной, когда в регистр данных ИНТ произойдет запись (чтение) слова данных. Обращение к ОЗУП АП-32 несколько сложнее в связи с тем, что этот тип памяти адресуется только через счетчик команд БУ. Поэтому для обмена с ОЗУП необходимо предварительно записать в счетчик команд адрес ячейки.

После исполнения командного цикла счетчик команд БУ (для ОЗУП) или регистр адреса (для ОЗУД) интерфейса автоматически инкрементируются.

## Регистры интерфейса

Со стороны ЭВМН ИНТ представляется как массив из 10 специализированных и адресуемых на ОШ «Электроника» регистров, в число которых входят регистры состояний и управления, данных, флагов и т. д. (таблица на Рис.505).

| Обозначение регистра | Назначение регистра | Кол-во разрядов | Тип опер. от ЭВМ | Тип опер. от АП-32 |
|----------------------|---------------------|-----------------|------------------|--------------------|
| РС                   | управление          | 16              | чтение/запись    | запись             |
| РД1                  | данные (мл.)        | 16              | чтение/запись    | чтение/запись      |
| РД2                  | данные (ст.)        | 16              | чтение/запись    | чтение/запись      |
| РК                   | команда             | 16              | запись           | —                  |
| РФ                   | флаги               | 13              | чтение           | —                  |
| РАД1                 | адрес ОЗУД (мл.)    | 16              | запись           | —                  |
| РАД2                 | адрес ОЗУД (ст.)    | 3               | запись           | —                  |
| РАП1                 | адрес ОЗУ ЭВМ (мл.) | 15              | запись           | —                  |
| РАП2                 | адрес ОЗУ ЭВМ (ст.) | 6               | запись           | —                  |
| РДМ                  | длина массива       | 16              | запись           | —                  |

Рис.505 – Регистры ИНТ.

Со стороны АП-32 интерфейс выглядит как один 32-разрядный регистр (PD1, PD2), играющий роль буферного при обмене данными между АП-32 и ЭВМН.

## Регистр состояний и управления

С помощью РС осуществляется управление режимами работы и взаимодействия АП-32 с ЭВМН. Формат РС показан ниже:

|     |    |     |     |    |     |   |   |     |     |     |     |     |     |     |     |
|-----|----|-----|-----|----|-----|---|---|-----|-----|-----|-----|-----|-----|-----|-----|
| 15  | 14 | 13  | 12  | 11 | 10  | 9 | 8 | 7   | 6   | 5   | 4   | 3   | 2   | 1   | 0   |
| FPE |    | IOE | PRQ |    | INP |   |   | END | INT | A17 | A16 | HLD | STP | STR | DMA |

Рис.506 – Регистр состояния и управления.

### ***Регистр данных***

Регистр данных (РД) используется в обмене данными между модулями АП-32 и ЭВМН. Он состоит из двух 16-разрядных регистров, однако со стороны АП-32 выглядит как 32-разрядный регистр, адресуемый на шине данных D магистрали АП-32.

Со стороны АП-32 операции чтения/записи регистра возможны только в режимах «ИСП», «КПД», причем 32-разрядным словом за один командный цикл. Обмен данными между ним и ЭВМН производится за два цикла обращения к каналу ввода-вывода ЭВМН на каждое 32-разрядное слово. Со стороны ЭВМ операции чтения-записи РД возможны как в режиме программного обмена, так и в режиме прямого доступа к оперативной памяти.

### ***Регистр команд***

Регистр команд служит для хранения 16-разрядного эквивалента команд обмена между ОЗУД и F-регистрами в режимах «ИСП», «КПД». Со стороны ЭВМН РК доступен только для записи (в режиме программного обмена). БУ АП-32 доступа к РК не имеет. При исполнении командного цикла, инициируемого ЭВМН с помощью РС и осуществляемого ИНТ, содержимое РК передается на шины CMD и воспринимается модулями АП-32 как команда. Одновременно с этим на шины AD (в случае обращения к ОЗУД) будет передано содержимое регистров адреса ОЗУД (РАД1, РАД2), где находится адресная часть команды.

### ***Регистр адреса ОЗУД***

Регистр адреса ОЗУД состоит из двух регистров: РАД1 (16 разрядов) и РАД2 (3 разряда). Со стороны ЭВМН эти регистры доступны только для записи. БУ АП-32 доступа к ним не имеет. Совместно с РК регистр адреса служит для хранения команд обращения к ОЗУД АП-32 и используется только в режимах «ИСП», «КПД». Выходы регистра подключены к шинам AD магистрали АП-32 и инициируются при исполнении команд обмена 32-разрядными данными между ОЗУД и F-регистрами модулей АП-32. В том случае, когда обмен производится между регистром данных ИНТ и регистрами модулей (регистр-регистр), ОЗУД должно быть отключено от шины данных D. Такой тип обмена задается содержимым старшего разряда EMD регистра РАД2 (EMD = 0 – ОЗУД «Включено», EMD=1 – ОЗУД «Отключено»). При обмене данными между ОЗУД и РД интерфейса РАД работает в инкрементном режиме.

### ***Регистр адреса ОЗУ ЭВМН***

Регистр адреса памяти ЭВМН (РАП) предназначен для задания адреса первого элемента массива данных. Используется только в режиме «КПД».

Со стороны ЭВМН РАП доступен только для записи. БУ АП-32 доступа к РАП не имеет.

РАП состоит из двух регистров: 15-разрядный РАП1 (16-разрядная Q-bus) и 6-разрядный РАП2 (22-разрядная Q-bus). Если ОШ ЭВМН 18-разрядная, то РАП2 отсутствует, а в качестве двух старших разрядов адреса (A17, A18) используются разряды регистра состояния и управления (РС).

Поскольку обмен данными производится 16-разрядными словами, то младший разряд (адрес байта) РАП не используется.

Адрес элемента массива записывается в РАП за два обращения к программному каналу ЭВМН. В процессе обмена к содержимому РАП добавляется единица после передачи каждого 16-разрядного слова.

### ***Регистр длины массива***

Регистр длины массива РДМ (16 разрядов) используется в режиме «КПД» для подсчёта количества слов, переданных по каналу прямого доступа к памяти ЭВМ. Необходимое количество слов (не более 32К) задаётся ЭВМ путем записи числа в РДМ за одно обращение к программному каналу.

После каждого переданного в (из) ЭВМ слова РДМ уменьшается на единицу. Когда число заданных слов окажется равным числу переданных, РДМ вырабатывает сигнал, который воспринимается автоматом КПД как конец передачи.

### Регистр флагов

При исполнении каждой арифметико-логической операции в режимах «АВТ», «ИНИЦ», «ИСП» признаки результата фиксируются в регистре флагов ИНТ синхросигналом S4 командного цикла. Формат РФ приведен ниже:

|    |    |    |    |     |     |     |    |    |   |   |   |   |
|----|----|----|----|-----|-----|-----|----|----|---|---|---|---|
| 12 | 11 | 10 | 9  | 8   | 7   | 6   | 5  | 4  | 3 | 2 | 1 | 0 |
| FA | FG | RQ | WT | FEQ | FGT | FVL | FZ | FN | N | Z | V | C |

Рис.507 – Регистр флагов.

Здесь N, Z, V, C – флаги ПЦЧ, остальные – флаги процессора чисел с плавающей запятой.

### Режимы работы и команды, исполняемые интерфейсом

В режиме «АВТ» (АП-32 работает автономно) функции ИНТ ограничены и используются только для синхронизации процессов АП-32 с процессами ЭВМН. Из всей системы команд АП-32 им исполняются всего две: «конец процесса», «программное прерывание». Они передаются БУ по шине CMD интерфейсу, где дешифрируются и исполняются.

В режиме «ЗАХВ» (исполнение программы АП-32 остановлено) регистры ИНТ доступны ЭВМН для обмена информацией по программному каналу ввода/вывода. Обращением ЭВМН к РС инициализируется исполнение команды, содержащейся в РК интерфейса (режим «ИСП»). Команды, исполняемые в «ИСП», можно разделить на несколько групп (таблица на Рис.508). Причем формат этих команд такой же, что и формат команд обращения к ОЗУД и I-, F-регистрам, исполняемых при автономной работе АП-32.

| N  | Группа команд           | Режимы работы АП-32 |
|----|-------------------------|---------------------|
| 1. | ОЗУД — I-, F-регистры   | «ИСП»               |
| 2. | ОЗУД - РД               | «ИСП», «КПД»        |
| 3. | I-, F-регистры — РД     | «ИСП», «КПД»        |
| 4. | F-регистры — F-регистры | «ИСП»               |

Рис.508 – Команды в режиме «ИСП».

В режиме «КПД» производится обмен массивами данных между АП-32 и ЭВМН в режиме прямого доступа к её памяти. Данные передаются двумя последовательными этапами. На первом этапе исполнением команды, содержащейся в РК, осуществляется запись передаваемого модулем АП-32 слова в РД интерфейса. Этот этап аналогичен режиму «ИСП» и отличается от него только тем, что в данном случае командный цикл интерфейса инициализируется автоматом КПД, а не регистром статуса. На втором этапе автомат передает содержимое РД в ЭВМН по каналу прямого доступа к памяти. Таким образом, рассматриваемый тип обмена осуществляется под управлением ИНТ, однако процесс передачи инициализируется ЭВМН посредством обращения её к статусному регистру.



## Алгоритмы операций канала ввода/вывода и управления

Аппаратные механизмы ИНТ60-АП32 и система команд, исполняемых в нём, обеспечивают ряд операций канала связи АП-32 — ЭВМН. Каждая операция выполняется по определенному алгоритму. Эти алгоритмы операций перечислены ниже.

1. А1 — чтение/запись I-регистров.
2. А2 — чтение/запись F-регистров.
3. А3 (ПР) — чтение/запись ОЗУД по программному каналу.
4. А3 (КПД) — чтение/запись ОЗУД по каналу прямого доступа.
5. А4 (ПР) — чтение/запись ОЗУП по программному каналу.
6. А4 (КПД) — чтение/запись ОЗУП по каналу прямого доступа.
7. А5 — управление модулями АП-32 в режиме «ИСП».
8. А6 — пуск АП-32 в режиме «АВТ».
9. А7 — пуск АП-32 на один шаг в режиме «ШАГ».
10. А8 — останов АП-32 (переход в режим «ЗАХВ»).

Ниже для иллюстрации приведены два алгоритма операций канала.

### Алгоритм А3 (ПР): ОЗУД → ОЗУ ЭВМН

1. Записать в РАД1, РАД2 ( $EMD = 0$ ) начальный адрес ОЗУД («ЗАХВ»).
2. Записать в РК команду из группы ОЗУД — РД («ЗАХВ»).
3. Записать в РС управляющее слово, соответствующее режиму «ИСП».
4. Выгрузить из РД (РД1, РД2) в ЭВМН данные (за два цикла обращения к каналу В/В в режиме «ЗАХВ»).

Если передача осуществляется массивами, то п.3, п.4 должны повторяться столько раз, сколько слов (32-разрядных) в массиве.

### Алгоритм А3 (КПД): ОЗУД → ОЗУ ЭВМН

1. Записать в РАД (РАД1, РАД2) начальный адрес ОЗУД ( $EMD = 0$ ).
2. Записать в РАП (РАП1, РАП2) начальный адрес памяти ЭВМН.
3. Записать в РДМ число 32-разрядных слов передачи.
4. Записать в РК команду из группы ОЗУД — РД.
5. Записать в РС управляющее слово, соответствующее режиму «КПД» и направлению передачи (разряд INP в РС).

Исполнением п.5 («ЗАХВ») инициализируется процесс передачи данных, который затем полностью управляется интерфейсом. Заканчивается он прерыванием ЭВМН и установкой разряда END («Конец») в регистре состояний и управления.

## Командный цикл интерфейса

При записи в РС управляющего слова, соответствующего, например, режиму «ИСП», устанавливается разряд STR. Положительный перепад выходного сигнала триггера STR инициализирует генератор состояний, который вырабатывает четыре синхросигнала — командный цикл (Рис.509).

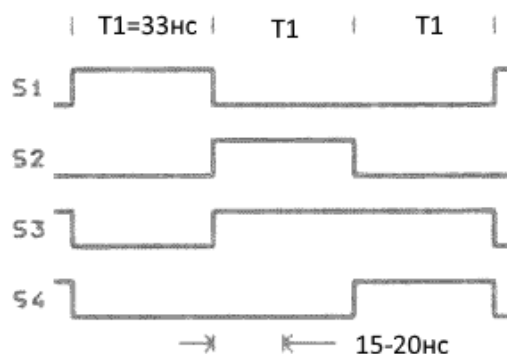


Рис.509 – Командный цикл интерфейса.

Диаграмма командного цикла интерфейса полностью идентична временным диаграммам команд обращения к ОЗУД. Общая длительность цикла равна  $4T1$  ( $T1 = 33 \text{ — } 40 \text{ нс}$ ).

В течение промежутка времени, равного длительности синхросигнала S1, все модули АП-32 дешифрируют команду. Если производится обращение к ОЗУД, то адрес памяти должен быть установлен на шинах AD регистром адреса (РАД) к моменту времени, примерно равному половине длительности синхросигнала S1. Дешифрация адреса микросхемами памяти начинается с момента появления сигнала S2. Синхросигналом S3 стробируются данные F-регистров, ОЗУД и РД интерфейса. Запись данных осуществляется S4. Им же инициируется сигнал, который сбрасывает триггер STR. На этом исполнение команды заканчивается. Следующий цикл начнётся только после новой установки STR. При пошаговом исполнении команд ОЗУП АП-32 (режим «ИНИЦ») триггером STR инициируется генератор состояний БУ, а не интерфейса. Соответственно, и сбрасывается триггер STR блоком управления (ЕХС).

## Драйвер АП-32

Существует два средства программирования ЭВМ семейства «Электроника» для работы с подключенной к ней аппаратурой: драйвер операционной системы и специализированная библиотека (пакет) подпрограмм доступа к этой аппаратуре, вызываемых непосредственно из программы. Оба подхода имеют свои достоинства и недостатки. К достоинствам драйвера относится следующее:

1. драйвер расположен в системном пространстве памяти, защищён от порчи пользовательской программой и не занимает виртуальное адресное пространство программы пользователя (верно только для ОС TSX-Plus);
2. одна копия драйвера в памяти может обслуживать несколько одинаковых устройств;
3. ОС ЭВМН соблюдает дисциплину очередности операций с устройством, не позволяет прерывать незаконченные операции следующими и предоставляет несколько вариантов синхронизации программы пользователя с завершением драйверной операции.

Главный недостаток драйвера — большой расход времени (порядка нескольких миллисекунд) на обработку операционной системой запроса к драйверу. При работе через пакет подпрограмм этого расхода нет, но и отсутствуют все достоинства драйвера.

Для решения задач ввода/вывода и управления АП-32 посредством интерфейса ИНТ60-АП32, а также некоторых других задач, был написан драйвер, работающий под операционной системой реального времени RT-11 версии 5.0 и более поздними (FB и XM-мониторы) и совместимой с ней многопользовательской операционной системой TSX-Plus V06.01.

Как упоминалось выше, требуется обеспечить режимы нормальной эксплуатации вычислительного комплекса и настройки аппаратуры, проведения регламентных работ.

В первом случае деятельность АП-32 характеризуется большим временем счёта и малым временем ввода/вывода. При этом потери, обусловленные ОС ЭВМН, несущественны. Достоинства (1) и (2) драйвера позволяют эксплуатировать АП-32 в многопользовательской ОС ЭВМН, а также в режиме «одна ЭВМН — несколько АП-32». Пункт 3 позволяет составить удобный протокол общения программы АП-32 и программы его обслуживания в ЭВМН.

При наладке, тестировании аппаратуры требования противоположные: время операций ввода/вывода превалирует над временем автономной деятельности АП-32, что удлиняет процесс тестирования или делает его невозможным. С другой стороны, такая деятельность ведётся в монопольном режиме ЭВМН и сложное взаимодействие АП-32 и ЭВМН отсутствует, так что достоинства драйвера здесь не важны.

Для удовлетворения этих противоречивых требований был написан драйвер, работающий как собственно драйвер и как пакет. В «нормальном» режиме можно работать без ограничений, а в режиме пакета («библиотечном») — только под RT-11, где драйвер после загрузки находится в адресном пространстве программы, которая может обращаться к нему специальным образом, минуя ОС ЭВМН. В последнем случае сохранены почти все возможности драйвера, работающего в «нормальном» режиме. Переключение из одного режима в другой происходит при изменении параметра.

В драйвер заложен ряд возможностей, опирающихся на аппаратные средства интерфейса ИНТ60-АП32. Одни возможности реализуются отдельными алгоритмами интерфейса, другие — комбинацией алгоритмов. Некоторые были добавлены по результатам практической работы (их список можно расширять по мере необходимости). В настоящее время драйвером обеспечивается следующий функционал.

1. Ввод/вывод информации в память АП-32 (регистровую, память программ и данных).
2. Управление работой АП-32: пуск, останов, пошаговое исполнение команд.
3. Управление работой отдельных модулей АП-32.
4. Доступ к памяти АП-32 при помощи стандартных запросов как к электронному диску.
5. Обработка аппаратных и программных прерываний от АП-32; диагностика его программных остановов, аппаратных и арифметических ошибок.
6. Возврат информации о состоянии АП-32 в вызвавшую программу ЭВМН по завершению любой операции.
7. Автоматическое определение объёма памяти АП-32 после загрузки драйвера.
8. Останов АП-32 при завершении работы связанной с ним программы ЭВМН.
9. Выбор режима операции ввода/вывода — по КПД или программному каналу — для памяти, нормальный или ускоренный (настроечный) для регистров.
10. Выполнение отладочных функций в программе АП-32 — установка и удаление точек останова, обработка этих остановов.
11. Таймирование операций — задание интервала времени работы АП-32 и установка тайм-аутов на ввод/вывод.
12. Передача блока данных из АП-32 по завершению работы его программы.

Часть перечисленных возможностей (п.1—4) реализованы в виде отдельных стандартных или специальных функций драйвера, доступных из программы ЭВМН. Другие являются обязательным элементом ряда (или всех) этих функций (п.5—8). Третьи (п.9—12) включаются в функции, если указаны соответствующие параметры.

Драйвер можно генерировать, устанавливая различные условия трансляции, при этом можно регулировать его размер и скорость исполнения функций путем включения или

исключения отдельных его частей. Его можно также подстраивать, задавая режимы его работы непосредственно при обращении к нему в качестве одного из параметров операции.

## Комплекс программного обеспечения

### Некоторые принципы программирования АП-32

Процессор АП-32 создан для обработки данных физического эксперимента и научных расчетов. Особенность его состоит в необходимости подключения к нему ЭВМ «накачки» (ЭВМН), в результате чего образуется, по сложившейся терминологии, много-машинный вычислительный комплекс (ММВК). В рамках этого комплекса можно считать АП-32 «ускорителем» арифметики ЭВМН, или же ЭВМН рассматривать как канал ввода/вывода АП-32.

Для программирования АП-32 созданы средства разработки его математического и программного обеспечения (МО и ПО) и средства исполнения и отладки программ. Характер и состав этих средств определялись условиями, в которых решаются задачи обработки данных в ИЯФ, архитектурой АП-32 и возможностями разработчиков. При программировании арифметических процессоров-сателлитов, таких, например, как АП-20, 3081/Е, АР-120 и аналогичных им характерно следующее:

1. рабочая программа комплекса «ЭВМН — арифметический процессор» состоит из двух частей: программы ЭВМН и программы процессора;
2. программа ЭВМН реализует логику управления вычислительным процессом, операции ввода/вывода информации и системно-зависимые действия;
3. программа процессора содержит только счётные подпрограммы.

Такой подход неудобен тем, что в каждом конкретном случае исходную программу нужно разбивать на две части для разных ЭВМ, каждую из них отлаживать независимо, а затем — во взаимодействии. Отсутствие интерактивных средств затрудняет процесс создания и отладки программы процессора. Аналогичный режим работы возможен и в ММВК «ЭВМН — АП-32», но он используется не в качестве основного, а только для наладочных и регламентных работ.

При разработке ПО АП-32 была поставлена задача обеспечить его проблемной программой возможности программы универсальной ЭВМ. При нормальной эксплуатации комплекса вся логика управления вычислительным процессом должна быть сосредоточена в проблемной программе АП-32, а на ЭВМН возложены только функции обслуживания этой программы. Таким образом, необходимость разделения исходной программы на части исчезает, и ММВК работает как единая ЭВМ.

Выбор способов взаимодействия ЭВМ, составляющих ММВК, и подходов к реализации ПО осуществлялись на основе ряда принципов, сформулированных ниже.

— **«Прозрачность».** Под этим понимается, что пользователь ММВК не должен ощущать разницы при работе с ЭВМН или АП-32, рассматривая комплекс как единую универсальную ЭВМ, работающую под операционной системой ОС ЭВМН, где вычислительные процессы в АП-32 и ЭВМН выглядят как равноправные задачи. При этом ресурсы периферийных устройств, подключенных к ЭВМН, должны быть доступны пользователю АП-32 в той же самой степени, что и пользователю ЭВМН.

Такой подход позволяет работать с АП-32 наиболее простым и естественным образом, исключая затраты времени на изучение его особенностей и адаптацию программ. Это, однако, предполагает, что как процессы разработки и эксплуатации программ этих ЭВМ, так и возможности, предоставляемые при их программировании (сервис операционной системы, ввод/вывод и т. д.) должны иметь много общего.

— **Автономность.** В состав ММВК входит ЭВМН одного семейства ЭВМ («Электроника»/СМ), программно совместимого с DEC PDP-11, и всё системное МО организации

комплекса со стороны ЭВМН выполнено для этого семейства. Средства разработки программ АП-32 также работают на ЭВМН, производя коды АП-32 (т.н. кросс-обеспечение). Таким образом, ММВК представляет собой функционально полную замкнутую вычислительную систему, которую можно эксплуатировать индивидуально или в составе других машинных комплексов, куда она включается посредством аппаратно-программных средств ЭВМН

— **Конкурентоспособность.** Главное достоинство АП-32 — высокое быстродействие — не является достаточным и должно сочетаться с высоким уровнем сервиса его программирования. Опыт показывает, что пользователь может пожертвовать быстродействием ради сокращения затрат времени на программирование своей задачи, перейдя на ЭВМ с лучшим сервисом. Решающим фактором является наличие языка программирования высокого уровня, для решения физических задач это обычно ФОРТРАН. Менее важно, но существенно, чтобы АП-32 был оснащен удобными средствами отладки программ, имелись библиотеки и пакеты подпрограмм для расчётов, машинной графики и т. п.

## Программный комплекс

Комплекс программного обеспечения ММВК — набор связанных между собой и взаимодействующих друг с другом программ ЭВМН и АП-32, позволяющих решать поставленные перед ММВК задачи. При его анализе можно воспользоваться моделью, предназначенной для описания множества возможностей вычислительной системы в совокупности с множеством правил их реализации. С помощью такой модели программный комплекс представляется как иерархическая система, где каждый уровень иерархии выполняет конкретные задачи, обеспечивая функционирование расположенного выше уровня и опираясь на результаты деятельности уровня ниже. Использовано понятие «операционная среда» (внешние по отношению к программе средства, здесь — уровни, расположенные ниже относительно какого-либо).

Комплекс ПО изображен при помощи схемы на Рис.510, описывающей такую иерархию. Каждый её уровень также представляет собой комплекс программных средств. Нижний уровень физически расположен в АП-32, остальные — в ЭВМН. Большую часть средств составляют стандартные программы ЭВМН. Для обеспечения требуемой «прозрачности» происходит внедрение специфических для АП-32 программ на всех уровнях, и средства ЭВМН и АП-32 взаимодействуют между собой по вертикали и горизонтали схемы.

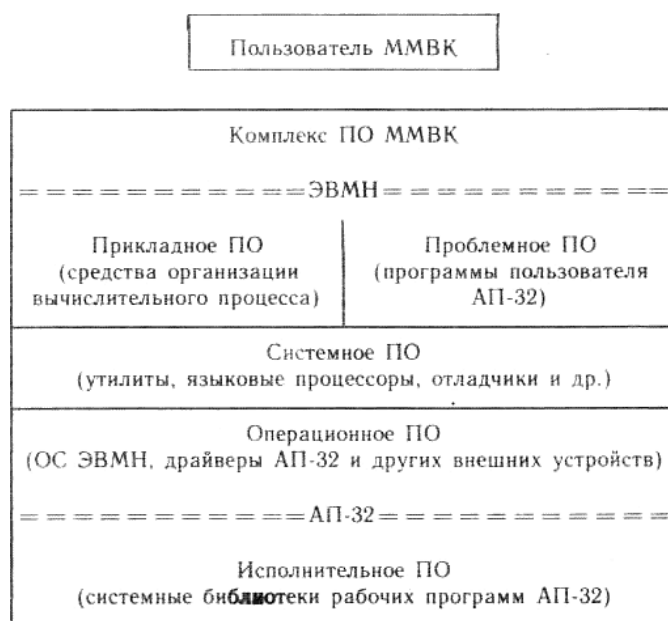


Рис.510 - Комплекс программного обеспечения ММВК.

## Средства разработки программ АП-32

Комплекс средств разработки программ АП-32, реализующих путь от алгоритма до машинных кодов, работает по традиционной схеме Рис.511, применяемой на многих современных ЭВМ. На Рис.511 прямоугольниками обозначены продукты деятельности средств программирования.

1. Ввод текста программы на языке ассемблера или ФОРТРАНе, его редактирование. Текст создается при помощи одного из работающих под операционными системами RT-11 или TSX-Plus редакторов. Результатом этой деятельности являются текстовые файлы типа .ASM или .FOR.
2. Трансляция текстового файла производится ассемблером A32 или компилятором FORTAP. Результатом работы этих программ является файл листинга типа .LST и файл типа .OBJ, содержащий перемещаемый объектный модуль.
3. Библиотечные файлы типа .OBJ создаются библиотекарем LIBRAP.
4. Сборка (построение задачи) из одного или несколько созданных в разное время объектных файлов и библиотек типа OBJ. Файлы поступают на вход редактора связей LINKAP, который создает файл образа задачи типа .JOB и файл карты распределения памяти АП-32 типа .MAP.
5. Загрузка файла типа .JOB в память программ и данных АП-32 и запуск программы со стартового адреса осуществляется программой обслуживания OTSAP.

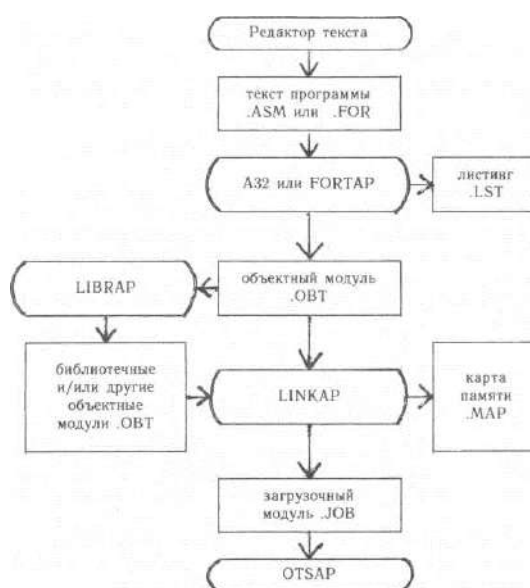


Рис.511 – Схема разработки ПО для АП-32.

Задача создания средств разработки состояла из следующих подзадач:

1. Разработка языков программирования (язык ассемблера и диалект ФОРТРАНа для АП-32).
2. Спецификация форматов объектного кода, файла образа задачи (загрузочного модуля), распределения памяти задачи АП-32, а также форматов листингов и карт распределения памяти.
3. Реализация перечисленных на схеме средств программирования АП-32.

## Операционная среда программы АП-32

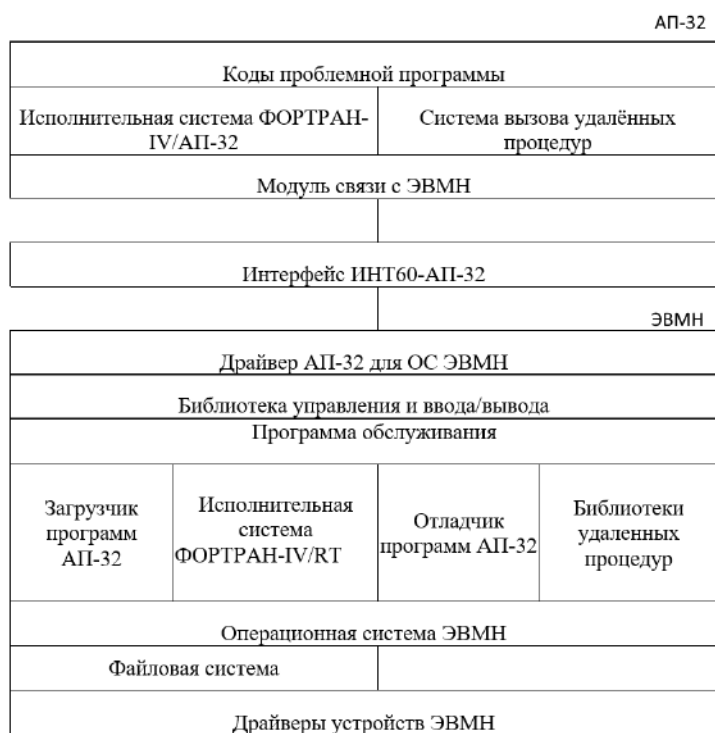
### Структура и связи программы АП-32

Как уже упоминалось, при разработке ПО АП-32 была поставлена задача обеспечить проблемной программе процессора возможности на уровне универсальной ЭВМ. Они сводятся к следующему:

1. реализация произвольных вычислительных алгоритмов;
2. исполнение фортрановских файловых операций и ввода/вывода;
3. выполнение системно-зависимых операций ЭВМН под управлением программ АП-32;
4. вызов подпрограмм ЭВМН из программ АП-32.

Первую из приведенных возможностей обеспечивает сам автор проблемной программы. Для реализации остальных требуется создание операционной среды программы — внешних по отношению к ней программных средств.

На Рис.512 приведена схема, описывающая структуру проблемной программы АП-32 и программное сопряжение АП-32 и ЭВМН. Это укрупненный фрагмент схемы Рис.510. Схема состоит из двух частей: программы, работающей в АП-32, и программы, работающей в ЭВМН. Машины связаны аппаратурой интерфейса ИНТ60-АП32.



**Рис.512 – Структура программы АП-32.**

При пуске программы терминал ЭВМН становится терминалом программы АП-32. Перевод терминала в контекст ОС ЭВМН возможен только при завершении работы программы АП-32, нормальном или принудительном. Принудительно программа АП-32 останавливается при останове OTSAP (например, двумя CTRL/C для ОС RT-11). При нормальном останове (например, исполнение фортрановского оператора STOP) на экран терминала выводится полное время исполнения программы АП-32 (с временем ожидания ввода/вывода) и процессорное время АП-32, после чего OTSAP завершается.

После пуска АП-32 нормальным состоянием OTSAP становится ожидание его останова. Причинами останова могут быть:

1. сбой аппаратуры интерфейса ИНТ60-АП32;
2. ошибка при обращении к АП-32 (например, запись данных в несуществующую память АП-32);



3. арифметическая ошибка АП-32 (переполнение, деление на нуль);
4. запрос на операцию исполнительной системы ФОРТРАН-IV;
5. вывод сообщения об ошибке исполнительной системы;
6. вызов удалённой подпрограммы;
7. оператор PAUSE;
8. оператор STOP.

В каждом случае OTSAP принимает решение о реакции на причину останова и запускает нужную процедуру. После её окончания OTSAP запускает программу АП-32, если это необходимо, и возвращается в состояние ожидания. Если пуск не нужен (например, исполнен STOP), работа OTSAP прекращается.

Параметры из АП-32 передаются подпрограммам ЭВМН без дополнительной обработки, за исключением форматного ввода/вывода, когда все форматные преобразования происходят в АП-32, а в ЭВМН передается уже текстовая информация в А-формате (то же самое в обратном направлении). Та же часть исполнительной системы ФОРТРАНа АП-32, которая находится в ЭВМН, состоит из фиксированного набора подпрограмм и является неизменяемой частью OTSAP.

Библиотеки удалённых процедур — динамическая компонента OTSAP, т. е. OTSAP можно строить с разными библиотеками. Число библиотек, включаемых в состав OTSAP, не ограничено. В настоящее время из программы АП-32 можно обращаться к следующему ряду библиотек:

1. SYSLIB RT-11—фортрановский интерфейс к системным директивам ОС RT-11;
2. SL7LIB — пакет для управления цветным графическим дисплеем ЭВМН;
3. VIDI — пакет для интерактивной работы с терминалом;
4. FMS-11 — библиотека для интерактивной работы с терминалом путем заполнения экранных бланков.

На установку новой библиотеки удалённых подпрограмм требуется время, необходимое для программирования описателей этих подпрограмм.

Следует упомянуть, что при пуске с ключом /D управление передается в отладчик АП-32 по любой из перечисленных причин останова, включая ошибки, нормально обрабатываемые исполнительной системой.

### ***Операционная система ЭВМН, файловая система, драйверы***

Исполнительная система и библиотеки OTSAP содержат системные директивы, посредством которых они обращаются за сервисом к ОС ЭВМН. В настоящее время для ЭВМН используются ОС RT-11 и TSX-Plus, позволяющие работать с любой моделью семейства «Электроника» или СМ ЭВМ, используемой в качестве ЭВМН. Мощные модели позволяют подключать более одного АП-32, каждый из которых обслуживается своей OTSAP. В принципе возможно перейти к ОС RSX-11M или RSX-11M-Plus, но для этого требуется модификация OTSAP и драйвера АП-32.

Обращение АП-32 к устройствам массовой памяти обрабатывается, как правило, файловым процессором ОС ЭВМН, т. е. частью ОС, взаимодействующей с файловой структурой магнитного диска или ленты. Если файловой структуры нет или требуется её игнорировать, запрос на операцию ввода/вывода передается непосредственно драйверу устройства, минуя файловый процессор. Файловая структура привязана к конкретной операционной системе, но зависимость от неё при работе АП-32 на ФОРТРАНе незначительна.

Драйверы ОС ЭВМН позволяют АП-32 обращаться к любому внешнему устройству, подключенному к ЭВМН (принтеры, графика, линии межмашинной связи и т. д.).

## ЗАКЛЮЧЕНИЕ

Важнейшим и главным компонентом любой вычислительной машины является её центральный процессор (CPU). За время, прошедшее с момента появления первого в мире микропроцессора Intel 4004 – 15 ноября 1971 года<sup>1</sup>, процессорная индустрия проделала огромный, часто кажущийся просто невероятным путь: количество элементов на кристалле выросло с 2300 у 4004 до десятков миллиардов(!) в современных процессорах Intel, AMD, Nvidia, ARM, IBM. Технологические нормы сократились с 10 мкм до сегодняшних 3...5...10нм, рабочие тактовые частоты выросли с сотен кГц до 3...6 ГГц.

В курсе была подробно рассмотрена эволюция вычислительных архитектур на основе процессоров семейства x86(-64) компании Intel – от первых персональных Altair-8800 на базе i8080, до современных сверхпроизводительных суперкомпьютеров из Top500. Эта линейка – компьютеры на основе Intel x86 – была выбрана для детального рассмотрения потому что, несмотря на изначальную очевидную нестройность, запутанность и сумбурность системы команд этого семейства, некоторые не очень удачные решения, принимавшиеся разработчиками этой архитектуры в отдельные моменты истории (NetBurst, например), x86 сумела на протяжении нескольких десятилетий быть номером один в процессорной гонке практически во всём диапазоне применений – от экономичных процессоров в супертонких ультрабуках до топовых CPU в суперкомпьютерах с мегаваттами потребляемой мощности. В «среднем» диапазоне – ноутбуки, десктопы, серверы – также долгое время фактически доминировала интеловская архитектура.

Но, подводя итоги многолетней истории развития компьютерной индустрии, сегодня, в 2021 году, необходимо отметить, что дела у Intel обстоят уже не так хорошо, как это было ещё пять-шесть лет назад. Восстал из пепла, как птица Феникс, их вечно второй конкурент<sup>2</sup> – компания AMD, сумевшая своими Ryzen и Epyc если ещё не заметно потеснить Intel, но заставившая всерьёз напрячься игрока номер один, как было когда-то, лет двадцать назад, в начале 2000-хх.

Несколько лет назад Intel фактически полностью проиграла мобильный рынок (кстати, как и Microsoft в части software), не сумев закрепиться на рынке смартфонов и лишь весьма незначительно обозначив своё присутствие на рынке планшетов – в мобильном мире сегодня с огромным отрывом доминирует платформа ARM, количество выпущенных устройств на её основе в разы превосходит количество x86-based систем.

Мобильный рынок для Intel фактически потерян, в «середине» – ноутбуки, десктопы, серверы – их всё больше и больше начинает теснить AMD. Но и на самом «верху», в суперкомпьютерной «высшей лиге», в июне 2020 года произошло знаковое событие для Intel: впервые за 9 лет номером один (top1) стал суперкомпьютер не из США или Китая, а японский Fugaku. И, что более существенно и, наверное, особенно обидно для Intel – построенный не на привычной в последние годы гибридной архитектуре x86/GPU (или IBM Power/GPU), а на ARM A64FX 48C 2.2GHz! На архитектуре ARM, удел которой, как казалось долгие годы – мобильные, энергоэффективные решения, но никак не суперкомпьютеры.

Подлила масла в огонь и компания Apple (в своё время, в 2004 году, перешедшая на платформу x86 с PowerPC), объявившая в том же июне 2020 года об уходе с x86 также на платформу ARM (собственный процессор arm-архитектуры A14).

Давление на Intel «по всему фронту» усугубляются ещё многолетними проблемами компании с переходом на новый 10нм техпроцесс (этап «Тик»). Intel выпускает по 10нм

---

<sup>1</sup> Любопытный факт: по сведениям из википедии ([Intel 4004](#)) – этот МП выпускался на протяжении 10-ти лет!

<sup>2</sup> «Вечно второй» компании AMD, к слову, принадлежит первенство в разработке расширения архитектуры x86-64. AMD первой, раньше, чем Intel, сделала RISC-ядро для процессора x86-архитектуры. Да и в области поддержки аппаратной виртуализации Intel заимствовала некоторые решения у «второго номера» – компании AMD.

технологии очень ограниченные партии процессоров ноутбучного класса, тогда как конкуренты давно работают на 7 нм (AMD Ryzen/Ерус), а некоторые (Apple) объявили уже даже о 5 нм техпроцессе.

Тут, правда, нужно разбираться внимательно: недавние сравнительные исследования под микроскопом 7нм кристаллов, произведённых TSMC, и 14+++ нм чипов от Intel показали вовсе не двукратную разницу в размере ширины затворов транзисторов, а всего лишь 24 нм против 26 нм. Давно известный факт, что свои нанометры каждый производитель измеряет по-своему.

Новости осени 2020 года заставили напрячься уже не только Intel, но и всю мировую индустрию – было объявлено о приобретении ARM компанией Nvidia. Один из основателей ARM опасается, что принципиально другая модель бизнеса у Nvidia может привести к возникновению супермонополиста для самой распространённой сегодня процессорной архитектуры Arm.

Новость от 26 сентября 2020, довольно претензиозно озаглавленная «*Arm утрёт нос Intel и AMD своим 128-ядерным процессором с поддержкой DDR5 и PCIe 5*», добавляет интриги в историю о том, как дальше будет развиваться компьютерная индустрия и её главный центральный элемент. Кстати, уже довольно давно, этот самый центральный элемент принято называть и относить не просто к CPU, а обозначать как «Система на кристалле» (SoC). Это давно ставшее общепринятым обозначение SoC для мобильных устройств (смартфоны), теперь оно распространилось и на серверные решения, например, AMD Ерус тоже называют SoC.

Ниже статья [Arm утрет нос Intel и AMD своим 128-ядерным процессором с поддержкой DDR5 и PCIe 5](#) приведена полностью, с небольшими несущественными правками.

### ***Формально новинку готовит Arm, но по факту она уже может выйти под брендом Nvidia***

Компания Arm, как пишет источник, работает над высокопроизводительной платформой Neoverse N2 для серверов, обеспечивающих работу облачных сервисов. Она получит 128 вычислительных ядер.

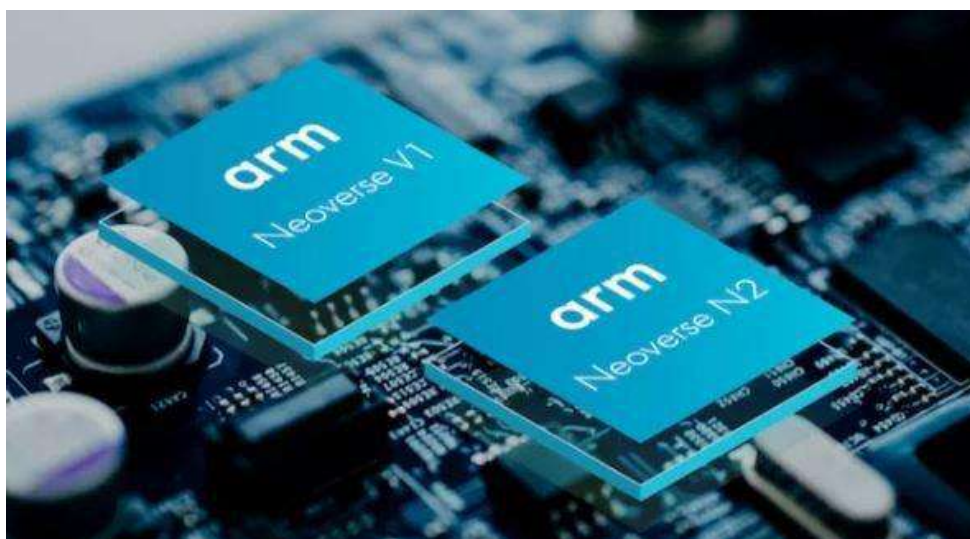


Рис.513 – Arm Neovease N2.

Neoverse N2 составит конкуренцию серверным решениям Intel и AMD, причем у неё масса других преимуществ, помимо огромного количества ядер. Так, заявлен 5-нанометровый техпроцесс, TDP всего лишь 350 Вт, поддержка памяти DDR5 и HBM2e, масштабируемого векторного расширения (SVE) и интерфейса PCIe 5. Одновременно с Neoverse N2

готовится более доступная платформа Neoverse V1, которая будет отличаться меньшим количеством ядер — 96. Правда, технология многопоточности не заявлена ни для Neoverse N2, ни для Neoverse V1.

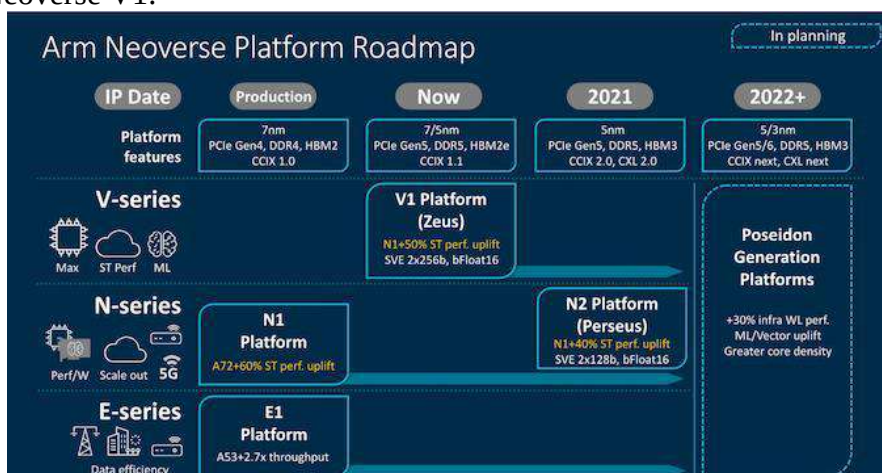


Рис.514 – Roadmap платформы Arm Neoverse.

Neoverse N2 выйдет, ориентировочно, в 2021 году. Как известно, Nvidia уже оформляет сделку по покупке Arm, она должна быть завершена в течение 18 месяцев. Поэтому может так случиться, что с процессорами Intel и AMD будет конкурировать уже не непосредственно сама Arm, а поглотившая её Nvidia.

Здесь можно отметить, что по Intel нанесён «залп» не только по части их «отсталого» техпроцесса и меньшего количества ядер в топовых серверных процессорах, но и по заявленной поддержке интерфейса PCIe 5 на платформе Arm Neoverse – ведь Intel только-только, с большим опозданием, объявила о поддержке четвертой версии PCIe 4, а тут уже заявлена поддержка версии пятой. Правда, стоит отметить, пока все эти «killer features» существуют только на бумаге – и 128 arm-ядер, и 5нм техпроцесс, и реально работающий PCIe 5 – насколько реальным окажутся эти заявки на лидерство, покажет ближайшее будущее.

Но наступление весьма непростых времён для компании Intel – уже давно не секрет, а давно свершившийся факт, даже пока без заявленного «утирания носа» со стороны Arm и AMD. Intel явно потеряла темп в развитии в последние несколько лет, а конкуренты, напротив, сумели именно в этот момент совершить прорывы и обойти «царя горы» по некоторым направлениям – может быть лишь на какой-то короткий отрезок времени, год-два, может быть надолго, на десятилетие-полтора, или даже навсегда – пока точно предсказать никто не берётся. Но тем интереснее эти времена для нас, как пользователей их высокотехнологичной продукции: что будет завтра, какая архитектура(-ы) станет лидирующей, кто возглавит процессорную гонку? Ждём и пользуемся результатами этой бесконечной (хочется верить) интереснейшей гонки инноваций и открытий, происходящей на наших глазах.

## СПИСОК ЛИТЕРАТУРЫ

1. *Принципы работы системы IBM/370*. П/р Л.Д.Райкова. Москва, «Мир», 1975.
2. *Супер-ЭВМ. Аппаратная и программная реализация*. П/р С.Фернбаха. Москва, «Радио и связь», 1991.
3. В.В.Сташин, А.В.Урусов, О.Ф.Молокогонцева. *Проектирование цифровых устройств на однокристалльных микроконтроллерах*. Москва, "Энергоатомиздат", 1990.
4. *Мини- и микроЭВМ семейства «Электроника»*. Производственное издание. Б.Л.Толстых и др. Москва, «Радио и связь», 1987.
5. М.Сингер. *Мини-ЭВМ PDP-11: программирование на языке ассемблера и организация машины*. Серия «Математическое обеспечение ЭВМ». Москва, «Мир», 1984.
6. *Центральный процессор М2. ТО и инструкция по эксплуатации*. ЦНИИ «Электроника», 1982.
7. С.П.Морс, Д.Д.Алберт. *Архитектура микропроцессора 80286*. Москва, «Радио и связь», 1990.
8. В.Л.Григорьев. *Архитектура микропроцессора 80486, в 4-х томах*. «ГРАНАЛ», 1993.
9. Ч.Кэпс, Р.Стаффорд. *VAX: программирование на языке ассемблера и архитектура*. Москва, «Радио и связь», 1991.
10. Э.Таненбаум, Т.Остин. *Архитектура компьютера*. 6-е издание. «Питер», 2013.
11. Э.Таненбаум, Х.Бос. *Современные операционные системы*. 4-ое издание. «Питер», 2015.
12. К.Хамахер, З.Вранешич, С.Заки. *Организация ЭВМ*. 5-ое издание. "ВНУ", «Питер», 2003.
13. William Stallings. *Computer Organization and Architecture (Designing for Performance)*. Tenth Edition. "PEARSON", 2016.
14. Джон Хеннесси, Дэвид Паттерсон. *Компьютерная архитектура. Количественный подход*, 6-е изд. «Техносфера», 2018.

Учебное издание

**Дубров** Сергей Валерьевич

## АРХИТЕКТУРА И ЭВОЛЮЦИЯ ЭВМ

Учебное пособие

Подготовка к печати

С. В. Исаковой, Е. В. Неклюдовой

Подписано в печать 05.04.2021 г.

Формат 60 х 84 1/8. Уч.-изд. л. 81,75. Усл. печ. л. 76.

Издательско-полиграфический центр НГУ  
630090, Новосибирск, ул. Пирогова, 2.